Tối ưu hoá, Optimizer và Hyperparameter

Lộ trình 51 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

Tối ưu hoá, optimizer và hyperparameter: lộ trình 51 bài . . . 4
Bài toán tối ưu trong học máy: biến dữ liệu và mô hình thành một mục tiêu có thể cải thiện . . . 12
Gradient descent: đi ngược gradient với bước đủ nhỏ để loss giảm . . . 20
SGD, mini-batch và nhiễu: học từ một phần dữ liệu ở mỗi bước . . . 28
Momentum và Nesterov: giữ hướng đi, rồi nhìn trước khi lấy gradient . . . 38
Optimizer thích nghi: mỗi tham số được điều chỉnh theo lịch sử gradient của nó . . . 47
Learning rate và lịch học: bước lớn để tiến, bước nhỏ để tinh chỉnh . . . 56
Batch size, critical batch và quy tắc scale learning rate: chọn lượng dữ liệu cho mỗi bước và điều chỉnh nhịp cập nhật . . . 65
Khởi tạo, normalization, gradient clipping và mixed precision: giữ tín hiệu học trong phạm vi ổn định . . . 75
00 Preface: Toán nền tảng và hướng dẫn đọc về Tối ưu hoá, Optimizer và Hyperparameter . . . 84
01 Convex Optimization: Algorithms and Complexity (2014) . . . 103
02 Optimization Methods for Large-Scale Machine Learning (2016) . . . 115
03 An overview of gradient descent optimization algorithms (2016) . . . 126
04 A Differential Equation for Modeling Nesterov’s Accelerated Gradient Method: Theory and Insights (2015) . . . 134
05 Qualitatively Characterizing Neural Network Optimization Problems (2014) . . . 142
06 Adam: A Method for Stochastic Optimization (2014) . . . 149
07 Decoupled Weight Decay Regularization (2017) . . . 157
08 On the Convergence of Adam and Beyond (2019) . . . 164
09 On the Variance of the Adaptive Learning Rate and Beyond (2019) . . . 171
10 Large Batch Optimization for Deep Learning: Training BERT in 76 minutes (2019) . . . 182
11 Symbolic Discovery of Optimization Algorithms (2023) . . . 189
12 Adafactor: Adaptive Learning Rates with Sublinear Memory Cost (2018) . . . 197
13 Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training (2023) . . . 204
14 Shampoo: Preconditioned Stochastic Tensor Optimization (2018) . . . 211
15 SOAP: Improving and Stabilizing Shampoo using Adam (2024) . . . 220
16 Muon is Scalable for LLM Training (2025) . . . 229
17 The Road Less Scheduled (2024) . . . 238
18 SGDR: Stochastic Gradient Descent with Warm Restarts (2016) . . . 247
19 Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates (2017) . . . 253
20 MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies (2024) 260
21 Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour (2017) . . . 267
22 An Empirical Model of Large-Batch Training (2018) . . . 273
23 Don’t Decay the Learning Rate, Increase the Batch Size (2017) . . . 280
24 On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima (2016) . . . 286
25 Sharpness-Aware Minimization for Efficiently Improving Generalization (2020) . . . 292
26 Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability (2021) . . . 298
27 Visualizing the Loss Landscape of Neural Nets (2017) . . . 304
28 On Empirical Comparisons of Optimizers for Deep Learning (2019) . . . 311
29 Descending through a Crowded Valley: Benchmarking Deep Learning Optimizers (2020) . . . 317
30 Benchmarking Neural Network Training Algorithms (2023) . . . 323
31 Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization (2016) . . . 334

32 Practical Bayesian Optimization of Machine Learning Algorithms (2012) . . . 341
33 A Tutorial on Bayesian Optimization (2018) . . . 348
34 Population Based Training of Neural Networks (2017) . . . 356
35 Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer (2022) . . . 364
36 Scaling Exponents Across Parameterizations and Optimizers (2024) . . . 375
37 Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification (2015) . . . 387
38 Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift (2015) 399
39 Layer Normalization (2016) . . . 410
40 On the difficulty of training Recurrent Neural Networks (2012) . . . 418
41 Why Gradient Clipping Accelerates Training: A Theoretical Justification for Adaptivity (2019) . . . 427
42 Mixed Precision Training (2017) . . . 435
43 The Marginal Value of Adaptive Gradient Methods in Machine Learning (2017) . . . 444
44 How Does Batch Normalization Help Optimization? (2018) . . . 453
45 Deconstructing What Makes a Good Optimizer for Language Models (2024) . . . 462
46 Practical Recommendations for Gradient-Based Training of Deep Architectures (2012) . . . 471
47 A Disciplined Approach to Neural Network Hyper-Parameters: Part 1 - Learning Rate, Batch Size, Momentum, and Weight Decay (2018) . . . 479
48 Old Optimizer, New Norm: An Anthology (2024) . . . 487
49 Gradient Descent Happens in a Tiny Subspace (2018) . . . 497
50 Why Do We Need Weight Decay in Modern Deep Learning? (2023) . . . 507
51 How to Set AdamW's Weight Decay as You Scale Model and Dataset Size (2024) . . . 516

Lộ trình

Tối ưu hoá, optimizer và hyperparameter: lộ trình 51 bài

Thư mục này chứa ghi chú học tập bằng tiếng Việt về **tối ưu hoá cho deep learning**: tối ưu lồi và SGD quy mô lớn, họ Adam, optimizer tiết kiệm bộ nhớ và bậc hai (Lion, Adafactor, Sophia, Shampoo, SOAP, Muon), learning rate schedule, batch size, sharpness và loss landscape, benchmark optimizer, tìm hyperparameter (Hyperband, Bayesian, PBT), muP, khởi tạo/chuẩn hoá/clipping/mixed precision, weight decay. Phần lồi cổ điển không có paper arXiv được dạy trong bài 00.

Mọi paper đều được đọc từ **full text trên arXiv** (PDF, tải ngày 09/11/2026). ID của từng bài đã được đối chiếu với tiêu đề trên trang arXiv. Một số paper là sách hoặc khảo sát rất dài: phạm vi đã đọc được ghi rõ trong header và mục 6 của từng bài.

> Đây là ghi chú diễn giải, không phải bản dịch. Mục ”Ứng dụng cho dự án” trong mỗi file là suy luận của chúng ta (cho dự án LightOnOCR-3-0.8B đã nén), **không phải** kết luận của paper. Các phép tính đánh dấu ”tự tính” là của người ghi chú.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	Toán và kiến thức nền không có trong paper, bảng thuật ngữ, cách đọc phản biện, lịch đọc, liên hệ dự án, 5 câu tự kiểm tra. **Đọc trước tiên.**

Giai đoạn 1: Nền tảng tối ưu và SGD

#	File	Bài	Ý chính
01	`01_2014_convex_optimization_bubeck.md`	Convex Optimization: Algorithms and Complexity (2014)	một cuốn sách chuyên khảo (monograph) trình bày hệ thống các định lý độ phức tạp (complexity) trung tâm của tối ưu lồi (convex optimization) cùng thuật toán tương ứng:…
02	`02_2016_optimization_methods_large_scale_ml.md`	Optimization Methods for Large-Scale Machine Learning (2016)	một khảo sát toàn diện về các phương pháp tối ưu số cho machine learning quy mô lớn: lý thuyết hội tụ/độ phức tạp đầy đủ của stochastic gradient (SG, tức SGD) cho cả hàm…
03	`03_2016_overview_gradient_descent_ruder.md`	An overview of gradient descent optimization algorithms (2016)	tổng quan mang tính trực giác (không đi sâu lý thuyết hội tụ hình thức) về các biến thể gradient descent (batch/stochastic/mini-batch), cách suy luận từng bước để đi từ…

#	File	Bài	Ý chính
04	`04_2015_nesterov_ode.md`	A Differential Equation for Modeling Nesterov's Accelerated Gradient Method: Theory and Insights (2015)	suy ra một phương trình vi phân thường (ODE) bậc hai, là giới hạn liên tục (continuous-time limit) của thuật toán Nesterov's Accelerated Gradient (NAG, Nesterov 1983)…
05	`05_2014_qualitative_optimization.md`	Qualitatively Characterizing Neural Network Optimization Problems (2014)	đề xuất một kỹ thuật thực nghiệm rất đơn giản (nội suy tuyến tính tham số từ điểm khởi tạo đến điểm nghiệm huấn luyện) để kiểm tra định tính xem mạng neural khi huấn…

Giai đoạn 2: Họ Adam

#	File	Bài	Ý chính
06	`06_2014_adam.md`	Adam: A Method for Stochastic Optimization (2014)	thuật toán tối ưu bậc một Adam (viết tắt của "adaptive moment estimation"): kết hợp momentum (ước lượng moment bậc 1 của gradient) với learning rate thích ứng theo từng…
07	`07_2017_adamw.md`	Decoupled Weight Decay Regularization (2017)	chỉ ra L2 regularization và weight decay **không tương đương** cho các optimizer thích ứng (như Adam), dù tương đương cho SGD;
08	`08_2019_amsgrad.md`	On the Convergence of Adam and Beyond (2019)	chỉ ra Adam (và RMSProp) có thể **không hội tụ** tới nghiệm tối ưu trong một số bài toán online convex đơn giản, xác định lỗi cụ thể trong chứng minh hội tụ gốc của…
09	`09_2019_radam.md`	On the Variance of the Adaptive Learning Rate and Beyond (2019)	lý giải bằng lý thuyết lẫn thực nghiệm tại sao heuristic *learning rate warmup* giúp ổn định huấn luyện với các optimizer thích ứng (RMSProp, Adam): nguyên nhân được xác…

Giai đoạn 3: Quy mô lớn, tiết kiệm bộ nhớ và bậc hai

#	File	Bài	Ý chính
10	`10_2019_lamb.md`	Large Batch Optimization for Deep Learning: Training BERT in 76 minutes (2019)	optimizer LAMB (Layerwise Adaptive Moments for Batch training), một chiến lược tổng quát biến optimizer nhỏ-batch (SGD, Adam) thành optimizer cho batch cực lớn bằng cách…

#	File	Bài	Ý chính
11	`11_2023_lion.md`	Symbolic Discovery of Optimization Algorithms (2023)	dùng program search (tìm kiếm chương trình, kiểu AutoML) để tự động phát hiện thuật toán optimizer, từ đó tìm ra **Lion** (EvoLved Sign Momentum): optimizer chỉ lưu 1…
12	`12_2018_adafactor.md`	Adafactor: Adaptive Learning Rates with Sublinear Memory Cost (2018)	Adafactor, biến thể của Adam giảm bộ nhớ lưu second-moment từ $O(nm)$ xuống $O(n+m)$ cho tham số dạng ma trận bằng cách chỉ lưu tổng theo hàng/cột (factored second…
13	`13_2023_sophia.md`	Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training (2023)	Sophia (Second-order Clipped Stochastic Optimization), optimizer bậc hai nhẹ cho pretraining LLM: dùng ước lượng đường chéo Hessian (Hutchinson hoặc…
14	`14_2018_shampoo.md`	Shampoo: Preconditioned Stochastic Tensor Optimization (2018)	thuật toán tối ưu ”bậc hai gần đúng” (approximate second-order / adaptive preconditioning) khai thác **cấu trúc tensor** của tham số mạng neural: thay vì một…
15	`15_2024_soap.md`	SOAP: Improving and Stabilizing Shampoo using Adam (2024)	chứng minh một kết nối hình thức (formal connection) giữa Shampoo (dùng luỹ thừa 1/2) và Adafactor: Shampoo (biến thể luỹ thừa 1/2) tương đương với chạy Adafactor trong…
16	`16_2025_muon.md`	Muon is Scalable for LLM Training (2025)	mở rộng optimizer Muon (K

Giai đoạn 4: Learning rate schedule

#	File	Bài	Ý chính
17	`17_2024_schedule_free.md`	The Road Less Scheduled (2024)	đề xuất ”Schedule-Free” learning: một họ phương pháp (Schedule-Free SGD, Schedule-Free AdamW) loại bỏ hoàn toàn nhu cầu đặt lịch learning rate (learning rate schedule,…
18	`18_2016_sgdr.md`	SGDR: Stochastic Gradient Descent with Warm Restarts (2016)	lịch learning rate dạng ”cosine annealing với warm restart” cho SGD: định kỳ tăng lại learning rate rồi giảm theo cosine, giúp cải thiện hiệu suất ”anytime” (đạt kết quả…
19	`19_2017_super_convergence.md`	Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates (2017)	hiện tượng ”super-convergence”: dùng cyclical learning rate (CLR) với một chu kỳ duy nhất (”1cycle”) và learning rate tối đa rất lớn giúp huấn luyện nhanh hơn một bậc độ…

#	File	Bài	Ý chính
20	`20_2024_minicpm.md`	MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies (2024)	chiến lược huấn luyện SLM (Small Language Model) có thể scale được, gồm ”Model Wind Tunnel Experiments” (dò siêu tham số/batch size/lr trên model nhỏ rồi transfer), và…

Giai đoạn 5: Batch size

#	File	Bài	Ý chính
21	`21_2017_large_minibatch_sgd.md`	Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour (2017)	quy tắc ”linear scaling rule” (tăng learning rate tuyến tính theo minibatch size) kết hợp chiến lược ”warmup” (tăng dần lr đầu huấn luyện) để huấn luyện ResNet-50 trên…
22	`22_2018_empirical_model_large_batch.md`	An Empirical Model of Large-Batch Training (2018)	định nghĩa ”gradient noise scale” B_{noise} (và bản đơn giản B_{simple}) để dự đoán kích thước batch tối đa còn hữu ích (”critical batch size” B_{crit}) trên nhiều…
23	`23_2017_dont_decay_lr_increase_batch.md`	Don’t Decay the Learning Rate, Increase the Batch Size (2017)	chứng minh giảm learning rate và tăng batch size trong quá trình huấn luyện tương đương nhau về mặt ”noise scale”;
24	`24_2016_sharp_minima.md`	On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima (2016)	quan sát thực nghiệm rằng SGD batch lớn (large-batch, LB) có xu hướng hội tụ về cực tiểu ”sắc” (sharp minima), còn SGD batch nhỏ (small-batch, SB) hội tụ về cực tiểu…

Giai đoạn 6: Sharpness và loss landscape

#	File	Bài	Ý chính
25	`25_2020_sam.md`	Sharpness-Aware Minimization for Efficiently Improving Generalization (2020)	đề xuất Sharpness-Aware Minimization (SAM), một thuật toán tối ưu tối thiểu hoá đồng thời giá trị loss và độ ”sắc” (sharpness) của loss, nhằm cải thiện generalization,…
26	`26_2021_edge_of_stability.md`	Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability (2021)	quan sát thực nghiệm rằng full-batch gradient descent (GD) trên neural network thường tự đẩy ”sharpness” (giá trị riêng lớn nhất của Hessian loss huấn luyện) lên ngưỡng…

#	File	Bài	Ý chính
27	`27_2017_visualizing_loss_landscape.md`	Visualizing the Loss Landscape of Neural Nets (2017)	kỹ thuật trực quan hoá (visualization) loss landscape của mạng neural bằng ”filter-wise normalization,” dùng để so sánh độ sắc/phẳng (sharpness/flatness) của các điểm…

Giai đoạn 7: So sánh và benchmark optimizer

#	File	Bài	Ý chính
28	`28_2019_empirical_comparison_optimizers.md`	On Empirical Comparisons of Optimizers for Deep Learning (2019)	đo lại độ tin cậy của các so sánh thực nghiệm giữa optimizer (SGD, Momentum, Nesterov, RMSProp, Adam, NAdam) cho deep learning;
29	`29_2020_crowded_valley_benchmark.md`	Descending through a Crowded Valley: Benchmarking Deep Learning Optimizers (2020)	benchmark thực nghiệm quy mô lớn (hơn 50.000 lượt chạy huấn luyện) so sánh 15 optimizer deep learning phổ biến trên 8 bài toán, 4 ngân sách tune, và 4 learning-rate…
30	`30_2023_algoperf_benchmark.md`	Benchmarking Neural Network Training Algorithms (2023)	benchmark cạnh tranh (competitive benchmark) đo tốc độ huấn luyện (time-to-result) cho ”training algorithm” nói chung (optimizer + tuning protocol + data selection),…

Giai đoạn 8: Tìm hyperparameter

#	File	Bài	Ý chính
31	`31_2016_hyperband.md`	Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization (2016)	tối ưu siêu tham số (hyperparameter optimization) bằng early-stopping và phân bổ tài nguyên thích ứng (adaptive resource allocation), đóng khung thành bài toán…
32	`32_2012_practical_bayesian_optimization.md`	Practical Bayesian Optimization of Machine Learning Algorithms (2012)	dùng Bayesian optimization với Gaussian Process (GP) để tự động tune siêu tham số của thuật toán ML;
33	`33_2018_bayesian_optimization_tutorial.md`	A Tutorial on Bayesian Optimization (2018)	tutorial lý thuyết về Bayesian optimization (BayesOpt) cho tối ưu hàm hộp đen, không có gradient, tốn thời gian đánh giá: Gaussian Process (GP) regression làm mô hình…

#	File	Bài	Ý chính
34	`34_2017_population_based_training.md`	Population Based Training of Neural Networks (2017)	Population Based Training (PBT): một thuật toán bất đồng bộ, tối ưu đồng thời trọng số θ và siêu tham số h của một quần thể (population) mô hình đang huấn luyện…

Giai đoạn 9: Parameterization và scaling

#	File	Bài	Ý chính
35	`35_2022_tensor_programs_v_mutransfer.md`	Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer (2022)	Maximal Update Parametrization (viết tắt muP): một quy tắc scale khởi tạo, learning rate và các hệ số nhân (multiplier) theo width sao cho HP optimum (learning rate,…
36	`36_2024_scaling_exponents_parameterizations_optimizers.md`	Scaling Exponents Across Parameterizations and Optimizers (2024)	mở rộng lý thuyết ”parameterization” kiểu Tensor Programs / muP sang một không gian tổng quát hơn về giả định ”alignment” giữa tham số và dữ liệu;

Giai đoạn 10: Khởi tạo, chuẩn hoá, clipping, độ chính xác số

#	File	Bài	Ý chính
37	`37_2015_delving_deep_rectifiers_kaiming_init.md`	Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification (2015)	đề xuất Parametric ReLU (PReLU), một bản tổng quát hoá của ReLU với hệ số dốc phần âm có thể học;
38	`38_2015_batch_normalization.md`	Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift (2015)	chuẩn hoá (normalize) activation của mỗi tầng theo thống kê mini-batch (mean, variance), kèm một phép scale/shift học được (γ, β), để giảm ”internal…
39	`39_2016_layer_normalization.md`	Layer Normalization (2016)	Layer Normalization (LN), chuẩn hoá tổng đầu vào (summed input) của các neuron trong một layer theo chiều feature, tính riêng cho từng training case (không theo batch…
40	`40_2012_difficulty_training_rnns_gradient_clipping.md`	On the difficulty of training Recurrent Neural Networks (2012)	phân tích nguyên nhân (đại số, hình học, hệ động lực) của vanishing/exploding gradient khi huấn luyện RNN bằng BPTT;

#	File	Bài	Ý chính
41	`41_2019_gradient_clipping_adaptivity_theory.md`	Why Gradient Clipping Accelerates Training: A Theoretical Justification for Adaptivity (2019)	đề xuất điều kiện độ trơn (smoothness) "(L_0, L_1)-smoothness", tổng quát hơn Lipschitz-smoothness thường gặp, để chứng minh gradient clipping (và normalized gradient…
42	`42_2017_mixed_precision_training.md`	Mixed Precision Training (2017)	huấn luyện mạng neural bằng số thực nửa độ chính xác (FP16, half-precision) kết hợp giữ một bản sao trọng số FP32 (master weights) và kỹ thuật "loss scaling", để đạt độ…

Giai đoạn 11: Adaptive so với SGD, BatchNorm, optimizer cho LM

#	File	Bài	Ý chính
43	`43_2017_marginal_value_adaptive_gradient_methods.md`	The Marginal Value of Adaptive Gradient Methods in Machine Learning (2017)	so sánh khả năng tổng quát hoá (generalization) giữa các optimizer thích ứng (AdaGrad, RMSProp, Adam) và các optimizer không thích ứng (SGD, SGD với momentum/Heavy…
44	`44_2018_batchnorm_helps_optimization.md`	How Does Batch Normalization Help Optimization? (2018)	lật lại lời giải thích phổ biến "Batch Normalization (BatchNorm) hoạt động vì nó giảm internal covariate shift (ICS)";
45	`45_2024_deconstructing_llm_optimizer.md`	Deconstructing What Makes a Good Optimizer for Language Models (2024)	so sánh thực nghiệm có hệ thống nhiều optimizer (SGD, Adam, Adafactor có momentum, Lion, Signum, Sophia) cho pretraining language model decoder-only, trên nhiều quy mô…

Giai đoạn 12: Hướng dẫn thực hành

#	File	Bài	Ý chính
46	`46_2012_practical_recommendations_deep_architectures.md`	Practical Recommendations for Gradient-Based Training of Deep Architectures (2012)	hướng dẫn thực hành (không phải một thí nghiệm/đề xuất thuật toán mới) về cách chọn và tinh chỉnh siêu tham số khi huấn luyện mạng neural/deep architecture bằng…

#	File	Bài	Ý chính
47	`47_2018_disciplined_hyperparameters_part1.md`	A Disciplined Approach to Neural Network Hyper-Parameters: Part 1 - Learning Rate, Batch Size, Momentum, and Weight Decay (2018)	quy trình thực hành (”checklist”) để chọn learning rate, batch size, momentum, và weight decay cho mạng neural, dựa trên việc đọc ”đầu mối” (clue) từ đường cong…

Giai đoạn 13: Hình học optimizer, không gian con, weight decay

#	File	Bài	Ý chính
48	`48_2024_old_optimizer_new_norm.md`	Old Optimizer, New Norm: An Anthology (2024)	diễn giải lại ba optimizer phổ biến (Adam, Shampoo, Prodigy), khi tắt exponential moving average (EMA), như các biến thể của ”steepest descent” (hạ dốc nhất) dưới một…
49	`49_2018_gradient_descent_tiny_subspace.md`	Gradient Descent Happens in a Tiny Subspace (2018)	quan sát thực nghiệm (trên nhiều kiến trúc/bộ dữ liệu) rằng gradient của hàm mất trong deep learning hội tụ nhanh vào một không gian con rất nhỏ (”tiny subspace”),…
50	`50_2023_why_weight_decay.md`	Why Do We Need Weight Decay in Modern Deep Learning? (2023)	xem lại vai trò thực sự của weight decay (WD) trong deep learning hiện đại qua hai chế độ huấn luyện khác nhau (huấn luyện nhiều epoch cho ảnh, và huấn luyện gần đúng 1…
51	`51_2024_adamw_weight_decay_scaling.md`	How to Set AdamW’s Weight Decay as You Scale Model and Dataset Size (2024)	diễn giải trọng số học được của AdamW như một exponential moving average (EMA, trung bình trượt theo hàm mũ) của các bước cập nhật gần đây;

Cách dùng

Đọc preface (00) trước. Cốt lõi cho người mới: 03, 06, 07, 17, 21, 22, 35, 50.

Mỗi bài có 11 mục cố định: bối cảnh, ý tưởng chính, phương pháp chi tiết, thí nghiệm, phân tích, giới hạn, liên hệ, ứng dụng cho dự án (suy luận), thí nghiệm gợi ý, tóm tắt công thức, kết luận. Mục 6 (giới hạn) là nơi ghi các mâu thuẫn nội tại phát hiện trong paper.

Liên quan

- `math content/` (35 bài): toán nền; `ml basics content/` (40 bài): ML cơ bản; `optimization content/` (51 bài): tối ưu, optimizer, hyperparameter.
- `llm content/`, `rl content/`, `distillation content/`, `pruning content/`, `interp content/`: các lộ trình chuyên đề xây trên ba thư mục nền này.

Nền tảng 01

Bài toán tối ưu trong học máy: biến dữ liệu và mô hình thành một mục tiêu có thể cải thiện

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: Không cần kiến thức tối ưu hoá; nên biết hàm số, tọa độ và phép bình phương. Chương này nhắc lại các ý cần dùng.

Dùng ở: bài optimization 01, optimization 02, optimization 03, optimization 05, optimization 24, optimization 26, optimization 27

> Đây là chương nền để học từ đầu, không phải bản dịch paper. Các ví dụ và phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao cần một bài toán tối ưu

Một mô hình học máy có thể tạo dự đoán, nhưng dự đoán đó tốt đến đâu phải được đo bằng một tiêu chuẩn. Bài toán tối ưu gom tiêu chuẩn này thành một hàm số, rồi tìm các tham số làm hàm nhỏ đi. Hàm đo lỗi thường được gọi là **hàm mất mát** (loss); hàm cần làm nhỏ cũng có thể gọi là hàm mục tiêu (objective).

Hãy nghĩ đến mô hình dự đoán giá nhà. Tham số của mô hình là các con số điều khiển dự đoán, chẳng hạn hệ số ứng với diện tích. Với một bộ tham số, ta dự đoán cho từng căn nhà, so dự đoán với giá đã biết, rồi gộp các sai lệch thành một con số. Nếu đổi tham số làm con số đó giảm, theo tiêu chuẩn đã chọn thì mô hình đang khớp dữ liệu tốt hơn.

Tối ưu hoá không tự biết một dự đoán có công bằng, an toàn hay hữu ích hay không. Nó chỉ xử lý tiêu chuẩn đã được đưa vào hàm mục tiêu. Nếu tiêu chuẩn bỏ sót một điều quan trọng, thuật toán vẫn có thể tối ưu rất tốt cho mục tiêu sai. Vì vậy, trước khi bàn đến optimizer, cần phân biệt rõ ta đang tối ưu đại lượng nào và nghiệm có nghĩa gì.

Trong các mạng nơ-ron, tham số có thể gồm hàng triệu trọng số. Ta thường không thể vẽ toàn bộ hàm mất mát, nhưng mọi ý tưởng vẫn bắt đầu từ bài toán nhỏ: có một biến đầu vào là tham số, có một con số đầu ra là loss. Các hình trong chương dùng một hoặc hai chiều để ta nhìn rõ điều mà không thể nhìn trực tiếp trong mô hình lớn.

2. Nhắc lại nền tảng: hàm, tham số và lỗi

Một **tham số** là đại lượng mà quá trình học được phép thay đổi. Gộp tất cả tham số thành một ký hiệu θ. Một **mô hình** nhận dữ liệu đầu vào x cùng tham số θ và tạo dự đoán $f_\theta(x)$. Ký hiệu này không nói mô hình nhất thiết là một mạng nơ-ron; nó có thể là đường thẳng, cây quyết định hay một phép biến đổi đơn giản.

Một **hàm mất mát cho một ví dụ** nhận dự đoán và đáp án đúng y, rồi trả về mức phạt $\ell(f_\theta(x), y)$. Chẳng hạn, nếu dự đoán là một số thực, bình phương sai lệch là một lựa chọn. Ví dụ (tự tính): dự đoán 4 cho đáp án 5 có sai lệch $4 - 5 = -1$ và bình phương sai lệch bằng 1. Sai lệch có dấu cho biết dự đoán cao hay thấp, còn bình phương không âm để lỗi cao và lỗi thấp đều bị phạt.

Với nhiều ví dụ, ta cần gộp nhiều loss thành một số. Phép trung bình dễ giải thích: mỗi ví dụ đóng góp ngang nhau. Tổng thay cho trung bình chỉ nhân mục tiêu với một hằng số dương, nên vị trí cực tiểu giữ nguyên, nhưng độ lớn gradient và cách chọn learning rate có thể đổi. Đây là một chi tiết dễ bị bỏ qua khi so công thức giữa hai tài liệu.

Ta cũng cần phân biệt **tham số** với **siêu tham số**. Tham số θ được cập nhật trong lúc học. Siêu tham số như learning rate hoặc mức phạt regularization thường do người thiết kế chọn trước hoặc tinh chỉnh riêng. Một bài toán tối ưu có thể viết trên không gian tham số cho sẵn, ký hiệu là Θ. Nếu có ràng buộc như θ phải nằm trong một miền, nghiệm chỉ được tìm trong miền đó.

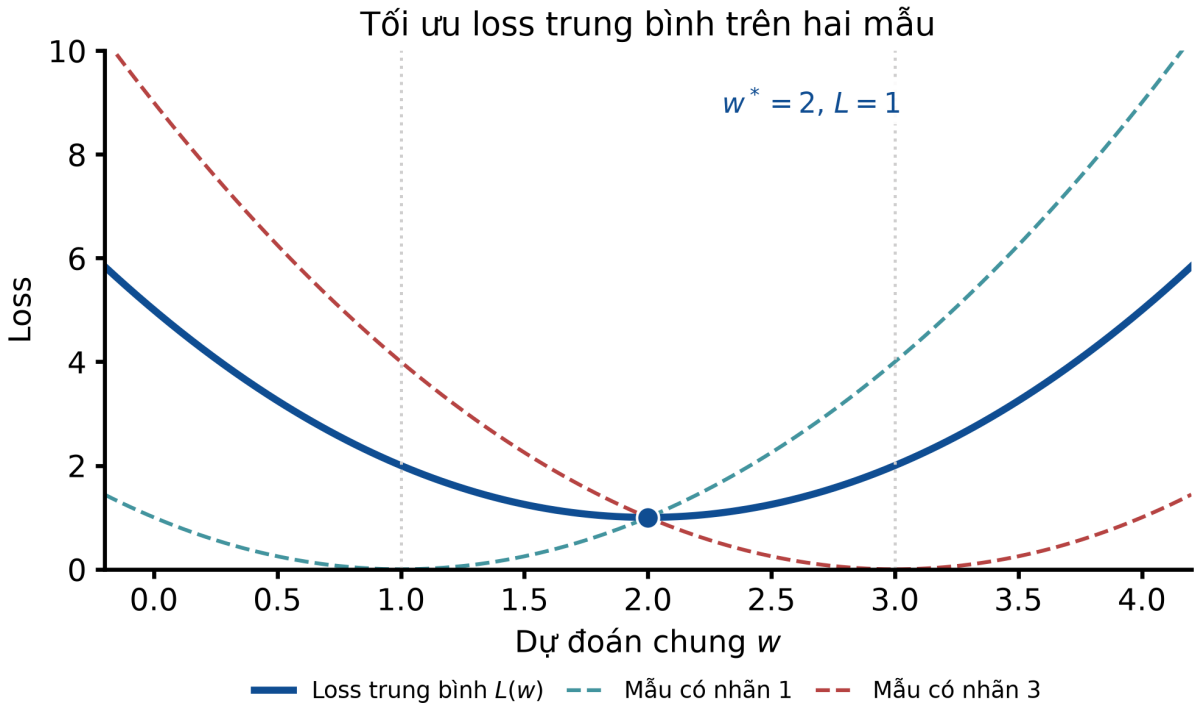
2.1 Nhắc nhanh đạo hàm, gradient và Hessian

Đạo hàm là độ dốc của hàm một biến tại một điểm. Với loss $L(w) = (w-2)^2 + 1$, đạo hàm là $L'(w) = 2(w-2)$; tại $w = 1$, độ dốc bằng -2, nên một dịch chuyển nhỏ sang phải làm loss giảm (tự tính). Đạo hàm bằng không nghĩa là đường cong phẳng ngay tại điểm đó, nhưng chưa đủ để kết luận ta đang ở đáy.

Khi tham số có nhiều tọa độ, ta gom các đạo hàm riêng thành **gradient** $\nabla J(\theta)$. Gradient cho biết mức thay đổi theo từng tọa độ. **Hessian** là ma trận các đạo hàm bậc hai, dùng để mô tả độ cong và cách các hướng tương tác. Chương này chỉ dùng chúng để phân biệt điểm dừng và cực trị; chương Gradient descent sẽ dùng gradient để tạo bước cập nhật.

3. Trực giác bằng hình: từ dự đoán đến một điểm thấp

Trước khi viết định nghĩa tổng quát, xét mô hình cực kỳ đơn giản: nó luôn dự đoán cùng một số w. Có hai nhãn huấn luyện, 1 và 3, và loss bình phương. Khi $1 < w < 3$, hai sai lệch $w-1$ và $w-3$ trái dấu, tức chúng bù nhau về hướng. Chúng chỉ triệt tiêu khi lấy trung bình sai lệch tại $w = 2$; bình phương mỗi sai lệch vẫn dương, nên loss nhỏ nhất không phải loss bằng không (tự tính). Bởi vậy ta tìm nơi trung bình bình phương lỗi nhỏ nhất, không tìm nơi lỗi từng ví dụ đều bằng không.



Hình 1: Loss trung bình của hai nhãn 1 và 3 có đáy tại giá trị dự đoán trung bình 2 (minh hoạ, tự tính)

Cách đọc hình 1: trục ngang là dự đoán chung w, trục dọc là loss trung bình trên hai nhãn. Đường cong có đáy duy nhất tại $w = 2$; hai đường gạch chỉ vị trí của nhãn 1 và 3. Tại $w = 2$, hai đường loss của từng mẫu cùng đi qua điểm xanh trên đường trung bình; vì hai giá trị trùng nhau nên hình chỉ vẽ một chấm. Mỗi nhãn vẫn có lỗi dù loss trung bình đã nhỏ nhất.

Hình này giúp tách hai câu hỏi. “Mô hình dự đoán đúng từng ví dụ chưa?” là câu hỏi về lỗi từng mẫu. “Trong các lựa chọn tham số, phương án nào làm tiêu chuẩn đã chọn tốt nhất?” là câu hỏi tối ưu hoá. Một nghiệm tối ưu cho bộ dữ liệu không nhất thiết làm mọi mẫu đúng, và cũng chưa nói nó sẽ dự đoán tốt trên dữ liệu mới.

4. Định nghĩa bài toán tối ưu trong học máy

Cho một tập huấn luyện gồm n cặp (x_i, y_i), mô hình f_θ, và loss trên một cặp dữ liệu ℓ. **Rủi ro thực nghiệm** (empirical risk), tức loss trung bình trên tập huấn luyện, là

$$L_{\text{train}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

Nếu bài toán có thêm mức phạt độ phức tạp $R(\theta)$ với hệ số $\lambda \geq 0$, mục tiêu có thể là

$$J(\theta) = L_{\text{train}}(\theta) + \lambda R(\theta).$$

Ở đây L_{train} đo khớp với dữ liệu huấn luyện, còn R thể hiện một ưu tiên bổ sung, ví dụ tham số không quá lớn. Tham số λ điều chỉnh mức ưu tiên đó. Công thức chỉ mô tả một thiết kế; chưa có định lý nào nói lựa chọn loss hoặc regularizer cụ thể phù hợp với mọi tác vụ.

Bài toán tối thiểu hoá là tìm

$$\theta^* \in \arg\min_{\theta \in \Theta} J(\theta).$$

Ký hiệu arg min là tập các tham số đạt giá trị nhỏ nhất, còn min là giá trị nhỏ nhất. Ta viết một nghiệm θ^* như một phần tử của tập nghiệm đó. Có thể có một nghiệm hoặc nhiều nghiệm cùng đạt đáy. Nếu hàm không đạt đáy trong miền, ký hiệu chính xác hơn là infimum (cận dưới đúng), chứ chưa chắc tồn tại một θ^*.

Trong học máy, ta thường chưa biết phân phối dữ liệu thực tế. Một quan điểm khái niệm là **rủi ro quần thể** (population risk), tức kỳ vọng loss trên ví dụ mới từ phân phối đó:

$$L_{\text{pop}}(\theta) = \mathbb{E}_{(x,y)\sim P}\left[\ell(f_\theta(x), y)\right].$$

Ta tối ưu trung bình mẫu L_{train} vì có dữ liệu hữu hạn; ta lại quan tâm đến L_{pop} vì mô hình sẽ được dùng trên ví dụ chưa thấy. Hai hàm này không đồng nhất. Học máy không chỉ là tìm đáy của đường cong training, mà còn cần đánh giá xem lựa chọn đó có hữu ích ngoài tập đã dùng hay không.

5. Cực tiểu toàn cục, địa phương và điểm dừng

Một tham số θ^* là **cực tiểu toàn cục** nếu không có điểm nào trong miền cho loss thấp hơn:

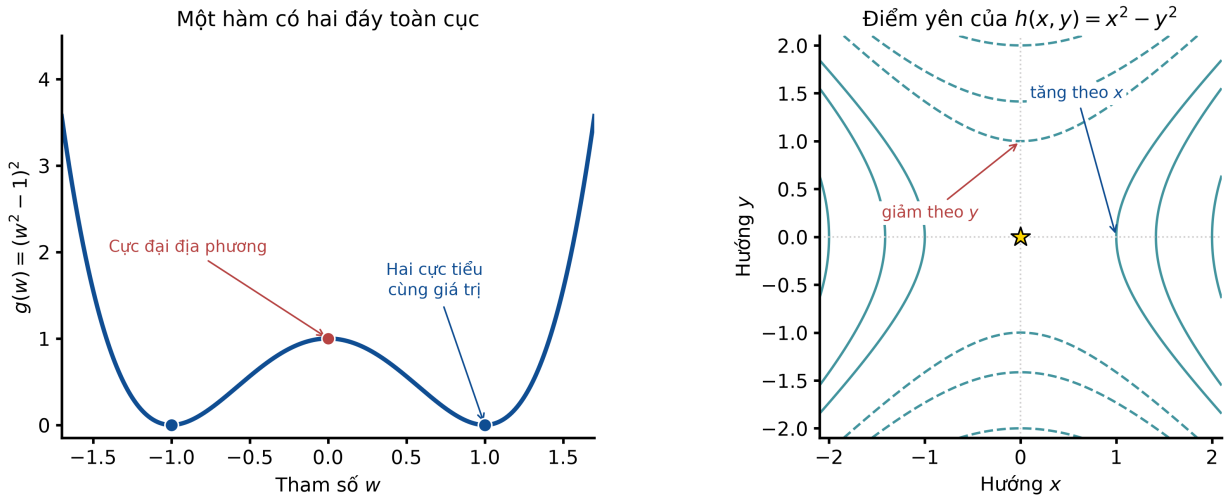
$$J(\theta^*) \leq J(\theta) \quad \text{với mọi } \theta \in \Theta.$$

Một điểm là **cực tiểu địa phương** nếu chỉ cần tốt hơn các điểm đủ gần nó. Có thể hình dung một người đứng trong thung lũng nhỏ: đi một đoạn đủ ngắn theo bất kỳ hướng nào cũng không xuống thấp hơn, nhưng ở xa có thể còn một thung lũng thấp hơn. Mọi cực tiểu toàn cục đều là cực tiểu địa phương theo nghĩa không nghiêm ngặt, nhưng chiều ngược lại không đúng.

Đối với hàm khả vi trên miền mở, điểm cực tiểu địa phương ở bên trong thường có gradient bằng không. Điều ngược lại sai: gradient bằng không chỉ gọi là **điểm dừng**. Điểm dừng có thể là cực tiểu, cực đại, hoặc điểm

yên. Nếu miền có biên, cực tiểu có thể nằm ngay trên biên dù gradient không bằng không; điều kiện gradient bằng không khi đó không áp dụng trực tiếp.

Trong chương này, **điểm yên** là điểm dừng, tức gradient bằng không, nhưng không phải cực tiểu địa phương cũng không phải cực đại địa phương. Mặt yên ngựa là hình dung chuẩn: gần tâm, hàm tăng khi dịch theo một hướng và giảm theo hướng khác. Tính tăng giảm theo các hướng là cách nhận ra ví dụ đó, còn điều kiện định nghĩa là điểm dừng nhưng không phải cực trị.



Hình 2: Hai hố cùng độ sâu và một điểm yên có hướng đi lên lẫn đi xuống (minh hoạ, tự tính)

Cách đọc hình 2: panel trái là mặt cắt một hàm có hai đáy cùng độ sâu, nên cả hai đều là cực tiểu toàn cục. Panel phải là các đường đồng mức của $x^2 - y^2$: tâm là điểm dừng, theo trục x loss tăng còn theo trục y loss giảm. Đường đồng mức không khép thành các vòng quanh tâm vì điểm đó không phải đáy.

6. Ví dụ tính tay: tìm đáy của loss bình phương

Ta làm từng bước với hai nhãn $y_1 = 1$ và $y_2 = 3$. Mô hình chỉ có một tham số w và dự đoán $f_w(x) = w$, không phụ thuộc x. Dùng bình phương sai lệch rồi lấy trung bình:

$$L(w) = \frac{1}{2}\left[(w-1)^2 + (w-3)^2\right].$$

Khai triển từng bình phương: $(w-1)^2 = w^2 - 2w + 1$ và $(w-3)^2 = w^2 - 6w + 9$. Cộng lại được $2w^2 - 8w + 10$; chia 2 được $L(w) = w^2 - 4w + 5$. Hoàn thành bình phương:

$$L(w) = (w-2)^2 + 1.$$

Do $(w-2)^2$ không âm với mọi số thực và bằng không khi $w = 2$, nghiệm toàn cục duy nhất là $w^* = 2$, loss nhỏ nhất bằng 1 (tự tính). Tính đạo hàm cho ta $L'(w) = 2w - 4$. Giải $L'(w) = 0$ cũng cho $w = 2$, và đạo hàm bậc hai là $L''(w) = 2 > 0$, xác nhận đường cong cong lên tại điểm đó (tự tính).

Thay ngược nghiệm vào từng mẫu: dự đoán là 2; sai lệch trên nhãn 1 bằng 1, trên nhãn 3 bằng -1. Bình phương mỗi lỗi đều bằng 1, nên trung bình cũng bằng 1 (tự tính). Điều này kiểm tra rằng nghiệm của công thức khớp với cách hiểu dữ liệu.

Ví dụ này còn cho thấy nghiệm tối ưu phụ thuộc vào lựa chọn loss. Nếu dùng sai lệch tuyệt đối thay vì bình phương, mọi w nằm giữa hai nhãn sẽ có cùng tổng sai lệch tuyệt đối; nghiệm không còn duy nhất. Chọn loss

không chỉ là thay ký hiệu trong phép tính, nó quyết định lỗi nào bị phạt nặng và hình dạng của mục tiêu.

7. Ba hàm nhỏ để phân biệt các loại điểm

Xét hàm một chiều $g(w) = (w^2 - 1)^2$. Hàm không âm, bằng không tại $w = -1$ và $w = 1$, nên hai điểm này là cực tiểu toàn cục cùng giá trị. Tại $w = 0$, hàm có giá trị 1; ở hai phía gần đó giá trị nhỏ hơn, nên tâm là cực đại địa phương. Đạo hàm là $g'(w) = 4w(w^2 - 1)$, do đó cả ba điểm đều dừng, nhưng chỉ hai trong số đó là cực tiểu (tự tính).

Để thấy điểm yên, xét $h(x, y) = x^2 - y^2$. Gradient là $(2x, -2y)$ và bằng $(0, 0)$ tại gốc. Nếu đi từ gốc theo trục x, giá trị hàm dương; nếu đi theo trục y, giá trị âm. Vì vậy gốc không thể là cực tiểu hay cực đại (tự tính).

Cuối cùng, $q(w) = w^2$ có một cực tiểu toàn cục tại $w = 0$. Đây là ví dụ lồi và đơn giản. Khi hàm lồi trên một miền lồi, mọi cực tiểu địa phương cũng là cực tiểu toàn cục. Với hàm không lồi, bảo đảm ấy mất đi; việc thuật toán tìm được một đáy không tự chứng minh rằng không còn đáy tốt hơn.

Các ví dụ này dùng đạo hàm để kiểm tra điểm dừng, nhưng không phải hàm loss nào cũng khả vi ở mọi nơi. Loss tuyệt đối có góc nhọn tại điểm bằng không, còn ReLU có điểm gãy tại gốc. Khi đó ta có thể dùng đạo hàm dưới hoặc quy tắc tối ưu thích hợp, thay vì giả vờ rằng đạo hàm thường luôn tồn tại.

8. Nghiệm có tồn tại không?

Một hàm có thể có cận dưới nhưng không đạt được cận dưới. Ví dụ $r(w) = e^w$ trên toàn trục thực luôn dương và tiến dần về 0 khi w giảm rất xa, nhưng không có w hữu hạn nào cho giá trị bằng 0. Infimum bằng 0, song minimizer không tồn tại. Ngược lại, hàm $s(w) = w$ trên toàn trục không bị chặn dưới, nên không có cả giá trị nhỏ nhất lẫn cận dưới hữu hạn.

Trong bài toán hữu hạn chiều, một kết quả quen thuộc cho ta điều kiện đủ dễ nhớ: hàm liên tục trên một miền đóng và bị chặn sẽ đạt cực tiểu toàn cục. Ý này không đảm bảo cách tìm nghiệm nhanh, và cũng không tự cho biết nghiệm đó hữu ích trên dữ liệu mới. Nó chỉ trả lời câu hỏi về sự tồn tại.

Nhiều mô hình thực tế có miền tham số không bị chặn. Regularization có thể làm mục tiêu tăng mạnh khi tham số lớn, nhờ đó tạo thêm tính chất thuận lợi, nhưng không phải phép cộng regularizer nào cũng bảo đảm mọi giả thiết cần thiết. Khi đọc một định lý hội tụ, cần xem rõ giả thiết về tính liên tục, lồi, trơn, miền tham số và việc có hay không có ràng buộc.

9. Các biến thể của mục tiêu

Tổng và trung bình. Nếu cùng một tập mẫu, tổng loss và loss trung bình có cùng minimizer khi số mẫu cố định và dương. Tuy nhiên, gradient của tổng lớn hơn gradient trung bình theo hệ số bằng số mẫu. Vì bước cập nhật dùng learning rate nhân gradient, giữ nguyên learning rate khi đổi tổng sang trung bình có thể làm quỹ đạo khác đáng kể.

Loss cộng regularization. Mục tiêu $J = L + \lambda R$ cân bằng độ khớp và một ưu tiên khác. Khi $\lambda = 0$, regularizer không ảnh hưởng. Khi tăng λ, ảnh hưởng của nó tăng tương đối so với loss dữ liệu. Không nên diễn giải R như một phép màu chống overfitting; hiệu quả phụ thuộc dạng regularizer, dữ liệu và mô hình.

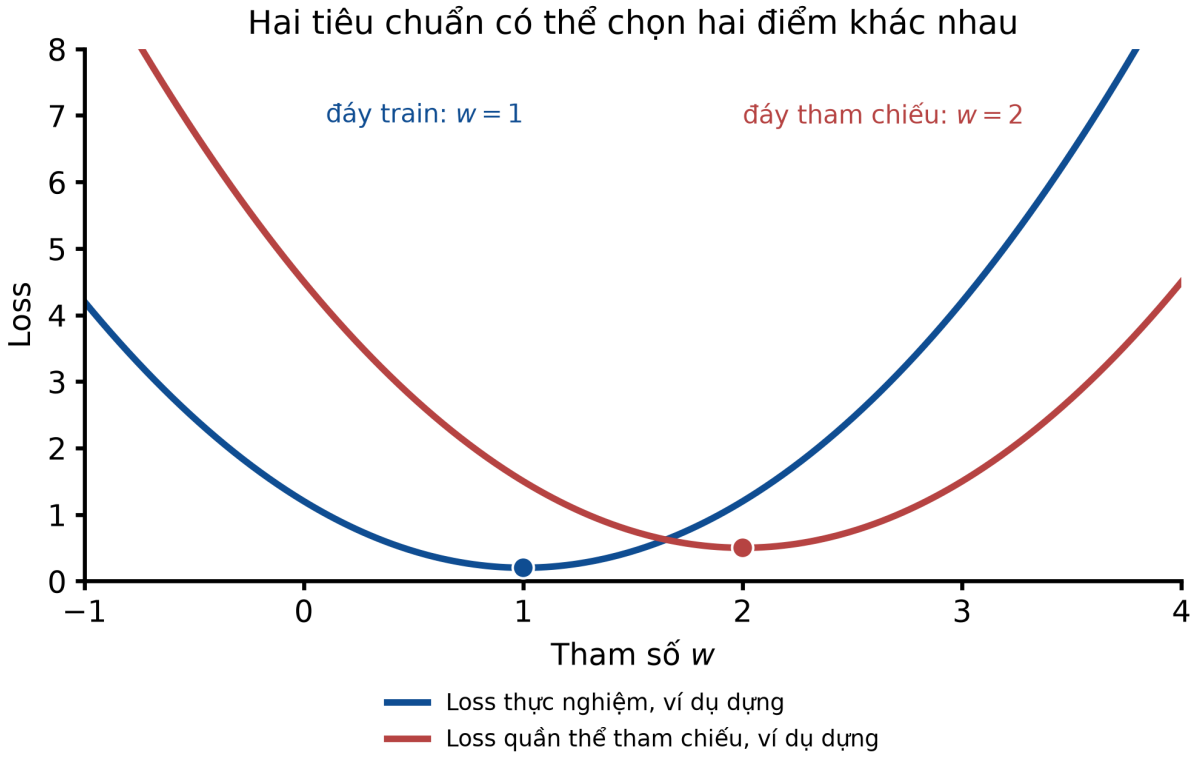
Ràng buộc tường minh. Thay vì phạt vi phạm, ta có thể chỉ cho phép θ thuộc một miền Θ, ví dụ trọng số có chuẩn không vượt một ngưỡng. Khi bước cập nhật đi ra ngoài miền, cần một cách xử lý như chiếu về miền hợp lệ. Đây là bài toán khác với tối ưu không ràng buộc và sẽ có điều kiện dừng khác.

Mục tiêu trơn và không trơn. Hàm trơn có đạo hàm biến thiên liên tục hoặc bị chặn theo cách phù hợp, giúp kiểm soát sai số của xấp xỉ tuyến tính. Hàm không trơn vẫn tối ưu được, nhưng điểm có góc hoặc thay đổi đột ngột cần ngôn ngữ và thuật toán tương ứng. Không nên gọi mọi hàm loss là “parabol” chỉ vì một ví dụ bình phương có dạng đó.

Nhiều nghiệm và đối xứng. Hai bộ tham số khác nhau có thể tạo cùng dự đoán hoặc cùng loss, chẳng hạn khi hoán đổi thứ tự neuron trong một tầng. Vì vậy, nghiệm tối ưu không nhất thiết duy nhất, và khoảng cách giữa hai vector tham số không phải lúc nào cũng phản ánh khoảng cách giữa hai mô hình.

10. Đọc hình dạng hàm mục tiêu

Khi có hai tham số, ta có thể vẽ đường đồng mức: mỗi đường nối các điểm có cùng loss. Các vòng gần tròn gợi ý độ cong giống nhau theo nhiều hướng; các vòng dẹt gợi ý có hướng thay đổi nhanh và hướng thay đổi chậm. Hình dạng này sẽ quyết định cách bước cập nhật hoạt động, là chủ đề của chương Gradient descent.



Hình 3: Các mức loss từ tập mẫu và một mục tiêu tham chiếu trong ví dụ một chiều (minh hoạ, tự tính)

Cách đọc hình 3: đường xanh là một loss thực nghiệm được dựng để minh hoạ, còn đường đỏ là một mục tiêu quần thể tham chiếu. Hai đường có thể đạt đáy tại các giá trị tham số khác nhau; vì thế giảm loss huấn luyện chưa tự động chứng minh đã giảm lỗi trên dữ liệu mới. Các đường không phải kết quả từ một thí nghiệm hay một paper cụ thể.

Trong mô hình lớn, mỗi tọa độ là một tham số và bề mặt có thể có rất nhiều chiều. Biểu đồ hai chiều chỉ là lát cắt hoặc ví dụ đồ chơi, không phải bản đồ đầy đủ. Cắt bề mặt theo các hướng khác nhau có thể cho hình rất khác; vì thế một hình contour đẹp không đủ để kết luận toàn bộ bài toán lồi hay dễ tối ưu.

11. Nối sang học máy

Huấn luyện là tối ưu một tiêu chuẩn cụ thể. Mô hình, dữ liệu, loss và regularizer cùng xác định hàm mục tiêu. Khi thay loss, thay cách lấy trung bình, hoặc thêm trọng số lớp, ta đã đổi bài toán. Khi so sánh optimizer, cần giữ rõ mục tiêu nào đang được đo.

Training loss khác khả năng tổng quát hoá. Nghiệm giảm loss trên các mẫu đã thấy có thể ghi nhớ chi tiết của tập huấn luyện. Validation và test dùng để đo chất lượng trên dữ liệu giữ lại theo quy trình riêng; chúng

không nên bị trộn vào mục tiêu huấn luyện một cách vô ý. Chủ đề overfitting được bàn sâu hơn trong sách ML cơ bản.

Mạng nơ-ron thường không lồi theo tham số. Tầng tuyến tính riêng lẻ có cấu trúc đơn giản, nhưng ghép nhiều tầng cùng hàm kích hoạt tạo ra mục tiêu phi tuyến và thường không lồi. Vì vậy, các bảo đảm dành cho bài toán lồi trong optimization 01 là nền tảng quan trọng, song không tự động áp dụng cho mọi mạng sâu.

Loss landscape và cực tiểu. Các bài optimization 24 và optimization 27 bàn cách nhìn hình dạng quanh điểm nghiệm; optimization 26 xem xét hành vi của gradient descent trong một chế độ huấn luyện cụ thể. Những chủ đề đó dùng ngôn ngữ điểm dừng và độ cong, nhưng không nên suy rộng kết quả một cấu hình thành định luật cho mọi mô hình.

Optimizer là phương pháp tìm kiếm. Mục tiêu quy định ta muốn giảm cái gì; optimizer quy định ta chọn các bước thay đổi tham số ra sao. Gradient descent ở chương kế tiếp dùng gradient của hàm mục tiêu để chọn hướng đi. Một optimizer tốt không sửa được mục tiêu được định nghĩa sai.

12. Sai lầm hay gặp

- **Đồng nhất minimizer với giá trị nhỏ nhất.** $\arg\min$ là tập các tham số đạt đáy, còn $\min$ là con số ở đáy. Hai đại lượng có đơn vị và ý nghĩa khác nhau.
- **Cho rằng gradient bằng không đồng nghĩa với cực tiểu.** Điểm dừng còn có thể là cực đại hoặc điểm yên. Cần xem hình dạng theo các hướng lân cận.
- **Gọi mọi cực tiểu địa phương là cực tiểu toàn cục.** Điều này đúng với hàm lồi trên miền lồi, nhưng không đúng chung cho hàm không lồi.
- **Nghĩ rằng thuật toán luôn tìm được nghiệm chính xác.** Ta thường chỉ chạy hữu hạn bước, nên nhận một điểm xấp xỉ. Điều kiện dừng và sai số chấp nhận phải được nói rõ.
- **Tin rằng loss train thấp đảm bảo dự đoán tốt ngoài mẫu.** Nó chỉ đo mức khớp với tiêu chuẩn và dữ liệu huấn luyện đã chọn.
- **Quên miền tối ưu.** Một điểm tốt trong không gian không ràng buộc có thể không hợp lệ nếu tham số phải thuộc một miền; ngược lại, nghiệm có thể nằm ở biên.

13. Tóm tắt ký hiệu và công thức

Khái niệm	Công thức	Cách hiểu
Dự đoán	$f_\theta(x)$	mô hình tại tham số θ nhận đầu vào x
Loss một mẫu	$\ell(f_\theta(x), y)$	độ phạt cho một dự đoán so với nhãn
Rủi ro thực nghiệm	$L_{\text{train}}(\theta) = \frac{1}{n}\sum_i \ell(f_\theta(x_i), y_i)$	loss trung bình trên mẫu huấn luyện
Mục tiêu có regularization	$J(\theta) = L_{\text{train}}(\theta) + \lambda R(\theta)$	khớp dữ liệu cộng một mức phạt
Nghiệm tối ưu	$\theta^* \in \arg\min_{\theta\in\Theta} J(\theta)$	tham số đạt giá trị nhỏ nhất trên miền
Cực tiểu địa phương	$J(\theta^*) \le J(\theta)$ với mọi θ đủ gần	không có điểm lân cận nào tốt hơn
Cực tiểu toàn cục	$J(\theta^*) \le J(\theta)$ với mọi $\theta \in \Theta$	không điểm hợp lệ nào tốt hơn
Điểm dừng khả vi	$\nabla J(\theta) = 0$	điều kiện cần ở một số cực tiểu bên trong, chưa đủ để kết luận
Rủi ro quần thể	$L_{\text{pop}}(\theta) = \mathbb{E}_{(x,y)\sim P}[\ell(f_\theta(x), y)]$	loss trung bình theo phân phối dữ liệu

14. Bài tập tự tính

Hãy thử làm từng câu trước khi xem lời giải. Các con số trong đáp án là ví dụ tự tính và đã được kiểm bằng numpy.

Bài 1 (dễ). Mô hình dự đoán một số w cho nhãn duy nhất $y = 4$, dùng bình phương sai lệch. Viết loss và tìm nghiệm toàn cục.

Đáp án. $L(w) = (w - 4)^2$. Bình phương luôn không âm và bằng không tại $w = 4$, nên $w^* = 4$ và loss nhỏ nhất là 0 (tự tính).

Bài 2 (dễ vừa). Với hai nhãn 2 và 6, mô hình dự đoán một số chung w. Dùng loss là trung bình bình phương sai lệch. Tìm loss dưới dạng bình phương hoàn chỉnh và tìm nghiệm.

Đáp án. $L(w) = \frac{1}{2}[(w - 2)^2 + (w - 6)^2] = w^2 - 8w + 20 = (w - 4)^2 + 4$. Do đó nghiệm toàn cục là $w^* = 4$ và loss nhỏ nhất bằng 4 (tự tính). Kiểm tra trực tiếp: hai sai lệch là 2 và -2, bình phương đều bằng 4.

Bài 3 (vừa). Với $g(w) = (w^2 - 4)^2$, tìm các điểm có đạo hàm bằng không và cho biết điểm nào là cực tiểu toàn cục.

Đáp án. $g'(w) = 4w(w^2 - 4)$, nên điểm dừng là $w = -2, 0, 2$. Hàm không âm và bằng không tại $w = -2$ và $w = 2$, nên hai điểm đó là cực tiểu toàn cục. Tại $w = 0$, giá trị là 16; điểm gần gốc nhưng khác gốc có giá trị nhỏ hơn, nên gốc là cực đại địa phương (tự tính).

Bài 4 (vừa khó). Xét $h(x, y) = 3x^2 - y^2$. Tìm gradient tại gốc và xét chiều thay đổi theo hai trục tọa độ.

Đáp án. $\nabla h(x, y) = (6x, -2y)$ nên gradient tại $(0, 0)$ bằng $(0, 0)$. Trên trục x, $h(x, 0) = 3x^2 > 0$ khi $x \neq 0$; trên trục y, $h(0, y) = -y^2 < 0$ khi $y \neq 0$. Do có hướng tăng và hướng giảm, gốc là điểm yên (tự tính).

Bài 5 (khó, lý thuyết). Giải thích vì sao một điểm đạt cực tiểu toàn cục cũng là cực tiểu địa phương, nhưng một cực tiểu địa phương chưa chắc toàn cục.

Đáp án. Điều kiện toàn cục so sánh với mọi điểm trong miền, nên đương nhiên đúng với tập con gồm các điểm đủ gần. Chiều ngược lại không đúng. Chẳng hạn, đặt $p(w) = \min\{(w + 1)^2 + 1, (w - 1)^2\}$. Gần $w = -1$, nhánh đầu thấp hơn nhánh hai, có đáy tại $w = -1$ với giá trị 1. Tại $w = 1$, nhánh hai bằng 0, là giá trị toàn cục nhỏ nhất vì cả hai nhánh đều không âm. Vậy $w = -1$ là cực tiểu địa phương nhưng không toàn cục (tự tính).

Tài liệu tham khảo nền: Bubeck, Convex Optimization: Algorithms and Complexity; Bottou, Curtis và Nocedal, Optimization Methods for Large-Scale Machine Learning (optimization 02); Goodfellow, Bengio và Courville, Deep Learning.

Nền tảng 02

Gradient descent: đi ngược gradient với bước đủ nhỏ để loss giảm

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: Chương nền 01 về hàm mục tiêu, nghiệm và điểm dừng; cần biết đạo hàm và vector ở mức cơ bản.

Dùng ở: bài optimization 01, optimization 02, optimization 03, optimization 04, optimization 26, optimization 47, optimization 49

> Đây là chương nền để học từ đầu, không phải bản dịch paper. Các ví dụ và phép tính ghi "tự tính" đã được kiểm bằng numpy.

1. Vì sao cần gradient descent

Chương trước biến việc học thành bài toán tìm tham số làm loss nhỏ. Với mô hình thực tế, ta hiếm khi giải được phương trình "đạo hàm bằng không" bằng tay: số tham số lớn, hàm phi tuyến, và dữ liệu cũng có thể rất lớn. Gradient descent là một quy tắc lặp đơn giản: tại tham số hiện tại, đo hướng loss tăng nhanh nhất rồi bước theo hướng ngược lại.

Ý tưởng có thể kiểm tra trên một con dốc. Nếu đứng tại một điểm và biết độ dốc, một bước nhỏ đi xuống dốc sẽ làm độ cao giảm. Nhưng bước dài quá có thể vượt qua đáy rồi leo lên phía bên kia. Vì thế gradient cho ta hướng cục bộ, còn độ dài bước quyết định ta đi xa đến đâu.

Trong huấn luyện máy học, "độ cao" là hàm mục tiêu, vị trí là vector tham số θ, còn hướng dốc là gradient $\nabla f(\theta)$. Thủ tục cập nhật được lặp lại nhiều lần. Nó không hứa tìm cực tiểu toàn cục cho mọi hàm; điều kiện hàm và learning rate quan trọng không kém công thức cập nhật.

2. Trực giác bằng hình: tiếp tuyến và bước đi

Trên một đường cong, đạo hàm tại một điểm là độ dốc của tiếp tuyến tại điểm đó. Nếu đạo hàm dương, đi một lượng nhỏ sang phải làm hàm tăng; đi sang trái làm hàm giảm. Gradient descent ghép các hướng một chiều thành quy tắc cho nhiều tham số.

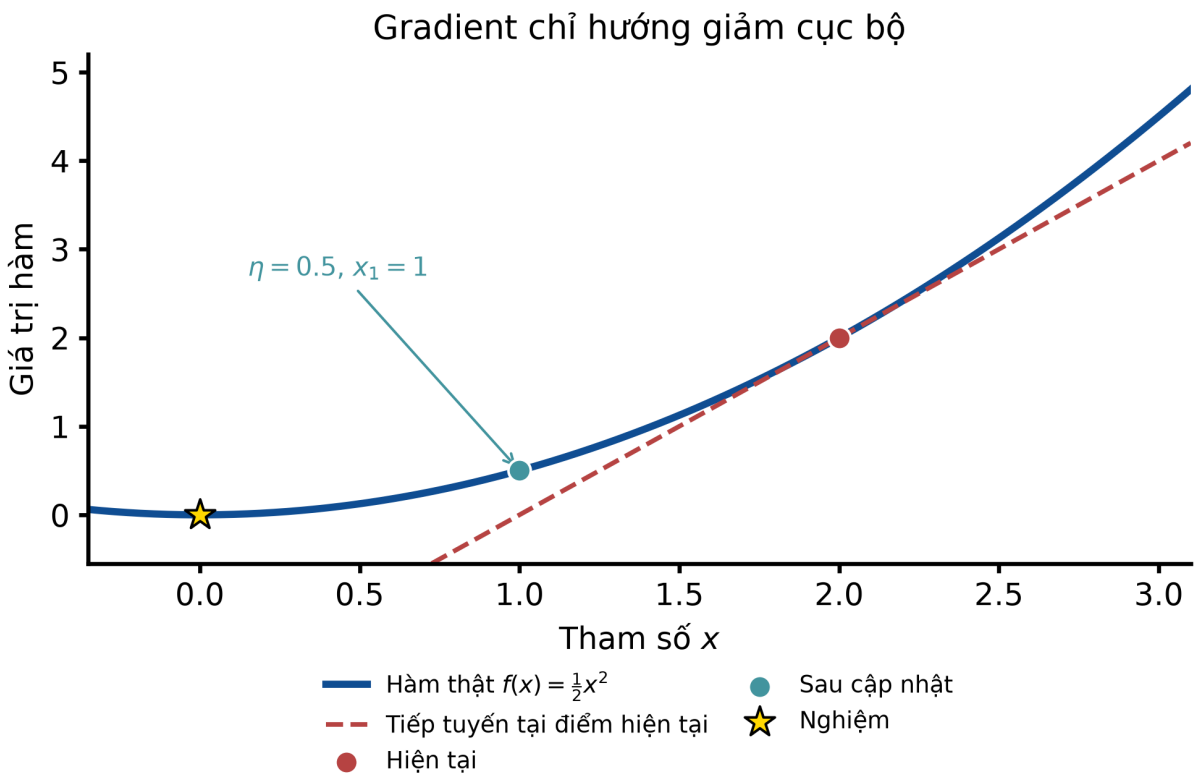
Cách đọc hình 1: chấm đỏ là tham số hiện tại, tiếp tuyến nét đứt đỏ có độ dốc dương nên hướng giảm là sang trái. Chấm xanh ngọc là điểm sau cập nhật; nó nằm gần đáy và loss giảm. Đường nét đứt là xấp xỉ tuyến tính tại điểm cũ; đường cong thật có độ cong nên giá trị ở điểm mới không nhất thiết bằng dự đoán của tiếp tuyến.

Đạo hàm không nói "đáy ở đâu" nếu ta chỉ biết nó tại một điểm. Nó chỉ cho biết hàm đổi ra sao khi dịch chuyển đủ nhỏ quanh điểm đó. Gradient descent dùng thông tin cục bộ lặp đi lặp lại, hy vọng mỗi bước hữu ích đưa ta đến vùng tốt hơn.

3. Quy tắc cập nhật

Với một hàm khả vi $f(\theta)$, gradient descent toàn lô (full-batch gradient descent) cập nhật

$$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t),$$



Hình 4: Tiếp tuyến cho biết hướng giảm cục bộ và một bước gradient descent tiến về đáy (minh hoạ, tự tính)

trong đó t là số bước, θ_t là tham số ở bước hiện tại, và $\eta > 0$ là **learning rate** hay kích thước bước. Dấu trừ chọn hướng giảm cục bộ. Gradient có cùng số tọa độ với tham số; mỗi tọa độ của gradient là đạo hàm riêng theo tham số tương ứng.

Trong bài toán học máy, f thường là loss trên toàn bộ tập huấn luyện hoặc một phiên bản trung bình đã định nghĩa. Tính gradient của cả tập rồi cập nhật một lần là full-batch. Dùng một phần dữ liệu để ước lượng gradient dẫn đến SGD hoặc mini-batch, được bàn trong chương nền kế tiếp. Công thức cập nhật giữ nguyên hình thức, nhưng gradient đưa vào có thể nhiễu.

Quãng đường thực đi trong một bước không chỉ bằng η. Độ dài cập nhật là $\|\theta_{t+1} - \theta_t\|_2 = \eta\|\nabla f(\theta_t)\|_2$. Vì vậy cùng learning rate có thể tạo bước lớn ở vùng dốc và bước nhỏ ở vùng gần phẳng.

4. Vì sao trừ gradient là hướng đi xuống

Gọi $g = \nabla f(\theta)$. Với một dịch chuyển nhỏ Δ, xấp xỉ Taylor bậc nhất cho

$$f(\theta + \Delta) \approx f(\theta) + g^\top \Delta.$$

Nếu ta chỉ được chọn một hướng đơn vị d, mức thay đổi ban đầu là $g^\top d$. Khi $g \neq 0$, bất đẳng thức Cauchy-Schwarz cho $g^\top d \geq -\|g\|_2\|d\|_2 = -\|g\|_2$, và dấu bằng đạt được khi $d = -g/\|g\|_2$. Vậy trong hình học Euclid, hướng âm gradient là hướng giảm nhanh nhất theo đơn vị độ dài. Nếu $g = 0$, mọi hướng có cùng biến thiên tuyến tính bằng không và biểu thức chuẩn hoá này không xác định.

Một cách nhìn khác là đặt $\Delta = -\eta g$. Khi đó số hạng tuyến tính là $-\eta\|g\|_2^2$, không dương. Nếu gradient khác không và bước đủ nhỏ, hàm khả vi sẽ giảm gần như dự đoán. Tuy nhiên, xấp xỉ tuyến tính chỉ đúng cục bộ; độ cong có thể làm thay đổi kết quả khi bước quá xa.

Ví dụ hướng (tự tính): nếu gradient tại một điểm là $(3, 4)$ thì độ dài gradient là 5, hướng đơn vị ngược gradient là $(-3/5, -4/5)$. Tích vô hướng của gradient với hướng này bằng -5, trong khi hướng cùng chiều cho tích vô hướng 5. Vậy một dịch chuyển nhỏ cùng chiều làm loss tăng, còn dịch chuyển ngược chiều làm loss giảm ở bậc nhất.

5. Ví dụ tính tay một chiều

Xét mục tiêu lồi $f(x) = \frac{1}{2}(x-3)^2$. Đạo hàm là $f'(x) = x - 3$, nên đáy ở $x^* = 3$. Bắt đầu tại $x_0 = 0$ và chọn $\eta = 0.5$. Từng bước được tính như sau:

1. $f'(x_0) = -3$, nên $x_1 = 0 - 0.5(-3) = 1.5$.
2. $f'(x_1) = -1.5$, nên $x_2 = 1.5 - 0.5(-1.5) = 2.25$.
3. $f'(x_2) = -0.75$, nên $x_3 = 2.25 - 0.5(-0.75) = 2.625$.

Các bước tiến gần 3 từ phía trái. Thay vào hàm, $f(x_0) = 4.5$, $f(x_1) = 1.125$, $f(x_2) = 0.28125$, và $f(x_3) = 0.0703125$ (tự tính). Mỗi lần sai số tới nghiệm bị nhân với 0.5: $x_{t+1} - 3 = (1-\eta)(x_t - 3)$. Do đó trong ví dụ này $x_t = 3(1 - 0.5^t)$ và loss tiến dần về 0 (tự tính).

Nếu thay learning rate thành 1, bước đầu đi thẳng từ 0 đến 3 và dừng ở nghiệm (tự tính). Nếu dùng 1.5, hệ số sai số là -0.5: các bước đổi bên qua nghiệm nhưng độ lệch giảm một nửa. Nếu dùng đúng 2, hệ số là -1, nên điểm qua lại với cùng khoảng cách và không hội tụ, trừ khi bắt đầu đúng nghiệm. Các con số này sẽ xuất hiện từ điều kiện ổn định của hàm bậc hai.

6. Learning rate: bước nhỏ, bước vừa, bước quá lớn

Learning rate nhỏ thường làm cập nhật bảo thủ: mỗi bước có thể giảm loss nhưng cần nhiều bước để đi xa. Learning rate lớn giúp đi xa hơn, nhưng có thể vượt quá vùng mà xấp xỉ tuyến tính còn đáng tin. Trên một hàm có độ cong mạnh, bước lớn có thể nhảy qua đáy, đổi phía liên tục, hoặc làm loss tăng.

Đây không phải quy tắc "càng nhỏ càng tốt". Nếu bước nhỏ đến mức loss gần như không đổi trong ngân sách thời gian, thuật toán không hữu ích về mặt thực hành. Mục tiêu là chọn bước vừa đủ lớn để tiến triển mà vẫn giữ được hành vi mong muốn. Chương về learning rate sau này sẽ bàn lịch thay đổi bước trong quá trình huấn luyện.

Với hàm có gradient Lipschitz liên tục hằng số L, một bất đẳng thức descent cho biết

$$f(\theta - \eta \nabla f(\theta)) \le f(\theta) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla f(\theta)\|_2^2.$$

Nếu $0 < \eta < 2/L$, vế phải thấp hơn hoặc bằng giá trị hiện tại, và thấp hơn nghiêm ngặt khi gradient khác không. Đây là bảo đảm giảm cho hàm trơn trong một bước, không phải lời hứa rằng ta đã đến cực tiểu toàn cục. Nếu hàm không lồi, ta có thể tiến đến điểm dừng không tối ưu.

7. Hội tụ trên hàm bậc hai

Xét hàm bậc hai quanh nghiệm θ^*:

$$f(\theta) = f(\theta^*) + \frac{1}{2}(\theta - \theta^*)^\top H (\theta - \theta^*),$$

trong đó H là ma trận Hessian đối xứng xác định dương. Gọi $e_t = \theta_t - \theta^*$ là sai số tham số. Gradient là He_t, nên cập nhật tạo

$$e_{t+1} = (I - \eta H) e_t.$$

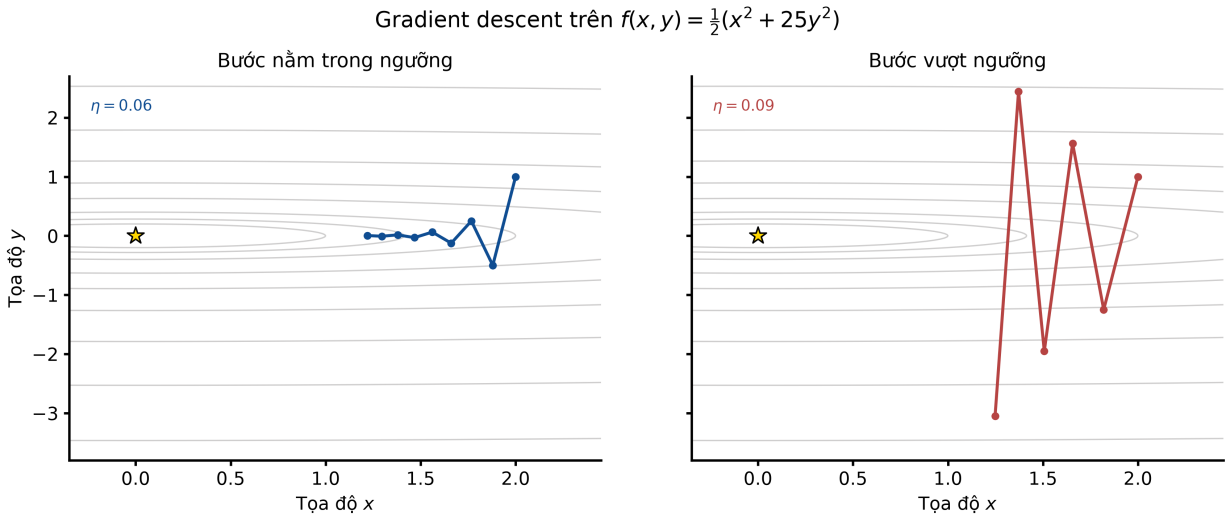
Vì H đối xứng, ta có thể phân tích theo các hướng riêng trực giao. Trên hướng có trị riêng $\lambda_i > 0$, thành phần sai số nhân với $1 - \eta\lambda_i$ sau mỗi bước. Muốn thành phần đó co lại về không cần $|1 - \eta\lambda_i| < 1$, tương đương $0 < \eta < 2/\lambda_i$.

Để mọi hướng đều co lại, bước cố định phải thỏa

$$0 < \eta < \frac{2}{\lambda_{\max}(H)}.$$

Ở biên $\eta = 2/\lambda_{\max}$, thành phần ứng với trị riêng lớn nhất đổi dấu mà không giảm độ lớn; bước lớn hơn làm thành phần đó tăng dần. Đây là phát biểu chính xác đằng sau lời khuyên "đừng chọn bước quá lớn" cho một quadratic xác định dương.

Hình contour làm rõ điểm này. Các đường đồng mức dẹt biểu thị độ cong rất khác nhau theo hai hướng. Bước đủ nhỏ có thể đi xuống nhưng zigzag; bước lớn hơn ngưỡng ổn định thì một tọa độ phóng đại qua mỗi lần cập nhật.



Hình 5: Quỹ đạo trên cùng một hàm bậc hai khi bước nằm trong và ngoài miền ổn định (minh hoạ, tự tính)

Cách đọc hình 2: hai panel dùng contour của cùng hàm $f(x, y) = \frac{1}{2}(x^2 + 25y^2)$ và cùng điểm bắt đầu. Bên trái, learning rate 0.06 nhỏ hơn ngưỡng $2/25 = 0.08$, nên tọa độ dốc hơn đổi dấu nhưng co lại. Bên phải, learning rate 0.09 vượt ngưỡng, các điểm rời xa đáy theo trục dốc; số ngưỡng được tính trực tiếp từ trị riêng lớn nhất (tự tính).

8. Số điều kiện: vì sao có hướng đi nhanh và hướng đi chậm

Với Hessian xác định dương, trị riêng lớn nhất $L = \lambda_{\max}$ là độ cong lớn nhất, còn trị riêng nhỏ nhất $\mu = \lambda_{\min}$ là độ cong nhỏ nhất. **Số điều kiện** của bài toán bậc hai là

$$\kappa = \frac{L}{\mu}.$$

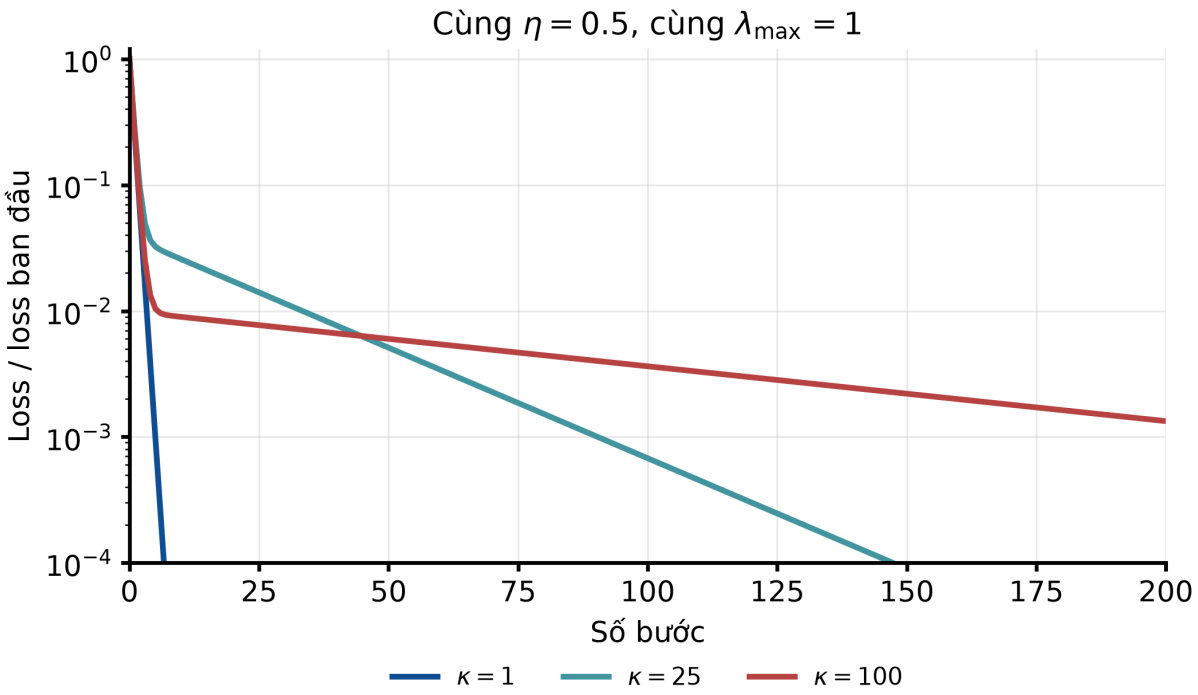
Nếu κ gần 1, độ cong các hướng tương tự nhau. Nếu κ lớn, có hướng rất dốc và hướng khác rất phẳng. Bước η phải đủ nhỏ cho hướng dốc nhất; cùng bước đó vì vậy có thể di chuyển chậm dọc hướng phẳng.

Với một learning rate cố định, hệ số co lớn nhất theo chuẩn phổ là

$$\rho(\eta) = \max_i |1 - \eta\lambda_i|.$$

Trên quadratic lồi mạnh, lựa chọn cố định tối ưu hệ số co tệ nhất theo các hướng riêng là $\eta = 2/(L+\mu)$, cho $\rho = (\kappa-1)/(\kappa+1)$. Khi số điều kiện lớn, hệ số này gần 1, tức mỗi bước chỉ giảm sai số một lượng nhỏ theo hướng chậm nhất. Công thức giả định ta biết các trị riêng cực trị và dùng một bước cố định; nó không phải lời khuyên áp nguyên xi vào mọi mạng nơ-ron, cũng không đảm bảo loss giảm nhiều nhất ngay trong bước đầu từ mọi điểm khởi tạo.

Trong mô hình nhiều tham số, hình học không nhất thiết là một chiếc bát bậc hai hoàn hảo. Hessian thay đổi theo vị trí, có thể có trị riêng âm nếu đang ở vùng lõm, và những hướng khác nhau tương tác. Số điều kiện vẫn là trực giác hữu ích cục bộ, nhưng cần nói rõ ma trận nào và tại điểm nào được dùng để tính nó.



Hình 6: Về cuối quỹ đạo, hướng phẳng co chậm hơn khi số điều kiện tăng (minh hoạ, tự tính)

Cách đọc hình 3: mỗi đường là một quadratic có trị riêng lớn nhất bằng 1, cùng điểm bắt đầu và cùng $\eta = 0.5$; trục dọc là loss chia cho loss ban đầu. Đường đỏ $\kappa = 100$ giảm mạnh ở đầu do thành phần theo hướng dốc bị triệt nhanh, nhưng về cuối nó chậm hơn đường xanh ngọc $\kappa = 25$ vì hướng phẳng co chậm hơn. Đây là các hàm dựng để minh hoạ số điều kiện, không phải phép đo trên một mô hình huấn luyện.

9. Ví dụ tính tay về số điều kiện

Xét

$$f(x, y) = \frac{1}{2}(x^2 + 25y^2).$$

Hessian là $H = \begin{pmatrix} 1 & 0 \\ 0 & 25 \end{pmatrix}$, nên độ cong theo hướng x bằng 1, theo hướng y bằng 25, và số điều kiện $\kappa = 25/1 = 25$ (tự tính). Từ điều kiện hội tụ, η phải nhỏ hơn $2/25 = 0.08$ (tự tính).

Bắt đầu tại $(x_0, y_0) = (2, 1)$ với $\eta = 0.06$. Gradient là $(x, 25y)$. Cập nhật đầu tiên cho

$$(x_1, y_1) = (2 - 0.06 \cdot 2,\ 1 - 0.06 \cdot 25) = (1.88, -0.5).$$

Ở bước kế tiếp, $x_2 = 1.88 - 0.06 \cdot 1.88 = 1.7672$ và $y_2 = -0.5 - 0.06 \cdot 25(-0.5) = 0.25$ (tự tính). Tọa độ y đổi dấu và co một nửa mỗi lần; tọa độ x chỉ co theo hệ số 0.94. Vì vậy đường đi ngoằn ngoèo quanh hướng phẳng thay vì thẳng đến gốc.

Giá trị mục tiêu ban đầu là $f(2, 1) = 14.5$; sau bước đầu là $f(1.88, -0.5) = 4.8922$ (tự tính). Bước làm loss giảm rõ rệt, nhưng điều đó không có nghĩa quỹ đạo đã hội tụ nhanh. Một vài bước đầu có thể giảm mạnh do hướng dốc được xử lý trước, rồi phần còn lại bị chi phối bởi hướng phẳng.

Nếu chọn bước tối ưu hệ số co tệ nhất dựa trên $L = 25$ và $\mu = 1$, ta được $\eta = 2/(25 + 1) = 1/13$ và hệ số phổ $12/13$ (tự tính). Nó để lại một hệ số co gần 1 do chênh lệch độ cong lớn, nhưng không nhất thiết cho loss nhỏ nhất sau đúng một bước. Với điểm bắt đầu $(2, 1)$, loss sau bước này xấp xỉ 12.355, cao hơn 4.8922 của bước $\eta = 0.06$ (tự tính).

10. Hội tụ không đồng nghĩa với tìm cực tiểu toàn cục

Trên quadratic xác định dương, điều kiện trên bảo đảm các thành phần sai số co về nghiệm. Với một hàm lồi trơn tổng quát, các kết quả hội tụ cũng có thể cho biết gradient hoặc khoảng cách tới tập nghiệm giảm nếu các giả thiết tương ứng được thỏa.

Với hàm không lồi nhưng có gradient L-Lipschitz với $L > 0$, dùng learning rate cố định thỏa $0 < \eta < 2/L$, và mục tiêu bị chặn dưới, bất đẳng thức ở mục 6 bảo đảm loss giảm. Cộng các bất đẳng thức qua các bước cho thấy $\frac{1}{T}\sum_{t=0}^{T-1} \|\nabla f(\theta_t)\|_2^2 \to 0$ khi T tăng (tự tính từ điều kiện trơn). Đây là kết quả về điểm gần dừng, không chứng minh điểm đó là cực tiểu toàn cục. Điều kiện bước là cần thiết: trên $f(x) = \frac{1}{2}x^2$, $\eta = 2$ làm điểm đổi dấu mà giữ nguyên độ lớn, nên không hội tụ. Một điểm yên cũng có gradient bằng không; ở vùng có độ cong âm, bước cập nhật có thể bị hút hoặc đẩy tùy hình học và nhiễu.

Trong huấn luyện mạng nơ-ron, thực hành thường dừng sau một ngân sách hữu hạn. Ta theo dõi loss, độ lớn gradient, validation và thời gian; ta không giải bài toán đến độ chính xác toán học vô hạn. Việc chọn điều kiện dừng là một phần của quy trình tối ưu, không phải chi tiết phụ.

11. Các biến thể và chi tiết hay bị bỏ qua

Gradient descent toàn lô. Mỗi bước dùng toàn bộ ví dụ để tính gradient của mục tiêu trung bình. Gradient ổn định theo dữ liệu hiện có, nhưng một lần cập nhật có thể tốn kém khi tập quá lớn.

Bước cố định và bước thay đổi. Phân tích quadratic thường giả định một η cố định. Thực tế có thể dùng bước thay đổi theo thời gian, giảm dần hoặc theo lịch. Mỗi lịch thay đổi cần được phân biệt với điều kiện ổn định của một bước cố định.

Chuẩn của gradient. Gradient phụ thuộc cách tham số hoá và đơn vị của từng tọa độ. Đổi đơn vị một tham số có thể làm gradient ở tọa độ đó đổi tỉ lệ, nên cùng một learning rate không còn tương đương. Điều này gợi ra các phương pháp chuẩn hoá hoặc tiền điều kiện, là chủ đề các chương optimizer sau.

Learning rate không phải khoảng cách cố định. Nếu gradient lớn thì cập nhật dài, nếu gradient nhỏ thì ngắn. Trên hàm không lồi, độ dốc có thể lớn ở nơi xa và nhỏ ở vùng phẳng; một learning rate cố định không đảm bảo mọi bước có cùng độ dài.

Hessian chỉ mô tả độ cong cục bộ. Trên quadratic, Hessian không đổi nên phân tích eigenvalue áp dụng ở mọi bước. Với hàm tổng quát, Hessian thay đổi, do đó ước lượng số điều kiện tại một điểm không nhất thiết đại diện cả hành trình.

12. Nối sang học máy

Tối ưu loss huấn luyện. Gradient descent áp dụng được khi có thể tính gradient của mục tiêu. Với mạng nơ-ron, backpropagation tính gradient hiệu quả; công thức cập nhật sau đó vẫn là phép trừ learning rate nhân gradient.

Mini-batch thêm nhiễu vào hướng đi. Khi gradient được tính từ một phần dữ liệu thay vì toàn bộ, hướng cập nhật biến động giữa các batch. Điều này có thể giảm chi phí mỗi bước và thay đổi hành vi tối ưu. Chương nền về SGD sẽ tách riêng phần ước lượng gradient và nguồn nhiễu.

Learning rate là siêu tham số quyết định động lực. Quá nhỏ, huấn luyện có thể tiến triển chậm; quá lớn, loss dao động hoặc mất ổn định. Bài optimization 47 bàn quy trình tinh chỉnh có kỷ luật, còn các bài về lịch learning rate xem xét cách thay đổi η theo bước.

Điều kiện số giúp giải thích hội tụ chậm. Hình dạng thung lũng hẹp và dài khiến gradient descent dùng một bước phải thỏa hướng dốc nhất nhưng đi chậm dọc hướng phẳng. Các optimizer thích nghi và momentum thay đổi cách kết hợp hướng hoặc thang đo, song vẫn phải kiểm tra trên mục tiêu và thiết lập cụ thể.

Không nên kết luận chỉ từ một lát cắt. Biểu đồ hai chiều giúp hiểu cơ chế nhưng mạng thực có nhiều tham số. Bài optimization 26 tập trung vào một chế độ cụ thể của full-batch gradient descent; đó không phải bằng chứng rằng mọi lần huấn luyện đều ở cùng chế độ.

13. Sai lầm hay gặp

- **Đi theo gradient thay vì ngược gradient.** Gradient chỉ hướng tăng nhanh nhất cục bộ; để giảm hàm, cập nhật dấu trừ.
- **Cho rằng dấu trừ tự bảo đảm loss giảm.** Cần bước đủ nhỏ so với độ cong và giả thiết trơn. Learning rate lớn có thể làm loss tăng.
- **Nhầm learning rate với độ dài cập nhật.** Độ dài còn phụ thuộc chuẩn gradient tại điểm hiện tại.
- **Dùng ngưỡng ổn định sai.** Với quadratic xác định dương, cần xét $\lambda_{\max}$ của Hessian; điều kiện là $0 < \eta < 2/\lambda_{\max}$ cho co trong mọi hướng.
- **Nghĩ rằng hội tụ đồng nghĩa với cực tiểu toàn cục.** Với hàm không lồi, điểm dừng có thể là điểm yên hoặc cực tiểu địa phương.
- **Dùng số điều kiện mà không nói ma trận.** κ của Hessian, ma trận dữ liệu và ma trận tiền điều kiện là các đại lượng khác nhau.

14. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Gradient descent	$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$	dịch tham số ngược gradient
Độ dài bước	$\|\theta_{t+1} - \theta_t\|_2 = \eta \|\nabla f(\theta_t)\|_2$	phụ thuộc cả learning rate và gradient
Xấp xỉ tuyến tính	$f(\theta + \Delta) \approx f(\theta) + \nabla f(\theta)^\top \Delta$	mô tả thay đổi cục bộ
Quadratic sai số	$e_{t+1} = (I - \eta H) e_t$	mỗi hướng riêng co theo một hệ số
Điều kiện ổn định	$0 < \eta < 2/\lambda_{\max}(H)$	các hướng của quadratic xác định dương đều co
Số điều kiện	$\kappa = \lambda_{\max}/\lambda_{\min}$	độ chênh độ cong giữa hướng dốc và phẳng
Bước cố định tối ưu	$\eta = 2/(L + \mu)$	tối ưu hệ số co phổ cho quadratic lồi mạnh
Hệ số co tối ưu	$(\kappa - 1)/(\kappa + 1)$	gần một khi bài toán điều kiện kém

15. Bài tập tự tính

Các phép tính số trong lời giải đã được kiểm bằng numpy. Hãy thử tự cập nhật trước, sau đó dùng đáp án để đối chiếu.

Bài 1 (dễ). Với $f(x) = \frac{1}{2}(x-2)^2$, bắt đầu $x_0 = 0$, dùng $\eta = 0.5$. Tính x_1 và $f(x_1)$.

Đáp án. $f'(x) = x - 2$, nên $f'(0) = -2$. Cập nhật cho $x_1 = 0 - 0.5(-2) = 1$. Loss là $f(1) = \frac{1}{2}(1-2)^2 = 0.5$ (tự tính).

Bài 2 (dễ vừa). Với $f(x) = \frac{1}{2}x^2$, viết sai số sau một bước từ x_t và tìm miền learning rate giúp sai số co về không.

Đáp án. Gradient là x_t, nên $x_{t+1} = (1-\eta)x_t$. Sai số co khi $|1-\eta| < 1$, tức $0 < \eta < 2$ (tự tính). Ở $\eta = 2$, dấu đổi nhưng độ lớn giữ nguyên.

Bài 3 (vừa). Với $f(x, y) = \frac{1}{2}(4x^2 + 9y^2)$, tìm Hessian, số điều kiện và điều kiện ổn định của bước cố định.

Đáp án. Hessian là $\mathrm{diag}(4, 9)$, nên $\lambda_{\min} = 4$, $\lambda_{\max} = 9$, $\kappa = 9/4 = 2.25$. Điều kiện ổn định là $0 < \eta < 2/9 \approx 0.2222$ (tự tính).

Bài 4 (vừa khó). Với quadratic ở bài 3, dùng $\eta = 0.1$ và điểm $(1, 1)$. Tính điểm sau một bước và loss trước, sau bước.

Đáp án. Gradient tại $(1, 1)$ là $(4, 9)$, nên điểm mới là $(1 - 0.4, 1 - 0.9) = (0.6, 0.1)$. Loss ban đầu là $\frac{1}{2}(4 + 9) = 6.5$. Loss mới là $\frac{1}{2}(4 \cdot 0.6^2 + 9 \cdot 0.1^2) = 0.765$ (tự tính).

Bài 5 (khó, lý thuyết). Giải thích vì sao tăng learning rate có thể làm hội tụ chậm hơn, dù mỗi lần cập nhật đi xa hơn.

Đáp án. Trên quadratic, mỗi hướng riêng nhân với $1 - \eta\lambda_i$. Khi η tiến gần $2/\lambda_{\max}$, thành phần ở hướng dốc nhất đổi dấu và có độ lớn gần như giữ nguyên. Nếu vượt ngưỡng, thành phần tăng lên. Vì vậy quãng cập nhật dài hơn không đồng nghĩa sai số giảm nhanh hơn; điều kiện ổn định phụ thuộc độ cong lớn nhất.

Tài liệu tham khảo nền: Bubeck, Convex Optimization: Algorithms and Complexity; Nocedal và Wright, Numerical Optimization; Bottou, Curtis và Nocedal, Optimization Methods for Large-Scale Machine Learning (optimization 02); Ruder, An Overview of Gradient Descent Optimization Algorithms (optimization 03).

Nền tảng 03

SGD, mini-batch và nhiễu: học từ một phần dữ liệu ở mỗi bước

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: hàm mất mát, đạo hàm, gradient descent ở mức cơ bản, trung bình và phương sai

Dùng ở: opt 02 (tối ưu quy mô lớn), opt 03 (tổng quan gradient descent), opt 21 (mini-batch lớn), opt 22 (gradient noise scale), opt 23 (batch size và learning rate)

> Đây là chương học từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" là ví dụ minh hoạ và đã được kiểm bằng numpy.

1. Vì sao cần SGD

Một câu để bắt đầu: **SGD dùng gradient ước lượng từ một phần dữ liệu để cập nhật mô hình, thay vì tính gradient trên toàn bộ tập ở mỗi bước.** Chữ stochastic nghĩa là ngẫu nhiên; mini-batch là nhóm mẫu nhỏ được dùng chung cho một lần cập nhật.

Khi huấn luyện một mạng, ta muốn thay đổi các tham số sao cho lỗi trung bình trên dữ liệu huấn luyện giảm. Nếu dữ liệu lớn, một phép tính gradient trên tất cả các mẫu có thể tốn nhiều công và bộ nhớ. Thay vào đó, ta đọc một nhóm, tính gradient của nhóm rồi cập nhật ngay. Sau đó ta lấy nhóm khác và tiếp tục. Một bước có thể rẻ hơn, nhưng hướng đi cũng phụ thuộc vào nhóm vừa được chọn.

Chương này trả lời bốn câu: một nhóm nhỏ nói gì về toàn bộ dữ liệu; vì sao hai nhóm cho hai gradient khác nhau; batch lớn làm độ dao động thay đổi ra sao; và tại sao một đường loss lên xuống không nhất thiết có nghĩa là code sai. Các câu trả lời dựa vào các giả thiết được nói rõ, chứ không biến mọi biểu đồ huấn luyện thành một quy luật chung.

2. Nhắc lại gradient và trung bình

Gọi θ là tham số của mô hình. Nó có thể là một số để dễ vẽ, hay là một vector chứa rất nhiều trọng số. Ký hiệu $\ell_i(\theta)$ là mất mát trên mẫu thứ i. Nếu có n mẫu và mỗi mẫu có trọng số bằng nhau, mất mát trung bình trên tập huấn luyện là

$$F(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta).$$

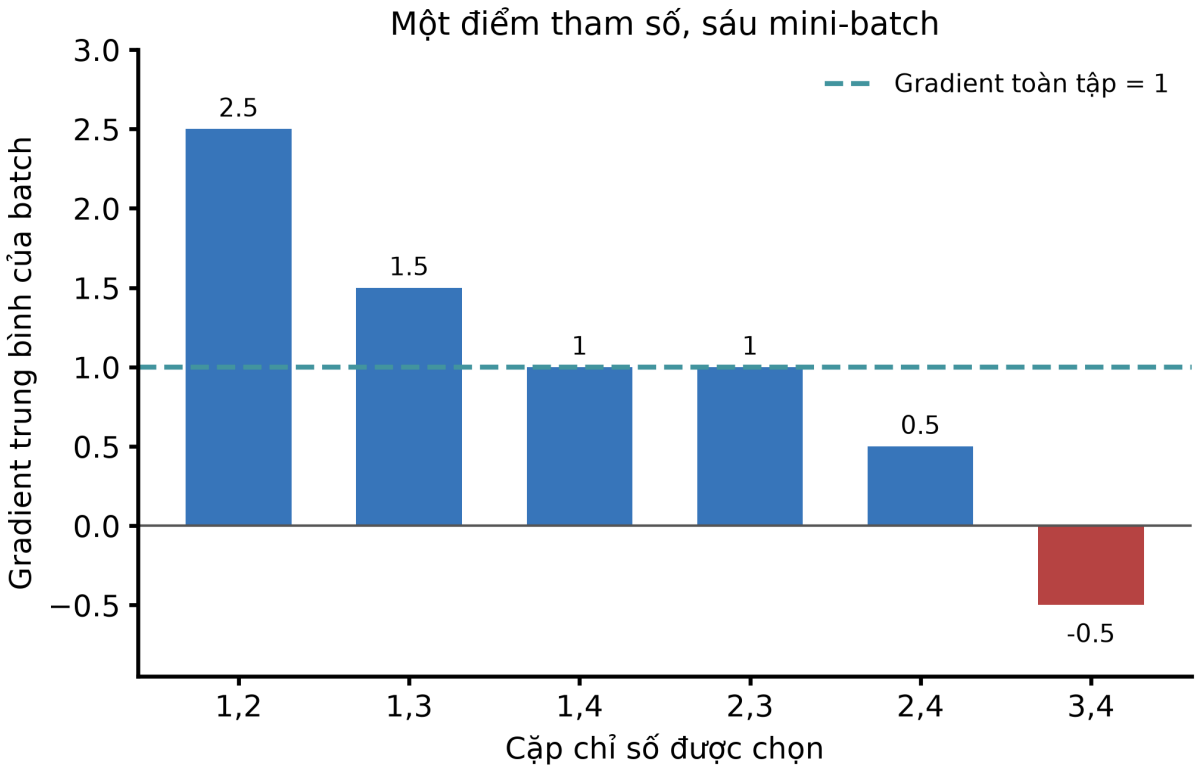
Gradient $\nabla F(\theta)$ chỉ chiều tăng nhanh nhất tại vị trí hiện tại, còn $-\nabla F(\theta)$ chỉ chiều giảm cục bộ. Gradient descent toàn tập, thường gọi full-batch GD, dùng cập nhật $\theta_{t+1} = \theta_t - \eta_t \nabla F(\theta_t)$. Số dương η_t là learning rate, tức độ lớn hệ số bước tại lần cập nhật thứ t.

Lấy ví dụ một chiều (tự tính): nếu $F(\theta) = \frac{1}{2}(\theta - 1)^2$ thì $F'(\theta) = \theta - 1$. Tại $\theta = 2$, đạo hàm bằng 1. Với bước học $0{,}2$, GD đi sang trái đến $1{,}8$. Tại $\theta = 0$, đạo hàm là -1, nên GD đi sang phải đến $0{,}2$. Dấu của đạo hàm cho hướng, nhưng độ lớn của bước còn phụ thuộc learning rate.

Trung bình của các gradient cũng tuân theo quy tắc cộng đạo hàm: $\nabla F(\theta) = \frac{1}{n} \sum_i \nabla \ell_i(\theta)$. Phương sai đo mức các giá trị lệch khỏi trung bình, không nói giá trị trung bình lớn hay nhỏ. Chẳng hạn các số $-2, 0, 2$ có trung bình 0 nhưng không giống nhau; phương sai trung bình của chúng là $\frac{8}{3}$ (tự tính). Điều này quan trọng vì gradient trung bình có thể bằng không trong khi gradient của từng mẫu vẫn khác không.

3. Trực giác qua sáu mini-batch

Giả sử tập có bốn mẫu và tại một vị trí nhất định, bốn gradient theo thứ tự là $3, 2, 0, -1$ (tự tính). Gradient toàn tập là trung bình của cả bốn, bằng 1. Ta chỉ chọn hai mẫu khác nhau cho một lần cập nhật. Có sáu cặp có thể chọn; biểu đồ sáu mini-batch cho thấy gradient trung bình của từng cặp.



Hình 7: Gradient của sáu mini-batch hai mẫu trong một tập bốn mẫu (minh hoạ, tự tính)

Cách đọc hình sáu mini-batch: mỗi cột là một cặp chỉ số, chiều cao là trung bình hai gradient của cặp ấy; đường nét đứt là gradient toàn tập. Cặp $1, 2$ cho $2{,}5$, còn cặp $3, 4$ cho $-0{,}5$, thậm chí trái dấu với gradient toàn tập. Nếu sáu cặp có cùng xác suất, trung bình chiều cao của chúng vẫn bằng 1. Một batch riêng lẻ có thể lệch rất xa, nhưng sự lệch đó không nhất thiết làm trung bình của phép lấy mẫu bị lệch.

Một cách hình dung khác: mỗi mẫu là một người chỉ đường đến chỗ họ thích, còn ta muốn đi theo hướng trung bình của cả nhóm. Hỏi hai người thường nhanh hơn hỏi tất cả, nhưng câu trả lời thay đổi theo hai người được hỏi. Hỏi nhiều người hơn thường làm trung bình bớt dao động, với điều kiện cách chọn và sự phụ thuộc giữa các câu trả lời được xét đúng.

4. Định nghĩa GD, SGD và mini-batch

Ta dùng các ký hiệu sau:

- θ_t: tham số trước lần cập nhật thứ t; $t = 0$ là lúc bắt đầu.
- n: số mẫu trong tập đang xét; B: số lượt lấy mẫu để tạo một mini-batch.
- $i_{t,j}$: chỉ số mẫu ở vị trí j của batch tại bước t.
- g_t: gradient ước lượng từ batch; $\eta_t > 0$: learning rate.

Với một hàm mất mát là trung bình của các mất mát riêng, ta tính

$$g_t = \frac{1}{B}\sum_{j=1}^{B} \nabla \ell_{i_{t,j}}(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta_t g_t.$$

SGD một mẫu là trường hợp $B = 1$. Tên SGD cũng thường được dùng rộng cho trường hợp mini-batch. **Full-batch GD** tính tất cả n gradient tại cùng một θ_t rồi lấy trung bình; điều đó tương ứng chọn toàn bộ tập đúng một lần mỗi mẫu. Nếu lấy $B = n$ **có hoàn lại**, có thể có mẫu lặp và mẫu vắng, nên phép tính chưa chắc là full-batch GD.

Giả sử tại mỗi bước ta lấy các chỉ số đều, độc lập, có hoàn lại và độc lập với lịch sử trước khi lấy batch. Giữ θ_t cố định khi lấy kỳ vọng theo batch mới, ta có

$$\mathbb{E}[g_t \mid \theta_t] = \nabla F(\theta_t).$$

Đó là tính **không chệch có điều kiện**: khi xét tất cả các batch có thể lấy tại cùng vị trí, trung bình gradient batch đúng bằng gradient toàn tập. Nó không nói một batch bất kỳ bằng gradient toàn tập, không nói từng lần cập nhật làm loss giảm, và cũng không nói tham số trung bình tự động là nghiệm tối ưu.

Nếu chọn một tập con đều, không hoàn lại, tại một vị trí cố định thì trung bình của gradient tập con cũng bằng gradient toàn tập. Tuy vậy, cách tạo batch ở các bước liên tiếp ảnh hưởng đến lập luận có điều kiện: mục 7.2 xét kỹ trường hợp xáo trộn dữ liệu theo epoch.

5. Ví dụ tay từ mất mát đến bước cập nhật

Ta dựng bài toán một tham số với bốn mục tiêu $a = (-1, 0, 2, 3)$, mỗi mẫu có mất mát

$$\ell_i(\theta) = \frac{1}{2}(\theta - a_i)^2, \qquad F(\theta) = \frac{1}{4}\sum_{i=1}^{4} \ell_i(\theta).$$

Mọi số trong mục này là ví dụ tự tính và đã kiểm bằng numpy. Ta sẽ không dùng dữ liệu hay kết quả đo từ một paper nào.

5.1 Tìm điểm tối ưu và gradient của từng mẫu

Đạo hàm của từng mất mát là $\ell_i'(\theta) = \theta - a_i$. Trung bình bốn giá trị a_i bằng 1, vì $(-1 + 0 + 2 + 3)/4 = 1$. Do đó $F'(\theta) = \theta - 1$, và nghiệm tối ưu là $\theta_* = 1$. Mất mát tối ưu vẫn lớn hơn không, vì một số θ không thể đồng thời bằng bốn giá trị a_i khác nhau.

Tại vị trí $\theta = 2$, các gradient theo thứ tự mẫu là $3, 2, 0, -1$. Tổng của chúng là 4, nên gradient toàn tập là $4/4 = 1$. Đây chính là bộ giá trị trong biểu đồ sáu mini-batch.

5.2 Liệt kê mọi batch cỡ hai

Chọn hai chỉ số khác nhau với xác suất đều. Bảng dưới liệt kê đủ sáu trường hợp (tự tính):

Hai chỉ số	Gradient của hai mẫu	Trung bình batch
$1, 2$	$3, 2$	2,5
$1, 3$	$3, 0$	1,5
$1, 4$	$3, -1$	1
$2, 3$	$2, 0$	1
$2, 4$	$2, -1$	0,5

Hai chỉ số	Gradient của hai mẫu	Trung bình batch
$3, 4$	$0, -1$	$-0{,}5$

Lấy trung bình sáu cột cuối: $(2{,}5 + 1{,}5 + 1 + 1 + 0{,}5 - 0{,}5)/6 = 1$. Mỗi mẫu xuất hiện ba lần trong sáu cặp, nên trọng số tổng của chúng bằng nhau. Đây là lý do kỳ vọng của gradient batch bằng gradient toàn tập trong ví dụ này.

5.3 So một bước GD với hai batch

Lấy cùng điểm xuất phát $\theta = 2$ và cùng learning rate $\eta = 0{,}2$ (tự tính):

- Toàn tập có gradient 1, nên $\theta_{\text{GD}} = 2 - 0{,}2 \cdot 1 = 1{,}8$.
- Batch $1, 2$ có gradient $2{,}5$, nên $\theta_{12} = 2 - 0{,}2 \cdot 2{,}5 = 1{,}5$.
- Batch $3, 4$ có gradient $-0{,}5$, nên $\theta_{34} = 2 - 0{,}2 \cdot (-0{,}5) = 2{,}1$.

Nghiệm là 1, vì vậy hai kết quả đầu tiến gần nghiệm, còn batch cuối tạm đi xa hơn. Tính trực tiếp toàn bộ bốn mất mát cho $F(2) = 1{,}75$, $F(1{,}8) = 1{,}57$, $F(1{,}5) = 1{,}375$, $F(2{,}1) = 1{,}855$ (tự tính). Một bước mini-batch có thể làm loss toàn tập **tăng**, ngay cả khi gradient của nó không chệch theo nghĩa kỳ vọng.

Không nên rút ra rằng batch $1, 2$ luôn tốt hơn GD. Nó tốt hơn trong đúng một bước và ở đúng điểm này; lần sau gradient mẫu đã thay đổi. Với các hàm khác, bước quá xa cũng có thể làm loss tăng. So sánh thuật toán phải tính cả nhiều bước, nhiều cách lấy mẫu và chi phí tính toán.

6. Nhiễu gradient và phương sai

6.1 Tín hiệu và phần sai khác

Tại cùng một vị trí, viết $\xi_t = g_t - \nabla F(\theta_t)$. Phần ξ_t là **nhiễu gradient do lấy mẫu**, tức sự khác biệt giữa gradient batch và gradient toàn tập. Nếu batch mới được chọn đều và độc lập với lịch sử trước bước hiện tại, thì $\mathbb{E}[\xi_t \mid \theta_t] = 0$, dù một giá trị ξ_t cụ thể thường không bằng không. Chỉ tính đều khi xét riêng phân phối biên của từng chỉ số là chưa đủ nếu chỉ số đó phụ thuộc vào trạng thái hiện tại, như có thể xảy ra khi xáo trộn theo epoch.

Trong ví dụ trước, batch $1, 2$ có nhiễu $2{,}5 - 1 = 1{,}5$; batch $3, 4$ có nhiễu $-0{,}5 - 1 = -1{,}5$ (tự tính). Nhiễu không chỉ là lỗi tính số trên máy: nó tồn tại cả khi mọi phép toán đều chính xác, vì mỗi nhóm quan sát một phần khác nhau của dữ liệu. Các nguồn ngẫu nhiên khác như augmentation hoặc dropout có thể bổ sung nhiễu, nhưng cần xác định chúng trong hàm mục tiêu trước khi nói về tính không chệch.

6.2 Lấy mẫu có hoàn lại

Xét một điểm θ cố định và một thành phần của gradient. Đặt G là gradient của một mẫu được chọn đều, $\mathbb{E}[G] = \mu$ và $\operatorname{Var}(G) = \sigma^2$. Nếu mỗi lượt chọn độc lập và có cùng phân phối, trung bình của B lượt có

$$\operatorname{Var}(g_B) = \operatorname{Var}\left(\frac{1}{B}\sum_{j=1}^{B} G_j\right) = \frac{\sigma^2}{B}.$$

Vì các lượt độc lập, các hiệp phương sai chéo bằng không. Phương sai giảm theo $1/B$, còn độ lệch chuẩn giảm theo $1/\sqrt{B}$. Ví dụ, tăng batch từ 1 lên 4 làm phương sai bằng một phần tư và độ lệch chuẩn bằng một nửa, trong đúng mô hình độc lập đó (tự tính). Cần phân biệt hai đại lượng trước khi nói ”nhiễu giảm bốn lần”.

Với vector gradient, phương sai trở thành ma trận hiệp phương sai $\Sigma(\theta)$. Dưới cùng giả thiết, hiệp phương sai của gradient batch là $\Sigma(\theta)/B$. Mỗi hướng tham số có thể có độ nhiễu khác nhau; một con số gộp không mô tả hết hình dạng nhiễu.

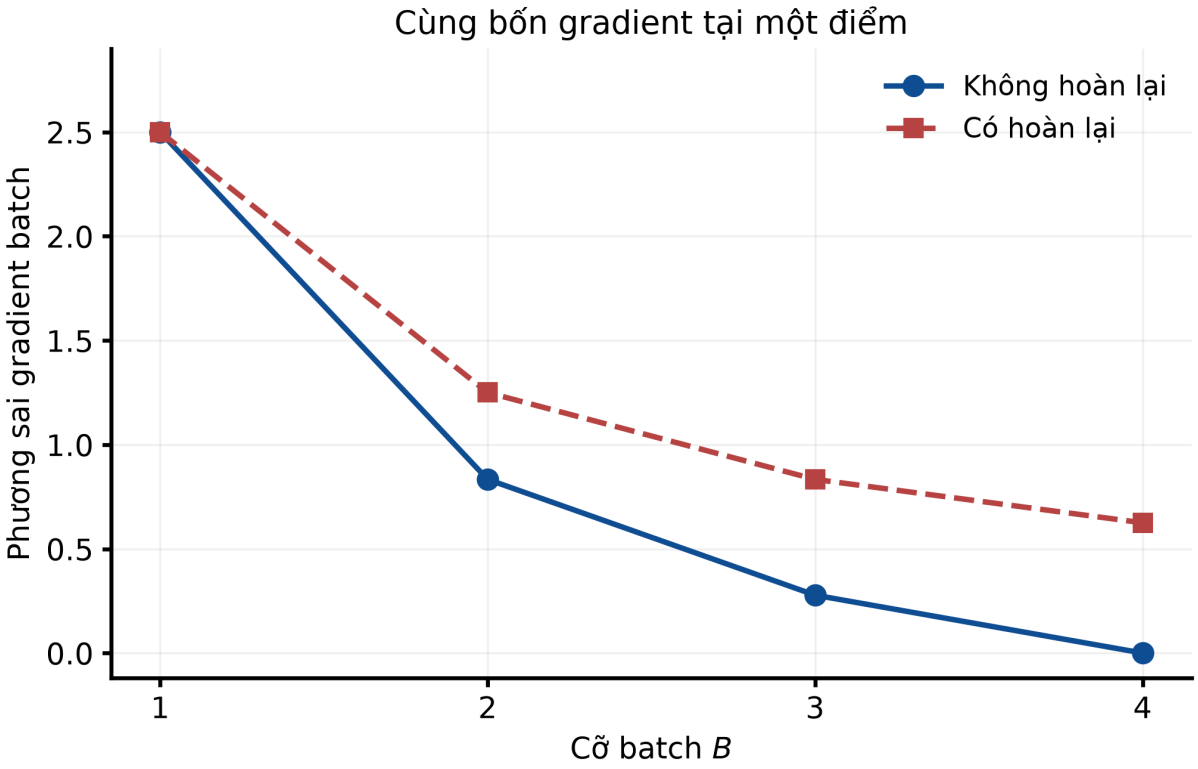
6.3 Lấy mẫu không hoàn lại từ tập hữu hạn

Nếu chọn đều B mẫu khác nhau từ tập $n > 1$ mẫu, các lượt không còn độc lập. Đặt $\sigma^2 = \frac{1}{n}\sum_i (G_i - \mu)^2$, tức chia cho n, ta được

$$\mathrm{Var}(g_B) = \frac{\sigma^2}{B}\,\frac{n-B}{n-1}, \qquad 1 \le B \le n.$$

Hệ số $(n-B)/(n-1)$ là hiệu chỉnh cho tập hữu hạn. Ở $B = 1$ nó bằng 1; khi $B = n$ nó bằng 0, vì ta đã dùng mỗi mẫu đúng một lần. Đừng nhầm công thức này với trường hợp chia phương sai mẫu cho $n-1$: khi đổi cách định nghĩa phương sai thì hệ số viết bên ngoài cũng đổi.

Với bốn gradient của mục 5, độ lệch so với trung bình là $2, 1, -1, -2$; phương sai chia cho n là $(4+1+1+4)/4 = 2{,}5$ (tự tính). Batch cỡ hai không hoàn lại có phương sai $\frac{2{,}5}{2}\frac{4-2}{4-1} = \frac{5}{6}$. Cũng với cỡ hai nhưng lấy có hoàn lại, phương sai là $2{,}5/2 = 1{,}25$ (tự tính). Ta cũng có thể tự kiểm bằng cách liệt kê tất cả batch ở mỗi cách chọn.



Hình 8: So sánh phương sai gradient batch khi lấy có và không hoàn lại (minh hoạ, tự tính)

Cách đọc hình phương sai: trục ngang là số lượt chọn, trục đứng là phương sai của trung bình batch tại cùng một điểm. Đường xanh dùng các mẫu khác nhau; đường đỏ cho phép một mẫu xuất hiện nhiều lần. Hai đường bằng nhau tại batch một mẫu; ở batch bốn mẫu, chỉ đường xanh về không. Bốn gradient ở đây cố định, nên hình không mô tả phương sai thay đổi theo thời gian huấn luyện.

7. Cách lấy dữ liệu quan trọng như công thức

7.1 Có hoàn lại, không hoàn lại và batch cuối

Có hoàn lại nghĩa là chọn xong vẫn có thể chọn lại cùng mẫu, kể cả ngay trong batch. Điều này dễ phân tích vì các lượt có thể độc lập, nhưng một epoch nếu được gọi là n lượt chọn có thể bỏ sót mẫu và lặp mẫu khác.

Không hoàn lại nghĩa là mẫu vừa được chọn không xuất hiện lại trong phần đang xét. Trong thực hành, ta thường xáo trộn thứ tự một lần rồi chia thành các nhóm liên tiếp, mỗi mẫu xuất hiện một lần trong epoch. Khi n không chia hết cho B, batch cuối có thể nhỏ hơn, hoặc bị bỏ đi theo cấu hình. Nếu batch cuối nhỏ, gradient batch vẫn là trung bình của số mẫu thực có, không phải tự chia cho kích thước batch dự kiến.

Epoch thường là một lượt đi qua tập dữ liệu, còn **step** là một lần cập nhật optimizer. Hai đại lượng này không hoán đổi được khi thay đổi batch. Với $n = 100$ và batch 20, đi qua mỗi mẫu một lần tạo năm bước nếu không có phần dư; batch 25 tạo bốn bước (tự tính). Trong thực nghiệm cần ghi rõ lịch học tính theo step, theo số mẫu hay theo epoch.

7.2 Xáo trộn một lần mỗi epoch không giống lấy độc lập

Tại đầu epoch, một hoán vị ngẫu nhiên có thể làm vị trí từng mẫu trông đều. Nhưng khi đã xem vài batch, các mẫu còn lại không còn là một mẫu đều độc lập từ toàn bộ tập. Hơn nữa, θ_t đã thay đổi theo các batch trước nên có liên hệ với thứ tự mẫu còn lại. Vì vậy, không được tự động áp dụng $\mathbb{E}[g_t \mid \theta_t] = \nabla F(\theta_t)$ cho từng bước của mọi quy trình xáo trộn, dù các batch được tạo bằng một hoán vị ngẫu nhiên.

Điều này không làm phương pháp xáo trộn trở nên sai. Nó chỉ cho biết một chứng minh dùng lấy mẫu độc lập không tự động là chứng minh cho hoán vị từng epoch. Trong từng tình huống, ta cần phân biệt giả thiết toán với lựa chọn của bộ nạp dữ liệu.

7.3 Lấy mẫu không đều, dữ liệu phụ thuộc và BatchNorm

Nếu các mẫu có xác suất chọn khác nhau, trung bình không trọng số của batch thường ước lượng mục tiêu có trọng số theo xác suất chọn, không còn là trung bình đều ban đầu. Có thể dùng trọng số hiệu chỉnh thích hợp nếu biết xác suất dương của từng mẫu, nhưng điều đó có thể làm phương sai tăng. Việc lấy mẫu cân bằng lớp vì vậy là một lựa chọn về mục tiêu và ước lượng, không chỉ là đổi thứ tự đọc dữ liệu.

Các đoạn cắt từ cùng một ảnh, token lân cận hoặc chuỗi thời gian gần nhau có thể mang thông tin trùng lặp. Riêng điều đó không làm mất tính độc lập khi ta cố định toàn bộ tập gradient rồi lấy các chỉ số độc lập có hoàn lại: công thức σ^2/B vẫn đúng. Tuy nhiên, nếu cơ chế tạo batch ghép các mẫu phụ thuộc, hoặc nếu xét dữ liệu ngẫu nhiên có phụ thuộc thay vì giữ tập cố định, gradient giữa các lượt có thể tương quan; khi ấy các số hạng hiệp phương sai có thể quan trọng và σ^2/B không còn tự động đúng.

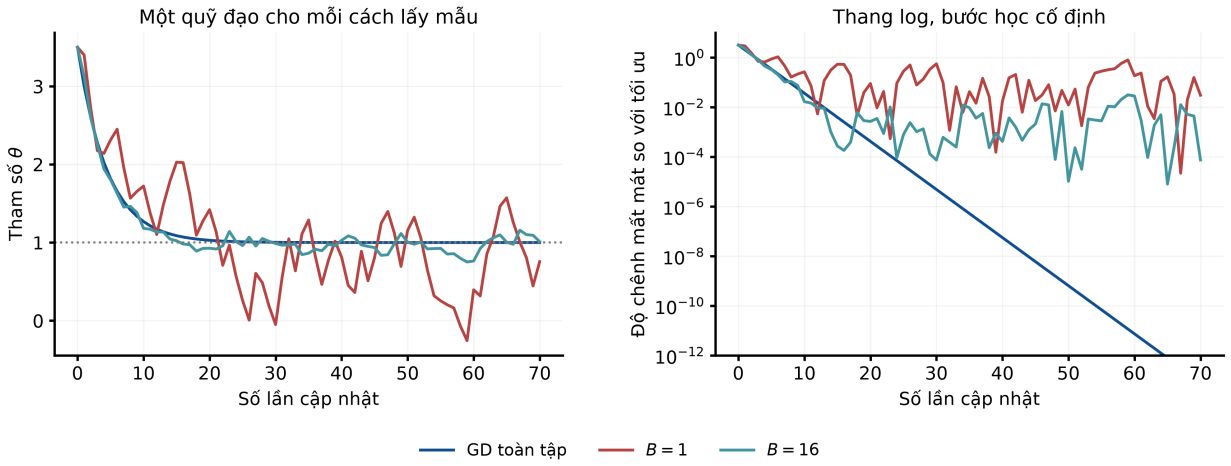
Công thức $F = \frac{1}{n}\sum_i \ell_i$ còn ngầm xem mất mát của một mẫu được xác định riêng tại cùng tham số. Với BatchNorm ở chế độ huấn luyện, đầu ra của một mẫu có thể phụ thuộc các mẫu khác trong batch thông qua thống kê chung. Ta vẫn tính được gradient của loss batch, nhưng không thể áp dụng nguyên xi công thức phương sai các gradient độc lập của từng mẫu như khi không có sự tương tác đó.

8. Một quỹ đạo nhiễu khác trung bình lý thuyết

Trở lại bài toán bốn mẫu với nghiệm $\theta_* = 1$. Bắt đầu ở $\theta_0 = 3{,}5$, dùng bước học $0{,}2$ cố định. Hình quỹ đạo vẽ GD toàn tập và hai quỹ đạo SGD lấy mẫu có hoàn lại, batch một và batch mười sáu. Mười sáu ở đây là số lượt chọn, có thể có các mẫu lặp, không phải một tập con mười sáu mẫu khác nhau. Các đường được tạo với hạt giống ngẫu nhiên cố định (tự tính).

Cách đọc hình quỹ đạo: panel trái là tham số theo số lần cập nhật, đường ngang chấm là nghiệm. Panel phải dùng thang log cho $F(\theta_t) - F(\theta_*)$, tức độ chênh mất mát với nghiệm, các vạch chính được ghi cách nhau hai bậc độ lớn, tức hệ số một trăm. GD trong ví dụ tiến đều đến nghiệm; các quỹ đạo SGD còn dao động, và batch lớn hơn dao động nhỏ hơn trong lần vẽ này. Mỗi đường chỉ là một lần lấy mẫu, không phải bảo đảm cho mọi lần chạy và cũng chưa so theo số mẫu đã xử lý.

Bởi $F'(\theta) = \theta - 1$, đặt sai số $e_t = \theta_t - 1$. Khi các lượt chọn độc lập có hoàn lại, gradient batch có dạng $g_t = e_t + \xi_t$; trong ví dụ này ξ_t có trung bình không và phương sai $2{,}5/B$, không phụ thuộc θ_t. Vì thế



Hình 9: Một quỹ đạo GD và hai quỹ đạo SGD với batch khác nhau (minh hoạ, tự tính)

$$e_{t+1} = (1-\eta)e_t - \eta\xi_t.$$

Nếu $0 < \eta < 2$, các mẫu mới độc lập với lịch sử và e_0 cố định, kỳ vọng sai số đi về không. Nhưng phương sai giới hạn của sai số là

$$\lim_{t\to\infty} \mathbb{E}[e_t^2] = \frac{\eta\,(2{,}5/B)}{2-\eta}.$$

Lý do: bình phương hệ thức trên, lấy kỳ vọng, số hạng chéo bằng không; tại giới hạn $V = (1-\eta)^2 V + \eta^2(2{,}5/B)$. Với $\eta = 0{,}2$, kết quả là $V = \frac{5}{18B}$ (tự tính). Batch một cho $V \approx 0{,}278$, batch bốn cho $V \approx 0{,}0694$. Đây là trung bình trên rất nhiều lần chạy về lâu dài, không phải độ lệch của một điểm bất kỳ trên hình.

Bài toán này cho thấy vì sao learning rate cố định cộng với nhiễu còn tồn tại có thể tạo một vùng dao động thay vì đi chính xác đến nghiệm. Đó là kết luận của mô hình cụ thể có các giả thiết ở trên, không phải định lý rằng mọi SGD đều mãi dao động. Nếu nhiễu tại nghiệm biến mất, nếu giảm learning rate phù hợp, hoặc nếu cơ chế lấy mẫu khác, hành vi có thể khác.

9. Learning rate, batch size và chi phí

Hai nút điều chỉnh tác động khác nhau. Learning rate nhân trực tiếp cả gradient trung bình lẫn nhiễu trong một bước. Batch lớn hơn làm ước lượng bớt nhiễu dưới các giả thiết phù hợp, nhưng cũng cần xử lý nhiều mẫu hơn cho mỗi lần cập nhật. Tăng đồng thời batch và learning rate có thể hữu ích trong một số thiết lập, song không có phép suy ra tổng quát rằng cứ nhân batch bao nhiêu thì nhân learning rate bấy nhiêu là an toàn. Độ cong, lịch học, kiến trúc và ngân sách đều quan trọng.

Cần hỏi **so theo trục nào**. Nếu chỉ so số step, batch lớn đã xử lý nhiều mẫu hơn sau mỗi step. Nếu so số mẫu hoặc số token, batch nhỏ có nhiều lần cập nhật hơn. Nếu so thời gian thực, phần cứng có thể xử lý nhiều mẫu song song nên chi phí thời gian không tăng tuyến tính. Báo cáo hữu ích ghi ít nhất cả batch hiệu dụng, số lần cập nhật, lượng dữ liệu và thời gian.

Giảm batch không bảo đảm tổng quát hoá tốt hơn. Nhiễu có thể thay đổi quỹ đạo và nghiệm tìm được, nhưng chất lượng dự đoán còn phụ thuộc dữ liệu, mô hình, regularization và cách điều chỉnh siêu tham số. Các quan sát thực nghiệm về batch lớn cần đọc trong đúng bối cảnh của chúng, không coi là định luật cho mọi bài toán.

10. Tích luỹ gradient và loss trung bình

Nếu một batch mong muốn không vừa bộ nhớ, ta có thể xử lý các **micro-batch** tại cùng bộ tham số, cộng gradient rồi mới gọi optimizer một lần. Giả sử micro-batch j có b_j mẫu và gradient trung bình g_j, thì gradient trung bình của toàn bộ các mẫu là

$$g_{\text{acc}} = \frac{\sum_j b_j g_j}{\sum_j b_j}.$$

Khi các micro-batch cùng kích thước, công thức là trung bình các g_j. Nếu kích thước khác, trung bình không trọng số sẽ cho micro-batch nhỏ trọng số quá lớn. Ví dụ hai micro-batch có kích thước 2 và 1, gradient trung bình lần lượt là 2 và 5: gradient đúng là $(2 \cdot 2 + 1 \cdot 5)/3 = 3$, còn trung bình hai số 2 và 5 là $3{,}5$ (tự tính).

Tích luỹ tương đương một batch gộp về gradient trong phép tính tách được theo mẫu, nếu giữ tham số cố định, dùng cùng quy ước lấy trung bình và xử lý ngẫu nhiên tương ứng. Với BatchNorm, thống kê tính riêng từng micro-batch có thể khác thống kê batch gộp nên kết quả không nhất thiết trùng. Không cập nhật optimizer giữa các micro-batch nếu mục tiêu là mô phỏng đúng một bước batch gộp; cập nhật ở giữa sẽ làm tham số tính gradient sau khác đi.

Một lỗi phổ biến là cộng các loss đã lấy trung bình mà quên chia hoặc quên trọng số kích thước. Khi đó độ lớn gradient thay đổi theo số micro-batch; so sánh learning rate giữa hai thiết lập trở nên sai nghĩa. Cũng cần lưu ý loss trên mỗi token, mỗi ảnh hay mỗi chuỗi có mẫu số khác nhau; hãy lấy trung bình theo đúng đơn vị mà mục tiêu yêu cầu.

11. Nối sang AI

Mạng phân loại và hồi quy. Mỗi mẫu cho một loss, backprop tính gradient của loss batch đối với các trọng số. Bộ tối ưu dùng gradient trung bình này cập nhật tham số. Mini-batch giúp phần cứng xử lý nhiều mẫu song song, còn tính ngẫu nhiên phụ thuộc cách xáo trộn và dữ liệu.

Mô hình ngôn ngữ. Một batch có thể được đếm theo chuỗi hoặc token. Các chuỗi có độ dài khác nhau, padding và mask làm mẫu số của loss quan trọng: trung bình đều theo chuỗi và trung bình đều theo token là hai mục tiêu khác nhau. Khi so batch giữa các lần chạy, cần nói rõ số token hợp lệ và số lần tích luỹ.

Dữ liệu mất cân bằng. Lấy mẫu để lớp hiếm xuất hiện thường xuyên hơn có thể giúp tín hiệu của lớp đó xuất hiện đều hơn, nhưng cũng đổi phân phối mà gradient không trọng số đang ước lượng. Hãy quyết định liệu muốn tối ưu trung bình theo tập gốc hay theo trọng số lớp đã chọn.

Quan sát khi huấn luyện. Loss từng batch lên xuống là bình thường ngay cả khi loss đánh giá trên một tập cố định giảm. Nếu muốn so các checkpoint, hãy dùng cùng cách tính và cùng tập đánh giá; đường mượt do trung bình trượt chỉ giúp nhìn xu hướng, không thay thế các giá trị gốc.

12. Sai lầm hay gặp

- Cho rằng SGD luôn đi xuống ở từng bước. Một batch có thể cho hướng ngược gradient toàn tập, như batch $3, 4$ ở mục 5.
- Cho rằng tính không chệch có nghĩa gradient của mọi batch đều giống nhau. Đó là phát biểu về kỳ vọng qua các batch tại một vị trí xác định.
- Dùng công thức σ^2/B mà bỏ qua lấy mẫu không hoàn lại, dữ liệu phụ thuộc hoặc tương tác giữa các mẫu.
- Gọi $B = n$ là full batch mà không kiểm xem có cho phép mẫu lặp hay không.
- So hai đường theo step rồi kết luận thuật toán dùng ít công hơn, dù số mẫu xử lý trong một step khác nhau.

- Quên trọng số micro-batch hoặc gọi optimizer giữa các micro-batch khi đang muốn tạo một cập nhật batch gộp.

13. Tóm tắt công thức

Ý nghĩa	Công thức và điều kiện
Mất mát toàn tập	$F(\theta) = \frac{1}{n}\sum_i \ell_i(\theta)$
Gradient batch	$g_t = \frac{1}{B}\sum_j \nabla \ell_{i_{t,j}}(\theta_t)$
Cập nhật SGD	$\theta_{t+1} = \theta_t - \eta_t g_t$
Không chệch	$\mathbb{E}[g_t \mid \theta_t] = \nabla F(\theta_t)$ khi batch mới được chọn đều độc lập với lịch sử
Phương sai có hoàn lại	σ^2/B với các lượt chọn độc lập tại cùng một điểm
Phương sai không hoàn lại	$\frac{\sigma^2}{B}\frac{n-B}{n-1}$ khi chọn tập con đều, σ^2 chia cho n
Tích luỹ	$g_{\text{acc}} = \frac{\sum_j b_j g_j}{\sum_j b_j}$ khi mọi gradient tính tại cùng tham số

14. Bài tập tự tính và đáp án

Hãy thử tự làm trước khi đọc đáp án. Mọi kết quả số trong phần này là tự tính và đã kiểm bằng numpy.

Bài 1: Một bước từ gradient đã cho

Tại $\theta = 3$, ba gradient mẫu là $2, 5, -1$. Tính gradient toàn tập, gradient batch gồm mẫu 1 và 3, rồi cập nhật từ cùng điểm với $\eta = 0{,}2$ theo từng gradient.

Đáp án: gradient toàn tập là $(2+5-1)/3 = 2$, nên đến $2{,}6$. Gradient batch là $(2-1)/2 = 0{,}5$, nên đến $2{,}9$. Hai kết quả khác nhau không mâu thuẫn vì một batch không nhất thiết bằng toàn tập.

Bài 2: Liệt kê phương sai

Vẫn tại điểm của bài 1, chọn đều hai mẫu khác nhau. Liệt kê ba gradient batch và tính phương sai của chúng, chia cho số batch. So với trường hợp chọn hai mẫu độc lập có hoàn lại.

Đáp án: ba trung bình là $3{,}5$, $0{,}5$, 2; trung bình của chúng là 2. Phương sai không hoàn lại là $(1{,}5^2 + (-1{,}5)^2 + 0^2)/3 = 1{,}5$. Phương sai của một gradient mẫu là $(0^2 + 3^2 + (-3)^2)/3 = 6$, nên phương sai batch có hoàn lại là $6/2 = 3$.

Bài 3: Một bước có thể tăng loss

Dùng lại bài toán mục 5, bắt đầu từ $\theta = 2$, chọn batch $3, 4$ và đặt $\eta = 0{,}2$. Tính tham số mới và so sánh loss toàn tập trước và sau.

Đáp án: gradient batch bằng $-0{,}5$, nên θ mới là $2{,}1$. Loss toàn tập tăng từ $1{,}75$ lên $1{,}855$. Việc tăng trong một bước không phủ nhận tính không chệch của phép chọn đều trong nhiều lần thử.

Bài 4: Hai micro-batch

Một micro-batch có hai gradient $1, 3$ và micro-batch kia có một gradient 5. Tính gradient batch gộp, rồi tính độ dịch chuyển với $\eta = 0{,}1$. Vì sao lấy trung bình hai gradient micro-batch lại sai?

Đáp án: gradient gộp là $(1+3+5)/3 = 3$, nên dịch chuyển bằng $-0{,}3$. Hai gradient trung bình riêng là 2 và 5; lấy trung bình không trọng số cho $3{,}5$, tức gán cho mẫu đơn ở micro-batch sau tổng trọng số bằng cả hai mẫu trước. Phải nhân từng trung bình với số mẫu trước khi chia tổng.

Bài 5: Vùng dao động trong bài toán bậc hai

Với thiết lập mục 8, $\eta = 0{,}2$, các mẫu độc lập có hoàn lại, tính giới hạn $\mathbb{E}[e_t^2]$ cho $B = 1$ và $B = 4$. Vì sao không dùng chính một đường trong hình quỹ đạo để kiểm chính xác các giá trị đó?

Đáp án: thay vào $5/(18B)$ cho $5/18 \approx 0{,}278$ và $5/72 \approx 0{,}0694$. Các giá trị là kỳ vọng ở giới hạn theo phân phối các lần chạy; một đường hữu hạn chỉ là một mẫu và còn có dao động ngẫu nhiên.

Tài liệu tham khảo nền: ghi chú opt 02, opt 03, opt 21, opt 22, opt 23; Bottou, Curtis và Nocedal, Optimization Methods for Large-Scale Machine Learning; Ruder, An Overview of Gradient Descent Optimization Algorithms.

Nền tảng 04

Momentum và Nesterov: giữ hướng đi, rồi nhìn trước khi lấy gradient

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: gradient, gradient descent, SGD và mini-batch ở mức cơ bản

Dùng ở: opt 02 (tối ưu quy mô lớn), opt 03 (các biến thể gradient descent), opt 04 (phương trình vi phân và Nesterov), opt 06 (Adam), opt 28 (so sánh optimizer)

> Đây là chương học từ đầu, không phải bản dịch paper. Các phép tính ghi ”tự tính” là ví dụ minh hoạ và đã được kiểm bằng numpy.

1. Vì sao cần momentum

Một câu để bắt đầu: **momentum bổ sung vào bước hiện tại một phần hướng dịch chuyển đã tích luỹ từ các bước trước**. Nhờ có ký ức đó, optimizer không phải phản ứng hoàn toàn theo gradient vừa thấy.

Hãy hình dung một viên bi lăn trong thung lũng. Sườn thung lũng dốc theo chiều ngang ngắn, nhưng lòng thung lũng lại dốc thoai thoải theo chiều dài. Gradient descent có thể đổi hướng liên tục giữa hai vách trong khi đi dọc đáy khá chậm. Momentum giữ một phần chuyển động từ trước, nên các gradient liên tiếp cùng hướng có thể được cộng dồn. Cách nói viên bi chỉ là trực giác: phép cập nhật thực là thuật toán rời rạc, không phải mô phỏng chính xác mọi hiện tượng vật lý.

Khi dùng mini-batch, hướng gradient từng bước còn dao động do dữ liệu được chọn. Việc kết hợp nhiều gradient trước có thể làm giảm ảnh hưởng của các dao động nhanh, nhưng cũng khiến optimizer phản ứng chậm hơn khi hướng thực sự đổi. Momentum không bảo đảm loss giảm từng bước và cũng không thay thế việc chọn learning rate hợp lý.

Nesterov là một họ cách thêm ý tưởng **nhìn trước**: dùng hướng đang tích luỹ để dự đoán một vị trí, rồi tính gradient ở vị trí dự đoán ấy. Có nhiều quy ước tương tự mang tên Nesterov trong tài liệu và thư viện. Chương này viết rõ một biến thể để bạn có thể tự tính, thay vì coi mọi công thức cùng tên là giống hệt nhau.

2. Nhắc lại gradient descent và mini-batch

Với hàm mất mát $F(\theta)$, gradient $\nabla F(\theta)$ cho hướng tăng cục bộ. Gradient descent cập nhật $\theta_{t+1} = \theta_t - \eta \nabla F(\theta_t)$, với $\eta > 0$ là learning rate. Khi θ là vector, mỗi phép cộng, trừ và nhân với số được thực hiện theo từng thành phần.

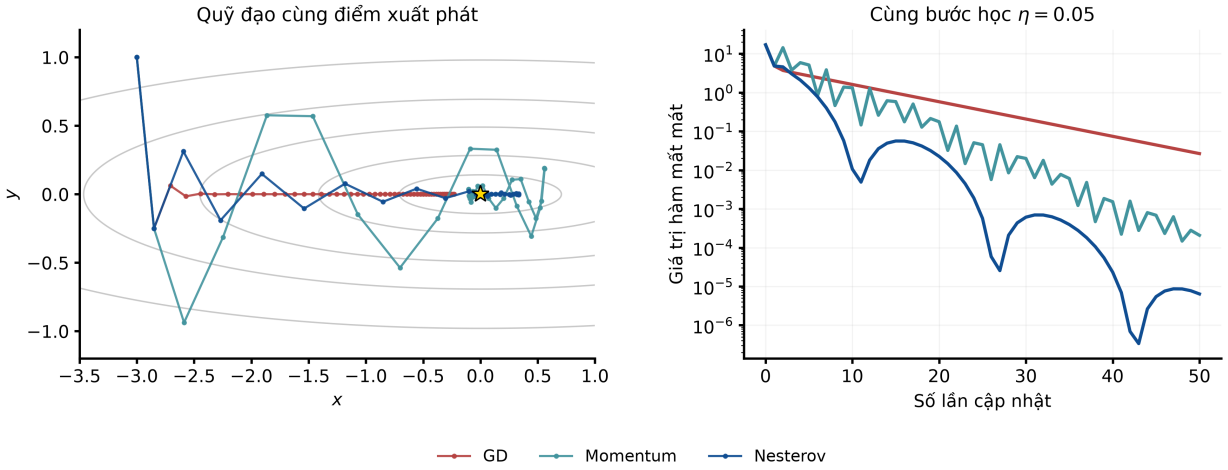
Ví dụ một chiều (tự tính): $F(\theta) = \theta^2$ có đạo hàm 2θ. Từ $\theta_0 = 1$ và $\eta = 0{,}2$, gradient ban đầu bằng 2, nên GD đến $\theta_1 = 1 - 0{,}2 \cdot 2 = 0{,}6$. Nếu lặp lại GD từ đó, gradient là $1{,}2$ và điểm tiếp theo là $0{,}36$. Gradient giảm theo vị trí, vì gradient được tính lại sau mỗi bước.

SGD hoặc mini-batch SGD thay gradient toàn tập bằng gradient tính từ một mẫu hoặc một nhóm mẫu. Hai batch khác nhau có thể cho hai gradient khác nhau ngay tại cùng vị trí. Phần dao động do chọn batch được giải thích trong nền tảng 03; chương này quan tâm đến việc optimizer xử lý chuỗi gradient đó như thế nào.

Chỉ số rất quan trọng: θ_t là vị trí **trước** lần cập nhật từ t sang $t+1$, còn gradient g_t được tính cho lần cập nhật đó. Ta sẽ luôn nêu nơi tính g_t, vì khác biệt giữa momentum thường và Nesterov nằm chính ở vị trí này.

3. Nhìn hình trước khi viết công thức

Hình thung lũng mô phỏng một thung lũng bậc hai với hai chiều: dốc theo trục đứng hơn theo trục ngang. Ba thuật toán khởi hành cùng điểm và cùng learning rate; hai thuật toán có momentum còn dùng cùng hệ số lưu hướng cũ (tự tính).



Hình 10: GD, momentum và Nesterov đi qua một thung lũng bậc hai (minh hoạ, tự tính)

Cách đọc hình thung lũng: các đường xám ở panel trái nối những điểm có cùng loss, ngôi sao là đáy, còn các chấm là vị trí sau mỗi lần cập nhật. Panel phải cho loss theo số cập nhật trên thang log; một đường đi xuống rồi lên vẫn có thể tiến về đáy về dài hạn. Với đúng hàm và siêu tham số ở đây, hai đường có momentum về gần đáy nhanh hơn GD sau nhiều bước, nhưng đường momentum thường dao động mạnh. Đây là một ví dụ được chọn để thấy cơ chế, không phải kết luận xếp hạng cố định cho mọi bài toán.

Trước khi dùng công thức, ta thấy ba ý. Thứ nhất, hướng đã đi là thông tin bổ sung ngoài gradient hiện tại. Thứ hai, tích luỹ cùng chiều giúp có thể tiến dọc thung lũng; mặt khác, quán tính có thể đưa ta qua bên kia đáy. Thứ ba, đặt cùng learning rate không có nghĩa các optimizer có cùng độ lớn bước: trạng thái tích luỹ cũng tham gia cập nhật.

4. Định nghĩa momentum dạng độ dịch chuyển

Gọi d_t là độ dịch chuyển đã thực hiện ở bước trước và đặt $d_0 = 0$. Với hệ số $0 \le \beta < 1$ và gradient hiện tại g_t, ta định nghĩa

$$d_{t+1} = \beta d_t - \eta g_t, \qquad \theta_{t+1} = \theta_t + d_{t+1}.$$

Trong **momentum thường**, còn gọi heavy-ball, $g_t = \nabla F(\theta_t)$ nếu tính toàn tập. Với mini-batch, g_t là gradient batch tại θ_t. Tên d được dùng vì nó mang cùng đơn vị với độ thay đổi tham số; từ lần đầu tiên trở đi, $d_t = \theta_t - \theta_{t-1}$.

Cách đọc cập nhật: lấy một phần βd_t của dịch chuyển cũ, cộng với dịch chuyển mới $-\eta g_t$. Nếu cả hai cùng chiều, chúng củng cố nhau. Nếu lực mới $-\eta g_t$ ngược chiều với dịch chuyển cũ d_t, nó hãm chuyển động cũ và có thể làm bước mới đảo chiều nếu đủ lớn. Việc gradient vừa đổi dấu không tự nó đủ để kết luận đang hãm: phải so hướng lực mới với chính dịch chuyển đang được lưu. Khi $\beta = 0$, ta trở về đúng GD hoặc SGD thường với cùng gradient và learning rate.

Đừng bỏ qua khởi tạo. Nếu $d_0 = 0$, bước đầu của momentum trùng bước GD vì chưa có hướng cũ. Nếu tiếp tục một lần huấn luyện đã lưu trạng thái, d_0 của đoạn chạy tiếp có thể khác không. Lưu và khôi phục riêng

trọng số mà bỏ trạng thái optimizer có thể làm quỹ đạo sau khi tiếp tục thay đổi.

5. Ví dụ tay: vì sao có thể vượt đáy

Ta dùng cùng hàm $F(\theta) = \theta^2$, nghiệm $\theta_* = 0$, điểm đầu $\theta_0 = 1$, hệ số $\beta = 0{,}8$, learning rate $\eta = 0{,}2$ và $d_0 = 0$. Mọi số trong mục này là tự tính và đã kiểm bằng numpy.

5.1 Bước thứ nhất

Gradient tại vị trí đầu là $g_0 = 2\theta_0 = 2$. Phần dịch chuyển cũ bằng 0, nên

$$d_1 = 0{,}8 \cdot 0 - 0{,}2 \cdot 2 = -0{,}4, \qquad \theta_1 = 1 - 0{,}4 = 0{,}6.$$

Điều này giống bước đầu của GD ở mục 2: chưa có quá khứ để tích luỹ.

5.2 Bước thứ hai

Tại $\theta_1 = 0{,}6$, gradient bây giờ là $g_1 = 2 \cdot 0{,}6 = 1{,}2$. Độ dịch chuyển được tạo từ hai phần:

$$d_2 = 0{,}8 \cdot (-0{,}4) - 0{,}2 \cdot 1{,}2 = -0{,}32 - 0{,}24 = -0{,}56.$$

Ta đến $\theta_2 = 0{,}6 - 0{,}56 = 0{,}04$. Tức là dù gradient mới nhỏ hơn trước, tổng dịch chuyển lại lớn hơn vì phần cũ vẫn đẩy sang trái. Nếu chỉ dùng GD từ $0{,}6$, ta đến $0{,}36$, không phải $0{,}04$.

5.3 Bước thứ ba

Gradient tại $\theta_2 = 0{,}04$ chỉ là $0{,}08$, nhưng hướng cũ vẫn còn mạnh:

$$d_3 = 0{,}8 \cdot (-0{,}56) - 0{,}2 \cdot 0{,}08 = -0{,}448 - 0{,}016 = -0{,}464.$$

Vì vậy $\theta_3 = 0{,}04 - 0{,}464 = -0{,}424$: ta đi qua đáy sang phía âm. Các loss tương ứng là $F(\theta_1) = 0{,}36$, $F(\theta_2) = 0{,}0016$ và $F(\theta_3) = 0{,}179776$ (tự tính). Loss tăng ở bước ba dù thuật toán không mắc lỗi số học.

Khi sang phía âm, gradient có dấu âm và lực mới hướng về bên phải. Nhưng nếu độ dịch chuyển tích luỹ sang trái vẫn đủ lớn, thuật toán có thể còn đi sang trái vài bước trước khi đảo chiều. Quá trình đó giải thích dao động, và cũng giải thích vì sao nhìn mỗi gradient tức thời chưa cho biết bước kế tiếp của momentum.

6. Nesterov: lấy gradient tại điểm nhìn trước

Dùng cùng d_t là độ dịch chuyển đã thực hiện ở bước trước, ta định nghĩa biến thể Nesterov của chương bằng ba dòng:

$$y_t = \theta_t + \beta d_t, \qquad g_t = \nabla F(y_t),$$

$$d_{t+1} = \beta d_t - \eta g_t, \qquad \theta_{t+1} = \theta_t + d_{t+1}.$$

Điểm y_t là **điểm nhìn trước**, không phải tham số đã cập nhật chính thức. Ta giả sử trước tiên tiếp tục theo phần dịch chuyển cũ, đo gradient tại đó, rồi dùng gradient này để tính bước từ θ_t. Trong phiên bản ngẫu nhiên của công thức, gradient batch cũng được đánh giá tại y_t. Phải giữ nguyên thứ tự: nhìn trước, tính gradient ở đó, ghép với hướng cũ, rồi mới cập nhật.

6.1 Tính chậm hai bước trên cùng ví dụ

Tại đầu, $d_0 = 0$ nên $y_0 = \theta_0 = 1$. Gradient bằng 2, vì vậy $d_1 = -0{,}4$ và $\theta_1 = 0{,}6$, giống momentum thường (tự tính).

Trước bước thứ hai, dùng $d_1 = -0{,}4$ để nhìn trước:

$$y_1 = 0{,}6 + 0{,}8 \cdot (-0{,}4) = 0{,}28, \qquad \nabla F(y_1) = 2 \cdot 0{,}28 = 0{,}56.$$

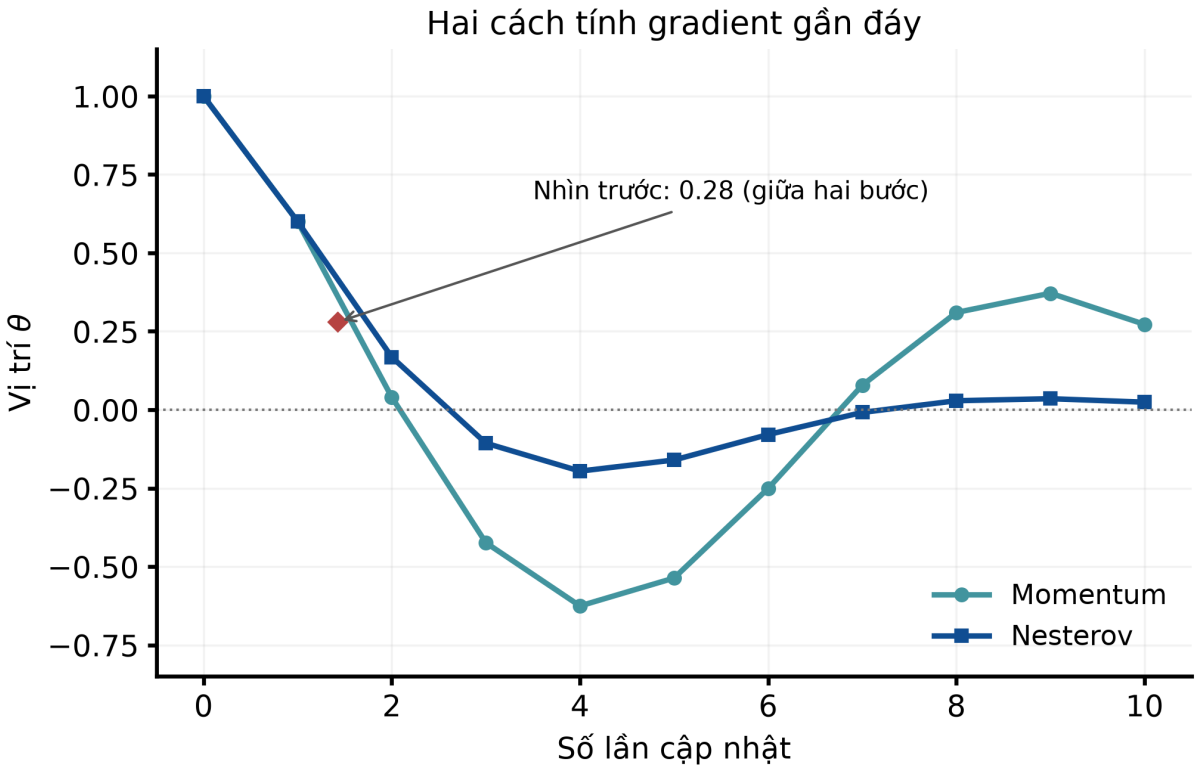
Bây giờ ghép hướng cũ với gradient ***ở*** $0{,}28$, không phải gradient ở $0{,}6$:

$$d_2 = 0{,}8 \cdot (-0{,}4) - 0{,}2 \cdot 0{,}56 = -0{,}432, \qquad \theta_2 = 0{,}6 - 0{,}432 = 0{,}168.$$

So với momentum thường, cùng phần cũ là $-0{,}32$, nhưng gradient mới nhỏ hơn vì điểm nhìn trước nằm gần đáy hơn. Điều này khiến độ dịch chuyển sang trái ở bước hai nhỏ hơn. Kết luận này đúng cho các số cụ thể vừa tính; ở bài toán khác điểm nhìn trước có thể nằm ở phía có gradient lớn hơn hoặc đổi dấu.

6.2 Một bước nữa và hình so sánh

Từ $\theta_2 = 0{,}168$ và $d_2 = -0{,}432$, ta có $y_2 = 0{,}168 + 0{,}8 \cdot (-0{,}432) = -0{,}1776$. Gradient nhìn trước là $-0{,}3552$. Do đó $d_3 = -0{,}27456$ và $\theta_3 = -0{,}10656$ (tự tính). Điểm nhìn trước đã ở phía âm, nên gradient mới góp phần hãm hướng đang đi sang trái.



Hình 11: So sánh vị trí của momentum và Nesterov cùng điểm nhìn trước (minh hoạ, tự tính)

Cách đọc hình nhìn trước: các điểm trên hai đường là vị trí sau số lần cập nhật ghi trên trục ngang; đường chấm ngang là nghiệm. Hình thoi đỏ cho $y_1 = 0{,}28$ được đặt giữa hai bước chỉ để chú thích, không phải một lần cập nhật tại thời điểm lẻ. Hai phương pháp trùng nhau sau bước đầu rồi tách ra, và cả hai đều có thể qua

đáy. Đường xanh dao động nhỏ hơn trong ví dụ này, nhưng không phải lúc nào nó cũng gần nghiệm hơn: sau đúng hai bước, $0{,}04$ của momentum thường gần đáy hơn $0{,}168$ của Nesterov.

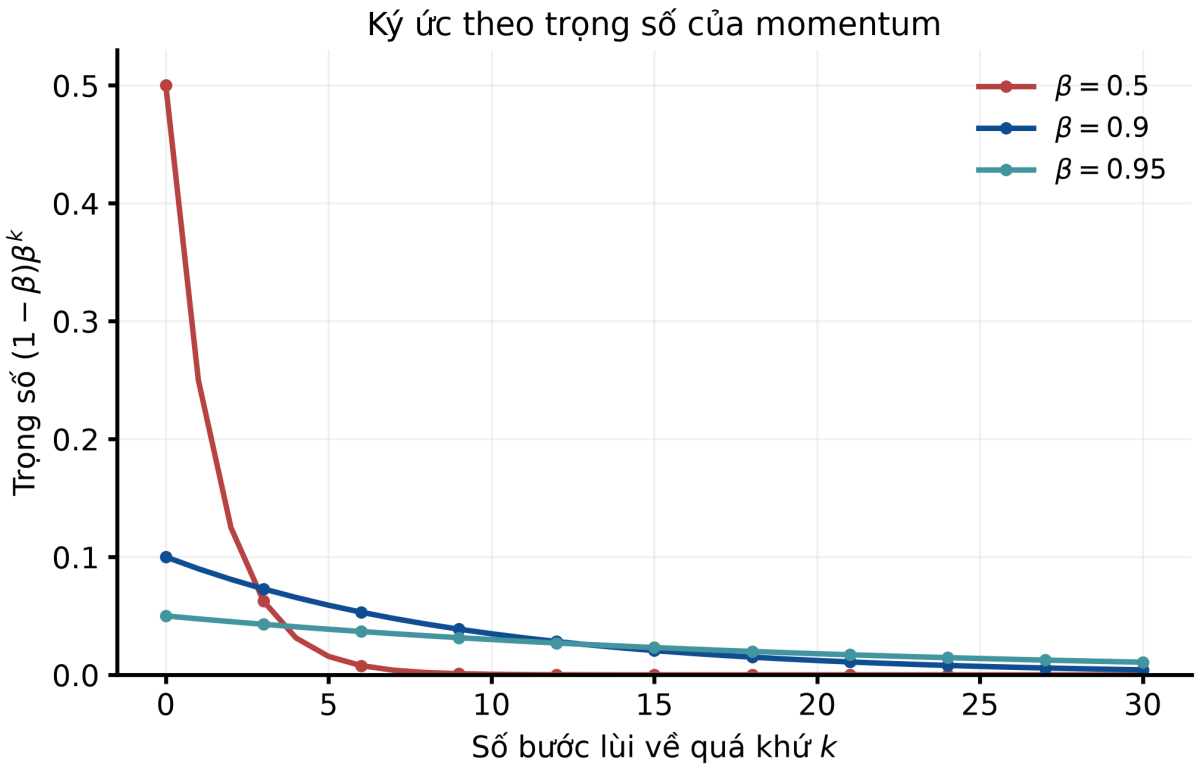
7. Momentum là ký ức có trọng số

Thay liên tiếp công thức của d_t vào chính nó, với $d_0 = 0$ và learning rate cố định, ta được

$$d_{t+1} = -\eta\big(g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \cdots + \beta^t g_0\big).$$

Gradient mới nhất có hệ số 1, gradient lùi một bước có hệ số β, lùi hai bước có hệ số β^2. Với $0 \le \beta < 1$, hệ số giảm theo tuổi của gradient. Đối với Nesterov cũng có phép khai triển đại số như thế nếu g_t được hiểu là gradient **tại điểm nhìn trước**, không phải tại θ_t.

Hình sau chuẩn hoá các hệ số bằng cách nhân thêm $(1-\beta)$. Ta có thể xem chúng như trọng số của một trung bình trượt mũ khi gradient được cấp vào đều đặn.



Hình 12: Trọng số gradient quá khứ theo ba hệ số momentum (minh hoạ, tự tính)

Cách đọc hình trọng số: trục ngang là số bước lùi về quá khứ, trục đứng là trọng số đã chuẩn hoá. Với β lớn, trọng số của một gradient rất mới nhỏ hơn nhưng các gradient cũ bị quên chậm hơn. Hình chỉ vẽ một phần quá khứ, nên tổng các trọng số hiện trên hình có thể nhỏ hơn một; phần còn lại nằm ngoài miền vẽ. Không được đọc trọng số đơn lẻ nhỏ hơn thành kết luận rằng tổng ảnh hưởng của lịch sử cũng nhỏ hơn.

Ví dụ (tự tính): với $\beta = 0{,}5$, trọng số đã chuẩn hoá của gradient hiện tại, trước một bước và trước hai bước là $0{,}5$, $0{,}25$, $0{,}125$. Với $\beta = 0{,}9$, ba trọng số tương ứng là $0{,}1$, $0{,}09$, $0{,}081$. Một quy tắc nhẩm hay dùng là độ dài ký ức cỡ $1/(1-\beta)$ bước: khoảng 10 bước khi $\beta = 0{,}9$. Đây là thang xấp xỉ chứ không phải ranh giới cứng, vì trọng số không đột ngột về không sau mười bước.

8. Hai quy ước buffer gradient và learning rate

Nhiều tài liệu viết momentum dưới dạng bộ đệm gradient v_t thay vì dịch chuyển d_t:

$$v_{t+1} = \beta v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_{t+1}, \qquad v_0 = 0.$$

Nếu η cố định và đặt $d_t = -\eta v_t$, công thức này đúng bằng dạng mục 4. Ví dụ với $\eta = 0{,}2$, $v_1 = 2$ tương ứng $d_1 = -0{,}4$ (tự tính). Bộ đệm v cùng đơn vị với gradient; d cùng đơn vị với tham số. Tên ”velocity” đôi khi được dùng cho cả hai nên phải kiểm công thức và dấu.

Một quy ước khác dùng trung bình trượt mũ đã nhân hệ số chuẩn hoá:

$$m_{t+1} = \beta m_t + (1-\beta) g_t, \qquad \theta_{t+1} = \theta_t - \alpha m_{t+1}.$$

Với cùng gradient, cùng trạng thái đầu bằng không và β cố định, $m_t = (1-\beta) v_t$. Để hai quy ước đi cùng quỹ đạo, phải chọn $\alpha = \eta/(1-\beta)$. Ví dụ $\beta = 0{,}8$, nếu dạng v dùng $\eta = 0{,}2$ thì dạng m dùng $\alpha = 1$ (tự tính). Dùng cùng con số learning rate cho hai công thức thì nói chung hai bước sẽ khác nhau.

Khi learning rate thay đổi, việc chuyển giữa buffer và dịch chuyển cần cẩn thận hơn. Ở dạng buffer với η_t, $d_{t+1} = -\eta_t v_{t+1}$; nếu $\eta_{t-1} > 0$ thì $d_{t+1} = \beta(\eta_t/\eta_{t-1}) d_t - \eta_t g_t$. Do đó lịch learning rate thay đổi có thể làm dạng này khác với việc giữ nguyên hệ số β trong quy tắc $d_{t+1} = \beta d_t - \eta_t g_t$. Khi so với code, kiểm cả trạng thái lẫn thứ tự áp learning rate.

Một ví dụ chỉ để hiểu độ lớn, không phải lịch huấn luyện thực: nếu $g_t = g$ không đổi mãi và dùng buffer chưa chuẩn hoá thì v_t tiến đến $g/(1-\beta)$. Với $\beta = 0{,}9$, giới hạn đó bằng $10g$ (tự tính). Bước tương ứng tiến đến $-10\eta g$. Trên hàm thực, gradient đổi theo vị trí và có thể đổi dấu, nên không thể thay learning rate bằng con số đó trong mọi hoàn cảnh.

9. Làm mượt nhiễu và cái giá phải trả

Để tách ý tưởng khỏi chuyển động của mô hình, xét một chuỗi số $g_t = \mu + \epsilon_t$, trong đó μ không đổi, các nhiễu ϵ_t độc lập qua thời gian, có trung bình không và phương sai σ^2. Dùng trung bình trượt mũ đã chuẩn hoá ở mục 8. Khi đã qua giai đoạn đầu, phương sai của phần nhiễu trong m là

$$\mathrm{Var}(m) = \frac{(1-\beta)^2}{1-\beta^2}\sigma^2 = \frac{1-\beta}{1+\beta}\sigma^2.$$

Cách thấy điều này: phương sai của tổng các nhiễu độc lập bằng tổng phương sai, nên ta cộng chuỗi hình học $(1-\beta)^2(1+\beta^2+\beta^4+\cdots)\sigma^2$. Với $\beta = 0{,}9$, hệ số bằng $1/19$, xấp xỉ $0{,}0526$ (tự tính). Đây là phương sai của **buffer chuẩn hoá**, không phải phát biểu rằng phương sai của tham số hay bước cập nhật trong mọi bài toán cũng giảm theo đúng hệ số đó.

Giả thiết độc lập và tín hiệu không đổi giúp làm phép tính đơn giản. Trong thực tế, các gradient được tính tại những vị trí khác nhau, dữ liệu có thể tương quan, và bản thân hướng thật cũng thay đổi. Gradient cũ lúc đó vừa chứa thông tin hữu ích vừa có thể trở nên lạc hậu. Vì vậy, tăng β có thể làm đường cập nhật mượt hơn nhưng cũng làm việc thích nghi với hướng mới chậm hơn.

9.1 Khởi tạo bằng không và trung bình trượt mũ

Vẫn với tín hiệu trung bình μ không đổi và $m_0 = 0$, ta có $\mathbb{E}[m_t] = (1-\beta^t)\mu$. Ban đầu tổng trọng số chưa đủ một, nên kỳ vọng buffer lệch về không. Chia cho $1-\beta^t$ cho một ước lượng không chệch của μ trong mô hình này. Chẳng hạn $\beta = 0{,}9$, $g_t = 2$ không đổi: $m_1 = 0{,}2$, $m_2 = 0{,}38$, $m_3 = 0{,}542$; chia m_3 cho $1 - 0{,}9^3 = 0{,}271$ được 2 (tự tính).

Bias correction là một lựa chọn cụ thể, không phải thao tác bắt buộc của mọi bản cài đặt momentum. Adam có công thức hiệu chỉnh cho các bộ đệm của nó; khi đọc một optimizer, xem nó thực sự dùng buffer nào và có hiệu chỉnh hay không. Trên một quỹ đạo mà gradient trung bình đang thay đổi, chia cho $1 - \beta^t$ cũng không biến trung bình lịch sử thành gradient hiện tại.

10. Độ cong, dao động và giới hạn ổn định

Để hiểu vì sao η và β phải đi cùng nhau, xét một chiều $F(x) = \frac{\lambda}{2}x^2$ với $\lambda > 0$. Gradient là λx. Với momentum thường và $d_t = x_t - x_{t-1}$, sau khi thế vào công thức ta được

$$x_{t+1} = (1 + \beta - \eta\lambda)x_t - \beta x_{t-1}.$$

Với $0 \leq \beta < 1$, quy tắc tuyến tính này ổn định tiệm cận cho mọi trạng thái đầu khi $0 < \eta\lambda < 2(1 + \beta)$. Đây là điều kiện riêng cho hàm bậc hai một chiều và momentum thường như đã định nghĩa; không phải điều kiện đủ cho một mạng nơ-ron bất kỳ. Khi hai nghiệm của phương trình đặc trưng có phần ảo, quỹ đạo có thể dao động ngay cả trong miền ổn định. Ổn định chỉ nói trạng thái tiến về đáy về lâu dài, không nói loss giảm ở từng bước.

Ví dụ (tự tính): nếu $\lambda = 4$ và $\beta = 0{,}5$, điều kiện trên thành $0 < \eta < 0{,}75$. Với $\eta = 0{,}5$, hai nghiệm có mô-đun $\sqrt{0{,}5} < 1$; với $\eta = 0{,}8$, một nghiệm xấp xỉ $-1{,}322$, có mô-đun lớn hơn một, nên tồn tại trạng thái đầu làm độ lớn tăng. Với GD không momentum trên cùng hàm, điều kiện là $0 < \eta < 0{,}5$. Các điều kiện biên không bảo đảm hội tụ tiệm cận.

Trong nhiều chiều bậc hai đối xứng xác định dương, mỗi hướng riêng có một độ cong tương ứng. Một lựa chọn phải xét tất cả các hướng, đặc biệt hướng có độ cong lớn. Momentum có thể giảm thời gian đi theo hướng thoải khi được điều chỉnh phù hợp, nhưng nếu chọn quá mạnh, dao động theo hướng dốc có thể áp đảo. Nesterov của mục 6 có quan hệ truy hồi và miền ổn định riêng, không được mặc định dùng cùng bất đẳng thức của heavy-ball.

11. Các biến thể và chi tiết thực hành

11.1 Nesterov trong lý thuyết và trong thư viện

Công thức nhìn trước của mục 6 rất hữu ích để tính tay. Các thuật toán Nesterov tăng tốc cổ điển còn có thể được viết bằng hai dãy điểm và hệ số thay đổi theo bước; một số dạng có thể đổi biến để liên hệ với dạng khác trong các điều kiện nhất định. Một cờ cấu hình tên ”Nesterov” trong thư viện cũng có thể dùng một cách lưu trạng thái và thứ tự áp bước riêng. Để đối chiếu, hãy viết ra điểm lấy gradient, trạng thái lưu, cách khởi tạo và thứ tự cập nhật, thay vì chỉ so tên.

Với hàm lồi trơn và lựa chọn thuật toán, tham số đúng điều kiện, một số phương pháp Nesterov có bảo đảm tốc độ hội tụ tốt hơn GD trong phân tích lý thuyết. Điều đó không tự động chứng minh biến thể SGD nhìn trước với hệ số cố định ở mục 6 nhanh hơn mọi optimizer trên mọi mạng phi lồi. Khi đọc opt 04, cần đối chiếu đúng dạng thuật toán và các giả thiết trong bài.

11.2 Hệ số momentum và lịch học

β gần một cho ký ức dài hơn, nhưng cũng giữ hướng cũ lâu hơn. η quyết định mức đóng góp của gradient trong từng bước, còn ảnh hưởng thực tế phụ thuộc quy ước buffer. Một cặp giá trị chạy ổn trên bài toán này không tự động ổn trên bài toán khác; thay batch, độ chuẩn hoá loss hoặc lịch learning rate có thể làm mức cập nhật thay đổi.

Đổi learning rate mà vẫn giữ bộ đệm là hành vi thường gặp, nhưng ảnh hưởng tức thời phụ thuộc cách cài đặt như mục 8. Nếu khôi phục checkpoint hoặc đổi thuật toán, cần kiểm xem các trạng thái optimizer có được

lưu và chuyển đúng hay không. So sánh hai lần chạy cũng nên ghi rõ lịch của cả η và β, không chỉ một con số ở cuối.

11.3 So sánh công bằng và nhìn đường huấn luyện

So sánh cùng learning rate có ích để xem sự khác nhau của quy tắc, như trong hình thung lũng, nhưng không bảo đảm hai thuật toán đã được chỉnh tốt như nhau. Một đánh giá thực hành nên dành ngân sách điều chỉnh hợp lý cho từng phương pháp và ghi rõ số lần cập nhật, dữ liệu đã xử lý, thời gian, cùng cách đánh giá trên tập cố định.

Nếu loss dao động, hãy xem cả learning rate, momentum, chuẩn gradient và trạng thái dữ liệu. Dao động có thể đến từ độ cong, nhiễu batch hoặc sự thay đổi dữ liệu. Một đường loss đẹp mắt sau khi làm mượt không chứng minh thuật toán tốt hơn; kết quả trên tập đánh giá và độ ổn định qua các lần chạy vẫn cần được kiểm tra.

12. Nối sang AI

Huấn luyện mạng thị giác. SGD với momentum thường được dùng khi cập nhật các bộ lọc và lớp phân loại. Gradient từ các mini-batch ảnh có thể biến thiên; momentum lưu một bản tóm tắt của các hướng trước. Chất lượng cuối còn phụ thuộc lịch learning rate, batch size, dữ liệu và lựa chọn regularization.

Mạng ngôn ngữ và optimizer thích nghi. Adam giữ một trung bình trượt mũ của gradient bên cạnh thông tin về bình phương gradient. Phần trung bình đó có quan hệ với ý tưởng momentum, nhưng bước Adam còn chia theo một đại lượng thích nghi và có bias correction. Vì thế không thể đồng nhất Adam với SGD có momentum chỉ bằng cách nhìn một bộ đệm.

Dữ liệu và mục tiêu thay đổi. Nếu phân phối mini-batch hoặc nhiệm vụ thay đổi trong lúc huấn luyện, ký ức dài có thể giúp làm mượt biến động ngắn nhưng cũng có thể mang thông tin của giai đoạn trước. Khi đó cần quan sát đáp ứng sau thay đổi và điều chỉnh theo thí nghiệm, thay vì kết luận hệ số lớn luôn tốt hơn.

Tích luỹ gradient. Nhiều micro-batch có thể đóng góp vào một gradient gộp trước khi thực hiện một bước optimizer. Khi làm vậy, bộ đệm momentum thường được cập nhật một lần cho batch gộp, không phải một lần cho mỗi micro-batch. Nếu cập nhật bộ đệm nhiều lần, thời gian ký ức tính theo số mẫu và kết quả cuối đều thay đổi.

13. Sai lầm hay gặp

- Dùng mỗi gradient hiện tại để tính bước momentum, quên trạng thái d_t hoặc v_t.
- Kết luận momentum luôn giảm dao động hoặc luôn làm loss giảm; ví dụ mục 5 vượt đáy và loss tăng.
- Đổi giữa buffer chưa chuẩn hoá và trung bình trượt đã chuẩn hoá mà giữ nguyên learning rate rồi cho rằng hai quy tắc tương đương.
- Tính gradient Nesterov tại θ_t thay vì điểm nhìn trước trong công thức cụ thể của chương.
- Cho rằng mọi optimizer có tên Nesterov dùng cùng ký hiệu, cùng cách khởi tạo và cùng thứ tự cập nhật.
- Lấy hệ số giảm phương sai của một trung bình nhiễu độc lập, ổn định rồi áp trực tiếp cho quỹ đạo SGD trên mạng thực.

14. Tóm tắt công thức

Ý nghĩa	Công thức và điều kiện
Momentum thường	$d_{t+1} = \beta d_t - \eta g_t,\ g_t = \nabla F(\theta_t)$
Cập nhật tham số	$\theta_{t+1} = \theta_t + d_{t+1}$
Nhìn trước Nesterov	$y_t = \theta_t + \beta d_t,\ g_t = \nabla F(y_t)$

Ý nghĩa	Công thức và điều kiện
Buffer chưa chuẩn hoá	$v_{t+1} = \beta v_t + g_t$, $d_{t+1} = -\eta v_{t+1}$ khi η cố định
Trung bình chuẩn hoá	$m_{t+1} = \beta m_t + (1-\beta) g_t$
Quy đổi bước	$m_t = (1-\beta) v_t$ và $\alpha = \eta/(1-\beta)$ khi hai trạng thái đầu bằng không
Phương sai buffer	$\frac{1-\beta}{1+\beta}\sigma^2$ với nhiễu độc lập, phương sai không đổi, sau giai đoạn đầu
Ổn định heavy-ball	$0 < \eta\lambda < 2(1+\beta)$ cho bậc hai một chiều, $0 \le \beta < 1$

15. Bài tập tự tính và đáp án

Hãy tự tính trước khi xem lời giải. Mọi kết quả số trong phần này là tự tính và đã kiểm bằng numpy.

Bài 1: Tích luỹ ba gradient

Cho $\theta_0 = 1$, $d_0 = 0$, $\eta = 0{,}1$, $\beta = 0{,}5$ và lần lượt nhận các gradient đã cho là $2, 1, -1$. Tính ba dịch chuyển và ba vị trí của momentum thường.

Đáp án: $d_1 = -0{,}2$, $\theta_1 = 0{,}8$; $d_2 = 0{,}5(-0{,}2) - 0{,}1(1) = -0{,}2$, $\theta_2 = 0{,}6$; $d_3 = 0{,}5(-0{,}2) - 0{,}1(-1) = 0$, $\theta_3 = 0{,}6$. Gradient cuối đã đổi dấu nhưng trước tiên chỉ vừa đủ triệt tiêu hướng cũ.

Bài 2: Điểm nhìn trước

Cho $F(\theta) = \theta^2$, $\theta_0 = 1$, $d_0 = 0$, $\eta = 0{,}2$, $\beta = 0{,}8$. Sau bước đầu của Nesterov, tính điểm nhìn trước và vị trí chính thức sau bước thứ hai. So với momentum thường.

Đáp án: sau bước đầu $\theta_1 = 0{,}6$, $d_1 = -0{,}4$. Điểm nhìn trước $y_1 = 0{,}28$, gradient tại đó là $0{,}56$. Vậy $d_2 = -0{,}432$ và $\theta_2 = 0{,}168$. Momentum thường dùng gradient $1{,}2$ ở θ_1, nên nhận $d_2 = -0{,}56$ và $\theta_2 = 0{,}04$.

Bài 3: Quy đổi hai buffer

Giả sử $v_0 = m_0 = 0$, $\beta = 0{,}5$, và ba gradient liên tiếp là $2, 1, -1$. Tính v_t và m_t. Nếu bước với v_t dùng $\eta = 0{,}1$, dạng m_t cần hệ số α nào để cùng dịch chuyển?

Đáp án: $v_1 = 2$, $v_2 = 2$, $v_3 = 0$; còn $m_1 = 1$, $m_2 = 1$, $m_3 = 0$. Quả thật $m_t = 0{,}5 v_t$. Chọn $\alpha = 0{,}1/(1-0{,}5) = 0{,}2$, vì $-0{,}2 m_t = -0{,}1 v_t$.

Bài 4: Trung bình nhiễu

Một gradient quan sát gồm tín hiệu không đổi và nhiễu độc lập có phương sai 9. Ở trạng thái ổn định của buffer chuẩn hoá, tính phương sai nhiễu khi $\beta = 0{,}5$ và khi $\beta = 0{,}9$.

Đáp án: áp dụng $\frac{1-\beta}{1+\beta}\sigma^2$, ta được $\frac{1}{3} \cdot 9 = 3$ và $\frac{1}{19} \cdot 9 = 9/19 \approx 0{,}474$. Các kết quả không tự động mô tả phương sai tham số của một mạng đang huấn luyện.

Bài 5: Kiểm điều kiện bậc hai

Với $F(x) = 2x^2$, $\beta = 0{,}5$ và momentum thường, hãy kiểm $\eta = 0{,}5$ và $\eta = 0{,}8$ có nằm trong miền ổn định tiệm cận cho mọi trạng thái đầu hay không.

Đáp án: ở đây $\lambda = 4$, nên điều kiện là $0 < \eta < 0{,}75$. Giá trị $0{,}5$ nằm trong miền, còn $0{,}8$ nằm ngoài. Ở $0{,}8$, một nghiệm đặc trưng có mô-đun lớn hơn một; do đó có trạng thái đầu khiến quỹ đạo không hội tụ. Kết luận này chỉ áp dụng trực tiếp cho bài toán và quy tắc đã cho.

Tài liệu tham khảo nền: ghi chú opt 02, opt 03, opt 04, opt 06, opt 28; Ruder, An Overview of Gradient Descent Optimization Algorithms; Bottou, Curtis và Nocedal, Optimization Methods for Large-Scale Machine Learning.

Nền tảng 05

Optimizer thích nghi: mỗi tham số được điều chỉnh theo lịch sử gradient của nó

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: gradient descent, SGD và mini-batch ở nền tảng 02, 03; momentum ở nền tảng 04; vector và đạo hàm riêng ở math nền tảng 02, 06

Dùng ở: optimization 03 (tổng quan), optimization 06 (Adam), optimization 07 (AdamW), optimization 08 (giới hạn hội tụ), optimization 09 (các bước đầu), optimization 12 (bộ nhớ của optimizer)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi ”tự tính” và các hình minh hoạ đều đã kiểm bằng numpy.

1. Vì sao cần optimizer thích nghi

Optimizer thích nghi là quy tắc cập nhật có thay đổi mức chia gradient theo lịch sử riêng của từng tham số. Nó giải quyết một bất tiện: trong cùng mạng nơ-ron, một trọng số có thể thường nhận gradient lớn, trong khi trọng số khác chỉ thỉnh thoảng nhận gradient. Một learning rate dùng chung nhân lên cả hai chưa chắc cho hai bước đi có tỉ lệ hữu ích.

Hãy hình dung bạn di chuyển trong một thung lũng rất hẹp. Theo chiều ngang, mặt đất thoai thoải; theo chiều dọc, mặt đất dốc hơn. Cùng một hệ số nhân gradient có thể khiến bước theo chiều dọc quá lớn, trong khi chiều ngang đi rất chậm. Thay đổi hệ số theo từng tọa độ là một cách giải quyết có thể thử. Đây chỉ là trực giác: lịch sử gradient không luôn mô tả chính xác độ cong của hàm mất mát, và optimizer thích nghi không tự bảo đảm tìm được cực tiểu tốt.

Trong AI, các phương pháp này được dùng để huấn luyện từ mô hình tuyến tính đến mạng sâu. Chương đi qua AdaGrad, RMSProp, Adam và AdamW. Mỗi phương pháp trả lời một câu hỏi: nhớ gradient trong bao lâu, có làm mượt hướng không, cần sửa gì ở bước đầu, và làm thế nào để giảm trọng số mà không lẫn vào bộ chia thích nghi.

2. Nhắc nền: gradient, bước đi và trung bình trượt

2.1 Một learning rate nhân cả vector

Gọi $\theta = (\theta^{(1)}, \theta^{(2)})$ là vector hai tham số, $L(\theta)$ là hàm mất mát, g_t là gradient tại bước t. Chỉ số trên trong ngoặc chỉ tọa độ; chỉ số dưới t trong θ_t chỉ toàn bộ vector ở một thời điểm. Với SGD không momentum, bước cập nhật theo quy ước chương này là

$$\theta_t = \theta_{t-1} - \eta g_t.$$

η là learning rate dương. Nếu dùng mini-batch, g_t chỉ là gradient tính từ batch đó, nên có thể thay đổi dù vị trí hiện tại gần giống nhau. Chỉ số t đếm các lần **cập nhật tham số**, bắt đầu từ 1; θ_0 là điểm trước cập nhật đầu tiên.

Ví dụ (tự tính), cho $L(\theta) = \frac{1}{2}((\theta^{(1)})^2 + 4(\theta^{(2)})^2)$, $\theta_0 = (2, 1)$, $\eta = 0{,}1$. Gradient là $(\theta^{(1)}, 4\theta^{(2)})$, nên $g_1 = (2, 4)$ và $\theta_1 = (1{,}8, 0{,}6)$. Loss giảm từ 4 xuống $2{,}34$ (tự tính). Hai thành phần gradient khác độ lớn, nhưng cả hai vẫn cùng nhân với $0{,}1$.

Trong optimizer thích nghi, ta cũng giữ một learning rate toàn cục η, nhưng sau đó nhân thêm một hệ số có thể khác nhau giữa các tọa độ. Vì vậy η quan trọng ngay cả khi tên thuật toán có chữ ”thích nghi”. Bộ chia phụ thuộc vào lịch sử, nên hai lần chạy cùng θ_t nhưng có lịch sử gradient khác nhau có thể đi bước tiếp theo khác nhau.

2.2 Bình phương, căn và trung bình trượt

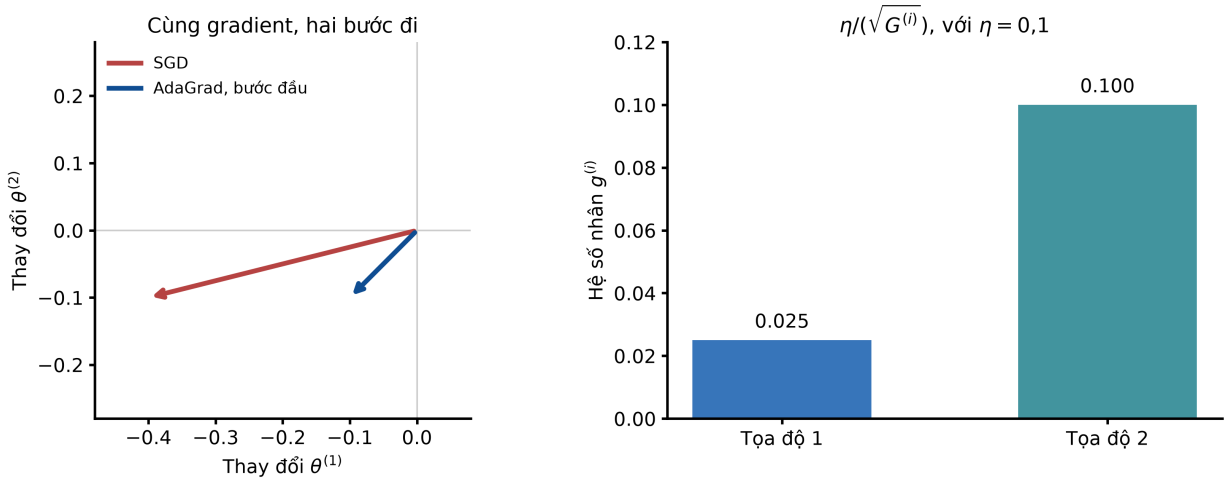
Với vector $g = (g^{(1)}, g^{(2)})$, ký hiệu g^2 trong chương có nghĩa **bình phương từng phần tử**: $((g^{(1)})^2, (g^{(2)})^2)$. Chỉ số trên trong ngoặc chỉ tọa độ, giống quy ước cho tham số; chỉ số dưới trong g_t chỉ bước. Phép căn và phép chia các vector cũng thực hiện từng phần tử, không phải phép chia ma trận. Bình phương bỏ dấu âm dương và đo độ lớn, do đó một gradient thường lớn sẽ để lại số lớn trong bộ nhớ bình phương.

Trung bình trượt hàm mũ, hay EMA, là quy tắc kết hợp bộ nhớ cũ a_{t-1} và giá trị mới z_t:

$$a_t = \beta a_{t-1} + (1-\beta) z_t, \qquad 0 \le \beta < 1.$$

Chẳng hạn (tự tính), với $a_0 = 0$, $\beta = 0{,}5$ và các giá trị mới lần lượt $2, 4, 0$, ta nhận $a_1 = 1$, $a_2 = 2{,}5$, $a_3 = 1{,}25$. Giá trị mới càng gần hiện tại càng có trọng số lớn; chọn β gần 1 cho bộ nhớ dài hơn và đáp ứng chậm hơn. EMA có thể làm mượt nhiễu, nhưng khi dữ liệu hoặc gradient đổi chế độ, bộ nhớ cũ cũng khiến nó phản ứng có độ trễ.

3. Trực giác bằng hình: cùng gradient, bước khác nhau



Hình 13: Cùng một gradient nhưng SGD và AdaGrad có thể đi theo hai hướng khác nhau (minh hoạ, tự tính)

Cách đọc hình: từ cùng gốc, mũi tên đỏ và xanh là **độ thay đổi tham số**, không phải hai điểm bắt đầu khác nhau; hai trục trái có cùng tỉ lệ. Ở ví dụ $g = (4, 1)$, $\eta = 0{,}1$, mũi tên SGD là $(-0{,}4, -0{,}1)$; mũi tên AdaGrad bước đầu là $(-0{,}1, -0{,}1)$ khi tạm bỏ ϵ (tự tính). Bên phải cho thấy hệ số nhân gradient lần lượt là $0{,}025$ và $0{,}1$: tọa độ có gradient lớn được chia nhiều hơn trong ví dụ này.

Mũi tên xanh không còn song song với mũi tên đỏ. Điều chỉnh từng tọa độ có thể đổi cả hướng lẫn độ dài của bước. Đây là ví dụ một bước, không chứng minh AdaGrad tốt hơn SGD: trên bài toán khác, các hướng, độ nhiễu và lịch sử có thể cho kết quả khác.

4. AdaGrad: cộng dồn bình phương từ đầu

4.1 Định nghĩa

Đặt $G_0 = 0$ cùng kích thước với θ. Ở mỗi bước t, AdaGrad dùng

$$G_t = G_{t-1} + g_t^2, \qquad \theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{G_t} + \epsilon}.$$

Tất cả phép toán trên vector thực hiện theo phần tử; $\epsilon > 0$ nhỏ giúp mẫu số khác 0 và cũng ảnh hưởng đến bước đi khi $\sqrt{G_t}$ rất nhỏ. Một số triển khai đặt hằng số ổn định bên trong căn hoặc khởi tạo bộ tích lũy bằng số dương: đó là quy ước khác, nên không thể chờ số hoàn toàn trùng nếu không đối chiếu công thức.

Vì G_t là tổng các số không âm, nó không giảm. Khi η cố định, hệ số $\eta/(\sqrt{G_t} + \epsilon)$ ở mỗi tọa độ không tăng. Một tọa độ thường có gradient lớn sẽ tích lũy nhanh và được nhân hệ số nhỏ hơn về sau. Nếu một tọa độ ít được kích hoạt, bộ tích lũy của nó có thể còn nhỏ, giúp lần cập nhật tiếp theo không bị giảm giống các tọa độ thường xuyên hoạt động. Đó là trực giác hữu ích cho đặc trưng thưa, nhưng độ lớn của bước còn phụ thuộc gradient hiện tại và ϵ.

4.2 Ví dụ tay: ba gradient liên tiếp

Xét **một tọa độ**, $\theta_0 = 1$, $\eta = 0{,}2$, $G_0 = 0$, gradient lần lượt là $3, 4, 0$. Để tính tay gọn, đặt $\epsilon = 0$ trong ví dụ này; các mẫu số đều khác 0. Các phép dưới đây đều do ta thực hiện (tự tính).

- Bước 1: $G_1 = 0 + 3^2 = 9$, $\sqrt{G_1} = 3$. Bước thay đổi là $-0{,}2 \cdot 3/3 = -0{,}2$, nên $\theta_1 = 0{,}8$.
- Bước 2: $G_2 = 9 + 4^2 = 25$, $\sqrt{G_2} = 5$. Bước thay đổi là $-0{,}2 \cdot 4/5 = -0{,}16$, nên $\theta_2 = 0{,}64$.
- Bước 3: $G_3 = 25 + 0^2 = 25$. Gradient bằng 0 nên tham số vẫn là $\theta_3 = 0{,}64$.

Dù gradient thứ hai lớn hơn, độ giảm tham số của nó nhỏ hơn vì mẫu số chứa cả hai gradient. Đây là chuỗi gradient **cho trước** để minh hoạ bộ nhớ, không giả định các số đó là gradient của một hàm cụ thể tại ba vị trí trên.

Điểm yếu đi cùng ưu điểm: nếu huấn luyện rất lâu và tiếp tục nhận gradient khác 0, tổng G_t có thể lớn khiến hệ số nhân trở nên rất nhỏ. Khi hướng hoặc tỉ lệ gradient thay đổi, AdaGrad vẫn mang cả lịch sử cũ. Sự giảm này không phải lỗi tính toán, mà là tính chất của bộ nhớ cộng dồn.

5. RMSProp: thay tổng vô hạn bằng bộ nhớ gần đây

5.1 Ý tưởng và công thức

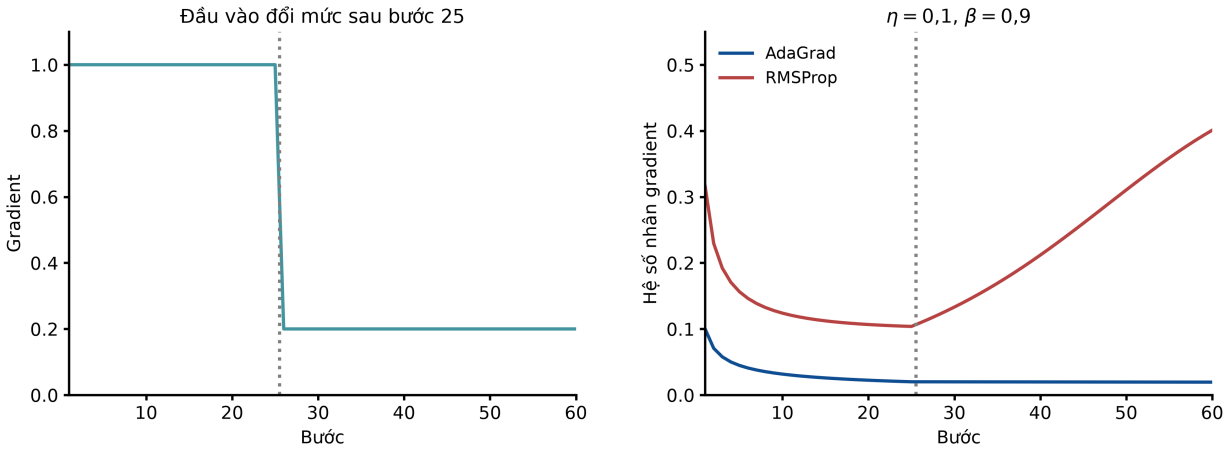
RMSProp thay tổng của AdaGrad bằng EMA của bình phương gradient. Với $v_0 = 0$ và $0 \le \beta < 1$,

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2, \qquad \theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}.$$

Tên RMS nhắc đến căn của trung bình bình phương. Chương dùng biến thể cơ bản, chưa thêm momentum và chưa sửa lệch khởi tạo; các thư viện có thể có lựa chọn khác. Không giống G_t của AdaGrad, v_t có thể giảm khi gradient gần đây nhỏ đi, vì mỗi bước làm nhẹ bớt các bình phương cũ. Do đó hệ số nhân gradient có thể tăng trở lại. Điều đó cũng có nghĩa bước sau không được bảo đảm an toàn chỉ vì bước trước ổn.

Ví dụ (tự tính), nếu $\beta = 0{,}5$, $v_0 = 0$ và gradient là $2, 0, 0$, thì $v_1 = 2$, $v_2 = 1$, $v_3 = 0{,}5$. Dù bộ chia nhỏ dần, ở hai bước có gradient 0 thì bước cập nhật RMSProp cơ bản bằng 0. Chỉ nói ”learning rate hiệu dụng tăng” chưa đủ để kết luận tham số đã di chuyển xa.

Cách đọc hình: bên trái gradient bằng 1 trong 25 bước rồi giảm xuống $0{,}2$; đường chấm dọc đánh dấu chỗ đổi chế độ (tự tính). Bên phải là hệ số nhân gradient với cùng $\eta = 0{,}1$, không phải độ dài bước thực tế: AdaGrad tiếp tục giảm nhẹ, còn hệ số RMSProp tăng khi bộ nhớ bình phương quên dần giá trị cũ. Bước đầu của RMSProp lớn hơn trong hình vì EMA bắt đầu từ 0 và biến thể này không có sửa lệch; không suy ra rằng thuật toán nào luôn tốt hơn.



Hình 14: AdaGrad cộng dồn còn RMSProp làm mượt lịch sử bình phương gradient (minh hoạ, tự tính)

5.2 Chọn độ dài bộ nhớ

β gần 1 tạo bộ nhớ dài: giá trị cũ giảm chậm, đường ước lượng mượt hơn nhưng phản ứng chậm khi phân phối gradient đổi. β nhỏ cho trọng số lớn hơn tới gradient gần nhất, nên phản ứng nhanh và có thể nhiễu hơn. Nếu ban đầu $v_0 = 0$ thì các bước đầu còn mang ảnh hưởng của điểm xuất phát, đặc biệt khi β gần 1. Adam dưới đây xử lý ảnh hưởng ấy bằng sửa lệch, đồng thời có thêm bộ nhớ hướng đi.

6. Adam: bộ nhớ hướng và bộ nhớ độ lớn

6.1 Hai moment có ý nghĩa gì

Adam viết tắt của adaptive moment estimation. Nó giữ hai EMA riêng: m_t nhớ gradient có dấu, còn v_t nhớ bình phương gradient. Với $m_0 = v_0 = 0$, $t \geq 1$:

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \\
\theta_t &= \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.
\end{aligned}
$$

$\hat{m}_t$ làm mượt hướng, gần tinh thần momentum; $\hat{v}_t$ đo mức bình phương gần đây của gradient. Đây là **moment bậc hai thô**, không phải phương sai: phương sai còn trừ bình phương trung bình. Bản Adam gốc đề xuất bộ giá trị khởi đầu thường dùng là $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\epsilon = 10^{-8}$; chúng không phải bảo đảm tối ưu cho mọi bài toán, xem optimization 06.

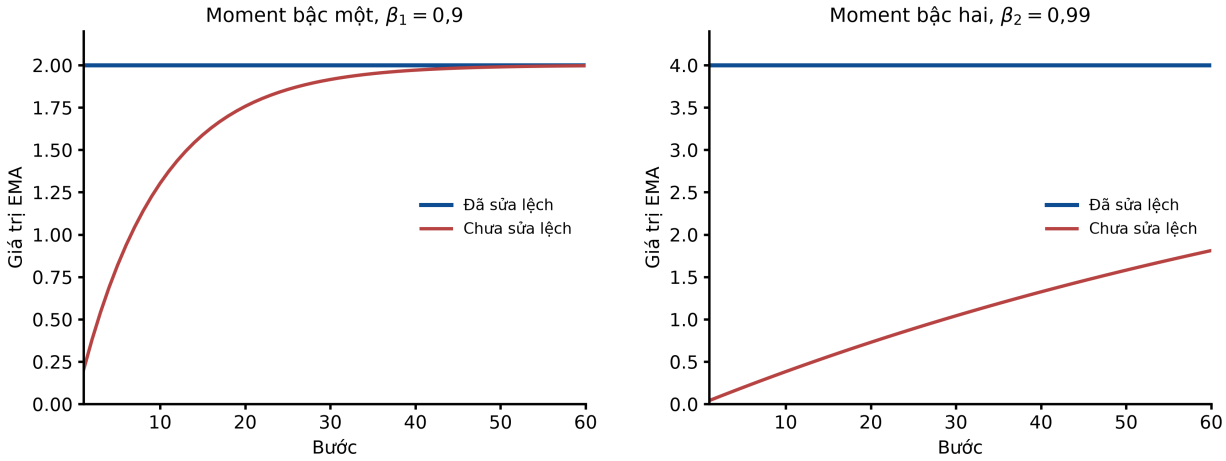
Khi các gradient cùng dấu liên tiếp, m_t có thể giữ hướng đó; khi chúng đổi dấu, các đóng góp có thể triệt tiêu. Mẫu số $\sqrt{\hat{v}_t} + \epsilon$ điều chỉnh độ lớn từng tọa độ. Với Adam, không nên gọi $\eta / (\sqrt{\hat{v}_t} + \epsilon)$ là hệ số nhân **gradient hiện tại**: nó nhân $\hat{m}_t$, tức lịch sử đã làm mượt, chứ không chỉ nhân g_t.

6.2 Vì sao phải sửa lệch

Từ $m_0 = 0$, nếu gradient là cùng một số c ở mọi bước thì khai triển truy hồi cho

$$
m_t = (1-\beta_1^t) c, \qquad v_t = (1-\beta_2^t) c^2.
$$

Do β_1, β_2 gần 1, ở những bước đầu hai số chưa sửa bị kéo về 0 so với c và c^2. Chia tương ứng cho $1-\beta_1^t$ và $1-\beta_2^t$ đưa chúng về đúng c và c^2 trong ví dụ gradient hằng. Ví dụ (tự tính), $g_1 = 2$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}99$ cho $m_1 = 0{,}2$, $v_1 = 0{,}04$; sau chia cho $0{,}1$ và $0{,}01$ ta được $\hat{m}_1 = 2$, $\hat{v}_1 = 4$.



Hình 15: Sửa lệch EMA khi gradient giữ nguyên tại giá trị 2 (minh hoạ, tự tính)

Cách đọc hình: trục ngang là bước; đường đỏ là EMA xuất phát từ 0, đường xanh là EMA sau khi chia hệ số sửa lệch. Bên trái giá trị đích bằng 2, bên phải bằng 4; hai hệ số nhớ khác nhau nên đường đỏ bên phải tăng chậm hơn (tự tính). Các đường xanh nằm ngang chỉ vì gradient trong hình giữ nguyên; với gradient thay đổi, sửa lệch không làm các moment trở thành giá trị tương lai hoặc giá trị chính xác không nhiễu.

Diễn giải bằng kỳ vọng: nếu kỳ vọng của g_t và g_t^2 tương ứng không đổi theo thời gian, phép chia loại được lệch do khởi tạo 0 của từng EMA. Khi phân phối gradient thay đổi, $\hat{m}_t$ và $\hat{v}_t$ vẫn là các trung bình có trọng số của quá khứ. Ngoài ra, dù từng moment có tính chất kỳ vọng thuận lợi dưới giả thiết ấy, **tỉ số** của chúng không tự động là một ước lượng không lệch.

6.3 Ví dụ tay hai bước Adam

Xét một tọa độ, $\theta_0 = 1$, $g_1 = 2$, $g_2 = -1$, $\beta_1 = \beta_2 = 0{,}5$, $\eta = 0{,}1$, $\epsilon = 0$. Gradient cho trước và mọi mẫu số khác 0; các số sau đều do ta thực hiện (tự tính).

Bước 1: $m_1 = 0{,}5 \cdot 0 + 0{,}5 \cdot 2 = 1$, $v_1 = 0{,}5 \cdot 0 + 0{,}5 \cdot 4 = 2$. Chia cho $1 - 0{,}5^1 = 0{,}5$, ta được $\hat{m}_1 = 2$, $\hat{v}_1 = 4$. Do đó $\theta_1 = 1 - 0{,}1 \cdot 2/\sqrt{4} = 0{,}9$.

Bước 2: $m_2 = 0{,}5 \cdot 1 + 0{,}5 \cdot (-1) = 0$; $v_2 = 0{,}5 \cdot 2 + 0{,}5 \cdot 1 = 1{,}5$. Chia cho $1 - 0{,}5^2 = 0{,}75$ cho $\hat{m}_2 = 0$ và $\hat{v}_2 = 2$. Tham số vẫn $\theta_2 = 0{,}9$: gradient hiện tại âm nhưng moment hướng đúng lúc triệt tiêu. Bước không phụ thuộc chỉ vào dấu của gradient mới nhất.

Nếu bỏ sửa lệch ở bước 1, tỉ số là $1/\sqrt{2}$ thay vì $2/\sqrt{4} = 1$, nên kết quả sẽ khác. Không thể chỉ nhìn một moment bị nhỏ rồi khẳng định bước Adam luôn lớn hơn hoặc luôn nhỏ hơn, vì cả tử và mẫu đều thay đổi.

7. AdamW: weight decay tách khỏi phần thích nghi

7.1 Phân biệt ba điều dễ gọi cùng tên

Một cách thêm regularization là cộng vào loss dữ liệu một hình phạt L2:

$$L_{\text{tong}}(\theta) = L_{\text{data}}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2.$$

Gradient của loss tổng là $g_t + \lambda\theta_{t-1}$, trong đó g_t là gradient **chỉ của loss dữ liệu**. Với SGD thuần không momentum, cập nhật dùng gradient tổng là

$$\theta_t = \theta_{t-1} - \eta(g_t + \lambda\theta_{t-1}) = (1 - \eta\lambda)\theta_{t-1} - \eta g_t.$$

Nó đúng bằng việc nhân tham số cũ với $1 - \eta\lambda$, rồi trừ bước dữ liệu. Hệ số λ ở đây định nghĩa theo quy ước có $\frac{1}{2}$ trong penalty. Nếu một tài liệu định nghĩa trực tiếp hệ số nhân là $1 - d$ thì cần đặt $d = \eta\lambda$ mới khớp. Tương đương này áp dụng cho SGD thuần với cùng các điều kiện vừa nêu; momentum và thuật toán thích nghi thay đổi cách gradient đi qua bộ nhớ hoặc bộ chia.

Với **Adam cộng L2 vào loss**, phần $\lambda\theta$ đi chung với gradient dữ liệu vào cả m_t và v_t. Nó vì thế bị làm mượt và chia theo từng tọa độ. Với **AdamW**, Adam tính $\hat{m}_t, \hat{v}_t$ từ gradient dữ liệu, còn weight decay áp thẳng vào tham số cũ:

$$\theta_t = (1 - \eta_t\lambda)\theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

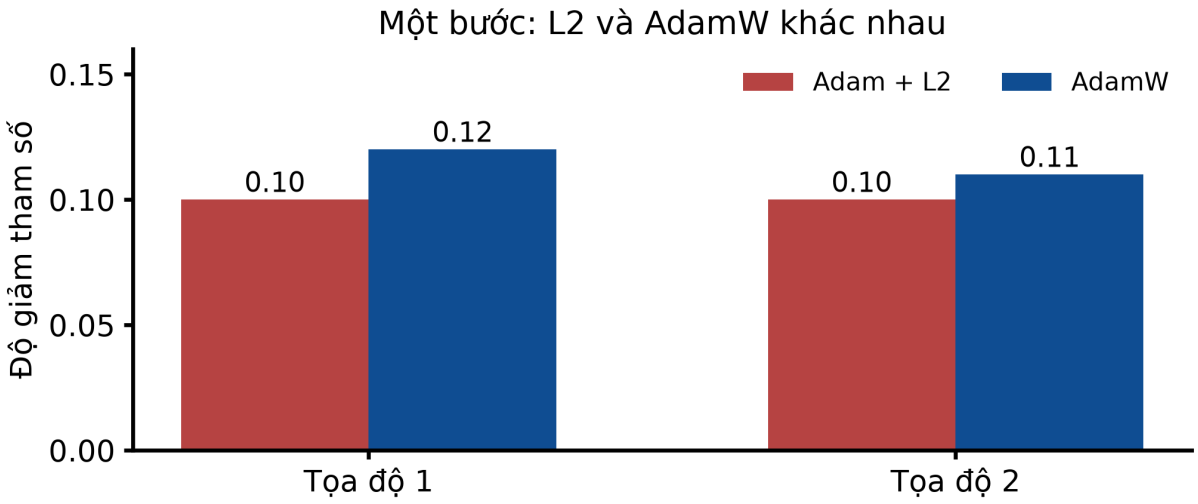
Đây là quy ước AdamW dùng trong chương; η_t có thể thay đổi theo lịch. Từ ”tách” nói đến **đường đi của decay**: nó không được đưa vào hai moment Adam. Nó không có nghĩa tổng tác động decay hoàn toàn độc lập với learning rate hay số bước, vì tích các hệ số $1 - \eta_t\lambda$ vẫn phụ thuộc cả hai.

7.2 Một phép tính cho thấy hai quy tắc không giống nhau

Lấy bước đầu, $\theta_0 = (2, 1)$, gradient dữ liệu $g_1 = (0{,}2, 0{,}2)$, $\eta = 0{,}1$, $\lambda = 0{,}1$, $\epsilon = 0$. Các hệ số β hợp lệ bất kỳ đều cho moment đã sửa ở bước đầu bằng gradient dùng để cập nhật và bình phương của nó. Tất cả số dưới đây đều do ta thực hiện (tự tính).

Adam với L2 trong loss: gradient vào Adam là $(0{,}2, 0{,}2) + 0{,}1(2, 1) = (0{,}4, 0{,}3)$. Ở bước đầu, chia mỗi thành phần dương cho trị tuyệt đối của nó cho $(1, 1)$. Bước giảm là $(0{,}1, 0{,}1)$, nên $\theta_1 = (1{,}9, 0{,}9)$.

AdamW: Adam chỉ nhìn gradient dữ liệu $(0{,}2, 0{,}2)$, nên bước theo dữ liệu là $(0{,}1, 0{,}1)$. Decay riêng là $\eta\lambda\theta_0 = (0{,}02, 0{,}01)$. Cộng hai tác động, $\theta_1 = (2, 1) - (0{,}12, 0{,}11) = (1{,}88, 0{,}89)$.



Hình 16: So sánh độ giảm tham số ở bước đầu của Adam với L2 và AdamW (minh hoạ, tự tính)

Cách đọc hình: mỗi nhóm cột ứng với một tọa độ, chiều cao là lượng tham số giảm trong ví dụ vừa tính. Cột đỏ bằng $0{,}1$ ở cả hai tọa độ, còn cột xanh lần lượt là $0{,}12$ và $0{,}11$ (tự tính). Đây là một phản ví dụ đủ để thấy

hai quy tắc không đồng nhất; nó không chứng minh AdamW luôn giảm tham số nhiều hơn hay luôn đạt loss thấp hơn.

Trong thực tế dùng $\epsilon > 0$; khi đó các giá trị gần, nhưng không đúng hoàn toàn, với ví dụ đã đặt $\epsilon = 0$. Một số thư viện gọi đối số là `weight_decay` nhưng có thể thực hiện kiểu cộng L2 hoặc kiểu tách tuỳ thuật toán và phiên bản. Đọc công thức hoặc tài liệu của chính optimizer, và kiểm nhóm tham số nào được áp decay. Nhiều cấu hình loại bias hoặc tham số của normalization khỏi decay, nhưng đó là lựa chọn thiết kế cần khai báo, không phải định luật áp dụng cho mọi mô hình.

8. Chọn và kiểm optimizer trong AI

Gradient thưa. Embedding và đặc trưng chỉ xuất hiện ở một số batch có thể làm các tọa độ nhận gradient với tần suất rất khác nhau. Bộ tích lũy từng tọa độ giúp biểu diễn sự khác nhau này, nhưng còn phải xét cách thư viện lưu gradient thưa, cập nhật trạng thái, regularization và chi phí bộ nhớ. Không phải mọi triển khai Adam đều hỗ trợ gradient thưa theo cùng cách.

Mạng sâu. Adam và AdamW cần giữ thêm hai tensor moment cùng kích thước các tham số được tối ưu. Điều này quan trọng khi mô hình lớn, vì bộ nhớ dùng cho trạng thái không chỉ có trọng số. Các biến thể như Adafactor ở optimization 12 là một hướng giảm chi phí, đổi lại có giả định và chi tiết cập nhật riêng.

Learning rate và warmup. Phép chia thích nghi không loại bỏ việc chọn η_t. Nếu learning rate quá lớn, hoặc bộ chia nhỏ bất thường, bước vẫn có thể mất ổn định. Warmup thay đổi learning rate toàn cục theo thời gian, còn Adam thay đổi hướng và tỉ lệ theo lịch sử từng tọa độ; chúng làm những việc khác nhau và có thể dùng cùng nhau. Xem nền tảng 06.

Đánh giá và so sánh. Optimizer không được xếp hạng bằng một giá trị loss đầu tiên. Cần giữ dữ liệu, mô hình, batch, ngân sách tính toán và cách đánh giá nhất quán; đồng thời điều chỉnh learning rate hợp lý cho từng phương pháp. So sánh một cấu hình mặc định với một cấu hình khác đã được tinh chỉnh có thể tạo kết luận sai. Loss train và chất lượng trên validation cũng trả lời hai câu hỏi khác nhau.

Giới hạn hội tụ. Các moment và bộ chia là cơ chế hữu ích chứ không phải bằng chứng mọi chuỗi gradient sẽ hội tụ. Adam có những ví dụ phản chứng dưới một số thiết lập; optimization 08 giải thích động lực của AMSGrad. Tương tự, việc gradient có độ lớn khác nhau không đủ kết luận đã biết Hessian hay đã tìm được phép biến đổi hình học tối ưu.

8.1 Checklist khi đọc một công thức

Trước khi thay optimizer trong một bài toán, hãy tìm câu trả lời cho các câu hỏi sau. Gradient được tính từ loss dữ liệu hay đã chứa penalty? Bình phương và phép chia có theo tọa độ không? ϵ nằm ở đâu? Moment có được sửa lệch hay không, và bộ đếm t được khôi phục khi tiếp tục từ checkpoint không? Decay đi vào moment hay tác động trực tiếp lên trọng số cũ?

Khi tiếp tục huấn luyện, chỉ lưu trọng số là không đủ để tái tạo cùng quỹ đạo. Ta còn cần trạng thái optimizer, bộ đếm bước, lịch learning rate và trạng thái dữ liệu/ngẫu nhiên khi muốn lặp lại chính xác. Reset moment tương đương đổi lịch sử, nên bước tiếp theo có thể khác dù tham số được nạp trùng nhau.

9. Sai lầm hay gặp

- **"Thích nghi" nghĩa là khỏi chọn learning rate.** Sai: η vẫn nhân toàn bộ bước, và lịch của nó vẫn ảnh hưởng đến quá trình huấn luyện.
- **Xem v_t là phương sai của gradient.** v_t là trung bình bình phương thô; phương sai còn phụ thuộc bình phương trung bình.
- **Cho rằng AdaGrad luôn có bước nhỏ dần.** Hệ số nhân không tăng với η cố định, nhưng gradient hiện tại thay đổi nên độ dài bước thực tế không nhất thiết giảm đều.

- **Cho rằng sửa lệch khôi phục gradient thật.** Nó sửa hệ số do khởi tạo EMA bằng 0 dưới lập luận đã nêu, không loại nhiễu mini-batch hoặc độ trễ của lịch sử.
- **Đồng nhất Adam cộng L2 với AdamW.** Với Adam, penalty L2 đi qua moment và bộ chia; AdamW tách decay khỏi chúng. Tương đương theo đại số ở mục 7 là cho SGD thuần.
- **Bỏ qua ϵ và trạng thái khi cài đặt.** Vị trí ϵ, nhóm tham số decay, moment và bộ đếm bước đều có thể làm hai lần chạy khác nhau.

10. Bảng tóm tắt

Mọi phép chia và căn trong bảng là theo từng phần tử; g_t là gradient dữ liệu trừ khi ghi khác.

Phương pháp	Bộ nhớ	Bước chính và ý nghĩa
SGD	Không có	$\theta_t = \theta_{t-1} - \eta g_t$, cùng hệ số cho mọi tọa độ
AdaGrad	$G_t = G_{t-1} + g_t^2$	Trừ $\eta g_t/(\sqrt{G_t} + \epsilon)$, tích luỹ cả lịch sử
RMSProp	$v_t = \beta v_{t-1} + (1-\beta) g_t^2$	Trừ $\eta g_t/(\sqrt{v_t} + \epsilon)$, nhớ gần đây hơn
Adam	EMA m_t và v_t, có sửa lệch	Trừ $\eta \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$
AdamW	Moment Adam từ gradient dữ liệu	Thêm riêng phần trừ $\eta\lambda\theta_{t-1}$

11. Bài tập tự tính

Các bài dưới đây là các phép tính độc lập, không phải số liệu thực nghiệm paper. Mỗi đáp án đã kiểm bằng numpy. Trừ khi nói khác, giả sử mọi phép toán theo từng tọa độ và $\epsilon = 0$ chỉ để tính tay.

Bài 1. SGD và gradient khác tỉ lệ

Cho $\theta_0 = (2, -1)$, $g_1 = (4, -2)$ và $\eta = 0{,}1$. Tính θ_1 của SGD và tính G_1, θ_1 của AdaGrad với $G_0 = 0$.

Đáp án (tự tính): SGD trừ $(0{,}4, -0{,}2)$, cho $(1{,}6, -0{,}8)$. AdaGrad có $G_1 = (16, 4)$, chia gradient cho $(4, 2)$ được $(1, -1)$, nên $\theta_1 = (1{,}9, -0{,}9)$.

Bài 2. Tổng và EMA

Với gradient một chiều $2, 0, 2$, tính G_1, G_2, G_3 của AdaGrad và v_1, v_2, v_3 của RMSProp, biết $G_0 = v_0 = 0$, $\beta = 0{,}5$.

Đáp án (tự tính): AdaGrad cho $G = (4, 4, 8)$ theo ba thời điểm. RMSProp cho $v_1 = 2$, $v_2 = 1$, $v_3 = 0{,}5 \cdot 1 + 0{,}5 \cdot 4 = 2{,}5$. Khác với G_t, v_t có thể giảm.

Bài 3. Sửa lệch Adam

Với một gradient không đổi $g = 3$, $m_0 = v_0 = 0$, $\beta_1 = 0{,}5$, $\beta_2 = 0{,}75$, tính m_1, v_1, $\hat{m}_1$, $\hat{v}_1$ và tỉ số dùng để cập nhật tại bước 1.

Đáp án (tự tính): $m_1 = 1{,}5$, $v_1 = 2{,}25$. Chia tương ứng cho $0{,}5$ và $0{,}25$ được $\hat{m}_1 = 3$, $\hat{v}_1 = 9$. Tỉ số $\hat{m}_1/\sqrt{\hat{v}_1} = 1$.

Bài 4. L2 và decay tách trong SGD

Cho $\theta_0 = 2$, gradient dữ liệu $g_1 = 3$, $\eta = 0{,}1$, $\lambda = 0{,}2$. Tính một bước SGD khi cộng $\frac{\lambda}{2}\theta^2$ vào loss và khi dùng decay tách $1 - \eta\lambda$.

Đáp án (tự tính): Theo L2, gradient tổng là $3 + 0{,}2 \cdot 2 = 3{,}4$, nên $\theta_1 = 2 - 0{,}1 \cdot 3{,}4 = 1{,}66$. Theo decay tách, $\theta_1 = (1 - 0{,}02) \cdot 2 - 0{,}1 \cdot 3 = 1{,}66$. Hai kết quả bằng nhau trong SGD thuần với quy ước này.

Bài 5. Một bước AdamW

Cho $\theta_0 = (1, -2)$, $g_1 = (2, -4)$, $\eta = 0{,}1$, $\lambda = 0{,}2$ và hai moment bắt đầu từ 0. Tính θ_1 của AdamW; giả sử các hệ số β hợp lệ và $\epsilon = 0$.

Đáp án (tự tính): Sửa lệch ở bước đầu cho $\hat{m}_1 = (2, -4)$, $\hat{v}_1 = (4, 16)$, nên tỉ số là $(1, -1)$. Phần cập nhật theo dữ liệu là $(-0{,}1, 0{,}1)$, phần decay là $(-0{,}02, 0{,}04)$. Vậy $\theta_1 = (0{,}88, -1{,}86)$.

Tài liệu tham khảo nền: Duchi, Hazan và Singer, Adaptive Subgradient Methods; Tieleman và Hinton, RMSProp; Kingma và Ba, Adam; Loshchilov và Hutter, Decoupled Weight Decay Regularization; các ghi chú optimization 03, 06, 07, 08 và 12 trong sách.

Nền tảng 06

Learning rate và lịch học: bước lớn để tiến, bước nhỏ để tinh chỉnh

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: gradient descent và SGD ở nền tảng 02, 03; momentum ở nền tảng 04; optimizer thích nghi ở nền tảng 05

Dùng ở: optimization 17 (schedule-free), optimization 18 (cosine và warm restart), optimization 19 (lịch theo chu kỳ), optimization 20 (WSD), optimization 21 (batch lớn và warmup), optimization 23 (batch size và decay)

> Đây là chương học từ đầu, không phải bản dịch một paper. Các phép tính ghi ”tự tính” và các hình minh hoạ đều đã kiểm bằng numpy.

1. Vì sao learning rate cần một lịch

Learning rate là hệ số điều khiển quy mô bước cập nhật; lịch learning rate là cách thay đổi hệ số đó theo tiến trình huấn luyện. Khi gradient khác 0, nó chỉ hướng tăng nhanh nhất tại chỗ, còn hướng âm của gradient là một hướng giảm cục bộ; learning rate điều chỉnh quy mô bước theo hướng được thuật toán đề xuất. Đi quá ngắn có thể lãng phí thời gian, đi quá dài có thể vượt qua vùng tốt hoặc mất ổn định.

Trong huấn luyện AI, tình huống còn thay đổi theo thời gian. Ban đầu các tham số và trạng thái optimizer vừa khởi tạo; giữa quá trình mô hình đang học từ dữ liệu; về cuối, nhiễu của từng mini-batch có thể khiến tham số dao động quanh một vùng. Một lịch học cho phép dành mức bước khác nhau cho các giai đoạn đó. Warmup tăng dần lúc đầu, cosine giảm trơn, còn WSD tách warmup, một pha ổn định và pha giảm cuối.

Không có lịch nào tốt nhất cho mọi mạng, dữ liệu và ngân sách. Mục tiêu của chương là giúp đọc được một lịch, tự tính vài mốc và hiểu vì sao kết quả phụ thuộc vào cả **hình dạng lịch lẫn tổng số lần cập nhật**.

2. Nhắc nền: một bước học và một lần đếm

Với hàm mất mát L và một tham số θ, gradient cho biết độ dốc tại vị trí hiện tại. Gradient descent không momentum dùng

$$\theta_{t+1} = \theta_t - \eta_t g_t, \qquad g_t = \nabla L(\theta_t),$$

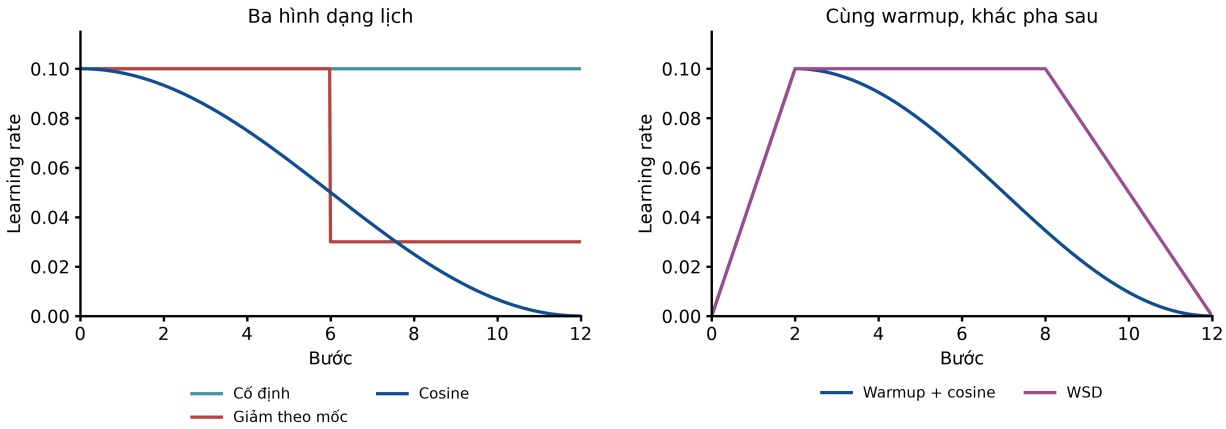
trong đó $\eta_t \geq 0$ là learning rate của bước. Nếu dùng mini-batch, g_t là gradient ước lượng từ batch hiện tại. Ở vector nhiều chiều, gradient và tham số là vector, còn η_t trong công thức SGD cơ bản là một hệ số vô hướng nhân mọi tọa độ.

Ví dụ (tự tính), lấy $L(\theta) = \frac{1}{2}\theta^2$ và $\theta_0 = 4$ thì $g_0 = 4$. Nếu $\eta_0 = 0{,}1$, bước đầu đưa θ đến $3{,}6$; nếu $\eta_0 = 0{,}01$, nó đến $3{,}96$. Cả hai đi theo cùng hướng, nhưng độ dài khác nhau mười lần. Không thể kết luận learning rate lớn hơn luôn tốt hơn: giới hạn ổn định sẽ được tính ở mục 5.

Một **optimizer step** là một lần cập nhật tham số. Một **epoch** thường là một lượt duyệt tập huấn luyện; một **mini-batch** là phần dữ liệu dùng cho một gradient. Nếu có 1.000 mẫu, batch gồm 25 mẫu và mỗi batch tạo một cập nhật thì một epoch có 40 cập nhật; bốn epoch có 160 cập nhật (tự tính). Ví dụ giả định dữ liệu chia hết, không bỏ batch và không có cập nhật bổ sung.

Nếu cộng gradient của bốn micro-batch trước khi cập nhật, ta chỉ có 10 optimizer step mỗi epoch trong ví dụ vừa rồi (tự tính). Khi đó lịch được tính theo optimizer step phải chạy theo số lần **cập nhật**, không phải mỗi lần tính gradient trên micro-batch. Các trạng thái phụ thuộc từng micro-batch, ví dụ normalization theo batch hoặc ngẫu nhiên hoá, vẫn có thể làm kết quả khác một batch lớn duy nhất.

3. Nhìn các lịch trước khi xem công thức



Hình 17: Các hình dạng learning rate cố định, theo mốc, cosine và WSD (minh hoạ, tự tính)

Cách đọc hình: trục ngang là tiến trình cập nhật từ 0 đến 12, trục dọc là learning rate, không phải loss hay độ chính xác. Bên trái, đường cố định giữ nguyên, đường theo mốc hạ đột ngột, còn cosine giảm trơn về 0. Bên phải, cả hai đường tăng trong warmup, sau đó cosine bắt đầu giảm còn WSD giữ một đoạn ổn định trước khi giảm tuyến tính; giá trị đỉnh và các mốc đều do ta chọn để minh hoạ (tự tính).

Hai lịch có cùng đỉnh vẫn không tương đương. Đường WSD ở đây nằm cao hơn cosine trong nhiều thời điểm, nhưng điều đó chỉ mô tả lượng learning rate được cấp, không tự chứng minh loss hay chất lượng của nó tốt hơn. Cùng một lịch cũng có thể cho kết quả khác khi đổi optimizer, batch size hoặc mô hình.

4. Định nghĩa và quy ước đếm thời gian

Ta mô tả lịch bằng một hàm $\eta(t)$ xác định trên đoạn từ 0 đến T. Để nối hình liên tục với chương trình rời rạc, giả sử có T cập nhật, đánh số $t = 0, \dots, T-1$, và cập nhật thứ t dùng $\eta_t = \eta(t)$. Điểm cuối $\eta(T)$ mô tả giới hạn của lịch sau cập nhật cuối, không nhất thiết được dùng cho một bước. Một thư viện khác có thể dùng chỉ số bắt đầu từ 1 hoặc lấy giá trị cuối trong các bước thực tế; hãy ghi rõ quy ước khi đối chiếu.

Một lịch thường cần quyết định:

1. **Đơn vị:** optimizer step, epoch, số mẫu hay số token đã xử lý. Các đơn vị chỉ đổi ra nhau khi biết batch và cách đếm.
2. **Mức đỉnh:** $\eta_{\max}$, learning rate lớn nhất của lịch được chọn.
3. **Mốc thời gian:** tổng thời lượng T, số bước warmup W, thời điểm bắt đầu decay S nếu có.
4. **Mức cuối:** $\eta_{\min}$ có thể bằng 0 hoặc dương.
5. **Quy tắc ở biên:** thời điểm nào thay pha và cập nhật nào dùng giá trị tại mốc.

Với SGD, η_t trực tiếp nhân gradient hiện tại. Với momentum hoặc Adam, nó nhân một hướng đã có bộ nhớ và/hoặc đã được chia theo tọa độ, nên cùng một η_t không có nghĩa là hai optimizer đi bước giống nhau. Với AdamW theo quy ước chương 05, nó còn nhân weight decay ở bước đó. Một đường learning rate vì vậy không tự cho biết toàn bộ quỹ đạo của tham số.

5. Ví dụ chính xác: quá lớn sẽ vượt điểm tốt

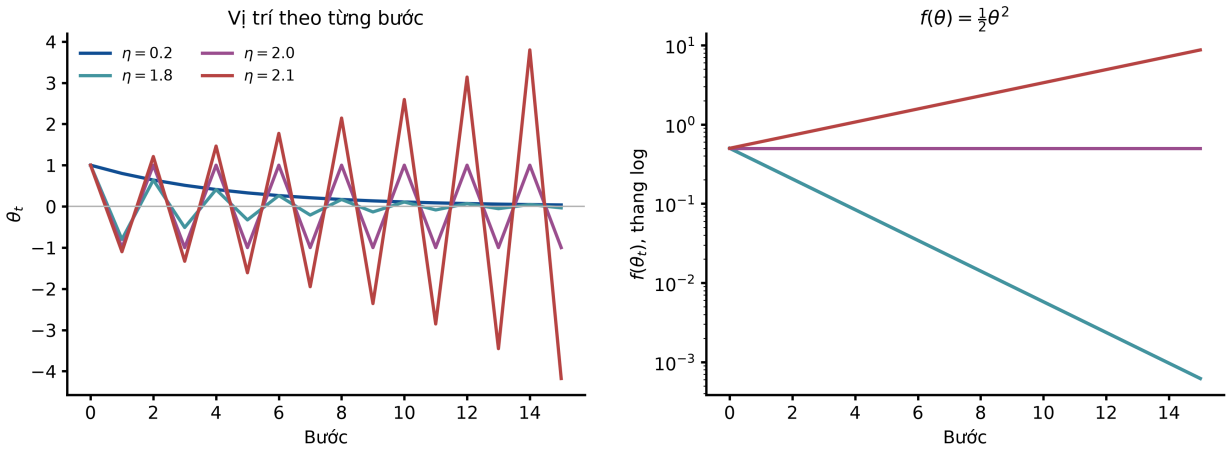
Xét lại $L(\theta) = \frac{1}{2}\theta^2$, gradient chính xác $g_t = \theta_t$ và $\theta_0 = 1$. Với learning rate cố định η,

$$\theta_{t+1} = (1-\eta)\theta_t, \qquad \theta_t = (1-\eta)^t \theta_0.$$

Khoảng cách đến nghiệm $\theta^* = 0$ giảm về 0 khi $|1-\eta| < 1$, tức $0 < \eta < 2$. Nếu $0 < \eta < 1$ thì điểm đi về 0 từ cùng một phía; nếu $1 < \eta < 2$ thì điểm đổi phía ở mỗi bước nhưng biên độ giảm. Tại $\eta = 2$ biên độ không giảm; trên 2 nó tăng. Đây là điều kiện chính xác cho **hàm và quy tắc đơn giản này**, không phải ngưỡng chung của mọi mạng hay Adam.

Các chuỗi sau xuất phát từ cùng $\theta_0 = 1$ (tự tính):

Learning rate	Bốn vị trí đầu $\theta_0, \theta_1, \theta_2, \theta_3$	Điều xảy ra
0,2	$1,\ 0{,}8,\ 0{,}64,\ 0{,}512$	Đi cùng phía và tiến về 0
1,8	$1,\ -0{,}8,\ 0{,}64,\ -0{,}512$	Đối phía, biên độ giảm
2	$1,\ -1,\ 1,\ -1$	Dao động không giảm
2,1	$1,\ -1{,}1,\ 1{,}21,\ -1{,}331$	Dao động lớn dần



Hình 18: Quỹ đạo và loss của bốn learning rate trên hàm bậc hai (minh hoạ, tự tính)

Cách đọc hình: bên trái là vị trí có thể vượt qua 0; đường đỏ đổi phía và vọt xa dần, còn đường tím lặp lại với biên độ không đổi. Bên phải là loss trên thang log: đường đỏ tăng, đường tím nằm ngang, các đường xanh giảm. Hai đường loss của $\eta = 0{,}2$ và $\eta = 1{,}8$ trùng nhau vì trị tuyệt đối hệ số đều bằng $0{,}8$; vị trí của chúng vẫn khác dấu ở các bước lẻ (tự tính).

Với hàm một chiều $L(\theta) = \frac{a}{2}\theta^2$ và $a > 0$, cách tính tương tự cho điều kiện $0 < \eta < 2/a$. Độ cong a càng lớn, miền bước cố định ổn định càng hẹp. Trong nhiều chiều, các hướng có thể có độ cong khác nhau; với hàm bậc hai dương xác định, hướng cong nhất giới hạn learning rate của gradient descent thông thường. Một mạng sâu thực tế không giữ độ cong cố định và gradient thường nhiễu, nên hình này là công cụ tư duy chứ không phải phép chọn ngưỡng phổ quát.

6. Warmup: tăng từ từ ở đầu

Warmup là giai đoạn ban đầu trong đó learning rate tăng từ một mức nhỏ lên mức đỉnh. Một lựa chọn đơn giản là tăng tuyến tính từ 0:

$$\eta(t) = \eta_{\max} \frac{t}{W}, \qquad 0 \le t \le W, \quad W > 0.$$

Ở $t = 0$ lịch bằng 0, ở $t = W$ đạt $\eta_{\max}$. Một số triển khai bắt đầu tại giá trị dương, chọn chỉ số khác hoặc dùng dạng tăng không tuyến tính, nên đồ thị và số bước đầu thực tế có thể khác. Trong quy ước cập nhật $t = 0, \ldots, T-1$ của chương, $\eta_0 = 0$ vẫn là một optimizer step nếu chương trình thực sự gọi cập nhật. Với optimizer có trạng thái, moment hoặc bộ đếm có thể thay đổi ngay cả khi tham số chưa di chuyển; cần xem cách triển khai.

Ví dụ (tự tính), $W = 2$, $\eta_{\max} = 0{,}1$: các giá trị tại $t = 0, 1, 2$ là 0, $0{,}05$, $0{,}1$. Mức tại $t = 2$ là đầu pha kế tiếp. Ví dụ quá ngắn để là khuyến nghị huấn luyện, nhưng giúp ta nhìn rõ điểm nối.

Vì sao dùng warmup? Một khả năng là tránh áp ngay bước lớn khi trọng số hoặc các thống kê optimizer còn đang hình thành; trong huấn luyện batch lớn, nó cũng là một công cụ ổn định thực nghiệm, xem optimization 21. Lý do chính xác còn phụ thuộc cấu hình, và không phải mọi mô hình đều cần warmup. Sửa lệch Adam và warmup cũng khác nhau: sửa lệch điều chỉnh moment khởi tạo bằng 0; warmup điều chỉnh learning rate toàn cục.

Warmup quá ngắn có thể không đủ nếu mức đỉnh đã chọn gây mất ổn định. Warmup quá dài dành nhiều ngân sách cho bước nhỏ và có thể làm chậm tiến bộ. Đây là lựa chọn để đo trên bài toán cụ thể, không có một tỉ lệ cố định đảm bảo tốt.

7. Cosine: giảm mượt sau warmup

Giả sử tổng tiến trình là T, warmup kết thúc tại $W < T$ và learning rate cuối là $\eta_{\min}$. Với phần sau warmup ta chọn

$$\eta(t) = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos\left(\pi \frac{t - W}{T - W}\right)\right), \qquad W \le t \le T.$$

Tại $t = W$, cos bằng 1 và lịch khớp mức đỉnh. Ở giữa phần decay, cos bằng 0 và lịch bằng trung bình của mức đỉnh với mức cuối. Tại $t = T$, cos bằng -1 và lịch chạm mức cuối. Chọn $\eta_{\min} > 0$ giữ một learning rate dương ở cuối; chọn 0 không có nghĩa mọi cập nhật thực tế nhất thiết dùng chính giá trị 0, do quy ước chỉ số ở mục 4.

Ví dụ (tự tính) khớp panel bên phải của hình các lịch: $W = 2$, $T = 12$, $\eta_{\max} = 0{,}1$, $\eta_{\min} = 0$. Khi $t = 2$, $4{,}5$, 7, $9{,}5$, 12, tỉ lệ hoàn thành decay lần lượt là 0, $\frac{1}{4}$, $\frac{1}{2}$, $\frac{3}{4}$, 1. Learning rate tương ứng xấp xỉ $0{,}1$, $0{,}08536$, $0{,}05$, $0{,}01464$, 0 (tự tính). Các mốc nửa bước dùng để đọc hàm liên tục; chương trình chỉ lấy những mốc phù hợp đơn vị cập nhật.

Cosine trong chương gồm một lần giảm, không tự động có restart. **Warm restart** là chủ động tăng learning rate trở lại rồi chạy một chu kỳ mới, thường vẫn tiếp tục từ trọng số đã học. Bài optimization 18 giới thiệu một thiết kế cosine với warm restart; độ dài chu kỳ và cách xử lý trạng thái optimizer là những quyết định bổ sung. Đừng nhầm warm restart với warmup chỉ vì cả hai có chữ ”warm”.

8. WSD: warmup, stable, decay

8.1 Ba pha và một công thức minh hoạ

WSD là viết tắt của **Warmup, Stable, Decay**: tăng ban đầu, giữ learning rate ở mức ổn định, rồi giảm khi chuẩn bị kết thúc. Một phiên bản đơn giản có warmup và decay tuyến tính được viết, với $0 < W < S < T$ và mức cuối 0, như sau:

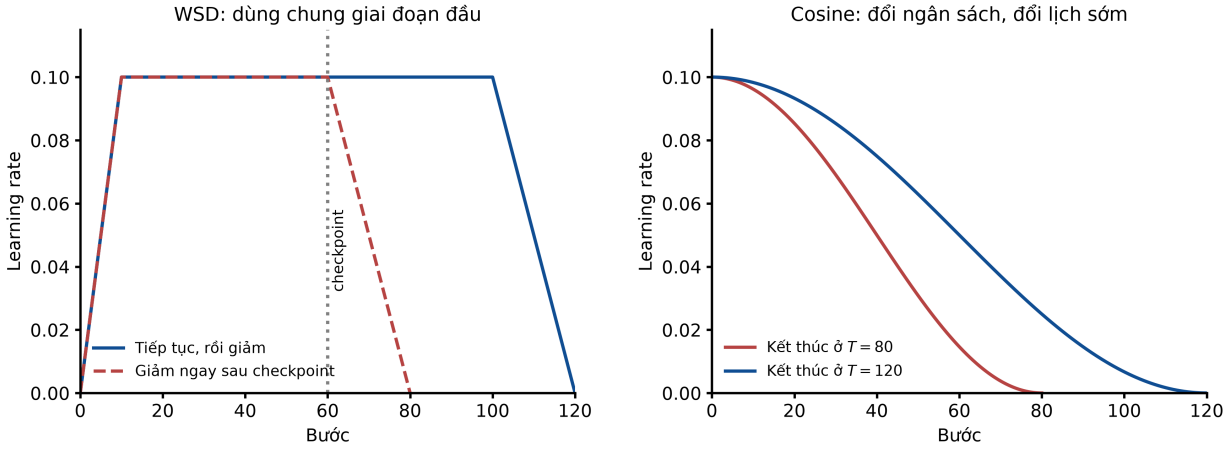
$$\eta(t) = \begin{cases} \eta_{\max} \dfrac{t}{W}, & 0 \le t < W, \\ \eta_{\max}, & W \le t < S, \\ \eta_{\max} \dfrac{T - t}{T - S}, & S \le t \le T. \end{cases}$$

Công thức này là **một lựa chọn WSD** để tính tay, không phải định nghĩa bắt buộc cho mọi triển khai. Pha decay có thể có hình dạng và mức cuối khác, miễn các quy ước được nêu rõ. Bài optimization 20 giới thiệu bối cảnh WSD trong MiniCPM; chương không gán các con số minh hoạ của mình cho kết quả paper.

Ví dụ (tự tính), với $W = 2$, $S = 8$, $T = 12$, $\eta_{\max} = 0{,}1$, các giá trị tại $t = 0, 1, 2, 8, 10, 12$ lần lượt là 0, $0{,}05$, $0{,}1$, $0{,}1$, $0{,}05$, 0. Công thức hai phía cho cùng giá trị tại các mốc nối, nên không có cú nhảy tại W hoặc S. Lịch này chính là đường tím trong hình các lịch.

8.2 Phân nhánh khi chưa chắc thời điểm dừng

Trong pha stable, có thể lưu một checkpoint, sau đó từ checkpoint ấy thử một nhánh decay để có mô hình kết thúc sớm hơn, trong khi một nhánh khác tiếp tục pha stable trước khi decay muộn hơn. Hai nhánh dùng chung công việc **trước** checkpoint; sau đó mỗi nhánh vẫn cần chạy các bước riêng. Trọng số, trạng thái optimizer và tiến trình lịch đều phải được lưu nếu muốn nối tiếp đúng trạng thái. Tuỳ mục tiêu tái lập, còn cần trạng thái dữ liệu và bộ sinh số ngẫu nhiên.



Hình 19: Dùng checkpoint chung của WSD và so sánh cosine với hai ngân sách (minh hoạ, tự tính)

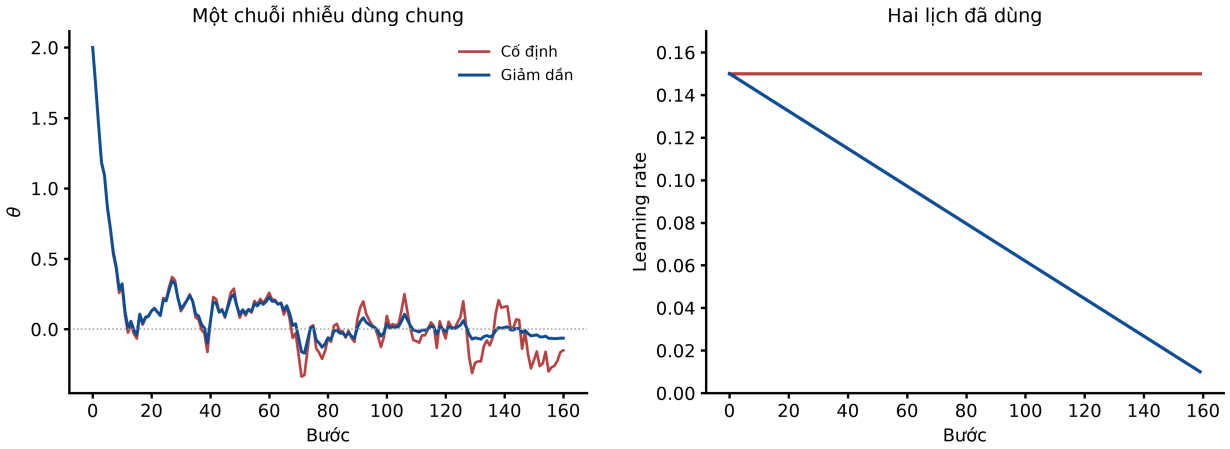
Cách đọc hình: bên trái, hai lịch trùng nhau trước checkpoint ở bước 60; nhánh đỏ bắt đầu giảm ngay và kết thúc ở 80, nhánh xanh giữ ổn định đến 100 rồi giảm đến 120 (tự tính). Bên phải, hai đường cosine được định sẵn cho các ngân sách khác nhau đã khác nhau từ trước bước 60: đổi thời điểm kết thúc cũng đổi lịch ở các thời điểm sớm. Hình chỉ so sánh lịch, không vẽ loss hay khẳng định các nhánh có cùng chất lượng cuối.

Tại bước 40 của hai cosine không warmup bên phải, lịch kết thúc tại $T = 80$ có $\eta = 0{,}05$; lịch kết thúc tại $T = 120$ có $\eta = 0{,}075$ (tự tính). Nếu đã chạy theo lịch ngắn, không thể coi trọng số tại bước 40 như thể đã chạy theo lịch dài. Một lịch WSD có thể hoãn quyết định bắt đầu decay, nhưng vẫn phải chọn mức ổn định, độ dài và hình dạng decay; nó không làm kết quả cuối tốt ngay lập tức tại mọi checkpoint.

9. Gradient nhiễu: vì sao cần bước nhỏ về cuối

Với mini-batch, ngay cả gần một nghiệm, gradient của mỗi batch thường không trùng gradient toàn dữ liệu. Nếu learning rate vẫn lớn, các cập nhật nhiễu có thể làm tham số tiếp tục dao động. Giảm learning rate làm

nhỏ cả tín hiệu lẫn nhiễu của cập nhật SGD: nó có thể giúp tinh chỉnh, đồng thời cũng khiến việc sửa sai hoặc thích ứng với dữ liệu mới chậm đi.



Hình 20: Hai lịch cùng sử dụng một chuỗi nhiễu gradient trên hàm bậc hai (minh hoạ, tự tính)

Cách đọc hình: bên trái là hai quỹ đạo tham số khởi đầu giống nhau, tính với cùng một chuỗi nhiễu được tạo sẵn; bên phải là learning rate đã cấp cho từng quỹ đạo. Đường xanh giảm bước tuyến tính và trong lần chạy này dao động cuối nhỏ hơn đường đỏ giữ bước cố định (tự tính). Đây chỉ là một chuỗi nhiễu minh hoạ, không phải trung bình nhiều lần chạy hay bằng chứng rằng giảm luôn cho nghiệm tốt hơn trong mọi nhiệm vụ.

9.1 Một mô hình xác suất đơn giản

Ta có thể thấy sự đánh đổi bằng đại số. Giả sử $L(\theta) = \frac{1}{2}\theta^2$, gradient ước lượng là $g_t = \theta_t + \xi_t$, trong đó ξ_t độc lập giữa các bước, có trung bình 0 và phương sai σ^2, đồng thời độc lập với trạng thái hiện tại. Với $0 < \eta < 2$ cố định:

$$\theta_{t+1} = (1 - \eta)\theta_t - \eta\xi_t.$$

Bình phương, lấy kỳ vọng và dùng giả thiết độc lập làm số hạng chéo có kỳ vọng 0:

$$\mathbb{E}[\theta_{t+1}^2] = (1 - \eta)^2\mathbb{E}[\theta_t^2] + \eta^2\sigma^2.$$

Về dài hạn, bình phương trung bình tiến đến giá trị V thoả $V = (1 - \eta)^2 V + \eta^2\sigma^2$. Giải ra

$$V = \frac{\eta\sigma^2}{2 - \eta}.$$

Ví dụ (tự tính), nếu $\sigma^2 = 0{,}36$, mức $\eta = 0{,}15$ cho $V \approx 0{,}02919$, còn $\eta = 0{,}03$ cho $V \approx 0{,}00548$. Trong mô hình này mức cố định nhỏ hơn có sàn dao động dài hạn nhỏ hơn, nhưng tiến về vùng gần nghiệm cũng có thể chậm hơn. Đây là kết luận dưới các giả thiết xác suất đã nêu, không phải công thức chung của mọi mạng, mọi optimizer hay lịch thay đổi theo thời gian.

10. Từ công thức đến một lần huấn luyện AI

10.1 Chọn đơn vị trước khi chọn con số

Nếu một lịch nói ”warmup 1.000 bước”, phải xác định bước là optimizer step, micro-batch hay đơn vị khác. Batch size và gradient accumulation thay đổi số optimizer step trên một lượng dữ liệu, trong khi độ dài chuỗi

có thể thay đổi số token trong một batch. Nếu tăng gấp đôi batch mà giữ nguyên số epoch và dữ liệu chia hết, số optimizer step giảm một nửa (tự tính); giữ nguyên warmup theo số bước khi ấy làm nó chiếm tỉ lệ khác trong toàn bộ lần chạy.

So sánh lịch theo epoch cũng cần ghi batch, số mẫu và cách xử lý batch cuối. So sánh theo token hữu ích khi độ dài chuỗi thay đổi, nhưng số token không đồng nhất với số cập nhật. Không có phép đổi đơn vị duy nhất nếu cách gom dữ liệu thay đổi giữa chừng.

10.2 Lịch và optimizer là hai bộ phận phối hợp

Với AdamW, learning rate nhân cả bước Adam đã thích nghi lẫn decay tách trong quy ước ở chương 05. Khi η_t thay đổi, lượng co tham số ở mỗi bước cũng thay đổi. So sánh hai lịch cùng λ nhưng khác learning rate hoặc số bước không cô lập hoàn toàn tác động tích luỹ của decay. Với momentum, hướng cập nhật mang bộ nhớ của các gradient trước; giảm η_t không xoá bộ nhớ đó.

Gradient clipping, khởi tạo và normalization cũng ảnh hưởng đến ổn định. Nếu loss nổ hoặc xuất hiện số không hữu hạn, cần xem gradient, dữ liệu và các thành phần ấy cùng learning rate, thay vì mặc định rằng chỉ cần thêm warmup. Ngược lại, một loss giảm rất chậm không tự chứng minh learning rate quá nhỏ: dữ liệu, mô hình và cách tính loss cũng có thể là nguyên nhân.

10.3 Checkpoint và tiếp tục chạy

Muốn tiếp tục đúng quỹ đạo, lưu trọng số, trạng thái optimizer, số optimizer step và trạng thái scheduler. Muốn tái lập chặt hơn, lưu vị trí dữ liệu, trạng thái bộ sinh ngẫu nhiên và các thành phần liên quan. Nếu chỉ nạp trọng số nhưng reset scheduler về đầu, learning rate có thể đột ngột tăng hoặc chạy lại warmup; nếu chỉ reset moment Adam, hướng bước cũng đổi.

Với cosine, quyết định kéo dài ngân sách sau khi lịch đã giảm cần một quy tắc mới được ghi rõ: tiếp tục với mức hiện tại, thiết kế một đoạn chuyển tiếp hay lên lịch lại là các thí nghiệm khác nhau. Không thể nói rằng chạy tiếp tự động tương đương đã biết ngân sách dài ngay từ đầu. Với WSD, nhánh decay và nhánh tiếp tục stable cũng là các lần chạy riêng sau checkpoint chung.

10.4 Kiểm một lựa chọn lịch

Trước khi chạy dài, vẽ chính các giá trị η_t mà chương trình sẽ dùng, kể cả bước đầu, hai phía điểm nối và bước cuối. Kiểm việc gọi scheduler trước hay sau optimizer theo quy ước của thư viện; sai thứ tự có thể làm lệch một bước hoặc bỏ qua một giá trị. Khi dùng accumulation, chỉ tăng bộ đếm lịch đúng lúc có optimizer step theo thiết kế.

Khi so sánh, giữ cùng mô hình, dữ liệu và ngân sách, cho mỗi lịch một mức đỉnh hợp lý, và theo dõi cả train lẫn validation. Nếu kết quả biến động theo khởi tạo hoặc thứ tự batch, chạy nhiều hạt giống và báo sự biến động giúp đọc kết quả thận trọng hơn. Loss thấp tại một thời điểm không đảm bảo validation tốt hơn, và một ví dụ một chiều không thay thế được phép đo trên bài toán thật.

11. Sai lầm hay gặp

- **Xem learning rate như độ dài bước thực tế.** Nó là hệ số trong quy tắc cập nhật; gradient, momentum, chia thích nghi và decay cũng quyết định độ dài.
- **Cho rằng Adam không cần lịch.** Adam có các bộ nhớ và hệ số theo tọa độ, nhưng learning rate toàn cục vẫn nhân lên bước cập nhật.
- **Coi warmup là sửa lệch Adam.** Chúng điều chỉnh những đại lượng khác nhau, có thể cùng dùng hoặc không.
- **Nhầm cosine đơn với warm restart.** Cosine một chu kỳ chỉ giảm; restart cần quy tắc tăng lại và các mốc chu kỳ.

- **Coi checkpoint WSD là mô hình đã kết thúc decay.** Checkpoint stable là điểm có thể phân nhánh; nhánh giảm vẫn cần thêm tính toán và đánh giá riêng.
- **Quên đơn vị đếm và trạng thái scheduler.** Epoch, micro-batch, optimizer step và token không luôn tương đương; khôi phục sai bộ đếm có thể tạo lịch khác dự định.

12. Bảng tóm tắt

Khái niệm	Công thức hoặc dấu hiệu nhận biết	Điều cần nhớ
SGD	$\theta_{t+1} = \theta_t - \eta_t g_t$	Bước thực tế còn phụ thuộc gradient
Warmup tuyến tính	$\eta(t) = \eta_{\max} t/W$	Tăng từ 0 lên đỉnh trên đoạn từ 0 đến W
Cosine decay	$\eta_{\min} + \frac{\eta_{\max}-\eta_{\min}}{2}(1+\cos(\pi(t-W)/(T-W)))$	Giảm trơn từ đỉnh về mức cuối
WSD	Warmup, giữ ổn định, rồi decay	Thời điểm decay và hình dạng cần được chọn
SGD trên $\frac{1}{2}\theta^2$	$\theta_{t+1} = (1-\eta)\theta_t$	Hội tụ khi $0 < \eta < 2$ trong ví dụ này

13. Bài tập tự tính

Các bài sau độc lập với thí nghiệm paper. Đáp án là tự tính và đã kiểm bằng numpy. Theo quy ước của chương, một cập nhật tại t dùng $\eta(t)$; hàm lịch cũng được định nghĩa tại mốc kết thúc T.

Bài 1. Hai bước SGD

Cho $L(\theta) = \frac{1}{2}\theta^2$, $\theta_0 = 2$, $\eta_0 = 0{,}5$ và $\eta_1 = 0{,}25$. Tính θ_1, θ_2 và $L(\theta_2)$.

Đáp án (tự tính): $\theta_1 = 2 - 0{,}5 \cdot 2 = 1$; $\theta_2 = 1 - 0{,}25 \cdot 1 = 0{,}75$. Loss cuối là $\frac{1}{2} \cdot 0{,}75^2 = 0{,}28125$.

Bài 2. Điều kiện ổn định

Cho $L(\theta) = 2\theta^2$, gradient descent chính xác và learning rate không đổi. Với $\theta_0 \neq 0$, khoảng learning rate dương nào làm θ_t hội tụ về 0? Điều gì xảy ra tại biên trên?

Đáp án (tự tính): Gradient bằng 4θ, nên $\theta_{t+1} = (1-4\eta)\theta_t$. Điều kiện $|1-4\eta| < 1$ cho $0 < \eta < 0{,}5$. Tại $\eta = 0{,}5$, hệ số là -1, tham số đổi dấu mỗi bước với biên độ không đổi.

Bài 3. Warmup

Dùng warmup tuyến tính từ 0 với $W = 4$ và $\eta_{\max} = 0{,}2$. Tính lịch tại $t = 0, 1, 2, 4$.

Đáp án (tự tính): Áp $\eta(t) = 0{,}2t/4$ cho các giá trị $0,\ 0{,}05,\ 0{,}1,\ 0{,}2$ tương ứng. Điểm $t = 4$ là mức đỉnh tại cuối warmup.

Bài 4. Cosine sau warmup

Cho $W = 2$, $T = 10$, $\eta_{\max} = 0{,}08$, $\eta_{\min} = 0$. Tính $\eta(t)$ tại $t = 2, 6, 10$ theo công thức cosine ở mục 7.

Đáp án (tự tính): Ba mốc lần lượt là đầu, giữa và cuối đoạn decay; các giá trị cos tương ứng là $1, 0, -1$. Kết quả là $0{,}08$, $0{,}04$, 0.

Bài 5. WSD và số cập nhật

Có 240 mẫu, batch 20 mẫu, huấn luyện 3 epoch, không tích luỹ gradient và dữ liệu chia hết. Lập lịch WSD tuyến tính theo optimizer step với $W = 3$, $S = 27$, T bằng tổng số cập nhật, $\eta_{\max} = 0{,}12$ và mức cuối bằng 0. Tính T, $\eta(1{,}5)$ và $\eta(31{,}5)$.

Đáp án (tự tính): Mỗi epoch có $240/20 = 12$ cập nhật, nên $T = 36$. Mốc $1{,}5$ ở giữa warmup cho $0{,}12 \cdot 1{,}5/3 = 0{,}06$. Decay kéo dài $36 - 27 = 9$ bước; $31{,}5$ là chính giữa, nên $\eta(31{,}5) = 0{,}06$. Hai mốc nửa bước dùng để tính giá trị hàm lịch, không nhất thiết được gọi bởi chương trình chỉ cập nhật tại bước nguyên.

Tài liệu tham khảo nền: Bottou, Curtis và Nocedal, Optimization Methods for Large-Scale Machine Learning; Loshchilov và Hutter, SGDR; Goyal và cộng sự, Accurate, Large Minibatch SGD; Hu và cộng sự, MiniCPM; các ghi chú optimization 17, 18, 20, 21 và 23 trong sách.

Nền tảng 07

Batch size, critical batch và quy tắc scale learning rate: chọn lượng dữ liệu cho mỗi bước và điều chỉnh nhịp cập nhật

Loại: chương nền tảng, không phải ghi chú paper

Đọc trước: gradient descent, SGD và mini-batch, learning rate ở các chương nền opt 02, 03, 06; có thể xem lại đạo hàm và gradient trong math nen06.

Dùng ở: optimization 10, 21, 22, 23, 30, 47; các bài về optimizer phân tán và huấn luyện mô hình lớn.

> Chương này giải thích khái niệm từ đầu, không phải bản dịch paper. Các phép tính tự làm được ghi "tự tính" và đã kiểm bằng NumPy.

1. Vì sao cần quan tâm đến batch size

Batch size là số mẫu mà ta dùng để tính một gradient trước khi cập nhật tham số. Một mẫu có thể là một ảnh, một câu, một đoạn văn bản hoặc một chuỗi token, tùy cách bài toán định nghĩa loss. Batch size tác động cùng lúc đến độ nhiễu của gradient, số lần cập nhật, mức dùng bộ nhớ và khả năng tận dụng phần cứng. Vì vậy, chọn batch size không chỉ là chọn "bao nhiêu mẫu vừa GPU".

Trong học máy, loss thường là trung bình của loss trên nhiều mẫu. Gradient của cả tập dữ liệu chỉ hướng tăng nhanh nhất cục bộ; hướng ngược gradient là hướng giảm loss cục bộ khi gradient khác 0. Tính gradient toàn tập có thể quá tốn kém. SGD thay gradient toàn tập bằng gradient của một mẫu hoặc một nhóm nhỏ mẫu. Nhóm đó gọi là mini-batch. Batch nhỏ rẻ hơn cho mỗi bước nhưng gradient dao động mạnh hơn. Batch lớn tốn nhiều phép tính và bộ nhớ cho mỗi bước, đổi lại phép ước lượng thường ổn định hơn.

Ta cần ba câu trả lời thực dụng. Batch lớn thêm có còn làm tiến độ nhanh lên không? Khi tăng batch, có nên tăng learning rate để bù cho số bước cập nhật ít đi không? Nếu bộ nhớ không đủ cho batch mục tiêu, chia thành nhiều micro-batch rồi cộng gradient có cho kết quả tương tự không? Chương này đi lần lượt từ một ví dụ số nhỏ đến các quy tắc và giới hạn.

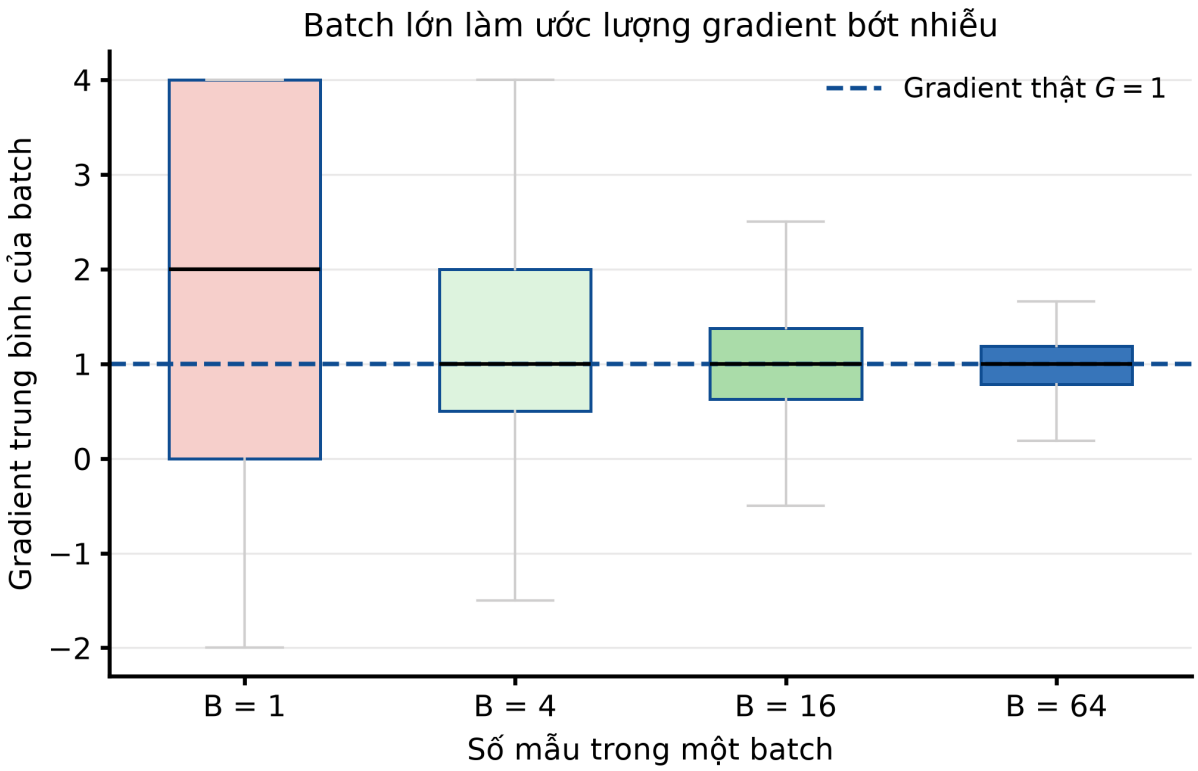
2. Nhắc lại: gradient của batch là một phép trung bình

Trước công thức, hãy nhìn phân bố các gradient trung bình trong hình. Mỗi lần ta chọn batch khác nhau thì mũi tên gradient có thể khác nhau, dù tất cả đều được lấy từ cùng dữ liệu. Mục tiêu của phép trung bình là làm dao động ngẫu nhiên bớt ảnh hưởng lên hướng cập nhật.

Cách đọc hình 1: đường gạch ngang xanh là gradient thật của ví dụ, còn mỗi hộp cho thấy phân bố các gradient trung bình được lấy nhiều lần từ cùng một tập gradient mẫu nhỏ. Các phép lấy mẫu là có hoàn lại từ bốn giá trị minh hoạ, nên batch size 64 có lặp mẫu. Hộp của batch size 1 trải rộng, trong khi batch size 64 tụ gần giá trị thật hơn. Hình này minh hoạ lấy mẫu IID và giảm nhiễu, không nói rằng một batch cụ thể luôn cho gradient tốt hơn.

2.1 Định nghĩa gradient batch

Gọi $\ell_i(\theta)$ là loss của mẫu thứ i tại tham số θ. Gradient của mẫu đó là $g_i = \nabla_\theta \ell_i(\theta)$. Với tập chỉ số mẫu trong batch $\mathcal{B}$ gồm b mẫu, gradient mini-batch là trung bình:



Hình 21: Các gradient trung bình tập trung gần gradient thật khi batch size tăng (minh hoạ, tự tính)

$$\widehat{g}_{\mathcal{B}} = \frac{1}{b} \sum_{i \in \mathcal{B}} g_i.$$

Ta dùng ký hiệu b cho batch size và $\mathcal{B}$ cho tập mẫu trong batch. Khi lấy mẫu ngẫu nhiên, $\widehat{g}_{\mathcal{B}}$ là một ước lượng của gradient thật $G = \mathbb{E}[g_i]$. Với cách lấy mẫu độc lập, kỳ vọng của gradient mini-batch vẫn là gradient thật, còn độ phân tán của nó giảm khi tăng số mẫu:

$$\mathbb{E}[\widehat{g}_{\mathcal{B}}] = G, \qquad \mathrm{Cov}(\widehat{g}_{\mathcal{B}}) = \frac{1}{b}\Sigma,$$

trong đó $\Sigma = \mathrm{Cov}(g_i)$ là ma trận hiệp phương sai của gradient từng mẫu. Phương sai giảm theo $1/b$, còn độ lệch chuẩn giảm theo $1/\sqrt{b}$. Tăng batch gấp bốn lần vì vậy làm độ lệch chuẩn giảm còn một nửa, chứ không giảm còn một phần tư.

Nếu lấy mẫu không hoàn lại từ một tập dữ liệu hữu hạn thì có thêm hệ số hiệu chỉnh hữu hạn. Khi batch gần bằng toàn bộ tập, trung bình gần như không còn nhiễu do chọn mẫu. Công thức Σ/b thường là xấp xỉ phù hợp khi tập dữ liệu lớn hoặc khi ta lấy mẫu có hoàn lại.

3. Ví dụ tay: trung bình giảm nhiễu ra sao

Xét một bài toán một chiều có bốn gradient mẫu là -2, 0, 2 và 4. Đây là một bộ số giả định để tính tay. Gradient trung bình của bốn mẫu là

$$G = \frac{-2 + 0 + 2 + 4}{4} = 1 \quad \text{(tự tính)}.$$

Độ lệch của từng gradient so với trung bình là -3, -1, 1 và 3. Bình phương các độ lệch lần lượt là 9, 1, 1 và 9, nên phương sai theo phân phối đều trên bốn mẫu bằng $20/4 = 5$ (tự tính). Nếu lấy một mẫu ngẫu nhiên, gradient chỉ có thể là một trong bốn số trên. Một mẫu có thể chỉ đúng hướng, nhưng độ lớn có thể khác xa $G = 1$.

Nếu chọn hai mẫu khác nhau mà không hoàn lại, có sáu cặp. Gradient trung bình của chúng lần lượt là -1, 0, 1, 1, 2 và 3 (tự tính). Trung bình của sáu kết quả vẫn là 1. Phương sai của sáu trung bình quanh 1 là

$$\frac{(-2)^2 + (-1)^2 + 0^2 + 0^2 + 1^2 + 2^2}{6} = \frac{5}{3} \quad \text{(tự tính)}.$$

Không phải cặp nào cũng gần 1, nhưng kết quả đã bớt phân tán so với lấy riêng một mẫu. Với lấy mẫu có hoàn lại, công thức xấp xỉ cho phương sai của trung bình batch là $5/b$. Khi $b = 4$, phương sai ước lượng là $1{,}25$ và độ lệch chuẩn là $\sqrt{1{,}25} \approx 1{,}118$ (tự tính). Con số này không bằng 0 vì trong phép lấy mẫu có hoàn lại ta có thể lấy lặp một mẫu. Nếu dùng cả bốn mẫu đúng một lần, gradient trung bình bằng chính xác 1.

Điểm quan trọng không phải batch nào cũng "đúng" hơn theo từng lần chạy. Điều ta có thể khẳng định là phương sai của ước lượng giảm trung bình khi tăng batch, dưới giả định lấy mẫu thích hợp.

4. Nhiễu, tối ưu và cách đọc batch

Hãy tưởng tượng gradient của mỗi mẫu là một mũi tên. Phần cùng hướng với nhiều mẫu tạo thành tín hiệu có ích, còn các thành phần trái chiều tạo thành nhiễu. Khi lấy trung bình một vài mũi tên, nhiễu ngẫu nhiên có thể chưa triệt tiêu. Khi lấy trung bình nhiều mũi tên độc lập hơn, phần tín hiệu thường còn lại, trong khi các dao động có dấu khác nhau triệt tiêu lẫn nhau.

Nếu gradient thật lớn so với nhiễu, batch nhỏ đã đủ để thấy hướng chung. Nếu gradient thật rất nhỏ nhưng từng mẫu tạo gradient lớn và khác hướng, cần batch lớn hơn mới phân biệt được hướng giảm loss với dao động do lấy mẫu. Trường hợp sau thường gặp gần vùng phẳng hoặc gần điểm dừng, nhưng không có một kích thước batch duy nhất phù hợp cho mọi bước huấn luyện.

Độ nhiễu không phải luôn là điều xấu. Với batch hữu hạn, SGD đi theo một quỹ đạo có dao động. Dao động đó có thể giúp tránh việc phụ thuộc quá sớm vào một hướng gradient ước lượng thiếu đại diện. Tuy nhiên, không nên kết luận rằng batch nhỏ luôn tổng quát hóa tốt hơn batch lớn. Kết quả còn phụ thuộc learning rate, lịch huấn luyện, kiến trúc, dữ liệu, optimizer và cách đánh giá. Các bài optimization 21-24 xem xét riêng các câu hỏi về batch lớn, độ nhiễu và tổng quát hóa.

5. Định nghĩa chính thức về batch và critical batch

Một tập dữ liệu có N mẫu. Nếu chia dữ liệu thành các nhóm cỡ b và cập nhật sau mỗi nhóm, số bước trong một epoch xấp xỉ N/b. Trong huấn luyện đồng bộ trên K worker, nếu mỗi worker xử lý b_{local} mẫu rồi gộp gradient, batch toàn cục là $b_{global} = Kb_{local}$. Đây là batch tạo ra một lần cập nhật tham số. Batch cục bộ và batch toàn cục có thể khác nhau đáng kể.

Để định lượng tương quan giữa tín hiệu gradient và nhiễu, một mô hình đơn giản dùng

$$B_{simple} = \frac{\operatorname{tr}(\Sigma)}{\|G\|_2^2}.$$

Tử số là tổng phương sai trên các tọa độ gradient, còn mẫu số là bình phương độ lớn của gradient trung bình. Công thức chỉ xác định khi $\|G\|_2 > 0$; tại điểm mà gradient thật bằng 0, tỷ số không được xác định. Nếu batch size b gần B_{simple}, tổng nhiễu của gradient batch ở cùng cỡ với bình phương tín hiệu gradient. Nếu b

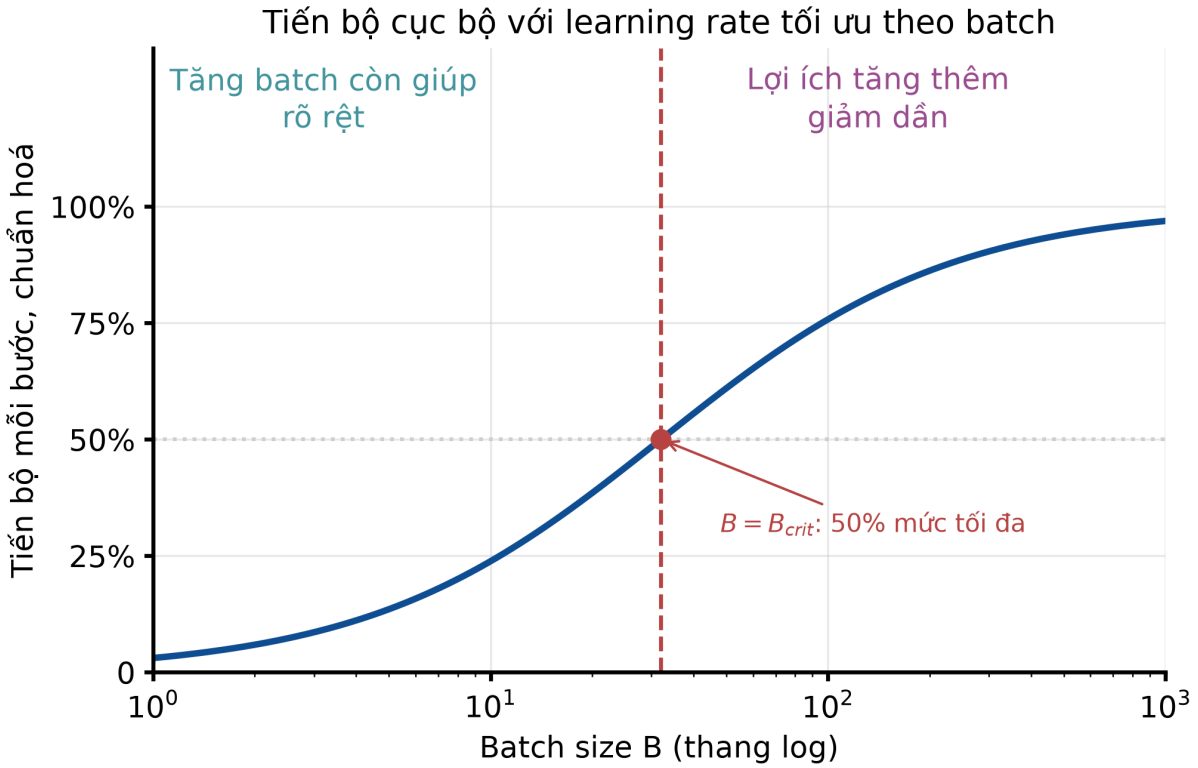
nhỏ hơn nhiều, gradient ước lượng có thể bị nhiễu chi phối. Nếu b lớn hơn nhiều, gradient ước lượng đã gần gradient thật và thêm mẫu có lợi ích giảm dần.

Một định nghĩa có xét độ cong cục bộ của loss là gradient noise scale:

$$B_{noise} = \frac{\mathrm{tr}(H\Sigma)}{G^\top H G},$$

trong đó H là Hessian loss tại vùng đang xét. Công thức có ý nghĩa trong mô hình cục bộ khi $G^\top H G > 0$ và $\mathrm{tr}(H\Sigma) > 0$, thường giả định vùng có độ cong dương theo hướng gradient. Nếu Hessian không dương bán xác định, tử số có thể âm và noise scale không còn diễn giải là tỷ lệ dương. Công thức này xuất phát từ xấp xỉ bậc hai cục bộ và không phải phép đo chính xác phổ quát cho mọi mạng. Trong một số mô hình đơn giản, critical batch size B_{crit} nằm cùng cỡ với noise scale. Có thể hiểu B_{crit} là mốc mà tăng batch thêm bắt đầu đổi hiệu quả lấy mẫu lấy được trên mỗi phép tính, chứ không phải ranh giới cứng mà ngay sau đó batch lớn "sai".

Ví dụ minh hoạ: giả sử ở một điểm huấn luyện, phép đo cho $\mathrm{tr}(\Sigma) = 80$ và $\|G\|_2^2 = 4$. Khi đó $B_{simple} = 80/4 = 20$ (tự tính). Đây chỉ là phép tính giả định để giải thích tỷ số. Nó không cho biết batch tối ưu chính xác trên một mô hình thực, cũng không thay thế đo số bước và số mẫu cần để đạt cùng mục tiêu huấn luyện.



Hình 22: Mô hình đơn giản cho thấy lợi ích mỗi bước bão hoà quanh critical batch (minh hoạ, tự tính)

Cách đọc hình 2: đường cong mô tả tiến bộ dự kiến trên mỗi bước trong xấp xỉ bậc hai cục bộ, với learning rate tối ưu hoá riêng cho từng batch size và $B_{crit} = 32$ được chọn để vẽ. Tại mốc đó, đường cong đạt một nửa mức tối đa của mô hình; phía bên trái, tăng batch làm đường cong tăng rõ, còn phía bên phải, đường cong dần phẳng. Đây là minh hoạ khái niệm với learning rate được tối ưu theo batch, không phải dự báo cho learning rate cố định hay số đo của một paper.

6. Quy tắc scale learning rate

Khi batch tăng k lần, số bước cập nhật trong một epoch giảm khoảng k lần. Nếu gradient không thay đổi nhiều trong khoảng đó, một bước batch lớn có thể xấp xỉ tổng của k bước batch nhỏ. Từ trực giác này xuất hiện quy tắc scale tuyến tính:

$$b' = kb, \qquad \eta' = k\eta.$$

Trong đó η là learning rate cũ, còn η' là learning rate mới. Quy tắc này chỉ là một phép xấp xỉ. Nó dựa trên ý rằng các gradient ở những bước nhỏ liên tiếp gần nhau. Khi tham số thay đổi nhanh, batch tăng quá mạnh, hoặc optimizer có trạng thái như momentum, xấp xỉ có thể hỏng. Tăng learning rate lớn ngay từ đầu cũng có thể tạo bước nhảy quá mạnh. Vì vậy các thí nghiệm scale batch lớn thường kết hợp warmup, tức tăng learning rate dần từ mức an toàn lên mức mục tiêu. Chương nền learning rate giải thích lịch warmup kỹ hơn; optimization 21 là ví dụ cụ thể trong tài liệu paper.

Một quy tắc khác là scale theo căn bậc hai:

$$\eta' = \eta\sqrt{k}.$$

Động cơ ở đây khác với quy tắc tuyến tính. Nếu nhiễu của gradient trung bình giảm theo $1/\sqrt{b}$, thì scale bước theo $\sqrt{b}$ có thể giữ gần cùng cỡ dao động ngẫu nhiên của cập nhật trong một số giả định. Quy tắc này không nhằm giữ cùng tổng độ dịch chuyển mỗi epoch. Hai quy tắc vì thế không thể thay thế nhau một cách máy móc. Chọn quy tắc nào còn tùy mục tiêu, optimizer, mức nhiễu, batch gốc và chất lượng tinh chỉnh.

Ví dụ (tự tính): một tập có 10.240 mẫu. Batch 64 tạo 160 bước trong một epoch, còn batch 256 tạo 40 bước (tự tính). Batch sau lớn gấp bốn lần, nên quy tắc tuyến tính đề xuất nhân learning rate với bốn. Nếu learning rate cũ là $0{,}01$, mức mới theo quy tắc là $0{,}04$ (tự tính). Trong mô hình lý tưởng gradient gần như không đổi, tổng hệ số cập nhật của hai lịch bằng nhau: $160 \times 0{,}01 = 40 \times 0{,}04 = 1{,}6$ (tự tính). Mạng thực không có gradient cố định như vậy, nên đây là lý do để thử nghiệm chứ không phải bảo đảm hai đường huấn luyện giống hệt nhau.

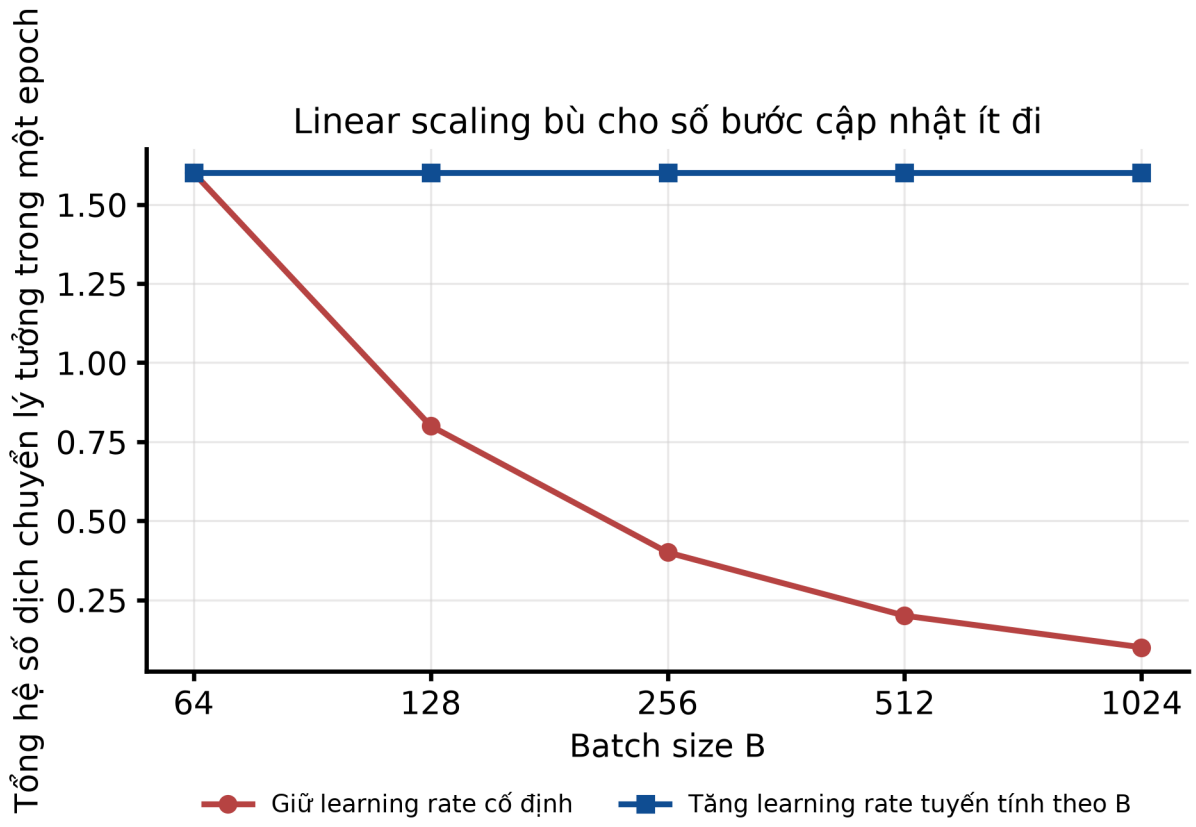
Cách đọc hình 3: đường đỏ giữ learning rate và giảm tổng độ dịch chuyển lý tưởng khi batch lớn lên vì số bước ít đi. Đường xanh tăng learning rate tỷ lệ với batch nên tổng hệ số dịch chuyển mỗi epoch không đổi trong giả định gradient đứng yên. Đường xanh không dự báo loss hay accuracy; thực tế còn cần kiểm tra độ ổn định và lịch learning rate.

7. Batch size và phần cứng

Batch tăng có thể giúp phần cứng chạy hiệu quả hơn nếu batch nhỏ chưa đủ việc để lấp đầy GPU. Trong huấn luyện phân tán, nhiều worker tính gradient song song rồi đồng bộ. Batch toàn cục lớn hơn cho phép mỗi worker làm nhiều phép toán trước mỗi lần đồng bộ, nhưng số byte truyền, thời gian all-reduce và độ mất cân bằng giữa worker cũng quan trọng. Tăng batch không đồng nghĩa với tăng tốc theo đúng tỷ lệ.

Batch cũng tiêu thụ bộ nhớ cho activation. Với một mô hình lớn, tăng batch có thể làm hết bộ nhớ trước khi đạt vùng hiệu suất tốt. Nếu mẫu có độ dài khác nhau, “batch 32” có thể xử lý số token rất khác nhau giữa hai batch. Nhiều hệ thống vì vậy theo dõi token trong batch thay vì chỉ theo số chuỗi. Một batch có cùng số mẫu chưa chắc có cùng công việc hay bộ nhớ.

Khi tăng số worker, phải phân biệt batch toàn cục với batch cục bộ dùng để tính thống kê. BatchNorm dựa vào thống kê của một tập activation cụ thể. Nếu mỗi worker dùng batch cục bộ nhỏ nhưng ta vô tình đổi cách gộp thống kê, phép toán của mô hình cũng thay đổi. Ghi chú optimization 21 chỉ rõ đây là một chi tiết triển khai cần giữ nhất quán khi scale huấn luyện phân tán.

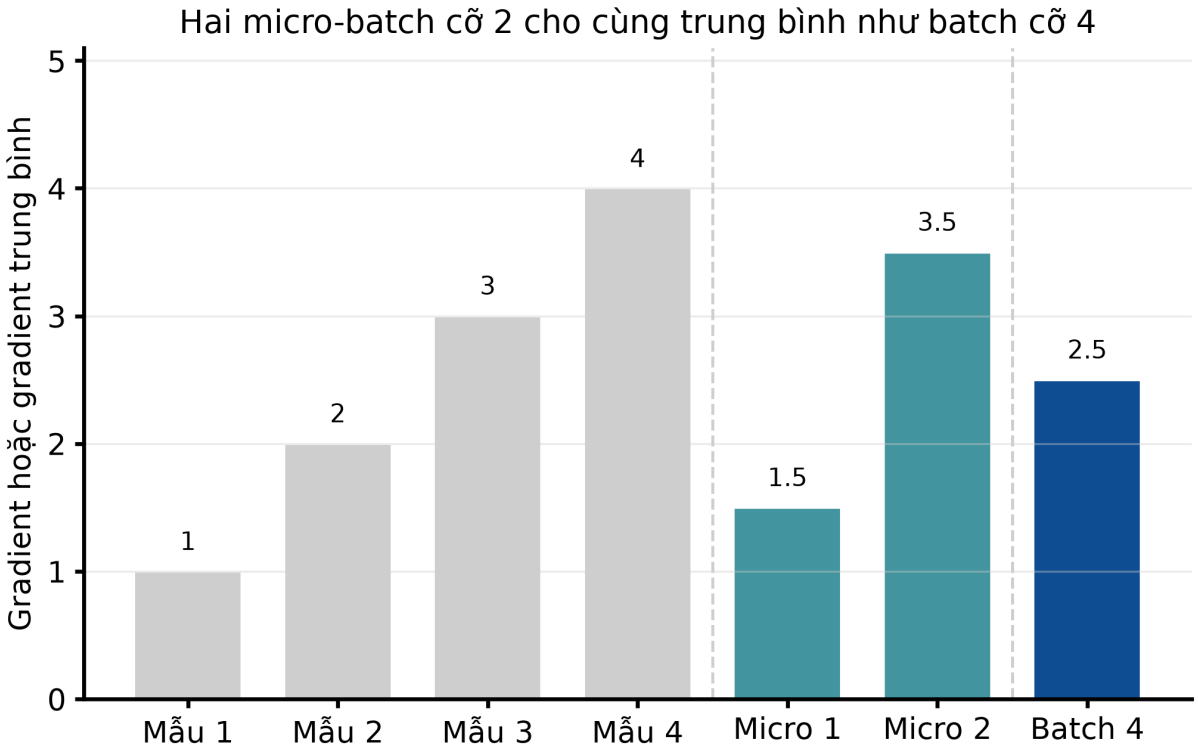


Hình 23: Quy tắc scale tuyến tính bù số bước giảm trong mô hình gradient cố định (minh hoạ, tự tính)

8. Gradient accumulation: batch lớn bằng nhiều micro-batch

Nếu một batch lớn không vừa bộ nhớ, ta có thể xử lý lần lượt nhiều micro-batch, cộng các gradient rồi mới cập nhật tham số. Với hai micro-batch có cùng số mẫu, gradient của batch lớn là trung bình của hai gradient micro-batch. Không cập nhật tham số giữa chừng là điều quan trọng: nếu cập nhật sau từng micro-batch, phép tính đã trở thành nhiều bước SGD nhỏ thay vì một bước với batch lớn.

Ví dụ (tự tính): bốn gradient mẫu là 1, 2, 3, 4. Hai micro-batch lần lượt có trung bình $(1+2)/2 = 1{,}5$ và $(3+4)/2 = 3{,}5$. Trung bình của hai micro-batch bằng $2{,}5$, đúng bằng trung bình của cả bốn mẫu $(1+2+3+4)/4 = 2{,}5$ (tự tính). Hình sau trình bày phép cộng trung bình này.



Hình 24: Hai micro-batch bằng nhau cho gradient trung bình giống một batch lớn (minh hoạ, tự tính)

Cách đọc hình 4: bốn cột xám là gradient từng mẫu, hai cột xanh ngọc là trung bình của từng micro-batch cỡ hai. Cột xanh đậm là trung bình batch cỡ bốn, khớp với trung bình của hai cột xanh ngọc vì micro-batch có cùng số mẫu. Nếu kích thước micro-batch không đều, cần gộp theo số mẫu hoặc số token, không lấy trung bình đơn giản giữa các trung bình.

Tương đương chỉ giữ trong những điều kiện phù hợp: loss cộng được theo mẫu, các micro-batch không làm thay đổi tham số giữa chừng, và các lớp không dùng thống kê phụ thuộc batch theo cách làm thay đổi đầu ra. BatchNorm là ví dụ điển hình khiến micro-batch có thống kê khác batch lớn. Dropout hay augmentation ngẫu nhiên có thể cho một lần chạy khác, dù kỳ vọng có thể tương tự. Cần chia loss đúng cho số mẫu/token tích lũy; cũng cần gọi optimizer và bộ lập lịch learning rate sau lần tích lũy hoàn chỉnh nếu mục tiêu là giữ một lần cập nhật.

9. Batch thay đổi trong suốt quá trình học

Không nhất thiết giữ một batch size cố định từ đầu tới cuối. Một số lịch huấn luyện tăng batch khi đang học, đồng thời giữ hoặc thay đổi learning rate. Lý do là gradient noise scale thay đổi theo vị trí trong quá trình tối ưu: đầu quá trình, bước đi lớn và nhiễu có thể hỗ trợ khám phá; gần điểm dừng, batch lớn hơn có thể ước

lượng gradient ổn định với ít lần cập nhật hơn. Đây là một cách nhìn để đặt giả thuyết, không phải quy tắc rằng "luôn tăng batch ở cuối".

Optimization 23 phân tích một quan hệ gần đúng giữa tăng batch và giảm learning rate trong mô hình nhiễu được paper xét. Không nên đồng nhất noise scale vô hướng trong bài đó với gradient noise scale B_{noise} ở optimization 22. Cùng dùng từ "noise scale" không có nghĩa là hai đại lượng cùng định nghĩa. Khi đọc paper, cần xem công thức cụ thể trước khi chuyển một quy tắc sang bài toán khác.

10. Cách chọn batch trong thực hành

Một quy trình thận trọng là đo trước khi scale xa. Bắt đầu từ batch vừa bộ nhớ, tăng theo các mức có thể so sánh, và ghi lại số bước cùng số mẫu/token cần để đạt cùng mục tiêu loss hoặc chất lượng. Đây là cách ước lượng giới hạn thống kê của batch: nó hỏi lượng dữ liệu và cập nhật nào cần thiết cho cùng một tiến độ. Riêng throughput phần cứng có đường bão hoà khác, phụ thuộc mức dùng GPU, dữ liệu vào, đồng bộ worker và bộ nhớ. Đo mẫu hoặc token mỗi giây để biết phần cứng còn được lợi khi tăng batch; sau đó kết hợp throughput với số mẫu tới mục tiêu để chọn thời gian hoàn tất thấp.

Khi tăng batch, thử learning rate cùng một lịch warmup hợp lý. Kiểm tra vài bước đầu vì mất ổn định thường lộ ra sớm qua loss tăng đột ngột, gradient không hữu hạn hoặc tham số đổi quá mạnh. Nếu batch lớn không cải thiện thời gian tới chất lượng mục tiêu, hãy quay lại batch nhỏ hơn hoặc điều chỉnh cách phân phối công việc. Không nên so hai cấu hình bằng cùng số bước nếu số mẫu đã xử lý khác nhau; cũng không nên chỉ so cùng số epoch nếu cách lấy mẫu hoặc dữ liệu bị bỏ lại khác nhau.

Critical batch size là khái niệm thống kê hữu ích để suy nghĩ về vùng lợi ích giảm dần, nhưng ước lượng nó trong mô hình thật khó. B_{simple} cần các gradient mẫu hoặc một ước lượng hiệp phương sai, vốn có thể tốn bộ nhớ và phép tính. Có thể ước lượng thực dụng bằng cách đo số bước và số mẫu cần tới một mục tiêu cố định, rồi tách riêng ảnh hưởng throughput của phần cứng. Batch phù hợp cuối cùng phụ thuộc thời gian tới mục tiêu, không chỉ một trong hai đại lượng. Cả noise scale và throughput đều có thể đổi trong quá trình học.

11. Liên hệ với AI

Trong huấn luyện phân loại, batch size quyết định bao nhiêu ảnh đóng góp vào mỗi lần cập nhật. Batch lớn có thể giúp GPU xử lý ma trận hiệu quả, nhưng cần learning rate và warmup phù hợp để tránh những bước đầu quá mạnh. Optimization 21 thảo luận chiến lược này trong huấn luyện minibatch lớn.

Trong huấn luyện ngôn ngữ, batch có thể được đếm bằng chuỗi, token hoặc tổng token không padding. Đếm theo token thường phản ánh compute sát hơn khi độ dài câu biến thiên. Gradient accumulation cho phép chia một batch token mục tiêu thành nhiều lần forward/backward nhỏ khi bộ nhớ hạn chế, nhưng vẫn cần tính đúng trung bình loss theo token.

Khi fine-tune, batch nhỏ có thể khiến gradient dao động nhiều vì mỗi bước nhìn thấy ít ví dụ hơn. Gradient clipping, lịch learning rate và chuẩn hoá có thể giúp một số run ổn định hơn, nhưng không thay đổi nghĩa của batch size. Các chương nền optimization 08 và ghi chú về clipping giải thích riêng những cơ chế đó.

12. Sai lầm hay gặp

- Nghĩ batch lớn luôn làm huấn luyện tổng thể nhanh hơn. Nó có thể giảm số bước nhưng tăng thời gian bước, bộ nhớ và chi phí đồng bộ.
- Cho rằng phương sai giảm theo $1/b$ thì sai số tuyệt đối cũng giảm đúng tỷ lệ đó. Độ lệch chuẩn giảm theo $1/\sqrt{b}$.
- Áp dụng learning rate tuyến tính vô hạn theo batch. Quy tắc dựa trên xấp xỉ gradient ổn định và có thể hỏng khi batch quá lớn hoặc learning rate quá cao.

- So sánh cấu hình bằng cùng số update mà bỏ qua số mẫu đã thấy. Hai cấu hình có thể đã dùng lượng dữ liệu rất khác nhau.
- Gọi batch toàn cục là batch cục bộ. Trong huấn luyện nhiều worker, hai số này khác nhau; BatchNorm có thể còn phụ thuộc batch cục bộ.
- Cho rằng gradient accumulation luôn giống batch lớn. Thống kê BatchNorm, loss chưa chuẩn hoá theo số mẫu, cập nhật optimizer giữa micro-batch và dropout có thể làm hai phép chạy khác nhau.

13. Tóm tắt công thức

Khái niệm	Công thức hoặc cách đọc
Gradient batch	$\widehat{g}_B = \frac{1}{b}\sum_{i\in B} g_i$
Kỳ vọng gradient	$\mathbb{E}[\widehat{g}_B] = G$ với lấy mẫu không thiên lệch
Hiệp phương sai batch IID	$\mathrm{Cov}(\widehat{g}_B) = \Sigma/b$
Độ lệch chuẩn theo batch	Tỷ lệ với $1/\sqrt{b}$
Noise scale đơn giản	$B_{simple} = \mathrm{tr}(\Sigma)/\|G\|_2^2$
Noise scale có độ cong	$B_{noise} = \mathrm{tr}(H\Sigma)/(G^\top HG)$ trong mô hình cục bộ
Batch toàn cục	$b_{global} = Kb_{local}$ khi K worker đồng bộ, batch cục bộ bằng nhau
Linear scaling	$\eta' = k\eta$ khi $b' = kb$, dùng như xấp xỉ và kiểm tra ổn định
Square-root scaling	$\eta' = \sqrt{k}\eta$ trong một số mục tiêu giữ cỡ nhiễu cập nhật
Gradient accumulation	Tích luỹ gradient các micro-batch, chỉ cập nhật tham số sau cùng

14. Bài tập tự tính

Bài 1

Gradient từng mẫu có phương sai 4. Theo giả định lấy mẫu độc lập, hãy tính phương sai và độ lệch chuẩn của gradient trung bình ở batch size 25 và 100.

Đáp án: phương sai lần lượt là $4/25 = 0{,}16$ và $4/100 = 0{,}04$ (tự tính). Độ lệch chuẩn là $0{,}4$ và $0{,}2$ (tự tính), giảm một nửa khi batch tăng bốn lần.

Bài 2

Tập dữ liệu có 8.192 mẫu. Tính số bước trong một epoch nếu batch size là 32 hoặc 128. Nếu learning rate ban đầu là $0{,}02$, linear scaling đề xuất learning rate nào cho batch lớn?

Đáp án: số bước là $8.192/32 = 256$ và $8.192/128 = 64$ (tự tính). Batch tăng bốn lần nên learning rate gợi ý là $4 \times 0{,}02 = 0{,}08$ (tự tính). Đây là điểm bắt đầu để kiểm tra, không phải cam kết hai cấu hình tương đương.

Bài 3

Ở một điểm huấn luyện, giả sử $\mathrm{tr}(\Sigma) = 36$ và $\|G\|_2^2 = 3$. Tính B_{simple}.

Đáp án: $B_{simple} = 36/3 = 12$ (tự tính). Nó gợi ý cỡ batch mà tổng phương sai và bình phương tín hiệu cùng cỡ trong mô hình đơn giản.

Bài 4

Một phân phối gradient có bốn giá trị đều xác suất: 0, 2, 4, 6. Tính trung bình, phương sai tổng thể, rồi ước lượng phương sai của trung bình batch cỡ 4 nếu lấy mẫu có hoàn lại.

Đáp án: trung bình là 3. Các độ lệch là -3, -1, 1, 3, nên phương sai là $(9+1+1+9)/4 = 5$ (tự tính). Với batch cỡ 4 lấy có hoàn lại, phương sai của trung bình là $5/4 = 1{,}25$ (tự tính).

Bài 5

Bốn gradient là 1, 2, 3, 4. Chia thành hai micro-batch bằng nhau, tính gradient trung bình sau khi tích luỹ và đối chiếu với gradient batch lớn.

Đáp án: hai trung bình micro-batch là $1{,}5$ và $3{,}5$. Trung bình của chúng là $2{,}5$, bằng trung bình $(1+2+3+4)/4 = 2{,}5$ của batch lớn (tự tính). Vì hai micro-batch có kích thước bằng nhau nên có thể lấy trung bình hai gradient micro-batch.

Tài liệu tham khảo nền: Bubeck, Convex Optimization: Algorithms and Complexity; McCandlish et al., An Empirical Model of Large-Batch Training; Goyal et al., Accurate, Large Minibatch SGD; Smith et al., Don’t Decay the Learning Rate, Increase the Batch Size; các ghi chú optimization 03, 21, 22, 23.

Nền tảng 08

Khởi tạo, normalization, gradient clipping và mixed precision: giữ tín hiệu học trong phạm vi ổn định

Loại: chương nền tảng, không phải ghi chú paper

Đọc trước: gradient descent và SGD ở optimization nen02-03; đạo hàm, chain rule và backprop ở math nen06-07; có thể xem lại hàm kích hoạt trong ml basics nen05.

Dùng ở: optimization 10, 21, 35, 37-42, 44, 46; các bài về mạng sâu, RNN, Transformer và huấn luyện độ chính xác thấp.

> Chương này giải thích các kỹ thuật từ đầu, không phải bản dịch paper. Các phép tính tự làm được ghi "tự tính" và đã kiểm bằng NumPy.

1. Vì sao huấn luyện cần nhiều lớp bảo vệ

Một mạng sâu biến đổi tín hiệu qua nhiều lớp. Lúc bắt đầu học, các trọng số được khởi tạo bằng một quy tắc nào đó. Trong lượt truyền xuôi, tín hiệu đi qua các phép nhân ma trận và hàm kích hoạt. Trong lượt truyền ngược, gradient đi ngược qua cùng chuỗi phép biến đổi. Nếu mỗi lớp làm tín hiệu lớn lên hoặc nhỏ đi quá nhiều, sau nhiều lớp activation hoặc gradient có thể bùng nổ hay biến mất.

Ngay cả khi độ lớn hợp lý, các feature vẫn có thể có thang đo rất khác nhau. Một lớp normalization có thể chuẩn hoá activation để tầng sau nhận đầu vào dễ xử lý hơn. Trong một bước huấn luyện, gradient đôi khi có độ lớn bất thường, chẳng hạn một batch chứa mẫu khó hoặc một chuỗi dài. Gradient clipping giới hạn độ lớn của bước cập nhật. Cuối cùng, tính toán toàn bộ bằng FP32 có thể tốn bộ nhớ và thời gian. Mixed precision dùng định dạng thấp bit cho phần lớn phép tính, đồng thời giữ một số phép tích luỹ và cập nhật ở độ chính xác cao.

Các kỹ thuật này giải quyết những vấn đề khác nhau. Khởi tạo chọn điểm bắt đầu và thang tín hiệu. Normalization thay đổi cách biểu diễn activation trong mạng. Clipping giới hạn một bước gradient quá lớn. Mixed precision giảm chi phí số học nhưng phải quản lý độ chính xác. Chúng bổ trợ nhau, không phải một nút "ổn định" duy nhất.

2. Nhắc nhanh nền tảng: tín hiệu đi qua nhiều tầng

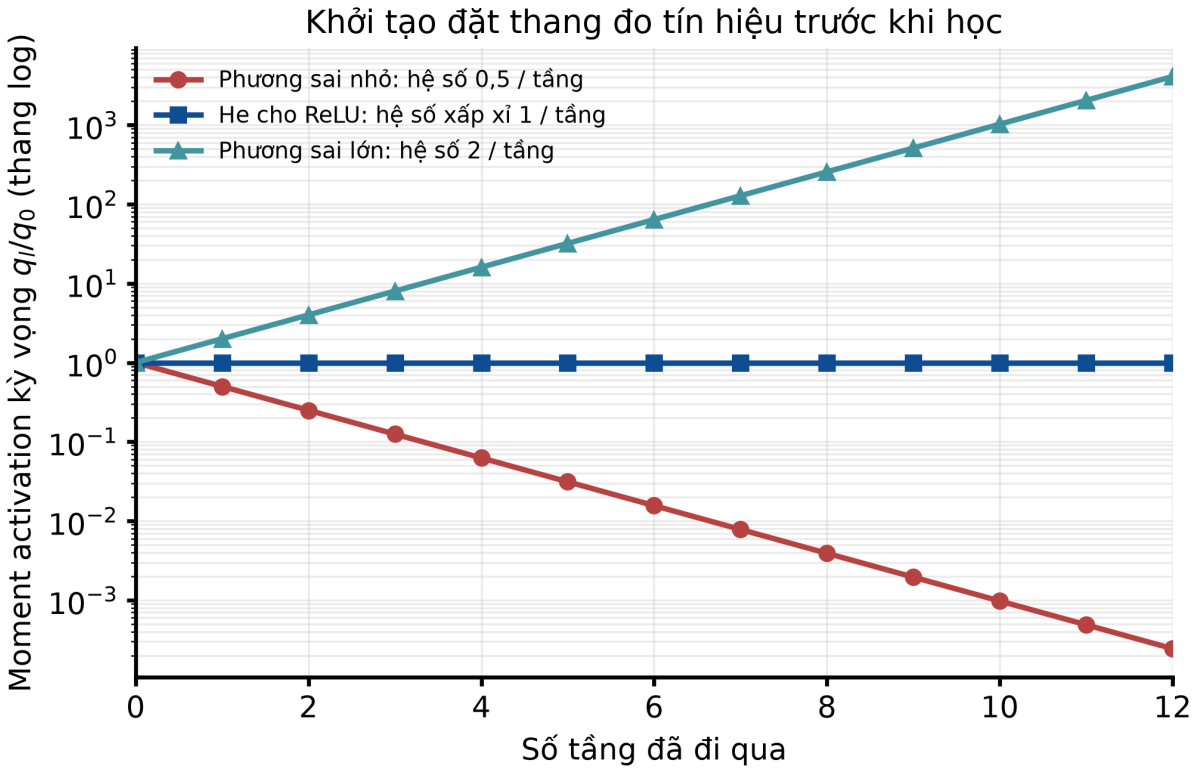
Một tầng tuyến tính nhận vector activation x và tính $z = Wx + b$, rồi hàm kích hoạt biến z thành activation tầng tiếp theo $a = \phi(z)$. Các tầng xếp liên tiếp tạo thành mạng sâu. Mỗi phép nhân ma trận cộng dồn nhiều đóng góp, nên độ lớn của activation phụ thuộc vào số đầu vào, thang đo trọng số và hàm kích hoạt.

Backprop tính gradient bằng chain rule. Nếu gradient qua một tầng được nhân với đạo hàm có độ lớn chủ yếu nhỏ hơn một, tích qua nhiều tầng có thể gần 0. Nếu các thừa số thường lớn hơn một, tích có thể tăng rất nhanh. Đây là trực giác của vanishing gradient và exploding gradient. Một mạng thực có ma trận và hướng khác nhau, nên không phải mọi gradient đều co hay nở giống nhau, nhưng việc kiểm soát thang đo lúc khởi tạo giúp tránh một số tình huống xấu ngay từ đầu.

Khởi tạo không phải là học trước. Nó là cách chọn các trọng số ban đầu để nhiều neuron không bắt đầu cùng một giá trị và tín hiệu có cơ hội truyền qua mạng. Các trọng số thường được lấy ngẫu nhiên quanh 0; độ phân tán của chúng cần phù hợp với số đầu vào của mỗi neuron và loại hàm kích hoạt.

3. Trực giác bằng hình: độ lớn activation qua độ sâu

Hình sau dùng một mô hình kỳ vọng rất đơn giản cho ReLU. Nếu moment activation cứ giảm một nửa ở mỗi tầng thì thông tin yếu dần; nếu tăng gấp đôi thì giá trị có thể lớn lên nhanh. Một phép khởi tạo phù hợp tìm cách giữ moment quanh cùng thang đo, dưới một số giả định về trọng số và activation.



Hình 25: Moment activation kỳ vọng qua các tầng ReLU theo ba mức phương sai khởi tạo (minh hoạ, tự tính)

Cách đọc hình 1: trục dọc dùng thang log để hiện cùng lúc quá trình suy giảm và tăng trưởng. Đường đỏ giảm theo hệ số một nửa mỗi tầng, đường xanh đậm giữ nguyên mức ban đầu, đường xanh ngọc tăng theo hệ số hai. Đây là mô hình lý tưởng để thấy tác động tích luỹ của một sai lệch nhỏ, không phải số đo activation của một kiến trúc cụ thể.

4. Khởi tạo: chọn đúng thang đo

Giả sử các đầu vào của neuron có trung bình gần 0, phương sai q, còn trọng số độc lập, trung bình 0 và phương sai s^2. Nếu neuron có n_{in} đầu vào thì phương sai của tổng tuyến tính xấp xỉ $n_{in}s^2q$. Đây là lý do độ phân tán trọng số không thể chọn độc lập với độ rộng tầng.

Với ReLU, $\phi(z) = \max(0, z)$. Nếu z đối xứng quanh 0, xấp xỉ một nửa tín hiệu bị đặt thành 0. Moment bậc hai sau ReLU gần bằng một nửa moment trước ReLU. Để bù phần này, khởi tạo He, còn gọi là Kaiming initialization, chọn

$$\mathrm{Var}(W_{ij}) = \frac{2}{n_{in}}, \qquad W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right).$$

Khi các giả định gần đúng đúng, **moment bậc hai** của tín hiệu sau tuyến tính rồi ReLU xấp xỉ được giữ ổn định. Cần phân biệt moment bậc hai với phương sai: ReLU tạo đầu ra có trung bình dương, nên phương sai đầu ra không bằng moment bậc hai. Có thể dùng phân phối đều với cùng phương sai trọng số. Với hàm kích

hoạt gần tuyến tính quanh vùng hoạt động, cách khởi tạo Xavier/Glorot thường cân bằng ảnh hưởng cả fan-in lẫn fan-out:

$$\mathrm{Var}(W_{ij}) \approx \frac{2}{n_{in} + n_{out}}.$$

`fan-in` là số đầu vào kết nối tới neuron, còn `fan-out` là số đầu ra. Trong lớp tích chập, fan-in tính cả kích thước kernel và số kênh đầu vào. Những quy tắc này là điểm bắt đầu: kiến trúc đặc biệt, residual connection, embedding, gate hoặc activation khác có thể cần cách xử lý riêng.

4.1 Ví dụ tay với ReLU

Xét một neuron nhận 64 đầu vào độc lập, mỗi đầu vào có trung bình 0 và moment bậc hai bằng 1. Theo He, phương sai trọng số là $2/64 = 1/32$ và độ lệch chuẩn là $\sqrt{1/32} \approx 0{,}177$ (tự tính). Trước ReLU, phương sai của tổng tuyến tính xấp xỉ $64 \times (1/32) \times 1 = 2$. Nếu tổng tuyến tính xấp xỉ Gaussian có trung bình 0, moment bậc hai sau ReLU bằng một nửa moment trước ReLU, nên bằng 1 (tự tính). Như vậy moment bậc hai đầu ra gần bằng moment đầu vào theo mô hình kỳ vọng. Phương sai của ReLU(Gaussian) là $(1/2 - 1/(2\pi))$ lần phương sai trước ReLU; với phương sai 2 ở đây, giá trị đó xấp xỉ $0{,}682$ (tự tính), không phải 1.

Nếu dùng phương sai trọng số bằng $1/64$ trong cùng giả định, moment sau ReLU sẽ còn xấp xỉ $0{,}5$ của tầng trước. Lặp lại qua nhiều tầng sẽ làm activation giảm nhanh. Nếu chọn phương sai gấp đôi He, moment sau mỗi tầng lại xấp xỉ gấp đôi. Hình 1 thể hiện đúng hai xu hướng đó qua nhiều tầng.

4.2 Khởi tạo đối xứng và các lựa chọn khác

Nếu mọi trọng số đi vào các neuron trong một tầng đều bằng nhau, tầng đó chưa phân biệt các neuron. Khi các phép tính và gradient phía sau cũng đối xứng với việc hoán đổi các neuron, chúng nhận gradient giống nhau và tiếp tục học giống nhau. Đây là lý do cần phá vỡ đối xứng ở các trọng số có thể hoán đổi, thường bằng khởi tạo ngẫu nhiên. Không thể kết luận chỉ từ việc các trọng số đầu vào bằng 0 rằng mọi neuron luôn có cùng gradient: các trọng số đi ra khác nhau có thể tạo tín hiệu lỗi khác nhau. Bias thường có thể khởi tạo bằng 0 vì bản thân bias chung ban đầu không làm các hàng trọng số trở nên giống nhau.

Khởi tạo trực giao chọn các hướng trọng số gần vuông góc, hữu ích trong một số mạng sâu hoặc mạng hồi quy. Khởi tạo orthogonal không loại bỏ nhu cầu chọn độ lớn phù hợp với activation. Với một số mô hình có residual, người thiết kế còn điều chỉnh nhánh residual hoặc scale ban đầu để tổng của các nhánh không quá lớn. Vì thế, “dùng He” không đảm bảo mọi mạng sẽ ổn định, nhưng là một quy tắc nền có cơ sở cho tầng ReLU thường.

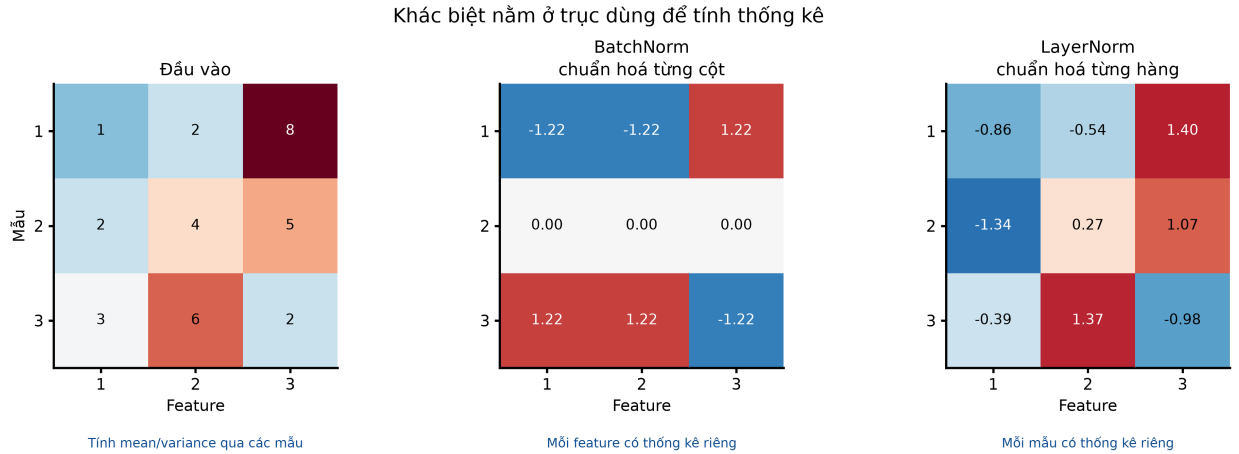
5. Normalization: chuẩn hoá activation theo một trục

Một cách khác để giữ thang đo là chuẩn hoá activation trong lúc chạy. Có nhiều loại normalization, và khác biệt quan trọng là chúng tính mean và variance trên những phần tử nào. Hình sau đặt BatchNorm và LayerNorm cạnh nhau trên cùng một ma trận ba mẫu, ba feature.

Cách đọc hình 2: ma trận bên trái là activation ban đầu, mỗi hàng là một mẫu và mỗi cột là một feature. BatchNorm chuẩn hoá từng cột, nên thống kê của một feature được tính qua các mẫu. LayerNorm chuẩn hoá từng hàng, nên mỗi mẫu có thống kê riêng qua các feature. Hai phép chuẩn hoá vì thế phụ thuộc vào các trục khác nhau.

Sau khi nhìn trục, ta viết phép chuẩn hoá cho một nhóm giá trị $x_1, \dots, x_m$:

$$\mu = \frac{1}{m}\sum_{j=1}^{m} x_j, \qquad v = \frac{1}{m}\sum_{j=1}^{m} (x_j - \mu)^2,$$



Hình 26: BatchNorm chuẩn hoá theo feature qua các mẫu, LayerNorm chuẩn hoá theo feature trong từng mẫu (minh hoạ, tự tính)

$$\widehat{x}_j = \frac{x_j - \mu}{\sqrt{v + \epsilon}}, \qquad y_j = \gamma \widehat{x}_j + \beta.$$

Hằng số nhỏ ϵ giúp mẫu số không bằng 0. Hệ số γ và β được học, cho phép mạng điều chỉnh lại thang đo và độ lệch sau chuẩn hoá. Đặt $\gamma = 1$ và $\beta = 0$ chỉ bỏ phần scale/shift học được; phép chuẩn hoá theo μ và v vẫn diễn ra.

5.1 BatchNorm

Batch Normalization tính mean và variance của từng kênh trên batch huấn luyện. Với ảnh, phép tính thường gộp cả vị trí không gian trong cùng kênh. Trong lúc huấn luyện, mỗi batch dùng thống kê của nó. Khi suy luận, mô hình thường dùng running mean và running variance tích luỹ trong huấn luyện thay vì thống kê của một batch mới. Nhờ vậy, đầu ra tại inference không đổi theo các mẫu khác được gửi cùng lúc.

Ví dụ tay (tự tính): activation của một kênh trong batch là $[1, 2, 3, 4]$. Mean là $2{,}5$, variance chia cho số phần tử là $1{,}25$. Nếu bỏ qua ϵ, các giá trị sau chuẩn hoá xấp xỉ $[-1{,}342, -0{,}447, 0{,}447, 1{,}342]$. Chọn $\gamma = 2$ và $\beta = 1$ cho đầu ra xấp xỉ $[-1{,}683, 0{,}106, 1{,}894, 3{,}683]$ (tự tính). Các số đã được tính lại bằng NumPy.

BatchNorm có thể phụ thuộc mạnh vào cách chia batch. Batch quá nhỏ làm mean và variance nhiễu hơn; batch một ở lớp fully connected có phương sai bằng 0 nếu chỉ còn một giá trị trên kênh. Một số kiến trúc dùng thống kê qua nhiều worker hoặc nhóm “ghost batch” để quản lý tác động này. Hãy phân biệt batch dùng cho gradient với tập activation dùng cho thống kê chuẩn hoá.

5.2 LayerNorm và chuẩn hoá liên quan

Layer Normalization tính mean và variance qua các feature trong một mẫu. Mỗi mẫu được chuẩn hoá riêng, nên kết quả không phụ thuộc batch chứa các mẫu khác. Điều đó thuận tiện cho chuỗi và batch nhỏ. Trong mô hình ngôn ngữ tự hồi quy, LayerNorm thường áp dụng riêng trên vector biểu diễn của từng token.

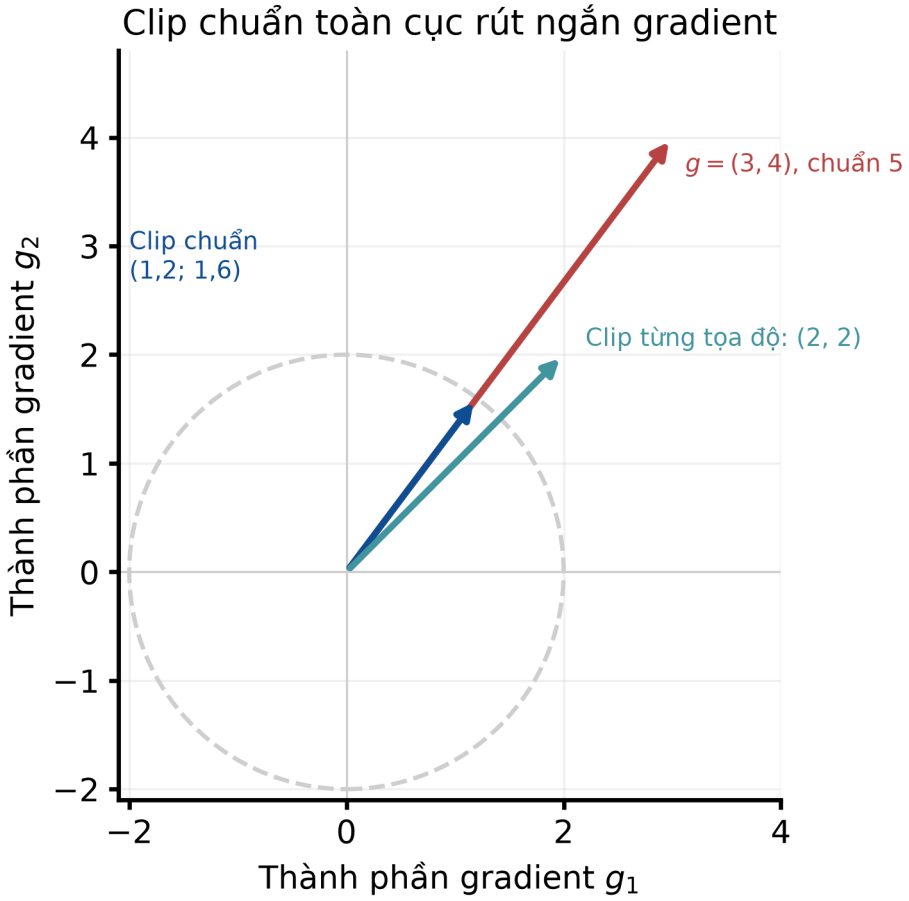
Ví dụ (tự tính): một vector feature là $[2, 4, 6]$. Mean là 4, variance là $8/3$, nên vector chuẩn hoá xấp xỉ $[-1{,}225, 0, 1{,}225]$. Nếu $\gamma = 1{,}5$ và $\beta = -1$, kết quả xấp xỉ $[-2{,}837, -1, 0{,}837]$ (tự tính). RMSNorm là một lựa chọn gần họ này, tính độ lớn RMS mà không trừ mean; nó không hoàn toàn giống LayerNorm.

Normalization giúp kiểm soát activation và có thể làm bài toán tối ưu dễ xử lý hơn, nhưng không làm mọi gradient trở nên tốt hay loại bỏ nhu cầu chọn learning rate. BatchNorm và LayerNorm cũng thay đổi hàm mô

hình theo cách riêng, vì vậy không nên đổi loại chuẩn hoá mà không xem xét cách nó được dùng lúc huấn luyện và suy luận.

6. Gradient clipping: giới hạn một bước quá lớn

Một gradient có độ lớn bất thường có thể tạo bước cập nhật lớn hơn dự định trong SGD thuần. Gradient clipping theo chuẩn toàn cục giữ hướng gradient nhưng rút ngắn nó nếu vượt ngưỡng. Trên mặt phẳng, vector $g = (3, 4)$ có chuẩn bằng 5; clipping về ngưỡng 2 đưa nó về điểm trên cùng hướng, nằm cách gốc đúng 2.



Hình 27: Clip chuẩn toàn cục giữ hướng gradient, clip từng tọa độ có thể đổi hướng (minh hoạ, tự tính)

Cách đọc hình 3: đường tròn nét đứt là các vector có chuẩn bằng ngưỡng 2. Mũi tên đỏ là gradient ban đầu, dài 5. Clip chuẩn toàn cục rút mũi tên về đường tròn theo đúng hướng đó; clip từng tọa độ cắt riêng thành phần và tạo hướng khác.

Với gradient toàn mô hình g, clipping chuẩn toàn cục được viết

$$g_{clip} = g \cdot \min\left(1, \frac{c}{\|g\|_2}\right),$$

trong đó c là ngưỡng. Nếu $\|g\|_2 \le c$, gradient giữ nguyên. Nếu vượt ngưỡng, ta nhân tất cả tọa độ với cùng tỷ lệ để chuẩn mới bằng c. Clipping từng tọa độ là quy tắc khác: mỗi thành phần được chặn riêng trong một khoảng, nên hướng chung có thể đổi.

Ví dụ tay (tự tính): với $g = (3, 4)$, chuẩn là $\sqrt{3^2 + 4^2} = 5$. Ngưỡng $c = 2$ cho tỷ lệ $2/5 = 0{,}4$ và $g_{clip} = (1{,}2, 1{,}6)$, có chuẩn 2. Clip từng tọa độ ở mức 2 lại cho $(2, 2)$, có chuẩn $\sqrt{8} \approx 2{,}828$ và hướng lệch

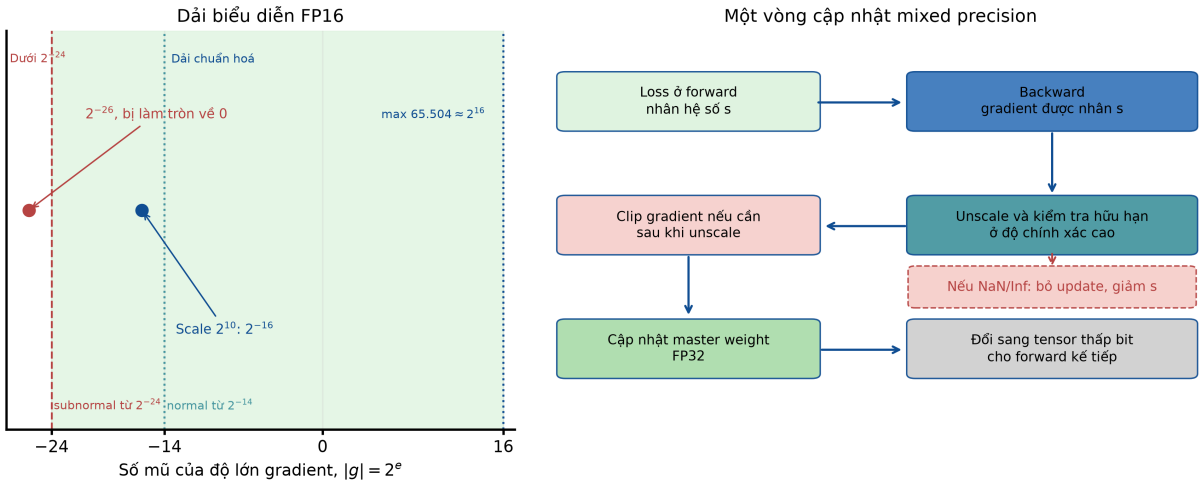
khỏi vector ban đầu (tự tính). Nếu learning rate là $0{,}1$, chuẩn của cập nhật trước clip là $0{,}5$, còn sau clip là $0{,}2$ (tự tính).

Clipping không sửa gradient biến mất, không đảm bảo loss giảm, và không thay thế việc điều chỉnh learning rate. Nó chỉ giới hạn gradient ở những bước vượt ngưỡng. Trong SGD thuần, gradient sau clip xác định trực tiếp hướng và độ dài bước. Với momentum hoặc optimizer thích ứng như Adam, clipping giới hạn gradient đầu vào của trạng thái optimizer, nhưng không đảm bảo chuẩn của bước tham số cuối cùng nhỏ hơn cùng ngưỡng; momentum, preconditioning và weight decay cũng góp phần vào bước đó. Hãy xác định rõ đang clip gradient nào và có clip trước hay sau unscale trong mixed precision.

7. Mixed precision: tính nhanh mà giữ phần nhạy ở độ chính xác cao

Một số dấu phẩy động dùng ít bit hơn FP32, giảm dung lượng lưu và có thể chạy nhanh hơn trên phần cứng phù hợp. Tuy nhiên, dải giá trị và độ phân giải của chúng cũng khác. Với FP16 chuẩn IEEE, số dương chuẩn nhỏ nhất có số mũ -14, còn số subnormal nhỏ nhất là 2^{-24}. Giá trị lớn nhất hữu hạn là 65.504. Gradient rất nhỏ có thể bị làm tròn thành 0; giá trị quá lớn có thể thành vô cực.

Hình sau phóng đại một ví dụ gradient nhỏ trên trục số mũ, rồi đặt cạnh thứ tự xử lý loss scaling trong một vòng cập nhật.



Hình 28: Loss scaling đưa gradient nhỏ vào dải FP16 rồi unscale trước cập nhật (minh hoạ, tự tính)

Cách đọc hình 4: điểm đỏ có số mũ thấp hơn ngưỡng subnormal nên có thể bị làm tròn về 0; nhân loss với 2^{10} đưa gradient minh hoạ sang vị trí xanh trong dải biểu diễn. Sơ đồ bên phải cho thấy phải unscale và kiểm tra gradient trước khi clip và cập nhật master weight; nhánh đỏ ghi bỏ bước cập nhật và giảm scale nếu gặp NaN/Inf. Mọi phép nhân/chia loss scale cần triệt tiêu đúng để giữ nguyên độ lớn gradient gốc.

7.1 Ví dụ tay: loss scaling

Xét gradient $g = 2^{-26}$. Nó nhỏ hơn số subnormal dương nhỏ nhất của FP16, 2^{-24}. Nếu chuyển trực tiếp sang FP16 với quy tắc làm tròn gần nhất, ví dụ này thành 0. Nhân loss với $s = 2^{10}$ làm gradient scaled thành 2^{-16}, nằm trong dải subnormal có thể biểu diễn. Sau backprop, chia lại cho 2^{10} ở độ chính xác cao khôi phục 2^{-26} (tự tính). Giá trị này được kiểm tra trực tiếp bằng NumPy.

Loss scaling dựa vào chain rule: nếu nhân loss với s, gradient của loss cũng được nhân với s. Sau backward phải unscale, tức chia gradient cho s, trước bước cập nhật. Nếu scale quá lớn, một giá trị trung gian có thể overflow thành infinity hoặc NaN. Dynamic loss scaling theo dõi overflow: khi gặp giá trị không hữu hạn thì bỏ bước cập nhật lỗi và giảm scale; sau các bước ổn định, scale có thể tăng. Đây là cơ chế thực hành phổ biến, trong khi bài mixed precision gốc 2017 chủ yếu khảo sát scale cố định.

7.2 FP16, BF16 và phần tích luỹ

FP16 có phần định trị nhiều bit hơn BF16 nhưng dải số mũ hẹp hơn. BF16 có dải số mũ gần FP32, nên thường chịu giá trị rất lớn và rất nhỏ tốt hơn, đổi lại làm tròn thô hơn. BF16 vì vậy thường không cần loss scaling để tránh underflow như FP16, nhưng vẫn cần tính toán ổn định. Khả năng hỗ trợ và hiệu năng tùy phần cứng.

Mixed precision không có nghĩa mọi phép toán đều chạy ở FP16. Một cách cài đặt phổ biến dùng định dạng thấp bit cho activation và toán nhân ma trận, tích luỹ tổng vào FP32, và giữ master weights cùng trạng thái optimizer ở FP32. Các phép reduction như chuẩn hoá, softmax hoặc tổng nhiều phần tử thường cần tích luỹ chính xác hơn. Việc chọn dtype chính xác phụ thuộc framework, thiết bị và toán tử; cần kiểm tra thay vì giả định.

Thứ tự quan trọng khi clipping cùng loss scaling là: scale loss, chạy backward, unscale gradient, phát hiện NaN/Inf, clip gradient chưa scale nếu cần, rồi gọi optimizer để cập nhật master weights. Nếu clip trước khi unscale với ngưỡng c trên gradient scaled, ngưỡng tương đương trên gradient gốc là c/s, nhỏ hơn c theo hệ số scale. Vì vậy cách đó làm thay đổi update ngoài ý muốn.

8. Ghép các cơ chế để huấn luyện ổn định

Khởi tạo và normalization chủ yếu ảnh hưởng đến thang tín hiệu trong mạng. Clipping xử lý một bước gradient lớn ở thời điểm cụ thể. Mixed precision tác động đến độ chính xác biểu diễn trong các phép tính. Một run ổn định thường cần một số lớp kiểm soát cùng lúc: khởi tạo hợp lý, loss và gradient không vô hạn, learning rate phù hợp, phép tích luỹ có độ chính xác đủ, và theo dõi các giá trị bất thường.

Ví dụ, nếu activation đã bùng nổ do scale khởi tạo sai, clipping có thể giảm gradient đưa vào optimizer nhưng không sửa các activation sai trong forward; với SGD thuần, nó trực tiếp giới hạn gradient của bước. Nếu gradient bị underflow trong FP16, clipping không thể khôi phục những giá trị đã biến thành 0. Nếu BatchNorm dùng thống kê thiếu mẫu, tăng độ chính xác số không làm thống kê đó đại diện hơn. Muốn chẩn đoán đúng, cần hỏi tín hiệu mất ở bước nào.

Trong thực hành, nên quan sát loss, norm gradient, tỷ lệ batch có overflow, giá trị activation ở vài tầng, và thống kê chuẩn hoá khi nghi ngờ. Một loss NaN có thể bắt đầu từ overflow, chia cho số gần 0, learning rate quá lớn hoặc dữ liệu không hợp lệ. Chỉ nhìn loss cuối cùng không đủ xác định nguyên nhân.

9. Liên hệ với AI

Trong mạng CNN với ReLU, He initialization giúp chọn độ lớn trọng số ban đầu có tính đến việc ReLU loại bỏ phần âm của activation. BatchNorm thường được dùng để chuẩn hoá activation theo kênh. Các ý này là nền để đọc optimization 37 và 38.

Trong mạng chuỗi và mô hình ngôn ngữ, gradient phải đi qua nhiều vị trí thời gian hoặc tầng. Exploding gradient là động lực lịch sử quan trọng cho clipping trong RNN. LayerNorm chuẩn hoá mỗi biểu diễn theo feature của chính nó, không phụ thuộc các mẫu còn lại trong batch. Xem optimization 39 và 40 để theo dõi hai kỹ thuật trong paper gốc.

Trong huấn luyện mô hình lớn, mixed precision giúp giảm chi phí lưu activation và tăng throughput trên phần cứng hỗ trợ. Các kỹ thuật này ghép với batch lớn và optimizer phân tán, nhưng không thay thế lựa chọn learning rate. Optimization 21 trình bày batch lớn, còn optimization 42 trình bày chi tiết FP16, master weights và loss scaling.

10. Sai lầm hay gặp

- Khởi tạo mọi trọng số bằng 0 vì muốn bắt đầu trung tính. Với các neuron hoán đổi được và phép tính phía sau đối xứng, chúng nhận gradient như nhau; zero init toàn mạng thường để nhiều tầng ẩn mà

không có khác biệt ban đầu. Zero init có thể được chọn có chủ ý ở một số tầng, nên cần xét cấu trúc xung quanh.

- Nghĩ He hoặc Xavier bảo đảm mạng sâu luôn ổn định. Quy tắc dựa trên giả định về phân phối, activation và cấu trúc; residual, normalization và gate có thể thay đổi thang đo.
- Xem BatchNorm và LayerNorm là cùng một phép toán với tên khác. Chúng lấy thống kê trên các trục khác nhau và phụ thuộc dữ liệu khác nhau.
- Dùng BatchNorm với batch bất kỳ mà không kiểm tra thống kê train/inference. Batch nhỏ, batch không đều hoặc phân phối giữa worker khác nhau có thể làm thống kê nhiễu.
- Clip từng tọa độ rồi cho rằng hướng gradient được giữ. Chỉ clip chuẩn toàn cục mới nhân mọi tọa độ với cùng hệ số và giữ hướng.
- Mong clipping chữa vanishing gradient. Clipping chỉ giảm gradient lớn; nó không làm gradient nhỏ quay lại.
- Scale loss rồi quên unscale trước optimizer hoặc clipping. Khi đó learning rate hay ngưỡng clip bị thay đổi ngoài ý muốn.
- Cho rằng mixed precision luôn chính xác như FP32 hoặc luôn nhanh hơn. Tích luỹ, overflow, underflow, memory bandwidth và hỗ trợ phần cứng đều ảnh hưởng kết quả.

11. Tóm tắt công thức

Khái niệm	Công thức hoặc cách đọc
Tầng tuyến tính	$z = Wx + b$
He cho ReLU	$\mathrm{Var}(W_{ij}) = 2/n_{in}$
Xavier/Glorot	$\mathrm{Var}(W_{ij}) \approx 2/(n_{in} + n_{out})$
Chuẩn hoá	$\widehat{x} = (x - \mu)/\sqrt{v + \epsilon}$
Scale và shift	$y = \gamma\widehat{x} + \beta$
Clipping chuẩn toàn cục	$g_{clip} = g \min(1, c/\|g\|_2)$
Loss scaling	$L' = sL$, gradient scaled là $s\nabla L$, unscale bằng cách chia s
Mixed precision	Toán nhân/lưu thấp bit, một số phép tích luỹ và trạng thái nhạy giữ ở độ chính xác cao

12. Bài tập tự tính

Bài 1

Một tầng ReLU có 64 đầu vào. Theo He initialization, hãy tính phương sai và độ lệch chuẩn của mỗi trọng số.

Đáp án: phương sai là $2/64 = 1/32 = 0{,}03125$. Độ lệch chuẩn là $\sqrt{1/32} \approx 0{,}1768$ (tự tính).

Bài 2

Các activation của một kênh trong batch là $[2, 4, 6]$. Tính mean, variance chia cho số giá trị và activation sau chuẩn hoá nếu bỏ qua ϵ.

Đáp án: mean bằng 4. Độ lệch là $[-2, 0, 2]$, nên variance là $(4 + 0 + 4)/3 = 8/3$ (tự tính). Sau chuẩn hoá, giá trị là khoảng $[-1{,}225, 0, 1{,}225]$ (tự tính).

Bài 3

Gradient là $g = (6, 8)$ và ngưỡng chuẩn là 5. Tính gradient sau clip chuẩn toàn cục.

Đáp án: chuẩn ban đầu là $\sqrt{36 + 64} = 10$. Tỷ lệ clip là $5/10 = 0{,}5$, nên gradient mới là $(3, 4)$ với chuẩn bằng 5 (tự tính).

Bài 4

Một gradient có độ lớn 2^{-26}. Nhân loss với 2^{10} trước backward thì gradient scaled có số mũ nào? Sau unscale, gradient trở lại số nào?

Đáp án: gradient scaled là $2^{-26}2^{10} = 2^{-16}$. Chia cho 2^{10} khôi phục 2^{-26} (tự tính). Giá trị scaled nằm trên ngưỡng subnormal thấp nhất của FP16, còn giá trị ban đầu nằm dưới ngưỡng đó.

Bài 5

Với activation ReLU có moment đầu vào bằng 1 và fan-in bằng 256, dùng phương sai He. Tính moment kỳ vọng sau tuyến tính rồi ReLU theo mô hình đơn giản.

Đáp án: phương sai trọng số là $2/256 = 1/128$. Moment trước ReLU là xấp xỉ $256 \times (1/128) \times 1 = 2$. ReLU giữ khoảng một nửa moment với đầu vào đối xứng, nên moment sau ReLU xấp xỉ 1 (tự tính).

Tài liệu tham khảo nền: Glorot và Bengio, Understanding the difficulty of training deep feedforward neural networks; He et al., Delving Deep into Rectifiers; Ioffe và Szegedy, Batch Normalization; Ba et al., Layer Normalization; Pascanu et al., On the difficulty of training recurrent neural networks; Micikevicius et al., Mixed Precision Training; các ghi chú optimization 37-42, 46.

Bài 00

Preface: Toán nền tảng và hướng dẫn đọc về Tối ưu hoá, Optimizer và Hyperparameter

Tác giả: ghi chú tổng hợp từ kiến thức nền của người biên soạn (không phải ghi chú của một paper cụ thể).
Venue / arXiv: không áp dụng.
Nguồn: không có nguồn đơn lẻ; các bài 01-51 trong thư mục này (tại thời điểm viết, đã kiểm bằng `ls`: toàn bộ 01-51 đã có, gồm bài 02, Bottou, Curtis, Nocedal, "Optimization Methods for Large-Scale Machine Learning", arXiv:1606.04838, xem mục 14); phần toán nền tảng của tối ưu lồi cổ điển (định nghĩa L-smooth, strongly convex, GD/Newton/quasi-Newton, ODE liên tục của momentum) không nằm riêng trong một bài nào mà tham khảo cấu trúc chuẩn của Boyd & Vandenberghe (*Convex Optimization*, 2004), Nocedal & Wright (*Numerical Optimization*, 2006), Nesterov (*Lectures on Convex Optimization*, 2018) và Goodfellow, Bengio & Courville (*Deep Learning*, 2016); không trích số trang cụ thể vì đây là tổng hợp kiến thức nền, không phải trích dẫn câu chữ.
Chủ đề: toán nền cho tối ưu hoá lồi cổ điển (hàm lồi, L-smooth, strongly convex, GD, Newton), SGD và nhiễu, momentum/Nesterov và góc nhìn ODE, các optimizer bậc hai và adaptive (từ AdaGrad đến Muon/Schedule-Free), learning rate schedule, chuẩn hoá/khởi tạo/clipping/mixed precision, tìm kiếm hyperparameter (grid/random/Bayesian/Hyperband/PBT) và muP, cạm bẫy khi so sánh optimizer, và cách đọc 51 bài theo lộ trình.
Vị trí trong lộ trình: Bài số 00, đọc **trước tất cả**. Phần lồi cổ điển (mục 1-3, 6) không nằm trong 51 bài còn lại (trừ bài 01, Bubeck, và bài 02, Bottou-Curtis-Nocedal) nên bạn cần đọc kỹ các mục đó trước khi sang bài 01 trở đi.

> Tài liệu học tập bằng tiếng Việt, viết từ kiến thức nền chung về tối ưu hoá, optimizer và hyperparameter, **không phải** ghi chú hay bản dịch của một paper cụ thể. Phần toán nền tảng (tập lồi, hàm lồi, GD, Newton, quasi-Newton, ODE liên tục của momentum) là kiến thức chuẩn trong các giáo trình tối ưu hoá, tham khảo cấu trúc trình bày của Boyd & Vandenberghe, Nocedal & Wright, Nesterov, và Goodfellow-Bengio-Courville; không trích dẫn số trang vì đây là tổng hợp, không phải trích dẫn từng câu. Mọi nhận định gắn với một bài cụ thể trong thư mục này được ghi rõ số bài (ví dụ "opt 06"). Mọi phép tính minh hoạ đánh dấu "(tự tính)" đã được kiểm bằng Python trước khi ghi vào đây. Mục liên hệ dự án (mục 15) là suy luận của người soạn, chưa được kiểm chứng thực nghiệm; khi một bài cụ thể nói khác với suy luận ở đây, hãy tin bài đó.

1. Bài toán tối ưu: loss landscape, cực tiểu, điểm yên

Bài toán tối ưu của deep learning là $\min_{\theta \in \mathbb{R}^d} L(\theta)$, với θ là toàn bộ tham số và L là loss trung bình trên dữ liệu huấn luyện. "**Loss landscape**" là cách nói hình ảnh cho đồ thị của L theo θ: một mặt d-chiều không vẽ được trực tiếp khi d lớn, chỉ chiếu xuống vài chiều (opt 27 dùng chuẩn hoá theo filter để chiếu xuống 2 chiều có ý nghĩa; opt 49 cho thấy gradient thực tế gần như nằm trong một không gian con rất nhỏ gồm vài chục eigenvector lớn nhất của Hessian).

Điểm dừng (critical/stationary point): $\nabla L(\theta) = 0$, điều kiện **cần** bậc một cho cực tiểu, không phải điều kiện đủ. Phân loại theo Hessian $H = \nabla^2 L(\theta)$:

- $H \succ 0$: cực tiểu địa phương chặt. $H \prec 0$: cực đại địa phương.
- H có cả eigenvalue dương và âm: **điểm yên (saddle point)**; đi theo eigenvector âm loss giảm tiếp, theo eigenvector dương loss tăng.
- H bán xác định (eigenvalue bằng 0): cần xét bậc cao hơn, hoặc là đường/mặt cực tiểu suy biến (đối xứng hoán vị neuron hoặc đối xứng scale giữa hai lớp liên tiếp).

Ví dụ tối giản. $f(x,y) = x^2 - y^2$ có $\nabla f(0,0) = (0,0)$, $H = \text{diag}(2,-2)$: điểm yên tại gốc, loss giảm dọc trục y, tăng dọc trục x.

Ví dụ sâu hơn, số liệu từ opt 05 (Goodfellow et al. 2014): mạng tuyến tính sâu tối giản $L(w_1,w_2) = (1-w_1w_2)^2$, $(0,0)$ là điểm yên (nghiệm toàn cục nằm trên cả hyperbol $w_1w_2 = 1$, do đối xứng scale giữa hai lớp). Dọc đường từ $(0,0)$ đến $(1,1)$: $L(\alpha) = (1-\alpha^2)^2$, $L''(\alpha) = -4 + 12\alpha^2$, âm tại gốc ($-4$, đúng tính chất điểm yên), dương tại nghiệm (8), đổi dấu tại $\alpha = 1/\sqrt{3} \approx 0.577$ (tự tính, từ opt 05). Minh hoạ cho hiện tượng quan sát rộng hơn: dọc đường từ khởi tạo đến nghiệm, loss mạng sâu thường giảm **đơn điệu** dù không gian tham số đầy điểm yên.

Vì sao điểm yên quan trọng hơn cực tiểu địa phương xấu (kiến thức nền chung): với mạng nhiều tham số, số điểm yên tăng nhanh hơn số cực tiểu địa phương khi độ sâu/rộng tăng (lý thuyết ma trận ngẫu nhiên); GD/SGD hiếm khi kẹt lâu ở cực tiểu địa phương tệ, thường chỉ chậm lại tạm thời gần điểm yên trước khi trôi ra theo hướng cong âm.

Local vs global: với hàm lồi (mục 2), cực tiểu địa phương **luôn** là cực tiểu toàn cục, nên lý thuyết hội tụ ở mục 3, 5, 6 chỉ cần nói "hội tụ về nghiệm". Với mạng neural (không lồi), phần lớn lý thuyết chỉ chứng minh hội tụ về **một điểm dừng** (gradient nhỏ), không chứng minh cực tiểu toàn cục; chất lượng cuối được đánh giá bằng thực nghiệm.

Sharp vs flat minima, liên quan đến độ cong quanh một cực tiểu, được bàn kỹ ở mục 11 cùng opt 24, 25 (SAM), 26 (edge of stability), 27.

---

2. Tập lồi, hàm lồi, L-smooth và strongly convex

Phần này là nền cho **toàn bộ** lý thuyết hội tụ ở mục 3, 5, 6. Không nằm trong 51 bài (ngoại trừ bài 01, Bubeck; định nghĩa dưới đây khớp mục 10 ghi chú đó, dùng β, α cho smooth/strongly convex; ở đây dùng ký hiệu phổ biến hơn, L và μ).

Tập lồi: $\mathcal{X} \subseteq \mathbb{R}^d$ lồi nếu $\forall x, y \in \mathcal{X}, t \in [0,1] : tx + (1-t)y \in \mathcal{X}$.

Hàm lồi: $f(tx + (1-t)y) \le tf(x) + (1-t)f(y)$ với mọi $x, y, t \in [0,1]$ (đồ thị nằm dưới mọi dây cung).

Điều kiện bậc một: f lồi $\iff$ $f(y) \ge f(x) + \nabla f(x)^\top (y-x)$ mọi x, y (tiếp tuyến luôn nằm dưới đồ thị).

Điều kiện bậc hai: f lồi $\iff$ $\nabla^2 f(x) \succeq 0$ mọi x. Đây là lý do mục 1 nói "cực tiểu địa phương của hàm lồi luôn là cực tiểu toàn cục": với θ^* cực tiểu địa phương, điều kiện bậc một cho $f(y) \ge f(\theta^*)$ với **mọi** y (vì $\nabla f(\theta^*) = 0$).

L-smooth: $\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\|$. Tương đương $\nabla^2 f(x) \preceq LI$. Hệ quả quan trọng cho mục 3, **cận trên dạng toàn phương**:

$$f(y) \le f(x) + \nabla f(x)^\top (y-x) + \frac{L}{2}\|y-x\|^2.$$

μ-strongly convex: $f(y) \ge f(x) + \nabla f(x)^\top (y-x) + \frac{\mu}{2}\|y-x\|^2$ ($\mu > 0$), tương đương $\nabla^2 f(x) \succeq \mu I$: **cận dưới dạng toàn phương**, đối xứng với L-smooth, đảm bảo nghiệm θ^* duy nhất.

Condition number:

$$\boxed{\kappa = \frac{L}{\mu} \ge 1}$$

κ lớn nghĩa là hàm "dẹt" một hướng, "dốc" hướng khác; $\kappa = 1$ là hàm "tròn đều" (ví dụ $f(x) = \frac{1}{2}\|x\|^2$). Mục 3, 5 cho thấy κ quyết định số bước hội tụ: GD cần $O(\kappa)$ bước, Nesterov/AGD $O(\sqrt{\kappa})$ bước, Newton $O(1)$ bước (opt 01, Theorem 3.3/3.18/3.19).

Ví dụ dùng xuyên suốt mục 3, 5, 6: $f(x) = \frac{1}{2}x^\top Ax$, $A = \text{diag}(L, \mu) \in \mathbb{R}^{2\times 2}$. $\nabla f(x) = Ax$, $\nabla^2 f(x) = A$, hai eigenvalue đúng là L, μ, nên f vừa L-smooth vừa μ-strongly convex với hằng số **chặt**. Nghiệm duy nhất $x^* = 0$: mô hình đơn giản nhất để tính tay mọi công thức ở mục sau.

Vì sao giả định này thường không đúng cho deep learning: loss mạng neural không lồi toàn cục, L thường rất lớn hoặc không đồng nhất. opt 41 đưa khái niệm tổng quát (L_0, L_1)-smooth: $\|\nabla^2 f(x)\| \le L_0 + L_1\|\nabla f(x)\|$; ví dụ $f(x) = x^3$ không L-smooth toàn cục nhưng là $(3, 1)$-smooth (tự tính, Lemma 2 opt 41). L, μ, κ kinh điển vẫn là từ vựng chuẩn để giải thích vì sao thuật toán hoạt động, và là điểm khởi đầu bắt buộc trước (L_0, L_1)-smooth.

---

3. Gradient descent và tốc độ hội tụ

Gradient descent (GD): $\theta_{t+1} = \theta_t - \eta\nabla f(\theta_t)$, với $\eta > 0$ là learning rate (bước học).

3.1 Hàm lồi, L-smooth (không cần strongly convex): tốc độ $O(1/t)$

Với $\eta = 1/L$, Theorem 3.3 của opt 01 cho $f(\theta_t) - f(\theta^*) \le \dfrac{2L\|\theta_1 - \theta^*\|^2}{t-1}$: sai số giảm theo $O(1/t)$, không phụ thuộc κ (chưa dùng strongly convex) - tốc độ "chậm" nhất trong các mục dưới đây.

3.2 Hàm toàn phương, L-smooth và μ-strongly convex: tốc độ tuyến tính (hội tụ hình học)

Dùng đúng ví dụ ở mục 2, $f(x) = \frac{1}{2}x^\top Ax$, $A = \text{diag}(L, \mu)$. Một bước GD:

$$x_{t+1} = x_t - \eta A x_t = (I - \eta A)x_t.$$

Vì A chéo, mỗi toạ độ co độc lập theo hệ số $(1 - \eta\lambda)$ với $\lambda \in \{L, \mu\}$. Sai số toạ độ ứng với eigenvalue λ sau t bước là $(1 - \eta\lambda)^t$ lần sai số ban đầu. Tốc độ hội tụ **toàn cục** (toạ độ chậm nhất) là

$$\rho(\eta) = \max\{|1 - \eta L|,\ |1 - \eta\mu|\}.$$

Chọn η tối ưu. $\rho(\eta)$ nhỏ nhất khi hai nhánh bằng nhau, $1 - \eta\mu = -(1 - \eta L)$, cho

$$\boxed{\eta^* = \frac{2}{L + \mu}}, \qquad \rho(\eta^*) = \frac{L - \mu}{L + \mu} = \frac{\kappa - 1}{\kappa + 1}.$$

Kiểm chứng số (tự tính, mô phỏng Python, $x_0 = (1, 1)$, dừng khi $\|x_t\| \le 10^{-6}\|x_0\|$):

κ (L/μ, $\mu = 1$)	ρ dự đoán	Số bước thực tế	Số bước dự đoán $\ln(10^{-6})/\ln\rho$
4	0.6000	28	27.0
10	0.8182	69	68.8
100	0.9802	691	690.8

Khớp gần như chính xác: số bước cần tăng **tuyến tính theo** κ (đúng "$O(\kappa)$ bước" nói ở mục 2) - lý do một bài toán "dẹt" (κ lớn) làm GD chậm hẳn so với bài toán "tròn" (κ nhỏ), dù cùng số chiều.

Ảnh hưởng của việc chọn η sai (tự tính, cùng ví dụ $\kappa = 10$):

Lựa chọn η	Giá trị	Kết quả
$1/L$ (an toàn, phổ biến trong lý thuyết)	0.1000	hội tụ sau 128 bước
$2/(L+\mu)$ (tối ưu)	0.1818	hội tụ sau 69 bước
$1.99/L$ (gần biên ổn định $2/L$)	0.1990	hội tụ sau 1341 bước (rất chậm, dù vẫn hội tụ)
$2.01/L$ (vượt biên $2/L$)	0.2010	**phân kỳ**

Hai điều cần nhớ: (1) $\eta = 1/L$ là lựa chọn ”an toàn” phổ biến trong chứng minh lý thuyết (mục 3.1) nhưng **không tối ưu** khi đã biết thêm μ; (2) có **biên ổn định cứng** $\eta < 2/L$: vượt biên này GD phân kỳ vô điều kiện, bất kể μ - cơ chế toán học phía sau ”learning rate quá cao làm loss nổ” (L thay đổi theo vùng tham số, nên vượt $2/L_{\text{cục bộ}}$ tại vùng đang đứng). Mục 11 nối điểm này với **edge of stability** (opt 26): huấn luyện mạng neural thực tế tự trôi đến vùng $\eta \cdot \lambda_{\max}(H)$ dao động quanh giá trị 2, đúng biên ổn định cổ điển này.

3.3 Trường hợp không lồi, chỉ L-smooth

Không còn khái niệm ”hội tụ về nghiệm θ^*”, chỉ còn chặn trên cho $\|\nabla f(\theta_t)\|$: GD với $\eta = 1/L$ đạt $\min_{t \le T} \|\nabla f(\theta_t)\|^2 = O(1/T)$ (cùng cấu trúc descent lemma như mục 3.1, chỉ cần f bị chặn dưới) - kiểu cận duy nhất chứng minh chặt được cho hầu hết bài toán deep learning thật, vì tính lồi hiếm khi đúng toàn cục.

---

4. SGD: nhiễu, batch size và lịch học Robbins-Monro

Vì sao SGD: tính $\nabla L(\theta)$ trên toàn bộ dữ liệu quá đắt. **Stochastic Gradient Descent (SGD)** dùng ước lượng không chệch từ một batch nhỏ: $\hat{g} = \frac{1}{B}\sum_{i=1}^{B} g_i$ với $g_i = \nabla \ell(\theta; x_i)$ là gradient trên một mẫu, B là batch size. $\mathbb{E}[\hat{g}] = \nabla L(\theta)$ (không chệch nếu mẫu được rút ngẫu nhiên, i.i.d. hoặc hoán vị đều).

Phương sai giảm theo $1/B$ (kiến thức nền chung). Nếu các g_i độc lập với phương sai (theo từng toạ độ) xấp xỉ σ^2:

$$\mathrm{Var}[\hat{g}] = \frac{\sigma^2}{B}.$$

Kiểm chứng số (tự tính, mô phỏng Python, nhiễu Gauss $\sigma = 2$, $200\,000$ mẫu):

B	Var đo được	σ^2/B lý thuyết
1	4.0126	4.0000
4	1.0002	1.0000
16	0.2498	0.2500
64	0.0619	0.0625
256	0.0155	0.0156

Hệ quả: **độ lệch chuẩn** của nhiễu gradient giảm theo $1/\sqrt{B}$, không phải $1/B$. Tăng B gấp 4 chỉ giảm độ lệch chuẩn một nửa: lợi ích giảm dần khi B đã lớn, ý tưởng ”noise scale” phía sau opt 22 (**critical batch size**: dưới ngưỡng này tăng B gần như tuyến tính giảm số bước cần, vượt ngưỡng thì không còn lợi nhiều) và opt

23 (tăng B và giảm η tác dụng tương đương lên nhiễu hiệu dụng, có thể thay giảm η theo lịch bằng tăng B theo lịch).

Quy tắc scale LR theo batch size (opt 21): tăng B lên k lần, scale η lên k lần theo **linear scaling rule**, kèm warmup vài epoch đầu (mục 8) để tránh instability lúc khởi động.

4.1 Điều kiện Robbins-Monro (1951, kinh điển, trước thời arXiv)

Cho lịch học giảm dần $\{\eta_t\}$, định lý hội tụ hầu chắc chắn (almost-sure) cổ điển của xấp xỉ ngẫu nhiên (stochastic approximation) yêu cầu hai điều kiện:

$$\boxed{\sum_{t=1}^{\infty} \eta_t = \infty} \qquad \text{và} \qquad \boxed{\sum_{t=1}^{\infty} \eta_t^2 < \infty}.$$

Điều kiện một đảm bảo tổng quãng đường di chuyển đủ lớn để tới được nghiệm dù khởi tạo xa đến đâu; điều kiện hai đảm bảo nhiễu cộng dồn qua các bước không phát nổ (phương sai tích lũy bị chặn).

Kiểm chứng số cho $\eta_t = 1/t$ (tự tính, $T = 100\,000$):

Đại lượng	Giá trị tính	Kỳ vọng lý thuyết
$\sum_{t \le T} 1/t$	12.090	$\approx \ln T + \gamma = 12.090$ (hằng số Euler-Mascheroni $\gamma \approx 0.5772$) $\to \infty$ khi $T \to \infty$
$\sum_{t \le T} 1/t^2$	1.644924	$\to \pi^2/6 = 1.644934$ (hội tụ)

$\eta_t = 1/t$ thỏa **cả hai** điều kiện Robbins-Monro: tổng phân kỳ (chậm, như $\ln T$) nhưng tổng bình phương hội tụ. Đây là lịch học ”kinh điển” được chứng minh hội tụ hầu chắc chắn.

Trường hợp $\eta_t = 1/\sqrt{t}$ (tự tính): $\sum 1/\sqrt{t}$ cũng phân kỳ (630.997 tại $T = 10^5$, thỏa điều kiện một), nhưng $\sum \eta_t^2 = \sum 1/t$ **cũng phân kỳ** (12.090, giống hàng trên) nên **vi phạm điều kiện hai**: $\eta_t = 1/\sqrt{t}$ không thỏa Robbins-Monro theo nghĩa cổ điển này.

Lưu ý, tránh nhầm lẫn hai khung lý thuyết: $\eta_t = 1/\sqrt{t}$ xuất hiện phổ biến trong tối ưu lồi (AdaGrad, mục 7; Theorem 6.3 opt 01 cho SGD trên hàm lồi Lipschitz, tốc độ $O(1/\sqrt{T})$) nhưng đó là khung **khác**: chặn sau T bước cố định biết trước (finite-horizon/regret), không phải hội tụ hầu chắc chắn khi $t \to \infty$ của Robbins-Monro. Đừng lẫn ”$\eta_t = 1/\sqrt{t}$ tối ưu trong T bước cố định” với ”thoả Robbins-Monro” (câu sau **sai**, như vừa kiểm chứng).

Thực hành (xem trước mục 8, 11): lịch học phổ biến (cosine, WSD) không thoả Robbins-Monro theo nghĩa chặt (giảm về sàn $\eta_{\min} > 0$ hoặc hoạt động trong horizon T biết trước), lý thuyết áp dụng thường là finite-horizon chứ không phải hội tụ tiệm cận cổ điển - khoảng trống lý thuyết-thực hành đáng lưu ý (mục 11, 13).

---

5. Momentum, Nesterov và góc nhìn ODE liên tục

5.1 Heavy-ball momentum (Polyak 1964, kinh điển)

$$v_{t+1} = \gamma v_t - \eta \nabla f(\theta_t), \qquad \theta_{t+1} = \theta_t + v_{t+1}$$

(opt 03, Eq. 4, quy ước dấu khác nhưng tương đương). Trực giác: v là ”vận tốc” tích lũy, giữ hướng đi nhất quán qua nhiều bước, giảm dao động trong ”khe hẹp” (hướng cong khác nhau nhiều, tình huống κ lớn ở mục 3).

5.2 Nesterov Accelerated Gradient (NAG)

$$v_{t+1} = \gamma v_t - \eta \nabla f(\theta_t + \gamma v_t), \qquad \theta_{t+1} = \theta_t + v_{t+1}$$

(opt 03, Eq. 5: gradient tính tại điểm "nhìn trước" $\theta_t + \gamma v_t$). Với hàm L-smooth và μ-strongly convex, chọn hệ số momentum đúng theo κ cho tốc độ $O(\sqrt{\kappa}\log(1/\varepsilon))$ bước, **nhanh hơn GD một hệ số** $\sqrt{\kappa}$ (so với $O(\kappa\log(1/\varepsilon))$, mục 3.2). Công thức (Theorem 3.18, opt 01):

$$y_{t+1} = x_t - \frac{1}{L}\nabla f(x_t), \qquad x_{t+1} = y_{t+1} + \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}(y_{t+1} - y_t).$$

Kiểm chứng số (tự tính bởi opt 01, tái dùng): với $\kappa = 10^4$ và $\varepsilon = 10^{-6}$ ($\ln(10^6) \approx 13.816$): số bước GD $\kappa\log(1/\varepsilon) \approx 138\,155$, số bước AGD $\sqrt{\kappa}\log(1/\varepsilon) \approx 1\,382$, tỉ lệ $\approx 100.0 = \sqrt{\kappa}$. Với bài toán rất "dẹt" ($\kappa = 10^4$), Nesterov AGD cần khoảng 100 lần ít bước hơn GD để đạt cùng độ chính xác.

5.3 Góc nhìn ODE liên tục (continuous-time view)

Heavy-ball liên tục. Khi $\eta \to 0$, đệ quy heavy-ball (5.1) xấp xỉ phương trình vi phân bậc hai với ma sát **hằng số**: $m\ddot{\theta} + \gamma\dot{\theta} + \nabla f(\theta) = 0$ (dao động tử điều hoà tắt dần). Với f toàn phương 1 chiều, hệ số cong λ: nghiệm đặc trưng $r = (-\gamma \pm \sqrt{\gamma^2 - 4\lambda})/2$. Ba trường hợp: $\gamma^2 > 4\lambda$ (overdamped, hội tụ không dao động nhưng chậm), $\gamma^2 = 4\lambda$ (critically damped, nhanh nhất không dao động), $\gamma^2 < 4\lambda$ (underdamped, dao động tắt dần, đúng "overshoot" của momentum khi κ lớn).

ODE của Nesterov có ma sát giảm dần theo $1/t$, phát hiện trung tâm của opt 04 (Su, Boyd, Candès). Giới hạn liên tục của lược đồ Nesterov gốc (opt 04, Eq. 1) khi $s \to 0$ là (Eq. 3):

$$\boxed{\ddot{X} + \frac{3}{t}\dot{X} + \nabla f(X) = 0}$$

tốc độ hội tụ $f(X(t)) - f^* \le 2\|x_0 - x^*\|^2/t^2$ (Theorem 3, hằng số 2 tối ưu). Tổng quát với ma sát r/t (Eq. 17): $r \ge 3$ giữ nguyên $O(1/t^2)$ (Theorem 5), $r < 3$ mất tốc độ này - **chuyển pha chặt** tại $r = 3$. Với hàm toàn phương, nghiệm ODE là hàm Bessel, dao động tắt dần với biên độ giảm theo $t^{-3/2}$ (Eq. 8-10).

Kiểm chứng chu kỳ dao động (tự tính bởi opt 04, tái dùng): hai độ cong khác nhau cho $T_1 = \pi/\sqrt{0.04} = 5\pi$ và $T_2 = \pi/\sqrt{0.01} = 10\pi$, khớp hình minh hoạ trong paper.

Vì sao quan trọng hơn một chi tiết lý thuyết: gia tốc Nesterov không phải "thêm quán tính" mơ hồ, mà là ma sát **giảm dần đúng tốc độ** $1/t$; ma sát hằng (heavy-ball) không đạt $O(1/t^2)$ tổng quát. Ý tưởng "ma sát/momentum nên đổi theo bước" là tiền đề của: restart theo tốc độ (opt 04, Theorem 10, hội tụ tuyến tính nhờ reset định kỳ) và ba dãy số nội suy x, y, z của Schedule-Free (opt 17, mục 7).

---

6. Bậc hai: Newton, quasi-Newton và natural gradient

6.1 Newton's method

Từ khai triển Taylor bậc hai $f(\theta + \delta) \approx f(\theta) + \nabla f(\theta)^\top \delta + \frac{1}{2}\delta^\top H \delta$, cực tiểu hoá theo δ cho bước Newton:

$$\boxed{\theta_{t+1} = \theta_t - H_t^{-1}\nabla f(\theta_t)}$$

Với hàm toàn phương, Newton hội tụ đúng nghiệm trong đúng 1 bước, bất kể κ (mô hình Taylor bậc hai chính là f thật). Kiểm chứng số (tự tính, cùng ví dụ $f(x) = \frac{1}{2}x^\top A x$): với $\kappa = 10$ và $\kappa = 1000$, cả hai

đều cho $x_1 = (0, 0)$ (đúng nghiệm) từ $x_0 = (3, -2)$ sau đúng 1 bước. So với GD cần $O(\kappa)$ bước (mục 3.2), Newton **không phụ thuộc** κ.

Vì sao không dùng Newton trực tiếp cho deep learning: cần lưu $H \in \mathbb{R}^{d\times d}$ và giải $H\delta = -\nabla f$, chi phí $O(d^2)$ bộ nhớ, $O(d^3)$ tính toán. Với $d \approx 0.8 \times 10^9$ (quy mô dự án, mục 15): H đầy đủ ở fp32 tốn $\approx 2.3 \times 10^6$ TiB (tự tính) - không khả thi trên bất kỳ cụm máy thực tế nào.

6.2 Quasi-Newton và L-BFGS

BFGS: xây dựng xấp xỉ $B_t \approx H_t$ chỉ từ hiệu gradient, qua phương trình cát tuyến $B_t s_t = y_t$ ($s_t = \theta_t - \theta_{t-1}$, $y_t = \nabla f(\theta_t) - \nabla f(\theta_{t-1})$), cập nhật hạng hai mỗi bước (công thức chuẩn, xem Nocedal & Wright). Vẫn lưu B_t dạng $O(d^2)$: không khả thi ở $d \sim 10^9$.

L-BFGS: chỉ giữ m cặp (s_i, y_i) gần nhất (m thường 5-20), tính $H^{-1}v$ gián tiếp qua "two-loop recursion", bộ nhớ $O(2md)$.

(Tự tính) Vì sao L-BFGS vẫn không khả thi ở quy mô dự án ($d \approx 0.8 \times 10^9$), bộ nhớ $2md \times 4$ byte (fp32):

m	2	5	10	20
Bộ nhớ	11.92 GiB	29.80 GiB	59.60 GiB	119.21 GiB

Ngay cả $m = 2$ đã tốn gần 12 GiB, vượt ngân sách 8 GB. "L-BFGS rẻ hơn Newton" đúng **so với Newton đầy đủ**, nhưng ở quy mô LLM/VLM hiện đại vẫn có thể "quá đắt cho một GPU 8 GB" - động lực thực tế cho Adafactor, Shampoo/SOAP, Muon (mục 7).

6.3 Gauss-Newton, Fisher information và natural gradient

Với loss là log-likelihood âm, Hessian thật tách gần đúng thành phần nửa dương xác định (Gauss-Newton) cộng phần dư; khi mô hình khớp tốt dữ liệu, $H \approx$ **ma trận Fisher** $F = \mathbb{E}[\nabla \log p \nabla \log p^\top]$. Dẫn xuất chi tiết (Gauss-Newton cho một tầng tuyến tính, K-FAC) đã trình bày ở **pruning content, mục 6.2** (dùng cho chấm điểm saliency, công thức toán giống hệt); ở đây thêm góc nhìn tối ưu:

Natural gradient: $\theta_{t+1} = \theta_t - \eta F_t^{-1} \nabla L(\theta_t)$, **bất biến với phép đổi biến tham số** (đo bước đi bằng KL-divergence $\approx \frac{1}{2}\delta^\top F \delta$ chứ không phải khoảng cách Euclid thô). **K-FAC** xấp xỉ F theo khối Kronecker mỗi lớp (cùng cấu trúc toán với Shampoo, mục 7.3). Đọc thêm (ngoài 51 bài): math content 06, math 07.

Sophia (opt 13) là cách "nhẹ" đưa thông tin bậc hai vào mà không cần F^{-1} đầy đủ: chỉ ước lượng **đường chéo** Hessian bằng Hutchinson ($\hat{h} = u \odot \nabla^2 L(\theta) u$) hoặc Gauss-Newton-Bartlett, cập nhật mỗi $k = 10$ bước, rồi clip update: $\theta_{t+1} = \theta_t - \eta\lambda\theta_t - \eta \cdot \text{clip}(m_t / \max\{\gamma h_t, \epsilon\}, 1)$. Overhead đo được (opt 13, Table 1): model 770M, AdamW 3.25s/bước, Sophia-H 3.40s/bước (dưới 5% chi phí thêm), đổi lại hội tụ nhanh hơn khoảng 2 lần trên GPT-2/GPT-NeoX tới 6.6B tham số: bậc hai **có** đáng dùng ở quy mô LLM, nếu chỉ lấy đường chéo và không cập nhật mỗi bước.

6.4 Preconditioning: khung nhìn thống nhất

Mọi phương pháp ở mục 3, 5, 6, 7 viết được dưới dạng $\theta_{t+1} = \theta_t - \eta P_t^{-1} g_t$ với P_t là "ma trận tiền điều kiện", tương đương nghiệm của "steepest descent dưới một chuẩn khác Euclid" (opt 48, "Old Optimizer, New Norm"). Bảng đối chiếu (Table 1, opt 48):

Không gian	Chuẩn	Optimizer tương ứng
$\mathbb{R}^n$	Euclid ℓ_2	GD / SGD ($P = I$)
$\mathbb{R}^n$	vô cùng ℓ_∞	sign descent / Adam khi tắt EMA

Không gian	Chuẩn	Optimizer tương ứng
$\mathbb{R}^{m\times n}$	Frobenius S_2	GD / SGD cho ma trận ($P=I$)
$\mathbb{R}^{m\times n}$	phổ S_∞	spectral descent / Shampoo khi tắt tích luỹ lịch sử

GD là $P=I$; Newton là $P=H$; AdaGrad/RMSProp/Adam là $P=\text{diag}(v_t)$ (xấp xỉ đường chéo thô của F hoặc H); Shampoo/SOAP là P dạng Kronecker; Muon trực giao hoá trực tiếp ma trận update. Đây là lý do mục 7 gom cả chục optimizer vào một bảng: chúng chỉ khác nhau ở cách chọn P_t, với ngân sách bộ nhớ/tính toán khác nhau.

---

7. Họ optimizer adaptive: từ AdaGrad đến Schedule-Free

7.1 Vì sao cần "adaptive": AdaGrad, RMSProp, Adadelta

AdaGrad (opt 03, Eq. 8-9): $\theta_{t+1,i} = \theta_{t,i} - \dfrac{\eta}{\sqrt{G_{t,ii}+\epsilon}} g_{t,i}$, $G_t = \sum_{\tau\le t} g_\tau g_\tau^\top$ (chỉ dùng đường chéo). Learning rate riêng từng toạ độ, tốt cho đặc trưng thưa, nhưng $G_{t,ii}$ **tích lũy không giảm** nên $\eta/\sqrt{G_{t,ii}}\to 0$: tự tính, $\eta=0.01$, gradient hằng số, hệ số $\eta/\sqrt{t}$ tại $t=1,10,100,10^4$ lần lượt $\approx 0.01, 0.00316, 0.001, 0.0001$, có thể "chết" quá sớm trên bài toán dài.

RMSProp / Adadelta (opt 03, Eq. 17-18) thay tổng cộng dồn bằng **EMA**: $v_t=\rho v_{t-1}+(1-\rho)g_t^2$, "quên" dần lịch sử cũ, không chết về 0.

7.2 Adam và các biến thể sửa lỗi: AdamW, AMSGrad, RAdam, LAMB

Adam (opt 06): EMA bậc một + bậc hai + bias-correction:

$$m_t=\beta_1 m_{t-1}+(1-\beta_1)g_t,\ v_t=\beta_2 v_{t-1}+(1-\beta_2)g_t^2,\ \hat m_t=\frac{m_t}{1-\beta_1^t},\ \hat v_t=\frac{v_t}{1-\beta_2^t},\ \theta_t=\theta_{t-1}-\eta\frac{\hat m_t}{\sqrt{\hat v_t}+\epsilon}$$

mặc định $\eta=0.001, \beta_1=0.9, \beta_2=0.999, \epsilon=10^{-8}$.

Bias-correction, tự tính (gradient hằng số $g=1$, $\beta_1=0.9$, $m_0=0$):

t	m_t thô	$\hat m_t$
1	0.1000	1.0000
2	0.1900	1.0000
3	0.2710	1.0000
4	0.3439	1.0000
5	0.4095	1.0000

m_t thô khởi động rất nhỏ (cold-start bias của EMA từ 0), nhưng $\hat m_t$ sửa đúng về 1.0000 ở mọi bước. Bias-correction sửa **kỳ vọng**, nhưng không sửa **phương sai** lớn của $\hat v_t$ ở vài bước đầu - đây chính là động lực của RAdam dưới đây.

AdamW (opt 07): tách weight decay khỏi phần adaptive: $\theta_t=\theta_{t-1}-\eta_t(\alpha\hat m_t/(\sqrt{\hat v_t}+\epsilon)+\lambda\theta_{t-1})$. Với SGD, cộng $\frac{\lambda}{2}\|\theta\|^2$ vào loss (L2) và trừ trực tiếp $\lambda\theta$ (weight decay) là **một việc** (Proposition 1, opt 07). Với Adam, gradient của số hạng L2 bị chia cho $\sqrt{\hat v_t}$ **giống phần gradient chính**, nên toạ độ có $\hat v_t$ lớn bị giảm decay hiệu dụng ngoài ý muốn; hai cách chỉ tương đương khi preconditioner là hằng số nhân đơn vị (Proposition 2),

điều không đúng với Adam. AdamW tách $\lambda\theta_{t-1}$ ra khỏi phần chia $\sqrt{\hat{v}_t}$ để khôi phục đúng ý nghĩa weight decay; kết quả: giảm khoảng 15% test error tương đối so với Adam ở LR mặc định 0.001 (opt 07, CIFAR-10/ImageNet32x32).

Vì sao vẫn cần weight decay, hai góc nhìn bổ sung: opt 50 cho thấy weight decay tương tác với nhiễu SGD tạo một dạng điều chuẩn ẩn lên chuẩn Jacobian, $\bar{L}_\lambda = L_\lambda + \eta\sigma^2_{\eta,\lambda}\|J\|_F^2$. opt 51 cho thấy weight decay của AdamW tương đương một EMA với hằng số thời gian $\tau_{\text{iter}} = 1/(\eta\lambda)$ (tự tính opt 51: $(0.99)^{10} \approx 0.9044$ khớp $e^{-0.1} \approx 0.9048$); giữ τ_{epoch} không đổi khi N tăng 4 lần cần λ giảm đúng 4 lần; khi scale width theo muP ($\eta \propto 1/s$), phải scale $\lambda \propto s$ để giữ τ_{iter} không đổi (tự tính opt 51, $s = 4$: τ_{iter} từ $40\,000$ về $10\,000$).

AMSGrad (opt 08): sửa phản ví dụ Adam hội tụ về điểm tệ nhất, bằng max tích luỹ $\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$ thay EMA debiased.

RAdam (opt 09): phương sai của $1/\sqrt{\hat{v}_t}$ có thể phân kỳ ở bước đầu; hệ số chỉnh r_t giữ phương sai hiệu dụng không đổi, tắt hẳn phần adaptive khi $\rho_t \leq 4$ (suy biến về SGD-momentum); $\rho_\infty = 1999$ với $\beta_2 = 0.999$ (tự tính opt 09).

LAMB (opt 10): thêm trust ratio theo lớp cho batch cực lớn (BERT trong 76 phút, batch 32768, opt 10); **không giảm bộ nhớ** so với Adam, lợi ích là khả năng dùng batch cực lớn trên hạ tầng song song, không phải lợi ích cho một GPU đơn.

7.3 Không EMA bậc hai: Lion, Adafactor, Sophia, Shampoo/SOAP, Muon, Schedule-Free

Lion (opt 11): chỉ 1 trạng thái momentum, update bằng **dấu** của EMA ($c_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$, $\theta_t = \theta_{t-1} - \eta(\text{sign}(c_t) + \lambda\theta_{t-1})$); lợi ích tăng theo batch size, **có thể kém AdamW ở batch dưới 64** (opt 11, Hình 8) - quan trọng cho dự án (mục 15).

Adafactor (opt 12): không lưu $v \in \mathbb{R}^{n\times m}$ đầy đủ, chỉ lưu tổng hàng/cột $R \in \mathbb{R}^n, C \in \mathbb{R}^m$, tái tạo $\hat{v} \approx RC/\mathbb{1}^\top R$: $O(nm) \to O(n+m)$. Tự tính: ma trận 1024×4096 giảm khoảng **819 lần** cho accumulator.

Sophia (opt 13): xem mục 6.3.

Shampoo (opt 14)/**SOAP** (opt 15): $L_t = L_{t-1} + G_t G_t^\top$, $R_t = R_{t-1} + G_t^\top G_t$, $W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}$: full-matrix AdaGrad xấp xỉ qua Kronecker, $O(m^2+n^2)$ thay $O(m^2n^2)$. SOAP chạy Adam **trong eigenbasis** của các preconditioner này, cập nhật eigenbasis chậm hơn (mỗi f bước).

Muon (opt 16): chỉ momentum M_t (không v_t), **trực giao hoá** bằng Newton-Schulz lặp (xấp xỉ $UV^\top$ không cần SVD): $W_t = W_{t-1} - \eta_t \,\text{NewtonSchulz}(M_t)$.

Schedule-Free (opt 17): ba dãy nội suy x, y, z thay EMA-momentum, giữ tốc độ $O(1/\sqrt{T})$ với **mọi** T, không cần biết trước tổng số bước (Theorem 1, 3) - giải đúng khoảng trống nêu ở cuối mục 4.1.

7.4 Bảng tổng hợp: trạng thái và bộ nhớ thêm (ngoài chính tham số)

Nhóm "theo toạ độ" (elementwise/diagonal), đo bằng số tensor cỡ đúng bằng số tham số:

Optimizer	Bài	Công thức cập nhật cốt lõi (rút gọn)	Số tensor trạng thái thêm
SGD	kinh điển	$\theta \leftarrow \theta - \eta g$	0
SGD + Momentum	opt 03	$v \leftarrow \gamma v + \eta g$; $\theta \leftarrow \theta - v$	1 (v)
Nesterov/NAG	opt 03	gradient tại $\theta + \gamma v$	1 (v)
AdaGrad	opt 03	$\theta \leftarrow \theta - \eta g/\sqrt{G+\epsilon}$, $G \mathrel{+}= g^2$	1 (G)
RMSProp	opt 03	$v \leftarrow \rho v + (1-\rho)g^2$; $\theta \leftarrow \theta - \eta g/\sqrt{v+\epsilon}$	1 (v)
Adam / AdamW	opt 06, 07	$\theta \leftarrow \theta - \eta\,\hat{m}/(\sqrt{\hat{v}}+\epsilon)$ ($+\lambda\theta$ tách riêng với AdamW)	2 (m, v)

Optimizer	Bài	Công thức cập nhật cốt lõi (rút gọn)	Số tensor trạng thái thêm
AMSGrad	opt 08	như Adam, $\hat{v} = \max(\hat{v}_{\text{cũ}}, v)$	3 $(m, v, \hat{v}_{\max})$
RAdam	opt 09	như Adam, thêm hệ số chỉnh r_t	2 (m, v)
LAMB	opt 10	như Adam, thêm trust ratio theo lớp	2 (m, v)
Lion	opt 11	$\theta \leftarrow \theta - \eta(\text{sign}(c) + \lambda\theta)$	1 (m)
Sophia	opt 13	$\theta \leftarrow \theta - \eta\lambda\theta - \eta\,\text{clip}(m/\max(\gamma h, \epsilon), 1)$	2 (m, h)
Muon	opt 16	$\theta \leftarrow \theta - \eta\,\text{NewtonSchulz}(M)$	1 (M, không có v)
Schedule-Free AdamW	opt 17	3 dãy x, y, z thay momentum; v EMA như Adam	2 (z, v; x dùng chung vai trò tham số)

Nhóm ”cấu trúc” (non-diagonal, phụ thuộc hình dạng ma trận $m \times n$, không phải đơn vị ”số tensor cỡ tham số”):

Optimizer	Bài	Trạng thái	Bộ nhớ thêm
Adafactor (factorized, không momentum)	opt 12	$R \in \mathbb{R}^m,\ C \in \mathbb{R}^n$	$O(m+n)$, cận dưới xa AdamW
Shampoo	opt 14	$L \in \mathbb{R}^{m\times m},\ R \in \mathbb{R}^{n\times n}$	$O(m^2 + n^2)$
SOAP (đầy đủ)	opt 15	$L, Q_L \in \mathbb{R}^{m\times m}$; $R, Q_R \in \mathbb{R}^{n\times n}$; $M, V \in \mathbb{R}^{m\times n}$	$2m^2 + 2n^2 + 3mn$
SOAP (factorized + one-sided)	opt 15	như trên nhưng chỉ xoay chiều nhỏ, dùng Adafactor cho V	$2\min(m,n)^2 + 2mn$

Ví dụ số cụ thể để so sánh trực giác (tự tính, một ma trận chiếu $m = 2048, n = 512$, điển hình cho một phần mô hình cỡ 0.8B, fp32):

Optimizer	Công thức bộ nhớ	Giá trị
AdamW ($2mn$, chỉ m, v)	$2 \times 2048 \times 512 \times 4$ byte	≈ 8.4 MB
Muon ($1 \times mn$, chỉ momentum)	$2048 \times 512 \times 4$ byte	≈ 4.2 MB
SOAP factorized + one-sided	$2(512)^2 + 2(2048)(512)$ số $\times 4$ byte	≈ 10.5 MB
Shampoo đầy đủ (L, R, $m^2 + n^2$)	$(2048^2 + 512^2) \times 4$ byte	≈ 17.2 MB

Kết luận trực tiếp cho dự án 8 GB (chi tiết hơn ở mục 15): **Muon** rẻ nhất (nửa AdamW); **Adafactor** còn rẻ hơn nữa cho ma trận lớn (tỉ lệ $O(m+n)$); **Schedule-Free AdamW** không tốn thêm so với AdamW (bằng định nghĩa); **Shampoo/SOAP đầy đủ** thường **đắt hơn** AdamW cho ma trận lớn gần vuông, chỉ biến thể factorized+one-sided của SOAP mới có cơ hội rẻ hơn AdamW, và chỉ với hình dạng ma trận đủ lệch ($\min(m,n)$ nhỏ).

---

8. Learning rate schedule: warmup, cosine, WSD, restart

Warmup: tăng η từ 0 lên giá trị đích trong vài trăm/nghìn bước đầu. Hai lý do: (a) RAdam (mục 7.2) cho thấy phương sai của $1/\sqrt{\hat{v}_t}$ rất lớn ở bước đầu khi ρ_t nhỏ; (b) khởi tạo ngẫu nhiên thường ở vùng $L_{\text{cục bộ}}$ lớn bất thường, η đích có thể vượt biên $2/L_{\text{cục bộ}}$ (mục 3.2) nếu áp dụng ngay.

Cosine annealing + warm restart (opt 18, SGDR, Eq. 5):

$$\eta_t = \eta^i_{\text{min}} + \frac{1}{2}(\eta^i_{\text{max}} - \eta^i_{\text{min}})\Big(1 + \cos\Big(\frac{T_{\text{cur}}}{T_i}\pi\Big)\Big)$$

Giá trị cụ thể (tự tính, $\eta_{\text{max}} = 10^{-3}$, $\eta_{\text{min}} = 10^{-5}$, $T_i = 1000$):

T_{cur}	η
0	0.001000
250	0.000855
500	0.000505
750	0.000155
1000	0.000010

Độ dài chu kỳ sau mỗi lần restart: $T_i = T_0 \cdot T^{i-1}_{\text{mult}}$. Nghiệm tại cuối chu kỳ (lúc $\eta = \eta_{\text{min}}$) làm checkpoint; nhiều checkpoint trước các lần restart cuối dùng làm **snapshot ensemble** gần miễn phí. Kết quả: CIFAR-10 3.14%, CIFAR-100 16.21% (opt 18, $N = 16$ run, $M = 3$ snapshot/run).

1cycle / super-convergence (opt 19): một chu kỳ **duy nhất** ngắn hơn tổng ngân sách, cộng đoạn "annihilation" ở cuối; LR tối đa xác định bằng **LR range test**. Nguyên lý cân bằng regularization: η lớn, batch nhỏ, weight decay, dropout chia nhau một "ngân sách regularization", tăng cái này phải giảm cái khác (opt 19, 47). Kết quả: ResNet-56/CIFAR-10 đạt 92.4% sau 10.000 iteration so với 91.2% sau 80.000 (lịch mặc định) - nhanh hơn 8 lần. Super-convergence quan sát được với Nesterov, AdaDelta, AdaGrad, nhưng **không** với Adam (opt 19): lịch học và optimizer không độc lập hoàn toàn.

WSD (Warmup-Stable-Decay) (opt 20, MiniCPM, Eq. 1):

$$WSD(T; s) = \begin{cases} \frac{s}{W}\eta, & s < W \\ \eta, & W < s < T \\ f(s-T)\eta, & T < s < S \end{cases}$$

Tách pha "ổn định" (η cao, dừng ở **bất kỳ** điểm nào) và pha "decay" ngắn ($\approx 10\%$ tổng token) hội tụ tinh tại điểm dừng, chất lượng tương đương cosine đặt $T =$ điểm dừng mà không cam kết trước tổng số bước - thuộc tính Schedule-Free (mục 7.3) cũng nhắm tới nhưng bằng nội suy ba dãy số thay vì ba pha rời rạc.

Liên hệ ngược mục 4.1: không lịch nào ở đây (cosine, 1cycle, WSD) thoả Robbins-Monro cổ điển (giảm về sàn $\eta_{\text{min}} > 0$ hoặc hoạt động trong horizon cố định T); lý thuyết áp dụng (khi có) là finite-horizon hoặc thực nghiệm, không phải hội tụ tiệm cận.

---

9. Chuẩn hoá, khởi tạo, clipping và mixed precision

Các kỹ thuật ở mục này không phải "optimizer", nhưng **thay đổi trực tiếp** L, μ, κ hiệu dụng mà optimizer ở mục 3-7 phải xử lý; đọc như "làm cho bài toán tối ưu dễ hơn trước khi optimizer vào việc".

Khởi tạo Kaiming (opt 37): giữ phương sai activation/gradient không đổi qua các lớp (tránh tích luỹ β^L gây tràn/tiêu biến số); với ReLU, điều kiện $\frac{1}{2}n_l\text{Var}[w_l] = 1$ cho std $= \sqrt{2/n_l}$ (opt 37, Eq. 8, 10) - tiền đề để optimizer thấy một bài toán có L hợp lý ngay từ bước 0.

Batch Normalization (opt 38): $\hat{x}_i = (x_i - \mu_B)/\sqrt{\sigma^2_B + \epsilon}$, $y_i = \gamma\hat{x}_i + \beta$. **Bất biến scale** $BN(Wu) = BN((aW)u)$ nên gradient theo W co theo $1/a$ khi a tăng - "tự-ổn-định" giảm độ nhạy với scale trọng số trôi

đi (opt 44: BN gián tiếp làm phẳng landscape theo hướng scale, giảm L hiệu dụng). Kết quả opt 38: 72.2% top-5 ImageNet với 14 lần ít bước hơn Inception gốc.

Layer Normalization (opt 39): chuẩn hoá theo unit của **một sample**; Riemannian metric $ds^2 \approx \frac{1}{2}\delta^\top F(\theta)\delta$ (Eq. 8, xấp xỉ KL bậc hai, mục 6.3) cho thấy tăng $\|w_i\|$ gấp 2 làm độ cong hiệu dụng giảm $\frac{1}{2}$ - "early-stopping ngầm" bằng kiến trúc, cùng tinh thần AdamW (mục 7.2) nhưng không qua optimizer.

Gradient clipping: norm clipping (opt 40): $\hat{g} \leftarrow (\text{threshold}/\|\hat{g}\|)\hat{g}$ nếu $\|\hat{g}\| \geq$ threshold, động lực ban đầu là gradient bùng nổ RNN. **Vì sao clipping tăng tốc** (opt 41): dưới (L_0, L_1)-smoothness (mục 2), clipped GD đạt cận **không phụ thuộc** hằng số Lipschitz toàn cục M (Theorem 3), trong khi GD thường phụ thuộc M trực tiếp - đúng tình huống $f(x) = x^3$ ở mục 2. Normalized GD tương đương clipped GD tới hằng số (Eq. 6): cầu nối giải thích vì sao optimizer chuẩn hoá theo toạ độ (Adam, Lion) cũng "ổn định tự nhiên" như clipping.

Mixed precision (opt 42): FP16 giá trị lớn nhất $65\,504$; để tránh mất update khi $|w|/|\Delta w| > 2.048 = 2^{11}$ (chỉ 10 bit mantissa), cần giữ **bản sao trọng số FP32** (master copy) và **loss scaling** ($L' = sL$) tránh gradient nhỏ underflow. Với ngân sách 8 GB (mục 15): giữ trạng thái optimizer (m, v) ở fp32 trong khi forward/backward ở bf16/fp16 là quyết định cần tính đúng precision từng tensor, không chỉ đếm số tensor.

Tổng kết: khởi tạo tốt giảm L ban đầu; BN/LN giảm κ hiệu dụng theo hướng scale trọng số suốt quá trình train; clipping chặn $L_{\text{cục bộ}}$ tăng vọt đột ngột; mixed precision là ràng buộc số học chồng lên trên tất cả. Bốn kỹ thuật **phối hợp** với optimizer chứ không thay thế nó.

---

10. Hyperparameter: grid, random, Bayesian, Hyperband, PBT và muP

Grid search: quét đều mọi tổ hợp trên một lưới giá trị. Chi phí tăng **theo lũy số**: k giá trị mỗi chiều, n chiều cho k^n tổ hợp - không khả thi khi $n \geq$ 3-4.

Random search (Bergstra & Bengio 2012, kiến thức nền chung, tiền đề cho opt 31-34): lấy mẫu ngẫu nhiên độc lập mỗi hyperparameter. Lập luận cốt lõi: chỉ vài hyperparameter thực sự quan trọng; grid "lãng phí" lần thử vào chiều không quan trọng, random luôn có xác suất khám phá mọi chiều. Tự tính, xác suất ít nhất một trong n lần thử rơi vào "vùng top 5%" là $1 - 0.95^n$: $n = 10 \to 0.4013$, $n = 20 \to 0.6415$, $n = 60 \to 0.9539$. Chỉ 60 lần thử độc lập đã có hơn 95% cơ hội chạm vùng tốt nhất 5%.

Bayesian Optimization (BO) (opt 32, opt 33): mô hình f bằng **Gaussian Process (GP)**, dùng acquisition function cân bằng khai thác/khám phá, ví dụ Expected Improvement $a_{EI}(x) = \sigma(x)\big(\gamma(x)\Phi(\gamma(x)) + \phi(\gamma(x))\big)$ với $\gamma(x) = (f(x_{\text{best}}) - \mu(x))/\sigma(x)$ (opt 32); biến thể nâng cao: Knowledge Gradient, Entropy Search (opt 33). Phù hợp khi mỗi lần đánh giá **rất đắt** (GP tốn $O(n^3)$ theo số điểm đã thử); kém phù hợp cho hàng nghìn thử nghiệm rẻ song song.

Hyperband (opt 31): bandit-based, phân phát ngân sách **không đều**, loại sớm cấu hình kém qua Successive-Halving, nhiều "bracket" để không phải đoán trước tỉ lệ đánh đổi tối ưu. $R = 81, \eta = 3$: $s_{\max} = 4$, tăng tốc tối đa $R/(s_{\max} + 1) \approx 16.2$ lần (tự tính, opt 31); nhanh hơn BO 5-30 lần trên CNN.

Population Based Training (PBT) (opt 34): kết hợp tối ưu trọng số **và** hyperparameter trong một lần chạy: quần thể worker train song song, định kỳ "exploit" (sao chép từ worker tốt hơn) và "explore" (nhiễu hyperparameter). Kết quả (opt 34): RL (DM Lab) 93% → 106% hiệu năng người ($+14.0\%$ tương đối, tự tính); BLEU validation $23.71 \to 24.23$.

muP và zero-shot hyperparameter transfer (opt 35, opt 36): dò trên **bản thu nhỏ**, **copy nguyên** sang model lớn qua tham số hoá muP (abc-parametrization). Quy tắc (Table 3, opt 35): output init $\Theta(1/\text{fan_in}^2)$, LR $\Theta(1/\text{fan_in})$; hidden init $\Theta(1/\text{fan_in})$, LR (Adam) $\Theta(1/\text{fan_in})$.

Kiểm chứng số, tự tính dựa trên quy tắc trên: đã dò $\eta = 0.001$ cho hidden layer trên proxy width 1024,

muP dự đoán η theo $1/\text{fan_in}$: width 2048 ($\times 2$) $\rightarrow 5.00 \times 10^{-4}$; width 4096 ($\times 4$) $\rightarrow 2.50 \times 10^{-4}$; width 8192 ($\times 8$) $\rightarrow 1.25 \times 10^{-4}$.

Caveat (opt 35, Table 1): muP chuyển giao tốt cho optimizer/init/multiplier và trục width/depth/batch/seq length, nhưng **không** chuyển giao tốt cho **weight decay và dropout** - cần dò riêng ở model đích. opt 36 mở rộng thêm trục optimizer (SGD/Adam/Adafactor khác quy tắc) và chỉ ra chênh lệch LR tối ưu giữa "fixed-step" và "compute-optimal" trung vị khoảng 0.46 đơn vị số mũ (tự tính).

Tuning playbook thực hành cho 1 GPU (tổng hợp opt 46, opt 47, gợi ý không phải quy tắc tuyệt đối):

1. Chọn optimizer **ổn định, có tài liệu rộng** làm baseline (AdamW mặc định, mục 7.2) trước khi thử họ khác.
2. **LR range test** (opt 19, opt 47): tăng η tuyến tính trên run ngắn, chọn $\eta_{\max}$ ngay trước điểm loss ngừng giảm.
3. Chọn batch size theo bộ nhớ cho phép trước, chỉnh η theo linear scaling rule (mục 4) hoặc gradient accumulation (trường hợp dự án, mục 15).
4. Momentum thử vài giá trị rời rạc $\{0.99, 0.97, 0.95, 0.9\}$ (opt 47) trước khi chuyển sang cyclical nếu dùng 1cycle.
5. Weight decay dò trên lưới log (opt 46, opt 47), nhớ quy tắc scale theo N (mục 7.2, opt 51) nếu đổi lượng dữ liệu.
6. Chỉ đổi **họ optimizer** (mục 7) sau khi loại trừ yếu tố 2-5, so sánh theo đúng checklist mục 11, 13 (cùng ngân sách tune, cùng số seed).
7. Nhiều thử nghiệm nhỏ trước train lớn: xem xét muP để tune rẻ trên proxy rồi transfer, nhưng dò riêng weight decay/dropout ở model đích.

---

11. Cạm bẫy khi so sánh và tune optimizer

So sánh không công bằng về ngân sách tune: opt 29 (Crowded Valley) và opt 28 đều cho thấy **thứ hạng** optimizer phụ thuộc mạnh vào ngân sách/giao thức tune, không có "optimizer tốt nhất" phổ quát. **AlgoPerf** (opt 30) chuẩn hoá benchmark này (cùng ngân sách, self-tuning track, target hiệu năng), đáng tin hơn bảng so sánh tự làm.

Adaptive không "luôn tốt hơn": opt 43 xây dựng bài toán phân loại tuyến tính $d > n$ cụ thể mà nghiệm SGD (min-norm, max-margin) tổng quát hoá tốt, còn AdaGrad hội tụ về hướng khác có lỗi **không giảm về 0** khi $N \rightarrow \infty$ (lỗi AdaGrad $\rightarrow 1 - p$, lỗi SGD $\rightarrow 0$, Appendix C). Phản ví dụ toán học chặt: **implicit bias** (hướng nghiệm tự nhiên khi có nhiều nghiệm cực tiểu, khác hẳn tốc độ hội tụ) là trục đánh giá độc lập với tốc độ hội tụ.

Sharp/flat minima và edge of stability: quan sát "cực tiểu phẳng tổng quát hoá tốt hơn" (opt 24) có lỗ hổng lý thuyết đã biết: độ "nhọn" có thể đổi tuỳ ý qua phép đổi biến không đổi hàm số (scale lại hai lớp liên tiếp, mục 1), nên không phải bất biến có ý nghĩa. SAM (opt 25) tránh vấn đề định nghĩa bằng **tối ưu trực tiếp** cho vùng lân cận loss thấp đều ($\min_\theta \max_{\|\epsilon\| \le \rho} L(\theta+\epsilon)$). **Edge of stability** (opt 26): huấn luyện mạng neural thực tế thường **không** ở sâu trong vùng ổn định cổ điển ($\eta \lambda_{\max}(H) \ll 2$), mà dao động quanh biên $\eta \lambda_{\max}(H) \approx 2$ suốt phần lớn quá trình train - loss vẫn giảm dù lý thuyết tuyến tính mục 3.2 dự đoán phân kỳ ở biên này; một trong những khoảng trống lớn nhất giữa lý thuyết GD cổ điển và thực hành, vẫn là chủ đề nghiên cứu mở.

Phương sai giữa các seed: một lần chạy không nói lên gì chắc chắn, đặc biệt khi chênh lệch chỉ vài phần trăm (sibling preface **rl content, mục 5.1** nêu cảnh báo tương tự cho RL).

Nhầm LR baseline chưa tune kỹ với "optimizer mới tốt hơn": so sánh optimizer mới (đã LR-range-test kỹ) với optimizer cũ ở LR mặc định "sách giáo khoa" là lỗi phổ biến nhất (liên hệ bước 2, 6 playbook mục 10).

Nhầm "ít bước hơn" với "nhanh hơn thật": optimizer cần ít bước hơn (Sophia, Shampoo, SOAP) không tự động nhanh hơn wall-clock nếu mỗi bước đắt hơn đáng kể (mục 6.3, 6.4, 7.3: Sophia dưới 5% overhead/bước nhưng Shampoo/SOAP tốn $O(m^3 + n^3)$ compute/bước).

Batch size vượt ngưỡng tới hạn: linear scaling rule (mục 4) chỉ đúng dưới **critical batch size** (opt 22); vượt ngưỡng này tăng η không còn lợi ích tương xứng, có thể làm xấu kết quả.

"Optimizer mismatch" khi fine-tune: model pretrain bằng optimizer A (thường AdamW), chuyển sang optimizer B (Muon) chỉ ở fine-tune có thể **không** lợi rõ rệt như dùng B từ đầu - rủi ro chính paper Muon tự nêu (opt 16), liên quan trực tiếp dự án mục 15.

---

12. Bảng thuật ngữ Anh-Việt

Thuật ngữ	Nghĩa ngắn	Bài/mục liên quan
Convex set / function	Tập lồi / hàm lồi	2
L-smooth	Gradient Lipschitz hằng số L; Hessian $\preceq LI$	2
Strongly convex (μ)	Hàm "cong" tối thiểu μ mọi hướng; Hessian $\succeq \mu I$	2
Condition number κ	L/μ; quyết định tốc độ hội tụ GD/Newton	2, 3, 6
Saddle point	Điểm yên: gradient 0, Hessian có cả eigenvalue dương/âm	1
Critical/stationary point	Điểm gradient bằng 0	1
Loss landscape	Hình ảnh hoá đồ thị hàm loss theo tham số	1
Gradient descent (GD)	$\theta \leftarrow \theta - \eta \nabla f(\theta)$	3
Stochastic GD (SGD)	GD dùng gradient ước lượng từ (mini-)batch	4
Noise scale / variance	Phương sai gradient ước lượng, giảm theo $1/B$	4
Critical batch size	Ngưỡng batch mà quá đó linear scaling không còn hiệu quả	4, opt 22
Robbins-Monro condition	$\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$: điều kiện hội tụ a.s.	4
Momentum (heavy-ball)	Tích lũy "vận tốc" theo gradient qua các bước	5
Nesterov / NAG	Gradient tại điểm "nhìn trước"; tốc độ $O(\sqrt{\kappa})$	5
Continuous-time ODE view	Giới hạn liên tục của đệ quy rời rạc khi $\eta \to 0$	5
Newton's method	$\theta \leftarrow \theta - H^{-1}\nabla f$; hội tụ $O(1)$ bước trên toàn phương	6
Quasi-Newton / BFGS	Xấp xỉ Hessian chỉ từ hiệu gradient	6
L-BFGS	BFGS giới hạn bộ nhớ, lưu m cặp (s, y) gần nhất	6
Gauss-Newton	Xấp xỉ PSD của Hessian cho bài toán dạng bình phương sai số	6
Fisher information F	$\mathbb{E}[\nabla \log p \nabla \log p^\top]$; xấp xỉ Hessian	6
Natural gradient	$\theta \leftarrow \theta - \eta F^{-1}\nabla L$; bất biến đổi biến	6
Preconditioning	Đổi biến/chuẩn trước khi lấy bước gradient	6
K-FAC	Xấp xỉ Fisher theo khối Kronecker mỗi lớp	6
AdaGrad	Tích luỹ $G = \sum g^2$, LR riêng từng toạ độ, giảm về 0	7
RMSProp / Adadelta	AdaGrad với EMA thay tích luỹ, không giảm về 0	7
Adam	EMA bậc 1 + bậc 2 + bias-correction	7
AdamW	Adam + weight decay tách khỏi phần adaptive	7
Weight decay vs L2	Chỉ tương đương với SGD, không tương đương với Adam	7
AMSGrad	$\hat{v} =$ max tích luỹ, sửa lỗi hội tụ của Adam	7

Thuật ngữ	Nghĩa ngắn	Bài/mục liên quan
RAdam	Hệ số chỉnh phương sai r_t, tắt adaptive ở bước đầu	7
LAMB	Adam + trust ratio theo lớp, cho batch cực lớn	7
Lion	Chỉ 1 trạng thái, update bằng sign của EMA	7
Adafactor	Factorize v theo hàng/cột, $O(n+m)$ bộ nhớ	7
Sophia	EMA Hessian-diag ước lượng mỗi k bước, clip update	6, 7
Shampoo	Preconditioner Kronecker $L^{-1/4} \cdot R^{-1/4}$	7
SOAP	Adam chạy trong eigenbasis của preconditioner Shampoo	7
Muon	Trực giao hoá momentum bằng Newton-Schulz	7
Schedule-Free	Ba dãy x, y, z thay EMA-momentum, không cần biết T	5, 7, 8
Warmup	Tăng dần η từ 0 ở các bước đầu	8
Cosine annealing	Lịch η giảm theo cos, có thể kèm warm restart	8
Warm restart (SGDR)	Quay lại $\eta_{\max}$ định kỳ, tăng dần độ dài chu kỳ	8
1cycle / super-convergence	Một chu kỳ LR lớn duy nhất + annihilation	8
WSD (Warmup-Stable-Decay)	3 pha: warmup / ổn định / decay ngắn khi cần dừng	8
LR range test	Tăng η tuyến tính, tìm $\eta_{\max}$ khả dụng	8, 10
Kaiming init	$\mathrm{Var}[w] = 2/n$, giữ phương sai qua lớp ReLU	9
Batch Normalization	Chuẩn hoá theo batch, bất biến scale trọng số	9
Layer Normalization	Chuẩn hoá theo các unit của một sample	9
Gradient clipping	Giới hạn chuẩn/giá trị gradient trước khi update	9
(L_0, L_1)-smoothness	Tổng quát hoá L-smooth, độ cong tăng theo độ dốc	2, 9
Mixed precision	Tính ở FP16/BF16, giữ master weight/optimizer state FP32	9
Loss scaling	Nhân loss với hệ số s trước backward FP16	9
Grid / random search	Quét lưới đều / lấy mẫu ngẫu nhiên hyperparameter	10
Bayesian Optimization	GP surrogate + acquisition function	10
Expected Improvement	Acquisition function phổ biến cho BO	10
Hyperband / SuccessiveHalving	Bandit-based, cấp ngân sách không đều, loại sớm	10
Population Based Training	Tiến hoá quần thể cả trọng số và hyperparameter	10
muP / zero-shot transfer	Tham số hoá để copy hyperparameter từ model nhỏ sang lớn	10
Sharp / flat minima	Độ cong quanh một cực tiểu; không bất biến đổi biến	1, 11
SAM	Tối ưu trực tiếp cho vùng lân cận loss thấp đều	11
Edge of stability	Train thực tế dao động quanh biên ổn định $\eta\lambda_{\max} \approx 2$	3, 11
Implicit bias	Hướng nghiệm tự nhiên khi có nhiều cực tiểu, khác tốc độ hội tụ	11
Optimizer mismatch	Đổi optimizer giữa pretrain và fine-tune có thể không lợi	11, 15

13. Cách đọc paper optimizer một cách phản biện

1. **Baseline có được tune công bằng không?** Optimizer mới thường được LR-range-test kỹ, còn baseline có khi chỉ dùng giá trị mặc định ”sách giáo khoa” (mục 10, 11). Tìm xem baseline có cùng ngân sách tune hay không.

2. **Cái gì thực sự bị đổi?** Một so sánh ”optimizer A tốt hơn B” nhiều khi thực chất đổi **cả** LR schedule, batch size, weight decay cùng lúc (confound); tìm bảng ablation tách riêng từng trục.
3. **Đo ở quy mô nào?** Nhiều kết quả tốt ở CNN nhỏ/CIFAR không chuyển được lên LLM pretraining (ví dụ Lion cần batch ≥ 64 mới có lợi rõ, opt 11; SOAP/Shampoo mọi bằng chứng đều ở batch lớn, opt 14/15).
4. **Ít bước hơn có phải nhanh hơn thật không?** Kiểm tra cả wall-clock **và** compute (FLOPs/TFLOPs), không chỉ số bước (mục 6.3, 11; ví dụ cụ thể Sophia opt 13 Table 1).
5. **Bộ nhớ được tính theo quy ước nào?** ”Trạng thái optimizer persistent” (chỉ $m, v, \ldots$) khác ”bộ nhớ làm việc mỗi bước” (gồm cả gradient tạm, buffer trung gian của QR/eigendecomposition); và bộ nhớ sau khi sharding (ZeRO) có thể che mất chi phí thật trên một GPU đơn (mục 7.4, ví dụ Muon dưới ZeRO-1, opt 16).
6. **Bao nhiêu seed, báo cáo độ biến thiên không?** Một con số không kèm khoảng tin cậy có thể chỉ là nhiễu seed (mục 11).
7. **Paper có báo cáo cả tốc độ VÀ chất lượng cuối, hay chỉ chọn một trục có lợi?** Ví dụ ”ít bước hơn” mà không nói wall-clock, hoặc ”cùng wall-clock” mà không nói đã cân bằng LR/schedule cho mọi phương pháp thế nào.
8. **Có kiểm tra ”optimizer mismatch” không?** Nếu claim áp dụng cho fine-tune một model đã pretrain sẵn, optimizer mới có được thử **từ đầu** (pretrain luôn bằng nó) hay chỉ ở giai đoạn fine-tune (mục 11, opt 16 tự nêu rủi ro này).
9. **Benchmark chuẩn hoá có tồn tại cho câu hỏi này chưa?** AlgoPerf (opt 30) là điểm tham chiếu đáng tin hơn một bảng so sánh tự làm của riêng một paper (mục 11).

---

14. Lịch đọc gợi ý theo 51 bài

Danh sách dưới dựa trên tên file thật, kiểm bằng `ls` tại thời điểm viết (toàn bộ bài 01-51 đã có).

Giai đoạn 0 - Lý thuyết lồi cổ điển (nền cho mọi phần sau, cùng mục 2-3, 5-6).

- `01_2014_convex_optimization_bubeck.md`: định nghĩa chặt L-smooth/strongly convex/κ, GD/Nesterov/Newton với chứng minh đầy đủ, dùng xuyên suốt mục 2-3-5-6.
- `02_2016_optimization_methods_large_scale_ml.md` (Bottou-Curtis-Nocedal): lý thuyết hội tụ/độ phức tạp đầy đủ của SG (SGD) cho cả hàm lồi mạnh và phi lồi, nối trực tiếp mục 4.

Giai đoạn 1 - Tổng quan và nền tảng optimizer bậc một.

- `03_2016_overview_gradient_descent_ruder.md`: bản đồ mọi công thức momentum/NAG/AdaGrad/RMSProp/Adam, đọc trước khi đi sâu từng bài.
- `04_2015_nesterov_ode.md`: góc nhìn ODE liên tục, giải thích ”vì sao” cho bài 03.
- `05_2014_qualitative_optimization.md`: saddle point, loss landscape định tính (mục 1).

Giai đoạn 2 - Họ Adam và các biến thể sửa lỗi (lõi mục 7).

- `06` (Adam) → `07` (AdamW) → `08` (AMSGrad) → `09` (RAdam): đúng thứ tự lịch sử sửa lỗi (bias-correction → decoupled weight decay → sửa hội tụ → sửa phương sai đầu run).
- `10` (LAMB) → `11` (Lion) → `12` (Adafactor) → `13` (Sophia): bốn hướng mở rộng độc lập (batch lớn, bộ nhớ tối thiểu, bộ nhớ sublinear, bậc hai nhẹ).
- `14` (Shampoo) → `15` (SOAP) → `16` (Muon) → `17` (Schedule-Free): hướng ”phi đường chéo”/không-lịch-trình, phần nặng toán nhất của mục 7.

Giai đoạn 3 - Learning rate schedule và batch size (mục 8, 4).

- `18` (SGDR) → `19` (super-convergence) → `20` (MiniCPM/WSD): cosine/warm-restart → 1cycle → WSD, đúng thứ tự phát triển lịch học.
- `21` → `22` → `23` → `24`: linear scaling rule → critical batch size → tương đương LR-decay/batch-increase → cái giá generalization của batch lớn.

Giai đoạn 4 - Landscape, flat minima, benchmark optimizer (mục 1, 11, 13).

- `25` (SAM) → `26` (edge of stability) → `27` (visualizing loss landscape): ba cách ”nhìn” độ cong/độ phẳng quanh nghiệm.
- `28` → `29` (Crowded Valley) → `30` (AlgoPerf): trực tiếp cho mục 11, 13 (vì sao so sánh optimizer khó công bằng, benchmark chuẩn hoá hiện có).

Giai đoạn 5 - Hyperparameter search (mục 10).

- `31` (Hyperband) → `32`, `33` (Bayesian optimization) → `34` (PBT): bốn phương pháp tìm kiếm chính, từ ”rẻ, đơn giản” đến ”phức tạp hơn”.
- `35` (muP/mutransfer) → `36` (scaling exponents): nền cho ”tune nhỏ, chuyển lớn”.

Giai đoạn 6 - Init, normalization, clipping, mixed precision (mục 9).

- `37` (Kaiming init) → `38` (BatchNorm) → `39` (LayerNorm): ba kỹ thuật ảnh hưởng trực tiếp L, κ hiệu dụng.
- `40` → `41` → `42`: động lực thực nghiệm clipping → lý giải lý thuyết → ràng buộc số học khi train.

Giai đoạn 7 - Phân tích sâu hơn và các góc nhìn thống nhất.

- `43` → `44`: implicit bias của adaptive methods, và vì sao BN giúp tối ưu (không chỉ generalization).
- `45` (deconstructing LLM optimizer): ablation có kiểm soát ”cái gì trong Adam thực sự quan trọng cho LLM”, ví dụ áp dụng checklist mục 13.
- `46` → `47`: hai bài thực hành/playbook kinh điển, nền cho mục 10.
- `48` (old optimizer new norm) → `49` (tiny subspace): khung thống nhất preconditioning (mục 6.4), và vì sao gradient thật sống trong không gian con rất nhỏ (liên hệ mục 1).
- `50` → `51`: hai góc nhìn bổ sung ”weight decay thực sự làm gì” (mục 7.2), đọc cuối vì cần quen AdamW và muP (bài 07, 35) trước.

Cốt lõi nếu thời gian hạn chế: 01 → 03 → 06, 07 → 18 → 35 → 48 → 50 hoặc 51. Đường tắt đủ hiểu phần lớn quyết định optimizer/LR thực hành, dù không đủ cho các optimizer chuyên biệt (Shampoo/SOAP/Muon/Sophia) nếu cần dùng trực tiếp.

---

15. Liên hệ với dự án LightOnOCR-3-0.8B và các thư mục khác (suy luận)

Dự án: **LightOnOCR-3-0.8B**, VLM OCR khoảng 0.8 tỷ tham số; theo llm 43 (`43_2026_lightonocr.md`), phiên bản 3 dùng decoder lai **GatedDeltaNet + full attention**, khác phiên bản paper gốc mô tả (LightOnOCR-2-1B, Qwen3 attention chuẩn). Công việc: fine-tune/QAT/distillation/RL trên **GPU $\leq$ 8 GB, batch 1**, gồm chọn optimizer/LR cho **recovery sau pruning**. Toàn bộ mục này là **suy luận**; khi một bài cụ thể trong `pruning content/` hoặc `distillation content/` nói khác, tin bài đó.

1. Ngân sách bộ nhớ optimizer-state là ràng buộc đầu tiên. Tự tính (theo opt 12, mục 8): coi gần hết 0.8 tỷ tham số nằm trong ma trận 2D, AdamW fp32 cần $2 \times 0.8 \times 10^9 \times 4$ byte $= 6.4$ GB chỉ cho m, v - gần hết ngân sách 8 GB, **trước khi** tính trọng số, activation (≈ 3.600 token/trang) và KV cache. Đây là lý do mục 7.4 xếp **Muon** (nửa bộ nhớ AdamW) và **Adafactor** ($O(m+n)$ thay $O(mn)$) là hai lựa chọn hấp dẫn nhất; **Schedule-Free AdamW** (cùng bộ nhớ AdamW) là lựa chọn rủi ro thấp nhất để thử trước.

2. Với LoRA/QAT thay vì full fine-tune, phép tính trên đổi hẳn (suy luận, chưa kiểm chứng). Nếu recovery dùng LoRA (llm 08) thay vì cập nhật toàn bộ 0.8B tham số, số tham số trainable nhỏ hơn hàng chục đến hàng trăm lần, AdamW đầy đủ không còn là vấn đề bộ nhớ lớn, câu hỏi "optimizer nào" trở lại là tốc độ hội tụ (mục 3, 5, 6) hơn là bộ nhớ. Cần xác định rõ đang fine-tune bao nhiêu tham số thật trước khi áp dụng bảng mục 7.4.

3. Caveat batch-1 xuất hiện lặp lại trong chính các bài optimizer: Lion (opt 11) tự nêu "có thể không tốt hơn AdamW ở batch < 64"; mọi bằng chứng Shampoo/SOAP/Muon đều ở chế độ batch lớn; Schedule-Free tự nêu "chưa có bằng chứng ở batch size 1" như khoảng trống. Với batch vật lý 1 (tăng batch hiệu dụng bằng gradient accumulation, mục 10 bước 3), suy luận: **Schedule-Free AdamW** và **Adafactor** rủi ro thấp nhất để thử trước; **Lion**/**Muon** có tiềm năng tốt hơn nhưng cần tự đo ở đúng batch 1; **Shampoo/SOAP** đầy đủ nhiều khả năng không khả thi, chỉ có giá trị nếu áp dụng chọn lọc cho vài lớp quan trọng (ý tưởng đã nêu trong opt 14).

4. LR và weight decay cho recovery sau pruning (suy luận, tổng hợp hai bài khác thư mục). `pruning content/00` (mục 5) khuyến nghị fine-tune sau cắt tỉa với LR nhỏ hơn pretrain 1/10-1/100 (pruning 02, 04), nhấn mạnh "ngân sách phục hồi quyết định kết quả không kém tiêu chí cắt" (mục 11). Kết hợp quy tắc $\lambda \propto 1/N$ của opt 51 (mục 7.2): recovery dùng ít token hơn pretrain gốc (N nhỏ hơn), nên nếu copy thẳng λ gốc, quy tắc này gợi ý nên **tăng** λ cho recovery để giữ cùng "cảm giác" điều chuẩn theo thời gian. Đây là suy luận kết hợp hai kết quả riêng biệt, **chưa** bài gốc nào phát biểu trực tiếp; nên coi là giả thuyết cần test (2-3 giá trị λ quanh mức gợi ý), không phải kết luận chắc chắn.

5. RL (GRPO/RLVR) và lịch học không biết trước tổng số bước (suy luận). llm 43: LightOnOCR dùng RLVR (GRPO) ở giai đoạn cuối; `rl content/00` (mục 4) mô tả chi tiết. RL fine-tune thường "thử rồi dừng khi đủ tốt" hơn là horizon T cố định, đúng tình huống Schedule-Free (mục 7.3, 8) được thiết kế xử lý tốt (không cần biết T trước). Lý do bổ sung, ngoài bộ nhớ ở điểm 1, để ưu tiên Schedule-Free cho giai đoạn RL.

6. Natural gradient/Fisher information ở cả "optimizer" và "RL" (suy luận, liên hệ liên ngành). Mục 6.3 định nghĩa natural gradient dùng ma trận Fisher F; `rl content/00` (mục 3.11) mô tả TRPO (rl 04) ràng buộc KL-divergence, xấp xỉ bậc hai $\approx \frac{1}{2}\delta^\top F\delta$ - **cùng một đối tượng toán học** xuất hiện ở "bậc hai cho supervised learning" (mục 6) và "trust region cho RL" (rl 04, 07). Không cần cài đặt natural gradient tường minh, nhưng nhận ra sự trùng khớp giúp đọc cả hai mảng với cùng từ vựng.

7. Thư mục toán/ML nền tảng liên quan: `math content/01` (đạo hàm ma trận, cần cho mục 3-7), `math content/06`, `07` (sâu hơn cho mục 6.3), `ml basics content/34` (k-fold cho validation hyperparameter nhỏ, mục 10).

---

16. Năm câu tự kiểm tra

Câu 1. Cho $f(x) = \frac{1}{2}x^\top Ax$ với $A = \text{diag}(100, 1)$. Tính κ, bước học tối ưu η^*, và hệ số co ρ. Ước lượng (bậc độ lớn) số bước GD cần để giảm $\|x_t\|$ xuống 10^{-6} lần $\|x_0\|$.

Đáp án.

$\kappa = L/\mu = 100$. $\eta^* = 2/(L+\mu) = 2/101 \approx 0.0198$. $\rho = (\kappa-1)/(\kappa+1) = 99/101 \approx 0.9802$. Số bước $\approx \ln(10^{-6})/\ln(0.9802) \approx 691$ (khớp đúng bảng tự tính ở mục 3.2). Bậc độ lớn: tuyến tính theo κ, tức khoảng 10 lần nhiều hơn so với trường hợp $\kappa = 10$ (69 bước).

Câu 2. Vì sao Newton's method hội tụ trong đúng 1 bước trên hàm toàn phương bất kể κ, và vì sao điều này không áp dụng trực tiếp được cho một model 0.8B tham số?

Đáp án.

Vì khai triển Taylor bậc hai của một hàm toàn phương chính là hàm đó, không có sai số xấp xỉ, nên bước Newton $\theta_{t+1} = \theta_t - H^{-1}\nabla f(\theta_t)$ nhảy thẳng tới nghiệm đúng ngay bước đầu. Không áp dụng được trực tiếp vì cần lưu và nghịch đảo $H \in \mathbb{R}^{d\times d}$: với $d \approx 0.8 \times 10^9$, H đầy đủ ở fp32 tốn khoảng 2.3×10^6 TiB (mục 6.1, tự tính) - không khả thi về bộ nhớ, và ngay cả L-BFGS (xấp xỉ rẻ hơn rất nhiều) với chỉ 2 cặp lịch sử cũng đã tốn gần 12 GiB (mục 6.2, tự tính), vượt ngân sách 8 GB của dự án.

Câu 3. Vì sao L2 regularization không tương đương weight decay khi dùng với Adam, trong khi chúng tương đương khi dùng với SGD?

Đáp án.

Với SGD, cộng $\frac{\lambda}{2}\|\theta\|^2$ vào loss (L2) tạo ra gradient thêm $\lambda\theta$, và trừ trực tiếp $\lambda\theta$ khỏi tham số (weight decay) là đúng một phép toán giống nhau (Proposition 1, opt 07). Với Adam, phần gradient thêm từ L2 ($\lambda\theta$) bị **chia cho** $\sqrt{\hat{v}_t}$ giống như phần gradient chính trước khi cập nhật, nên các toạ độ có $\hat{v}_t$ (lịch sử gradient bình phương) lớn bị giảm decay hiệu dụng một cách không chủ ý, khác hẳn ý nghĩa "co đều mọi tham số về 0" của weight decay nguyên bản. AdamW tách riêng số hạng $\lambda\theta_{t-1}$ ra khỏi phần chia $\sqrt{\hat{v}_t}$ để khôi phục đúng ý nghĩa weight decay (mục 7.2).

Câu 4. Một paper báo cáo optimizer mới X cần một nửa số bước của AdamW trên một model 7B. Liệt kê ít nhất 3 câu hỏi bạn sẽ đặt ra trước khi tin "X tốt hơn AdamW" (mục 11, 13).

Đáp án.

Ví dụ: (1) Chi phí mỗi bước của X so với AdamW thế nào (wall-clock/FLOPs), hay chỉ "ít bước hơn" mà mỗi bước đắt hơn nhiều (như Shampoo/SOAP, mục 6.4, 7.4)? (2) AdamW có được tune LR/schedule/weight decay kỹ ngang X, hay dùng giá trị mặc định? (3) Bộ nhớ của X được tính theo quy ước nào (trạng thái persistent hay cả buffer làm việc, có sharding/ZeRO che mất chi phí không, mục 7.4, 13)? (4) Kết quả có seed/khoảng tin cậy không? (5) X có được test ở cả hai chế độ pretrain-từ-đầu và fine-tune-từ-checkpoint-AdamW không (rủi ro optimizer mismatch, mục 11)?

Câu 5. Theo quy tắc muP cho lớp hidden với Adam ($\eta \propto 1/\text{fan_in}$), nếu đã dò $\eta = 2 \times 10^{-3}$ trên một proxy model có hidden width 512, muP dự đoán η nào cho target model width 4096?

Đáp án.

$\eta_{\text{target}} = \eta_{\text{proxy}} \times (512/4096) = 2 \times 10^{-3} \times 0.125 = 2.5 \times 10^{-4}$ (tự tính, theo đúng quy tắc $\Theta(1/\text{fan_in})$ ở Table 3 của opt 35, mục 10).

Kết luận ngắn

Sáu ý cần nhớ: (1) L-smooth, μ-strongly convex và $\kappa = L/\mu$ là bộ từ vựng gốc giải thích vì sao GD chậm ($O(\kappa)$ bước), Nesterov nhanh hơn ($O(\sqrt{\kappa})$ bước), và Newton không phụ thuộc κ (nhưng không khả thi ở quy mô tỷ tham số); (2) mọi optimizer từ AdaGrad đến Muon/SOAP có thể nhìn như "GD dưới một preconditioner/chuẩn khác" (P^{-1} hoặc chuẩn không-Euclid), khác nhau chủ yếu ở việc chọn P nào với ngân sách bộ nhớ/tính toán nào; (3) nhiễu SGD giảm theo $1/\sqrt{B}$ chứ không phải $1/B$, và hầu hết lịch học thực hành (cosine, WSD) không thoả điều kiện hội tụ hầu chắc chắn cổ điển (Robbins-Monro) mà dựa trên lý thuyết finite-horizon hoặc thực nghiệm; (4) chuẩn hoá/khởi tạo/clipping/mixed precision không thay optimizer nhưng thay đổi trực tiếp L, κ hiệu dụng mà optimizer phải xử lý; (5) so sánh optimizer công bằng là khó (ngân sách tune, quy mô, wall-clock vs số bước, optimizer mismatch) - luôn áp dụng checklist mục 13 trước khi tin một tuyên bố "optimizer X tốt hơn"; (6) với ràng buộc 8 GB/batch 1 của dự án, bộ nhớ optimizer-state thường là nút thắt đầu tiên cần tính trước khi hỏi "optimizer nào nhanh nhất". Từ đây mở `01_2014_convex_optimization_bubeck.md` rồi `02_2016_optimization_methods_large_scale_ml.md`.

Bài 01

Convex Optimization: Algorithms and Complexity (2014)

Tác giả: Sébastien Bubeck (Theory Group, Microsoft Research).
Venue / arXiv: arXiv:1405.4980 (v1 nộp 05/2014; bản đọc là v2, 16/11/2015); xuất bản tại Foundations and Trends in Machine Learning, vol. 8, no. 3-4, pp. 231-358, 2015.
Nguồn: https://arxiv.org/abs/1405.4980
Chủ đề: một cuốn sách chuyên khảo (monograph) trình bày hệ thống các định lý độ phức tạp (complexity) trung tâm của tối ưu lồi (convex optimization) cùng thuật toán tương ứng: mô hình black-box oracle, các phương pháp cutting-plane tối ưu trong không gian hữu hạn chiều, các phương pháp bậc một "không phụ thuộc chiều" (dimension-free) cho hàm Lipschitz/trơn (smooth)/lồi mạnh (strongly convex) kèm cận dưới (lower bound) khớp, mở rộng ra không gian non-Euclidean (mirror descent), các phương pháp "vượt ra ngoài black-box" (FISTA, saddle-point smoothing, interior point), và tối ưu dưới nhiễu ngẫu nhiên (stochastic, bao gồm SGD, mini-batch, SVRG).
Vị trí trong lộ trình: đây là bài nền tảng lý thuyết cho cả cụm opt 03-09 (Ruder, Su-Boyd-Candès, Goodfellow et al., Adam, AdamW, AMSGrad, RAdam): mọi khái niệm "smoothness", "strong convexity", "condition number", "rate $O(1/t^2)$ của Nesterov" dùng xuyên suốt các bài đó đều được định nghĩa và chứng minh chặt ở đây. Nên đọc bài này sớm (hoặc đối chiếu lại) khi đọc opt 03, opt 04, opt 09.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026). Đây là một monograph dài 127 trang gồm 6 chương; theo đúng quy định cho bài khảo sát/sách dài, phạm vi đọc được khai báo rõ ngay sau đây, không giả vờ đã đọc hết chi tiết chứng minh của mọi chương. **Đã đọc đầy đủ, chi tiết (toàn bộ định nghĩa, định lý, và cấu trúc chứng minh):** Chương 1 (Introduction, toàn bộ 1.1-1.6, bao gồm Table 1.1 tổng hợp kết quả), Chương 3 (Dimension-free convex optimization, toàn bộ 3.1-3.7: subgradient descent, GD cho hàm smooth, Frank-Wolfe, strong convexity, lower bounds, geometric descent, Nesterov's AGD), Chương 6 mục 6.1-6.3 (Non-smooth/smooth stochastic optimization, mini-batch SGD, SVRG). **Đọc ở mức tổng quan (đọc đoạn mở đầu, định nghĩa cốt lõi, và các định lý/tốc độ hội tụ chính qua Table 1.1, KHÔNG đọc chi tiết từng dòng chứng minh):** Chương 2 (center of gravity, ellipsoid, Vaidya's cutting plane, conjugate gradient, đều là phương pháp phụ thuộc chiều, không phải trọng tâm của chuỗi bài "optimizer cho deep learning"), Chương 4 (mirror maps, mirror descent, dual averaging, mirror prox, vector field view), Chương 5 (ISTA/FISTA, saddle-point smoothing, interior point methods), Chương 6 mục 6.4-6.7 (random coordinate descent, acceleration ngẫu nhiên cho saddle point, convex relaxation/randomized rounding, random walk methods). Các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Convex optimization (tối ưu lồi) nghiên cứu bài toán $\min_{x \in \mathcal{X}} f(x)$ khi f là hàm lồi và $\mathcal{X}$ là tập lồi. Tính lồi tạo ra một hiện tượng "cục bộ suy ra toàn cục" (local to global) đặc biệt quan trọng: cực tiểu địa phương luôn là cực tiểu toàn cục (Proposition 1.2 của paper), và gradient tại một điểm chứa thông tin toàn cục dưới dạng một chặn dưới tuyến tính cho toàn hàm (thông qua khái niệm subgradient). Chính tính chất này cho phép xây dựng lý thuyết độ phức tạp (complexity theory) hoàn chỉnh: với từng lớp hàm con (chỉ Lipschitz, Lipschitz và lồi mạnh, smooth, smooth và lồi mạnh...), paper chứng minh cả **chặn trên** (một thuật toán cụ thể đạt được tốc độ đó) và **chặn dưới** (không thuật toán bậc một nào làm tốt hơn, bằng một hàm phản ví dụ cụ thể) - gọi là "matching upper and lower bounds".

Mô hình chủ đạo của 4 chương đầu là **mô hình black-box (oracle)**: giả sử có tài nguyên tính toán không hạn chế, tập ràng buộc $\mathcal{X}$ đã biết, còn hàm mục tiêu f chỉ được truy cập qua oracle (bậc 0: trả giá trị $f(x)$; bậc 1: trả một subgradient $g \in \partial f(x)$). Câu hỏi trung tâm là: cần bao nhiêu lần gọi oracle để tìm một điểm ε-tối ưu? Mô hình này, phát triển từ Nemirovski và Yudin [1983], hấp dẫn vì hai lý do mà paper nêu rõ ở mục 1.4: (a) có thể xây thuật toán với độ phức tạp **không phụ thuộc số chiều** n trong không gian rất lớn chiều, và

(b) các thuật toán kiểu này thường **bền với nhiễu** trong oracle, điều cực kỳ quan trọng cho tối ưu ngẫu nhiên (stochastic) trong machine learning.

Tác giả, Sébastien Bubeck (lúc viết sách thuộc Theory Group, Microsoft Research), tự mô tả rõ phạm vi và các điểm **bị bỏ qua có chủ đích** ngay ở mục 1.6 (”Overview of the results and disclaimer”): sách tập trung vào 5 kết quả lớn (lần lượt là nội dung của Chương 2 đến Chương 6), và **không** bao gồm các lập luận đối ngẫu (duality), tối ưu không lồi (trong đó paper liệt kê nguyên văn ”neural networks optimization” như một ví dụ bị bỏ qua), các thuật toán thực hành quy mô lớn (heuristic, hệ phân tán, tính thích nghi với tham số chưa biết). Đây là một tuyên bố giới hạn rất rõ ràng, quan trọng cho Mục 6 và 8 của ghi chú này.

2. Ý tưởng chính

Ý tưởng tổ chức xuyên suốt cuốn sách là: **mỗi giả thiết cấu trúc bổ sung lên hàm mục tiêu (Lipschitz, smoothness, strong convexity, cấu trúc không gian non-Euclidean, cấu trúc composite, nhiễu ngẫu nhiên) sẽ cho phép cải thiện tốc độ hội tụ, và paper luôn đi kèm một chặn dưới chứng minh mức cải thiện đó là tối ưu**. Cụ thể 5 kết quả lớn (mục 1.6):

1. **Chương 2:** tồn tại các phương pháp cutting-plane (center of gravity, ellipsoid, Vaidya) với độ phức tạp oracle tối ưu trong không gian hữu hạn chiều (phụ thuộc n, nhưng chỉ $O(n \log(1/\varepsilon))$, tức rất nhanh theo ε).
2. **Chương 3:** một đặc trưng hoàn chỉnh (complete characterization) giữa độ phức tạp oracle bậc một và độ cong (curvature, nghĩa là smoothness β và strong convexity α) của hàm mục tiêu, không phụ thuộc số chiều n.
3. **Chương 4:** các phương pháp bậc một vượt ra ngoài không gian Euclidean (mirror descent và các biến thể), cho phép giảm sự phụ thuộc vào chiều từ $\sqrt{n}$ xuống $\sqrt{\log n}$ trong một số bài toán (ví dụ tối ưu trên simplex).
4. **Chương 5:** các phương pháp phi-black-box (interior point methods) có thể cải thiện **bậc hai** (quadratic improvement) số bước lặp so với phương pháp black-box tối ưu.
5. **Chương 6:** tính bền với nhiễu (noise robustness) của các phương pháp bậc một, cho phép suy ra lý thuyết của stochastic gradient descent, mini-batch, và các phương pháp giảm phương sai (variance reduction).

Một thông điệp quan trọng khác, được nêu ở cuối Chương 1 (Table 1.1) và xuyên suốt Chương 3: **”strong convexity là giả thiết đối ngẫu (dual) của smoothness”** (paper viết rõ ở mục 3.4: ”strong convexity is a dual assumption to smoothness”), và tỉ số $\kappa = \beta/\alpha$ (gọi là **condition number**, số điều kiện) chính là đại lượng quyết định tốc độ hội tụ tuyến tính (linear/exponential) của các phương pháp cho hàm smooth và lồi mạnh. Gia tốc Nesterov (Nesterov’s accelerated gradient descent, AGD) cải thiện sự phụ thuộc vào κ từ $O(\kappa)$ xuống $O(\sqrt{\kappa})$, một trong những kết quả nổi bật nhất của toàn cuốn sách.

3. Phương pháp chi tiết

3.1 Các định nghĩa nền tảng (Chương 1)

- **Tập lồi / hàm lồi (Định nghĩa 1.1):** X lồi nếu chứa mọi đoạn thẳng nối hai điểm của nó; f lồi nếu $f((1-\gamma)x + \gamma y) \le (1-\gamma)f(x) + \gamma f(y)$ với mọi $\gamma \in [0,1]$.
- **Subgradient (Định nghĩa 1.2):** $g \in \partial f(x)$ nếu $f(x) - f(y) \le g^\top (x - y)$ với mọi $y \in X$. Mọi hàm lồi có subgradient tại các điểm trong (Proposition 1.1); nếu f khả vi thì $\nabla f(x) \in \partial f(x)$.
- **Cực tiểu địa phương = cực tiểu toàn cục (Proposition 1.2):** đúng khi và chỉ khi $0 \in \partial f(x)$. Đây là hệ quả trực tiếp của tính lồi, và là lý do convex optimization ”dễ” hơn hẳn optimization phi lồi.
- **β-smooth:** ∇f là β-Lipschitz, tức $\|\nabla f(x) - \nabla f(y)\| \le \beta \|x - y\|$. Nếu f khả vi hai lần, tương đương các trị riêng của Hessian $\le \beta$.

- **α-strongly convex (Eq. 3.13):** $f(x) - f(y) \leq \nabla f(x)^\top (x-y) - \frac{\alpha}{2}\|x-y\|^2$. Tương đương $x \mapsto f(x) - \frac{\alpha}{2}\|x\|^2$ lồi; nếu khả vi hai lần, trị riêng Hessian $\geq \alpha$.
- **Condition number $\kappa = \beta/\alpha$:** luôn có $\beta \geq \alpha$, nên $\kappa \geq 1$.

3.2 Chương 3, phần Lipschitz và smooth (không strongly convex)

Projected subgradient descent (Eq. 3.2-3.3): $y_{t+1} = x_t - \eta g_t$, $x_{t+1} = \Pi_X(y_{t+1})$. Với f là L-Lipschitz, $\eta = R/(L\sqrt{t})$, Theorem 3.2 cho

$$f\left(\frac{1}{t}\sum_{s=1}^{t} x_s\right) - f(x^*) \leq \frac{RL}{\sqrt{t}}.$$

Đây là tốc độ $O(1/\sqrt{t})$, không phụ thuộc chiều n, nhưng paper chỉ ra ngay (mục 3.5) đây là **tối ưu**: không thuật toán bậc một nào làm tốt hơn trong trường hợp tổng quát này.

Khi f là β-smooth, gradient descent (không cần subgradient, dùng ∇f trực tiếp) với $\eta = 1/\beta$ đạt tốc độ nhanh hơn nhiều (Theorem 3.3, không gian không ràng buộc):

$$f(x_t) - f(x^*) \leq \frac{2\beta\|x_1 - x^*\|^2}{t-1}.$$

Tức $O(1/t)$ thay vì $O(1/\sqrt{t})$. Trong trường hợp có ràng buộc X (projected gradient descent), Theorem 3.7 cho một dạng tương tự $O(1/t)$ dùng đại lượng "gradient mapping" $g_X(x) = \beta(x - x^+)$ với $x^+ = \Pi_X(x - \frac{1}{\beta}\nabla f(x))$.

Frank-Wolfe / conditional gradient descent (Eq. 3.8-3.9, mục 3.3): thay bước chiếu (projection) bằng một bài toán tối ưu tuyến tính $y_t \in \arg\min_{y \in X} \nabla f(x_t)^\top y$, rồi $x_{t+1} = (1-\gamma_t)x_t + \gamma_t y_t$ với $\gamma_t = 2/(t+1)$. Theorem 3.8 cho $f(x_t) - f(x^*) \leq 2\beta R^2/(t+1)$, cùng bậc $O(1/t)$ như projected gradient descent, nhưng có hai ưu điểm quan trọng: (a) hoạt động với **mọi norm** (không chỉ Euclidean, vì không cần phép chiếu), và (b) tạo ra **iterate thưa (sparse)**: trên một polytope, x_t là tổ hợp lồi của tối đa t đỉnh (nhờ định lý Carathéodory). Paper minh hoạ ứng dụng này với bài toán LASSO cấu trúc thưa trên một từ điển (dictionary) rất lớn chiều N (thậm chí N cỡ mũ theo n), cho thấy Frank-Wolfe có thể giải trong thời gian đa thức theo n nếu có một oracle tối ưu tuyến tính nhanh trên từ điển.

3.3 Strong convexity (mục 3.4)

Với f vừa α-strongly convex vừa L-Lipschitz, projected subgradient descent với bước $\eta_s = 2/(\alpha(s+1))$ đạt (Theorem 3.9) tốc độ cải thiện từ $O(1/\sqrt{t})$ xuống $O(1/t)$: $f(\cdot) - f(x^*) = O(L^2/(\alpha t))$.

Khi có **cả** strong convexity và smoothness, gradient descent với bước cố định $\eta = 1/\beta$ đạt tốc độ **tuyến tính (hội tụ hàm mũ)** (Theorem 3.10):

$$\|x_{t+1} - x^*\|^2 \leq \exp\left(-\frac{t}{\kappa}\right)\|x_1 - x^*\|^2.$$

Với bước tối ưu hơn $\eta = 2/(\alpha+\beta)$ (Theorem 3.12, không gian không ràng buộc), hằng số được cải thiện nhẹ: $f(x_{t+1}) - f(x^*) \leq \frac{\beta}{2}\exp(-\frac{4t}{\kappa+1})\|x_1 - x^*\|^2$.

3.4 Lower bounds (mục 3.5)

Paper chứng minh các cận dưới khớp với mọi tốc độ nêu trên, theo khung "thủ tục black-box" (Eq. 3.15: $x_{t+1} \in \text{Span}(g_1, \ldots, g_t)$, $x_1 = 0$):

- **Theorem 3.13 (không smooth):** tồn tại hàm L-Lipschitz (lồi, hoặc lồi mạnh) sao cho mọi thủ tục black-box với $t \le n$ bước có $\min_{s\le t} f(x_s) - f(x^*) \ge \frac{RL}{2(1+\sqrt{t})}$ (trường hợp lồi thường) hoặc $\ge L^2/(8\alpha t)$ (trường hợp lồi mạnh). Hàm phản ví dụ cụ thể là $f(x) = \gamma \max_{i\le t} x(i) + \frac{\alpha}{2}\|x\|^2$, một hàm "max" buộc oracle chỉ tiết lộ một toạ độ mới mỗi lần gọi.
- **Theorem 3.14 (smooth, không strongly convex):** dùng ma trận ba đường chéo (tridiagonal) A_{2t+1}, hàm $f(x) = \frac{\beta}{8}x^\top A_{2t+1}x - \frac{\beta}{4}x^\top e_1$ cho $\min_{s\le t} f(x_s) - f(x^*) \ge \frac{3\beta\|x_1 - x^*\|^2}{32(t+1)^2}$, khớp đúng bậc $O(1/t^2)$ của Nesterov's AGD.
- **Theorem 3.15 (smooth và strongly convex, không gian ℓ_2 vô hạn chiều):** $f(x_t) - f(x^*) \ge \frac{\alpha}{2}\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^{2(t-1)}\|x_1 - x^*\|^2$, xấp xỉ $\exp(-4(t-1)/\sqrt{\kappa})$ khi κ lớn, khớp đúng bậc $O(\sqrt{\kappa})$ (không phải $O(\kappa)$) của Nesterov's AGD.

Những cận dưới này quan trọng: chúng chứng minh **không có khoảng trống lý thuyết** để cải thiện thêm các thuật toán bậc một trong các lớp hàm này (ngoại trừ cải thiện hằng số).

3.5 Geometric descent và Nesterov's AGD (mục 3.6-3.7)

Trước Nesterov's AGD, gradient descent cho hàm smooth + strongly convex chỉ đạt $O(\kappa \log(1/\varepsilon))$ bước, trong khi cận dưới (Theorem 3.15) chỉ ra $O(\sqrt{\kappa}\log(1/\varepsilon))$ là khả thi. Paper trình bày hai phương pháp đạt cận này: **geometric descent** (Bubeck et al. 2015, mục 3.6, dựa trên một lập luận hình học "giao hai hình cầu chứa nghiệm", lấy cảm hứng từ ellipsoid method) và **Nesterov's accelerated gradient descent** (mục 3.7, thuật toán gốc Nesterov 1983).

AGD cho hàm smooth + strongly convex (mục 3.7.1): khởi tạo $x_1 = y_1$, lặp

$$y_{t+1} = x_t - \frac{1}{\beta}\nabla f(x_t), \qquad x_{t+1} = \left(1 + \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right) y_{t+1} - \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} y_t.$$

Theorem 3.18: $f(y_t) - f(x^*) \le \frac{\alpha+\beta}{2}\|x_1 - x^*\|^2 \exp\left(-\frac{t-1}{\sqrt{\kappa}}\right)$. Chứng minh dùng một dãy hàm bậc hai phụ $\Phi_s(x)$ "xấp xỉ từ dưới" f ngày càng chặt (Eq. 3.17-3.18), một kỹ thuật ("estimate sequence") khá trừu tượng mà chính paper thừa nhận "không hoàn toàn trực giác" ngay ở đầu mục 3.6 (lý do paper giới thiệu thêm geometric descent, dễ hình dung hơn, như một thuật toán thay thế đạt cùng tốc độ).

AGD cho hàm chỉ smooth (không strongly convex, mục 3.7.2): dùng dãy hệ số $\lambda_0 = 0$, $\lambda_t = \frac{1+\sqrt{1+4\lambda_{t-1}^2}}{2}$, $\gamma_t = \frac{1-\lambda_t}{\lambda_{t+1}}$, lặp $y_{t+1} = x_t - \frac{1}{\beta}\nabla f(x_t)$, $x_{t+1} = (1-\gamma_t)y_{t+1} + \gamma_t y_t$. Theorem 3.19 (theo chứng minh của Beck và Teboulle 2009): $f(y_t) - f(x^*) \le \frac{2\beta\|x_1 - x^*\|^2}{t^2}$, tức đúng bậc $O(1/t^2)$ khớp cận dưới Theorem 3.14. Paper trích dẫn trực tiếp "Su et al. [2014]" (chính là bài opt 04 trong bộ ghi chú này) ngay ở đầu mục 3.7 như một cách diễn giải gần đây của phương pháp này dưới dạng phương trình vi phân.

3.6 Chương 2 (tóm tắt ở mức tổng quan): các phương pháp cutting-plane phụ thuộc chiều

Chương 2 trình bày các phương pháp hoạt động trên tập ràng buộc X qua một **separation oracle** (không cần cấu trúc Lipschitz/smooth của f), đạt tốc độ hội tụ hàm mũ $\exp(-t/n)$ hoặc tốt hơn nhưng với số bước phụ thuộc chiều n (ngược với Chương 3 là "dimension-free" nhưng tốc độ chỉ đa thức theo $1/\varepsilon$). Bốn phương pháp, theo Table 1.1 của paper: **center of gravity method** (cắt tập khả thi qua trọng tâm (centroid) của nó mỗi bước, về lý thuyết tối ưu nhưng tính trọng tâm là bài toán tích phân n-chiều rất đắt); **ellipsoid method** (thay trọng tâm bằng tâm của ellipsoid nhỏ nhất chứa tập khả thi, rẻ hơn (chỉ cần nhân ma trận-vector) nhưng tốc độ $\exp(-t/n^2)$ chậm hơn một bậc theo n); **Vaidya's cutting plane method** (dùng một "volumetric barrier", đạt lại tốc độ $\exp(-t/n)$ của center of gravity nhưng với chi phí mỗi bước chỉ là nhân ma trận-ma trận); **conjugate gradient (CG)** cho hàm bậc hai, hội tụ **chính xác** trong đúng n bước khi tính toán không có lỗi số (do n hướng liên hợp trực giao hoá không gian). Lưu ý: bảng Table 1.1 của paper có một số công thức mũ (ví dụ dòng CG, dòng ellipsoid) bị lỗi hiển thị dấu căn khi trích từ PDF (mất dấu $\sqrt{\cdot}$ ở một số số hạng), nên các

con số mũ chính xác cho CG trong trường hợp không chính xác tuyệt đối không được khẳng định lại ở đây; người đọc cần tham khảo trực tiếp mục 2.4 của paper nếu cần số liệu chính xác.

3.7 Chương 4 (tóm tắt): mirror descent, dual averaging, mirror prox

Ý tưởng cốt lõi (mục 4.1-4.2): khi norm đo khoảng cách không phải Euclidean (ví dụ ℓ_1), phép trừ "$x - \eta\nabla f(x)$" không còn ý nghĩa tự nhiên vì gradient sống trong không gian đối ngẫu (dual space). **Mirror map** Φ là một cầu nối: ánh xạ điểm x sang không gian đối ngẫu qua $\nabla\Phi(x)$, thực hiện bước gradient ở đó, rồi ánh xạ ngược lại và chiếu bằng Bregman divergence D_Φ. Với "ball setup" ($\Phi(x) = \frac{1}{2}\|x\|_2^2$), mirror descent suy biến đúng về projected subgradient descent. Với "simplex setup" (negative entropy $\Phi(x) = \sum_i x(i)\log x(i)$), mirror descent trở thành một dạng "exponentiated gradient" ($y_{t+1}(i) = x_t(i)\exp(-\eta\nabla f(x_t)(i))$) và cải thiện tốc độ từ $O(\sqrt{n/t})$ xuống $O(\sqrt{\log n/t})$ khi tối ưu trên simplex, Theorem 4.2. **Dual averaging** (hay "lazy mirror descent", mục 4.4, chính là phương pháp của Nesterov) thay vì đi lại giữa primal/dual ở mỗi bước, chỉ tích luỹ (average) gradient trong không gian đối ngẫu: $x_t = \arg\min_x \eta\sum_{s<t} g_s^\top x + \Phi(x)$. **Mirror prox** (mục 4.5, Nemirovski 2004) là biến thể hai bước (dự đoán rồi hiệu chỉnh) đạt tốc độ $O(1/t)$ cho hàm smooth, dùng nhiều ở Chương 5 cho bài toán saddle-point.

3.8 Chương 5 (tóm tắt): ISTA/FISTA, saddle-point smoothing, interior point

ISTA/FISTA (mục 5.1): cho bài toán $\min f(x) + g(x)$ với f smooth (có oracle) và g "đơn giản" (ví dụ $g(x) = \lambda\|x\|_1$), ISTA lặp $x_{t+1} = \arg\min_x g(x) + \frac{1}{2\eta}\|x - (x_t - \eta\nabla f(x_t))\|_2^2$ (một "proximal gradient step"), đạt cùng tốc độ $O(1/t)$ như gradient descent trên f một mình. **FISTA** kết hợp ý tưởng này với Nesterov's AGD, đạt $O(1/t^2)$: $f(y_t) + g(y_t) - (f(x^*) + g(x^*)) \le 2\beta\|x_1 - x^*\|^2/t^2$. Với $g(x) = \lambda\|x\|_1$, bước proximal có nghiệm đóng dạng toán tử "shrinkage" $\tau_\alpha(x) = (|x| - \alpha)_+\text{sign}(x)$.

Saddle-point smoothing (mục 5.2): nhiều hàm không smooth có dạng $f(x) = \max_{1\le i\le m} f_i(x)$ với f_i smooth (ví dụ hàm dùng để chứng minh lower bound ở Theorem 3.13); biểu diễn lại thành bài toán saddle-point $\min_x \max_y \varphi(x, y)$ rồi dùng mirror prox cho phép đạt $O(1/t)$ thay vì $O(1/\sqrt{t})$ của subgradient descent trực tiếp trên f.

Interior point methods (mục 5.3): giải bài toán tuyến tính $\min c^\top x$ s.t. $x \in X$ bằng cách đi theo "central path" $x^*(t) = \arg\min tc^\top x + F(x)$ với F là một "self-concordant barrier", dùng phương pháp Newton (hội tụ toàn phương, quadratic convergence, khi đủ gần nghiệm) để "nhảy" dọc theo đường trung tâm này. Kết quả: $O(\sqrt{\nu}\log(\nu/\varepsilon))$ bước Newton, trong đó ν là "độ phức tạp" (complexity value) của hàm chắn (barrier). Đây là cải thiện **bậc hai** (quadratic speed-up) về số bước so với các phương pháp black-box tối ưu của Chương 3, đổi lại mỗi bước đắt hơn (một bước Newton đầy đủ).

3.9 Chương 6, mục 6.1-6.3: tối ưu ngẫu nhiên (đọc đầy đủ)

Chương này mở đầu bằng một quan sát quan trọng (trích gần nguyên văn): một key insight có từ Robbins và Monro [1951] là các phương pháp bậc một khá bền (robust): gradient không cần tính chính xác để đảm bảo tiến triển, miễn là đúng **trung bình**. **Stochastic oracle** trả về $\tilde{g}(x)$ với $\mathbb{E}[\tilde{g}(x)] \in \partial f(x)$. Paper phân biệt rõ hai tình huống (rất quan trọng, thường bị nhập nhằng trong thực hành machine learning):

- **Expected risk** $f(x) = \mathbb{E}_\xi\,\ell(x, \xi)$: oracle lấy mẫu ξ mới mỗi lần, do đó với m mẫu i.i.d. có sẵn, về lý thuyết chỉ được phép **một lượt duyệt dữ liệu (one pass)** để các kỳ vọng có điều kiện còn đúng.
- **Empirical risk** $f(x) = \frac{1}{m}\sum_i f_i(x)$ (hữu hạn tổng, finite-sum): oracle chọn ngẫu nhiên chỉ số $I \in [m]$ và trả $\nabla f_I(x)$; ở đây được phép duyệt **nhiều epoch** vì f đã cố định, không còn vấn đề "dùng lại mẫu làm hỏng kỳ vọng".

S-MD / SGD không smooth (Theorem 6.1, 6.2): với oracle có $\mathbb{E}\|\tilde{g}(x)\|_*^2 \le B^2$, stochastic mirror descent đạt $\mathbb{E}f(\cdot) - \min f \le RB\sqrt{2/t}$; trường hợp Euclidean + strongly convex (chính là SGD cổ điển) với $\eta_s = 2/(\alpha(s+1))$ đạt $O(B^2/(\alpha t))$ (Theorem 6.2), cùng dạng với Theorem 3.9 (bản xác định/deterministic) nhưng L (Lipschitz) được thay bằng B (biên của phương sai gradient ngẫu nhiên).

Smooth stochastic và mini-batch SGD (mục 6.2, Theorem 6.3): đây là một điểm rất quan trọng mà paper nhấn mạnh: với oracle ngẫu nhiên tổng quát, **smoothness không mang lại gia tốc** (không thể đạt $O(1/t)$ hay $O(1/t^2)$ như trường hợp xác định, chỉ còn $O(1/\sqrt{t})$). Theorem 6.3 cho công thức nội suy giữa hai chế độ, với $\mathbb{E}\|\nabla f(x) - \tilde{g}(x)\|_*^2 \le \sigma^2$:

$$\mathbb{E} f\left(\frac{1}{t}\sum_{s} x_{s+1}\right) - f(x^*) \le R\sigma\sqrt{\frac{2}{t}} + \frac{\beta R^2}{t}.$$

Khi $\sigma = 0$ (oracle chính xác), số hạng đầu biến mất, trở về đúng tốc độ $O(1/t)$ của Theorem 3.7. **Mini-batch SGD** (trung bình m truy vấn độc lập tại cùng một điểm) giảm phương sai hiệu dụng xuống σ^2/m (hay B^2/m trong công thức không smooth, tính trực tiếp từ công thức phương sai của trung bình m biến độc lập cùng phân phối). Paper chỉ rõ hai lý do thực tế để dùng mini-batch: (i) tính toán có thể **phân tán** trên nhiều máy, và (ii) ngay cả tuần tự, một số phép tính có thể **dùng lại** khi tính nhiều gradient tại cùng một điểm.

Variance reduction, SVRG (mục 6.3, Theorem 6.5): cho bài toán empirical risk $f = \frac{1}{m}\sum_i f_i$ (α-strongly convex, f_i β-smooth), gradient descent thường cần $O(m\kappa \log(1/\varepsilon))$ lượt tính gradient (mỗi bước cần cả m gradient), SGD cần $O(1/(\alpha\varepsilon))$ (độc lập với độ chính xác theo kiểu log, nên chậm khi cần độ chính xác cao). **SVRG** (Stochastic Variance Reduced Gradient, Johnson và Zhang 2013, cùng họ với SAG của Le Roux et al. 2012 và SDCA của Shalev-Shwartz và Zhang 2013) dùng một điểm "căn" (centering point) y cập nhật thưa (mỗi epoch), thay $\nabla f_i(x)$ bằng $\nabla f_i(x) - \nabla f_i(y) + \nabla f(y)$ (không thiên lệch kỳ vọng, nhưng phương sai nhỏ hơn khi x, y gần nhau và gần nghiệm). Theorem 6.5 (với $\eta = 1/(10\beta)$, $k = 20\kappa$): $\mathbb{E} f(y^{(s+1)}) - f(x^*) \le 0.9^s(f(y^{(1)}) - f(x^*))$, tức hội tụ tuyến tính với chỉ $O((m+\kappa)\log(1/\varepsilon))$ lượt tính gradient tổng, tốt hơn cả gradient descent (loại bỏ hệ số nhân $m \times \kappa$ thành cộng $m + \kappa$).

3.10 Chương 6, mục 6.4-6.7 (tóm tắt ở mức tổng quan)

Random coordinate descent (6.4): cập nhật chỉ một toạ độ ngẫu nhiên mỗi bước; Theorem 6.6 cho thấy nó cần n lần nhiều bước hơn gradient descent để đạt cùng độ chính xác (có lý, vì mỗi bước chỉ "nhìn thấy" $1/n$ thông tin), nhưng mục 6.4.1 cho thấy nếu tận dụng được "directional smoothness" riêng từng toạ độ (β_i khác nhau), có thể lấy lại phần lớn hiệu quả đã mất. **Acceleration ngẫu nhiên cho saddle point (6.5):** áp dụng ý tưởng stochastic oracle cho bài toán minimax (ví dụ matrix games), đạt độ phức tạp **sublinear theo kích thước ma trận** $O(\|A\|_{\max}^2 (n+m)\log(n+m)/\varepsilon^2)$, nhỏ hơn hẳn $O(nm)$ phần tử của ma trận. **Convex relaxation và randomized rounding (6.6):** ví dụ kinh điển MAXCUT (NP-hard), lấy xấp xỉ $1/2$ bằng cách sample ngẫu nhiên đều trên hypercube. **Random walk methods (6.7):** thay việc tính chính xác trọng tâm (centroid, đắt) trong center of gravity method bằng việc lấy mẫu ngẫu nhiên các điểm trong tập khả thi (qua "hit-and-run walk") rồi lấy trung bình mẫu, giữ được tốc độ hội tụ gần như phương pháp gốc.

4. Thí nghiệm và kết quả chính

Đây là một **monograph lý thuyết thuần tuý**: không có bất kỳ thực nghiệm số/benchmark máy học nào (không dataset, không bảng kết quả thực nghiệm). "Kết quả chính" của paper là các **định lý độ phức tạp oracle** (upper bound đi kèm thuật toán, và lower bound đi kèm hàm phản ví dụ), được chính paper tổng hợp trong Table 1.1 (Chương 1). Bảng dưới đây trích lại đầy đủ Table 1.1 (giữ đúng ký hiệu của paper; R = bán kính chứa x_1, x^*; r = bán kính hình cầu trong tập khả thi; L = hằng số Lipschitz; β = hằng số smoothness; α = hằng số strong convexity; $\kappa = \beta/\alpha$; ν = độ phức tạp hàm chắn; B, σ = biên phương sai stochastic oracle; m = số hàm con trong finite-sum):

Lớp hàm f	Thuật toán	Tốc độ hội tụ	Số bước để đạt ε	Chi phí/bước
non-smooth	center of gravity	$\exp(-t/n)$	$n\log(1/\varepsilon)$	1 oracle bậc 1, giải tích phân n chiều
non-smooth	ellipsoid method	$\exp(-t/n^2)$	$n^2\log(R/(r\varepsilon))$	1 oracle bậc 1, nhân ma trận-vector
non-smooth	Vaidya	$\exp(-t/n)$	$n\log(R/(r\varepsilon))$	1 oracle bậc 1, nhân ma trận-ma trận
quadratic	CG	chính xác (arithmetic đúng)	n (chính xác); dạng mũ khi xấp xỉ (công thức OCR không rõ, xem mục 3.6)	1 oracle bậc 1
non-smooth, Lipschitz	PGD	$RL/\sqrt{t}$	R^2L^2/ε^2	1 gradient, 1 phép chiếu
smooth	PGD	$\beta R^2/t$	$\beta R^2/\varepsilon$	1 gradient, 1 phép chiếu
smooth	AGD	$\beta R^2/t^2$	$R\sqrt{\beta/\varepsilon}$	1 gradient
smooth (mọi norm)	Frank-Wolfe	$\beta R^2/t$	$\beta R^2/\varepsilon$	1 gradient, 1 bài toán LP
strongly convex, Lipschitz	PGD	$L^2/(\alpha t)$	$L^2/(\alpha\varepsilon)$	1 gradient, 1 phép chiếu
strongly convex, smooth	PGD	$R^2\exp(-t/\kappa)$	$\kappa\log(R^2/\varepsilon)$	1 gradient, 1 phép chiếu
strongly convex, smooth	AGD	$R^2\exp(-t/\sqrt{\kappa})$	$\sqrt{\kappa}\log(R^2/\varepsilon)$	1 gradient
$f+g$, f smooth, g đơn giản	FISTA	$\beta R^2/t^2$	$R\sqrt{\beta/\varepsilon}$	1 gradient của f, 1 proximal của g
saddle $\max_y \varphi(x,y)$, φ smooth	SP-MP	$\beta R^2/t$	$\beta R^2/\varepsilon$	MD trên X và trên Y
linear, X với barrier ν-self-concordant	IPM	$\nu\exp(-t/\sqrt{\nu})$	$\sqrt{\nu}\log(\nu/\varepsilon)$	1 bước Newton trên F
non-smooth, stochastic	SGD	$BL/\sqrt{t}$	B^2L^2/ε^2	1 stochastic gradient, 1 phép chiếu
non-smooth, strongly convex, stochastic	SGD	$B^2/(\alpha t)$	$B^2/(\alpha\varepsilon)$	1 stochastic gradient, 1 phép chiếu
$f=\frac{1}{m}\sum f_i$, f_i smooth, f strongly convex	SVRG	(không nêu rate dạng đóng)	$(m+\kappa)\log(1/\varepsilon)$	1 stochastic gradient

Vì đây là bảng do chính paper tổng hợp (không phải thực nghiệm), ”kết quả chính” thực chất là chuỗi 19 định lý (Theorem 1.1 đến Theorem 6.14 rải trong 6 chương) chứng minh các dòng trên, cùng các cận dưới khớp (Theorem 3.13, 3.14, 3.15) chứng minh không thể cải thiện thêm trong lớp hàm tương ứng (ngoại trừ hằng số). Không có bảng số liệu thực nghiệm nào để đối chiếu; paper hoàn toàn dựa trên chứng minh toán học.

5. Phân tích / ablation

”Ablation” tự nhiên của cuốn sách chính là cấu trúc toàn bộ Chương 3 và Table 1.1: mỗi hàng của bảng tương ứng với việc **thêm/bớt một giả thiết** (smoothness? strong convexity? norm nào? oracle chính xác hay ngẫu nhiên?) và quan sát hệ quả lên tốc độ:

- **Chỉ Lipschitz** $\to O(1/\sqrt{t})$ (PGD, SGD không smooth).
- **Thêm smoothness** $\to O(1/t)$ (PGD smooth), rồi **thêm gia tốc Nesterov** $\to O(1/t^2)$ (AGD).

- **Chỉ Lipschitz + strongly convex** $\rightarrow O(1/t)$ (so với $O(1/\sqrt{t})$ của chỉ Lipschitz, nhanh hơn một bậc căn).
- **Smooth + strongly convex** $\rightarrow$ hội tụ hàm mũ $O(\kappa \log(1/\varepsilon))$ (PGD), rồi **thêm gia tốc** $\rightarrow O(\sqrt{\kappa}\log(1/\varepsilon))$ (AGD/geometric descent), cải thiện căn bậc hai theo κ.
- **Oracle chính xác** $\rightarrow$ **oracle ngẫu nhiên (cùng giả thiết smooth):** mất hẳn khả năng vượt $O(1/\sqrt{t})$ (Theorem 6.3) trừ khi dùng cấu trúc finite-sum (SVRG).
- **Oracle đơn (1 mẫu)** $\rightarrow$ **mini-batch (**m **mẫu):** phương sai hiệu dụng giảm theo $1/m$ (mục 6.2), không đổi về bậc hội tụ theo t nhưng giảm hằng số.
- **Dùng cấu trúc finite-sum** $f = \frac{1}{m}\sum f_i$ **(SVRG) thay vì stochastic oracle tổng quát:** khôi phục lại hội tụ tuyến tính $O((m+\kappa)\log(1/\varepsilon))$, điều bất khả thi với oracle ngẫu nhiên tổng quát.
- **Euclidean** $\rightarrow$ **non-Euclidean (mirror descent trên simplex):** giảm phụ thuộc chiều từ $\sqrt{n}$ xuống $\sqrt{\log n}$ (Theorem 4.2 áp dụng cho negative-entropy mirror map).

Đây đồng thời là bản đồ "điều kiện nào mua được tốc độ nào" mà toàn bộ các bài opt 03-09 (về optimizer thực hành cho deep learning) ngầm giả định hoặc cố gắng mô phỏng lại dưới dạng heuristic, dù hầu hết không còn giữ được chứng minh chặt khi hàm mục tiêu không còn lồi.

6. Giới hạn

- **Paper tự công khai phạm vi bị loại trừ** (mục 1.6, trích gần nguyên văn): không bao gồm duality, và đặc biệt bỏ qua "(i) sử dụng convex optimization trong bối cảnh non-convex" trong đó liệt kê rõ ví dụ "neural networks optimization" như một trường hợp **không được covered**, và "(ii) các thuật toán thực hành quy mô lớn" (heuristic như quasi-Newton/BFGS, hệ phân tán, line search thay cho biết trước α, β). Đây là giới hạn quan trọng nhất đối với việc áp dụng trực tiếp cho dự án deep learning của người dùng (xem Mục 8).
- **Mọi thuật toán giả định biết trước các tham số cấu trúc** (L, β, α) để chỉnh bước học η. Paper thừa nhận đây là một đơn giản hoá (mục 1.6): trong thực hành, line search thay thế việc biết trước này, nhưng không được phân tích chi tiết trong sách; paper cũng chỉ ra rằng "chạy song song nhiều phiên bản với các giả định tham số khác nhau" là một cách giải quyết về lý thuyết, nhưng tự ghi chú (footnote 1, trang 242) rằng **mánh này không hoạt động trong bối cảnh của Chương 6** (stochastic optimization) - một hạn chế kỹ thuật được chính paper chỉ rõ, không phải điều người ghi chú suy luận ra.
- **Các cận dưới (lower bound) chỉ áp dụng cho "thủ tục black-box tuyến tính"** (Eq. 3.15: $x_{t+1} \in \text{Span}(g_1, \ldots, g_t)$), một lớp thủ tục xác định (deterministic) khá rộng nhưng không hoàn toàn tổng quát; paper không bàn chi tiết liệu các thủ tục ngẫu nhiên hoặc phi tuyến có thể vượt qua các cận này hay không (dù phần Chương 6 gợi ý câu trả lời là không, với các chứng minh riêng cho trường hợp ngẫu nhiên).
- **Bảng Table 1.1 có vài công thức bị lỗi hiển thị dấu căn khi chuyển từ PDF sang văn bản** (ví dụ hàng CG ở Chương 2, hàng ellipsoid): không phải lỗi của paper, mà là hạn chế của quy trình trích xuất văn bản (pdftotext) trong ghi chú này; các số liệu không chắc chắn đã được nêu rõ ở mục 3.6 và 4 thay vì khẳng định nhầm.
- **Chương 2 (cutting-plane methods) chỉ đọc ở mức tổng quan** trong ghi chú này (xem khai báo phạm vi ở đầu bài): các phát biểu về center of gravity/ellipsoid/Vaidya/CG ở mục 3.6 của ghi chú này dựa trên đoạn mở đầu chương, Table 1.1, và kiến thức phổ thông về các thuật toán cổ điển này "(ngoài paper, kiến thức chung)" cho phần diễn giải trực giác (ví dụ lý do ellipsoid chậm hơn center of gravity một bậc theo n), không phải chép lại chứng minh chi tiết của chính Bubeck.

7. Liên hệ với các bài khác

- **opt 04 (Su, Boyd, Candès, ODE cho Nesterov's AGD):** liên hệ được paper **xác nhận trực tiếp bằng trích dẫn**: ngay đầu mục 3.7 ("Nesterov's accelerated gradient descent"), Bubeck viết "We refer to Su et al. [2014] for a recent interpretation of the method in terms of differential equations", và "Su,

W., Boyd, S., and Candès, E." xuất hiện nguyên văn trong danh mục tài liệu tham khảo của sách. Hai bài bổ sung cho nhau: Bubeck chứng minh tốc độ $O(1/t^2)$/$O(\exp(-t/\sqrt{\kappa}))$ bằng kỹ thuật "estimate sequence" đại số (mục 3.7.1-3.7.2 ở trên), còn opt 04 giải thích CÙNG các công thức đó (hệ số động lượng $(k-1)/(k+2)$, hằng số $\sqrt{\kappa}$) qua một ODE liên tục-thời-gian, cho trực giác hình học/vật lý mà Bubeck tự nhận là thiếu ở phương pháp estimate-sequence gốc.

- **opt 03 (Ruder, overview gradient descent):** cùng trình bày họ thuật toán SGD/Momentum/NAG/Adagrad/Adam, nhưng ở hai mức độ hoàn toàn khác nhau: Ruder mô tả trực giác, không chứng minh, cho bối cảnh deep learning phi lồi (không có cận trên/cận dưới khớp); Bubeck chứng minh chặt (với cả upper và matching lower bound) nhưng chỉ cho hàm **lồi**. Ví dụ cụ thể: Ruder nêu NAG với hệ số $\gamma \approx 0{,}9$ cố định (mục 4.2 bài đó), còn Bubeck (mục 3.7.1 ở trên) cho công thức chính xác $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ (phụ thuộc κ, không phải một hằng số cố định) mà hệ số $0{,}9$ của thực hành deep learning chỉ là một xấp xỉ heuristic không có nguồn gốc lý thuyết trực tiếp từ công thức này.
- **opt 09 (RAdam) và opt 06/07/08 (Adam/AdamW/AMSGrad):** các optimizer này là thuật toán **thích nghi theo từng toạ độ** (coordinate-wise adaptive), về bản chất là một dạng heuristic hoá của ý tưởng "directional smoothness" mà Bubeck trình bày chặt cho random coordinate descent (mục 6.4.1 ở trên, với bước $1/\beta_i$ riêng từng toạ độ i): Adam ước lượng một "độ nhạy" v_t riêng từng toạ độ từ bình phương gradient quan sát được (thay vì biết trước β_i như Bubeck giả định), nhưng **không có** một định lý dạng Theorem 6.6/6.4.1 nào trong Bubeck chứng minh tốc độ cho trường hợp "β_i không biết trước, phải ước lượng online" trong bối cảnh phi lồi của Adam; đây là một khoảng trống lý thuyết thực sự (suy luận của người ghi chú, dựa trên việc không tìm thấy phát biểu tương ứng trong text đã đọc).
- **opt 07 (AdamW, decoupled weight decay):** cấu trúc $\min_x f(x) + g(x)$ với f smooth và g "đơn giản" (ISTA/FISTA, mục 5.1 ở trên, với ví dụ $g(x) = \lambda\|x\|_1$) là tiền đề toán học chính xác cho ý tưởng "decoupled" của AdamW: tách phần regularizer g ra khỏi gradient của f và xử lý riêng (ở ISTA/FISTA là một bước proximal đóng dạng; ở AdamW là trừ trực tiếp $\lambda\theta$ vào tham số sau bước Adam, không đi qua v_t). Đây là một quan sát của người ghi chú (suy luận): Bubeck's book không hề nhắc tới AdamW (sách viết 2014-2015, AdamW là 2017), nhưng cấu trúc toán học $f + g$ tách biệt là chung giữa hai bối cảnh.
- **pruning 09 (Linear Mode Connectivity):** liên hệ qua đối lập, đã kiểm tra tồn tại bằng `ls`: Proposition 1.2 của Bubeck ("cực tiểu địa phương của hàm lồi luôn là cực tiểu toàn cục") là chính xác lý do convex optimization "dễ"; pruning 09 (và opt 05 trong cùng bộ bài) nghiên cứu các hiện tượng (nhiều cực tiểu địa phương, "rào cản" (barrier) khi nội suy tuyến tính giữa hai nghiệm) chỉ có thể xảy ra vì mạng neural **không lồi**, đúng như Bubeck's disclaimer ở mục 1.6 đã cảnh báo trước.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là suy luận áp dụng cho việc post-training/nén **LightOnOCR-3-0.8B** (VLM OCR, kiến trúc hybrid GatedDeltaNet linear attention + full attention, GPU mục tiêu ≤ 8 GB, batch size $= 1$ khi inference). Cần nói rõ ngay: paper **tự khai báo không bao gồm** neural network optimization (mục 1.6), nên không có định lý nào trong sách áp dụng trực tiếp, không cần chứng minh, cho việc huấn luyện LightOnOCR (một bài toán tối ưu phi lồi). Tuy vậy sách là nền tảng lý thuyết của toàn bộ "ngôn ngữ" mà các optimizer thực hành (opt 06-09) dùng, nên có một số áp dụng gián tiếp, mức tin cậy từ trung bình đến thấp, cần gắn nhãn rõ:

1. **Lý giải mini-batch / gradient accumulation dưới ràng buộc batch size = 1 (suy luận, tin cậy trung bình):** Theorem 6.3 (mục 3.9 ở trên) cho thấy phương sai hiệu dụng của một bước cập nhật giảm theo $1/m$ khi trung bình m mẫu gradient độc lập **tại cùng một điểm tham số**, và chứng minh của theorem này không phân biệt việc m gradient được tính song song (vật lý trong một batch) hay tuần tự rồi cộng dồn (gradient accumulation, cách bắt buộc khi batch size vật lý chỉ là 1 do giới hạn 8GB). Suy luận trực tiếp: lý thuyết của Bubeck (dù chứng minh cho hàm lồi) **ủng hộ về mặt toán học** rằng gradient accumulation qua nhiều micro-batch kích thước 1 trước khi cập nhật tham số có hiệu quả giảm nhiễu tương đương một batch lớn thật, miễn các micro-batch độc lập (hoặc gần độc lập) với nhau, đây là lý

do lý thuyết cho một kỹ thuật kỹ thuật phổ biến khi fine-tune LightOnOCR dưới ràng buộc VRAM.

2. **Trực giác condition number cho việc chọn optimizer/lịch learning rate (suy luận, tin cậy thấp đến trung bình):** toàn bộ mạch lý luận Chương 3 (mục 3.3-3.5 ở trên) chỉ ra: hàm có $\kappa = \beta/\alpha$ lớn (ill-conditioned) hội tụ rất chậm với gradient descent thường ($O(\kappa)$ bước) nhưng nhanh hơn nhiều với gia tốc ($O(\sqrt{\kappa})$). Loss landscape của một mạng transformer/SSM hybrid như LightOnOCR gần như chắc chắn có độ cong rất không đồng đều giữa các hướng (một dạng "ill-conditioning" phi lồi), và đây chính là lý do trực giác (không có chứng minh tương đương cho phi lồi) khiến AdamW (với preconditioning theo toạ độ, một dạng rất thô của việc "chuẩn hoá" độ cong khác nhau theo từng hướng) và learning rate warmup/cosine decay (một dạng kiểm soát bước đi khi chưa biết "hiệu dụng β, α" cục bộ) đều hữu ích trong thực hành, dù sách này không hề nói gì về trường hợp phi lồi.
3. **SVRG/variance reduction ít khả thi cho fine-tune LightOnOCR dưới 8GB (suy luận, tin cậy trung bình, có cơ sở nêu trực tiếp từ paper):** mục 6.3 (Theorem 6.5) cho SVRG cần lưu một gradient "căn" $\nabla f(y)$ tính trên **toàn bộ** dữ liệu mỗi epoch, một chi phí bộ nhớ/tính toán không nhỏ; dưới ràng buộc GPU $\leq$ 8GB và batch $= 1$, chi phí thêm một lượt duyệt đầy đủ dữ liệu mỗi epoch để tính $\nabla f(y)$ là tốn kém không cân xứng so với lợi ích (vốn chỉ được chứng minh cho hàm lồi). Đây là lý do hợp lý giải thích vì sao SVRG/SAG/SDCA hầu như không được dùng trong huấn luyện deep learning thực tế, kể cả khi tập fine-tune của LightOnOCR là hữu hạn (finite-sum, đúng giả thiết của SVRG).
4. **Frank-Wolfe/mirror descent: ít liên quan trực tiếp** (suy luận có cơ sở, tin cậy trung bình): các kỹ thuật này nhằm giải quyết ràng buộc hình học đặc biệt (polytope, simplex, spectrahedron) hoặc norm phi-Euclidean, không khớp với bối cảnh huấn luyện mạng neural không có ràng buộc cứng lên tham số (chỉ có regularization "mềm" như weight decay). Nêu thẳng: đây là phần ít liên quan nhất của cuốn sách đối với dự án.

9. Thí nghiệm gợi ý

1. **Kiểm chứng tỉ lệ tốc độ κ so với $\sqrt{\kappa}$ (GD vs AGD) trên hàm bậc hai, CPU, không cần GPU.** Cài đặt gradient descent (Theorem 3.10) và Nesterov's AGD (mục 3.7.1) trên $f(x) = \frac{1}{2}x^\top Ax$ với A đường chéo có điều kiện số κ do người dùng chọn (ví dụ $\kappa = 10^4$). Đo số bước cần để đạt $\|x_t - x^*\| \leq 10^{-6}\|x_1 - x^*\|$ cho cả hai thuật toán; so với dự đoán lý thuyết (mục 10 dưới, bảng tự tính). Diễn giải: xác nhận trực tiếp định lý cốt lõi của cả cuốn sách, độc lập với deep learning.
2. **Kiểm chứng gradient accumulation = batch lớn thật, trên một mô hình nhỏ 1 GPU 8GB.** Huấn luyện một mạng nhỏ (hoặc LoRA nhỏ gắn vào LightOnOCR) với (a) batch vật lý $= B$ và (b) batch vật lý $= 1$ tích luỹ B bước trước khi cập nhật, cùng tổng số mẫu đã xem; đo đường cong loss theo số mẫu đã xem (không theo số lần cập nhật). Dự đoán theo mục 8.1: hai đường cong gần như trùng khớp (sai khác chỉ do nhiễu ngẫu nhiên của thứ tự lấy mẫu, không do cơ chế accumulation). Nếu hai đường cong khác biệt rõ rệt, đó là dấu hiệu có lỗi cài đặt (ví dụ quên chia gradient theo số bước accumulate) chứ không phải hiện tượng lý thuyết mới.
3. **So sánh SGD, mini-batch SGD, và SVRG trên hồi quy logistic (lồi, nhỏ, CPU), KHÔNG claim chuyển sang deep learning.** Tạo một tập dữ liệu tổng hợp tuyến tính-phân-tách-được-gần-đúng, $m = 1000$ mẫu, $n = 50$ chiều; so sánh ba thuật toán theo số lượt tính gradient cần để đạt độ chính xác cho trước. Dự đoán: SVRG hội tụ tuyến tính (đường thẳng trên thang log), SGD không (chậm dần theo $1/t$). Diễn giải: minh hoạ trực tiếp Theorem 6.2 vs Theorem 6.5 bằng số, một kiểm tra sạch, có chứng minh đi kèm.
4. **Thăm dò (exploratory, không có bảo đảm lý thuyết) liệu trực giác $\sqrt{\kappa}$ còn giữ được trên một mạng nhỏ phi lồi.** Trên một MLP 2-3 lớp nhỏ (có thể thử trên GPU 8GB nhanh, hoặc CPU), ước lượng thô "condition number cục bộ" quanh điểm khởi tạo (ví dụ qua tỉ số trị riêng lớn nhất/nhỏ nhất của Hessian xấp xỉ bằng power iteration trên một batch nhỏ), rồi so sánh tốc độ hội tụ của SGD thường vs SGD+Nesterov-momentum vs Adam. Đo xem thứ tự tốc độ có khớp định tính với dự đoán (Adam/Nesterov nhanh hơn SGD thường khi condition number ước lượng lớn) hay không. Diễn giải: đây CHỈ là kiểm tra định tính, phi lồi không có định lý tương ứng trong paper này, kết quả (nếu không

khớp) không phản bác gì về Bubeck's book (vì sách không tuyên bố áp dụng cho trường hợp này).

5. **Đo phương sai gradient theo kích thước micro-batch trên chính kiến trúc hybrid GatedDeltaNet của LightOnOCR (nếu khả thi trong 8GB).** Tính gradient trên nhiều micro-batch kích thước 1 khác nhau (cùng layer cuối, ví dụ chỉ qua LoRA adapter để nhẹ bộ nhớ), đo phương sai thực nghiệm của gradient theo số micro-batch tích luỹ $m \in \{1, 2, 4, 8, 16\}$. Dự đoán theo Theorem 6.3: phương sai giảm theo xấp xỉ $1/m$. Diễn giải: nếu quan sát khớp, cho biết tích luỹ bao nhiêu bước là "đủ" trước khi lợi ích giảm nhiễu không còn đáng kể so với chi phí thời gian, một quyết định thực dụng cho pipeline huấn luyện dưới 8GB.

10. Tóm tắt công thức / khái niệm

Khái niệm / công thức	Ý nghĩa
β-smooth: $\|\nabla f(x) - \nabla f(y)\| \le \beta\|x - y\|$	Gradient không đổi quá nhanh; cho phép bước dài hơn ($\eta = 1/\beta$)
α-strongly convex: $f(x) - f(y) \le \nabla f(x)^\top (x - y) - \frac{\alpha}{2}\|x - y\|^2$	Hàm "cong" tối thiểu α; đối ngẫu của smoothness
$\kappa = \beta/\alpha$ (condition number)	Quyết định tốc độ hội tụ tuyến tính: $O(\kappa)$ (GD) hay $O(\sqrt{\kappa})$ (AGD) bước
Theorem 3.3 (GD, smooth): $f(x_t) - f(x^*) \le 2\beta\|x_1 - x^*\|^2/(t-1)$	$O(1/t)$
Theorem 3.18 (AGD, smooth + strongly convex): $f(y_t) - f(x^*) \le \frac{\alpha+\beta}{2}\|x_1 - x^*\|^2 \exp(-(t-1)/\sqrt{\kappa})$	$O(\sqrt{\kappa}\log(1/\varepsilon))$ bước
Theorem 3.19 (AGD, chỉ smooth): $f(y_t) - f(x^*) \le 2\beta\|x_1 - x^*\|^2/t^2$	$O(1/t^2)$, khớp cận dưới Theorem 3.14
Nesterov AGD (smooth+str.conv.): $y_{t+1} = x_t - \frac{1}{\beta}\nabla f(x_t)$, $x_{t+1} = (1 + \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1})y_{t+1} - \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}y_t$	Công thức động lượng chính xác, phụ thuộc κ
Theorem 6.3 (mini-batch): $\mathbb{E}f(\cdot) - f(x^*) \le R\sigma\sqrt{2/t} + \beta R^2/t$	Phương sai σ^2 giảm $1/m$ khi trung bình m mẫu
Theorem 6.5 (SVRG): $\mathbb{E}f(y^{(s+1)}) - f(x^*) \le 0{,}9^s(\ldots)$	$O((m + \kappa)\log(1/\varepsilon))$, thay $m \times \kappa$ bằng $m + \kappa$
FISTA (Chương 5): $f(y_t) + g(y_t) - \ldots \le 2\beta\|x_1 - x^*\|^2/t^2$	Tách f (smooth) và g (đơn giản, ví dụ ℓ_1) vẫn giữ $O(1/t^2)$

(Tự tính) minh hoạ bằng số cho mục 9, thí nghiệm 1, dùng trực tiếp công thức "# Iter" của Table 1.1 (paper) cho dòng "strongly convex, smooth": PGD cần $\kappa\log(1/\varepsilon)$ bước, AGD cần $\sqrt{\kappa}\log(1/\varepsilon)$ bước. Với $\kappa = 10^4$ (một con số hợp lý cho hồi quy logistic L_2-regularized với $m = 10^4$ mẫu, như paper gợi ý ở mục 3.6) và $\varepsilon = 10^{-6}$ (tức $\log(1/\varepsilon) = \ln(10^6) \approx 13{,}816$):

Đại lượng	Giá trị (tự tính)
$\kappa\log(1/\varepsilon)$ (PGD, số bước)	$10^4 \times 13{,}816 \approx 138\,155$
$\sqrt{\kappa}\log(1/\varepsilon)$ (AGD, số bước)	$100 \times 13{,}816 \approx 1\,382$
Tỉ lệ số bước PGD / AGD	$\approx 100{,}0$, khớp đúng $\sqrt{\kappa} = \sqrt{10^4} = 100$

Phép tính xác nhận trực tiếp mức cải thiện định tính mà paper phát biểu ("$\sqrt{\kappa}$ lần nhanh hơn", mục 3.6): với $\kappa = 10^4$, Nesterov's AGD cần khoảng 100 lần ít bước hơn gradient descent thường để đạt cùng độ chính xác 10^{-6}.

11. Kết luận ngắn

Cuốn monograph của Bubeck xây dựng một lý thuyết độ phức tạp gần như hoàn chỉnh cho tối ưu lồi bậc một: với mỗi tổ hợp giả thiết (Lipschitz/smooth/strongly convex, Euclidean/non-Euclidean, chính xác/ngẫu nhiên), paper cho một thuật toán đạt tốc độ tối ưu và một cận dưới chứng minh không thể làm tốt hơn. Các kết quả trung tâm nhất cho chuỗi bài opt 03-09 là: định nghĩa chính xác của smoothness/strong convexity/condition number (Chương 1, 3), công thức và chứng minh đầy đủ của Nesterov's accelerated gradient descent cho cả hai trường hợp strongly convex và chỉ smooth (mục 3.7, trực tiếp trích dẫn opt 04 như một cách diễn giải bổ sung), và lý thuyết SGD/mini-batch/SVRG (Chương 6.1-6.3) làm nền cho mọi phân tích phương sai gradient sau này. Paper tự giới hạn phạm vi rất rõ ràng: không bao gồm tối ưu phi lồi (trong đó liệt kê rõ "neural networks optimization"), không bàn heuristic/line-search/hệ phân tán. Vì vậy giá trị đối với dự án LightOnOCR là **nền tảng khái niệm** (condition number, phương sai mini-batch, cấu trúc $f + g$) hơn là **định lý áp dụng trực tiếp**, và mọi liên hệ ở Mục 8 đều cần được hiểu là suy luận định tính, không có bảo đảm toán học tương đương trong bối cảnh phi lồi của mạng neural thật.

Nguồn gốc: https://arxiv.org/abs/1405.4980

Bài 02

Optimization Methods for Large-Scale Machine Learning (2016)

Tác giả: Léon Bottou (Facebook AI Research), Frank E. Curtis (Lehigh University), Jorge Nocedal (Northwestern University).
Venue / arXiv: arXiv:1606.04838 (v1 nộp 06/2016; bản đọc là v3, ghi ”February 12, 2018” ở đầu trang và ”8 Feb 2018” trên dòng arXiv). Paper không ghi rõ venue xuất bản trong phần văn bản đã đọc; được biết đến rộng rãi là xuất bản tại SIAM Review năm 2018 ”(ngoài paper, kiến thức chung, không xác nhận được trong văn bản arXiv đã tải)”.
Nguồn: https://arxiv.org/abs/1606.04838
Chủ đề: một khảo sát toàn diện về các phương pháp tối ưu số cho machine learning quy mô lớn: lý thuyết hội tụ/độ phức tạp đầy đủ của stochastic gradient (SG, tức SGD) cho cả hàm lồi mạnh và hàm phi lồi tổng quát (bao gồm deep neural network), cùng hai hướng cải tiến chính được tổ chức theo một sơ đồ 2 chiều: các phương pháp giảm nhiễu (noise reduction: dynamic sampling, gradient aggregation như SVRG/SAGA, iterate averaging) và các phương pháp bậc hai (second-order: Hessian-free Newton, quasi-Newton ngẫu nhiên, Gauss-Newton, natural gradient, các kỹ thuật scaling đường chéo như AdaGrad/RMSprop/batch normalization), cùng các phương pháp cho mô hình có regularization không trơn.
Vị trí trong lộ trình: đọc sau opt 01 (Bubeck, nền tảng lý thuyết tối ưu lồi thuần tuý) và cùng nhóm với opt 03 (Ruder). Đây là bài ”cầu nối” giữa lý thuyết tối ưu lồi (opt 01) và các optimizer thực hành cho deep learning phi lồi (opt 06-09): là khảo sát đầu tiên trong bộ (theo nội dung) trình bày AdaGrad/RMSprop/batch normalization trong cùng một khung lý thuyết SG duy nhất, kèm phân tích work-complexity lý giải vì sao SG thắng thế trong machine learning quy mô lớn.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026). Đây là một khảo sát dài khoảng 84 trang (không kể tài liệu tham khảo), gồm 9 chương và 2 phụ lục; theo đúng quy định cho bài khảo sát dài, phạm vi đọc được khai báo rõ ngay sau đây. **Đã đọc đầy đủ, chi tiết:** §1 (Introduction), Chương 2 (toàn bộ 2.1-2.3, case studies), Chương 3 (toàn bộ 3.1-3.4, overview + động cơ cho SG), Chương 4 (toàn bộ 4.1-4.5, lý thuyết hội tụ SG, đây là chương lý thuyết trung tâm của cả bài), Chương 7 (toàn bộ 7.1-7.3, momentum/accelerated/coordinate descent), Chương 9 (Summary and Perspectives, toàn bộ), Phụ lục A (Convexity and Analyses of SG, toàn bộ, ngắn). **Đọc ở mức tổng quan/chọn lọc công thức chính, KHÔNG đọc hết mọi so sánh biến thể và chứng minh phụ:** Chương 5 (Noise Reduction: 5.1 nắm nguyên lý; 5.2 Dynamic Sample Size đọc công thức (5.6)-(5.7) và phần ”Practical Implementation” ở mức tổng quan; 5.3 Gradient Aggregation đọc công thức SVRG (5.13) và nguyên lý SAGA, không đọc hết phần so sánh/commentary chi tiết; 5.4 Iterate Averaging đọc đầy đủ vì ngắn), Chương 6 (Second-Order Methods: 6.1-6.3 Hessian-free/quasi-Newton/Gauss-Newton chỉ đọc đoạn mở đầu và Hình 6.1 (sơ đồ phân loại), không đọc chi tiết thuật toán; 6.4 Natural Gradient đọc phần định nghĩa Fisher information/KL divergence, không đọc hết phần ứng dụng cuối mục; 6.5 Diagonal Scalings đọc đầy đủ vì liên quan trực tiếp AdaGrad/RMSprop/batch normalization), Chương 8 (Methods for Regularized Models: chỉ đọc đoạn mở đầu, công thức proximal gradient (8.1)-(8.3) và Theorem 8.1, không đọc 8.2 Proximal Newton/Orthant-Based chi tiết), Phụ lục B (Proofs: chỉ xem 2-3 chứng minh ngắn làm mẫu đối chiếu, không chép lại toàn bộ). Các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Paper mở đầu bằng ba câu hỏi định hướng toàn bộ nội dung: (1) các bài toán tối ưu trong machine learning nảy sinh như thế nào và điều gì làm chúng khó; (2) phương pháp tối ưu nào đã thành công nhất cho machine learning quy mô lớn và tại sao; (3) những tiến bộ gần đây nào đã được thực hiện và câu hỏi mở nào còn tồn tại. Để trả lời câu hỏi (1), paper dùng hai case study:

- **Phân loại văn bản (text classification, mục 2.1):** bài toán lồi, ví dụ hồi quy logistic có regularization $\frac{1}{n}\sum_i \ell(h(x_i;w,\tau),y_i) + \frac{\lambda}{2}\|w\|_2^2$ (Eq. 2.3), biểu diễn văn bản dạng "bag of words" thưa (ví dụ tập RCV1 có từ điển $d = 47\,152$ từ).
- **Nhiệm vụ nhận thức qua deep neural network (mục 2.2):** bài toán phi lồi, mạng nhiều lớp $x_i^{(j)} = s(W_j x_i^{(j-1)} + b_j)$ (Eq. 2.4), minh hoạ bằng các kết quả nổi tiếng từ các tài liệu khác: tỉ lệ lỗi nhận dạng giọng nói giảm mạnh nhờ DNN thay GMM (Hình 2.2, so sánh 3 nhóm nghiên cứu 2010-2012), kiến trúc CNN vô địch ImageNet 2012 với khoảng 60 triệu tham số (Hình 2.3), và lịch sử tỉ lệ lỗi top-5 của ImageNet qua các năm (Hình 2.4). Paper nhấn mạnh: đây là các kết quả **trích dẫn từ công trình khác**, không phải thực nghiệm mới của chính Bottou/Curtis/Nocedal.

Mục 2.3 hình thức hoá bài toán: minimize **expected risk** $R(w) = \mathbb{E}[\ell(h(x;w),y)]$ (Eq. 2.6) là mục tiêu thật, nhưng chỉ có thể tối ưu **empirical risk** $R_n(w) = \frac{1}{n}\sum_i \ell(h(x_i;w),y_i)$ (Eq. 2.1) vì không biết phân phối thật $P(x,y)$. Paper dẫn bất đẳng thức Hoeffding và cận VC-dimension (Eq. 2.7): $\sup_{h\in H}|R(h) - R_n(h)| \le O\left(\sqrt{\frac{1}{2n}\log\frac{2}{\eta}} + \sqrt{\frac{d_H}{n}\log\frac{n}{d_H}}\right)$, nền tảng cho nguyên lý **structural risk minimization** (chọn họ hàm $H_C = \{h : \Omega(h) \le C\}$ sao cho vừa đủ phức tạp, Hình 2.5) và kỹ thuật **early stopping** (dừng tối ưu R_n sớm cũng đóng vai trò tương tự một siêu tham số regularization, Hình 2.6).

2. Ý tưởng chính

Luận điểm trung tâm (nêu ngay trong Abstract): **machine learning quy mô lớn là một bối cảnh đặc thù, trong đó stochastic gradient (SG) đóng vai trò trung tâm, còn các kỹ thuật tối ưu phi tuyến cổ điển (batch, quasi-Newton đầy đủ...) thường "chật vật" (falter)**. Lý do không phải SG có tốc độ hội tụ nhanh hơn (ngược lại, SG hội tụ **chậm hơn** batch gradient về số bước), mà vì phân tích **work complexity** (tổng công tính toán, không chỉ số bước) cho một ngân sách thời gian cố định lại ủng hộ SG khi n (số mẫu) rất lớn.

Để tổ chức các phương pháp "vượt ra ngoài SG", paper đề xuất một **sơ đồ 2 chiều** (Hình 3.3, nhắc lại ở Hình 5.1 và 6.1): trục ngang là "mức độ giảm nhiễu ngẫu nhiên" (từ SG thuần tới batch gradient đầy đủ, đi qua mini-batch, dynamic sampling, gradient aggregation, iterate averaging), trục đứng là "mức độ xử lý ill-conditioning/phi tuyến bằng thông tin bậc hai" (từ SG thuần tới Newton đầy đủ, đi qua quasi-Newton, Gauss-Newton, natural gradient, diagonal scaling). Paper tự nhận sơ đồ này "không nên hiểu quá cứng nhắc" (mục 3.4) nhưng nó cho một bản đồ hữu ích để định vị gần như mọi optimizer thực hành (bao gồm cả AdaGrad, RMSprop, momentum, Adam-dòng sau này) trong một không gian chung.

3. Phương pháp chi tiết

3.1 Phát biểu bài toán (mục 3.1)

Ký hiệu đơn giản hoá: mẫu (hoặc tập mẫu) là một "seed" ngẫu nhiên ξ; hàm mất mát tổng hợp $f(w;\xi)$ (Eq. 3.3). Expected risk: $R(w) = \mathbb{E}[f(w;\xi)]$ (Eq. 3.4). Empirical risk: $R_n(w) = \frac{1}{n}\sum_i f_i(w)$ (Eq. 3.6).

3.2 SG vs batch, và ba động cơ cho SG (mục 3.2-3.3)

SG (Eq. 3.7): $w_{k+1} \leftarrow w_k - \alpha_k \nabla f_{i_k}(w_k)$, với i_k chọn ngẫu nhiên từ $\{1,\dots,n\}$. **Batch gradient (Eq. 3.8):** $w_{k+1} \leftarrow w_k - \alpha_k \nabla R_n(w_k)$.

Ba động cơ cho SG mà paper trình bày:

- **Trực giác:** nếu tập huấn luyện S chứa 10 bản sao của một tập con S_{sub}, batch gradient tốn gấp 10 lần công nhưng SG không hề chậm hơn (các xác suất chọn mẫu không đổi). Dữ liệu thật hiếm khi trùng lặp hoàn toàn nhưng thường dư thừa, nên dùng toàn bộ dữ liệu mỗi bước là lãng phí.
- **Thực nghiệm (Hình 3.1, kết quả gốc của chính paper):** so sánh batch L-BFGS và SG (bước cố định $\alpha = 4$) trên bài toán phân loại nhị phân, hồi quy logistic, tập RCV1. Trục hoành là số lượt truy cập dữ liệu (ADP, accessed data points), trục tung là empirical risk R_n, vẽ trong 10 epoch đầu. SG giảm nhanh

ngay từ đầu, vượt xa L-BFGS trong phạm vi 10 epoch này; paper ghi rõ đã phải dò nhiều giá trị α để tìm ra bước tốt cho ví dụ này. Ví dụ 3.1 (trích Bertsekas): mô phỏng bằng các hàm bậc hai với cực tiểu rải trên $[-1, 1]$, giải thích tại sao SG tiến nhanh lúc đầu (hướng chắc chắn là đúng khi còn xa nghiệm) rồi chậm hẳn khi gần nghiệm (”vùng nhầm lẫn”, confusion region).

- **Lý thuyết (xem trước, chứng minh đầy đủ ở mục 3.3-3.4 dưới):** batch gradient trên hàm lồi mạnh đạt $O(\rho^k)$ (Eq. 3.9, hội tụ tuyến tính) nhưng tốn $O(n)$ mỗi bước, tổng công $O(n \log(1/\varepsilon))$; SG đạt chỉ $O(1/k)$ (Eq. 3.10, chậm hơn) nhưng mỗi bước không phụ thuộc n, tổng công $O(1/\varepsilon)$ không phụ thuộc n - thắng thế khi n rất lớn.

3.3 Lý thuyết hội tụ SG cho hàm lồi mạnh (mục 4.1-4.2, đọc đầy đủ)

Thuật toán SG tổng quát hoá (Algorithm 4.1): $w_{k+1} = w_k - \alpha_k g(w_k, \xi_k)$, với $g(w_k, \xi_k)$ có thể là gradient một mẫu, trung bình mini-batch, hoặc một hướng kiểu Newton/quasi-Newton (Eq. 4.2) - cách trình bày này cho phép MỘT lý thuyết áp dụng cho rất nhiều biến thể.

Giả thiết 4.1 (smoothness): $\|\nabla F(w) - \nabla F(\bar{w})\| \le L\|w - \bar{w}\|$, kéo theo $F(w) \le F(\bar{w}) + \nabla F(\bar{w})^\top (w - \bar{w}) + \frac{1}{2}L\|w - \bar{w}\|^2$ (Eq. 4.3).

Giả thiết 4.3 (moment bậc 1 và 2): tồn tại $\mu_G \ge \mu > 0$ với $\nabla F(w_k)^\top \mathbb{E}_{\xi_k}[g(w_k, \xi_k)] \ge \mu \|\nabla F(w_k)\|^2$ (hướng $-g$ là hướng giảm ”đủ tốt” kỳ vọng), và tồn tại $M, M_V \ge 0$ với $\mathbb{V}_{\xi_k}[g(w_k, \xi_k)] \le M + M_V \|\nabla F(w_k)\|^2$ (phương sai bị chặn).

Lemma 4.2, 4.4: kết hợp hai giả thiết trên cho $\mathbb{E}_{\xi_k}[F(w_{k+1})] - F(w_k) \le -(\mu - \frac{1}{2}\alpha_k L M_G)\alpha_k \|\nabla F(w_k)\|^2 + \frac{1}{2}\alpha_k^2 LM$ (Eq. 4.10b), nền tảng của mọi định lý sau.

Theorem 4.6 (strongly convex, fixed stepsize): với $0 < \bar{\alpha} \le \mu/(LM_G)$,

$$\mathbb{E}[F(w_k) - F_*] \le \frac{\bar{\alpha} LM}{2c\mu} + (1 - \bar{\alpha} c\mu)^{k-1} \left(F(w_1) - F_* - \frac{\bar{\alpha} LM}{2c\mu} \right) \xrightarrow{k \to \infty} \frac{\bar{\alpha} LM}{2c\mu}.$$

Diễn giải quan trọng: hội tụ **tuyến tính tới một vùng lân cận** của nghiệm, không tới chính nghiệm, vì số hạng nhiễu $\frac{1}{2}\bar{\alpha}^2 LM$ không giảm theo k. Đây là lý do thực hành ”chạy SG với bước cố định, khi thấy chững lại thì giảm nửa bước” (Hình 4.1).

Theorem 4.7 (strongly convex, diminishing stepsize, kiểu Robbins-Monro): với $\alpha_k = \beta/(\gamma + k)$, $\beta > 1/(c\mu)$ (điều kiện (4.20)):

$$\mathbb{E}[F(w_k) - F_*] \le \frac{\nu}{\gamma + k}, \qquad \nu := \max\left\{ \frac{\beta^2 LM}{2(\beta c\mu - 1)}, (\gamma + 1)(F(w_1) - F_*) \right\}.$$

Đây chính xác dạng điều kiện bước $\sum \alpha_k = \infty, \sum \alpha_k^2 < \infty$ (Eq. 4.19) mà Robbins và Monro [1951] đề ra. Paper nhấn mạnh vai trò β: nếu β bị chọn quá nhỏ (do ước lượng sai hệ số lồi mạnh c), tốc độ hội tụ có thể cực kỳ chậm trong thực tế (paper trích một ví dụ từ tài liệu khác: sau 10^9 bước, khoảng cách tới nghiệm vẫn $> 10^{-2}$).

Trade-off mini-batch (cuối mục 4.2): so sánh SG đơn mẫu và mini-batch cỡ n_{mb}. Mini-batch tốn n_{mb} lần công mỗi bước nhưng giảm phương sai M, M_V đúng n_{mb} lần; áp Theorem 4.6 cho cả hai cho thấy **tổng công tính toán gần như bằng nhau** (”this amounts to effectively the same total computation as for the mini-batch SG method”, trích gần nguyên văn). Nhưng paper cảnh báo ngay sau: **không được tự động dùng bước học lớn hơn n_{mb} lần** cho mini-batch để ”đổi” công tính toán, vì cận trên của bước cho phép ($\bar{\alpha} \le \mu/(LM_G)$) không phụ thuộc n_{mb} theo cách cho phép điều đó; lợi ích thực tế của mini-batch nằm ở khả năng **song song hoá/tận dụng GPU**, không phải ở việc ”mua” bước học lớn hơn miễn phí.

3.4 Lý thuyết hội tụ SG cho hàm phi lồi tổng quát (mục 4.3, đọc đầy đủ, liên quan trực tiếp deep learning)

Không còn giả thiết lồi; chỉ dùng Giả thiết 4.1 và 4.3.

Theorem 4.8 (fixed stepsize): $\mathbb{E}\left[\frac{1}{K}\sum_{k=1}^{K}\|\nabla F(w_k)\|^2\right] \le \frac{\bar{\alpha}LM}{\mu} + \frac{2(F(w_1)-F_{\text{inf}})}{K\mu\bar{\alpha}} \xrightarrow{K\to\infty} \frac{\bar{\alpha}LM}{\mu}$ (Eq. 4.28). Nếu không nhiễu ($M=0$), đây là kết quả cổ điển: tổng bình phương gradient hữu hạn, nên $\|\nabla F(w_k)\| \to 0$. Có nhiễu thì chỉ hội tụ tới một vùng lân cận (tương tự trường hợp lồi mạnh).

Theorem 4.9, 4.10 (diminishing stepsize): với $\sum \alpha_k = \infty, \sum \alpha_k^2 < \infty$, $\liminf_k \mathbb{E}[\|\nabla F(w_k)\|^2] = 0$ (Eq. 4.29), và một kết quả mạnh hơn về trung bình có trọng số (Eq. 4.30). **Corollary 4.11:** chọn ngẫu nhiên một chỉ số $k(K)$ với xác suất tỉ lệ α_k, thì $\|\nabla F(w_{k(K)})\| \to 0$ theo xác suất. **Corollary 4.12:** với thêm điều kiện chính quy (Lipschitz của $\|\nabla F\|^2$), có $\lim_k \mathbb{E}[\|\nabla F(w_k)\|^2] = 0$ (hội tụ thực sự, không chỉ lim inf).

Đây là toàn bộ "bảo đảm lý thuyết" mà SG có thể cho khi áp dụng cho một mạng neural sâu: **không đảm bảo tới cực tiểu toàn cục** (vì hàm phi lồi), chỉ đảm bảo **gradient trung bình tiến về 0**, tức thuật toán cuối cùng dừng lại ở (hoặc gần) một điểm dừng (stationary point, có thể là cực tiểu địa phương, saddle point, hoặc điểm yên ngựa thoái hoá).

3.5 Work complexity cho học máy quy mô lớn (mục 4.4, đọc đầy đủ)

Phân rã lỗi tổng (Eq. 4.31): $\mathbb{E}[R(\tilde{w}_n)] = \underbrace{R(w_*)}_{\text{approx. } E_{app}} + \underbrace{\mathbb{E}[R(w_n) - R(w_*)]}_{\text{estimation } E_{est}} + \underbrace{\mathbb{E}[R(\tilde{w}_n) - R(w_n)]}_{\text{optimization } E_{opt}}$. Bài toán macroscopic: $\min_{n,\varepsilon} E(n,\varepsilon)$ sao cho $T(n,\varepsilon) \le T_{max}$ (Eq. 4.32), với xấp xỉ $E(n,\varepsilon) \sim 1/n + \varepsilon$ (Eq. 4.34, tốc độ nhanh nhất còn hợp lý với lý thuyết thống kê sơ cấp).

- **Batch gradient:** $T_{batch}(n,\varepsilon) \sim n\log(1/\varepsilon)$ (vì mỗi bước tốn n, cần $\log(1/\varepsilon)$ bước). Trong ngân sách T_{max}: $n \sim T_{max}/\log(1/\varepsilon)$, tối ưu hoá cho $\varepsilon \sim 1/T_{max}$, lỗi cuối $E_* \sim \log(T_{max})/T_{max} + 1/T_{max}$.
- **SG đơn giản:** $T_{stoch}(n,\varepsilon) \sim 1/\varepsilon$ (không phụ thuộc n!). Trong ngân sách T_{max}: nên dùng n càng lớn càng tốt, lỗi cuối $E_* \sim 1/T_{max}$.

Kết luận tổng hợp ở Table 4.1 (trình bày lại ở Mục 4 dưới): **SG có lỗi cuối tỉ lệ $1/T_{max}$, tốt hơn batch gradient (có thêm số hạng $\log(T_{max})/T_{max}$)**, dù batch có tốc độ hội tụ (theo số bước) nhanh hơn. Paper còn chỉ ra đây khớp một **cận dưới** cho mọi phương pháp chỉ truy cập hàm mục tiêu qua oracle nhiễu (theo mô hình Nemirovsky-Yudin): không thuật toán nào vượt quá $\Omega(1/k)$ độ chính xác sau k lần gọi oracle nhiễu khi tối ưu hàm lồi mạnh, và SG đạt đúng cận này (tới hằng số).

3.6 Commentary (mục 4.5, đọc đầy đủ): những gì lý thuyết trên chưa nói hết

- **"Fragility":** tốc độ của Theorem 4.7 cần $\beta > 1/(c\mu)$, đòi hỏi biết (ước lượng) hệ số lồi mạnh c - không hiển nhiên ngoài trường hợp có regularizer ℓ_2 tường minh.
- **Ill-conditioning:** hằng số trong Theorem 4.7 tăng theo cả condition number L/c và tỉ số M/c; cả hai có thể cải thiện lớn bằng cách **rescale thích nghi theo thông tin độ cong cục bộ** - đây chính là động cơ cho Chương 6 (second-order/diagonal scaling).
- **Cơ hội phân tán:** SG "khó phân tán hiệu quả" vì truy cập tham số dùng chung với tần suất cao, dù bền với nhiễu/đồng bộ lỏng.
- **Phương án hội tụ nhanh hơn:** cận dưới $\Omega(1/k)$ chỉ đúng khi mục tiêu là expected risk R (oracle nhiễu không kiểm soát được); khi tối ưu **empirical risk** R_n (finite-sum), một thuật toán "biết" khi nào một mẫu đã được dùng lại có thể vượt cận dưới đó bằng gradient aggregation (SVRG/SAGA/SAG, Chương 5) - động cơ trực tiếp cho hướng này.

3.7 Các phương pháp giảm nhiễu (Chương 5, tổng quan)

- **Dynamic sample size (mục 5.2, Eq. 5.6-5.7):** mini-batch SG với cỡ mẫu tăng dần hình học $n_k = \lceil \tau^{k-1} \rceil$ ($\tau > 1$); lý thuyết cho thấy đây là cách "mua lại" tốc độ tuyến tính (giống batch) mà vẫn giữ

phần nào tính rẻ của SG lúc đầu. Phần "Practical Implementation" đề xuất một tiêu chí thích nghi (Eq. 5.11-5.12: tăng n_k khi phương sai ước lượng ϕ_k còn quá lớn so với $\|g(w_k, \xi_k)\|^2$) thay cho lịch tăng cố định trước.

- **Gradient aggregation, SVRG (mục 5.3.1, Eq. 5.13):** hoạt động theo "chu kỳ" (epoch): tại điểm neo w_k, tính gradient đầy đủ $\nabla R_n(w_k)$ một lần; trong chu kỳ, lặp $\tilde{w}_{j+1} \leftarrow \tilde{w}_j - \alpha \tilde{g}_j$ với $\tilde{g}_j = \nabla f_{i_j}(\tilde{w}_j) - (\nabla f_{i_j}(w_k) - \nabla R_n(w_k))$ (không thiên lệch kỳ vọng, phương sai nhỏ khi $\tilde{w}_j$ gần w_k). **SAGA** (nêu ngắn) tương tự nhưng lưu một bảng gradient riêng cho mỗi mẫu, cập nhật dần, tránh phải tính lại gradient đầy đủ mỗi chu kỳ (đổi lại cần lưu trữ $O(n)$ vector gradient).
- **Iterate averaging (mục 5.4, Eq. 5.16):** chạy SG bình thường nhưng đồng thời giữ trung bình $\tilde{w}_{k+1} = \frac{1}{k+1}\sum_{j \le k+1} w_j$. Paper trích kết quả Polyak-Ruppert: nếu dùng bước giảm chậm hơn cổ điển ($O(1/k^a)$, $a \in (1/2, 1)$, thay vì $O(1/k)$) thì $\mathbb{E}\|w_k - w_*\|^2 = O(1/k^a)$ nhưng $\mathbb{E}\|\tilde{w}_k - w_*\|^2 = O(1/k)$ với **hằng số tối ưu** mà không một phép rescale (dùng ma trận xác định dương) nào cải thiện thêm được - nghĩa là averaging "miễn phí" loại bỏ tác động xấu của ill-conditioning, một hiệu ứng paper so sánh với dùng một phương pháp bậc hai lý tưởng.

3.8 Các phương pháp bậc hai (Chương 6, tổng quan, mục 6.5 đọc đầy đủ)

Động cơ (Eq. 6.1-6.3): gradient descent thường tương đương tối thiểu hoá một mô hình bậc hai $q_k(w) = F(w_k) + \nabla F(w_k)^\top (w - w_k) + \frac{1}{2}(w - w_k)^\top B^{-2}(w - w_k)$ với $B^{-2} = I$; Newton's method dùng $B^{-2} = \nabla^2 F(w_k)$. Hình 6.1 xếp Hessian-free Newton, quasi-Newton ngẫu nhiên, Gauss-Newton, natural gradient, diagonal scaling trên trục "bậc hai" của sơ đồ 2D.

Diagonal scalings (mục 6.5), bao gồm các công thức tiền thân trực tiếp của Adam/RMSprop/AdaGrad:

- **Scaling theo độ cong đường chéo đã tính (Becker-LeCun 1989):** $[G_k]_i = (1-\lambda)[G_{k-1}]_i + \lambda [J_h(w_k; \xi_k)^\top J_h(w_k; \xi_k)]_{ii}$, cập nhật $[w_{k+1}]_i = [w_k]_i - \frac{\alpha}{[G_k]_i + \mu}[g(w_k, \xi_k)]_i$. Paper trích nhận xét gốc: cải thiện tốc độ chỉ khoảng 3 lần so với SG, nên "nên coi là một cách định kỳ hiệu chỉnh lại SG hơn là một phương pháp bậc hai hoàn chỉnh".
- **RMSprop (trích dẫn [152]):** $[R_k]_i = (1-\lambda)[R_{k-1}]_i + \lambda [g(w_k, \xi_k)]_i^2$, cập nhật $[w_{k+1}]_i = [w_k]_i - \frac{\alpha}{\sqrt{[R_k]_i + \mu}}[g(w_k, \xi_k)]_i$. Paper ghi rõ: "cách tiếp cận đơn giản đến bất ngờ này đã được chứng minh rất hiệu quả cho việc tối ưu deep neural network", nhưng "giải thích lý thuyết cho hiệu quả này trên các bài toán phi lồi vẫn là đối tượng nghiên cứu tích cực" - một thừa nhận rõ ràng về khoảng trống lý thuyết.
- **AdaGrad [54]:** thay trung bình động bằng **tổng**: $[R_k]_i = [R_{k-1}]_i + [g(w_k, \xi_k)]_i^2$, tạo một dãy bước giảm dần $\alpha / \sqrt{[R_k]_i + \mu}$ riêng cho mỗi toạ độ. Ban đầu được phân tích cho hàm lồi (không cần lồi mạnh) với lý thuyết SG gợi ý bước $O(1/\sqrt{k})$; paper ghi nhận: AdaGrad hoạt động tốt trên deep learning nhưng bước giảm "quá hung hãn" (too aggressively) ở giai đoạn đầu.
- **Batch normalization [80] như một kỹ thuật "cấu trúc" liên quan diagonal scaling:** phân tích khối chéo của ma trận Gauss-Newton cho một lớp fully-connected (Eq. 6.25) cho thấy nó là một ma trận moment bậc hai có trọng số của input đã augment; nếu input được "trắng hoá" (whitened, trung bình 0 phương sai 1), khối này được conditioning tốt. Batch normalization thực hiện chuẩn hoá này bằng thống kê của chính mini-batch hiện tại, cải thiện conditioning nhưng (paper chỉ rõ) đưa vào một nguồn nhiễu phụ (vì chuẩn hoá phụ thuộc các mẫu khác trong mini-batch) có phương sai "được kiểm soát kém" (poorly controlled), dù hiệu quả thực nghiệm rất tốt.
- **Natural gradient (mục 6.4, đọc phần định nghĩa):** xây gradient descent trong không gian hàm dự đoán (không gian các phân phối h_w), dùng Kullback-Leibler divergence $D_{KL}(h_w \| h_{w+\delta w}) \approx \frac{1}{2}\delta w^\top G(w) \delta w$ (Eq. 6.19-6.20) với $G(w)$ là ma trận thông tin Fisher $G(w) = -\mathbb{E}_{h_w}[\partial^2 \log h_w / \partial w^2]$ (Eq. 6.21). Bất biến với mọi phép biến đổi khả nghịch (mạnh hơn Newton, chỉ bất biến với biến đổi tuyến tính).

3.9 Momentum, gia tốc, coordinate descent (Chương 7, đọc đầy đủ)

Heavy ball / momentum (Eq. 7.1): $w_{k+1} \leftarrow w_k - \alpha_k \nabla F(w_k) + \beta_k (w_k - w_{k-1})$. Với F bậc hai lồi mạnh (trị riêng c, L), steepest descent và heavy ball đạt tốc độ tuyến tính với hằng số co tương ứng (Eq. 7.2):

$$\frac{\kappa-1}{\kappa+1} \quad \text{(steepest descent)} \qquad \text{và} \qquad \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \quad \text{(heavy ball)}, \qquad \kappa := L/c.$$

Đây **chính xác cùng công thức** mà opt 01 (Bubeck, Theorem 3.10 và 3.18) chứng minh cho projected gradient descent và Nesterov's AGD - một sự khớp đẹp giữa hai bài, xác nhận tính nhất quán của lý thuyết. Khi (α_k, β_k) chọn tối ưu từng bước (Eq. 7.3), (7.1) trở thành chính xác **conjugate gradient (CG)** tuyến tính. Diễn giải thay thế của heavy ball: mỗi bước là trung bình suy giảm hàm mũ của các gradient quá khứ ($w_{k+1} \leftarrow w_k - \alpha \sum_{j\le k} \beta^{k-j} \nabla F(w_j)$) - các hướng dao động triệt tiêu nhau, hướng nhất quán được khuếch đại. Paper ghi rõ: "bảo đảm lý thuyết về việc thêm momentum trong bối cảnh ngẫu nhiên vẫn còn mơ hồ" (elusive), dù lợi ích thực tế đã được báo cáo nhiều.

Accelerated gradient (Nesterov, Eq. 7.4-7.5): $\tilde{w}_k \leftarrow w_k + \beta_k(w_k - w_{k-1})$, $w_{k+1} \leftarrow \tilde{w}_k - \alpha_k \nabla F(\tilde{w}_k)$ (khác heavy ball ở **thứ tự**: lấy "đà" trước, tính gradient tại điểm đã dịch, không phải tại w_k). Đạt $O(1/k^2)$ cho hàm lồi smooth (so với $O(1/k)$ của steepest descent), được chứng minh tối ưu trong lớp phương pháp bậc một (giống Chương 3 của opt 01). Paper thừa nhận thẳng: "không có giải thích trực giác nào được chấp nhận rộng rãi về cách Nesterov's method đạt được tốc độ tối ưu này" - đúng là câu hỏi mà opt 04 (Su, Boyd, Candès) cố gắng trả lời bằng một ODE, dù Bottou/Curtis/Nocedal (bài này) không trích dẫn opt 04. Trong bối cảnh ngẫu nhiên, gia tốc "chỉ có thể cải thiện hằng số, không cải thiện được bậc của tốc độ" (trích dẫn [1], cùng tài liệu tham khảo cho cận dưới ở mục 4.4).

Coordinate descent (Eq. 7.6, Theorem 7.1): $w_{k+1} \leftarrow w_k - \alpha_k \nabla_{i_k} F(w_k) e_{i_k}$. Với F lồi mạnh hệ số c, mỗi toạ độ Lipschitz hệ số L_i, $\hat{L} = \max_i L_i$, $\alpha_k = 1/\hat{L}$, i_k chọn ngẫu nhiên đều:

$$\mathbb{E}[F(w_{k+1})] - F_* \le \left(1 - \frac{c}{d\hat{L}}\right)^k (F(w_1) - F_*).$$

Chậm hơn gradient descent đầy đủ bởi hệ số d (số chiều), nhưng **cạnh tranh được** nếu d bước coordinate descent rẻ tương đương 1 bước gradient đầy đủ (đúng với các cấu trúc dạng $F(w) = \frac{1}{n}\sum_j \tilde{F}_j(x_j^\top w) + \sum_i \hat{F}_i(w_i)$, Eq. 7.9, như hồi quy logistic/Lasso với dữ liệu thưa).

3.10 Mô hình có regularization không trơn (Chương 8, đọc tổng quan)

Bài toán $\min_w \Phi(w) := F(w) + \lambda\Omega(w)$ (Eq. 8.1), đặc biệt $\Omega(w) = \|w\|_1$ (Eq. 8.2, LASSO khi F bình phương). **Proximal gradient (Eq. 8.3):** $w_{k+1} \leftarrow \arg\min_w F(w_k) + \nabla F(w_k)^\top (w - w_k) + \frac{1}{2\alpha_k}\|w - w_k\|^2 + \lambda\Omega(w)$, giống chính xác cấu trúc ISTA/FISTA của opt 01 (mục 3.8 bài đó). **Theorem 8.1:** với F lồi mạnh hệ số c, smooth hệ số L, $\alpha = \alpha \in (0, 1/L]$: $\Phi(w_{k+1}) - \Phi(w_*) \le (1 - \alpha c)^k (\Phi(w_1) - \Phi(w_*))$, hội tụ tuyến tính dù Ω không trơn - đúng kết luận mà ISTA của opt 01 cũng đạt được (hội tụ như gradient descent trên F một mình).

4. Thí nghiệm và kết quả chính

Cần phân biệt rõ hai loại "kết quả" trong bài, vì bản thân paper là một khảo sát (survey/commentary) chứ không phải một bài báo thực nghiệm mới:

1. **Kết quả minh hoạ trích dẫn từ công trình khác** (không phải thực nghiệm mới của Bottou/Curtis/Nocedal): Hình 2.2 (tỉ lệ lỗi nhận dạng giọng nói, DNN vs GMM, 3 nhóm nghiên cứu, 2010-2012), Hình 2.3 (kiến trúc CNN vô địch ILSVRC 2012, khoảng 60 triệu tham số), Hình 2.4 (lịch sử tỉ lệ lỗi top-5 ImageNet qua các năm, bước nhảy lớn năm 2012 nhờ CNN).
2. **Một kết quả minh hoạ số gốc của chính paper** (Hình 3.1, mục 3.3): so sánh batch L-BFGS và SG (bước cố định $\alpha = 4$) trên hồi quy logistic, tập RCV1, trong 10 epoch đầu, trục hoành là số lượt truy cập dữ liệu. SG giảm empirical risk nhanh hơn rõ rệt trong phạm vi này; paper tự ghi nhận đây cần dò

bước cẩn thận và chỉ đúng trong một số epoch đầu (nếu chạy lâu hơn, batch sẽ vượt qua, như paper nói rõ ở mục 3.3).

3. **Kết quả chính thực sự của bài là các định lý độ phức tạp** (không phải thực nghiệm): Theorem 4.6, 4.7 (hội tụ SG, hàm lồi mạnh), Theorem 4.8-4.10 (hội tụ SG, hàm phi lồi tổng quát), và quan trọng nhất là **Table 4.1** (kết quả work-complexity, mục 3.5 ở trên), nguyên văn:

	Batch	Stochastic
$T(n, \varepsilon)$	$\sim n \log(1/\varepsilon)$	$\sim 1/\varepsilon$
E_*	$\sim \log(T_{max})/T_{max} + 1/T_{max}$	$\sim 1/T_{max}$

Đây là bảng trung tâm của toàn bài: chứng minh bằng toán học (không phải thực nghiệm) lý do SG được ưa chuộng trong machine learning quy mô lớn, dưới một khung giả thiết cụ thể (Eq. 4.34, tốc độ thống kê nhanh nhất hợp lý). Paper tự thừa nhận ngay sau đó (mục 4.5, Commentary) rằng khung này "có thể khá thuyết phục nhưng sẽ là vội vàng khi kết luận SG là giải pháp hoàn hảo" - một cảnh báo rõ ràng về khoảng cách giữa lý thuyết tiệm cận và thực tế.

5. Phân tích / ablation

- **Bước cố định vs bước giảm dần (Theorem 4.6 vs 4.7):** bước cố định hội tụ nhanh (tuyến tính) nhưng chỉ tới một **vùng lân cận** nghiệm có kích thước tỉ lệ $\bar{\alpha}$; bước giảm dần hội tụ **tới chính nghiệm** nhưng với tốc độ chậm hơn ($O(1/k)$). Đây là một trade-off kinh điển, và paper đề xuất chiến lược lai (giảm nửa bước khi chững lại, Hình 4.1) như một "cầu nối" thực dụng giữa hai chế độ.
- **Nhiễu $M = 0$ vs $M > 0$ (trong cả Theorem 4.6 và 4.8):** không nhiễu, phục hồi đúng kết quả cổ điển của gradient descent đầy đủ (hội tụ hẳn, không chỉ tới vùng lân cận); có nhiễu, luôn có một "sàn" nhiễu không thể vượt qua chỉ bằng chỉnh bước (phải giảm nhiễu bằng cấu trúc khác, dẫn tới Chương 5).
- **Kích thước mini-batch $n_{mb} \ll n$ vs $n_{mb} \approx n$ (mục 4.2):** khi $n_{mb} \ll n$, về lý thuyết tổng công gần như không đổi (mục 3.3 ở trên); khi $n_{mb} \approx n$, mini-batch SG tiến gần hành vi batch gradient (trade-off phức tạp hơn, paper để ngỏ phân tích chi tiết).
- **Lồi mạnh vs phi lồi tổng quát (Theorem 4.6/4.7 vs 4.8-4.10):** cùng khung giả thiết Lipschitz-gradient + moment-bị-chặn, nhưng kết luận yếu hẳn đi khi mất tính lồi: từ "hội tụ giá trị hàm mục tiêu" (lồi mạnh) xuống chỉ còn "gradient trung bình tiến về 0" (phi lồi) - không có gì đảm bảo về giá trị hàm mục tiêu đạt được tốt tới đâu.
- **Trung bình đơn giản vs trung bình Polyak-Ruppert (mục 5.4):** trung bình với bước cổ điển $O(1/k)$ không cải thiện gì; nhưng kết hợp bước giảm chậm hơn ($O(1/k^a)$) với trung bình lại cho hằng số **tối ưu**, không thể cải thiện thêm bằng rescale ma trận - một "ablation" tinh tế cho thấy lợi ích không nằm ở bước giảm chậm hay ở việc trung bình riêng lẻ, mà ở **sự kết hợp** của cả hai.

6. Giới hạn

- **Paper tự công khai những gì KHÔNG thảo luận chi tiết** (mục 9, Summary and Perspectives, trích gần nguyên văn): "chúng tôi chưa thảo luận cũng chưa phân tích sâu các cơ hội từ tính toán song song và phân tán" (dù nhìn nhận SG "có thể không phải phương pháp phù hợp nhất cho các kiến trúc máy tính mới nổi"); và liệt kê rõ **ADMM** (alternating direction method of multipliers) và **EM** (expectation-maximization) là những phương pháp quan trọng **không được bao gồm** trong khảo sát.
- **Lý thuyết cho momentum trong bối cảnh ngẫu nhiên "vẫn còn mơ hồ" (elusive)** - trích trực tiếp từ mục 7.1: "Theoretical guarantees about the inclusion of momentum in stochastic settings are elusive". Đây là một khoảng trống được chính paper chỉ rõ, trực tiếp liên quan tới momentum/Adam dùng trong thực hành (opt 06-09).

- **RMSprop hoạt động tốt nhưng chưa có giải thích lý thuyết cho trường hợp phi lồi** - trích mục 6.5: ”giải thích lý thuyết của hiệu suất này trên các bài toán phi lồi vẫn là đối tượng nghiên cứu tích cực”. Một khoảng trống lý thuyết quan trọng khác, nằm ngay ở gốc rễ của Adam (opt 06) và các biến thể.
- **Dynamic sample size methods ”chưa được dùng rộng rãi trong machine learning”** (mục 5.2.1, trích gần nguyên văn): paper tự nhận kỹ thuật trình bày ”có thể chỉ là điểm khởi đầu cho các nghiên cứu sau này”, không phải một phương pháp đã chín.
- **Mọi lý thuyết hội tụ dựa trên Giả thiết 4.3 (chặn moment bậc 1/2)**, một giả thiết khá trừu tượng; paper cung cấp Phụ lục A như một cách phân tích thay thế (dựa convexity, không dựa smoothness), nhưng tự nhận cách thay thế này ”không áp dụng dễ dàng cho bối cảnh phi lồi hoặc khi dùng bước kiểu (quasi-)Newton” - mỗi khung phân tích có đánh đổi riêng, không có khung nào ”miễn phí”.
- **Phần work-complexity (mục 4.4) dựa trên một xấp xỉ mạnh** $E(n, \varepsilon) \sim 1/n + \varepsilon$ (Eq. 4.34) được paper tự gọi là ”tốc độ nhanh nhất còn hợp lý với lý thuyết thống kê sơ cấp” - đây là trường hợp tốt nhất cho batch method trong so sánh, nên kết luận ”SG thắng” càng mạnh hơn khi giả thiết thực tế kém thuận lợi hơn cho batch, nhưng người đọc cần hiểu đây là một so sánh **lý tưởng hoá** (cả hai phương pháp đều ”idealized” theo đúng chữ của Table 4.1’s caption), không phải đo trên cài đặt thực tế cụ thể.

7. Liên hệ với các bài khác

- **opt 01 (Bubeck):** cả hai bài trích dẫn Robbins và Monro [1951] như nguồn gốc của SG. Hằng số co $(\kappa-1)/(\kappa+1)$ của steepest descent và $(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$ của heavy ball/accelerated (mục 3.9 ở trên, Eq. 7.2 của bài này) khớp **chính xác** với Theorem 3.10 và 3.18 của Bubeck (opt 01) - hai nhóm tác giả độc lập đạt cùng công thức, một sự xác nhận chéo mạnh. Cấu trúc proximal gradient cho regularizer không trơn (mục 3.10 ở trên, Theorem 8.1) cũng khớp đúng ISTA/FISTA của opt 01 (mục 3.8 bài đó).
- **opt 03 (Ruder, overview gradient descent, cùng năm 2016):** cả hai khảo sát Momentum, NAG, Adagrad, RMSprop trong cùng giai đoạn, nhưng góc nhìn khác nhau: bài này (Bottou et al.) cho động cơ **Gauss-Newton/diagonal-curvature** cụ thể cho AdaGrad/RMSprop (mục 3.8 ở trên, Eq. 6.25 và các công thức R_k), còn Ruder (opt 03) trình bày các công thức tương tự chủ yếu từ góc nhìn ”lịch sử phát triển ý tưởng” không đi sâu vào liên hệ Gauss-Newton. Hai bài bổ sung lẫn nhau về góc nhìn cho cùng một nhóm thuật toán.
- **opt 06 (Adam):** công thức RMSprop ($[R_k]_i = (1-\lambda)[R_{k-1}]_i + \lambda g_i^2$, chia gradient cho $\sqrt{R_k + \mu}$) và AdaGrad ($[R_k]_i = [R_{k-1}]_i + g_i^2$, tổng tích luỹ) trích dẫn nguyên văn ở mục 3.8 của ghi chú này chính là hai thành phần Adam kết hợp lại (EMA bậc hai của RMSprop + bias correction + EMA bậc một kiểu momentum). Paper này (2016/2018) đã mô tả rõ hai tiền thân trước khi Adam (2014/2015) được đưa vào bối cảnh ”diagonal scaling” tổng quát.
- **opt 07 (AdamW):** ý tưởng ”decoupled weight decay” của AdamW có cùng tổ tiên toán học với cấu trúc $F + \lambda\Omega$ tách biệt của Chương 8 bài này (mục 3.10 ở trên): xử lý phần regularization bằng một bước riêng (proximal) thay vì gộp vào gradient của F. Đây là quan sát của người ghi chú (suy luận): bài Bottou et al. không hề nhắc AdamW (ra đời sau), nhưng cấu trúc toán học tách biệt là chung.
- **rl 07 (PPO, đã kiểm tra tồn tại bằng `ls` trong thư mục `rl content/`):** PPO dùng Adam với ”annealed stepsize” (bước giảm dần theo tiến trình huấn luyện) - đúng kiểu stepsize schedule mà Theorem 4.7 (mục 3.3 ở trên) phân tích lý thuyết cho SG (dù PPO dùng Adam, không phải SG thuần, nên không áp dụng trực tiếp định lý, chỉ là cùng triết lý ”giảm bước theo thời gian để kiểm soát nhiễu”).

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là suy luận cho việc post-training/nén **LightOnOCR-3-0.8B** (VLM OCR, kiến trúc hybrid GatedDeltaNet linear attention + full attention, GPU mục tiêu ≤ 8 GB, batch size $= 1$ khi inference). Đây là bài có liên hệ **trực tiếp và cụ thể hơn** opt 01 vì bàn thẳng về công việc tính toán thực tế (work complexity) thay vì chỉ lý thuyết lồi trừu tượng:

1. **Gradient accumulation dưới ràng buộc batch=1 (suy luận, tin cậy cao, có trích dẫn trực tiếp từ**

paper): mục 3.3 ở trên (cuối mục 4.2 của paper) kết luận rõ: mini-batch cỡ n_{mb} và SG đơn mẫu lặp n_{mb} lần nhiều hơn với bước nhỏ hơn n_{mb} lần cho **"effectively the same total computation"**. Đây là cơ sở lý thuyết trực tiếp (không cần suy luận bắc cầu như ở opt 01) cho việc dùng gradient accumulation khi batch vật lý chỉ là 1 do giới hạn 8GB. Nhưng paper cũng cảnh báo rõ: **không thể tự động tăng learning rate theo** n_{mb} để "mua lại" tốc độ, vì cận trên cho phép của bước học không phụ thuộc n_{mb} theo cách đó - một lưu ý thực dụng quan trọng khi thiết kế lịch learning rate cho fine-tune LightOnOCR với gradient accumulation.

2. **Khung "ngân sách thời gian cố định" (mục 3.5/4 ở trên, Table 4.1) khớp đúng bối cảnh dự án:** post-training trên 1 GPU 8GB với thời gian hữu hạn là chính xác bối cảnh T_{max} cố định mà paper phân tích. Kết luận của paper ("nên dùng càng nhiều dữ liệu càng tốt, chấp nhận mỗi mẫu chỉ được xử lý sơ sài bởi một thuật toán nhiễu" cho SG) gợi ý ưu tiên **thông lượng (throughput, tokens/giây)** hơn **độ chính xác mỗi bước** khi fine-tune LightOnOCR dưới ràng buộc tài nguyên - ủng hộ việc dùng SG/AdamW đơn giản, giảm thiểu overhead mỗi bước (ví dụ tránh các bước bậc hai đắt đỏ như Hessian-free Newton) hơn là tìm một optimizer "chính xác hơn nhưng chậm hơn mỗi bước".
3. **Theorem 4.8-4.10 (SG cho hàm phi lồi) là nền lý thuyết chính thức, yếu nhưng có thật, cho việc huấn luyện LightOnOCR:** không có gì đảm bảo tới cực tiểu toàn cục, chỉ đảm bảo gradient trung bình tiến về 0 (điểm dừng). Suy luận thực dụng: theo dõi **chuẩn gradient trung bình** (không chỉ loss) qua quá trình fine-tune là một chỉ báo hợp lý (có cơ sở lý thuyết, dù chỉ cho trường hợp tổng quát) để quyết định khi nào huấn luyện "đã dừng tiến triển", bổ sung cho việc theo dõi loss/validation đơn thuần.
4. **Batch normalization không áp dụng trực tiếp (suy luận có cơ sở, tin cậy trung bình):** kiến trúc hybrid GatedDeltaNet + full attention của LightOnOCR gần như chắc chắn dùng LayerNorm/RMSNorm (chuẩn chuẩn cho kiến trúc Transformer/SSM hiện đại) chứ không dùng BatchNorm theo thống kê mini-batch như mô tả ở mục 3.8 ở trên; với batch size $= 1$ khi inference, BatchNorm kiểu mini-batch sẽ không có ý nghĩa thống kê. Tuy vậy **động cơ nền tảng** (Eq. 6.25: chuẩn hoá input của một lớp để khối Gauss-Newton tương ứng được conditioning tốt hơn) vẫn là lý do khái niệm (không phải thống kê theo batch) giải thích tại sao LayerNorm/RMSNorm giúp huấn luyện ổn định hơn, dù đây là suy luận ngoài phạm vi trực tiếp của paper.
5. **Iterate averaging / Polyak-Ruppert (mục 3.7 ở trên) như một kỹ thuật gần như miễn phí cho fine-tune ngắn (suy luận, tin cậy trung bình):** giữ trung bình các checkpoint cuối của quá trình fine-tune LightOnOCR (tương tự kỹ thuật "weight averaging"/EMA-of-weights phổ biến trong thực hành hiện đại) có cơ sở lý thuyết từ paper này (giảm tác động ill-conditioning mà không cần tính toán bậc hai đắt đỏ), đáng thử nghiệm vì chi phí thực hiện gần như bằng không (chỉ cần lưu và trung bình vài checkpoint).

9. Thí nghiệm gợi ý

1. **Tái dựng tinh thần Hình 3.1 (SG vs phương pháp batch) trên một bài toán nhỏ, CPU, không cần GPU.** Trên hồi quy logistic ℓ_2-regularized, dữ liệu tổng hợp hoặc một tập nhỏ công khai, so sánh SG (bước cố định, dò vài giá trị α) với một phương pháp batch (gradient descent đầy đủ, hoặc L-BFGS nếu có sẵn thư viện), vẽ loss theo số lượt truy cập dữ liệu. Đo: SG có vượt trội trong vài epoch đầu như Hình 3.1 không? Diễn giải: xác nhận trực tiếp hiện tượng gốc của bài, độc lập với deep learning.
2. **Kiểm chứng cảnh báo "không tăng learning rate theo** n_{mb} **một cách tự do" (mục 8.1) trên một bài toán lồi nhỏ.** Chạy mini-batch SG với cỡ batch $n_{mb} \in \{1, 4, 16, 64\}$, với learning rate (a) giữ cố định và (b) scale tuyến tính theo n_{mb}; đo tổng số lượt truy cập mẫu cần để đạt một ngưỡng loss cho trước. Dự đoán theo Theorem 4.6: scale tuyến tính vượt quá cận $\bar{\alpha} \leq \mu/(LM_G)$ sẽ làm chậm hoặc làm mất ổn định hội tụ ở n_{mb} lớn, trong khi giữ cố định cho tổng công gần như không đổi theo n_{mb} (trong vùng $n_{mb} \ll n$). Diễn giải: kiểm tra trực tiếp một khuyến nghị thực dụng rút ra từ lý thuyết của chính paper, liên quan trực tiếp tới việc chọn learning rate khi dùng gradient accumulation cho LightOnOCR.
3. **Cài đặt từ đầu công thức AdaGrad và RMSprop đúng như Eq. ở mục 3.8 (không dùng thư viện có sẵn), so sánh với Adam (opt 06), trên một mạng nhỏ.** Đo tốc độ giảm loss và độ ổn định trong các

bước đầu (nơi AdaGrad được paper ghi nhận ”giảm bước quá hung hãn”). Diễn giải: xác nhận bằng thực nghiệm nhỏ nhận xét của paper, và làm rõ trực quan ”khoảng cách” giữa AdaGrad/RMSprop và Adam đầy đủ.

4. **Đo chuẩn gradient trung bình trong một lượt fine-tune LoRA ngắn của LightOnOCR (hoặc một mô hình nhỏ thay thế), trên 1 GPU $\leq$ 8GB.** Ghi lại $\|\nabla F(w_k)\|^2$ (hoặc một ước lượng xấp xỉ qua gradient của mini-batch) mỗi vài bước; vẽ theo k. Dự đoán theo Theorem 4.8/4.10: xu hướng giảm dần (dù có nhiễu, không đơn điệu). Diễn giải: kiểm tra định tính liệu lý thuyết SG-phi-lồi (Theorem 4.8-4.10) còn phản ánh đúng hành vi thực tế trên một mô hình/tập dữ liệu thật, một bước kiểm chứng thận trọng trước khi tin vào gradient norm làm tiêu chí dừng.
5. **Thử iterate averaging (Polyak-Ruppert, Eq. 5.16) trên chính lượt fine-tune LightOnOCR ở thí nghiệm 4, so sánh checkpoint cuối cùng vs checkpoint trung bình của N checkpoint cuối.** Đo validation loss/metric OCR của cả hai. Dự đoán theo mục 3.7/8.5: checkpoint trung bình ổn định hơn (ít nhạy với ”may rủi” của vài bước cuối), đặc biệt nếu learning rate cuối chưa giảm hẳn về 0. Diễn giải: một kỹ thuật gần như miễn phí, nếu có hiệu quả rõ, có thể áp dụng ngay cho pipeline fine-tune thật của dự án.

10. Tóm tắt công thức / khái niệm

Khái niệm / công thức	Ý nghĩa
SG (Eq. 3.7): $w_{k+1} = w_k - \alpha_k \nabla f_{i_k}(w_k)$	Cập nhật dựa trên 1 mẫu ngẫu nhiên mỗi bước
Algorithm 4.1 tổng quát: $w_{k+1} = w_k - \alpha_k g(w_k, \xi_k)$	Khung chung cho SG, mini-batch, Newton ngẫu nhiên
Theorem 4.6 (fixed step, strongly convex): $\mathbb{E}[F(w_k) - F_*] \to \bar{\alpha} LM/(2c\mu)$	Hội tụ tuyến tính tới vùng lân cận, không tới nghiệm
Theorem 4.7 (diminishing step, Robbins-Monro): $\mathbb{E}[F(w_k) - F_*] \leq \nu/(\gamma + k)$	$O(1/k)$, cần $\sum \alpha_k = \infty, \sum \alpha_k^2 < \infty$
Theorem 4.8-4.10 (phi lồi): $\liminf \mathbb{E}[\|\nabla F(w_k)\|^2] = 0$	Chỉ đảm bảo tới điểm dừng, không tới cực tiểu toàn cục
Table 4.1: $E_*^{batch} \sim \log(T_{max})/T_{max} + 1/T_{max}$, $E_*^{SG} \sim 1/T_{max}$	SG thắng batch trong chế độ ”ngân sách thời gian cố định, n lớn”
Mini-batch trade-off (mục 4.2 cuối): n_{mb} lần đắt hơn, giảm phương sai n_{mb} lần	Tổng công gần như không đổi, lợi ích thực là song song hoá
RMSprop: $[R_k]_i = (1-\lambda)[R_{k-1}]_i + \lambda g_i^2$, chia $\sqrt{R_k + \mu}$	Tiền thân trực tiếp của phần v_t trong Adam (opt 06)
AdaGrad: $[R_k]_i = [R_{k-1}]_i + g_i^2$	Tổng tích luỹ, bước giảm dần $O(1/\sqrt{k})$ mỗi toạ độ
Heavy ball (Eq. 7.1-7.2): hệ số co $(\kappa-1)/(\kappa+1)$; accelerated: $(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$	Khớp đúng Theorem 3.10/3.18 của opt 01
Theorem 7.1 (coordinate descent): $\mathbb{E}[F(w_{k+1})] - F_* \leq (1 - c/(d\hat{L}))^k (\ldots)$	Chậm hơn GD đầy đủ bởi hệ số d, trừ khi có cấu trúc đặc biệt
Polyak-Ruppert averaging (Eq. 5.16)	Bước giảm chậm $O(1/k^a)$ + trung bình iterate cho hằng số tối ưu

(Tự tính) minh hoạ bằng số cho hằng số co của steepest descent so với heavy ball/accelerated (Eq. 7.2), với $\kappa = 100$ (một condition number vừa phải, hợp lý cho một lớp được regularize nhẹ):

Đại lượng	Giá trị (tự tính)
$(\kappa-1)/(\kappa+1) = 99/101$	0,9802
$(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1) = 9/11$	0,8182

Đại lượng	Giá trị (tự tính)
Số bước cần để giảm sai số 1000 lần, steepest descent: $\ln(1000)/(-\ln 0{,}9802)$	$6{,}9078/0{,}0200 \approx 345$
Số bước cần, heavy ball/accelerated: $\ln(1000)/(-\ln 0{,}8182)$	$6{,}9078/0{,}2007 \approx 34$
Tỉ lệ số bước (steepest / accelerated)	$\approx 10{,}0$, khớp đúng $\sqrt{\kappa} = \sqrt{100} = 10$

Phép tính xác nhận đúng mức cải thiện $\sqrt{\kappa}$ lần mà cả paper này (Eq. 7.2) và opt 01 (Bubeck, Theorem 3.18) cùng khẳng định, tính độc lập từ hai bộ công thức khác nhau nhưng cho cùng kết luận định lượng.

11. Kết luận ngắn

Bài khảo sát của Bottou, Curtis và Nocedal xây dựng một lý thuyết hội tụ/work-complexity khá đầy đủ cho stochastic gradient (SG) trong cả bối cảnh lồi mạnh và phi lồi tổng quát (Chương 4, đọc đầy đủ trong ghi chú này), rồi dùng nó để giải thích vì sao SG thắng thế trong machine learning quy mô lớn (Table 4.1), trước khi khảo sát hai hướng cải tiến có tổ chức rõ theo một sơ đồ 2 chiều: giảm nhiễu (Chương 5: dynamic sampling, SVRG/SAGA, iterate averaging) và xử lý bậc hai/ill-conditioning (Chương 6: quasi-Newton, Gauss-Newton, natural gradient, và quan trọng nhất cho chuỗi bài opt là diagonal scaling với công thức tường minh của AdaGrad/RMSprop và liên hệ tới batch normalization). Paper tự thừa nhận rõ các khoảng trống lý thuyết quan trọng (momentum ngẫu nhiên ”elusive”, RMSprop cho phi lồi ”vẫn đang nghiên cứu”, dynamic sampling ”chưa chín”), và tự công khai những gì không bàn (ADMM, EM, phân tán/song song). Đối với dự án LightOnOCR, bài này cho nền tảng lý thuyết **trực tiếp hơn opt 01** cho các quyết định thực dụng: gradient accumulation dưới ràng buộc batch=1 (có trích dẫn cụ thể, kèm cảnh báo về learning rate), khung ”ngân sách thời gian cố định” cho việc đánh đổi thông lượng, và một kỹ thuật gần miễn phí (iterate averaging) đáng thử nghiệm.

Nguồn gốc: https://arxiv.org/abs/1606.04838

Bài 03

An overview of gradient descent optimization algorithms (2016)

Tác giả: Sebastian Ruder (Insight Centre for Data Analytics, NUI Galway; Aylien Ltd., Dublin).
Venue / arXiv: arXiv:1609.04747 (v1 nộp 09/2016; bản đọc ở đây là v2, 15/06/2017). Bài vốn xuất bản dạng blog post tại sebastianruder.com ngày 19/01/2016 trước khi đưa lên arXiv; không gắn với hội nghị/journal cụ thể nào, không qua bình duyệt peer-review truyền thống (survey/tutorial dạng blog cá nhân, dù được trích dẫn rất rộng rãi trong cộng đồng deep learning). Theo quy ước bộ ghi chú, YYYY lấy theo chữ số arXiv ID (1609 -> 2016), dù có một chuỗi mốc thời gian khác nhau: blog gốc 01/2016, arXiv v1 09/2016, bản text đọc được là v2 06/2017; nêu rõ để tránh nhầm giữa các mốc này.
Nguồn: https://arxiv.org/abs/1609.04747
Chủ đề: tổng quan mang tính trực giác (không đi sâu lý thuyết hội tụ hình thức) về các biến thể gradient descent (batch/stochastic/mini-batch), cách suy luận từng bước để đi từ Momentum/NAG tới Adagrad, Adadelta, RMSprop, Adam, AdaMax, Nadam, kiến trúc song song hoá/phân tán SGD, và các chiến lược bổ sung (shuffling, curriculum learning, batch normalization, early stopping, gradient noise).
Vị trí trong lộ trình: nên đọc trước hoặc cùng lúc với opt 06 (Adam), opt 07 (AdamW), opt 08 (AMSGrad), opt 09 (RAdam) như một bản đồ tổng quan dễ tiếp cận trước khi đọc chi tiết từng bài gốc. Lưu ý quan trọng: bài này viết năm 2016, **trước** cả ba bài opt 07-09 (lần lượt công bố 2017-2019), nên khuyến nghị ở mục 4.10 của chính bài (”Adam có thể là lựa chọn tốt nhất nói chung”) đã bị các phát hiện sau đó làm lỗi thời một phần - xem mục 6-7.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026, đọc toàn văn). Các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Đến năm 2016, gradient descent đã là cách phổ biến nhất để tối ưu mạng neural, và mọi thư viện deep learning phổ biến lúc đó (Lasagne, Caffe, Keras) đều cung cấp sẵn nhiều biến thể thuật toán cập nhật. Nhưng theo tác giả, các thuật toán này thường được dùng như ”hộp đen” (black-box optimizers): người thực hành chọn một cái tên quen (SGD, Adam...) mà không hiểu rõ động lực, ưu/nhược điểm của từng thuật toán, vì các giải thích thực tế, dễ hiểu khó tìm được ở thời điểm đó. Mục tiêu paper: xây dựng trực giác, trình bày **cách một thuật toán được suy ra từ thuật toán trước nó** (để giải quyết một nhược điểm cụ thể), hơn là chỉ liệt kê công thức rời rạc.

2. Ý tưởng chính

Toàn bài tổ chức theo một chuỗi suy luận ”nhược điểm -> sửa -> nhược điểm mới -> sửa tiếp”: bắt đầu từ gradient descent thường (batch), nêu nhược điểm tốc độ/bộ nhớ, dẫn tới SGD và mini-batch SGD; nêu các thách thức của mini-batch SGD (chọn learning rate, cùng một learning rate cho mọi tham số, saddle point), rồi lần lượt đưa Momentum (sửa dao động trong ravine), NAG (sửa ”mù” của Momentum bằng cách nhìn trước), Adagrad (learning rate riêng cho từng tham số), Adadelta/RMSprop (sửa nhược điểm learning rate của Adagrad giảm về 0), Adam (kết hợp ý tưởng momentum vào RMSprop/Adadelta kèm bias-correction), AdaMax (tổng quát hoá chuẩn ℓ_2 sang ℓ_∞), Nadam (kết hợp NAG vào Adam). Sau đó bài mở rộng sang song song hoá/phân tán SGD và các chiến lược bổ sung không gắn với một optimizer cụ thể nào (shuffling, batch norm, early stopping, gradient noise).

3. Phương pháp chi tiết

3.1 Ba biến thể gradient descent (§2)

$$\text{Batch: } \theta = \theta - \eta \cdot \nabla_\theta J(\theta) \quad (1) \qquad \text{SGD: } \theta = \theta - \eta \cdot \nabla_\theta J(\theta; x^{(i)}; y^{(i)}) \quad (2)$$

$$\text{Mini-batch: } \theta = \theta - \eta \cdot \nabla_\theta J(\theta; x^{(i:i+n)}; y^{(i:i+n)}) \quad (3)$$

Batch GD: chính xác nhưng chậm, không fit được dataset lớn vào bộ nhớ, không học online được. SGD (từng mẫu một): nhanh, học online được, nhưng nhiễu lớn làm hàm mục tiêu dao động mạnh; dao động này một mặt giúp nhảy sang vùng cực tiểu khác tốt hơn, mặt khác làm khó hội tụ chính xác vào đáy cực tiểu (dù nếu giảm dần learning rate, SGD gần như chắc chắn hội tụ tương tự batch GD). Mini-batch (kích thước phổ biến 50-256): kết hợp cả hai, giảm phương sai cập nhật và tận dụng được các phép toán ma trận tối ưu hoá của thư viện deep learning; trong thực hành, ”SGD” thường được dùng để chỉ cả mini-batch GD.

3.2 Thách thức của mini-batch SGD thường (§3)

- Chọn learning rate khó: quá nhỏ hội tụ chậm, quá lớn gây dao động/phân kỳ.
- Lịch learning rate (annealing) phải định sẵn trước, không tự thích ứng theo đặc điểm dữ liệu.
- Cùng một learning rate áp cho mọi tham số, không phù hợp khi dữ liệu sparse (một số đặc trưng hiếm gặp cần bước cập nhật lớn hơn).
- **Saddle point**, không phải local minima, mới là khó khăn chính khi tối ưu hàm phi lồi của neural network (paper trích dẫn trực tiếp Dauphin et al. 2014: ”the difficulty arises in fact not from local minima but from saddle points” - một chiều dốc lên, chiều khác dốc xuống, thường được một vùng cao nguyên (plateau) cùng mức lỗi bao quanh, khiến gradient gần 0 ở mọi chiều, rất khó thoát).

3.3 Momentum và Nesterov Accelerated Gradient (§4.1-4.2)

Momentum (Qian 1999) thêm một phần γ của vector cập nhật bước trước vào cập nhật hiện tại, giống hình ảnh một quả bóng lăn xuống dốc, tích luỹ ”vận tốc”:

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta), \qquad \theta = \theta - v_t. \quad (4)$$

γ thường chọn 0,9. Giúp tăng tốc theo hướng nhất quán (gradient cùng hướng nhiều bước liên tiếp), giảm dao động theo hướng gradient đổi chiều liên tục (điển hình trong ravine, nơi mặt lỗi dốc hơn nhiều theo một chiều so với chiều khác).

NAG (Nesterov 1983) cho ”quả bóng” này một chút ”linh cảm”: tính gradient không phải tại vị trí hiện tại θ, mà tại vị trí xấp xỉ **sau khi đã** áp dụng phần động lượng ($\theta - \gamma v_{t-1}$, một ước lượng thô vị trí tiếp theo):

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta - \gamma v_{t-1}), \qquad \theta = \theta - v_t. \quad (5)$$

Nhờ ”nhìn trước” này, NAG phản ứng nhanh hơn khi gần tới đáy dốc (biết ”giảm tốc” sớm hơn Momentum thường), paper nêu NAG đã cải thiện rõ hiệu suất RNN trong một số tác vụ (trích Bengio et al. 2012).

3.4 Adagrad (§4.3)

Thích ứng learning rate theo từng tham số riêng: cập nhật lớn cho tham số hiếm gặp, nhỏ cho tham số thường gặp, phù hợp dữ liệu sparse (paper nêu ví dụ Dean et al. dùng Adagrad huấn luyện mạng lớn nhận diện mèo trên YouTube tại Google; Pennington et al. dùng Adagrad huấn luyện GloVe word embeddings). Với $g_{t,i} = \nabla_{\theta_t} J(\theta_{t,i})$:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i} \quad (8), \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t \quad (9),$$

với G_t là ma trận đường chéo, phần tử i là **tổng** bình phương mọi gradient của θ_i từ đầu tới bước t. Ưu điểm: không cần chỉnh tay learning rate (dùng mặc định 0,01 là đủ trong nhiều implementation). Nhược điểm chính (paper tự nêu, dẫn tới các thuật toán sau): G_t chỉ tăng (mọi hạng tử cộng vào đều dương), nên learning rate hiệu dụng **giảm đơn điệu về 0**, tới một lúc thuật toán không còn học thêm được gì.

3.5 Adadelta và RMSprop (§4.4-4.5)

Cả hai cùng mục tiêu: chặn vấn đề "learning rate giảm về 0" của Adagrad, bằng cách **không** cộng dồn toàn bộ lịch sử, mà dùng trung bình trượt mũ (decay $\gamma \approx 0{,}9$) của gradient bình phương:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma) g_t^2. \quad (10)$$

RMSprop (Hinton, chưa công bố chính thức, chỉ có trong slide bài giảng Coursera Lecture 6e) dừng lại ở đây, dùng trực tiếp:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t, \quad \gamma = 0{,}9,\ \eta = 0{,}001 \text{ (mặc định đề xuất)}. \quad (18)$$

Adadelta (Zeiler 2012) đi xa hơn một bước: nhận xét đơn vị của cập nhật $\Delta\theta_t = -\eta g_{t,i}$ (SGD/Momentum/Adagrad) không khớp đơn vị của θ (vì gradient có đơn vị khác tham số, giống đơn vị vận tốc so với vị trí trong vật lý). Để sửa, định nghĩa thêm một trung bình trượt mũ của **chính các bước cập nhật bình phương** $E[\Delta\theta^2]_t = \gamma E[\Delta\theta^2]_{t-1} + (1-\gamma)\Delta\theta_t^2$ (15), rồi xấp xỉ $RMS[\Delta\theta]_t$ (chưa biết ở bước t) bằng $RMS[\Delta\theta]_{t-1}$ (đã biết), thay trực tiếp vào vị trí của η:

$$\Delta\theta_t = -\frac{RMS[\Delta\theta]_{t-1}}{RMS[g]_t} g_t, \qquad \theta_{t+1} = \theta_t + \Delta\theta_t. \quad (17)$$

Kết quả: Adadelta **không cần đặt learning rate** nào cả, nó bị loại khỏi công thức cập nhật hoàn toàn.

3.6 Adam, AdaMax, Nadam (§4.6-4.8)

Adam (Kingma & Ba) giữ cả trung bình trượt mũ bậc một m_t (giống momentum) **và** bậc hai v_t (giống RMSprop/Adadelta), coi là ước lượng mean và (uncentered) variance của gradient:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2. \quad (19)$$

Vì khởi tạo $m_0 = v_0 = 0$, hai ước lượng này bị lệch (bias) về 0, đặc biệt ở các bước đầu và khi β_1, β_2 gần 1; Adam sửa bằng bias-correction:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t. \quad (20\text{-}21)$$

Mặc định đề xuất: $\beta_1 = 0{,}9,\ \beta_2 = 0{,}999,\ \epsilon = 10^{-8}$ (đúng với giá trị dùng trong opt 06).

AdaMax tổng quát hoá chuẩn ℓ_2 trong v_t sang chuẩn ℓ_p bất kỳ ($v_t = \beta_2^p v_{t-1} + (1-\beta_2^p)|g_t|^p$, Eq. 23), rồi lấy giới hạn $p \to \infty$: chuẩn ℓ_∞ cho một giá trị ổn định hơn các chuẩn p lớn khác, cụ thể $u_t = \max(\beta_2 \cdot v_{t-1}, |g_t|)$ (Eq. 24) - một phép **max tích luỹ**, về hình thức khá gần với ý tưởng "lấy max thay EMA" mà opt 08 (AMSGrad) dùng để sửa vấn đề hội tụ của Adam hai năm sau, dù mục tiêu thiết kế ban đầu của AdaMax (ổn định số học của chuẩn ℓ_p) khác mục tiêu của AMSGrad (đảm bảo hội tụ lý thuyết). Vì u_t dựa trên max, không bị bias về 0 như m_t, v_t, nên **không cần bias-correction** cho u_t:

$$\theta_{t+1} = \theta_t - \frac{\eta}{u_t}\hat{m}_t, \qquad \eta = 0{,}002 \text{ (mặc định)}, \beta_1 = 0{,}9,\ \beta_2 = 0{,}999. \quad (25)$$

Nadam (Dozat) ghép NAG vào Adam. Paper trình bày phép suy luận từng bước: viết lại Momentum dạng "mở rộng" $\theta_{t+1} = \theta_t - (\gamma m_{t-1} + \eta g_t)$ (27); NAG sửa bằng cách tính gradient tại vị trí nhìn trước; Dozat đề xuất một biến thể NAG khác, áp "vector động lượng nhìn trước" trực tiếp để cập nhật tham số hiện tại thay vì tính lại gradient hai lần (29); áp cùng mẹo vào Adam: khai triển $\hat{m}_t$ trong công thức Adam, nhận ra số hạng $\beta_1 m_{t-1}/(1-\beta_1^t)$ chính là $\hat{m}_{t-1}$ (bias-corrected của bước **trước**) (31-32), rồi thay $\hat{m}_{t-1}$ bằng $\hat{m}_t$ (bias-corrected của bước **hiện tại**, đúng mẹo "nhìn trước" của Dozat) để ra công thức Nadam cuối:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\left(\beta_1 \hat{m}_t + \frac{(1-\beta_1)g_t}{1-\beta_1^t}\right). \quad (33)$$

3.7 Ví dụ số (tự tính): learning rate hiệu dụng của Adagrad giảm nhanh ra sao

Paper nêu định tính "the accumulated sum keeps growing... learning rate shrink and eventually become infinitesimally small" nhưng không cho số cụ thể. Giả sử (tự tính, ví dụ minh hoạ) gradient không đổi $g_t = 1$ ở mọi bước (trường hợp đơn giản nhất để thấy xu hướng), $\eta = 0{,}01$, bỏ qua ϵ (rất nhỏ so với G_t sau vài bước): $G_{t,ii} = \sum_{k=1}^{t} g_k^2 = t$, nên learning rate hiệu dụng là $\eta/\sqrt{t} = 0{,}01/\sqrt{t}$:

t	1	10	100	10.000
$\eta/\sqrt{t}$ (tự tính)	0,01	0,00316	0,001	0,0001

Chỉ sau 10.000 bước với gradient đơn vị không đổi, learning rate hiệu dụng đã giảm 100 lần - minh hoạ trực tiếp, bằng số, lý do Adadelta/RMSprop phải thay tổng cộng dồn bằng trung bình trượt mũ (EMA không tăng vô hạn, "quên" dần gradient cũ, nên learning rate hiệu dụng không bị ép về 0 theo kiểu này).

4. Thí nghiệm và kết quả chính

Đây là một bài tổng quan, không có benchmark định lượng tự thực hiện (không có bảng số liệu train/test accuracy của riêng tác giả); "kết quả" chủ yếu là các minh hoạ trực quan (do Alec Radford tạo, paper trích nguồn rõ) và các phát hiện định lượng **được dẫn lại** từ các paper gốc (Adagrad cho Google Cat NN và GloVe, EASGD, gradient noise...).

Hình 4a (đường đi trên contour hàm Beale): Adagrad, Adadelta, RMSprop lập tức đi đúng hướng và hội tụ nhanh tương đương nhau; Momentum và NAG bị trớn đi lệch hướng trước (giống hình ảnh quả bóng lăn), nhưng NAG tự sửa hướng nhanh hơn Momentum nhờ cơ chế nhìn trước.

Hình 4b (hành vi tại saddle point): SGD, Momentum, NAG đều khó phá vỡ đối xứng tại saddle point (Momentum và NAG cuối cùng thoát được nhưng chậm); Adagrad, RMSprop, Adadelta đi xuống theo chiều dốc âm rất nhanh, Adadelta nhanh nhất trong nhóm này. Paper kết luận: các phương pháp adaptive learning rate phù hợp nhất cho các tình huống này.

Mục 4.10 "Which optimizer to use?" (khuyến nghị tổng hợp của tác giả, không phải một thí nghiệm riêng): dữ liệu sparse nên dùng adaptive method (không cần tune learning rate kỹ); RMSprop là một mở rộng của Adagrad; RMSprop gần như giống Adadelta (chỉ khác: Adadelta dùng RMS của chính các bước cập nhật ở tử số); Adam = RMSprop + bias-correction + momentum; paper trích lại tuyên bố của Kingma & Ba rằng bias-correction giúp Adam vượt nhẹ RMSprop về cuối quá trình huấn luyện khi gradient trở nên sparser; kết luận (của paper, năm 2016): **"Adam might be the best overall choice."** Paper cũng ghi nhận nhiều bài báo gần đó vẫn dùng SGD thường + momentum + lịch anneal đơn giản, đạt cực tiểu tốt nhưng chậm hơn, nhạy hơn với khởi tạo/lịch learning rate, dễ "mắc kẹt" ở saddle point hơn.

5. Phân tích / ablation

Không có ablation theo nghĩa thực nghiệm có kiểm soát (không phải nội dung chính của một bài tổng quan dạng này); phần ”phân tích” ở đây là chuỗi suy luận **từng bước sửa lỗi của thuật toán trước** mà bản thân cấu trúc bài đã trình bày (đã thuật lại ở mục 3): mỗi thuật toán mới được paper giới thiệu đúng ngay sau khi chỉ ra một nhược điểm cụ thể, cố tình đọc được của thuật toán ngay trước nó (Adagrad sửa ”một learning rate cho mọi tham số”; Adadelta/RMSprop sửa ”learning rate Adagrad giảm về 0”; Adam thêm ”động lượng” vào RMSprop/Adadelta; AdaMax đổi chuẩn để ổn định số học hơn; Nadam thêm ”nhìn trước” kiểu NAG vào Adam). Đây là một cách trình bày sư phạm hiệu quả, nhưng cũng là một **giản lược hoá lịch sử**: trong thực tế nhiều thuật toán này (ví dụ Adadelta và RMSprop, theo đúng lời paper) được phát triển **độc lập, gần như đồng thời**, không phải theo đúng trình tự nhân-quả tuyến tính mà cách trình bày ”A sửa B” gợi ý.

Phần §5-6 (song song hoá, chiến lược bổ sung) cũng không có so sánh định lượng trực tiếp giữa các phương án với nhau trong chính bài; mỗi mục chỉ tóm tắt lại một paper khác (Hogwild!, Downpour SGD, EASGD, gradient noise...) và dẫn lại kết luận của paper đó.

6. Giới hạn

- **Không có lý thuyết hội tụ hình thức nào** (không theorem, không regret bound) - khác hẳn opt 06 (Adam có Theorem 4.1) và opt 08 (AMSGrad có Theorem 4). Đây là một bài tổng quan trực giác, không phải một đóng góp lý thuyết mới.
- **Lỗi thời một phần theo thời gian, do chính thời điểm xuất bản (2016), trước các phát hiện quan trọng sau đó về cùng Adam:** khuyến nghị ”Adam might be the best overall choice” (mục 4.10) không thể tính đến ba vấn đề được phát hiện sau: (i) Adam có thể **không hội tụ** trong một số bài toán online convex đơn giản, do lỗi trong chứng minh gốc (opt 08, AMSGrad, 2018); (ii) cách Adam kết hợp weight decay với gradient thích ứng **vô tình làm yếu** hiệu lực của weight decay (opt 07, AdamW, 2017/2019); (iii) learning rate thích ứng của Adam có phương sai lớn bất thường ở giai đoạn đầu huấn luyện, lý giải vì sao thường cần warmup (opt 09, RAdam, 2019). Đây không phải ”mâu thuẫn nội tại” của paper này (paper không sai ở thời điểm viết), mà là một khuyến nghị đã bị vượt qua bởi nghiên cứu sau - cần nêu rõ khi dùng bài này làm tài liệu tham khảo hiện tại.
- **AdaMax và Nadam ít phổ biến trong thực hành hiện đại** (quan sát chung, ngoài paper): dù được trình bày chi tiết ngang với Adam, cả hai hiếm khi là lựa chọn mặc định trong các pipeline huấn luyện lớn hiện nay (phần lớn dùng Adam hoặc AdamW); paper không thể biết trước điều này vào năm 2016.
- **Không thảo luận second-order method** (paper tự loại trừ ngay từ đầu mục 4, vì không khả thi tính toán cho dữ liệu nhiều chiều) - là một giới hạn phạm vi được công khai, không phải thiếu sót giấu kín.
- **Phần song song hoá/phân tán (§5) đã cũ về công nghệ:** TensorFlow được mô tả ở giai đoạn ”vừa mã nguồn mở, bản phân tán mới ra 04/2016” - framework và kiến trúc phân tán huấn luyện hiện đại (multi-node training cho LLM, ZeRO, FSDP...) đã phát triển rất xa so với Downpour SGD/Hogwild!/EASGD thời điểm bài viết.

7. Liên hệ với các bài khác

- **opt 05 (Goodfellow, Vinyals, Saxe):** cả hai bài cùng trích dẫn **đúng một paper**, Dauphin et al. 2014 (”Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”) để lập luận saddle point, không phải local minima, mới là khó khăn chính của tối ưu hoá phi lồi trong neural network - một liên hệ được xác nhận trực tiếp qua danh sách tham khảo của cả hai bài, không phải suy đoán.
- **opt 06 (Adam, Kingma & Ba 2014):** bài này (opt 03) trình bày lại đúng công thức Adam (Eq. 19-21) với cùng giá trị mặc định $\beta_1 = 0{,}9, \beta_2 = 0{,}999, \epsilon = 10^{-8}$ như trong opt 06, nhưng không có phần chứng minh hội tụ (Theorem 4.1) hay thực nghiệm MNIST/CIFAR-10/CNN đầy đủ của opt 06 - là một bản tóm tắt công thức, không phải thay thế cho việc đọc bài gốc.

- **opt 07 (AdamW), opt 08 (AMSGrad), opt 09 (RAdam):** ba bài này xuất bản **sau** bài tổng quan này (2017, 2018, 2019 so với 2016), nên hoàn toàn không được đề cập - đã nêu ở mục 6. Đáng chú ý: công thức AdaMax trong chính bài này ($u_t = \max(\beta_2 \cdot v_{t-1}, |g_t|)$, Eq. 24) có dạng toán học khá gần với cơ chế "lấy max tích luỹ" mà AMSGrad (opt 08) dùng để sửa vấn đề hội tụ của Adam, dù hai thuật toán được thiết kế cho hai mục tiêu khác nhau (ổn định số học của chuẩn ℓ_p với AdaMax, đảm bảo hội tụ lý thuyết với AMSGrad) và bài khảo sát này không có cách nào biết trước mối liên hệ đó (AMSGrad ra sau).
- **opt 04 (Su, Boyd, Candès, góc nhìn ODE cho gia tốc Nesterov, cùng batch, chưa đọc chi tiết):** Nesterov Accelerated Gradient được giới thiệu ở mục 3.3 trên (trích Nesterov 1983) chính là thuật toán mà opt 04 phân tích tốc độ hội tụ bằng công cụ phương trình vi phân (ODE) - bài này (opt 03) chỉ cho công thức cập nhật và trực giác "nhìn trước", không có phân tích tốc độ hội tụ $O(1/k^2)$ mà Nesterov/opt 04 chứng minh.
- **llm 43 (LightOnOCR):** dùng AdamW (không phải Adam/RMSprop/Adadelta như các thuật toán chính của bài tổng quan này), với cosine schedule và warmup - cho thấy thực hành hiện đại (2026) đã tiến xa hơn khuyến nghị "Adam is probably the best choice" của bài 2016 này; AdamW (opt 07) mới là lựa chọn mặc định phổ biến hiện nay, không phải Adam gốc.
- **rl 07 (PPO):** dùng Adam/SGD bậc một với stepsize suy giảm tuyến tính (annealing) - đúng loại "lịch learning rate định sẵn trước" mà mục 3.2 (thách thức SGD) của bài này mô tả, không dùng adaptive-method để tự động hoá việc này, dù PPO ra sau Adagrad/RMSprop/Adam nhiều năm - một ví dụ thực tế rằng lựa chọn optimizer trong thực hành không luôn theo đúng "thứ bậc tốt nhất" mà một bài tổng quan gợi ý, mà còn phụ thuộc đặc thù bài toán (ở đây là RL on-policy, batch dữ liệu reuse nhiều epoch).

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là suy luận cho việc post-training/compressing **LightOnOCR-3-0.8B** (VLM OCR, kiến trúc hybrid GatedDeltaNet linear attention + full attention, GPU mục tiêu ≤8GB, batch size = 1 khi inference), không phải nội dung paper tuyên bố.

- **Giá trị chính của bài này cho dự án là một "bản đồ" để định vị quyết định đã có, không phải một khuyến nghị mới:** LightOnOCR đã chọn AdamW (opt 07), một lựa chọn **tốt hơn** khuyến nghị "Adam" thuần của bài tổng quan 2016 này, đúng theo đúng hướng mà các phát hiện sau 2016 (đặc biệt opt 07) chỉ ra. Bài này không đổi quyết định đó, chỉ giúp hiểu **tại sao** Adam lại được ưu tiên hơn SGD/Adagrad/Adadelta/RMSprop trong bối cảnh huấn luyện mạng sâu, phức tạp như một VLM OCR: theo mục 4.10, các phương pháp adaptive learning rate hội tụ nhanh hơn, ít nhạy khởi tạo/lịch learning rate hơn, phù hợp hơn khi "quan tâm tốc độ hội tụ và huấn luyện mạng sâu/phức tạp" - đúng tình huống của một VLM 0,8B có kiến trúc hybrid.
- **Mục §6.1 (shuffling, curriculum learning) có thể liên quan trực tiếp đến dữ liệu OCR:** nếu dữ liệu post-training của LightOnOCR được tổ chức theo độ khó (ví dụ: trang in rõ trước, trang scan mờ/nghiêng sau), mục này gợi ý thử curriculum learning có thể cải thiện, nhưng paper cũng dẫn lại một phát hiện quan trọng (Zaremba & Sutskever 2014, LSTM học thực thi chương trình đơn giản): **chiến lược trộn (mixed)** - không hoàn toàn ngẫu nhiên, không hoàn toàn theo đúng thứ tự độ khó - tốt hơn cả hai cực. Đây là một gợi ý cụ thể, dù cần kiểm chứng riêng cho OCR (ngoài paper, suy luận).
- **Mục §6.2 (batch normalization) ít liên quan trực tiếp:** các mô hình Transformer/VLM hiện đại (bao gồm LightOnOCR) thường dùng LayerNorm/RMSNorm, không dùng BatchNorm như bài 2016 này mô tả (BatchNorm nhạy với batch size nhỏ, mà batch size=1 khi inference của dự án càng làm BatchNorm không phù hợp vì thống kê theo batch không ổn định). Nên phần này **không áp dụng trực tiếp** cho LightOnOCR.
- **Mục §6.3 (early stopping) và §6.4 (gradient noise) đều là chiến lược rẻ, dễ thêm vào bất kỳ optimizer nào:** early stopping luôn hợp lý khi fine-tune với tài nguyên hạn chế (GPU ≤8GB, dễ overfit nếu huấn luyện quá lâu trên tập fine-tune nhỏ); gradient noise (Neelakantan et al. 2015, thêm $N(0, \sigma_t^2)$

vào gradient, $\sigma_t^2 = \eta/(1+t)^\gamma$) được paper dẫn lại là giúp mạng **sâu, phức tạp** ổn định hơn với khởi tạo kém - có thể là một thử nghiệm rẻ (suy luận, chưa kiểm chứng cho VLM OCR) nếu gặp vấn đề ổn định khi fine-tune các phần mới/khởi tạo lại của kiến trúc hybrid.

9. Thí nghiệm gợi ý

1. **Tái hiện bảng (tự tính) learning rate Adagrad giảm theo $1/\sqrt{t}$ trên một mô hình nhỏ thật, không chỉ gradient hằng số giả định.** Trên 1 GPU ≤8GB: huấn luyện một MLP nhỏ bằng Adagrad trên MNIST, log giá trị $\sqrt{G_{t,ii}}$ trung bình theo thời gian cho vài tham số đại diện. Đo: tốc độ giảm thực tế của learning rate hiệu dụng so với xấp xỉ $1/\sqrt{t}$ lý thuyết. Diễn giải: kiểm tra mức độ chính xác của xấp xỉ "tự tính" ở mục 3.7 trên dữ liệu/gradient thật (không hằng số như giả định đơn giản hoá).
2. **So sánh trực tiếp 6 optimizer (SGD, Momentum, NAG, Adagrad, RMSprop, Adam) trên cùng một mạng nhỏ, tái dựng ý tưởng Hình 4a/4b ở quy mô thật (không chỉ animation 2D).** Trên 1 GPU ≤8GB: một MLP nhỏ trên Fashion-MNIST hoặc CIFAR-10 thu nhỏ. Đo: tốc độ giảm loss những epoch đầu, độ nhạy với learning rate (thử 3-4 giá trị cho mỗi optimizer). Kỳ vọng: adaptive methods (Adagrad, RMSprop, Adam) ít nhạy learning rate hơn, hội tụ nhanh hơn ở giai đoạn đầu, khớp định tính với mục 4.10 và Hình 4.
3. **Kiểm tra claim "RMSprop gần giống Adadelta" bằng số thật.** Cài cả hai optimizer trên cùng mạng nhỏ, cùng khởi tạo, cùng dữ liệu, đặt $\gamma = 0{,}9$ cho cả hai, nhưng cho Adadelta learning rate tự thích ứng (theo đúng Eq. 17, không cần đặt η) còn RMSprop đặt $\eta = 0{,}001$ như mặc định đề xuất. Đo: quỹ đạo loss hai thuật toán có gần nhau không trong vài trăm bước đầu (khi $RMS[\Delta\theta]$ của Adadelta còn chưa "tự học" ra một learning rate hiệu dụng tương tự 0,001). Diễn giải: kiểm chứng trực tiếp phát biểu "RMSprop giống bước đầu của Adadelta" của chính bài.
4. **Thử nghiệm gradient noise (§6.4) trên một mạng khởi tạo kém có chủ đích.** Trên 1 GPU ≤8GB: huấn luyện một mạng sâu vừa (6-10 lớp fully-connected) với khởi tạo trọng số xấu (ví dụ std quá lớn hoặc quá nhỏ so với Xavier/He), so sánh có/không thêm gradient noise theo Eq. 34-35. Đo: tốc độ hội tụ, có tránh được hiện tượng kẹt sớm không. Diễn giải: kiểm tra claim "gradient noise giúp mạng sâu/phức tạp ổn định hơn với khởi tạo kém" ở quy mô nhỏ, liên hệ tới các module mới/khởi tạo lại trong kiến trúc hybrid của LightOnOCR (mục 8).
5. **Curriculum learning vs shuffling ngẫu nhiên vs chiến lược trộn, trên một tập dữ liệu phân theo độ khó rõ ràng.** Tạo một tập ảnh/OCR nhỏ tổng hợp với 2 mức độ khó rõ rệt (ví dụ chữ in rõ vs chữ bị làm mờ/xoay nhẹ). So sánh 3 thứ tự: ngẫu nhiên (baseline), dễ-tới-khó (curriculum thuần), và trộn (mixed, theo gợi ý Zaremba & Sutskever). Đo: tốc độ hội tụ và chất lượng cuối. Diễn giải: kiểm tra trực tiếp liệu phát hiện "mixed tốt hơn cả hai cực" (dẫn lại trong §6.1) có lặp lại trên một bài toán giống OCR không, trước khi áp dụng suy luận ở mục 8 vào dữ liệu OCR thật.

10. Tóm tắt công thức / khái niệm

Thuật toán	Công thức cập nhật chính	Ý tưởng sửa lỗi so với thuật toán trước
Batch / SGD / Mini-batch GD (Eq. 1-3)	$\theta = \theta - \eta\nabla_\theta J(\theta;\dots)$	-
Momentum (Eq. 4)	$v_t = \gamma v_{t-1} + \eta\nabla_\theta J(\theta)$; $\theta = \theta - v_t$	Giảm dao động trong ravine, tăng tốc theo hướng nhất quán
NAG (Eq. 5)	$v_t = \gamma v_{t-1} + \eta\nabla_\theta J(\theta - \gamma v_{t-1})$	Tính gradient tại vị trí "nhìn trước", phản ứng nhanh hơn
Adagrad (Eq. 8-9)	$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii}+\epsilon}} g_{t,i}$	Learning rate riêng từng tham số, tốt cho dữ liệu sparse
Adadelta (Eq. 17)	$\Delta\theta_t = -\frac{RMS[\Delta\theta]_{t-1}}{RMS[g]_t} g_t$	Chặn Adagrad giảm lr về 0; không cần đặt η

Thuật toán	Công thức cập nhật chính	Ý tưởng sửa lỗi so với thuật toán trước
RMSprop (Eq. 18)	$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$	Cùng mục tiêu Adadelta, dừng ở bước EMA gradient bình phương
Adam (Eq. 19-21)	$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$	Thêm EMA bậc một (momentum) + bias-correction vào RMSprop
AdaMax (Eq. 24-25)	$\theta_{t+1} = \theta_t - \frac{\eta}{u_t} \hat{m}_t$, $u_t = \max(\beta_2 v_{t-1}, \lvert g_t \rvert)$	Chuẩn ℓ_∞ ổn định số học hơn ℓ_2
Nadam (Eq. 33)	$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} (\beta_1 \hat{m}_t + \frac{(1-\beta_1) g_t}{1-\beta_1^t})$	Thêm cơ chế nhìn-trước kiểu NAG vào Adam
(tự tính) $\eta/\sqrt{t}$, $\eta = 0{,}01$	$t = 1 \to 0{,}01$; $t = 10 \to 0{,}00316$; $t = 100 \to 0{,}001$; $t = 10^4 \to 0{,}0001$	Minh hoạ bằng số tốc độ learning rate Adagrad giảm về 0
Hogwild! / Downpour SGD / EASGD (§5)	-	Các kiến trúc song song/phân tán hoá SGD (lock-free, parameter server, elastic force)
Shuffling, curriculum, batch norm, early stopping, gradient noise (§6)	-	Các chiến lược bổ sung, dùng kèm bất kỳ optimizer nào ở trên

11. Kết luận ngắn

Bài là một bản tổng quan trực giác, dễ đọc, trình bày cách suy luận từng bước từ SGD/Momentum/NAG tới các phương pháp adaptive learning rate (Adagrad, Adadelta, RMSprop, Adam, AdaMax, Nadam), cùng các kiến trúc song song hoá SGD và các chiến lược bổ sung không gắn với optimizer cụ thể. Không có đóng góp lý thuyết mới, không có thực nghiệm riêng định lượng; giá trị chính là tổ chức lại kiến thức đã có thành một câu chuyện dễ theo dõi. Vì viết năm 2016, khuyến nghị "Adam có thể là lựa chọn tốt nhất" của bài đã bị các phát hiện sau đó (opt 07 AdamW, opt 08 AMSGrad, opt 09 RAdam) bổ sung/điều chỉnh đáng kể - nên dùng bài này như một bản đồ nhập môn, không phải khuyến nghị cuối cùng cho thực hành hiện tại. Với dự án LightOnOCR, giá trị thực dụng lớn nhất (suy luận, mục 8) nằm ở các chiến lược bổ sung ít được chú ý hơn (curriculum/mixed-order dữ liệu, gradient noise cho phần mới khởi tạo, early stopping cho fine-tune tài nguyên hạn chế), vì phần lựa chọn optimizer chính (AdamW) đã vượt xa khuyến nghị gốc của bài này rồi.

Nguồn gốc: https://arxiv.org/abs/1609.04747

Bài 04

A Differential Equation for Modeling Nesterov's Accelerated Gradient Method: Theory and Insights (2015)

Tác giả: Weijie Su (Dept. Statistics), Stephen Boyd (Dept. Electrical Engineering), Emmanuel J. Candès (Dept. Statistics và Mathematics) - Stanford University.
Venue / arXiv: arXiv:1503.01243 (v1 nộp 03/2015; bản đọc ở đây là v2, 27/10/2015); về sau xuất bản tại Journal of Machine Learning Research (JMLR), vol. 17 (2016). Theo quy ước bộ ghi chú, YYYY lấy theo chữ số arXiv ID (1503 -> 2015), dù bản xuất bản chính thức ở JMLR là năm 2016.
Nguồn: https://arxiv.org/abs/1503.01243
Chủ đề: suy ra một phương trình vi phân thường (ODE) bậc hai, là giới hạn liên tục (continuous-time limit) của thuật toán Nesterov's Accelerated Gradient (NAG, Nesterov 1983) khi step size tiến về 0; dùng ODE này để giải thích hiện tượng dao động (oscillation) quan sát được trong NAG, lý giải ý nghĩa của hằng số "3" đặc biệt trong công thức động lượng gốc, và đề xuất một lược đồ "restart" (khởi động lại) đạt tốc độ hội tụ tuyến tính (exponential) khi hàm mục tiêu strongly convex.
Vị trí trong lộ trình: liên quan trực tiếp tới NAG đã nêu ở opt 03 (mục 4.2 bài đó, cùng trích Nesterov 1983) và tới cách opt 05 dùng khai triển Taylor liên tục-thời-gian để giải thích hành vi tối ưu hoá phi lồi (dù ở mức sơ lược hơn hẳn, chỉ một "cách giải thích khả dĩ", không phải một ODE chứng minh chặt như bài này). Khác hẳn opt 06-09 (optimizer thích ứng kiểu Adam cho deep learning phi lồi, có thực nghiệm neural network thật): bài này thuần lý thuyết tối ưu **lồi** (convex optimization), thực nghiệm chỉ là các bài toán tối ưu lồi mô phỏng (Lasso, hồi quy logistic, matrix completion...), không có mạng neural nào.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026, đọc toàn văn phần chính 1-6; các phụ lục A-E (chứng minh chi tiết Theorem 1, 2, 9 và các Lemma của mục 5) được đọc lướt để nắm đúng nội dung/điều kiện, không chép lại toàn bộ đại số chứng minh vì quá dài và không cần cho trực giác chính - nêu rõ ở đây theo đúng quy ước cho bài lý thuyết nặng chứng minh). Các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Năm 1983, Nesterov đề xuất một thuật toán bậc một (chỉ dùng gradient, không dùng Hessian) để tối ưu hàm lồi f, đạt tốc độ hội tụ $O(1/k^2)$ - nhanh hơn gradient descent thường (chỉ $O(1/k)$) với cùng độ phức tạp tính toán mỗi bước, và tốc độ này được biết là **tối ưu** trong lớp phương pháp chỉ dùng thông tin gradient tại các điểm liên tiếp. Bí quyết nằm ở một số hạng "động lượng" $x_k - x_{k-1}$ với hệ số được tinh chỉnh cẩn thận $(k-1)/(k+2)$.

Dù đã 30 năm (tính đến 2015) kể từ công bố gốc, hành vi của thuật toán này vẫn còn nhiều điều chưa được hiểu trực giác rõ: (a) nó dao động (oscillate), không đơn điệu giảm, khi tiến gần nghiệm, trái với gradient descent thường; (b) hằng số "3" xuất hiện trong phần phân tích bằng ODE của chính bài (giải thích dưới đây) có vẻ đặc biệt, không rõ vì sao chính xác là 3; (c) với hàm strongly convex, phương pháp vẫn chỉ đạt rate đa thức, không tận dụng được tốc độ hội tụ tuyến tính (exponential) mà gradient descent thường **có thể** đạt được khi biết hệ số strong-convexity μ (nhưng ước lượng μ trong thực hành thường rất khó, trong khi ước lượng Lipschitz constant L dễ hơn nhiều, ví dụ bằng backtracking).

Ý tưởng cốt lõi của paper: thay vì phân tích trực tiếp lược đồ rời rạc (discrete) phức tạp, suy ra **giới hạn liên tục** của nó (một ODE), vì lý thuyết ODE đơn giản, súc tích, và được hiểu sâu từ trước - rồi dùng ODE đó như một công cụ để phân tích/mở rộng chính lược đồ rời rạc.

2. Ý tưởng chính

Lược đồ Nesterov gốc (Eq. 1 của paper):

$$x_k = y_{k-1} - s\nabla f(y_{k-1}), \qquad y_k = x_k + \frac{k-1}{k+2}(x_k - x_{k-1}),$$

với $s \le 1/L$ (L = hằng số Lipschitz của ∇f), đạt $f(x_k) - f^\star \le O(\|x_0 - x^\star\|^2/(sk^2))$.

Paper chứng minh: khi cho step size $s \to 0$ (với ansatz $x_k \approx X(k\sqrt{s})$ cho một đường cong trơn $X(t)$), lược đồ này hội tụ chính xác tới nghiệm của ODE bậc hai:

$$\ddot{X} + \frac{3}{t}\dot{X} + \nabla f(X) = 0, \qquad X(0) = x_0,\ \dot{X}(0) = 0. \tag{3}$$

Một điểm bất ngờ paper nhấn mạnh ngay: **một thuật toán bậc một (discrete) lại được mô hình hoá bằng một ODE bậc hai** (continuous). Hệ số $3/t$ đóng vai trò một "lực cản" (damping/friction) giảm dần theo thời gian: ban đầu (t nhỏ) lực cản lớn (overdamped, không dao động); về sau (t lớn) lực cản nhỏ dần (underdamped), hệ dao động với biên độ giảm dần về 0 - đây chính là cách giải thích trực tiếp cho hiện tượng dao động (a) nêu ở mục 1. Hằng số "3" (từ $(k+2) - (k-1) = 3$) hoá ra không tuỳ tiện: nó là hằng số **nhỏ nhất** đảm bảo tốc độ $O(1/t^2)$ vẫn giữ được - một "chuyển pha" (phase transition) tại đúng giá trị 3, giải thích câu hỏi (b). Cuối cùng, nhìn qua lăng kính ODE, vấn đề (c) (không tận dụng được strong convexity) được diễn giải là "quá nhiều động lượng" (hệ số $3/t$ giảm dần làm mất lực cản cần thiết ở giai đoạn sau) - từ đó paper đề xuất một lược đồ **restart theo tốc độ (speed restarting)**: mỗi khi vận tốc $\|\dot{X}\|$ bắt đầu giảm, "khởi động lại" bộ đếm thời gian t về 0, lấy lại lực cản lớn, và chứng minh được tốc độ hội tụ **tuyến tính (exponential)** cho hàm strongly convex.

3. Phương pháp chi tiết

3.1 Suy ra ODE (§2, Eq. 3-5)

Kết hợp hai phương trình của lược đồ Nesterov, đặt $k = t/\sqrt{s}$, dùng ansatz $X(t) \approx x_{t/\sqrt{s}}$, khai triển Taylor theo $\sqrt{s}$ cho cả hai phía của phương trình sai phân, rồi so khớp hệ số của $\sqrt{s}$ ở hai phía - paper trình bày phép suy luận này một cách "informal" (không hoàn toàn chặt) ở đây, nhưng sau đó chứng minh chặt bằng hai định lý:

- **Theorem 1:** với mọi f có gradient Lipschitz (trên toàn không gian, tập hợp các hàm này paper gọi $\mathcal{F}_\infty$) và mọi x_0, ODE (3) có nghiệm toàn cục duy nhất, dù hệ số $3/t$ có kỳ dị (singular) tại $t = 0$ và ∇f thường không giải tích (analytic) tại x_0, khiến lý thuyết ODE cổ điển không áp dụng trực tiếp (paper dùng kỹ thuật xây dựng dãy ODE "làm trơn" gần $t = 0$ rồi lấy giới hạn hội tụ qua định lý Arzelà-Ascoli).
- **Theorem 2:** khi $s \to 0$, lược đồ rời rạc hội tụ **đều** (uniformly) tới nghiệm ODE trên mọi khoảng thời gian hữu hạn: $\lim_{s\to 0} \max_{0 \le k \le T/\sqrt{s}} \|x_k - X(k\sqrt{s})\| = 0$.

Điều kiện đầu $\dot{X}(0) = 0$ (vận tốc ban đầu bằng 0) được paper suy ra trực tiếp từ việc $x_2 - x_1 = O(s)$ nhỏ. Paper cũng chỉ ra một khai triển tiệm cận cho t nhỏ: $X(t) = x_0 - \nabla f(x_0)t^2/8 + o(t^2)$ - khớp với quan sát thực nghiệm rằng lược đồ Nesterov "di chuyển chậm" ở các bước đầu.

ODE (3) có hai tính chất bất biến tự nhiên, khớp với tính chất của chính lược đồ gốc: **bất biến theo thời gian** (chỉ đúng khi hệ số của $\dot{X}$ có dạng C/t, một lý do khác cho thấy dạng $3/t$ không tuỳ tiện) và **bất biến theo phép quay** (orthogonal transformation).

3.2 Tốc độ hội tụ của ODE và hàm bậc hai với Bessel function (§3.1-3.2)

Theorem 3 (chứng minh bằng một "energy functional" $E(t) = t^2(f(X(t)) - f^\star) + 2\|X + t\dot{X}/2 - x^\star\|^2$, chỉ ra $\dot{E} \le 0$ nhờ tính lồi của f): với mọi $f \in \mathcal{F}_\infty$,

$$f(X(t)) - f^\star \le \frac{2\|x_0 - x^\star\|^2}{t^2},$$

cùng dạng $O(1/t^2)$ như lược đồ rời rạc, và paper chứng minh hằng số "2" ở tử số là **tối ưu** (không thể thay bằng số nhỏ hơn, bằng một phản ví dụ cụ thể $f(x) = x$ với $x \ge 0$).

Với f là hàm **bậc hai** $f(x) = \frac{1}{2}\langle x, Ax\rangle$ (A đối xứng dương, phân rã theo trị riêng λ_i), ODE (3) có **nghiệm dạng đóng** bằng hàm Bessel bậc một J_1: $X_i(t) = \frac{2x_{0,i}}{t\sqrt{\lambda_i}} J_1(t\sqrt{\lambda_i})$ (Eq. 8). Dùng khai triển tiệm cận $J_1(u) \approx \sqrt{2/(\pi u)}\cos(u - 3\pi/4)$ khi u lớn, paper tính được $f(X(t)) - f^\star = O(\|x_0 - x^\star\|^2/(t^3\sqrt{\min \lambda_i}))$ - nhanh hơn cận $O(1/t^2)$ tổng quát! Nhưng cũng chứng minh một cận dưới $\limsup t^3(f(X(t)) - f^\star) \ge 2\|x_0 - x^\star\|^2/(\pi\sqrt{L}) > 0$, nghĩa là **không thể nhanh hơn** $O(1/t^3)$ - loại trừ khả năng có bậc cao hơn nữa cho trường hợp bậc hai.

3.3 Dao động của hàm strongly convex, lý thuyết Sturm-Picone (§3.3)

Mở rộng sang hàm strongly convex 1 chiều bất kỳ (không chỉ bậc hai), dùng lý thuyết so sánh Sturm-Picone cho phương trình vi phân tự liên hợp (self-adjoint). **Theorem 4** cho các cận cụ thể trên vị trí các điểm t_i mà $X(t)$ cắt qua $x^\star$ (tức các lần dao động qua nghiệm): $t_1 < 7{,}6635/\sqrt{\mu}$, $t_{i+1} - t_i < 7{,}6635/\sqrt{\mu}$, $t_{i+2} - t_i > \pi/\sqrt{L}$ - nói cách khác, tần số dao động của ODE nằm giữa $O(\sqrt{\mu})$ và $O(\sqrt{L})$ (với μ, L là hệ số strong-convexity và Lipschitz).

3.4 So sánh Nesterov với gradient descent thường, và ổn định số (§3.4)

Theorem 2 cho thấy $x_k \approx X(k\sqrt{s})$: mỗi bước lặp của Nesterov tương đương di chuyển $\sqrt{s}$ theo "thời gian" dọc đường cong ODE (3), còn gradient descent thường tương đương di chuyển chỉ s dọc đường cong của ODE bậc một $\dot X + \nabla f(X) = 0$. Vì $\sqrt{s} \gg s$ khi s nhỏ, đây là một lời giải thích trực giác (không phải chứng minh chặt tốc độ, chỉ là một quan sát định tính có minh hoạ số) cho việc Nesterov "đi nhanh hơn" gradient descent mỗi bước. Paper cũng phân tích ổn định số khi rời rạc hoá ODE bằng forward Euler: bước Δt ổn định nếu $\Delta t < 2/\sqrt{L}$ (tương đương $s = 1/L$ của lược đồ gốc), trong khi gradient descent (rời rạc hoá ODE bậc một) chỉ ổn định với $\Delta t \le 2/L$ - nhỏ hơn hẳn khi L lớn, một lý do kỹ thuật khác giải thích vì sao Nesterov "chịu được" bước lớn hơn gradient descent.

3.5 Hằng số 3 có thể thay bằng r bất kỳ, nhưng chỉ $r \ge 3$ giữ được $O(1/t^2)$ (§4)

Tổng quát hoá ODE (3) với hệ số r tuỳ ý: $\ddot X + \frac{r}{t}\dot X + \nabla f(X) = 0$ (Eq. 17), tương ứng lược đồ rời rạc tổng quát với hệ số động lượng $(k-1)/(k+r-1)$ (Eq. 19).

- $r > 3$ **(nhiều "ma sát" hơn):** **Theorem 5** (ODE) và **Theorem 6** (lược đồ rời rạc) vẫn cho $O(1/t^2)$/$O(1/k^2)$, với hằng số lớn hơn $(r-1)^2$ thay cho 4; thêm một cận tích phân (trung bình) chặt hơn.
- $2 \le r < 3$ **(ít "ma sát" hơn):** paper xây phản ví dụ cụ thể ($f(x) = \frac{1}{2}\|x\|^2$ cho $r < 2$; $f(x) = |x|$ cho $2 \le r < 3$, giải chính xác bằng tay) chứng tỏ **mất** tốc độ $O(1/t^2)$, chỉ còn $O(1/t^r)$ - khẳng định $r = 3$ là **ngưỡng chuyển pha chặt** (tight phase transition), không phải một lựa chọn tuỳ ý.
- $1 < r < 3$ **với điều kiện thêm** ($(f - f^\star)^{(r-1)/2}$ lồi): **Theorem 7** vẫn khôi phục được $O(1/t^2)$ - cho thấy ranh giới $r = 3$ chỉ chặt trong trường hợp tổng quát, không chặt nếu hàm có thêm cấu trúc đặc biệt.
- **Hàm strongly convex,** $r \ge 9/2$**: Theorem 8** (ODE) và **Theorem 9** (lược đồ rời rạc, composite $f = g + h$) chứng minh tốc độ $O(1/t^{2r/3})$/$O(1/k^3)$ **không cần biết** hệ số điều kiện L/μ - nhanh hơn hẳn Theorem 6 khi k đủ lớn so với $\sqrt{L/\mu}$.

3.6 Lược đồ restart theo tốc độ và hội tụ tuyến tính (§5)

Với hàm strongly convex, phương pháp "chính xác" (biết μ, L) dùng hệ số động lượng cố định $\frac{\sqrt{L}-\sqrt{\mu}}{\sqrt{L}+\sqrt{\mu}}$ đạt tốc độ tuyến tính $O((1-\sqrt{\mu/L})^k)$ (Nesterov 2004) - nhưng ước lượng μ trong thực hành rất khó. Paper định nghĩa **thời điểm restart theo tốc độ** $T = \sup\{t : \forall u \in (0,t),\ \frac{d\|\dot{X}(u)\|^2}{du} > 0\}$ (thời điểm đầu tiên tốc độ $\|\dot{X}\|$ **bắt đầu giảm**) - về phía rời rạc, là bước đầu tiên $\|x_{k+1} - x_k\| < \|x_k - x_{k-1}\|$. Mỗi khi chạm mốc này, "đặt lại" bộ đếm thời gian cục bộ $t_{sr} = 0$ trong hệ số $3/t_{sr}$ (Eq. 29), rồi lặp lại.

Theorem 10 (kết quả chính của mục này): với $f \in \mathcal{S}_{\mu,L}$ (strongly convex, gradient Lipschitz), nghiệm X^{sr} của ODE restart theo tốc độ thoả:

$$f(X^{sr}(t)) - f^\star \le \frac{c_1 L\|x_0 - x^\star\|^2}{2} e^{-c_2 t\sqrt{L}},$$

với c_1, c_2 chỉ phụ thuộc hệ số điều kiện L/μ - tức **hội tụ tuyến tính (exponential)**, điều mà lược đồ Nesterov gốc (hay các tổng quát hoá $r \ge 3$ ở mục 3.5) **không** đạt được cho strongly convex (xem phản ví dụ $f(x) = |x|$ ở mục 3.5). Đây là kết quả lý thuyết mới so với lược đồ "restart theo gradient" trước đó (O'Donoghue & Candès 2013, chỉ quan sát thực nghiệm tốt, chưa có lý thuyết hội tụ tổng quát).

3.7 Ví dụ số (tự tính): kiểm chứng xấp xỉ tiệm cận của hệ số động lượng

Paper nêu (không kèm bảng số): hệ số động lượng thay thế $\theta_k(\theta_{k-1}^{-1} - 1)$ (với $\theta_{k+1} = (\sqrt{\theta_k^4 + 4\theta_k^2} - \theta_k^2)/2, \theta_0 = 1$, một công thức khác tương đương cũng dùng trong thực hành, ví dụ bởi Beck & Teboulle 2009) xấp xỉ $(k-1)/(k+2)$ với sai số $O(1/k^2)$. Bảng dưới (tự tính) kiểm chứng xấp xỉ $(k-1)/(k+2) \approx 1 - 3/k$ (công thức paper dùng ngay ở phần dẫn nhập) cho vài giá trị k:

k	$(k-1)/(k+2)$ (tự tính)	$1-3/k$ (tự tính)	Sai số tuyệt đối (tự tính)	$1/k^2$ (để so sánh bậc)
10	0,75000	0,70000	0,05000	0,01
100	0,97059	0,97000	0,00059	0,0001
1000	0,99700	0,99700	0,0000060	0,000001

Sai số giảm rất nhanh, nhất quán với bậc $O(1/k^2)$ mà paper khẳng định (ví dụ từ $k = 10 \to 100$, $1/k^2$ giảm 100 lần, sai số tự tính giảm khoảng $0{,}05/0{,}00059 \approx 85$ lần - cùng bậc độ lớn).

Kiểm chứng thêm (tự tính) ví dụ Hình 1 của paper ($f = 2\times10^{-2}x_1^2 + 5\times10^{-3}x_2^2 = \frac{1}{2}(0{,}04x_1^2 + 0{,}01x_2^2)$, nên $\lambda_1 = 0{,}04, \lambda_2 = 0{,}01$): $T_1 = \pi/\sqrt{\lambda_1} = \pi/0{,}2 = 5\pi$ và $T_2 = \pi/\sqrt{\lambda_2} = \pi/0{,}1 = 10\pi$ - khớp đúng hai giá trị $5\pi, 10\pi$ mà paper nêu, xác nhận cách đọc λ_1, λ_2 từ hệ số hàm f là đúng.

4. Thí nghiệm và kết quả chính

Đây là một paper lý thuyết (convex optimization): "thí nghiệm" ở đây là các **mô phỏng số trên bài toán tối ưu lồi tổng hợp** (synthetic), không phải benchmark machine learning thật (không MNIST/ImageNet, không mạng neural) - khác hẳn opt 06-09.

Mục 4.4, so sánh lược đồ tổng quát hoá với các r khác nhau trên 6 bài toán tổng hợp: Lasso (thiết kế "fat" 100×500 và "square" 500×500), nonnegative least squares (NLS, thiết kế fat và sparse 1000×10000), hồi quy logistic, và hồi quy logistic có ℓ_1-regularization. Quan sát chung: r nhỏ hơn (ít "ma sát") cho tiến triển nhanh hơn ở giai đoạn đầu, nhưng r lớn hơn ưu thế hơn khi k lớn (ít overshoot hơn khi đã gần nghiệm) - khớp đúng diễn giải "lực cản" của mục 3.2.

Mục 5.3, so sánh 4 phương pháp: speed restarting (srN, đề xuất của paper), gradient restarting (grN, O'Donoghue & Candès 2013), Nesterov gốc không restart (oN), và proximal gradient thường (PG) trên 4 bài toán tổng hợp thêm: hàm bậc hai strongly convex (ma trận 500×500, trị riêng trong $[0{,}001, 1]$), log-sum-exp (không strongly convex - **cố ý vi phạm giả thuyết của Theorem 10** để kiểm tra tính mạnh mẽ (robustness) ngoài phạm vi lý thuyết), matrix completion (ma trận hạng 5, nuclear norm penalty), và Lasso dạng ℓ_1-constrained với thiết kế sparse lớn (5000×50000). Trên cả 4 bài toán (kể cả 2 bài **không** thoả giả thuyết strongly convex/smooth của Theorem 10), **cả hai lược đồ restart đều quan sát thấy hội tụ tuyến tính thực nghiệm**, giảm rõ các "bước nhảy" (bumps) trong giá trị hàm mục tiêu so với Nesterov gốc không restart; speed restarting (của paper) cho tốc độ tuyến tính **ổn định hơn** gradient restarting trong các ví dụ này. Paper tự nêu đây để lại một câu hỏi mở: liệu gradient restarting có tốc độ tuyến tính được chứng minh **tổng quát** như Theorem 10 (chỉ chứng minh cho speed restarting) hay không.

Thêm 4 bài toán mô phỏng khác ở mục 5.3 phần mở rộng: ℓ_1-regularized logistic regression, logistic regression với thiết kế sparse rất lớn ($10^7 \times 20000$, khoảng 2×10^8 phần tử khác 0), sorted ℓ_1 penalized estimation (SLOPE, Bogdan et al. 2015, một kỹ thuật mới lúc đó), và Lasso thường - tất cả đều mô phỏng số liệu tổng hợp (Gaussian random design), không dùng dữ liệu thật.

5. Phân tích / ablation

Phần "ablation" tự nhiên nhất của bài này là chuỗi thí nghiệm thay đổi **một hằng số duy nhất** r (hệ số lực cản r/t) và quan sát hệ quả - đây gần như đúng nghĩa ablation kinh điển (giữ mọi thứ khác cố định, chỉ đổi một siêu tham số), được cả lý thuyết (mục 3.5, Theorem 5-9) và thực nghiệm (mục 4.4, Hình 5) xác nhận song song: $r < 3$ phá vỡ tốc độ $O(1/t^2)$ (có phản ví dụ cụ thể, không chỉ là chưa chứng minh được); $r = 3$ là ngưỡng tối thiểu; $r > 3$ vẫn giữ tốc độ nhưng với hằng số lớn hơn (chậm hơn về hằng số, dù cùng bậc).

Ablation thứ hai: **so sánh 4 biến thể restart/không-restart** (srN/grN/oN/PG, mục 5.3) trên cùng bài toán - cho thấy restart (bất kỳ kiểu nào) cải thiện rõ so với không restart, và giữa hai kiểu restart, speed restarting (dựa trên vận tốc, không cần tính lại giá trị hàm mục tiêu mỗi bước để so sánh như gradient restarting) cho kết quả ổn định hơn trong các ví dụ thử - mặc dù về lý thuyết paper chỉ chứng minh chặt cho speed restarting, không phải vì gradient restarting kém hơn về bản chất mà vì (paper tự nói) nó "để lại một câu hỏi mở" chưa có công cụ chứng minh.

6. Giới hạn

- **ODE chỉ là một xấp xỉ liên tục, việc "dịch" các tính chất ODE sang kết quả rời rạc tương ứng đòi hỏi chứng minh riêng, tẻ nhạt, không tự động.** Paper tự nói rõ ở mục Discussion: "the translation... involves parameter tuning and tedious calculations" và mong có "a general theory mapping properties of ODEs into corresponding properties for discrete updates" trong tương lai - nghĩa là bản thân tác giả coi đây là một hạn chế chưa giải quyết của phương pháp, không phải điểm mạnh tuyệt đối.
- **Phần lớn kết quả chỉ đúng cho hàm lồi (hoặc strongly convex) trơn (hoặc composite với phần không trơn xử lý qua proximal), không áp dụng cho hàm phi lồi** như hàm mất mát của mạng neural sâu hiện đại (khác hẳn phạm vi của opt 06-09). Bài không tuyên bố áp dụng cho non-convex, nên đây là giới hạn phạm vi được công khai, không phải thiếu sót giấu kín.
- **Lược đồ restart theo tốc độ cần biết trước μ, L (qua hằng số c_1, c_2 trong Theorem 10 phụ thuộc hệ số điều kiện L/μ)** - paper không hoàn toàn giải quyết được khó khăn gốc (ước lượng μ khó trong thực hành) mà chỉ **giảm nhẹ** nó: không cần biết μ, L để **chạy** thuật toán restart (restart dựa trên quan sát $\|\dot{X}\|$ giảm, không cần biết μ, L), nhưng **tốc độ hội tụ được chứng minh** vẫn phụ thuộc các hằng số này - một sự phân biệt tinh tế giữa "thuật toán không cần tham số" và "chứng minh lý thuyết không cần tham số" mà người đọc cần phân biệt rõ.
- **Thực nghiệm chỉ trên bài toán tổng hợp (synthetic), quy mô vừa phải, không có dữ liệu thật.** Không có gì sai (phù hợp mục tiêu minh hoạ lý thuyết), nhưng giới hạn khả năng suy rộng trực tiếp

sang các bài toán tối ưu quy mô lớn, cấu trúc phức tạp trong thực hành deep learning hiện đại.

- **Lược đồ restart theo gradient (grN, dùng để so sánh) chưa có chứng minh hội tụ tuyến tính tổng quát** (chỉ speed restarting của chính paper có Theorem 10) - paper tự nêu đây là câu hỏi mở, chưa giải quyết trong chính bài.
- **"Hằng số 3" chỉ được paper giải thích là ngưỡng chuyển pha (phase transition) bằng phản ví dụ cụ thể, chưa có một "nguyên lý sâu hơn" nào lý giải TẠI SAO lại đúng 3** (ngoài việc nó bằng $(k+2)-(k-1)$ trong công thức gốc của Nesterov) - paper tự thừa nhận: "this striking phase transition at 3 deserves more attention as an interesting research challenge", tức là một câu hỏi mở được chính các tác giả nêu ra, chưa trả lời dứt điểm trong bài.

7. Liên hệ với các bài khác

- **opt 03 (Ruder, overview gradient descent):** NAG trình bày ở mục 4.2 của opt 03 (trích đúng Nesterov 1983, công thức $v_t = \gamma v_{t-1} + \eta \nabla f(\theta - \gamma v_{t-1})$) dùng hệ số γ **cố định** (thường 0,9), khác với công thức **gốc** của Nesterov mà bài này (opt 04) phân tích, trong đó hệ số động lượng $(k-1)/(k+2)$ **thay đổi theo** k và chính sự thay đổi này mới tạo ra tốc độ $O(1/k^2)$ đã được chứng minh tối ưu. Đây là một sự khác biệt quan trọng, thường bị nhập nhằng trong thực hành: "Nesterov momentum" cài trong các thư viện deep học hiện đại (ví dụ tuỳ chọn `nesterov=True` của SGD trong PyTorch) thường dùng γ cố định kiểu opt 03, **không hoàn toàn** là lược đồ có lịch $(k-1)/(k+2)$ mà Theorem gốc của Nesterov (và ODE của bài này) đảm bảo tốc độ $O(1/k^2)$.
- **opt 05 (Goodfellow, Vinyals, Saxe):** cả hai bài cùng dùng ý tưởng **xấp xỉ liên tục thời gian (continuous-time)** của một thuật toán gradient-based rời rạc như một công cụ phân tích/giải thích, nhưng ở hai mức độ chặt rất khác nhau. opt 05 chỉ dùng khai triển Taylor bậc hai theo thời gian của gradient descent liên tục $\dot{X} = -\nabla f(X)$ như "một cách giải thích khả dĩ" (paper 05 tự dùng từ "one possible explanation") cho việc né saddle point, không có định lý hội tụ nào gắn với nó. Bài này (opt 04) đi xa hơn rất nhiều: suy ra chính xác ODE là **giới hạn** của lược đồ Nesterov (Theorem 2, hội tụ đều, không chỉ là một phép xấp xỉ gợi ý), và chứng minh một loạt định lý hội tụ chặt (Theorem 3-10) dựa trên ODE đó. Hai bài cho thấy hai mức độ sử dụng ý tưởng "continuous-time limit": một ở mức minh hoạ trực giác (opt 05), một ở mức công cụ phân tích chặt, có định lý (opt 04).
- **opt 06 (Adam):** opt 06 cũng dùng một dạng "động lượng" (m_t, EMA của gradient) nhưng với hệ số β_1 **cố định** (0,9 mặc định), giống kiểu thực hành ở opt 03 hơn là lược đồ Nesterov gốc có lịch hệ số biến đổi theo t mà bài này phân tích; Nadam (một biến thể của Adam, nêu ở opt 03 mục 4.8) là nơi ý tưởng "nhìn trước" kiểu NAG được ghép vào Adam, nhưng cũng dùng β_1 cố định, không dùng đúng lịch $(k-1)/(k+2)$.
- **opt 01, opt 02 (Bubeck; Bottou, Curtis, Nocedal - cùng batch, chưa đọc chi tiết tại thời điểm viết ghi chú này):** theo mô tả nhiệm vụ, đây là hai khảo sát lớn về lý thuyết tối ưu lồi/học máy; nhiều khả năng cả hai đều có phần trình bày phương pháp gia tốc Nesterov và tốc độ hội tụ $O(1/k^2)$ như một kết quả kinh điển của tối ưu lồi bậc một - bài này (opt 04) cung cấp một góc nhìn bổ sung (ODE) cho đúng kết quả đó, không thay thế phần trình bày chuẩn (dựa trên bất đẳng thức rời rạc) mà các khảo sát đó có thể dùng.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là suy luận cho việc post-training/compressing **LightOnOCR-3-0.8B** (VLM OCR, kiến trúc hybrid GatedDeltaNet linear attention + full attention, GPU mục tiêu ≤8GB, batch size = 1 khi inference), không phải nội dung paper tuyên bố. Cần nói thẳng ngay từ đầu: đây là bài **ít liên quan trực tiếp nhất** trong số 9 bài được giao đến quy trình huấn luyện/fine-tune cụ thể của dự án, vì ba lý do: (i) lý thuyết chỉ áp dụng cho hàm **lồi**, còn huấn luyện mạng neural (bao gồm LightOnOCR) là bài toán phi lồi; (ii) optimizer thực tế của LightOnOCR là AdamW với β_1 cố định (opt 07), không phải lược đồ Nesterov có lịch hệ số biến đổi theo k mà bài này phân tích; (iii) không có thực nghiệm neural network nào trong bài để đối chiếu trực tiếp.

Tuy vậy, có hai điểm suy luận gián tiếp, đáng ghi nhận với mức độ tin cậy thấp đến trung bình:

- **Ý tưởng "restart" có một tương tự xa trong thực hành huấn luyện hiện đại:** cosine learning rate schedule **với warm restarts** (SGDR, nêu gián tiếp ở opt 07 §3.5 khi thảo luận AdamWR) định kỳ "khởi động lại" lịch learning rate - về hình thức tương tự (không giống hệt về cơ chế hay chứng minh) ý tưởng "restart" của bài này (khởi động lại bộ đếm t trong hệ số $3/t$ khi vận tốc giảm). LightOnOCR hiện dùng cosine schedule **không** có restart (một lần duy nhất, theo llm 43), nên đây chỉ là một liên hệ về hình thức/ngữ cảnh lịch sử, không phải một khuyến nghị cụ thể cho dự án (ngoài paper, suy luận yếu).
- **Bài học định tính có thể chuyển giao (yếu, cần cẩn trọng khi ngoại suy từ lồi sang phi lồi): "quá nhiều động lượng gây overshoot khi gần nghiệm; ít động lượng hội tụ nhanh hơn lúc đầu nhưng dao động nhiều hơn lúc cuối"** (mục 3.5, Hình 5) là một trực giác có thể áp dụng định tính (không định lượng) khi chọn β_1 cho AdamW lúc fine-tune LightOnOCR với số bước rất ít (ràng buộc GPU ≤8GB thường đi kèm ít bước cập nhật): nếu fine-tune rất ngắn, có thể hợp lý thử **giảm nhẹ** β_1 (ít động lượng hơn mặc định 0,9) để tiến triển nhanh hơn trong ít bước, đánh đổi lấy khả năng dao động nhiều hơn gần cực tiểu - nhưng đây là một suy luận định tính, chuyển từ một kết quả **lồi, có chứng minh** sang một bối cảnh **phi lồi, không có chứng minh tương ứng**, nên độ tin cậy thấp, chỉ nên coi là một giả thuyết để thử nghiệm nhỏ (mục 9), không phải một khuyến nghị.

9. Thí nghiệm gợi ý

1. **Tái dựng trực tiếp ODE và lược đồ rời rạc trên một hàm bậc hai 2 chiều, kiểm tra nghiệm Bessel (tái dựng Hình 1).** Không cần GPU: cài đặt lược đồ Nesterov (Eq. 1) với vài step size s nhỏ dần, và giải số ODE (3) bằng một bộ giải ODE chuẩn (ví dụ `scipy.integrate.solve_ivp`), trên $f = 0{,}02x_1^2 + 0{,}005x_2^2$ giống đúng paper. Đo: quỹ đạo lược đồ rời rạc có tiến gần nghiệm ODE khi $s \to 0$ không (kiểm chứng Theorem 2), và nghiệm ODE có khớp công thức Bessel (Eq. 8) không. Diễn giải: xác nhận trực tiếp, rẻ, độ chính xác của phần toán cốt lõi mà cả bài xây trên đó.
2. **Kiểm tra ngưỡng chuyển pha $r = 3$ bằng số (tái dựng Hình 2/4).** Trên 1 CPU: chạy lược đồ tổng quát hoá (Eq. 19) với $r \in \{1, 2, 2{,}5, 3, 4, 5\}$ trên $f(x) = \frac{1}{2}x^2$ và $f(x) = |x|$ (hoặc một bản làm trơn nhẹ gần 0 để tránh vấn đề không khả vi, như paper gợi ý ở footnote). Đo: $k^2(f(x_k) - f^\star)$ có hội tụ về một giá trị hữu hạn (ứng $r \geq 3$) hay tăng không giới hạn (ứng $r < 3$)? Diễn giải: xác nhận bằng số ngưỡng chuyển pha lý thuyết, một kết quả rất sạch, dễ kiểm tra độc lập.
3. **Thử lược đồ speed-restarting (Algorithm 1 của paper) trên một bài toán strongly convex nhỏ và một bài toán KHÔNG strongly convex (log-sum-exp), tái dựng ý tưởng Hình 6.** Không cần GPU. Đo: tốc độ hội tụ (log-scale) của srN so với oN (Nesterov gốc không restart). Diễn giải: kiểm tra trực tiếp tuyên bố "hội tụ tuyến tính thực nghiệm cả khi giả thuyết lý thuyết (strongly convex) không thoả" - một tuyên bố mạnh mà paper chỉ minh hoạ bằng số, chưa chứng minh cho trường hợp không strongly convex.
4. **Phiên bản "suy luận §8" cho AdamW: quét β_1 trên một lượt fine-tune rất ngắn của một mạng nhỏ (proxy cho LightOnOCR).** Trên 1 GPU ≤8GB: fine-tune một mạng nhỏ (hoặc LoRA nhỏ) trong một số bước rất hạn chế (ví dụ 50-200 bước, mô phỏng ràng buộc GPU nhỏ), thử $\beta_1 \in \{0{,}5; 0{,}7; 0{,}9; 0{,}95\}$ với AdamW. Đo: tốc độ giảm loss trong các bước đầu, và độ dao động/overshoot của loss về cuối. Diễn giải: kiểm tra (một cách thận trọng, phi lồi, không có bảo đảm lý thuyết) liệu trực giác "ít động lượng nhanh lúc đầu, nhiều động lượng ổn lúc cuối" từ bài toán lồi của paper này có còn đúng định tính trên một bài toán phi lồi nhỏ, ngắn hạn giống tình huống thực tế của dự án không.
5. **So sánh trực quan "quãng đường mỗi bước": Nesterov vs gradient descent (tái dựng Hình 3b).** Không cần GPU: trên một hàm lồi 2D đơn giản, chạy cả hai thuật toán với cùng s nhỏ, đánh dấu vị trí sau cùng số bước lặp, so với vị trí $X(t)$ trên đường cong ODE tương ứng tại $t = k\sqrt{s}$ (Nesterov) và $t = ks$ (gradient descent). Đo: khoảng cách đã đi được sau cùng số bước. Diễn giải: minh hoạ trực tiếp, bằng mắt, lý do định tính "mỗi bước Nesterov tương đương đi $\sqrt{s}$ thời gian ODE, nhanh hơn s của gradient descent".

10. Tóm tắt công thức / khái niệm

Khái niệm / công thức	Ý nghĩa
Lược đồ Nesterov (Eq. 1): $x_k = y_{k-1} - s\nabla f(y_{k-1})$, $y_k = x_k + \frac{k-1}{k+2}(x_k - x_{k-1})$	Thuật toán gốc (1983), tốc độ $O(1/k^2)$, tối ưu trong lớp phương pháp bậc một
ODE giới hạn (Eq. 3): $\ddot{X} + \frac{3}{t}\dot{X} + \nabla f(X) = 0$	Giới hạn liên tục khi $s \to 0$ (Theorem 1-2); hệ số $3/t$ là "lực cản" giảm dần
Theorem 3: $f(X(t)) - f^\star \leq 2\|x_0 - x^\star\|^2/t^2$	Tốc độ hội tụ của ODE, hằng số 2 là tối ưu
Nghiệm Bessel cho f bậc hai (Eq. 8-10)	$f(X(t)) - f^\star = O(\|x_0 - x^\star\|^2/(t^3\sqrt{\min \lambda_i}))$ cho hàm bậc hai, giải thích dao động tắt dần
ODE tổng quát (Eq. 17): $\ddot{X} + \frac{r}{t}\dot{X} + \nabla f(X) = 0$	$r \geq 3$ giữ $O(1/t^2)$ (Theorem 5); $r < 3$ có phản ví dụ mất tốc độ này (mục 3.5) - chuyển pha chặt tại $r = 3$
Theorem 9 (strongly convex, composite): $f(x_k) - f^\star \lesssim 1/k^3$ (với $r \geq 9/2$)	Không cần biết hệ số điều kiện L/μ
Thời điểm restart theo tốc độ $T = \sup\{t : \forall u < t,\ d\|\dot{X}(u)\|^2/du > 0\}$	Khởi động lại khi vận tốc bắt đầu giảm
Theorem 10: $f(X^{sr}(t)) - f^\star \leq \frac{c_1 L\|x_0 - x^\star\|^2}{2} e^{-c_2 t\sqrt{L}}$	Hội tụ tuyến tính (exponential) cho strongly convex với speed restarting
(tự tính) $(k-1)/(k+2)$ vs $1 - 3/k$	Sai số giảm theo $O(1/k^2)$: 0,05 ($k = 10$) -> 0,0006 ($k = 100$) -> 0,000006 ($k = 1000$)
(tự tính, kiểm chứng) $T_1 = \pi/\sqrt{0{,}04} = 5\pi$, $T_2 = \pi/\sqrt{0{,}01} = 10\pi$	Khớp đúng chu kỳ dao động nêu trong Hình 1 của paper

11. Kết luận ngắn

Bài suy ra một ODE bậc hai là giới hạn liên tục chính xác (chứng minh chặt, Theorem 1-2) của lược đồ Nesterov's Accelerated Gradient, rồi dùng ODE này để: giải thích hiện tượng dao động (qua hệ số lực cản $3/t$ giảm dần và nghiệm dạng Bessel cho hàm bậc hai), chứng minh hằng số "3" là ngưỡng chuyển pha chặt cho tốc độ $O(1/t^2)$ (mục 3.5), và đề xuất một lược đồ restart theo tốc độ đạt hội tụ tuyến tính chứng minh được cho hàm strongly convex (Theorem 10) - vượt qua giới hạn cố hữu của lược đồ gốc (không tận dụng được strong convexity, mục 3.5-3.6). Đây là một bài lý thuyết tối ưu lồi chặt chẽ, không có thực nghiệm neural network, và không trực tiếp đề xuất optimizer mới cho deep learning. Với dự án LightOnOCR, mức độ liên quan trực tiếp là thấp (suy luận, mục 8); giá trị chủ yếu là một bài học định tính, cần kiểm chứng riêng, không có bảo đảm lý thuyết tương ứng khi chuyển từ bối cảnh lồi sang phi lồi.

Nguồn gốc: https://arxiv.org/abs/1503.01243

Bài 05

Qualitatively Characterizing Neural Network Optimization Problems (2014)

Tác giả: Ian J. Goodfellow, Oriol Vinyals (Google Inc.), Andrew M. Saxe (Stanford, Dept. Electrical Engineering).
Venue / arXiv: arXiv:1412.6544 (v1 nộp 12/2014; bản đọc ở đây là v6, 21/05/2015); trang đầu PDF ghi ”Published as a conference paper at ICLR 2015”. Theo quy ước bộ ghi chú, YYYY lấy theo chữ số arXiv ID (1412 -> 2014); hội nghị công bố chính thức là ICLR 2015, một chênh lệch nhỏ và khá thường gặp (bài nộp cuối năm trước, duyệt/trình bày đầu năm sau), nêu rõ để tránh nhầm, không nghiêm trọng như trường hợp opt 08.
Nguồn: https://arxiv.org/abs/1412.6544
Chủ đề: đề xuất một kỹ thuật thực nghiệm rất đơn giản (nội suy tuyến tính tham số từ điểm khởi tạo đến điểm nghiệm huấn luyện) để kiểm tra định tính xem mạng neural khi huấn luyện bằng SGD có thực sự phải ”vượt qua” local minima/saddle point hay không. Kết luận chính: trên đường thẳng nối điểm khởi tạo và nghiệm, hầu hết các mô hình thực tế (7 mô hình khác loại) không gặp vật cản đáng kể.
Vị trí trong lộ trình: không phụ thuộc trực tiếp hai chiều với opt 06-09 (các bài đó là về thuật toán cập nhật tham số cụ thể - Adam/AdamW/AMSGrad/RAdam; bài này là một nghiên cứu chẩn đoán/trực quan hoá hình dạng bề mặt mất mát mà các thuật toán đó phải đi qua). Có thể đọc độc lập, trước hoặc sau opt 01-04 và opt 06-09 tuỳ ý; liên quan gián tiếp đến opt 04 (góc nhìn continuous-time/ODE cho động lực tối ưu hoá) và đến dòng nghiên cứu ”linear mode connectivity” ở bài pruning 09.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026, đọc toàn văn kể cả phụ lục A-D). Các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Huấn luyện mạng neural là giải một bài toán tối ưu hoá **không lồi (non-convex)** quy mô lớn. Trong nhiều năm, giới nghiên cứu lo ngại bài toán này rất khó: sợ local minima (cực tiểu cục bộ ”tệ”), sợ saddle point (điểm yên ngựa) làm optimizer đứng im, và điều này từng là động lực cho nhiều kỹ thuật như unsupervised pretraining (khởi tạo thông minh trước khi fine-tune bằng gradient descent, để tránh phải đi qua các vùng khó). Nhưng đến thời điểm paper này ra đời (2014), các mạng neural hiện đại đã đạt lỗi huấn luyện gần như bằng 0 trên các bài toán phức tạp **chỉ bằng SGD trực tiếp**, không cần các mẹo khởi tạo phức tạp đó.

Câu hỏi paper đặt ra: các mạng này có thực sự đang ”vượt qua” (escape) local minima và saddle point một cách khéo léo nào đó, hay đơn giản là chúng **không gặp vật cản đáng kể** trên đường đi của mình? Hai khả năng này dẫn tới hai cách nhìn khác hẳn nhau về lý do SGD thành công trên NN.

Paper tự đặt rõ giới hạn phạm vi ngay từ đầu: chỉ xét hàm mất mát **tổng trên toàn bộ tập huấn luyện** (total cost function), không phải các xấp xỉ stochastic minibatch mà SGD thực sự dùng ở mỗi bước; nên vẫn có khả năng khó khăn thực sự nằm ở cấu trúc riêng của từng minibatch hoặc ở nhiễu do lấy mẫu minibatch, chứ không nằm ở hình dạng của hàm mất mát tổng mà bài này khảo sát.

2. Ý tưởng chính

Kỹ thuật trung tâm: gọi θ_i là tham số khởi tạo (nhỏ, ngẫu nhiên) và θ_f là tham số sau khi huấn luyện xong (hội tụ, chọn bằng early stopping). Thay vì cố tóm tắt toàn bộ quỹ đạo SGD cao chiều, phức tạp (rất khó trực quan hoá), paper chỉ nhìn vào **đường thẳng nối hai điểm này**:

$$\theta(\alpha) = (1-\alpha)\theta_i + \alpha\theta_f,$$

rồi tính $J(\theta(\alpha))$ (hàm mất mát tổng, trên cả train và validation) với α chạy qua một dải giá trị (thường $[0, 1]$, có khi mở rộng ra $[-0.2, 2]$ để xem thêm hai phía). Đây là một phép "cắt lát" 1 chiều của một không gian tham số rất nhiều chiều: rẻ để tính (chỉ cần forward pass, không cần gradient riêng cho thí nghiệm này), và dễ đọc trên một đồ thị 2 chiều.

Áp dụng kỹ thuật này cho 7 mô hình khác loại (mạng fully-connected với nhiều hàm kích hoạt, mạng tích chập, mô hình sinh (generative), mô hình hồi quy tuần tự (LSTM), và một mô hình tuyến tính sâu phân tích được), paper tìm thấy một đặc điểm chung nổi bật: đường cắt $J(\theta(\alpha))$ **gần như lồi, trơn**, không có vật cản rõ rệt, ngoại trừ một vùng **bằng phẳng gần** $\alpha = 0$ (gần điểm khởi tạo), được giải thích là hệ quả của **phá vỡ đối xứng sớm (early symmetry breaking)**: ở điểm khởi tạo đối xứng (các unit/tham số tương đương nhau), hàm mất mát có một saddle point, và SGD cần vài bước đầu để "chọn" một hướng phá vỡ đối xứng trước khi thực sự tiến triển.

3. Phương pháp chi tiết

3.1 Kỹ thuật nội suy tuyến tính và giới hạn tự nhiên của nó (§2)

Nếu khi đặt $\theta_0 = \theta_i, \theta_1 = \theta_f$, đường cắt $J(\theta(\alpha))$ trơn và gần lồi, điều đó có nghĩa: **nếu biết đúng hướng cần đi, một line search thô (coarse) duy nhất đã có thể huấn luyện xong mạng**. Đây là một phát hiện mạnh về mặt định tính, nhưng paper tự nhấn mạnh ngay giới hạn: đây chỉ là MỘT lát cắt 1 chiều của một không gian rất nhiều chiều; không nói gì chắc chắn về cấu trúc nằm ngoài đường thẳng này. Phần còn lại của paper (kể cả phụ lục) dành nhiều công sức để tự kiểm tra xem lát cắt này có "đại diện" hay bị chọn một cách tình cờ tạo cảm giác trơn giả tạo không (xem mục 3.3, 5).

3.2 Toạ độ thứ hai: đo SGD lệch khỏi đường thẳng bao nhiêu (Phụ lục B)

SGD thực tế **không** đi theo đường thẳng $\theta(\alpha)$ (dù bắt đầu và kết thúc trùng với nó). Để định lượng độ lệch, paper định nghĩa vector đơn vị u theo hướng $\theta_f - \theta_i$, toạ độ $\alpha(t) = (\theta(t) - \theta_i)^\top u$ (vị trí dọc đường thẳng), vector đơn vị v vuông góc với u (theo hướng còn lại của $\theta(t)$ sau khi đã trừ phần chiếu lên u), và toạ độ dư $\beta(t) = (\theta(t) - \theta_i - \alpha(t)u)^\top v$ (độ lớn phần "lệch" khỏi đường thẳng). Cặp $(\alpha(t), \beta(t))$ cho một mặt cắt 2 chiều, vẽ được dạng 3D (α, β, J).

3.3 Thí nghiệm kiểm soát (control): không gian nhiều chiều tự "làm mượt" ngẫu nhiên (Phụ lục B, Hình 11-12)

Đây là điểm phương pháp luận quan trọng nhất của toàn bài, paper tự đưa ra để cảnh báo người đọc: nếu lấy một **random walk hoàn toàn ngẫu nhiên** (không có cấu trúc hàm mất mát nào cả) trong không gian D chiều, rồi vẽ (α, β) theo đúng công thức ở mục 3.2, hình dạng đường cong **hội tụ về một dạng hình học đơn giản, gần giống nhau**, chỉ do hiệu ứng số chiều cao (concentration of measure), không liên quan gì đến có hay không có vật cản thật. Nghĩa là: một đường (α, β) "đẹp, mượt" khi số chiều lớn **không tự nó là bằng chứng** rằng hàm mất mát trơn, paper tự nhắc người đọc điều này thay vì giấu đi.

3.4 Mô hình tuyến tính sâu: trường hợp phân tích được bằng tay (§5)

Để có một mô hình toán học đơn giản mô phỏng đúng các hiện tượng định tính quan sát ở mạng thật, paper xét **mạng tuyến tính sâu (deep linear network)**: ghép nhiều phép biến đổi affine đã học, bỏ hết phi tuyến. Tổng hợp các phép biến đổi tuyến tính vẫn là một phép biến đổi tuyến tính (nên mô hình này **không có khả năng biểu diễn** như mạng sâu thật), nhưng vì trọng số bị "phân rã" (factored) thành tích nhiều ma trận, động lực học huấn luyện (shape của hàm mất mát theo tham số) vẫn giống mạng sâu thật: đầu ra tuyến tính theo input, nhưng **phi tuyến theo tham số** - hồi quy tuyến tính thường là lồi, còn hồi quy tuyến tính "sâu" (tham số phân rã) lại **không lồi**.

Mô hình đơn giản nhất: $y = w_1 w_2 x$, huấn luyện để $y = 1$ khi $x = 1$ bằng mean squared error, cho:

$$L(w_1, w_2) = (1 - w_1 w_2)^2.$$

Hàm này có saddle point tại gốc $(0, 0)$, và **không có local minima khác chất lượng**: mọi điểm $w_1 w_2 = 1$ đều là cực tiểu toàn cục ($L = 0$), nằm trên một đa tạp (manifold) liên tục - với 1 unit/lớp, đa tạp đó chính là hyperbola $w_2 = 1/w_1$.

3.5 Ví dụ số (tự tính): dấu độ cong dọc đường nội suy của mô hình tuyến tính sâu

Paper không cho sẵn công thức $L(\alpha)$ cụ thể dọc đường nội suy, chỉ minh hoạ bằng hình (Hình 14) với chú thích định tính "độ cong âm gần điểm khởi tạo, độ cong dương gần điểm nghiệm". Dưới đây là phép tính tay (tự tính) để kiểm chứng chính xác câu đó bằng công thức $(1 - w_1 w_2)^2$ ở mục 3.4.

Chọn $\theta_i = (0, 0)$ (gốc, điểm saddle) và $\theta_f = (1, 1)$ (một nghiệm trên hyperbola $w_1 w_2 = 1$). Do $\theta(\alpha) = (1 - \alpha)\theta_i + \alpha\theta_f$, với $\theta_i = (0, 0)$ ta có đơn giản $\theta(\alpha) = (\alpha, \alpha)$. Thay vào L:

$$L(\alpha) = (1 - \alpha \cdot \alpha)^2 = (1 - \alpha^2)^2 = 1 - 2\alpha^2 + \alpha^4.$$

Đạo hàm bậc 1 và bậc 2 theo α (tự tính):

$$L'(\alpha) = -4\alpha + 4\alpha^3 = -4\alpha(1 - \alpha^2), \qquad L''(\alpha) = -4 + 12\alpha^2.$$

Kiểm tra tại $\alpha = 0$ (điểm khởi tạo): $L'(0) = 0$ (đúng là một điểm tới hạn/bằng phẳng, khớp mô tả "flattening near $\alpha = 0$" ở nhiều hình khác của paper, ví dụ Hình 9), và $L''(0) = -4 < 0$ (**độ cong âm**, khớp đúng chú thích Hình 14 "negative curvature near its initial point"). Kiểm tra tại $\alpha = 1$ (điểm nghiệm): $L''(1) = -4 + 12 = 8 > 0$ (**độ cong dương**, khớp đúng "positive curvature near its final point"). Tự tính thêm: độ cong đổi dấu khi $L''(\alpha) = 0 \Leftrightarrow \alpha^2 = 1/3 \Leftrightarrow \alpha = 1/\sqrt{3} \approx 0{,}577$ - đây là một điểm uốn cụ thể mà paper không nêu số, suy ra trực tiếp từ công thức của chính paper.

4. Thí nghiệm và kết quả chính

Ghi chú: đây là một paper thuần thực nghiệm/trực quan hoá, không có theorem hội tụ hay regret bound (khác hẳn opt 06-09); "kết quả chính" ở đây là các quan sát định tính nhất quán trên 7 mô hình, cùng một vài số liệu định lượng phụ (lỗi test) để xác nhận các mô hình thật sự đạt chất lượng SOTA tại thời điểm đó.

Mạng fully-connected trên MNIST (§3, Hình 1-4): so sánh 4 cấu hình kích hoạt: maxout có huấn luyện đối kháng (adversarial training, SOTA lúc đó, lỗi test trung bình 78,2/10.000), maxout không đối kháng (94/10.000 lỗi), sigmoid (lỗi test 1,66%), và ReLU. Cả 4 đều cho đường nội suy trơn, gần lồi; cột bên phải Hình 1 cho thấy đối lập rõ: đường cong SGD thật theo thời gian (epoch) **không trơn, phần lớn tiến bộ xảy ra trong vài epoch đầu, sau đó chật vật tiến thêm** - nhấn mạnh rằng đường nội suy trơn không đồng nghĩa SGD "đi thẳng" theo đường đó. Thí nghiệm độ phân giải cao (Hình 4, chia $[0, 1]$ thành 200 điểm, soi kỹ $[0, 0.01]$ và $[0.99, 1]$) không phát hiện thêm cấu trúc nhỏ khó khăn nào.

So sánh hai nghiệm khác nhau (Hình 5): nội suy giữa hai nghiệm SGD từ hai random seed khác nhau cho thấy **có một rào cản** (chi phí cao hơn) ở giữa - hai nghiệm này là hai điểm cực tiểu khác nhau trong lát cắt 1 chiều này, nhưng paper diễn giải đây là do chúng phá vỡ đối xứng theo hai cách khác nhau, không phải vì chất lượng nghiệm khác nhau. Ngược lại, nội suy từ một điểm **ngẫu nhiên** (không phải nghiệm) đến một nghiệm SGD **không** tìm thấy local minimum nào khác ngoài chính nghiệm đó, gợi ý cực tiểu cục bộ hiếm gặp trong không gian tham số nói chung.

Mạng tích chập trên CIFAR-10 (Hình 6): nhìn tổng thể trơn, nhưng khi soi gần điểm khởi tạo có một rào cản **nông nhưng rộng** - paper suy đoán có thể do khởi tạo trọng số ngẫu nhiên quá lớn.

Mô hình sinh MP-DBM (Hình 7): phát hiện một cực tiểu cục bộ phụ, lồi cao, nằm ngay trên lát cắt 1 chiều chính - nhưng khi vẽ quỹ đạo SGD thật (toạ độ α, β), SGD đi **vòng xa** khỏi điểm này, không hề bị ảnh hưởng trong thực tế.

LSTM trên Penn Treebank (Hình 8): không phát hiện cấu trúc khó nào - cho thấy các đặc điểm ”exotic” của tối ưu hoá không lồi dường như cũng không gây khó khăn cho mô hình hồi quy tuần tự.

Mô hình tuyến tính sâu (§5, Hình 9-10): nội suy từ khởi tạo nhỏ ngẫu nhiên tới nghiệm cho cùng đặc điểm định tính như các mạng thật (bằng phẳng gần saddle point tại gốc, một cực tiểu duy nhất trong lát cắt). Nội suy giữa **hai nghiệm khác nhau** của mô hình tuyến tính sâu (Hình 10) cũng cho một rào cản ở giữa, dù cả hai nghiệm cùng nằm trên một đa tạp cực tiểu toàn cục liên tục - khớp hoàn toàn với hiện tượng quan sát ở mạng thật (Hình 5), nhưng lần này **giải thích được chính xác bằng công thức giải tích** (mục 3.4-3.5).

5. Phân tích / ablation

Paper không có ”ablation” theo nghĩa cổ điển (bỏ một thành phần mô hình/optimizer), nhưng có một loạt kiểm định tính đúng đắn/độ tin cậy của chính phương pháp nội suy, đóng vai trò tương đương:

- **Kiểm định bằng random walk (Phụ lục B, Hình 11-12, đã nêu ở mục 3.3):** xác nhận hình dạng (α, β) ”mượt” có thể chỉ là hiệu ứng số chiều cao, không chắc là bằng chứng về cấu trúc hàm mất mát.
- **Kiểm định bằng subspace ngẫu nhiên (Phụ lục D, Hình 21-22):** nếu chọn một mặt phẳng 2 chiều **ngẫu nhiên** đi qua điểm nghiệm (thay vì mặt phẳng được chọn có chủ đích như mục 3.2), hàm mất mát gần như **không biến thiên gì**, còn quỹ đạo SGD thật trông **rất nhiễu, phức tạp**. Paper tự giải thích: các hình 3D ”đẹp” trong phần chính thực ra không phải một mặt phẳng 2 chiều cố định, mà là **nhiều lát cắt 1 chiều xoay song song với nhau**, được chọn có chủ đích để lộ biến thiên - nên không nên suy rộng rằng ”mọi lát cắt của bề mặt mất mát neural network đều trơn”.
- **Phản ví dụ paper tự tìm ra (Phụ lục C, Hình 18-19):** mạng maxout huấn luyện đối kháng **có** một vật cản thật: hẹp (narrow) nhưng dốc (steep), ép quỹ đạo SGD đi trong một ”hẻm núi” hẹp gần cuối quá trình huấn luyện. Vì vậy tuyên bố chính của paper **không phải** ”không bao giờ có vật cản nào”, mà chính xác hơn là: ”không tìm thấy bằng chứng local minima hoặc saddle point làm **chậm** quỹ đạo SGD” - có thể tồn tại vật cản dạng khác (ravine/canyon hẹp) mà không nhất thiết là local minimum hay saddle point cổ điển.
- **Lý giải khả năng né saddle point bằng khai triển Taylor liên tục theo thời gian (Phụ lục C):** với gradient descent liên tục $\frac{d}{dt}\theta = -\nabla_\theta J(\theta)$, khai triển Taylor bậc hai theo thời gian quanh $t = 0$ cho $\theta(t) \approx \theta(0) - t\nabla_{\theta(0)}J(\theta(0)) + \frac{1}{2}t^2 H(0)\nabla_{\theta(0)}J(\theta(0))$, trong đó H là ma trận Hessian. Paper chỉ ra số hạng bậc hai này **tương đương việc đi theo hướng gradient của** $\|\nabla_\theta J(\theta)\|^2$ - nói cách khác, số hạng bậc một bảo ”đi xuống dốc”, còn số hạng bậc hai bảo ”làm gradient lớn hơn”, tức khuyến khích đi theo hướng **độ cong âm**, giúp né khỏi saddle point một cách tự nhiên. Đây chỉ là **một cách giải thích khả dĩ**, không phải một chứng minh chặt, paper dùng từ ”one possible explanation”.

6. Giới hạn

- **Chỉ xét hàm mất mát tổng, không phải xấp xỉ minibatch thật mà SGD dùng** (paper tự nêu rõ ngay phần mở đầu) - khó khăn thật của SGD có thể nằm ở cấu trúc minibatch riêng lẻ hoặc nhiễu lấy mẫu, không nằm ở hình dạng hàm mất mát tổng mà bài này khảo sát.
- **Nguy cơ survivorship bias, paper tự thừa nhận ở phần Discussion:** 7 mô hình được chọn **vì đã đạt kết quả SOTA** trên benchmark cạnh tranh - có khả năng quá trình dò hyperparameter đã (vô tình) lọc ra chính các cấu hình ”dễ” cho SGD tối ưu hoá, không đại diện cho mọi lựa chọn hyperparameter có thể có.

Paper nêu rõ: ”It is possible that these neural networks perform well because extensive hyperparameter search has found problems that SGD is able to optimize easily.”

- **Chỉ là một lát cắt 1 chiều (hoặc 2 chiều với β) của một không gian rất nhiều chiều** - không nói gì chắc chắn về cấu trúc ngoài lát cắt đó; mục 5 (Phân tích) cho thấy chính paper cũng tự cảnh báo điều này bằng thí nghiệm kiểm soát.
- **Không phải mọi mô hình đều ”sạch”:** mạng maxout đối kháng có vật cản hẹp/dốc thật; MP-DBM có cực tiểu cục bộ phụ nằm ngay trên lát cắt chính (dù SGD tránh được trong thực tế). Nên phát biểu đúng của paper là có sắc thái, không tuyệt đối.
- **Không có bảo đảm toán học tổng quát, không có theorem/regret bound** - khác hẳn opt 06 (Theorem 4.1), opt 08 (Theorem 4, Corollary 1). Phần giải tích duy nhất (mục 3.4-3.5) chỉ áp dụng cho một mô hình tuyến tính sâu đơn giản hoá (1 unit mỗi lớp), không tổng quát hoá hình thức cho mạng sâu phi tuyến thật.
- **Dữ liệu/mô hình đã cũ (2014):** MNIST, CIFAR-10 thời early CNN/maxout, LSTM nhỏ trên Penn Treebank - không có Transformer, không có mô hình quy mô lớn hiện đại. Việc ngoại suy sang kiến trúc hybrid linear-attention/full-attention hiện đại (liên quan dự án, mục 8) hoàn toàn là suy luận, chưa có bằng chứng thực nghiệm trong paper.

7. Liên hệ với các bài khác

- **pruning 09 (Linear Mode Connectivity and the Lottery Ticket Hypothesis, Frankle et al. 2019/2020):** đây là liên hệ trực tiếp và rõ nhất. Ghi chú pruning 09 mô tả ”mode connectivity”: hai nghiệm huấn luyện độc lập được gọi là **kết nối tuyến tính (linearly connected)** nếu trên đoạn thẳng nối hai bộ trọng số, lỗi không có ”rào cản” (barrier) tăng lên - đây **chính xác** là câu hỏi Hình 5 và Hình 10 của bài này (opt 05) đặt ra năm 2014/2015 (nội suy giữa hai nghiệm SGD khác seed, hoặc hai nghiệm của mô hình tuyến tính sâu, tìm rào cản ở giữa). Ghi chú pruning 09 còn nhắc tới ”mode connectivity (Draxler 2018, Garipov 2018): các mạng train từ khởi tạo khác nhau nối được bằng đường phi tuyến có lỗi hằng, nhưng không nối được bằng đường thẳng” - cùng hiện tượng rào cản trên đường thẳng mà opt 05 quan sát trực tiếp trước đó. Bài opt 05 không được pruning 09 trích dẫn tên trực tiếp trong đoạn đã kiểm tra, nên đây là một liên hệ về **chung kỹ thuật và chung hiện tượng** (nội suy tuyến tính giữa hai nghiệm, rào cản/barrier), không phải một trích dẫn xác nhận trực tiếp giữa hai bài.
- **pruning 08 (Lottery Ticket Hypothesis, Frankle & Carbin 2018):** pruning 09 (vừa nêu trên) được viết như một lời giải thích mở rộng trực tiếp cho pruning 08; gián tiếp, opt 05 cung cấp một trong những quan sát nền tảng sớm nhất (nội suy giữa khởi tạo và nghiệm là trơn; nội suy giữa hai nghiệm khác nhau có rào cản) mà dòng nghiên cứu lottery-ticket/mode-connectivity sau này dùng để lý giải vì sao reset về đúng khởi tạo (hoặc một điểm rewind sớm) lại quan trọng.
- **opt 04 (Su, Boyd, Candès - góc nhìn ODE cho Nesterov’s accelerated gradient, cùng batch, chưa đọc chi tiết tại thời điểm viết ghi chú này):** theo tiêu đề và mô tả nhiệm vụ, opt 04 cũng dùng góc nhìn continuous-time/ODE để phân tích động lực tối ưu hoá, giống đúng kiểu lập luận continuous-time Taylor-expansion ở mục 3.3/5 của bài này (opt 05) khi giải thích vì sao SGD né được saddle point. Hai bài dùng công cụ toán tương tự (giới hạn continuous-time của một thuật toán rời rạc) nhưng nhắm tới hai câu hỏi khác nhau: opt 04 về tốc độ hội tụ của gia tốc Nesterov, opt 05 về hình dạng định tính của bề mặt mất mát không lồi.
- **opt 02 (Bottou, Curtis, Nocedal - survey lý thuyết tối ưu hoá cho machine learning, cùng batch, chưa đọc chi tiết):** theo mô tả, đây là survey tổng quan về lý thuyết hội tụ của SGD cho các bài toán (bao gồm) không lồi - cung cấp khung lý thuyết (tốc độ hội tụ, điều kiện trên hàm mục tiêu) mà bài opt 05 này không có (opt 05 thuần thực nghiệm/trực quan), nên hai bài bổ sung cho nhau: một cho lý thuyết hội tụ tổng quát, một cho quan sát thực nghiệm cụ thể về hình dạng bề mặt mất mát thật.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là suy luận cho việc post-training/compressing **LightOnOCR-3-0.8B** (VLM OCR, kiến trúc hybrid GatedDeltaNet linear attention + full attention, GPU mục tiêu ≤8GB, batch size = 1 khi inference), không phải nội dung paper tuyên bố. Khác với opt 06-09, bài này **không đề xuất optimizer hay hyperparameter** nào, nên mức độ liên quan trực tiếp tới việc "chọn optimizer" cho dự án là **thấp**; giá trị chính nằm ở một kỹ thuật **chẩn đoán rẻ**:

- **Đề xuất cụ thể:** sau mỗi lượt post-training/fine-tune (LoRA hoặc fine-tune toàn phần quy mô nhỏ) trên LightOnOCR, có thể nội suy tuyến tính giữa checkpoint gốc (pretrained, hoặc SFT) và checkpoint vừa fine-tune, đánh giá loss validation tại một dải α (ví dụ 10-20 điểm). Kỹ thuật này **chỉ cần forward pass** (không cần gradient riêng cho chẩn đoán này), nên tương thích tốt với ràng buộc GPU ≤8GB và với việc đang tối ưu cho inference. Nếu đường cong trơn, đơn điệu (giống đa số mô hình trong paper), gợi ý hai checkpoint nằm trong cùng một "lưu vực" (basin) không có rào cản; nếu xuất hiện rào cản rõ (giống Hình 5/10, hoặc giống pha "hẻm núi" của mạng maxout đối kháng ở Hình 18-19), đó là tín hiệu fine-tune đã di chuyển mô hình sang một vùng khác biệt về chất, có thể đáng xem xét kỹ hơn (ổn định huấn luyện, catastrophic forgetting).
- **Liên hệ cụ thể tới giai đoạn RL/GRPO của LightOnOCR (llm 43):** giai đoạn GRPO xuất phát từ checkpoint pretrain (`-base`/`-bbox-base`) và cập nhật bằng AdamW lr 4×10^{-5}. Theo khung "mode connectivity" (liên hệ mục 7, pruning 09), có thể kiểm tra (suy luận, chưa thử) xem checkpoint sau GRPO có còn **kết nối tuyến tính không rào cản** với checkpoint SFT gốc hay không - nếu có, gợi ý RL chỉ "tinh chỉnh nhẹ" trong cùng vùng nghiệm; nếu không, gợi ý RL đã đưa mô hình sang một vùng tham số khác biệt rõ, điều này có thể liên quan đến các hiện tượng quen thuộc khi RLHF/RLVR làm trôi (drift) quá xa so với policy gốc.
- **Về kiến trúc hybrid GatedDeltaNet:** paper không thử nghiệm kiến trúc attention tuyến tính/gating nào (ngoài paper, kiến thức chung) - không có cơ sở để khẳng định các tham số gating của GatedDeltaNet có hình dạng bề mặt mất mát "trơn" tương tự các mạng fully-connected/CNN/LSTM cũ mà paper khảo sát. Đây là một khoảng trống ngoại suy hoàn toàn, chỉ nên coi là gợi ý thử nghiệm (mục 9), không phải kết luận.
- **Nhận định trung thực:** so với opt 06-09 (trực tiếp quyết định công thức cập nhật tham số khi huấn luyện LightOnOCR), bài này **ít liên quan trực tiếp** đến việc chọn/tinh chỉnh optimizer; giá trị chủ yếu là một công cụ chẩn đoán phụ trợ, rẻ, có thể dùng **sau khi** đã huấn luyện xong một lượt, không phải một thay đổi vào chính quy trình huấn luyện.

9. Thí nghiệm gợi ý

1. **Tái dựng Hình 1-3 ở quy mô nhỏ.** Trên 1 GPU ≤8GB: huấn luyện một MLP nhỏ (2-3 lớp) trên MNIST/Fashion-MNIST bằng SGD, lưu θ_i và θ_f, tính $J(\theta(\alpha))$ (train và validation) tại ~50 điểm $\alpha \in [-0.2, 1.5]$. Đo: hình dạng đường cong. Kỳ vọng theo paper: trơn, gần lồi, bằng phẳng gần $\alpha = 0$. Diễn giải: nếu tái hiện được ở quy mô rất nhỏ hiện đại, xác nhận tính phổ quát của hiện tượng; nếu không, gợi ý hiện tượng nhạy với kiến trúc/khởi tạo cụ thể.
2. **Đối chiếu đường nội suy (trơn) với đường cong SGD thật theo thời gian (bumpy), tái dựng đối lập 2 cột của Hình 1.** Trên cùng mô hình ở thí nghiệm 1: vẽ loss theo epoch thật của SGD, cạnh đường $J(\theta(\alpha))$. Đo: SGD "di chuyển" bao nhiêu dọc trục α sau mỗi epoch thật (chiếu $\theta(t)$ lên đường thẳng). Diễn giải: định lượng SGD "lãng phí" bao nhiêu bước để lệch khỏi đường thẳng ngắn nhất, minh hoạ khoảng cách giữa "có một con đường trơn tồn tại" và "optimizer tìm thấy/đi theo con đường đó nhanh".
3. **Tái dựng mô hình tuyến tính sâu bằng tay (mục 3.4-3.5), kiểm chứng dấu độ cong.** Cài đặt PyTorch 2 tham số scalar w_1, w_2, $L = (1 - w_1 w_2)^2$, khởi tạo gần gốc (ví dụ $w_1 = w_2 = 0{,}01$, phá vỡ đối xứng nhẹ), chạy gradient descent thường, đồng thời vẽ heatmap $L(w_1, w_2)$ (tái dựng Hình 14 ở quy mô tối giản). Đo: quỹ đạo có đi gần saddle point tại gốc không, dấu độ cong dọc quỹ đạo có khớp phép tính tay ở mục 3.5 (âm gần gốc, dương gần nghiệm, đổi dấu tại $\alpha \approx 1/\sqrt{3}$) không. Diễn giải: kiểm chứng

trực tiếp, rẻ, phần giải tích duy nhất của toàn bài.

4. **Kiểm định bằng subspace ngẫu nhiên (tái dựng Phụ lục D, Hình 21).** Trên mô hình ở thí nghiệm 1: ngoài đường $\theta_i \to \theta_f$, chọn thêm một hướng 1 chiều **ngẫu nhiên** đi qua θ_f, vẽ J dọc hướng đó. Đo: độ biến thiên và độ nhiễu so với đường $\theta_i \to \theta_f$. Diễn giải: kiểm tra xem hiện tượng "trơn" có đặc thù cho đúng lát cắt được chọn (như paper tự cảnh báo) hay là tính chất chung của mọi hướng gần nghiệm.
5. **Kiểm tra "kết nối tuyến tính" giữa hai nghiệm cùng khởi tạo, khác nhiễu SGD (liên hệ mục 7, pruning 09).** Trên mô hình ở thí nghiệm 1: huấn luyện hai lần từ **cùng** θ_i, chỉ đổi thứ tự minibatch (seed khác), được $\theta_f^{(1)}, \theta_f^{(2)}$; nội suy giữa hai nghiệm này (tái dựng ý tưởng Hình 5, nhưng kiểm soát chặt hơn: cùng khởi tạo, chỉ khác nhiễu SGD, đúng khung "instability analysis" của pruning 09). Đo: có rào cản ở giữa không. Diễn giải: nếu không có rào cản (nghiệm "ổn định" với nhiễu SGD theo định nghĩa pruning 09), đó là tín hiệu tốt cho việc một lượt fine-tune ngắn của LightOnOCR sẽ cho kết quả ổn định, ít phụ thuộc seed; nếu có rào cản rõ, gợi ý nên cẩn trọng hơn khi so sánh các lần fine-tune khác seed.

10. Tóm tắt công thức / khái niệm

Khái niệm / công thức	Ý nghĩa
$\theta(\alpha) = (1-\alpha)\theta_i + \alpha\theta_f$	Nội suy tuyến tính giữa điểm khởi tạo θ_i và điểm nghiệm θ_f; công thức trung tâm của toàn bài
$\alpha(t) = (\theta(t) - \theta_i)^\top u$, $\beta(t) = (\theta(t) - \theta_i - \alpha(t)u)^\top v$ (Phụ lục B)	Hai toạ độ: vị trí dọc đường thẳng và phần dư vuông góc, dùng để đo SGD lệch khỏi đường thẳng bao nhiêu
$L(w_1, w_2) = (1 - w_1 w_2)^2$	Mô hình tuyến tính sâu tối giản (1 unit/lớp); saddle tại gốc, cực tiểu toàn cục trên hyperbola $w_1 w_2 = 1$
$L(\alpha) = (1-\alpha^2)^2$, $L'(\alpha) = -4\alpha(1-\alpha^2)$, $L''(\alpha) = -4 + 12\alpha^2$ (tự tính, dọc $\theta_i = (0,0) \to \theta_f = (1,1)$)	$L''(0) = -4 < 0$ (cong âm tại khởi tạo), $L''(1) = 8 > 0$ (cong dương tại nghiệm), đổi dấu tại $\alpha = 1/\sqrt{3} \approx 0{,}577$ (tự tính)
$\theta(t) \approx \theta(0) - t\nabla J(\theta(0)) + \frac{1}{2}t^2 H(0)\nabla J(\theta(0))$	Khai triển Taylor bậc 2 theo thời gian của gradient descent liên tục; số hạng bậc 2 tương đương ascend gradient của $\|\nabla J\|^2$, gợi ý cơ chế né saddle point
7 mô hình thử nghiệm	Fully-connected (maxout±adversarial, sigmoid, ReLU) trên MNIST; CNN maxout trên CIFAR-10; MP-DBM (generative); LSTM trên Penn Treebank; mô hình tuyến tính sâu (phân tích được)
Rào cản (barrier) giữa hai nghiệm khác nhau (Hình 5, Hình 10)	Hiện tượng tiền thân của "linear mode connectivity" (liên hệ mục 7, pruning 09)

11. Kết luận ngắn

Bằng một kỹ thuật rất đơn giản (nội suy tuyến tính $\theta(\alpha)$ giữa điểm khởi tạo và điểm nghiệm, chỉ cần forward pass), paper cho thấy trên 7 mô hình neural network khác loại đã đạt kết quả SOTA (feedforward, CNN, generative, LSTM, và một mô hình tuyến tính sâu phân tích được bằng tay), đường cắt 1 chiều của hàm mất mát từ khởi tạo tới nghiệm gần như luôn trơn, gần lồi, không có local minima/saddle point làm SGD chậm lại một cách đáng kể, ngoại trừ một vùng bằng phẳng gần khởi tạo do phá vỡ đối xứng sớm. Paper tự kiểm định cẩn thận phương pháp của mình (thí nghiệm kiểm soát bằng random walk và subspace ngẫu nhiên) và tự tìm ra ít nhất một phản ví dụ (mạng maxout đối kháng có vật cản hẹp thật) - một mẫu mực về cách trình bày trung thực giới hạn của phương pháp. Với dự án LightOnOCR, giá trị thực dụng chính (suy luận, mục 8) là một công cụ chẩn đoán rẻ, dùng sau khi fine-tune, để kiểm tra checkpoint mới có còn "kết nối tuyến tính không rào cản" với checkpoint gốc hay không, hơn là một thay đổi trực tiếp vào công thức huấn luyện/optimizer.

Nguồn gốc: https://arxiv.org/abs/1412.6544

Bài 06

Adam: A Method for Stochastic Optimization (2014)

Tác giả: Diederik P. Kingma (University of Amsterdam, OpenAI), Jimmy Lei Ba (University of Toronto).
Venue / arXiv: arXiv:1412.6980 (v9, 30/01/2017; bản đầu tiên nộp tháng 12/2014, trang đầu PDF ghi ”Published as a conference paper at ICLR 2015”).
Nguồn: https://arxiv.org/abs/1412.6980
Chủ đề: thuật toán tối ưu bậc một Adam (viết tắt của ”adaptive moment estimation”): kết hợp momentum (ước lượng moment bậc 1 của gradient) với learning rate thích ứng theo từng tham số (ước lượng moment bậc 2, kiểu RMSProp/AdaGrad), có bias-correction cho cả hai ước lượng; kèm biến thể AdaMax dựa trên norm vô cùng.
Vị trí trong lộ trình: đọc sau opt 03 (Ruder, tổng quan các optimizer đi trước: SGD, momentum, NAG, Adagrad, Adadelta, RMSprop, đặt Adam vào bối cảnh) và opt 05 (Goodfellow, quan sát định tính landscape tối ưu của neural network, lý do cần optimizer thích ứng hơn GD trơn). Đọc trước opt 07 (AdamW, sửa cách Adam xử lý weight decay), opt 08 (AMSGrad, chỉ ra Adam có thể không hội tụ trong một số trường hợp) và opt 09 (RAdam, sửa vấn đề variance/warmup ở các bước đầu): ba bài này đều sửa trực tiếp các nhược điểm cụ thể của chính bài này.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Tối ưu hoá bậc một dựa trên gradient là công cụ trung tâm của machine learning: hàm mục tiêu thường là **ngẫu nhiên** (stochastic) vì được tính trên các minibatch dữ liệu con, hoặc có nhiễu từ các nguồn khác như dropout. Khi không gian tham số có số chiều rất lớn, các phương pháp bậc hai (dùng Hessian) quá tốn kém, nên paper giới hạn vào **phương pháp bậc một** (chỉ dùng gradient).

Hai phương pháp thích ứng phổ biến trước Adam (paper gọi là nguồn cảm hứng trực tiếp):

- **AdaGrad** (Duchi, Hazan, Singer 2011): chia learning rate cho $\sqrt{\sum_{i=1}^{t} g_i^2}$ tích lũy từ đầu, hoạt động tốt với gradient **thưa** (sparse).
- **RMSProp** (Tieleman & Hinton, 2012): dùng moving average của g^2 (không tích lũy toàn bộ lịch sử) nên hoạt động tốt trong bối cảnh **online, không tĩnh** (non-stationary).

Câu hỏi đặt ra: có thể kết hợp ưu điểm của cả hai (sparse-friendly của AdaGrad, non-stationary-friendly của RMSProp) và thêm momentum, trong một thuật toán đơn giản, ít cần chỉnh tay, tốn ít bộ nhớ, không?

2. Ý tưởng chính

Adam duy trì hai **exponential moving average (EMA)**:

- m_t: EMA của gradient g_t (ước lượng moment bậc 1, tức giá trị trung bình, giống momentum).
- v_t: EMA của g_t^2 (ước lượng moment bậc 2 thô, tức ”uncentered variance”, giống RMSProp).

Vì m_t, v_t được khởi tạo bằng 0, chúng bị **lệch (bias) về 0** ở các bước đầu, đặc biệt khi hệ số decay gần 1. Adam sửa bằng **bias-correction**: chia m_t, v_t cho $(1-\beta_1^t)$, $(1-\beta_2^t)$ để được ước lượng không lệch (tại stationary). Bước cập nhật tham số là m đã sửa lệch, chia cho căn bậc hai của v đã sửa lệch, nhân với learning rate α. Tên ”Adam” = ”adaptive moment estimation”: ước lượng moment thích ứng.

3. Phương pháp chi tiết

3.1 Thuật toán (Algorithm 1)

Với $g_t = \nabla_\theta f_t(\theta_{t-1})$, Adam cập nhật (paper dùng $\odot$ ngầm cho mọi phép toán, là element-wise):

$$
\begin{aligned}
m_t &= \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t \\
v_t &= \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2 \\
\hat{m}_t &= m_t / (1-\beta_1^t) \\
\hat{v}_t &= v_t / (1-\beta_2^t) \\
\theta_t &= \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)
\end{aligned}
$$

Giá trị mặc định do paper đề xuất (nguyên văn, cho ”các bài toán ML đã thử”): $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Paper cũng nêu dạng tính toán hiệu quả hơn (ít rõ hơn) bằng cách gộp hệ số: $\alpha_t = \alpha \cdot \sqrt{1-\beta_2^t}/(1-\beta_1^t)$ rồi $\theta_t \leftarrow \theta_{t-1} - \alpha_t \cdot m_t/(\sqrt{v_t} + \hat{\epsilon})$ (tránh tính $\hat{m}_t, \hat{v}_t$ tường minh).

3.2 Tính chất của bước cập nhật (§2.1)

Đặt $\Delta_t = \alpha \cdot \hat{m}_t/\sqrt{\hat{v}_t}$ (giả sử $\epsilon = 0$). Paper chỉ ra hai chặn trên: $|\Delta_t| \le \alpha \cdot (1-\beta_1)/\sqrt{1-\beta_2}$ khi $(1-\beta_1) > \sqrt{1-\beta_2}$, và $|\Delta_t| \le \alpha$ trong trường hợp còn lại (trường hợp đầu chỉ xảy ra khi gradient bằng 0 ở mọi bước trừ bước hiện tại, tức sparsity cực đoan). Trong đa số tình huống thực tế $\hat{m}_t/\sqrt{\hat{v}_t} \approx \pm 1$ (vì $|\mathbb{E}[g]/\sqrt{\mathbb{E}[g^2]}| \le 1$), nên $|\Delta_t| \lesssim \alpha$: α đóng vai trò **trust region** quanh tham số hiện tại. Paper gọi $SNR_t = \hat{m}_t/\sqrt{\hat{v}_t}$ là ”signal-to-noise ratio”: SNR nhỏ (gần 0) nghĩa là hướng của $\hat{m}_t$ không chắc khớp hướng gradient thật, bước đi tự động co lại, tạo hiệu ứng **annealing tự động** khi gần optimum. Một tính chất khác: bước cập nhật **bất biến với việc rescale gradient** theo hằng số c, vì $\hat{m}_t$ scale theo c còn $\sqrt{\hat{v}_t}$ scale theo c (từ $\sqrt{c^2}$), hai hệ số này triệt tiêu nhau.

3.3 Bias-correction (§3), với ví dụ số tự tính

Vì $v_0 = 0$, khai triển $v_t = (1-\beta_2)\sum_{i=1}^{t} \beta_2^{t-i} g_i^2$ (công thức 1). Lấy kỳ vọng và giả sử g_i^2 có kỳ vọng xấp xỉ hằng số $\mathbb{E}[g_t^2]$ (dừng, stationary) thì:

$$
\mathbb{E}[v_t] = \mathbb{E}[g_t^2] \cdot (1-\beta_2) \sum_{i=1}^{t} \beta_2^{t-i} + \zeta = \mathbb{E}[g_t^2] \cdot (1-\beta_2^t) + \zeta
$$

với $\zeta = 0$ nếu $\mathbb{E}[g_i^2]$ không đổi theo i. Hệ số $(1-\beta_2^t)$ là độ lệch do khởi tạo 0; chia v_t cho đúng hệ số này triệt tiêu lệch. Lập luận cho m_t hoàn toàn tương tự.

(Tự tính) Với $\beta_2 = 0.999$ (mặc định), hệ số sửa lệch $(1-\beta_2^t)$ theo t:

t	β_2^t	$1-\beta_2^t$	Hệ số khuếch đại $1/(1-\beta_2^t)$
1	0.999	0.001	1000
10	≈ 0.990	≈ 0.00996	$\approx 100,4$
100	≈ 0.9048	≈ 0.0952	$\approx 10,5$
1000	≈ 0.3677	≈ 0.6323	$\approx 1,58$
10000	$\approx 4,5 \times 10^{-5}$	≈ 0.99995	$\approx 1,00005$

Ở bước $t = 1$, nếu không sửa lệch, $v_1 = (1-\beta_2)g_1^2 = 0.001 \cdot g_1^2$ chỉ bằng 0.1% giá trị thật của g_1^2; sửa lệch nhân lại đúng 1000 lần để $\hat{v}_1 = g_1^2$. Đây là lý do paper nhấn mạnh: với β_2 gần 1 (cần cho gradient thưa), thiếu bias-correction sẽ làm bước đầu **quá lớn** (vì $\sqrt{\hat{v}_t}$ ở mẫu bị đánh giá thấp).

3.4 Phân tích hội tụ (§4, định lý 4.1)

Dùng khung **online convex optimization** (Zinkevich 2003): dãy hàm lồi $f_1, \ldots, f_T$ không biết trước, regret $R(T) = \sum_{t=1}^{T}[f_t(\theta_t) - f_t(\theta^*)]$ với $\theta^* = \arg\min_{\theta \in \mathcal{X}} \sum_t f_t(\theta)$ (công thức 5). Đặt $\gamma \triangleq \beta_1^2/\sqrt{\beta_2}$ (paper viết $\gamma \triangleq \beta_1/\sqrt{\beta_2}$ ở định nghĩa đầu dòng 235-236 nhưng dùng với mũ 2 trong điều kiện định lý "$\beta_1^2/\sqrt{\beta_2} < 1$"; xem mục 6 về điểm không khớp nhỏ này). Giả sử gradient chặn ($\|\nabla f_t(\theta)\|_2 \le G$, $\|\nabla f_t(\theta)\|_\infty \le G_\infty$), khoảng cách tham số chặn ($\|\theta_n - \theta_m\| \le D, D_\infty$), learning rate giảm theo $\alpha_t = \alpha/\sqrt{t}$, và $\beta_{1,t} = \beta_1 \lambda^{t-1}$ giảm dần theo $\lambda \in (0,1)$ (ví dụ $\lambda = 1 - 10^{-8}$):

Định lý 4.1 (regret bound):

$$\boxed{R(T) \le \sum_{i=1}^{d} \frac{D^2}{2\alpha(1-\beta_1)} \sqrt{T\hat{v}_{T,i}} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 + \sum_{i=1}^{d} \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha(1-\beta_1)(1-\lambda)^2}}$$

Đây là bound $O(\sqrt{T})$ theo T (số hạng thứ hai, chứa $\|g_{1:T,i}\|_2 \le G_\infty\sqrt{T}$, chiếm phần chính), **tương đương bậc** với bound tốt nhất đã biết cho online convex optimization; khi dữ liệu thưa (sparse), tổng $\sum_i \|g_{1:T,i}\|_2$ có thể nhỏ hơn hẳn chặn trên $dG_\infty\sqrt{T}$, cho Adam (và AdaGrad) đạt $O(\log d\sqrt{T})$ thay vì $O(\sqrt{dT})$ của phương pháp không thích ứng.

Hệ quả 4.2: $R(T)/T = O(1/\sqrt{T}) \to 0$ khi $T \to \infty$, suy trực tiếp từ định lý 4.1.

Chứng minh đầy đủ ở Phụ lục (định nghĩa hàm lồi, Lemma 10.2-10.4, Định lý 10.5 lặp lại định lý 4.1): dùng quy nạp trên T và khai triển bất đẳng thức Young $ab \le a^2/2 + b^2/2$; không chép lại toàn bộ các bước đại số ở đây vì không thêm trực giác, chỉ giữ phát biểu và giả thiết.

3.5 So sánh với RMSProp và AdaGrad (§5)

- **RMSProp + momentum:** khác Adam ở chỗ momentum áp lên gradient đã rescale (chia $\sqrt{v_t}$) rồi mới cộng, còn Adam ước lượng trực tiếp EMA của m_t và v_t riêng; RMSProp **không có bias-correction**, gây bước đầu quá lớn (có thể phân kỳ) khi β_2 gần 1, như thực nghiệm ở mục 4 dưới xác nhận.
- **AdaGrad:** là trường hợp riêng của Adam khi $\beta_1 = 0$, $(1-\beta_2)$ vô cùng nhỏ, và α được thay bằng $\alpha_t = \alpha \cdot t^{-1/2}$: paper chứng minh $\lim_{\beta_2 \to 1} \hat{v}_t = t^{-1}\sum_{i=1}^{t} g_i^2$, khớp đúng mẫu số của AdaGrad. Tương ứng này **không còn đúng** nếu bỏ bias-correction (như ở RMSProp): khi đó $\beta_2 \to 1$ sẽ gây bias vô hạn, bước cập nhật vô hạn.
- Adam cũng được ví như một dạng **natural gradient descent** (Amari 1998) vì $\hat{v}_t$ xấp xỉ đường chéo ma trận Fisher information, nhưng "thận trọng" hơn NGD thuần vì chia cho căn bậc hai nghịch đảo đường chéo đó (giống AdaGrad) thay vì nghịch đảo trực tiếp.

3.6 Mở rộng: AdaMax và temporal averaging (§7)

AdaMax (Algorithm 2): tổng quát hoá chuẩn L_2 của v_t thành L_p: $v_t = \beta_2^p v_{t-1} + (1-\beta_2^p)|g_t|^p$; khi $p \to \infty$, giới hạn $u_t = \lim_{p\to\infty}(v_t)^{1/p}$ có công thức đóng rất đơn giản (paper suy ra qua các bước 6-12):

$$\boxed{u_t = \max(\beta_2 \cdot u_{t-1},\ |g_t|)}$$

không cần bias-correction cho u_t (vì không có hệ số chuẩn hoá $(1-\beta_2^p)$ tích luỹ kiểu EMA cộng). Cập nhật: $\theta_t \leftarrow \theta_{t-1} - (\alpha/(1-\beta_1^t)) \cdot m_t/u_t$. Mặc định đề xuất: $\alpha = 0.002$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. Chặn bước đơn giản hơn Adam: $|\Delta_t| \le \alpha$ (không có hai trường hợp như Adam). Lời cảm ơn (Acknowledgments) của paper ghi rõ: "Thanks to Kai Fan ... for spotting an error in the original AdaMax derivation" , tức bản gốc (trước v9) có lỗi suy diễn AdaMax đã được phát hiện và sửa; bản text đọc ở đây (v9) là bản đã sửa, không có cách nào kiểm tra lại lỗi gốc cụ thể là gì từ chính văn bản này.

Temporal averaging (§7.2): giống Polyak-Ruppert averaging cho SGD (Moulines & Bach 2011): trung bình các tham số dọc theo quá trình train để giảm nhiễu của ”last iterate”. Cài đặt bằng EMA tham số $\bar{\theta}_t \leftarrow \beta_2 \bar{\theta}_{t-1} + (1-\beta_2)\theta_t$ (dùng lại β_2), sửa lệch bằng $\hat{\theta}_t = \bar{\theta}_t/(1-\beta_2^t)$, chỉ cần thêm 1 dòng vào loop của Algorithm 1/2. Paper chỉ nêu ý tưởng, không có thực nghiệm riêng cho phần này.

4. Thí nghiệm và kết quả chính

Tất cả thí nghiệm dùng **cùng cách khởi tạo tham số** giữa các optimizer so sánh; hyperparameter (learning rate, momentum) được ”search over a dense grid”, báo cáo kết quả với **cấu hình tốt nhất** (không phải một lần chạy ngẫu nhiên).

§6.1 Logistic regression (hàm lồi, loại bỏ vấn đề local minimum): L2-regularized multi-class logistic regression.

- **MNIST** (ảnh 784 chiều), minibatch 128, learning rate Adam giảm theo $\alpha_t = \alpha/\sqrt{t}$ (khớp lý thuyết ở mục 3.4): so Adam với SGD+Nesterov momentum và Adagrad. Kết quả (Hình 1): Adam có tốc độ hội tụ **tương đương** SGD+momentum, cả hai nhanh hơn Adagrad.
- **IMDB movie review** (bag-of-words 10.000 từ phổ biến nhất, rất **thưa**, có/không dropout 50% trên feature): Adagrad vượt SGD+Nesterov ”bằng một khoảng cách lớn” cả có và không dropout; **Adam hội tụ nhanh như Adagrad**. Paper diễn giải: giống Adagrad, Adam khai thác được tính thưa của feature để hội tụ nhanh hơn SGD+momentum thường.

§6.2 Multi-layer neural network (MLP 2 tầng ẩn, 1000 unit/tầng, ReLU, minibatch 128, MNIST): so với **SFO** (Sum-of-Functions Optimizer, một phương pháp quasi-Newton theo minibatch). Hình 2: Adam tiến bộ nhanh hơn cả về số iteration và wall-clock time; SFO chậm hơn Adam 5-10 lần mỗi iteration (do chi phí cập nhật thông tin độ cong) và có bộ nhớ tuyến tính theo số minibatch; SFO giả định subfunction tất định nên **thất bại hội tụ** khi có regularization ngẫu nhiên (dropout), còn Adam vẫn hội tụ tốt.

§6.3 CNN trên CIFAR-10 (3 tầng xen kẽ conv 5x5 + max-pool 3x3 stride 2, tiếp theo 1 tầng FC 1000 ReLU, tiền xử lý whitening, dropout ở input và FC, minibatch 128): Hình 3 cho thấy ở vài epoch đầu Adam và Adagrad giảm cost rất nhanh, nhưng về sau Adam và SGD hội tụ nhanh hơn hẳn Adagrad. Paper giải thích: $\hat{v}_t$ ”vanishes to zeros after a few epochs” và bị ϵ chi phối, nên ước lượng moment bậc 2 trở thành xấp xỉ **kém** cho hình học hàm mất mát trong CNN (so với MLP ở §6.2); ngược lại, giảm variance minibatch qua moment bậc 1 (momentum) quan trọng hơn cho tốc độ ở CNN.

§6.4 Ablation bias-correction (Variational Autoencoder, kiến trúc 1 tầng ẩn 500 unit, softplus, latent Gaussian 50 chiều): quét $\beta_1 \in [0, 0.9]$, $\beta_2 \in \{0.99, 0.999, 0.9999\}$, $\log_{10}\alpha \in [-5, \ldots, -1]$. Hình 4 (sau 10 và 100 epoch): β_2 gần 1 (cần cho gradient thưa/bền) **không có** bias-correction gây mất ổn định huấn luyện, nhất là ở các epoch đầu; **có** bias-correction, Adam đạt kết quả bằng hoặc tốt hơn RMSProp **với mọi cấu hình hyperparameter đã thử**.

Paper không có bảng số liệu tổng hợp (loss/accuracy cuối cùng) ngoài các đường cong Hình 1-4; không có benchmark kiểu test accuracy trên tập test được báo cáo tường minh trong phần chính (chỉ ”training cost”/”training negative log likelihood”).

5. Phân tích / ablation

- **SNR và ”trust region” (mục 3.2):** cùng một cơ chế giải thích cả tốc độ hội tụ nhanh ban đầu lẫn khả năng tự giảm bước gần optimum; đây là lý giải trực giác hơn là định lý chặt.
- **Lý thuyết chỉ bao phủ hàm lồi, thực nghiệm toàn phi lồi:** Định lý 4.1/Hệ quả 4.2 chứng minh cho online convex optimization; §6.2, §6.3 dùng MLP và CNN (hàm mất mát không lồi). Paper **tự nêu rõ** ”Although our convergence analysis does not apply to non-convex problems, we empirically found

that Adam often outperforms other methods in such cases" : đây là khoảng cách lý thuyết-thực nghiệm được chính tác giả thừa nhận, không phải điều người ghi chú suy ra.

- ϵ **không chỉ là hằng số tránh chia 0:** thí nghiệm CNN (§6.3) cho thấy ϵ có thể **chi phối** mẫu số khi $\hat{v}_t$ nhỏ, làm thay đổi hành vi optimizer một cách thực chất (không chỉ về số học) tuỳ kiến trúc; đây là điểm thường bị bỏ qua khi chỉ đọc công thức Algorithm 1.
- **Quan hệ AdaGrad chỉ đúng khi có bias-correction** (mục 3.5): nếu một người tự cài lại Adam mà bỏ bias-correction (tưởng là chi tiết phụ), họ sẽ mất cả tính chất khớp AdaGrad lúc $\beta_2 \to 1$ lẫn ổn định ở β_2 gần 1 nói chung, như thực nghiệm §6.4 cho thấy.
- **Đã kiểm chứng trực tiếp trong opt 08 (AMSGrad, "On the Convergence of Adam and Beyond"):** opt 08 xây dựng cụ thể các dãy hàm lồi mà Adam hội tụ tới điểm **tệ nhất** trong tập khả thi (ví dụ $f_t(x) = Cx$ khi $t \bmod 3 = 1$, $f_t(x) = -x$ còn lại, trên $[-1, 1]$), chứng minh average regret của Adam **không** tiến về 0 (Theorem 1, 2 của opt 08), tức Định lý 4.1 ở bài này không đúng như phát biểu. Nguyên văn opt 08 chỉ rõ nguyên nhân: "The main issue in their proof is the incorrect assumption that Γ_t defined in their equation (3) is positive semidefinite, and we also identified problems in lemmas 10.3 and 10.4 in their paper" , tức chính Lemma 10.4 và đại lượng $\Gamma_t = \sqrt{\hat{v}_{t+1}}/\alpha_{t+1} - \sqrt{\hat{v}_t}/\alpha_t$ (ngầm định không âm trong chứng minh Định lý 4.1 ở mục 3.4 trên) có thể **âm** trong thực tế vì EMA của v_t không đơn điệu, phá vỡ chứng minh. Khớp với điểm người ghi chú tự phát hiện ở mục 6 (không khớp định nghĩa γ giữa thân bài và Lemma 10.4): hai điểm này cùng nằm trong phần chứng minh bị opt 08 xác nhận có vấn đề.

6. Giới hạn

Do tác giả nêu:

- Phân tích hội tụ chỉ áp dụng cho bài toán lồi (online convex programming), không chứng minh gì cho phi lồi, dù thực nghiệm dùng MLP/CNN phi lồi.
- Hyperparameter (learning rate, momentum của baseline) được chọn bằng "dense grid search", báo cáo cấu hình tốt nhất: không rõ độ nhạy của Adam với hyperparameter trên các bài toán khác ngoài 4 thí nghiệm đã thử.
- $\hat{v}_t$ có thể "vanish" và bị ϵ chi phối trong CNN (§6.3), làm giảm hiệu quả phần thích ứng theo tham số của Adam trong kiến trúc đó.
- Bản gốc có lỗi trong suy diễn AdaMax (ghi nhận ở Acknowledgments), đã sửa ở bản đọc đây (v9) nhưng không có mô tả cụ thể lỗi là gì.

Người ghi chú nhận xét:

- Ký hiệu γ trong Định lý 4.1 ở thân bài (dòng định nghĩa "$\gamma \triangleq \beta_1/\sqrt{\beta_2}$") và điều kiện của chính định lý đó ("$\beta_1^2/\sqrt{\beta_2} < 1$") dùng hai luỹ thừa khác nhau của β_1 (mũ 1 so với mũ 2); phần Phụ lục (Lemma 10.4) định nghĩa lại $\gamma \triangleq \beta_2/\sqrt{\beta_1}$ (vai trò tử/mẫu đổi chỗ so với thân bài) rồi dùng điều kiện "$\beta_2/\sqrt{\beta_1} < 1$". Đây là một điểm không khớp giữa các lần định nghĩa cùng một ký hiệu trong văn bản gốc. Quan trọng hơn: bài opt 08 (đọc ngay sau, xem mục 5 trên) xác nhận bằng chứng minh phản ví dụ cụ thể rằng toàn bộ Định lý 4.1/chứng minh ở Phụ lục (gồm đúng Lemma 10.4 này) có lỗi thực chất, không chỉ lỗi đánh máy ký hiệu: giả thiết ngầm $\Gamma_t \succeq 0$ dùng trong chứng minh **không** luôn đúng.
- Không có thước đo test accuracy/test loss tường minh, chỉ có training cost: khó đánh giá overfitting hay khả năng tổng quát hoá của Adam so với các baseline từ chính paper này.
- Không có thí nghiệm ở quy mô lớn (theo chuẩn 2014-2015: không có Transformer, không có mô hình > chục triệu tham số); mọi suy luận áp dụng Adam cho mô hình hiện đại (mục 8 dưới) đều là ngoại suy, không phải từ paper.

7. Liên hệ với các bài khác

- **opt 03** (Ruder, overview): đặt Adam trong bối cảnh dòng họ optimizer SGD-momentum, NAG, Adagrad, Adadelta, RMSprop; đọc trước bài này để có bản đồ tổng quan.
- **opt 05** (Goodfellow, qualitatively characterizing...): quan sát định tính landscape tối ưu neural network (không lồi, có saddle point) là một phần động lực cho việc cần optimizer thích ứng mạnh hơn GD trơn.
- **opt 07** (AdamW, Decoupled Weight Decay Regularization): chỉ ra rằng cộng L2 regularization vào gradient trước khi chia $\sqrt{\hat{v}_t}$ (cách thường dùng để thêm "weight decay" vào Adam) làm suy giảm hiệu quả weight decay không đều giữa các tham số; AdamW tách biệt (decouple) hai cơ chế này, sửa trực tiếp một thực hành phổ biến khi dùng chính thuật toán ở bài này.
- **opt 08** (AMSGrad, On the Convergence of Adam and Beyond): phân tích sâu hơn phần hội tụ ở mục 3.4 trên, chỉ ra Adam có thể thất bại hội tụ trong một số bài toán online convex cụ thể, đề xuất sửa bằng cách lấy max tích luỹ của v_t.
- **opt 09** (RAdam, On the Variance of the Adaptive Learning Rate and Beyond): phân tích variance của learning rate thích ứng ở các bước đầu (khi mẫu ít), đề xuất warmup thích ứng thay cho warmup thủ công thường thấy khi dùng Adam cho Transformer.
- **llm 01** (Attention Is All You Need): dùng chính thuật toán Adam ở bài này nhưng với hyperparameter khác mặc định ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$) cộng với lịch learning rate warmup rồi suy giảm theo step$^{-0.5}$, một minh chứng cụ thể rằng $\beta_2 = 0.999$ không phải lựa chọn phổ quát cho mọi kiến trúc.
- **rl 07** (PPO): dùng Adam với stepsize 3×10^{-4} (bảng siêu tham số MuJoCo) làm optimizer mặc định cho cả policy và value network, một ví dụ Adam được dùng "mặc định không cần chỉnh" đúng như kỳ vọng ban đầu của paper này.
- **llm 43** (LightOnOCR, mô hình nền của dự án người dùng): theo ghi chú đã viết trong thư mục llm, mô hình này KHÔNG dùng Adam thuần mà dùng **AdamW** (opt 07) với lr đỉnh 1e-4, cosine schedule, warmup 100 bước cho pretrain, và AdamW lr 4e-5 cho giai đoạn RL (GRPO); đây là bằng chứng trực tiếp rằng biến thể của Adam ở bài này (qua AdamW) là optimizer thực tế của chính mô hình mà dự án người dùng đang nén/hậu huấn luyện.

8. Ứng dụng cho dự án (suy luận)

Dự án người dùng là nén + hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, mỗi trang tài liệu khoảng 3.600 token gồm ~1.500 token text và ~1.600-2.100 token visual, GPU $\leq$ 8 GB, batch size = 1 khi suy luận). Theo đối chiếu ở mục 7, mô hình nền thực tế dùng **AdamW** (biến thể decoupled weight decay của chính Adam), nên mọi suy luận dưới đây là về **cơ chế lõi** của Adam mà AdamW vẫn giữ nguyên (EMA của m_t, v_t, bias-correction), không phải về phần weight decay (xem opt 07 cho phần đó).

1. **Bias-correction quan trọng hơn khi fine-tune/distill với ít bước và batch nhỏ (suy luận).** Mục 3.3 cho thấy hệ số khuếch đại $1/(1-\beta_2^t)$ chỉ giảm về gần 1 sau khoảng vài nghìn bước (với $\beta_2 = 0.999$ mặc định). Nếu quy trình distillation chỉ chạy vài trăm đến vài nghìn bước cập nhật (phổ biến khi fine-tune/distill một mô hình 0.8B trên dữ liệu OCR giới hạn), phần lớn quá trình huấn luyện có thể nằm trong vùng "chưa hội tụ hệ số sửa lệch", tức bias-correction không phải chi tiết phụ mà ảnh hưởng thực chất tới quy mô bước cập nhật xuyên suốt. Gợi ý thực tế: nếu dùng warmup ngắn (ví dụ 100 bước như pretrain gốc), cần kiểm tra bước cập nhật ở các bước đầu không bị quá lớn/quá nhỏ bất thường, đặc biệt nếu đổi β_2 khác mặc định.
2. **Batch = 1 làm gradient rất nhiễu, ảnh hưởng tới cả m_t và v_t (suy luận).** Với batch 1, mỗi g_t là một ước lượng rất nhiễu của gradient thật; theo lập luận SNR ở mục 3.2, $SNR_t = \hat{m}_t/\sqrt{\hat{v}_t}$ sẽ có xu hướng nhỏ và nhiễu hơn so với batch lớn, khiến bước cập nhật tự động co lại (tính chất "annealing tự động" của paper) nhưng cũng làm hướng đi kém tin cậy hơn. Nếu không dùng gradient accumulation, nên theo dõi SNR_t trong log huấn luyện (tính được từ chính $\hat{m}_t, \hat{v}_t$ nội bộ của optimizer) để phát hiện giai đoạn optimizer "gần như chỉ đi theo nhiễu".

3. **Cảnh báo từ thí nghiệm CNN (§6.3): $\hat{v}_t$ có thể vanish và bị ϵ chi phối (suy luận, ngoại suy từ CNN sang hybrid attention/SSM).** Kiến trúc GatedDeltaNet (linear attention dạng gated, tương tự SSM/RNN tuyến tính hoá) có động lực gradient khác tầng full attention; nếu một số nhóm tham số (ví dụ gate của GatedDeltaNet) có gradient rất nhỏ/ổn định theo thời gian, $\hat{v}_t$ của nhóm đó có thể giảm về gần ϵ giống hiện tượng CNN ở bài này, khiến phần "thích ứng" của Adam/AdamW gần như vô hiệu cho nhóm tham số đó. Đây chỉ là một giả thuyết dựa trên loại suy, cần kiểm tra thực nghiệm riêng trên chính mô hình, paper này không nói gì về SSM/linear attention.
4. **Không nên kỳ vọng lại các con số tốc độ hội tụ cụ thể của paper.** Mọi hình 1-4 của paper là trên MNIST/IMDB/CIFAR-10/VAE quy mô rất nhỏ (2014); không dùng trực tiếp làm "ngưỡng tham chiếu" cho VLM OCR 0.8B, chỉ dùng được **cơ chế** (bias-correction, SNR, vai trò ϵ) như phân tích ở trên.

Điều ít liên quan trực tiếp: phần AdaMax (mục 3.6) và temporal averaging không thấy được dùng trong pipeline LightOnOCR theo ghi chú llm 43 đã đối chiếu; phần chứng minh regret bound (mục 3.4) là lý thuyết online-convex, không áp dụng trực tiếp cho huấn luyện phi lồi thực tế.

9. Thí nghiệm gợi ý

1. **Tái lập hiệu ứng bias-correction (ablation của chính §6.4) ở quy mô tối thiểu:** cài Adam bằng tay (không dùng `torch.optim.Adam`) cho logistic regression hoặc MLP nhỏ trên MNIST (CPU hoặc GPU nhỏ), chạy hai phiên bản có/không chia cho $(1-\beta_1^t), (1-\beta_2^t)$, với $\beta_2 \in \{0.99, 0.999, 0.9999\}$. Đo: training loss ở 50-200 bước đầu. Kỳ vọng: bản không bias-correction có bước đầu dao động/tăng loss rõ hơn khi β_2 gần 1, khớp hướng Hình 4 của paper (ở quy mô nhỏ hơn).
2. **Theo dõi SNR theo bước huấn luyện:** trong cùng cài đặt Adam tay ở (1), log $SNR_t = \hat{m}_t/\sqrt{\hat{v}_t}$ (trung bình trị tuyệt đối trên toàn bộ tham số) mỗi bước, so sánh giữa batch size 1 và batch size 32 trên cùng mô hình nhỏ. Đo: phân phối $|SNR_t|$. Kỳ vọng (suy luận ở mục 8.2): batch 1 cho $|SNR_t|$ trung bình thấp hơn/nhiễu hơn ở các bước đầu.
3. **Kiểm tra hiện tượng $\hat{v}_t$ "vanish" trên CNN nhỏ:** huấn luyện một CNN nhỏ (2-3 tầng conv) trên MNIST với Adam, log giá trị trung bình $\hat{v}_t$ của tầng conv đầu theo epoch. Đo: $\hat{v}_t$ có giảm về cỡ $\epsilon = 10^{-8}$ sau vài epoch như paper mô tả cho CIFAR-10 không. Diễn giải: nếu có, xác nhận hiện tượng không chỉ đặc thù CIFAR-10/kiến trúc cụ thể của paper.
4. **So Adam với AdaGrad trên tác vụ sparse-feature tổng hợp (mô phỏng IMDB BoW §6.1):** tạo logistic regression trên feature giả thưa (ví dụ 90% giá trị 0), so tốc độ hội tụ Adam vs Adagrad vs SGD+Nesterov. Đo: training loss theo epoch. Kỳ vọng: Adam và Adagrad hội tụ nhanh hơn SGD+Nesterov rõ rệt, khớp claim §6.1 của paper.
5. **Kiểm chứng công thức AdaMax bằng số:** với một dãy $g_1, \ldots, g_T$ ngẫu nhiên cố định, tính trực tiếp $v_t^{1/p} = \left((1-\beta_2^p)\sum_i \beta_2^{p(t-i)}|g_i|^p\right)^{1/p}$ với p lớn (ví dụ $p = 1000$) và so với công thức rút gọn $u_t = \max(\beta_2 u_{t-1}, |g_t|)$ áp dụng tuần tự. Đo: sai số tuyệt đối giữa hai cách tính. Kỳ vọng: sai số rất nhỏ, xác nhận phép lấy giới hạn $p \to \infty$ ở công thức (6)-(12) của paper là đúng về mặt số học.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
EMA gradient	$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$
EMA gradient bình phương	$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
Bias-correction	$\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$
Cập nhật tham số	$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$
Mặc định Adam	$\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$
Chặn bước (sparsity cực đoan)	$\lvert\Delta_t\rvert \le \alpha(1-\beta_1)/\sqrt{1-\beta_2}$
Chặn bước (thường)	$\lvert\Delta_t\rvert \lesssim \alpha$

Khái niệm	Công thức / giá trị
SNR	$\hat{m}_t/\sqrt{\hat{v}_t}$
Regret bound (Định lý 4.1)	$R(T) = O(\sqrt{T})$, xem công thức đầy đủ ở mục 3.4
Average regret (Hệ quả 4.2)	$R(T)/T = O(1/\sqrt{T}) \to 0$
AdaMax	$u_t = \max(\beta_2 u_{t-1}, \lvert g_t \rvert)$, $\theta_t = \theta_{t-1} - \frac{\alpha}{1-\beta_1^t} \cdot \frac{m_t}{u_t}$, mặc định $\alpha = 0.002$
Temporal averaging	$\bar{\theta}_t = \beta_2 \bar{\theta}_{t-1} + (1-\beta_2)\theta_t$, $\hat{\theta}_t = \bar{\theta}_t/(1-\beta_2^t)$
AdaGrad như trường hợp riêng	$\beta_1 = 0$, $\beta_2 \to 1$, $\alpha_t = \alpha t^{-1/2}$

11. Kết luận ngắn

Adam kết hợp momentum (EMA của gradient) với learning rate thích ứng theo tham số (EMA của gradient bình phương, kiểu RMSProp/AdaGrad), cộng thêm bias-correction để sửa lệch khởi tạo 0 của hai EMA này. Paper chứng minh regret bound $O(\sqrt{T})$ cho online convex optimization và thực nghiệm trên logistic regression, MLP, CNN cho thấy Adam hội tụ nhanh, ổn định, ít cần chỉnh hyperparameter so với AdaGrad/RMSProp/SGD+momentum ở quy mô 2014-2015; ablation xác nhận bias-correction là thành phần thiết yếu, không phải chi tiết phụ. Thuật toán (với các biến thể AdamW, AMSGrad, RAdam ở các bài 07-09 cùng thư mục) trở thành optimizer mặc định cho phần lớn deep learning hiện đại, bao gồm chính mô hình nền của dự án người dùng (qua AdamW, xem llm 43).

Nguồn gốc: https://arxiv.org/abs/1412.6980

Bài 07

Decoupled Weight Decay Regularization (2017)

Tác giả: Ilya Loshchilov, Frank Hutter (University of Freiburg).
Venue / arXiv: arXiv:1711.05101 (v3, 04/01/2019; bản đầu nộp tháng 11/2017); trang đầu PDF ghi ”Published as a conference paper at ICLR 2019”.
Nguồn: https://arxiv.org/abs/1711.05101
Chủ đề: chỉ ra L2 regularization và weight decay **không tương đương** cho các optimizer thích ứng (như Adam), dù tương đương cho SGD; đề xuất AdamW (và SGDW) tách (decouple) bước weight decay khỏi bước cập nhật theo gradient, cộng thêm normalized weight decay và AdamWR (warm restarts).
Vị trí trong lộ trình: đọc ngay sau opt 06 (Adam, thuật toán gốc mà bài này sửa trực tiếp một phần: cách xử lý regularization). Đọc cùng nhóm với opt 08 (AMSGrad) và opt 09 (RAdam), hai bài khác cũng sửa các nhược điểm khác của Adam (hội tụ, warmup) một cách độc lập với sửa đổi ở bài này (weight decay). Có thể đọc trước hoặc sau 08/09 vì ba sửa đổi không phụ thuộc lẫn nhau.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Các optimizer thích ứng (AdaGrad, RMSProp, Adam, AMSGrad) đã trở thành lựa chọn mặc định cho mạng feed-forward và RNN, nhưng kết quả state-of-the-art trên các bộ dữ liệu phân loại ảnh phổ biến (CIFAR-10, CIFAR-100) vẫn thường do **SGD + momentum** đạt được. Wilson et al. (2017, trích trong paper) từng cho rằng optimizer thích ứng tổng quát hoá kém hơn SGD+momentum trên nhiều tác vụ (phân loại ảnh, language modeling ở mức ký tự, constituency parsing), với các giả thuyết như ”sharp local minima” hay ”vấn đề cố hữu của gradient thích ứng”.

Paper đặt câu hỏi cụ thể hơn: nên dùng **L2 regularization** hay **weight decay** khi train với SGD và với Adam? Hai kỹ thuật này được dùng thay thế cho nhau trong hầu hết thư viện deep learning (gọi chung là ”weight decay”), dựa trên một tính chất quen thuộc của SGD (xem Proposition 1 ở mục 3): với SGD, cộng số hạng L2 $\frac{\lambda'}{2}\|\theta\|_2^2$ vào loss **tương đương hoàn toàn** với nhân θ bởi $(1-\lambda)$ mỗi bước (weight decay), miễn $\lambda' = \alpha\lambda$ (α = learning rate). Paper chứng minh tính tương đương này **không còn đúng** cho optimizer thích ứng như Adam.

2. Ý tưởng chính

Với optimizer thích ứng, cập nhật tham số có dạng $\theta_{t+1} \leftarrow \theta_t - \alpha M_t \nabla f_t(\theta_t)$, trong đó M_t là ma trận ”preconditioner” thay đổi theo thời gian (với Adam, $M_t \approx \text{diag}(1/(\sqrt{\hat{v}_t}+\epsilon))$). Nếu cộng L2 regularization vào hàm mục tiêu trước khi lấy gradient, số hạng regularization $\lambda'\theta_t$ **cũng bị nhân với** M_t giống gradient của loss gốc; vì M_t không phải kI (không phải một hằng số nhân đồng nhất mọi chiều), tham số có gradient lịch sử lớn sẽ bị M_t thu nhỏ **cả phần cập nhật loss lẫn phần regularization**, tức bị regularize **ít hơn** tương đối so với tham số có gradient nhỏ. Weight decay ”nguyên bản” (Hanson & Pratt, 1988): $\theta_{t+1} = (1-\lambda)\theta_t - \alpha\nabla f_t(\theta_t)$, lại áp dụng cùng tỉ lệ λ cho **mọi** tham số, không đi qua M_t. Ý tưởng của paper: tách hẳn bước weight decay ra khỏi bước cập nhật dựa-trên-gradient (decoupling), cho ra **AdamW** (và **SGDW** cho SGD, dù với SGD hai cách này tương đương về mặt lý thuyết).

3. Phương pháp chi tiết

3.1 Weight decay nguyên bản và tính tương đương cho SGD (§2, Proposition 1)

Weight decay (Hanson & Pratt, 1988), công thức (1):

$$\theta_{t+1} = (1-\lambda)\theta_t - \alpha\nabla f_t(\theta_t)$$

Proposition 1: SGD chuẩn với learning rate α, chạy trên $f_t^{reg}(\theta) = f_t(\theta) + \frac{\lambda'}{2}\|\theta\|_2^2$ (không weight decay) tạo ra đúng cùng bước cập nhật như SGD chạy trên $f_t(\theta)$ với weight decay λ, khi $\lambda' = \alpha\lambda$. Chứng minh (Phụ lục A, công thức 5-6): cả hai cho $\theta_{t+1} = \theta_t - \alpha\nabla f_t(\theta_t) - \alpha\lambda'\theta_t$, giống nhau khi $\lambda' = \alpha\lambda$.

Điểm hay bị bỏ qua (paper nhấn mạnh): vì $\lambda' = \alpha\lambda$, muốn giữ cùng mức regularization hiệu quả khi đổi α thì phải đổi λ theo cùng tỉ lệ: λ và α **bị khoá chặt** (coupled) với nhau. Đây là nguồn gốc của phần "decoupling" trong tên bài.

(Tự tính) Minh hoạ coupling: giả sử ban đầu $\alpha = 0.01, \lambda = 0.1 \Rightarrow \lambda' = \alpha\lambda = 0.001$. Nếu sau đó giảm α xuống 0.005 (ví dụ do learning rate schedule) nhưng **giữ nguyên** $\lambda = 0.1$ (theo đúng công thức weight decay viết bằng λ), thì λ' mới $= 0.005 \times 0.1 = 0.0005$, **giảm một nửa** dù người dùng không chủ ý giảm cường độ regularization: hệ số L2 hiệu dụng tự động thay đổi theo α. Đây chính là lý do paper nói tìm α, λ tốt cho L2 regularization ở SGD "không tách biệt" được (xem Hình 2 ở mục 4).

3.2 Thuật toán SGDW và AdamW (Algorithm 1 và 2)

Algorithm 1 (SGDW, với momentum β_1, hệ số lịch η_t):

```
cho: alpha, beta1, lambda
khoi tao: t=0, theta_0, m_0=0, eta_0
lap:
  t = t+1
  g_t = grad f_t(theta_{t-1}) + lambda*theta_{t-1}      # (*) dòng này = L2, KHÔNG phải
  ↪  weight decay tách biệt
  eta_t = SetScheduleMultiplier(t)
  m_t = beta1*m_{t-1} + eta_t*alpha*g_t
  theta_t = theta_{t-1} - m_t - eta_t*lambda*theta_{t-1}
```

Dòng cuối cộng thêm $-\eta_t\lambda\theta_{t-1}$ **tách biệt** khỏi m_t: đây chính là bước weight decay decoupled (SGDW). Nếu bỏ dòng cuối và chỉ dùng g_t đã cộng $\lambda\theta_{t-1}$ ở bước (*), ta có SGD với L2 regularization thông thường.

Algorithm 2 (AdamW): mặc định $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$ (giống Adam gốc, opt 06), thêm λ:

```
lap:
  t = t+1
  g_t = grad f_t(theta_{t-1})                 # KHÔNG cộng lambda*theta vào gradient
  m_t = beta1*m_{t-1} + (1-beta1)*g_t
  v_t = beta2*v_{t-1} + (1-beta2)*g_t^2
  m_hat_t = m_t/(1-beta1^t)
  v_hat_t = v_t/(1-beta2^t)
  eta_t = SetScheduleMultiplier(t)
  theta_t = theta_{t-1} - eta_t*( alpha*m_hat_t/(sqrt(v_hat_t)+eps) + lambda*theta_{t-1}
  ↪  )
```

Khác biệt cốt lõi so với "Adam + L2" (cách cài đặt phổ biến trong nhiều thư viện tại thời điểm paper công bố): số hạng $\lambda\theta_{t-1}$ **không** đi vào g_t (nên không bị chia cho $\sqrt{\hat{v}_t}$), mà cộng trực tiếp vào bước cập nhật cuối, nhân với η_t (hệ số lịch) nhưng không nhân với phần thích ứng $1/(\sqrt{\hat{v}_t} + \epsilon)$.

3.3 Vì sao hai cách không tương đương với gradient thích ứng (Proposition 2, 3)

Proposition 2: cho optimizer O tổng quát với cập nhật $\theta_{t+1} \leftarrow \theta_t - \alpha M_t \nabla f_t(\theta_t)$ (không weight decay) và $\theta_{t+1} \leftarrow (1-\lambda)\theta_t - \alpha M_t \nabla f_t(\theta_t)$ (có weight decay), với $M_t \neq kI$. Khi đó **không tồn tại** hệ số L2 λ' nào để chạy O không-weight-decay trên $f_t(\theta) + \frac{\lambda'}{2}\|\theta\|_2^2$ cho ra đúng cùng iterate. Chứng minh (Phụ lục A, công

thức 7-8): điều kiện tương đương đòi hỏi $\lambda\theta_t = \alpha\lambda' M_t\theta_t$ với mọi θ_t, chỉ thoả khi $M_t = kI$, mâu thuẫn giả thiết.

Proposition 3 (trường hợp đặc biệt, preconditioner **cố định** $M_t = \text{diag}(s)^{-1}$, $s_i > 0$): khi đó weight decay λ tương đương L2 regularization trên hàm mất mát đã chỉnh tỉ lệ:

$$f_t^{sreg}(\theta) = f_t(\theta) + \frac{\lambda'}{2}\left\|\theta \odot \sqrt{s}\right\|_2^2, \qquad \lambda' = \alpha\lambda$$

Paper tự nêu rõ: Proposition 3 **không áp dụng trực tiếp** cho optimizer thích ứng thực tế (vì M_t của Adam thay đổi mỗi bước, không cố định), nhưng vẫn cho trực giác: tham số θ_i có nghịch-preconditioner s_i lớn (tương ứng gradient lịch sử lớn ở chiều i) bị regularize **tương đối nhiều hơn** dưới weight decay so với dưới L2 thường; paper phát biểu "regularization tỉ lệ với $\sqrt{s_i}$" (nguyên văn, xem công thức (2) ở trên) khi so hai cơ chế trên cùng thang đo chuẩn hoá của họ, người ghi chú không tự suy diễn lại thêm vì cách định nghĩa "lượng regularize" cụ thể trong câu đó không được paper viết rõ từng bước.

3.4 Biện minh qua Bayesian filtering (§3, theo Aitchison 2018)

Sau khi paper đăng bản sơ bộ lên arXiv, Aitchison (2018) chỉ ra một khung lý thuyết thống nhất: coi tối ưu ngẫu nhiên là một bài toán **Bayesian filtering** theo dõi phân phối hậu nghiệm $P(\theta_t \mid y_{1:t})$ qua các minibatch. Với giả thiết Gaussian cho state-transition và likelihood xấp xỉ liên hợp, công thức cập nhật trung bình hậu nghiệm (công thức 3):

$$\mu_{post} = \mu_{prior} + \Sigma_{post} \times g$$

cho thấy preconditioner chính là độ không chắc chắn hậu nghiệm Σ_{post} (cập nhật lớn hơn cho tham số ta ít chắc chắn hơn); Adam, RMSProp và các phương pháp Kronecker-factorized là trường hợp riêng của khung này (theo Aitchison, paper này không tự chứng minh lại). Nếu giả thiết state-transition là $P(\theta_{t+1} \mid \theta_t) = \mathcal{N}((I - A)\theta_t, Q)$ (công thức 4) với $A = \lambda I$, thì A đóng đúng vai trò weight decay decoupled (nhân μ_t với $(1 - \lambda)$ ở mỗi bước) và **không phụ thuộc** vào độ không chắc chắn riêng từng tham số, tức không đi qua preconditioner, khác hẳn L2 regularization (vốn phải đi qua gradient, nên phải đi qua preconditioner). Đây là lý giải vì sao weight decay "khớp tự nhiên" hơn L2 trong khung Bayesian filtering.

3.5 Hai cải tiến thực hành bổ sung (Phụ lục B)

Normalized weight decay (B.1): hệ số λ tối ưu phụ thuộc vào tổng số lượt cập nhật (số epoch $\times$ số batch), paper đề xuất chuẩn hoá bằng λ_{norm}:

$$\boxed{\lambda = \lambda_{norm}\sqrt{\frac{b}{BT}}}$$

với b = batch size, B = tổng số điểm dữ liệu train, T = tổng số epoch (hoặc số epoch của lần restart hiện tại nếu dùng AdamWR). λ_{norm} có thể hiểu là "weight decay nếu chỉ chạy 1 lượt qua 1 batch".

(Tự tính) ví dụ minh hoạ công thức trên với số liệu kiểu CIFAR-10 mà paper dùng: $B = 50.000$ ảnh train, $b = 128$, $T = 100$ epoch, $\lambda_{norm} = 0.025$ (một giá trị paper báo cáo tối ưu ở §4.3/Phụ lục D):

$$\lambda = 0.025 \times \sqrt{\frac{128}{50000 \times 100}} = 0.025 \times \sqrt{2.56 \times 10^{-5}} \approx 0.025 \times 0.005060 \approx 1.26 \times 10^{-4}$$

Nếu chạy lâu hơn, $T = 1800$ epoch (như thí nghiệm §4.3), với cùng $\lambda_{norm} = 0.025$: $\lambda = 0.025 \times \sqrt{128/(50000 \times 1800)} = 0.025 \times \sqrt{1.422 \times 10^{-6}} \approx 0.025 \times 0.001193 \approx 2.98 \times 10^{-5}$, nhỏ hơn khoảng

$4,2$ lần so với $T = 100$ (tỉ lệ đúng $\sqrt{1800/100} = \sqrt{18} \approx 4.24$, tự tính): xác nhận trực giác "chạy lâu hơn thì λ tối ưu nên nhỏ hơn" mà paper nêu ở §B.1.

AdamWR, cosine annealing + warm restarts (B.2, kế thừa SGDR, Loshchilov & Hutter 2016): hệ số lịch η_t giảm theo cosine trong mỗi "run" dài T_i epoch rồi **restart** (tăng lại η_t về 1, giữ nguyên θ_t, m_t, v_t đã học, không train lại từ đầu), công thức (14)-(15):

$$\eta_t = \eta_{min}^{(i)} + 0.5(\eta_{max}^{(i)} - \eta_{min}^{(i)})(1+\cos(\pi T_{cur}/T_i)), \quad \text{rút gọn khi } \eta_{max} = 1, \eta_{min} = 0: \ \eta_t = 0.5 + 0.5\cos(\pi T_{cur}/T_i)$$

T_{cur} đếm số epoch từ lần restart gần nhất (không cần nguyên); sau mỗi restart, T_i thường nhân với T_{mult} (ví dụ 2) để lần chạy sau dài gấp đôi. AdamWR = AdamW với η_t theo công thức trên và λ theo normalized weight decay.

4. Thí nghiệm và kết quả chính

Kiến trúc chính: **ResNet 26 lớp, 2 nhánh residual, Shake-Shake regularization** (Gastaldi, 2017), hai biến thể **2x64d** (11.6M tham số) và **2x96d** (25.6M tham số), batch size 128, trên **CIFAR-10** và **ImageNet32x32** (bản ImageNet hạ xuống 32×32 pixel, 1.2 triệu ảnh).

- **§4.1 (Hình 1), so 3 lịch learning rate (fixed, drop-step tại epoch 30/60/80, cosine annealing), 100 epoch:** weight decay decoupled (AdamW) vượt L2 regularization (Adam) ở **cả ba** lịch, khoảng cách lớn hơn khi lịch tốt hơn; cosine annealing cho kết quả tốt nhất trong ba lịch, được dùng cho các thí nghiệm sau.
- **§4.2 (Hình 2), hình dạng không gian siêu tham số (α, λ):** với SGD + L2 (Adam + L2 tương tự), vùng siêu tham số tốt nằm trên đường **chéo** (phải đổi cả α và λ cùng nhau để cải thiện) đúng như hệ quả coupling ở Proposition 1/mục 3.1; với SGDW/AdamW (decoupled), vùng tốt **song song với trục**, tức có thể cố định α chưa tối ưu và chỉ chỉnh λ vẫn ra kết quả tốt. Adam+L2 cho kết quả **tệ hơn rõ** so với SGD+L2 tốt nhất, và gần như "không benefit gì" từ L2 (kết quả tốt nhất với $\lambda \neq 0$ xấp xỉ kết quả với $\lambda = 0$); AdamW thu hẹp khoảng cách này, cạnh tranh được với SGD/SGDW.
- **§4.3 (Hình 3), chạy dài 1800 epoch, $\alpha = 0.001$ cố định (mặc định Adam):** AdamW cho **training loss thấp hơn** và **test error thấp hơn** Adam+L2; quan trọng hơn, ở **cùng mức training loss**, AdamW vẫn cho test error tốt hơn, tức cải thiện không chỉ do hội tụ nhanh hơn mà do **tổng quát hoá tốt hơn thực sự**. Kết quả tương tự trên ImageNet32x32 (SuppFigure 4).
- **§4.4 (Hình 4), AdamWR:** warm restarts giúp tăng tốc "anytime performance" của AdamW tới hệ số **10 lần** (đo ở lần restart đầu). Với $\alpha = 0.001$ mặc định, **AdamW đạt cải thiện 15% tương đối về test error** so với Adam, trên cả CIFAR-10 và ImageNet32x32; AdamWR giữ mức cải thiện này với anytime performance tốt hơn, thu hẹp hầu hết khoảng cách còn lại giữa Adam và SGDWR trên CIFAR-10, và cho hiệu suất tương đương trên ImageNet32x32.
- **§4.5, các ứng dụng độc lập được paper trích dẫn:** Wang et al. (2018, face detection, WIDER FACE) dùng AdamW; Völker et al. (2018, phân loại tín hiệu não EEG) báo cáo AdamW cho test accuracy cao hơn Adam trên Deep4Net (73.68% so với 71.37%) và ResNet (72.04% so với 61.34%, có ý nghĩa thống kê); **Radford et al. (2018) dùng AdamW để huấn luyện kiến trúc Transformer, đạt state-of-the-art trên nhiều benchmark natural language understanding** (đây chính là GPT-1, tiền thân của llm 02 GPT-3); Zhang et al. (2018) so L2 vs weight decay cho SGD/Adam/K-FAC trên CIFAR với ResNet/VGG, báo cáo weight decay decoupled luôn thắng khi hai cách khác nhau.

Đây là paper có thực nghiệm thật (không phải lý thuyết thuần), toàn bộ trên phân loại ảnh; không có benchmark NLP/Transformer do chính nhóm tác giả thực hiện (chỉ trích dẫn công trình khác dùng AdamW cho Transformer).

5. Phân tích / ablation

- **Tách biệt hai nguồn cải thiện:** Hình 3 (mục 4) cho thấy rõ cải thiện của AdamW có **hai phần**: tốc độ hội tụ (training loss thấp hơn ở cùng epoch) và tổng quát hoá thật (test error thấp hơn ở cùng training loss); nhiều người chỉ đọc "AdamW tốt hơn Adam" sẽ bỏ sót phần thứ hai, nhưng chính phần thứ hai mới là đóng góp quan trọng của paper (trả lời trực tiếp câu hỏi "Adam tổng quát hoá kém hơn SGD" của Wilson et al. 2017).
- **Chuẩn hoá weight decay là điều kiện cần để so sánh công bằng giữa các budget khác nhau:** không có chuẩn hoá, λ tối ưu cho 100 epoch khác hẳn 1800 epoch (tỉ lệ theo $\sqrt{T}$ như minh hoạ "tự tính" ở mục 3.5); nếu không biết điều này, một người thử AdamW với λ copy từ một lần chạy ngắn sang lần chạy dài hơn rất nhiều sẽ thấy kết quả "tệ hơn" và có thể kết luận sai rằng AdamW không tốt.
- **Warm restarts hữu ích cho "anytime performance" nhưng không phải cải thiện cơ bản của AdamW:** Hình 4 tách biệt hai đóng góp (decoupled weight decay cho 15% cải thiện test error; warm restarts cho tốc độ đạt tới mức đó nhanh hơn tới 10 lần ở điểm restart đầu), hai cơ chế độc lập có thể dùng riêng.
- Proposition 3 (mục 3.3) chỉ là xấp xỉ trực giác (preconditioner cố định), bản thân paper thừa nhận "không áp dụng trực tiếp" cho Adam thật; không nên dùng công thức đó để tính số "chính xác" cho Adam, chỉ dùng để hiểu hướng ảnh hưởng.

6. Giới hạn

Do tác giả nêu (§5, Kết luận):

- Kết quả chỉ kiểm chứng trên phân loại ảnh (CIFAR-10, ImageNet32x32); tác giả tự viết "phải được kiểm chứng trên phạm vi tác vụ rộng hơn, đặc biệt những tác vụ mà regularization được kỳ vọng quan trọng".
- Chỉ thực nghiệm trên Adam; tác giả **tin** (không chứng minh trong bài) kết quả tương tự cũng đúng cho AdaGrad và AMSGrad.
- Việc tích hợp phát hiện về weight decay với các cải tiến khác của Adam (ví dụ "normalized direction-preserving Adam", Zhang et al. 2017) được để ngỏ cho công việc tương lai.
- Proposition 3 chỉ đúng với preconditioner cố định, không khớp hoàn toàn với Adam thật (preconditioner đổi mỗi bước); tác giả tự nêu rõ hạn chế này ngay sau khi phát biểu.

Người ghi chú nhận xét:

- Batch size cố định ở 128 trong toàn bộ thí nghiệm chính; không có ablation về batch size, nên không rõ công thức normalized weight decay ($\sqrt{b/(BT)}$) có giữ đúng tỉ lệ ở batch size rất nhỏ (ví dụ 1) hay rất lớn hay không, dù công thức có chứa b một cách tường minh.
- Grid siêu tham số cho thí nghiệm 1800 epoch (§4.3, Phụ lục D) được paper tự nhận là "thô" (coarse, do chi phí tính toán), nên mức chênh lệch chính xác giữa AdamW và Adam ở chạy dài có độ tin cậy thấp hơn so với thí nghiệm 100 epoch.
- Câu "regularization tỉ lệ $\sqrt{s_i}$" ở Proposition 3 (mục 3.3) không có dẫn xuất từng bước rõ ràng trong văn bản chính; người ghi chú không tự suy ra lại được từ công thức (2) một cách không mơ hồ, nên chỉ trích dẫn nguyên văn, không diễn giải thêm.

7. Liên hệ với các bài khác

- **opt 06** (Adam): là thuật toán gốc; AdamW chỉ thay đổi cách xử lý weight decay (mục 3.2), giữ nguyên toàn bộ cơ chế m_t, v_t, bias-correction của bài 06.
- **opt 08** (AMSGrad): paper này tự nêu tin rằng kết quả tương tự cũng đúng cho AMSGrad (§5, Kết luận); AMSGrad và AdamW sửa hai vấn đề **độc lập** của Adam (hội tụ vs. regularization), về nguyên tắc có thể kết hợp cả hai.

- **opt 09** (RAdam): sửa vấn đề warmup/variance ở các bước đầu, một vấn đề khác nữa; cũng độc lập về mặt cơ chế với decoupled weight decay ở đây, có thể dùng chung.
- **llm 01** (Attention Is All You Need): kiến trúc Transformer gốc; Radford et al. (2018, GPT-1, không phải bài trong thư mục này) dùng chính AdamW được trích trong §4.5 để huấn luyện Transformer đạt state-of-the-art NLU, một liên kết trực tiếp giữa AdamW và dòng mô hình llm 01-02 dẫn tới ngày nay.
- **llm 43** (LightOnOCR, mô hình nền của dự án người dùng): theo ghi chú đã viết trong thư mục llm, mô hình này dùng đúng AdamW, **không weight decay** cho giai đoạn pretrain (lr đỉnh 1e-4, cosine, warmup 100 bước), và AdamW lr 4e-5 cho giai đoạn RL (GRPO); tức dự án người dùng thực sự dùng optimizer của chính bài này.
- Cơ chế cosine annealing + warm restarts ở mục 3.5 kế thừa trực tiếp từ một paper trước đó của cùng hai tác giả (Loshchilov & Hutter, 2016, về warm restarts cho SGD); bài đó không thuộc 9 bài opt 01-09 nên không trích số cụ thể ở đây, chỉ nêu tên.

8. Ứng dụng cho dự án (suy luận)

Dự án người dùng là nén + hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang gồm ~1.500 text + ~1.6-2.1K visual, GPU $\leq$ 8 GB, batch size = 1 khi suy luận). Theo mục 7, mô hình nền **đã dùng chính AdamW** của bài này, nên đây là bài liên quan **trực tiếp nhất** trong cả thư mục 'opt' cho việc chọn cấu hình optimizer khi fine-tune/distill.

1. **Lưu ý "không weight decay" ở pretrain gốc (quan sát từ llm 43, không phải suy luận).** Nếu quy trình distillation/fine-tune của người dùng giữ nguyên thiết lập $\lambda = 0$ như pretrain gốc, phần phân tích decoupled-vs-L2 của bài này **không phát huy tác dụng** (vì $\lambda = 0$ thì AdamW và Adam trùng nhau về công thức cập nhật). Nếu người dùng **thêm** weight decay khi fine-tune (một lựa chọn phổ biến để chống overfitting khi dữ liệu OCR ít), nên dùng đúng dạng decoupled (đã có sẵn trong `torch.optim.AdamW`) chứ không cộng L2 vào loss thủ công, để tránh đúng vấn đề mục 3.2-3.3 mô tả.
2. **Normalized weight decay (mục 3.5) đặc biệt liên quan khi thay đổi quy mô dữ liệu/epoch giữa các lần fine-tune (suy luận).** Nếu người dùng thử nhiều tập dữ liệu OCR với số lượng trang khác nhau, hoặc đổi số epoch, công thức $\lambda = \lambda_{norm}\sqrt{b/(BT)}$ cho một cách có nguyên tắc để giữ cường độ regularization ổn định thay vì dò lại λ từ đầu mỗi lần; với batch effective = 1 (không accumulation) công thức này cho λ rất nhỏ (vì b nhỏ), cần kiểm tra bằng thực nghiệm chứ không suy ra máy móc vì paper chỉ kiểm chứng ở $b = 128$.
3. **AdamWR (warm restarts, mục 3.5) có thể không cần thiết cho fine-tune ngắn (suy luận).** Cơ chế này hữu ích khi "anytime performance" quan trọng (muốn có checkpoint tốt sớm, nhiều lần, như huấn luyện 1800 epoch từ đầu); với fine-tune/distill chỉ vài nghìn bước trên GPU nhỏ, lợi ích có thể không rõ bằng khi huấn luyện từ đầu quy mô lớn như thí nghiệm gốc, vì không có đủ "budget" để nhiều restart.
4. **Proposition 2/3 (mục 3.3) nhắc rằng với kiến trúc hybrid (GatedDeltaNet + full attention), các nhóm tham số khác nhau (gate, attention, FFN) có thể có thang gradient lịch sử rất khác nhau (suy luận, ngoại suy từ phân tích chung của paper).** Nếu người dùng thêm L2 thủ công thay vì dùng AdamW, các nhóm tham số với gradient nhỏ (ví dụ một số tầng linear-attention/gate) có thể bị regularize quá tay so với tầng full attention; dùng weight decay decoupled (uniform theo λ cho mọi tham số, như paper đề xuất) tránh được thiên lệch này.

Điều ít liên quan: các thí nghiệm cụ thể trên ResNet/Shake-Shake/CIFAR-10/ImageNet32x32 (mục 4) không chuyển trực tiếp sang VLM OCR; các con số "15% cải thiện test error" là đặc thù bài toán phân loại ảnh, không nên trích dẫn như một kỳ vọng chung cho mọi tác vụ.

9. Thí nghiệm gợi ý

1. **Tái lập hiệu ứng coupling ở quy mô nhỏ:** trên MLP nhỏ (MNIST), quét lưới (α, λ) cho "Adam + L2 thủ công" và cho `AdamW` có sẵn của PyTorch, vẽ bản đồ nhiệt test error theo $(\log \alpha, \log \lambda)$ như Hình 2

của paper. Đo: hình dạng vùng tốt (chéo so với song song trục). Kỳ vọng: Adam+L2 cho vùng chéo (coupled), AdamW cho vùng gần như song song trục.

2. **Kiểm tra tách biệt hội tụ vs tổng quát hoá:** huấn luyện cùng một mạng nhỏ với Adam+L2 và AdamW tới **cùng mức training loss** (dừng sớm phiên bản hội tụ nhanh hơn), so test error ở điểm đó. Đo: test error tại cùng training loss. Kỳ vọng: AdamW thấp hơn, khớp claim mục 4 (Hình 3).
3. **Kiểm chứng công thức normalized weight decay bằng thực nghiệm nhỏ:** cố định λ_{norm}, chạy cùng mô hình với hai số epoch khác nhau (T và $4T$), tính λ theo công thức mục 3.5 cho mỗi lần chạy, so test error cuối cùng với trường hợp dùng cùng λ (không chuẩn hoá) cho cả hai T. Đo: test error. Kỳ vọng: dùng λ chuẩn hoá theo T ổn định hơn dùng λ cố định.
4. **Batch size rất nhỏ (liên hệ mục 8.2):** thử AdamW với λ cố định ở batch 1 và batch 32 (dùng gradient accumulation để so sánh công bằng số bước cập nhật), đo test loss sau cùng số lượt xem dữ liệu. Đo: ảnh hưởng của b trong công thức normalized weight decay có khớp thực nghiệm hay không ở b rất nhỏ.
5. **Warm restarts trên bài toán nhỏ:** cài η_t theo công thức (15) với $T_0 = 10$ epoch, $T_{mult} = 2$, so tốc độ đạt một ngưỡng test error cho trước giữa AdamW thường và AdamWR trên MNIST/CIFAR nhỏ. Đo: epoch cần để đạt ngưỡng ở mỗi lần restart. Kỳ vọng: AdamWR đạt ngưỡng sớm hơn ở các điểm restart đầu, minh hoạ lại Hình 4 ở quy mô nhỏ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
Weight decay nguyên bản	$\theta_{t+1} = (1-\lambda)\theta_t - \alpha \nabla f_t(\theta_t)$
Tương đương L2 cho SGD (Prop. 1)	$\lambda' = \alpha\lambda$
Không tương đương cho gradient thích ứng (Prop. 2)	cần $M_t = kI$, không đúng với Adam
Scale-adjusted L2, preconditioner cố định (Prop. 3)	$f_t(\theta) + \frac{\lambda'}{2}\|\theta \odot \sqrt{s}\|_2^2$
AdamW, bước cập nhật	$\theta_t = \theta_{t-1} - \eta_t\big(\alpha\hat{m}_t/(\sqrt{\hat{v}_t}+\epsilon) + \lambda\theta_{t-1}\big)$
Normalized weight decay	$\lambda = \lambda_{norm}\sqrt{b/(BT)}$
Lịch cosine (warm restarts)	$\eta_t = 0.5 + 0.5\cos(\pi T_{cur}/T_i)$ (khi $\eta_{max}=1, \eta_{min}=0$)
Cải thiện thực nghiệm chính	AdamW: -15% test error tương đối so Adam (lr mặc định 0.001, CIFAR-10/ImageNet32x32)

11. Kết luận ngắn

L2 regularization và weight decay tương đương cho SGD nhưng **không** tương đương cho Adam, vì số hạng regularization trong cách cài "Adam + L2" bị chia cho $\sqrt{\hat{v}_t}$ giống gradient của loss, làm giảm hiệu lực regularize ở tham số có gradient lịch sử lớn. AdamW tách bước weight decay khỏi cơ chế thích ứng, cải thiện 15% test error tương đối trên CIFAR-10/ImageNet32x32 với cùng learning rate mặc định, đồng thời làm không gian siêu tham số (α, λ) dễ dò hơn (tách biệt thay vì chéo); normalized weight decay và AdamWR (warm restarts) là hai cải tiến thực hành bổ sung. AdamW đã trở thành lựa chọn mặc định cho fine-tune/pretrain Transformer hiện đại, bao gồm chính mô hình nền LightOnOCR của dự án người dùng (xem llm 43).

Nguồn gốc: https://arxiv.org/abs/1711.05101

Bài 08

On the Convergence of Adam and Beyond (2019)

Tác giả: Sashank J. Reddi, Satyen Kale, Sanjiv Kumar (Google New York).
Venue / arXiv: arXiv:1904.09237 (v1, 19/04/2019); trang đầu PDF ghi ”Published as a conference paper at ICLR 2018”. Lưu ý: ngày arXiv (2019) muộn hơn hẳn thời điểm hội nghị ICLR 2018 (thường diễn ra tháng 4-5/2018); bài đạt **ICLR 2018 Best Paper Award**. Rất có thể bản arXiv này được đăng tải (hoặc đăng lại) hơn một năm sau khi đã công bố ở hội nghị, nên ”YYYY” theo quy ước của bộ ghi chú (lấy theo chữ số arXiv ID) là **2019**, dù nội dung khoa học thuộc về ICLR 2018; đây là điểm bất thường về ngày, nêu rõ ở đây theo yêu cầu kiểm tra.
Nguồn: https://arxiv.org/abs/1904.09237
Chủ đề: chỉ ra Adam (và RMSProp) có thể **không hội tụ** tới nghiệm tối ưu trong một số bài toán online convex đơn giản, xác định lỗi cụ thể trong chứng minh hội tụ gốc của Adam, và đề xuất AMSGrad (dùng max tích luỹ của v_t thay vì EMA trực tiếp) để đảm bảo hội tụ.
Vị trí trong lộ trình: đọc ngay sau opt 06 (Adam, bài bị phân tích và sửa trực tiếp ở đây) và opt 07 (AdamW, sửa một vấn đề khác, độc lập, của cùng Adam). Đọc trước hoặc sau opt 09 (RAdam) đều được, vì RAdam giải quyết một vấn đề thứ ba (variance/warmup) không phụ thuộc vào sửa đổi ở bài này.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

RMSProp, Adam, Adadelta, Nadam đều dùng gradient đã scale bởi căn bậc hai của **exponential moving average (EMA)** của gradient bình phương trong lịch sử gần. Thực nghiệm (ở nhiều ứng dụng, ví dụ học với không gian output lớn) đã quan sát các thuật toán này **thất bại hội tụ** tới nghiệm tối ưu (hoặc điểm tới hạn trong bài toán phi lồi). Paper đặt câu hỏi: nguyên nhân gốc là gì, và có sửa được không?

Trực giác được paper trình bày: một số minibatch cho gradient **lớn nhưng hiếm**; các gradient này rất thông tin (informative) nhưng ảnh hưởng của chúng **mất đi nhanh** do phép trung bình theo kiểu exponential (chỉ phụ thuộc thực chất vào vài gradient gần nhất, giống một ”cửa sổ” kích thước cố định). Paper chứng minh chặt trực giác này: giới hạn sự phụ thuộc của bước cập nhật vào chỉ vài gradient gần nhất **có thể** gây ra vấn đề hội tụ nghiêm trọng.

2. Ý tưởng chính

Paper đưa ra một khung tổng quát (Algorithm 1: Generic Adaptive Method) với hai hàm ϕ_t (tạo m_t từ lịch sử gradient) và ψ_t (tạo V_t, ma trận ”preconditioner”), bao trùm SGD, AdaGrad, RMSProp, Adam như các trường hợp riêng. Đại lượng chủ chốt:

$$\Gamma_{t+1} = \frac{\sqrt{V_{t+1}}}{\alpha_{t+1}} - \frac{\sqrt{V_t}}{\alpha_t}$$

đo sự thay đổi của **nghịch learning rate** theo thời gian. Với SGD và AdaGrad, $\Gamma_t \succeq 0$ luôn đúng (learning rate không tăng). Với Adam/RMSProp (EMA), Γ_t **có thể âm** (tức learning rate hiệu dụng ở một chiều nào đó có thể **tăng** đột ngột trở lại sau khi đã giảm), và chính tính không đơn điệu này làm hỏng chứng minh hội tụ gốc của Adam, đồng thời có thể khiến thuật toán hội tụ tới điểm **tệ nhất** trong tập khả thi ở các ví dụ được xây dựng cụ thể. Để sửa: cần cho optimizer ”bộ nhớ dài hạn” (long-term memory) về gradient lớn trong quá

khứ, bằng cách lấy **max tích luỹ** của v_t thay vì EMA thuần, mà vẫn giữ đúng chi phí thời gian/bộ nhớ như Adam, gọi là **AMSGrad**.

3. Phương pháp chi tiết

3.1 Khung Generic Adaptive Method (Algorithm 1) và các trường hợp riêng (§2)

$$m_t = \phi_t(g_1, \dots, g_t), \quad V_t = \psi_t(g_1, \dots, g_t), \quad \hat{x}_{t+1} = x_t - \alpha_t m_t / \sqrt{V_t}, \quad x_{t+1} = \Pi_{\mathcal{F}, \sqrt{V_t}}(\hat{x}_{t+1})$$

(Π là phép chiếu lên tập khả thi $\mathcal{F}$ theo chuẩn do $\sqrt{V_t}$ định nghĩa; khi $\mathcal{F} = \mathbb{R}^d$ phép chiếu là identity, trùng khớp Adam gốc). Ba trường hợp riêng (ký hiệu d = số chiều):

Thuật toán	ϕ_t	ψ_t (đường chéo v_t)	α_t
SGD	g_t	I	$\alpha/\sqrt{t}$
AdaGrad	g_t	$\mathrm{diag}\big(\frac{1}{t}\sum_{i=1}^{t} g_i^2\big)$	$\alpha/\sqrt{t}$
Adam	$(1-\beta_1)\sum_{i=1}^{t}\beta_1^{t-i} g_i$	$(1-\beta_2)\mathrm{diag}\big(\sum_{i=1}^{t}\beta_2^{t-i} g_i^2\big)$	theo lý thuyết $1/\sqrt{t}$, thực tế dùng hằng số

Paper nêu rõ chú thích: để đơn giản trình bày, họ **bỏ bước debias** của Adam gốc (opt 06) trong phần phân tích chính, nhưng ”lập luận của chúng tôi cũng áp dụng cho phiên bản đã debias”. $\beta_1 = 0.9, \beta_2 = 0.999$ là cấu hình ”thường được khuyến nghị trong thực tế” (paper trích dẫn lại, không phải giá trị tự đề xuất).

3.2 Ví dụ không hội tụ cụ thể (§3, Theorem 1-3)

Ví dụ online, $\mathcal{F} = [-1, 1]$:

$$f_t(x) = \begin{cases} Cx & t \bmod 3 = 1 \\ -x & \text{còn lại} \end{cases}, \quad C > 2$$

Điểm tối ưu (regret nhỏ nhất) là $x = -1$. Với $\beta_1 = 0$, $\beta_2 = 1/(1+C^2)$, paper chứng minh (Theorem 1, Phụ lục A, quy nạp) rằng Adam cho $x_{3t+1} = 1$ với mọi t, tức **hội tụ tới điểm tệ nhất** $x = 1$. Trực giác (nguyên văn diễn giải): gradient lớn C xuất hiện 1 lần mỗi 3 bước, bị β_2 (chọn đúng theo C) làm giảm quy mô đi một hệ số gần C, không đủ để triệt tiêu ảnh hưởng tích luỹ của 2 bước gradient -1 còn lại, nên điểm cứ trôi dần về $+1$.

(Tự tính) Vết số cụ thể cho 3 bước đầu của ví dụ Theorem 1, chọn $C = 5$ (thoả $C > 2$), $\beta_1 = 0$ (nên $m_t = g_t$ đúng từng bước, không có quán tính làm rối phép tính), $\beta_2 = 1/(1+C^2) = 1/26 \approx 0.038462$, $\alpha = 0.5$ (thoả điều kiện $\alpha < \sqrt{1-\beta_2} \approx 0.9806$ của chứng minh), $x_1 = 1$, $v_0 = 0$:

t	$t \bmod 3$	g_t	$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$	$\alpha_t = \alpha/\sqrt{t}$	$\hat{x}_{t+1} = x_t - \alpha_t g_t / \sqrt{v_t}$	x_{t+1} (chiếu vào $[-1,1]$)
1	1	5	$0{,}9615 \times 25 = 24{,}04$	0,5	$1 - 0{,}5 \cdot 5/4{,}903 = 0{,}4901$	0,4901
2	2	-1	$0{,}03846 \cdot 24{,}04 + 0{,}9615 \cdot 1 = 1{,}886$	0,3536	$0{,}4901 + 0{,}3536/1{,}3734 = 0{,}7475$	0,7475
3	0	-1	$0{,}03846 \cdot 1{,}886 + 0{,}9615 = 1{,}0341$	0,2887	$0{,}7475 + 0{,}2887/1{,}0169 = 1{,}0314$	**1,000** (bị chặn/kẹp)

Sau đúng 3 bước, $\hat{x}_4 \approx 1{,}031 > 1$ bị phép chiếu kẹp về $x_4 = 1$: khớp chính xác tuyên bố của Theorem 1 rằng $x_{3t+1} = 1$ với $t = 1$ (ở đây $x_4 = x_{3\cdot 1+1}$). Vì β_2 nhỏ (do C lớn), một gradient $+5$ hiếm gặp (1 trong 3 bước) không đủ để thắng hai gradient -1 liên tiếp sau đó, nên điểm trôi dần về biên dương $x = 1$, đúng là điểm **tệ nhất** trong $[-1, 1]$ (vì hàm $f_t(x) = -x$ ở 2/3 số bước muốn x càng lớn thì loss càng nhỏ cục bộ, nhưng regret tổng thể tính theo điểm tối ưu cố định $x^* = -1$).

Tổng quát hoá (Theorem 2): với **mọi** $\beta_1, \beta_2 \in [0, 1)$ hằng số sao cho $\beta_1 < \sqrt{\beta_2}$ (điều kiện chính Adam gốc giả định trong chứng minh của nó), luôn **tồn tại** một bài toán online convex khiến Adam có average regret không về 0. Paper cũng chỉ rõ: dù Adam gốc dùng $\beta_{1,t}$ **giảm dần** theo thời gian (không hằng số), lập luận vẫn mở rộng được vì tham số "tới hạn" gây lỗi là β_2 (hằng), không phải β_1; paper chỉ chứng minh trường hợp β_1 hằng "để rõ ràng".

Thêm ϵ không cứu được (Theorem 6, Phụ lục F): paper cũng chỉ ra với **bất kỳ** hằng số $\epsilon > 0$ thêm vào mẫu số (như Adam gốc dùng trong thực tế, công thức (3) của bài này: $\hat{x}_{t+1} = x_t - \alpha_t m_t / \sqrt{V_t + \epsilon I}$), vẫn tồn tại một bài toán khiến Adam có average regret không về 0 (không chứng minh chi tiết trong phần chính, chỉ dẫn Theorem 6).

3.3 Nguyên nhân cụ thể trong chứng minh gốc (§4, Phụ lục D)

Nguyên văn (Phụ lục D): "The main issue in their proof is the incorrect assumption that Γ_t defined in their equation (3) is positive semidefinite, and we also identified problems in lemmas 10.3 and 10.4 in their paper." Tức: chứng minh gốc của Adam (opt 06, Định lý 4.1 và các Lemma 10.3, 10.4 ở Phụ lục của bài đó) ngầm giả định $\Gamma_t \succeq 0$ (learning rate hiệu dụng không tăng theo thời gian), nhưng với v_t là EMA (không đơn điệu), giả định này **không luôn đúng**; các ví dụ ở mục 3.2 trên chính là minh chứng cụ thể cho việc giả định sai này dẫn tới kết luận sai.

3.4 Thuật toán AMSGrad (Algorithm 2) và hội tụ (Theorem 4)

```
Input: x1, step size {alpha_t}, {beta1_t}, beta2
m0 = 0, v0 = 0, v_hat0 = 0
for t = 1..T:
  g_t = grad f_t(x_t)
  m_t = beta1_t * m_{t-1} + (1 - beta1_t) * g_t
  v_t = beta2 * v_{t-1} + (1 - beta2) * g_t^2
  v_hat_t = max(v_hat_{t-1}, v_t)             # <-- khác biệt cốt lõi so với Adam
  x_{t+1} = Proj_F( x_t - alpha_t * m_t / sqrt(v_hat_t) )
```

Khác biệt duy nhất so với Adam (opt 06): thay $\hat{v}_t = v_t/(1-\beta_2^t)$ (EMA debiased) bằng $\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$ (**max tích luỹ**, không giảm theo thời gian). Vì $\hat{v}_t$ không giảm, $\sqrt{\hat{v}_t}/\alpha_t$ không giảm (nếu α_t cũng không giảm nhanh hơn), đảm bảo $\Gamma_t \succeq 0$ **ngay cả với** β_2 **hằng số**, khắc phục đúng lỗ hổng ở mục 3.3; không cần thay đổi cấu trúc thuật toán gốc, chi phí thời gian/bộ nhớ giữ nguyên (chỉ thêm một phép `max` theo từng toạ độ).

Theorem 4 (regret bound AMSGrad): với $\alpha_t = \alpha/\sqrt{t}$, $\beta_1 = \beta_{1,1}$, $\beta_{1,t} \le \beta_1$, $\gamma = \beta_1/\sqrt{\beta_2} < 1$, gradient và miền bị chặn (D_∞, G_∞):

$$R_T \le \frac{D_\infty^2 \sqrt{T}}{\alpha(1-\beta_1)} \sum_{i=1}^{d} \hat{v}_{T,i}^{1/2} + \frac{D_\infty^2}{(1-\beta_1)^2} \sum_{t=1}^{T}\sum_{i=1}^{d} \frac{\beta_{1,t}\hat{v}_{t,i}^{1/2}}{\alpha_t} + \frac{\alpha(1+\log T)}{(1-\beta_1)^2(1-\gamma)\sqrt{1-\beta_2}} \sum_{i=1}^{d} \|g_{1:T,i}\|_2$$

Hệ quả 1 (với $\beta_{1,t} = \beta_1 \lambda^{t-1}$): bound rút gọn còn 3 số hạng, số hạng chính $O\big(\alpha(1+\log T)\sum_i \|g_{1:T,i}\|_2\big)$, có thể **tốt hơn hẳn** $O(\sqrt{dT})$ của SGD khi dữ liệu thưa (giống lý do Adam/AdaGrad tốt với sparse gradient ở opt 06). Paper cũng nêu: có thể dùng $\beta_{1,t} = \beta_1/t$ (suy giảm nhẹ hơn) mà vẫn giữ $O(\sqrt{T})$.

Chứng minh (Phụ lục D) dùng đúng kỹ thuật của Adam gốc (Lemma 4 kiểu bất đẳng thức chiếu + Young) nhưng **sửa đúng chỗ lỗi**: vì $\hat{v}_t$ không giảm, các bất đẳng thức trung gian không cần giả định $\Gamma_t \succeq 0$ sai nữa.

3.5 Biến thể thứ hai: AdamNC (§5.1, Algorithm 3, Theorem 5)

Thay vì đổi cấu trúc (như AMSGrad), **AdamNC** ("non-constant") giữ cấu trúc Adam nguyên bản nhưng cho $\beta_{2,t}$ **tăng dần theo thời gian** (thay vì hằng số) sao cho hai điều kiện kỹ thuật được thoả (đủ để $\Gamma_t \succeq 0$). Trường hợp riêng đáng chú ý (Hệ quả 2): $\beta_{2,t} = 1 - 1/t$ làm $\hat{v}_t$ trở thành **trung bình cộng thường** $v_{t,i} = \frac{1}{t}\sum_{j=1}^{t} g_{j,i}^2$ (không còn exponential), tức AdamNC với lựa chọn này gần như một biến thể "có momentum" của AdaGrad, với regret bound data-dependent tương tự Hệ quả 1 ở AMSGrad.

4. Thí nghiệm và kết quả chính

Thí nghiệm tổng hợp (synthetic), minh hoạ trực tiếp ví dụ không hội tụ (§5, Hình 1):

- **Online:** $f_t(x) = 10^{10}x$ khi $t \bmod 101 = 1$, $f_t(x) = -10x$ còn lại, $\mathcal{F} = [-1, 1]$, $\beta_1 = 0.9, \beta_2 = 0.99$ (cấu hình "thường dùng trong thực tế", theo paper). Kết quả: average regret của Adam **không** về 0, iterate $x_t \to 1$ (điểm tệ nhất); average regret AMSGrad về 0, iterate $x_t \to -1$ (điểm tối ưu).
- **Stochastic:** $f_t(x) = 10^{10}x$ với xác suất 0.01, $f_t(x) = -10x$ còn lại. Cùng kết luận: x_t của Adam trôi về 1.

Logistic regression (MNIST, 784 chiều, minibatch 128): $\alpha_t = \alpha/\sqrt{t}$ (đúng lý thuyết) cho cả Adam và AMSGrad, $\beta_1 = 0.9$, $\beta_2 \in \{0.99, 0.999\}$ (cố định suốt thí nghiệm, chọn bằng grid search cùng α). AMSGrad cho train loss và test loss **tốt hơn** Adam; paper quan sát thêm AMSGrad "tương đối bền hơn với thay đổi hyperparameter" so với Adam.

Mạng neural 1 tầng ẩn (MNIST, 100 unit ReLU): $\alpha_t = \alpha$ hằng số (giống thực hành deep learning thông thường, khác lý thuyết), $\beta_1 = 0.9$, $\beta_2 \in \{0.99, 0.999\}$, grid search chọn cấu hình tốt nhất.

CifarNet (CIFAR-10, 60.000 ảnh 32×32): CNN 2 tầng conv (64 kênh, kernel 6×6) + 2 tầng FC (384, 192), max-pool 2×2, layer response normalization giữa các tầng conv, dropout (keep prob 0.5) giữa các tầng FC, minibatch 128. Kết quả: AMSGrad tốt hơn Adam **đáng kể** về train loss và accuracy; cải thiện này cũng chuyển thành kết quả tốt hơn ở test loss.

Paper không báo cáo con số cụ thể (%, giá trị loss) trong văn bản chính cho các thí nghiệm MNIST/CIFAR, chỉ dẫn chiếu Hình 1-2 (đường cong); không có bảng số liệu tổng hợp.

5. Phân tích / ablation

- **Phân biệt rõ hai mức độ của kết quả:** (a) các ví dụ tổng hợp (mục 3.2, 4) là **chứng minh chặt** (Theorem 1-3, 6) rằng Adam có thể thất bại hội tụ, không phải quan sát thực nghiệm đơn thuần; (b) các thí nghiệm MNIST/CIFAR (mục 4) chỉ là **thực nghiệm sơ bộ** (nguyên văn abstract: "a preliminary empirical study"), cho thấy AMSGrad "thường bằng hoặc tốt hơn" Adam, không phải bằng chứng AMSGrad luôn thắng trong thực tế.
- **Vai trò β_2 lớn:** ví dụ Theorem 1 cho thấy trực giác "nên dùng β_2 lớn khi dùng Adam" (một khuyến nghị quen thuộc trong thực hành) chính là cách giảm thiểu (không loại bỏ hẳn) nguy cơ không hội tụ; nhưng Theorem 2 chứng minh **không có hằng số** β_1, β_2 **nào** (thoả điều kiện $\beta_1 < \sqrt{\beta_2}$ của Adam gốc) loại bỏ hoàn toàn nguy cơ này.
- **AMSGrad có thể dùng learning rate nhỏ hơn hiệu dụng:** vì $\hat{v}_t = \max(\dots)$ không giảm, mẫu số $\sqrt{\hat{v}_t}$ có xu hướng lớn hơn hoặc bằng $\sqrt{v_t}$ của Adam tại cùng bước, nên bước cập nhật AMSGrad nói chung **nhỏ hơn hoặc bằng** Adam tại cùng α; đây là đánh đổi (an toàn hội tụ hơn, nhưng có thể chậm hơn nếu v_t của Adam từng giảm "đúng" theo đúng hình học bài toán, không phải lúc nào giảm cũng là lỗi).

- **AdamNC (mục 3.5) là một hướng sửa khác về bản chất:** không đổi cấu trúc thuật toán (vẫn EMA), chỉ đổi lịch β_2; cho thấy "long-term memory" có thể đạt bằng nhiều cách (max tích luỹ, hoặc EMA với hệ số tiến dần về trung bình cộng), không chỉ một cách duy nhất.

6. Giới hạn

Do tác giả nêu:

- Thực nghiệm (§5) tự nhận là "sơ bộ" (preliminary), chỉ trên logistic regression, MLP 1 tầng, 1 CNN nhỏ (CifarNet); không có thí nghiệm ở quy mô hiện đại (RNN lớn, Transformer).
- Phân tích tập trung vào Adam và RMSProp "vì tính tổng quát", nhưng paper tự nói lập luận cũng áp dụng cho Adadelta, Nadam và "các thuật toán dùng trung bình gradient bình phương trên một cửa sổ kích thước xấp xỉ cố định" nói chung, tuy **không trình bày chứng minh tổng quát đó** trong bài ("we omit the general analysis in this paper for the sake of clarity").
- Chỉ chứng minh trường hợp β_1 hằng số (không chứng minh tường minh trường hợp β_1 giảm dần theo thời gian như Adam gốc dùng), dù tác giả lập luận việc mở rộng là "dễ" vì tham số tới hạn là β_2.
- Tác giả tự nhận ví dụ không hội tụ là "được xây dựng cẩn thận" (carefully constructed) để minh hoạ vấn đề, không khẳng định các tình huống thực tế luôn tệ như vậy, chỉ cảnh báo rằng vấn đề "ít nhất có thể làm chậm hội tụ" trong tình huống thực.

Người ghi chú nhận xét:

- Vì $\hat{v}_t$ của AMSGrad không giảm theo thời gian, về lý thuyết có nguy cơ mẫu số "mắc kẹt" ở một giá trị lớn do một gradient ngoại lệ hiếm gặp ở đầu huấn luyện, làm bước cập nhật nhỏ dần vĩnh viễn sau đó dù gradient sau này đều nhỏ hơn; paper không thảo luận trực tiếp rủi ro này (nó là mặt đối lập của chính vấn đề mà AMSGrad sửa: đánh đổi "quên quá nhanh" của Adam lấy "nhớ vĩnh viễn" của AMSGrad).
- Không có so sánh trực tiếp AMSGrad với AdamW (opt 07, ra đời muộn hơn về mặt công bố arXiv dù AMSGrad được trích dẫn trong AdamW) hay với các kỹ thuật warmup; hai hướng sửa Adam này đến từ các nhóm khác nhau, được kết hợp hay không là câu hỏi mở từ chính các bài báo riêng lẻ.

7. Liên hệ với các bài khác

- **opt 06** (Adam): là đối tượng chính bị phân tích; bài này xác định chính xác dòng lỗi trong Phụ lục chứng minh của opt 06 (Lemma 10.3, 10.4, và giả định $\Gamma_t \succeq 0$).
- **opt 07** (AdamW): sửa một vấn đề khác, độc lập (weight decay), của cùng Adam; chính AdamW (công bố sau AMSGrad về mặt ICLR) trích dẫn AMSGrad và "tin rằng" phát hiện về weight decay cũng áp dụng được cho AMSGrad, gợi ý hai sửa đổi có thể kết hợp.
- **opt 09** (RAdam): sửa vấn đề thứ ba (variance của $\hat{v}_t$ lớn ở các bước đầu, cần warmup), cũng độc lập về cơ chế với max tích luỹ của AMSGrad ở đây.
- **opt 03** (Ruder, overview): liệt kê AdaGrad, RMSProp, Adam như các bước phát triển liên tiếp; bài này (và opt 09) là các bước "sửa lỗi" tiếp theo không có trong overview đó (Ruder công bố 2016, trước AMSGrad).
- **rl 07** (PPO): dùng Adam làm optimizer mặc định; vấn đề không hội tụ ở đây là trên bài toán **online convex** xây dựng riêng, không trực tiếp nói gì về việc Adam có ổn với RL phi lồi hay không, nhưng là lời nhắc rằng "Adam hội tụ" không phải định lý vô điều kiện khi áp dụng PPO hoặc các pipeline RLHF/RLVR khác trong thư mục rl.

8. Ứng dụng cho dự án (suy luận)

Dự án người dùng là nén + hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, GPU $\leq$ 8 GB, batch size = 1). Mô hình nền dùng AdamW (opt 07), không phải AMSGrad; mức độ liên quan trực tiếp của bài này thấp hơn opt 06/07, nhưng có giá trị **chẩn đoán** khi gặp bất ổn huấn luyện.

1. **Dấu hiệu nhận biết kiểu lỗi ở mục 3.2 khi debug (suy luận).** Nếu trong quá trình fine-tune/distill, loss đột ngột trôi về một hướng xấu sau một vài gradient "đột biến" hiếm gặp (ví dụ một trang tài liệu rất khác thường trong tập OCR gây gradient lớn bất thường), và hiện tượng lặp lại có tính tuần hoàn thô (giống cấu trúc ví dụ $t \bmod 3 = 1$ của paper), đây là dấu hiệu phù hợp với đúng cơ chế thất bại hội tụ mà bài này mô tả cho optimizer kiểu EMA (Adam/AdamW). Hướng kiểm tra: thử tăng β_2 (gần 1 hơn) hoặc thử tạm thời đổi sang biến thể kiểu AMSGrad (`torch.optim.Adam(..., amsgrad=True)`, PyTorch có sẵn) để xem bất ổn có giảm không.
2. **Với batch = 1, gradient "đột biến" hiếm gặp dễ xảy ra hơn (suy luận).** Batch lớn trung bình hoá nhiều mẫu, làm mượt các gradient bất thường; batch 1 giữ nguyên độ lớn gradient của từng mẫu đơn lẻ, tăng khả năng gặp đúng tình huống "gradient lớn nhưng hiếm" mà cơ chế EMA của Adam/AdamW xử lý kém theo phân tích ở mục 2-3. Đây là lý do gợi ý cân nhắc thử AMSGrad hoặc tăng β_2 cụ thể hơn cho pipeline batch-1 của dự án, thay vì chỉ dùng mặc định $\beta_2 = 0.999$.
3. **Không nên kỳ vọng chuyển trực tiếp cải thiện từ CifarNet/MNIST (mục 4) sang VLM OCR (suy luận).** Các con số cải thiện chỉ mang tính minh hoạ ở quy mô rất nhỏ (2018); không dùng làm ngưỡng kỳ vọng cụ thể.

Điều ít liên quan: chứng minh regret bound chi tiết (mục 3.4, Phụ lục D) là lý thuyết online-convex, không trực tiếp hướng dẫn cách chọn hyperparameter cho huấn luyện phi lồi thực tế; AdamNC (mục 3.5) hiếm thấy dùng trong thực hành hiện đại so với AMSGrad hoặc AdamW.

9. Thí nghiệm gợi ý

1. **Tái lập chính xác ví dụ không hội tụ (Theorem 1):** cài đặt trực tiếp $f_t(x) = Cx$ khi $t \bmod 3 = 1$, $f_t(x) = -x$ còn lại, $\mathcal{F} = [-1, 1]$, $C = 5$, $\beta_1 = 0, \beta_2 = 1/(1 + C^2)$, chạy cả Adam và AMSGrad bằng tay (NumPy, không cần GPU). Đo: giá trị x_t sau 1000-10000 bước. Kỳ vọng: Adam về gần $x = 1$, AMSGrad về gần $x = -1$, đúng Hình 1 của paper.
2. **Khảo sát ảnh hưởng của β_2 lên mức độ "tệ" của Adam trên cùng ví dụ:** quét $\beta_2 \in \{0.9, 0.99, 0.999, 0.9999\}$ (giữ C cố định, không chọn β_2 "đúng công thức lý thuyết" của Theorem 1), đo điểm hội tụ x_T cuối cùng. Diễn giải: kiểm tra trực giác "β_2 lớn giảm nhẹ vấn đề" có đúng theo mức độ hay không, và liệu có luôn tránh được vấn đề (kỳ vọng dựa theo Theorem 2: không, dù β_2 lớn tới đâu nếu vẫn hằng số và cấu hình C phù hợp sẽ có một C khác làm vấn đề quay lại).
3. **So Adam vs AMSGrad trên MLP nhỏ với một tỉ lệ nhỏ gradient "đột biến" nhân tạo:** trên MNIST, thỉnh thoảng (ví dụ 1% số bước) nhân gradient của một batch với hệ số lớn (giả lập outlier), còn lại giữ nguyên; so training loss cuối cùng của Adam và AMSGrad (`amsgrad=True`). Đo: độ ổn định train loss. Kỳ vọng: AMSGrad ổn định hơn khi tỉ lệ outlier và độ lớn outlier tăng.
4. **Batch size 1 vs batch lớn trên cùng ví dụ stochastic (mục 4):** mô phỏng $f_t(x) = 10^{10}x$ với xác suất nhỏ p, $f_t(x) = -10x$ còn lại, thử hai cách "batch": lấy 1 mẫu mỗi bước (batch 1, giữ nguyên độ lớn ngẫu) so với lấy trung bình 32 mẫu mỗi bước (giảm variance). Đo: tốc độ/khả năng Adam trôi về điểm xấu theo batch size. Liên hệ trực tiếp mục 8.2.
5. **AdamNC với $\beta_{2,t} = 1 - 1/t$ (Hệ quả 2, mục 3.5):** cài biến thể này trên logistic regression MNIST nhỏ, so với Adam và AMSGrad gốc. Đo: training loss, và giá trị v_t theo thời gian (nên tiến dần giống trung bình cộng thay vì EMA). Diễn giải: xác nhận công thức rút gọn $v_{t,i} = \frac{1}{t}\sum_j g_{j,i}^2$ bằng quan sát trực tiếp.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
Đại lượng Γ_t	$\Gamma_{t+1} = \sqrt{V_{t+1}}/\alpha_{t+1} - \sqrt{V_t}/\alpha_t$; cần $\Gamma_t \succeq 0$ để chứng minh hội tụ kiểu Adam

Khái niệm	Công thức
Ví dụ không hội tụ (online)	$f_t(x) = Cx$ ($t \bmod 3 = 1$), $= -x$ còn lại, $\mathcal{F} = [-1, 1]$, Adam $\to x = 1$ (tệ nhất)
AMSGrad, điểm khác Adam	$\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$ thay cho $\hat{v}_t = v_t/(1-\beta_2^t)$
Cập nhật AMSGrad	$x_{t+1} = \Pi_{\mathcal{F}}(x_t - \alpha_t m_t/\sqrt{\hat{v}_t})$
Regret bound AMSGrad (Theorem 4)	$O\big(\alpha(1+\log T)\sum_i \|g_{1:T,i}\|_2\big)$ (số hạng chính), xem đầy đủ ở mục 3.4
Lỗi gốc đã xác định	giả định $\Gamma_t \succeq 0$ trong chứng minh Adam (opt 06), vấn đề ở Lemma 10.3, 10.4 của opt 06
AdamNC	$\beta_{2,t}$ tăng dần (ví dụ $1 - 1/t$) thay cho β_2 hằng số

11. Kết luận ngắn

Adam (và RMSProp) có thể không hội tụ tới nghiệm tối ưu trong các bài toán online convex được xây dựng cụ thể, vì EMA của v_t khiến learning rate hiệu dụng có thể **tăng** trở lại (đại lượng Γ_t không luôn dương), phá vỡ một giả định ngầm trong chứng minh hội tụ gốc (chỉ ra chính xác ở Lemma 10.3, 10.4 của opt 06). AMSGrad sửa bằng cách lấy max tích luỹ của v_t (đảm bảo learning rate không tăng), giữ nguyên chi phí tính toán, có regret bound chứng minh chặt và thực nghiệm sơ bộ cho kết quả bằng hoặc tốt hơn Adam trên logistic regression/MLP/CifarNet. Đây là một trong ba hướng sửa độc lập của Adam trong thư mục này (cùng AdamW ở opt 07 cho weight decay, RAdam ở opt 09 cho warmup/variance), đạt giải Best Paper ICLR 2018.

Nguồn gốc: https://arxiv.org/abs/1904.09237

Bài 09

On the Variance of the Adaptive Learning Rate and Beyond (2019)

Tác giả: Liyuan Liu (University of Illinois Urbana-Champaign), Haoming Jiang (Georgia Tech), Pengcheng He, Weizhu Chen (Microsoft Dynamics 365 AI), Xiaodong Liu, Jianfeng Gao (Microsoft Research), Jiawei Han (UIUC).
Venue / arXiv: arXiv:1908.03265 (v1 nộp 09/08/2019; bản đọc trong ghi chú này là v4, 26/10/2021); trang đầu PDF ghi ”Published as a conference paper at ICLR 2020”. Theo quy ước của bộ ghi chú (năm lấy theo chữ số trong arXiv ID, 1908 -> 2019), YYYY = 2019, dù hội nghị chính thức là ICLR 2020 và bản text đọc được lại là v4 (2021). Đây là một bất thường ngày nhỏ, cùng dạng với opt 08 (AMSGrad), nêu rõ ở đây để tránh nhầm giữa ”năm arXiv ID”, ”năm hội nghị” và ”năm của bản PDF cụ thể đang đọc”.
Nguồn: https://arxiv.org/abs/1908.03265
Chủ đề: lý giải bằng lý thuyết lẫn thực nghiệm tại sao heuristic *learning rate warmup* giúp ổn định huấn luyện với các optimizer thích ứng (RMSProp, Adam): nguyên nhân được xác định là phương sai của *adaptive learning rate* quá lớn ở giai đoạn đầu huấn luyện (do thiếu mẫu). Từ đó đề xuất **Rectified Adam (RAdam)**, biến thể của Adam tự động chỉnh (rectify) phương sai này mà không cần hyperparameter warmup bổ sung.
Vị trí trong lộ trình: đọc sau opt 06 (Adam), opt 07 (AdamW) và opt 08 (AMSGrad). Đây là sửa đổi thứ ba, hoàn toàn độc lập, nhắm vào một vấn đề khác của cùng họ Adam: không phải weight decay (opt 07) hay chứng minh hội tụ lý thuyết (opt 08), mà là phương sai của learning rate thích ứng trong các bước đầu và cách giải thích heuristic warmup vẫn hay dùng trong thực hành (ví dụ ở llm 01, llm 43).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026, đọc toàn văn kể cả phụ lục A, B, C). Các phép tính đánh dấu ”tự tính” là của người ghi chú, dùng công thức paper nhưng số liệu cụ thể không có sẵn trong bài. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Từ khi Adam (opt 06) trở thành optimizer phổ biến, một heuristic đi kèm gần như bắt buộc trong nhiều ứng dụng, đặc biệt huấn luyện Transformer (bài llm 01) và BERT, là **learning rate warmup**: bắt đầu bằng learning rate rất nhỏ, tăng dần (thường tuyến tính) trong vài nghìn bước đầu, trước khi áp dụng lịch suy giảm chính. Warmup được quan sát mang lại cải thiện rõ rệt về ổn định huấn luyện, tốc độ hội tụ, và khả năng tổng quát hoá cho các optimizer thích ứng (RMSProp, Adam, Adadelta, Nadam) nhưng tại thời điểm paper ra đời, chưa có lý thuyết giải thích **tại sao** warmup hoạt động, nên người thực hành phải dò (trial-and-error) độ dài warmup cho từng bài toán, kiến trúc, learning rate.

Paper dẫn ví dụ cụ thể: huấn luyện Transformer cho dịch máy (NMT) trên IWSLT’14 German-to-English, theo công thức gốc của Vaswani et al. 2017 (bài llm 01) và Popel & Bojar 2018, nếu **bỏ warmup**, training loss hội tụ về khoảng 10; nếu **có warmup**, training loss giảm xuống dưới 3. Hiện tượng tương tự cũng được quan sát khi huấn luyện trước BERT (Devlin et al., 2019). Paper mô tả đây là optimizer ”mắc kẹt” ở một điểm tối ưu cục bộ tồi/đáng nghi (suspicious/bad local optima) khi không có warmup.

Câu hỏi gốc của paper: nguyên nhân sâu của hiện tượng này là gì, và liệu có thể thay heuristic warmup (cần dò một hyperparameter mới, độ dài warmup T_w) bằng một cơ chế tự động, có cơ sở lý thuyết, không cần thêm hyperparameter?

2. Ý tưởng chính

Paper dùng lại khung tổng quát **Generic Adaptive Method** (Algorithm 1), vốn cũng là khung mà opt 08 (AMSGrad, Reddi et al. 2018) dùng để thống nhất SGD, AdaGrad, RMSProp, Adam: ở bước t, tính gradient g_t, rồi dùng hai hàm $\phi_t(g_1, \dots, g_t)$ (tính "động lượng" m_t) và $\psi_t(g_1, \dots, g_t)$ (tính "adaptive learning rate" l_t), cập nhật $\theta_t \leftarrow \theta_{t-1} - \alpha_t m_t l_t$. Với Adam cụ thể (Equation 1 của paper):

$$\phi(g_1, \dots, g_t) = \frac{(1-\beta_1)\sum_{i=1}^{t} \beta_1^{t-i} g_i}{1-\beta_1^t}, \qquad \psi(g_1, \dots, g_t) = \sqrt{\frac{1-\beta_2^t}{(1-\beta_2)\sum_{i=1}^{t} \beta_2^{t-i} g_i^2}}. \tag{1}$$

Ý tưởng trực giác mở đầu: xét trường hợp đặc biệt $t = 1$, khi đó $\psi(g_1) = \sqrt{1/g_1^2}$. Nếu coi $g_1, \dots, g_t$ là các biến ngẫu nhiên Gauss độc lập cùng phân phối $N(0, \sigma^2)$, thì $1/g_1^2$ tuân theo phân phối scaled inverse chi-squared Scale-inv-$\chi^2(1, 1/\sigma^2)$, và Var$[\sqrt{1/g_1^2}]$ là **phân kỳ** (vô hạn). Nghĩa là tỉ số thích ứng (ψ) có thể lớn bất thường ở giai đoạn đầu huấn luyện, trong khi đặt learning rate nhỏ ở giai đoạn này làm giảm phương sai (Var$[\alpha x] = \alpha^2$Var$[x]$) - đây chính là lý do (giả thuyết của paper) warmup hoạt động: **warmup là một kỹ thuật giảm phương sai (variance reduction), không chỉ là một "mẹo ổn định" tuỳ tiện.**

Từ giả thuyết này, paper: (a) xây dựng thí nghiệm kiểm chứng trực tiếp (Adam-2k, Adam-eps - xem mục 4), (b) suy ra dạng giải tích của Var$[\psi(.)]$ theo số bước đã dùng (ký hiệu ρ, tương tự "độ dài hiệu dụng" của trung bình trượt), (c) từ dạng giải tích đó, thiết kế một **số hạng chỉnh (rectification term)** r_t nhân trực tiếp vào adaptive learning rate để giữ phương sai luôn ổn định (không phụ thuộc t) ngay từ bước đầu, không cần warmup thủ công. Biến thể Adam có số hạng này được gọi là **Rectified Adam (RAdam)**.

3. Phương pháp chi tiết

3.1 Thí nghiệm kiểm chứng trực tiếp giả thuyết phương sai (§3.1)

Paper thiết kế hai biến thể của Adam để giảm phương sai của $\psi(.)$ theo hai cách khác nhau, so sánh với Adam có/không warmup trên IWSLT'14 De-En:

- **Adam-2k:** chỉ cập nhật $\psi(.)$ trong 2000 bước đầu, còn $\phi(.)$ (động lượng) và θ (tham số) **giữ cố định**; sau đó mới chạy Adam bình thường (các bước này đánh số từ -1999 để so sánh cùng trục với các baseline khác). Ý tưởng: lấy đủ mẫu để ước lượng $\psi(.)$ ổn định trước khi dùng nó cập nhật tham số thật. Kết quả: Adam-2k tránh được vấn đề hội tụ của Adam-vanilla.
- **Adam-eps:** tăng ϵ (hằng số ổn định số học trong $\hat{\psi}(.) = \sqrt{(1-\beta_2^t)}/\sqrt{\epsilon + (1-\beta_2)\sum \beta_2^{t-i} g_i^2}$) từ 10^{-8} (Adam gốc) lên 10^{-4}. Lý do: nếu $\hat{\psi}(.)$ gần như phân phối đều, phương sai của nó $\approx \frac{1}{12}\epsilon^2$, nên tăng ϵ cũng giảm phương sai. Kết quả: Adam-eps cũng tránh được vấn đề hội tụ tồi, nhưng **hiệu suất kém hơn** Adam-2k và Adam-warmup - paper suy đoán vì ϵ lớn gây **bias** lớn vào adaptive learning rate, làm chậm quá trình tối ưu.

Cả hai biến thể đều cho thấy: khi phương sai của $\psi(.)$ được giảm (bằng cách nào cũng được), vấn đề hội tụ tồi biến mất, và phân phối gradient (quan sát qua histogram log-scale, Figure 2-3 của paper) không còn bị "biến dạng" (distorted) trong các bước đầu như khi dùng Adam-vanilla. Đây là bằng chứng thực nghiệm trực tiếp cho giả thuyết phương sai, trước khi đi vào phân tích lý thuyết.

3.2 Phân tích giải tích phương sai của $\psi(.)$ (§3.2, Theorem 1)

Do EMA (exponential moving average) khó phân tích trực tiếp, paper xấp xỉ EMA bằng SMA (simple moving average, trung bình trượt đơn giản, theo Nau 2014) khi t nhỏ (lúc đó trọng số mũ của các g_i chưa khác nhau nhiều):

$$\sqrt{\frac{(1-\beta_2)\sum_{i=1}^{t}\beta_2^{t-i}g_i^2}{1-\beta_2^t}} \approx \sqrt{\frac{\sum_{i=1}^{t}g_i^2}{t}}.$$

Với $g_i \sim N(0,\sigma^2)$ i.i.d., $t/\sum_{i=1}^{t}g_i^2 \sim$ Scale-inv-$\chi^2(t,1/\sigma^2)$. Paper tổng quát hoá: giả sử $\psi^2(.) \sim$ Scale-inv-$\chi^2(\rho,1/\sigma^2)$ với ρ "số bậc tự do" (degree of freedom), đóng vai trò số bước hiệu dụng đã dùng để ước lượng ψ.

Theorem 1. Nếu $\psi^2(.) \sim$ Scale-inv-$\chi^2(\rho,1/\sigma^2)$, thì Var$[\psi(.)]$ giảm đơn điệu khi ρ tăng. Với $\forall\rho > 4$:

$$\text{Var}[\psi(.)] = \tau^2\Big(\frac{\rho}{\rho-2} - \frac{\rho\,2^{2\rho-5}}{\pi}B\big(\tfrac{\rho-1}{2},\tfrac{\rho-1}{2}\big)^2\Big), \tag{2}$$

trong đó $B(.,.)$ là hàm beta, $\tau = 1/\sigma$. Chứng minh đầy đủ (Phụ lục A) dùng công thức mật độ scaled inverse chi-squared, tích phân $E[\sqrt{x}]$ bằng hàm gamma, và một chuỗi bất đẳng thức (bất đẳng thức Gautschi, công thức trùng lặp Legendre) để chỉ ra đạo hàm của biểu thức trong (2) luôn âm với $\rho \geq 4$. Ghi chú đã đọc toàn bộ phần chứng minh này (khá dài, thuần đại số/giải tích hàm đặc biệt) nhưng không chép lại từng bước ở đây vì không cần cho trực giác chính: kết luận quan trọng là **phương sai của adaptive learning rate giảm đơn điệu khi có nhiều "bậc tự do" (tức nhiều mẫu/bước) hơn**, và ở ρ nhỏ (giai đoạn đầu), phương sai này **lớn hơn hẳn** (paper nêu minh hoạ: phương sai tại $\rho = 5$ lớn hơn 100 lần so với tại $\rho = 500$).

3.3 Ước lượng ρ_t: tương đương số bước EMA ≈ SMA (§4.1)

Để áp dụng Theorem 1 vào EMA thật, paper tìm độ dài $f(t,\beta_2)$ của một SMA có cùng "trọng tâm" (center of mass) với EMA tại bước t:

$$\frac{(1-\beta_2)\sum_{i=1}^{t}\beta_2^{t-i}\cdot i}{1-\beta_2^t} = \frac{\sum_{i=1}^{f(t,\beta_2)}(t+1-i)}{f(t,\beta_2)}. \tag{4}$$

Giải ra: $f(t,\beta_2) = \dfrac{2}{1-\beta_2} - 1 - \dfrac{2t\beta_2^t}{1-\beta_2^t}$. Đặt $\rho_t \triangleq f(t,\beta_2)$ (ước lượng số bậc tự do tại bước t), và $\rho_\infty \triangleq \lim_{t\to\infty}\rho_t = \dfrac{2}{1-\beta_2} - 1$ (độ dài SMA tương đương tối đa). Vậy:

$$\rho_\infty = \frac{2}{1-\beta_2} - 1, \qquad \rho_t = \rho_\infty - \frac{2t\beta_2^t}{1-\beta_2^t}. \tag{3'}$$

(Paper gọi cặp công thức này là Equation 3-4; ghi chú gộp dạng cuối cùng của ρ_t thành một dòng, đánh số (3') để tránh nhầm với các equation gốc ở trên.)

3.4 Số hạng chỉnh phương sai và Rectified Adam (§4.2, Algorithm 2)

Dùng xấp xỉ bậc một của $\sqrt{\psi^2(.)}$ quanh $E[\psi^2(.)]$ (Wolter, 2007), paper thu được dạng xấp xỉ đơn giản hơn của phương sai:

$$\text{Var}[\psi(.)] \approx \frac{\rho_t}{2(\rho_t-2)(\rho_t-4)\sigma^2}. \tag{5}$$

Paper kiểm chứng bằng mô phỏng (§5.3, Figure 8) rằng xấp xỉ bậc một này rất khớp với dạng giải tích đầy đủ (2) trên dải $\rho \in \{5,\ldots,500\}$.

Mục tiêu: chọn số hạng chỉnh r_t sao cho Var$[r_t\psi(.)] = C_{var}$ không đổi theo t, với $C_{var} = \min_{\rho_t}\text{Var}[\psi(.)] = \text{Var}[\psi(.)]|_{\rho_t=\rho_\infty}$ (phương sai nhỏ nhất, đạt khi dùng hết "bộ nhớ" ρ_∞). Từ (5), số hạng chỉnh có dạng đóng:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}.$$

Algorithm 2 (RAdam), viết lại theo đúng cấu trúc paper:

```
Input: {alpha_t}, beta1, beta2, theta_0, f_t(theta)
m_0, v_0 <- 0, 0
rho_inf <- 2/(1-beta2) - 1
for t = 1..T:
    g_t      <- grad f_t(theta_{t-1})
    v_t      <- beta2 * v_{t-1} + (1-beta2) * g_t^2
    m_t      <- beta1 * m_{t-1} + (1-beta1) * g_t
    m_hat_t  <- m_t / (1 - beta1^t)              # bias-correction (giống Adam, opt 06)
    rho_t    <- rho_inf - 2*t*beta2^t / (1-beta2^t)
    if rho_t > 4:                                 # phuong sai con tinh toan duoc
        l_t  <- sqrt((1-beta2^t) / v_t)
        r_t  <- sqrt( (rho_t-4)*(rho_t-2)*rho_inf / ((rho_inf-4)*(rho_inf-2)*rho_t) )
        theta_t <- theta_{t-1} - alpha_t * r_t * m_hat_t * l_t
    else:                                          # phuong sai khong tinh duoc -> tat
    ↪  adaptive term
        theta_t <- theta_{t-1} - alpha_t * m_hat_t
```

Hai điểm quan trọng paper nhấn mạnh, cả hai đều được chứng minh trực tiếp từ công thức (không phải suy đoán):

- Khi $\rho_t \leq 4$ (giai đoạn rất đầu), r_t không tính được (căn của số âm/chia 0), nên RAdam **tắt hẳn** adaptive learning rate và chỉ dùng m_t đã bias-correct, tức **suy biến đúng thành SGD có động lượng (SGDM)** ở các bước này.
- Nếu $\beta_2 \leq 0.6$ thì $\rho_\infty \leq 4$ **với mọi** t, nghĩa là RAdam **luôn luôn** suy biến thành SGDM, không bao giờ kích hoạt phần thích ứng - một điều kiện biên rõ ràng, không mơ hồ.

3.5 So sánh với warmup heuristic (§4.3)

Paper nhận xét r_t có dạng tương tự warmup tuyến tính heuristic (có thể viết warmup tuyến tính như $r_t^{heuristic} = \min(t/T_w, 1)$), nhưng khác hai điểm: (i) RAdam **tắt hẳn** phần thích ứng khi phương sai còn phân kỳ (thay vì chỉ nhân với một hệ số nhỏ như warmup làm), tránh hẳn các bước cập nhật mất ổn định đầu tiên; (ii) RAdam **không cần thêm hyperparameter** T_w, tự thích ứng theo β_2 đang dùng. Paper cũng nói rõ cơ chế chỉnh phương sai này **độc lập** (orthogonal) với các kỹ thuật ổn định khác như gradient clipping, tăng ϵ, range constraints (AdaBound, Luo et al. 2019), khởi tạo tốt, hay batch/layer normalization - có thể kết hợp chứ không thay thế chúng.

3.6 Ví dụ số (tự tính): ρ_t và r_t với $\beta_2 = 0.999$, liên hệ warmup 100 bước

Paper không cho sẵn bảng số cụ thể của ρ_t, r_t theo t; bảng dưới là **tự tính** bằng công thức (3') và r_t ở trên, với $\beta_2 = 0.999$ (giá trị mặc định dùng trong hầu hết thí nghiệm của paper, và cũng là giá trị mặc định phổ biến nhất trong thực hành, bao gồm cấu hình AdamW của LightOnOCR - xem mục 7, 8). Với $\beta_2 = 0.999$: $\rho_\infty = 2/(1 - 0{,}999) - 1 = 1999$.

t	ρ_t (tự tính)	$\rho_t > 4$?	r_t (tự tính)	$\min(t/100, 1)$ (warmup tuyến tính 100 bước, để so sánh)
1	1,000	không	- (SGDM thuần)	0,01
2	1,999	không	- (SGDM thuần)	0,02
3	2,999	không	- (SGDM thuần)	0,03

t	ρ_t (tự tính)	$\rho_t > 4$?	r_t (tự tính)	$\min(t/100, 1)$ (warmup tuyến tính 100 bước, để so sánh)
4	3,997	không	- (SGDM thuần)	0,04
5	4,996	có	0,0173	0,05
10	9,983	có	0,0490	0,10
50	49,583	có	0,1482	0,50
100	98,333	có	0,2153	1,00
500	458,485	có	0,4765	1,00 (đã bão hoà)
1000	835,967	có	0,6453	1,00
4000	1850,045	có	0,9619	1,00
10000	1998,096	có	0,9998	1,00

Vài quan sát (tự tính, suy ra từ bảng, không phải câu paper viết trực tiếp): với $\beta_2 = 0.999$, RAdam suy biến thành SGDM thuần đúng 4 bước đầu ($\rho_t \leq 4$ tại $t \leq 4$, $\rho_5 \approx 4{,}996 > 4$ là bước đầu tiên phần thích ứng được kích hoạt) - ngắn hơn nhiều so với một warmup thủ công 100 bước. Nhưng sau khi kích hoạt, r_t tăng **rất chậm**: tại $t = 100$ (đúng độ dài warmup mà LightOnOCR dùng ở giai đoạn pretrain, xem mục 7-8), $r_t \approx 0{,}215$, tức mới đạt khoảng 21,5% "cường độ" thích ứng đầy đủ, trong khi warmup tuyến tính 100 bước đã đạt 100% ngay tại $t = 100$. r_t chỉ tiến gần 1 (cường độ đầy đủ) khi t cỡ vài nghìn đến chục nghìn bước ($r_t \approx 0{,}96$ tại $t = 4000$, $\approx 1{,}00$ tại $t = 10000$). Điều này cho thấy RAdam với β_2 mặc định **không tương đương** một warmup ngắn: nó áp một "warmup ngầm" dài hơn rất nhiều, chỉ nhanh ở vài bước đầu tiên (nhờ tắt hẳn adaptive term) rồi chậm dần. Đây là điểm quan trọng cần nhớ khi định thay AdamW+warmup ngắn bằng RAdam trực tiếp (xem mục 8).

4. Thí nghiệm và kết quả chính

Ghi chú: đây là một paper có cả lý thuyết (mục 3) và thực nghiệm đầy đủ trên nhiều tác vụ thật; không phải trường hợp "không có thực nghiệm thật" nên không cần áp dụng quy ước "chỉ có định lý/giới hạn lý thuyết" của bộ ghi chú.

Paper đánh giá RAdam trên ba nhóm bài toán: **language modeling** (One Billion Word, LSTM 2 lớp 2048 hidden), **image classification** (CIFAR-10 với ResNet-20, ImageNet với ResNet-18), và **neural machine translation** (IWSLT'14 De-En/En-De, WMT'16 En-De, Transformer qua fairseq). Theo Loshchilov & Hutter 2018 (AdamW, bài opt 07), paper **decouple weight decay** trong tất cả biến thể Adam dùng để so sánh (Adam-vanilla, Adam-warmup, RAdam) - nghĩa là baseline thực nghiệm của RAdam vốn đã dùng kỹ thuật của opt 07, không phải Adam "gốc" kiểu opt 06.

So với Adam-vanilla (§5.1): trên cả 3 benchmark (One Billion Word, CIFAR-10, ImageNet), RAdam cho cải thiện nhất quán so với Adam-vanilla, xác nhận vấn đề phương sai tồn tại rộng trên nhiều loại tác vụ/kiến trúc, không riêng NMT. Trên ImageNet/CIFAR-10, paper cung cấp một bảng kết quả test accuracy (Table 1 trong paper); do văn bản PDF-to-text của bảng này bị xáo trộn thứ tự cột/dòng khá nặng (lỗi OCR/trích layout, không phải lỗi nội dung paper), ghi chú diễn giải lại dựa trên việc ghép các số với câu văn mô tả đi kèm (cách đọc này có khả năng sai nhỏ ở vị trí, nêu rõ để người đọc tự kiểm tra lại nếu cần):

Optimizer	ImageNet (test acc., %)	CIFAR-10 (test acc., %)
SGD	69,86	91,51
Adam	66,54	90,54
RAdam	67,62	91,38

Paper tự nhận: ”RAdam fails to outperform SGD in terms of test accuracy” trên cả hai bộ dữ liệu ảnh - khớp với bảng trên (RAdam thấp hơn SGD ở cả hai). Nhưng về **training accuracy** trên ImageNet, paper cho số liệu rõ trong văn bản (không bị OCR xáo trộn): SGD 69,57; Adam 69,12; RAdam 70,30 - ở đây RAdam **vượt** cả SGD và Adam. Ghi chú nhận xét: đây là một điểm paper tự nêu rõ sự khác biệt train/test accuracy (RAdam tốt hơn trên train nhưng không tốt hơn SGD trên test), không phải mâu thuẫn nội tại, nhưng là chi tiết dễ bị bỏ sót nếu chỉ đọc bảng test accuracy.

So với warmup heuristic (§5.2): trên NMT (BLEU, Table 2 của paper, số liệu đọc rõ, không bị OCR xáo trộn):

Phương pháp	IWSLT’14 DE-EN	IWSLT’14 EN-DE	WMT’16 EN-DE
Adam + warmup	$34{,}66 \pm 0{,}014$	$28{,}56 \pm 0{,}067$	27,03
RAdam	$34{,}76 \pm 0{,}003$	$28{,}48 \pm 0{,}054$	27,27

Hai phương pháp cho hiệu suất **tương đương** (RAdam nhích hơn ở DE-EN và WMT’16, nhích kém hơn ở EN-DE, chênh lệch nhỏ trong khoảng độ lệch chuẩn) - xác nhận giả thuyết: vấn đề hội tụ tồi của Adam-vanilla đúng là do phương sai của adaptive learning rate, vì chỉnh đúng phương sai (RAdam) đạt hiệu quả tương đương với ”chữa cháy” bằng warmup thủ công.

Trên CIFAR-10, Adam+warmup đạt accuracy tốt nhất 91,29% (tương đương RAdam 91,38%), nhưng paper nhấn mạnh RAdam **ít nhạy với learning rate hơn**: với 4 giá trị learning rate khác nhau $\{0{,}1; 0{,}03; 0{,}01; 0{,}003\}$ (Figure 6), các đường test accuracy của RAdam khớp nhau rất sát; Adam và SGD nhạy hơn rõ với lựa chọn learning rate. Với warmup thủ công (Figure 7, thử 4 độ dài warmup $\{100, 200, 500, 1000\}$ bước), paper cho một ví dụ cụ thể: tại learning rate 0,1, Adam với 100 bước warmup chỉ đạt accuracy 90,13%, còn RAdam (không cần chỉnh độ dài warmup) đạt 91,06% - minh chứng RAdam cần ít công dò hyperparameter hơn.

5. Phân tích / ablation

Kiểm chứng xấp xỉ bậc một (§5.3, Figure 8): so sánh dạng giải tích đầy đủ (2) và xấp xỉ bậc một (5) trên dải $\rho \in \{5, \dots, 500\}$ (với $\tau = 1$): hai đường gần như trùng nhau, sai khác tuyệt đối nhỏ hơn hẳn giá trị gốc - xác nhận xấp xỉ (5) đáng tin cậy, không chỉ là một phép đơn giản hoá tuỳ tiện.

Kiểm chứng giả định phân phối (§5.3, Figure 9): paper thử nghiệm với gradient $g_i \sim N(\mu, 1)$ với $\mu \in \{0; 0{,}001; 0{,}01; 0{,}1; 1; 10\}$ (không còn zero-mean như giả định gốc), mô phỏng 5000 trajectory, 6000 bước, $\beta_2 = 0.999$, so sánh phương sai của $1/\sqrt{v_t}$ (chưa chỉnh) và $r_t/\sqrt{v_t}$ (đã chỉnh). Kết luận paper nêu: trên cả 6 giá trị μ, adaptive learning rate chưa chỉnh có phương sai lớn ở giai đoạn đầu, còn adaptive learning rate đã chỉnh có phương sai ”tương đối ổn định” (relative consistent) - xác nhận độ tin cậy của giả định (dù $\mu \neq 0$ thực tế hay xảy ra khi huấn luyện tiến triển, không còn đúng hoàn toàn giả định ”trọng số khởi tạo zero-mean” ban đầu).

Ablation ”downgrade về SGDM” (Phụ lục C, Table 3): đây thực chất là một ablation trực tiếp về **việc xử lý vài bước đầu tiên** có quan trọng không. Trên CIFAR-10 ($lr = 0.1$, trung bình 5 lần chạy):

1-4 bước	5-8 bước	Từ bước 9	Test acc.	Train loss	Train error
RAdam	RAdam	RAdam	91,08	0,021	0,74
Adam (phương sai phân kỳ)	RAdam	RAdam	89,98	0,060	2,12
SGD	Adam (phương sai đã tụ)	RAdam	90,29	0,038	1,23

Paper kết luận: optimizer không đạt được chất lượng tương đương khi 4 bước đầu bị đổi thành Adam có phương sai phân kỳ (dòng 2, kém nhất); đổi 5-8 bước ít tai hại hơn (dòng 3, đỡ hơn dòng 2 nhưng vẫn kém dòng 1). Paper tự nhận định đúng với lý thuyết của mình: vài bước đầu tiên gây hại nhiều hơn các bước sau. Tuy vậy paper cũng tự hạ thấp tầm quan trọng của phần "downgrade về SGDM" này, nói rõ đây chỉ là "a minor part of our algorithm design", trọng tâm chính của bài là cơ chế warmup và số hạng chỉnh phương sai.

6. Giới hạn

Các giới hạn dưới đây lấy trực tiếp từ những gì paper tự thừa nhận hoặc từ việc đọc toàn văn kỹ, không suy đoán thêm:

- **Không có định lý hội tụ/regret bound cho RAdam.** Khác biệt rõ với opt 06 (Adam có Theorem 4.1, regret bound dạng $O(\sqrt{T})$) và opt 08 (AMSGrad có Theorem 4 và Corollary 1, cũng regret bound). Paper này chỉ có Theorem 1 (về tính đơn điệu của $\mathrm{Var}[\psi(.)]$ theo ρ), không có theorem nào đảm bảo RAdam hội tụ tới nghiệm tối ưu hoặc đạt regret giới hạn nào. Đóng góp của bài chủ yếu mang tính **giải thích + thực nghiệm**, không phải chứng minh lý thuyết tối ưu hoá theo nghĩa online-learning như hai bài kia.
- **Nhiều lớp xấp xỉ chồng lên nhau, mỗi lớp paper tự thừa nhận là xấp xỉ, không chính xác tuyệt đối:** (i) xấp xỉ EMA bằng SMA (hợp lý khi t nhỏ, nhưng là xấp xỉ); (ii) giả định $g_i \sim N(0, \sigma^2)$ độc lập cùng phân phối, zero-mean - paper tự ghi chú (footnote 2) giả định zero-mean "hợp lệ ở giai đoạn đầu huấn luyện" (vì trọng số khởi tạo từ phân phối zero-mean, theo Balduzzi et al. 2017), ngầm ý giả định **yếu dần khi huấn luyện tiến triển**; (iii) xấp xỉ bậc một của $\sqrt{\psi^2(.)}$ để ra công thức r_t khả thi tính toán (Equation 5), paper kiểm chứng bằng mô phỏng nhưng không chứng minh sai số giới hạn chặt dưới dạng giải tích.
- **Điều kiện suy biến về SGDM phụ thuộc hoàn toàn vào β_2, không phải một lựa chọn được tối ưu hoá:** nếu $\beta_2 \leq 0.6$, RAdam luôn luôn là SGDM, không bao giờ kích hoạt phần thích ứng - nghĩa là với β_2 nhỏ (ít khi dùng trong thực hành, nhưng có thể xảy ra khi tinh chỉnh hyperparameter), RAdam thực chất không còn là một "Adam được chỉnh", mà là SGDM hoàn toàn mà người dùng có thể không nhận ra.
- **Giai đoạn "ramp-up ngầm" có thể rất dài với β_2 lớn** (như bảng tự tính ở mục 3.6 cho thấy, $\beta_2 = 0.999$ cần tới hàng nghìn bước để r_t tiến gần 1) - đây không phải một "lỗi" của paper, nhưng là một đặc tính ít được nhấn mạnh trong phần kết luận chính, có thể gây hiểu nhầm nếu chỉ đọc "RAdam không cần warmup" mà không để ý cơ chế chỉnh phương sai của nó vẫn ngầm chứa một quá trình tăng dần tương tự, chỉ khác là **tự động theo** β_2, không phải theo một hyperparameter T_w tách biệt.
- **Phạm vi kiến trúc thử nghiệm giới hạn ở 2019:** LSTM (ngôn ngữ), ResNet (ảnh), Transformer (NMT, qua fairseq) - không có mô hình quy mô lớn kiểu LLM hiện đại, không có kiến trúc attention tuyến tính/hybrid như GatedDeltaNet (liên quan trực tiếp đến dự án, xem mục 8). Việc ngoại suy sang các kiến trúc này là suy luận, chưa có bằng chứng thực nghiệm trong paper.
- **Bảng Table 1 (image classification) trong bản PDF-to-text bị xáo trộn layout khá nặng** (đã nêu ở mục 4) - đây là hạn chế của quy trình trích xuất văn bản (pdftotext trên PDF có layout 2 cột xen hình), không phải hạn chế của paper, nhưng ghi chú này không thể khẳng định 100% các số trong bảng tái dựng ở mục 4 đúng theo đúng vị trí gốc; các số **có khớp** với mô tả bằng lời của paper nên độ tin cậy ở mức khá cao, nhưng không tuyệt đối.

7. Liên hệ với các bài khác

- **opt 06 (Adam, Kingma & Ba 2014):** RAdam là một biến thể trực tiếp của Adam - cùng m_t, v_t bias-corrected, chỉ thêm số hạng chỉnh r_t nhân vào adaptive learning rate. Khi $\rho_t \leq 4$, RAdam dùng $\hat{m}_t$ (giống bias-correction của Adam) nhưng tắt hẳn phần $\sqrt{v_t}$ ở mẫu.
- **opt 07 (AdamW, Loshchilov & Hutter 2017/2019):** liên kết trực tiếp, không chỉ là trích dẫn tổng quát - paper RAdam ghi rõ: "Following Loshchilov & Hutter (2018), we decouple weight decays in

the vanilla Adam, Adam with warmup and RAdam in our experiments" (mục đầu của §5 Experiments). Nghĩa là **mọi baseline Adam trong các thí nghiệm của RAdam đã dùng kỹ thuật decoupled weight decay của AdamW**, không phải Adam nguyên bản. Hai kỹ thuật (decouple weight decay, và rectify variance) độc lập, có thể cộng dồn, và paper RAdam thực tế đã cộng dồn chúng trong mọi thí nghiệm.

- **opt 08 (AMSGrad, Reddi, Kale & Kumar 2018):** cả hai bài dùng **cùng khung Generic Adaptive Method** (ϕ_t, ψ_t như ở mục 2-3 trên) do chính Reddi et al. 2018 đề xuất - paper RAdam trích rõ "describes various popular stochastic gradient descent algorithms (Reddi et al., 2018)". Hai bài nhắm vào hai khiếm khuyết **độc lập** của Adam: AMSGrad sửa khả năng không hội tụ (do Γ_t có thể âm, bộ nhớ ngắn hạn về gradient lớn hiếm gặp), còn RAdam sửa phương sai lớn của adaptive learning rate ở giai đoạn đầu (không liên quan đến dấu của Γ_t). Hai cơ chế sửa không xung đột, có thể kết hợp (ví dụ AMSGrad + rectification), tuy paper không thử nghiệm trực tiếp sự kết hợp này.
- **llm 01 (Attention is All You Need, Vaswani et al. 2017):** chính là nguồn gốc của hiện tượng mà RAdam phân tích - công thức warmup gốc trong llm 01 là $lrate = d_{model}^{-0.5} \cdot \min(step^{-0.5}, step \cdot warmup^{-1.5})$ với $warmup = 4000$ bước, dùng Adam $\beta_1 = 0.9, \beta_2 = 0.98$. Paper RAdam dẫn trực tiếp Vaswani et al. 2017 ngay phần mở đầu làm ví dụ về sự cần thiết của warmup trong Transformer.
- **llm 09 (Pre-LN Transformer, Xiong et al. 2020):** đưa ra một **lời giải thích cạnh tranh/khác góc nhìn** cho cùng hiện tượng warmup. Ghi chú llm 09 cho biết: kiến trúc Post-LN gốc (giống llm 01) "bắt buộc có learning rate warm-up"; không warmup, Adam chỉ đạt BLEU 8,45, có warmup ($T_{warmup} = 4000$) đạt khoảng BLEU 34 - rất gần với con số mà RAdam dẫn (loss hội tụ ~10 không warmup, dưới 3 có warmup, cùng kiểu bài toán IWSLT'14 De-En). Nhưng lời giải thích của llm 09 **khác hẳn**: Xiong et al. quy nguyên nhân về **độ lớn gradient tại lớp output khi khởi tạo** dưới kiến trúc Post-LN (vấn đề kiến trúc), còn RAdam quy nguyên nhân về **phương sai của adaptive learning rate do optimizer** (vấn đề optimizer, không phụ thuộc kiến trúc cụ thể). Đây là một điểm thú vị: hai bài giải thích **hai cơ chế khác nhau** cho cùng một hiện tượng quan sát được (cần warmup để Adam không hội tụ tồi trên Transformer NMT) - paper RAdam không đề cập đến hướng giải thích kiểu Pre-LN/gradient-tại-init, nên đây là một khoảng trống chưa được đối chiếu trực tiếp giữa hai dòng nghiên cứu, không phải một mâu thuẫn đã được paper nào xử lý.
- **llm 43 (LightOnOCR):** dự án thực tế của người dùng, dùng chính xác công thức mà RAdam nhắm tới thay thế: giai đoạn pretrain dùng AdamW, weight decay = 0, learning rate đỉnh 10^{-4}, lịch cosine, **warmup 100 bước**; giai đoạn RL (GRPO) dùng AdamW learning rate 4×10^{-5} (ghi chú llm 43 không nêu rõ RL có warmup hay không). Đây là cơ sở thực tế cho mục 8 dưới đây.
- **rl 07 (PPO, Schulman et al. 2017):** dùng Adam/SGD với stepsize suy giảm tuyến tính theo tiến trình huấn luyện (ví dụ 3×10^{-4} giảm dần), nhưng **không nêu dùng warmup**. Đây là một điểm đối lập nhẹ, đáng chú ý: PPO tái dùng cùng batch rollout cho nhiều epoch cập nhật trong một vòng ngắn, chế độ huấn luyện khác hẳn pretrain ngôn ngữ dài hạn - việc PPO không cần warmup có thể (suy luận, paper RAdam không xác nhận) liên quan đến số bước cập nhật tổng thể ít hơn nhiều so với pretrain NMT/LM quy mô lớn mà RAdam khảo sát, nên vấn đề "phương sai lớn ở vài nghìn bước đầu" ít có cơ hội bộc lộ rõ trong một vòng PPO ngắn.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là suy luận áp dụng cho việc post-training/compressing **LightOnOCR-3-0.8B** (VLM OCR, kiến trúc hybrid GatedDeltaNet linear attention + full attention, ~3600 token/trang, mục tiêu chạy vừa GPU ≤8GB, batch size = 1 khi inference), không phải nội dung paper tuyên bố.

- **Recipe pretrain gốc của LightOnOCR (llm 43) vốn đã dùng chính "heuristic" mà RAdam muốn thay thế:** AdamW, warmup 100 bước, cosine. Nếu người dùng định fine-tune/distill tiếp (ví dụ LoRA, hoặc fine-tune toàn phần quy mô nhỏ) trên GPU ≤8GB, một câu hỏi tự nhiên là: có nên giữ warmup 100 bước, hay thử RAdam để bỏ hẳn hyperparameter này? Theo bảng tự tính ở mục 3.6: với $\beta_2 = 0.999$ (giá trị mặc định, nhiều khả năng cũng là giá trị AdamW đang dùng cho LightOnOCR vì ghi chú

llm 43 không nêu β_2 khác), RAdam chỉ đạt $r_t \approx 0{,}215$ tại $t = 100$ - nghĩa là **nếu đổi trực tiếp AdamW+warmup(100 bước) sang RAdam mà không suy nghĩ lại**, với một lượt fine-tune ngắn (vài trăm đến vài nghìn bước, điều rất có thể xảy ra khi tài nguyên chỉ là 1 GPU ≤8GB), RAdam sẽ dùng adaptive learning rate ở cường độ thấp hơn nhiều so với kỳ vọng trong suốt phần lớn quá trình fine-tune (vì r_t chỉ tiến gần 1 ở hàng nghìn bước, như bảng mục 3.6 cho thấy) - có thể làm quá trình fine-tune **chậm hội tụ hơn** dự tính, không phải vì RAdam "sai", mà vì tốc độ "ramp-up ngầm" của nó gắn chặt với β_2, không gắn với số bước fine-tune thực tế dự kiến (khác bản chất với warmup 100 bước, vốn được chọn **đúng theo** độ dài lịch huấn luyện).

- **Hệ quả thực hành (suy luận):** nếu muốn thử RAdam cho một vòng fine-tune ngắn, nên **giảm** β_2 để ρ_∞ nhỏ hơn, giúp r_t tiến gần 1 nhanh hơn, tương xứng với tổng số bước fine-tune dự kiến - có thể dùng công thức mục 3.4 ($\rho_\infty = 2/(1-\beta_2) - 1$) để chọn β_2 sao cho $\rho_t \approx \rho_\infty$ (ví dụ $r_t \gtrsim 0.9$) đạt được ở khoảng 10-20% tổng số bước fine-tune, tương tự cách paper chọn $\beta_2 = 0.9995$ (tương ứng $\nu = 4000$) để khớp với warmup 4000 bước trong thí nghiệm NMT (mục 4, Phụ lục B.3) của chính RAdam. Đây là một thủ thuật đã có tiền lệ trong paper (họ tự làm với NMT), không phải suy đoán tuỳ tiện, nhưng số β_2 cụ thể cho LightOnOCR vẫn cần dò nghiệm thực tế, không có công thức "đúng duy nhất".
- **Liên hệ với opt 07 (AdamW) đã viết trước:** LightOnOCR pretrain đặt weight decay = 0, nên AdamW tại đó toán học tương đương Adam thường (xem opt 07 §3.1). Nếu post-training về sau có dùng weight decay khác 0 (ví dụ để regularize một fine-tune nhỏ, tránh overfit trên tập dữ liệu hẹp), kết hợp cả hai kỹ thuật (decoupled weight decay của opt 07 + rectification của RAdam) là tự nhiên và **đã được paper RAdam tự làm trong mọi thí nghiệm của chính họ** (xem mục 7) - nên đây không phải một kết hợp chưa kiểm chứng, dù kiểm chứng đó ở quy mô LSTM/ResNet/Transformer-NMT 2019, không phải VLM OCR hybrid.
- **Về kiến trúc hybrid GatedDeltaNet (linear attention) + full attention:** RAdam phân tích phương sai của $\psi(.)$ theo từng **tham số, element-wise**, không phụ thuộc kiến trúc sinh ra gradient đó - nên về lý thuyết không có lý do để phương sai sớm của các tham số gating trong GatedDeltaNet khác về bản chất so với tham số attention thường. Tuy nhiên đây là suy luận ngoại suy hoàn toàn (ngoài paper, kiến thức chung): paper không thử nghiệm kiến trúc attention tuyến tính/hybrid nào, và các cơ chế gating (ví dụ sigmoid/softplus trên "forget gate") có thể có phân phối gradient khác hẳn giả định $N(0, \sigma^2)$ mà paper dùng để suy ra Theorem 1, nên mức độ tin cậy của suy luận này ở mức thấp, chỉ nên coi là gợi ý để thử nghiệm (xem mục 9), không phải kết luận.
- **Về batch size nhỏ do giới hạn GPU ≤8GB:** RAdam's ρ_t chỉ phụ thuộc t (số bước) và β_2, không phụ thuộc batch size trực tiếp trong công thức. Nhưng batch size nhỏ làm tăng σ^2 hiệu dụng của gradient mỗi bước (nhiễu minibatch lớn hơn), và theo (2)/(5), $\text{Var}[\psi(.)] \propto \tau^2 = 1/\sigma^2$ - vậy khi σ^2 lớn hơn (do batch nhỏ), phương sai tuyệt đối của adaptive learning rate ở giai đoạn đầu **lớn hơn**, không nhỏ hơn. Suy luận: với ràng buộc GPU ≤8GB buộc dùng batch nhỏ (có thể cần gradient accumulation để đạt batch hiệu dụng lớn hơn), vấn đề "phương sai lớn ở giai đoạn đầu" mà RAdam mô tả **có khả năng nghiêm trọng hơn**, không phải ít quan trọng hơn, so với lúc LightOnOCR pretrain ở quy mô 96×H100 với batch 384. Đây là lý do hợp lý để **giữ warmup (hoặc RAdam)** khi fine-tune ở quy mô nhỏ, hơn là bỏ hẳn nó để "đơn giản hoá" công thức huấn luyện.

9. Thí nghiệm gợi ý

1. **Vẽ đường ρ_t, r_t cho cấu hình thật dự kiến dùng.** Viết một script PyTorch/Python nhỏ (không cần GPU) tính ρ_t, r_t theo công thức mục 3.4 cho β_2 và tổng số bước fine-tune LightOnOCR dự kiến (ví dụ vài trăm đến vài nghìn bước). Đo: giá trị r_t tại bước cuối dự kiến. Kỳ vọng (dựa trên bảng mục 3.6): nếu $\beta_2 = 0.999$ và tổng số bước dưới ~2000, r_t sẽ chưa đạt gần 1 khi kết thúc fine-tune. Diễn giải: nếu đúng vậy, nên hạ β_2 theo công thức $\rho_\infty = 2/(1-\beta_2) - 1$ trước khi dùng RAdam, hoặc đơn giản giữ warmup thủ công thay vì RAdam.
2. **Tái dựng thí nghiệm Adam-vanilla vs Adam-warmup vs RAdam ở quy mô nhỏ.** Trên 1 GPU ≤8GB: huấn luyện một Transformer nhỏ (2-4 lớp, $d_{model} = 128$) cho một tác vụ seq2seq đồ chơi (ví dụ

copy/reverse sequence, hoặc một tập con nhỏ IWSLT'14). So sánh 3 optimizer: Adam không warmup, Adam + warmup tuyến tính (ví dụ 400 bước), RAdam. Đo: training loss ở 50-200 bước đầu. Kỳ vọng theo paper: Adam không warmup có thể mắc ở loss cao hơn hẳn (dạng giống Figure 1); hai phương pháp còn lại tránh được. Diễn giải: nếu quy mô nhỏ KHÔNG tái hiện được hiện tượng (loss hội tụ tương đương cả 3 cách), gợi ý rằng vấn đề phương sai chủ yếu nổi rõ ở quy mô lớn/learning rate lớn, và với fine-tune nhỏ của LightOnOCR có thể không cần lo lắng nhiều về việc này.

3. **Histogram gradient đầu huấn luyện (tái dựng ý tưởng Figure 2/3) trên một lượt fine-tune nhỏ thật của LightOnOCR (hoặc một proxy nhỏ hơn).** Log histogram (log-scale) của $|g|$ ở 10-20 bước đầu, so sánh có/không warmup. Đo: hình dạng histogram có "co lại" bất thường trong vài bước đầu không. Kỳ vọng: nếu không warmup, phân phối gradient collapse về giá trị nhỏ sớm (giống paper mô tả); nếu không thấy hiện tượng này ở mô hình/learning rate đang dùng, đó là tín hiệu thực nghiệm cho thấy có thể bỏ qua warmup/RAdam an toàn cho setup cụ thể đó.
4. **So sánh độ nhạy learning rate (tái dựng ý tưởng Figure 6/7) ở quy mô nhỏ.** Fine-tune (hoặc một LoRA nhỏ trên LightOnOCR, hoặc một mạng nhỏ thay thế) với 3-4 learning rate khác nhau, dùng (a) AdamW + warmup 100 bước (recipe gốc) và (b) RAdam (đã chỉnh β_2 theo gợi ý thí nghiệm 1). Đo: phương sai của metric validation cuối cùng giữa các learning rate khác nhau, cho mỗi cách. Kỳ vọng theo paper: RAdam cho kết quả ít phụ thuộc learning rate hơn warmup thủ công. Diễn giải: nếu đúng, RAdam có giá trị thực dụng là giảm công dò learning rate khi chuyển sang một recipe post-training khác hẳn recipe pretrain gốc (dữ liệu khác, batch khác, số bước khác).
5. **Ablation "vài bước đầu" (tái dựng ý tưởng Table 3, Phụ lục C) ở quy mô nhỏ.** Trên một tác vụ nhỏ (CIFAR-10 thu nhỏ, hoặc một classification proxy), so sánh: RAdam toàn bộ; Adam-vanilla (phương sai phân kỳ) ở 4 bước đầu rồi chuyển RAdam; SGD ở 4 bước đầu rồi Adam-converged rồi RAdam. Đo: test accuracy cuối, training loss. Kỳ vọng theo paper: cách xử lý vài bước đầu ảnh hưởng không nhỏ tới kết quả cuối. Diễn giải: giúp đánh giá xem có cần đầu tư thời gian "tối ưu" cách khởi động optimizer cho các lượt fine-tune ngắn của LightOnOCR, hay chỉ cần một warmup ngắn đơn giản là đủ (biên độ cải thiện nhỏ không đáng công sức thêm).

10. Tóm tắt công thức / khái niệm

Khái niệm / công thức	Ý nghĩa
$\phi_t(g_1,\dots,g_t)=m_t$, $\psi_t(g_1,\dots,g_t)=l_t$ (Eq. 1, Generic Adaptive Method, cùng khung với opt 08)	Tách optimizer thích ứng thành hàm tính động lượng và hàm tính adaptive learning rate
$\mathrm{Var}[\sqrt{1/g_1^2}]$ phân kỳ khi $t=1$	Trực giác mở đầu: phương sai của adaptive learning rate có thể vô hạn ở bước đầu
Theorem 1, Eq. (2): $\mathrm{Var}[\psi(.)]=\tau^2\big(\frac{\rho}{\rho-2}-\frac{\rho 2^{2\rho-5}}{\pi}B(\frac{\rho-1}{2},\frac{\rho-1}{2})^2\big)$, $\rho>4$	Dạng giải tích đầy đủ của phương sai, giảm đơn điệu theo ρ (số bậc tự do/số bước hiệu dụng)
$\rho_\infty=\dfrac{2}{1-\beta_2}-1$	Độ dài SMA tương đương tối đa (giới hạn khi $t\to\infty$); với $\beta_2=0.999$: $\rho_\infty=1999$ (tự tính)
$\rho_t=\rho_\infty-\dfrac{2t\beta_2^t}{1-\beta_2^t}$ (Eq. 3-4)	Độ dài SMA tương đương tại bước t (ước lượng số bậc tự do hiện có)
Eq. (5) xấp xỉ bậc 1: $\mathrm{Var}[\psi(.)]\approx\dfrac{\rho_t}{2(\rho_t-2)(\rho_t-4)\sigma^2}$	Dạng đơn giản, tính được trực tiếp, dùng để suy ra r_t
$r_t=\sqrt{\dfrac{(\rho_t-4)(\rho_t-2)\rho_\infty}{(\rho_\infty-4)(\rho_\infty-2)\rho_t}}$	Số hạng chỉnh phương sai (rectification term), cốt lõi của RAdam
Nếu $\rho_t\le 4$: tắt adaptive term, $\theta_t\leftarrow\theta_{t-1}-\alpha_t\hat{m}_t$	RAdam suy biến đúng thành SGD có động lượng ở các bước đầu
Nếu $\beta_2\le 0.6$: $\rho_\infty\le 4$ với mọi t	RAdam suy biến hoàn toàn thành SGDM, không bao giờ kích hoạt phần thích ứng

Khái niệm / công thức	Ý nghĩa
Algorithm 2 (RAdam)	$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$; $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$; $\hat{m}_t = m_t/(1-\beta_1^t)$; cập nhật $\theta_t \leftarrow \theta_{t-1} - \alpha_t r_t \hat{m}_t \sqrt{(1-\beta_2^t)/v_t}$ khi $\rho_t > 4$
Bảng ρ_t, r_t với $\beta_2 = 0.999$ (tự tính, mục 3.6)	$r_t \approx 0{,}017$ tại $t = 5$; $\approx 0{,}215$ tại $t = 100$; $\approx 0{,}96$ tại $t = 4000$; $\approx 1{,}00$ tại $t = 10000$

11. Kết luận ngắn

Paper xác định nguyên nhân sâu của hiện tượng ”cần warmup để Adam không hội tụ tồi”: phương sai của adaptive learning rate quá lớn khi số mẫu/bước còn ít, chứng minh bằng cả thực nghiệm kiểm chứng trực tiếp (Adam-2k, Adam-eps) và phân tích giải tích (Theorem 1, giả định scaled inverse chi-squared). Từ phân tích đó, đề xuất RAdam: tắt hẳn adaptive learning rate khi phương sai còn phân kỳ (vài bước đầu, suy biến thành SGDM), rồi nhân một số hạng chỉnh r_t tính trực tiếp từ β_2, t, không cần hyperparameter warmup riêng. Thực nghiệm trên ngôn ngữ, ảnh, dịch máy cho kết quả tương đương hoặc tốt hơn warmup thủ công, và ít nhạy hơn với lựa chọn learning rate. Khác với opt 06 (Adam) và opt 08 (AMSGrad), RAdam không kèm chứng minh hội tụ/regret bound hình thức; đóng góp chủ yếu là giải thích + cơ chế thực hành. Với dự án LightOnOCR, điểm quan trọng nhất cần nhớ (suy luận, mục 8) là: tốc độ ”ramp-up ngầm” của RAdam gắn chặt với β_2, không tự động khớp với độ dài lịch huấn luyện cụ thể như một warmup được chọn tay - nên nếu muốn dùng RAdam cho một lượt fine-tune ngắn trên GPU hạn chế, cần tính lại β_2 (hoặc kiểm tra bằng thí nghiệm 1, mục 9) trước khi thay thế trực tiếp recipe AdamW+warmup hiện có.

Nguồn gốc: https://arxiv.org/abs/1908.03265

Bài 10

Large Batch Optimization for Deep Learning: Training BERT in 76 minutes (2019)

Tác giả: Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, Kurt Keutzer, Cho-Jui Hsieh (Google, UC Berkeley, UCLA).
Venue / arXiv: arXiv:1904.00962 (v5, 03/01/2020); paper ghi "Published as a conference paper at ICLR 2020".
Nguồn: https://arxiv.org/abs/1904.00962
Chủ đề: optimizer LAMB (Layerwise Adaptive Moments for Batch training), một chiến lược tổng quát biến optimizer nhỏ-batch (SGD, Adam) thành optimizer cho batch cực lớn bằng cách chuẩn hoá và co giãn update theo từng layer; dùng để huấn luyện BERT với batch tới 64K trong 76 phút và ResNet-50 đạt độ chính xác state-of-the-art ở batch 32K.
Vị trí trong lộ trình: đọc sau opt 06-09 (Adam, AdamW, hội tụ của Adam/AMSGrad, RAdam) vì LAMB dùng Adam làm "base algorithm" rồi thêm lớp chuẩn hoá layerwise lên trên; đọc trước opt 11 (Lion, một optimizer "mới" khác cho giai đoạn pretraining LLM). Cũng liên quan tới opt 21-24 (large-batch training nhìn từ góc SGD/generalization gap) tuy các bài đó chưa được viết trong lộ trình này tại thời điểm ghi chú.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Huấn luyện các mô hình sâu trên tập dữ liệu lớn rất tốn thời gian: paper nêu ví dụ BERT-Large mất 3 ngày trên 16 chip TPUv3 và ResNet-50 mất 29 giờ trên 8 GPU Tesla P100 (trích Devlin et al. 2018; He et al. 2016). Để giảm thời gian, một hướng là **tăng batch size** và chạy song song đồng bộ (synchronous SGD) trên nhiều máy, vì khi đó mỗi bước cập nhật xử lý nhiều dữ liệu hơn cùng lúc. Nhưng tăng batch size "ngây thơ" (giữ learning rate cũ) thường làm giảm khả năng tổng quát hoá (generalization) và giảm lợi ích tính toán (Goyal et al., 2017, dẫn trong paper).

Hướng khắc phục: scale learning rate tuyến tính theo batch size, cộng với warmup (tăng dần LR ở đầu) và decay. Goyal et al. (2017) dùng cách này đưa ResNet-50 từ 29 giờ xuống 1 giờ với batch 8192. Nhưng kỹ thuật này cần tay chỉnh (hand-tuned) nhiều, và tới một batch size nào đó thì linear scaling không còn hiệu quả.

Một hướng khác là **learning rate thích nghi theo từng layer** (layerwise adaptive learning rate). Thuật toán tiêu biểu là **LARS** (You et al., 2017), cho phép train ResNet-50 trên ImageNet chỉ trong vài phút. Nhưng nhóm tác giả quan sát: LARS hoạt động kém với mô hình attention như BERT, tức lợi ích của nó không nhất quán giữa các kiến trúc. Hơn nữa, LARS thiếu phân tích lý thuyết (convergence) rõ ràng.

Câu hỏi của paper: có thể thiết kế một optimizer tổng quát, vừa adaptive theo layer vừa adaptive theo từng toạ độ tham số (như Adam), hoạt động tốt cho cả CNN (ResNet) và Transformer (BERT) ở batch cực lớn, kèm chứng minh hội tụ, không? Đó là động lực cho **LAMB** (Layerwise Adaptive Moments optimizer for Batch training).

Khái niệm cần nắm trước khi đọc (xem opt 06 Adam, opt 07 AdamW): momentum bậc 1 m_t (trung bình trượt của gradient), moment bậc 2 v_t (trung bình trượt của gradient bình phương theo từng toạ độ), bias correction $m_t/(1-\beta_1^t)$, $v_t/(1-\beta_2^t)$, và khái niệm "layer" ở đây là một nhóm tham số (ví dụ 1 ma trận trọng số của 1 tầng mạng).

2. Ý tưởng chính

Paper đề xuất một **chiến lược tổng quát (general strategy)** để biến bất kỳ optimizer cơ sở A (SGD hay Adam) có update layerwise $x_{t+1} = x_t + \eta_t u_t$ thành một optimizer phù hợp batch lớn, bằng 2 thay đổi áp dụng riêng cho từng layer i:

1. **Chuẩn hoá update về norm đơn vị**: thay u_t bằng $u_t/\|u_t\|$ (theo norm ℓ_2, tính riêng cho mỗi layer).
2. **Co giãn learning rate theo "trust ratio"** $\phi(\|x_t\|)$: learning rate hiệu dụng của layer đó tỉ lệ với norm của tham số hiện tại của layer, qua một hàm ϕ (paper dùng $\phi(z) = \min\{\max\{z, \gamma_l\}, \gamma_u\}$, tức clip vào $[\gamma_l, \gamma_u]$).

Trực giác: chuẩn hoá giúp optimizer bền với gradient nổ (exploding) hoặc biến mất (vanishing/plateau) vì bỏ qua độ lớn gradient, chỉ giữ hướng; còn trust ratio đảm bảo kích thước bước cập nhật của mỗi layer tỉ lệ với kích thước tham số của layer đó, thay vì dùng chung 1 learning rate toàn cục cho mọi layer (các layer có tham số lớn/nhỏ khác nhau nên "tốc độ học" khác nhau).

Áp dụng chiến lược này với A = SGD-với-momentum cho ra **LARS** (đã có từ trước, paper này phân tích lại); áp dụng với A = Adam cho ra **LAMB**, optimizer mới của paper. LAMB có 2 lớp adaptivity: (i) theo từng toạ độ như Adam (chia cho $\sqrt{v_t}$), và (ii) theo layer (trust ratio). Khi $\beta_1 = \beta_2 = 0$, LAMB suy biến thành SignSGD co giãn theo $\sqrt{d_i}$ (số chiều của layer i).

Kết quả chính: dùng LAMB, batch size của BERT pretraining có thể tăng tới 64K (giai đoạn 1, sequence length 128) / 32K (giai đoạn 2, sequence length 512) mà **không giảm** độ chính xác downstream (F1 trên SQuAD), đưa thời gian train BERT-Large từ 3 ngày xuống còn 76 phút trên TPUv3 Pod. Với ResNet-50/ImageNet, LAMB là optimizer thích nghi (adaptive) **đầu tiên** đạt độ chính xác ngang SGD-momentum/LARS ở batch lớn, trong khi Adam/AdaGrad/AdamW "thuần" đều thất bại rõ rệt.

3. Phương pháp chi tiết

3.1 Thiết lập bài toán (Mục 2 của paper)

Bài toán tối ưu không lồi (nonconvex) có regularization ℓ_2:

$$\min_{x \in \mathbb{R}^d} f(x) := \mathbb{E}_{s \sim P}[\ell(x, s)] + \frac{\lambda}{2}\|x\|^2 \tag{1}$$

với x là tham số mô hình, ℓ là loss, P là phân phối dữ liệu (không biết trước), λ là hệ số weight decay. Tham số được chia thành h block theo layer: $x = (x^{(1)}, \ldots, x^{(h)})$, mỗi block $x^{(i)} \in \mathbb{R}^{d_i}$. Giả định chuẩn trong lý thuyết tối ưu stochastic: hàm ℓ là L_i-smooth theo từng layer i (gradient Lipschitz với hằng số L_i), phương sai gradient stochastic bị chặn ($\mathbb{E}\|\nabla_i \ell(x, s) - \nabla_i f(x)\|^2 \le \sigma_i^2$), và gradient bị chặn theo từng toạ độ ($|[\nabla \ell(x, s)]_j| \le G$).

SGD cơ bản: $x_{t+1} = x_t - \eta_t \frac{1}{|S_t|} \sum_{s \in S_t} \nabla \ell(x_t, s) + \lambda x_t$. Với batch rất lớn $b = T$ (paper dùng ký hiệu này để phân tích lý thuyết, không phải cách chạy thực nghiệm), tốc độ hội tụ của SGD (Theorem 1, trích Ghadimi & Lan 2013b) là $\mathbb{E}\|\nabla f(x_a)\|^2 \le O\Big(\frac{(f(x_1) - f(x^*))L_\infty}{T} + \frac{\|\sigma\|^2}{T}\Big)$, phụ thuộc $L_\infty = \max_i L_i$ (giá trị smoothness xấu nhất trong mọi layer) nên hội tụ chậm khi các layer rất không đồng nhất.

3.2 Chiến lược tổng quát và thuật toán LARS

Với base algorithm A = SGD-momentum, áp dụng 2 thay đổi ở Mục 2 trên, LARS (Algorithm 1 trong paper) có update:

$$x_{t+1}^{(i)} = x_t^{(i)} - \eta_t \frac{\phi(\|x_t^{(i)}\|)}{\|m_t^{(i)}\|} m_t^{(i)}, \quad m_t = \beta_1 m_{t-1} + (1 - \beta_1)(g_t + \lambda x_t)$$

Định lý 2 (hội tụ LARS, trường hợp $\beta_1 = \lambda = 0$ để đơn giản) cho:

$$\mathbb{E}\left[\left(\frac{1}{\sqrt{h}}\sum_{i=1}^{h}\|\nabla_i f(x_a)\|\right)^2\right] \le O\left(\frac{(f(x_1)-f(x^*))L_{avg}}{T} + \frac{\|\sigma\|_1^2}{Th}\right)$$

với $L_{avg} = \frac{1}{h}\sum_i L_i$ thay vì L_∞: đây là điểm mấu chốt, vì $L_{avg} \le L_\infty$ nên bound này có thể tốt hơn SGD khi độ smoothness giữa các layer lệch nhau nhiều (một vài layer rất ”gai góc” nhưng phần lớn layer mượt).

3.3 Thuật toán LAMB (Algorithm 2, Mục 3.2)

LAMB dùng Adam làm base algorithm. Với $0 < \beta_1, \beta_2 < 1$, $\epsilon > 0$, tại mỗi bước $t = 1, \dots, T$:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = m_t/(1-\beta_1^t), \qquad \hat{v}_t = v_t/(1-\beta_2^t)$$

$$r_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} \qquad \text{(update ”kiểu Adam”, theo từng toạ độ)}$$

$$x_{t+1}^{(i)} = x_t^{(i)} - \eta_t \frac{\phi(\|x_t^{(i)}\|)}{\|r_t^{(i)} + \lambda x_t^{(i)}\|}\left(r_t^{(i)} + \lambda x_t^{(i)}\right)$$

Tỉ số $\frac{\phi(\|x_t^{(i)}\|)}{\|r_t^{(i)}+\lambda x_t^{(i)}\|}$ chính là **trust ratio** của layer i: nó so sánh ”độ lớn tham số hiện tại” với ”độ lớn update được đề xuất”, rồi co giãn để update không quá lớn/nhỏ so với tham số. Khi $\beta_1 = \beta_2 = 0$, $r_t = \text{sign}(g_t)$ và LAMB trở thành SignSGD co giãn theo $\sqrt{d_i}$ (trích dẫn Bernstein et al., 2018).

Siêu tham số mặc định (Appendix H, paper không cần tune khi tăng batch size): $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-6}$, weight decay $\lambda = 0.01$; chỉ learning rate η_t (dùng lịch polynomial decay $\eta_t = \eta_0 \times (1 - t/T)$, giống baseline BERT) và warmup ratio được co giãn theo batch size bằng ”square-root LR scaling rule” + ”linear-epoch warmup”.

Ví dụ tự tính (minh hoạ trust ratio, số tự chọn để dễ hình dung, không phải số trong paper): giả sử một layer có vector tham số với norm $\|x_t^{(i)}\| = 4$, và sau khi tính $r_t^{(i)} + \lambda x_t^{(i)}$ ta được norm $\|r_t^{(i)} + \lambda x_t^{(i)}\| = 0.5$. Nếu ϕ không clip (coi $\phi(z) = z$), trust ratio $= 4/0.5 = 8$. Với $\eta_t = 10^{-3}$, bước cập nhật hiệu dụng cho layer này có hệ số $\eta_t \times 8 = 8 \times 10^{-3}$, lớn hơn 8 lần learning rate ”thô”. Nếu một layer khác có $\|x_t^{(i)}\| = 0.1$ và $\|r_t^{(i)} + \lambda x_t^{(i)}\| = 0.5$ (cùng độ lớn update chuẩn hoá), trust ratio $= 0.1/0.5 = 0.2$, bước hiệu dụng chỉ 2×10^{-4}. Vậy layer có tham số ”to” được phép đi bước lớn hơn layer có tham số ”nhỏ”, dù update thô (trước chuẩn hoá) có độ lớn giống nhau: đây chính là ý nghĩa của ”layerwise adaptive”.

3.4 Biến thể và lựa chọn thiết kế khác (Appendix D, E, F)

- **N-LAMB / NN-LAMB** (Appendix D): thay momentum thường bằng Nesterov momentum (NAG, Nesterov 1983; áp dụng vào Adam theo Nadam, Dozat 2016) cho moment bậc 1 (N-LAMB) hoặc cả 2 moment (NN-LAMB), dùng $\beta_1 = 0.975, \beta_2 = 0.999, \epsilon = 10^{-8}$ (giống khuyến nghị Nadam). Kết quả: comparable với LAMB gốc, tốt hơn momentum SGD.
- **Loại bỏ ”adam-correction”** (Appendix E): phép bias-correction $m_t/(1-\beta_1^t)$, $v_t/(1-\beta_2^t)$ có tác dụng tương tự warmup learning rate (paper quan sát thực nghiệm hình dạng 2 đường cong trùng nhau); vì hệ thống deep learning hiện đại đã có warmup riêng, có thể bỏ adam-correction mà không giảm accuracy BERT/ImageNet.
- **Chọn norm khác cho bước chuẩn hoá** (Appendix F): thử các norm khác ngoài ℓ_2 cho trust ratio, khác biệt accuracy ImageNet dưới 0.1 điểm phần trăm; paper giữ ℓ_2 làm default.

4. Thí nghiệm và kết quả chính

4.1 BERT pretraining + fine-tune SQuAD (Mục 4.1, Bảng 1 và 2)

Dữ liệu: Wikipedia + BooksCorpus (2.5 tỷ + 800 triệu từ, giống Devlin et al. 2018). Đánh giá bằng F1 trên SQuAD-v1 sau khi fine-tune theo đúng quy trình gốc của BERT (không đổi epoch/hyperparameter fine-tune, trừ khi nói rõ). Baseline BERT-Large (checkpoint công khai) đạt F1 = 90.395, train 1000k bước, batch 512, 16 TPUv3, 81.4 giờ.

Bảng 1 (trích nguyên số liệu paper):

Optimizer	Batch	Steps	F1 (dev)	TPU	Thời gian
Baseline	512	1000k	90.395	16	81.4h
LAMB	512	1000k	91.752	16	82.8h
LAMB	1k	500k	91.761	32	43.2h
LAMB	2k	250k	91.946	64	21.4h
LAMB	4k	125k	91.137	128	693.6m
LAMB	8k	62500	91.263	256	390.5m
LAMB	16k	31250	91.345	512	200.0m
LAMB	32k	15625	91.475	1024	101.2m
LAMB	64k/32k (mixed)	8599	90.584	1024	**76.19m**

Ở batch 32768, LAMB đạt tăng tốc 49.1 lần với 64 lần tài nguyên tính toán (hiệu suất scaling 76.7%). Với kỹ thuật **mixed-batch training** (giai đoạn 1 sequence-length 128 dùng batch 65536, giai đoạn 2 sequence-length 512 dùng batch 32768, vì giới hạn memory TPU Pod ở sequence dài) cộng với **re-warm-up** (ở đầu giai đoạn 2, không decay tiếp learning rate cũ mà warmup lại từ 0), tổng thời gian giảm xuống 76 phút (hiệu suất scaling 100.2%, tức vượt mức lý thuyết tuyến tính do cách tính). Paper cũng nói rõ: nếu chỉnh lại learning rate fine-tune, LAMB đạt F1 91.688 ở batch 16K (cao hơn số 91.345 ”untuned” báo cáo trong bảng).

So sánh với AdamW và LARS: AdamW (3 cách tune khác nhau, grid search) **dừng scale ở batch 16K** vì không đạt target F1 (báo cáo 88.1 so với baseline 90.4); ở batch 64K/32K mixed, AdamW thất bại hoàn toàn dù đã tune kỹ (Bảng 8, Appendix: nhiều cấu hình AdamW ở batch 16K bị ”diverged”). Bảng 2: LAMB tốt hơn LARS ở **mọi** batch size, LARS bắt đầu ”diverge” ở batch 32K còn LAMB vẫn giữ F1 91.475.

4.2 ImageNet / ResNet-50 (Mục 4.2, Bảng 3)

Baseline (momentum SGD, Goyal et al. 2017): 76.3% top-1 sau 90 epoch, batch 16K. Trước paper này, chưa có công bố nào cho Adam/AdaGrad/AdamW đạt SOTA accuracy ở batch lớn trên ResNet-50. Thực nghiệm của paper (batch 16K, 90 epoch): AdaGrad/Adam/AdamW thuần chỉ đạt 55.38%/66.04%/67.27% top-1; thêm lịch LR của Goyal et al. (warmup 5 epoch + giảm LR ×0.1 ở epoch 30/60/80) cải thiện lên 72.0%/73.48%/73.07%, vẫn thấp hơn hẳn 76.3%. LAMB đạt 76.66% ở batch 16K (vượt baseline momentum 75.20% theo bảng), và ở batch 32K đạt 76.4% (gần LARS 76.3%); ở batch 2K, LAMB đạt tới 77.11% so với LARS 76.6%.

4.3 Tính robust về hyperparameter (Bảng 4, 5)

Dùng ”square-root LR scaling” (LR tăng theo $\sqrt{\text{batch size}}$) + ”linear-epoch warmup” (số bước warmup tăng tuyến tính theo epoch), LAMB **không cần tune lại** khi đổi batch size cho cả BERT (Bảng 4: batch 512 tới 32K, F1 giữ trong khoảng 91.1-91.9) và ResNet-50 (Bảng 5: batch 512 tới 32K, top-1 giữ 0.764-0.771, top-5 giữ quanh 0.93).

4.4 Thí nghiệm batch nhỏ, dataset nhỏ (Appendix G, Bảng 6, 7)

CIFAR-10 với DavidNet (batch 512, 24 epoch): LAMB đạt 94.08% test accuracy, cao hơn AdaGrad (90.74%), Adam (92.25%), AdamW (92.71%), momentum SGD (93.72% TF / 94.06% PyTorch baseline DAWNBench). MNIST với LeNet (batch 1024, 30 epoch, trung bình 5 lần chạy): LAMB 99.45% so với momentum 99.33%, AdaGrad 99.28%, Adam 99.36%, AdamW 99.41%. Vậy LAMB không chỉ tốt ở batch cực lớn mà còn ít nhất ngang bằng/tốt hơn ở batch "bình thường" trên bài toán nhỏ.

5. Phân tích / ablation

- **So sánh lý thuyết hội tụ** (Mục 3, Appendix C): bound của LARS/LAMB phụ thuộc L_{avg} (trung bình độ smoothness theo layer) thay vì L_∞ (giá trị lớn nhất) như SGD; định nghĩa $\psi_L, \psi_g, \psi_\sigma$ (Appendix C) cho thấy LARS tốt hơn SGD khi "gradient dày hơn đường cong và nhiễu" (gradient denser than curvature/stochasticity, $\psi_L \ll \psi_g^2$ và $\psi_\sigma \ll \psi_g^2$); ngược lại SGD có bound tốt hơn. Đây là so sánh **bound trên lý thuyết**, không phải chứng minh LAMB luôn thắng thực tế.
- Với LAMB (Định lý 3): khi $\beta_2 = 0$ bound tương tự dạng LARS (theo $\|\cdot\|_1$); khi $\beta_2 > 0$ (trường hợp dùng thực tế) bound có thêm hệ số $\sqrt{G^2 d/(h(1-\beta_2))}$, tức **lý thuyết cho thấy rate hơi kém hơn** so với $\beta_2 = 0$, dù tác giả tin thực nghiệm $\beta_2 > 0$ vẫn tốt hơn (paper nói rõ: "chúng tôi tin hành vi của nó nên tốt hơn trường hợp $\beta_2 = 0$, chúng tôi để việc điều tra này cho công việc tương lai" - đây là một khoảng hở lý thuyết/thực hành chưa khép lại trong chính bài báo).
- **Ablation Nesterov (N-LAMB/NN-LAMB)**: không cải thiện rõ so với LAMB gốc, chỉ "comparable", nhưng tốt hơn momentum thường.
- **Ablation norm khác**: không có khác biệt đáng kể (<0.1%).
- **Ablation bỏ adam-correction**: không giảm accuracy, cho thấy bias-correction của Adam có thể dư thừa nếu đã có warmup, một quan sát thú vị về sự chồng chéo công dụng giữa 2 kỹ thuật khác nguồn gốc.
- **Trust ratio theo layer** (Appendix H.0.2, Hình 9-14): trust ratio khác nhau rất nhiều giữa các layer của ResNet-50, xác nhận trực giác rằng cơ chế này giúp các layer "học chậm" (slow learners) bắt kịp.
- **Validation loss không đáng tin ở batch lớn** (Hình 5): loss thấp hơn không đồng nghĩa accuracy cao hơn, nên phải dùng accuracy/F1 thật để đánh giá, không chỉ loss.

6. Giới hạn

- Phân tích hội tụ đầy đủ chỉ làm cho trường hợp đơn giản hoá ($\beta_1 = 0, \lambda = 0$ cho LARS; $\beta_2 = 0$ cho phần chính của LAMB); trường hợp tổng quát ($\beta_1, \beta_2 > 0$ cùng lúc) được nói là "mở rộng được nhưng phép tính trở nên rối" và bound lý thuyết cho $\beta_2 > 0$ kém hơn $\beta_2 = 0$, một khoảng hở giữa lý thuyết và thực hành mà paper tự nhận và để lại cho tương lai.
- Kết quả tiêu đề (76 phút) đạt được trên **TPUv3 Pod 1024 chip**, hạ tầng mà tuyệt đại đa số người dùng/nhóm nghiên cứu nhỏ không có; LAMB về cấu trúc thuật toán không giảm memory so với Adam (vẫn giữ 2 moment m_t, v_t cùng kích thước tham số), lợi ích chính là cho phép batch rất lớn chạy song song, không phải giảm chi phí trên 1 GPU đơn.
- Thực nghiệm giới hạn ở 2 kiến trúc chính (BERT encoder, ResNet-50 CNN) cộng thêm bài toán nhỏ (CIFAR-10, MNIST); không có thử nghiệm trên decoder-only LLM hiện đại, mô hình đa phương thức (VLM), hay các kiến trúc attention tuyến tính/hybrid.
- Trust ratio dùng norm toàn layer, không xét tới cấu trúc nội tại ma trận (so với Shampoo/SOAP ở opt 14/15 sau này, dùng preconditioning theo cấu trúc tensor chi tiết hơn).
- Baseline AdamW trong paper được tune khá công bằng (grid search, nhiều cấu hình weight decay), nhưng vẫn là một nhóm nghiên cứu khác tự tune cho đối thủ của chính optimizer mình đề xuất, nên nên đọc số liệu so sánh với một mức độ cẩn trọng thông thường (không có gì mâu thuẫn rõ trong text, chỉ là điểm cần lưu ý khi diễn giải).

7. Liên hệ với các bài khác

- **opt 06 (Adam) và opt 07 (AdamW)**: LAMB dùng Adam làm ”base algorithm A”, thêm lớp trust-ratio layerwise lên trên; AdamW là baseline chính bị LAMB vượt qua ở batch lớn trong thí nghiệm BERT/ImageNet của paper này.
- **opt 11 (Lion, Symbolic Discovery of Optimization Algorithms)**: bài tiếp theo trong lộ trình opt, cũng là một optimizer thay thế Adam cho pretraining quy mô lớn, nhưng theo hướng tìm kiếm symbolic thay vì thiết kế layerwise adaptive; hai bài cho thấy 2 triết lý khác nhau để cải tiến Adam ở quy mô lớn.
- **opt 21-24 (theo titles.tsv: ”Accurate, Large Minibatch SGD” Goyal et al. 2017; ”An Empirical Model of Large-Batch Training”; ”Don’t Decay the Learning Rate, Increase the Batch Size”; ”On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima”)**: đây chính là các công trình được paper LAMB trích dẫn trực tiếp làm nền (Goyal et al. 2017 cho linear-scaling + warmup; Keskar et al. 2016 cho vấn đề generalization gap ở batch lớn); các bài này (chưa tồn tại thành file tại thời điểm viết ghi chú này, nhưng có số thứ tự cố định trong titles.tsv) sẽ đi sâu hơn vào khía cạnh SGD/generalization của cùng chủ đề ”large-batch training” mà LAMB tiếp cận từ phía adaptive-optimizer.
- **llm content/03_2020_scaling_laws_kaplan.md** (đã kiểm tra tồn tại bằng `ls`): bài scaling-laws của Kaplan et al. dùng **Adafactor** (không phải LAMB) cho các model >1 tỷ tham số và Adam cho các model nhỏ hơn; đây là một ví dụ khác về việc optimizer ”chuẩn” (Adam) không scale tốt khi kích thước mô hình/batch tăng, cùng động lực với LAMB nhưng giải pháp khác (Adafactor giảm memory, LAMB tăng khả năng dùng batch lớn).

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid Gated-DeltaNet + full attention, ~3.600 token/trang, chạy trên GPU <= 8 GB, **batch 1**). Đây là **(suy luận)**: LAMB được thiết kế và chứng minh lợi ích ở batch từ vài nghìn tới vài chục nghìn (512 tới 64K trong paper); ở batch 1 trên 1 GPU, lý do tồn tại của LAMB (chuẩn hoá update để chịu được learning rate lớn khi batch lớn, và trust ratio để cân bằng tốc độ học giữa các layer khi gradient noise giảm nhờ batch lớn) **không áp dụng trực tiếp**, vì với batch 1, phương sai gradient cao, learning rate hiệu dụng đã bị hạn chế bởi lý do khác (ổn định số, không phải do ”layer nào học nhanh/chậm ở quy mô lớn”). Do đó LAMB **ít liên quan trực tiếp** cho pipeline huấn luyện chính (batch 1) của dự án.

Tuy vậy, có 2 điểm có thể tham khảo gián tiếp (vẫn là suy luận, không phải khuyến nghị trực tiếp từ paper): (a) nếu trong giai đoạn distillation/finetune có dùng **gradient accumulation** để mô phỏng batch hiệu dụng lớn hơn (ví dụ accumulate 32-128 bước trước khi update, vẫn forward/backward batch 1 mỗi lần vì giới hạn 8 GB VRAM với ảnh VLM ~3.600 token), thì ý tưởng trust ratio (giới hạn update theo tỉ lệ norm tham số/update của từng layer, chứ không dùng 1 LR chung) có thể giúp ổn định hơn so với AdamW thuần khi batch hiệu dụng lớn lên, đặc biệt nếu các module khác nhau trong kiến trúc hybrid (GatedDeltaNet vs full-attention) có độ lớn tham số/gradient rất khác nhau; (b) quan sát của paper rằng validation loss không đáng tin ở batch lớn (Hình 5, Mục 5 trên) là một lời nhắc chung hữu ích: khi distill/finetune model OCR nhỏ, nên theo dõi metric đích thực (ví dụ accuracy đọc chữ, CER/WER) thay vì chỉ nhìn loss, dù đây không phải phát hiện đặc thù của batch lớn mà là nguyên tắc chung.

9. Thí nghiệm gợi ý

1. **So sánh AdamW vs ”LAMB-toy” khi tăng batch hiệu dụng bằng gradient accumulation**: huấn luyện 1 MLP/CNN nhỏ (ví dụ LeNet trên 1 phần MNIST, hoặc 1 Transformer nhỏ) trên GPU 8GB, cố định tổng số sample đã thấy, so sánh batch thật 32 (accumulate 1) với batch hiệu dụng 1024 (accumulate 32) dùng AdamW thường và dùng 1 bản tự cài trust-ratio theo Algorithm 2 của paper. Đo: accuracy cuối, có bị giảm khi batch hiệu dụng tăng với AdamW không, LAMB-toy có giữ accuracy tốt hơn không.

Kỳ vọng: AdamW giảm nhẹ ở batch hiệu dụng lớn nếu không chỉnh LR, LAMB-toy ổn định hơn nhờ tự co giãn theo layer.

2. **Đo trust ratio thực tế trên model nhỏ**: cài đặt công thức trust ratio (Mục 3.3) cho một mạng nhỏ vài layer, log giá trị $\phi(\|x_t^{(i)}\|)/\|r_t^{(i)}\|$ theo từng layer qua các bước huấn luyện. Đo: các layer có dao động trust ratio khác nhau bao nhiêu? Diễn giải: layer embedding/norm thường có norm tham số nhỏ, dễ có trust ratio cực trị, cần xem có cần clip $[\gamma_l, \gamma_u]$ hay không.
3. **Ablation adam-correction vs warmup**: trên cùng 1 bài toán nhỏ, so sánh 4 cấu hình (có/không bias-correction) x (có/không warmup). Đo: tốc độ hội tụ ở vài trăm bước đầu. Kỳ vọng theo paper: có warmup mà bỏ bias-correction không làm chậm hội tụ hơn so với có bias-correction mà không warmup, tức 2 kỹ thuật ”thế chỗ” được cho nhau ở mức độ nào đó trên bài toán nhỏ.
4. **So sánh norm ℓ_1, ℓ_2, ℓ_∞ cho bước chuẩn hoá update**: thử 3 norm khác nhau trong bước 1 của ”general strategy” (Mục 2) trên 1 CNN nhỏ, đo accuracy cuối. Kỳ vọng theo paper gốc (trên ResNet-50 lớn): khác biệt rất nhỏ (<0.1 điểm %); kiểm tra xem điều này còn đúng ở quy mô nhỏ/batch nhỏ không.
5. **Quan sát mối liên hệ validation loss vs accuracy**: huấn luyện 1 model nhỏ ở 2 batch size khác nhau (nhỏ và lớn hơn ~16-32 lần qua accumulation), vẽ cả loss và accuracy theo bước. Đo: có xảy ra hiện tượng ”loss thấp hơn nhưng accuracy không cao hơn” như Hình 5 của paper không, dù ở quy mô rất nhỏ so với thí nghiệm gốc.

10. Tóm tắt công thức / khái niệm

- **Bài toán** (eq. 1): $\min_x f(x) = \mathbb{E}_{s\sim P}[\ell(x,s)] + \frac{\lambda}{2}\|x\|^2$.
- **Chiến lược tổng quát** (Mục 2): $x_{t+1}^{(i)} = x_t^{(i)} - \eta_t \dfrac{\phi(\|x_t^{(i)}\|)}{\|u_t^{(i)}\|} u_t^{(i)}$, với u_t là update của optimizer cơ sở A, $\phi(z) = \min\{\max(z, \gamma_l), \gamma_u\}$.
- **LARS** (Algorithm 1): $u_t = m_t = \beta_1 m_{t-1} + (1-\beta_1)(g_t + \lambda x_t)$ (momentum SGD).
- **LAMB** (Algorithm 2): $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$; $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$; bias-correct $\hat{m}_t, \hat{v}_t$; $r_t = \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$; update layer i: $x_{t+1}^{(i)} = x_t^{(i)} - \eta_t \dfrac{\phi(\|x_t^{(i)}\|)}{\|r_t^{(i)} + \lambda x_t^{(i)}\|}(r_t^{(i)} + \lambda x_t^{(i)})$.
- **”Trust ratio”**: tên gọi (không chính thức, dùng trong Appendix H) cho hệ số $\phi(\|x_t^{(i)}\|)/\|r_t^{(i)} + \lambda x_t^{(i)}\|$ nhân vào mỗi layer.
- **Siêu tham số mặc định**: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-6}$, $\lambda = 0.01$ (weight decay); chỉ tune η_0 (learning rate gốc), dùng square-root LR scaling + linear-epoch warmup khi đổi batch size.
- **Định lý hội tụ** (Theorem 1, 2, 3): cho SGD, LARS, LAMB tương ứng, dạng $O\left(\frac{(f(x_1)-f(x^*))L}{T} + \frac{\text{noise}}{T}\right)$ với L là L_∞ (SGD) hay L_{avg} (LARS/LAMB khi $\beta_2 = 0$); LAMB với $\beta_2 > 0$ có thêm hệ số $\sqrt{G^2 d/(h(1-\beta_2))}$.
- **Khi** $\beta_1 = \beta_2 = 0$: LAMB $\equiv$ SignSGD co giãn theo $\sqrt{d_i}$ của layer.

11. Kết luận ngắn

LAMB mở rộng ý tưởng layerwise-adaptive (từ LARS) sang Adam, cho phép huấn luyện BERT ở batch tới 64K và ResNet-50 ở batch 32K mà hầu như không mất độ chính xác, đưa thời gian train BERT từ 3 ngày xuống 76 phút trên hạ tầng TPU Pod lớn, trong khi Adam/AdamW/AdaGrad/LARS ”thuần” đều thất bại hoặc kém hơn rõ rệt ở các batch size này. Cơ chế cốt lõi là trust ratio: chuẩn hoá update về norm đơn vị rồi co giãn lại theo tỉ lệ norm-tham-số/norm-update của từng layer. Paper cũng thẳng thắn để lại khoảng hở lý thuyết (bound cho $\beta_2 > 0$ kém hơn $\beta_2 = 0$) và giới hạn thực nghiệm ở 2 kiến trúc chính thời điểm 2019 (BERT, ResNet-50). Với dự án nén/hậu huấn luyện VLM OCR nhỏ ở batch 1, LAMB bản thân ít liên quan trực tiếp, nhưng ý tưởng trust-ratio vẫn là một tham chiếu hữu ích nếu dự án có giai đoạn dùng gradient accumulation để mô phỏng batch lớn hơn.

Nguồn gốc: https://arxiv.org/abs/1904.00962

Bài 11

Symbolic Discovery of Optimization Algorithms (2023)

Tác giả: Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, Quoc V. Le (Google, UCLA).
Venue / arXiv: arXiv:2302.06675 (v4, 08/05/2023); paper không nêu venue hội nghị cụ thể trên trang bìa (bản này công bố song song với NeurIPS 2023, nhưng văn bản tải không in dòng venue).
Nguồn: https://arxiv.org/abs/2302.06675
Chủ đề: dùng program search (tìm kiếm chương trình, kiểu AutoML) để tự động phát hiện thuật toán optimizer, từ đó tìm ra **Lion** (EvoLved Sign Momentum): optimizer chỉ lưu 1 trạng thái momentum, cập nhật bằng hàm sign, nhẹ hơn Adam về memory nhưng mạnh hơn trên nhiều benchmark lớn (ảnh, vision-language, diffusion, ngôn ngữ).
Vị trí trong lộ trình: đọc sau opt 10 (LAMB, cũng là một cách cải tiến Adam cho quy mô lớn nhưng bằng thiết kế thủ công layerwise) và opt 06-07 (Adam/AdamW, baseline chính mà Lion so sánh); đọc trước opt 12 (Adafactor). Cũng liên quan opt 16 (Muon) và opt 48 (”Old Optimizer, New Norm: An Anthology”), hai bài sau này tiếp tục bàn về các optimizer thay thế Adam cho LLM.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Dù có rất nhiều optimizer ”thủ công” (handcrafted) được đề xuất, Adam (kèm decoupled weight decay, gọi là AdamW) và Adafactor (second moment được factorize để giảm memory, xem opt 12) vẫn là lựa chọn mặc định cho hầu hết mô hình ngôn ngữ, ảnh, đa phương thức hiện đại (paper dẫn GPT-3, BERT, Transformer gốc, ViT, CLIP, Imagen...). Một hướng khác để tìm optimizer mới là **”learning to optimize” (L2O)**: huấn luyện một mạng neural để sinh ra update, nhưng các optimizer kiểu này thường chỉ train trên vài task nhỏ nên khó tổng quát hoá lên quy mô SOTA. Một hướng nữa dùng reinforcement learning hoặc Monte Carlo để tìm trong không gian cây biểu thức (ví dụ NOS của Bello et al. 2017), nhưng không gian tìm kiếm thường bị giới hạn (toán hạng/cấu trúc cố định) nên các optimizer tìm ra (PowerSign, AddSign) chưa đạt SOTA thực sự.

Paper này đặt câu hỏi: có thể biểu diễn việc tìm optimizer như tìm kiếm **chương trình (program)** (không phải mạng neural tham số hoá) trong một không gian đủ linh hoạt, rồi xử lý được vấn đề ”khoảng cách tổng quát hoá” (generalization gap) giữa proxy task nhỏ dùng để tìm kiếm và target task lớn, không? Đây là bài toán mở rộng trực tiếp từ AutoML-Zero (Real et al., 2020), vốn tìm toàn bộ pipeline ML trên toy task; ở đây tác giả thu hẹp phạm vi về riêng optimizer nhưng mở rộng để chạy được trên các bài toán SOTA thực sự (ảnh, ngôn ngữ, đa phương thức).

2. Ý tưởng chính

Biểu diễn một optimizer như một **chương trình** (hàm `train(w, g, m, v, lr)` nhận trọng số, gradient, các biến trạng thái, learning rate, trả ra update và trạng thái mới), gồm một chuỗi câu lệnh gán, mỗi câu lệnh gọi 1 trong 45 hàm toán học cơ bản (giống NumPy/JAX) lên các biến/hằng số. Dùng **tiến hoá (evolutionary search), có warm-start từ AdamW**, kết hợp 2 kỹ thuật chính để xử lý không gian vô hạn và thưa (sparse):

- **Pruning bằng abstract execution**: trước khi chạy thật, suy luận kiểu/shape để loại chương trình lỗi, hash chương trình để cache/loại chương trình trùng ngữ nghĩa, và loại câu lệnh dư (redundant).
- **Funnel selection + simplification**: chọn chương trình tốt trên proxy task nhỏ (chạy 20 phút/1 TPU V2), rồi kiểm tra trên các ”meta-validation task” tăng dần kích thước (để tránh ”meta-overfit” vào proxy), cuối cùng đơn giản hoá chương trình còn lại bằng tay.

Kết quả tìm kiếm là **Lion (EvoLved Sign Momentum)**: chỉ theo dõi 1 biến trạng thái (momentum m), cập nhật bằng toán tử `sign`, nên **nhẹ hơn Adam** (Adam cần cả m và v). Khác với các optimizer adaptive (Adam, AdaGrad...), **mọi toạ độ của update đều có cùng độ lớn** (chỉ là $\pm$ learning rate, trước khi cộng weight decay) vì đi qua hàm sign. Lion vượt AdamW/Adafactor trên nhiều benchmark lớn: ViT trên ImageNet (+tới 2%), tiết kiệm tới 5x compute pretraining trên JFT, đạt 88.3% zero-shot / 91.1% fine-tune ImageNet cho mô hình vision-language BASIC-L (vượt SOTA trước đó 2% và 0.1%), và cải thiện FID cho diffusion model (giảm tới 2.3x compute). Trên ngôn ngữ (autoregressive LM, masked LM, fine-tune GLUE), Lion bằng hoặc tốt hơn AdamW.

3. Phương pháp chi tiết

3.1 Không gian tìm kiếm chương trình (Mục 2.1)

Chương trình gồm hàm `train` với input chính là `weight (w)`, `gradient (g)`, giá trị `learning rate schedule (lr)` tại bước hiện tại; output là update cho trọng số; có thêm các biến phụ khởi tạo 0 để lưu thông tin lịch sử (ví dụ Adam cần 2 biến phụ cho m, v). Chương trình là chuỗi câu lệnh gán không hạn chế số lượng, mỗi câu lệnh gọi 1 trong 45 hàm (tương ứng hàm NumPy hoặc phép toán đại số tuyến tính), trong đó có hàm tiện ích `interp(x, y, a) = (1-a)x + ay` (nội suy tuyến tính). Paper **không** cho phép câu lệnh điều kiện/vòng lặp hay định nghĩa hàm mới vì thử nghiệm sơ bộ không cải thiện kết quả. Số chương trình khả dĩ là $n_p = n_f^l n_v^{n_a \cdot l}$ (paper ước lượng thô, với n_f số hàm, n_v số biến, n_a số đối số trung bình/câu lệnh, l độ dài chương trình), tức **vô hạn/rất lớn**, và phần lớn chương trình trong không gian này hoạt động rất kém (thử random search 2 triệu chương trình trên 1 proxy task rẻ, kết quả tốt nhất vẫn kém AdamW đáng kể).

3.2 Kỹ thuật tìm kiếm hiệu quả (Mục 2.2)

- **Regularized evolution** (quần thể $P = 1000$ chương trình, tournament size $T = 2$), warm-start quần thể ban đầu bằng AdamW (không khởi tạo random) để tăng tốc.
- **Restart**: (1) restart từ chương trình gốc (AdamW) nhiều lần song song để tránh kẹt ở 1 local optimum; (2) restart từ chương trình tốt nhất tìm được để tiếp tục khai thác (exploit) khi fitness đã chững lại (paper: sau khoảng 300K "progress").
- **Abstract execution**: suy luận kiểu/shape để lọc chương trình lỗi, hash để cache chương trình trùng ngữ nghĩa và phát hiện câu lệnh dư. Theo 5 lần search, mỗi lần 300K chương trình: tỉ lệ câu lệnh dư cuối quá trình là $69.8\% \pm 1.9\%$ (làm chương trình ngắn lại khoảng 3 lần), tỉ lệ cache hit $89.1\% \pm 0.6\%$ (giảm chi phí tìm kiếm khoảng 10 lần).
- **Proxy task**: giảm kích thước model/dữ liệu/bước huấn luyện để chạy xong trong 20 phút/1 chip TPU V2; mỗi lần search dùng 100 chip TPU V2 chạy khoảng 72 giờ, sinh 200-300K chương trình (nhưng chỉ thật sự evaluate khoảng 20-30K nhờ cache), tổng chi phí khoảng 3.000 ngày-TPU-V2 (bao gồm cả restart).

3.3 Chống "meta-overfitting" và đơn giản hoá (Mục 2.3)

Chương trình có thể fit tốt proxy task nhưng không tổng quát hoá: gọi là **meta-overfitting**, khi fitness trên search tiếp tục tăng nhưng metric trên "meta-validation task" (task lớn hơn proxy) lại giảm. Paper dùng **funnel selection**: tăng dần kích thước meta-validation task (ví dụ proxy A, rồi task B lớn hơn 10 lần, chỉ chương trình vượt baseline trên B mới được đánh giá tiếp trên task C lớn hơn 100 lần), giúp lọc dần các chương trình tổng quát kém. Quan sát: search nào "meta-overfit muộn hơn" (ở % progress cao hơn) thường cho ra chương trình tổng quát hoá tốt hơn.

Sau khi chọn được chương trình tốt, **đơn giản hoá** bằng cách: xoá câu lệnh dư (xác định bằng abstract execution), xoá câu lệnh không dư nhưng ảnh hưởng rất nhỏ tới output, rồi sắp xếp lại và đặt tên biến bằng tay, viết lại dạng toán học tương đương đơn giản hơn.

3.4 Suy ra thuật toán Lion (Mục 3.1, Program 1-4)

Chương trình thô tìm được (Program 8, Appendix) sau khi loại câu lệnh dư cho ra Program 4, rồi đơn giản hoá tiếp (bỏ hàm `cosh` vì m sẽ bị ghi lại ở vòng lặp sau; bỏ `arcsin` và `clip` vì không giảm chất lượng khi bỏ; 3 câu lệnh "đỏ" gộp lại thành 1 phép `sign`; nhận ra v chỉ ảnh hưởng tới cách cập nhật momentum, 2 phép `interp` với hằng số ≈ 0.9 và ≈ 1.1 tương đương 1 phép `interp` với hằng số ≈ 0.99 nên không cần lưu v riêng; bỏ bias-correction vì không đổi hướng update) cho ra **Program 1 (Lion)**:

```
def train(weight, gradient, momentum, lr):
    update = interp(gradient, momentum, beta1)      # noi suy: (1-beta1)*g + beta1*m
    update = sign(update)
    momentum = interp(gradient, momentum, beta2)     # noi suy: (1-beta2)*g + beta2*m
    weight_decay = weight * lambda_
    update = update + weight_decay
    update = update * lr
    return update, momentum
```

với $\beta_1 = 0.9,\ \beta_2 = 0.99$ mặc định (tự "tìm ra" từ quá trình search, không phải chọn tay). Viết lại dưới dạng công thức toán quen thuộc hơn: gọi $c_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ (nội suy giữa momentum cũ và gradient hiện tại với hệ số β_1), update tham số:

$$x_t = x_{t-1} - \eta_t \big(\text{sign}(c_t) + \lambda x_{t-1}\big), \qquad m_t = \beta_2 m_{t-1} + (1-\beta_2) g_t$$

Lưu ý: c_t (dùng để tính hướng update qua `sign`) và m_t (trạng thái lưu lại cho bước sau) dùng **hai hệ số nội suy khác nhau** (β_1 cho c_t, β_2 cho việc cập nhật m lưu trữ) chứ không phải chỉ 1 EMA như Adam/SGD-momentum thông thường. Đây là điểm khác biệt quan trọng với NAdam (Dozat, 2016): NAdam kết hợp moment bậc 1 đã cập nhật với gradient hiện tại để tính update, còn Lion "tách" việc theo dõi momentum (qua β_2) khỏi việc dùng nó trong update (qua β_1).

Ví dụ tự tính: giả sử tại bước t, $m_{t-1} = 0.6$ (giá trị trung bình trượt cũ của 1 toạ độ gradient), gradient hiện tại $g_t = -0.2$, $\beta_1 = 0.9, \beta_2 = 0.99$, learning rate $\eta_t = 10^{-4}$, weight decay $\lambda = 0.01$, tham số hiện tại $x_{t-1} = 2.0$.

- Tính $c_t = \beta_1 m_{t-1} + (1-\beta_1) g_t = 0.9 \times 0.6 + 0.1 \times (-0.2) = 0.54 - 0.02 = 0.52$.
- $\text{sign}(c_t) = \text{sign}(0.52) = +1$.
- Weight decay term: $\lambda x_{t-1} = 0.01 \times 2.0 = 0.02$.
- Update (trước khi nhân η_t): $+1 + 0.02 = 1.02$.
- Bước cập nhật tham số: $x_t = x_{t-1} - \eta_t \times 1.02 = 2.0 - 10^{-4} \times 1.02 = 2.0 - 0.000102 = 1.999898$.
- Cập nhật trạng thái lưu cho bước sau: $m_t = \beta_2 m_{t-1} + (1-\beta_2) g_t = 0.99 \times 0.6 + 0.01 \times (-0.2) = 0.594 - 0.002 = 0.592$.

Quan sát từ ví dụ: dù g_t âm, vì m_{t-1} dương và $\beta_1 = 0.9$ áp đảo, hướng update vẫn là $+1$ (tức tham số **giảm** đi, vì dấu trừ trong công thức cập nhật) theo chiều momentum cũ, minh hoạ cách Lion "nhớ" xu hướng dài hạn hơn Adam dù hàm sign bỏ hết thông tin độ lớn gradient.

3.5 Hướng dẫn chọn siêu tham số (Mục 5)

Vì update của Lion là ± 1 theo từng toạ độ (trước khi cộng weight decay), **norm của update lớn hơn** so với Adam/SGD (vốn có độ lớn phụ thuộc gradient thật). Do đó:

- Learning rate của Lion nên **nhỏ hơn AdamW khoảng 3-10 lần**.
- Để giữ "cường độ" weight decay hiệu dụng không đổi (vì cách tính là `update += w*λ; update *= lr`, tức weight decay hiệu dụng tỉ lệ $lr \times \lambda$), weight decay λ của Lion nên **lớn hơn AdamW khoảng 3-10 lần**.

- Ví dụ cụ thể paper cho (trích đúng số liệu):
  - ViT-B/16 trên ImageNet với augmentation mạnh: Lion $lr = 10^{-4}, \lambda = 10.0$ so với AdamW $lr = 10^{-3}, \lambda = 1.0$.
  - Diffusion model: Lion $lr = 3 \times 10^{-5}, \lambda = 0.1$ so với AdamW $lr = 3 \times 10^{-4}, \lambda = 0.01$.
  - Mô hình ngôn ngữ 7.5B: Lion $lr = 10^{-4}, \lambda = 0.01$ so với Adafactor $lr = 10^{-3}, \lambda = 0.001$.
- $\beta_1 = 0.9, \beta_2 = 0.99$ là mặc định; với bài toán ngôn ngữ, paper quan sát việc tune lại ($\beta_1 = 0.95, \beta_2 = 0.98$ cho Lion; $\beta_1 = 0.9, \beta_2 = 0.99$ cho AdamW, so với mặc định $\beta_2 = 0.999$) cải thiện chất lượng cho cả 2 optimizer; ϵ của AdamW được đặt 10^{-6} (thay mặc định 10^{-8}) để ổn định hơn, tương tự quan sát trong RoBERTa.
- Lion không cần ϵ hay các tham số liên quan tới factorization (so với Adafactor), nên về số siêu tham số thì đơn giản hơn.

4. Thí nghiệm và kết quả chính

4.1 Phân loại ảnh (Mục 4.1)

- Train from scratch trên ImageNet (ResNet-50, 90 epoch, batch 1024; các mô hình khác 300 epoch, batch 4096), Bảng 2: Lion vượt AdamW trên hầu hết kiến trúc, ví dụ ViT-B/16 (không RandAug/Mixup): AdamW 75.48% vs Lion 77.44% top-1 ImageNet (+1.96 điểm theo cách paper diễn giải "1.96%"); ViT-S/16: AdamW 76.12% vs Lion 76.70% (+0.58%). Với augmentation mạnh (RandAug+Mixup), ViT-B/16: AdamW 80.12% vs Lion 80.77%; CoAtNet-3: AdamW 84.45% vs Lion 84.87% (+0.42 điểm, dù dùng regularization mạnh).
- Pretrain trên JFT rồi fine-tune ImageNet (Bảng 3): với ViT-L/16, Lion cho phép đạt độ chính xác ngang ViT-H/14 huấn luyện bằng AdamW nhưng với chi phí pretraining ít hơn 3 lần (ImageNet, ImageNet V2); trên ImageNet ReaL mức tiết kiệm lên tới 5 lần. Ở scale lớn hơn (JFT-3B), ViT-g/14 huấn luyện bằng Lion vượt ViT-G/14 (AdamW/Adafactor) trước đó dù có 1.8 lần ít tham số hơn; ViT-G/14 (Lion) đạt 90.71% trên ImageNet.

4.2 Vision-language contrastive (Mục 4.2)

- LiT (locked-image text tuning), huấn luyện 1 tỷ cặp ảnh-văn bản, batch 16384 (Bảng 4): Lion cải thiện zero-shot ImageNet accuracy +1.10% (LiT-B/32-B), +1.13% (LiT-B/16-B), +0.66% (LiT-g/14-L) so với AdamW.
- **BASIC-L** (Pham et al., 2021), batch 65536, pretrain trên JFT-5B, contrastive trên 6.6 tỷ cặp ảnh-văn bản (Bảng 1, trích nguyên văn): Adafactor 85.7% zero-shot ImageNet / 90.9% fine-tune, Lion 88.3% zero-shot / 91.1% fine-tune, tức Lion vượt SOTA trước đó (Yu et al. 2022: 86.3% zero-shot, 91.0% fine-tune) 2.0 điểm và 0.1 điểm tương ứng. Các benchmark robustness khác (V2, A, R, Sketch, ObjectNet) cũng đều tăng khi dùng Lion.

4.3 Diffusion model (Mục 4.3)

- Sinh ảnh ImageNet (U-Net cải tiến theo Dhariwal & Nichol 2021, batch 2048, learning rate hằng số, DDPM 1000 bước decode không guidance): ở độ phân giải 256x256, Lion đạt FID cuối của AdamW chỉ với số bước ít hơn 2.3 lần; FID cuối cùng: Lion 4.1 so với AdamW 4.7 (tham khảo: FID của ADM là 10.94).
- Text-to-image theo setup Imagen: không có cải thiện rõ trên model base 64x64, nhưng Lion tốt hơn AdamW trên model super-resolution 64x64 → 256x256 (CLIP score cao hơn, FID ít noise hơn).

4.4 Mô hình ngôn ngữ và fine-tuning (Mục 4.4)

- Autoregressive LM trên Wiki-40B, PG-19 (Transformer 110M/336M/731M, batch 2^18 token, 125K bước): Lion luôn đạt perplexity validation thấp hơn AdamW; tăng tốc 1.6x (Wiki-40B) và 1.5x (PG-19) ở model medium, tăng lên 2x ở model large trên PG-19.

- LM quy mô lớn (1.1B-7.5B tham số, 300B token, batch 3M token, giống setup GLaM/GPT-3), Bảng 5: trên cùng bộ dữ liệu khổng lồ này, **không thấy khác biệt perplexity** giữa Adafactor và Lion trong suốt training, nhưng Lion vẫn tốt hơn trên khả năng in-context learning (Avg NLG/NLU): ví dụ model 7.5B, Avg NLG Adafactor 24.1 vs Lion 24.7, Avg NLU 61.3 vs 61.7; model 7.5B train 300B token (Lion) vượt PaLM 8B train 780B token trên các benchmark này.
- Masked LM trên C4 (giống BERT): Lion tốt hơn AdamW về validation perplexity: 4.18 vs 4.25 (small), 3.42 vs 3.54 (medium), 3.18 vs 3.25 (large).
- Fine-tune T5 (Base/Large/11B) trên GLUE (Bảng 6, 500K bước, batch 128, LR hằng số): Lion thắng AdamW trên trung bình cả 3 scale, thắng 10/12, 12/12, 10/12 trong các cột điểm số tương ứng Base/Large/11B.

4.5 So sánh với các optimizer khác (Mục 4.5, Bảng 7)

So với RAdam, NAdam, AdaBelief, AMSGrad (handcrafted) và PowerSign, AddSign (tìm bằng AutoML trước đây), trên ViT-S/16 và ViT-B/16/ImageNet (có RandAug+Mixup, mọi optimizer đều được tune kỹ lr và λ): Lion vẫn là tốt nhất (ví dụ ViT-S/16 ImageNet: Lion 79.46% so với AMSGrad 79.01%, AdamW 78.89%, các optimizer tìm tự động cũ PowerSign/AddSign chỉ 77.36%/77.37%); không có ”người thắng rõ ràng” giữa các baseline còn lại.

5. Phân tích / ablation

- **Ablation cả β_1, β_2 (Mục 4.6, Bảng 7, cột Ablation0.9/Ablation0.99)**: thay Lion bằng rule đơn giản hơn `m = interp(g, m, β); update = sign(m)` (chỉ 1 hệ số β, không tách β_1/β_2). Cả 2 biến thể ($\beta = 0.9$ và $\beta = 0.99$) đều **kém hơn** tất cả 5 baseline đã so sánh, chứ không chỉ kém Lion đầy đủ, đây là bằng chứng rõ paper đưa ra cho sự cần thiết của việc tách 2 hệ số nội suy.
- **Ảnh hưởng batch size (Mục 4.6, Hình 8 trái)**: huấn luyện ViT-B/16 trên ImageNet với các batch size khác nhau (giữ tổng 300 epoch, có RandAug+Mixup): batch tối ưu của AdamW là 256, còn của Lion là 4096, tức **Lion ”thích” batch lớn hơn**; tuy vậy Lion vẫn hoạt động ổn ở batch nhỏ 64; ở batch 32K (chỉ 11K bước), Lion đạt 77.9% so với AdamW 75.4% (+2.5 điểm), cho thấy Lion đặc biệt mạnh ở chế độ batch rất lớn.
- **Độ nhạy siêu tham số (Hình 8 giữa/phải)**: heatmap theo lr, λ cho thấy Lion **ít nhạy hơn** AdamW với lựa chọn lr, λ (vùng ”tốt” rộng hơn trên heatmap).
- **Diễn giải cơ chế regularization**: hàm sign làm norm update đồng nhất theo mọi toạ độ, được paper coi là một dạng nhiễu đóng vai trò regularization; minh chứng: ViT-B/16 huấn luyện bằng Lion có **training error cao hơn** AdamW nhưng **validation accuracy cao hơn 2%** (Hình 11 phải, Phụ lục) - một dấu hiệu gián tiếp của regularization giúp tổng quát hoá tốt hơn, không phải chỉ fit tốt hơn trên train.
- **Memory/runtime** (Mục 3.2): Lion chỉ lưu m (so với m, v của AdamW) nên nhẹ hơn; ví dụ cụ thể paper cho: AdamW cần tối thiểu 16 chip TPU V4 để train ViT-B/16 (resolution 224, batch 4096), Lion chỉ cần 8 chip (cả 2 dùng bfloat16 cho momentum); runtime nhanh hơn AdamW/Adafactor khoảng 2-15% tuỳ task/codebase/hardware.

6. Giới hạn

Paper có mục ”Limitations” riêng (Mục 6), nêu rõ (trích lại, không bịa thêm):

- **Giới hạn của phương pháp search**: không gian tìm kiếm vẫn bị ảnh hưởng bởi các optimizer bậc nhất đã biết nên có thiên vị (bias) về phía các thuật toán tương tự; thiếu các hàm cần để xây dựng thuật toán bậc hai (second-order, như Shampoo opt 14); chi phí search vẫn lớn và việc đơn giản hoá chương trình cuối cùng cần can thiệp tay; cấu trúc chương trình (không có điều kiện/vòng lặp/hàm con) còn đơn giản.
- **Giới hạn của Lion**: trên các tác vụ ảnh dùng ResNet, khác biệt giữa Lion/AdamW/momentum SGD khá nhỏ (paper giải thích: ConvNet dễ optimize hơn Transformer). Lợi ích của Lion giảm khi dùng augmentation mạnh. Có 3 trường hợp Lion chỉ **tương đương** AdamW: (1) model text-to-image base

64x64 của Imagen, (2) perplexity của autoregressive LM trên dataset nội bộ quy mô rất lớn (paper coi đây là metric kém tin cậy hơn benchmark in-context learning), (3) masked LM trên C4; đặc điểm chung của các trường hợp này là dataset khổng lồ và chất lượng cao, làm giảm khác biệt giữa optimizer. Lion cũng **có thể không tốt hơn** AdamW ở batch nhỏ (<64). Cuối cùng, Lion vẫn cần lưu momentum ở bfloat16, có thể tốn kém với model khổng lồ; paper gợi ý factorize momentum (giống Adafactor) là một hướng giải quyết khả thi nhưng chưa làm trong bài.

- Một điểm người ghi chú nhận xét thêm (không phải paper nói): phần lớn kết quả nổi bật nhất (BASIC-L, JFT-3B, LM 7.5B) đều ở quy mô hạ tầng rất lớn (hàng trăm tới hàng nghìn chip TPU), nên các số liệu tiết kiệm compute ”X lần” cần hiểu trong ngữ cảnh pretraining quy mô lớn, chưa chắc còn đúng tỉ lệ ở quy mô finetune nhỏ trên 1 GPU.

7. Liên hệ với các bài khác

- **opt 06 (Adam) / opt 07 (AdamW)**: Lion được ”tiến hoá” từ chương trình khởi tạo chính là AdamW (Program 3 trong paper), và là baseline so sánh chính trong hầu hết thí nghiệm; hiểu Adam/AdamW là điều kiện cần để đọc paper này.
- **opt 10 (LAMB)**: cùng mục tiêu ”cải tiến Adam ở quy mô lớn” nhưng theo 2 triết lý đối lập: LAMB dùng thiết kế layerwise thủ công (trust ratio) để chịu được batch cực lớn mà không đổi hướng update cơ bản của Adam; Lion dùng tìm kiếm tự động (program search) để tìm ra một công thức update hoàn toàn khác (sign-based, bỏ hẳn second moment).
- **opt 12 (Adafactor)**: Adafactor là baseline so sánh thứ hai trong paper này (dùng khi Adam/AdamW tốn memory quá nhiều, ví dụ cho LM 1.1B-7.5B và BASIC-L); paper Lion trích rõ Adafactor factorize second moment để giảm memory, một cách tiếp cận khác với việc Lion bỏ hẳn second moment.
- **opt 48 (”Old Optimizer, New Norm: An Anthology”)**: theo tên bài này trong titles.tsv, nhiều khả năng phân tích lại Lion (và SignSGD, Muon) dưới một khung lý thuyết thống nhất (steepest descent theo norm nào đó); đây là một liên hệ về mặt lộ trình, chưa thể xác nhận nội dung cụ thể vì chưa đọc bài đó.
- **opt 16 (Muon is Scalable for LLM Training)**: một optimizer khác nổi lên sau Lion (2025) cũng nhằm thay thế AdamW cho pretraining LLM, dùng cơ chế orthogonalization (Newton-Schulz) khác hẳn sign-based; đáng so sánh triết lý ”đơn giản hoá/chuẩn hoá update” giữa 2 bài.
- **llm content/01_2017_attention_is_all_you_need.md** (đã kiểm tra tồn tại bằng `ls`): Lion được thử nghiệm chủ yếu trên kiến trúc Transformer (ViT, text Transformer, T5); hiểu kiến trúc attention cơ bản (bài 01 llm content) giúp hình dung rõ hơn các mô hình trong Bảng 2, 5, 6 của paper này, dù paper Lion không đi sâu vào chi tiết kiến trúc.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU <= 8 GB, batch 1). **(suy luận)** Lion có 2 đặc điểm phù hợp khá tốt với ràng buộc này:

- **Memory**: Lion chỉ lưu 1 trạng thái (momentum) thay vì 2 như AdamW. Với model 0.8B tham số, AdamW cần thêm xấp xỉ $2 \times 0.8 \times 10^9 \times 4$ byte ≈ 6.4 GB cho m, v ở fp32 (tự tính, giả định lưu optimizer state ở fp32; nếu dùng bf16 thì khoảng 3.2GB), trong khi Lion chỉ cần khoảng một nửa số đó cho m (khoảng 3.2 GB ở fp32, hoặc 1.6GB ở bf16 như paper dùng). Trên GPU 8GB, khoản tiết kiệm này (tự tính, ước lượng thô, chưa tính activation/gradient/weights) có thể là khác biệt giữa ”chạy được” và ”OOM” khi finetune/distill full-parameter.
- **Batch nhỏ**: đây là điểm cần lưu ý ngược lại: paper nói rõ (Mục 6, Hình 8) lợi ích của Lion **tăng theo batch size** và có thể **không tốt hơn AdamW ở batch < 64**; dự án train với batch 1 (có thể dùng gradient accumulation để batch hiệu dụng lớn hơn), nên nếu accumulate ít bước thì theo đúng paper, Lion chưa

chắc có lợi thế về chất lượng so với AdamW, dù vẫn lợi về memory. Nếu accumulate đủ nhiều (batch hiệu dụng hàng trăm, hàng nghìn) thì cả lợi thế memory và chất lượng có thể cùng xuất hiện.

- **Tuning**: Lion cần learning rate nhỏ hơn AdamW 3-10 lần và weight decay lớn hơn 3-10 lần tương ứng (Mục 5); nếu thử Lion cho dự án, không nên tái dùng trực tiếp config AdamW cũ mà phải chỉnh theo tỉ lệ này, nếu không rất dễ bị học quá nhanh/chậm so với kỳ vọng.

Tóm lại (suy luận): Lion là một lựa chọn **đáng thử** cho giai đoạn finetune/distill nếu VRAM là nút thắt chính, nhưng cần thử nghiệm thực tế trên chính model/dữ liệu OCR vì paper không có benchmark nào ở quy mô 0.8B/batch nhỏ giống dự án.

9. Thí nghiệm gợi ý

1. **So sánh memory thực tế AdamW vs Lion** trên 1 mô hình nhỏ (ví dụ 1 Transformer vài trăm triệu tham số) trên GPU 8GB: đo VRAM optimizer-state thực tế (qua `torch.cuda.max_memory_allocated`), kỳ vọng Lion thấp hơn AdamW gần 2 lần cho phần optimizer state, khớp với "tự tính" ở Mục 8.
2. **Ablation batch size nhỏ**: finetune 1 model nhỏ ở batch thật 1, 8, 64 (accumulate để mô phỏng batch hiệu dụng khác nhau), so sánh AdamW (đã tune) với Lion (đã chỉnh lr/λ theo tỉ lệ 1/5 và x5 gợi ý ở Mục 3.5). Đo: chất lượng cuối (loss/accuracy) theo batch. Kỳ vọng theo Mục 5-6 của paper: khoảng cách Lion-AdamW thu hẹp hoặc đảo chiều ở batch rất nhỏ.
3. **Grid nhỏ quét lr/λ cho Lion**: trên 1 bài toán nhỏ, quét lr trong {1e-5, 3e-5, 1e-4, 3e-4} và λ trong {0.01, 0.1, 1.0, 10.0}, vẽ heatmap tương tự Hình 8 (giữa/phải) của paper. Đo: vùng "tốt" của Lion có rộng hơn vùng "tốt" của AdamW (quét tương tự) không, với scale bài toán nhỏ hơn hẳn paper gốc.
4. **Kiểm tra giả thuyết sign update = regularization**: huấn luyện 1 model nhỏ bằng AdamW và Lion tới cùng mức training loss (dừng sớm AdamW nếu cần), so sánh validation loss/accuracy. Kỳ vọng theo Mục 5 (paper): Lion có thể có training loss cao hơn nhưng validation tốt hơn hoặc ngang, phản ánh hiệu ứng regularization của hàm sign.
5. **Thử Lion cho giai đoạn distill LightOnOCR** (thực nghiệm sát dự án nhất, cần code riêng): dùng Lion thay AdamW cho 1 lần finetune/distill ngắn (ví dụ vài trăm bước) trên 1 tập nhỏ dữ liệu OCR, so sánh tốc độ giảm loss và VRAM peak với cấu hình AdamW hiện tại của dự án. Diễn giải: nếu VRAM giảm rõ và chất lượng không tệ hơn, Lion đáng xem xét làm optimizer mặc định cho các lần finetune tiếp theo.

10. Tóm tắt công thức / khái niệm

- **Chương trình tìm kiếm**: hàm `train(w, g, trạng thái, lr) -> (update, trạng thái mới)`, mỗi câu lệnh gọi 1 trong 45 hàm cơ bản; dùng evolutionary search + abstract execution (hash, lọc lỗi) + funnel selection (chống meta-overfitting) + simplification tay.
- **Lion (Program 1)**: với $\beta_1 = 0.9, \beta_2 = 0.99$ mặc định,

$$c_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad x_t = x_{t-1} - \eta_t\big(\text{sign}(c_t) + \lambda x_{t-1}\big), \qquad m_t = \beta_2 m_{t-1} + (1-\beta_2) g_t$$

  (2 hệ số nội suy khác nhau cho "hướng update" c_t và "trạng thái lưu" m_t; không cần bias-correction vì hàm sign không đổi theo scale).
- **Quy đổi siêu tham số so với AdamW**: $lr_{Lion} \approx lr_{AdamW}/3$ đến $lr_{AdamW}/10$; $\lambda_{Lion} \approx 3\lambda_{AdamW}$ đến $10\lambda_{AdamW}$ (giữ $lr \times \lambda$ gần tương đương).
- **Memory**: Lion lưu 1 tensor trạng thái (m), AdamW lưu 2 (m, v).
- **Quan sát thực nghiệm chính**: lợi ích Lion tăng theo batch size; batch tối ưu Lion (4096) > batch tối ưu AdamW (256) trên ViT-B/16/ImageNet (thí nghiệm cụ thể của paper); Lion kém lợi thế rõ ở batch < 64 hoặc dataset khổng lồ/chất lượng rất cao.

11. Kết luận ngắn

Bằng cách biểu diễn việc thiết kế optimizer như tìm kiếm chương trình và giải quyết bài toán ”khoảng cách tổng quát hoá” giữa proxy task và target task (qua funnel selection, abstract execution, restart trong evolutionary search), paper tìm ra Lion: một optimizer rất đơn giản (chỉ 1 trạng thái momentum, update bằng `sign`), nhẹ hơn Adam về memory, và vượt hoặc ngang AdamW/Adafactor trên hầu hết benchmark lớn đã thử (ảnh, vision-language, diffusion, ngôn ngữ), đặc biệt mạnh khi batch size lớn. Giới hạn chính: lợi ích thu hẹp ở batch nhỏ (<64), ở dataset khổng lồ chất lượng rất cao, và ở ConvNet (so với Transformer); cần chỉnh lại lr/weight decay theo tỉ lệ khác AdamW khi dùng. Với dự án nén/hậu huấn luyện VLM OCR 0.8B batch 1 trên GPU 8GB, lợi thế memory của Lion là điểm hấp dẫn nhất (suy luận), nhưng vì paper không khuyến nghị mạnh ở batch rất nhỏ, nên cần thử nghiệm thực tế trước khi thay AdamW.

Nguồn gốc: https://arxiv.org/abs/2302.06675

Bài 12

Adafactor: Adaptive Learning Rates with Sublinear Memory Cost (2018)

Tác giả: Noam Shazeer, Mitchell Stern (Google Brain, UC Berkeley).
Venue / arXiv: arXiv:1804.04235 (v1, 11/04/2018); bản PDF arXiv tải không in banner hội nghị, paper không nêu venue trong văn bản (công trình được biết công bố tại ICML 2018, nhưng thông tin này không xuất hiện trong text đã tải nên không ghi vào như dữ kiện của paper).
Nguồn: https://arxiv.org/abs/1804.04235
Chủ đề: Adafactor, biến thể của Adam giảm bộ nhớ lưu second-moment từ $O(nm)$ xuống $O(n+m)$ cho tham số dạng ma trận bằng cách chỉ lưu tổng theo hàng/cột (factored second moment), kèm 2 kỹ thuật ổn định (update clipping, decay rate tăng dần) để bỏ được momentum, và relative step size (learning rate co giãn theo scale tham số).
Vị trí trong lộ trình: đọc sau opt 10 (LAMB) và opt 11 (Lion), đều là các cách khác để thích nghi Adam cho quy mô lớn; đọc trước opt 13 (Sophia). Cũng liên quan opt 06/07 (Adam/AdamW, nền tảng trực tiếp bị sửa đổi) và opt 14 (Shampoo, một cách khác để giảm/structure hoá second-moment).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Các optimizer adaptive (RMSProp, Adam, Adadelta) co giãn update bằng nghịch đảo căn bậc hai của trung bình trượt (moving average) bình phương gradient quá khứ. Trên bài toán lồi, một số phương pháp này có lợi thế lý thuyết khi gradient thưa (sparse); trên bài toán không lồi/dense (như huấn luyện mạng neural), chưa có bảo đảm hội tụ nhưng thực nghiệm vẫn thường tốt hơn SGD.

Cái giá phải trả: Adam cần lưu **2 accumulator** (moment bậc 1 m, moment bậc 2 v) cho **mỗi tham số**, tức gấp 3 lần memory so với chỉ lưu tham số (bản thân x, cộng m, v). Trong khi khả năng tính toán (compute) để train model lớn tăng nhanh, memory để lưu tham số/optimizer-state không tăng tương ứng, khiến memory trở thành nút thắt khi muốn tăng kích thước model. Paper đặt câu hỏi: có thể giữ lợi ích thực nghiệm của adaptive method nhưng giảm mạnh bộ nhớ accumulator không?

Ngoài vấn đề memory, paper còn phát hiện và giải quyết thêm 2 vấn đề liên quan tới Adam: (a) bỏ momentum (để giảm thêm 1 accumulator) có thể gây mất ổn định huấn luyện nếu không sửa gì khác, nguyên nhân được chỉ ra là **second-moment estimator bị ”lỗi thời”** (out-of-date) khi decay rate β_2 lớn; (b) learning rate dạng ”absolute step size” của Adam không tính đến việc các ma trận tham số khác nhau có **scale rất khác nhau**, nên paper đề xuất ”relative step size”.

2. Ý tưởng chính

Ba đóng góp chính, kết hợp lại thành **Adafactor**:

1. **Factored second moment estimation** (Mục 3): với tham số dạng ma trận $X \in \mathbb{R}^{n \times m}$, thay vì lưu toàn bộ accumulator $V \in \mathbb{R}^{n \times m}$ (trung bình trượt của $(\nabla_X f)^2$), chỉ lưu **tổng theo hàng** $R \in \mathbb{R}^n$ và **tổng theo cột** $C \in \mathbb{R}^m$, rồi tái tạo $V \approx RC / \mathbb{1}^\top R$ (xấp xỉ hạng 1, tối ưu theo divergence Kullback-Leibler tổng quát hoá). Giảm memory từ $O(nm)$ xuống $O(n+m)$.
2. **Loại bỏ momentum ($\beta_1 = 0$) kèm 2 kỹ thuật ổn định** (Mục 4-7): (a) **update clipping**: giới hạn RMS của update không vượt ngưỡng d; (b) **decay rate** $\hat{\beta}_{2t}$ **tăng dần theo thời gian** (dạng $1 - t^{-c}$) thay vì

hằng số, giúp accumulator ”theo kịp” gradient ở early training mà vẫn ổn định ở late training, và tiện lợi thêm: loại luôn được bước bias-correction của Adam.

3. **Relative step size** (Mục 8): định nghĩa step size $\alpha_t = \max(\epsilon_2, \text{RMS}(X_{t-1})) \cdot \rho_t$, tức learning rate hiệu dụng co giãn theo ”scale” (RMS) của tham số hiện tại, với ρ_t là một **relative step size** nhỏ dùng chung cho mọi tham số (không cần tune riêng theo scale của từng lớp/ma trận).

Kết hợp 3 ý tưởng trên cho ra thuật toán **Adafactor**, cho kết quả BLEU tương đương Adam trên bài dịch máy Transformer (WMT’14 En-De) trong khi dùng lượng bộ nhớ accumulator **sublinear** theo số tham số (thay vì linear như Adam).

3. Phương pháp chi tiết

3.1 Nhắc lại Adam (Mục 2, Algorithm 1)

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1-\beta_1^t},\ \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad x_t = x_{t-1} - \alpha_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

3.2 Factored second moment (Mục 3)

Với $W \in \mathbb{R}^{n \times m}$ là 1 ma trận tham số, muốn xấp xỉ accumulator đầy đủ $V \in \mathbb{R}^{n \times m}$ (trung bình trượt của $(\nabla_W f)^2$) bằng tích RS với $R \in \mathbb{R}^{n \times k}, S \in \mathbb{R}^{k \times m}$, $k \ll n, m$. Thay vì dùng chuẩn Frobenius (tương ứng SVD cắt ngọn, nhưng SVD không tương thích với exponential smoothing và có thể cho giá trị âm trong khi ta cần căn bậc hai), paper dùng **divergence Kullback-Leibler tổng quát hoá (I-divergence)**:

$$d(p, q) = p \log \frac{p}{q} - p + q, \qquad \text{minimize}_{R,S \geq 0} \sum_{i,j} d(V_{ij}, [RS]_{ij}) \tag{1}$$

Lemma 1 (paper chứng minh bằng đạo hàm = 0): với $k = 1$ (hạng 1), nghiệm tối ưu là $R = V\mathbb{1}_m$ (tổng theo hàng), $S = \mathbb{1}_n^\top V / (\mathbb{1}_n^\top V \mathbb{1}_m)$ (tổng theo cột, chuẩn hoá), tức $RS = V\mathbb{1}_m \mathbb{1}_n^\top V / (\mathbb{1}_n^\top V \mathbb{1}_m)$. Tính chất quan trọng: nghiệm này chỉ cần biết **tổng theo hàng và tổng theo cột** của V, là các hàm **tuyến tính** của V, nên tương thích với exponential smoothing (tổng theo hàng của trung bình trượt = trung bình trượt của tổng theo hàng, tương tự cho cột). Nếu V thực sự có hạng 1, nghiệm sẽ khôi phục đúng V.

Thuật toán cụ thể (Algorithm 2 trong paper, trường hợp $\beta_1 = 0$ để tập trung vào second moment):

$$R_t = \beta_2 R_{t-1} + (1-\beta_2)(G_t^2)\mathbb{1}_m, \qquad C_t = \beta_2 C_{t-1} + (1-\beta_2)\mathbb{1}_n^\top (G_t^2)$$
$$\hat{V}_t = \frac{R_t C_t / \mathbb{1}_n^\top R_t}{1-\beta_2^t}, \qquad X_t = X_{t-1} - \alpha_t\, G_t / \sqrt{\hat{V}_t + \epsilon}$$

Ví dụ tự tính (minh hoạ memory saving, không phải số trong paper): một ma trận trọng số 1024×4096 có $1024 \times 4096 = 4.194.304$ phần tử. Lưu accumulator đầy đủ V ở fp32 tốn $4.194.304 \times 4$ byte $\approx 16{,}0$ MB. Lưu $R \in \mathbb{R}^{1024}$ và $C \in \mathbb{R}^{4096}$ chỉ tốn $(1024 + 4096) \times 4$ byte $= 5120 \times 4 \approx 0{,}02$ MB, tức giảm khoảng **820 lần** cho riêng accumulator này (tự tính: $4.194.304 / 5120 \approx 819$). Nếu Adam (có cả m và v) cần $2 \times 16{,}0 = 32{,}0$ MB cho ma trận này, Adafactor factorized (không momentum, chỉ R, C) cần khoảng $0{,}02$ MB, một khoản tiết kiệm rất lớn ở quy mô 1 layer, và cộng dồn rất đáng kể khi model có hàng trăm ma trận như vậy.

3.3 Vấn đề ”second moment lỗi thời” và update clipping (Mục 5-6)

Định nghĩa $\text{RMS}(U_t) = \text{Mean}_{x \in X} \sqrt{(g_{xt})^2 / \hat{v}_{xt}}$ (căn trung bình bình phương của update **chưa co giãn theo step size**, trên toàn bộ 1 ma trận/vector tham số). Nếu accumulator $\hat{v}$ hoạt động đúng, $(g_{xt})^2 / \hat{v}_{xt}$ phải gần 1 với mỗi toạ độ, nên $\text{RMS}(U_t)$ của cả ma trận cũng nên gần 1. Thực nghiệm (Hình 1 của paper, trên 1 ma trận

trọng số Transformer dịch máy): với $\beta_2 = 0.9$ (decay nhanh), $\text{RMS}(U_t)$ giữ gần 1; với $\beta_2 = 0.999$ (decay chậm), $\text{RMS}(U_t)$ dao động mạnh, có lúc vượt xa 1, tức **update lớn hơn mong muốn**, trùng với lúc quan sát mất ổn định khi bỏ momentum và bỏ warmup.

Để sửa: **update clipping**, định nghĩa update đã chuẩn hoá:

$$\hat{U}_t = \frac{U_t}{\max(1, \text{RMS}(U_t)/d)}$$

(với d là ngưỡng, paper dùng $d = 1$) rồi $x_t = x_{t-1} - \alpha_t \hat{U}_t$. Khác với gradient clipping thông thường (giới hạn norm gradient trước khi chia cho accumulator), update clipping giới hạn **chính update cuối cùng sau khi đã chia** cho accumulator, nên kiểm soát trực tiếp độ lớn bước đi.

3.4 Decay rate tăng dần và loại bỏ bias correction (Mục 7)

Paper viết lại Adam dưới dạng dùng decay-rate hiệu chỉnh $\hat{\beta}_{2t} = \beta_2 \frac{1-\beta_2^{t-1}}{1-\beta_2^t}$ thay cho bias-correction (Algorithm 3, paper chứng minh 2 cách viết tương đương), từ đó nhận ra Adam **vốn đã** dùng một schedule $\hat{\beta}_{2t}$ tăng dần từ 0 lên β_2. Paper đề xuất họ schedule tổng quát hơn:

$$\hat{\beta}_{2t} = 1 - t^{-c}, \qquad t \geq 1,\ c > 0$$

và chứng minh (Mục 7.2): điều kiện $\sum_{i=1}^{t}(1-\hat{\beta}_{2i}) \prod_{j=i+1}^{t} \hat{\beta}_{2j} = 1$ (đảm bảo $\mathbb{E}[v_t] \approx \mathbb{E}[g_t^2]$, không cần bias-correction) đúng với **mọi** schedule có $\hat{\beta}_{2,1} = 0$; còn điều kiện $\lim_{t\to\infty}(1-\hat{\beta}_{2i}) \prod_{j=i+1}^{t} \hat{\beta}_{2j} = 0$ (đóng góp gradient quá khứ phải giảm về 0) chỉ đúng khi $c \leq 1$ (vì cần chuỗi $\sum 1/j^c$ phân kỳ). Trường hợp đặc biệt $c = 1$: v_t trở thành trung bình số học đơn giản của lịch sử g_i^2. Thực nghiệm chọn $c = 0.8$ (từ Table 2 dòng K/L/M, $c = 0.5$ ổn định+hội tụ tốt, $c = 1.0$ hội tụ nhưng kém ổn định hơn khi không warmup; paper chọn công bố cuối dùng $\hat{\beta}_{2t} = 1 - t^{-0.8}$ trong Algorithm 6).

3.5 Relative step size (Mục 8) và thuật toán Adafactor hoàn chỉnh

Thay vì absolute step size α_t, định nghĩa scale của 1 vector/ma trận X là $\text{RMS}(X)$ (căn trung bình bình phương các phần tử), chặn dưới bởi hằng số nhỏ ϵ_2 (để tham số khởi tạo 0 vẫn "thoát" được 0), rồi:

$$\alpha_t = \max(\epsilon_2, \text{RMS}(X_{t-1})) \cdot \rho_t$$

với ρ_t là **relative step size** dùng chung mọi tham số (không cần tune riêng theo scale từng lớp). Kết hợp tất cả, **Adafactor cho ma trận** (Algorithm 4, trích đầy đủ):

$$\alpha_t = \max(\epsilon_2, \text{RMS}(X_{t-1}))\rho_t, \quad G_t = \nabla f_t(X_{t-1})$$

$$R_t = \hat{\beta}_{2t} R_{t-1} + (1-\hat{\beta}_{2t})(G_t^2 + \epsilon_1 \mathbb{1}_n \mathbb{1}_m^\top)\mathbb{1}_m, \qquad C_t = \hat{\beta}_{2t} C_{t-1} + (1-\hat{\beta}_{2t})\mathbb{1}_n^\top (G_t^2 + \epsilon_1 \mathbb{1}_n \mathbb{1}_m^\top)$$

$$\hat{V}_t = R_t C_t / \mathbb{1}_n^\top R_t, \qquad U_t = G_t / \sqrt{\hat{V}_t}, \qquad \hat{U}_t = U_t / \max(1, \text{RMS}(U_t)/d), \qquad X_t = X_{t-1} - \alpha_t \hat{U}_t$$

(với vector/tham số 1 chiều, Algorithm 5 dùng trực tiếp accumulator đầy đủ $\hat{V}_t$ không factorize, vì không có cấu trúc hàng/cột để tách). **Siêu tham số đề xuất (Algorithm 6, trích đúng số liệu paper)**: $\epsilon_1 = 10^{-30}$, $\epsilon_2 = 10^{-3}$, $d = 1$, $\rho_t = \min(10^{-2}, 1/\sqrt{t})$, $\hat{\beta}_{2t} = 1 - t^{-0.8}$. Không có hệ số β_1/momentum trong công thức mặc định (dù paper nói có thể thêm lại nếu muốn).

4. Thí nghiệm và kết quả chính

Toàn bộ thí nghiệm dùng Transformer (Vaswani et al., 2017) trên **WMT 2014 English-to-German**, đo BLEU trên newstest2013 (dev set), beam search beam=4, length penalty $\alpha = 0.6$; huấn luyện 100.000 bước, batch khoảng 4096+4096 token (khoảng 8 lần nhỏ hơn batch của paper gốc Transformer, nên BLEU tuyệt đối thấp hơn, paper nói rõ không nên so sánh trực tiếp với số của Vaswani et al. 2017).

Bảng 1 (trích, so sánh decay nhanh/chậm, có/không warmup): $\beta_2 = 0.999$ (chậm): BLEU 25.6 có warmup, **0.1** không warmup (mất ổn định hoàn toàn); $\beta_2 = 0.9$ (nhanh): BLEU 18.4 có warmup, 15.6 không warmup (ổn định hơn nhưng BLEU thấp hơn do decay quá nhanh). Đây minh họa rõ "fast decay có vấn đề hội tụ, slow decay có vấn đề ổn định" khi không có cơ chế khác hỗ trợ.

Bảng 2 (trích các dòng chính, BLEU có-warmup / không-warmup):

Dòng	Factored	$\hat{\beta}_1$	$\hat{\beta}_2$	Update clip d	Relative step	BLEU (warmup)	BLEU (no warmup)
(A) baseline	không	0	0.999	-	-	25.6	0.1
(B) +momentum	không	0.9	0.999	-	-	25.4	23.1
(C) +factored	có	0	0.999	-	-	25.4	0.2
(G) fast decay	không	0	0.9	-	-	18.4	15.6
(H) +clip d=1	không	0	0.999	1.0	-	25.4	21.5
(I) +clip d=2	không	0	0.999	2.0	-	25.7	0.2
(J) factored+clip	có	0	0.999	1.0	-	25.6	22.4
(K) $1 - t^{-0.5}$	không	0	$1 - t^{-0.5}$	-	-	25.6	21.1
(N) $1 - t^{-0.8}$+clip	không	0	$1 - t^{-0.8}$	1.0	-	25.9	22.4
(O) = Adafactor (không momentum)	có	0	$1 - t^{-0.8}$	1.0	$\rho_t = s_t$	25.0	**25.5**
(P) Adafactor+momentum	có	0.9	$1 - t^{-0.8}$	1.0	$\rho_t = s_t$	24.9	25.3

(trong đó $s_t = \min(10^{-6}t, 1/\sqrt{t})$ với warmup hoặc $\min(10^{-2}, 1/\sqrt{t})$ không warmup). Điểm nổi bật: dòng (O), chính là **Adafactor** hoàn chỉnh (factored + decay tăng dần + clipping + relative step size, không momentum), đạt BLEU **25.5 khi không có warmup**, cao nhất trong số các cấu hình "không warmup" được thử, trong khi baseline (A) không-warmup chỉ đạt 0.1 (hoàn toàn mất ổn định). Có warmup, các scheme khác nhau cho kết quả gần nhau (quanh 25-26), ngoại trừ fast-decay (G) kém hơn rõ.

Bảng 3 (độ bền với cách khởi tạo embedding, trích đúng số): với embedding khởi tạo $\sigma = 1/\sqrt{d_{model}}$ không scale lại (thiết lập "ngây thơ" nhất): BLEU Adam 24.2 so với Adafactor 25.4; với embedding khởi tạo $\sigma = 1$ không scale: Adam 25.8 so với Adafactor 26.4; với cách khởi tạo "khéo" gốc của Transformer ($\sigma = 1/\sqrt{d_{model}}$ rồi scale lên $\sqrt{d_{model}}$): Adam 26.4 so với Adafactor 26.6. Kết luận của paper: **relative step size giúp Adafactor bền hơn Adam** khi gặp cách khởi tạo/scale tham số kém tối ưu, vì learning rate hiệu dụng tự co giãn theo scale thật của tham số.

5. Phân tích / ablation

- **Factored vs full accumulator** (Bảng 2, (A) vs (C), (H) vs (J)): kết quả "tương tự" (25.6 vs 25.4; 25.4(no warmup not shown for H/J warmup case) vs 25.6 có warmup, 21.5 vs 22.4 không warmup) nên việc factorize **không** làm giảm chất lượng rõ rệt so với giữ accumulator đầy đủ, trong cùng cấu hình $\hat{\beta}_2$/clipping.
- **Factorize theo row-mean/col-mean riêng lẻ** (Mục 3.2, không phải outer product RS): kết quả row-mean "tương tự baseline", nhưng col-mean "much worse" (Bảng 2, dòng D/E: BLEU col-mean chỉ

0.3). Paper giải thích: do kiến trúc Transformer dùng **chung 1 ma trận** cho token-embedding và lớp output softmax, mỗi hàng của ma trận này tương ứng 1 token trong vocab; token phổ biến có gradient lớn hơn hẳn token hiếm, nên chỉ lấy trung bình cột (bỏ thông tin theo hàng) làm mất đặc điểm "mỗi token-row cần learning rate riêng". Đây là ví dụ cụ thể cho thấy **outer-product rank-1** (giữ cả thông tin hàng và cột) quan trọng hơn chỉ lấy 1 trong 2 chiều.

- **Update clipping, d=1 vs d=2** (Bảng 2, H vs I): $d = 1$ cải thiện rõ ổn định khi không warmup (21.5 vs baseline 0.1); $d = 2$ (ngưỡng lỏng hơn) **không** cải thiện (0.2, gần như baseline), cho thấy ngưỡng clip phải đủ chặt mới có tác dụng.
- **Decay rate tăng dần, các giá trị** c (Bảng 2, K/L/M với $c = 0.5, 0.8, 1.0$): $c = 0.5$ ổn định cả 2 trường hợp (warmup/no-warmup: 25.6/21.1); $c = 0.8$ không clip (không có trong bảng ở dạng riêng ngoài dòng N có kèm clip); $c = 1.0$ (L) mất ổn định khi không warmup (BLEU 0.1), trùng với phân tích lý thuyết ở Mục 3.4 rằng $c \leq 1$ là điều kiện cần nhưng c gần 1 vẫn có thể kém ổn định hơn c nhỏ hơn.
- **Momentum có giúp thêm không khi đã có đủ cơ chế khác?** (Bảng 2, O vs P): Adafactor không-momentum (O) đạt 25.0/25.5 (warmup/no-warmup), thêm momentum $\hat{\beta}_1 = 0.9$ (P) cho 24.9/25.3, tức **không cải thiện**, thậm chí nhích xuống nhẹ trong thí nghiệm này; đây là cơ sở thực nghiệm cho quyết định bỏ momentum trong Adafactor mặc định.
- **SGD** (Bảng 2, dòng Q, nhiều mức learning rate): SGD cần learning rate rất lớn (gấp 100-300 lần so với scale s_t dùng cho Adam) mới đạt BLEU gần được (22.9-24.3 ở lr=100-200$\times s_t$), và dễ "diverge" khi không warmup hoặc lr quá lớn (lr=300$\times s_t$ diverge cả 2 trường hợp); kết quả SGD **kém và kém ổn định hơn** Adam/Adafactor trong thí nghiệm này, phụ thuộc mạnh vào lựa chọn learning rate.

6. Giới hạn

- Thí nghiệm của paper **chỉ** trên 1 bài toán (dịch máy WMT'14 En-De, kiến trúc Transformer gốc), không có thí nghiệm trên ảnh, các LM decoder-only hiện đại, hay mô hình đa phương thức; paper tự nhận batch size thí nghiệm (khoảng 4096 token) nhỏ hơn 8 lần so với paper Transformer gốc nên BLEU tuyệt đối thấp hơn và "không nên so sánh trực tiếp".
- Phần phân tích lý thuyết (Mục 7.2) chỉ chứng minh 2 điều kiện cần cho $\mathbb{E}[v_t] \approx \mathbb{E}[g_t^2]$ dưới giả định gradient **dừng (stationary)** hoặc sai số nhỏ; không có bảo đảm hội tụ tổng thể (dạng regret bound như các optimizer khác, ví dụ LAMB ở opt 10) cho toàn bộ Adafactor.
- Factorization hạng 1 (rank-1) chỉ khôi phục đúng V nếu V **thực sự** có hạng 1; với ma trận gradient bình phương có cấu trúc phức tạp hơn, đây chỉ là xấp xỉ, và ablation row-mean/col-mean ở Mục 5 cho thấy xấp xỉ có thể thất bại nặng nếu bỏ 1 chiều thông tin (trường hợp ma trận embedding/softmax chia sẻ của Transformer).
- Relative step size dùng RMS(X_{t-1}) làm thước đo scale tham số; với tham số **khởi tạo bằng 0** hoặc rất nhỏ, cần hằng số chặn dưới $\epsilon_2 = 10^{-3}$ để tránh learning rate về 0 hoàn toàn, một heuristic cần tự chọn tay (paper không phân tích độ nhạy của ϵ_2).
- Không có so sánh trực tiếp với LAMB (chưa công bố cùng thời điểm) hay các phương pháp preconditioning theo cấu trúc tensor đầy đủ hơn (Shampoo, công bố gần như cùng năm 2018), nên không rõ Adafactor đứng ở đâu so với các hướng "giảm memory nhưng giữ preconditioning chi tiết hơn" này; đây là so sánh mà các bài sau (opt 14, 15) trong lộ trình mới cung cấp.

7. Liên hệ với các bài khác

- **opt 06 (Adam) / opt 07 (AdamW)**: Adafactor là một biến thể trực tiếp của Adam (Algorithm 1 được paper nhắc lại làm điểm xuất phát), thay accumulator đầy đủ bằng factored accumulator và bỏ momentum; hiểu rõ công thức Adam là điều kiện để theo được toàn bộ Mục 3-8 của bài này.
- **opt 10 (LAMB)**: cùng động lực "optimizer cho model/batch lớn, Adam thuần không đủ", nhưng LAMB nhắm vào vấn đề **batch size** (ổn định khi batch cực lớn) bằng layerwise trust-ratio, còn Adafactor nhắm vào vấn đề **memory** (giảm kích thước accumulator); hai hướng bổ sung nhau hơn là cạnh tranh.

- **opt 13 (Sophia)** và **opt 14 (Shampoo)**: các optimizer ”bậc cao hơn” dùng thông tin curvature (Hessian ước lượng, hoặc preconditioner theo cấu trúc tensor), đi xa hơn Adafactor về độ phức tạp nhưng cùng chia sẻ mối quan tâm ”chi phí accumulator/preconditioner so với lợi ích”. Shampoo đặc biệt đáng so sánh: cả 2 cùng khai thác cấu trúc ma trận của tham số, nhưng Adafactor chỉ lấy tổng hàng/cột (hạng 1, rẻ), còn Shampoo tính preconditioner đầy đủ hơn theo mỗi chiều tensor (tốn hơn, chính xác hơn).
- **llm content/03_2020_scaling_laws_kaplan.md** (đã kiểm tra tồn tại bằng `ls`, và đã `grep -a -i adafactor` xác nhận nội dung): bài scaling-laws của Kaplan et al. (2020) ghi rõ ”các mô hình >1B dùng Adafactor” (còn mô hình nhỏ hơn dùng Adam) khi huấn luyện các Transformer để đo quy luật tỉ lệ loss theo tham số/dữ liệu/compute; đây là một minh chứng thực tế (ngoài paper Adafactor) cho việc lợi ích giảm-memory của Adafactor được áp dụng đúng như mục đích thiết kế: cho phép train model tỉ tham số trên hạ tầng cố định.
- **opt 51 (”How to set AdamW’s weight decay as you scale model and dataset size”)**: theo tên bài trong titles.tsv, bàn về cách chọn siêu tham số AdamW khi scale model/data; Adafactor cũng có vấn đề tương tự (chọn ρ_t, ϵ_2) nên 2 bài có thể đối chiếu về triết lý ”siêu tham số nào cần co giãn theo scale, cái nào giữ cố định”.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU <= 8 GB, batch 1). **(suy luận)** Đây là một trong những bài liên quan **trực tiếp nhất** tới ràng buộc phần cứng của dự án trong số 8 bài opt 10-17, vì lý do tồn tại cốt lõi của Adafactor chính là giảm memory optimizer-state, đúng vấn đề của GPU 8GB:

- **Tự tính minh hoạ cho model 0.8B**: nếu coi gần hết 0.8 tỷ tham số nằm trong các ma trận 2D (xấp xỉ, bỏ qua embedding/norm/bias nhỏ), AdamW ở fp32 cần lưu m, v tốn khoảng $2 \times 0.8 \times 10^9 \times 4$ byte $= 6{,}4$ GB (tự tính). Adafactor (bỏ momentum, chỉ lưu R, C theo hàng/cột mỗi ma trận) cho mỗi ma trận $n \times m$ chỉ tốn $O(n+m)$ thay vì $O(nm)$; với các ma trận ”vuông vức” cỡ vài nghìn x vài nghìn (phổ biến trong attention/MLP của model 0.8B), hệ số giảm có thể lên tới hàng trăm/nghìn lần cho riêng accumulator (xem ví dụ tự tính ở Mục 3.2 cho 1 ma trận 1024×4096: giảm khoảng 819 lần). Thực tế tổng memory accumulator của Adafactor cho cả model 0.8B nhiều khả năng chỉ còn vài MB tới vài chục MB (tự tính thô, chưa tính ϵ_1, ϵ_2 hay phần không factorize được như vector 1 chiều), so với 6.4GB của AdamW fp32: đây là khoản VRAM rất đáng kể trên GPU 8GB khi còn phải chứa activation cho ~3.600 token/trang.
- **Relative step size** cũng hữu ích về mặt thực hành: trong 1 kiến trúc hybrid (GatedDeltaNet + full attention), các khối khác nhau (state-space-like layer vs attention) có thể có scale tham số rất khác nhau; learning rate tự co giãn theo $\text{RMS}(X_{t-1})$ giúp giảm nhu cầu tự tay tune LR riêng cho mỗi loại khối, một lợi ích trực tiếp khi finetune/distill kiến trúc hybrid lạ.
- **Rủi ro cần lưu ý (suy luận)**: Adafactor trong paper được thiết kế và kiểm chứng không-momentum; với finetune ngắn (ít bước, như các lần distill nhỏ), thiếu momentum có thể làm hội tụ chậm hơn so với AdamW nếu số bước ít; paper gợi ý có thể thêm lại momentum (dòng P trong Bảng 2, tuy không cải thiện rõ trên bài dịch máy của paper) nên nếu dùng cho OCR, nên thử cả 2 biến thể (có/không momentum) trên chính dữ liệu của dự án trước khi chọn.

9. Thí nghiệm gợi ý

1. **Đo VRAM optimizer-state thực tế**: trên 1 bản LightOnOCR-3-0.8B (hoặc 1 model nhỏ cùng cỡ), đo `torch.cuda.max_memory_allocated` khi dùng AdamW (fp32 và bf16 cho state) so với 1 cài đặt Adafactor (ví dụ dùng implementation có sẵn trong HuggingFace `transformers.optimization.Adafactor`). Đối chiếu với con số ”tự tính” ở Mục 8 (6.4GB AdamW fp32 so với vài chục MB Adafactor).

2. **Ablation momentum trong Adafactor** trên 1 tác vụ finetune ngắn (vài trăm-vài nghìn bước, giống quy mô 1 lần distill OCR), so sánh Adafactor có/không $\hat{\beta}_1 = 0.9$ (giống dòng O/P trong Bảng 2 của paper). Đo: tốc độ giảm loss những bước đầu. Diễn giải: nếu không khác biệt đáng kể (giống kết quả paper), có thể bỏ momentum để tiết kiệm thêm memory.
3. **Tái tạo thí nghiệm "second moment lỗi thời"**: trên 1 model nhỏ, log $\text{RMS}(U_t)$ (căn trung bình bình phương của $g_{xt}^2/\hat{v}_{xt}$) cho 1 ma trận trọng số theo thời gian, với $\beta_2 = 0.9$ và $\beta_2 = 0.999$ cố định (không dùng schedule tăng dần). Đo: $\text{RMS}(U_t)$ có dao động mạnh với β_2 lớn như Hình 1 của paper không, ở quy mô nhỏ.
4. **So sánh factorized vs full second moment** trên 1 mạng nhỏ có ít nhất 1 ma trận lớn (ví dụ embedding dùng chung input/output như thí nghiệm gốc, nếu kiến trúc toy cho phép), đo chất lượng cuối cùng. Đo thêm: thử phá vỡ factorization bằng cách chỉ giữ row-mean hoặc chỉ col-mean (bỏ outer product), xem có tái hiện hiện tượng "col-mean much worse" của paper (do cấu trúc chia sẻ hàng/token) không.
5. **Đo độ bền với cách khởi tạo**: trên 1 model nhỏ, thử 2-3 cách khởi tạo tham số khác scale nhau (ví dụ std 1.0 vs std $1/\sqrt{d}$ không scale lại), huấn luyện bằng AdamW và bằng Adafactor (relative step size), so sánh chất lượng cuối. Kỳ vọng theo Bảng 3 của paper: Adafactor ít nhạy hơn AdamW với cách khởi tạo "ngây thơ".

10. Tóm tắt công thức / khái niệm

- **Factorization hạng 1 tối ưu I-divergence** (Lemma 1): $R = V\mathbb{1}_m$, $S = \mathbb{1}_n^\top V / (\mathbb{1}_n^\top V \mathbb{1}_m)$; chỉ cần tổng hàng/cột, tương thích exponential smoothing. Memory: $O(nm) \to O(n+m)$.
- **RMS của 1 ma trận/vector** X: $\text{RMS}(X) = \sqrt{\text{Mean}(X_{ij}^2)}$; dùng cho cả chẩn đoán update ($\text{RMS}(U_t)$) và relative step size ($\text{RMS}(X_{t-1})$).
- **Update clipping**: $\hat{U}_t = U_t / \max(1, \text{RMS}(U_t)/d)$, $d = 1$ mặc định.
- **Decay rate tăng dần**: $\hat{\beta}_{2t} = 1 - t^{-c}$ ($c \leq 1$ để đảm bảo đóng góp gradient quá khứ giảm về 0); $c = 0.8$ dùng trong công bố cuối; $c = 1$ cho v_t = trung bình số học đơn giản của lịch sử g_i^2.
- **Relative step size**: $\alpha_t = \max(\epsilon_2, \text{RMS}(X_{t-1})) \cdot \rho_t$.
- **Adafactor (Algorithm 4, ma trận)**: R_t, C_t theo hàng/cột của $(G_t^2 + \epsilon_1 \mathbb{1}\mathbb{1}^\top)$ với decay $\hat{\beta}_{2t}$; $\hat{V}_t = R_t C_t / \mathbb{1}^\top R_t$; $U_t = G_t / \sqrt{\hat{V}_t}$; $\hat{U}_t$ sau update-clip; $X_t = X_{t-1} - \alpha_t \hat{U}_t$.
- **Siêu tham số đề xuất**: $\epsilon_1 = 10^{-30}$, $\epsilon_2 = 10^{-3}$, $d = 1$, $\rho_t = \min(10^{-2}, t^{-1/2})$, $\hat{\beta}_{2t} = 1 - t^{-0.8}$, không momentum mặc định.

11. Kết luận ngắn

Adafactor giải quyết vấn đề memory của Adam bằng cách factorize accumulator second-moment theo hàng/cột (tối ưu I-divergence, nghiệm hạng-1 dạng tích tổng hàng x tổng cột), giảm memory từ $O(nm)$ xuống $O(n+m)$ cho mỗi ma trận tham số; để bỏ được momentum (tiết kiệm thêm) mà không mất ổn định, paper bổ sung update clipping và decay rate $\hat{\beta}_{2t}$ tăng dần theo $1 - t^{-c}$ (loại luôn được bias-correction); cuối cùng dùng relative step size để tránh phải tune learning rate riêng theo scale từng lớp. Trên dịch máy Transformer WMT'14 En-De, Adafactor đạt chất lượng tương đương Adam (và bền hơn khi không có warmup hoặc khởi tạo kém tối ưu), với lượng accumulator sublinear. Với dự án nén/hậu huấn luyện VLM OCR 0.8B trên GPU 8GB, đây là một trong các bài liên quan trực tiếp nhất về mặt động lực (memory-constrained), đáng thử nghiệm thực tế thay AdamW, đặc biệt khi VRAM là nút thắt chính (suy luận).

Nguồn gốc: https://arxiv.org/abs/1804.04235

Bài 13

Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training (2023)

Tác giả: Hong Liu, Zhiyuan Li, David Hall, Percy Liang, Tengyu Ma (Stanford University).
Venue / arXiv: arXiv:2305.14342 (v4, 05/03/2024); paper không nêu venue hội nghị trong văn bản tải. Ghi chú này đọc bản v4 (có thí nghiệm tới GPT NeoX 6.6B, Hình 5(e); các version đầu (v1-v2, 2023) chỉ có tới 770M/1.5B, nên nếu đọc lại bản khác số liệu quy mô lớn có thể khác).
Nguồn: https://arxiv.org/abs/2305.14342
Chủ đề: Sophia (Second-order Clipped Stochastic Optimization), optimizer bậc hai nhẹ cho pretraining LLM: dùng ước lượng đường chéo Hessian (Hutchinson hoặc Gauss-Newton-Bartlett) làm preconditioner, chỉ cập nhật Hessian mỗi k bước, và clip update theo từng toạ độ để chống nhiễu/độ cong âm; đạt tốc độ gấp khoảng 2 lần AdamW (số bước, compute, wall-clock) trên GPT-2/GPT NeoX.
Vị trí trong lộ trình: đọc sau opt 12 (Adafactor, một cách khác giảm chi phí optimizer-state) và opt 06/07 (Adam/AdamW, baseline chính); đọc trước opt 14 (Shampoo). Cũng liên quan opt 11 (Lion, baseline so sánh trực tiếp trong paper) và opt 45 (”Deconstructing What Makes a Good Optimizer for Language Models”).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Pretraining LLM rất đắt: paper dẫn ví dụ PaLM huấn luyện 2 tháng trên 6144 TPU, tốn 10 triệu USD (Chowdhery et al., 2022). Một cải tiến ”không tầm thường” về optimizer có thể giảm đáng kể thời gian/chi phí. Adam (và biến thể AdamW, Adafactor, LAMB) đã là lựa chọn mặc định nhiều năm cho GPT, OPT, Gopher, LLaMA; các optimizer bậc hai (dùng thông tin Hessian/độ cong) về lý thuyết có thể tốt hơn vì thích nghi với độ cong khác nhau giữa các chiều tham số, nhưng thường tốn chi phí mỗi bước quá lớn (ví dụ K-FAC tốn kém cho LLM) nên chưa có báo cáo nào (theo paper, tính tới thời điểm viết) cho thấy optimizer bậc hai **thực sự nhanh hơn** Adam trên LLM decoder-only về wall-clock/compute. Một hướng khác gần đây: Lion (opt 11, Chen et al. 2023) tìm bằng symbolic search, nhanh hơn Adam rõ trên ảnh/diffusion nhưng paper này trích dẫn Lion ”chỉ đạt speedup hạn chế trên LLM”.

Câu hỏi: có thể thiết kế optimizer bậc hai **nhẹ** (overhead mỗi bước nhỏ) mà vẫn tận dụng được thông tin Hessian để tăng tốc hội tụ trên LLM thật, đạt tốc độ nhanh hơn cả về số bước **và** wall-clock/compute (không chỉ số bước, vì mỗi bước optimizer bậc hai thường đắt hơn), không?

2. Ý tưởng chính

Sophia ước lượng đường chéo (diagonal) của Hessian bằng 1 trong 2 estimator nhẹ, chỉ tính lại Hessian mỗi $k = 10$ bước (không phải mọi bước), rồi dùng EMA của Hessian ước lượng làm preconditioner cho EMA của gradient, cuối cùng **clip update theo từng toạ độ** để chặn trường hợp Hessian ước lượng sai/âm/thay đổi nhanh gây update quá lớn. Vì chỉ tính Hessian 1/10 số bước và dùng estimator rẻ, overhead trung bình mỗi bước chỉ khoảng 5%, nên tốc độ hội tụ nhanh hơn theo số bước **cũng dịch thành** nhanh hơn theo wall-clock/compute, điều mà các optimizer bậc hai trước đó (cần Hessian mỗi bước) không đạt được.

Kết quả chính: trên GPT-2 (125M-770M) và GPT NeoX (1.5B, 6.6B), Sophia đạt validation loss tương đương AdamW với khoảng **50% số bước** (tức gấp đôi tốc độ) về cả số bước, tổng compute, và wall-clock time; khoảng cách này **tăng theo kích thước model** (scaling law thiên về Sophia). Về lý thuyết, trên 1 bản đơn giản hoá (hàm lồi, deterministic), paper chứng minh runtime bound của Sophia **không phụ thuộc condition**

number (tỉ lệ độ cong lớn nhất/nhỏ nhất), trong khi GD/SignGD (gần giống Adam đơn giản hoá) có runtime phụ thuộc condition number này.

3. Phương pháp chi tiết

3.1 Ví dụ minh hoạ động lực (Mục 2.1)

Xét hàm 2 chiều $L(\theta_{[1]}, \theta_{[2]}) = L_1(\theta_{[1]}) + L_2(\theta_{[2]})$ với L_1 "sắc" (sharp, độ cong h_1 lớn) và L_2 "bẹt" (flat, độ cong h_2 nhỏ, $h_1 \gg h_2$):

- **GD**: $\theta_{[1]} \leftarrow \theta_{[1]} - \eta L_1'(\theta_{[1]})$ và tương tự cho $\theta_{[2]}$. Learning rate tối ưu cho chiều 1 là $\approx 1/h_1$ (rất nhỏ vì h_1 lớn), nhưng vì dùng chung η cho cả 2 chiều, chiều $\theta_{[2]}$ (cần $\eta \approx 1/h_2$ lớn hơn) học rất chậm.
- **SignGD** (xấp xỉ Adam khi bỏ EMA): $\theta \leftarrow \theta - \eta\,\text{sign}(\nabla L(\theta))$, độ lớn update **giống nhau** mọi chiều ($= \eta$). Ở chiều sắc, update "nảy" (bounce) qua lại quanh đáy; ở chiều bẹt, mỗi bước chỉ giảm loss rất ít vì update nhỏ so với độ dốc cần đi.
- **Newton's method**: $\theta_{[i]} \leftarrow \theta_{[i]} - \eta L_i'(\theta_{[i]})/h_i$, chia theo đúng độ cong từng chiều nên lý thuyết tối ưu cho hàm lồi, nhưng trên hàm **không lồi** có thể hội tụ về điểm yên ngựa (saddle point) hoặc cực đại nếu độ cong âm ở đâu đó (minh hoạ Hình 2 của paper: Newton's method hội tụ nhầm vào saddle).
- **Sophia (công thức động lực, eq. 4)**: $\theta_{[i]} \leftarrow \theta_{[i]} - \eta \cdot \text{clip}\big(L_i'(\theta_{[i]})/\max\{h_i, \epsilon\},\ \rho\big)$: chia theo độ cong (như Newton) nhưng **clip** kết quả vào $[-\rho, \rho]$. Khi độ cong ước lượng âm/rất nhỏ/không tin cậy, clip khiến optimizer "lùi về" hành vi SignGD (an toàn hơn Newton thuần), còn khi độ cong dương và tin cậy, optimizer đi nhanh ở chiều bẹt và kiềm lại ở chiều sắc, tránh "nảy".

3.2 Thuật toán Sophia (Mục 2.2, Algorithm 3)

Ký hiệu θ_t tham số, $g_t = \nabla L_t(\theta_t)$ gradient minibatch. EMA gradient: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$. EMA Hessian ước lượng, chỉ cập nhật mỗi k bước ($k = 10$ trong cài đặt):

$$h_t = \begin{cases} \beta_2 h_{t-k} + (1-\beta_2)\hat{h}_t & \text{nếu } t \bmod k = 1 \\ h_{t-1} & \text{ngược lại} \end{cases}$$

Update (dùng clip theo từng toạ độ, $\text{clip}(z, \rho) = \max(\min(z, \rho), -\rho)$):

$$\theta_t \leftarrow \theta_t - \eta_t \lambda \theta_t \text{ (weight decay)}, \qquad \theta_{t+1} = \theta_t - \eta_t \cdot \text{clip}\Big(\frac{m_t}{\max\{\gamma h_t, \epsilon\}},\ 1\Big) \tag{6}$$

Paper chỉ ra đồng nhất thức $\eta_t \cdot \text{clip}(m_t/\max\{\gamma h_t, \epsilon\}, 1) = (\eta_t/\gamma) \cdot \text{clip}(m_t/\max\{h_t, \epsilon/\gamma\}, \gamma)$: tức về bản chất là clip update gốc m_t/h_t theo ngưỡng γ, rồi co giãn lại bằng $1/\gamma$; nhờ vậy γ chỉ quyết định **tỉ lệ toạ độ bị clip**, còn độ lớn update cuối cùng ít phụ thuộc γ hơn (khi γ rất nhỏ, mọi toạ độ bị clip và update tiến tới η_t đồng nhất, giống SignGD). Khi $h_t[i] < 0$ (Hessian âm hoặc ước lượng sai thành âm), $m_t[i]/\max\{\gamma h_t[i], \epsilon\} = m_t[i]/\epsilon$ rất lớn và cùng dấu $m_t[i]$, nên sau clip, update trở thành $\eta_t \cdot \text{sign}(m_t[i])$: **Sophia tự động lùi về "stochastic momentum SignSGD"** ở những toạ độ có độ cong âm/không tin cậy, đây là cơ chế an toàn chính của thuật toán.

Ví dụ tự tính: giả sử tại 1 toạ độ, $m_t = 0.08$, $h_t = 4$ (độ cong ước lượng dương, "sắc"), $\gamma = 0.01$, $\epsilon = 10^{-12}$, $\eta_t = 2 \times 10^{-4}$.

- $\gamma h_t = 0.01 \times 4 = 0.04$ (lớn hơn ϵ rất nhiều nên lấy $\max\{\gamma h_t, \epsilon\} = 0.04$).
- $m_t/\max\{\gamma h_t, \epsilon\} = 0.08/0.04 = 2.0$.
- Vì $2.0 > 1$ (ngưỡng clip), giá trị bị clip về 1 (bị kiềm lại vì toạ độ này "sắc").
- Update cuối: $\theta_{t+1} = \theta_t - \eta_t \times 1 = \theta_t - 2 \times 10^{-4}$.

So sánh với 1 toạ độ "bẹt" khác có cùng $m_t = 0.08$ nhưng $h_t = 0.1$: $\gamma h_t = 0.001$, $m_t/\max\{\gamma h_t, \epsilon\} = 0.08/0.001 = 80$, bị clip về 1 **giống hệt** toạ độ trước (vì $80 > 1$ cũng clip về 1) - nhưng nếu m_t nhỏ hơn, ví dụ $m_t = 0.0003$: toạ độ sắc ($h_t = 4$) cho $0.0003/0.04 = 0.0075 < 1$ (không bị clip, update nhỏ $\eta_t \times 0.0075$), còn toạ độ bẹt ($h_t = 0.1$) cho $0.0003/0.001 = 0.3 < 1$ (cũng không bị clip, nhưng update lớn hơn 40 lần so với toạ độ sắc, $\eta_t \times 0.3$ so với $\eta_t \times 0.0075$): đây minh hoạ chính xác ý "chia theo độ cong khi chưa bị clip, kiềm đều về η_t khi đã bị clip".

3.3 Hai bộ ước lượng đường chéo Hessian (Mục 2.3)

- **Hutchinson's estimator** (Algorithm 1): lấy $u \sim \mathcal{N}(0, I_d)$, trả về $\hat{h} = u \odot \nabla(\langle \nabla L(\theta), u \rangle)$ (tích element-wise giữa u và Hessian-vector-product $\nabla^2 L(\theta) u$). Đây là estimator **không chệch** (unbiased) cho đường chéo Hessian ($\mathbb{E}[\hat{h}] = \text{diag}(\nabla^2 L(\theta))$), chỉ cần 1 Hessian-vector-product (hỗ trợ sẵn trong PyTorch/JAX qua autodiff), chi phí runtime cùng cấp 1 lần tính gradient minibatch.
- **Gauss-Newton-Bartlett (GNB) estimator** (Algorithm 2): tận dụng cấu trúc loss cross-entropy $\ell_{ce}(f(\theta, x), y)$. Hessian của loss có phân tích Gauss-Newton $\nabla^2_\theta \ell = J_\theta f \cdot S \cdot J_\theta f^\top + J_{\theta\theta} f[q]$ (eq. 8), với số hạng thứ 2 thường nhỏ hơn số hạng Gauss-Newton (trích dẫn Sankar et al. 2021). Dùng identity Bartlett thứ nhất/thứ hai (Bartlett, 1953), paper suy ra: lấy mẫu label giả $\hat{y}_b \sim \text{softmax}(f(\theta, x_b))$ (không dùng label thật y_b), tính gradient $\hat{g}$ trên loss với label giả này, thì $B \cdot \hat{g} \odot \hat{g}$ là estimator **không chệch** cho đường chéo ma trận Gauss-Newton (là 1 estimator **chệch** cho đường chéo Hessian đầy đủ, vì bỏ số hạng $J_{\theta\theta} f[q]$). Ưu điểm estimator này: luôn cho giá trị **bán xác định dương (PSD)**, đảm bảo hướng update luôn là hướng giảm (descent direction).
- Paper gọi 2 biến thể tương ứng là **Sophia-H** (Hutchinson) và **Sophia-G** (GNB). Cả 2 estimator chỉ tốn chi phí runtime cùng cấp 1 lần tính gradient (không cần full Hessian $d \times d$).

3.4 Siêu tham số thực nghiệm (Mục 3.1)

- $\beta_1 = 0.96$, $\beta_2 = 0.99$, $\epsilon = 10^{-12}$, cập nhật Hessian mỗi $k = 10$ bước; $\gamma = 0.01$ (Sophia-H), $\gamma = 0.05$ (Sophia-G).
- Sophia-H dùng **32 example** (subset nhỏ của minibatch) để tính Hessian-vector-product mỗi lần cập nhật Hessian; Sophia-G dùng **240 example**.
- Batch size: 480 (GPT-2), 2048 (GPT NeoX). LR schedule cosine, LR cuối $= 0.05\times$ LR đỉnh (theo Rae et al. 2021). Gradient clipping theo norm, ngưỡng 1.0. Warmup 2000 bước cố định.
- Huấn luyện ở bfloat16; GPU A5000 (125M, 355M), A100 (770M), TPU v3-128 (1.5B, 6.6B).
- Baseline AdamW dùng $\beta_1 = 0.9, \beta_2 = 0.95$ (paper nói đây là "well-established practice", không phải 0.999 mặc định gốc); Lion dùng $\beta_1 = 0.95, \beta_2 = 0.98$ (theo Chen et al. 2023), nhưng paper lưu ý: với LLM, learning rate của Lion nên **lớn hơn** gợi ý gốc cho vision (0.1x AdamW), tìm được qua grid search riêng trên LLM; AdaHessian dùng $\beta_1 = 0.92, \beta_2 = 0.99$.
- **Chiến lược chọn γ và learning rate** (thực hành gợi ý trong Mục 3.1): trên model nhỏ, chỉnh γ sao cho tỉ lệ toạ độ **không bị clip** rơi vào khoảng 10%-50% (nếu tỉ lệ quá cao/thấp thì nhân/chia γ cho 2 rồi thử lại); γ này có thể "transfer" sang model cùng kiến trúc/dữ liệu nhưng khác kích thước. Sau khi có γ, learning rate của Sophia có thể đặt bằng **3-5 lần** LR mà người dùng sẽ dùng cho Lion, hoặc **0.8 lần** LR dùng cho AdamW.

3.5 Phương pháp luận so sánh "nhanh gấp k lần" (Mục 3.2)

Paper lập luận cách so sánh "đúng": O_2 nhanh hơn O_1 k lần nếu tồn tại siêu tham số H_2 sao cho $\min_{H_1} \text{Eval}(O_1, T, H_1) \geq \text{Eval}(O_2, T/k, H_2)$ (eq. 14), tức **phải tune** O_1 **cho đúng budget** T (vì LR schedule như cosine rất nhạy với tổng số bước đã định trước), rồi so với O_2 chạy **chỉ** T/k bước. Paper cảnh báo 1 cách so sánh "cám dỗ" nhưng sai: lấy checkpoint ở bước $T/2$ của 1 lần chạy T bước rồi so với lần chạy T bước đầy đủ của O_1 (phương án 2'); paper chỉ ra phương án này dễ bị "đánh lừa" (có thể dựng ra O_2' thoả tiêu chí 2' chỉ bằng cách chạy O_2 hết $T/2$ bước rồi "đứng im"), nên chủ yếu dùng tiêu chí chính xác hơn (phương án 1&2), có bổ sung so sánh với 2' ở Hình 5 cho tham khảo.

4. Thí nghiệm và kết quả chính

Model: GPT-2 (125M, 355M, 770M) trên OpenWebText, GPT NeoX (1.5B, 6.6B) trên the Pile; context length 1024 (GPT-2) / 2048 (NeoX).

Validation loss ở cùng số bước (Hình 5, trích đúng số):

- GPT-2 Small (125M): AdamW 2.921, Lion 2.924, Sophia-H 2.901, Sophia-G 2.875.
- GPT-2 Medium (355M): AdamW 2.691, Lion 2.678, Sophia-H 2.645, Sophia-G 2.627.
- GPT-2 Large (770M): AdamW 2.613, Sophia-H 2.554, Sophia-G 2.524.
- GPT NeoX 1.5B: AdamW 2.250, Sophia-G 2.218.
- GPT NeoX 6.6B: AdamW 1.992, Sophia-G 1.969.

Khác biệt loss tăng theo kích thước model (ví dụ 0.04-0.05 ở 355M/770M). Paper dẫn quy luật scaling (Kaplan et al. 2020; Hoffmann et al. 2022) để diễn giải: ở vùng quy mô này, **chênh 0.05 loss tương đương gấp đôi số bước/compute** để đạt cùng loss, khớp với số liệu tốc độ ở Hình 1/4.

Tốc độ gấp đôi (Hình 1, 4): so sánh số bước cần để đạt cùng validation loss, Sophia-H/-G đạt **2x speedup** so với AdamW trên mọi kích thước model đã thử (355M, 770M, 1.5B). Scaling theo kích thước model (100K bước, Hình 1(d)): model 540M dùng Sophia đạt loss bằng model 770M dùng AdamW (ở cùng 100K bước), trong khi model 770M cần 40% nhiều thời gian train và 40% nhiều compute inference hơn model 540M.

Wall-clock/compute thực tế (Table 1, trích đúng số): trên A100, model 770M: AdamW 3.25s/bước (không tính Hessian), Sophia-H 3.40s/bước (0.12s cho Hessian), Sophia-G 3.42s/bước (0.17s); compute (TFLOPs ước lượng theo Chowdhery et al. 2022): AdamW 2550, Sophia-H 2708, Sophia-G 2678. Với model 355M: AdamW 1.77s (1195 compute), Sophia-H 1.88s (0.09s Hessian, 1249 compute), Sophia-G 1.86s (0.09s, 1255 compute). Vì tính Hessian trên 1 subset nhỏ và chỉ mỗi $k = 10$ bước, chi phí Hessian chiếm khoảng 6% tổng compute, tổng overhead wall-clock dưới 5% so với AdamW. Memory: Sophia có 2 trạng thái (m, h), giống AdamW (m, v) về chi phí memory.

Few-shot SuperGLUE (Hình 6): cải thiện validation loss dịch thành cải thiện accuracy few-shot (2-shot, greedy decode, trung bình 5 prompt) trên hầu hết subtask (BoolQ, CB, COPA, RTE); model train bằng Sophia 200K bước có accuracy tương đương AdamW train 400K bước (ở GPT NeoX 1.5B).

Ổn định huấn luyện (Hình 7): tỉ lệ bước bị gradient-clipping (theo norm, ngưỡng 1.0 cho mọi optimizer) kích hoạt trên GPT-2 Small: AdamW và Lion kích hoạt **hơn 10%** số bước, Sophia-H **hiếm khi** kích hoạt. Trick ”scale key-value theo nghịch đảo layer index” (Karamcheti et al. 2021, dùng trong Mistral/HuggingFace để train ổn định hơn): AdamW/Lion không có trick này chỉ chạy ổn định tới LR=1.5e-4 (so với 3e-4 có trick) trên GPT-2 Medium; Sophia-H **không cần** trick này.

5. Phân tích / ablation

- **Tần suất cập nhật Hessian** k (Mục 3.5, Hình 8a, trên model 30M, chạy 100K/200K/400K bước): $k = 1$ cho validation loss tốt nhất ***ở cùng số bước*** nhưng overhead compute tăng 50%, nên tốc độ hội tụ **theo compute** lại kém hơn $k = 10$; $k = 100$ vẫn vượt AdamW nhưng kém $k = 10$. Kết luận: $k = 10$ là điểm cân bằng tốt giữa chất lượng ước lượng Hessian và overhead.
- **So sánh preconditioner đường chéo khác nhau** (Hình 8b, cùng $k = 10$): GNB (Sophia-G) tốt hơn Empirical Fisher (EMA bình phương gradient, khác GNB ở chỗ không lấy mẫu label giả) - khớp quan sát của Kunstner et al. (2019); Sophia-H tốt hơn AdaHessian (dùng EMA của **bình phương** Hessian ước lượng ở mẫu, khác Sophia-H dùng EMA trực tiếp của Hessian ước lượng); paper giả thuyết (chưa chứng minh chặt) rằng EMA trực tiếp có hiệu ứng khử nhiễu tốt hơn EMA của bình phương.
- **Vai trò của clipping** (Hình 8c): so sánh clip không preconditioner (”Clip”, tương đương sign-momentum/Lion với 1 β), normalize không preconditioner (”Normalize”), AdaHessian không clip ($k = 1$, vì không clip thì $k = 2$ đã ”diverge”), GNB không clip ($k = 2$, vì $k = 5$ ”diverge”). Kết

quả: **clip theo từng toạ độ, dù không có preconditioner, đã tốt hơn AdamW**; thêm preconditioner GNB vào thì Sophia-G vượt xa các baseline. Đây là bằng chứng rõ ràng cho việc 2 thành phần (preconditioner Hessian + clipping) **đều cần thiết**, không chỉ 1 trong 2.

- **Độ nhạy siêu tham số và tính "transfer"** (Hình 7c, model 30M, quét $\gamma \in \{0.9, 0.99, 0.999\}$ - đây thực chất là nhãn trục cho β_2 theo Hình 7(c) gốc - và giá trị khác): các tổ hợp cho kết quả khá giống nhau (validation loss dao động rất nhỏ, ví dụ 3.341-3.356 trong bảng con của Hình 7c), $\beta_2 = 0.99, \gamma = 0.01$ tốt nhất; bộ siêu tham số tìm trên model 30M **transfer tốt** sang 125M/355M/770M (chỉ cần tune lại LR đỉnh).
- **Lý thuyết (Định lý 4.3, Mục 4)**: trên hàm lồi, 2 lần khả vi liên tục, với giả định độ cong "liên tục đủ tốt" trong 1 vùng bán kính R (Assumption 4.2), chọn $\eta = 1/2$, $\rho = R/(2\sqrt{d})$, số bước để đạt loss sai số ϵ là $T \lesssim d \cdot \frac{L(\theta_0) - \min L}{\mu R^2} + \ln \frac{\mu R^2}{8d\epsilon}$ (paper viết dạng 2 giai đoạn: burn-in tuyến tính rồi decay hàm mũ). Điểm mấu chốt: bound này **không phụ thuộc condition number** (tỉ lệ $\lambda_{max}/\lambda_{min}$ của Hessian), trong khi GD/SignGD (phân tích ở Appendix D.1, không trích chi tiết số ở đây) có runtime phụ thuộc condition number ngay cả trên hàm quadratic đơn giản.

6. Giới hạn

Paper có Appendix C "Limitations" riêng, nêu rõ (trích lại):

- **Chưa scale thí nghiệm so sánh lên model/dataset lớn hơn**: Sophia được chứng minh scale tới 6.6B tham số (và "không có trở ngại căn bản" để scale tiếp), nhưng paper **không** so sánh với AdamW/Lion ở quy mô lớn hơn do hạn chế tài nguyên; nhóm tác giả tin Sophia sẽ còn nhanh hơn ở quy mô lớn hơn (dựa trên xu hướng scaling law quan sát được) nhưng đây là niềm tin, chưa phải kết quả thực nghiệm trong paper.
- **Đánh giá downstream còn hạn chế**: chỉ đánh giá trên validation loss pretraining và 4 subtask SuperGLUE, một phần vì model ở quy mô này (tới 6.6B) chưa có đủ năng lực cho các đánh giá phức tạp hơn (in-context learning phức tạp, suy luận toán học).
- **Chưa đánh giá ngoài miền ngôn ngữ**: paper tập trung hoàn toàn vào LLM pretraining; một optimizer "tổng quát" cần được kiểm tra thêm trên ảnh, RL, đa phương thức, nhưng việc này để lại cho công việc tương lai do hạn chế tài nguyên tính toán.
- Nhận xét thêm của người ghi chú (không phải paper nói): paper này có nhiều version (v1-v4) với phạm vi thí nghiệm khác nhau (bản đầu nhỏ hơn, bản v4 thêm GPT NeoX 6.6B); khi trích dẫn Sophia ở nơi khác, nên ghi rõ version để tránh nhầm quy mô thí nghiệm đã kiểm chứng.

7. Liên hệ với các bài khác

- **opt 06 (Adam) / opt 07 (AdamW)**: AdamW là baseline chính xuyên suốt paper; Sophia có cùng cấu trúc 2 trạng thái (m và h, tương tự m và v của Adam) nên chi phí memory ngang AdamW.
- **opt 11 (Lion)**: Lion là baseline so sánh thứ hai (Hình 5, 6, 7); paper chỉ ra Lion cần LR lớn hơn gợi ý gốc (cho vision) khi áp dụng cho LLM, và theo kết quả của chính paper Sophia, Lion chỉ nhanh hơn AdamW rất ít trên GPT-2 Small/Medium (ví dụ 125M: Lion 2.924 so với AdamW 2.921, hầu như không khác biệt), khác hẳn mức cải thiện lớn Lion đạt được trên ảnh/diffusion (theo paper Lion, opt 11) - một điểm đối chiếu thú vị giữa 2 bài về việc lợi ích optimizer có thể rất khác nhau giữa miền ứng dụng.
- **opt 12 (Adafactor)**: paper Sophia trích dẫn Adafactor như "một phiên bản low-rank của Adam" được dùng trong T5/PaLM (Raffel et al. 2020; Chowdhery et al. 2022); khác hướng với Sophia (Adafactor giảm memory, Sophia tăng tốc hội tụ bằng thông tin bậc 2), 2 hướng về nguyên tắc có thể kết hợp (dùng factorized accumulator cho phần m, thêm Hessian-based preconditioner) nhưng paper không thử nghiệm việc này.
- **opt 14 (Shampoo)**: cùng là optimizer "bậc hai"/preconditioning-based cho deep learning quy mô lớn; paper Sophia trích Shampoo-style optimizer (Anil et al. 2020, "Scalable second order optimization")

như công trình trước đó cố gắng đưa thông tin bậc 2 vào training lớn nhưng cần offload tính toán Hessian sang CPU; khác với Sophia (ước lượng rẻ, cập nhật mỗi 10 bước, chạy hoàn toàn trên accelerator).

- **llm content/01_2017_attention_is_all_you_need.md** (đã kiểm tra tồn tại bằng `ls`): Sophia thử nghiệm trên GPT-2/GPT NeoX, cả hai đều là Transformer decoder-only; hiểu kiến trúc attention cơ bản giúp hình dung rõ các mô hình trong thí nghiệm, dù bài Sophia không đi sâu vào chi tiết kiến trúc ngoài việc nhắc trick ”scale key-value theo layer index” (Mục 5 ở trên).

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU <= 8 GB, batch 1). **(suy luận)**:

- **Memory**: Sophia có cùng số trạng thái (2: m, h) như AdamW, nên **không giúp giảm VRAM** so với AdamW (khác Adafactor ở opt 12, hoặc Lion ở opt 11 vốn chỉ cần 1 trạng thái). Đây là điểm khác biệt quan trọng cần lưu ý: nếu mục tiêu chính là tiết kiệm VRAM trên GPU 8GB, Sophia **không** phải lựa chọn phù hợp hơn AdamW về mặt này.
- **Overhead compute thêm**: cần thêm 1 Hessian-vector-product (Sophia-H) hoặc 1 lượt forward/backward với label lấy mẫu (Sophia-G) mỗi $k = 10$ bước, trên 1 subset nhỏ của batch (32 hoặc 240 example trong paper, nhưng dự án chạy batch 1, nên ”subset” này cũng chỉ có thể là chính batch 1 đó hoặc vài bước gradient-accumulation gần nhất); với batch 1, việc ước lượng Hessian-vector-product có thể rất nhiễu (paper luôn thử nghiệm với batch cỡ hàng trăm tới hàng nghìn, chưa có bằng chứng về hành vi ở batch rất nhỏ).
- **Lợi ích tốc độ hội tụ**: nếu mục tiêu là **giảm số bước** cần để distill/finetune (ví dụ muốn hội tụ nhanh hơn trong ngân sách thời gian cố định, không phải giới hạn VRAM), cơ chế ”thích nghi theo độ cong + clip an toàn” của Sophia về nguyên tắc vẫn áp dụng được cho finetune quy mô nhỏ, nhưng paper **chỉ** kiểm chứng ở pretraining từ đầu, batch lớn (480-2048); chưa có cơ sở trong paper để khẳng định lợi ích tương tự khi finetune ngắn, batch rất nhỏ như dự án.
- Tóm lại (suy luận): trong số các optimizer đã đọc (LAMB, Lion, Adafactor, Sophia), Sophia có vẻ **ít phù hợp nhất** với ràng buộc ”GPU 8GB, batch 1” của dự án này, vì không giảm memory và cơ chế cốt lõi (ước lượng Hessian trên subset) giả định có nhiều example để lấy mẫu; Adafactor hoặc Lion (opt 11, opt 12) có vẻ là lựa chọn thực tế hơn cho dự án nếu cần đổi optimizer.

9. Thí nghiệm gợi ý

1. **Tái hiện ablation clipping ở quy mô nhỏ**: trên 1 mô hình nhỏ (vài chục triệu tham số), so sánh 4 cấu hình giống Hình 8c của paper: (a) AdamW, (b) clip theo toạ độ không preconditioner, (c) preconditioner Hessian (Hutchinson) không clip, (d) Sophia-H đầy đủ (preconditioner + clip). Đo: validation loss theo bước. Kỳ vọng theo paper: (b) đã tốt hơn AdamW, (d) tốt nhất; (c) có thể mất ổn định nếu k nhỏ.
2. **Đo tỉ lệ toạ độ bị clip theo γ**: với 1 model nhỏ, quét γ qua vài giá trị (x0.5, x1, x2 quanh 1 điểm khởi đầu), đo tỉ lệ toạ độ có $|m_t/\max\{\gamma h_t, \epsilon\}| < 1$ (không bị clip). Mục tiêu: tìm γ cho tỉ lệ trong khoảng 10-50% theo đúng gợi ý thực hành ở Mục 3.4, rồi kiểm tra có transfer được sang model lớn hơn chút không.
3. **So sánh Hutchinson vs GNB ở batch rất nhỏ**: vì dự án chạy batch 1, đáng kiểm tra riêng: ước lượng Hutchinson/GNB có nhiễu thế nào khi chỉ có 1-vài example (so với 32/240 example paper dùng). Đo: phương sai của $\hat{h}$ qua nhiều lần lấy mẫu độc lập ở cùng 1 điểm θ, với batch 1 so với batch giả định lớn hơn (dùng gradient-accumulation để mô phỏng). Diễn giải: nếu phương sai quá lớn ở batch 1, Sophia khó áp dụng trực tiếp cho finetune batch 1 thật.
4. **Đo overhead thực tế của Hessian-vector-product trên GPU 8GB**: với model cỡ LightOnOCR-3-0.8B (hoặc model thay thế cùng cỡ), đo thời gian 1 lần Hutchinson Hessian-vector-product so với 1 lần backward thường, và VRAM peak tăng thêm bao nhiêu (vì Hessian-vector-product cần 1 lượt ”double

backward"). Đối chiếu với con số 0.12s/0.09s "overhead Hessian" của paper (Table 1) để xem tỉ lệ overhead có tương đương trên phần cứng nhỏ hơn không.

5. **So sánh tốc độ hội tụ finetune ngắn**: finetune 1 model nhỏ vài trăm-vài nghìn bước bằng AdamW (đã tune) và Sophia-H (dùng gợi ý γ, β_2 của paper, tune lại LR), đo số bước cần để đạt 1 mức loss cố định. Diễn giải: nếu Sophia không cho lợi ích rõ ở quy mô nhỏ/finetune ngắn (khác hẳn pretraining từ đầu quy mô lớn trong paper), đó là tín hiệu paper này ít áp dụng được trực tiếp cho giai đoạn finetune của dự án.

10. Tóm tắt công thức / khái niệm

- **Update Sophia** (Algorithm 3): $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$; mỗi k bước: $h_t = \beta_2 h_{t-k} + (1-\beta_2)\hat{h}_t$ (else $h_t = h_{t-1}$); $\theta_{t+1} = \theta_t - \eta_t \lambda \theta_t - \eta_t \cdot \text{clip}(m_t / \max\{\gamma h_t, \epsilon\}, 1)$.
- **Hutchinson estimator**: $\hat{h} = u \odot \nabla^2 L(\theta) u$, $u \sim \mathcal{N}(0, I)$, không chệch cho $\text{diag}(\nabla^2 L)$.
- **GNB estimator**: $\hat{h} = B \cdot \hat{g} \odot \hat{g}$ với $\hat{g}$ là gradient trên label lấy mẫu từ $\text{softmax}(f(\theta, x))$; không chệch cho đường chéo ma trận Gauss-Newton (chệch cho Hessian đầy đủ), luôn PSD.
- **Siêu tham số khuyến nghị**: $\beta_1 = 0.96, \beta_2 = 0.99, \epsilon = 10^{-12}$, $k = 10$, $\gamma = 0.01$ (Sophia-H) hoặc 0.05 (Sophia-G); LR Sophia $\approx 0.8\times$ LR AdamW hoặc 3-5$\times$ LR Lion.
- **Tiêu chí "k lần nhanh hơn"**: $\exists H_2,\ \min_{H_1} \text{Eval}(O_1, T, H_1) \geq \text{Eval}(O_2, T/k, H_2)$.
- **Định lý 4.3 (bản đơn giản hoá, lồi, deterministic)**: với $\eta = 1/2, \rho = R/(2\sqrt{d}), T \lesssim d \cdot \frac{L(\theta_0) - \min L}{\mu R^2} + \ln \frac{\mu R^2}{8d\epsilon}$ bước để đạt sai số ϵ; không phụ thuộc condition number.

11. Kết luận ngắn

Sophia là một optimizer bậc hai "nhẹ" cho pretraining LLM: ước lượng đường chéo Hessian bằng Hutchinson hoặc Gauss-Newton-Bartlett, chỉ cập nhật mỗi 10 bước, và clip update theo từng toạ độ để an toàn khi Hessian ước lượng sai/âm. Nhờ overhead mỗi bước nhỏ (dưới 5% wall-clock), tốc độ hội tụ nhanh hơn theo số bước (khoảng 2x so với AdamW trên GPT-2/GPT NeoX tới 6.6B tham số) dịch thành tốc độ nhanh hơn thật theo wall-clock và compute, điều mà nhiều optimizer bậc hai trước đó không đạt được; lợi thế này còn tăng theo kích thước model. Paper cũng chứng minh 1 bound lý thuyết không phụ thuộc condition number trên bản đơn giản hoá lồi. Giới hạn chính (paper tự nhận): chưa so sánh ở quy mô lớn hơn 6.6B, đánh giá downstream còn hẹp, và chưa thử ngoài miền ngôn ngữ. Với dự án nén/hậu huấn luyện VLM OCR 0.8B trên GPU 8GB batch 1, Sophia có vẻ ít phù hợp hơn Adafactor/Lion (đã đọc ở opt 11-12) vì không giảm memory optimizer-state và cơ chế ước lượng Hessian giả định có đủ example để lấy mẫu ổn định (suy luận).

Nguồn gốc: https://arxiv.org/abs/2305.14342

Bài 14

Shampoo: Preconditioned Stochastic Tensor Optimization (2018)

Tác giả: Vineet Gupta, Tomer Koren, Yoram Singer (Google Brain / Princeton).
Venue / arXiv: arXiv:1802.09568 (v2, 02/03/2018); công bố tại ICML 2018.
Nguồn: https://arxiv.org/abs/1802.09568
Chủ đề: thuật toán tối ưu ”bậc hai gần đúng” (approximate second-order / adaptive preconditioning) khai thác **cấu trúc tensor** của tham số mạng neural: thay vì một preconditioner đầy đủ kích thước $mn \times mn$ cho một ma trận tham số $m \times n$ (như full-matrix AdaGrad, quá tốn để dùng thực tế), Shampoo duy trì hai ma trận nhỏ hơn nhiều, $m \times m$ và $n \times n$ (hoặc một ma trận $n_i \times n_i$ cho mỗi chiều i của một tensor bậc k tổng quát), và chứng minh đây là một **xấp xỉ ngầm (implicit approximation)** có cơ sở lý thuyết của full-matrix AdaGrad, kèm chặn hối hận (regret bound) trong khung online convex optimization.
Vị trí trong lộ trình: đọc sau opt 12 (Adafactor, một cách khác để giảm chi phí bộ nhớ của second-moment bằng phân rã hạng 1 theo hàng/cột) và opt 13 (Sophia, một cách khác để đưa thông tin curvature/Hessian vào training quy mô lớn mà không tốn preconditioner đầy đủ). Đọc trước opt 15 (SOAP, cải tiến trực tiếp của chính Shampoo bằng cách chạy Adam trong không gian eigenbasis của các preconditioner Shampoo) và opt 16 (Muon, một hướng khác dùng orthogonalization thay cho preconditioning kiểu Kronecker). Cũng liên quan opt 06/07 (Adam/AdamW, nền so sánh chính trong các bài theo sau Shampoo).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Các phương pháp tối ưu bậc một thích ứng (adaptive first-order), như AdaGrad, dùng một **ma trận preconditioner** để biến đổi gradient trước khi cập nhật tham số: thay vì bước $w_{t+1} = w_t - \eta g_t$ (gradient descent trơn), ta dùng $w_{t+1} = w_t - \eta H_t^{-1} g_t$ với H_t là một ma trận dương xác định (positive definite) được tích luỹ từ lịch sử gradient. Paper gọi đây là ”adaptive regularization in online optimization”: H_t đóng vai trò một ”metric” thích ứng, co giãn khác nhau theo từng hướng trong không gian tham số.

Full-matrix AdaGrad định nghĩa H_t dựa trên $M_t = \sum_{s \le t} g_s g_s^T$ (tổng ngoại tích của tất cả gradient theo thời gian, với g_t là **vector hoá** (flatten) toàn bộ gradient). Đây là preconditioner ”đúng” theo nghĩa đạt chặn hối hận chặt nhất trong khung online convex optimization, nhưng nếu tham số có d chiều (ví dụ $d = m \times n$ cho một ma trận trọng số $m \times n$), M_t là ma trận $d \times d$: với d cỡ hàng triệu (bình thường cho một layer neural network), việc lưu trữ $O(d^2)$ số và tính nghịch đảo/luỹ thừa ma trận ($O(d^3)$ hoặc tệ hơn) là bất khả thi. Vì vậy, trong thực tế mọi người chỉ dùng phiên bản **đường chéo (diagonal)** của AdaGrad (chỉ giữ các phần tử trên đường chéo của M_t, bỏ hoàn toàn thông tin tương quan chéo giữa các toạ độ), như Adam/AdaGrad/RMSProp đều làm.

Câu hỏi của paper: có cách nào giữ lại **một phần** thông tin tương quan chéo (không chỉ đường chéo) mà vẫn khả thi về bộ nhớ/tính toán, cho tham số có cấu trúc **tensor** (ma trận, hoặc tensor bậc cao hơn, như trong các layer tích chập 4 chiều)?

Một hướng liên quan đã có trước đó là K-FAC (Martens & Grosse, 2015, Kronecker-Factored Approximate Curvature): xấp xỉ ma trận Fisher information bằng tích Kronecker của hai ma trận nhỏ hơn, dựa trên giả thiết độc lập thống kê của gradient cụ thể cho mạng feed-forward, và cần lấy mẫu (sample) từ phân phối dự đoán (predictive distribution) của mô hình. Paper nói rõ sự khác biệt: Shampoo **không** dựa vào giả thiết cấu trúc

mô hình cụ thể nào, chỉ cần thông tin gradient tiêu chuẩn, áp dụng được trong bối cảnh tối ưu stochastic/online tổng quát, và đi kèm chứng minh hội tụ cho trường hợp lồi (convex).

2. Ý tưởng chính

Với một ma trận tham số $W \in \mathbb{R}^{m\times n}$ và gradient $G_t \in \mathbb{R}^{m\times n}$ tại bước t, Shampoo duy trì **hai** ma trận tích luỹ nhỏ, một cho mỗi chiều:

$$L_t = L_{t-1} + G_t G_t^T \quad (\text{kích thước } m\times m), \qquad R_t = R_{t-1} + G_t^T G_t \quad (\text{kích thước } n\times n)$$

và cập nhật tham số bằng

$$W_{t+1} = W_t - \eta\, L_t^{-1/4}\, G_t\, R_t^{-1/4}.$$

Trực giác: L_t "tóm" tương quan giữa các **hàng** của gradient (tức giữa các neuron đầu ra, nếu W là ma trận trọng số của một layer fully-connected), R_t tóm tương quan giữa các **cột** (giữa các đầu vào). Nhân gradient với $L_t^{-1/4}$ ở bên trái và $R_t^{-1/4}$ ở bên phải tương đương (sau khi vector hoá/flatten cả hai phía) với nhân $\mathrm{vec}(G_t)$ với tích Kronecker $L_t^{-1/4} \otimes R_t^{-1/4}$, một ma trận $mn \times mn$ **không bao giờ được tạo ra một cách tường minh**. Đây là điểm mấu chốt: ta được hiệu ứng của một preconditioner đầy đủ $mn \times mn$ mà chi phí bộ nhớ chỉ là $m^2 + n^2$ (so với m^2n^2 của preconditioner đầy đủ) và chi phí tính toán là $O(m^3 + n^3)$ (so với $O(m^3n^3)$).

Số mũ $1/4$ (không phải $1/2$ như người ta có thể đoán ban đầu) được chọn vì: khi ghép cả hai preconditioner $L_t^{1/4}$ và $R_t^{1/4}$ thông qua tích Kronecker thành $H_t = L_t^{1/4} \otimes R_t^{1/4}$, H_t đóng vai trò preconditioner tổng, và vì mỗi "nửa" đã được áp dụng một lần ở bên trái và một lần ở bên phải, số mũ hiệu dụng tác động lên G_t tương đương số mũ $-1/2$ của một preconditioner AdaGrad kiểu chuẩn (tạo ra tốc độ suy giảm bước $O(1/\sqrt{t})$ tối ưu trong online convex optimization, giống cơ chế bên dưới AdaGrad).

Paper mở rộng ý tưởng này lên **tensor bậc** k **tổng quát** (không chỉ ma trận bậc 2): với tensor tham số k chiều, kích thước $n_1 \times \cdots \times n_k$, ta giữ k ma trận preconditioner, mỗi cái kích thước $n_i \times n_i$ cho chiều i, và áp dụng luỹ thừa $-1/(2k)$ cho mỗi chiều (thay vì $-1/4$ cố định của trường hợp ma trận $k = 2$, vì $1/(2\cdot 2) = 1/4$ khớp với công thức trên).

3. Phương pháp chi tiết

3.1 Thuật toán cho ma trận (Algorithm 1)

```
Khởi tạo: W1 = 0 (m×n); L0 = εI_m; R0 = εI_n
for t = 1, ..., T:
    nhận hàm mất mát f_t
    G_t = ∇f_t(W_t)
    L_t = L_{t-1} + G_t G_t^T
    R_t = R_{t-1} + G_t^T G_t
    W_{t+1} = W_t − η · L_t^{-1/4} G_t R_t^{-1/4}
```

ε là một hằng số nhỏ để tránh chia cho 0 khi L_0, R_0 còn là ma trận không (giống ε trong AdaGrad/Adam). Luỹ thừa ma trận A^α (với A đối xứng dương xác định) được định nghĩa qua phân rã trị riêng: nếu $A = \sum_i \lambda_i u_i u_i^T$ thì $A^\alpha = \sum_i \lambda_i^\alpha u_i u_i^T$ (trong thực hành, paper tính qua SVD).

3.2 Chặn hối hận cho trường hợp ma trận (Theorem 7)

Giả sử các gradient $G_1, \ldots, G_T$ có hạng (rank) tối đa r. Khi đó hối hận của Algorithm 1 so với bất kỳ W^* bị chặn bởi

$$\sum_{t=1}^{T} f_t(W_t) - \sum_{t=1}^{T} f_t(W^*) \le 2\sqrt{r}\, D \sqrt{\mathrm{Tr}(L_T^{1/4})\,\mathrm{Tr}(R_T^{1/4})},$$

với $D = \max_t \|W_t - W^*\|_F$ (norm Frobenius). Dưới điều kiện nhẹ (ví dụ gradient Lipschitz theo spectral norm), mỗi số hạng $\mathrm{Tr}(\cdot)$ tăng theo $O(T^{1/4})$, nên cận trên tổng thể là $O(\sqrt{T})$, đây là tốc độ tối ưu có thể đạt được trong online/stochastic convex optimization (cùng bậc với full-matrix AdaGrad và SGD lồi chuẩn).

Bước chứng minh quan trọng nhất là **Lemma 8**: với $g_t = \mathrm{vec}(G_t)$,

$$\varepsilon I_{mn} + \frac{1}{r}\sum_{t=1}^{T} g_t g_t^T \preceq \left(\varepsilon I_m + \sum_{t=1}^{T} G_t G_t^T\right)^{1/2} \otimes \left(\varepsilon I_n + \sum_{t=1}^{T} G_t^T G_t\right)^{1/2}.$$

Nói cách khác: tích Kronecker của hai preconditioner $L_T^{1/2}$ và $R_T^{1/2}$ (của Shampoo) **chặn trên** (theo thứ tự nửa xác định dương, Loewner order $\preceq$) preconditioner đầy đủ $mn \times mn$ của full-matrix AdaGrad (chia tỉ lệ bởi $1/r$). Điều này đảm bảo các trị riêng nhỏ của preconditioner đầy đủ (vốn quan trọng nhất cho việc preconditioning hiệu quả) không bị ”biến mất” trong xấp xỉ Kronecker; đây chính là ý nghĩa cụm từ ”xấp xỉ ngầm có cơ sở” của full-matrix AdaGrad mà paper nhấn mạnh trong phần giới thiệu.

3.3 Thuật toán và chặn hối hận cho tensor bậc k (Algorithm 2, Theorem 10)

Với tensor k chiều, định nghĩa $A^{(i)} = \mathrm{mat}_i(A)\,\mathrm{mat}_i(A)^T$ (ma trận $n_i \times n_i$ thu được bằng cách ”matricize” tensor theo chiều i rồi nhân với chuyển vị của chính nó, tổng quát hoá GG^T/G^TG của trường hợp ma trận). Thuật toán:

```
Khởi tạo: W1 = 0 (n1×...×nk); H0^i = εI_{ni} cho mọi i
for t = 1, ..., T:
    G_t = ∇f_t(W_t)
    G̃_t ← G_t
    for i = 1, ..., k:
        H_t^i = H_{t-1}^i + G_t^{(i)}
        G̃_t ← G̃_t ×_i (H_t^i)^{-1/(2k)}        # ×_i: tensor-matrix product theo chiều i
    W_{t+1} = W_t − η G̃_t
```

($\times_i$ là phép nhân tensor-ma trận theo chiều i: tổng quát hoá phép nhân ma trận thông thường cho tensor nhiều chiều hơn 2; các phép nhân theo các chiều khác nhau giao hoán được với nhau, nên thứ tự lặp qua $i = 1, \ldots, k$ không ảnh hưởng kết quả.) Chặn hối hận (Theorem 10) có dạng tương tự Theorem 7, với $r = (\prod_{i=1}^{k} r_i)^{1/k}$ (r_i là chặn hạng của matricization theo chiều i), và mỗi số hạng vết (trace) co lại theo $O(T^{1/2k})$, nên tích của k số hạng vẫn cho tổng thể $O(\sqrt{T})$ bất kể bậc tensor k.

3.4 Biến thể đường chéo (Algorithm 3, Appendix A)

Khi một chiều n_i quá lớn để lưu ma trận $n_i \times n_i$ đầy đủ hoặc tính SVD của nó (paper dùng ngưỡng khoảng **1200** cho mỗi chiều trong thực nghiệm), Shampoo tự động chuyển chiều đó sang **đường chéo**: $L_t = L_{t-1} + \mathrm{diag}(G_t G_t^T)$ (chỉ lưu $O(n_i)$ số thay vì $O(n_i^2)$). Các chiều khác của cùng tensor không bị ảnh hưởng và vẫn có thể dùng preconditioner đầy đủ. Theorem 13 chứng minh một chặn hối hận tương tự cho biến thể này (dùng norm ℓ_∞ thay Frobenius cho D), cho phép paper áp dụng Shampoo lên các layer rất lớn (ví dụ embedding) mà không phải sửa kiến trúc hay code huấn luyện.

3.5 Hai điều chỉnh thực nghiệm (heuristic, Section 5)

1. **Cập nhật trễ (delayed update)**: không tính lại luỹ thừa ma trận ($L_t^{-1/4}$, $R_t^{-1/4}$...) ở mỗi bước, mà chỉ tính lại sau mỗi 20-100 bước; paper báo cáo điều này ”hầu như không ảnh hưởng độ chính xác” nhưng cải thiện đáng kể thời gian chạy trung bình mỗi bước.

2. **Momentum**: tính trung bình trượt (running average) của gradient $\bar{G}_t = \alpha\bar{G}_{t-1} + (1-\alpha)G_t$ với $\alpha = 0.9$ cố định, dùng $\bar{G}_t$ thay G_t trong bước cập nhật tham số (nhưng L_t, R_t vẫn tích luỹ từ G_t gốc, theo cách đọc Algorithm 1/2 nguyên bản). Paper nói điều này "cải thiện nhẹ" tốc độ hội tụ.

Cả hai điều chỉnh này là **heuristic thực hành**, không có chứng minh lý thuyết mở rộng kèm theo trong bài (quan sát của người ghi chú: các định lý 7, 10, 13 đều chứng minh cho thuật toán *không* có hai điều chỉnh này).

(tự tính) Ví dụ số nhỏ minh hoạ bước cập nhật: giả sử $m = n = 2$ (ma trận 2×2), và tại một bước t, $L_t = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, $R_t = \begin{pmatrix} 9 & 0 \\ 0 & 1 \end{pmatrix}$ (giả định đã là ma trận đường chéo để luỹ thừa tính tay dễ dàng). Vì L_t đường chéo, $L_t^{-1/4} = \begin{pmatrix} 4^{-1/4} & 0 \\ 0 & 1^{-1/4} \end{pmatrix} = \begin{pmatrix} 0.7071 & 0 \\ 0 & 1 \end{pmatrix}$ (vì $4^{1/4} = \sqrt{2} \approx 1.4142$, nên $4^{-1/4} \approx 0.7071$). Tương tự $R_t^{-1/4} = \begin{pmatrix} 9^{-1/4} & 0 \\ 0 & 1 \end{pmatrix} \approx \begin{pmatrix} 0.5774 & 0 \\ 0 & 1 \end{pmatrix}$ (vì $9^{1/4} = \sqrt{3} \approx 1.7321$, $9^{-1/4} \approx 0.5774$). Nếu $G_t = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, thì $L_t^{-1/4} G_t R_t^{-1/4} = \begin{pmatrix} 0.7071 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0.5774 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.7071 & 0.7071 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0.5774 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.4084 & 0.7071 \\ 0.5774 & 1 \end{pmatrix}$. So với gradient thô G_t (mọi phần tử bằng 1), bước cập nhật đã được co giãn khác nhau theo hàng và cột: hàng 1 (ứng với trị riêng L nhỏ hơn, 4) bị co lại nhiều hơn hàng 2, cột 1 (ứng với trị riêng R nhỏ hơn, 9, nhưng lớn hơn 4 nên co ít hơn theo quy tắc luỹ thừa âm) bị co khác cột 2. Đây chính là hiệu ứng "preconditioning không đồng nhất theo hướng" mà Shampoo nhằm đạt được, tương tự AdaGrad nhưng theo cấu trúc hàng/cột thay vì từng toạ độ riêng lẻ.

4. Thí nghiệm và kết quả chính

Paper thử nghiệm trên hai domain: phân loại ảnh (CIFAR-10, CIFAR-100) và statistical language modeling (LM1B, One Billion Word benchmark). Mọi thí nghiệm dùng mini-batch size 128; với Shampoo, batch chỉ ảnh hưởng việc G_t là gradient trung bình hoá qua 128 ví dụ (không có cơ chế đặc biệt khác cho batch). Baseline so sánh: SGD (với momentum, paper gọi "Momentum" trong hình), Adam, AdaGrad.

Bảng 1 (tốc độ, số bước/giây, batch 128, GPU Tesla K40):

Dataset (kiến trúc)	SGD	Adam	AdaGrad	Shampoo
CIFAR-10 (ResNet-32)	2.184	2.184	2.197	2.151
CIFAR-10 (Inception, 20 lớp)	3.638	3.667	3.682	3.506
CIFAR-100 (ResNet-55)	1.210	1.203	1.210	1.249
LM1B (Attention)	4.919	4.871	4.908	3.509

Quan sát đáng chú ý của paper: dù Shampoo thực hiện nhiều phép tính hơn hẳn mỗi bước (luỹ thừa ma trận, không chỉ phép toán theo toạ độ như SGD/Adam/AdaGrad), thời gian chạy thực tế **không tệ hơn nhiều**, và với ResNet-55 trên CIFAR-100, Shampoo còn **nhanh hơn** cả ba baseline (1.249 so với 1.203-1.210 bước/giây).

Thí nghiệm ảnh (Section 6.1): ResNet-32, 2.4M tham số, trên CIFAR-10, đạt lỗi test 5%; Inception nhỏ 20 lớp (Zhang et al. 2017), 1.65M tham số, đạt lỗi test 7.5% trên CIFAR-10. ResNet-55 trên CIFAR-100 (13.5M tham số, mọi biến đều là tensor bậc 4 vì toàn bộ là layer tích chập, lớp lớn nhất kích thước $(256, 3, 3, 256)$, không dùng batchnorm/dropout) đạt lỗi test 24%. Với mỗi thuật toán, paper quét 10 learning rate khác nhau giữa 0.01 và 10.0, chọn kết quả tốt nhất. Hình 2 và 3 (loss theo epoch) cho thấy Shampoo hội tụ nhanh hơn rõ rệt so với AdaGrad/Adam/Momentum trên cả ResNet và Inception ở CIFAR-10.

Thí nghiệm language model (Section 6.2): mô hình Attention (Vaswani et al. 2017, chính là kiến trúc Transformer) với 9.8M tham số trên LM1B, mọi tensor có bậc tối đa 2 (lớp lớn nhất $(2000, 256)$). Với Shampoo, paper dùng trực tiếp learning rate mặc định $\eta = 1.0$ (không cần dò); các thuật toán khác phải dò learning rate riêng. Hình 4 (test log-perplexity theo số bước, tới 5×10^5 bước) cho thấy Shampoo đạt log-perplexity thấp hơn rõ rệt so với AdaGrad, Momentum, Adam ở cùng số bước huấn luyện. Đây là kết quả về "tốc độ hội tụ theo số bước", paper không báo cáo so sánh theo wall-clock time cho riêng thí nghiệm LM1B này (Bảng 1 chỉ cho số bước/giây, phải tự kết hợp với Hình 4 để suy ra tốc độ theo thời gian thực).

5. Phân tích / ablation

Paper có ít ablation có hệ thống so với các bài sau này (opt 11 Lion, opt 13 Sophia); phần phân tích chính của bài này là **lý thuyết** (chứng minh chặn hối hận), không phải ablation thực nghiệm theo từng thành phần. Một số điểm gần nhất với "ablation":

- **So sánh kiến trúc có/không batchnorm**: mô hình ResNet-55 trên CIFAR-100 được chọn "không dùng batchnorm, dropout..." (trích nguyên văn Section 6.1) có thể là lựa chọn có chủ đích để kiểm tra Shampoo trong điều kiện khó hơn (không có các kỹ thuật ổn định hoá khác hỗ trợ), nhưng paper không nêu rõ động cơ hay so sánh với phiên bản có batchnorm.
- **Biến thể đường chéo (Appendix A)**: được mô tả là kích hoạt tự động khi một chiều vượt ngưỡng ~1200, và paper nói "không có hy sinh hiệu năng rõ rệt nào" khi dùng biến thể này cho các chiều lớn, nhưng không có bảng số liệu so sánh trực tiếp đầy đủ Shampoo-đầy-đủ so với Shampoo-đường-chéo trên cùng mô hình để người đọc tự kiểm tra con số cụ thể; đây là một khẳng định định tính không kèm bảng số (quan sát của người ghi chú, không phải lỗi của paper, nhưng cần nêu rõ vì brief yêu cầu không gán cho paper điều nó không trình bày bằng số liệu).
- **Hai điều chỉnh heuristic (delayed update, momentum)**: paper nói "gần như không ảnh hưởng độ chính xác" (cho delayed update) và "cải thiện nhẹ" (cho momentum), nhưng cũng không có bảng ablation riêng tách hai yếu tố này; các con số trong Bảng 1 và Hình 2-4 đều đã bao gồm cả hai điều chỉnh cùng lúc.

6. Giới hạn

Bài báo này (khác với các bài sau trong lộ trình, như opt 11 Lion hay opt 13 Sophia) **không có một mục "Limitations" tường minh**; các giới hạn dưới đây được người ghi chú tổng hợp từ nội dung Section 5 (Implementation details) và cấu trúc thuật toán, không phải trích dẫn trực tiếp một đoạn "Limitations" của paper:

- **Preconditioner khối-đường-chéo giữa các tensor**: optimizer coi mỗi tensor (mỗi layer) là một biến tối ưu độc lập và áp Shampoo riêng cho từng tensor; paper tự nói điều này "chỉ nắm được tương quan **trong** một tensor, hoàn toàn bỏ qua tương quan **giữa** các tensor khác nhau trong mô hình" (trích gần nguyên văn Section 5). Đây là một xấp xỉ khối-đường-chéo (block-diagonal) ở cấp độ toàn mô hình.
- **Chiều quá lớn vẫn phải dùng đường chéo**: với ngưỡng ~1200, các layer rất lớn (ví dụ embedding của vocabulary lớn) buộc phải rơi về biến thể đường chéo cho chiều đó, mất phần lớn lợi ích "preconditioning đầy đủ" mà Shampoo hướng tới.
- **Chi phí tính luỹ thừa ma trận**: dù $O(m^3 + n^3)$ nhỏ hơn hẳn $O(m^3 n^3)$, phép tính SVD/luỹ thừa ma trận vẫn đáng kể hơn phép toán theo toạ độ của Adam/AdaGrad; hai điều chỉnh heuristic (delayed update, tính lại mỗi 20-100 bước) là cách paper giảm chi phí này trong thực hành, nhưng đây là một thoả hiệp (trade-off) không được paper phân tích lý thuyết đầy đủ (chặn hối hận Theorem 7/10 là cho thuật toán cập nhật preconditioner ở **mọi** bước, không phải phiên bản trễ dùng trong thực nghiệm).
- **Chứng minh lý thuyết chỉ cho hàm lồi (convex)**: giống hầu hết các chứng minh regret-bound trong online convex optimization, Theorem 7/10/13 áp dụng cho hàm mất mát lồi; mạng neural sâu thực tế có hàm mất mát không lồi, nên các con số "tối ưu $O(\sqrt{T})$" chỉ là động lực lý thuyết (theoretical

motivation), không phải cam kết hội tụ cho trường hợp thực nghiệm.

- **Chỉ thử trên mô hình tương đối nhỏ**: mô hình lớn nhất trong thí nghiệm là 13.5M tham số (ResNet-55); không có thí nghiệm ở quy mô hàng trăm triệu hay hàng tỷ tham số như các optimizer công bố sau này (LAMB, Lion, Sophia). Đây là hạn chế về **thời điểm công bố** (2018) hơn là hạn chế lý thuyết của phương pháp, nhưng ảnh hưởng tới mức độ "đã được kiểm chứng ở quy mô LLM hiện đại" của chính bài gốc.

7. Liên hệ với các bài khác

- **opt 06/07 (Adam, AdamW)**: Shampoo không thay cơ chế EMA hai moment của Adam, mà thay hẳn **dạng** của preconditioner: Adam dùng preconditioner đường chéo (một số vô hướng cho mỗi toạ độ tham số), Shampoo dùng hai ma trận đầy đủ (không đường chéo) cho hai chiều của một tensor bậc 2. Về mặt lý thuyết, cả hai đều là trường hợp đặc biệt của khung "adaptive regularization" (Section 2.2 của paper này); Adam/AdaGrad đường chéo tương ứng chọn $\Phi(H)$ sao cho nghiệm tối ưu là ma trận đường chéo, Shampoo chọn cách xấp xỉ khác (Kronecker) để giữ lại tương quan chéo mà vẫn rẻ.
- **opt 12 (Adafactor)**: cùng khai thác cấu trúc ma trận 2 chiều của tham số, nhưng theo cách khác hẳn: Adafactor chỉ lấy **tổng theo hàng và theo cột** của bình phương gradient (một phân rã hạng 1, rất rẻ, $O(m+n)$ số) để tái tạo gần đúng second-moment đường chéo đầy đủ; Shampoo tích luỹ **toàn bộ** ma trận $G_t G_t^T$ và $G_t^T G_t$ ($O(m^2+n^2)$ số, đắt hơn Adafactor nhưng rẻ hơn preconditioner đầy đủ $O(m^2 n^2)$) rồi lấy luỹ thừa $-1/4$ của toàn ma trận đó, giữ lại tương quan chéo giữa các hàng/cột mà Adafactor bỏ qua hoàn toàn. Nói cách khác, Adafactor đánh đổi độ chính xác của preconditioner để lấy bộ nhớ tối thiểu; Shampoo đánh đổi nhiều bộ nhớ/compute hơn Adafactor để lấy preconditioner chính xác hơn (đã được xác nhận trong `12_2018_adafactor.md`, dòng liên hệ với opt 14: "Shampoo đặc biệt đáng so sánh: cả 2 cùng khai thác cấu trúc ma trận của tham số, nhưng Adafactor chỉ lấy tổng hàng/cột (hạng 1, rẻ), còn Shampoo tính preconditioner đầy đủ hơn theo mỗi chiều tensor (tốn hơn, chính xác hơn)").
- **opt 13 (Sophia)**: cả hai đều được paper Sophia (đã đọc ở opt 13) xếp vào nhóm "optimizer bậc hai/preconditioning-based cho deep learning quy mô lớn"; Sophia trích dẫn một công trình Shampoo-style (Anil et al. 2020) như một hướng trước đó cần offload tính toán sang CPU, khác với cách tiếp cận "ước lượng Hessian rẻ, chạy hoàn toàn trên accelerator" của Sophia (đã xác nhận trong `13_2023_sophia.md`, dòng: "paper Sophia trích Shampoo-style optimizer (Anil et al. 2020, 'Scalable second order optimization') như công trình trước đó cố gắng đưa thông tin bậc 2 vào training lớn nhưng cần offload tính toán Hessian sang CPU").
- **opt 15 (SOAP)**: theo tên và vị trí lộ trình (chưa đọc đầy đủ khi viết ghi chú này, chỉ dựa trên tiêu đề cố định trong titles.tsv, "SOAP: Improving and Stabilizing Shampoo using Adam"), đây là cải tiến trực tiếp của chính Shampoo, dự kiến chạy Adam trong không gian các eigenvector của preconditioner Shampoo để ổn định hơn; bài này sẽ cần đọc kỹ Shampoo (bài hiện tại) làm nền.
- **opt 45 (Deconstructing What Makes a Good Optimizer for Language Models)**: paper này (đã có trong thư mục, đã đọc một phần khi kiểm tra liên hệ) chỉ so sánh thực nghiệm chính giữa các optimizer **đường chéo** (Adam, Adafactor, Lion, Signum, SGD) và nói rõ Shampoo (optimizer phi đường chéo, đã từng được Gupta et al. 2018 - tức chính là bài hiện tại - báo cáo tốt hơn Adam) không được đưa vào so sánh thực nghiệm chính của họ (đã xác nhận trong `45_2024_deconstructing_llm_optimizer.md`, dòng: "các optimizer phi đường chéo như Shampoo (đã được dẫn là có hiệu năng tốt hơn Adam trong công trình khác, Gupta et al., 2018a) không được đưa vào so sánh thực nghiệm chính").
- **opt 10 (LAMB)**: `10_2019_lamb.md` (đã viết trong lộ trình này) tự nhận xét trust ratio của LARS/LAMB "dùng norm toàn layer, không xét tới cấu trúc nội tại ma trận (so với Shampoo/SOAP ở opt 14/15 sau này, dùng preconditioning theo cấu trúc tensor chi tiết hơn)" - xác nhận Shampoo đứng ở một "mức" preconditioning chi tiết hơn LAMB trong phân loại các optimizer thích ứng theo cấu trúc tham số.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là **suy luận** của người ghi chú, áp cho bài toán cụ thể: nén/huấn luyện tiếp (post-training) mô hình LightOnOCR-3-0.8B, một VLM OCR lai GatedDeltaNet + full attention, khoảng 3600 token/trang, ràng buộc GPU $\leq$ 8GB, batch size 1. Paper không nhắc đến mô hình này hay VLM OCR nói chung.

- **Chi phí bộ nhớ của chính Shampoo là rào cản chính cho use case này.** Với tham số là ma trận $m \times n$, Shampoo cần lưu hai ma trận $m \times m$ và $n \times n$ cho **mỗi** layer, cộng thêm chi phí tính SVD/luỹ thừa định kỳ. (tự tính) Với một linear layer 4096×4096 (kích thước phổ biến trong các LLM/VLM cỡ vài trăm triệu đến vài tỷ tham số, dùng để minh hoạ bậc độ lớn), L_t, R_t mỗi ma trận có $4096^2 \approx 16.8$ triệu phần tử; ở fp32, mỗi ma trận tốn $16.8 \times 10^6 \times 4$ byte $\approx$ 67MB, hai ma trận (L_t và R_t) tốn $\approx$ 134MB **cho một layer duy nhất**. Một mô hình 0.8B tham số có hàng chục layer như vậy; tổng bộ nhớ cho preconditioner Shampoo full có thể dễ dàng vượt quá, hoặc ít nhất cạnh tranh gay gắt với, toàn bộ 8GB VRAM khả dụng, trước cả khi tính tới trọng số mô hình, activation, và optimizer state khác. Đây là lý giải trực tiếp (tự tính, dựa trên đúng công thức $m^2 + n^2$ mà paper nêu) cho việc Shampoo **nguyên bản, không có biến thể đường chéo**, khó dùng trực tiếp trong ràng buộc 8GB/batch 1 của dự án.
- **Biến thể đường chéo (Section 3.4/Appendix A) là lối thoát khả dĩ nhưng làm mất phần lớn lợi ích.** Nếu áp ngưỡng (ví dụ 1200, như paper dùng) để tự động chuyển các chiều lớn (ví dụ chiều ẩn 4096 của mô hình giả định trên) về đường chéo, chi phí bộ nhớ giảm mạnh ($O(n)$ thay $O(n^2)$), nhưng khi đó phần lớn các chiều "quan trọng" của một mô hình cỡ 0.8B (vốn thường có hidden dim hàng nghìn) sẽ rơi về đường chéo, khiến Shampoo gần như suy biến thành một biến thể của AdaGrad đường chéo, không còn giữ được ưu điểm chính (tương quan chéo giữa các hàng/cột) mà paper quảng bá.
- **Kết luận suy luận của người ghi chú**: Shampoo (bản đầy đủ) **ít phù hợp trực tiếp** cho việc fine-tune/nén LightOnOCR-3-0.8B trong 8GB/batch 1, vì chi phí hai ma trận preconditioner mỗi layer quá lớn so với ngân sách bộ nhớ hạn hẹp, và biến thể đường chéo cần thiết để vừa bộ nhớ sẽ xoá bỏ phần giá trị cốt lõi của phương pháp. Thứ có giá trị hơn để mang sang dự án không phải là chạy trực tiếp Shampoo, mà là **ý tưởng cấu trúc Kronecker/theo-chiều-tensor**: nếu muốn một preconditioner tốt hơn đường chéo nhưng vẫn rẻ, có thể chỉ áp dụng Shampoo-style preconditioning cho một tập con rất nhỏ các layer "đáng giá nhất" (ví dụ chỉ layer đầu ra hoặc một vài layer attention quan trọng nhất đối với chất lượng OCR), giữ các layer còn lại ở preconditioning đường chéo kiểu AdamW/Adafactor, một chiến lược lai mà paper không thử nhưng cấu trúc thuật toán (mỗi tensor xử lý độc lập) cho phép làm được dễ dàng về mặt kỹ thuật.
- **Liên hệ ngược với Adafactor (opt 12)**: nếu mục tiêu là tiết kiệm bộ nhớ tối đa cho optimizer state trên 8GB GPU (đã được đánh giá ở `12_2018_adafactor.md` là "một trong những bài liên quan trực tiếp nhất" tới dự án), hướng của Adafactor (phân rã hạng 1, $O(m+n)$) vẫn khả thi hơn nhiều so với hướng của Shampoo ($O(m^2+n^2)$) cho đúng ràng buộc bộ nhớ của dự án này; Shampoo đánh đổi quá nhiều bộ nhớ để lấy preconditioning chính xác hơn, một đánh đổi không hợp với ràng buộc "GPU $\leq$ 8GB, batch 1" đang có.

9. Thí nghiệm gợi ý

1. **Đo chi phí bộ nhớ thực tế của preconditioner Shampoo trên một layer nhỏ.** Tạo một `nn.Linear(512, 512)` bằng PyTorch, tự cài đặt vòng lặp Algorithm 1 (tính L_t, R_t, luỹ thừa $-1/4$ qua `torch.linalg.eigh` hoặc SVD) trên vài batch dữ liệu ngẫu nhiên, đo bộ nhớ GPU qua `torch.cuda.max_memory_allocated()`. Kỳ vọng: bộ nhớ cho $L_t + R_t$ xấp xỉ $2 \times 512^2 \times 4$ byte $\approx$ 2.1MB cho layer này (tự tính, theo công thức $m^2 + n^2$ với $m = n = 512$), so với preconditioner đầy đủ lý thuyết $512^2 \times 512^2 \times 4$ byte (khổng lồ, không khả thi để thực sự cấp phát) - minh hoạ trực tiếp tỉ lệ tiết kiệm mà paper nêu. Diễn giải: nếu số đo khớp công thức $m^2 + n^2$, xác nhận hiểu đúng cơ chế; nếu lệch nhiều, có thể do PyTorch giữ thêm buffer trung gian (SVD, ma trận tạm) cần tính riêng.
2. **So sánh tốc độ hội tụ Shampoo vs Adam trên một bài toán hồi quy tuyến tính nhỏ (toy, không cần GPU mạnh).** Dùng một ma trận trọng số $W \in \mathbb{R}^{64 \times 64}$, loss là mean-squared-error của WX so

với Y (sinh ngẫu nhiên có cấu trúc tương quan giữa các cột X, ví dụ $X = AZ$ với A cố định, Z nhiễu Gaussian độc lập), chạy cả hai optimizer cùng số bước, cùng learning rate đã dò riêng cho mỗi thuật toán. Kỳ vọng: khi X có tương quan mạnh giữa các cột (preconditioner R_t của Shampoo "thấy" được tương quan này), Shampoo hội tụ (giảm loss) nhanh hơn rõ rệt theo số bước so với Adam/AdaGrad đường chéo, minh hoạ chính luận điểm cốt lõi của Lemma 8 (giữ lại thông tin tương quan chéo). Diễn giải: nếu không thấy khác biệt, kiểm tra lại dữ liệu có đủ tương quan cột hay không (dữ liệu độc lập theo cột sẽ không cho Shampoo lợi thế).

3. **Kiểm tra hiệu ứng ngưỡng chuyển sang đường chéo (biến thể Appendix A) trên một embedding layer lớn.** Tạo `nn.Embedding(50000, 256)` (vocab lớn, chiều ẩn nhỏ), cài đặt Shampoo với ngưỡng tự chọn (ví dụ 1000) để chiều vocab (50000) tự động dùng đường chéo còn chiều 256 dùng preconditioner đầy đủ, đo thời gian một bước cập nhật và so với việc dùng preconditioner đầy đủ (giả, không khả thi thực tế) cho cả hai chiều. Kỳ vọng: biến thể lai (đường chéo cho chiều lớn, đầy đủ cho chiều nhỏ) có thời gian mỗi bước thấp hơn nhiều so với cố gắng dùng preconditioner 50000×50000 (vốn sẽ out-of-memory ngay). Diễn giải: xác nhận tính thực tế của cơ chế ngưỡng tự động mà paper mô tả cho các layer embedding lớn, loại layer phổ biến trong VLM OCR (token embedding, vocabulary lớn cho văn bản).
4. **Đo ảnh hưởng của "delayed update" (tính lại luỹ thừa mỗi k bước) lên tốc độ wall-clock trên GPU 8GB.** Trên cùng bài toán hồi quy tuyến tính 64×64 ở thí nghiệm 2 (hoặc một mạng nhỏ hơn, ví dụ MLP 2 lớp), chạy Shampoo với $k \in \{1, 10, 50, 100\}$ (tính lại $L_t^{-1/4}, R_t^{-1/4}$ mỗi k bước), đo thời gian tổng và loss cuối cùng sau cùng số bước huấn luyện. Kỳ vọng: k lớn hơn giảm thời gian mỗi bước rõ rệt (ít phải tính SVD lại) với ảnh hưởng nhỏ lên loss cuối, phù hợp tuyên bố "hầu như không ảnh hưởng độ chính xác" của paper (Section 5). Diễn giải: nếu loss cuối xấu đi nhiều ở k lớn, thuật toán nhạy hơn paper nêu (ít nhất trên bài toán toy này) với tần suất cập nhật preconditioner, một điểm paper không có bảng ablation riêng để so sánh (đã nêu ở Mục 5), nên thí nghiệm này có thể bổ sung đúng khoảng trống đó.
5. **So sánh trực tiếp Shampoo-khối-đường-chéo-giữa-layer với một phiên bản "có tương quan giữa layer" giả định (toy).** Dùng một MLP 2 lớp nhỏ ($64 \to 64 \to 64$), chạy Shampoo chuẩn (mỗi layer một cặp L_t, R_t độc lập, đúng như paper) so với một phiên bản tuỳ biến trong đó preconditioner của layer 2 được tính luôn từ gradient nối (concatenate) của cả layer 1 và layer 2 (mô phỏng việc "nắm tương quan giữa tensor", điều paper tự nhận là không làm). Kỳ vọng: nếu hai layer có tương quan gradient mạnh (có thể tạo bằng cách cho layer 2 là một hàm tuyến tính gần như sao chép layer 1), phiên bản "nối" hội tụ nhanh hơn đôi chút, minh hoạ cụ thể cái giá phải trả của giả thiết khối-đường-chéo-giữa-layer mà Mục 6 (Giới hạn) đã nêu. Diễn giải: giúp cảm nhận định lượng mức độ quan trọng của giới hạn "chỉ nắm tương quan trong tensor, không giữa các tensor" trên một ví dụ cụ thể, dễ kiểm chứng trên GPU 8GB hoặc thậm chí CPU.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Cập nhật Shampoo (ma trận)	$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}$	Preconditioning hai phía, một cho hàng (L_t), một cho cột (R_t)
Tích luỹ preconditioner	$L_t = L_{t-1} + G_t G_t^T$, $R_t = R_{t-1} + G_t^T G_t$	Tương tự v_t của AdaGrad nhưng là ma trận, không phải vector
Luỹ thừa ma trận	$A^\alpha = \sum_i \lambda_i^\alpha u_i u_i^T$ (qua phân rã trị riêng $A = \sum_i \lambda_i u_i u_i^T$)	Định nghĩa phép toán "căn bậc 4 nghịch đảo" của ma trận
Preconditioner tổng (dạng Kronecker)	$H_t = L_t^{1/4} \otimes R_t^{1/4} \in \mathbb{R}^{mn \times mn}$	Không bao giờ tạo tường minh; là "diễn giải" cho thấy Shampoo tương đương một bước gradient descent được tiền điều kiện bởi H_t

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Chặn hối hận (ma trận, Theorem 7)	$\sum_t f_t(W_t) - \sum_t f_t(W^*) \leq 2\sqrt{r}D\sqrt{\mathrm{Tr}(L_T^{1/4})\mathrm{Tr}(R_T^{1/4})}$	Tốc độ $O(\sqrt{T})$, tối ưu trong online convex optimization
Thuật toán tensor bậc k	$\tilde{G}_t \leftarrow G_t$; với mỗi i: $H_t^i = H_{t-1}^i + G_t^{(i)}$, $\tilde{G}_t \leftarrow \tilde{G}_t \times_i (H_t^i)^{-1/(2k)}$	Tổng quát hoá Algorithm 1 cho tensor nhiều chiều hơn 2 (ví dụ layer tích chập 4D)
Biến thể đường chéo	$L_t = L_{t-1} + \mathrm{diag}(G_t G_t^T)$	Dùng khi một chiều quá lớn để lưu/luỹ thừa ma trận đầy đủ (ngưỡng ~1200 trong thực nghiệm của paper)
Chi phí bộ nhớ/compute	$O(m^2 + n^2)$ bộ nhớ, $O(m^3 + n^3)$ compute (so với $O(m^2n^2)$/$O(m^3n^3)$ của preconditioner đầy đủ)	Lý do thực tế để Shampoo khả thi hơn full-matrix AdaGrad

11. Kết luận ngắn

Shampoo chỉ ra rằng có thể xấp xỉ preconditioner đầy đủ (full-matrix AdaGrad), vốn bị coi là bất khả thi về bộ nhớ/tính toán cho mạng neural, bằng cách khai thác cấu trúc tensor của tham số: dùng tích Kronecker của các ma trận nhỏ hơn nhiều (một cho mỗi chiều), có chứng minh lý thuyết (Lemma 8, Theorem 7/10/13) rằng xấp xỉ này không làm mất các trị riêng nhỏ quan trọng của preconditioner đầy đủ, và đạt chặn hối hận $O(\sqrt{T})$ tối ưu trong khung online convex optimization. Thực nghiệm trên CIFAR-10/100 và LM1B (quy mô nhỏ theo chuẩn hiện tại, tới 13.5M tham số) cho thấy Shampoo hội tụ nhanh hơn AdaGrad/Adam/SGD theo số bước, với chi phí thời gian mỗi bước không tệ hơn nhiều nhờ hai điều chỉnh heuristic (delayed update, momentum) và biến thể đường chéo cho các chiều quá lớn. Hạn chế chính (tổng hợp từ nội dung bài, không phải một mục Limitations tường minh) là xấp xỉ khối-đường-chéo giữa các tensor khác nhau trong mô hình, chi phí bộ nhớ $O(m^2 + n^2)$ vẫn có thể lớn cho các layer rộng, và chứng minh lý thuyết chỉ áp dụng cho hàm lồi. Đối với dự án nén/huấn luyện LightOnOCR-3-0.8B trong 8GB/batch 1, Shampoo bản đầy đủ khó dùng trực tiếp do chi phí bộ nhớ preconditioner mỗi layer, nhưng ý tưởng cấu trúc Kronecker/theo-chiều-tensor có thể truyền cảm hứng cho một chiến lược lai (áp dụng chọn lọc cho vài layer quan trọng nhất), một hướng mà paper gốc không thử nhưng cấu trúc thuật toán cho phép.

Nguồn gốc: https://arxiv.org/abs/1802.09568

Bài 15

SOAP: Improving and Stabilizing Shampoo using Adam (2024)

Tác giả: Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira (Harvard University); David Brandfonbrener, Lucas Janson, Sham Kakade (Kempner Institute at Harvard University / Harvard University).
Venue / arXiv: arXiv:2409.11321 (v2, 31/01/2025; bản đầu nộp tháng 09/2024).
Nguồn: https://arxiv.org/abs/2409.11321
Chủ đề: chứng minh một kết nối hình thức (formal connection) giữa Shampoo (dùng luỹ thừa 1/2) và Adafactor: Shampoo (biến thể luỹ thừa 1/2) tương đương với chạy Adafactor trong **eigenbasis** (cơ sở trị riêng) của preconditioner Shampoo. Từ đó đề xuất SOAP (ShampoO with Adam in the Preconditioner's eigenbasis): chạy AdamW trong không gian xoay (rotated space) do Shampoo cung cấp, chỉ cập nhật lại eigenbasis định kỳ (mỗi f bước) nhưng cập nhật second-moment Adam ở **mọi** bước trong cơ sở hiện tại, giúp giảm suy giảm hiệu năng khi tần suất cập nhật preconditioner thấp.
Vị trí trong lộ trình: đọc ngay sau opt 14 (Shampoo, nền tảng trực tiếp mà SOAP mở rộng và phân tích lại). Đọc cùng nhóm với opt 06/07 (Adam/AdamW, "lõi" chạy bên trong eigenbasis của SOAP) và opt 12 (Adafactor, dùng trong biến thể "SOAP factorized" để giảm bộ nhớ). Đọc trước opt 16 (Muon, một hướng khác không dùng eigenbasis mà dùng orthogonalization trực tiếp). Cũng liên quan opt 45 ("Deconstructing What Makes a Good Optimizer for Language Models", cùng nhóm tác giả, được trích trong bài này như Zhao et al. 2024c).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Paper mở đầu bằng việc nhắc lại thành công thực nghiệm của Shampoo (opt 14): trong AlgoPerf (Dahl et al. 2023, một benchmark hiệu quả tối ưu), Shampoo (qua bản cài đặt phân tán DistributedShampoo của Shi et al. 2023) vượt mọi bài dự thi khác, bao gồm Adam, giảm 28% wall-clock time (theo công bố MLCommons 2024 mà paper trích dẫn); Shampoo cũng đã được dùng trong các lần huấn luyện quy mô lớn như Gemini-1.5 Flash (Gemini Team 2024). Tuy nhiên, Shampoo có nhược điểm: nhiều hyperparameter hơn Adam, và chi phí tính toán cao hơn do phải tính nghịch đảo/luỹ thừa ma trận của các preconditioner L_t, R_t định kỳ.

Câu hỏi nghiên cứu: có cách nào đơn giản hoá Shampoo, giảm chi phí tính toán, mà vẫn giữ (hoặc cải thiện) hiệu năng? Ý tưởng khởi điểm của paper là một quan sát lý thuyết: nếu dùng luỹ thừa $1/2$ (thay $1/4$ như bản gốc Shampoo, một thay đổi đã được Anil et al. 2020 khuyến nghị thực nghiệm và được Morwani et al. 2024 - cùng một phần tác giả với bài này - chứng minh là xấp xỉ Kronecker tối ưu của AdaGrad), Shampoo có thể được viết lại hoàn toàn tương đương với việc chạy Adafactor (opt 12, phiên bản rút gọn của Adam dùng second-moment hạng 1) trong **cơ sở trị riêng (eigenbasis)** của các ma trận L, R. Nói cách khác: phần "khó" của Shampoo (ma trận preconditioner L, R) chỉ đóng vai trò xác định một **phép xoay toạ độ**; sau khi xoay, phần "dễ" (preconditioning theo toạ độ, kiểu Adam/Adafactor) có thể tách riêng và cập nhật rẻ hơn nhiều, độc lập với tần suất xoay lại.

2. Ý tưởng chính

Shampoo nguyên bản (với luỹ thừa $1/2$, dùng kỳ vọng theo batch $\mathbb{E}_B[\cdot]$ thay tích luỹ theo thời gian, để đơn giản hoá phân tích lý thuyết):

$$L = \mathbb{E}_B[G_B G_B^T], \quad R = \mathbb{E}_B[G_B^T G_B], \quad W_t \leftarrow W_{t-1} - \eta\, L^{-1/2} G_t R^{-1/2} / \mathrm{Tr}(L)^{-1/2}.$$

Claim 1 của paper: thuật toán trên tương đương với thuật toán sau (được gọi là "Adafactor trong eigenspace của Shampoo"): tính các eigenvector Q_L, Q_R của L, R; xoay gradient $G'_t = Q_L^T G_t Q_R$; chạy Adafactor (lấy trung bình hàng/cột của G'^2 để tạo second-moment hạng 1 $\hat{V}_t$) trên G'_t trong không gian đã xoay; chia $G'_t / \sqrt{\hat{V}_t + \epsilon}$; xoay kết quả về lại không gian gốc bằng Q_L, Q_R.

Chứng minh (Mục 4.1, trích gọn): gọi $\lambda_1, \ldots, \lambda_m$ và $\mu_1, \ldots, \mu_n$ là trị riêng của L và R. Thuật toán Shampoo co giãn toạ độ (i, j) (trong cơ sở Q_L, Q_R) theo hệ số $(\lambda_i \mu_j / \sum_k \lambda_k)^{-1/2}$; thuật toán Adafactor-trong-eigenspace co giãn theo $(A_i C_j / \sum_k A_k)^{-1/2}$ với A_i, C_j là ước lượng hạng-1 của Adafactor trong cơ sở đã xoay. Paper chứng minh $A_i = \lambda_i$ (và tương tự $C_j = \mu_j$) bằng cách khai triển $A_i = \mathbb{E}_B[\|u_i^T G_B\|^2] = \mathbb{E}_B[u_i^T G_B G_B^T u_i] = \lambda_i$ (dùng u_i là eigenvector thứ i của L, và v_j của R tạo thành một cơ sở trực chuẩn nên $\sum_j (G'_B)_{ij}^2 = \|u_i^T G_B\|^2$). Vậy hai thuật toán co giãn từng toạ độ bằng **đúng** hệ số, nên tương đương.

Từ đây, paper nhận ra: thứ khiến Shampoo đắt không phải là bước preconditioning theo toạ độ (rẻ), mà là bước **tính lại eigenbasis** (Q_L, Q_R, qua phân rã trị riêng/SVD). Nếu tách hai việc này ra, ta có thể cập nhật eigenbasis **không thường xuyên** (ví dụ mỗi $f = 10$ bước) mà vẫn cập nhật ước lượng second-moment (giờ không cần là Adafactor hạng 1 nữa, có thể dùng Adam đầy đủ, tổng quát hơn) ở **mọi** bước, miễn là tính trong cơ sở xoay hiện tại (dù cơ sở đó đã "cũ" vài bước). Đây chính là SOAP: "Shampoo nhưng chạy Adam trong eigenbasis của nó, thay vì chỉ tính lại preconditioner theo lịch Shampoo".

3. Phương pháp chi tiết

3.1 Thuật toán SOAP (Algorithm 3)

Với mỗi layer có tham số $W \in \mathbb{R}^{m \times n}$, SOAP duy trì 4 ma trận: $L \in \mathbb{R}^{m \times m}$, $R \in \mathbb{R}^{n \times n}$ (và các eigenvector tương ứng Q_L, Q_R), cùng $M, V \in \mathbb{R}^{m \times n}$ (momentum và second-moment, giống Adam nhưng trong không gian đã xoay).

```
Mỗi bước t:
    G ← gradient tại W (trừ dấu, theo quy ước paper)
    G' ← Q_L^T G Q_R                      # xoay gradient vào eigenbasis
    M ← β1 M + (1−β1) G                   # momentum trong không gian GỐC
    M' ← Q_L^T M Q_R                      # xoay momentum vào eigenbasis
    # "chạy Adam" trên G', M' đã xoay:
    V ← β2 V + (1−β2) (G' ⊙ G')           # EMA bình phương theo toạ độ (như Adam)
    N' ← M' / sqrt(V̂ + ε)
    N ← Q_L N' Q_R^T                      # xoay ngược về không gian gốc
    W ← W − η N
    # cập nhật preconditioner (và, định kỳ, eigenbasis):
    L ← β2 L + (1−β2) G G^T
    R ← β2 R + (1−β2) G^T G
    nếu t chia hết cho f:
        Q_L ← Eigenvectors(L, Q_L_cũ)     # power iteration, xem Algorithm 4
        Q_R ← Eigenvectors(R, Q_R_cũ)
```

Điểm mấu chốt (khác Shampoo gốc): V (second-moment kiểu Adam) được cập nhật ở **mọi** bước t, dù Q_L, Q_R (eigenbasis) chỉ được cập nhật mỗi f bước. Ở Shampoo gốc, toàn bộ "tính thích ứng" (adaptivity) của learning rate chỉ xảy ra khi L, R (và luỹ thừa của chúng) được tính lại, nên nếu f lớn, Shampoo "đứng yên" về mặt thích ứng trong f bước; SOAP vẫn tiếp tục thích ứng mỗi bước nhờ V, chỉ riêng *hướng xoay* là cũ.

3.2 Tính eigenvector bằng power iteration (Algorithm 4)

Thay vì gọi `torch.linalg.eigh` (phân rã trị riêng đầy đủ, chuẩn) mỗi lần cần Q_L, Q_R, paper (theo Wang et al. 2024) dùng một bước của **power method**: $S \leftarrow PQ_{\text{cũ}}$ (nhân ma trận PSD P - chính là L hoặc R - với ước lượng eigenvector trước đó), rồi $Q_{\text{mới}} \leftarrow \text{QR}(S)$ (phân rã QR để trực chuẩn hoá lại). Vì chỉ làm **một** bước lặp (không lặp tới hội tụ), Q chỉ "trôi" dần theo mỗi lần gọi, dựa trên giả thiết preconditioner thay đổi chậm theo thời gian huấn luyện; paper dùng `torch.linalg.qr` thay `torch.linalg.eigh` vì nhanh hơn trong PyTorch (theo tài liệu PyTorch mà paper trích). Lần đầu tiên (chưa có Q cũ), paper dùng phân rã trị riêng chuẩn một lần.

3.3 Các chi tiết cài đặt khác

1. **Layer 1 chiều (bias, LayerNorm scale...)**: chạy AdamW tiêu chuẩn, không làm phép xoay (tránh phải giải bài toán eigenvector cho các layer này, điều mà cách cài đặt Shampoo chuẩn vẫn làm).
2. **Layer có chiều quá lớn** (ví dụ layer embedding đầu/cuối của transformer ngôn ngữ, vốn có kích thước vocabulary rất lớn): cố định ma trận xoay (Q_L hoặc Q_R) bằng ma trận đơn vị cho chiều đó (không xoay). Nếu cả hai chiều của một layer 2D đều cố định đơn vị, SOAP suy biến đúng thành AdamW (paper nêu rõ điều này như một tính chất "nhất quán" của thiết kế).
3. **Bias correction và weight decay**: bị bỏ qua trong pseudocode (Algorithm 3) "để đơn giản", nhưng paper nói rõ **có** dùng trong cài đặt thực tế, giống cách AdamW dùng.
4. **Biến thể một phía (one-sided, Mục 7.1)**: lấy cảm hứng từ GaLore (Zhao et al. 2024a), chỉ xoay theo chiều **nhỏ hơn** của ma trận ($\min(m, n)$), giữ chiều lớn hơn ở ma trận đơn vị (không xoay). Giảm cả bộ nhớ và chi phí tính, đổi lại hiệu năng giảm nhẹ (xem Mục 5 bên dưới).
5. **Biến thể "factorized" (dùng Adafactor thay Adam, Mục 7.2.1)**: sau khi xoay vào eigenbasis, dùng ước lượng second-moment hạng 1 (kiểu Adafactor) thay EMA đầy đủ theo toạ độ của Adam, giảm bộ nhớ thêm mn (không cần lưu V đầy đủ $m \times n$).

3.4 Chi phí bộ nhớ và tính toán (Mục 7.2, 7.3)

Với ma trận $m \times n$ ($m > n$), SOAP cần (ngoài trọng số/activation): $2m^2$ (cho L, Q_L) $+2n^2$ (cho R, Q_R) $+3mn$ (cho gradient, M, V) - **giống hệt** DistributedShampoo về bộ nhớ, trong khi AdamW chỉ cần $3mn$. Kết hợp biến thể Adafactor (bớt mn) và một phía (bớt từ $2m^2 + 2n^2$ xuống $2\min(m, n)^2$), bộ nhớ tổng giảm xuống $2\min(m, n)^2 + 2mn$, **ít hơn** $3mn$ của AdamW cho các hình dạng ma trận điển hình của transformer (ví dụ gần vuông hoặc $m \gg n$ hay $n \gg m$ với $\min(m, n)$ đủ nhỏ).

Chi phí tính mỗi bước: SOAP cần m^3 (cập nhật L) $+n^3$ (cập nhật R) $+2m^2n + 2mn^2$ (phép xoay và xoay ngược ở cả hai chiều), **nhiều hơn** Shampoo gốc ($m^3 + n^3 + m^2n + n^2m$, chỉ phải nhân preconditioner một lần mỗi bên, không phải xoay-rồi-xoay-ngược của một tensor đầy đủ theo cả hai chiều cùng lúc). Đây là đánh đổi: SOAP tốn hơn Shampoo *mỗi bước* nhưng tốn ít hơn hẳn khi tính đến chi phí tính lại eigenbasis (loại chi phí thứ hai, chiếm ưu thế khi f nhỏ), vì SOAP chịu được f lớn tốt hơn Shampoo (xem Mục 4).

(tự tính) Ví dụ số minh hoạ sự khác biệt bộ nhớ: với một layer $m = 4096, n = 1024$ (kích thước điển hình của một ma trận chiếu trong một mạng transformer cỡ vài trăm triệu đến vài tỷ tham số, dùng để minh hoạ, không phải số từ paper), SOAP đầy đủ cần $2(4096)^2 + 2(1024)^2 + 3(4096)(1024) = 2 \times 16{,}777{,}216 + 2 \times 1{,}048{,}576 + 3 \times 4{,}194{,}304 = 33{,}554{,}432 + 2{,}097{,}152 + 12{,}582{,}912 = 48{,}234{,}496$ số (ở fp32, $\times 4$ byte $\approx$ 193MB), so với AdamW $3 \times 4{,}194{,}304 = 12{,}582{,}912$ số ($\approx$ 50MB). Biến thể một phía (chỉ xoay theo chiều nhỏ $n = 1024$): $2(1024)^2 + 3(4096)(1024) = 2{,}097{,}152 + 12{,}582{,}912 = 14{,}680{,}064$ số ($\approx$ 59MB), vẫn hơn AdamW nhưng gần hơn nhiều so với bản đầy đủ. Số liệu này minh hoạ trực tiếp công thức Mục 7.2 của paper, áp cho một kích thước cụ thể do người ghi chú chọn.

4. Thí nghiệm và kết quả chính

Thiết lập (Appendix A, trích verbatim theo paper): huấn luyện ngôn ngữ trên C4, tokenizer T5, dùng codebase OLMo (Groeneveld et al. 2024), mô hình decoder-only transformer 3 kích thước: 210m, 360m, 660m (số tham số không tính embedding); width 1024/1024/1408, depth 12/24/24; MLP hidden = 4x width; activation GeLU; positional encoding RoPE; có QK layer norm; không học bias cho linear layer/LayerNorm; huấn luyện mixed precision bfloat16. Batch size mặc định $\approx$ 2 triệu token, số bước "chinchilla-optimal" ($\approx 20\times$ số tham số): 210m $\rightarrow$ 1600 bước (≈ 3.3 tỷ token), 360m $\rightarrow$ 3200 bước, 660m $\rightarrow$ 6400 bước. Hyperparameter mặc định: $\beta_1 = 0.95$ (tốt hơn 0.9 trong các lần dò ở mô hình 360m), weight decay decoupled 10^{-4}, z-loss 10^{-4}, $\epsilon = 10^{-8}$ (AdamW/SOAP/GaLore); cho Shampoo: luỹ thừa mặc định $-1/2.5$ (dựa trên quan sát sơ bộ gần với khuyến nghị $-1/2$ hoặc $-1/2.2$ của DistributedShampoo), $\epsilon_{\text{shampoo}} = 10^{-12}$, $\beta_{\text{shampoo}} = 0.95$.

Kết quả chính (Hình 1-3, batch 2 triệu token, tần suất preconditioning $f = 10$): so với AdamW, SOAP giảm $\geq 40\%$ số bước huấn luyện và $\geq 35\%$ wall-clock time để đạt cùng loss cuối; so với Shampoo, SOAP giảm thêm $\approx 20\%$ ở cả hai chỉ số (số bước và wall-clock). Phương pháp đo "hiệu quả" (Mục 5): vì dùng cosine decay, paper chạy SOAP ở 4 tỉ lệ dữ liệu khác nhau (0.5/0.625/0.75/0.875 tổng số bước), khớp một hàm luật-scaling $a + bN^{-\beta}$ qua loss cuối (N = số điểm huấn luyện), rồi dùng hàm này để tính chính xác điểm SOAP đạt cùng loss với AdamW/Shampoo đã chạy đủ.

Critical batch size (Mục 6.3): mô hình 360m, AdamW ở batch 256 nghìn token đạt loss 2.842 sau số bước "chinchilla-optimal" (dùng làm loss mục tiêu để so sánh). Khi tăng batch size từ 256 nghìn tới 2048 nghìn token, SOAP luôn cần ít bước hơn AdamW để đạt loss mục tiêu này, và lợi thế tăng theo batch size; SOAP bám sát đường "linear scaling lý tưởng" (gấp đôi batch thì giảm một nửa số bước) tốt hơn AdamW, tức SOAP có **critical batch size** lớn hơn AdamW. Ở batch nhỏ nhất (256 nghìn token), SOAP giảm 25% số bước so với AdamW và $\approx 10\%$ so với Shampoo; cải thiện wall-clock $\geq 15\%$ so với AdamW và $\approx 10\%$ so với Shampoo. Paper tự lưu ý đây là "phân tích sơ bộ cho batch nhỏ" và cách giữ tích (batch size × precondition frequency) = hằng số "có thể chưa phải tối ưu".

Mở rộng số token (Mục 6.4): với 100 lần số tham số (ví dụ 33.5 tỷ token cho mô hình 360m, thay vì 20x "chinchilla-optimal" chuẩn), SOAP vẫn duy trì lợi thế so với AdamW (Hình 5), cho thấy lợi ích không chỉ giới hạn ở các lần huấn luyện ngắn.

5. Phân tích / ablation

Ảnh hưởng tần suất cập nhật eigenbasis (Mục 6.2, Hình 1 phải): với mọi tần suất f từ 1 đến 100, cả SOAP và Shampoo đều vượt AdamW. Ở $f = 1$ (cập nhật mỗi bước), SOAP và Shampoo gần tương đương (khớp với Claim 1: khi $f = 1$, SOAP gần giống Shampoo-dùng-Adam-thay-Adafactor). Ở f cao hơn, hiệu năng cả hai giảm, nhưng **Shampoo giảm nhanh hơn SOAP rõ rệt** - đây là kết quả cốt lõi mà paper dùng để biện minh cho thiết kế SOAP: tách việc "cập nhật eigenbasis" (đắt, có thể làm hiếm) khỏi việc "cập nhật second-moment" (rẻ, nên làm mỗi bước) giúp SOAP bền hơn với f lớn.

One-sided (Mục 7.1, Hình 6): chỉ xoay theo chiều nhỏ hơn làm giảm hiệu năng nhẹ so với SOAP đầy đủ, nhưng vẫn ngang hoặc nhích hơn Shampoo, với chi phí tính/bộ nhớ thấp hơn. Paper để việc tìm biến thể vừa nhanh vừa không giảm hiệu năng cho công trình tương lai.

Factorized (Adafactor thay Adam, Mục 7.2.1, Hình 6): gây tăng loss "rất nhỏ" (theo mô tả của paper, không có bảng số cụ thể trong phần văn bản chính, chỉ Hình 6 minh hoạ trực quan); kết hợp cả factorized và one-sided vẫn vượt AdamW dù bộ nhớ ít hơn AdamW.

Chi phí tính theo tần suất (Hình 7 trái): overhead của SOAP so với AdamW giảm khi tần suất precondition tăng, nhưng tiến tới một "đường tiệm cận lớn hơn 0" (không về 0), do các phép nhân ma trận để cập nhật L, R và xoay/xoay-ngược gradient xảy ra ở **mọi** bước (không phụ thuộc f), hiện đang chạy ở fp32; paper gợi

ý giảm độ chính xác số (theo Wang et al. 2024) có thể hạ đường tiệm cận này.

So sánh cách tính eigenvector (Hình 7 phải): dùng power iteration + QR (Algorithm 4, rẻ hơn) cho hiệu năng "tương đương" với dùng phân rã trị riêng đầy đủ (`torch.linalg.eigh`) mỗi lần, trên toàn dải tần suất precondition đã thử.

So sánh với GaLore (Appendix B): paper tự thử GaLore (Zhao et al. 2024a, phương pháp giảm bộ nhớ Adam bằng chiếu gradient xuống không gian hạng thấp từ SVD của gradient hiện tại) cho mô hình 210m: GaLore vượt AdamW nhưng **kém hơn** Shampoo (loss tốt nhất của GaLore là 3.12 so với 3.10 của Shampoo tốt nhất); vì vậy paper không mở rộng so sánh GaLore lên các mô hình lớn hơn. Hai khác biệt chính giữa GaLore và SOAP mà paper nêu: (a) không gian chiếu của GaLore lấy từ SVD của gradient **hiện tại**, còn SOAP dùng trung bình trượt (EMA) của GG^T, G^TG qua thời gian; (b) GaLore giữ momentum **trong** không gian đã chiếu và không xoay lại khi không gian cập nhật, còn SOAP giữ momentum ở không gian gốc và xoay vào/ra mỗi bước.

6. Giới hạn

Paper có mục riêng tên **"9. Discussion and Limitations"** (trùng lặp gần như nguyên văn với mục 8 "Discussion and Future Work" ngay trước đó, một điểm lặp lại khá bất thường về cấu trúc bài mà người ghi chú thấy cần nêu, dù không ảnh hưởng nội dung khoa học). Các giới hạn paper tự nêu, gần nguyên văn:

- **Quy mô thí nghiệm nhỏ**: "chúng tôi thừa nhận nghiên cứu của mình tập trung ở quy mô tương đối nhỏ so với các LLM gần đây (Touvron et al. 2023, tức LLaMA) lớn hơn hai bậc độ lớn (hai orders of magnitude)". Mô hình lớn nhất thử nghiệm là 660m tham số, còn các LLM hiện đại ở quy mô hàng chục tỷ tham số.
- **Giả thuyết ngoại suy chưa được kiểm chứng**: paper "giả thuyết (hypothesize) rằng các kết quả về hiệu năng của SOAP sẽ tổng quát hoá (generalize) tới quy mô lớn hơn nhờ nền tảng lý thuyết của nó", lý do: SOAP tương đương chạy Adam trong không gian xoay, và Adam đã được chứng minh hiệu quả qua nhiều quy mô/tác vụ khác nhau. Nhưng paper tự nói rõ: "tuy nhiên, giả thuyết này vẫn cần được kiểm chứng (this hypothesis remains to be validated)" - đây là một giới hạn được paper phát biểu minh bạch, không phải suy luận của người ghi chú.
- **Chưa thử ở domain khác ngoài ngôn ngữ**: toàn bộ thí nghiệm là language modeling trên C4; paper nói muốn thử SOAP trên các domain khác như vision trong công trình tương lai, nhưng chưa có số liệu.
- **Chưa có cài đặt phân tán (distributed) tối ưu**: paper nói muốn cải thiện "better distributed implementation" trong tương lai; các con số hiệu quả hiện tại đo trên một GPU H100 duy nhất với gradient accumulation để đạt batch lớn, và paper tự lưu ý cách đo này "có thể trông như bất lợi cho AdamW" (vì overhead của Shampoo/SOAP có thể amortize qua nhiều GPU trong triển khai phân tán thực tế) nhưng để việc so sánh triển khai phân tán đầy đủ cho công trình tương lai.
- **Chi phí vẫn chạy ở fp32**: các phép nhân ma trận cập nhật L, R và các phép xoay hiện chạy fp32 vì `torch.linalg.qr` của PyTorch (thời điểm viết bài) không hỗ trợ độ chính xác thấp hơn; đây là một hạn chế về công cụ/framework hơn là về thuật toán, nhưng ảnh hưởng trực tiếp tới overhead đo được.
- **Discrepancy với Anil et al. (2020)**: paper tự nêu một điểm không khớp giữa công trình của mình và tài liệu trước: Anil et al. (2020), trong Appendix B, đã mô tả một thuật toán "về cơ bản tương đương SOAP cho layer 2D" nhưng không có thí nghiệm, và các tác giả đó "tuyên bố các thí nghiệm chưa công bố cho thấy không cải thiện gì so với Shampoo". Paper này suy đoán "sự khác biệt giữa hai phát hiện có thể do một số chi tiết cài đặt của SOAP" nhưng không xác định được chính xác chi tiết nào gây ra khác biệt - đây là một điểm chưa được giải thích triệt để mà paper tự thừa nhận.

7. Liên hệ với các bài khác

- **opt 14 (Shampoo)**: SOAP là mở rộng/diễn giải lại trực tiếp của Shampoo. Về công thức, Shampoo (opt 14) dùng $L_t = L_{t-1} + G_tG_t^T$ (tích luỹ **tổng**, không suy giảm) và luỹ thừa $-1/4$; SOAP (và biến

thể Shampoo dùng trong bài này) dùng EMA β_{shampoo} (suy giảm theo thời gian, giống Adam) và luỹ thừa $-1/2$ (hoặc $-1/2.5$ mặc định thực nghiệm của paper này) cho L, R - một sự khác biệt cụ thể về công thức giữa bản Shampoo "gốc" (opt 14) và bản Shampoo "thực hành hiện đại" mà SOAP so sánh trực tiếp.

- **opt 06/07 (Adam, AdamW)**: "lõi" thuật toán chạy bên trong eigenbasis của SOAP (bước 6-8 của Algorithm 3) chính là công thức Adam/AdamW nguyên bản (M, V, chia căn bậc hai), chỉ khác là áp dụng lên gradient/momentum đã được xoay G', M' thay vì G, M gốc. Nếu cố định $Q_L = I, Q_R = I$ (không xoay), SOAP suy biến đúng thành AdamW, điều paper tự nêu rõ (Mục 4.1, điểm 3).
- **opt 12 (Adafactor)**: dùng theo hai cách trong bài này: (a) là "nhân vật trung gian" trong chứng minh tương đương Shampoo-Adafactor-trong-eigenbasis (Mục 4.1); (b) là lựa chọn thay thế Adam trong biến thể "SOAP factorized" (Mục 7.2.1) để giảm bộ nhớ thêm mn, với cái giá hiệu năng giảm "rất nhỏ" theo quan sát thực nghiệm của paper.
- **opt 45 (Deconstructing What Makes a Good Optimizer for Language Models)**: cùng nhóm tác giả (Rosie Zhao, Depen Morwani, David Brandfonbrener, Nikhil Vyas, Sham Kakade - trùng 5/8 tác giả với bài SOAP này), được trích trong danh mục tham khảo của SOAP dưới tên "Zhao et al., 2024c". `45_2024_deconstructing_llm_optimizer.md` (đã xác nhận trong thư mục) tự nêu: các optimizer đường chéo (Adam, Adafactor, Lion, Signum, SGD) có hiệu năng tương đương nhau, và loại trừ các optimizer phi đường chéo như Shampoo khỏi so sánh thực nghiệm chính của họ. SOAP (bài hiện tại), dù cũng thuộc nhóm tác giả đó, lại cho thấy đúng optimizer phi đường chéo (Shampoo/SOAP) mới là nơi có cải thiện rõ rệt so với AdamW - hai bài, dù cùng nhóm tác giả, nhắm tới hai câu hỏi khác nhau (một hỏi "giữa các optimizer đường chéo có khác biệt gì" - không có khác biệt lớn; một hỏi "liệu rời khỏi đường chéo có lợi gì" - SOAP cho thấy có).
- **opt 11 (Lion)**: paper liệt kê Lion cùng Sophia, Adafactor như các "optimizer dùng diagonal preconditioning khác AdamW", và tự nhận định (Mục 3, Related Work) rằng các nghiên cứu trước (Kaddour et al. 2023, Zhao et al. 2024c = opt 45) cho thấy các optimizer này "có hiệu năng so sánh được với AdamW cho LLM pretraining nhưng không vượt qua nó", từ đó lập luận cần khám sát các preconditioner **phi đường chéo** (như Shampoo/SOAP) - chính là động cơ trực tiếp dẫn tới bài SOAP này.
- **opt 16 (Muon)**: theo tên và vị trí lộ trình (chưa đọc đầy đủ khi viết ghi chú này, chỉ dựa theo tiêu đề cố định trong titles.tsv, "Muon is Scalable for LLM Training"), đây là một hướng khác không dùng eigenbasis/Kronecker mà dùng orthogonalization của ma trận cập nhật; cả Muon và SOAP đều thuộc nhóm "optimizer phi đường chéo mới cho LLM pretraining quy mô vừa/lớn" nổi lên sau Shampoo, nên việc so sánh hai bài này (khi opt 16 được viết xong) sẽ là một liên hệ tự nhiên cho ghi chú đó.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là **suy luận** của người ghi chú, áp cho bài toán nén/huấn luyện tiếp (post-training) LightOnOCR-3-0.8B (VLM OCR lai GatedDeltaNet + full attention, $\approx$ 3600 token/trang, GPU $\leq$ 8GB, batch size 1). Paper không nhắc tới mô hình này hay VLM OCR.

- **Bộ nhớ của SOAP đầy đủ còn tốn hơn cả Shampoo (opt 14) ở một khía cạnh**: như paper tự nêu (Mục 7.2), SOAP cần $2m^2 + 2n^2 + 3mn$ so với $2m^2 + 2n^2 + mn$ (xấp xỉ) của DistributedShampoo thông thường (SOAP cần thêm lưu cả M và V dạng đầy đủ $m \times n$, trong khi Shampoo cơ bản không cần lưu V dạng ma trận đầy đủ theo kiểu Adam). Với ràng buộc 8GB/batch 1 của dự án, đây là một tín hiệu bất lợi tương tự Shampoo (đã phân tích ở `14_2018_shampoo.md`): chi phí $O(m^2 + n^2)$ cho mỗi layer 2D lớn (ví dụ các ma trận chiếu trong attention/MLP của một mô hình 0.8B) có thể cạnh tranh gay gắt với ngân sách 8GB.
- **Biến thể kết hợp (factorized + one-sided) là điểm khả thi nhất để mượn ý tưởng cho dự án**: paper tự tính biến thể này đạt bộ nhớ $2\min(m,n)^2 + 2mn$, **ít hơn** AdamW ($3mn$) cho hình dạng ma trận điển hình. (tự tính) Với một ma trận $m = 2048, n = 512$ (giả định quy mô phù hợp một phần của mô hình 0.8B, để minh hoạ, không phải số từ paper): biến thể kết hợp cần $2(512)^2 + 2(2048)(512) =$

$524{,}288 + 2{,}097{,}152 = 2{,}621{,}440$ số (≈ 10.5MB ở fp32), so với AdamW $3(2048)(512) = 3{,}145{,}728$ số (≈ 12.6MB); tức biến thể kết hợp này **tiết kiệm hơn** AdamW một chút cho hình dạng ma trận này, trái với cảm nhận ban đầu rằng "optimizer phi đường chéo luôn tốn bộ nhớ hơn". Đây là lý do khiến biến thể factorized+one-sided (không phải SOAP đầy đủ) đáng xem xét nhất cho ràng buộc 8GB, nếu muốn thử ý tưởng "preconditioning theo eigenbasis" cho dự án.

- **Tần suất cập nhật eigenbasis thấp (f lớn) là điểm phù hợp trực tiếp với "huấn luyện/fine-tune ngắn hơi, ít bước" của một dự án cá nhân trên 1 GPU**: kết quả Mục 6.2 (SOAP bền hơn Shampoo khi f tăng) gợi ý rằng nếu chỉ có ngân sách tính toán hạn chế để tính lại eigenvector (power iteration + QR, dù rẻ hơn eigh đầy đủ, vẫn tốn thời gian mỗi lần gọi), có thể đặt f khá lớn (ví dụ 50-100) mà không mất quá nhiều hiệu năng so với Shampoo nguyên bản, điều mà Shampoo (opt 14) không làm tốt bằng.
- **Batch size 1 là một hạn chế quan trọng cần lưu ý**: toàn bộ thí nghiệm SOAP dùng batch rất lớn (256 nghìn đến 2 triệu token); phần "critical batch size" (Mục 6.3) chính là nói về việc lợi thế của SOAP **tăng** theo batch size, và paper (giống Shampoo, Sophia đã đọc trước đó) tự trích dẫn quan sát từ công trình khác rằng "lợi ích hiệu quả của phương pháp bậc hai giảm ở batch size nhỏ" (Zhang et al. 2019; Ishikawa & Yokota 2024, trích trong Mục 6.3 của paper). Với batch size 1 của dự án, không có cơ sở nào trong paper để kỳ vọng SOAP mang lại lợi ích tương tự những con số 20-40% báo cáo ở batch hàng trăm nghìn/triệu token; đây là khoảng trống thực nghiệm lớn nhất khi áp bài này vào đúng ràng buộc của dự án.
- **Kết luận suy luận của người ghi chú**: SOAP (dù là cải tiến thực tế hơn Shampoo về mặt tần suất cập nhật) vẫn **ít phù hợp trực tiếp** để chạy "như-paper" trên GPU 8GB/batch 1, vì (a) chi phí bộ nhớ của bản đầy đủ còn cao hơn cả Shampoo ở khía cạnh lưu M, V dạng ma trận; (b) mọi bằng chứng thực nghiệm về lợi ích đều ở chế độ batch rất lớn, trái hẳn ràng buộc batch 1 của dự án. Ý tưởng khả dụng nhất để mang sang là biến thể "factorized + one-sided", vì nó là biến thể **duy nhất** trong bài này được paper tự tính là tiết kiệm bộ nhớ hơn AdamW, dù vẫn cần thử nghiệm thực tế ở batch nhỏ (không có trong paper) trước khi kết luận có lợi hay không cho dự án cụ thể này.

9. Thí nghiệm gợi ý

1. **Tái tạo Claim 1 (tương đương Shampoo-Adafactor-trong-eigenbasis) trên một layer toy.** Tạo ma trận gradient giả $G \in \mathbb{R}^{32\times 16}$ (vài batch ngẫu nhiên có cấu trúc tương quan), tính $L = \mathbb{E}[GG^T], R = \mathbb{E}[G^TG]$, lấy eigenvector Q_L, Q_R bằng `torch.linalg.eigh`; so sánh (a) áp trực tiếp $L^{-1/2}GR^{-1/2}$ và (b) xoay $G' = Q_L^TGQ_R$, chia theo Adafactor hạng-1 trong không gian xoay, xoay ngược. Đo sai số tuyệt đối giữa hai kết quả. Kỳ vọng: sai số rất nhỏ (xấp xỉ 0, chỉ còn lỗi số học dấu phẩy động), xác nhận Claim 1 trên dữ liệu thực tế nhỏ. Diễn giải: nếu sai số lớn, kiểm tra lại việc chuẩn hoá $\mathrm{Tr}(L)$ ở mẫu số (Algorithm 1, bước 5-6) có được áp đúng ở cả hai phía hay không.
2. **Đo ảnh hưởng của tần suất cập nhật eigenbasis (f) trên một MLP nhỏ, mô phỏng Hình 1 (phải) ở quy mô GPU 8GB.** Huấn luyện một MLP 3 lớp ($256 \to 512 \to 256 \to 10$) trên một bài phân loại toy (ví dụ dữ liệu Gaussian tổng hợp nhiều cụm), chạy SOAP (tự cài đặt Algorithm 3, dùng `torch.linalg.eigh` cho đơn giản thay power iteration) và Shampoo (Algorithm 1 của opt 14) với $f \in \{1, 10, 50, 100\}$, đo loss cuối sau cùng số bước. Kỳ vọng: cả hai giảm hiệu năng khi f tăng, nhưng SOAP giảm chậm hơn Shampoo rõ rệt, giống xu hướng Hình 1 phải của paper. Diễn giải: nếu không thấy khác biệt rõ ở quy mô toy này, có thể vì bài toán quá đơn giản để lộ sự khác biệt giữa "thích ứng mỗi bước" và "thích ứng mỗi f bước" - thử tăng độ khó bài toán (nhiều cụm hơn, nhiễu hơn) để khuếch đại hiệu ứng.
3. **Đo bộ nhớ thực tế của 4 biến thể SOAP (đầy đủ, one-sided, factorized, kết hợp) trên một `nn.Linear` lớn, trong ngân sách 8GB.** Với `nn.Linear(2048, 512)` (hoặc kích thước tương tự một layer chiếu của mô hình 0.8B), cài đặt cả 4 biến thể bộ nhớ (theo công thức Mục 7.2), đo `torch.cuda.max_memory_allocated()` thực tế khi chạy vài bước optimizer, so với con số lý thuyết đã tính ở Mục 8 (tự tính, ≈ 10.5MB cho biến thể kết hợp ở kích thước 2048×512). Kỳ vọng: số đo thực tế gần khớp công thức lý thuyết (sai lệch do buffer tạm của QR/eigh). Diễn giải: nếu số đo thực tế lớn hơn hẳn lý thuyết, có thể do framework giữ bản sao trung gian trong phép xoay (G', M') không được tính trong

công thức ”beyond weights and activations” của paper - một chi phí thực tế mà paper không tính vào Mục 7.2.

4. **Kiểm tra hành vi SOAP ở batch size rất nhỏ (batch 1-8), khác hẳn chế độ batch lớn mà paper thử nghiệm.** Trên bài toán MLP toy ở thí nghiệm 2, chạy SOAP và AdamW với batch size $\in \{1, 4, 16, 256\}$ (giữ tổng số example đã thấy không đổi để so sánh công bằng), đo loss theo số example đã xử lý (không phải theo bước optimizer, vì batch khác nhau). Kỳ vọng, dựa trên cảnh báo của chính paper (trích dẫn Zhang et al. 2019; Ishikawa & Yokota 2024 ở Mục 6.3) rằng lợi ích của phương pháp bậc hai giảm ở batch nhỏ: khoảng cách giữa SOAP và AdamW sẽ hẹp lại đáng kể, có thể biến mất hoàn toàn, ở batch 1-4 so với batch 256. Diễn giải: đây là thí nghiệm trực tiếp nhất kiểm tra giả thuyết (suy luận ở Mục 8) rằng SOAP ”ít phù hợp” với đúng ràng buộc batch 1 của dự án; nếu thực tế khoảng cách không hẹp lại nhiều, cần xem lại suy luận đó.
5. **So sánh power iteration (Algorithm 4) với eigh đầy đủ về thời gian chạy trên GPU 8GB, với kích thước ma trận tăng dần.** Với các ma trận PSD ngẫu nhiên kích thước 256×256 tới 4096×4096, đo thời gian của một lần gọi `torch.linalg.eigh` so với một bước power iteration (`P@Q` rồi `torch.linalg.qr`), lặp lại nhiều lần để lấy trung bình. Kỳ vọng: power iteration nhanh hơn eigh rõ rệt, đặc biệt ở ma trận lớn, khớp nhận định ”`torch.linalg.qr` nhanh hơn `torch.linalg.eigh`” mà paper trích từ tài liệu PyTorch (Mục 7.3). Diễn giải: nếu chênh lệch tốc độ nhỏ hơn nhiều so với kỳ vọng trên GPU cụ thể của dự án (so với GPU H100 dùng trong paper), kết luận ”power iteration luôn đáng dùng” của paper có thể không chuyển thẳng sang phần cứng 8GB khác (ví dụ GPU tiêu dùng), cần đo lại trước khi áp dụng.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Shampoo luỹ thừa $1/2$ (idealized)	$W_t \leftarrow W_{t-1} - \eta L^{-1/2} G_t R^{-1/2} / \mathrm{Tr}(L)^{-1/2}$	Dạng Shampoo dùng để chứng minh tương đương với Adafactor-trong-eigenbasis
Tương đương (Claim 1)	co giãn toạ độ (i,j): Shampoo dùng $(\lambda_i \mu_j / \sum \lambda_k)^{-1/2}$, Adafactor-trong-eigenbasis dùng $(A_i C_j / \sum A_k)^{-1/2}$, và $A_i = \lambda_i$	Hai thuật toán co giãn đúng cùng hệ số theo từng toạ độ trong eigenbasis
Algorithm 3 (SOAP, cốt lõi)	xoay $G' = Q_L^T G Q_R$; chạy Adam trên G', M'; xoay ngược $N = Q_L N' Q_R^T$	Tách ”thích ứng mỗi bước” (rẻ) khỏi ”cập nhật eigenbasis” (đắt, làm mỗi f bước)
Eigenvectors qua power iteration (Algorithm 4)	$S \leftarrow P Q_{\text{cũ}}$; $Q_{\text{mới}} \leftarrow \mathrm{QR}(S)$	Một bước lặp rẻ hơn phân rã trị riêng đầy đủ, dựa trên preconditioner đổi chậm
Bộ nhớ SOAP đầy đủ ($m > n$)	$2m^2 + 2n^2 + 3mn$	Bằng DistributedShampoo, hơn AdamW ($3mn$)
Bộ nhớ biến thể kết hợp (factorized + one-sided)	$2\min(m,n)^2 + 2mn$	Có thể ít hơn AdamW cho hình dạng ma trận điển hình
Overhead mỗi bước SOAP	$m^3 + n^3 + 2m^2 n + 2mn^2$	Hơn Shampoo ($m^3 + n^3 + m^2 n + n^2 m$) vì phải xoay/xoay-ngược cả hai chiều
Hiệu quả thực nghiệm chính	$\geq 40\%$ giảm số bước, $\geq 35\%$ giảm wall-clock so AdamW; $\approx 20\%$ so Shampoo (batch 2 triệu token, $f = 10$)	Kết quả đầu mục của paper trên mô hình 360m/660m

11. Kết luận ngắn

SOAP xuất phát từ một quan sát lý thuyết chặt: Shampoo (với luỹ thừa $1/2$) tương đương với chạy Adafactor trong eigenbasis của preconditioner Shampoo (Claim 1, Mục 4.1). Từ đó, paper tách ”cập nhật eigenbasis” (đắt) khỏi ”cập nhật second-moment” (rẻ, nên chạy mỗi bước theo kiểu Adam đầy đủ, không chỉ Adafactor hạng 1), cho ra thuật toán SOAP: chạy AdamW trong một không gian xoay được cập nhật định kỳ mỗi f bước. Thực nghiệm trên mô hình ngôn ngữ 360m/660m (huấn luyện trên C4) cho thấy SOAP vượt cả AdamW ($\geq 40\%$ giảm số bước, $\geq 35\%$ giảm wall-clock) và Shampoo ($\approx 20\%$ ở cả hai chỉ số), đặc biệt bền hơn Shampoo khi tần suất cập nhật eigenbasis thấp, và cải thiện critical batch size so với AdamW. Paper tự nêu giới hạn rõ ràng nhất: quy mô thí nghiệm (tới 660m tham số) còn nhỏ hơn hai bậc độ lớn so với LLM hiện đại, và giả thuyết ”kết quả sẽ tổng quát hoá lên quy mô lớn hơn” chưa được kiểm chứng. Đối với dự án nén/huấn luyện LightOnOCR-3-0.8B trong 8GB/batch 1, SOAP bản đầy đủ tốn bộ nhớ hơn cả Shampoo (opt 14) ở khía cạnh lưu M, V dạng ma trận đầy đủ, và toàn bộ bằng chứng lợi ích của paper đến từ chế độ batch rất lớn (256 nghìn đến 2 triệu token), trái với batch 1 của dự án; biến thể ”factorized + one-sided” là lựa chọn khả thi nhất về bộ nhớ nếu muốn thử nghiệm ý tưởng này, nhưng cần kiểm chứng riêng ở batch nhỏ vì paper không có số liệu ở chế độ đó.

Nguồn gốc: https://arxiv.org/abs/2409.11321

Bài 16

Muon is Scalable for LLM Training (2025)

Tác giả: Jingyuan Liu, Jianlin Su, Xingcheng Yao (UCLA), Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, Zhilin Yang (còn lại: Moonshot AI).
Venue / arXiv: arXiv:2502.16982 (v1, 24/02/2025); trang đầu PDF ghi ”Technical Report” (không phải bài hội nghị/tạp chí có peer review, mà là báo cáo kỹ thuật mở của Moonshot AI).
Nguồn: https://arxiv.org/abs/2502.16982
Chủ đề: mở rộng optimizer Muon (K. Jordan et al. 2024, ban đầu chỉ là một blog post, cho kết quả tốt ở quy mô nhỏ) lên quy mô huấn luyện LLM lớn (hàng tỷ tham số, hàng nghìn tỷ token): xác định và giải quyết hai vấn đề cản trở việc mở rộng quy mô (thiếu weight decay, và update RMS không đồng nhất giữa các ma trận có hình dạng khác nhau), đề xuất một cài đặt phân tán (Distributed Muon, kiểu ZeRO-1), và huấn luyện mô hình MoE ”Moonlight” (3B/16B tham số) với 5.7 nghìn tỷ token để kiểm chứng.
Vị trí trong lộ trình: đọc sau opt 14 (Shampoo) và opt 15 (SOAP, cả hai dùng preconditioning kiểu eigenbasis/Kronecker), vì Muon là một hướng khác (orthogonalization trực tiếp của update, không qua eigenbasis/Kronecker) cùng thuộc nhóm ”optimizer phi đường chéo mới cho LLM pretraining” nổi lên gần đây. Đọc cùng opt 48 (”Old Optimizer, New Norm: An Anthology”, khung lý thuyết ”steepest descent dưới một norm” mà bài này trích dẫn trực tiếp để giải thích Muon) và opt 36 (scaling exponents/parameterization cho optimizer, được `36_2024_scaling_exponents_parameterizations_optimizers.md` tự nhận là chưa phân tích Muon, một câu hỏi mở).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Optimizer Muon (K. Jordan et al. 2024, công bố dưới dạng một bài blog, không phải paper chính thức) cập nhật các tham số dạng **ma trận** bằng gradient momentum đã được **trực giao hoá (orthogonalized)** qua phép lặp Newton-Schulz, và cho kết quả mạnh ở quy mô nhỏ (huấn luyện mô hình ngôn ngữ nhỏ, nổi tiếng qua các kỷ lục tốc độ huấn luyện nanoGPT). Tuy nhiên, như chính blog gốc đã thừa nhận (paper này trích lại), ba câu hỏi còn mở: (1) làm sao mở rộng optimizer dựa trên orthogonalization ma trận lên mô hình hàng tỷ tham số huấn luyện với hàng nghìn tỷ token; (2) làm sao tính orthogonalization gần đúng trong bối cảnh phân tán (nhiều GPU); (3) optimizer này có tổng quát hoá tốt qua các giai đoạn huấn luyện khác nhau (pretraining và supervised finetuning/SFT) không.

Paper này (của nhóm Moonshot AI, tác giả khác với nhóm đề xuất Muon gốc) là một báo cáo kỹ thuật giải quyết trực tiếp ba câu hỏi trên, đồng thời công bố mô hình MoE ”Moonlight” (3B tham số được kích hoạt/16B tổng, 5.7 nghìn tỷ token) làm minh chứng thực nghiệm ở quy mô lớn.

2. Ý tưởng chính

Công thức Muon gốc (Eq. 1 của paper, trích từ K. Jordan et al. 2024): với trọng số W_{t-1}, momentum μ, learning rate η_t, hàm mất mát L_t:

$$M_t = \mu M_{t-1} + \nabla L_t(W_{t-1}), \qquad O_t = \text{NewtonSchulz}(M_t), \qquad W_t = W_{t-1} - \eta_t O_t.$$

Newton-Schulz là một phép lặp xấp xỉ $(M_t M_t^T)^{-1/2} M_t$: nếu $M_t = U\Sigma V^T$ (phân rã SVD), thì

$(M_t M_t^T)^{-1/2} M_t = UV^T$, tức **trực giao hoá** M_t (giữ hướng của các vector singular nhưng đặt mọi trị singular về 1). Trực giác mà paper nêu: trực giao hoá đảm bảo ma trận cập nhật "isomorphic" (đẳng cấu, mọi hướng được cập nhật với cùng độ lớn), tránh việc trọng số chỉ học theo một vài hướng chiếm ưu thế (dominant directions) trong gradient.

Diễn giải theo "steepest descent dưới ràng buộc norm" (trích Bernstein et al. 2024, tức paper đã đọc ở opt 48 "Old Optimizer, New Norm"): paper này diễn giải sự khác biệt giữa Muon và Adam như sự khác biệt về **norm ràng buộc** trong bài toán steepest descent: Adam là steepest descent dưới một norm được điều chỉnh động từ "Max-of-Max norm", còn Muon cho một ràng buộc norm nằm trong một khoảng tĩnh của **Schatten-p norm** với p lớn; khi công thức (1) được tính chính xác, ràng buộc norm của Muon chính là **spectral norm** (norm toán tử $\ell_2 \to \ell_2$). Vì trọng số mạng neural đóng vai trò toán tử (operator) trên không gian đầu vào/ẩn, mà các không gian này thường (xét cục bộ) là không gian Euclid, paper lập luận ràng buộc dạng toán tử-norm (spectral norm) "hợp lý hơn" so với ràng buộc của Adam.

Từ nền này, hai cải tiến chính để mở rộng quy mô Muon:

1. **Thêm weight decay** vào công thức cập nhật (Muon gốc không có weight decay).
2. **Điều chỉnh độ lớn cập nhật theo hình dạng tham số**: Muon gốc có update RMS phụ thuộc hình dạng ma trận (Lemma 1 của paper, xem Mục 3), không đồng nhất như Adam; paper đề xuất co giãn lại để mọi ma trận có cùng update RMS, và để RMS đó khớp với RMS điển hình của AdamW, cho phép dùng lại trực tiếp learning rate/weight decay đã tinh chỉnh cho AdamW.

3. Phương pháp chi tiết

3.1 Phép lặp Newton-Schulz (Eq. 2)

$$X_0 = \frac{M_t}{\|M_t\|_F}, \qquad X_k = aX_{k-1} + b(X_{k-1}X_{k-1}^T)X_{k-1} + c(X_{k-1}X_{k-1}^T)^2 X_{k-1},$$

với hệ số $a = 3.4445$, $b = -4.7750$, $c = 2.0315$ (giữ nguyên từ thiết kế gốc của K. Jordan et al. 2024), chọn sao cho đa thức $f(x) = ax + bx^3 + cx^5$ có điểm bất động (fixed point) gần 1, giúp quá trình lặp hội tụ nhanh hơn với các trị singular ban đầu nhỏ. Paper này **giữ nguyên** các hệ số này, không tinh chỉnh lại. Trong thực hành, paper dùng số bước lặp $N = 5$ (không phải $N = 10$: quan sát thực nghiệm cho thấy $N = 10$ cho kết quả trực giao hoá chính xác hơn nhưng **không** cải thiện hiệu năng huấn luyện cuối cùng, nên chọn $N = 5$ để tiết kiệm chi phí tính). Paper cũng dùng biến thể Nesterov-style: đưa $\mu M_t + \nabla L_t(W_{t-1})$ (không phải M_t thuần) vào phép lặp Newton-Schulz, theo đúng cách K. Jordan et al. (2024) làm.

3.2 Thêm weight decay (Eq. 3)

$$W_t = W_{t-1} - \eta_t(O_t + \lambda W_{t-1}).$$

Lý do: paper quan sát rằng khi huấn luyện mô hình lớn hơn với nhiều token hơn, nếu không có weight decay, RMS của cả trọng số và output của layer tiếp tục tăng, vượt khỏi dải chính xác cao của bf16, có thể làm hại hiệu năng mô hình (độ chính xác số bị mất khi giá trị quá lớn trong định dạng bf16). Thí nghiệm trên mô hình 800M tham số, huấn luyện 100 tỷ token ($\approx 5\times$ số token "tối ưu" theo scaling law dùng trong bài): Muon không weight decay hội tụ nhanh hơn ban đầu, nhưng một số trọng số tăng quá lớn theo thời gian; Muon có weight decay vượt cả AdamW và Muon-không-weight-decay trong chế độ huấn luyện "quá mức" (over-train).

3.3 Update RMS không đồng nhất theo hình dạng (Lemma 1)

Lemma 1 (paper): với ma trận tham số hạng đầy đủ (full-rank) hình dạng $[A, B]$, update RMS lý thuyết của Muon là $1/\sqrt{\max(A, B)}$.

Chứng minh (Phụ lục A, tóm gọn): xét $X = U_{[:,:r]} V_{[:r,:]}$ (dạng của update Muon sau trực giao hoá hoàn hảo, với $U \in \mathbb{R}^{n\times n}, V \in \mathbb{R}^{m\times m}$ trực giao, $n \geq m \geq r$). Khai triển $\text{RMS}(X)^2 = \frac{1}{mn}\sum_{i,j}(\sum_{k=1}^{r} U_{i,k}V_{k,j})^2$;

dùng tính trực giao của U, V (các cột/hàng có norm 1, trực chuẩn với nhau) để đơn giản hoá tổng chéo, kết quả cuối $\text{RMS}(X)^2 = r/(mn)$. Với ma trận hạng đầy đủ $r = \min(A, B) = m$ (giả sử $n \geq m$), $\text{RMS}(X) = \sqrt{m/(mn)} = 1/\sqrt{n} = 1/\sqrt{\max(A, B)}$.

Hệ quả: nếu $\max(A, B)$ quá lớn (ví dụ ma trận MLP dày), update quá nhỏ, hạn chế khả năng biểu diễn (representational capacity) của mô hình; nếu $\max(A, B)$ quá nhỏ (ví dụ coi mỗi KV head trong GQA/MLA là một tham số riêng), update quá lớn, gây mất ổn định huấn luyện. Đây khác hẳn Adam/AdamW, vốn có update RMS lý thuyết luôn xoay quanh $1/3$ (thực tế thường nhỏ hơn do $\beta_1 < \beta_2$ và $\epsilon > 0$), **không phụ thuộc** hình dạng tham số.

3.4 Cân chỉnh update RMS đồng nhất (Eq. 4) và khớp với AdamW

Để cân chỉnh đồng nhất giữa các ma trận: co giãn update của mỗi ma trận theo $\sqrt{\max(A, B)}$ để triệt tiêu hiệu ứng Lemma 1. Để khớp luôn với AdamW (quan sát thực nghiệm: update RMS của AdamW thường trong khoảng 0.2-0.4):

$$W_t = W_{t-1} - \eta_t \Big(0.2 \cdot O_t \cdot \sqrt{\max(A, B)} + \lambda W_{t-1} \Big).$$

Lợi ích trực tiếp: vì update RMS của Muon giờ khớp AdamW, paper có thể **dùng lại trực tiếp** learning rate và weight decay đã tinh chỉnh cho baseline AdamW, không cần dò lại riêng cho Muon - một điểm quan trọng giúp Muon "hoạt động ngay, không cần tinh chỉnh hyperparameter" (out-of-the-box), đúng như tuyên bố trong abstract.

3.5 Distributed Muon (Algorithm 1, kiểu ZeRO-1)

ZeRO-1 (Rajbhandari et al. 2020) phân chia (partition) các trạng thái optimizer tốn bộ nhớ (master weight, momentum) ra toàn cluster, hiệu quả với AdamW vì AdamW tính update theo kiểu element-wise (không cần toàn bộ ma trận cùng lúc). Muon thì cần **toàn bộ ma trận gradient** để tính Newton-Schulz, nên ZeRO-1 nguyên bản không áp dụng trực tiếp được. Giải pháp (Algorithm 1): mỗi thiết bị reduce-scatter gradient G trên nhóm data-parallel (DP) để có gradient cục bộ đúng; áp momentum cục bộ; **gather** lại gradient đã có momentum thành ma trận đầy đủ trên mỗi thiết bị; tính Newton-Schulz trên ma trận đầy đủ đó; chỉ giữ lại phần tương ứng với tham số cục bộ của thiết bị đó (bỏ phần còn lại); all-gather tham số đã cập nhật.

So sánh với Distributed AdamW (ZeRO-1 kiểu cũ):

- **Bộ nhớ**: Muon chỉ cần một buffer momentum (so với hai của AdamW: m và v), nên bộ nhớ thêm của Muon chỉ bằng **một nửa** so với AdamW.
- **Giao tiếp (communication)**: Distributed Muon cần thêm bước DP-gather (chỉ cho tham số cục bộ, không phải toàn mô hình); vì các bước Newton-Schulz chạy ở bf16 (giảm thêm 50% khối lượng giao tiếp so với fp32), tổng khối lượng giao tiếp của Distributed Muon nằm trong khoảng $(1, 1.25]$ lần so với Distributed AdamW (cận trên tính từ: Distributed Muon $= 4$ (fp32 reduce-scatter G) $+2$ (bf16 gather Muon) $+4$ (fp32 all-gather P) $= 10$, so với Distributed AdamW $= 4 + 4 = 8$, tỉ lệ $10/8 = 1.25$; trong thực hành với nhiều nhóm DP, chi phí thực tế gần cận dưới 1 hơn).
- **Độ trễ (latency)**: lớn hơn AdamW do thêm giao tiếp và các bước Newton-Schulz, nhưng không đáng kể vì (a) chỉ cần khoảng 5 bước Newton-Schulz; (b) độ trễ do optimizer gây ra thường chỉ chiếm 1-3% tổng thời gian forward-backward của mô hình; có thể giảm thêm bằng cách chồng lấp (overlap) các bước gather/compute.

4. Thí nghiệm và kết quả chính

Scaling law (Mục 3.2): so sánh Muon với một baseline AdamW đã được dò hyperparameter kỹ (grid search đầy đủ, Phụ lục B), trên 5 kích thước mô hình dense kiến trúc Llama: 399M, 545M, 822M, 1.1B, 1.5B tham số (không tính embedding), độ rộng (hidden) 1536-2560, độ sâu 12-20 lớp, số token huấn luyện 8.92 tỷ đến 38.91

tỷ (tỉ lệ compute-optimal theo Kaplan et al. 2020). Vì update RMS của Muon đã khớp AdamW, paper dùng lại trực tiếp learning rate tối ưu của AdamW cho Muon, không dò riêng. Kết quả khớp đường luật scaling: $\text{loss}_{\text{Muon}} = 2.506 \times C^{-0.052}$ so với $\text{loss}_{\text{AdamW}} = 2.608 \times C^{-0.054}$ (C là compute). Theo các đường cong này, Muon chỉ cần $\approx 52\%$ FLOPs huấn luyện của AdamW để đạt cùng hiệu năng ở chế độ compute-optimal (tương đương "khoảng gấp đôi hiệu quả tính toán" như abstract nêu).

Huấn luyện Moonlight (Mục 3.3): kiến trúc giống deepseek-v3-small (MoE), 2.24B tham số kích hoạt/15.29B tổng (tính cả embedding: 3B kích hoạt/16B tổng, khớp tên "Moonlight 3B/16B" trong abstract), huấn luyện 5.7 nghìn tỷ token, độ dài ngữ cảnh tối đa 8K, qua 3 giai đoạn: (1) 0-33 tỷ token, LR tăng tuyến tính tới 4.2×10^{-4} trong 2 nghìn bước, batch 2048 ví dụ; (2) 33 tỷ-5.2 nghìn tỷ token, LR giảm cosine từ 4.2×10^{-4} xuống 4.2×10^{-5}, batch giữ 2048 tới 200 tỷ token rồi tăng gấp đôi lên 4096; (3) 5.2-5.7 nghìn tỷ token (giai đoạn "cooldown"), LR tăng lên 10^{-4} trong 100 bước rồi giảm tuyến tính về 0 trong 500 tỷ token, batch 4096, dùng dữ liệu chất lượng cao nhất (toán, code, reasoning). Weight decay 0.1 xuyên suốt.

So sánh ở checkpoint 1.2 nghìn tỷ token (Bảng 4), giữa DSV3-Small (AdamW, 1.33T token), Moonlight-A (cùng thiết lập Moonlight nhưng dùng AdamW), và Moonlight (Muon), cùng 2.24B tham số kích hoạt:

Benchmark	DSV3-Small	Moonlight-A (AdamW)	Moonlight (Muon)
MMLU	53.3	60.2	60.4
BBH	41.4	45.3	43.2
HumanEval	26.8	29.3	37.2
MBPP	36.8	49.2	52.9
GSM8K	31.4	43.8	45.0
MATH	10.7	16.1	19.8

Moonlight (Muon) vượt Moonlight-A (AdamW) trên hầu hết benchmark, đặc biệt mạnh ở code (HumanEval $+7.9$, MBPP $+3.7$ điểm) và toán (MATH $+3.7$ điểm), nhưng **thua** trên BBH (43.2 so với 45.3, tức -2.1 điểm) - một kết quả không đồng nhất mà bản thân bảng số liệu paper đưa ra (không phải suy luận của người ghi chú), paper không bàn riêng về điểm BBH giảm này, chỉ nói chung "Muon đặc biệt mạnh ở các tác vụ liên quan Toán và Code, khuyến khích cộng đồng tìm hiểu thêm hiện tượng này".

So sánh ở checkpoint đầy đủ 5.7 nghìn tỷ token (Bảng 5), Moonlight so với Llama3.2-3B (AdamW, 9T token), Qwen2.5-3B (optimizer không công bố, 18T token), DSV2-Lite (AdamW, 5.7T token): Moonlight dẫn đầu gần như mọi benchmark (MMLU 70.0 so với $54.7/65.6/58.3$; HumanEval 48.1 so với $28.0/42.1/29.9$; MATH 45.3 so với $8.5/42.6/17.1$), dù dùng ít token hơn Qwen2.5-3B (18T) và ít tham số kích hoạt hơn Llama3.2-3B về danh nghĩa tham số dense; ngoại lệ là GSM8K, Moonlight (77.4) thấp hơn Qwen2.5-3B (79.1) nhưng vẫn vượt Llama3.2-3B và DSV2-Lite rõ rệt. Paper kết luận Moonlight nằm trên đường Pareto frontier hiệu năng-so-với-FLOPs huấn luyện (Hình 1b).

Phân tích phổ singular value (Mục 3.4): định nghĩa SVD entropy của một ma trận trọng số có trị singular $\sigma = (\sigma_1, \ldots, \sigma_n)$:

$$H(\sigma) = -\frac{1}{\log n} \sum_{i=1}^{n} \frac{\sigma_i^2}{\sum_j \sigma_j^2} \log \frac{\sigma_i^2}{\sum_j \sigma_j^2}.$$

(tự tính) Ví dụ minh hoạ công thức này trên trường hợp cực đoan, $n = 2$: nếu $\sigma = (1, 1)$ (hai trị singular bằng nhau, "trải đều" hoàn toàn), xác suất chuẩn hoá $p_1 = p_2 = \sigma_i^2 / \sum \sigma_j^2 = 0.5$; $H = -\frac{1}{\log 2}(0.5 \log 0.5 + 0.5 \log 0.5) = -\frac{1}{\log 2} \log 0.5 = -\frac{-\log 2}{\log 2} = 1$ (entropy tối đa, bất kể dùng log cơ số nào vì đã chuẩn hoá bởi $\log n$). Ngược lại, nếu $\sigma = (1, 0)$ (toàn bộ "năng lượng" tập trung vào một hướng duy nhất), $p_1 = 1, p_2 = 0$, và theo quy ước $0 \log 0 = 0$: $H = -\frac{1}{\log 2}(1 \log 1 + 0) = 0$ (entropy tối thiểu). Vậy $H(\sigma) \in [0, 1]$ đo mức độ

"trải đều" của phổ trị singular: gần 1 nghĩa là update gần như đẳng hướng (isomorphic, đúng mục tiêu thiết kế của Muon), gần 0 nghĩa là update tập trung vào một vài hướng chiếm ưu thế (điều Muon muốn tránh).

Kết quả thực nghiệm: trên mọi checkpoint huấn luyện và cả 6 nhóm ma trận trọng số (AttnQO, AttnKV, Experts, SharedExperts, Router, Dense), SVD entropy của Muon **luôn cao hơn** AdamW; sự khác biệt rõ nhất ở ma trận router (chọn expert trong MoE), gợi ý mô hình MoE có thể hưởng lợi nhiều hơn từ Muon. Ở checkpoint 1.2 nghìn tỷ token, hơn 90% ma trận trọng số có SVD entropy cao hơn khi dùng Muon so với AdamW.

SFT (Mục 3.5): finetune Moonlight@1.2T và Moonlight-A@1.2T (2 epoch, tulu-3-sft-mixture, 4K token/chuỗi) bằng cả Muon và AdamW cho mỗi mô hình. Kết quả (Bảng 6): tổ hợp Muon-pretrain + Muon-SFT tốt nhất (MMLU 55.7, HumanEval 57.3, GSM8K 68.0), vượt ba tổ hợp còn lại; nhưng khi optimizer SFT **khác** optimizer pretraining, Muon không cho lợi thế rõ rệt so với AdamW riêng ở giai đoạn SFT. Thí nghiệm thứ hai (Bảng 7): SFT mô hình công khai Qwen2.5-7B (đã pretrain bằng AdamW) bằng Adam-SFT so với Muon-SFT: Adam-SFT nhích hơn Muon-SFT trên mọi benchmark đo được (MMLU 71.4 vs 70.8, HumanEval 79.3 vs 77.4, GSM8K 89.8 vs 85.8) - paper kết luận Muon hiệu quả hơn khi dùng **từ giai đoạn pretraining**, không phải chỉ áp ở SFT sau một mô hình đã pretrain bằng optimizer khác.

5. Phân tích / ablation

Bảng 1 (so sánh 3 cách cân chỉnh update RMS, trích nguyên số liệu), trên mô hình 800M đã sửa kiến trúc MLP (từ SwiGLU hình dạng $[H, 2.6H]$ sang MLP 2 lớp chuẩn hình dạng $[H, 4H]$ để khuếch đại sự khác biệt về hình dạng ma trận), đánh giá ở 4 tỷ token trong lịch 20 tỷ token:

Phương pháp	Train loss	Val loss	Query weight RMS	MLP weight RMS
Baseline (co giãn $0.2\sqrt{H}$ cố định)	2.734	2.812	3.586×10^{-2}	2.52×10^{-2}
Update Norm (chuẩn hoá RMS đúng 0.2)	2.72	2.789	4.918×10^{-2}	5.01×10^{-2}
Adjusted LR (co giãn theo $\sqrt{\max(A, B)}$ riêng từng ma trận)	2.721	2.789	3.496×10^{-2}	4.89×10^{-2}

Cả Update Norm và Adjusted LR đều tốt hơn Baseline (loss thấp hơn); với ma trận MLP $[H, 4H]$, cả hai cho RMS trọng số gấp khoảng 2 lần Baseline (vì $\sqrt{\max(H, 4H)}/\sqrt{H} = 2$, đúng dự đoán của Lemma 1); với ma trận query $[H, H]$, Update Norm vẫn chuẩn hoá RMS (cho RMS trọng số lớn hơn, giống MLP), còn Adjusted LR **không** co giãn thêm (vì $\sqrt{\max(H, H)}/\sqrt{H} = 1$), nên cho RMS trọng số tương tự Baseline. Paper chọn **Adjusted LR** cho các thí nghiệm sau vì chi phí tính thấp hơn (chỉ cần một hệ số vô hướng theo hình dạng, không cần tính RMS thực của update mỗi bước).

Ablation hệ số Newton-Schulz N: $N = 10$ cho kết quả trực giao hoá chính xác hơn $N = 5$ nhưng **không** cải thiện hiệu năng huấn luyện cuối, nên paper chọn $N = 5$ (rẻ hơn). Đây là một quan sát quan trọng: độ chính xác trực giao hoá hoàn hảo không cần thiết, chỉ cần "đủ gần" trực giao để có lợi ích thực nghiệm.

Ablation momentum μ: không quan sát được cải thiện hiệu năng nhất quán khi dò μ, nên giữ $\mu = 0.95$ như thiết kế gốc của K. Jordan et al. (2024).

Quan sát không đồng nhất đáng chú ý (không phải tự mâu thuẫn của paper, nhưng là một kết quả lệch khỏi câu chuyện "Muon luôn tốt hơn" mà người ghi chú thấy cần nêu rõ): ở Bảng 4, Moonlight (Muon) thua Moonlight-A (AdamW) trên BBH (43.2 so với 45.3); ở Bảng 7, Adam-SFT nhích hơn Muon-SFT trên toàn bộ 4 benchmark đo khi áp lên một mô hình đã pretrain bằng AdamW. Hai điểm này không được paper flag như một "giới hạn" tường minh, nhưng tự các bảng số liệu thể hiện rõ: Muon không phải lúc nào cũng vượt AdamW, đặc biệt khi optimizer pretraining/SFT không khớp nhau.

6. Giới hạn

Paper không có mục ”Limitations” riêng, nhưng Mục 4 (”Discussions”) tự liệt kê các hướng chưa giải quyết, có thể đọc như giới hạn hiện tại của phương pháp:

- **Muon chưa phải một optimizer ”toàn phần”**: hiện tại Muon chỉ áp dụng cho tham số dạng ma trận; các tham số không phải ma trận (RMSNorm, LM head, embedding) vẫn phải dùng AdamW riêng, tạo ra một thiết lập **lai (hybrid)** hai optimizer cùng lúc trong một lần huấn luyện, thay vì một optimizer thống nhất.
- **Chỉ giới hạn ở spectral norm**: Muon tương đương steepest descent dưới spectral norm (một trường hợp đặc biệt của Schatten-p norm với p lớn); paper tự nêu việc mở rộng sang Schatten norm tổng quát hơn là hướng nghiên cứu ”đầy hứa hẹn” nhưng **chưa thực hiện** trong bài này.
- **Vấn đề không khớp optimizer pretrain-finetune (pretraining-finetuning mismatch)**: paper tự thừa nhận hiện tượng ”mô hình pretrain bằng AdamW rồi finetune bằng Muon có hiệu năng dưới tối ưu, và ngược lại”, gọi đây là ”rào cản đáng kể” cho việc tận dụng các checkpoint đã pretrain bằng AdamW có sẵn, và nói rõ cần ”một nghiên cứu lý thuyết chặt chẽ” để hiểu cơ chế bên dưới - hiện bài báo **chưa có** lời giải cho vấn đề này, chỉ xác nhận nó tồn tại qua Bảng 6/7.
- **Hệ số Newton-Schulz không được tinh chỉnh lại**: paper giữ nguyên hệ số a, b, c từ thiết kế gốc K. Jordan et al. (2024), không tự suy ra hoặc tối ưu lại cho các quy mô mô hình mới trong bài.
- **Quy mô kiểm chứng vẫn hạn chế so với LLM hàng đầu hiện nay**: dù Moonlight (16B tổng tham số, 5.7 nghìn tỷ token) lớn hơn hẳn các thí nghiệm Muon gốc, đây vẫn là một MoE tương đối nhỏ so với các LLM hàng đầu cỡ hàng trăm tỷ tham số; kết quả scaling law chính (Mục 3.2) chỉ chạy tới 1.5B tham số dense, Moonlight là điểm dữ liệu duy nhất ở quy mô lớn hơn, không có một đường scaling law đầy đủ ở quy mô MoE 16B.
- **Báo cáo kỹ thuật, không qua bình duyệt đầy đủ**: bản PDF tự ghi ”Technical Report”, không phải một bài hội nghị/tạp chí đã qua peer review đầy đủ tại thời điểm bài được viết (khác các paper khác đã đọc trong lộ trình như Shampoo/SOAP, dù SOAP sau đó cũng được paper này trích dẫn như đã công bố tại ICLR 2025).

7. Liên hệ với các bài khác

- **opt 14 (Shampoo) và opt 15 (SOAP)**: cùng thuộc nhóm ”optimizer phi đường chéo cho LLM pretraining quy mô lớn”, nhưng theo cơ chế khác hẳn: Shampoo/SOAP dùng preconditioning qua ma trận Kronecker/eigenbasis (L, R tích luỹ từ GG^T, G^TG), còn Muon dùng **trực giao hoá trực tiếp** của chính ma trận update (qua Newton-Schulz, không cần tích luỹ lịch sử dạng ma trận L, R nào). Về bộ nhớ, Muon chỉ cần một buffer momentum (giống SGD-momentum, rẻ hơn hẳn cả Adam lẫn Shampoo/SOAP vốn cần L, R cỡ $O(m^2 + n^2)$); đây là một khác biệt quan trọng về chi phí bộ nhớ giữa hai nhánh ”optimizer phi đường chéo” đã đọc trong lộ trình.
- **opt 48 (Old Optimizer, New Norm: An Anthology)**: paper hiện tại trích dẫn trực tiếp ”Bernstein et al. 2024” (chính là paper đã đọc ở opt 48) cho diễn giải ”Muon = steepest descent dưới spectral norm”. Đã xác nhận qua `48_2024_old_optimizer_new_norm.md`: bài đó (viết trước Muon paper này, công bố 09/2024) đã tự trình bày một ”Newton-Schulz iteration” của riêng hai tác giả Bernstein & Newhouse ($X_{t+1} = \frac{3}{2}X_t - \frac{1}{2}X_tX_t^TX_t$) để tính UV^T mà **không hề nhắc tên ”Muon”**, và ghi chú của bài đó đã tự nhận xét (ngoài paper, kiến thức chung) đây chính là phép trực giao hoá được Muon dùng sau này - một sự trùng khớp lý thuyết đã được xác nhận hai chiều giữa hai bài trong cùng lộ trình.
- **opt 36 (Scaling exponents / parameterizations for optimizers)**: `36_2024_scaling_exponents_parameterizations_optimizers.md` (đã xác nhận trong thư mục) tự liệt kê Muon trong phần liên hệ, nói rõ Muon ”không nằm trong ba họ SGD/Adam/Adafactor” mà bài 36 phân tích, và việc suy ra exponent learning-rate-theo-width cho Muon (theo khung muP-tổng-quát của bài 36) vẫn là ”câu hỏi mở, chưa có câu trả lời trong cả hai bài” - xác nhận rằng tới thời điểm viết ghi chú này, chưa có phân tích parameterization/muP chính thức nào cho Muon trong lộ trình.

- **opt 06/07 (Adam, AdamW)**: Muon trong bài này luôn được dùng **kết hợp** với AdamW (cho tham số không phải ma trận: RMSNorm, LM head, embedding), nên baseline so sánh chính xuyên suốt bài là AdamW; Mục 3.4 (update RMS của AdamW "thường trong khoảng 0.2-0.4") là một con số thực nghiệm được paper này tự đo, không trích từ paper AdamW gốc (opt 07).
- **opt 11 (Lion)**: cùng là optimizer được symbolic-search/thiết kế để thay AdamW cho LLM pretraining với ít trạng thái bộ nhớ hơn (Lion: 1 trạng thái; Muon: 1 trạng thái momentum, không cần v_t kiểu Adam); khác biệt chính là Lion hoạt động hoàn toàn theo toạ độ (sign của EMA), còn Muon hoạt động ở cấp độ **toàn ma trận** (trực giao hoá SVD-xấp-xỉ).

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là **suy luận** của người ghi chú, áp cho bài toán nén/huấn luyện tiếp (post-training) LightOnOCR-3-0.8B (VLM OCR lai GatedDeltaNet + full attention, $\approx$ 3600 token/trang, GPU $\leq$ 8GB, batch size 1). Paper không nhắc tới mô hình này, VLM, hay kiến trúc lai GatedDeltaNet.

- **Chi phí bộ nhớ của Muon là điểm mạnh rõ rệt nhất cho ràng buộc 8GB**: Muon chỉ cần **một** buffer momentum (giống SGD-momentum, kích thước $m \times n$ cho mỗi ma trận), không cần buffer v_t thứ hai như AdamW/Lion-hai-beta, và **không** cần các ma trận preconditioner cỡ $O(m^2 + n^2)$ như Shampoo/SOAP (opt 14/15). (tự tính) Với một ma trận chiếu 2048×512 (giả định một phần kích thước của mô hình 0.8B, để minh hoạ, không phải số từ paper): Muon cần $2048 \times 512 \times 4$ byte ≈ 4.2MB (một buffer momentum fp32), so với AdamW cần gấp đôi (≈ 8.4MB cho m và v), và so với SOAP bản kết hợp (đã tính ở `15_2024_soap.md` cho cùng kích thước là ≈ 10.5MB). Đây là optimizer **tiết kiệm bộ nhớ nhất** trong 3 optimizer phi đường chéo (Shampoo/SOAP/Muon) đã đọc trong lộ trình, một lợi thế trực tiếp và rõ ràng cho ràng buộc 8GB.
- **Chi phí tính Newton-Schulz mỗi bước phụ thuộc kích thước ma trận, cần cân nhắc với batch 1**: mỗi bước Newton-Schulz cần vài phép nhân ma trận $m \times n$ (khoảng 5 bước lặp theo paper); với batch size 1, số bước optimizer cho mỗi epoch dữ liệu sẽ nhiều hơn hẳn so với batch lớn (ít gradient accumulation), nên tổng chi phí tương đối của Newton-Schulz (vốn không phụ thuộc batch size, chỉ phụ thuộc kích thước ma trận) so với một bước forward-backward đơn lẻ (cũng rất rẻ ở batch 1) có thể **chiếm tỉ trọng lớn hơn** so với tỉ lệ "1-3% tổng thời gian" mà paper báo cáo ở quy mô huấn luyện lớn (batch hàng nghìn ví dụ) - đây là một khoảng trống thực nghiệm paper không đề cập (paper chỉ đo ở quy mô batch rất lớn), cần tự đo lại ở quy mô batch 1 trước khi áp dụng.
- **Cảnh báo pretrain-finetune mismatch trực tiếp liên quan tới dự án**: dự án là **post-training/nén** một mô hình đã pretrain sẵn (LightOnOCR-3-0.8B); paper tự nêu rõ (Mục 4, và xác nhận qua Bảng 6/7) rằng nếu mô hình được pretrain bằng AdamW (rất có thể là trường hợp của LightOnOCR, vì AdamW là lựa chọn phổ biến nhất, dù paper gốc của mô hình này không được đọc trong phạm vi bài ghi chú này) mà sau đó finetune bằng Muon, hiệu năng **không** có lợi thế rõ rệt so với tiếp tục dùng AdamW, và có thể còn kém hơn đôi chút (như Bảng 7 cho thấy trên Qwen2.5-7B). Đây là một suy luận quan trọng: nếu không biết chắc optimizer pretraining gốc của LightOnOCR-3-0.8B, chuyển sang Muon cho giai đoạn fine-tune/nén tiếp theo có rủi ro thực nghiệm cụ thể mà paper này đã ghi nhận, không chỉ là rủi ro lý thuyết.
- **Kết luận suy luận của người ghi chú**: trong ba optimizer phi đường chéo mới (Shampoo/SOAP/Muon) đã đọc, Muon là ứng viên **khả thi nhất về bộ nhớ** cho ràng buộc 8GB (chỉ 1 buffer momentum, không ma trận preconditioner cỡ $O(m^2+n^2)$), nhưng mang hai rủi ro cụ thể mà paper tự ghi nhận: (a) toàn bộ bằng chứng hiệu quả tính toán (52% FLOPs so với AdamW) đo ở chế độ batch lớn, chưa có dữ liệu ở batch 1; (b) nếu mô hình gốc đã pretrain bằng AdamW, chuyển sang Muon cho fine-tune có thể không mang lại lợi ích rõ rệt, theo đúng hiện tượng "optimizer mismatch" mà paper nêu. Nên coi Muon là một hướng đáng thử nghiệm nhỏ (không phải áp dụng mặc định), ưu tiên đo tỉ trọng chi phí Newton-Schulz ở batch 1 trước khi quyết định dùng cho toàn bộ quá trình fine-tune.

9. Thí nghiệm gợi ý

1. **Tái tạo Lemma 1 (update RMS theo hình dạng ma trận) bằng thực nghiệm số trên GPU 8GB.** Tạo các ma trận ngẫu nhiên gradient hạng đầy đủ với các hình dạng khác nhau (ví dụ 256×256, 256×1024, 1024×256), áp Newton-Schulz (tự cài đặt Eq. 2, 5 bước lặp) để trực giao hoá, đo RMS thực tế của kết quả, so với công thức lý thuyết $1/\sqrt{\max(A, B)}$. Kỳ vọng: khớp sát công thức (sai số nhỏ do chỉ xấp xỉ 5 bước Newton-Schulz, không trực giao hoá hoàn hảo). Diễn giải: nếu sai số lớn ở ma trận hình dạng lệch nhiều (ví dụ 256×4096), có thể do 5 bước Newton-Schulz chưa đủ hội tụ cho tỉ lệ khía cạnh (aspect ratio) lớn, cần tăng số bước N hoặc kiểm tra lại hệ số a, b, c có còn phù hợp không.
2. **Đo tỉ trọng chi phí Newton-Schulz ở batch size 1, so với báo cáo "1-3%" của paper ở batch lớn.** Trên một mạng nhỏ (MLP 4 lớp, mỗi lớp 512×512, chạy trên GPU 8GB), đo thời gian một bước forward-backward ở batch size 1 so với thời gian 5 bước Newton-Schulz cho optimizer Muon áp lên cùng các ma trận đó, tính tỉ lệ phần trăm thời gian optimizer/tổng thời gian một bước huấn luyện. Kỳ vọng: tỉ lệ này **cao hơn** 1-3% mà paper báo cáo ở quy mô lớn, vì ở batch 1 thời gian forward-backward mỗi bước ngắn hơn hẳn (ít việc tính hơn) trong khi Newton-Schulz không phụ thuộc batch size. Diễn giải: nếu tỉ lệ đo được vẫn nhỏ (dưới 10%), Muon vẫn thực tế cho dự án về mặt tốc độ; nếu lớn hơn nhiều (ví dụ trên 30%), đây là một rào cản tốc độ cụ thể cho việc dùng Muon ở batch 1, cần cân nhắc kỹ trước khi áp dụng.
3. **So sánh SVD entropy của update giữa Muon và AdamW trên một mạng nhỏ, kiểm tra trực giác "trải đều hướng cập nhật".** Huấn luyện một MLP nhỏ (ví dụ $128 \to 256 \to 128$) trên một bài toán hồi quy toy bằng cả Muon và AdamW, định kỳ lưu lại ma trận update (không phải trọng số) của một lớp, tính SVD entropy theo công thức Mục 3.4 (dùng `torch.linalg.svdvals`). Kỳ vọng: SVD entropy của update Muon cao hơn AdamW qua các bước huấn luyện, khớp xu hướng Hình 4 của paper (dù ở quy mô nhỏ hơn rất nhiều). Diễn giải: nếu không thấy khác biệt rõ, kiểm tra lại bài toán có đủ phức tạp (nhiều hướng gradient quan trọng) để lộ sự khác biệt hay không; bài toán quá đơn giản (gradient gần hạng-1) có thể khiến cả hai optimizer cho entropy tương tự.
4. **Kiểm tra hiện tượng "pretrain-finetune mismatch" ở quy mô nhỏ, liên quan trực tiếp tới rủi ro đã nêu ở Mục 8.** Pretrain một mô hình nhỏ (ví dụ một Transformer vài triệu tham số trên một tập văn bản nhỏ) bằng AdamW tới một checkpoint cố định, rồi finetune tiếp checkpoint đó bằng (a) AdamW và (b) Muon trên một tập dữ liệu downstream khác, đo loss/accuracy sau cùng số bước finetune. Kỳ vọng, theo đúng cảnh báo của paper (Bảng 7): finetune bằng Muon trên một mô hình pretrain-bằng-AdamW sẽ **không** vượt trội finetune bằng AdamW, có thể còn kém hơn đôi chút. Diễn giải: nếu kết quả khớp, xác nhận rủi ro cụ thể cho việc chọn Muon làm optimizer fine-tune cho LightOnOCR-3-0.8B (đã pretrain bằng optimizer không rõ, khả năng cao là AdamW); nếu không khớp (Muon vẫn tốt hơn), có thể hiện tượng mismatch của paper phụ thuộc quy mô mô hình/dữ liệu cụ thể, cần thêm thí nghiệm ở quy mô gần hơn với mô hình thật.
5. **Đo bộ nhớ thực tế của Muon so với AdamW trên một `nn.Linear` lớn, xác nhận lợi thế "một buffer momentum".** Với `nn.Linear(2048, 8192)` (kích thước MLP điển hình), cài đặt vòng lặp optimizer Muon (chỉ lưu một ma trận momentum $m \times n$) và AdamW (lưu hai ma trận m, v), đo `torch.cuda.max_memory_allocated()` sau vài bước. Kỳ vọng: bộ nhớ optimizer của Muon xấp xỉ một nửa AdamW, khớp phân tích Mục 3.5 của paper (dù paper đo cho bối cảnh phân tán ZeRO-1, không phải một GPU đơn như đề xuất thí nghiệm này). Diễn giải: xác nhận trực tiếp lợi thế bộ nhớ nêu ở Mục 8, con số cụ thể cho kích thước ma trận gần với mô hình 0.8B thật của dự án.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Update Muon gốc (Eq. 1)	$M_t = \mu M_{t-1} + \nabla L_t(W_{t-1})$; $O_t = \text{NewtonSchulz}(M_t)$; $W_t = W_{t-1} - \eta_t O_t$	Trực giao hoá momentum trước khi cập nhật, thay vì dùng trực tiếp như SGD-momentum
Newton-Schulz (Eq. 2)	$X_k = aX_{k-1} + b(X_{k-1}X_{k-1}^T)X_{k-1} + c(X_{k-1}X_{k-1}^T)^2X_{k-1}$, a=3.4445, b=−4.7750, c=2.0315	Xấp xỉ UV^T (trực giao hoá) không cần SVD tường minh, vài phép nhân ma trận
Update RMS theo hình dạng (Lemma 1)	RMS $= 1/\sqrt{\max(A,B)}$ cho ma trận $[A,B]$ hạng đầy đủ	Giải thích vì sao Muon gốc bất ổn khi scale lên các hình dạng tham số khác nhau
Update đã cân chỉnh, khớp AdamW (Eq. 4)	$W_t = W_{t-1} - \eta_t(0.2 \cdot O_t \cdot \sqrt{\max(A,B)} + \lambda W_{t-1})$	Cho phép dùng lại learning rate/weight decay đã dò cho AdamW
Weight decay thêm vào Muon (Eq. 3)	$W_t = W_{t-1} - \eta_t(O_t + \lambda W_{t-1})$	Ngăn trọng số/RMS output tăng không kiểm soát khi huấn luyện dài
SVD entropy	$H(\sigma) = -\frac{1}{\log n}\sum_i \frac{\sigma_i^2}{\sum_j \sigma_j^2} \log \frac{\sigma_i^2}{\sum_j \sigma_j^2} \in [0,1]$	Đo mức "trải đều" của phổ trị singular một ma trận update/trọng số
Hiệu quả tính toán chính	$\approx 52\%$ FLOPs của AdamW để đạt cùng hiệu năng (compute-optimal)	Kết quả đầu mục scaling law, mô hình dense tới 1.5B tham số
Bộ nhớ Distributed Muon	bằng một nửa phần bộ nhớ thêm của Distributed AdamW (1 buffer momentum so với 2)	Lợi thế bộ nhớ chính so với AdamW (và so với Shampoo/SOAP cần $O(m^2+n^2)$ thêm)

11. Kết luận ngắn

Paper mở rộng optimizer Muon (dựa trên trực giao hoá momentum qua Newton-Schulz) từ quy mô nhỏ lên quy mô LLM lớn, bằng hai sửa đổi cụ thể: thêm weight decay (ngăn trọng số tăng không kiểm soát khi huấn luyện dài) và cân chỉnh update RMS đồng nhất theo hình dạng ma trận, khớp với AdamW (cho phép dùng lại hyperparameter đã tinh chỉnh, không cần dò riêng). Kết hợp với một cài đặt phân tán kiểu ZeRO-1 (bộ nhớ thêm chỉ bằng nửa AdamW), Muon đạt hiệu quả tính toán $\approx$ 52% FLOPs của AdamW ở chế độ compute-optimal (scaling law tới 1.5B tham số dense), và mô hình MoE Moonlight (3B/16B tham số, 5.7 nghìn tỷ token, huấn luyện bằng Muon) vượt các mô hình cùng kiến trúc/quy mô token dùng AdamW trên hầu hết benchmark, đặc biệt mạnh ở code và toán, dù có một vài ngoại lệ không đồng nhất (BBH kém hơn AdamW, SFT bằng Muon trên mô hình pretrain-bằng-AdamW không vượt trội). Giới hạn chính paper tự nêu: Muon vẫn cần kết hợp với AdamW cho tham số phi-ma-trận (chưa phải optimizer toàn phần), và hiện tượng "pretrain-finetune optimizer mismatch" chưa có lời giải lý thuyết. Đối với dự án nén/huấn luyện LightOnOCR-3-0.8B trong 8GB/batch 1, Muon là optimizer tiết kiệm bộ nhớ nhất trong ba optimizer phi đường chéo đã đọc (Shampoo/SOAP/Muon), nhưng mang hai rủi ro cụ thể cần tự kiểm chứng trước khi dùng: chi phí Newton-Schulz tương đối ở batch 1 (chưa có số liệu trong paper) và rủi ro mismatch nếu mô hình gốc đã pretrain bằng một optimizer khác (nhiều khả năng AdamW).

Nguồn gốc: https://arxiv.org/abs/2502.16982

Bài 17

The Road Less Scheduled (2024)

Tác giả: Aaron Defazio, Xingyu (Alice) Yang (Fundamental AI Research Team, Meta); Harsh Mehta (Google Research); Konstantin Mishchenko (Samsung AI Center); Ahmed Khaled (Princeton University); Ashok Cutkosky (Boston University, tác giả chính/senior author).
Venue / arXiv: arXiv:2405.15682 (v4, 29/10/2024); công bố tại NeurIPS 2024 (38th Conference on Neural Information Processing Systems, ghi ở chân trang 3 của PDF).
Nguồn: https://arxiv.org/abs/2405.15682
Chủ đề: đề xuất ”Schedule-Free” learning: một họ phương pháp (Schedule-Free SGD, Schedule-Free AdamW) loại bỏ hoàn toàn nhu cầu đặt lịch learning rate (learning rate schedule, ví dụ cosine decay) phụ thuộc vào tổng số bước huấn luyện T đã biết trước, bằng một dạng nội suy momentum mới giữa điểm ”cơ sở” (base) và điểm trung bình (average), dựa trên một định lý online-to-batch conversion tổng quát mới mà paper chứng minh; đạt hiệu năng ngang hoặc vượt các lịch đã tinh chỉnh kỹ trên 28 bài toán, từ hồi quy logistic lồi tới huấn luyện mô hình ngôn ngữ lớn.
Vị trí trong lộ trình: đọc sau opt 18 (SGDR, lịch cosine-với-warm-restart mà bài này phê bình trực tiếp vì cần biết trước tổng số bước) và opt 19 (Super-Convergence, một lịch cyclical khác cũng phụ thuộc thời gian huấn luyện). Đọc cùng opt 06/07 (Adam/AdamW, nền tảng mà Schedule-Free AdamW mở rộng trực tiếp, xem Algorithm 1) và opt 46/47 (các khuyến nghị thực hành về learning rate/warmup, cùng chủ đề ”chọn lịch lr” nhưng theo hướng ngược lại: bài này thay thế hoàn toàn việc chọn lịch).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Trong tối ưu lồi (convex optimization), lý thuyết kinh điển nói rằng với SGD bước $z_{t+1} = z_t - \gamma g_t$, kỳ vọng loss của chính dãy z là **không tối ưu**; cần lấy trung bình Polyak-Ruppert (PR) $x_{t+1} = (1-c_{t+1})x_t + c_{t+1}z_{t+1}$ (với $c_{t+1} = 1/(t+1)$, tức trung bình đều theo thời gian) để đạt tốc độ hội tụ tối ưu về lý thuyết. Tuy nhiên, đây là một ”khoảng cách lý thuyết-thực hành” nổi tiếng trong ngành (”folk-law”): trong thực hành, trung bình PR cho kết quả **tệ hơn hẳn** so với chỉ dùng điểm cuối (last-iterate) z_T của SGD (paper minh hoạ qua Hình 2 và Hình 11/Appendix).

Gần đây, Zamani và Glineur (2023) và Defazio et al. (2023) (một công trình có chung tác giả với bài này) chỉ ra rằng tốc độ tối ưu worst-case có thể đạt được chỉ bằng cách chọn **lịch learning rate** (learning rate schedule) phù hợp, không cần trung bình hoá gì cả. Điều này gợi ý lịch learning rate đóng vai trò tương đương trung bình PR về mặt lý thuyết. Nhưng lịch learning rate có một nhược điểm nghiêm trọng: phải **biết trước** thời điểm dừng T để thiết kế lịch (ví dụ cosine decay cần biết T để tính điểm ”về 0”).

Câu hỏi nghiên cứu của paper: có tồn tại một cách trung bình hoá (averaging) nào vừa khớp hiệu năng thực nghiệm của lịch learning rate, vừa giữ được bảo đảm lý thuyết, mà **không cần biết trước** T? Paper trả lời khẳng định bằng cách xây dựng một liên kết mới giữa trung bình hoá và lịch learning rate.

2. Ý tưởng chính

Công thức Schedule-Free SGD (Eq. 3-5):

$$y_t = (1-\beta)z_t + \beta x_t, \qquad z_{t+1} = z_t - \gamma \nabla f(y_t, \zeta_t), \qquad x_{t+1} = (1-c_{t+1})x_t + c_{t+1}z_{t+1},$$

với $c_{t+1} = 1/(t+1)$, $z_1 = x_1$ là điểm khởi tạo. Ba dãy có vai trò riêng: z là "dãy cơ sở" (nơi bước gradient descent thực sự xảy ra, giống SGD thuần); x là "dãy ước lượng" (trung bình đều theo thời gian của dãy z, chính là dãy được **trả về** làm tham số cuối cùng); y là "dãy vị trí tính gradient" (nơi gradient được đánh giá tại mỗi bước), nội suy giữa z và x theo trọng số β.

Tham số β nội suy giữa hai trường hợp đã biết: $\beta = 0$ cho đúng trung bình Polyak-Ruppert (gradient luôn tính tại z); $\beta = 1$ cho "Primal averaging" (gradient luôn tính tại x, công thức Eq. 6-7, hội tụ tối ưu về lý thuyết nhưng được paper mô tả là "nói chung quá chậm để dùng thực tế"). Giá trị $\beta \approx 0.9$ (giống giá trị momentum tiêu chuẩn) cho "những điều tốt của cả hai thế giới": hội tụ nhanh như PR averaging (vì dãy z di chuyển nhanh hơn hẳn x) mà vẫn giữ sự kết nối giữa dãy trả về x và vị trí tính gradient y, tăng ổn định.

Trực giác momentum: với $\beta = 0.9$, ảnh hưởng ngay lập tức của gradient g_t vào dãy y là $(1-\beta)g_t = 0.1g_t$, giống hệt hệ số của EMA-momentum chuẩn ($\beta = 0.9$). Khác biệt: phần còn lại của g_t (chiếm $0.9g_t$) được "đưa vào" y rất chậm qua vai trò của nó trong trung bình x theo thời gian, trong khi EMA-momentum chuẩn đưa phần lớn gradient vào trong khoảng 10 bước tiếp theo. Vậy Schedule-Free là "một dạng momentum có cùng hiệu ứng ngay lập tức nhưng trì hoãn lâu hơn phần còn lại của gradient".

Cơ chế "thay thế lịch" (Mục 2.3): paper minh hoạ (Hình 4) rằng dãy x của mọi phương pháp trung bình hoá (PR, Primal, Schedule-Free) đều mô phỏng hiệu ứng của một **lịch Linear Decay** (lịch giảm tuyến tính, đã biết là tối ưu worst-case) kết thúc tại bước **hiện tại** t, chứ không phải bước cuối T. Đây chính là lý do các phương pháp trung bình hoá có thể "thay" một lịch: chúng đang ngầm áp dụng một lịch, tại mọi thời điểm t, như thể t là bước cuối cùng. Khác biệt giữa ba phương pháp nằm ở việc gradient gần đây góp phần vào dãy y (nơi gradient được tính) nhiều hay ít: PR averaging giống Linear Decay ở đầu huấn luyện nhưng lệch nhiều ở cuối (gradient gần đây ảnh hưởng quá mạnh lên y); Primal averaging ngược lại (giống Linear Decay ở cuối nhưng hội tụ chậm ở đầu vì hạ-trọng-số gradient gần đây quá mạnh); Schedule-Free khắc phục nhược điểm của Primal averaging bằng cách tăng nhẹ trọng số gradient gần đây xuyên suốt quá trình huấn luyện.

(tự tính) Ví dụ số minh hoạ công thức nội suy $y_t = (1-\beta)z_t + \beta x_t$ với $\beta = 0.9$: giả sử tại một bước t nào đó, dãy cơ sở (di chuyển nhanh) đang ở $z_t = 5.0$ và dãy trung bình (di chuyển chậm) đang ở $x_t = 3.0$ (một số thực bất kỳ, chỉ để minh hoạ, không phải số từ paper). Khi đó $y_t = 0.1 \times 5.0 + 0.9 \times 3.0 = 0.5 + 2.7 = 3.2$. Gradient tại bước này được tính tại $y_t = 3.2$, rất gần với $x_t = 3.0$ (dãy sẽ được trả về) nhưng đã dịch nhẹ 0.2 về phía $z_t = 5.0$ (dãy di chuyển nhanh theo gradient mới nhất) - đúng tinh thần "cùng hiệu ứng ngay lập tức như EMA-momentum nhưng phần còn lại được trì hoãn", vì chỉ 10% khoảng cách giữa z_t và x_t được "đưa vào" y_t ngay lập tức.

3. Phương pháp chi tiết

3.1 Lý thuyết tổng quát: định lý online-to-batch mới (Theorem 2)

Với F là hàm lồi, $\zeta_1, \dots, \zeta_T$ là dãy iid sao cho $F(x) = \mathbb{E}_\zeta[f(x,\zeta)]$, và các dãy số bất kỳ $z_1, \dots, z_T$, $w_1, \dots, w_T, \beta_1, \dots, \beta_T \in [0,1]$ (độc lập với $\zeta_t, \dots, \zeta_T$), định nghĩa:

$$x_t = \frac{\sum_{i=1}^t w_i z_i}{\sum_{i=1}^t w_i}, \qquad y_t = \beta_t x_t + (1-\beta_t) z_t, \qquad g_t = \nabla f(y_t, \zeta_t).$$

Khi đó: $\mathbb{E}[F(x_T) - F(x^\star)] \le \dfrac{\mathbb{E}[\sum_{t=1}^T w_t \langle g_t, z_t - x^\star\rangle]}{\sum_{i=1}^T w_i}$ (Eq. 12).

Vế phải chính là **regret** (hối hận, đại lượng trung tâm của online convex optimization) chia cho tổng trọng số, nên Theorem 2 là một **phép chuyển đổi online-to-batch (online-to-batch conversion)**: biến một thuật toán tối ưu online (chỉ có chặn regret) thành một thuật toán tối ưu stochastic (có chặn hội tụ kỳ vọng). Paper chỉ ra định lý này **thống nhất (unify)** nhiều phép chuyển đổi online-to-batch đã biết trước đây, đơn giản bằng cách chọn w_t, β_t khác nhau:

- $w_t = 1, \beta_t = 0 \; \forall t$: cho lại trung bình Polyak chuẩn (Cesa-Bianchi et al. 2004).
- $w_t = 1, \beta_t = 1 \; \forall t$: cho lại Primal averaging, đã được chỉ ra tương đương "heavy-ball momentum" (Defazio 2020; Defazio và Gower 2021).
- $w_t = 1, \beta_t = t/T$: cho lại kết quả gần đây của Zamani và Glineur (2023)/Defazio et al. (2023) về lịch giảm tuyến tính.

Từ Theorem 2 với $w_t = 1, \beta_t = \beta$ (hằng số) suy ra **Theorem 1** (trường hợp đặc biệt cho SGD): với $D = \|x_1 - x^\star\|$, $\gamma = D/(G\sqrt{T})$, với **mọi** $\beta \in [0, 1]$:

$$\mathbb{E}[F(x_T) - F(x^\star)] \leq \frac{DG}{\sqrt{T}}.$$

Điểm quan trọng: đây là tốc độ tối ưu worst-case cho hàm lồi Lipschitz không trơn (non-smooth), **với mọi lựa chọn** β, bao gồm cả hằng số. Paper đối lập điều này với EMA-momentum chuẩn: trong cùng bối cảnh không trơn, EMA-momentum thực sự **làm xấu đi** tốc độ hội tụ worst-case theo lý thuyết, còn Schedule-Free giữ nguyên tính tối ưu dù vẫn có "cảm giác" của momentum.

3.2 Learning rate lớn (Theorem 3)

Theo lý thuyết worst-case cổ điển, learning rate tối ưu cho SGD với thời hạn T cố định là $\gamma = D/(G\sqrt{T})$ (giảm theo $1/\sqrt{T}$); cho hội tụ "any-time" (dừng bất cứ lúc nào), lý thuyết gợi ý dùng dãy $\gamma_t = D/(G\sqrt{t})$. Nhưng paper nói rõ dãy learning rate dạng này "có hiệu năng thực hành kém" (trích Defazio et al. 2023); thay vào đó, bước lớn hơn nhiều, dạng hằng số D/G (không giảm theo t), cho hiệu năng tốt hơn hẳn trong hầu hết ứng dụng thực tế - một khoảng cách lý thuyết-thực hành khác mà paper nói "hầu như chưa được bàn tới trong tài liệu".

Theorem 3: với gradient bị chặn g_t, $z_{t+1} = z_t - \gamma g_t$, $D = \|z_1 - z^\star\|$, $G = \max_{t \leq T} \|g_t\|$, nếu chọn $\gamma = D/G$ và **điều kiện** $\sum_{t=1}^{T} \langle g_t, z_t - z_1 \rangle \leq D\sqrt{\sum_{t=1}^{T} \|g_t\|^2}$ (Eq. 15) được thoả, thì regret trung bình đạt $O\left(\frac{D}{T}\sqrt{\sum_t \|g_t\|^2}\right)$ (Eq. 16), tức tốc độ tối ưu dù dùng learning rate lớn hằng số. Điều kiện này **có thể kiểm tra được trong lúc huấn luyện** (dùng điểm tham chiếu $z^\star = x_T$), và paper báo cáo nó thoả mãn cho **mọi** bài toán trong các thí nghiệm ở Mục 4.1. Paper suy đoán (giả thuyết, không chứng minh đầy đủ cho trường hợp stochastic tổng quát) rằng thành công của learning rate lớn có thể liên quan tới hành vi tiệm cận-toàn phương (asymptotic quadratic behavior) của quá trình học, dựa trên kết quả đã biết của Bach và Moulines (2013) cho bài toán toàn phương (quadratic).

3.3 Thuật toán cụ thể: Schedule-Free AdamW (Algorithm 1)

```
Input: x1, learning rate γ, decay λ, số bước warmup T_warmup, β1, β2, ε
z1 = x1; v0 = 0
for t = 1 to T:
    y_t = (1-β1) z_t + β1 x_t                          # momentum qua nội suy
    g_t = ∇f(y_t, ζ_t)                                  # gradient tính tại y
    v_t = β2 v_{t-1} + (1-β2) g_t^2
    γ_t = γ · sqrt(1 - β2^t) · min(1, t/T_warmup)       # LR gồm warmup +
    ↪  bias-correction Adam
    z_{t+1} = z_t - γ_t g_t / (sqrt(v_t) + ε) - γ_t λ y_t  # weight decay tính tại y
    c_{t+1} = γ_t^2 / Σ_{i=1}^t γ_i^2
    x_{t+1} = (1-c_{t+1}) x_t + c_{t+1} z_{t+1}
Return x_T
```

Hai chi tiết cài đặt quan trọng:

1. **Trọng số c_{t+1} khi có warmup (Eq. 23)**: không dùng $c_{t+1} = 1/(t+1)$ đơn giản, mà dùng $c_{t+1} = \gamma_t^2 / \sum_{i=1}^{t} \gamma_i^2$ (trọng số theo bình phương learning rate tại mỗi bước). Paper báo cáo điều này "cải thiện

đáng kể hiệu năng" khi có warmup; sau khi warmup kết thúc, dãy này suy giảm với tốc độ $1/t$ (giống c_{t+1} gốc), nhưng trong giai đoạn warmup nó tránh việc các bước đầu (learning rate còn nhỏ, gradient kém tin cậy) có trọng số quá lớn trong trung bình.

2. **Weight decay tính tại y, không tại z**: paper chọn tính decay tại y (khớp cách hiểu weight decay là một regularizer L2 cộng vào loss, vì gradient của loss cũng được tính tại y), và báo cáo điều này cho hiệu năng "tốt hơn đáng kể" trên một số bài toán, cụ thể được nêu tên: ImageNet và NanoGPT.

3.4 Các vấn đề cài đặt khác (Mục 4.4)

- **Bộ nhớ**: biến thể Schedule-Free thường có **cùng** yêu cầu bộ nhớ với optimizer gốc. Ví dụ Schedule-Free SGD không cần bộ nhớ thêm so với SGD-momentum chuẩn: SGDM lưu điểm hiện tại x và buffer momentum m; Schedule-Free lưu x và z thay vào đó (y tính trực tiếp từ x, z khi cần, không cần lưu riêng). Cài đặt PyTorch hiệu quả của paper dùng một buffer cho z, và buffer tham số chính luân chuyển giữa lưu x (khi test/inference) và y (khi huấn luyện).
- **BatchNorm cần xử lý đặc biệt**: các lớp BatchNorm duy trì `running_mean`/`running_var` được tính tại y trong lúc huấn luyện; để đánh giá mô hình đúng, các buffer này cần được cập nhật lại theo thống kê của x (chạy một số batch huấn luyện nhỏ qua x ngay trước mỗi lần đánh giá, hoặc dùng kỹ thuật tinh vi hơn như PreciseBN). Các lớp normalization **không** dùng batch-statistics (ví dụ LayerNorm) không cần xử lý này.
- **Warmup vẫn cần thiết**: dù không cần lịch decay, paper vẫn dùng linear warmup cho một số bước cố định T_{warmup} (đã biết trước, không phải "tổng số bước huấn luyện" T - đây là sự khác biệt quan trọng: warmup chỉ cần biết một hằng số nhỏ ở đầu, không cần biết điểm kết thúc).

4. Thí nghiệm và kết quả chính

Bài toán deep learning (Mục 4.1, 9 bài toán, Hình 5), so sánh Schedule-Free với lịch cosine/step-wise/linear-decay đã tinh chỉnh riêng cho từng bài toán:

Bài toán	Schedule-Free	Lịch tốt nhất (cosine/step-wise/LD)
CIFAR-10 (WRN-16-8), test accuracy	96.03% SE 0.04	Cosine 95.73% SE 0.04
CIFAR-100 (DenseNet), test accuracy	78.71% SE 0.06	Cosine 77.41% SE 0.09
SVHN (ResNet-3-96), test accuracy	98.40% SE 0.01	Cosine 98.27% SE 0.02
ImageNet (ResNet-50), test accuracy	76.90% SE 0.03	Cosine 76.90% SE 0.06 (ngang nhau)
fastMRI (U-Net VarNet 2.0), test SSIM	0.9112 SE 0.00012	Cosine 0.9110 SE 0.00016
Criteo Kaggle (DLRM), test accuracy	0.7915 SE 0.00003	Linear Decay 0.7906 SE 0.00006
MAE finetune (ViT), test accuracy	83.54% SE 0.03	Cosine 83.52% SE 0.02 (ngang nhau)
OpenWebText (GPT-2 124M), test loss	2.831 SE 0.008	Cosine 2.853 SE 0.004
IWSLT14 (LSTM), test loss	4.18 SE 0.001	Cosine 4.23 SE 0.003

Schedule-Free vượt rõ cosine/step-wise schedule trên 5/9 bài toán (CIFAR-10, CIFAR-100, SVHN, IWSLT14, OpenWebText), vượt linear-decay schedule trên fastMRI và Criteo, và **ngang bằng** (tie) trên ImageNet ResNet-50 và MAE fine-tune. Với mỗi bài toán, cả baseline và Schedule-Free được dò trên cùng một lưới weight decay/learning rate, và β được dò qua 2 giá trị $\{0.9, 0.98\}$.

Hiện tượng "sụp gradient norm" (Hình 6): khi huấn luyện CIFAR-10 với SGD và lịch cosine, ở giai đoạn cuối huấn luyện, cả training loss và gradient norm đều sụp gần về 0 (dù weight decay đã được tinh chỉnh tối ưu cho test accuracy), khiến việc học chậm lại đáng kể. Schedule-Free SGD **không** thể hiện hiện tượng sụp này ở cả loss và gradient norm, tiếp tục học ổn định. Paper cho rằng đây là một yếu tố góp phần giải thích kết quả tốt đặc biệt trên CIFAR-10/100/SVHN.

Độ nhạy β: giá trị mặc định $\beta = 0.9$ hoạt động tốt cho mọi bài toán **trừ** NanoGPT, nơi loss tăng nhanh khi dùng $\beta = 0.9$ (tương tự sự bất ổn quan sát được ở Polyak Averaging, Appendix H); $\beta = 0.98$ (giá trị lớn hơn trong lưới dò) ổn định cho bài toán này.

MLCommons AlgoPerf Self-Tuning Track (Mục 4.2): benchmark lớn gồm 8 bài toán (Transformer, ConvNet, U-Net trên ảnh/ngôn ngữ/đồ thị/âm thanh); bài ResNet-50 bị loại vì cả AdamW và NAdamW (baseline tham chiếu) đều không đạt ngưỡng accuracy mục tiêu trên bài này. Track ”self-tuning” yêu cầu **một bộ hyperparameter duy nhất** cho toàn bộ 8 bài toán (khác hẳn Mục 4.1, nơi mỗi bài toán được dò riêng) - đây là một bài kiểm tra khó và thực tế hơn cho tính ”không cần tinh chỉnh”. Schedule-Free AdamW vượt baseline NAdamW trên WMT, ViT, fastMRI, OGBG; hoà trên Conformer và Criteo; **thua nhẹ** trên DeepSpeech. Paper tự giải thích hai bài thua/hoà (Conformer, DeepSpeech) là do cả hai dùng BatchNorm, mà khuôn khổ đánh giá của AlgoPerf ”không dễ cho phép” cập nhật running statistics của BatchNorm theo dãy x (điều cần thiết để đạt hiệu năng tốt nhất với phương pháp này, như đã nêu ở Mục 3.4). Schedule-Free AdamW **thắng** track Self-Tuning của cuộc thi AlgoPerf 2024 (tuyên bố chính của abstract).

Bài toán lồi (Mục 4.3, Hình 8): 12 bài hồi quy logistic ngẫu nhiên từ LibSVM repository, dò learning rate trên lưới lũy-thừa-2 cho từng phương pháp riêng. Schedule-Free vượt cả hai phương pháp trung bình hoá cổ điển (Polyak, Primal) và lịch Linear Decay tốt nhất: hội tụ nhanh hơn trên 11/12 bài, accuracy cao hơn trên 6/12 bài, hoà ở các bài còn lại. Paper dùng kết quả này để khẳng định lợi thế của Schedule-Free **không** chỉ giới hạn ở bài toán không lồi (deep learning) mà cả bài toán lồi thuần.

5. Phân tích / ablation

Độ nhạy thời gian-huấn-luyện của β (Mục 5, Hình 9): chạy ImageNet với Schedule-Free SGD tới 200 epoch (dài hơn chuẩn), learning rate cố định 1.5, với các giá trị momentum $\beta \in \{0.75, 0.9, 0.95, 0.98\}$. Kết quả: $\beta = 0.9$ cho accuracy tốt nhất (77.78% SE 0.04) và **giữ nguyên là lựa chọn tốt nhất ở mọi độ dài huấn luyện** đã thử (không phụ thuộc vào thời hạn T). Đây là một kiểm tra quan trọng về tính ”thật sự schedule-free”: nếu β tối ưu thay đổi theo T, vấn đề ”phải biết trước thời hạn” chỉ bị chuyển từ learning rate schedule sang tham số β, làm mất đi toàn bộ lợi ích chính của phương pháp.

Độ nhạy learning rate (Hình 10): learning rate tối ưu ổn định qua một khoảng rộng, tương tự cả cho Schedule-Free và phương pháp dùng lịch cosine; với huấn luyện ngắn (≤ 25 epoch), learning rate lớn hơn cho kết quả tốt hơn ở cả hai cách (một hình thái phụ thuộc thời hạn nhẹ, paper nói rõ **cả hai** phương pháp đều có hiện tượng này tương tự nhau, không phải nhược điểm riêng của Schedule-Free).

Ablation trọng số c_{t+1} khi có warmup: paper báo cáo (Mục 4.4, không kèm bảng số riêng trong văn bản chính) rằng dùng trọng số $c_{t+1} = \gamma_t^2 / \sum \gamma_i^2$ (Eq. 23) thay vì $c_{t+1} = 1/(t+1)$ đơn giản ”cải thiện đáng kể hiệu năng” khi có warmup, nhưng không trình bày số liệu so sánh trực tiếp hai lựa chọn trong phần chính của bài (quan sát của người ghi chú, không phải thiếu sót của paper, chỉ là một khẳng định định tính không kèm bảng số trong văn bản chính mà người đọc cần lưu ý).

6. Giới hạn

Paper không có mục ”Limitations” riêng, nhưng Mục 6 (Kết luận) nêu rõ hai điểm, trích gần nguyên văn:

- **”Hạn chế thực tế chính (the primary practical limitation) là cần dò (sweep) learning rate và weight decay, vì giá trị tốt nhất khác với giá trị dùng cùng một lịch.”** Tức Schedule-Free **không** loại bỏ hoàn toàn việc tinh chỉnh hyperparameter: nó chỉ loại bỏ việc phải biết trước T để thiết kế lịch, nhưng learning rate/weight decay tối ưu cho Schedule-Free khác với giá trị tối ưu khi dùng lịch, nên không thể ”tái dùng” trực tiếp cấu hình đã tinh chỉnh cho một optimizer có lịch.
- **”Chúng tôi cung cấp một khảo sát lý thuyết sơ bộ (preliminary) cho phương pháp, nhưng cần thêm lý thuyết để hiểu đầy đủ phương pháp.”** Paper tự thừa nhận phần lý thuyết (Theorem 1-3) chỉ

bao phủ trường hợp lồi (convex); toàn bộ kết quả trên các bài toán deep learning không lồi (Mục 4.1, 4.2) là thực nghiệm, không có bảo đảm hội tụ hình thức tương ứng.

Thêm các điểm người ghi chú tổng hợp từ nội dung bài (không phải trích dẫn trực tiếp mục Limitations, vì bài không có mục này):

- $\beta = 0.9$ **mặc định không phải lúc nào cũng đúng**: như chính Mục 4.1 nêu, NanoGPT cần $\beta = 0.98$ vì $\beta = 0.9$ gây tăng loss nhanh - một ngoại lệ cụ thể (không phải suy luận của người ghi chú, mà là dữ liệu paper tự báo cáo) cho thấy ”mặc định hoạt động cho mọi bài toán” không hoàn toàn đúng trên mọi trường hợp đã thử; cần dò β giữa ít nhất 2 giá trị trong thực hành, như chính paper làm.
- **BatchNorm là một điểm yếu cài đặt cụ thể, không chỉ lý thuyết**: kết quả AlgoPerf (Mục 4.2) cho thấy hai trong số các trường hợp thua/hoà của Schedule-Free (Conformer, DeepSpeech) đều là các bài dùng BatchNorm, nơi khuôn khổ thi không hỗ trợ tốt việc cập nhật running statistics theo dãy x - một hạn chế triển khai thực tế (không phải hạn chế lý thuyết của phương pháp), nhưng ảnh hưởng tới kết quả benchmark cụ thể.
- **Điều kiện cho Theorem 3 (learning rate lớn) chỉ được kiểm chứng thực nghiệm, không chứng minh tổng quát**: paper nói rõ ”các điều kiện đầy đủ để learning rate lớn hoạt động vẫn chưa được hiểu đầy đủ cho bài toán stochastic”, chỉ xác nhận điều kiện (Eq. 15) đúng cho các bài toán đã thử, và chỉ ”suy đoán” (conjecture, không chứng minh) về cơ chế tiệm-cận-toàn-phương đứng sau.

7. Liên hệ với các bài khác

- **opt 18 (SGDR)**: `18_2016_sgdr.md` (đã xác nhận trong thư mục) tự nêu trong phần ”Vị trí trong lộ trình” của chính nó: ”...đọc trước opt 17 (Road Less Scheduled, phê bình việc phải biết trước tổng số bước mà SGDR cần)” - xác nhận hai chiều quan hệ phê bình trực tiếp giữa hai bài: SGDR (và các lịch warm-restart nói chung) cần đặt trước độ dài mỗi chu kỳ/tổng số chu kỳ, đúng loại nhược điểm mà bài hiện tại (opt 17) nhằm loại bỏ.
- **opt 19 (Super-Convergence)**: cùng thuộc nhóm ”các phương pháp điều khiển learning rate theo thời gian” (1cycle policy của Super-Convergence cũng cần biết trước T để đặt điểm giữa/điểm cuối chu kỳ), nhưng theo hướng ngược hẳn triết lý của bài hiện tại: Super-Convergence **tận dụng** việc biết trước T để đạt tốc độ hội tụ nhanh hơn một bậc độ lớn bằng learning rate rất lớn có kiểm soát theo chu kỳ, còn Schedule-Free loại bỏ hoàn toàn nhu cầu biết trước T. Thú vị là cả hai bài đều khai thác learning rate **lớn hơn** mức lý thuyết cổ điển gợi ý (Super-Convergence qua 1cycle, Schedule-Free qua Theorem 3) như một yếu tố quan trọng cho tốc độ hội tụ tốt.
- **opt 06/07 (Adam, AdamW)**: Algorithm 1 (Schedule-Free AdamW) của bài này là một sửa đổi trực tiếp lên công thức AdamW gốc (opt 07): giữ nguyên cơ chế EMA bậc hai $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ và bias-correction $\sqrt{1-\beta_2^t}$, nhưng thay thế **hoàn toàn** cơ chế momentum bậc một (EMA của gradient, m_t) bằng dãy nội suy ba phần x, y, z, và thay lịch learning rate decay theo thời gian bằng dãy trọng số c_{t+1} tính từ chính γ_t.
- **opt 46/47 (khuyến nghị thực hành hyperparameter, cùng chủ đề warmup/learning rate)**: bài hiện tại vẫn giữ nhu cầu linear warmup (Mục 3.4, 4.4) - một thực hành phổ biến được các bài khuyến nghị thực hành khác (opt 46/47, theo tên tiêu đề cố định trong titles.tsv, chưa đọc đầy đủ khi viết ghi chú này) cũng đề cập; điểm khác biệt là bài hiện tại chỉ cần biết **độ dài warmup** (một hằng số nhỏ, cố định, không phụ thuộc tổng thời gian huấn luyện), không cần biết điểm kết thúc toàn bộ quá trình huấn luyện như các lịch decay thông thường.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là **suy luận** của người ghi chú, áp cho bài toán nén/huấn luyện tiếp (post-training) LightOnOCR-3-0.8B (VLM OCR lai GatedDeltaNet + full attention, $\approx$ 3600 token/trang, GPU $\leq$ 8GB, batch size 1). Paper không nhắc tới mô hình này, VLM, hay nén/pruning.

- **Đây là optimizer "rẻ" nhất về bộ nhớ trong toàn bộ nhóm bài opt 10-17 đã đọc**: như Mục 4.4 của paper tự nêu, Schedule-Free AdamW có **cùng** yêu cầu bộ nhớ với AdamW gốc (chỉ cần lưu x, z thay vì x, m; không cần ma trận preconditioner như Shampoo/SOAP, không cần thêm buffer so với Lion/Muon). Với ràng buộc 8GB/batch 1 của dự án, đây là một lợi thế rõ ràng: Schedule-Free AdamW có thể được dùng làm "thay thế trực tiếp" (drop-in replacement) cho AdamW hiện có trong pipeline huấn luyện, **không** phát sinh thêm chi phí bộ nhớ nào, khác hẳn Shampoo/SOAP (opt 14/15, cần thêm $O(m^2 + n^2)$ mỗi layer).
- **Lợi ích thực tế lớn nhất cho một dự án cá nhân/nghiên cứu nhỏ: không cần biết trước tổng số bước fine-tune**: khi nén/fine-tune một mô hình đã pretrain (trường hợp của dự án), số bước huấn luyện tối ưu thường **không biết trước** (phụ thuộc vào khi nào đạt hội tụ, khi nào hết dữ liệu nén/distillation, hoặc khi nào cần dừng để đánh giá/thử nghiệm tiếp). Với một lịch cosine thông thường, việc không biết trước T buộc phải chọn T "ước lượng" trước, và nếu huấn luyện thực tế cần nhiều/ít hơn ước lượng, lịch cosine trở nên không tối ưu (quá dốc hoặc quá nông). Schedule-Free loại bỏ hoàn toàn nhu cầu này: có thể huấn luyện, theo dõi loss, và dừng ở bất kỳ điểm nào mà vẫn gần với hiệu năng tối ưu cho khoảng thời gian đã huấn luyện tới đó (đúng tinh thần "theo dõi sát đường Pareto frontier của loss và thời gian huấn luyện trong một lần chạy duy nhất" mà Hình 1 của paper minh hoạ).
- **Cảnh báo cụ thể từ paper (Mục 6) vẫn áp dụng: cần dò lại learning rate/weight decay riêng**: không thể lấy trực tiếp learning rate/weight decay đã dùng cho AdamW-với-cosine-schedule của LightOnOCR gốc (nếu biết) và áp thẳng vào Schedule-Free AdamW; cần một lượt dò riêng (dù nhỏ, ví dụ quét vài giá trị learning rate quanh giá trị gốc) cho phiên bản Schedule-Free, đúng như hạn chế paper tự nêu.
- **Batch size 1 là một khoảng trống thực nghiệm của paper, cần tự kiểm chứng**: toàn bộ thí nghiệm của paper dùng batch vừa/lớn (CIFAR tới ImageNet, NanoGPT với batch nhiều ví dụ); không có thí nghiệm ở batch size 1. Về mặt công thức, Schedule-Free AdamW (Algorithm 1) không có ràng buộc toán học nào cấm batch 1 (vẫn là một EMA bậc hai cộng một dãy nội suy ba phần, hoạt động per-step bình thường ở batch bất kỳ), nhưng gradient ở batch 1 nhiễu (noisy) hơn hẳn, có thể ảnh hưởng tới độ ổn định của β mặc định 0.9 (tương tự cách NanoGPT, một bài toán nhạy cảm hơn về nhiễu gradient do kiến trúc Transformer, đã cần $\beta = 0.98$ thay 0.9) - đây là một suy luận hợp lý (không có bằng chứng trực tiếp trong paper) rằng batch rất nhỏ có thể cần β lớn hơn mặc định, cần tự dò khi áp dụng.
- **Kết luận suy luận của người ghi chú**: trong số các optimizer đã đọc ở opt 10-17, Schedule-Free AdamW là ứng viên có **chi phí thấp nhất để thử nghiệm** cho dự án (không tốn bộ nhớ thêm, dễ thay thế AdamW hiện có), và mang lợi ích thực tế rõ ràng nhất cho đúng tình huống "không biết trước tổng số bước fine-tune/nén" của một dự án nghiên cứu cá nhân. Rủi ro chính là cần dò lại hyperparameter riêng (paper tự nêu) và chưa có bằng chứng ở batch size 1 (khoảng trống của paper, không phải nhược điểm đã xác nhận).

9. Thí nghiệm gợi ý

1. **Tái tạo hiện tượng "theo dõi Pareto frontier" (Hình 1 của paper) ở quy mô nhỏ trên GPU 8GB.** Huấn luyện một mô hình nhỏ (ví dụ MLP hoặc CNN nhỏ trên CIFAR-10 thu nhỏ, hoặc một Transformer nhỏ trên một tập văn bản nhỏ) bằng Schedule-Free AdamW (tự cài đặt Algorithm 1) và bằng AdamW với vài lịch cosine có độ dài T khác nhau ($T = 1000, 2000, 5000$ bước), vẽ loss theo số bước cho tất cả các lần chạy trên cùng một đồ thị. Kỳ vọng: đường loss của Schedule-Free bám sát "đường biên dưới" (envelope) của các đường cosine ở mọi điểm dừng, giống Hình 1 của paper. Diễn giải: nếu đường Schedule-Free không bám sát biên dưới ở một đoạn nào, kiểm tra lại learning rate đã dò đủ kỹ chưa (paper nhấn mạnh cần dò riêng, không dùng lại giá trị của cosine).
2. **Đo độ nhạy của β ở batch size rất nhỏ (1-4), so với batch lớn, để kiểm tra suy luận ở Mục 8.** Trên cùng mô hình nhỏ ở thí nghiệm 1, chạy Schedule-Free AdamW với batch $\in \{1, 4, 64\}$ và $\beta \in \{0.9, 0.95, 0.98\}$ (giữ tổng số example không đổi để so sánh công bằng), đo loss cuối cùng. Kỳ vọng, theo suy luận ở Mục 8: ở batch 1, $\beta = 0.9$ có thể kém ổn định hơn (loss tăng/dao động) so với β lớn hơn, tương tự hiện tượng NanoGPT mà paper báo cáo. Diễn giải: nếu $\beta = 0.9$ vẫn ổn ở batch 1, suy

luận ở Mục 8 có thể quá thận trọng; nếu không ổn, xác nhận cần dò β lớn hơn mặc định cho batch rất nhỏ.

3. **Kiểm tra hiện tượng "sụp gradient norm" (Hình 6 của paper) có xuất hiện khi fine-tune một mô hình pretrained nhỏ hay không.** Fine-tune một mô hình nhỏ đã pretrain (ví dụ một checkpoint CNN nhỏ có sẵn) trên một tập dữ liệu downstream nhỏ, bằng AdamW-với-cosine và bằng Schedule-Free AdamW, theo dõi gradient norm theo thời gian huấn luyện cho cả hai. Kỳ vọng: nếu hiện tượng sụp gradient-norm-cuối-huấn-luyện cũng xảy ra trong bối cảnh fine-tune (không chỉ pretrain-from-scratch như trong paper), Schedule-Free AdamW sẽ không thể hiện sự sụp này, có thể tiếp tục cải thiện loss ở giai đoạn cuối trong khi AdamW-cosine đã "bão hoà". Diễn giải: đây là kiểm tra trực tiếp liệu lợi ích chính của Schedule-Free (tránh sụp gradient norm) có chuyển sang đúng bối cảnh của dự án (fine-tune/nén một mô hình đã pretrain, không phải pretrain từ đầu) hay không, điều paper không thử nghiệm trực tiếp.
4. **So sánh bộ nhớ thực tế Schedule-Free AdamW so với AdamW chuẩn trên một `nn.Linear` lớn, xác nhận tuyên bố "không tốn thêm bộ nhớ".** Với `nn.Linear(2048,2048)`, cài đặt cả hai optimizer (AdamW chuẩn lưu m, v; Schedule-Free AdamW lưu x, z cộng v theo Algorithm 1), đo `torch.cuda.max_memory_allocated()` sau vài bước cho cả hai. Kỳ vọng: bộ nhớ gần như bằng nhau (cả hai cần đúng 2 buffer cỡ $m \times n$ cộng tham số chính, khác nhau không đáng kể). Diễn giải: xác nhận trực tiếp tuyên bố Mục 4.4 của paper ("cùng yêu cầu bộ nhớ với optimizer gốc"), một điều kiện tiên quyết cho việc coi đây là lựa chọn "miễn phí" về bộ nhớ cho ràng buộc 8GB của dự án.
5. **Đo ảnh hưởng của việc dừng huấn luyện "giữa đường" (không hoàn thành lịch) khi dùng Schedule-Free so với cosine, mô phỏng tình huống thực tế của một dự án cá nhân phải dừng sớm vì hết thời gian/ngân sách tính toán.** Trên mô hình nhỏ ở thí nghiệm 1, chạy cả hai phương pháp với một "ngân sách dự kiến" $T_{\text{dự kiến}} = 5000$ bước (cosine được thiết kế cho $T = 5000$), nhưng **dừng thực tế** ở các điểm $t \in \{1000, 2500, 5000, 7500\}$ (một số điểm sớm hơn, một số muộn hơn dự kiến), so sánh loss tại mỗi điểm dừng. Kỳ vọng: cosine (thiết kế cho $T = 5000$) cho kết quả tốt nhất đúng tại $t = 5000$ nhưng tệ hơn hẳn ở $t = 1000$ (còn ở pha lr cao) và ở $t = 7500$ (lr đã về 0, không học thêm được); Schedule-Free cho kết quả tốt tương đối đều ở mọi điểm dừng. Diễn giải: đây là minh chứng trực tiếp, cụ thể cho lợi ích đã nêu ở Mục 8 (dự án không biết trước chính xác số bước fine-tune cần, nên tính "bất biến theo điểm dừng" của Schedule-Free có giá trị thực tế).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Ba dãy của Schedule-Free	$y_t = (1-\beta)z_t + \beta x_t$; $z_{t+1} = z_t - \gamma \nabla f(y_t, \zeta_t)$; $x_{t+1} = (1-c_{t+1})x_t + c_{t+1} z_{t+1}$	z: cơ sở (nhanh); x: ước lượng trả về (chậm, trung bình); y: nơi tính gradient (nội suy)
Trường hợp đặc biệt	$\beta = 0$: Polyak-Ruppert averaging; $\beta = 1$: Primal averaging	Schedule-Free nội suy giữa hai phương pháp trung bình hoá cổ điển
Theorem 1 (regret bound)	$\mathbb{E}[F(x_T) - F(x^\star)] \le DG/\sqrt{T}$, đúng với mọi $\beta \in [0,1]$	Tốc độ tối ưu worst-case cho hàm lồi Lipschitz, không suy giảm vì có "momentum"
Theorem 2 (online-to-batch tổng quát)	$\mathbb{E}[F(x_T) - F(x^\star)] \le \frac{\mathbb{E}[\sum_t w_t \langle g_t, z_t - x^\star \rangle]}{\sum_t w_i}$	Thống nhất Polyak averaging, Primal averaging/heavy-ball, lịch linear-decay thành một khung duy nhất
Theorem 3 (learning rate lớn)	$\gamma = D/G$ (hằng số, không giảm theo t) vẫn tối ưu nếu Eq. 15 thoả	Giải thích tại sao bước lớn hơn lý thuyết cổ điển gợi ý lại hoạt động tốt trong thực hành
Trọng số c_{t+1} có warmup	$c_{t+1} = \gamma_t^2 \big/ \sum_{i=1}^{t} \gamma_i^2$	Tránh các bước đầu (lr nhỏ, gradient kém tin cậy) có trọng số quá lớn trong trung bình x

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Algorithm 1 (Schedule-Free AdamW)	xem Mục 3.3	Thay EMA-momentum bậc một của AdamW bằng nội suy x, y, z; giữ nguyên EMA bậc hai v_t
Kết quả đầu mục	Thắng track Self-Tuning của MLCommons AlgoPerf 2024; vượt/hoà cosine trên 7-9 bài toán deep learning	Bằng chứng thực nghiệm chính cho tuyên bố ”khớp hoặc vượt lịch đã tinh chỉnh kỹ”

11. Kết luận ngắn

Schedule-Free learning giải quyết một mâu thuẫn lý thuyết-thực hành lâu đời (trung bình Polyak-Ruppert tối ưu về lý thuyết nhưng tệ trong thực hành; lịch learning rate tốt trong thực hành nhưng cần biết trước T) bằng một dạng nội suy momentum mới giữa ba dãy x, y, z, dựa trên một định lý online-to-batch conversion tổng quát (Theorem 2) thống nhất nhiều kết quả lý thuyết trước đó. Phương pháp giữ nguyên tốc độ hội tụ tối ưu lý thuyết cho hàm lồi (Theorem 1) với mọi giá trị momentum β, không cần thêm hyperparameter so với optimizer gốc (ngoài β, vốn đã có ý nghĩa quen thuộc như momentum chuẩn), và không cần biết trước tổng số bước huấn luyện. Thực nghiệm trên 28 bài toán (từ hồi quy logistic lồi tới GPT-2/ImageNet/AlgoPerf) cho thấy Schedule-Free khớp hoặc vượt các lịch learning rate đã tinh chỉnh kỹ, và thắng track Self-Tuning của cuộc thi AlgoPerf 2024. Hạn chế chính mà paper tự nêu: vẫn cần dò learning rate/weight decay riêng (giá trị tối ưu khác với khi dùng lịch), và lý thuyết hiện tại mới ở mức sơ bộ, chỉ bao phủ trường hợp lồi. Đối với dự án nén/huấn luyện LightOnOCR-3-0.8B trong 8GB/batch 1, Schedule-Free AdamW là lựa chọn có chi phí thử nghiệm thấp nhất trong các optimizer đã đọc ở opt 10-17 (không tốn bộ nhớ thêm so với AdamW), và giải quyết trực tiếp vấn đề thực tế ”không biết trước số bước fine-tune/nén cần thiết” của một dự án nghiên cứu cá nhân, dù cần tự kiểm chứng ở batch size 1 vì paper chỉ thử nghiệm ở batch vừa/lớn.

Nguồn gốc: https://arxiv.org/abs/2405.15682

Bài 18

SGDR: Stochastic Gradient Descent with Warm Restarts (2016)

Tác giả: Ilya Loshchilov, Frank Hutter (University of Freiburg).
Venue / arXiv: arXiv:1608.03983 (v5, 03/05/2017); venue: Published as a conference paper at ICLR 2017.
Nguồn: https://arxiv.org/abs/1608.03983
Chủ đề: lịch learning rate dạng ”cosine annealing với warm restart” cho SGD: định kỳ tăng lại learning rate rồi giảm theo cosine, giúp cải thiện hiệu suất ”anytime” (đạt kết quả tốt sớm hơn) khi huấn luyện mạng sâu.
Vị trí trong lộ trình: đọc sau opt 06 (Adam, optimizer nền để so sánh lịch lr) và cùng nhóm với opt 19 (Super-Convergence, cũng dùng lịch lr dạng cycle); đọc trước opt 20 (MiniCPM, dùng lịch WSD thay cosine) và opt 17 (Road Less Scheduled, phê bình việc phải biết trước tổng số bước mà SGDR cần).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Huấn luyện mạng sâu (DNN) thường tốn nhiều ngày trên GPU mạnh, nên bất kỳ cách nào giúp hội tụ nhanh hơn đều có giá trị. Bài toán là tối thiểu hoá $f : \mathbb{R}^n \to \mathbb{R}$ bằng gradient descent ngẫu nhiên (SGD):

$$x_{t+1} = x_t - \eta_t \nabla f_t(x_t) \tag{1}$$

Lý tưởng muốn dùng thông tin bậc hai $x_{t+1} = x_t - \eta_t H_t^{-1} \nabla f_t(x_t)$ (2), nhưng nghịch đảo Hessian H_t^{-1} không khả thi với n lớn (hàng triệu tham số), nên các phương pháp thực tế (AdaDelta, Adam) chỉ xấp xỉ phần chéo của Hessian để có learning rate thích ứng.

Tại thời điểm paper này (2016), các kết quả SOTA trên CIFAR-10/100, SVHN, ImageNet, PASCAL VOC, MS COCO lại đến từ Residual Network dùng SGD với Nesterov’s momentum (không dùng Adam/AdaDelta):

$$v_{t+1} = \mu_t v_t - \eta_t \nabla f_t(x_t), \qquad x_{t+1} = x_t + v_{t+1} \tag{3,4}$$

Khó khăn chính là chọn lịch learning rate η_t và lượng weight decay. Lịch phổ biến nhất lúc đó (”default” của Zagoruyko & Komodakis 2016, Wide ResNet - WRN): $\eta_0 = 0.1$, giảm theo hệ số 0.2 tại các epoch 60, 120, 160, tổng ngân sách 200 epoch cố định. Vấn đề: phải biết trước tổng số epoch và các mốc giảm, và hiệu suất ”anytime” (giữa chừng huấn luyện) kém vì phần lớn thời gian lr vẫn cao.

Restart trong tối ưu không gradient: CMA-ES và các phương pháp niching dùng restart để dò nhiều cực trị của hàm đa cực trị, nhưng không scale với số chiều lớn.

Restart trong tối ưu có gradient: Fletcher & Reeves (1964), Powell (1977) restart conjugate gradient theo chu kỳ hoặc khi mất tính trực giao. O’Donoghue & Candes (2012) chỉ ra: với gradient accelerated scheme (Nesterov), nghiệm dao động theo chu kỳ tỉ lệ $\sqrt{\text{số điều kiện (condition number)}}$ của hàm lồi trơn; restart đúng chu kỳ này đạt tốc độ hội tụ tuyến tính tối ưu. Vì số điều kiện không biết trước và có thể thay đổi, họ đề xuất 2 kiểu restart thích ứng: restart khi hàm mục tiêu tăng lên (”function scheme”), hoặc khi góc giữa momentum và gradient âm là góc tù (”gradient scheme”). Smith (2015, 2016) đưa ra cyclical learning rate, gần về tinh thần với SGDR nhưng không tập trung vào ”restart” (không coi mỗi chu kỳ là một lần bắt đầu lại).

2. Ý tưởng chính

SGDR mô phỏng ”warm restart” (khởi động lại nhưng KHÔNG từ đầu, giữ nguyên x_t hiện tại, chỉ tăng lại learning rate) theo chu kỳ cố định hoặc tăng dần. Trong mỗi chu kỳ (run) thứ i, learning rate giảm từ η^i_{max} xuống η^i_{min} theo cosine annealing. Khi hết chu kỳ, lr được ”bật” lại lên η^i_{max} (restart), dùng tiếp tham số x_t đã học được làm điểm khởi đầu mới (không reset trọng số). Hai biến thể chính: (a) chu kỳ cố định T_0 (restart đều đặn mỗi T_0 epoch); (b) chu kỳ tăng dần, T_0 ban đầu rồi nhân T_{mult} sau mỗi lần restart, để vừa hội tụ nhanh ở các chu kỳ đầu (tốt cho anytime performance) vừa có chu kỳ dài ở cuối (tinh chỉnh kỹ). Kết quả thực nghiệm: SGDR đạt kết quả tương đương hoặc tốt hơn lịch ”default” nhưng chỉ cần 2-4 lần ít epoch hơn, và các ”snapshot” mô hình ngay trước mỗi restart có thể ghép thành ensemble gần như miễn phí, cho SOTA mới trên CIFAR-10 (3.14%) và CIFAR-100 (16.21%).

3. Phương pháp chi tiết

3.1 Công thức cosine annealing với warm restart (eq. 5)

Trong chu kỳ (run) thứ i:

$$\boxed{\eta_t = \eta^i_{min} + \frac{1}{2}(\eta^i_{max} - \eta^i_{min})\Big(1 + \cos\big(\frac{T_{cur}}{T_i}\pi\big)\Big)} \tag{5}$$

- $\eta^i_{max}, \eta^i_{min}$: biên trên/dưới của learning rate trong chu kỳ i (paper giữ nguyên không đổi qua các i ”để giảm số siêu tham số”, tuy ghi rõ ”có thể sẽ hữu ích” nếu giảm dần theo i).
- T_i: số epoch của chu kỳ i (độ dài run).
- T_{cur}: số epoch đã trôi qua từ lần restart gần nhất; **cập nhật ở mỗi batch** (không phải mỗi epoch) nên có thể nhận giá trị không nguyên như 0.1, 0.2...
- Tại $T_{cur} = 0$ (vừa restart): $\eta_t = \eta^i_{max}$. Tại $T_{cur} = T_i$ (cuối chu kỳ): $\cos(\pi) = -1 \Rightarrow \eta_t = \eta^i_{min}$.

Ví dụ tự tính (minh hoạ công thức 5, dùng $\eta_{max} = 0.05$, $\eta_{min} = 0$, $T_i = 10$, giống thiết lập EEG của paper):

T_{cur}	$T_{cur}/T_i \cdot \pi$	$\cos(\cdot)$	$\eta_t = 0.5 \times 0.05 \times (1 + \cos)$
0	0	1	0.0500
2.5	$\pi/4 \approx 0.785$	0.7071	0.0427
5	$\pi/2 \approx 1.571$	0	0.0250
7.5	$3\pi/4 \approx 2.356$	-0.7071	0.0073
10	π	-1	0.0000

(tự tính, theo đúng eq. 5; khớp với mô tả ”$\eta_t = \eta_{max}$ khi $t = 0$” và ”$\eta_t = \eta_{min}$ khi $T_{cur} = T_i$” mà paper nêu.)

3.2 Tăng chu kỳ theo T_{mult} và tổng epoch tích luỹ

Khi dùng T_0, T_{mult}: chu kỳ thứ i (đếm từ 1) có độ dài $T_i = T_0 \cdot T_{mult}^{i-1}$. Thời điểm restart (tính theo epoch tích luỹ từ đầu) là $\sum_{j=1}^{i} T_j = T_0 \dfrac{T_{mult}^i - 1}{T_{mult} - 1}$ (khi $T_{mult} \neq 1$).

Ví dụ tự tính, đối chiếu với thí nghiệm ensemble của paper (mục 4.3): paper dùng $T_0 = 10, T_{mult} = 2$ làm mô hình gốc cho ensemble, và lấy $M = 3$ snapshot ”ngay trước 3 lần restart cuối” tại epoch 150, 70, 30. Kiểm lại bằng công thức: $T_1 = 10$ (restart tại epoch tích luỹ 10), $T_2 = 10 \cdot 2 = 20$ (restart tại $10 + 20 = 30$), $T_3 = 10 \cdot 4 = 40$ (restart tại $30 + 40 = 70$), $T_4 = 10 \cdot 8 = 80$ (restart tại $70 + 80 = 150$). Vậy các mốc restart tích luỹ là epoch 10, 30, 70, 150 (tự tính từ eq. $T_i = T_0 T_{mult}^{i-1}$) - khớp chính xác với 3 mốc 30, 70, 150 mà paper dùng làm điểm lấy snapshot (bỏ mốc đầu tiên 10 vì chỉ lấy $M = 3$ snapshot cuối). Điều này xác nhận cách đọc đúng công thức tăng chu kỳ.

3.3 Quy tắc chọn nghiệm trả về (recommendation)

Vì bản thân việc tăng lr để restart thường làm xấu tạm thời hiệu suất, paper **không** luôn lấy x_t cuối cùng làm nghiệm đề xuất. Trong run đầu tiên (trước restart đầu), nghiệm đề xuất là x_t cuối của run đó; sau đó, nghiệm đề xuất là x_t tại cuối lần run **đã hoàn thành gần nhất** (ở $\eta_t = \eta_{min}^i$), không phải x_t hiện tại có thể đang giữa một chu kỳ mới (lr đang cao). Nhờ vậy phương pháp không cần một tập validation riêng để chọn điểm dừng.

3.4 Thiết lập thực nghiệm chính

Wide ResNet (WRN-d-k, d=độ sâu, k=hệ số mở rộng độ rộng) trên CIFAR-10/100 (50.000 train, 10.000 test, ảnh 32x32). Baseline (Zagoruyko & Komodakis 2016): SGD + Nesterov momentum, $\eta_0 = 0.1$, weight decay 0.0005, momentum 0.9, batch 128, giảm lr theo hệ số 0.2 tại epoch 60/120/160, tổng 200 epoch. SGDR dùng chính xác budget 200 epoch nhưng chia theo T_0, T_{mult} khác nhau (ví dụ $T_0 = 50, 100, 200$ với $T_{mult} = 1$; hoặc $T_0 = 1$ hay 10 với $T_{mult} = 2$, khi đó $200 = T_0(2^n - 1)$ xác định số lần restart n).

4. Thí nghiệm và kết quả chính

4.1 CIFAR-10 / CIFAR-100, mô hình đơn (Bảng 1, trích)

Cấu hình	WRN-28-10, CIFAR-10	WRN-28-10, CIFAR-100	WRN-28-20, CIFAR-10	WRN-28-20, CIFAR-100
default $\eta_0 = 0.1$	4.24	20.33	4.08	19.53
default $\eta_0 = 0.05$	4.13	20.21	3.96	19.67
$T_0 = 50, T_{mult} = 1$	4.17	19.99	4.01	19.28
$T_0 = 100, T_{mult} = 1$	4.07	19.87	3.77	19.24
$T_0 = 200, T_{mult} = 1$	3.86	19.98	3.66	19.69
$T_0 = 1, T_{mult} = 2$	4.09	19.74	3.91	18.90
$T_0 = 10, T_{mult} = 2$	4.03	19.58	3.74	18.70

(đơn vị: % lỗi test, median của 5 lần chạy cho WRN-28-10, median của 2 cho WRN-28-20; số liệu copy nguyên văn Bảng 1). Paper nhấn mạnh: $T_0 = 200, T_{mult} = 1$ (tức một chu kỳ cosine kéo dài cả 200 epoch, **không có restart nào**) đạt test error tốt nhất trên CIFAR-10 trong nhóm WRN-28-10 (3.86%), nhưng lại có anytime performance **tệ nhất** (vì phải đợi hết cả 200 epoch lr mới về 0); ngược lại $T_0 = 1, T_{mult} = 2$ đạt error tương đương default chỉ sau khoảng 1/2 đến 1/4 số epoch.

4.2 Ensemble từ snapshot (Bảng 2, mục 4.3)

Lấy M snapshot ngay trước các lần restart của một lần chạy SGDR $T_0 = 10, T_{mult} = 2$ trên WRN-28-10, cộng xác suất softmax các snapshot với trọng số đều:

Cấu hình	CIFAR-10	CIFAR-100
$N = 1$ run, $M = 1$ (median 16 run)	4.03	19.57
$N = 1$, $M = 3$ (epoch 30,70,150)	3.51	17.75
$N = 3$ run, $M = 3$	3.25	16.64
$N = 16$ run, $M = 3$	**3.14**	**16.21**

Đây là 2 số được nêu ngay ở abstract làm SOTA mới tại thời điểm paper. Điểm thú vị: $N = 3, M = 3$ (9 model tổng) cho kết quả tốt hơn cả $N = 18, M = 1$ hay $N = 21, M = 1$ (nhiều model hơn nhưng không lấy từ cùng một quỹ đạo SGDR); nếu chỉ lấy 3 snapshot ở 3 epoch cuối liên tiếp (148,149,150, không phải 3 mốc trước restart) thì **không** cải thiện so với $M = 1$, khẳng định chính các điểm ”trước restart” mới tạo đa dạng có ích.

4.3 EEG và ImageNet thu nhỏ

- **EEG** (14 subjects, ~1000 trial/người, kiến trúc CNN của Schirrmeister et al. 2017): lỗi tham chiếu (median) khoảng 9%; dùng snapshot của 1 lần chạy cải thiện 1-2 điểm %, dùng snapshot từ cả 2 thiết lập siêu tham số ($N = 2$) cải thiện 2-3 điểm % (paper không cho số tuyệt đối chính xác hơn, chỉ nêu khoảng).
- **ImageNet thu nhỏ về 32x32** (giữ đủ 1000 lớp, chỉ downsample ảnh, khác với Tiny-ImageNet trước đó): SGDR $T_0 = 10, T_{mult} = 2$, $\eta_{max} = 0.01$ đạt top-1 39.24%, top-5 17.17%, ”khớp” kết quả AlexNet gốc (40.7%/18.2%) dù ảnh có khoảng 50 lần ít pixel hơn.

5. Phân tích / ablation

- **Overfitting (Hình 7, phụ lục):** SGDR giảm training loss nhanh hơn default tới khoảng epoch 120; sau đó default bắt đầu overfit (test error tăng) còn SGDR ”chỉ overfit rất nhẹ” (nguyên văn ”very mild overfitting”).
- **50k vs 100k mẫu/epoch (mục 8.1, phụ lục):** vì data augmentation (lật ảnh) nhân đôi số mẫu train trong code kế thừa, paper kiểm tra riêng trên WRN-28-1 với epoch định nghĩa theo đúng 50k mẫu gốc; SGDR vẫn cho anytime performance tốt (Hình 6), nên kết luận không bị nhiễu bởi định nghĩa epoch.
- **Số siêu tham số:** trong trường hợp đơn giản nhất (1 chu kỳ dài, không restart, chính là trường hợp $T_0 =$ tổng epoch, $T_{mult} = 1$), SGDR chỉ cần 2 siêu tham số: learning rate khởi đầu và tổng số epoch, ít hơn lịch ”giảm theo mốc” (cần chọn các mốc giảm và hệ số giảm).
- **ImageNet thu nhỏ, permute theo subgroup (Hình 8, phụ lục):** khi hoán vị nhãn chỉ trong 10 nhóm con (khó tổng quát hoá hơn), SGDR vẫn tốt hơn default; paper diễn giải: learning rate khởi đầu ”rất quan trọng” nhưng SGDR giảm độ nhạy với việc chọn sai nó, vì nó quét (anneal) qua một dải giá trị lr từ max về 0 trong mỗi chu kỳ.

6. Giới hạn

Tác giả tự nêu:

- Không khẳng định quan sát được hiệu ứng đa cực trị (multi-modality) nào thực sự được giải quyết bởi restart, dù đó là động lực ban đầu của kỹ thuật restart nói chung.
- Giữ $\eta^i_{max}, \eta^i_{min}$ không đổi qua các i ”để giảm số siêu tham số”, nhưng tự nhận ”có thể hữu ích” nếu giảm dần theo từng lần restart.
- Thí nghiệm ImageNet thu nhỏ chỉ là ”preliminary” (sơ bộ), 40 epoch, không tinh chỉnh kỹ siêu tham số.
- Không có tập validation riêng dùng để chọn η_{max} tối ưu; nếu dùng, ”có thể cải thiện kết quả thêm”.
- Chưa thử kết hợp warm restart với AdaDelta hay Adam (chỉ thử với SGD+momentum); để lại cho ”future work”.

Người ghi chú nhận xét:

- Việc ”default $\eta_0 = 0.1$” được paper dùng làm đối chứng chính lại kém hơn ”default $\eta_0 = 0.05$” (Bảng 1: 4.24 vs 4.13 trên WRN-28-10/CIFAR-10), cho thấy một phần lợi ích đo được của SGDR có thể lẫn với việc đơn giản là chọn η_{max} phù hợp hơn baseline gốc, không chỉ nhờ cơ chế restart; paper có ý thức về điều này (dùng cả hai baseline) nhưng không tách bạch hoàn toàn 2 hiệu ứng.
- $T_0 = 200, T_{mult} = 1$ (không restart) lại cho kết quả **tốt nhất** ở một số cấu hình (Bảng 1), nghĩa là bản thân cosine annealing (không cần restart) đã là một cải tiến so với ”giảm theo mốc”; phần ”restart” chủ yếu đóng góp vào anytime performance và khả năng tạo ensemble miễn phí, chứ không chắc luôn cho test error tốt nhất.
- Tất cả so sánh số trong Bảng 1 không có kiểm định thống kê (paper tự ghi ”Bold text ... is not based on statistical tests (too few runs)”), nên các khác biệt nhỏ (dưới 0.2 điểm %) cần được xem là chỉ tham khảo.

7. Liên hệ với các bài khác

- **opt 06 (Adam):** paper chỉ thử nghiệm với SGD+momentum, nêu rõ ”future work” nên thử warm restart với AdaDelta/Adam; các framework hiện đại (`CosineAnnealingWarmRestarts` trong PyTorch) áp dụng được với mọi optimizer bậc một, kể cả Adam/AdamW.
- **opt 19 (Super-Convergence):** cũng dùng một lịch learning rate dạng hình học theo chu kỳ (ở đó là ”1cycle”, một tam giác/cosine duy nhất với lr lớn ở giữa), nhưng động lực khác: Smith coi lr lớn là một dạng regularization, còn SGDR coi mỗi chu kỳ là một ”run” tối ưu độc lập có thể tạo snapshot.
- **opt 17 (Road Less Scheduled):** phê bình trực tiếp các lịch như cosine/SGDR vì cần biết trước tổng số bước T (ở đây là T_0, T_{mult} và số lần restart); đề xuất lịch ”schedule-free” không cần thông tin đó.
- **opt 20 (MiniCPM):** dùng lịch Warmup-Stable-Decay (WSD) thay cho cosine/restart, cho phép lấy checkpoint tốt ở bất kỳ thời điểm sau một đoạn ”decay” ngắn, giải quyết đúng nhược điểm ”phải biết trước budget” mà cả cosine và SGDR gặp phải.
- **distillation content/ (thư mục cũ):** đã kiểm tra bằng `ls`, các file trong thư mục này (ví dụ các ghi chú về recovery sau pruning) không đề cập trực tiếp SGDR nên không trỏ số bài cụ thể ở đây để tránh gán sai.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU ≤ 8 GB, batch 1). Đây là suy luận của người ghi chú, không phải kết luận của paper:

1. **Warm restart cho fine-tune/distill nhiều giai đoạn:** nếu quy trình phục hồi sau pruning gồm nhiều ”vòng” (ví dụ: cắt một phần rồi fine-tune, lặp lại), mỗi vòng fine-tune có thể dùng một chu kỳ cosine riêng (giống một ”run” của SGDR) thay vì tiếp tục giảm lr tuyến tính xuyên suốt; điều này giữ đúng tinh thần ”mỗi lần fine-tune lại là một bài toán tối ưu mới cần quét lại một dải lr”.
2. **Snapshot ensemble không hợp với batch 1 / inference đơn mô hình:** lợi ích lớn nhất của SGDR trong paper (ensemble gần miễn phí) đòi hỏi giữ nhiều snapshot và suy luận nhiều lần, mâu thuẫn với mục tiêu một mô hình nhỏ, batch 1, GPU hạn chế của dự án; phần này ”ít liên quan” trực tiếp.
3. **Công thức cosine annealing (eq. 5) vẫn hữu ích** như lịch lr đơn giản (không cần restart, tức $T_{mult} = 1$ và chỉ 1 chu kỳ) cho các lần fine-tune ngắn trên GPU nhỏ, vì chỉ cần 2 siêu tham số (lr đầu, tổng bước) như paper nêu ở mục 5, phù hợp khi không có ngân sách để dò nhiều mốc giảm.
4. **Quy tắc ”không lấy x_t cuối cùng mà lấy x_t cuối chu kỳ đã hoàn thành”** là một gợi ý thực hành tốt: khi checkpoint cuối cùng của một lần train/distill rơi đúng lúc lr đang cao (ví dụ do dừng giữa chừng), nên quay lại checkpoint gần nhất tại lr thấp để đánh giá, tránh đánh giá nhầm một điểm ”chưa hội tụ cục bộ”.

9. Thí nghiệm gợi ý

1. **Tái tạo công thức cosine annealing trên toy model:** huấn luyện một MLP nhỏ (hoặc CNN nhỏ) trên MNIST/CIFAR-10 con với $T_0 = 10, T_{mult} = 2$ so với lịch ”giảm theo mốc” cùng tổng số epoch; đo test error theo epoch (đường cong ”anytime”) và kiểm tra có đạt error thấp nhanh hơn ở các epoch giữa không, như Hình 2 của paper.
2. **Kiểm tra lợi ích snapshot ensemble ở quy mô nhỏ:** với cùng cấu hình trên, lấy 3 snapshot ngay trước 3 lần restart cuối (tính theo công thức mục 3.2), ghép ensemble bằng trung bình softmax, so với 1 model đơn; đo mức giảm error, kỳ vọng cải thiện nhỏ hơn paper (do model/dataset nhỏ hơn rất nhiều) nhưng cùng chiều.
3. **Ablation không-restart ($T_{mult} = 1$, $T_0 =$ tổng epoch) vs có-restart:** trên cùng 1 GPU, so sánh test error cuối cùng và tốc độ hội tụ giữa một chu kỳ cosine dài không restart và nhiều restart ngắn cùng tổng epoch, để tự kiểm chứng nhận xét ở mục 6 rằng ”restart chủ yếu lợi ở anytime performance, không

chắc lợi ở error cuối".

4. **Warm restart với Adam/AdamW:** thử đúng công thức (5) nhưng optimizer là AdamW thay SGD+momentum (paper không thử điều này), đo xem hành vi "giảm nhẹ rồi tăng lại lr" có gây mất ổn định với bộ nhớ bậc hai của Adam (second moment) hay không, so với SGD.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Cosine annealing + warm restart	$\eta_t = \eta^i_{min} + \frac{1}{2}(\eta^i_{max} - \eta^i_{min})(1 + \cos(\frac{T_{cur}}{T_i}\pi))$	eq. (5); T_{cur} cập nhật theo batch
Độ dài chu kỳ i	$T_i = T_0 \cdot T_{mult}^{i-1}$	$T_{mult} = 1$: chu kỳ cố định
Epoch tích luỹ khi restart thứ i	$\sum_{j \le i} T_j = T_0 \dfrac{T_{mult}^i - 1}{T_{mult} - 1}$	tự tính, khớp mốc 10/30/70/150 ở mục 3.2
Nghiệm đề xuất	x_t tại cuối run **đã hoàn thành** gần nhất ($\eta = \eta^i_{min}$)	không cần tập validation riêng
Số siêu tham số tối thiểu	2 (lr khởi đầu, tổng số epoch)	trường hợp 1 chu kỳ, không restart
Snapshot ensemble	lấy M model ngay trước M restart cuối, trung bình softmax	theo Huang et al. (2016a), "free" vì không cần train thêm
Kết quả SOTA chính	CIFAR-10 3.14%, CIFAR-100 16.21%	$N = 16$ run, $M = 3$ snapshot/run

11. Kết luận ngắn

SGDR thay lịch "giảm lr theo mốc cố định" bằng cosine annealing lặp lại theo chu kỳ, trong đó mỗi lần hết chu kỳ lr được bật lại (warm restart) mà không reset trọng số; chu kỳ có thể cố định ($T_{mult} = 1$) hoặc tăng dần ($T_{mult} > 1$) để vừa có anytime performance tốt vừa có giai đoạn tinh chỉnh dài cuối cùng. Thực nghiệm trên WRN cho CIFAR-10/100 cho thấy tốc độ hội tụ nhanh hơn 2-4 lần so với lịch default ở cùng chất lượng, và các snapshot ngay trước restart ghép lại thành ensemble gần miễn phí, đạt SOTA 3.14%/16.21%. Ý tưởng cosine annealing (không nhất thiết cần restart) sau này trở thành lịch lr mặc định phổ biến nhất cho huấn luyện mạng sâu và LLM, trước khi bị các lịch linh hoạt hơn như WSD (opt 20) hay schedule-free (opt 17) thách thế vì nhược điểm phải biết trước tổng số bước.

Nguồn gốc: https://arxiv.org/abs/1608.03983

Bài 19

Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates (2017)

Tác giả: Leslie N. Smith (U.S. Naval Research Laboratory), Nicholay Topin (University of Maryland, Baltimore County).
Venue / arXiv: arXiv:1708.07120 (v3, 17/05/2018); paper không nêu venue hội nghị/tạp chí, tự ghi chú ngay dưới Hình 1 là ”Preprint. Work in progress.”
Nguồn: https://arxiv.org/abs/1708.07120
Chủ đề: hiện tượng ”super-convergence”: dùng cyclical learning rate (CLR) với một chu kỳ duy nhất (”1cycle”) và learning rate tối đa rất lớn giúp huấn luyện nhanh hơn một bậc độ lớn và đôi khi chính xác hơn, kèm lý giải rằng lr lớn đóng vai trò regularization nên phải giảm các regularization khác.
Vị trí trong lộ trình: đọc sau opt 18 (SGDR, cũng dùng cosine theo chu kỳ nhưng paper này chứng minh thực nghiệm rằng mẫu sawtooth của SGDR KHÔNG tạo ra super-convergence); cùng nhóm với opt 06 (Adam, được paper so sánh và không đạt super-convergence); đọc trước opt 46/47 (hướng dẫn thực hành lr/batch/momentum/weight decay, cùng tác giả Leslie Smith) và opt 24 (Keskar, sharp minima, mà kết quả batch size ở đây đi ngược lại gợi ý của Keskar).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Lịch learning rate ”điển hình” (standard/piecewise-constant) mà paper mô tả: dùng một lr toàn cục (ví dụ ≈ 0.1) trong nhiều epoch tới khi test accuracy chững lại (plateau), rồi giảm lr theo hệ số 0.1 và tiếp tục; lặp lại 2-3 lần. Cách này an toàn nhưng chậm.

Cyclical Learning Rate (CLR) và LR range test đã được Smith (2015, công bố chính thức 2017) giới thiệu trước đó. Paper này quan sát: với một số giá trị siêu tham số, dùng CLR với lr **cực lớn** (gấp khoảng 10 lần giá trị ”điển hình”) có thể huấn luyện nhanh hơn một bậc độ lớn, hiện tượng được đặt tên là ”super-convergence”. Ví dụ mở đầu (Hình 1, Resnet-56 trên Cifar-10): lịch piecewise-constant đạt đỉnh 91.2% sau khoảng 80.000 iteration; super-convergence đạt 92.4% chỉ sau 10.000 iteration. Với một biến thể CLR khác đạt lr tối đa 3 (gấp ~10 lần lr thường dùng khoảng 0.35), đạt test accuracy 92.1% chỉ sau 6.000 iteration (so với 91.2% của lịch điển hình); khi tăng tổng số iteration từ 2.000 lên 20.000, accuracy cải thiện từ 89.7% lên 92.7%.

Bốn đóng góp paper tự liệt kê: (1) khảo sát hệ thống một phương pháp huấn luyện mới nhanh và tốt hơn; (2) chứng minh lr lớn có tác dụng regularization, nên phải giảm các dạng regularization khác để giữ cân bằng; (3) suy ra một bản đơn giản hoá của phương pháp Hessian-free bậc hai để ước lượng lr tối ưu, cho thấy lr lớn tìm được minima rộng và phẳng; (4) hiệu ứng super-convergence càng rõ rệt khi dữ liệu có nhãn càng hạn chế.

Paper nêu rõ việc dùng lr lớn này **đi ngược** gợi ý trong literature về một giá trị lr tối đa an toàn (Bottou et al. 2016, cùng paper với opt 02 Optimization Methods for Large-Scale ML).

2. Ý tưởng chính

CLR cần 2 biên lr (min, max) và một ”stepsize” (số iteration cho mỗi nửa chu kỳ: tăng hoặc giảm). **LR range test**: chạy huấn luyện với lr tăng dần tuyến tính từ rất nhỏ tới lớn trong một lần chạy thử; vẽ test accuracy theo lr. Đường cong thường có một đỉnh rõ: đó là lr tối đa nên dùng cho CLR; lr tối thiểu lấy bằng lr tối đa chia 3 hoặc 4. Nếu đường cong **không giảm** mà vẫn giữ accuracy cao trên một dải lr ”bất thường lớn” (ví dụ tới 3.0, gấp 10 lần giá trị thường dùng), đó là dấu hiệu tiềm năng super-convergence.

1cycle: biến thể CLR đề xuất riêng cho super-convergence: dùng **đúng một chu kỳ**, có độ dài **nhỏ hơn** tổng số iteration/epoch huấn luyện, và cho phần iteration còn lại, giảm tiếp lr xuống **vài bậc độ lớn thấp hơn** giá trị lr khởi đầu (gọi là giai đoạn ”annihilation”). Đây khác CLR gốc (lặp nhiều chu kỳ cố định biên trên/dưới) và khác SGDR (giảm cosine rồi **nhảy lại lên** giá trị ban đầu). Paper khẳng định: thử đúng mẫu sawtooth của SGDR (Loshchilov & Hutter, 2016) thì ”không thể quan sát hiện tượng super-convergence”.

Lý giải bằng lý thuyết regularization: coi lr lớn, batch nhỏ, weight decay, dropout đều là các dạng ”nhiễu”/regularization; tổng lượng regularization cho một bài toán/kiến trúc phải được cân bằng. Nếu tăng lr rất lớn (một dạng regularization mạnh) mà không giảm các dạng khác (weight decay, dropout...) thì sẽ underfit; giảm đúng mức các regularization khác sẽ cho phép lr lớn phát huy tác dụng và huấn luyện hiệu quả hơn nhiều.

3. Phương pháp chi tiết

3.1 Cơ sở toán học: xấp xỉ Hessian-free để ước lượng lr tối ưu

Gradient descent cơ bản: $x = x_0 - \epsilon \nabla_x f(x)$ (1). Xấp xỉ bậc hai cục bộ của loss (Martens 2010, Hessian-free optimization):

$$f(\theta) \approx f(\theta_0) + (\theta - \theta_0)^T \nabla_\theta f(\theta_0) + \frac{1}{2}(\theta - \theta_0)^T H (\theta - \theta_0) \tag{3}$$

Vì tính và lưu H với N tham số tốn $\Omega(N^2)$ là bất khả thi, Martens (2010) dùng xấp xỉ sai phân hữu hạn theo hướng gradient:

$$H(\theta) = \lim_{\delta \to 0} \frac{\nabla f(\theta + \delta) - \nabla f(\theta)}{\delta} \tag{6}$$

AdaSecant (Gulcehre et al.) suy ra lr tối ưu cho mỗi neuron $\epsilon^* \approx \dfrac{\theta_{i+1} - \theta_i}{\nabla f(\theta_{i+1}) - \nabla f(\theta_i)}$ (7); viết lại theo 3 giá trị trọng số liên tiếp:

$$\boxed{\epsilon^* = \frac{\theta_{i+1} - \theta_i}{2\theta_{i+1} - \theta_i - \theta_{i+2}}} \tag{8}$$

trong đó ϵ bên phải là lr **thực tế** đã dùng để cập nhật trọng số. Để có một lr toàn cục từ các lr riêng từng trọng số, paper lấy tổng (giá trị tuyệt đối, khác Schaul et al. 2013 dùng bình phương) của tử và mẫu công thức (8) trên toàn bộ trọng số.

Paper tính thử ϵ^* trên Resnet-56/Cifar-10 cho 2 lần chạy: lr cố định 0.1, và CLR trong khoảng 0.1-3 với stepsize 5.000 iteration. Lấy trung bình trượt $LR = \alpha\epsilon^* + (1-\alpha)LR$, $\alpha = 0.1$. Kết quả (Hình 3): lr tối ưu ước lượng nằm trong khoảng **2 đến 6**; với lr cố định, ước lượng ban đầu tăng vọt tới khoảng 3 rồi tụt về gần 0.2 (300-10.000 iteration), còn với CLR ước lượng giữ cao tới cuối, dừng ở khoảng 0.5. Paper diễn giải: lr tối ưu lớn ⇔ giá trị Hessian xấp xỉ nhỏ ⇔ SGD đang ở vùng minima **rộng và phẳng** (flat, wide minima). Paper nói rõ **không** đánh giá đầy đủ tính hiệu quả của chính kỹ thuật ước lượng này, chỉ dùng để minh hoạ rằng lr lớn là hợp lý; để lại ”future work”.

3.2 Chính sách ”1cycle” cụ thể trong thí nghiệm

Ví dụ tự tính 1 (annihilation vượt quá biên dưới ban đầu): thí nghiệm Resnet-50/ImageNet dùng 1cycle 20 epoch, lr đi từ 0.05 lên 1.0 rồi xuống **0.00005**. Tỉ lệ lr đầu / lr cuối $= 0.05/0.00005 = 1000 = 10^3$ (tự tính): đúng với mô tả ”giảm vài bậc độ lớn thấp hơn giá trị khởi đầu” ở mục 3 của paper (ở đây là 3 bậc), không chỉ quay về đúng lr khởi đầu như một tam giác đối xứng thuần.

Ví dụ tự tính 2 (chu kỳ nhỏ hơn tổng ngân sách): với densenet trên Cifar-10 (Bảng 2, dòng cuối), paper dùng "0.1-4/70" (stepsize = 70 epoch mỗi nửa chu kỳ) trong tổng 150 epoch. Một chu kỳ đầy đủ (tăng + giảm) dài $2 \times 70 = 140$ epoch; phần còn lại cho annihilation là $150 - 140 = 10$ epoch (tự tính). Điều này khớp đúng định nghĩa 1cycle: "luôn dùng một chu kỳ nhỏ hơn tổng số iteration/epoch".

3.3 Chuẩn hoá batch (Batch Norm) và nhịp cập nhật thống kê

Paper phát hiện: tham số `moving_average_fraction` (MAF) mặc định của Caffe (0.999, phù hợp cho huấn luyện dài) ngăn cản super-convergence khi dùng `use_global_stats=true` ở test phase, vì huấn luyện quá nhanh nên thống kê toàn cục không kịp cập nhật. Dùng MAF nhỏ hơn (ví dụ 0.95-0.97 cho các thí nghiệm CLR 10.000-20.000 iteration) giải quyết vấn đề mà không cần "ghost statistics" (kỹ thuật của Hoffer et al. 2017, cũng được opt 21 Goyal et al. nhắc tới).

4. Thí nghiệm và kết quả chính

4.1 Cifar-10, Resnet-56, dữ liệu hạn chế (Bảng 1)

So sánh PC-LR=0.35 (piecewise constant, 80.000 iteration) với CLR(0.1-3) (10.000 iteration, BN MAF 0.95):

# mẫu train	PC-LR (80k iter)	CLR 0.1-3 (10k iter)	Chênh lệch
40.000	89.1%	91.1%	+2.0
30.000	85.7%	89.6%	+3.9
20.000	82.7%	87.9%	+5.2
10.000	71.4%	80.6%	+9.2

Mục 5 của paper (văn bản chính) cũng nêu: "super-convergence test accuracy tốt hơn 1.2%, 5.2% và 9.2% cho các trường hợp 50.000, 20.000, 10.000 mẫu train, tương ứng". Số 5.2% và 9.2% khớp với cột chênh lệch (tự tính) ở 20.000 và 10.000 mẫu trong Bảng 1; số 1.2% ứng với trường hợp 50.000 mẫu (toàn bộ Cifar-10) chính là chênh lệch 92.4% - 91.2% đã nêu ở Hình 1 (abstract). Paper dẫn nguồn các số này là "Bảng 6", nhưng Bảng 6 thực tế trong phụ lục lại là bảng ablation biên lr tối đa (1.0-3.5), **không phải** bảng dữ liệu hạn chế; đây là một lỗi trỏ nhầm số bảng của paper (xem mục 6). Kết luận chính: lợi ích của super-convergence **tăng lên** khi dữ liệu càng hạn chế.

4.2 Độ sâu mạng (Hình 4b)

Resnet với $l = 20 + 9n$ lớp ($n = 0, \ldots, 10$, tức 20 đến 110 lớp), CLR stepsize 10.000 iteration so với piecewise-constant lr=0.35: Resnet-20: CLR 90.4% so với 88.6% (chênh +1.8, tự tính); Resnet-110: CLR 92.1% so với 91.0% (chênh +1.1, tự tính). Cải thiện lớn hơn ở mạng **nông hơn**.

4.3 Kích thước batch (Hình 5)

Batch tổng chủ yếu dùng là 1.000 (chia 8 GPU, mỗi GPU 125). Paper quan sát: batch lớn hơn cho cải thiện nhỏ về accuracy (Hình 5a), và **generalization gap** (hiệu train-test accuracy) **gần như không đổi** giữa batch nhỏ và lớn (Hình 5b) - khác với các kết quả khác trong literature (paper dẫn trực tiếp Keskar et al. 2016, opt 24). Kết luận: "batch lớn hiệu quả hơn batch nhỏ cho super-convergence training" (ngược trực giác thông thường), vì batch lớn cho phép dùng lr lớn hơn.

4.4 Các kiến trúc/dataset khác (Bảng 2, trích một phần)

Dataset	Kiến trúc	Lịch	Epoch	Accuracy
Cifar-10	wide resnet	step, $\eta = 0.1$	800	90.3±1.0

Dataset	Kiến trúc	Lịch	Epoch	Accuracy
Cifar-10	wide resnet	1cycle 0.1-1/45	100	91.9±0.2
Cifar-10	densenet	step, $\eta = 0.1$	400	92.7±0.2
Cifar-10	densenet	1cycle 0.1-4/70	150	92.8±0.1
MNIST	LeNet	step, $\eta = 0.01$	85	99.00±0.04
MNIST	LeNet	1cycle 0.02-0.2/40	85	99.35±0.03
Cifar-100	resnet-56	step, $\eta = 0.005$	400	61.0±0.2
Cifar-100	resnet-56	1cycle 0.09-0.9/45	100	69.0±0.4

Điểm nổi bật: wide-resnet đạt 91.9% chỉ sau 100 epoch bằng 1cycle, so với 90.3% sau **800 epoch** của lịch step (8 lần số epoch); MNIST LeNet đạt 99.25-99.28% chỉ sau 12-25 epoch bằng 1cycle so với 99.00-99.03% sau 85 epoch của lịch gốc; Cifar-100 resnet-56 với 1cycle 100 epoch (69.0%) vượt xa lịch step dù lịch step chạy tới 400 epoch (61.0%), chênh tới 8 điểm % (tự tính).

4.5 ImageNet (Hình 6, mục 5.2)

- **Resnet-50:** baseline (Szegedy et al. 2017: momentum 0.9, lr=0.045 giảm expo 0.94 mỗi 2 epoch, WD=10^{-4}) đạt top-1 63.7% (đường cong còn đang tiến tới ~65%). 1cycle 20 epoch, lr $0.05 \to 1.0 \to 0.00005$, cần **giảm weight decay** xuống 3×10^{-6} đến 10^{-5} để dùng được lr lớn, đạt top-1 tốt nhất **67.6%** (chênh +3.9 điểm % tự tính so với baseline 63.7%, trong 20 so với nhiều hơn epoch).
- **Inception-resnet-v2:** baseline sau 100 epoch đạt 67.6% (ngoại suy tới 69-70%). 1cycle 20 epoch, lr $0.05 \to 1.0 \to 0.00005$, WD 3×10^{-6} đến 10^{-6}, tốt nhất 3×10^{-6} cho accuracy **74.0%**.

Cả hai trường hợp ImageNet đều có dấu hiệu overfit nhẹ ở cấu hình WD tốt nhất; paper diễn giải: ”một lượng overfit nhỏ là dấu hiệu tốt giúp tìm WD tối ưu sớm trong quá trình huấn luyện”.

5. Phân tích / ablation

- **Biên lr tối đa (Bảng 6, phụ lục):** CLR(0.1-3.5): 92.1%; (0.1-3): 92.4% (tốt nhất); (0.1-2.5): 92.3%; (0.1-2): 91.7%; (0.1-1.5): 90.9%; (0.1-1): 91.3%. Lr tối đa khoảng 3 là tốt nhất cho Resnet-56/Cifar-10 trong các giá trị thử.
- **BN MAF và số iteration (cùng Bảng 6):** MAF 0.97 (20.000 iter): 92.7%; 0.95 (10.000): 92.4%; 0.93 (8.000): 91.7%; 0.90 (6.000): 92.1%; 0.85 (4.000): 91.1%; 0.80 (2.000): 89.7%. Xu hướng chung: nhiều iteration hơn (MAF lớn hơn, cập nhật thống kê chậm hơn) cho kết quả tốt hơn, nhưng không đơn điệu tuyệt đối (0.90 > 0.93 ngược thứ tự).
- **Mini-batch size (Bảng 7, phụ lục, CLR=0.1-3, stepsize 5.000):** 1536: 92.1%; 1000: 92.4% (tốt nhất, cũng là batch dùng trong hầu hết thí nghiệm chính); 512: 91.7%; 256: 89.5%. GPU memory giới hạn không thử được batch > 1530.
- **Momentum (Bảng 8, phụ lục, cùng CLR):** 0.80: 92.1%; 0.85: 91.9%; 0.90: 92.4% (tốt nhất); 0.95: 90.7%.
- **Weight decay (Hình 12b):** WD=10^{-3} **ngăn** super-convergence xảy ra; 10^{-4} hoạt động tốt; các giá trị nhỏ hơn ($10^{-5}, 10^{-6}$) không ngăn. Minh chứng trực tiếp nguyên lý ”phải giảm regularization khác để bù cho lr lớn”.
- **Dropout (Hình 11):** super-convergence với dropout=0.2 cho cải thiện nhỏ so với không dropout.
- **Optimizer thích ứng (Hình 9, phụ lục):** với lịch piecewise-constant, không optimizer nào trong Nesterov, AdaDelta, AdaGrad, Adam tự ”phát hiện” lợi ích của lr lớn hay có dấu hiệu super-convergence. Khi chuyển sang 1cycle: Nesterov, AdaDelta, AdaGrad **đạt được** super-convergence (ví dụ Nesterov+1cycle: 92.1% sau 10.000 iteration so với 90.9% của piecewise-constant sau 80.000 iteration), nhưng **Adam thì không** (”we were unable to create this phenomenon with Adam”).

- **Bằng chứng regularization (Hình 8, phụ lục A.2):** trong LR range test, khi lr tăng từ khoảng 0.2 đến 2.0 (văn bản) / ”0.3 đến 1.5” (chú thích Hình 8, số liệu không khớp hoàn toàn giữa văn bản và caption, xem mục 6): training loss **tăng** còn test loss **giảm**, tức khoảng cách generalization giảm dần, bằng chứng trực tiếp cho vai trò regularization của lr lớn.

6. Giới hạn

Tác giả tự nêu:

- Đây là ”Preprint. Work in progress.” (tự ghi ngay trong bài), không công bố ở venue nào.
- Không đánh giá đầy đủ phương pháp ước lượng lr tối ưu bằng Hessian-free đơn giản hoá (mục 3.1), để lại ”future work”.
- Thừa nhận mâu thuẫn nội tại chưa giải quyết: kết quả super-convergence (ở giữa huấn luyện) khớp với giả thuyết ”minima rộng, phẳng tổng quát hoá tốt hơn” (Hochreiter & Schmidhuber 1997, Keskar et al. 2016), nhưng **cần lr nhỏ ở cuối huấn luyện**, ngụ ý minima cuối cùng lại **hẹp**; paper viết thẳng ”this needs to be reconciled” (cần được làm rõ, chưa có lời giải).
- Chưa có khung lý thuyết thống nhất cho ”nhiễu tốt” (good noise) so với ”nhiễu xấu” (bad noise) trong SGD; đề xuất 3 hướng nghiên cứu tương lai (khung thống nhất cho các loại nhiễu, áp dụng nhiễu biến đổi theo thời gian có hệ thống hơn, cách ổn định tối ưu hoá khi nhiễu rất lớn).
- Kết quả đi ngược gợi ý ”warm start bằng batch nhỏ rồi chuyển sang batch lớn” của Keskar et al. (2016); paper nói rõ dữ liệu của họ ”contradicts this suggestion” và đề xuất ngược lại: bắt đầu ít nhiễu, tăng nhiễu lên đỉnh, rồi giảm nhiễu ở cuối (curriculum + simulated annealing).

Người ghi chú nhận xét:

- Trích dẫn số liệu ở mục 5 (”Table 6”) không khớp với Bảng 6 thực tế trong phụ lục (đó là bảng ablation biên lr, không phải bảng dữ liệu hạn chế); đây là lỗi trỏ nhầm số bảng trong chính văn bản paper, người ghi chú không tự sửa mà chỉ đối chiếu số liệu bằng cách khác (xem mục 4.1).
- Chú thích Hình 8 ghi ”lr tăng từ 0.3 đến 1.5” trong khi văn bản mục A.2 mô tả ”lr tăng từ khoảng 0.2 đến 2.0” cho cùng hiện tượng; hai con số không khớp nhau, có thể do caption và văn bản được viết ở thời điểm khác nhau.
- Tác giả bài này là **Leslie N. Smith**; một tác giả khác tên **Samuel L. Smith** (không liên quan) viết các paper được trích dẫn ở đây về ”noise scale” và về ”Don’t Decay the Learning Rate, Increase the Batch Size” (chính là opt 23). Hai tên ”Smith” dễ gây nhầm lẫn khi đọc nhanh phần tài liệu tham khảo.
- Hầu hết thí nghiệm dùng framework Caffe (2016), hardware Titan Black/Power8+P100 đã khá cũ; các số liệu tuyệt đối (ví dụ accuracy ImageNet) không đại diện cho mức SOTA hiện tại, chỉ có giá trị so sánh **tương đối** giữa 2 lịch lr trong cùng thí nghiệm.
- Phạm vi đọc: đã đọc toàn bộ văn bản chính (mục 1-6) và phụ lục A.1-A.6; không có phần nào bị bỏ qua vì paper có độ dài vừa phải (~18 trang gồm phụ lục).

7. Liên hệ với các bài khác

- **opt 18 (SGDR):** paper này thử trực tiếp mẫu sawtooth-cosine của SGDR và báo cáo **không** quan sát được super-convergence với mẫu đó; sự khác biệt mấu chốt là 1cycle dùng **đúng một chu kỳ nhỏ hơn tổng ngân sách** cộng thêm giai đoạn ”annihilation” xuống rất thấp, còn SGDR lặp lại nhiều chu kỳ và nhảy lại lên đúng η_{max} ban đầu.
- **opt 21 (Goyal, Large Minibatch SGD):** cả hai paper đều coi warmup lr là một dạng rời rạc hoá của CLR; paper này đồng thời thách thức một phần kết luận thường gặp về generalization gap theo batch size mà opt 21 phải xử lý bằng linear scaling rule + warmup.
- **opt 24 (Keskar, sharp minima):** paper này trích dẫn trực tiếp và nói kết quả của mình ”đi ngược” gợi ý batch-size-warm-start của Keskar et al., đồng thời quan sát generalization gap gần như không đổi theo batch size (khác mô tả ”batch lớn gây sharp minima” của Keskar).

- **opt 06 (Adam):** là một trong 4 optimizer thích ứng được thử; không đạt super-convergence dù dùng 1cycle, trong khi Nesterov/AdaDelta/AdaGrad đạt được, gợi ý sự khác biệt về cách Adam dùng bộ nhớ bậc hai (second moment) tương tác xấu với lr cực lớn.
- **opt 46, opt 47 (hướng dẫn thực hành lr/batch/momentum/weight decay, cùng tác giả Leslie Smith):** là các bài tiếp nối trực tiếp, hệ thống hoá các ablation (momentum, weight decay, batch size) đã làm sơ bộ ở phụ lục bài này thành một ”disciplined approach”.
- **llm content/, rl content/, distillation content/, pruning content/, interp content/ (thư mục cũ):** đã `ls` kiểm tra, không có bài nào trong các thư mục này bàn trực tiếp về CLR/1cycle/super-convergence nên không trỏ số bài cụ thể ở đây.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU ≤ 8 GB, batch 1). Toàn bộ mục này là suy luận của người ghi chú:

1. **1cycle có thể tăng tốc fine-tune/distill trên ngân sách GPU nhỏ**, vì paper cho thấy cùng chất lượng đạt được với ít hơn tới 4-8 lần epoch so với lịch cố định; với GPU ≤ 8 GB và cần lặp nhiều thí nghiệm, giảm thời gian mỗi lần chạy là lợi ích trực tiếp.
2. **Xung đột tiềm ẩn với mục tiêu ”giữ sát model gốc” khi distill:** lr cực lớn (gấp ~10 lần thường dùng) được paper coi là một dạng regularization mạnh, có thể làm model học sinh di chuyển xa khỏi lời giải gần giảng viên hơn cần thiết, đặc biệt nguy hiểm khi mục tiêu là phục hồi gần một điểm cố định trong không gian trọng số sau pruning, không phải tìm một minima mới tốt nhất từ đầu; cần giảm biên lr tối đa khi áp dụng cho fine-tune/distill thay vì train-from-scratch.
3. **Kết quả ”Adam không đạt super-convergence”** đáng chú ý vì AdamW là optimizer phổ biến nhất cho fine-tune LLM/VLM hiện nay (bao gồm khả năng cho dự án này); nếu muốn thử 1cycle, nên thử trước với SGD+momentum hoặc Nesterov trên một phần nhỏ dữ liệu để kiểm tra hiện tượng có xảy ra không, trước khi giả định nó chuyển thẳng sang AdamW.
4. **Batch 1 nằm ngoài phạm vi thí nghiệm của paper** (paper dùng batch 256-1536, nhiều GPU); không có cơ sở trực tiếp để suy ra 1cycle có còn hiệu quả ở batch 1 (gradient noise rất cao sẵn có thể đã đóng vai trò tương tự ”lr lớn”, nên cộng thêm lr lớn có thể dư thừa regularization, gây underfit); cần kiểm chứng thực nghiệm riêng (xem mục 9).

9. Thí nghiệm gợi ý

1. **LR range test trên model nhỏ:** chạy một LR range test (lr tăng tuyến tính từ rất nhỏ tới lớn trong một lần train ngắn) cho một CNN nhỏ hoặc một đầu fine-tune nhỏ của VLM OCR, đo test accuracy/loss theo lr để tìm lr tối đa khả dụng, giống Hình 2 của paper.
2. **So sánh 1cycle vs cosine đơn (opt 18) trên cùng ngân sách:** với cùng tổng số bước fine-tune trên 1 GPU, so sánh 1cycle (biên lr rộng, có annihilation) với cosine annealing đơn giản (biên hẹp hơn, không annihilation mạnh), đo tốc độ hội tụ và chất lượng cuối.
3. **Ablation weight decay khi dùng lr lớn:** cố định 1cycle với lr tối đa cao, thử vài giá trị weight decay (như Hình 12b: cao/trung bình/thấp) để kiểm tra lại nguyên lý ”WD lớn ngăn super-convergence” ở quy mô nhỏ.
4. **Batch 1 có còn hưởng lợi từ lr lớn không:** so sánh 1cycle ở batch rất nhỏ (1, 4, 16) trên cùng model nhỏ, đo xem lợi ích (nếu có) giảm đi thế nào khi batch giảm, đối chiếu với nhận xét ở mục 8.4 rằng nhiễu gradient tự nhiên ở batch nhỏ có thể đã đóng vai trò gần giống lr lớn.
5. **Kiểm tra optimizer nào đạt ”super-convergence” ở quy mô nhỏ:** lặp lại so sánh SGD+Nesterov vs AdamW với cùng 1cycle trên một bài toán nhỏ, xem liệu quan sát ”Adam không đạt super-convergence” của paper có lặp lại không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
CLR	2 biên lr + 1 stepsize (số iteration/nửa chu kỳ)	Smith 2015/2017
LR range test	tăng lr tuyến tính, tìm đỉnh accuracy	xác định lr_{max}; $lr_{min} = lr_{max}/3$ hoặc $/4$
1cycle	1 chu kỳ $<$ tổng ngân sách + annihilation sau đó	khác CLR lặp nhiều chu kỳ, khác SGDR (nhảy lại η_{max})
Xấp xỉ Hessian-free	$\epsilon^* = \frac{\theta_{i+1} - \theta_i}{2\theta_{i+1} - \theta_i - \theta_{i+2}}$	eq. (8); ϵ^* lớn $\Rightarrow$ Hessian nhỏ $\Rightarrow$ minima phẳng
Nguyên lý regularization	lr lớn, batch nhỏ, WD, dropout cùng ”ngân sách” regularization	tăng cái này phải giảm cái khác
Kết quả chính Cifar-10	92.4% / 10.000 iter (super-conv) vs 91.2% / 80.000 iter (default)	Resnet-56
Kết quả ImageNet tốt nhất	Resnet-50: 67.6% top-1; Inception-resnet-v2: 74.0%	1cycle 20 epoch, WD giảm
Optimizer đạt super-convergence	Nesterov, AdaDelta, AdaGrad: có; Adam: không	với 1cycle

11. Kết luận ngắn

Super-convergence là hiện tượng huấn luyện nhanh hơn một bậc độ lớn (và có khi chính xác hơn) bằng cách dùng đúng một chu kỳ CLR (”1cycle”) với lr tối đa rất lớn, kèm giảm các regularization khác (weight decay, dropout) để giữ cân bằng, dựa trên lý giải rằng lr lớn tự nó là một dạng regularization và tương ứng với việc SGD tìm minima rộng, phẳng (ước lượng qua một bản đơn giản hoá của Hessian-free). Thực nghiệm trên Cifar-10/100, MNIST, ImageNet với nhiều kiến trúc (resnet, wide-resnet, densenet, inception) xác nhận hiện tượng, đặc biệt rõ khi dữ liệu hạn chế; tuy nhiên hiện tượng chỉ xuất hiện với SGD/Nesterov/AdaDelta/AdaGrad, không với Adam, và không xuất hiện với mẫu sawtooth của SGDR, cho thấy chi tiết chính xác của lịch lr (không chỉ việc ”có cycle”) mới là yếu tố quyết định.

Nguồn gốc: https://arxiv.org/abs/1708.07120

Bài 20

MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies (2024)

Tác giả: Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, Maosong Sun (Tsinghua University, Modelbest Inc.).
Venue / arXiv: arXiv:2404.06395 (v3, 03/06/2024); paper không nêu venue hội nghị/tạp chí chính thức - tự ghi chú ban đầu công bố dưới dạng technical blog ngày 01/02/2024.
Nguồn: https://arxiv.org/abs/2404.06395
Chủ đề: chiến lược huấn luyện SLM (Small Language Model) có thể scale được, gồm ”Model Wind Tunnel Experiments” (dò siêu tham số/batch size/lr trên model nhỏ rồi transfer), và lịch lr Warmup-Stable-Decay (WSD) cho phép đo scaling law data-model với chi phí tuyến tính.
Vị trí trong lộ trình: đọc sau opt 18 (SGDR/cosine decay, nền so sánh chính của WSD), opt 21-23 (batch size scaling, được trích dẫn trực tiếp và áp dụng thực tế trong bài này); liên quan chặt tới llm 03 (Kaplan Scaling Laws) và llm 04 (Chinchilla) vì bài này đo lại tỉ lệ data/model tối ưu và tìm ra khác biệt lớn so với Chinchilla.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

Phạm vi ghi chú (quan trọng): MiniCPM là một paper rất dài (~184.000 ký tự), bao gồm nhiều nội dung không thuộc chủ đề ”optimization” của thư mục này: kiến trúc model (embedding sharing, GQA, deep-and-thin), MiniCPM-V (đa phương thức), MiniCPM-DPO (alignment), MiniCPM-MoE, MiniCPM-128K (context dài), và chi tiết decontamination/benchmark. Ghi chú này **chỉ tập trung** vào phần tối ưu hoá/huấn luyện: Model Wind Tunnel Experiments (mục 3 của paper), WSD learning rate scheduler (mục 4), chiến lược 2-giai đoạn pretraining liên quan tới lịch lr (mục 5), và scaling law data-model (mục 4.5) - đã đọc đầy đủ các mục 1-5 (giới thiệu, related work, 3 mục trên) cùng Phụ lục A.1-A.3, B.1-B.4 liên quan trực tiếp; đã đọc lướt mục 6-8 (Model, MiniCPM Family, Conclusion) chỉ để lấy thông tin nền (cấu hình batch size/lr thực tế đã dùng, phần ”Limitations” tác giả tự nêu), không đi sâu các nội dung ngoài phạm vi trên.

1. Bối cảnh

Huấn luyện LLM tốn kém tới mức hầu hết các phòng thí nghiệm không thể làm nhiều lần thí nghiệm dò siêu tham số trên quy mô lớn. Ý tưởng ”wind tunnel” (hầm gió, mượn từ ngành hàng không): dùng model nhỏ (SLM) để dò trước các quyết định huấn luyện (siêu tham số, batch size, lr, lịch lr), rồi **transfer** sang model lớn mà không cần dò lại từ đầu - dựa trên Tensor Program / μP (Yang et al. 2022, 2023), framework đảm bảo siêu tham số tối ưu ổn định qua các mức độ rộng (width) và sâu (depth) khác nhau của mạng.

Cosine LRS (Loshchilov & Hutter, 2016 = opt 18) là lịch lr phổ biến nhất cho pretraining LLM hiện nay (Kaplan 2020, Hoffmann/Chinchilla 2022, LLaMA, Qwen,...). Nhưng cosine LRS có một hạn chế thực tế: phải **biết trước tổng số bước huấn luyện** S để đặt $T = S$ (điểm cosine giảm về mức tối thiểu) - không phù hợp với continual pretraining (huấn luyện tiếp tục không biết trước khi nào dừng) hay việc tái sử dụng checkpoint giữa đường để thử các độ dài dữ liệu khác nhau.

2. Ý tưởng chính

Hai đóng góp chính (trong phạm vi ghi chú này):

1. **Model Wind Tunnel Experiments (MWTE, mục 3):** gồm 3 phần - (a) ổn định siêu tham số qua các scale bằng Tensor Program (width scaling + depth scaling); (b) luật scale batch size tối ưu theo loss; (c) xác nhận lr tối ưu ổn định qua các scale (nhờ Tensor Program).
2. **WSD (Warmup-Stable-Decay) LRS (mục 4):** tách rõ pha "lr cao, ổn định" (để khám phá không gian tham số) và pha "giảm lr" (để hội tụ tinh), thay vì trộn lẫn liên tục như cosine. Lợi ích: có thể **dừng pha ổn định ở bất kỳ điểm nào**, rồi "decay" (giảm dần) riêng trong một khoảng ngắn (~10% tổng token) để đạt loss tối ưu tương đương cosine tại chính điểm đó - cho phép đo scaling law data-model với chi phí **tuyến tính** $O(mC)$ (so với $O(m^2C)$ truyền thống phải train lại từ đầu cho mỗi cặp kích thước model/dữ liệu).

3. Phương pháp chi tiết

3.1 Tensor Program / μP (mục 3.1, Phụ lục A.1, Bảng 7)

5 phép scale cụ thể được áp dụng (Bảng 7 của paper): (1) scale output embedding bởi `scale_emb`; (2) scale output mỗi block trước khi cộng residual bởi `scale_depth`$/\sqrt{\text{num_layers}}$; (3) khởi tạo std của tensor 2D bằng `init_std`$/\sqrt{d_m/d_{base}}$ (các tham số khác khởi tạo std=0,1); (4) lr của tensor 2D = $1/(d_m/d_{base})\times$ lr tổng thể; (5) scale logit đầu ra LM head bởi $1/(d_m/d_{base})$. Paper **không** dùng attention softmax scaling (một phần khác của Tensor Program). Qua Bayesian search trên model 0.009B (train với $10N = 0{,}09$B token), tìm được bộ siêu tham số tốt nhất: `scale_depth=1,4`, `scale_emb=12`, `init_std=0,1`, `lr=0,01`. Paper cũng thử QK-Norm + weight decay độc lập (giảm độ nhạy lr, giống Wortsman et al. 2023) nhưng **không dùng** cho MiniCPM vì với SLM không cần độ nhạy lr thấp (đã tìm được lr tốt bằng Tensor Program).

3.2 Luật batch size tối ưu (mục 3.2, Phụ lục A.2)

Train 3 model (0,009B, 0,03B, 0,17B) × 6 batch size mỗi model, lr cố định 0,01, cosine LRS, đo loss trên C4. Vẽ batch size (trục x) vs số token tiêu thụ (trục y), màu = loss; dùng parabola khớp các điểm cùng loss, nối các cực tiểu parabola giữa các model size → quan hệ tuyến tính trong không gian log, cho công thức:

$$\boxed{bs = \frac{1{,}21 \times 10^9}{L^{6{,}24}}}$$

(bs = batch size tối ưu, L = loss trên C4).

Ví dụ tự tính (kiểm chứng công thức với loss thực tế của MiniCPM cuối huấn luyện): với $L = 2{,}6$ (gần giá trị loss C4 cuối huấn luyện MiniCPM-2.4B theo Hình 12 của paper, khoảng 2,4-2,6): $L^{6{,}24} = 2{,}6^{6{,}24}$. Tính $\ln(2{,}6) \approx 0{,}9555$, $\times 6{,}24 \approx 5{,}962$, $e^{5{,}962} \approx 388{,}7$ (tự tính). Vậy $bs \approx 1{,}21\times10^9/388{,}7 \approx 3{,}11\times10^6 \approx 3{,}11$ triệu (tự tính) - khá gần với batch size thực tế MiniCPM-2.4B dùng (3,93 triệu, Bảng 2 mục 6.2 của paper), xác nhận công thức cho dự đoán đúng cấp độ lớn khi áp vào chính model cuối cùng của họ.

Khác biệt quan trọng với Kaplan et al. (2020) (= liên quan opt 22, Phụ lục A.2 của paper): Kaplan giả định có **số GPU không giới hạn**, nên tăng batch size không làm chậm mỗi bước (chỉ giảm tổng số bước) - hợp lý để định nghĩa "critical batch size" tối thiểu hoá cả bước và token. MiniCPM có **số GPU cố định**, nên gấp đôi batch size ≈ gấp đôi thời gian mỗi bước - tăng batch để giảm số bước **không** tiết kiệm thời gian thực tế. Do đó MiniCPM **bỏ** mục tiêu "giảm số bước" của Kaplan, chỉ còn mục tiêu tối thiểu hoá **số token** để đạt loss thấp nhất. Paper cũng tự nhận đây là một "nghịch lý con gà - quả trứng" (ước lượng batch tối ưu từ loss mà loss chỉ biết được sau khi train), và mô tả quy trình coordinate-descent-như (tối ưu lr trước ở 1 batch, rồi tối ưu batch, rồi tối ưu lại lr) để xử lý tương quan lr-batch, tự nhận "các phương pháp chặt chẽ hơn vẫn được chào đón ở công trình tương lai".

3.3 Công thức WSD (eq. 1, mục 4.2)

$$WSD(T;s) = \begin{cases} \frac{s}{W}\eta, & s < W \\ \eta, & W < s < T \\ f(s-T)\eta, & T < s < S \end{cases} \quad (1)$$

(W=hết warmup, T=hết pha ổn định, S=tổng số bước, f là hàm giảm, $0 < f \leq 1$). Dạng cụ thể dùng cho MiniCPM chính thức (mục 6.2): decay **exponential**, $f(s-T) = 0{,}5^{(s-S)/T}$ với $T = 5000$ bước (20B token) - **lưu ý:** cách dùng biến S, T trong công thức cụ thể này ở mục 6.2 của paper có vẻ không hoàn toàn khớp quy ước T=hết-pha-ổn-định/S=tổng-bước đã định nghĩa ở eq. (1); nhiều khả năng đây là lỗi mất định dạng số mũ/chỉ số khi PDF được chuyển thành văn bản (không có cách khôi phục chắc chắn công thức chính xác từ bản text thuần đã trích xuất) - ghi chú này trình bày đúng như paper viết, không tự suy diễn lại công thức.

4. Thí nghiệm và kết quả chính

4.1 Xác nhận lr ổn định qua scale (mục 3.3, Hình 3)

Thử lr ở 0,04B/0,1B/0,3B/0,5B (tăng 10 lần kích thước): lr tối ưu **không đổi đáng kể**, giữ khoảng 0,01 - xác nhận hiệu quả của Tensor Program. Kiểm chứng thêm ở 2,1B: lr=0,01 vẫn cho loss thấp nhất.

4.2 So sánh Cosine(T) vs CosineLoop(T) (mục 4.1, Hình 4)

Train model 0,036B ở $S \in \{20N, 40N, 60N, 80N\}$: loss thấp nhất **luôn** đạt được khi $T = S$ (cả $T < S$ và $T > S$ đều kém hơn) - xác nhận cả quan sát của Kaplan (lr cao kéo dài giúp loss thấp, nên không nên $T < S$) và của Hoffmann/Chinchilla (lr không nên giữ cao mãi, nên không nên $T > S$) đều đúng, cùng chỉ về $T = S$ là tối ưu.

4.3 Thí nghiệm WSD (mục 4.3, Hình 5, 6)

- **Loss giảm mạnh ở pha decay:** trên model 0,036B, ngay khi lr bắt đầu giảm, loss giảm nhanh, đạt bằng hoặc thấp hơn Cosine($T = S$) tại đúng điểm đó.
- **10% bước là đủ:** so sánh decay 10% vs 2,5% tổng token từ cùng checkpoint pha ổn định (ở 40N/60N/80N dữ liệu): decay 10% đủ để hội tụ tốt nhất, 2,5% chưa đủ. MiniCPM dùng decay ~10% cho các thí nghiệm tiếp theo.
- **Scale dữ liệu hiệu quả:** tiếp tục train 1 model 0,036B tới hội tụ cực hạn (nhiều hơn cả mức Chinchilla Optimal), so sánh với model 0,17B train 40N dữ liệu; khớp đường cong "optimal loss envelope" (tại mỗi D, loss = WSD(D, 0,1D)) bằng 2 dạng hàm (exponential $L(C) = \alpha e^{-\beta C} + L_0$; power-law $L(C) = \beta C^{-\alpha} + L_0$) - **power-law khớp tốt hơn** (tương tự phát hiện power-law của Kaplan cho Cosine LRS). Kết quả: model 0,036B train tiếp tục (continual training) đạt hiệu năng **bằng** model 0,17B, với chi phí train tăng ~4 lần nhưng **giảm chi phí inference mỗi lần gọi ~5 lần** - một cấu hình inference-compute-optimal tốt hơn (trích Sardana & Frankle 2023).

4.4 Phân tích pha decay (mục 4.4, Hình 7, 8)

Đo $\max_{ij}(W_{ij}^{(t+1)} - W_{ij}^{(t)})$ (độ thay đổi trọng số lớn nhất) trên MiniCPM-2.4B, mọi layer/submodule: tương quan mạnh với độ lớn lr. Phát hiện **không hiển nhiên** (nguyên văn paper nhấn mạnh "may not be trivial"): pha ổn định (lr cao) có thay đổi trọng số **lớn** nhưng loss giảm **rất ít**; pha decay (lr giảm) có thay đổi trọng số **nhỏ hơn** nhưng loss giảm **nhanh hơn**. Phân tích gradient chi tiết hơn trên model 0,2B (ghi mọi bước gradient): chuẩn gradient giảm dần trong pha decay; cosine góc giữa gradient 2 bước liên tiếp trở nên **dương** (hướng cập nhật nhất quán hơn) khi vào pha decay; đạo hàm bậc 1 theo hướng quỹ đạo $D_1 = \frac{g^{(t+1)} \cdot v^{(t)}}{\|v^{(t)}\|}$ giảm theo hàm mũ (khớp lr); đạo hàm bậc 2 $D_2 = \frac{(g^{(t+1)} - g^{(t)}) \cdot v^{(t)}}{\|v^{(t)}\|^2}$ tăng nhẹ về độ lớn; độ cong $K = |D_2|/(1 + D_1^2)^{3/2}$ tăng khoảng 1 bậc độ lớn - paper diễn giải là dấu hiệu tiến gần cực tiểu local, và tự nhận "dành cho nghiên cứu tương lai".

4.5 Đo scaling law bằng WSD (mục 4.5, Hình 9, 10)

Train 6 kích thước model (0,04B-2B) × 6 checkpoint decay (bắt đầu từ 10N-60N dữ liệu pha ổn định), đánh giá trên 5 tập held-out, loss tính theo byte (không theo token, để so sánh công bằng giữa tokenizer khác nhau, theo Achiam et al. 2023). Khớp $L(N, D) = C_N N^{-\alpha} + C_D D^{-\beta} + L_0$ (eq. 2, theo đúng form Hoffmann/Chinchilla). Kết quả trung bình qua các tập: $\alpha = 0{,}29$, $\beta = 0{,}23$, và (từ panel "Average" của Hình 10) $D_{opt}/N_{opt}|_{C=10^{21}} = 191{,}87 \approx 192$ - **gấp khoảng 9,6 lần tỉ lệ 20 của Chinchilla** (tự tính: $192/20 = 9{,}6$). Paper diễn giải: vì $\beta < \alpha$ (0,23<0,29), nên khi tăng compute, "nên nhấn nhẹ hơn vào data scaling so với model scaling" - theo đúng hướng Chinchilla khuyến nghị, nhưng độ lớn tỉ lệ D_{opt}/N_{opt} đo được **khác biệt rất lớn** so với con số 20 gốc của Chinchilla.

So sánh với Llama 2: paper trích xuất loss curve công khai của Llama 2 (13B/34B/70B) từ chính paper Llama 2, ước lượng điểm giao nhau giữa các đường loss theo compute để suy ra $D_{opt}/N_{opt} \approx$ 70-100 cho Llama 2 - "gần với tỉ lệ của MiniCPM hơn là tỉ lệ 20 của Chinchilla". **Lưu ý về độ tin cậy số liệu này:** phần tính FLOPs cụ thể trong đoạn này của bản text trích xuất có dấu hiệu bị lỗi định dạng số mũ (ví dụ cụm "10^5 EFlops (10^{18} Flops)" không tự nhất quán về đơn vị khi đọc theo nghĩa đen - 1 EFlop = 10^{18} FLOP nên "10^5 EFlops" phải là 10^{23} FLOP, không phải 10^{18}); ghi chú này **không** tái tạo lại các số FLOP trung gian cụ thể vì không có cách xác nhận chắc chắn từ bản PDF-to-text, chỉ ghi lại kết luận định tính mà paper tự nêu ($D_{opt}/N_{opt} \approx$ 70-100 cho Llama 2, tự nhận "đây chỉ là ước lượng thô").

4.6 Chiến lược 2-giai đoạn pretraining (mục 5, Bảng 1)

Giả thuyết: trộn dữ liệu SFT chất lượng cao **vào pha decay** (không chỉ ở pha SFT riêng) giúp model học tốt hơn, vì (a) phân phối dữ liệu annealing gần với tình huống sử dụng thật hơn pretraining thô; (b) tránh lặp lại 1 tập SFT nhỏ nhiều lần trong suốt pretraining dài (có thể gây hại). Ablation (Bảng 1, đọc được đầy đủ số liệu từ bản PDF-text):

Cấu hình	C-Eval	CMMLU	MMLU	GSM8K	MATH	HumanEval	MBPP
A-1 (2,4B, decay chỉ pretrain, +SFT 4B token)	40,0	41,5	44,6	27,7	5,1	27,7	24,4
A-2 (2,4B, decay trộn SFT, +SFT 4B token)	52,6	51,1	50,9	42,3	5,4	30,4	30,3
B-1 (1,2B, decay chỉ pretrain, +SFT 6B token)	40,9	41,5	47,9	34,2	7,9	43,9	30,5
B-2 (1,2B, decay chỉ pretrain, +SFT 12B token)	41,2	42,0	47,9	34,4	7,3	43,9	29,8
B-3 (1,2B, decay trộn SFT, +SFT 6B token)	49,1	46,8	49,6	31,8	10,5	44,5	32,8

A-2 vượt A-1 rõ rệt trên hầu hết benchmark (C-Eval +12,6, CMMLU +9,6, GSM8K +14,6). B-3 vượt B-1/B-2 trên C-Eval, CMMLU, MATH, HumanEval, MBPP, nhưng **thấp hơn** B-1/B-2 ở GSM8K (31,8 so với 34,2/34,4) - một ngoại lệ cụ thể trong chính bảng số liệu của paper mà văn bản chính không nhắc riêng (chỉ nói chung "đẩy được ranh giới hiệu năng"); ghi chú này nêu rõ ngoại lệ này theo đúng số liệu, không bỏ qua. So B-1 và B-2 (cùng decay chỉ pretrain, chỉ khác lượng token SFT 6B vs 12B): kết quả gần như không đổi, xác nhận việc B-1/B-2 kém B-3 không đơn giản do thiếu token SFT.

5. Phân tích / ablation

- Hiệu ứng "mâu thuẫn hoà giải" giữa Kaplan (ủng hộ lr cao kéo dài) và Chinchilla (ủng hộ $T = S$, không kéo dài quá) được giải quyết gọn bằng WSD: tách 2 pha rõ ràng, thay vì cố gắng chọn 1 điểm T duy nhất cho cosine.

- Tỉ lệ D_{opt}/N_{opt} đo bằng WSD (≈192) cao hơn Chinchilla (20) gần 10 lần - cùng hướng với quan sát thực tế rằng các LLM hiện đại (Llama 2, trích dẫn trực tiếp) train nhiều dữ liệu hơn nhiều so với khuyến nghị Chinchilla mà vẫn cải thiện hiệu năng, cho thấy công thức gốc Chinchilla (2022) có thể đã đánh giá thấp lượng dữ liệu tối ưu cho các cấu hình huấn luyện/kiến trúc hiện đại hơn.
- Bảng 1 cho thấy lợi ích "trộn SFT vào decay" **không đồng nhất trên mọi benchmark** (ngoại lệ GSM8K ở cấu hình B) - gợi ý hiệu ứng có thể phụ thuộc loại năng lực (kiến thức tổng hợp vs suy luận toán) chứ không phải lợi ích tuyệt đối trên mọi tác vụ.

6. Giới hạn

Tác giả tự nêu (mục "Limitations" cuối bài, trích nguyên văn ý): "Mặc dù đã đề xuất một nghiên cứu kỹ về scaling law với SLM, bài báo này không mở rộng tới việc train một LLM (thực sự lớn) để xác nhận lại scaling law đó. Việc áp dụng WSD LRS cho LLM chưa được khám phá đầy đủ tới nay." Đây là giới hạn quan trọng nhất tác giả tự thừa nhận: toàn bộ luật scale batch/data-model/WSD được đo trên model tối đa 2,4B, ngoại suy sang LLM (hàng chục-hàng trăm tỷ tham số) chưa được kiểm chứng trực tiếp trong bài.

Người ghi chú nhận xét thêm:

- Công thức WSD decay cụ thể ở mục 6.2 (dùng cho chính MiniCPM-2.4B/1.2B) có vẻ không khớp hoàn toàn quy ước biến trong eq. (1) (xem mục 3.3) - khả năng cao là lỗi mất định dạng số mũ khi chuyển PDF sang text, không khẳng định đây là lỗi thật của paper gốc.
- Phần ước lượng FLOPs cho Llama 2 (mục 4.5, so sánh D_{opt}/N_{opt}) chứa số liệu có dấu hiệu lỗi định dạng đơn vị (EFlops/Flops không nhất quán trong bản text trích xuất) - đã nêu rõ ở mục 4.5, không tái dựng số liệu cụ thể.
- Bảng 1 (mục 4.6) có 1 ngoại lệ (GSM8K) không được paper thảo luận riêng dù dữ liệu chính paper công bố thể hiện rõ.
- Batch size formula $bs = 1{,}21 \times 10^9 / L^{6{,}24}$ được fit từ model rất nhỏ (0,009B-0,17B); mức độ ngoại suy đúng cho model 2,4B (dù kiểm chứng tốt ở mục 3.2 phần tự tính trên) lẫn cho model lớn hơn (LLM) chưa được paper xác nhận trực tiếp bằng thực nghiệm riêng ở quy mô LLM.

7. Liên hệ với các bài khác

- **opt 18 (SGDR/Cosine Annealing):** WSD là cải tiến trực tiếp thay thế Cosine LRS tiêu chuẩn mà opt 18 đã phổ biến hoá cho deep learning; paper trích dẫn Loshchilov & Hutter (2016) trực tiếp khi giới thiệu Cosine LRS làm baseline so sánh.
- **opt 21/22/23 (large-batch, noise scale, tăng batch thay giảm lr):** mục 3.2/Phụ lục A.2 trích dẫn trực tiếp Kaplan et al. (2020) (liên quan mật thiết tới cách đo critical batch size của opt 22) và **chủ động chỉnh sửa giả định** của Kaplan cho phù hợp với ràng buộc GPU cố định; mục 6.4 trích dẫn trực tiếp Smith et al. (2017) (= opt 23) khi giải thích hiện tượng loss giảm đột ngột lúc tăng batch size (2M → 4M) cho MiniCPM-1.2B, "có thể có hiệu ứng tương tự giảm lr".
- **llm 03 (Kaplan, Scaling Laws) và llm 04 (Chinchilla):** mục 4.5 là phần đối thoại trực tiếp nhất với 2 bài này - đo lại D_{opt}/N_{opt} bằng phương pháp rẻ hơn (WSD) và tìm tỉ lệ cao hơn gần 10 lần so với Chinchilla, dùng chính công thức $L(N, D)$ của Hoffmann et al. (2022).
- **opt 24-26 (sharp minima, SAM, Edge of Stability):** liên hệ gián tiếp qua phần phân tích pha decay (mục 4.4) - quan sát độ cong K tăng và cosine gradient dương khi vào pha decay có thể được nhìn như một dạng "tiến gần cực tiểu" tương tự tinh thần theo dõi curvature của opt 26, dù MiniCPM không dùng Hessian trực tiếp mà dùng xấp xỉ qua 2 đạo hàm phương hướng liên tiếp.
- Đã `ls` kiểm tra 5 thư mục cũ (llm/rl/distillation/pruning/interp content); ngoài các file llm 03/04 đã nêu (xác nhận tồn tại qua `ls` trước đó trong phiên làm việc), không có bài khác liên quan trực tiếp tới WSD/wind-tunnel scaling nên không trích dẫn thêm.

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, batch 1, GPU ≤ 8GB). Toàn bộ mục này là suy luận:

1. **WSD scheduler phù hợp tự nhiên với continual fine-tune/distill không biết trước tổng số bước** (suy luận, liên quan trực tiếp): nếu dự án không cố định trước số bước distill (ví dụ muốn dừng sớm nếu đủ tốt, hoặc tiếp tục nếu còn ngân sách), WSD cho phép giữ lr ổn định trong pha chính rồi ”decay” ngắn (10% số bước còn lại) bất cứ khi nào cần dừng để lấy checkpoint tốt, thay vì phải cam kết trước tổng số bước như cosine - có thể áp dụng trực tiếp cho vòng lặp fine-tune OCR của dự án.
2. **Tỉ lệ data/model rất cao (192 so với 20)** gợi ý rằng với model nhỏ như LightOnOCR-3-0.8B, có thể còn ”dư địa” để tiếp tục train/distill thêm nhiều dữ liệu OCR hơn so với trực giác kiểu Chinchilla (dữ liệu/tham số = 20) trước khi bão hoà - nhưng đây là ngoại suy từ domain text/C4 sang domain OCR/VLM, **độ tin cậy thấp** (suy luận yếu, hai domain khác nhau nhiều).
3. **Chiến lược ”trộn dữ liệu chất lượng cao vào pha decay” (mục 4.6) có thể áp dụng trực tiếp cho giai đoạn cuối distill:** nếu quy trình nén có một pha ”giảm lr cuối” trước khi đóng băng model, trộn một tỉ lệ dữ liệu OCR khó/đa dạng (tương tự SFT chất lượng cao ở MiniCPM) vào đúng giai đoạn giảm lr này có thể hiệu quả hơn là chỉ thêm chúng vào 1 pha fine-tune riêng cuối cùng (suy luận, dựa trên kết quả Bảng 1, chưa kiểm chứng cho OCR).
4. **Công thức batch size cụ thể ($bs \propto L^{-6,24}$) ít liên quan trực tiếp** vì được fit cho C4/text LLM ở quy mô rất khác (và dự án bị khoá cứng ở batch 1 do VRAM) - không nên áp dụng hệ số 6,24 này cho domain OCR mà không fit lại riêng (suy luận, ”ít liên quan” với lý do rõ).

9. Thí nghiệm gợi ý

1. **Thử WSD scheduler cho 1 lần fine-tune/distill OCR nhỏ:** so với cosine LRS hiện có (nếu đang dùng), giữ lr ổn định một khoảng, rồi decay ngắn (~10% bước còn lại) tại vài điểm dừng khác nhau, so sánh validation CER/WER tại mỗi điểm dừng với cosine đặt $T =$ đúng điểm dừng đó.
2. **Thử trộn một phần dữ liệu khó/OOD vào pha decay cuối** (giống mục 4.6 của bài) của 1 lần fine-tune nhỏ, so với thêm cùng lượng dữ liệu đó vào 1 pha riêng sau cùng, đo chênh lệch benchmark OCR.
3. **Đo max weight update và gradient cosine similarity** (như mục 4.4) trong 1 lần fine-tune OCR nhỏ, xem có pattern tương tự ”update lớn nhưng loss giảm ít” ở pha đầu, ”update nhỏ nhưng loss giảm nhanh” ở pha cuối hay không - một cách rẻ để chẩn đoán khi nào nên bắt đầu decay lr.
4. **Kiểm tra lại công thức batch-size-theo-loss ở quy mô nhỏ của dự án:** nếu có thể chạy vài cấu hình batch (qua gradient accumulation) ở model nhỏ fine-tune OCR, thử fit dạng $bs = aL^{-b}$ riêng cho domain này, so sánh hệ số b đo được với $6{,}24$ của MiniCPM (khác domain nên khó kỳ vọng giống, nhưng là một kiểm tra thú vị).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Nguồn
WSD scheduler	3 pha: warmup / stable η / decay $f(s-T)\eta$	eq. (1)
Batch size tối ưu theo loss	$bs = 1{,}21 \times 10^9 / L^{6,24}$	mục 3.2
Pha decay cần	~10% tổng token	mục 4.3
Scaling law data-model	$L(N,D) = C_N N^{-\alpha} + C_D D^{-\beta} + L_0$	eq. (2), theo Hoffmann et al. 2022
Hệ số đo được (trung bình)	$\alpha = 0{,}29$, $\beta = 0{,}23$	mục 4.5
D_{opt}/N_{opt} tại $C = 10^{21}$	$191{,}87 \approx 192$ (so với 20 của Chinchilla)	Hình 10, mục 4.5

Khái niệm	Công thức / giá trị	Nguồn
Ước lượng Llama 2	$D_{opt}/N_{opt} \approx$ 70-100	mục 4.5 (số FLOP trung gian không xác nhận được do lỗi trích xuất)
MiniCPM-2.4B batch/token thực tế	batch 3,93M, tổng 1,1T token	Bảng 2, mục 6.2
MiniCPM-1.2B batch	2M → 4M (tăng dần, hiệu ứng giống giảm lr - Smith et al. 2017)	mục 6.4

11. Kết luận ngắn

MiniCPM đóng góp 2 ý tưởng tối ưu hoá chính: (1) ”Model Wind Tunnel Experiments” - dùng model nhỏ để dò trước siêu tham số (qua Tensor Program/μP), batch size tối ưu (công thức $bs \propto L^{-6,24}$), và lr ổn định, rồi transfer sang model đích mà không cần dò lại; (2) WSD learning rate scheduler - tách rõ pha ổn định và pha decay, cho phép reuse checkpoint và đo scaling law data-model với chi phí tuyến tính thay vì bậc hai. Kết quả đo được cho thấy tỉ lệ data/model tối ưu cao hơn Chinchilla gần 10 lần (192 so với 20), phần nào giải thích vì sao các LLM hiện đại train nhiều dữ liệu hơn khuyến nghị Chinchilla mà vẫn cải thiện. Tác giả tự nhận hạn chế lớn nhất: chưa train một LLM thực sự lớn để xác nhận lại các luật scale này.

Nguồn gốc: https://arxiv.org/abs/2404.06395

Bài 21

Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour (2017)

Tác giả: Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, Kaiming He (Facebook).
Venue / arXiv: arXiv:1706.02677 (v2, 30/04/2018); paper không nêu venue hội nghị/tạp chí.
Nguồn: https://arxiv.org/abs/1706.02677
Chủ đề: quy tắc ”linear scaling rule” (tăng learning rate tuyến tính theo minibatch size) kết hợp chiến lược ”warmup” (tăng dần lr đầu huấn luyện) để huấn luyện ResNet-50 trên ImageNet với minibatch tới 8.192 ảnh mà không mất accuracy, cho phép huấn luyện trong 1 giờ trên 256 GPU.
Vị trí trong lộ trình: đọc sau opt 06 (Adam/SGD nền) và cùng nhóm với opt 22 (McCandlish, giải thích vì sao linear scaling rule có giới hạn qua gradient noise scale), opt 23 (Don’t Decay the LR, hướng tiếp cận ngược: tăng batch thay giảm lr), opt 24 (Keskar, sharp minima, lo ngại mà bài này đối chứng thực nghiệm); đọc trước opt 10 (LAMB, mở rộng ý tưởng batch lớn sang BERT ở quy mô lớn hơn nữa).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Mô hình và dữ liệu càng lớn, thời gian huấn luyện càng dài. Distributed synchronous SGD (chia minibatch cho nhiều worker/GPU) là giải pháp tự nhiên, nhưng để hiệu quả, workload mỗi worker phải lớn, kéo theo minibatch tổng phải lớn theo. Trước paper này, chưa có kết quả nào cho thấy accuracy được giữ nguyên với minibatch lớn tới 8.192 trên ImageNet, hay huấn luyện ở độ chính xác cao trong thời gian ngắn như vậy.

Bài toán SGD cơ bản: minimize $L(w) = \frac{1}{|X|}\sum_{x\in X} l(x,w)$ (1), cập nhật minibatch $w_{t+1} = w_t - \eta\frac{1}{n}\sum_{x\in B}\nabla l(x,w_t)$ (2), $n = |B|$ là minibatch size. Baseline tham chiếu: ResNet-50 gốc huấn luyện với $kn = 256$ (8 GPU Tesla P100, $n = 32$/worker), 29 giờ.

2. Ý tưởng chính

Linear Scaling Rule (không cần siêu tham số mới): khi minibatch size nhân k lần, nhân learning rate lên k lần, giữ mọi siêu tham số khác (weight decay...) không đổi. Quy tắc này không mới (đã có ở Krizhevsky và Bottou et al.), nhưng paper là người đầu tiên khảo sát thực nghiệm toàn diện giới hạn của nó, và chỉ ra rằng nó chỉ hoạt động tốt khi kết hợp với một chiến lược **warmup gradual** (tăng dần lr từ nhỏ lên giá trị mục tiêu trong vài epoch đầu) để xử lý giai đoạn đầu huấn luyện khi mạng thay đổi nhanh (giả định $\nabla l(x,w_t) \approx \nabla l(x,w_{t+j})$ dùng để suy ra quy tắc không còn đúng). Kết quả: với minibatch tới 8k, không mất accuracy so với baseline 256, và cả đường cong training error cũng khớp sát baseline (không chỉ accuracy cuối), cho phép debug nhanh. Ngoài ra, paper phát hiện và sửa một loạt ”cạm bẫy” cài đặt (weight decay, momentum, chuẩn hoá loss khi gradient aggregation, data shuffling, Batch Norm) có thể âm thầm làm sai kết quả nếu không cẩn thận khi scale lên nhiều GPU.

3. Phương pháp chi tiết

3.1 Suy luận trực giác cho linear scaling rule (mục 2.1)

So sánh k bước SGD minibatch nhỏ n, lr η với 1 bước minibatch lớn kn, lr $\hat{\eta}$:

$$w_{t+k} = w_t - \eta \frac{1}{n} \sum_{j<k} \sum_{x \in B_j} \nabla l(x, w_{t+j}) \tag{3}$$

$$\hat{w}_{t+1} = w_t - \hat{\eta} \frac{1}{kn} \sum_{j<k} \sum_{x \in B_j} \nabla l(x, w_t) \tag{4}$$

Nếu giả định (mạnh) $\nabla l(x, w_t) \approx \nabla l(x, w_{t+j})$ với $j < k$ (gradient ít đổi qua k bước nhỏ), thì đặt $\hat{\eta} = k\eta$ cho $\hat{w}_{t+1} \approx w_{t+k}$. Giả định này rõ ràng sai ở 2 trường hợp: (a) đầu huấn luyện khi mạng thay đổi nhanh (giải quyết bằng warmup); (b) khi k quá lớn, minibatch không thể scale vô hạn (thực nghiệm cho thấy điểm gãy ở khoảng 8k trên ImageNet).

3.2 Warmup (mục 2.2)

- **Constant warmup:** dùng lr thấp cố định η (không phải $\hat{\eta} = k\eta$) trong vài epoch đầu, sau đó nhảy thẳng lên $\hat{\eta}$. Paper thấy: với k lớn, cách này **không đủ**, lỗi training "tăng vọt" (spike) ngay khi chuyển và không hồi phục hoàn toàn.
- **Gradual warmup (dùng chính thức):** bắt đầu từ η, tăng dần tuyến tính mỗi iteration tới khi đạt $\hat{\eta} = k\eta$ sau 5 epoch, rồi theo lịch lr gốc. Tránh cú nhảy đột ngột.

3.3 Batch Normalization và minibatch size (mục 2.3)

BN tính thống kê theo minibatch, làm loss của một mẫu phụ thuộc cả minibatch chứa nó: $L_B(x, w)$ không độc lập giữa các mẫu nữa. Paper định nghĩa lại loss trên "siêu mẫu" là cả minibatch cỡ n: $L(w) = \frac{1}{|X^n|} \sum_{B \in X^n} L(B, w)$ (5). Mấu chốt: n **(kích thước minibatch mỗi worker dùng để tính thống kê BN) phải giữ cố định** khi tăng số worker k, không phải minibatch tổng kn; nếu không, hàm loss bị thay đổi theo nghĩa khác, không chỉ là ước lượng gradient khác. Paper dùng $n = 32$ cho mọi cấu hình. Thống kê BN không được tính gộp qua các worker.

3.4 Các cạm bẫy cài đặt phân tán (mục 3, 4 "Remark")

- **Weight decay (Remark 1):** loss viết $l(x, w) = \frac{\lambda}{2}\|w\|^2 + \epsilon(x, w)$; cập nhật đầy đủ là $w_{t+1} = w_t - \eta\lambda w_t - \eta \frac{1}{n} \sum \nabla \epsilon(x, w_t)$ (8). **Scaling cross-entropy loss KHÔNG tương đương scaling learning rate** vì số hạng weight-decay $\eta\lambda w_t$ scale khác số hạng gradient; nhầm lẫn này dễ gây lỗi âm thầm.
- **Momentum correction (Remark 2):** với dạng $v_{t+1} = mv_t + \eta \frac{1}{n} \sum \nabla l$, $w_{t+1} = w_t - v_{t+1}$ (10), khi η thay đổi (ví dụ trong warmup), cần nhân thêm hệ số hiệu chỉnh η_{t+1}/η_t vào v_t để giữ tương đương với công thức tham chiếu u_t (độc lập η); đặc biệt quan trọng khi $\eta_{t+1} \gg \eta_t$ (warmup đi lên), kém quan trọng khi η giảm.
- **Gradient aggregation (Remark 3):** phải chuẩn hoá loss mỗi worker theo **tổng minibatch** kn, không phải n riêng từng worker, vì allreduce chỉ cộng (sum) không chia trung bình; "huỷ k" bằng cách đặt $\hat{\eta} = \eta$ (không nhân k) và chuẩn hoá theo $1/n$ là **sai**, gây sai weight decay (liên hệ Remark 1).
- **Data shuffling (Remark 4):** dùng đúng MỘT phép xáo trộn ngẫu nhiên của toàn bộ tập train mỗi epoch, chia cho k worker, không để mỗi worker xáo trộn độc lập.

3.5 Thiết lập thực nghiệm chính

ResNet-50 (biến thể có stride-2 trên conv 3x3), Nesterov momentum $m = 0.9$, weight decay $\lambda = 10^{-4}$ (không áp dụng cho hệ số BN γ, β), $n = 32$/worker cố định, 90 epoch, $\eta = 0.1 \cdot \frac{kn}{256}$ (reference lr), giảm $\times 0.1$ tại epoch 30/60/80. Khởi tạo đặc biệt: hệ số scale γ của **BN cuối mỗi residual block** đặt $= 0$ (khác chuẩn $\gamma = 1$) giúp tín hiệu lan truyền qua identity shortcut lúc đầu, dễ optimize hơn, đặc biệt với minibatch lớn. Hardware: Facebook Big Basin, 8 Tesla P100/server (NVLink), 50Gbit Ethernet, Caffe2 + Gloo (allreduce dùng NCCL nội server, thuật toán halving/doubling hoặc binary-blocks giữa server).

4. Thí nghiệm và kết quả chính

4.1 Kết quả chính: so sánh 3 chiến lược warmup (Bảng 1)

Cấu hình	k	kn	η	Top-1 error (%)
Baseline (1 server)	8	256	0.1	23.60 ± 0.12
Không warmup	256	8k	3.2	24.84 ± 0.37
Constant warmup	256	8k	3.2	25.88 ± 0.56
Gradual warmup	256	8k	3.2	**23.74 ± 0.09**

Ví dụ tự tính (linear scaling rule): từ baseline $\eta = 0.1$ tại $kn = 256$, với $kn = 8192$ ta có $k = 8192/256 = 32$ (tự tính); áp dụng quy tắc $\hat{\eta} = k\eta = 32 \times 0.1 = 3.2$ (tự tính), đúng khớp giá trị $\eta = 3.2$ paper dùng cho mọi thí nghiệm 8k ở Bảng 1 và Bảng 2.

Gradual warmup đạt error chỉ cao hơn baseline 0.14 điểm % (23.74 so với 23.60), trong khi không-warmup cao hơn 1.24 điểm % và constant-warmup cao hơn 2.28 điểm %; constant warmup thậm chí **tệ hơn** không-warmup, vì cú nhảy lr đột ngột sau giai đoạn thấp gây lỗi tăng vọt mà huấn luyện ”không hồi phục hoàn toàn” (nguyên văn).

4.2 Minibatch size vs. error (Hình 1, mục 5.3)

Quét minibatch 64 đến 65.536 (64k) với linear scaling rule + gradual warmup cho $kn > 256$: validation error **ổn định** từ 64 đến 8k, sau đó **tăng nhanh**; vượt 64k thì **phân kỳ** (không hội tụ) với linear scaling rule thuần.

4.3 Các quy tắc scaling lr thay thế (Bảng 2a)

kn	η	Top-1 error (%)
256	0.10	23.60 ± 0.12 (tốt nhất ở 256)
8k	0.10×32 (linear)	23.74 ± 0.09 (tốt nhất ở 8k)
8k	0.10 (không scale)	41.67 ± 0.10
8k	$0.10 \times \sqrt{32}$ (square-root rule, Krizhevsky)	26.22 ± 0.03

Ví dụ tự tính (square-root rule): $\sqrt{32} \approx 5.657$, nên $\eta = 0.1 \times 5.657 \approx 0.566$ (tự tính), thấp hơn nhiều so với 3.2 của linear scaling; kết quả 26.22% rõ ràng kém hơn linear scaling (23.74%) dù vẫn tốt hơn hẳn trường hợp không scale (41.67%, gần như thất bại hoàn toàn vì lr quá thấp so với minibatch).

4.4 BN γ-init (Bảng 2b) và ResNet-101 (Bảng 2c)

$\gamma = 0$ ở BN cuối mỗi residual block cải thiện cả 2 minibatch size, nhưng cải thiện **lớn hơn ở minibatch lớn**: tại $kn = 256$ giảm từ 23.84% xuống 23.60% (chênh 0.24, tự tính); tại $kn = 8k$ giảm từ 24.11% xuống 23.74% (chênh 0.37, tự tính). ResNet-101: baseline $kn = 256$ đạt 22.08±0.06%, $kn = 8k$ đạt 22.36±0.09%, chênh 0.28 điểm % (tự tính, paper ghi ”khoảng 0.3%”); huấn luyện ResNet-101 với $kn = 8k$ trên 256 GPU tốn 92.5 phút.

4.5 ImageNet-5k (6,8 triệu ảnh, Hình 6)

Dùng tập mở rộng 5x dữ liệu (ImageNet-1k + 4k lớp từ ImageNet-22k) để kiểm tra ”điểm gãy” (elbow, khoảng 8k-16k) có phụ thuộc lượng dữ liệu không. Kết quả: đường cong minibatch-vs-error định tính **giống** ImageNet-1k; batch 8k làm tăng error từ 25.83% (batch 256) lên 26.09% (batch 8k), chênh 0.26 điểm % (tự tính). Kết luận: tăng dữ liệu 5x **không** làm ”điểm gãy” minibatch tăng đáng kể; paper để ngỏ mối quan hệ chính xác giữa generalization error, minibatch size, và lượng thông tin dữ liệu cho ”future work”.

4.6 Transfer sang detection/segmentation (Mask R-CNN trên COCO, Bảng 3)

- **Pretraining transfer (Bảng 3a):** ResNet-50 pretrain với kn từ 256 đến 16k, dùng để init Mask R-CNN. Box/mask AP giữ gần như không đổi (35.9/33.9 ở 256 xuống 35.8/33.9 ở 8k) chừng nào ImageNet error còn thấp (tới 8k); ở $kn = 16k$ cả ImageNet error (24.79%) và COCO AP (35.1/33.2) đều **suy giảm** đồng thời. Kết luận: miễn ImageNet error được giữ, minibatch lớn **không** làm hại transfer learning.
- **Trực tiếp scale Mask R-CNN (Bảng 3b):** train với 1, 2, 4, 8 GPU ($kn = 2, 4, 8, 16$; lưu ý minibatch ở đây là số ảnh, khác với 512 RoI/ảnh cho loss phân loại/box/mask), linear lr scaling, box/mask AP gần như không đổi (35.6-35.7 / 33.5-33.7) qua mọi cấu hình.

4.7 Hiệu năng hệ thống (Hình 7, 8, mục 5.5)

Thời gian mỗi iteration chỉ tăng 12% khi minibatch tăng 44 lần (256 đến 11k) nhờ communication được pipeline song song với backprop; thời gian mỗi epoch giảm gần tuyến tính từ hơn 16 phút xuống 30 giây; với 352 GPU (44 server), một epoch đầy đủ ImageNet (~1,28 triệu ảnh) chỉ mất khoảng 30 giây. Hiệu suất scale từ 8 lên 256 GPU đạt khoảng 90% (so với lý tưởng tuyến tính).

5. Phân tích / ablation

- **Đường cong training khớp nhau là chỉ báo sớm cho thành công (mục 5.3, Hình 3):** khi validation error cuối khớp baseline ($kn \leq 8k$), đường cong training error cũng khớp sát sau giai đoạn warmup; khi validation error KHÔNG khớp ($kn \geq 16k$), đường cong training cũng lệch rõ xuyên suốt. Gợi ý dùng đường cong training sớm để chẩn đoán mà không cần chờ hội tụ.
- **Thay đổi η (không đi kèm đổi minibatch) làm lệch hình dạng đường cong (Hình 5)**, dù error cuối có thể tương đương; ngược lại thay đổi minibatch kèm scale η tuyến tính giữ **cả hình dạng đường cong** gần như nguyên vẹn (Hình 3) - gợi ý một bất biến ẩn (invariance) được bảo toàn bởi linear scaling rule mà chỉ đổi η đơn thuần không có.
- **Optimization, không phải generalization, là vấn đề chính (mục 5.2):** phần lớn chênh lệch validation error khi không dùng warmup đến từ chênh lệch training error tương ứng (~2.1% training error cao hơn khớp với ~1.2% validation error cao hơn ở cấu hình không-warmup), không phải do overfit hay nguyên nhân khác liên quan tổng quát hoá. Đây là kết luận **đối lập trực tiếp** với một số nghiên cứu trước đó mà paper trích dẫn (bao gồm Keskar et al. 2016, opt 24) về ”generalization gap” cố hữu của batch lớn: paper này cho rằng (ít nhất trên ImageNet, với warmup đúng) đó chủ yếu là vấn đề optimization có thể khắc phục, không phải một giới hạn generalization cơ bản.

6. Giới hạn

Tác giả tự nêu:

- Quan hệ chính xác giữa generalization error, minibatch size và ”lượng thông tin” của dataset vẫn ”mở cho future work” (mục 5.3).
- Communication aggregation có thể trở thành bottleneck khi số tham số tiếp tục tăng hoặc GPU nhanh hơn; các kỹ thuật như gradient lượng tử hoá hay Block-Momentum SGD được nêu nhưng ”ngoài phạm vi” bài này.
- Không giải thích lý thuyết đầy đủ cho chính linear scaling rule, chỉ đưa ”thảo luận không chính thức” (informal discussion) dựa trên giả định mạnh $\nabla l(x, w_t) \approx \nabla l(x, w_{t+j})$, và tự nhận giả định này rõ ràng sai ở 2 trường hợp (đầu huấn luyện, k quá lớn).
- Cấu hình $kn \geq 12k$ được **mô phỏng** bằng gradient accumulation trên 1 server (không phải train phân tán thật) do hạn chế phần cứng lúc đó; paper nói đã ”kiểm chứng kỹ” 2 cách cho kết quả tương đương nhưng đây vẫn là một giả định cần lưu ý khi đọc Hình 1 ở vùng minibatch rất lớn.

Người ghi chú nhận xét:

- Kết luận ”optimization, không phải generalization, là vấn đề chính” chỉ được kiểm trên ImageNet/ResNet với đúng một kiến trúc họ; paper tự thừa nhận điều này ”ít nhất trên ImageNet” (nguyên văn ”at least on ImageNet”), nên không nên tổng quát hoá sang mọi bài toán/kiến trúc mà không kiểm lại.
- Khoảng tin cậy của nhiều kết quả khá rộng so với chênh lệch đang so sánh: ví dụ chênh lệch baseline/gradual-warmup là 0.14 điểm % nhưng std của ước lượng là ~0.1 điểm % (paper tự nói), nên khác biệt này nằm gần biên nhiễu thống kê, cần cẩn trọng khi diễn giải là ”khớp hoàn toàn”.

7. Liên hệ với các bài khác

- **opt 22 (McCandlish, Empirical Model of Large-Batch Training):** cung cấp lý thuyết (gradient noise scale, critical batch size) giải thích *vì sao* có một ”điểm gãy” minibatch (ở đây thực nghiệm thấy khoảng 8k-16k trên ImageNet) mà bài Goyal chỉ quan sát thực nghiệm.
- **opt 23 (Don’t Decay the LR, Increase the Batch Size):** lấy chính linear scaling rule của bài này làm nền, nhưng đảo vai trò: thay vì cố định epoch và tăng batch để **tăng tốc độ song song hoá** (mục tiêu của Goyal), Smith et al. tăng batch **để thay thế việc giảm lr** trong suốt một lịch huấn luyện cố định.
- **opt 24 (Keskar, sharp minima):** paper này trích dẫn trực tiếp Keskar et al. (2016) và đi đến kết luận ngược: với warmup đúng, generalization gap giữa batch nhỏ và 8k là tối thiểu, mâu thuẫn với tường thuật ”batch lớn luôn gây sharp minima/generalization gap lớn”.
- **opt 19 (Super-Convergence, file tại optimization content/19_2017_super_convergence.md):** paper super-convergence trích dẫn trực tiếp Goyal et al. ở phần thảo luận, coi warmup là ”một bản rời rạc hoá của CLR”, và cũng tìm thấy generalization gap không đổi theo batch size, cùng chiều với phát hiện mục 5.2 ở đây.
- **llm content/, rl content/, distillation content/, pruning content/, interp content/ (thư mục cũ):** đã `ls` kiểm tra, không thấy bài nào bàn trực tiếp về linear scaling rule hay các Remark cài đặt phân tán ở đây nên không trỏ số bài cụ thể.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU ≤ 8 GB, batch 1). Toàn bộ mục này là suy luận của người ghi chú:

1. **Linear scaling rule và warmup cho batch cực lớn ”ít liên quan trực tiếp”**, vì dự án không có cụm nhiều GPU để tăng batch song song; mục tiêu của paper (giữ nguyên accuracy khi tăng batch để tăng tốc song song hoá) không khớp với ràng buộc batch 1 cố định của dự án.
2. **Ý tưởng gradual warmup vẫn hữu ích ở quy mô nhỏ**, bất kể batch size: dùng vài trăm/vài nghìn bước đầu với lr thấp tăng dần trước khi vào lr mục tiêu giúp tránh ”lỗi tăng vọt” khi fine-tune/distill, đúng như quan sát ở mục 4.1 (constant warmup còn tệ hơn không-warmup vì cú nhảy đột ngột) - gợi ý luôn dùng **gradual**, không dùng constant, khi cần warmup cho bất kỳ lr nào.
3. **Các ”Remark” về cài đặt (weight decay, chuẩn hoá loss theo batch, momentum correction khi lr đổi) đáng tham khảo cho bất kỳ pipeline train/distill nào**, dù chạy trên 1 GPU: ví dụ nếu lr thay đổi theo lịch (warmup hay decay) mà dùng optimizer có momentum dạng (10) thay vì dạng tham chiếu (9), nên áp dụng ”momentum correction” (Remark 2) để tránh bất ổn khi lr tăng nhanh.
4. **BN γ-init = 0 ở lớp cuối mỗi block** là một mẹo khởi tạo cụ thể cho kiến trúc có residual/skip connection; nếu VLM OCR có thành phần dùng BN (ít khả năng với kiến trúc Transformer/SSM hiện đại, thường dùng LayerNorm/RMSNorm), mẹo này không áp dụng trực tiếp, nhưng nguyên lý chung ”giúp tín hiệu lan truyền qua identity/skip path lúc đầu, dễ optimize” có thể gợi ý cách khởi tạo tương tự cho gate của GatedDeltaNet.

9. Thí nghiệm gợi ý

1. **So sánh 3 chiến lược warmup ở quy mô nhỏ:** trên 1 GPU, fine-tune một model nhỏ với không-warmup, constant-warmup, gradual-warmup (cùng lr mục tiêu), đo training loss những bước đầu để kiểm tra có hiện tượng ”spike không hồi phục” của constant-warmup như Hình 2b không.
2. **Kiểm tra linear vs square-root scaling ở batch nhỏ (4, 16, 64):** vì không có nhiều GPU, có thể mô phỏng ”tăng batch” bằng gradient accumulation (đúng như paper mô phỏng $kn \geq 12k$), so sánh 2 quy tắc scale lr để xem công thức nào khớp đường cong training tốt hơn ở quy mô nhỏ.
3. **Đo độ nhạy momentum correction:** bật/tắt hệ số hiệu chỉnh η_{t+1}/η_t (Remark 2) khi dùng warmup có độ dốc lớn (ví dụ tăng lr gấp 10 lần trong vài bước), đo xem có xuất hiện bất ổn (loss tăng vọt) khi tắt hiệu chỉnh không.
4. **Transfer learning nhỏ:** pretrain/fine-tune 2 model với 2 batch size khác nhau (cùng linear scaling rule), rồi dùng cả hai làm checkpoint khởi đầu cho một bước fine-tune OCR nhỏ, so sánh chất lượng cuối để xem có lặp lại hiện tượng ”miễn loss pretrain thấp thì transfer không bị ảnh hưởng” (mục 4.6) ở quy mô nhỏ không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Linear Scaling Rule	$\hat{\eta} = k\eta$ khi minibatch $\times k$	không cần siêu tham số mới
Reference lr	$\eta = 0.1 \cdot kn/256$	ResNet-50, baseline $kn = 256, \eta = 0.1$
Gradual warmup	$\eta : \eta_0 \to \hat{\eta}$ tuyến tính trong 5 epoch	tốt hơn hẳn constant warmup
Weight decay đúng (Remark 1)	$w_{t+1} = w_t - \eta\lambda w_t - \eta\frac{1}{n}\sum \nabla\epsilon$	scale loss $\neq$ scale lr
Momentum correction (Remark 2)	hệ số η_{t+1}/η_t nhân vào v_t khi η đổi	quan trọng khi η tăng
Chuẩn hoá gradient (Remark 3)	chia theo kn (tổng), không chia n	vì allreduce chỉ sum
BN batch size n	cố định ($n = 32$) dù k đổi	kn đổi, n không đổi
Kết quả chính	$kn = 8k$: 23.74±0.09% so với baseline $kn = 256$: 23.60±0.12%	gradual warmup, 256 GPU, 1 giờ
Điểm gãy minibatch	ổn định tới ~8k, tăng mạnh sau đó, phân kỳ sau ~64k	ImageNet-1k và ImageNet-5k tương tự

11. Kết luận ngắn

Bằng linear scaling rule (nhân lr theo đúng hệ số tăng minibatch) cộng với gradual warmup (tăng dần lr vài epoch đầu) và một loạt chỉnh sửa cài đặt cẩn thận (weight decay, momentum correction, chuẩn hoá gradient theo tổng minibatch, giữ cố định batch-size cho BN, shuffle đúng cách), Goyal et al. huấn luyện ResNet-50 trên ImageNet với minibatch 8.192 trong 1 giờ trên 256 GPU, không mất accuracy so với baseline minibatch 256, cả về validation error cuối lẫn hình dạng đường cong training. Kết quả cho thấy, ít nhất trên ImageNet với warmup đúng, trở ngại chính của batch lớn là **optimization** (có thể khắc phục) hơn là một giới hạn **generalization** cơ bản, và quy tắc này tiếp tục đúng khi chuyển sang object detection/segmentation (Mask R-CNN). Giới hạn thực nghiệm (điểm gãy khoảng 8k-16k trên ImageNet) không đổi nhiều khi tăng dữ liệu 5x, và được giải thích sâu hơn ở opt 22 qua khái niệm gradient noise scale.

Nguồn gốc: https://arxiv.org/abs/1706.02677

Bài 22

An Empirical Model of Large-Batch Training (2018)

Tác giả: Sam McCandlish, Jared Kaplan (Johns Hopkins University, OpenAI), Dario Amodei (OpenAI), và OpenAI Dota Team (đo đạc riêng phần RL Dota).
Venue / arXiv: arXiv:1812.06162 (v1, 14/12/2018); paper không nêu venue.
Nguồn: https://arxiv.org/abs/1812.06162
Chủ đề: định nghĩa "gradient noise scale" B_{noise} (và bản đơn giản B_{simple}) để dự đoán kích thước batch tối đa còn hữu ích ("critical batch size" B_{crit}) trên nhiều domain (supervised, RL, generative), giải thích vì sao giới hạn này khác nhau giữa các bài toán.
Vị trí trong lộ trình: đọc sau opt 21 (Goyal, Large Minibatch SGD, được trích dẫn trực tiếp trong bài này làm minh chứng thực nghiệm cho batch lớn trên ImageNet) và cùng nhóm với opt 23 (Don't Decay the LR, Increase the Batch Size, cũng được trích dẫn trực tiếp ở đây); đọc trước opt 10 (LAMB, áp dụng ý tưởng batch lớn cho BERT) và liên quan chặt tới llm 03 (Scaling Laws Kaplan, cùng tác giả Kaplan, dùng trực tiếp công thức $B_{crit}(L)$ của bài này).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Nhiều paper (bao gồm opt 21, Goyal et al., batch 8 nghìn trên ImageNet) đã chứng minh **thực nghiệm** rằng với một số domain, batch rất lớn vẫn song song hoá gần như tuyến tính mà không hại data efficiency. Nhưng giới hạn này khác nhau rất nhiều giữa domain: vài chục nghìn cho ImageNet, tới hàng triệu cho agent RL chơi Dota 2. Trước paper này, "hiểu biết khái niệm" (conceptual understanding) về lý do khác biệt này, hoặc cách chọn trước batch size cho một domain mới, còn rất hạn chế; thực hành phổ biến là dò thử (trial-and-error), tốn kém vì batch lớn thường cần warmup hay lịch lr đặc biệt mới hoạt động tốt.

Bài toán cơ bản: tối thiểu hoá $L(\theta) = \mathbb{E}_{x\sim\rho}[L_x(\theta)]$; ước lượng gradient từ 1 batch kích thước B:

$$G_{est}(\theta) = \frac{1}{B}\sum_{i=1}^{B} \nabla_\theta L_{x_i}(\theta), \quad x_i \sim \rho \tag{2.1}$$

$$\mathbb{E}[G_{est}] = G(\theta), \qquad \text{cov}(G_{est}) = \frac{1}{B}\Sigma(\theta) \tag{2.2}$$

với $\Sigma(\theta) = \text{cov}_{x\sim\rho}(\nabla_\theta L_x(\theta))$ là ma trận hiệp phương sai gradient từng mẫu (2.3). Batch nhỏ cho ước lượng gradient nhiễu cao; batch lớn cho ước lượng gần đúng gradient thật nhưng tốn nhiều compute hơn cho mỗi bước cập nhật. Câu hỏi: điểm chuyển tiếp giữa 2 vùng này ở đâu, và nó phụ thuộc gì?

2. Ý tưởng chính

Hình dung trực giác (Hình 1-3 của paper): khi batch nhỏ hơn nhiều so với "quy mô nhiễu", tăng batch giúp **tuyến tính** tăng tiến bộ tối ưu mỗi bước (vùng "time-efficient", batch nhỏ lãng phí vì bước nào cũng gần như nhiễu thuần). Khi batch lớn hơn nhiều quy mô nhiễu, 2 batch ngẫu nhiên cho gradient gần như giống nhau, tăng batch thêm không giúp gì (vùng "compute-efficient" bị lãng phí). Điểm chuyển tiếp xảy ra khoảng khi phương sai gradient và độ lớn gradient cân bằng nhau, định lượng hoá thành **gradient noise scale** B_{noise}. Paper đưa ra một mô hình toán đơn giản (xấp xỉ bậc hai cục bộ) cho tiến bộ tối ưu kỳ vọng mỗi bước theo B, suy ra công thức tường minh cho B_{noise}, và kiểm chứng trên 8 bài toán rất khác nhau (từ MNIST tới Dota

5v5, chênh nhau 6 bậc độ lớn về B_{crit}) rằng phiên bản đơn giản B_{simple} **dự đoán đúng cấp độ lớn** của batch size tối ưu thực nghiệm B_{crit}.

3. Phương pháp chi tiết

3.1 Suy ra noise scale (mục 2.2)

Khai triển Taylor bậc hai của loss thật quanh θ theo bước ϵV: $L(\theta - \epsilon V) \approx L(\theta) - \epsilon G^T V + \frac{1}{2}\epsilon^2 V^T H V$ (2.4), G, H là gradient/Hessian thật. Với gradient thật không nhiễu, bước tối ưu là $\epsilon_{max} = |G|^2/(G^T H G)$. Với gradient nhiễu G_{est} từ batch B, tối thiểu hoá $\mathbb{E}[L(\theta - \epsilon G_{est})]$ (dùng (2.2)) cho bước tối ưu:

$$\epsilon_{opt}(B) = \frac{\epsilon_{max}}{1 + B_{noise}/B} \tag{2.6}$$

và cải thiện loss kỳ vọng tương ứng:

$$\Delta L_{opt}(B) = \frac{\Delta L_{max}}{1 + B_{noise}/B}, \qquad \Delta L_{max} = \frac{1}{2}\frac{|G|^4}{G^T H G} \tag{2.7}$$

$$\boxed{B_{noise} = \frac{\mathrm{tr}(H\Sigma)}{G^T H G}} \tag{2.8}$$

B_{noise} không phụ thuộc kích thước tập huấn luyện (chỉ phụ thuộc phân phối dữ liệu). Nếu dùng bước $> 2\epsilon_{opt}$, loss có thể tăng (phân kỳ).

3.2 Bản đơn giản hoá B_{simple} (dùng trong thực nghiệm)

Giả định H tỉ lệ với ma trận đơn vị (điều kiện hoá hoàn hảo, không thực tế nhưng rẻ để tính):

$$\boxed{B_{simple} = \frac{\mathrm{tr}(\Sigma)}{|G|^2}} \tag{2.9}$$

tức tổng phương sai các thành phần gradient chia cho bình phương độ lớn gradient thật: đo ”gradient lớn so với nhiễu của nó” cỡ nào. Khoảng cách L2 chuẩn hoá kỳ vọng giữa gradient ước lượng và thật: $\mathbb{E}[|G_{est} - G|^2]/|G|^2 = B_{simple}/B$ (2.10). Paper đo được B_{simple} và B_{noise} thường chỉ khác nhau một hệ số hằng nhỏ trong thực nghiệm (đặc biệt khi dùng các kỹ thuật cải thiện conditioning), nên ưu tiên dùng B_{simple} vì rẻ hơn tính (Phụ lục A.1: ước lượng không thiên lệch bằng cách so sánh norm gradient trước/sau khi average giữa các worker trong training phân tán, gần như ”miễn phí” về compute).

3.3 Đánh đổi thời gian/compute và critical batch size (mục 2.3)

Trung bình hoá công thức (2.7) qua nhiều bước tối ưu (Phụ lục D) cho quan hệ giữa tốc độ huấn luyện S (số bước) và hiệu quả dữ liệu E (số mẫu đã xử lý) để đạt một mức hiệu năng cho trước:

$$\boxed{\left(\frac{S}{S_{min}} - 1\right)\left(\frac{E}{E_{min}} - 1\right) = 1} \tag{2.11}$$

với S_{min}, E_{min} là số bước/số mẫu tối thiểu lý thuyết (batch rất lớn và rất nhỏ tương ứng). Vì train ở batch cố định, $E = BS$. Định nghĩa **critical batch size**:

$$B_{crit} = \frac{E_{min}}{S_{min}} \tag{2.12}$$

Mô hình dự đoán $B_{crit} \approx B_{noise}$ (lấy trung bình phù hợp qua quá trình train, Phụ lục D). Tại $B = B_{crit}$, cả 2 vế của (2.11) đều bằng 1, nghĩa là train tốn **gấp đôi** số bước tối thiểu và **gấp đôi** số mẫu tối thiểu so với 2 cực (train hoàn toàn tối ưu thời gian hoặc hoàn toàn tối ưu dữ liệu).

Ví dụ tự tính (liên hệ công thức 2.7 với định nghĩa "50% hiệu quả" ở Hình 4): từ (2.7), tỉ lệ cải thiện loss thực tế so với tối đa là $\Delta L_{opt}(B)/\Delta L_{max} = 1/(1 + B_{noise}/B)$. Tại $B = B_{noise}$: tỉ lệ $= 1/(1+1) = 0.5 = 50\%$ (tự tính) - khớp đúng chú thích Hình 4 của paper định nghĩa critical batch size là "điểm mà compute efficiency rơi xuống dưới 50% tối ưu". Tại $B = 0.1\, B_{noise}$: tỉ lệ $= 1/(1 + 10) \approx 0.091 = 9.1\%$ (tự tính, batch nhỏ, mỗi bước chỉ đạt ~9% tiến bộ tối đa nhưng dùng rất ít dữ liệu mỗi bước). Tại $B = 10\, B_{noise}$: tỉ lệ $= 1/(1 + 0.1) \approx 0.909 = 90.9\%$ (tự tính, batch lớn, gần tối đa nhưng cần 10 lần dữ liệu mỗi bước để đạt đó).

Ví dụ tự tính thứ hai (minh hoạ B_{simple} với số giả định): nếu $\text{tr}(\Sigma) = 100$ và $|G|^2 = 5$ (số giả định, không phải số trong paper), thì $B_{simple} = 100/5 = 20$ (tự tính) - cùng cấp độ lớn với giá trị $B_{crit} = 20$ nhỏ nhất mà paper thực sự đo được (cho bộ tự mã hoá SVHN, mục 3.2), minh hoạ cách một $\text{tr}(\Sigma)/|G|^2$ nhỏ tương ứng batch tối ưu rất nhỏ.

3.4 Các giả định/caveat (mục 2.4, paper tự liệt kê 6 điểm)

1. **Short-horizon bias:** chỉ là bức tranh cục bộ (1 bước), tối ưu tham lam không chắc tối ưu toàn cục.
2. **Poor conditioning:** nếu Hessian xấu, tham số có thể dao động theo hướng độ cong lớn, (2.7) không phản ánh đúng tiến bộ thật mỗi bước (nhưng vẫn phản ánh đúng **tốc độ tương đối** giữa các batch size).
3. B_{simple} **không chính xác** khi Hessian không gần đồng nhất.
4. **Learning rate chưa tối ưu hoàn hảo** trong thực tế làm sai lệch (2.7); B_{noise} phụ thuộc mạnh vào "nhiệt độ" (temperature) qua lr (Phụ lục C).
5. **Xấp xỉ bậc hai** có thể bỏ qua hiệu ứng bậc ba quan trọng.
6. **Generalization không được xét:** mô hình chỉ nói về tối ưu hoá training loss, không về generalization. Paper nêu rõ: "một số paper báo cáo 'generalization gap' nơi batch lớn cho training loss tốt nhưng test loss kém đi, dường như không liên quan overfitting [Keskar et al. 2016]"; nhưng **cũng nêu trực tiếp**: "công việc gần đây [Smith, Kindermans, Chiang, Le 2018 - chính là opt 21, Goyal et al., "SLA+18"] không thấy bằng chứng generalization gap khi siêu tham số được tinh chỉnh đúng".

4. Thí nghiệm và kết quả chính

4.1 Tổng quan (Hình 4)

Trên 8 bài toán (SVHN, MNIST, CIFAR-10, ImageNet [supervised]; Billion Word LSTM [ngôn ngữ]; Atari, Dota [RL]; VAE/Autoencoder SVHN [generative]), B_{simple} dự đoán đúng **cấp độ lớn** của B_{crit} thực nghiệm, dù B_{crit} trải từ **20** (autoencoder SVHN) tới **trên 10 triệu** (Dota 5v5) - chênh khoảng 6 bậc độ lớn.

4.2 Supervised learning

- **SVHN (CNN + SGD):** đo đủ cả B_{crit}, B_{simple}, B_{noise}; B_{noise} (đầy đủ, có trọng số Hessian) dự đoán B_{crit} tốt hơn B_{simple} (đơn giản hoá). Với Adam, B_{simple} lại cho dự đoán "rất chính xác" cho B_{crit}.
- **MNIST (CNN + SGD):** B_{simple} "ước lượng thô" B_{crit}, nhưng B_{crit} thực tế "nhỏ hơn đáng kể".
- **CIFAR-10 (ResNet-32 + Momentum):** B_{simple} dự đoán tốt B_{crit}.
- **ImageNet (ResNet-50 + Momentum, lr giảm 3 lần trong huấn luyện):** cả B_{simple} và B_{crit} thay đổi mạnh theo giai đoạn huấn luyện (phải đo riêng ở mỗi mức lr, xem Phụ lục C). Noise scale dao động từ **2.000 đến 100.000** trong giai đoạn chính của huấn luyện, khớp với kết quả thực nghiệm khác cho thấy batch cố định tới 64.000 có thể dùng mà không mất hiệu quả. Ở giai đoạn fine-tuning cuối, noise scale tăng tiếp lên **hàng trăm nghìn tới hàng triệu**, gợi ý batch còn lớn hơn có thể hữu ích ở cuối huấn luyện. B_{crit} đo được ở đây **thấp hơn** (15k) so với literature (64k), paper tự giải thích vì không dùng kỹ thuật layer-wise adaptive learning rate (LARS) mới nhất.

4.3 Generative modeling

VAE và Autoencoder trên SVHN: VAE có B_{crit} và B_{simple} **lớn hơn** Autoencoder (do VAE có thêm stochasticity); cả hai đều có B_{simple} thấp hơn nhiều so với các bộ phân loại ảnh SVHN. Đây là trường hợp duy nhất (cùng Dota 1v1) mà B_{simple} **nhỏ hơn đáng kể** B_{crit} (xem mục 5).

4.4 Language modeling (LSTM 1 lớp, Billion Word)

B_{crit} và B_{simple} khớp tốt. Theo kích thước LSTM (Hình 8): noise scale **gần như độc lập** kích thước model tại cùng giá trị loss cố định, nhưng model lớn hơn cuối cùng đạt loss thấp hơn nên đạt noise scale cao hơn - **chỉ qua cơ chế loss thấp hơn**, không phải qua số tham số trực tiếp (số tham số ”triệt tiêu” khỏi định nghĩa noise scale).

4.5 Reinforcement learning

- **Atari (A2C, 7 game: Alien, Beamrider, Breakout, Pong, Qbert, Seaquest, Space Invaders):** đường cong đánh đổi khớp khá tốt với (2.11), có nhiễu (ví dụ Beam Rider lệch ở batch rất nhỏ, có thể do nhạy với siêu tham số khác như momentum).
- **Dota (PPO, 1v1 và 5v5 - 5v5 là ”công việc sơ bộ đang tiếp diễn”):** Dota 5v5 có noise scale ”**ít nhất 10 triệu**”, lớn hơn hẳn Dota 1v1, lý giải bởi đa dạng tình huống lớn hơn (5 hero mỗi bên, toàn bản đồ) sinh ra phương sai gradient lớn hơn. Do hạn chế tài nguyên, không đo được Pareto front đầy đủ cho 5v5, chỉ báo cáo batch size mà team Dota đã dùng (như cận dưới) và noise scale đo được.

4.6 Model size dependence

Số tham số không xuất hiện trực tiếp trong định nghĩa noise scale. Với LSTM (kích thước có thể tăng mà không đổi kiến trúc về chất), noise scale ”gần như độc lập kích thước model tại loss cố định” được xác nhận (Hình 8); model lớn đạt noise scale cao hơn **chỉ vì** đạt loss thấp hơn.

5. Phân tích / ablation

- **Tỉ lệ B_{simple}/B_{crit} thay đổi khoảng một bậc độ lớn giữa các task** (mục 5, Discussion): nhỏ hơn 1 cho Autoencoder, VAE, và Dota 1v1; xấp xỉ 1 cho LSTM; lớn hơn 1 cho cả 2 bài toán phân loại ảnh và Atari. Paper tự nhận ”chúng tôi thiếu lời giải thích cho những biến thiên này” (nguyên văn ”we lack an explanation”).
- **So sánh RL Dota vs Atari:** noise scale của Dota ”khoảng 1000 lần” lớn hơn Atari, nhưng tổng số bước tối ưu cần để huấn luyện Dota agent ”không lớn hơn nhiều” so với Atari theo tỉ lệ đó; paper suy đoán phần lớn compute bổ sung cần cho model mạnh hơn ”có thể song song hoá được” (nghĩa là nằm ở chiều E/batch, không phải chiều S/số bước tuần tự).
- **Optimal learning rate theo batch size (Hình 5):** với SGD/momentum, lr tối ưu theo đúng dạng hàm của (2.6) (tăng tuyến tính rồi bão hoà); với Adam/RMSProp, lr tối ưu ban đầu theo luật luỹ thừa $\epsilon(B) \propto B^{\alpha}$ với α giữa 0.5 và 1.0 (tuỳ task), sau đó gần như hằng số; điểm bão hoà lr thường **nhỏ hơn một chút** so với noise scale đo được.
- **Overfitting không ảnh hưởng nhiều kết luận:** trên MNIST/CIFAR-10/SVHN (dataset nhỏ, dễ overfit), đo test error (Phụ lục E.3) cho Pareto front và B_{crit} ”nói chung không thay đổi nhiều” so với dùng train loss.

6. Giới hạn

Tác giả tự nêu (mục 2.4, 5, và rải trong bài):

- Không giải thích lý thuyết đầy đủ cho các biến thiên tỉ lệ B_{simple}/B_{crit} giữa task (nêu rõ ở mục 5).
- Mô hình hoàn toàn bỏ qua generalization (chỉ về tối ưu training loss), dù thừa nhận có tồn tại lo ngại ”generalization gap” trong literature (Keskar et al. 2016) mà bài này không loại trừ nhưng cũng dẫn chứng công trình khác (Goyal et al., opt 21) không thấy bằng chứng khi tune đúng.

- B_{noise} phụ thuộc mạnh vào lr qua ”temperature” (Phụ lục C); đo không đúng khi lr chưa được tune tốt sẽ cho số bị thổi phồng.
- Với Dota 5v5, do hạn chế tài nguyên, không đo được B_{crit} thật, chỉ có cận dưới từ batch mà team Dota dùng thực tế, nên con số ”ít nhất 10 triệu” là chưa đầy đủ kiểm chứng như các task khác.
- Thí nghiệm dùng grid search riêng cho lr ở mỗi batch size (không phải một công thức đóng), nên quy trình đo không hoàn toàn ”hyperparameter-free” như linear scaling rule của opt 21 mà chỉ là một phương pháp đo thực nghiệm.

Người ghi chú nhận xét:

- Bản thân công thức (2.7)-(2.9) và (2.11)-(2.12) sau này được tái sử dụng gần như nguyên văn trong llm 03 (Kaplan et al., Scaling Laws for Neural Language Models, mục 5.1 ở đó), nhưng Kaplan et al. khớp $B_{crit}(L) = B_*/L^{1/\alpha_B}$ theo một luật luỹ thừa cụ thể cho riêng LM, trong khi bài này đo trực tiếp trên nhiều domain khác nhau mà không khớp một hàm chung theo loss xuyên-domain; hai bài bổ sung cho nhau (một cho khung lý thuyết + đa domain, một cho luật thực nghiệm chi tiết trên 1 domain).
- Phạm vi đọc: đã đọc toàn bộ mục 1-5 (Introduction, Theory, Experiments, Related Work, Discussion) và lướt qua tiêu đề/nội dung tổng quát các phụ lục A-E (Methods, Results for All Tasks, Temperature, Dynamically Varying Batch Size, Comments on Optimization) để xác nhận phạm vi và kết luận chung; không đi sâu vào mọi hình/bảng số liệu chi tiết trong Phụ lục B (kết quả đầy đủ từng task) do paper tự coi đó là phần mở rộng của Hình 4 ở thân bài.

7. Liên hệ với các bài khác

- **opt 21 (Goyal, Large Minibatch SGD):** được trích dẫn trực tiếp ngay trong đoạn mở đầu (”batch sizes of 8 thousand [GDG+17]”) làm một trong các minh chứng thực nghiệm cho batch lớn; đồng thời được dẫn lại ở mục 2.4 (giả định 6, generalization) như công trình ”không thấy generalization gap khi tune đúng”, gián tiếp ủng hộ kết luận mục 5.2 của opt 21.
- **opt 23 (Don’t Decay the LR, Increase the Batch Size):** được trích dẫn trực tiếp ở Related Work (”[SKYL17] showed that decaying the learning rate and increasing the batch size have the same effect, motivated by the SGD training temperature”); bài này cung cấp đúng khái niệm ”temperature” mà opt 23 dùng để lý giải sự tương đương lr/batch.
- **llm 03 (Scaling Laws Kaplan, đã tồn tại tại llm content/03_2020_scaling_laws_kaplan.md):** dùng trực tiếp công thức B_{crit}, $(S/S_{min} - 1)(E/E_{min} - 1) = 1$ của bài này (với α_S, α_B khớp riêng cho LM), cùng đồng tác giả Kaplan.
- **opt 10 (LAMB, Large Batch Optimization for Deep Learning):** áp dụng và mở rộng ý tưởng batch rất lớn (tới hàng chục nghìn) sang BERT, nằm đúng trong vùng ”batch gần/vượt B_{crit}” mà khung lý thuyết ở đây dự đoán sẽ gặp hiệu quả giảm dần.
- **llm content/, rl content/, distillation content/, pruning content/, interp content/ (thư mục cũ):** đã `ls` kiểm tra, không có bài nào bàn trực tiếp gradient noise scale/critical batch size nên không trỏ số bài cụ thể ở các thư mục này.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU ≤ 8 GB, batch 1). Toàn bộ mục này là suy luận của người ghi chú:

1. **Batch 1 nằm RẤT XA B_{crit} của hầu hết task thực tế** (paper đo B_{crit} nhỏ nhất là 20, cho một bài toán generative đơn giản trên SVHN); với một VLM OCR phức tạp hơn nhiều, B_{crit} gần như chắc chắn lớn hơn 1 nhiều bậc độ lớn, nên **tăng batch từ 1 lên vài chục/vài trăm (nếu GPU cho phép) sẽ gần như tuyến tính hữu ích** (vùng $B \ll B_{noise}$ trong mô hình của paper), không lãng phí compute. Đây không phải lợi ích tốc độ thực tế của dự án (vì GPU bị khoá cứng ở khả năng chứa batch 1 do VRAM), nhưng

là một lưu ý nếu có thể dùng gradient accumulation để ”giả lập” batch lớn hơn 1 mà không tăng VRAM đỉnh.

2. **Gradient noise scale như một chỉ số chẩn đoán (suy luận):** có thể đo $B_{simple} = \text{tr}(\Sigma)/|G|^2$ với chi phí gần như miễn phí (Phụ lục A.1: so sánh norm gradient trước/sau average) ngay cả ở batch nhỏ, để biết liệu bước cập nhật ở batch 1 đang chủ yếu là ”nhiễu” (nếu $B_{simple} \gg 1$) hay đã gần ”tín hiệu đầy đủ” (nếu $B_{simple} \approx 1$); nếu B_{simple} rất lớn, có thể cân nhắc gradient accumulation vài bước trước khi update để giảm lãng phí các bước gần như thuần nhiễu.
3. **Noise scale tăng theo quá trình huấn luyện (mục 2.5, 4.2):** gợi ý rằng early trong fine-tune/distill, batch nhỏ (hoặc ít accumulation) đã đủ hiệu quả; nhưng khi model gần hội tụ (loss thấp), B_{crit} tăng lên, có thể cân nhắc tăng accumulation ở giai đoạn cuối fine-tune nếu muốn tận dụng tốt hơn các bước cập nhật hiếm hoi trên GPU nhỏ.
4. **Caveat về generalization (mục 6 ở đây):** paper chỉ mô hình hoá tối ưu training loss, không đảm bảo gì về chất lượng OCR thực tế (CER/WER); việc đo B_{simple} chỉ cho biết ”nên tăng batch/accumulation không về mặt tối ưu hoá”, không thay thế việc đánh giá trên tập validation OCR.

9. Thí nghiệm gợi ý

1. **Đo B_{simple} trên một model nhỏ:** cài công thức (A.1)-(A.2) của Phụ lục A.1 (so sánh norm gradient ở 2 batch size, ví dụ $B_{small} = 1$ và $B_{big} = 8$ qua gradient accumulation) trên một MLP/CNN nhỏ huấn luyện MNIST/CIFAR-10, theo dõi B_{simple} thay đổi theo thời gian huấn luyện, kiểm tra có tăng dần như paper dự đoán (mục 2.5) không.
2. **Pareto front nhỏ:** với cùng model, train ở vài batch size nhỏ khác nhau (1, 2, 4, 8, 16 qua accumulation), đo số bước và số mẫu cần để đạt một mức loss cố định, fit vào công thức (2.11) để ước lượng B_{crit} thực nghiệm, so với B_{simple} đo ở mục 1.
3. **Kiểm tra độ nhạy lr theo batch (Hình 5):** quét lr tối ưu cho vài batch size nhỏ, kiểm tra có theo dạng (2.6) (tuyến tính rồi bão hoà) với SGD/momentum, và dạng luỹ thừa với Adam/AdamW như paper quan sát không.
4. **So sánh noise scale ở các giai đoạn fine-tune khác nhau:** đo B_{simple} ở đầu và cuối một lần fine-tune/distill thật (trên bài toán OCR nhỏ), kiểm tra xu hướng tăng theo paper, và xem liệu gradient accumulation lớn hơn ở cuối fine-tune có cải thiện tốc độ hội tụ hiệu dụng không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Gradient noise scale (đầy đủ)	$B_{noise} = \text{tr}(H\Sigma)/(G^T HG)$	eq. (2.8)
Gradient noise scale (đơn giản)	$B_{simple} = \text{tr}(\Sigma)/\|G\|^2$	eq. (2.9), dùng chính trong thực nghiệm
Cải thiện loss mỗi bước	$\Delta L_{opt}(B) = \Delta L_{max}/(1 + B_{noise}/B)$	eq. (2.7)
Đánh đổi thời gian/compute	$(S/S_{min} - 1)(E/E_{min} - 1) = 1$	eq. (2.11)
Critical batch size	$B_{crit} = E_{min}/S_{min} \approx B_{noise}$	eq. (2.12); tại $B = B_{crit}$: 2x S_{min}, 2x E_{min}
50% hiệu quả	$B = B_{noise} \Rightarrow \Delta L_{opt}/\Delta L_{max} = 0.5$	tự tính từ (2.7); định nghĩa B_{crit} ở Hình 4
B_{crit} đo được, trải rộng	20 (SVHN autoencoder) đến $> 10^7$ (Dota 5v5)	Hình 4, 6 bậc độ lớn
Phụ thuộc kích thước model	gần như độc lập tại loss cố định	chỉ tăng qua đạt loss thấp hơn

11. Kết luận ngắn

Bài báo xây dựng một mô hình toán đơn giản (xấp xỉ bậc hai local) cho thấy tiến bộ tối ưu mỗi bước bão hoà dần khi batch vượt quá ”gradient noise scale” $B_{noise} \approx \text{tr}(H\Sigma)/(G^T HG)$, hoặc bản đơn giản $B_{simple} = \text{tr}(\Sigma)/|G|^2$; đại lượng này dự đoán đúng cấp độ lớn của ”critical batch size” B_{crit} (điểm compute-efficiency rơi xuống 50%) trên 8 bài toán rất khác nhau, trải 6 bậc độ lớn từ 20 đến hơn 10 triệu. Noise scale tăng theo quá trình huấn luyện (khi loss giảm), gần như độc lập số tham số model tại loss cố định, và lớn hơn cho task phức tạp hơn. Khung lý thuyết này, cùng với các khái niệm B_{crit}, temperature, được tái sử dụng trực tiếp trong các paper sau (đáng chú ý nhất là llm 03, Scaling Laws, cùng tác giả Kaplan) và giải thích về mặt lý thuyết cho các kết quả thực nghiệm batch lớn như opt 21.

Nguồn gốc: https://arxiv.org/abs/1812.06162

Bài 23

Don’t Decay the Learning Rate, Increase the Batch Size (2017)

Tác giả: Samuel L. Smith, Pieter-Jan Kindermans (đóng góp ngang nhau), Chris Ying, Quoc V. Le (Google Brain).
Venue / arXiv: arXiv:1711.00489 (v2, 24/02/2018); venue: Published as a conference paper at ICLR 2018.
Nguồn: https://arxiv.org/abs/1711.00489
Chủ đề: chứng minh giảm learning rate và tăng batch size trong quá trình huấn luyện tương đương nhau về mặt ”noise scale”; dùng tăng batch (thay giảm lr) để giảm số lần cập nhật tham số, tăng khả năng song song hoá.
Vị trí trong lộ trình: đọc sau opt 21 (Goyal, Large Minibatch SGD - được lặp lại thiết lập trực tiếp ở đây) và opt 22 (McCandlish, Empirical Model of Large-Batch Training - cùng dùng từ ”noise scale” nhưng định nghĩa khác); đọc cùng nhóm với opt 18 (SGDR - cosine decay được nhắc tới như minh chứng cho xu hướng ”anneal dốc hơn”), opt 24 (Keskar - generalization gap, được trích dẫn trực tiếp), opt 19 (Super-Convergence - tác giả Samuel L. Smith khác Leslie N. Smith của bài đó).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Thực hành phổ biến: giảm learning rate (lr) dần trong quá trình huấn luyện (step decay, cosine decay như opt 18, v.v.) để hội tụ tốt. Đồng thời, nhiều paper (Keskar et al. = opt 24; Goyal et al. = opt 21) cho thấy batch lớn thường làm giảm test accuracy (”generalization gap”), trừ khi tune kỹ (như opt 21 đã làm). Câu hỏi của bài này: nếu lý do lr cần giảm không phải vì giá trị lr bản thân nó, mà vì **mức độ nhiễu ngẫu nhiên** mà SGD đưa vào quá trình tối ưu, thì có thể đạt hiệu ứng tương tự bằng cách **tăng batch size** (giảm nhiễu bằng cách khác) mà giữ lr cố định không? Nếu đúng, có thể tận dụng được batch rất lớn (gần bằng tập huấn luyện) mà không mất độ chính xác, đồng thời giảm mạnh số lần cập nhật tham số (song song hoá tốt hơn, huấn luyện nhanh hơn).

Cơ sở lý thuyết (Robbins & Monro, 1951) cho SGD trên hàm lồi chặt: để hội tụ cần

$$\sum_{i=1}^{\infty} \epsilon_i = \infty \tag{1}$$

$$\sum_{i=1}^{\infty} \epsilon_i^2 < \infty \tag{2}$$

ϵ_i là lr ở bước cập nhật i. (1) đảm bảo có thể đi tới cực tiểu dù khởi tạo xa, (2) đảm bảo lr giảm đủ nhanh để không ”nảy” quanh cực tiểu do nhiễu gradient. Nhưng (2) **chỉ đúng khi batch size không đổi** - đây là khe hở lý thuyết mà bài báo khai thác: nếu batch size có thể thay đổi, điều kiện thật sự cần có thể được thoả mãn bằng cách tăng batch chứ không cần giảm lr.

2. Ý tưởng chính

Dựa trên phân tích SGD như một phương trình vi phân ngẫu nhiên (SDE) của Smith & Le (2017): $\frac{d\omega}{dt} = -\frac{dC}{d\omega} + \eta(t)$ (3), với $\eta(t)$ là nhiễu Gauss mô phỏng việc ước lượng gradient trên 1 mini-batch, có kỳ vọng $\langle \eta(t) \rangle = 0$ và phương sai $\langle \eta(t)\eta(t') \rangle = g\, F(\omega)\, \delta(t - t')$. Đại lượng g gọi là **noise scale**:

$$g = \epsilon\Big(\frac{N}{B} - 1\Big) \approx \frac{\epsilon N}{B} \quad (B \ll N) \tag{3'}$$

(N = số mẫu tập huấn luyện, B = batch size, ϵ = lr). Ý tưởng cốt lõi: **giảm lr tương đương giảm noise scale** g; nhưng cũng có thể giảm g bằng cách **tăng** B **ở lr cố định** - theo công thức (3'), $g \propto \epsilon/B$, nên tăng B gấp k lần có tác dụng giống giảm ϵ đi k lần. Nếu cách diễn giải "giảm lr = simulated annealing của noise scale" đúng, thì 3 lịch trình: (a) giảm lr giữ batch cố định, (b) giữ lr cố định tăng batch, (c) pha trộn cả hai - miễn là cùng lịch trình giảm g theo cùng hệ số ở cùng mốc epoch - phải cho **đúng cùng learning curve** theo epoch và cùng test accuracy cuối, nhưng (b)/(c) cần **ít lần cập nhật tham số hơn** vì mỗi lần cập nhật xử lý nhiều mẫu hơn.

3. Phương pháp chi tiết

3.1 Noise scale với momentum (mục 4)

Mở rộng cho SGD + momentum m (Smith & Le, 2017):

$$g = \frac{\epsilon}{1-m}\Big(\frac{N}{B} - 1\Big) \approx \frac{\epsilon N}{B(1-m)} \tag{4-5}$$

Định nghĩa **effective learning rate** $\epsilon_{eff} = \epsilon/(1-m)$. Để giảm số lần cập nhật hơn nữa, có thể **tăng** ϵ (giữ $B \propto \epsilon$) hoặc **tăng** m (giữ $B \propto 1/(1-m)$, tức $B \propto \epsilon_{eff}$), nhưng tăng m làm giảm nhẹ test accuracy (lý giải ở mục 3.2 dưới).

Ví dụ tự tính (kiểm chứng công thức 3'): với $\epsilon = 0.1$, $N = 50000$ (CIFAR-10), $B = 128$ (batch ban đầu của "Original training schedule", mục 5.2): $g \approx 0.1 \times 50000/128 \approx 39{,}06$ (tự tính). Nếu tăng batch gấp 5 lần lên $B = 640$ ở cùng $\epsilon = 0.1$: $g' \approx 0.1 \times 50000/640 \approx 7{,}81$ (tự tính), đúng bằng $39{,}06/5$ - **giống chính xác** hiệu ứng của việc giảm ϵ đi 5 lần ở batch cố định, minh hoạ trực tiếp tuyên bố trung tâm của bài báo.

Ví dụ tự tính thứ hai (effective lr và quy tắc scale batch theo momentum): lịch trình "Increased momentum coefficient" (mục 5.2) dùng $\epsilon = 0.5$, $m = 0.98$, batch đầu $B = 3200$; lịch trình gốc dùng $\epsilon = 0.1$, $m = 0.9$, $B = 128$. Effective lr gốc: $\epsilon_{eff} = 0{,}1/(1-0{,}9) = 1{,}0$ (tự tính); effective lr mới: $\epsilon'_{eff} = 0{,}5/(1-0{,}98) = 25$ (tự tính) - gấp 25 lần. Theo quy tắc $B \propto \epsilon_{eff}$: $128 \times 25 = 3200$ (tự tính) - khớp **chính xác** với batch 3200 mà paper chọn, xác nhận quy tắc scale được áp dụng đúng như công thức.

3.2 Biến "accumulation" và số epoch bị "mất" (Phụ lục A)

Phương trình cập nhật SGD + momentum: $\Delta A = -(1-m)A + \frac{d\hat{C}}{d\omega}$ (6), $\Delta\omega = -A$ (7). A ("accumulation") khởi tạo 0, cần thời gian để "lớn lên" tới giá trị ổn định, trong lúc đó bước cập nhật $\Delta\omega$ bị suy giảm biên độ (effective lr nhỏ hơn danh định). Giải phương trình vi phân liên tục hoá (giả định gradient $\approx G$ hằng số đầu huấn luyện): $A = \frac{G}{1-m}\big(1 - e^{-(1-m)(N/B)N_{epochs}}\big)$ (11). Số "epoch bị mất" tương đương:

$$N_{lost} = \int_0^\infty e^{-(1-m)(N/B)N_{epochs}}\, dN_{epochs} = \frac{B}{N(1-m)} \tag{12-13}$$

Vì $B \propto \epsilon/(1-m)$, suy ra $N_{lost} \propto \epsilon/(N(1-m)^2)$ - tăng rất nhanh theo m (bậc 2) khi $m \to 1$.

Ví dụ tự tính (áp vào 2 lịch trình mục 5.2): lịch trình gốc, $B = 128, N = 50000, m = 0{,}9$: $N_{lost} = 128/(50000 \times 0{,}1) \approx 0{,}0026$ epoch (tự tính) - không đáng kể. Lịch trình "Increased momentum", $B = 3200, N = 50000, m = 0{,}98$: $N_{lost} = 3200/(50000 \times 0{,}02) = 3{,}2$ epoch (tự tính) - đáng kể, giải thích phần nào vì sao lịch trình này đạt test accuracy thấp hơn (93,3% so với 94,2-94,5% của các lịch trình khác, mục 4 dưới).

3.3 Chiến lược chuyển đổi lịch trình

Thay vì tune lại từ đầu, paper đề xuất: lấy lịch trình lr-decay đã có sẵn trong literature, **mỗi khi lr giảm hệ số α thì tăng batch gấp α** (giữ lr không đổi), cho tới khi batch đạt khoảng $B \sim N/10$; sau điểm đó quay lại giảm lr như thường (vì quy tắc $B \ll N$ không còn đúng). Paper nhấn mạnh phương pháp này **không cần tune hyperparameter mới**.

4. Thí nghiệm và kết quả chính

4.1 CIFAR-10, Wide ResNet "16-4" (mục 5.1)

Dùng ghost batch norm (ghost batch=128, để gradient trung bình độc lập batch size). So sánh 3 lịch trình cùng giảm noise scale 5 lần ở mỗi "step": "Decaying learning rate" (batch cố định, lr giảm ×5), "Hybrid" (step đầu tăng batch ×5, các step sau giảm lr ×5), "Increasing batch size" (lr cố định, batch tăng ×5 mỗi step). Kết quả: cross-entropy huấn luyện theo **epoch** gần như trùng khớp hoàn toàn cho cả 3 (Hình 2a), nhưng theo **số lần cập nhật tham số**, "Increasing batch size" giảm mạnh số update sau step đầu (Hình 2b). Test accuracy theo epoch cũng gần trùng khớp cho SGD+momentum 0,9 (Hình 3a), Nesterov momentum 0,9 (Hình 3b), vanilla SGD (Hình 4a), và cả Adam với tham số TF mặc định $lr = 10^{-3}, \beta_1 = 0{,}9, \beta_2 = 0{,}999$ (Hình 4b) - khẳng định tính tổng quát qua nhiều optimizer.

4.2 Tăng effective learning rate (mục 5.2)

CIFAR-10, $N = 50000$, đặt $B_{max} = 5120$ (ràng buộc $B \ll N$). 4 lịch trình, cùng giảm noise scale ×5 ở 3 step, cùng số epoch:

Lịch trình	ϵ_0	B_0	m	Số update	Test acc. (1 run)	Test acc. (median 5 runs)
Original training schedule	0,1	128	0,9	~80.000	94,3%	94,3%
Increasing batch size	0,1	128	0,9	~29.000	94,4%	94,2%
Increased initial lr	0,5	640	0,9	<6.500	94,5%	94,2%
Increased momentum coefficient	0,5	3200	0,98	<2.500	93,3%	93,5%

(Paper tự ghi chú: bản gốc Zagoruyko & Komodakis báo cáo 95% nhưng nhóm tác giả "chưa thể tái lập" được số này, chỉ đạt 94,3% - sự khác biệt được chính paper nêu rõ, không che giấu.) Phụ lục B (Hình 8) quét lr ban đầu từ 0,1 đến 3,2 (giữ batch cố định): accuracy giảm khi lr ban đầu lớn hơn khoảng 0,4.

4.3 ImageNet, Inception-ResNet-V2, giảm xuống 2500 update (mục 5.3)

Lặp lại thiết lập Goyal et al. 2017 (opt 21: batch 8192, lr 3,2, momentum 0,9, 90 epoch, giảm ×10 ở epoch 30/60/80, ~14.000 update) nhưng không dùng warm-up, đổi sang Inception-ResNet-V2 làm baseline mạnh hơn, ghost batch=32, lr=3,0: "Decaying learning rate" (batch cố định 8192) đạt 78,7%/77,8% (2 lần chạy); "Increasing batch size" (batch tăng tới 81920 ở step đầu, rồi giảm lr) đạt 78,1%/76,8%, giảm số update từ hơn 14.000 xuống dưới 6.000.

Tiếp tục tăng momentum, $B_{max} = 2^{16} = 65536$, ghost batch=64, lr=3 cố định cho cả 3 lịch trình: "Momentum 0,9" ($B_0 = 8192$) đạt 78,8% trong <6.000 update; "Momentum 0,975" ($B_0 = 16384$) và "Momentum 0,9875" ($B_0 = 32768$) được định nghĩa nhưng **đoạn kết quả lại báo cáo cho "Momentum 0,95" (78,1%/77,8%, <3.500 update) và "Momentum 0,975" (77,5%/76,8%, <2.500 update)** - xem mục 6 (Giới hạn) về sự không khớp tên gọi này.

4.4 ResNet-50 trên TPU trong 30 phút (mục 5.4)

Nửa TPU pod (256 tensorcore): batch 8192 cố định đạt 76,1% accuracy trong <45 phút; tăng batch lên 16384 sau 30 epoch đầu, vẫn đạt đúng 76,1% nhưng trong <30 phút (30 epoch đầu và 60 epoch sau đều mất ~15 phút - song song hoá gần hoàn hảo). Đối chứng: tăng gấp đôi lr ban đầu và dùng batch 16384 **suốt toàn bộ** quá trình (không tăng dần) chỉ đạt 75,0% trong 22 phút - thấp hơn, xác nhận **việc tăng batch dần trong lúc huấn luyện** (không chỉ dùng batch lớn từ đầu) mới là yếu tố quan trọng.

5. Phân tích / ablation

- Hiệu ứng nhất quán qua **4 optimizer khác nhau** (SGD, Momentum, Nesterov, Adam) ở mục 5.1 - đây là bằng chứng mạnh cho tổng quát hoá của ý tưởng "noise scale", không chỉ đặc thù SGD vanilla.
- Tăng ϵ (giữ $B \propto \epsilon$) không làm giảm accuracy đáng kể; tăng m (giữ $B \propto \epsilon/(1-m)$) **làm giảm** accuracy một chút - được paper lý giải bằng phân tích "epoch bị mất" (mục 3.2 ở đây, Phụ lục A của paper): mất ổn định sớm, do biến accumulation cần nhiều epoch hơn để "bắt kịp" khi m lớn.
- Thử nghiệm Kingma & Ba (2014)'s initialization bias correction (tăng lr đầu huấn luyện để bù accumulation nhỏ) nhưng paper nhận thấy với batch lớn cách này "thường gây mất ổn định" - nên Goyal et al. (opt 21) đã chọn một giải pháp khác (giảm lr vài epoch đầu, warm-up).

6. Giới hạn

Tác giả tự nêu:

- Quy tắc $g \approx \epsilon N/B$ chỉ là xấp xỉ khi $B \ll N$; khi B gần N (paper dùng ngưỡng thực hành $B \sim N/10$) công thức không còn chính xác, buộc phải quay lại giảm lr.
- Không tái lập được accuracy 95% của bài gốc Wide ResNet (Zagoruyko & Komodakis), chỉ đạt 94,3% - tự thừa nhận không giải thích được khác biệt này.
- Không xét asynchronous-SGD (vì quy tắc scale cho phép dùng batch cỡ tập huấn luyện trực tiếp).
- Kết quả ImageNet "khá nhiễu" (noisy) vì giảm số lần đánh giá trên test set để tiết kiệm thời gian huấn luyện (mục 5.3).

Người ghi chú phát hiện (không có trong phần "tác giả tự nêu", mục 5.3 của paper): có **sự không khớp tên gọi 3 lịch trình momentum** trong đoạn mô tả thí nghiệm ImageNet mở rộng: paragraph định nghĩa 3 lịch trình là "Momentum 0,9" ($B_0 = 8192$), "Momentum 0,975" ($B_0 = 16384$), "Momentum 0,9875" ($B_0 = 32768$); nhưng paragraph báo cáo kết quả ngay sau đó lại nói "Momentum 0,9" (78,8%), rồi "Momentum 0,95" (78,1%/77,8%), rồi "Momentum 0,975" (77,5%/76,8%) - giá trị momentum thứ 2 và thứ 3 trong kết quả (0,95 và 0,975) không khớp với giá trị thứ 2 và thứ 3 đã định nghĩa (0,975 và 0,9875), và không có kết quả nào được báo cáo cho "Momentum 0,9875". Đây rất có thể là lỗi đánh máy trong paper gốc (có thể ý là 0,9/0,95/0,975 ngay từ đầu, hoặc 0,975/0,9875 ở phần kết quả); ghi chú này **không tự sửa** mà chỉ trình bày đúng như 2 đoạn văn mâu thuẫn trong bản PDF.

- Khái niệm "noise scale g" ở đây ($g = \epsilon N/B$, một đại lượng vô hướng tỉ lệ nghịch batch, tỉ lệ thuận lr) là **khác hoàn toàn** về định nghĩa với "gradient noise scale B_{noise}" của opt 22 (McCandlish et al., $B_{noise} = \text{tr}(H\Sigma)/(G^T H G)$, một đại lượng có đơn vị "số mẫu", không phụ thuộc lr) - cùng tên "noise scale" nhưng là 2 khái niệm riêng biệt, không nên nhầm lẫn khi đọc song song 2 bài.

7. Liên hệ với các bài khác

- **opt 21 (Goyal, Large Minibatch SGD):** thiết lập ImageNet/Inception-ResNet-V2/ResNet-50 ở mục 5.3-5.4 của bài này **lặp lại trực tiếp** cấu hình của opt 21 (batch 8192, lr 3,2, 90 epoch, decay ở epoch 30/60/80) rồi mở rộng bằng kỹ thuật tăng batch; linear scaling rule $B \propto \epsilon$ của opt 21 được paper này trích dẫn trực tiếp và tái sử dụng.

- **opt 24 (Keskar, Generalization Gap and Sharp Minima):** được trích dẫn trực tiếp ở mục 3 và 6 (Related Work) về "generalization gap" và nhận xét "lợi ích của nhiễu rõ nhất ở đầu huấn luyện" - cùng chủ đề batch lớn/generalization nhưng bài này đề xuất một cách giải quyết khác (tăng batch dần, không phải cố định batch lớn từ đầu).
- **opt 22 (McCandlish, Empirical Model of Large-Batch Training):** cùng ngữ "noise scale" nhưng định nghĩa khác (xem mục 6); cả hai bài cùng dự đoán "có một batch size mà vượt qua đó lợi ích giảm", theo 2 công thức toán khác nhau.
- **opt 18 (SGDR):** được nhắc gián tiếp ở mục 3 ("researchers increasingly favor sharper decay schedules like cosine decay (Loshchilov & Hutter, 2016)") - chính là bài SGDR, dùng làm ví dụ cho xu hướng "anneal dốc" mà bài này diễn giải lại theo góc nhìn simulated annealing.
- **opt 19 (Super-Convergence):** tác giả **Leslie N. Smith** của opt 19 là người khác với tác giả **Samuel L. Smith** (đồng tác giả chính của bài này) - đã ghi chú sự trùng họ này ở opt 19, nhắc lại ở đây để tránh nhầm lẫn.
- Đã `ls` kiểm tra 5 thư mục cũ (llm/rl/distillation/pruning/interp content), không tìm thấy bài nào bàn trực tiếp về momentum-SDE hay noise-scale-equivalence nên không trích dẫn bài cụ thể ở đó.

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, batch 1, GPU ≤ 8GB). Toàn bộ mục này là suy luận:

1. **Batch 1 cố định, không thể "tăng batch" vật lý do VRAM** - kỹ thuật trung tâm của bài này (thay giảm lr bằng tăng batch) **không áp dụng trực tiếp** được cho vòng lặp huấn luyện thật của dự án (suy luận, "ít liên quan" theo nghĩa đen). Tuy nhiên, nếu dùng **gradient accumulation** (tích lũy gradient qua nhiều bước forward/backward trước khi update, không tăng VRAM đỉnh), có thể coi đó là "tăng batch hiệu dụng" mà công thức $g \approx \epsilon N/B$ ở đây vẫn áp dụng được cho batch hiệu dụng (not batch vật lý) - nghĩa là có thể **giảm số lần "update thật" bằng cách tăng số bước accumulation**, giữ lr không đổi, theo đúng logic bài báo (suy luận).
2. **Ý tưởng "early-training noise có lợi, late-training nên giảm noise"** (trích Keskar et al., mục 3) gợi ý: trong giai đoạn đầu fine-tune/distill, batch nhỏ/accumulation ít có thể còn hữu ích (thăm dò không gian tham số); về cuối, nên tăng accumulation (hoặc giảm lr, 2 cách tương đương theo bài này) để "đóng băng" vào một vùng tốt - đây là một gợi ý lịch trình cụ thể cho fine-tune model nhỏ trên GPU hạn chế, dù chưa kiểm chứng thực nghiệm cho riêng OCR task (suy luận).
3. **Cảnh báo về momentum cao + batch nhỏ (đảo ngược của mục 5.2):** phân tích "epoch bị mất" (mục 3.2 ở đây) cho thấy $N_{lost} \propto B/(N(1-m))$; với N (kích thước tập fine-tune OCR) **nhỏ hơn nhiều** so với ImageNet, dùng momentum cao (ví dụ 0,98-0,99, phổ biến trong AdamW mặc định cho β_1 đóng vai trò tương tự) có thể gây "mất" một phần đáng kể epoch hữu dụng nếu tập fine-tune nhỏ - gợi ý nên cân nhắc giảm β_1 hoặc tăng warm-up khi fine-tune trên tập dữ liệu OCR nhỏ (suy luận, dựa trên ngoại suy công thức, paper chỉ kiểm chứng trên CIFAR-10/ImageNet với N lớn).

9. Thí nghiệm gợi ý

1. **Kiểm chứng tương đương lr-decay vs accumulation-increase trên model nhỏ:** fine-tune một mô hình nhỏ (ví dụ ResNet nhỏ hoặc 1 lớp attention) trên 1 GPU 8GB với 2 lịch trình cùng tổng số epoch: (a) batch vật lý cố định, lr giảm theo step; (b) lr cố định, số bước gradient-accumulation tăng theo cùng hệ số ở cùng mốc epoch; đo train/test loss theo epoch, kỳ vọng 2 đường cong gần trùng khớp (theo lý thuyết bài báo).
2. **Đo noise scale thực nghiệm** $g \approx \epsilon N/B$ ở batch hiệu dụng khác nhau (qua accumulation 1, 4, 16 bước) trên một tập fine-tune nhỏ, xem độ nhạy thực tế của train loss theo công thức này có khớp cấp độ lớn không.

3. **Thử tăng effective lr + batch hiệu dụng cùng lúc** (ϵ tăng, accumulation tăng theo ϵ) để giảm số lần update thật, đo xem có giảm được wall-clock time cho 1 epoch fine-tune mà vẫn giữ chất lượng OCR (CER/WER) không đổi hay không.
4. **Kiểm tra ảnh hưởng β_1 (AdamW) khi tập fine-tune nhỏ:** quét $\beta_1 \in \{0{,}9; 0{,}95; 0{,}99\}$ với batch/accumulation cố định trên tập OCR nhỏ, theo dõi tốc độ hội tụ sớm, kiểm tra có dấu hiệu "lost epoch" (hội tụ chậm bất thường ở đầu huấn luyện) giống phân tích Phụ lục A của bài không.
5. **So sánh warm-up ngắn vs không warm-up** khi dùng accumulation lớn ngay từ đầu, đo độ ổn định loss những bước đầu (liên hệ trực tiếp nhận xét mục 5 của bài: bias correction của Adam "gây mất ổn định" ở batch lớn).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Nguồn
Điều kiện Robbins-Monro	$\sum \epsilon_i = \infty,\ \sum \epsilon_i^2 < \infty$	eq. (1)-(2), chỉ đúng khi B cố định
Noise scale (SGD vanilla)	$g = \epsilon(N/B - 1) \approx \epsilon N/B$	eq. (3)
Noise scale (có momentum)	$g \approx \epsilon N/(B(1-m))$	eq. (4)-(5)
Effective lr	$\epsilon_{eff} = \epsilon/(1-m)$	mục 4
Quy tắc chuyển đổi	giảm lr $\times\alpha \Leftrightarrow$ tăng B $\times\alpha$ (giữ ϵ)	ý tưởng trung tâm
Ngưỡng dùng được	$B \lesssim N/10$	mục 2
Số epoch "bị mất"	$N_{lost} = B/(N(1-m))$	eq. (13), Phụ lục A
Kết quả nổi bật	Inception-ResNet-V2, 65536 batch, <2500 update, 77% acc	mục 5.3
Kết quả nổi bật 2	ResNet-50 TPU, batch tăng 8192 → 16384, 76,1% acc trong <30 phút	mục 5.4

11. Kết luận ngắn

Bài báo chứng minh (lý thuyết qua SDE-noise-scale và thực nghiệm trên CIFAR-10/ImageNet, nhiều optimizer) rằng giảm lr trong huấn luyện tương đương về mặt noise scale với tăng batch size ở lr cố định, nên có thể chuyển trực tiếp lịch trình lr-decay có sẵn thành lịch trình tăng-batch mà không cần tune lại, giảm mạnh số lần cập nhật tham số và thời gian huấn luyện (ResNet-50 ImageNet trong 30 phút, Inception-ResNet-V2 trong dưới 2500 update). Có thể tăng thêm hiệu quả bằng tăng effective lr (qua ϵ hoặc m), nhưng tăng m quá cao làm giảm nhẹ accuracy do hiệu ứng "epoch bị mất" trong biến accumulation. Phát hiện một chỗ không khớp tên gọi lịch trình momentum trong mục 5.3 của bản PDF gốc, được ghi lại như một mâu thuẫn nội bộ của paper.

Nguồn gốc: https://arxiv.org/abs/1711.00489

Bài 24

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima (2016)

Tác giả: Nitish Shirish Keskar (Northwestern University), Dheevatsa Mudigere (Intel), Jorge Nocedal (Northwestern University), Mikhail Smelyanskiy (Intel), Ping Tak Peter Tang (Intel).
Venue / arXiv: arXiv:1609.04836 (v2, 09/02/2017); venue: Published as a conference paper at ICLR 2017.
Nguồn: https://arxiv.org/abs/1609.04836
Chủ đề: quan sát thực nghiệm rằng SGD batch lớn (large-batch, LB) có xu hướng hội tụ về cực tiểu ”sắc” (sharp minima), còn SGD batch nhỏ (small-batch, SB) hội tụ về cực tiểu ”bằng” (flat minima); đề xuất ”sharpness metric” để đo và so sánh, cùng vài biện pháp khắc phục chưa triệt để.
Vị trí trong lộ trình: đọc sau opt 18-19 (lr schedule/noise) và trước opt 25 (SAM - trực tiếp lấy động lực từ đúng bài này) và opt 26 (Edge of Stability - cũng dùng khái niệm độ cong/sharpness); liên quan chặt tới opt 21 (Goyal, phản biện trực tiếp phần ”generalization gap” trên ImageNet) và opt 23 (Smith et al., trích dẫn trực tiếp bài này và đề xuất giải pháp ”tăng batch dần” mà chính bài này đã gợi ý ở Hình 5).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

SGD với batch nhỏ (SB, thường 32-512 mẫu) là phương pháp mặc định cho deep learning, nhưng nhược điểm là khó song song hoá do tính tuần tự và batch nhỏ. Hướng tự nhiên để tăng song song hoá: tăng batch size (LB). Nhưng thực hành cho thấy khi tăng batch, **test accuracy giảm** dù train accuracy vẫn cao - đây là ”generalization gap”. Bài báo này (trước opt 21 Goyal et al. về mặt thời gian công bố, và được opt 21 trích dẫn như một trong các bằng chứng về vấn đề này) điều tra **nguyên nhân** của generalization gap, chứ không đề xuất cách huấn luyện LB hiệu quả như opt 21.

Bài toán tối ưu: $\min_{x\in\mathbb{R}^n} f(x) := \frac{1}{M}\sum_{i=1}^{M} f_i(x)$ (1), SGD cập nhật $x_{k+1} = x_k - \alpha_k\big(\frac{1}{|B_k|}\sum_{i\in B_k}\nabla f_i(x_k)\big)$ (2). Paper định nghĩa: SB = $|B_k| = 256$; LB = $|B_k| = 10\%$ số mẫu huấn luyện. Cả hai dùng optimizer Adam (thí nghiệm bổ sung với AdaGrad, SGD, adaQN cho kết quả tương tự, không được in chi tiết).

2. Ý tưởng chính

Quan sát chính (nguyên văn, mục 2.1, đóng khung trong paper):

> ”Việc thiếu khả năng tổng quát hoá là do các phương pháp batch lớn có xu hướng hội tụ về các cực tiểu sắc (sharp) của hàm huấn luyện. Các cực tiểu này đặc trưng bởi một số đáng kể các giá trị riêng (eigenvalue) dương lớn của $\nabla^2 f(x)$, và có xu hướng tổng quát hoá kém hơn. Ngược lại, phương pháp batch nhỏ hội tụ về cực tiểu bằng (flat), đặc trưng bởi nhiều eigenvalue nhỏ. Cảnh quan hàm mất mát của deep neural network khiến LB bị hút vào vùng cực tiểu sắc và, khác với SB, không thể thoát khỏi lưu vực hút (basin of attraction) của các cực tiểu này.”

Trực giác: cực tiểu ”bằng” có thể mô tả với độ chính xác thấp (hàm thay đổi chậm quanh đó), còn cực tiểu ”sắc” cần độ chính xác cao để mô tả đúng - theo lý thuyết minimum description length (MDL, Rissanen 1983), model mô tả được với ít bit hơn (độ phức tạp thấp hơn) tổng quát hoá tốt hơn, nên cực tiểu bằng tổng quát hoá tốt hơn cực tiểu sắc. Paper **không chứng minh toán học** điều này, mà đưa ra **bằng chứng số (numerical evidence)** ủng hộ quan sát trên qua 6 bài toán phân loại.

3. Phương pháp chi tiết

3.1 Sharpness metric (mục 2.2.2)

Vì tính trực tiếp eigenvalue của Hessian $\nabla^2 f(x)$ quá tốn kém cho deep network, paper đề xuất đo độ nhạy cục bộ bằng cách tối đa hoá f trong một lân cận nhỏ của nghiệm x. Để tránh bị "lừa" bởi trường hợp giá trị lớn chỉ đạt được trên 1 không gian con rất nhỏ, phép tối đa hoá được thực hiện cả trên toàn không gian $\mathbb{R}^n$ và trên một manifold ngẫu nhiên chiều $p = 100$ (qua ma trận $A \in \mathbb{R}^{n\times p}$ cột ngẫu nhiên). Tập ràng buộc:

$$C_\epsilon = \{z \in \mathbb{R}^p : -(|(A^+x)_i| + 1) \le z_i \le (|(A^+x)_i| + 1)\ \forall i\} \tag{3}$$

(A^+ = giả nghịch đảo của A, ϵ điều khiển kích thước "hộp"). **Định nghĩa** (C_ϵ, A)**-sharpness**:

$$\boxed{\varphi_{x,f}(\epsilon, A) := \frac{\left(\max_{y\in C_\epsilon} f(x + Ay)\right) - f(x)}{1 + f(x)} \times 100} \tag{4}$$

Nếu $A = I_n$ (toàn không gian): φ liên quan trực tiếp eigenvalue lớn nhất của $\nabla^2 f(x)$ (khi ϵ nhỏ); nếu A ngẫu nhiên: φ xấp xỉ giá trị Ritz của $\nabla^2 f(x)$ chiếu lên không gian cột của A. Bài toán tối đa hoá trong (4) được giải **không chính xác** bằng 10 vòng lặp L-BFGS-B (vì chi phí đánh giá f thật quá lớn).

Ví dụ tự tính (minh hoạ công thức 4 với số giả định): nếu $f(x) = 0{,}05$ (loss tại cực tiểu) và giá trị lớn nhất tìm được trong lân cận là $\max f = 0{,}55$, thì $\varphi = \frac{0{,}55-0{,}05}{1+0{,}05} \times 100 = \frac{0{,}5}{1{,}05} \times 100 \approx 47{,}62$ (tự tính) - để so sánh cấp độ lớn: paper đo được (Bảng 3, $\epsilon = 10^{-3}$, full space) $\varphi^{SB}_{C1} = 28{,}58$ và $\varphi^{LB}_{C1} = 707{,}23$ - chênh nhau khoảng **25 lần**, một ví dụ cho "chênh 1-2 bậc độ lớn" mà paper báo cáo giữa SB và LB.

3.2 Thiết kế thí nghiệm (mục 2.2)

6 bài toán (Bảng 1): F1 (FC, MNIST), F2 (FC, TIMIT - nhận diện giọng nói, 1973 lớp), C1 (conv nông kiểu AlexNet, CIFAR-10), C2 (conv sâu kiểu VGG, CIFAR-10), C3 (như C1, CIFAR-100), C4 (như C2, CIFAR-100). LB = 10% dữ liệu huấn luyện mỗi batch; SB = 256. Mỗi cấu hình chạy 5 lần từ điểm khởi tạo ngẫu nhiên khác nhau, train tới khi loss không cải thiện nữa (không giới hạn budget).

4. Thí nghiệm và kết quả chính

4.1 Generalization gap cơ bản (Bảng 2)

Mạng	Train SB	Train LB	Test SB	Test LB	Gap (test, tự tính)
F1	99,66%	99,92%	98,03%	97,81%	0,22
F2	99,99%	98,35%	64,02%	59,45%	4,57
C1	99,89%	99,66%	80,04%	77,26%	2,78
C2	99,99%	99,99%	89,24%	87,26%	1,98
C3	99,56%	99,88%	49,58%	46,45%	3,13
C4	99,10%	99,57%	63,08%	57,81%	5,27

Train accuracy **cao và tương đương** ở cả 2 chế độ trên mọi mạng, nhưng test accuracy LB thấp hơn SB rõ rệt trên 5/6 mạng (F1 gần như không đổi). Gap test lớn nhất đo được (tự tính từ bảng, mạng C4) $\approx 5{,}27$ điểm phần trăm, khớp với tuyên bố của paper "gap cao tới 5%". Paper nhấn mạnh: đường cong train/test theo epoch (Hình 2, mạng F2 và C1) **không** có hình dạng "đỉnh rồi giảm" đặc trưng của overfitting - nên early-stopping không giúp giảm gap này; đây là một khẳng định quan trọng để loại trừ giả thuyết "LB chỉ overfit nhiều hơn".

4.2 Parametric plots (mục 2.2.1, Hình 3)

Nội suy tuyến tính $f(\alpha x_\ell^* + (1-\alpha)x_s^*)$ cho $\alpha \in [-1, 2]$ giữa nghiệm SB (x_s^*, $\alpha = 0$) và LB (x_ℓ^*, $\alpha = 1$), trên cả train và test loss, cho cả 6 mạng: vùng quanh $\alpha = 1$ (nghiệm LB) hiện rõ "sắc" hơn nhiều so với vùng quanh $\alpha = 0$ (nghiệm SB) - minh hoạ trực quan khớp với sharpness metric định lượng. Phụ lục D lặp lại với đường cong phi tuyến $f(\sin(\frac{\alpha\pi}{2})x_\ell^* + \cos(\frac{\alpha\pi}{2})x_s^*)$, kết quả tương tự.

4.3 Sharpness định lượng (Bảng 3, 4)

Đo φ ở $\epsilon \in \{10^{-3}, 5\times10^{-4}\}$, cả full space ($A = I$) và random subspace $p = 100$: **mọi** cặp SB/LB trên 6 mạng cho LB sắc hơn SB **1-2 bậc độ lớn**. Ví dụ (Bảng 3, $\epsilon = 10^{-3}$, full space): C2: SB $8{,}68 \pm 1{,}32$ vs LB $925{,}32 \pm 38{,}29$ (chênh ~107 lần). Ghi chú: cực tiểu sắc đo được **không giống hình nón** - hàm chỉ tăng nhanh theo một tập con chiều thấp (khoảng 5% không gian), còn hầu hết các hướng khác tương đối bằng.

4.4 Ngưỡng batch size và thí nghiệm warm-start (mục 3, Hình 4, 5, 6)

- **Ngưỡng batch:** quét batch size liên tục cho F2 và C1 (Hình 4): có một ngưỡng rõ rệt (~15.000 cho F2, ~500 cho C1) sau đó test accuracy giảm mạnh **và** độ tăng sharpness cũng chững lại; các mạng khác có ngưỡng tương tự (không công bố số cụ thể).
- **Thí nghiệm warm-start/piggyback (Hình 5):** train SB 100 epoch, lưu iterate sau mỗi epoch; từ mỗi iterate đó, chạy tiếp LB 100 epoch ("piggybacked"). Kết quả: nếu warm-start chỉ với vài epoch SB đầu, LB **không** cải thiện generalization, sharpness vẫn cao; nhưng sau một số epoch SB đủ lớn (khi SB đã "kết thúc giai đoạn thăm dò" và tìm được cực tiểu bằng), LB warm-start từ đó đạt test accuracy cao **và** sharpness giảm. Đây là gợi ý thực nghiệm sớm nhất trong paper cho ý tưởng "tăng batch dần trong lúc huấn luyện" mà sau này opt 23 phát triển thành phương pháp chính thức.
- **Tỉ số khoảng cách tới điểm khởi tạo:** $\|x_s^* - x_0\|_2 / \|x_\ell^* - x_0\|_2$ đo được trong khoảng 3-10, ủng hộ giả thuyết LB bị hút về cực tiểu **gần** điểm khởi tạo hơn SB.
- **Sharpness theo loss (Hình 6):** ở giá trị loss lớn (gần điểm khởi tạo), SB và LB có sharpness tương đương; khi loss giảm, sharpness của LB tăng nhanh còn SB giữ tương đối hằng rồi giảm - phù hợp giả thuyết "SB thăm dò trước, rồi hội tụ về cực tiểu bằng".

4.5 Mô hình hiệu năng lý tưởng hoá (Phụ lục C)

Để LB nhanh hơn SB: $\frac{I_\ell}{I_s} < \frac{B_s}{f_s(P)B_\ell}$, với I=số iteration, B=batch size, P=số processor, $f_s(P)$=hiệu suất song song của SB (giả định $f_\ell(P) = 1$ cho LB).

Ví dụ tự tính (kiểm lại số ví dụ chính paper đưa ra): paper nêu "nếu $f_s(P) = 0{,}2$ và $B_s/B_\ell = 0{,}1$, LB phải hội tụ trong tối đa một nửa số iteration của SB". Kiểm tra: $\frac{B_s}{f_s(P)B_\ell} = \frac{0{,}1}{0{,}2} = 0{,}5$ (tự tính) - khớp đúng "một nửa" như paper phát biểu bằng lời, không có sai số.

4.6 Các biện pháp khắc phục thử nghiệm (Phụ lục E)

- **Data augmentation** (Bảng 6): LB với augmentation mạnh (lật ngang, xoay tới 10°, dịch tới 0,2 kích thước ảnh) đạt test accuracy **gần bằng** SB baseline trên 4 mạng conv, nhưng sharpness **vẫn cao** (ví dụ C2: $468{,}65 \pm 47{,}86$ ở $\epsilon = 10^{-3}$) - paper kết luận: cải thiện accuracy nhưng không giải quyết độ nhạy.
- **Conservative training** (Bảng 7, công thức (5): $x_{k+1} = \arg\min_x \frac{1}{|B_k|}\sum f_i(x) + \frac{\lambda}{2}\|x - x_k\|^2$, giải gần đúng bằng 3 bước Adam, $\lambda = 10^{-3}$): cải thiện test accuracy có ý nghĩa thống kê nhưng tương tự, sharpness vẫn cao.
- **Robust/adversarial training** (mục E.3): **không** cải thiện generalization trong thí nghiệm của paper (cả adversarial training kiểu Goodfellow et al. 2014a và "stability training" của Zheng et al. 2016 cho kết quả tương tự baseline).

5. Phân tích / ablation

Quan sát của người ghi chú (không phải kết luận paper tự nêu, rút ra từ chính Bảng 6 và 7): dữ liệu của paper có **một số trường hợp ngoại lệ** với narrative chính ”sharp = generalization kém”: ở Bảng 6, mạng C4 với data augmentation đạt test accuracy LB = 65,88% **cao hơn** baseline SB = 63,05% (dù sharpness vẫn ở mức cao, 271,06), nghĩa là ở riêng trường hợp này LB sắc hơn SB nhưng lại tổng quát hoá **tốt hơn** chứ không kém hơn. Tương tự ở Bảng 7, mạng F1 với conservative training đạt 98,12% so với baseline SB 98,03% (cao hơn, dù sharpness LB=232,25 rất cao). Paper không thảo luận riêng các điểm dữ liệu này (chỉ nói chung ”cải thiện có ý nghĩa thống kê nhưng không giải quyết được sharpness”), nhưng đây là bằng chứng (từ dữ liệu paper chính công bố) cho thấy mối quan hệ sharpness-generalization **không hoàn toàn đơn điệu** trong mọi trường hợp, dù xu hướng chung (5/6 mạng ở thí nghiệm gốc Bảng 2) vẫn ủng hộ luận điểm chính.

6. Giới hạn

Tác giả tự nêu (mục 4, Discussion, liệt kê trực tiếp 5 câu hỏi mở):

- Chỉ đưa ra **bằng chứng số**, không chứng minh toán học rằng LB ”luôn” hội tụ về cực tiểu sắc.
- Chưa biết mật độ tương đối giữa cực tiểu sắc và bằng trên loss landscape.
- Chưa rõ có thể thiết kế kiến trúc network phù hợp hơn với LB không.
- Chưa rõ có cách khởi tạo nào giúp LB thành công không.
- Chưa rõ có cách nào (thuật toán hay regularizer) ”lái” LB tránh xa cực tiểu sắc không - đây chính là câu hỏi mà opt 25 (SAM) trả lời trực tiếp vài năm sau.
- Cả 3 biện pháp khắc phục thử (augmentation, conservative training, robust training) đều **không giải quyết triệt để** vấn đề sharpness, chỉ cải thiện accuracy phần nào.
- Sharpness metric φ chỉ là một ”proxy” không hoàn hảo (do tối đa hoá không chính xác bằng 10 bước L-BFGS-B, do chi phí tính f thật quá cao) cho eigenvalue thật của Hessian.

Người ghi chú nhận xét thêm:

- Kết luận về generalization gap của bài này sau đó **bị thách thức trực tiếp** bởi opt 21 (Goyal et al. 2017), vốn cho thấy trên ImageNet/ResNet-50, nếu tune đúng linear scaling rule + warmup, generalization gap **biến mất** (sai khác chỉ ~0,26-0,28 điểm %, trong khoảng nhiễu giữa các lần chạy) - gợi ý rằng gap quan sát ở bài Keskar có thể một phần do **thiếu tune hyperparameter cho LB** (không áp dụng linear scaling rule cho lr theo batch, không có warmup) chứ không hoàn toàn là thuộc tính nội tại ”sharp minima luôn generalize kém”. Bài Keskar không thử linear scaling rule cho lr theo batch size trong các thí nghiệm chính (mục 2.2) của mình.
- Phạm vi đọc: đã đọc toàn bộ thân bài (mục 1-4) và Phụ lục A-E.

7. Liên hệ với các bài khác

- **opt 25 (Sharpness-Aware Minimization):** trả lời trực tiếp câu hỏi mở (e) của bài này (”có thể lái LB tránh sharp minima không?”) bằng một thuật toán tối ưu cụ thể (minimize worst-case loss trong lân cận ϵ-ball) - về bản chất là phiên bản ”có thể huấn luyện được” của chính sharpness metric (4) ở đây.
- **opt 26 (Edge of Stability):** tiếp tục nghiên cứu độ cong (sharpness, qua giá trị riêng lớn nhất Hessian) của quá trình huấn luyện gradient descent, mở rộng khái niệm ”sharpness” sang động lực học huấn luyện theo thời gian thực (không chỉ so sánh 2 cực tiểu cuối cùng như bài này).
- **opt 21 (Goyal, Large Minibatch SGD):** phản biện trực tiếp phần kết luận về ”generalization gap” của bài này trên ImageNet khi dùng linear scaling rule + warmup đúng cách (xem mục 6).
- **opt 23 (Smith et al., Don’t Decay the LR, Increase the Batch Size):** trích dẫn trực tiếp bài này; đồng thời cách giải quyết chính của opt 23 (tăng batch dần) là một cách hiện thực hoá ý tưởng ”warm-start/piggyback” mà chính bài Keskar đã thử nghiệm sơ bộ ở Hình 5.

- **opt 22 (McCandlish, Empirical Model of Large-Batch Training):** trích dẫn bài này khi thảo luận giả định về generalization (mục 2.4 giả định 6), đồng thời dẫn Goyal et al. như phản biện.
- Đã `ls` kiểm tra 5 thư mục cũ (llm/rl/distillation/pruning/interp content); không tìm bài nào bàn trực tiếp sharp/flat minima hay sharpness metric, nên không trích dẫn bài cụ thể ở đó.

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, batch 1, GPU ≤ 8GB). Toàn bộ mục này là suy luận:

1. **Batch 1 nằm ở cực SB tuyệt đối** của phổ SB-LB mà bài này nghiên cứu, nên theo logic của paper, huấn luyện/fine-tune ở batch 1 (nếu dùng SGD-kiểu, không averaging gradient) có xu hướng **tự nhiên hướng tới cực tiểu bằng (flat)** - về lý thuyết đây là điều có lợi cho generalization, nên quy trình fine-tune batch 1 của dự án **không cần** thêm biện pháp đặc biệt để tránh sharp minima theo tinh thần bài này (suy luận: batch nhỏ "miễn nhiễm" với vấn đề mà bài báo mô tả, vì vấn đề chỉ xảy ra *ở* batch *lớn*).
2. **Sharpness metric (4) như công cụ chẩn đoán độc lập với batch size:** có thể đo $\varphi_{x,f}$ của checkpoint model sau khi distill/prune để kiểm tra mức độ "sắc" của cực tiểu đạt được, bất kể batch huấn luyện là bao nhiêu - nếu sharpness cao bất thường sau pruning, có thể là dấu hiệu model dễ mất ổn định khi fine-tune tiếp hoặc khi gặp phân phối dữ liệu lệch (out-of-distribution OCR scan) (suy luận, chưa kiểm chứng cho riêng domain OCR).
3. **Liên hệ gián tiếp tới pruning/distillation:** nếu quá trình prune+distill (nằm ngoài phạm vi bài này, thuộc thư mục pruning/distillation content) tạo ra một landscape "sắc" hơn quanh nghiệm cuối (vì xoá tham số làm hàm mất mát nhạy hơn với phần còn lại), sharpness metric của bài này có thể dùng làm chỉ số phụ để so sánh các chiến lược prune khác nhau xem cái nào giữ được cực tiểu "bằng" hơn (ít liên quan trực tiếp tới batch size, suy luận mở rộng ứng dụng công cụ chứ không áp dụng kết luận chính của bài).
4. **Cảnh báo khi dùng gradient accumulation lớn (nếu áp dụng gợi ý từ opt 23):** nếu dự án dùng accumulation để "giả lập" batch lớn hơn 1 (xem opt 23), bài này gợi ý nên theo dõi sharpness/test performance cẩn thận hơn khi batch hiệu dụng tăng, vì đó chính là vùng mà hiện tượng "sharp minima" của bài này có thể xuất hiện trở lại (suy luận, nối 2 bài).

9. Thí nghiệm gợi ý

1. **Đo sharpness của checkpoint fine-tune OCR ở các batch hiệu dụng khác nhau:** cài công thức (3)-(4) (đơn giản hoá: random subspace p nhỏ, ví dụ $p = 20$, do giới hạn GPU 8GB) trên model nhỏ fine-tune với batch hiệu dụng 1 và với accumulation (ví dụ batch hiệu dụng 32), so sánh φ giữa 2 cách.
2. **Parametric plot giữa 2 checkpoint** (ví dụ checkpoint pruned vs checkpoint gốc, hoặc checkpoint batch 1 vs checkpoint accumulation lớn) theo công thức nội suy tuyến tính của mục 2.2.1, quan sát hình dạng loss landscape giữa 2 điểm.
3. **Thí nghiệm warm-start kiểu Hình 5** trên quy mô nhỏ: train vài epoch đầu ở batch nhỏ (ví dụ 1-4), rồi chuyển sang batch lớn hơn (accumulation 16-32) cho các epoch sau, đo xem test performance/sharpness có cải thiện so với dùng batch lớn ngay từ đầu không.
4. **Quét ngưỡng batch hiệu dụng** (như Hình 4 của bài) trên tập fine-tune OCR nhỏ: tăng batch hiệu dụng dần (qua accumulation: 1, 4, 16, 64, 256...), đo validation CER/WER, tìm điểm "gãy" nếu có.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Nguồn
Bài toán tối ưu	$\min_x f(x) = \frac{1}{M}\sum_i f_i(x)$	eq. (1)
Sharpness metric	$\varphi_{x,f}(\epsilon, A) = \frac{\max_{y \in C_\epsilon} f(x+Ay) - f(x)}{1+f(x)} \times 100$	eq. (4)

Khái niệm	Công thức / giá trị	Nguồn
Hộp ràng buộc	$C_\epsilon = \{z : \lvert z_i\rvert \leq \lvert(A^+x)_i\rvert + 1\}$	eq. (3)
LB, SB batch	LB = 10% dữ liệu; SB = 256	mục 2.2
Gap test lớn nhất đo được	~5,27 điểm % (mạng C4)	Bảng 2, tự tính
Chênh sharpness SB/LB	1-2 bậc độ lớn	Bảng 3, 4
Điều kiện LB nhanh hơn SB	$I_\ell/I_s < B_s/(f_s(P)B_\ell)$	Phụ lục C
Ngưỡng batch (ví dụ)	~15.000 (F2), ~500 (C1)	Hình 4
Giải pháp gợi ý nhưng chưa triệt để	data augmentation, conservative training, robust training	Phụ lục E

11. Kết luận ngắn

Bài báo cung cấp bằng chứng số mạnh (6 bài toán, parametric plot, sharpness metric định lượng, thí nghiệm warm-start) cho thấy SGD batch lớn có xu hướng hội tụ về cực tiểu sắc, tương quan với generalization gap lên tới ~5 điểm % so với batch nhỏ, và hiện tượng này **không phải** do overfitting. Các biện pháp khắc phục thử (data augmentation, conservative training, robust training) cải thiện accuracy phần nào nhưng không loại bỏ được sharpness. Bài đặt ra nhiều câu hỏi mở mà các công trình sau giải quyết trực tiếp: opt 21 phản biện generalization gap khi tune đúng, opt 23 hiện thực hoá ý tưởng ”tăng batch dần” đã được gợi ý ở thí nghiệm warm-start, và opt 25 (SAM) trực tiếp xây một thuật toán tối ưu nhằm ”lái” nghiệm về vùng bằng hơn - đúng như câu hỏi mở cuối cùng của bài này.

Nguồn gốc: https://arxiv.org/abs/1609.04836

Bài 25

Sharpness-Aware Minimization for Efficiently Improving Generalization (2020)

Tác giả: Pierre Foret, Ariel Kleiner, Hossein Mobahi (Google Research), Behnam Neyshabur (Blueshift, Alphabet).
Venue / arXiv: arXiv:2010.01412 (v3, 29/04/2021); venue: Published as a conference paper at ICLR 2021.
Nguồn: https://arxiv.org/abs/2010.01412
Chủ đề: đề xuất Sharpness-Aware Minimization (SAM), một thuật toán tối ưu tối thiểu hoá đồng thời giá trị loss và độ "sắc" (sharpness) của loss, nhằm cải thiện generalization, có thể cắm thay trực tiếp cho SGD/Adam.
Vị trí trong lộ trình: trả lời trực tiếp câu hỏi mở cuối cùng của opt 24 (Keskar - "có thể lái large-batch training tránh sharp minima không?"), đọc sau opt 24 và trước opt 26 (Edge of Stability - cũng dùng phổ Hessian/λ_{max} nhưng theo góc nhìn động lực học huấn luyện); liên quan tới opt 21-23 (batch size/noise, vì chi phí tính toán của SAM tương tác với mini-batch qua khái niệm "m-sharpness").

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

opt 24 (Keskar et al. 2016) đã chỉ ra tương quan giữa sharp minima và generalization gap kém, nhưng chỉ đưa ra bằng chứng số, không đề xuất được thuật toán huấn luyện trực tiếp "lái" nghiệm tránh sharp minima mà vẫn hiệu quả ở quy mô lớn (các biện pháp thử - augmentation, conservative training, robust training - đều không giải quyết triệt để). Jiang et al. (2019), nghiên cứu lớn so sánh 40 "complexity measure" khác nhau để dự đoán generalization gap, tìm thấy **sharpness-based measure có tương quan cao nhất** - đây là động lực trực tiếp cho bài báo này: nếu sharpness dự đoán tốt generalization, tại sao không tối ưu hoá trực tiếp để giảm sharpness?

Model với tham số $w \in \mathcal{W} \subseteq \mathbb{R}^d$, loss từng điểm l, loss huấn luyện $L_S(w) = \frac{1}{n}\sum_i l(w, x_i, y_i)$, loss thật (population) $L_D(w) = \mathbb{E}_{(x,y)\sim D}[l(w, x, y)]$. Cách làm chuẩn: tối thiểu $L_S(w)$ bằng SGD/Adam. Vấn đề: với model overparameterized hiện đại, L_S không lồi, có nhiều cực tiểu local/global cho giá trị train loss tương đương nhau nhưng $L_D(w)$ (test performance) **rất khác nhau**.

2. Ý tưởng chính

Thay vì tìm w có $L_S(w)$ thấp, SAM tìm w sao cho **toàn bộ lân cận** của w có L_S thấp đều (tương đương: lân cận vừa có loss thấp vừa có độ cong thấp). Cơ sở lý thuyết (Định lý 1, phát biểu không hình thức, chứng minh đầy đủ kiểu PAC-Bayes ở Phụ lục A):

$$L_D(w) \leq \max_{\|\epsilon\|_2 \leq \rho} L_S(w+\epsilon) + h(\|w\|_2^2/\rho^2) \qquad \text{(bất đẳng thức, Theorem 1)}$$

với h là hàm tăng ngặt. Viết lại vế phải: $[\max_{\|\epsilon\|_2\leq\rho} L_S(w+\epsilon) - L_S(w)] + L_S(w) + h(\|w\|_2^2/\rho^2)$ - số hạng trong ngoặc vuông chính là **thước đo sharpness** (tăng loss tối đa khi nhiễu w trong bán kính ρ). Paper thay $h(\cdot)$ bằng $\lambda\|w\|_2^2$ (L2 regularization chuẩn, vì h cụ thể phụ thuộc chi tiết chứng minh), dẫn tới bài toán **Sharpness-Aware Minimization**:

$$\boxed{\min_w L_S^{SAM}(w) + \lambda\|w\|_2^2, \qquad L_S^{SAM}(w) := \max_{\|\epsilon\|_p\leq\rho} L_S(w+\epsilon)} \qquad (1)$$

($\rho \geq 0$ siêu tham số duy nhất, $p \in [1, \infty]$, thực nghiệm cho thấy $p = 2$ là tối ưu, Phụ lục C.5).

3. Phương pháp chi tiết

3.1 Xấp xỉ bước trong (inner maximization)

Để tối thiểu $L_S^{SAM}(w)$ bằng gradient descent, cần vi phân qua phép max bên trong. Xấp xỉ Taylor bậc 1: $\epsilon^*(w) = \arg\max_{\|\epsilon\|_p \leq \rho} L_S(w+\epsilon) \approx \arg\max_{\|\epsilon\|_p \leq \rho} \epsilon^T \nabla_w L_S(w)$ - đây là bài toán dual-norm cổ điển, có nghiệm đóng:

$$\hat{\epsilon}(w) = \rho \, \text{sign}(\nabla_w L_S(w)) \, |\nabla_w L_S(w)|^{q-1} \Big/ \|\nabla_w L_S(w)\|_q^{q/p} \quad (1/p + 1/q = 1) \tag{2}$$

Với $p = 2$ (trường hợp dùng chính trong thực nghiệm): công thức rút gọn thành **chỉ cần scale gradient về norm** ρ:

$$\hat{\epsilon}(w) = \rho \frac{\nabla_w L_S(w)}{\|\nabla_w L_S(w)\|_2}$$

Ví dụ tự tính: cho gradient toy 2 chiều $\nabla_w L_S(w) = (3, 4)$, $\|\nabla_w L_S(w)\|_2 = \sqrt{3^2 + 4^2} = 5$. Với $\rho = 0{,}1$: $\hat{\epsilon} = 0{,}1 \times (3, 4)/5 = (0{,}06;\ 0{,}08)$ (tự tính). Kiểm tra: $\|\hat{\epsilon}\|_2 = \sqrt{0{,}06^2 + 0{,}08^2} = \sqrt{0{,}0036 + 0{,}0064} = \sqrt{0{,}01} = 0{,}1$ (tự tính) - đúng bằng ρ, xác nhận đúng chú thích của paper ("với $p = 2$, việc này chỉ đơn giản là scale gradient sao cho norm của nó bằng ρ").

3.2 Gradient cuối và thuật toán

Thế $\hat{\epsilon}(w)$ vào (1) rồi lấy gradient: $\nabla_w L_S^{SAM}(w) \approx \nabla_w L_S(w)|_{w+\hat{\epsilon}(w)} + \frac{d\hat{\epsilon}(w)}{dw} \nabla_w L_S(w)|_{w+\hat{\epsilon}(w)}$. Số hạng thứ 2 chứa thông tin Hessian (qua Hessian-vector product, tính được nhưng tốn hơn); paper **bỏ số hạng bậc 2** để tăng tốc, thu được xấp xỉ cuối dùng trong thực nghiệm chính:

$$\nabla_w L_S^{SAM}(w) \approx \nabla_w L_S(w)|_{w+\hat{\epsilon}(w)} \tag{3}$$

Thuật toán SAM (Algorithm 1): mỗi bước lấy batch B, (i) tính gradient tại w, (ii) tính $\hat{\epsilon}(w)$ theo (2) từ gradient đó, (iii) tính gradient **lần 2** tại điểm nhiễu $w + \hat{\epsilon}(w)$, (iv) cập nhật $w_{t+1} = w_t - \eta g$ với g là gradient lần 2. Mỗi bước SAM cần **2 lần backpropagation** (chi phí gấp đôi 1 bước SGD thường).

4. Thí nghiệm và kết quả chính

4.1 Image classification from scratch (CIFAR-10/100, mục 3.1)

Bảng 1 (trích một phần, augmentation AutoAugment - AA, số run=5, mean±95% CI): WRN-28-10 (1800 epoch): CIFAR-10 SAM $1{,}6 \pm 0{,}1$% vs SGD $2{,}2 \pm 0{,}1$%; CIFAR-100 SAM $12{,}8 \pm 0{,}2$% vs SGD $16{,}1 \pm 0{,}2$%. PyramidNet+ShakeDrop (AA): CIFAR-10 SAM $1{,}4$% vs SGD $1{,}6$%; CIFAR-100 SAM $10{,}3$% vs SGD $10{,}6$% - theo paper, **SOTA mới** trên CIFAR-100 không dùng thêm dữ liệu, tại thời điểm công bố. SAM cải thiện **mọi** cấu hình model/augmentation được thử (16 dòng trong Bảng 1).

Bài toán cân bằng chi phí: vì mỗi bước SAM tốn gấp đôi backprop, các baseline không-SAM được cho chạy **gấp đôi số epoch** và lấy điểm tốt nhất giữa 2 mức epoch, để so sánh công bằng theo compute - SAM vẫn thắng.

4.2 ImageNet (mục 3.1, Bảng 2)

ResNet-{50,101,152}, batch 4096, lr khởi điểm 1,0, cosine lr schedule, SGD momentum 0,9, label smoothing 0,1, weight decay 10^{-4}, $\rho = 0{,}05$ (tune bằng grid search trên ResNet-50/100 epoch). Kết quả nổi bật: SAM

cải thiện top-1 error ResNet-152 từ 20,3% (chuẩn, 200 epoch) xuống 18,4% (SAM, 400 epoch). Quan trọng hơn: **huấn luyện lâu hơn (200→400 epoch) tiếp tục cải thiện với SAM nhưng làm TỆ hơn với training chuẩn** (ResNet-152 chuẩn: 20,4%→20,3%→20,9% khi tăng 100→200→400 epoch, tức **tăng trở lại** ở 400 epoch - dấu hiệu overfitting rõ ràng; còn SAM: 19,2%→18,5%→18,4%, liên tục giảm). Đây là bằng chứng mạnh cho khả năng chống overfit của SAM, không chỉ cải thiện điểm số tuyệt đối.

4.3 Finetuning (mục 3.2, Bảng 3)

Finetune EfficientNet-b7 (pretrain ImageNet, checkpoint RandAugment 84,7%) và EfficientNet-L2 (pretrain ImageNet+JFT, checkpoint NoisyStudent 88,2%) trên 9 downstream task (FGVC Aircraft, Flowers, Oxford Pets, Stanford Cars, CIFAR-10/100, Birdsnap, Food101, ImageNet). SAM cải thiện **đồng loạt** so với không-SAM; SOTA mới tại thời điểm công bố: CIFAR-10 0,30% (EffNet-L2+SAM), CIFAR-100 3,92%, ImageNet 11,39%.

4.4 Robust với nhiễu label (mục 3.3, Bảng 4)

CIFAR-10, label bị đảo ngẫu nhiên 20/40/60/80%, ResNet-32, 200 epoch, test set giữ nguyên (clean). SAM ($\rho = 0{,}1$, hoặc $\rho = 0{,}05$ ở mức nhiễu 80% để ổn định hơn): accuracy 95,1/93,4/90,5/77,9% ở 4 mức nhiễu, **vượt mọi phương pháp chuyên biệt cho noisy-label** trừ MentorMix (95,6/94,2/91,3/81,0%); Bootstrap+SAM (95,4/94,2/91,8/79,9%) đạt gần tương đương MentorMix dù đơn giản hơn nhiều. So với SGD chuẩn (84,8/68,8/48,2/26,2%): ở mức nhiễu 80%, SAM cao hơn SGD **51,7 điểm %** ($77{,}9 - 26{,}2 = 51{,}7$, tự tính) - mức cải thiện rất lớn, minh chứng mạnh cho tính chất "robust tự nhiên với nhiễu label" mà paper nhấn mạnh.

5. Phân tích / ablation

5.1 m-sharpness (mục 4.1)

Khi chạy phân tán trên nhiều accelerator, mỗi accelerator chỉ thấy một tập con m điểm dữ liệu để tính $\hat{\epsilon}$ riêng (rồi average gradient SAM) - tương đương đổi định nghĩa sharpness trong (1) từ max trên toàn tập ($m = n$) thành tổng của nhiều max trên các tập con kích thước m ("m-sharpness"). Thực nghiệm (Hình 3 giữa): m **nhỏ hơn** cho test error tốt hơn. Đặc biệt (Hình 3 phải, đo bằng mutual information theo khung Jiang et al. 2019): m-sharpness với $m < n$ **tương quan tốt hơn** với generalization gap thật so với sharpness toàn tập ($m = n$, chính là đại lượng được Định lý 1 biện minh về lý thuyết) - paper tự nhận đây là một "hướng nghiên cứu mới thú vị", nghĩa là đại lượng lý thuyết đúng (toàn tập) không phải đại lượng thực nghiệm tốt nhất (tập con nhỏ).

5.2 Phổ Hessian (mục 4.2, Hình 3 trái)

Đo phổ Hessian (xấp xỉ bằng thuật toán Lanczos, Ghorbani et al. 2019) của WideResNet40-10 (không batch norm) trên CIFAR-10, tại epoch 1/50/300: tại epoch 300 (hội tụ), $\lambda_{max} \approx 24$ (không SAM) vs $\approx 1{,}0$ (SAM); tỉ số λ_{max}/λ_5 (proxy sharpness phổ biến, Jastrzebski et al. 2020) lên tới 11,4 (không SAM) vs 2,6 (SAM) - xác nhận SAM hội tụ về vùng độ cong thấp hơn rõ rệt. **Quan sát riêng của người ghi chú** (không phải kết luận chính paper nêu, chỉ đọc từ chi tiết Hình 3): tại epoch 1 (rất sớm), tỉ số λ_{max}/λ_5 lại **cao hơn** với SAM (3,6) so với không SAM (2,5), dù giá trị λ_{max} tuyệt đối đã thấp hơn (18,6 vs 62,9) - tức ở giai đoạn đầu huấn luyện, hiệu ứng "san bằng phổ" của SAM chưa nhất quán theo cả 2 chỉ số cùng lúc; paper không thảo luận riêng điểm này.

5.3 Ablation trong Phụ lục C

- **Độ nhạy ρ:** dùng $\rho = 0{,}05$ cố định (không tune riêng từng task) cho mọi thí nghiệm CIFAR/finetuning (Bảng 9, 10) cho kết quả **gần như không đổi** so với ρ tune riêng - SAM không quá nhạy với lựa chọn ρ.

- **Bỏ số hạng bậc 2 (C.4):** paper tự thấy một kết quả đáng ngạc nhiên - mô hình **bỏ** số hạng bậc 2 (công thức 3, dùng trong thuật toán chính) đạt test error **thấp hơn** mô hình **giữ** số hạng bậc 2 đầy đủ, dù 2 bản update gần giống nhau (cosine similarity gần 1) ở nửa đầu huấn luyện và tách xa nhau dần về cuối. Paper viết nguyên văn: ”lý do cho điều này khá chưa rõ và nên được phân tích ở công trình tiếp theo” - tức phiên bản rẻ hơn về compute lại tốt hơn về generalization, mà bản thân nhóm tác giả **không giải thích được**.
- **Chọn p-norm (C.5):** $p = 2$ (nhiễu theo hướng gradient, chuẩn hoá Euclid) tốt hơn $p = \infty$ (nhiễu dấu từng chiều) và tốt hơn nhiễu ngẫu nhiên cùng norm - xác nhận bản chất ”đối nghịch” (adversarial, không phải ngẫu nhiên) của $\hat{\epsilon}$ là quan trọng.
- **Nhiều bước inner maximization (C.6):** xấp xỉ tuyến tính 1 bước (dùng trong thuật toán chính) cho test accuracy gần tương đương chạy nhiều bước projected gradient ascent để tìm ϵ chính xác hơn, dù giá trị sharpness ước lượng được khi dùng 5 bước **cao hơn tới ~3%** so với 1 bước lúc gần hội tụ (CIFAR-10) - nghĩa là xấp xỉ bậc 1 ”sai” về giá trị sharpness tuyệt đối nhưng vẫn ”đủ tốt” để optimizer hoạt động hiệu quả.

6. Giới hạn

Tác giả tự nêu:

- Mỗi bước SAM cần 2 lần backprop - chi phí tính toán gấp đôi SGD thường cho cùng số bước (paper bù lại bằng cách so sánh ở compute budget công bằng, cho baseline gấp đôi epoch).
- Không giải thích được vì sao bỏ số hạng bậc 2 lại cho kết quả **tốt hơn** giữ nguyên (mục C.4) - tự nhận là câu hỏi mở.
- m-sharpness (đại lượng tương quan tốt nhất với generalization gap thực tế) **không phải** đại lượng mà Định lý 1 biện minh về lý thuyết (vốn dùng toàn tập, $m = n$) - khoảng cách giữa lý thuyết và thực nghiệm tốt nhất, tự nhận là hướng nghiên cứu mở.
- Định lý 1 (và chứng minh đầy đủ ở Phụ lục A) chỉ là một chặn trên xác suất cao, không phải đẳng thức; các hằng số trong chặn phụ thuộc nhiều giả định kỹ thuật (ví dụ $L_D(w) \leq \mathbb{E}_{\epsilon \sim N(0,\rho)}[L_D(w+\epsilon)]$, điều kiện ”nhiễu Gauss không làm giảm test error”, chỉ kỳ vọng đúng ở nghiệm cuối, không đúng cho w bất kỳ).

Người ghi chú nhận xét thêm (không có trong chính văn bài SAM, cần biết khi đánh giá toàn cảnh):

- Phần Related Work (mục 5) của chính bài SAM trích dẫn Dinh et al. (2017), ”Sharp minima can generalize for deep nets” - một kết quả **lý thuyết nổi tiếng mâu thuẫn trực tiếp** với giả thuyết nền tảng ”sharp minima luôn generalize kém” mà cả opt 24 và SAM dựa vào: Dinh et al. chỉ ra rằng có thể tái tham số hoá (ví dụ scale lại layer) để tạo ra cực tiểu ”sắc hơn” theo metric Hessian thông thường mà **không** đổi hàm mà network tính (do đó không đổi generalization thật). SAM **không phản biện trực tiếp** luận điểm này trong phần chính văn (chỉ liệt kê Dinh et al. như một tài liệu liên quan, không thảo luận sâu); đây là một tranh luận mở quan trọng trong literature mà người đọc nên biết, không phải lỗi hay thiếu sót cụ thể của SAM nhưng là giới hạn chung của cách đo sharpness theo kiểu Hessian/perturbation mà cả 2 bài (opt 24, opt 25) dùng.
- Phạm vi đọc: đã đọc toàn bộ thân bài (mục 1-6) và các phần ablation chính của Phụ lục C (C.3-C.6); đã đọc tổng quan Phụ lục A (chứng minh Định lý 2/PAC-Bayes) nhưng không diễn giải lại từng dòng đại số của chứng minh (không cần thiết cho mục đích ghi chú ứng dụng).

7. Liên hệ với các bài khác

- **opt 24 (Keskar, Sharp Minima):** SAM trả lời trực tiếp câu hỏi mở cuối cùng của opt 24 (”có cách nào lái LB method tránh sharp minima không?”) bằng một thuật toán cụ thể; cùng dùng ý tưởng ”đo loss trong lân cận” nhưng SAM tối ưu hoá trực tiếp đại lượng đó thay vì chỉ đo sau khi huấn luyện xong.

- **opt 26 (Edge of Stability):** cùng quan tâm tới λ_{max} của Hessian trong quá trình huấn luyện (dùng Lanczos ở cả 2 bài), nhưng góc nhìn khác: SAM so sánh **điểm cuối** của 2 phương pháp huấn luyện khác nhau, còn Edge of Stability theo dõi **quỹ đạo** λ_{max} theo thời gian của cùng 1 phương pháp.
- **opt 21/22/23 (large-batch, noise scale):** khái niệm ”m-sharpness” (tính sharpness trên tập con m mẫu rồi trung bình) có họ hàng khái niệm với ”critical batch size”/”noise scale” của opt 22-23: cả hai đều cho thấy tính toán trên tập con nhỏ hơn toàn batch/toàn dataset mang lại tín hiệu hữu ích khác với tính trên toàn bộ.
- **opt 18/19 (lr schedule, cyclical/warm restarts):** là một hướng khác (gián tiếp, qua nhiễu gradient tự nhiên của SGD) để tránh sharp minima, khác với SAM (trực tiếp, qua tối ưu hoá worst-case trong lân cận ρ-ball).
- Đã `ls` kiểm tra 5 thư mục cũ (llm/rl/distillation/pruning/interp content) tìm từ khoá sharp/flat/minima/generaliz/hessian; không có bài nào bàn trực tiếp chủ đề này (chỉ trùng từ ”minima” trong tên file `72_2026_minimax_sparse_attention.md`, không liên quan), nên không trích dẫn bài cụ thể ở các thư mục đó.

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, batch 1, GPU ≤ 8GB). Toàn bộ mục này là suy luận:

1. **Chi phí 2x backprop mỗi bước là rào cản chính cho GPU 8GB batch 1** (suy luận, ”ít liên quan trực tiếp” nếu áp dụng nguyên bản): với batch 1 và ngân sách compute đã chật (do model OCR + visual token dài), thêm một lần backprop đầy đủ mỗi bước có thể làm chậm gấp đôi thời gian fine-tune/distill - cần cân nhắc kỹ có đáng đánh đổi hay không so với lợi ích generalization, đặc biệt khi tập fine-tune OCR nhỏ (ít rủi ro overfit nghiêm trọng như ImageNet 400 epoch).
2. **SAM như công cụ chẩn đoán độ ”sắc” sau prune/distill (không cần dùng SAM để huấn luyện):** có thể dùng riêng công thức sharpness $\max_{\|\epsilon\|_2 \leq \rho} L_S(w+\epsilon) - L_S(w)$ (không cần chạy toàn bộ thuật toán SAM để train) để đo xem checkpoint sau khi prune có bị ”sắc” hơn checkpoint gốc không - một chỉ số rẻ hơn tính cả phổ Hessian, có thể tính bằng 1-vài lần forward/backward thêm trên 1 batch nhỏ (suy luận, liên hệ với thư mục pruning content, chưa kiểm chứng cho OCR).
3. **Khả năng chống overfit của SAM (mục 4.2) có thể hữu ích nếu tập fine-tune OCR nhỏ và cần train nhiều epoch:** nếu dữ liệu OCR domain-specific hạn chế (dễ overfit khi train lâu), SAM có thể giúp tiếp tục cải thiện mà không ”quay đầu” như ImageNet 400-epoch baseline không-SAM trong Bảng 2 - nhưng cần đánh giá với chi phí 2x compute có đáng trên GPU 8GB không (suy luận, cần thử nghiệm thực tế).
4. **Phiên bản rẻ hơn (bỏ số hạng bậc 2, công thức 3) đã là phiên bản chính paper dùng** - nên nếu áp dụng SAM cho dự án, không cần cân nhắc thêm việc ”có nên tính Hessian-vector product” vì chính paper đã xác nhận bản rẻ này tốt hơn bản đầy đủ (mục 5.3), giảm lo ngại về chi phí bổ sung ngoài 2x backprop cơ bản.

9. Thí nghiệm gợi ý

1. **So sánh fine-tune OCR với SAM vs AdamW chuẩn** trên cùng 1 tập nhỏ, cùng số epoch, đo validation CER/WER và thời gian wall-clock, để định lượng đánh đổi ”2x backprop” vs ”cải thiện generalization” thực tế trên GPU 8GB.
2. **Đo sharpness (không cần train bằng SAM)** của checkpoint trước/sau khi prune một phần model (liên hệ thư mục pruning content): tính $\max_{\|\epsilon\|_2 \leq \rho} L_S(w+\epsilon) - L_S(w)$ trên 1 batch validation nhỏ, với vài giá trị $\rho \in \{0{,}01, 0{,}05, 0{,}1\}$, xem checkpoint pruned có ”sắc” hơn không.
3. **Quét** ρ (grid nhỏ như paper: 0,01 đến 0,5) trên 1 bài toán fine-tune nhỏ để xác nhận paper’s claim ”$\rho = 0{,}05$ là default tốt, không nhạy cảm” có đúng ở domain OCR hay không.

4. **Thử m-sharpness với m nhỏ (mục 5.1):** nếu có thể dùng gradient-accumulation để mô phỏng nhiều ”sub-batch” nhỏ, tính $\hat{\epsilon}$ riêng cho mỗi sub-batch (kiểu m-sharpness) rồi gộp, so với tính $\hat{\epsilon}$ trên cả batch effective lớn, xem ảnh hưởng tới validation performance.
5. **Ablation bỏ/giữ số hạng bậc 2** (mục C.4 của paper) trên bài toán nhỏ của dự án, xác nhận lại hiện tượng ”bỏ bậc 2 tốt hơn” có lặp lại ở domain khác (OCR) hay là đặc thù CIFAR-10/WideResNet.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Nguồn
Chặn generalization (Theorem 1)	$L_D(w) \leq \max_{\|\epsilon\|_2 \leq \rho} L_S(w+\epsilon) + h(\|w\|_2^2/\rho^2)$	mục 2
Bài toán SAM	$\min_w \max_{\|\epsilon\|_p \leq \rho} L_S(w+\epsilon) + \lambda\|w\|_2^2$	eq. (1)
Nghiệm xấp xỉ bậc 1 ($p=2$)	$\hat{\epsilon}(w) = \rho \nabla_w L_S(w)/\|\nabla_w L_S(w)\|_2$	eq. (2), footnote 2
Gradient SAM dùng trong thuật toán	$\nabla_w L_S^{SAM}(w) \approx \nabla_w L_S(w)\|_{w+\hat{\epsilon}(w)}$	eq. (3)
Chi phí	2 lần backprop/bước	Algorithm 1
ρ mặc định	0,05	mục 3.1, C.3
Kết quả nổi bật ImageNet	ResNet-152: 20,3% → 18,4% top-1 error	Bảng 2
Chống overfit dài hạn	SAM cải thiện liên tục 100 → 400 epoch; chuẩn thì xấu đi	Bảng 2
Cải thiện robust noisy-label (80% nhiễu)	SGD 26,2% → SAM 77,9% (chênh 51,7 điểm %, tự tính)	Bảng 4
m-sharpness	m nhỏ hơn → generalization tốt hơn và tương quan với gap thật tốt hơn	mục 4.1

11. Kết luận ngắn

SAM là một thuật toán tối ưu đơn giản (thêm 1 bước nhiễu gradient trước khi tính gradient thật) nhưng hiệu quả rộng, cải thiện generalization nhất quán trên CIFAR-10/100, ImageNet, nhiều bài finetuning, và cho robust tốt với nhiễu label - đồng thời xác nhận bằng phổ Hessian rằng nó thực sự hội tụ về vùng độ cong thấp hơn. Bài trả lời trực tiếp câu hỏi mở của opt 24 bằng một công thức cụ thể, nhưng cũng tự bộc lộ vài điểm chưa giải thích được (bỏ số hạng bậc 2 lại tốt hơn giữ; m-sharpness tương quan tốt hơn sharpness toàn tập mà lý thuyết biện minh) và chưa đối chiếu trực tiếp với phản biện lý thuyết ”sharp minima vẫn có thể generalize tốt” (Dinh et al. 2017) mà chính bài trích dẫn trong Related Work.

Nguồn gốc: https://arxiv.org/abs/2010.01412

Bài 26

Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability (2021)

Tác giả: Jeremy Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter (Carnegie Mellon University, Bosch AI), Ameet Talwalkar (Carnegie Mellon University, Determined AI).
Venue / arXiv: arXiv:2103.00065 (v3, 23/11/2022); venue: Published as a conference paper at ICLR 2021.
Nguồn: https://arxiv.org/abs/2103.00065
Chủ đề: quan sát thực nghiệm rằng full-batch gradient descent (GD) trên neural network thường tự đẩy "sharpness" (giá trị riêng lớn nhất của Hessian loss huấn luyện) lên ngưỡng ổn định $2/\eta$ rồi "mắc kẹt" ngay tại đó ("Edge of Stability"), mâu thuẫn với nhiều giả định cổ điển của lý thuyết tối ưu hoá.
Vị trí trong lộ trình: đọc sau opt 24 (Keskar) và opt 25 (SAM) - cả 3 bài cùng dùng khái niệm "sharpness" nhưng với 3 định nghĩa/mục đích khác nhau (bài này tự phân biệt rõ trong chú thích 2); mở rộng câu chuyện sharpness từ "so sánh 2 cực tiểu cuối cùng" (opt 24) hay "tối ưu hoá trực tiếp" (opt 25) sang "theo dõi động lực học huấn luyện theo thời gian thực".

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

Lưu ý về phạm vi đọc: đây là một trong các bài dài nhất trong lộ trình (file text ~482.000 ký tự, ~7.100 dòng). Người ghi chú đã đọc toàn bộ thân bài chính (mục 1-8: Introduction, Background, Gradient Descent on Neural Networks, Further Experiments, Discussion, SGD, Conclusion, Acknowledgements - khoảng 950 dòng đầu) một cách đầy đủ, bao gồm tất cả hình/số liệu được trích trực tiếp trong thân bài. Phần Phụ lục (Appendix A đến Q, chiếm phần lớn còn lại của file, ~6.100 dòng) chứa các thí nghiệm mở rộng (quét kiến trúc, batch norm, nhiều task, chứng minh toán học chi tiết) được **thân bài trích dẫn và tóm tắt kết quả chính**, nhưng không được đọc toàn văn từng phụ lục - ghi chú này chỉ dựa vào những gì thân bài tự báo cáo về nội dung phụ lục (có trích dẫn rõ "Appendix X" khi nêu).

1. Bối cảnh

Hầu như không ai huấn luyện neural network bằng **full-batch** gradient descent (GD) thuần tuý trong thực tế (luôn dùng SGD/variant), nhưng GD là "nền tảng khái niệm" của mọi biến thể đó. Lý thuyết tối ưu hoá cổ điển có một quy tắc rõ ràng cho GD trên hàm **bậc hai (quadratic)**: GD với step size η phân kỳ nếu bất kỳ eigenvalue nào của ma trận Hessian A vượt ngưỡng $2/\eta$. Với deep network, hàm mất mát không phải quadratic toàn cục, nhưng xấp xỉ Taylor bậc 2 tại một điểm x_0 bất kỳ **là** quadratic với ma trận "A" chính là Hessian tại x_0. Câu hỏi bài báo đặt ra: khi train network thật bằng GD, "sharpness" (định nghĩa **chặt**: eigenvalue lớn nhất của Hessian loss huấn luyện, λ_{max}) diễn biến ra sao so với ngưỡng $2/\eta$ này?

Lưu ý thuật ngữ quan trọng (chú thích 2 của paper, trích nguyên văn ý): "Trong literature, từ 'sharpness' đã được dùng cho nhiều đại lượng khác nhau, thường gắn với generalization (ví dụ Keskar et al. 2016 = opt 24). Trong bài này, 'sharpness' **chỉ** có nghĩa là eigenvalue lớn nhất của Hessian loss huấn luyện. Chúng tôi **không** khẳng định đại lượng này có liên hệ gì với generalization." Đây là một điểm quan trọng cần phân biệt khi đọc song song với opt 24/opt 25 (xem mục 6, 7 bên dưới).

2. Ý tưởng chính

Hai phát hiện chính (đóng khung trong Introduction):

1. **Progressive sharpening (§3.1):** miễn là sharpness còn nhỏ hơn $2/\eta$ (còn "ổn định"), GD có xu hướng **áp đảo** là liên tục làm **tăng** sharpness theo thời gian.

2. **Edge of Stability (§3.2):** khi sharpness chạm tới $2/\eta$ ("breakeven point", thuật ngữ của Jastrzębski et al. 2020), GD trở nên mất ổn định cục bộ (dao động tăng dần theo hướng eigenvector lớn nhất) - nhưng **không phân kỳ hoàn toàn**. Thay vào đó, nó đi vào một "chế độ" mới: sharpness dao động ngay tại, hoặc chỉ cao hơn một chút, giá trị $2/\eta$; còn train loss dao động không đơn điệu trong ngắn hạn nhưng **vẫn giảm đều** trong dài hạn. GD dường như "muốn" tăng sharpness tiếp nhưng bị "kìm lại" - như một bài toán tối ưu hoá có ràng buộc ẩn sharpness $\leq 2/\eta$.

Hiện tượng này **mâu thuẫn trực tiếp** với nhiều giả định nền tảng của lý thuyết tối ưu hoá (L-smoothness, hội tụ đơn điệu, xấp xỉ quadratic cục bộ) - xem mục 5.

3. Phương pháp chi tiết

3.1 Cơ sở: ổn định của GD trên hàm bậc hai (§2)

Với $f(x) = \frac{1}{2}x^T Ax + b^T x + c$, trường hợp 1D ($f(x) = \frac{1}{2}ax^2 + bx + c$, $a > 0$, $x^* = -b/a$): cập nhật $x_{t+1} = x_t - \eta(ax_t + b)$ cho $x_t - x^* = (1 - \eta a)^t(x_0 - x^*)$. Nếu $a > 2/\eta$ thì $(1 - \eta a) < -1$, dãy $\{x_t\}$ dao động với biên độ **tăng theo cấp số nhân** - phân kỳ. Trường hợp d chiều: phân tích theo hệ trục riêng (eigenvector) của A, mỗi thành phần tiến hoá độc lập theo đúng công thức 1D với $a = a_i$ (eigenvalue tương ứng).

Ví dụ tự tính (tái tạo đúng Hình 2 của paper, $a_1 = 20, a_2 = 1$): với $\eta = 0{,}09$: $1 - \eta a_1 = 1 - 0{,}09 \times 20 = 1 - 1{,}8 = -0{,}8$, $|-0{,}8| < 1$ nên hội tụ theo hướng q_1 (tự tính); kiểm tra ngưỡng: $2/\eta = 2/0{,}09 \approx 22{,}22 > 20 = a_1$ - khớp đúng điều kiện ổn định. Với $\eta = 0{,}11$: $1 - \eta a_1 = 1 - 0{,}11 \times 20 = 1 - 2{,}2 = -1{,}2$, $|-1{,}2| > 1$ nên **phân kỳ** theo q_1 (tự tính); $2/\eta = 2/0{,}11 \approx 18{,}18 < 20 = a_1$ - khớp đúng mô tả Hình 2(b) của paper (GD hội tụ theo q_2 nhưng phân kỳ theo q_1, nên phân kỳ tổng thể).

Với momentum (Phụ lục B, chứng minh đầy đủ): GD + Polyak hoặc Nesterov momentum β phân kỳ nếu sharpness vượt "maximum stable sharpness" (MSS):

$$\text{MSS}_{Polyak}(\eta, \beta) = \frac{2 + 2\beta}{\eta}, \qquad \text{MSS}_{Nesterov}(\eta, \beta) = \frac{2}{\eta} \cdot \frac{2 + 2\beta}{1 + 2\beta} \tag{1}$$

(Công thức Polyak đã xuất hiện ở Goh 2017; công thức Nesterov là mới của bài này.)

Ví dụ tự tính (minh hoạ công thức 1): $\eta = 0{,}01$, $\beta = 0{,}9$: $\text{MSS}_{Polyak} = (2 + 1{,}8)/0{,}01 = 380$ (tự tính); $\text{MSS}_{Nesterov} = (2/0{,}01) \times (3{,}8/2{,}8) = 200 \times 1{,}3571 \approx 271{,}43$ (tự tính) - thấp hơn Polyak ở cùng β, nhất quán với việc Nesterov "nhìn trước" nên nhạy hơn với độ cong lớn.

3.2 Đo sharpness thực tế

Sharpness = $\lambda_{max}(\nabla^2 L(w))$, không tính trực tiếp (quá tốn) mà dùng phương pháp Lanczos/power-iteration qua Hessian-vector product (không cần vật chất hoá Hessian đầy đủ) - kỹ thuật chuẩn trong literature (Ghorbani et al. 2019, cũng được opt 25/SAM dùng).

4. Thí nghiệm và kết quả chính

4.1 Progressive sharpening (§3.1, Hình 3)

Mạng FC 2 lớp ẩn (width 200, tanh), train trên subset 5.000 ảnh CIFAR-10, $\eta = 2/600$ (nhỏ, để sharpness không bao giờ chạm $2/\eta$). Với MSE loss: sharpness tăng liên tục suốt huấn luyện (trừ một đợt giảm ngắn lúc đầu). Với cross-entropy loss: có **ngoại lệ** - sharpness **giảm** ở cuối huấn luyện, do số hạng Gauss-Newton trong Hessian của cross-entropy nhỏ dần khi margin của classifier tăng lên (giải thích chi tiết ở Phụ lục C, dẫn Soudry et al. 2018 về hiện tượng margin tăng ở "terminal phase"); tuy sharpness (Hessian) giảm, eigenvalue lớn nhất của Neural Tangent Kernel vẫn **tiếp tục tăng** (Hình 10d, 11d, không đọc chi tiết).

Hiệu ứng độ rộng mạng (Phụ lục D, trích dẫn trong thân bài): mạng càng rộng (đặc biệt dưới NTK parameterization, nơi lý thuyết dự đoán Hessian hầu như không đổi khi width $\to \infty$), progressive sharpening xảy ra **ít hơn**; cũng ít hơn nếu task dễ hoặc mạng nông.

4.2 Edge of Stability (§3.2, Hình 4-7)

Cùng mạng, $\eta = 0{,}01$ (lớn hơn): train tới khi sharpness chạm $2/\eta = 200$ (”breakeven point”). Sau đó (Hình 4b): thành phần $\langle q_1, x_t \rangle$ (chiếu lên eigenvector lớn nhất) dao động biên độ tăng dần - giống đúng ví dụ quadratic phân kỳ ở Hình 2b - train loss ban đầu vẫn giảm đơn điệu (dao động còn nhỏ) rồi ”nhảy vọt” (spike) khi dao động đủ lớn. Nhưng sau đó (Hình 4c, 1000 bước): GD **không** phân kỳ hẳn - loss tiếp tục giảm, dù không đơn điệu. Hình 5 (nhiều η, cả MSE và cross-entropy): mọi η ”đủ lớn” đều cho sharpness tăng tới đúng $2/\eta$ rồi dừng ở đó (hoặc sát trên) cho hết phần còn lại của huấn luyện.

Thí nghiệm giảm lr (Hình 6): train ở $\eta = 2/200$ tới Edge of Stability, rồi tại iteration 6000 giảm η xuống $2/300$: sharpness **ngay lập tức** tăng tiếp cho tới khi chạm ngưỡng mới $2/300$, rồi dừng lại - minh chứng trực tiếp cho diễn giải ”constrained optimization”: ràng buộc sharpness $\leq 2/\eta$ là ràng buộc ”active” (đang bị chặn), không phải GD tình cờ dừng ở đó.

Momentum (Hình 7): GD + Polyak/Nesterov cũng train tại Edge of Stability, nhưng plateau ở đúng giá trị MSS (công thức 1) thay vì $2/\eta$ thuần. Phụ lục P (trích dẫn): các eigenvalue tiếp theo ($\lambda_2, \lambda_3, ...$) cũng tăng dần rồi plateau gần $2/\eta$ (không chỉ λ_{max}).

4.3 Gradient flow trajectory (§3.4)

Vẽ sharpness theo ”thời gian” = iteration × η (thay vì theo iteration thô): với mạng này, GD ở các η khác nhau đi theo **đúng cùng một quỹ đạo** (tính gần đúng bằng Runge-Kutta RK4 cho gradient flow ODE, $\eta \to 0$) cho tới đúng điểm trên quỹ đạo đó mà sharpness chạm $2/\eta$, rồi mới ”rời khỏi” quỹ đạo để vào Edge of Stability. Hiện tượng này đúng cho ”nhiều nhưng không phải tất cả” kiến trúc đã thử (đúng cho activation liên tục khả vi; đôi khi đúng, đôi khi không với ReLU/max-pooling).

4.4 Tổng quát hoá rộng (§4)

- **11 kiến trúc** (FC và CNN, max/avg pooling, tanh/ReLU/ELU/softplus/hardtanh) × 2 loss (MSE/CE) trên subset 5k CIFAR-10 (Phụ lục J, Hình 7.1): **mọi** tổ hợp đều tái lập được pattern Edge of Stability.
- **Batch normalization** (Phụ lục K): hiện tượng vẫn giữ đúng.
- **3 task khác** (Phụ lục L, Hình 7.2): Transformer trên WikiText-2 (language modeling), hồi quy 1D tổng hợp, deep linear network trên dữ liệu Gaussian - cả 3 đều cho sharpness tăng tới $2/\eta$ rồi dừng.
- **Kiến trúc chuẩn trên full CIFAR-10** (Phụ lục M): ResNet+BN, VGG+BN, VGG không BN - progressive sharpening xảy ra ”ở mức độ ấn tượng”. Ví dụ cụ thể (số liệu của chính paper, Hình 7.3): VGG-19 train bằng gradient flow (Runge-Kutta), sharpness tăng từ $6{,}3$ (hoặc $6{,}4$ theo chú thích Hình 7.3 - paper dùng cả 2 giá trị gần nhau ở 2 chỗ khác nhau trong văn bản) lên đỉnh $2227{,}6$.

Kiểm chứng tự tính (lặp lại đúng phép tính mà chính paper trình bày để xác nhận không có sai số): step size ổn định cần $\eta < 2/2227{,}6$. Tính: $2/2227{,}6 \approx 0{,}0008976$, khớp với số paper ghi $0{,}000897$ (tự tính, sai số làm tròn). Thời gian gradient-flow để huấn luyện xong là $14{,}91$; số iteration cần ở step size ổn định: $14{,}91/0{,}000897 \approx 16.622{,}1$, khớp con số 16.622 của paper (tự tính). Nhưng ở step size ”Edge of Stability” $\eta = 0{,}16$, chỉ cần 329 iteration. Tỉ lệ chậm hơn: $16622/329 \approx 50{,}52$, khớp ”chậm hơn ít nhất 50,5 lần” (tự tính) - paper tự tính đúng, không có sai số.

5. Phân tích / ablation

Paper tự chỉ ra 4 điểm mâu thuẫn với ”quy ước” (conventional wisdom) trong tối ưu hoá (mục 5, Discussion):

1. **L-smoothness không áp dụng được (dù cục bộ):** điều kiện L-smoothness (Nesterov 1998) - nền tảng của hầu hết chứng minh hội tụ GD - đòi hỏi sharpness dọc quỹ đạo $< 2/\eta$ ở mức tối thiểu. Nhưng tại

Edge of Stability, sharpness lại **ngay trên** $2/\eta$, nên các phân tích hội tụ dựa trên L-smoothness **không thể** áp dụng cho GD ở bất kỳ step size "hợp lý" nào trên kiến trúc thực tế.

2. **GD không giảm loss đơn điệu ở step size hợp lý:** nhiều lý thuyết hội tụ (ví dụ "relaxed L-smoothness" của Zhang et al. 2020, hay Allen-Zhu et al. 2019) giả định/khẳng định train loss giảm đơn điệu cho full-batch GD - sai tại Edge of Stability.
3. **Edge of Stability về bản chất "phi-quadratic":** hành vi thật (tiến bộ liên tục dù lộn xộn) khác hẳn dự đoán của xấp xỉ Taylor bậc 2 cục bộ (sẽ phân kỳ nhanh, chứng minh ở Phụ lục E).
4. **Quy tắc chọn step size cổ điển ($\eta_t = 1/\lambda_t$, "tối ưu" theo lý thuyết quadratic) bị "đánh bại" bởi quy tắc cố định** $\eta_t = 1/\lambda_0$ (Phụ lục F, trích dẫn): quy tắc thích nghi đảm bảo loss giảm mỗi bước nhưng **chậm hơn** quy tắc cố định (vốn "theo lý thuyết là không được phép" vì có lúc tăng loss) - cho thấy việc chấp nhận dao động không đơn điệu "đáng giá" để đi bước lớn hơn.

Phần SGD (§6): các kết quả **chỉ chứng minh chặt cho full-batch GD**; với SGD, sharpness **không** ổn định ở một giá trị cố định (Phụ lục G). Nhưng literature trước (Jastrzębski et al. 2017/2019/2020) cho thấy: lr lớn hơn và/hoặc batch size nhỏ hơn "lái" SGD vào vùng sharpness thấp hơn - đây chính là phiên bản SGD của quy tắc $2/\eta$ cho GD. Paper trích dẫn trực tiếp **Keskar et al. 2016 (= opt 24)** ở chỗ này khi nói về hiệu ứng batch size nhỏ → sharpness thấp hơn. Phụ lục H minh hoạ SGD "acclimates" (thích nghi) với η/batch size theo nghĩa tương tự Edge of Stability nhưng chỉ đúng **theo kỳ vọng** (expectation), không phải mỗi bước.

6. Giới hạn

Tác giả tự nêu (mục 7, Conclusion, 4 câu hỏi mở trực tiếp):

- Không biết **tại sao** progressive sharpening xảy ra.
- Không biết **cơ chế** nào khiến GD tránh phân kỳ hoàn toàn tại Edge of Stability.
- Không biết nên chọn step size thế nào cho đúng, khi quy ước cổ điển đã sai.
- Không biết Edge of Stability có "tổng quát hoá" theo cách nào cho SGD hay các thuật toán khác không - vẫn là "vùng nghiên cứu rất sôi động" (mục 6).
- Kết quả chỉ chứng minh chặt cho **full-batch** GD; với SGD chỉ có quan sát gián tiếp, chưa có đặc trưng định lượng tương đương.
- Quỹ đạo gradient-flow (Runge-Kutta) không trùng khớp với GD thật ở **mọi** kiến trúc (chỉ đúng cho "nhiều nhưng không phải tất cả" - cụ thể: đúng cho activation liên tục khả vi, đúng cho tanh network và 3 kiến trúc chuẩn CIFAR-10 (Phụ lục M) dù dùng ReLU, nhưng **không** đúng cho deep linear network hoặc Transformer trong Phụ lục L).

Người ghi chú nhận xét thêm:

- Có một chi tiết số liệu nhỏ không hoàn toàn nhất quán trong văn bản: giá trị sharpness ban đầu của VGG-19 được ghi là $6{,}3$ ở một chỗ (gần cuối mục 4, ngay trước Hình 7.3) và $6{,}4$ ở chú thích Hình 7.3 ngay sau đó - khả năng cao là lỗi đánh máy/làm tròn nhỏ của chính paper (2 số liệu rất gần nhau, không ảnh hưởng tới kết luận chính về việc sharpness tăng lên trên 2000), nhưng được ghi lại đây theo đúng yêu cầu không tự sửa các khác biệt trong văn bản gốc.
- Định nghĩa "sharpness" ở đây (eigenvalue lớn nhất, đo tại một điểm huấn luyện cụ thể theo thời gian) khác biệt rõ với 2 bài liên quan: opt 24 (Keskar) dùng heuristic "phóng đại f trong vùng lân cận" làm proxy cho eigenvalue, nhằm **so sánh 2 nghiệm cuối** của SB/LB; opt 25 (SAM) dùng "tăng loss tối đa trong ρ-ball" làm **mục tiêu để tối ưu hoá trực tiếp**. Bài này đo chính xác λ_{max} bằng Lanczos và theo dõi **theo thời gian trong suốt quá trình huấn luyện**, không so sánh 2 nghiệm cuối cũng không dùng để regularize. Paper tự nói rõ **không** khẳng định liên hệ gì với generalization (chú thích 2) - khác hẳn mục tiêu chính của cả opt 24 và opt 25.

7. Liên hệ với các bài khác

- **opt 24 (Keskar, Sharp Minima):** được trích dẫn trực tiếp ở §6 (SGD) cho quan sát ”batch nhỏ lái SGD vào vùng sharpness thấp hơn”; đồng thời là nguồn của thuật ngữ ”sharpness” mà bài này chủ động phân biệt (chú thích 2) để tránh nhầm với generalization.
- **opt 25 (SAM):** cùng công cụ (Lanczos đo phổ Hessian) nhưng mục đích khác (so sánh điểm cuối SGD vs SAM, không theo dõi động lực học theo thời gian như bài này); việc SAM hội tụ về λ_{max} thấp hơn (opt 25, mục 4.2: ~1,0 vs ~24) có thể được nhìn qua lăng kính bài này như SAM ”ép” quá trình huấn luyện giữ sharpness ở ngưỡng thấp hơn ngưỡng Edge-of-Stability tự nhiên của SGD thường.
- **opt 18/19 (lr schedule, warm restarts/super-convergence):** liên hệ gián tiếp - nếu progressive sharpening luôn đẩy sharpness tới gần $2/\eta$, một lr schedule giảm dần (như cosine annealing, opt 18) tương đương liên tục ”hạ ngưỡng ổn định”, buộc sharpness phải giảm theo (tương tự thí nghiệm Hình 6 của bài này, vốn cho thấy giảm η khiến sharpness lại tăng tiếp tới ngưỡng mới - đây là động lực ngược, nhưng cùng logic ràng buộc).
- **opt 21/22/23 (large-batch, noise scale):** §6 của bài này gợi ý rằng khái niệm ”Edge of Stability” có thể tồn tại dạng tương tự cho SGD, phụ thuộc cả lr **và** batch size - đúng tinh thần ”noise scale” của opt 22/23 (nhiễu gradient quy mô batch ảnh hưởng tới vùng mà quá trình tối ưu ”mắc kẹt”).
- Đã `ls` kiểm tra 5 thư mục cũ (llm/rl/distillation/pruning/interp content) cho từ khoá sharp/hessian/eigenvalue/stability; không tìm bài nào bàn trực tiếp Edge of Stability hay phổ Hessian theo thời gian, nên không trích dẫn bài cụ thể ở đó.

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, batch 1, GPU ≤ 8GB). Toàn bộ mục này là suy luận, và cần nhấn mạnh: bài báo nghiên cứu **full-batch GD**, trong khi dự án dùng **SGD/Adam batch 1** - khoảng cách lớn giữa điều kiện lý thuyết và thực hành dự án, nên mọi suy luận dưới đây có độ tin cậy thấp hơn so với các bài opt khác trong loạt này:

1. **Chọn lr không nên dựa ”mù” vào công thức $\eta = 1/\lambda_t$ kiểu cổ điển** (phát hiện 4, mục 5): nếu dùng bất kỳ phương pháp ước lượng sharpness/curvature để auto-tune lr khi fine-tune OCR, kết quả của bài này cảnh báo rằng cách ”tối ưu theo lý thuyết quadratic” có thể **chậm hơn** một lr cố định đơn giản - gợi ý nên luôn so sánh thực nghiệm, không tin tưởng tuyệt đối vào công thức lý thuyết tối ưu theo mô hình bậc 2 (suy luận, ngoại suy từ full-batch GD sang SGD batch 1, chưa kiểm chứng).
2. **”Edge of Stability” như một chỉ báo khi lr quá lớn/quá nhỏ cho fine-tune:** nếu đo được sharpness (hoặc proxy rẻ hơn) của mô hình đang fine-tune và thấy nó liên tục tăng không dừng (progressive sharpening không giới hạn) mà chưa thấy loss dao động, có thể là dấu hiệu lr đang **quá nhỏ** (ở xa Edge of Stability, ”lãng phí” tốc độ hội tụ theo đúng phát hiện mục 4.4 của bài: stable step size có thể chậm hơn 50x); ngược lại nếu loss dao động dữ dội không giảm, có thể lr đã **vượt xa** ngưỡng ổn định cho batch 1 (suy luận, ít chắc chắn vì bài không nghiên cứu batch 1 trực tiếp).
3. **Ít liên quan trực tiếp tới việc chọn kiến trúc nén/prune:** các phát hiện về progressive sharpening/Edge of Stability không nói gì trực tiếp về nên prune cái gì hay distill thế nào; đây là một bài về **động lực học tối ưu hoá**, không phải về kiến trúc hay compression - phần ”ứng dụng” ở đây chủ yếu nằm ở việc **chọn/hiểu lr schedule** khi fine-tune sau prune, không ở quyết định nén (suy luận, đánh giá mức độ liên quan).

9. Thí nghiệm gợi ý

1. **Đo sharpness theo thời gian khi fine-tune model nhỏ:** dùng Lanczos/power-iteration qua Hessian-vector product (tính được bằng autodiff, không cần vật chất hoá Hessian) để theo dõi λ_{max} của loss trên 1 batch validation cố định, trong suốt quá trình fine-tune OCR batch 1, xem có xuất hiện pattern ”tăng dần rồi dao động quanh một ngưỡng” giống Edge of Stability không (dù lý thuyết chỉ chứng minh cho

full-batch).

2. **Thử nghiệm giảm lr giữa chừng** (như Hình 6 của bài) trên 1 model nhỏ fine-tune OCR: giảm lr tại 1 điểm cố định, quan sát sharpness có ”tăng lại” ngay sau đó không (nếu có, gợi ý vẫn có ràng buộc ẩn kiểu Edge-of-Stability dù dùng SGD/Adam batch nhỏ).
3. **So sánh lr cố định vs lr thích nghi theo** $1/\lambda_t$ (mô phỏng thí nghiệm Phụ lục F) trên 1 bài toán nhỏ, đo tốc độ hội tụ thực tế, kiểm tra lại phát hiện ”lr cố định thắng lr thích nghi theo lý thuyết” có lặp lại ở quy mô nhỏ/batch nhỏ không.
4. **Tái lập ví dụ 1D/2D quadratic (Hình 2) bằng code vài dòng** để tự kiểm chứng trực giác về ngưỡng $2/\eta$, trước khi áp dụng trực giác đó cho model thật - một bài tập nhỏ giúp nắm chắc nền tảng lý thuyết trước khi đo trên model lớn.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Nguồn
Ngưỡng ổn định GD (quadratic)	phân kỳ nếu eigenvalue $> 2/\eta$	§2
MSS Polyak momentum	$(2+2\beta)/\eta$	eq. (1)
MSS Nesterov momentum	$(2/\eta)\cdot(2+2\beta)/(1+2\beta)$	eq. (1)
Progressive sharpening	sharpness tăng liên tục khi còn $< 2/\eta$	§3.1
Edge of Stability	sharpness dao động tại/trên $2/\eta$; loss không đơn điệu ngắn hạn, giảm dài hạn	§3.2
Ví dụ VGG-19 (gradient flow)	sharpness $6{,}3 \to 2227{,}6$; stable step chậm hơn $\geq 50{,}5\times$	Hình 7.3, tự tính kiểm lại
Breakeven point	thời điểm sharpness lần đầu chạm $2/\eta$	thuật ngữ của Jastrzębski et al. 2020
Giới hạn SGD	sharpness không ổn định cố định; lr lớn/batch nhỏ → sharpness thấp hơn	§6

11. Kết luận ngắn

Bài báo cung cấp bằng chứng thực nghiệm rộng (11+ kiến trúc, nhiều task, cả kiến trúc chuẩn trên CIFAR-10 đầy đủ) cho thấy full-batch gradient descent trên neural network có xu hướng áp đảo tự đẩy sharpness (λ_{max} Hessian) lên ngưỡng ổn định lý thuyết $2/\eta$ (”progressive sharpening”), rồi ”mắc kẹt” ngay tại ngưỡng đó (”Edge of Stability”) mà không phân kỳ hoàn toàn - hành vi mâu thuẫn trực tiếp với L-smoothness, hội tụ đơn điệu, và dogma chọn step size cổ điển. Phát hiện này đặt ra nhiều câu hỏi mở (cơ chế, cách chọn lr đúng, tổng quát hoá cho SGD) mà tới thời điểm công bố chưa có lời giải, và chủ động phân biệt khái niệm ”sharpness” của mình (eigenvalue thuần, theo thời gian) với khái niệm cùng tên nhưng khác nghĩa ở opt 24/opt 25 (gắn với generalization, so sánh 2 nghiệm cuối hoặc dùng làm mục tiêu tối ưu).

Nguồn gốc: https://arxiv.org/abs/2103.00065

Bài 27

Visualizing the Loss Landscape of Neural Nets (2017)

Tác giả: Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, Tom Goldstein (University of Maryland; United States Naval Academy; Cornell University).
Venue / arXiv: 32nd Conference on Neural Information Processing Systems (NeurIPS/NIPS 2018); arXiv:1712.09913 (v3, 07/11/2018).
Nguồn: https://arxiv.org/abs/1712.09913
Chủ đề: kỹ thuật trực quan hoá (visualization) loss landscape của mạng neural bằng ”filter-wise normalization,” dùng để so sánh độ sắc/phẳng (sharpness/flatness) của các điểm cực tiểu, hiệu ứng kiến trúc (skip connection, độ sâu, độ rộng) lên cấu trúc không lồi, và trực quan hoá quỹ đạo tối ưu bằng PCA.
Vị trí trong lộ trình: đọc bổ sung cho nhóm benchmark optimizer (opt 28, 29, 30): các bài đó đo ”optimizer nào tốt hơn” bằng số, còn bài này giải thích **vì sao** kiến trúc (skip connection, độ rộng) và siêu tham số (batch size, weight decay) ảnh hưởng tới khả năng huấn luyện và khả năng tổng quát hoá, thông qua hình dạng hình học của loss landscape.

Lưu ý phạm vi đọc: bài báo có rất nhiều hình (contour plot, bản đồ nhiệt Hessian) khiến bản trích xuất văn bản từ PDF (`pdftotext -layout`) chứa nhiều đoạn nhãn số trên biểu đồ bị xáo trộn không đọc được (ví dụ các nhãn mức contour chồng chéo). Ghi chú này dựa trên toàn bộ phần văn xuôi chính văn (Mục 1-8, Kết luận) đã đọc đầy đủ và trích xuất được rõ ràng; phần Phụ lục (A.1, A.2: hình bổ sung như ResNet-56 ở Hình 15, so sánh định lượng filter-wise/layer-wise/không chuẩn hoá) chỉ được xác định có tồn tại qua một câu trích dẫn trong chính văn, **không đọc chi tiết nội dung phụ lục**.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Huấn luyện mạng neural đòi hỏi tối thiểu hoá một hàm loss không lồi trong không gian nhiều chiều, về lý thuyết là NP-hard (Blum & Rivest 1989, dẫn trong Mục 1), nhưng trong thực hành các phương pháp gradient đơn giản thường tìm được cực tiểu gần như toàn cục, ngay cả khi nhãn dữ liệu bị xáo trộn ngẫu nhiên trước khi huấn luyện. Tuy nhiên hành vi ”tốt” này **không phổ biến**: khả năng huấn luyện được phụ thuộc mạnh vào lựa chọn kiến trúc, optimizer, cách khởi tạo tham số, và nhiều yếu tố khác, mà ảnh hưởng cụ thể của từng yếu tố lên cấu trúc loss surface vẫn chưa rõ ràng. Vì việc đánh giá hàm loss tốn kém (cần lặp qua toàn bộ tập huấn luyện), các nghiên cứu trong lĩnh vực này chủ yếu mang tính lý thuyết, với các giả định hạn chế về phân phối dữ liệu hoặc cấu trúc phi tuyến.

2. Ý tưởng chính

Đóng góp chính (Mục 1.1):

1. Các phương pháp trực quan hoá loss hiện có (nội suy tuyến tính 1D, contour plot với hướng ngẫu nhiên) **có lỗi**, không nắm bắt đúng hình học cục bộ (sharpness/flatness) của cực tiểu.
2. Đề xuất **”filter-wise normalization”**: chuẩn hoá hướng ngẫu nhiên theo từng filter để loại bỏ hiệu ứng bất biến tỉ lệ (scale invariance) của mạng ReLU/batch-norm, giúp độ sắc đo được tương quan tốt với lỗi tổng quát hoá, ngay cả khi so sánh giữa các kiến trúc/phương pháp huấn luyện khác nhau.
3. Khi mạng đủ sâu, loss landscape chuyển đổi nhanh từ gần lồi sang ”hỗn loạn” (chaotic); chuyển đổi này đi kèm sụt giảm khả năng tổng quát hoá và cuối cùng mất khả năng huấn luyện được.
4. **Skip connection** thúc đẩy cực tiểu phẳng và ngăn chuyển đổi sang hỗn loạn, giải thích vì sao skip connection cần thiết để huấn luyện mạng rất sâu.

5. Đo định lượng độ không lồi bằng trị riêng nhỏ nhất (âm nhất) của Hessian quanh cực tiểu, trực quan hoá dưới dạng bản đồ nhiệt.
6. Quỹ đạo tối ưu của SGD nằm trong một không gian **rất ít chiều**; hướng ngẫu nhiên thất bại trong việc trực quan hoá quỹ đạo vì lý do hình học (hai vector Gauss ngẫu nhiên trong không gian nhiều chiều gần như vuông góc với xác suất cao).

3. Phương pháp chi tiết

3.1 Các phương pháp trực quan hoá cơ bản (Mục 3)

Hàm loss $L(\theta) = \frac{1}{m}\sum_{i=1}^{m} \ell(x_i, y_i; \theta)$. Vì θ nhiều chiều, phải giảm chiều để vẽ:

- **Nội suy tuyến tính 1D** (Goodfellow et al. 2015): chọn 2 bộ tham số θ, θ', vẽ $f(\alpha) = L((1-\alpha)\theta + \alpha\theta')$. Nhược điểm: khó thấy độ không lồi trên đồ thị 1D; không tính tới batch normalization hay tính bất biến đối xứng của mạng.
- **Contour plot với hướng ngẫu nhiên:** chọn điểm trung tâm θ^* và 2 vector hướng ngẫu nhiên δ, η (phân phối Gauss), vẽ
$$f(\alpha, \beta) = L(\theta^* + \alpha\delta + \beta\eta). \qquad (1)$$
Khi vẽ 2D, các tham số batch normalization được giữ cố định (hướng ngẫu nhiên không áp dụng lên chúng).

3.2 Filter-wise normalization (Mục 4, đóng góp chính)

Vấn đề cốt lõi: mạng dùng ReLU **bất biến tỉ lệ**: nhân trọng số một lớp với 10 và chia lớp kế tiếp cho 10 không làm thay đổi hành vi mạng; hiệu ứng này càng rõ khi dùng batch normalization (norm của filter trở nên vô nghĩa vì output mỗi lớp được tái chuẩn hoá). Do đó, hướng ngẫu nhiên thô không chuẩn hoá có thể cho **ấn tượng sai lệch**: mạng có trọng số lớn trông "phẳng" (nhiễu đơn vị không ảnh hưởng nhiều vì trọng số sống ở quy mô lớn hơn 1), còn mạng có trọng số nhỏ (dù tương đương về hành vi, chỉ là tái tỉ lệ) trông "sắc." Dinh et al. (2017) đã khai thác chính hiệu ứng này để tạo các cặp mạng tương đương nhưng có độ sắc quan sát khác nhau.

Để loại hiệu ứng này, paper đề xuất chuẩn hoá hướng ngẫu nhiên d (cùng kích thước với θ) theo **từng filter**: với $d_{i,j}$ là filter thứ j của lớp i trong d (không phải từng trọng số riêng lẻ), thay thế

$$d_{i,j} \leftarrow \frac{d_{i,j}}{\|d_{i,j}\|}\|\theta_{i,j}\|,$$

với $\|\cdot\|$ là chuẩn Frobenius. Chuẩn hoá này áp dụng cho cả lớp tích chập (Conv) và lớp kết nối đầy đủ (FC, coi như Conv với output 1×1, mỗi filter tương ứng trọng số sinh ra một neuron). Điểm khác biệt so với cách chuẩn hoá của Im et al. (2016): cách đó chỉ chuẩn hoá **toàn bộ** hướng theo một norm chung, không xét riêng từng filter.

3.3 Đo độ không lồi bằng Hessian (Mục 6)

Tính tỉ lệ $|\lambda_{\min}/\lambda_{\max}|$ (trị riêng nhỏ nhất và lớn nhất của Hessian) tại từng điểm trên lưới 2D, dùng **phương pháp Lanczos khởi động lại ngầm (implicitly restarted Lanczos)**, chỉ cần tích Hessian-vector (tính trực tiếp bằng automatic differentiation), không cần biểu diễn Hessian đầy đủ hay phân tích trị riêng toàn phần. Vùng "xanh" (tỉ lệ gần 0) biểu thị vùng lồi hơn; vùng "vàng" biểu thị độ cong âm đáng kể.

3.4 Trực quan hoá quỹ đạo tối ưu bằng PCA (Mục 7.2)

Hướng ngẫu nhiên thất bại khi trực quan hoá quỹ đạo tối ưu vì lý do hình học: kỳ vọng cosine similarity giữa 2 vector Gauss ngẫu nhiên trong không gian n chiều xấp xỉ $\sqrt{2/(\pi n)}$ (Goldstein & Studer 2016, Lemma 5), tức gần vuông góc khi n lớn; nếu quỹ đạo tối ưu nằm trong một không gian con hạng thấp, một hướng ngẫu

nhiên hầu như luôn vuông góc với không gian con đó và không bắt được biến động thật. Giải pháp: gọi θ_i là tham số tại epoch i, θ_n là tham số cuối sau n epoch; áp PCA lên ma trận $M = [\theta_0 - \theta_n; \ldots; \theta_{n-1} - \theta_n]$, chọn 2 thành phần giải thích nhiều biến động nhất làm 2 trục vẽ, rồi chiếu quỹ đạo lẫn contour loss lên 2 trục đó.

4. Thí nghiệm và kết quả chính

4.1 "Sharp vs. Flat dilemma" (Mục 5)

Huấn luyện VGG-9 (có batch norm) trên CIFAR-10, batch nhỏ (128) và batch lớn (8192, tương đương 16,4% dữ liệu huấn luyện), với/không weight decay (WD):

Thiết lập	Test error batch nhỏ	Test error batch lớn	Độ sắc quan sát (không chuẩn hoá)
WD $= 0$	7,37%	11,07%	batch nhỏ "phẳng", batch lớn "sắc"
WD $= 5 \times 10^{-4}$	6,00%	10,19%	**đảo ngược**: batch lớn "phẳng" hơn batch nhỏ

Điểm mấu chốt: **batch nhỏ luôn tổng quát hoá tốt hơn** trong cả hai thiết lập, nhưng độ sắc quan sát bằng nội suy tuyến tính không chuẩn hoá **đảo chiều** tuỳ có weight decay hay không, tức không tương quan thật với khả năng tổng quát hoá. Nguyên nhân (xác nhận qua histogram trọng số): batch nhỏ có nhiều update/epoch hơn, nên hiệu ứng co trọng số của weight decay rõ hơn, làm thay đổi **quy mô tuyệt đối** của trọng số (không liên quan tới hình học thật) chứ không phản ánh độ sắc nội tại. Khi dùng **filter-normalized plots** (Hình 3), sự khác biệt sharpness vẫn còn nhưng "tinh tế hơn nhiều," và lần này batch lớn **nhất quán** sắc hơn (dù không quá kịch tính) và có test error cao hơn, khớp với kỳ vọng.

4.2 Hiệu ứng skip connection và độ sâu (Mục 6)

Huấn luyện ResNet-20/56/110 và các bản "noshort" (bỏ skip connection, giống VGG) trên CIFAR-10 (SGD + Nesterov momentum, batch 128, weight decay 0,0005, 300 epoch, LR khởi tạo 0,1 giảm 10 lần ở epoch 150/225/275; các mạng noshort sâu cần LR khởi tạo nhỏ hơn, 0,01):

Mạng	Test error	Mạng (noshort)	Test error
ResNet-20	7,37%	ResNet-20-NS	8,18%
ResNet-56	5,89%	ResNet-56-NS	13,31%
ResNet-110	5,79%	ResNet-110-NS	16,44%

Quan sát chính: không có skip connection, độ sâu làm loss landscape chuyển từ "khá lành" (ResNet-20-noshort) sang "hỗn loạn mạnh" (ResNet-56/110-noshort, vùng rộng mà hướng gradient không chỉ về cực tiểu, loss tăng rất nhanh theo một số hướng). Có skip connection, hình dạng và độ rộng của đường contour mức 0,1 **gần như giống nhau** từ 20 tới 110 lớp, nghĩa là skip connection ngăn hoàn toàn chuyển đổi sang hỗn loạn khi tăng độ sâu; hiệu ứng này rõ nhất ở mạng sâu, gần như không đáng kể ở mạng nông (ResNet-20 vs ResNet-20-noshort). DenseNet-121 (một kiểu skip connection khác) cũng không có độ không lồi đáng kể. Wide-ResNet-56 (tăng số filter mỗi lớp $k = 2, 4, 8$) cho landscape ngày càng lồi, cực tiểu phẳng hơn, không có hành vi hỗn loạn, và tổng quát hoá tốt nhất trong các thí nghiệm của paper.

4.3 Xác nhận bằng trị riêng Hessian (Hình 7)

Tỉ lệ $|\lambda_{\text{min}}/\lambda_{\text{max}}|$ xác nhận trực quan: vùng "trông lồi" trên contour plot thực sự có trị riêng âm không đáng kể (không có "độ không lồi ẩn" bị bỏ lỡ bởi phép chiếu giảm chiều); vùng "hỗn loạn" có độ cong âm lớn. Với landscape trông lồi như DenseNet, trị riêng âm vẫn nhỏ (dưới 1% độ lớn trị riêng dương) trên một vùng rộng.

4.4 Quỹ đạo tối ưu (Mục 7)

Hướng ngẫu nhiên (Hình 8a) hoặc 1 hướng ngẫu nhiên + 1 hướng chỉ từ điểm khởi tạo tới nghiệm (Hình 8b, theo Goodfellow et al. 2015) đều **không bắt được** biến động thật của quỹ đạo (trục bị "siêu zoom," trông như random walk). Dùng PCA (Hình 9, VGG-9, SGD và Adam, WD $\in \{0, 5 \times 10^{-4}\}$, batch $\in \{128, 8192\}$): thành phần PCA thứ nhất giải thích từ **24,57% đến 87,78%** biến động tuỳ thiết lập; tổng 2 thành phần PCA đầu giải thích **40% đến 90%** biến động của toàn bộ quỹ đạo huấn luyện, xác nhận quỹ đạo tối ưu nằm trong không gian rất ít chiều. Ở giai đoạn đầu huấn luyện, quỹ đạo di chuyển vuông góc với đường contour (giống gradient descent không ngẫu nhiên); ở giai đoạn sau, với weight decay và batch nhỏ, quỹ đạo gần như song song với contour và "quay quanh" nghiệm khi bước học (stepsize) còn lớn, rồi "gãy" vào cực tiểu gần nhất khi LR giảm (điểm đỏ).

5. Phân tích / ablation

- **Weight decay bật/tắt** là ablation then chốt cho thấy độ sắc quan sát không chuẩn hoá là giả tạo (đảo chiều hoàn toàn khi bật weight decay), còn filter-normalized plots giữ kết luận nhất quán.
- **Độ sâu (depth) có/không skip connection**: đây là ablation chính của Mục 6, cho thấy chuyển đổi lồi → hỗn loạn chỉ xảy ra ở mạng **sâu** và **không có** skip connection; ở mạng nông hiệu ứng này không đáng kể.
- **Độ rộng (width)** $k = 2, 4, 8$: ablation phụ cho thấy tăng độ rộng cũng ngăn hành vi hỗn loạn, độc lập một phần với hiệu ứng skip connection.
- **"Note of caution" (tự phản biện trong paper, Mục 6):** paper tự đặt câu hỏi liệu các plot 2D có thực sự phản ánh độ lồi hay không. Về mặt toán, trị riêng của mặt cắt giảm chiều (hướng Gauss ngẫu nhiên) là **trung bình có trọng số** (trọng số là biến ngẫu nhiên Chi-square) của trị riêng đầy đủ chiều. Hệ quả: nếu plot giảm chiều cho thấy không lồi, chắc chắn không gian đầy đủ chiều cũng không lồi; nhưng nếu plot giảm chiều trông lồi, **không suy ra** hàm đầy đủ chiều thực sự lồi, chỉ suy ra độ cong trung bình (trị riêng trung bình) là dương. Đây là lý do paper bổ sung phân tích Hessian eigenvalue (Mục 4.3 của ghi chú này) để kiểm tra "độ không lồi ẩn."
- **SGD vs Adam trong quỹ đạo PCA**: cả hai optimizer đều cho quỹ đạo thấp chiều tương tự về bản chất (40-90% biến động trong 2 chiều), gợi ý tính thấp-chiều của quỹ đạo là thuộc tính của **landscape**, không riêng của optimizer nào.

6. Giới hạn

Do tác giả tự nêu:

- Lập luận bất biến tỉ lệ của filter normalization **chỉ áp dụng cho mạng dùng ReLU** (rectified networks); không tổng quát rõ ràng cho các hàm kích hoạt khác.
- Các tham số batch normalization bị **giữ cố định** khi vẽ 2D (không áp dụng hướng ngẫu nhiên lên chúng), một đơn giản hoá có chủ đích, không phải mô hình hoá đầy đủ không gian tham số.
- Apparent convexity trong plot giảm chiều **không đảm bảo** hàm đầy đủ chiều lồi thật (chỉ đảm bảo độ cong trung bình dương); đây là giới hạn cố hữu của mọi phương pháp giảm chiều để trực quan hoá, paper tự nêu rõ trong "note of caution."
- Phương pháp PCA trajectory cần lưu tham số mô hình ở **mọi epoch** huấn luyện để tính M, tốn bộ nhớ/I-O, không áp dụng tức thời trong khi huấn luyện.
- Paper không đưa ra lý thuyết chặt chẽ giải thích đầy đủ **tại sao** chuyển đổi lồi → hỗn loạn xảy ra ở một độ sâu/độ rộng cụ thể; chỉ quan sát thực nghiệm và đưa giả thuyết định tính (vùng hấp dẫn lồi "nông" hơn ở mạng sâu không skip connection khiến điểm khởi tạo ngẫu nhiên dễ rơi vào vùng hỗn loạn).

Do chúng ta nhận thấy:

- Phần phụ lục (so sánh định lượng filter-wise/layer-wise/không chuẩn hoá, và một số hình bổ sung như Hình 15 cho ResNet-56) **không được đọc chi tiết** trong lượt làm ghi chú này (xem khai báo phạm vi

đọc ở đầu file); các kết luận trích ở Mục 4-5 của ghi chú đều lấy từ chính văn, không suy diễn từ phụ lục chưa đọc.
- Toàn bộ thí nghiệm dùng CIFAR-10/CIFAR-10-scale models (VGG-9, ResNet-20/56/110, Wide-ResNet, DenseNet-121); không có thí nghiệm trên mô hình ngôn ngữ, Transformer, hay VLM, nên khả năng tổng quát hoá kết luận sang kiến trúc hoàn toàn khác (như mô hình hybrid attention/SSM của dự án) là suy luận, không phải kết quả trực tiếp của paper.

7. Liên hệ với các bài khác

- **opt 28/29/30** (benchmark optimizer): các bài đó đo ”optimizer nào thắng” bằng số trên nhiều bài toán; bài này cung cấp một lời giải thích **hình học** bổ sung cho lý do batch size, weight decay, kiến trúc ảnh hưởng tới kết quả huấn luyện/tổng quát hoá, một góc nhìn khác (cơ chế) so với góc nhìn benchmark (hiện tượng) của 3 bài kia.
- **pruning content/01 (Optimal Brain Damage/Surgeon) và pruning content/22 (Optimal Brain Compression)** (đã xác nhận tồn tại qua `ls`): cả hai dùng thông tin Hessian (độ cong bậc 2) để ước lượng tầm quan trọng của trọng số khi cắt tỉa; bài này dùng chính trị riêng Hessian để đo độ không lồi của landscape, cùng công cụ toán (Hessian-vector product, Lanczos) nhưng mục đích khác (giải thích hình học thay vì quyết định cắt tỉa). Liên hệ tự nhiên: nếu một vùng loss landscape ”hỗn loạn” (trị riêng âm lớn) sau khi cắt tỉa, các phương pháp pruning dựa trên xấp xỉ Hessian bậc 2 (giả định cục bộ gần lồi) có thể kém chính xác hơn ở vùng đó.
- **math content** (theo titles.tsv, thư mục đang viết đồng thời): kỹ thuật PCA dùng để trực quan hoá quỹ đạo tối ưu (Mục 7.2 của ghi chú này) là ứng dụng trực tiếp của PCA/phân tích trị riêng ma trận hiệp phương sai, nội dung cơ bản thường có trong các bài toán nền tảng về đại số tuyến tính/thống kê của thư mục này (tham khảo theo số thứ tự titles.tsv, chưa xác nhận bằng `ls` vì thư mục math đang được viết đồng thời).

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, chạy trên GPU <= 8GB, batch 1). Nhận định dưới đây là suy luận:

1. **Batch size 1 (ràng buộc bắt buộc do GPU 8GB) có thể là một điểm ”có lợi bất ngờ”** theo logic của paper: batch nhỏ có xu hướng tìm cực tiểu phẳng hơn, tổng quát hoá tốt hơn (Mục 4.1 của ghi chú này). Đây là lý do để **không quá lo lắng** về việc buộc phải dùng batch 1, miễn là learning rate và weight decay được điều chỉnh hợp lý cho batch rất nhỏ (gradient rất nhiễu).
2. **Cảnh báo quan trọng cho chiến lược pruning**: phát hiện ”skip connection ngăn chuyển đổi sang hỗn loạn ở mạng sâu” gợi ý rằng **cắt tỉa làm hỏng hoặc làm yếu đường skip/residual** (ví dụ cắt tỉa toàn bộ một khối residual, hoặc cắt kênh theo cách phá vỡ shortcut) có nguy cơ đẩy loss landscape của mô hình đã cắt tỉa về phía ”hỗn loạn,” làm khó phục hồi (recovery training) hơn so với cắt tỉa cách khác giữ nguyên cấu trúc skip. Nên ưu tiên các chiến lược cắt tỉa (structured pruning) giữ nguyên topology của đường residual/shortcut trong kiến trúc hybrid GatedDeltaNet + attention.
3. **Kỹ thuật filter-normalized visualization có thể dùng như công cụ chẩn đoán rẻ**: sau khi cắt tỉa/distill, vẽ loss landscape 1D/2D quanh checkpoint cuối (dùng filter normalization) để kiểm tra trực quan mô hình đã cắt tỉa có rơi vào vùng ”sắc/hỗn loạn” hay vẫn ở vùng ”phẳng/lồi” như mô hình gốc, như một chỉ báo sớm về rủi ro tổng quát hoá kém, trước khi tốn công đánh giá đầy đủ trên benchmark OCR.
4. **PCA trajectory cho bước recovery training**: có thể áp dụng kỹ thuật Mục 3.4 cho chính giai đoạn fine-tune/distill phục hồi sau cắt tỉa, để kiểm tra liệu quỹ đạo phục hồi có thấp chiều (gợi ý hội tụ nhanh, ít cần nhiều bước) hay không, giúp quyết định số bước huấn luyện phục hồi cần thiết (liên hệ trực tiếp tới opt 31, Hyperband, về phân bổ tài nguyên).

9. Thí nghiệm gợi ý

1. **So sánh filter-normalized loss landscape của mô hình gốc và mô hình đã cắt tỉa (pruned)** quanh checkpoint cuối cùng, trên một batch nhỏ dữ liệu OCR validation. Đo và so sánh độ "sắc" (dao động của loss khi di chuyển dọc 2 hướng chuẩn hoá) giữa 2 mô hình; kỳ vọng theo paper: mô hình có cấu trúc skip bị phá vỡ nhiều hơn sẽ trông sắc/hỗn loạn hơn.
2. **Tính tỉ lệ trị riêng Hessian** $|\lambda_{\min}/\lambda_{\max}|$ (dùng thư viện Hessian-vector product như PyHessian, không cần Hessian đầy đủ) tại checkpoint cuối của vài cấu hình pruning khác nhau (giữ/không giữ skip connection), để định lượng độ không lồi, bổ sung cho quan sát trực quan ở thí nghiệm 1.
3. **Tái hiện thí nghiệm "sharp vs. flat dilemma" ở quy mô nhỏ**: fine-tune một mô hình nhỏ của dự án với 2 batch size khác nhau (ví dụ batch 1 so với gradient-accumulation tương đương batch 16) và 2 mức weight decay, dùng nội suy tuyến tính không chuẩn hoá và có chuẩn hoá filter; kiểm tra liệu kết luận "độ sắc không chuẩn hoá bị đảo chiều theo weight decay" có tái hiện ở quy mô/kiến trúc khác không.
4. **PCA trajectory cho bước recovery training sau pruning**: lưu checkpoint tham số mỗi vài trăm bước trong giai đoạn phục hồi, áp PCA lên các hiệu $\theta_i - \theta_n$, đo phần trăm biến động giải thích bởi 2 thành phần đầu. Nếu tỉ lệ cao (giống mức 40-90% của paper), đây là tín hiệu quỹ đạo phục hồi đơn giản, có thể dùng ít bước hơn.
5. **Ablation skip connection trên một mạng con nhỏ** (ví dụ một khối attention/GatedDeltaNet đơn giản hoá, đủ nhỏ để chạy nhiều cấu hình trên 8GB GPU): so sánh landscape có/không đường residual quanh cùng một điểm khởi tạo, để kiểm tra liệu hiệu ứng "skip connection ngăn hỗn loạn" của paper (quan sát trên ResNet/CNN) có xuất hiện tương tự trên kiến trúc hybrid attention/SSM của dự án hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Nội suy tuyến tính 1D	$f(\alpha) = L((1-\alpha)\theta + \alpha\theta')$
Contour plot 2D	$f(\alpha,\beta) = L(\theta^* + \alpha\delta + \beta\eta)$ (công thức 1)
Filter-wise normalization	$d_{i,j} \leftarrow \frac{d_{i,j}}{\|d_{i,j}\|}\|\theta_{i,j}\|$
Cosine similarity kỳ vọng (2 vector Gauss, n chiều)	$\approx \sqrt{2/(\pi n)}$
Đo độ không lồi	tỉ lệ $\|\lambda_{\min}/\lambda_{\max}\|$ của Hessian, tính qua Lanczos + Hessian-vector product
PCA trajectory	$M = [\theta_0 - \theta_n; \ldots; \theta_{n-1} - \theta_n]$, chọn 2 thành phần chính
Test error ResNet vs noshort (CIFAR-10)	20 lớp: 7,37% vs 8,18%; 56 lớp: 5,89% vs 13,31%; 110 lớp: 5,79% vs 16,44%
Biến động giải thích bởi 2 PCA đầu (Hình 9)	khoảng 40% đến 90% tuỳ thiết lập (SGD/Adam, batch, weight decay)

11. Kết luận ngắn

Paper đề xuất filter-wise normalization để sửa lỗi hệ thống của các phương pháp trực quan hoá loss landscape trước đó (bị nhiễu bởi tính bất biến tỉ lệ của mạng ReLU/batch-norm), từ đó cho thấy độ sắc của cực tiểu tương quan thật với khả năng tổng quát hoá khi so sánh đúng cách. Bằng trực quan hoá có kiểm soát, paper chỉ ra mạng sâu không có skip connection chuyển nhanh từ loss landscape gần lồi sang hỗn loạn, trong khi skip connection (và tăng độ rộng) ngăn chặn chuyển đổi này, giải thích một phần vì sao các kiến trúc đó dễ huấn luyện và tổng quát hoá tốt hơn. Quỹ đạo tối ưu của SGD/Adam được chứng minh thực nghiệm là nằm trong không gian rất ít chiều, giải thích vì sao các phương pháp trực quan hoá dựa trên hướng ngẫu nhiên trước đó thất bại. Các kỹ thuật (filter normalization, bản đồ Hessian eigenvalue, PCA trajectory) có thể dùng như công cụ chẩn đoán định tính cho các quyết định thiết kế kiến trúc và pruning trong các dự án nén mô hình.

Nguồn gốc: https://arxiv.org/abs/1712.09913

Bài 28

On Empirical Comparisons of Optimizers for Deep Learning (2019)

Tác giả: Dami Choi, Christopher J. Shallue, Zachary Nado, Jaehoon Lee, Chris J. Maddison, George E. Dahl (Google Research/Brain Team; một phần đồng tác giả thuộc Vector Institute, University of Toronto, DeepMind, Institute for Advanced Study).
Venue / arXiv: arXiv:1910.05446 (v3, 16/06/2020); paper không nêu venue hội nghị/tạp chí trong chính văn bản, chỉ có nhãn arXiv.
Nguồn: https://arxiv.org/abs/1910.05446
Chủ đề: đo lại độ tin cậy của các so sánh thực nghiệm giữa optimizer (SGD, Momentum, Nesterov, RMSProp, Adam, NAdam) cho deep learning; chỉ ra rằng quan hệ ”bao hàm” (inclusion) giữa các optimizer dự đoán đúng thứ hạng khi tune đủ kỹ, và thứ hạng optimizer rất nhạy với protocol tuning siêu tham số.
Vị trí trong lộ trình: đọc cùng nhóm với opt 29 (Crowded Valley, benchmark optimizer quy mô lớn hơn) và opt 30 (AlgoPerf, benchmark chuẩn hoá ra đời sau để giải quyết đúng vấn đề phương pháp luận mà bài này nêu ra); cũng liên hệ opt 31/32/33 (Hyperband, Bayesian optimization) vì protocol tuning ở đây dùng quasi-random search, một baseline mà các bài đó so sánh với.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Chọn optimizer là một bước trung tâm trong pipeline deep learning, nhưng **chưa có lý thuyết** giải thích đầy đủ nên chọn optimizer nào (Mục 1). Cộng đồng do đó dựa vào các nghiên cứu thực nghiệm, ví dụ Wilson et al. (2017, ”The Marginal Value of Adaptive Gradient Methods”) và Schneider et al. (2019, DeepOBS). Hai công trình này báo cáo rằng optimizer ”thích ứng” (adaptive, như Adam, RMSProp) **không** vượt trội, thậm chí thua, các optimizer đơn giản hơn (SGD, Momentum) trên nhiều workload.

Paper này chỉ ra một quan sát toán học quan trọng bị các nghiên cứu trên bỏ qua: các optimizer bậc một phổ biến tạo thành một **hệ thống bao hàm tự nhiên** (inclusion hierarchy). Ví dụ, Adam và RMSProp có thể **xấp xỉ** Momentum nếu hệ số ε ở mẫu số công thức cập nhật được cho lớn tuỳ ý. Nhưng điều này không có nghĩa quan hệ bao hàm luôn ”có ý nghĩa thực hành”: các giá trị siêu tham số cho phép Adam khớp Momentum có thể quá khó tìm (ví dụ chỉ đạt trong giới hạn $\varepsilon \to \infty$) nếu không tune đúng cách.

2. Ý tưởng chính

Hai phát hiện chính, liên quan chặt chẽ với nhau (Tóm tắt, Mục 1):

1. **Quan hệ bao hàm giữa optimizer dự đoán đúng kết quả so sánh trong thực tế** khi tune đủ kỹ: optimizer tổng quát hơn (ví dụ Adam) **không bao giờ** thua kém optimizer là trường hợp đặc biệt của nó (ví dụ Momentum) trong các thí nghiệm của paper, trái với ”quan điểm thông thường” từ Wilson et al. (2017) và Balles & Hennig (2017).
2. **So sánh optimizer rất nhạy với protocol tune siêu tham số.** Paper chứng minh điều này bằng cách tái lập kết quả của Wilson et al. (2017) và Schneider et al. (2019), rồi chỉ bằng cách **mở rộng không gian tìm kiếm siêu tham số**, đảo ngược hoặc xoá bỏ hoàn toàn thứ hạng optimizer mà hai công trình đó báo cáo.

Ý nghĩa thực hành: nếu một paper báo cáo optimizer tổng quát (Adam) thua optimizer đặc biệt (Momentum/SGD), điều đó có khả năng cao là do **tune chưa đủ**, không phải do bản chất thuật toán.

3. Phương pháp chi tiết

3.1 Định nghĩa optimizer và quan hệ bao hàm (Mục 3)

Một optimizer bậc một được xác định bởi **update rule** M: $\theta_{t+1} = M(H_t, \varphi_t)$, trong đó $H_t = \{\theta_s, \nabla\ell(\theta_s), \ell(\theta_s)\}_{s=0}^{t}$ là lịch sử các điểm đã đi qua, và φ_t là thiết lập siêu tham số tại bước t. Bảng 1 của paper liệt kê 6 update rule:

Optimizer	Công thức cập nhật
SGD(H_t, η_t)	$\theta_{t+1} = \theta_t - \eta_t \nabla\ell(\theta_t)$
Momentum(H_t, η_t, γ)	$v_{t+1} = \gamma v_t + \nabla\ell(\theta_t)$, $\theta_{t+1} = \theta_t - \eta_t v_{t+1}$
Nesterov(H_t, η_t, γ)	$v_{t+1} = \gamma v_t + \nabla\ell(\theta_t)$, $\theta_{t+1} = \theta_t - \eta_t(\gamma v_{t+1} + \nabla\ell(\theta_t))$
RMSProp$(H_t, \eta_t, \gamma, \rho, \varepsilon)$	$v_{t+1} = \rho v_t + (1-\rho)\nabla\ell(\theta_t)^2$, $m_{t+1} = \gamma m_t + \frac{\eta_t}{\sqrt{v_{t+1}} + \varepsilon}\nabla\ell(\theta_t)$, $\theta_{t+1} = \theta_t - m_{t+1}$
Adam$(H_t, \alpha_t, \beta_1, \beta_2, \varepsilon)$	$m_{t+1} = \beta_1 m_t + (1-\beta_1)\nabla\ell(\theta_t)$, $v_{t+1} = \beta_2 v_t + (1-\beta_2)\nabla\ell(\theta_t)^2$, $\hat{b}_{t+1} = \sqrt{1-\beta_2^{t+1}}/(1-\beta_1^{t+1})$, $\theta_{t+1} = \theta_t - \alpha_t \frac{m_{t+1}}{\sqrt{v_{t+1}} + \varepsilon}\hat{b}_{t+1}$
NAdam$(H_t, \alpha_t, \beta_1, \beta_2, \varepsilon)$	giống Adam nhưng tử số dùng $\beta_1 m_{t+1} + (1-\beta_1)\nabla\ell(\theta_t)$ (kiểu Nesterov) thay vì m_{t+1}

Định nghĩa 1 (quan hệ bao hàm, Mục 3.1): M là tập con/trường hợp đặc biệt của N (viết $M \subseteq N$) nếu với mọi thiết lập φ của M, tồn tại một dãy thiết lập ψ^i của N sao cho $\lim_{i\to\infty} N(H_t, \psi_t^i) = M(H_t, \varphi_t)$ với mọi t và mọi lịch sử H_t. Paper chứng minh (Phụ lục A) các quan hệ:

$$\text{SGD} \subseteq \text{Momentum} \subseteq \text{RMSProp}, \qquad \text{SGD} \subseteq \text{Momentum} \subseteq \text{Adam}, \qquad \text{SGD} \subseteq \text{Nesterov} \subseteq \text{NAdam}.$$

Chứng minh trực giác cho Momentum $\subseteq$ Adam (Phụ lục A, paper đặt $\beta_2 = 0$): cho Adam$(H_t, \alpha_t = \varepsilon\,\eta_t(1-\gamma^t), \beta_1 = \gamma, \beta_2 = 0, \varepsilon)$, khi đó $m_{t+1} = \gamma m_t + (1-\gamma)\nabla\ell(\theta_t)$ và

$$\theta_{t+1} = \theta_t - \frac{\eta_t}{1-\gamma} \cdot \frac{m_{t+1}}{|\nabla\ell(\theta_t)|/\varepsilon + 1}.$$

Khi ε rất lớn (nên $|\nabla\ell(\theta_t)|/\varepsilon \to 0$), biểu thức trên hội tụ về $\theta_{t+1} = \theta_t - \frac{\eta_t}{1-\gamma} m_{t+1}$, và vì $m_{t+1}^{(\text{Adam})} \equiv (1-\gamma) v_{t+1}^{(\text{Momentum})}$, đây chính là công thức cập nhật của Momentum.

Kiểm tra bằng số (tự tính): lấy $\gamma = 0{,}9$, $\eta_t = 0{,}1$, $\nabla\ell(\theta_t) = 4$, $m_t = v_t = 0$ (bước đầu). Với Adam ($\varepsilon = 10^6$, rất lớn): $m_{t+1} = 0{,}9 \times 0 + 0{,}1 \times 4 = 0{,}4$; vì $|\nabla\ell(\theta_t)|/\varepsilon = 4/10^6 \approx 0$, độ thay đổi tham số là $\Delta\theta = -\frac{0{,}1}{1-0{,}9} \times 0{,}4 = -1 \times 0{,}4 = -0{,}4$. Với Momentum: $v_{t+1} = 0{,}9 \times 0 + 4 = 4$, $\Delta\theta = -\eta_t v_{t+1} = -0{,}1 \times 4 = -0{,}4$. Hai giá trị $\Delta\theta = -0{,}4$ khớp nhau chính xác (tự tính), xác nhận bằng số quan hệ bao hàm Momentum $\subseteq$ Adam cho một bước cập nhật với ε lớn.

3.2 Vai trò của ε (Mục 2)

ε ban đầu chỉ để tránh chia cho 0, nhưng paper lưu ý nhiều cách diễn giải khác: xem Adam như xấp xỉ natural gradient descent đường chéo, ε đóng vai trò hệ số "damping" cải thiện điều kiện số của ma trận Fisher; hoặc xem ε như bán kính trust-region, kiểm soát nội suy giữa Momentum và natural gradient descent đường chéo. Dưới cách hiểu nào, giá trị ε tối ưu đều **phụ thuộc bài toán**. Thực tế, nhiều paper trước chỉ dùng ε mặc định (10^{-8} cho Adam, 10^{-10} cho RMSProp) hoặc không báo cáo giá trị ε dùng; một số ngoại lệ hiếm (Zaheer et al. 2018; De et al. 2018; Szegedy et al. 2016 dùng $\varepsilon = 1$ cho RMSProp; Liu et al. 2019 dùng $\varepsilon = 10^{-6}$; Tan et al. 2019 dùng $\varepsilon = 10^{-3}$ và đạt state-of-the-art ImageNet) đã thử giá trị khác mặc định và thường thấy cải thiện.

3.3 Protocol tuning (Mục 4)

Để tránh thiên vị, paper tune **độc lập** không gian siêu tham số cho mỗi optimizer (không dùng "cùng một không gian" cho mọi optimizer, vì siêu tham số cùng tên có thể cần giá trị khác nhau hoàn toàn giữa các optimizer, ví dụ Nesterov chịu được γ lớn hơn Momentum). Các siêu tham số $\eta_0, 1-\gamma, 1-\beta_1, 1-\beta_2$ được tune trên thang log. ε được **tách khỏi (decoupled)** learning rate ban đầu: tune trên thang log của $(\varepsilon, \alpha_0/\sqrt{\varepsilon})$ cho RMSProp và $(\varepsilon, \alpha_0/\varepsilon)$ cho Adam/NAdam, dựa trên quan sát của Savarese et al. (2019) rằng ε lớn thường cần learning rate lớn đi kèm.

Dùng **quasi-random uniform search** (Bousquet et al. 2017), chạy tới khi đủ K "trial khả thi" (không bị NaN/diverge); ước lượng trung bình và độ không chắc chắn bằng **bootstrap** (lấy $N > K$ trial, resample K trong số đó 100 lần, Phụ lục C). (K, N) dao động từ $(10, 50)$ (LSTM, tune chỉ learning rate) tới $(100, 500)$ (CNN/ResNet-32, tune đầy đủ).

Bảng 2: các workload thí nghiệm (Mục 4.1):

Tác vụ	Mô hình	Dữ liệu	Batch size	Ngân sách
Phân loại ảnh	Simple CNN	Fashion MNIST	256	10k bước
Phân loại ảnh	ResNet-32	CIFAR-10	256	50k bước
Phân loại ảnh	CNN (All-CNN-C)	CIFAR-100	256	350 epoch
Phân loại ảnh	VGG-16	CIFAR-10	128	250 epoch
Phân loại ảnh	ResNet-50	ImageNet	1024	150k bước
Mô hình ngôn ngữ	LSTM 2 lớp	War and Peace	50	200 epoch
Mô hình ngôn ngữ	Transformer	LM1B	256	750k bước

4. Thí nghiệm và kết quả chính

Hình 1-2 (kết quả chính, Mục 4.2): trên cả 6 optimizer (SGD, Momentum, Nesterov, RMSProp, Adam, NAdam) và các workload đại diện, quan hệ bao hàm **luôn đúng** trong khoảng sai số (error bar), bất kể dùng validation error, test error, hay training error làm thước đo; cũng đúng khi đo **tốc độ huấn luyện** (số bước để đạt ngưỡng validation error mục tiêu, Bảng 2 cột "Target error"). Cụ thể trên Transformer/LM1B (workload có khoảng cách rõ rệt nhất giữa optimizer): Adam chỉ cần **60%** số bước của Momentum, và **25%** số bước của SGD, để đạt cùng ngưỡng lỗi mục tiêu.

ResNet-50/ImageNet: NAdam đạt validation accuracy **77,1%**, vượt **76,5%** mà Goyal et al. (2017) đạt được bằng Momentum trên cùng kiến trúc, với $\varepsilon = 9475$ (lớn hơn giá trị mặc định 10^{-8} khoảng 10^{12} lần). Trên 4 workload chính, dải giá trị ε tối ưu trải **10 bậc độ lớn** (order of magnitude), xác nhận ε phụ thuộc bài toán mạnh.

Mức độ khác biệt giữa optimizer không đồng nhất giữa workload: với ResNet-32/CIFAR-10, mọi optimizer đạt hiệu năng và tốc độ tương tự nhau (không có sự khác biệt đáng kể); với Transformer/LM1B, có thứ hạng rõ rệt như đã nêu trên. Với ba optimizer "tổng quát nhất" không bao hàm lẫn nhau (RMSProp, Adam, NAdam), không có optimizer nào luôn thắng rõ rệt, dù Adam và NAdam thường có xu hướng nhích hơn RMSProp.

5. Phân tích / ablation

Tái lập và "lật ngược" kết quả của Wilson et al. 2017 (Hình 3, Mục 4.3): dùng chính code của Wilson et al. (2017) trên VGG/CIFAR-10: (a) grid search chỉ trên learning rate (đúng protocol gốc) tái lập được kết quả gốc: RMSProp và Adam có test error **tệ hơn** SGD/Momentum; (b) khi tune thêm γ và ε bằng random search, cả 4 optimizer đạt test error **gần như giống nhau**; (c) khi mở rộng thêm tune learning rate schedule,

và (d) tune đầy đủ thêm $\{\gamma, \beta_1, \beta_2, \varepsilon\}$ và weight decay, mọi optimizer vẫn đạt test error tương đương nhau trong khoảng sai số, **không còn dấu hiệu nào** cho thấy quan hệ bao hàm bị vi phạm.

Tái lập và "lật ngược" kết quả của Schneider et al. 2019/DeepOBS (Hình 4, Mục 4.3): trên CNN/CIFAR-100 và LSTM/War-and-Peace: (a) chỉ tune constant learning rate (đúng protocol DeepOBS) tái lập đúng thứ hạng của Schneider et al.: Adam **tệ hơn** Momentum trên CNN; SGD **nhích hơn** Adam và Momentum trên War-and-Peace; (b) khi tune thêm learning rate schedule và $\{\gamma, \beta_1, \beta_2, \varepsilon\}$, thứ hạng đổi thành Adam $>$ Momentum $>$ SGD, **đúng theo quan hệ bao hàm**.

Kết luận của Mục 4.3: cả hai công trình trước (Wilson et al. 2017, Schneider et al. 2019) đưa ra thứ hạng mâu thuẫn với quan hệ bao hàm vì họ **không tune đủ số siêu tham số**; khi đặt lại vào thuật ngữ của bài này (Adam với ε mặc định là một optimizer "khác" Adam với ε đã tune), mâu thuẫn biến mất, và rõ ràng các nghiên cứu trước chưa xét dạng "thú vị nhất" của Adam đối với người thực hành.

6. Giới hạn

Do tác giả tự nêu (Mục 4.4, Mục 5):

- **Không đo ảnh hưởng của batch size:** các công trình khác (Shallue et al. 2019; Zhang et al. 2019) cho thấy khoảng cách giữa SGD/Momentum và giữa Momentum/Adam **tăng** theo batch size; paper chỉ suy đoán (không chứng minh) rằng quan hệ bao hàm vẫn dự đoán đúng ở mọi batch size dưới protocol tương tự.
- **Kết quả phụ thuộc protocol và workload đã xét:** chỉ nên tổng quát hoá cho workload tương tự dưới protocol tương tự (quasi-random tuning hàng chục tới hàng trăm trial, không gian tìm kiếm dạng hypercube, cách tham số hoá learning rate schedule cụ thể của paper).
- **Không có xu hướng nhất quán** khi so sánh các optimizer **không** có quan hệ bao hàm lẫn nhau (RMSProp, Adam, NAdam).
- Một số workload **không có sự khác biệt có ý nghĩa thống kê** giữa các optimizer (ví dụ ResNet-32/CIFAR-10); bài học rút ra không áp dụng đều cho mọi workload.
- Các kỹ thuật biến đổi không gian tìm kiếm (tách ε khỏi learning rate) chỉ để tăng hiệu quả tìm kiếm, "về lý thuyết" không đổi kết quả nếu có ngân sách trial lớn hơn và tìm kiếm trên thang tuyến tính; đây là giả định chưa được kiểm chứng kỹ với ngân sách giới hạn.

Do chúng ta nhận thấy:

- Paper không thử các optimizer hiện đại hơn như AdamW (Loshchilov & Hutter 2017, có nhắc tên trong tài liệu tham khảo nhưng không thử nghiệm), LAMB, hay Lion; kết luận chỉ áp dụng cho 6 optimizer bậc một "cổ điển" trong Bảng 1.
- Chi phí tune "đầy đủ" (hàng trăm trial mỗi workload, nhiều workload tốn tới 150k-750k bước) rất cao; paper tự nhận đây gần với "ngân sách tune của chuyên gia", không phải lúc nào cũng khả thi, đặc biệt trên phần cứng hạn chế.

7. Liên hệ với các bài khác

- **opt 29** (Descending through a Crowded Valley, benchmark optimizer quy mô lớn hơn, nhiều workload và nhiều optimizer hơn): cùng chủ đề phương pháp luận (độ nhạy của ranking theo protocol tune), có thể xem là bước tiếp theo mở rộng quy mô benchmark sau phát hiện của bài này.
- **opt 30** (AlgoPerf/MLCommons, Benchmarking Neural Network Training Algorithms): là phản hồi trực tiếp (ra đời sau) cho đúng vấn đề mà bài này nêu ra, chuẩn hoá protocol tune (ví dụ giới hạn ngân sách tune theo "ngân sách cố định" hoặc "tự do") để các so sánh optimizer có thể lặp lại và công bằng hơn.
- **opt 31/32/33** (Hyperband, Bayesian optimization): đều là công cụ **chọn siêu tham số**; bài này dùng quasi-random search đơn giản hơn các phương pháp đó, nhưng bài học "tune chưa đủ sẽ cho kết luận

sai lệch" áp dụng trực tiếp cho người dùng bất kỳ công cụ HPO nào, bao gồm cả BayesOpt/Hyperband, nếu không gian tìm kiếm bị hạn chế quá mức.

- Đã kiểm tra bằng `ls` các thư mục cũ ("llm content/", "rl content/", "distillation content/", "pruning content/", "interp content/"): không thấy bài trực tiếp so sánh optimizer; một số bài trong "llm content/" (ví dụ về mô hình hoá ngôn ngữ nói chung) có thể gián tiếp liên quan tới việc chọn optimizer khi huấn luyện nhưng không thảo luận chủ đề này.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, chạy trên GPU <= 8GB, batch 1). Nhận định dưới đây là suy luận:

1. **Bài học về ε rất đáng áp dụng trực tiếp**: nếu dự án dùng AdamW (họ hàng Adam) để fine-tune/distill, paper cho thấy ε mặc định (10^{-8}) có thể cách xa giá trị tối ưu tới nhiều bậc độ lớn, đặc biệt với mô hình/kiến trúc khác với các benchmark chuẩn trong paper. Nên thử vài giá trị ε khác mặc định (ví dụ $10^{-6}, 10^{-4}$) khi gặp huấn luyện không ổn định hoặc hiệu năng kém hơn kỳ vọng, trước khi đổi hẳn optimizer.
2. **Cảnh báo về kết luận "optimizer A tốt hơn B" rút ra từ ít thí nghiệm**: với ngân sách GPU 8GB hạn chế, dự án khó có thể tune "đầy đủ" theo tiêu chuẩn của paper (hàng chục tới hàng trăm trial mỗi workload); do đó nếu so sánh 2 optimizer trên pipeline cắt/distill của dự án mà thấy một optimizer thắng, nên diễn giải cẩn trọng: khác biệt có thể do protocol tune chưa đủ sâu cho optimizer thua, chứ không chắc phản ánh bản chất thuật toán.
3. **Batch size 1 là một điểm đáng lưu ý riêng**: paper chỉ nói tới ảnh hưởng batch size tăng (gap optimizer rộng ra), không trực tiếp nói về batch size rất nhỏ (=1) như trong dự án; đây là suy luận ngoài phạm vi thực nghiệm của paper, cần tự kiểm chứng (xem Mục 9) vì chế độ gradient rất nhiễu ở batch 1 có thể thay đổi hoàn toàn động lực giữa SGD/Momentum/Adam.
4. **Tách ε khỏi learning rate khi tune** (tune $(\varepsilon, \alpha_0/\varepsilon)$ thay vì (ε, α_0) trực tiếp) là một kỹ thuật thực hành dễ áp dụng, không tốn thêm chi phí, chỉ cần đổi cách tham số hoá không gian tìm kiếm.

9. Thí nghiệm gợi ý

1. **Quét ε của AdamW trên một bước distill/fine-tune nhỏ** (5-10 giá trị log-scale từ 10^{-10} đến 10^{2}, giữ learning rate tối ưu tương ứng theo tỉ lệ α_0/ε), đo validation loss cuối. Kỳ vọng: tồn tại một vùng ε tốt hơn hẳn mặc định, giống phát hiện 10 bậc độ lớn của paper.
2. **Lặp lại thí nghiệm "lật ngược ranking" kiểu Hình 3/4 ở quy mô nhỏ**: so sánh SGD/Momentum/AdamW trên cùng một tác vụ nhỏ của dự án, lần đầu chỉ tune learning rate (protocol "nghèo"), lần hai tune thêm $\gamma, \beta_1, \beta_2, \varepsilon$ (protocol "giàu"). Kỳ vọng theo paper: ranking có thể đổi hoặc thứ hạng optimizer tổng quát hơn sẽ cải thiện tương đối khi protocol giàu hơn.
3. **Kiểm tra quan hệ bao hàm bằng số ở batch size 1** (tình huống thực tế của dự án): chạy Momentum với γ cố định, và AdamW với ε rất lớn (mô phỏng xấp xỉ Momentum theo Mục 3.1), so sánh quỹ đạo loss của 2 lần chạy trên vài trăm bước. Đo độ lệch giữa hai quỹ đạo; kỳ vọng độ lệch nhỏ nếu ε đủ lớn, xác nhận quan hệ bao hàm vẫn giữ ở chế độ batch rất nhỏ.
4. **Đo "số bước để đạt ngưỡng" (steps-to-target) thay vì chỉ loss cuối** cho 2-3 optimizer trên tác vụ của dự án, theo đúng cách đo ở Hình 2 của paper. Đây là thước đo thực tế hơn cho ràng buộc thời gian/GPU-giờ hạn chế của dự án so với chỉ so sánh loss cuối cùng ở ngân sách cố định.
5. **So sánh chi phí tune "nghèo" và "giàu"**: đo tổng GPU-giờ cần để tune đủ tốt AdamW (nhiều siêu tham số) so với Momentum (ít siêu tham số hơn) trên cùng tác vụ, để ước lượng liệu lợi ích tiềm năng của optimizer tổng quát hơn có đáng chi phí tune thêm trong điều kiện GPU 8GB hạn chế của dự án hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Update rule tổng quát	$\theta_{t+1} = M(H_t, \varphi_t)$
Quan hệ bao hàm	$M \subseteq N$ nếu $\exists\{\psi^i\} : \lim_{i\to\infty} N(H_t, \psi_t^i) = M(H_t, \varphi_t)\ \forall t, H_t$
Chuỗi bao hàm chính	SGD $\subseteq$ Momentum $\subseteq$ RMSProp; SGD $\subseteq$ Momentum $\subseteq$ Adam; SGD $\subseteq$ Nesterov $\subseteq$ NAdam
Momentum $\subseteq$ Adam (giới hạn $\varepsilon \to \infty$)	$\alpha_t = \varepsilon\eta_t(1-\gamma^t)$, $\beta_1 = \gamma$, $\beta_2 = 0$
Decoupling ε khi tune	RMSProp: tune $(\varepsilon, \alpha_0/\sqrt{\varepsilon})$; Adam/NAdam: tune $(\varepsilon, \alpha_0/\varepsilon)$
Kết quả chính (Transformer/LM1B)	Adam cần 60% số bước của Momentum, 25% số bước của SGD để đạt cùng ngưỡng lỗi
Kết quả chính (ResNet-50/ImageNet)	NAdam với $\varepsilon = 9475$ đạt 77,1% val-accuracy, vượt Momentum (76,5%, Goyal et al. 2017)
Dải ε tối ưu quan sát được	trải khoảng 10 bậc độ lớn giữa các workload

11. Kết luận ngắn

Paper chỉ ra rằng các so sánh thực nghiệm phổ biến giữa optimizer deep learning (kết luận Adam/RMSProp thua SGD/Momentum) thường là hệ quả của **protocol tune chưa đủ sâu**, không phải bản chất thuật toán: khi tune đầy đủ mọi siêu tham số (bao gồm ε, thường bị bỏ quên hoặc cố định ở giá trị mặc định), optimizer tổng quát hơn theo quan hệ bao hàm **không bao giờ** thua kém trường hợp đặc biệt của nó, đúng như lý thuyết dự đoán. Bài học thực hành quan trọng nhất: luôn báo cáo rõ không gian tìm kiếm siêu tham số đã dùng khi so sánh optimizer, đặc biệt là có tune ε hay không, và nên nghi ngờ các so sánh nơi một optimizer tổng quát bị báo cáo thua một trường hợp đặc biệt của chính nó.

Nguồn gốc: https://arxiv.org/abs/1910.05446

Bài 29

Descending through a Crowded Valley: Benchmarking Deep Learning Optimizers (2020)

Tác giả: Robin M. Schmidt, Frank Schneider, Philipp Hennig (University of Tübingen; Max Planck Institute for Intelligent Systems).
Venue / arXiv: Proceedings of the 38th International Conference on Machine Learning (ICML 2021), PMLR 139; arXiv:2007.01547 (v6, 10/08/2021).
Nguồn: https://arxiv.org/abs/2007.01547
Chủ đề: benchmark thực nghiệm quy mô lớn (hơn 50.000 lượt chạy huấn luyện) so sánh 15 optimizer deep learning phổ biến trên 8 bài toán, 4 ngân sách tune, và 4 learning-rate schedule.
Vị trí trong lộ trình: đọc nối tiếp opt 28 (Choi et al. 2019, On Empirical Comparisons of Optimizers), mà bài này trực tiếp trích dẫn và mở rộng quy mô; cũng dọn đường cho opt 30 (AlgoPerf/MLCommons), ra đời sau để chuẩn hoá chính xác vấn đề phương pháp luận mà bài này nêu ra ở phần kết luận (”có đủ optimizer rồi, cần benchmark có ý nghĩa hơn”).

Lưu ý phạm vi đọc (bài khảo sát/benchmark dài): đã đọc đầy đủ toàn bộ phần chính văn (Tóm tắt, Mục 1-5, Tài liệu tham khảo) của bài báo. Phần phụ lục (Appendix A đến G: Bảng 2 danh sách hơn 100 optimizer, Bảng 3 danh sách hơn 20 họ learning-rate schedule, Bảng 4 không gian tìm kiếm/giá trị mặc định cho 15 optimizer, và các hình/bảng chi tiết bổ sung theo từng bài toán) **chỉ được xem lướt một phần** (trang đầu của Bảng 2), không được đọc chi tiết toàn bộ do độ dài rất lớn của phụ lục so với phạm vi 8 bài cần hoàn thành. Các số liệu trích dẫn trong ghi chú này đều lấy từ phần chính văn, không suy ra từ phụ lục chưa đọc.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Chọn optimizer được xem là một trong những quyết định thiết kế quan trọng nhất của deep learning, nhưng không dễ: tài liệu liệt kê **hàng trăm** phương pháp tối ưu (Bảng 2 trong phụ lục của paper, hơn 100 tên). Thiếu lý thuyết rõ ràng và bằng chứng thực nghiệm thuyết phục, lựa chọn thường dựa trên ”giai thoại” (anecdote). Các so sánh thực nghiệm trước đó có quy mô nhỏ hơn: Wilson et al. (2017) chỉ tune một phần siêu tham số; Choi et al. (2019, opt 28 trong thư mục này) chỉ ra rằng không gian tìm kiếm siêu tham số có thể là yếu tố quan trọng nhất quyết định thứ hạng optimizer, và quan hệ bao hàm giữa optimizer dự đoán đúng kết quả khi tune đủ; Sivaprasad et al. (2020) cho thấy thứ hạng optimizer phụ thuộc mạnh vào ngân sách tune. Mỗi nghiên cứu này chỉ xét một tập nhỏ optimizer trên một tập nhỏ bài toán.

Bài này đặt mục tiêu làm một benchmark **quy mô lớn hơn hẳn**: 15 optimizer phổ biến, 8 bài toán deep learning đa dạng (từ DeepOBS, Schneider et al. 2019), 4 ngân sách tune, 4 learning-rate schedule, tổng hợp từ **hơn 50.000 lượt chạy huấn luyện** và **53.760 đường cong huấn luyện** công khai, nhằm thay ”giai thoại” bằng ”heuristic có bằng chứng thực nghiệm.”

2. Ý tưởng chính

Ba đóng góp chính (Tóm tắt, Mục 1):

1. **Hiệu năng optimizer thay đổi rất mạnh theo bài toán** (gợi nhớ tới ”No Free Lunch Theorem” của Wolpert & Macready 1997, dù paper không chứng minh chặt điều này mà chỉ quan sát thực nghiệm).
2. **Đánh giá nhiều optimizer với siêu tham số mặc định (one-shot) cho hiệu năng xấp xỉ tương đương với việc tune kỹ một optimizer cố định.** Đây là phát hiện thực hành quan trọng nhất: một người

thực hành với ngân sách tính toán cố định có thể đạt kết quả tương đương bằng cách (a) chạy thử vài optimizer với tham số mặc định và chọn cái tốt nhất, hoặc (b) đầu tư toàn bộ ngân sách đó để tune kỹ một optimizer duy nhất.

3. **Không có optimizer nào thống trị rõ rệt** trên mọi bài toán đã thử, nhưng có một tập con rút gọn các optimizer/lựa chọn tham số thường cho kết quả cạnh tranh: Adam vẫn là một lựa chọn mạnh, các phương pháp mới hơn (AdaBound, AdaBelief, RAdam...) không vượt trội nhất quán và có ý nghĩa thống kê so với Adam.

3. Phương pháp chi tiết

3.1 Không gian tổ hợp thí nghiệm (Mục 2)

Benchmark tổ hợp: **8 bài toán × 15 optimizer × 4 ngân sách tune × 4 learning-rate schedule**. Kiểm tra bằng số (tự tính): $15 \times 8 \times 4 \times 4 = 1920$ cấu hình (khớp đúng con số paper nêu ở Mục 2), mỗi optimizer được đánh giá trong $8 \times 4 \times 4 = 128$ thiết lập.

8 bài toán (Bảng 1, lấy từ DeepOBS ”small” P1-P4 và ”large” P5-P8):

Mã	Dữ liệu	Mô hình	Nhiệm vụ	Thước đo	Batch	Ngân sách (epoch)
P1	Nhân tạo	Noisy quadratic	Minimization	Loss	128	100
P2	MNIST	VAE	Generative	Loss	64	50
P3	Fashion-MNIST	CNN đơn giản (2c2d)	Phân loại	Accuracy	128	100
P4	CIFAR-10	CNN đơn giản (3c3d)	Phân loại	Accuracy	128	100
P5	Fashion-MNIST	VAE	Generative	Loss	64	100
P6	CIFAR-100	All-CNN-C	Phân loại	Accuracy	256	350
P7	SVHN	Wide ResNet 16-4	Phân loại	Accuracy	128	160
P8	War and Peace	RNN (char-level)	Dự đoán ký tự	Accuracy	50	200

15 optimizer (chọn từ hơn 100 trong Bảng 2 phụ lục, theo mức độ phổ biến ước lượng qua số lần được nhắc tới trên arXiv, Hình 1): AMSBound, AMSGrad, AdaBelief, AdaBound, Adadelta, Adagrad, Adam, LookAhead(Momentum), LookAhead(RAdam), Momentum, NAG (Nesterov), NAdam, RAdam, RMSProp, SGD. Các optimizer bị loại khỏi danh sách 15 chỉ chiếm **dưới 4%** số lần được nhắc tới trên arXiv năm 2020 (Hình 1 trong paper).

3.2 Ngân sách tune (Mục 2.3)

Bốn mức ngân sách: **one-shot** (1 lần chạy, dùng giá trị mặc định do tác giả gốc đề xuất); **small** (25 lần chạy, random search); **medium** (50 lần); **large** (75 lần, trong đó dùng lại phân phối của medium nhưng **thu hẹp** không gian tìm kiếm chỉ còn vùng chứa 20% cấu hình tốt nhất của medium). Dùng **random search** (không phải Bayesian optimization) vì đơn giản, dễ song song hoá, và để tránh thêm các quyết định thiết kế (kernel, acquisition function...) làm phức tạp việc phân tích. Mỗi cấu hình tốt nhất (theo validation) được chạy lại với **10 seed ngẫu nhiên khác nhau** để báo cáo trung bình và độ lệch chuẩn, nhằm đánh giá độ ổn định; các ”lượt chạy hỏng” (failing seeds) chiếm dưới 0,5% tổng số lượt tune.

3.3 Learning-rate schedule (Mục 2.4)

Bốn schedule được áp dụng (chỉ cho learning rate, không cho các siêu tham số khác): hằng số (constant); cosine decay (Loshchilov & Hutter 2017); cosine với warm restarts (SGDR, cũng Loshchilov & Hutter 2017); hình thang (trapezoidal, có warm-up, theo Goyal et al. 2017 và Xing et al. 2018).

3.4 Vấn đề định nghĩa "optimizer" và ε (Mục 2.3)

Paper kế thừa trực tiếp cách nhìn của Choi et al. (2019, opt 28): biên giới giữa "siêu tham số có thể tune" và "tham số thiết kế cố định" là mờ. ε của Adam, ban đầu chỉ để tránh chia 0, nay được xem là siêu tham số phụ thuộc bài toán. Theo logic đó, "Adam$_\alpha$" (chỉ tune learning rate) và "Adam$_{\alpha,\beta_1,\beta_2,\varepsilon}$" (tune đầy đủ) được xem là hai optimizer khác nhau. Paper chọn **một** lựa chọn phổ biến các siêu tham số có thể tune cho mỗi update rule (Bảng 4 phụ lục), không thử tất cả biến thể mức độ tune như bài opt 28 đã làm.

4. Thí nghiệm và kết quả chính

Ma trận one-shot vs. tuned (Hình 2, Mục 3): với 5 bài toán minh hoạ, paper dựng ma trận 15×15 so sánh hiệu năng one-shot (mặc định, trục y) của mỗi optimizer với hiệu năng đã tune (ngân sách small, trục x) của mọi optimizer khác. Ví dụ cụ thể paper nêu: AMSBound đã tune (ngân sách small) đạt test accuracy cao hơn SGD mặc định **2,4%** trên một bài toán cụ thể. Phát hiện chính từ ma trận này:

- Siêu tham số mặc định **hoạt động kém** (dễ bị bất kỳ optimizer đã tune nào vượt qua) cho Momentum, NAG, SGD, và cả AMSGrad, Adadelta.
- Siêu tham số mặc định **hoạt động tốt** (ngay cả optimizer đã tune cũng không vượt rõ) cho Adam, NAdam, RAdam, cũng như AMSBound, AdaBound, AdaBelief, trên một số (không phải tất cả) bài toán, ủng hộ "quy tắc ngón tay cái": các phương pháp thích ứng (adaptive) có xu hướng có giá trị mặc định hoạt động tốt hơn các phương pháp không thích ứng.
- **Không có cột** nào nhất quán trên mọi bài toán (nghĩa là không có optimizer nào, khi tune, luôn vượt tất cả optimizer mặc định khác trên mọi bài toán), ủng hộ kết luận hiệu năng optimizer phụ thuộc mạnh vào bài toán.

Tác dụng của tune và schedule (Hình 3, Mục 3): không có xu hướng rõ cho từng thiết lập đơn lẻ (đường màu xám, biến động mạnh, có thể còn **giảm** hiệu năng), nhưng **trung vị** cho thấy xu hướng cải thiện: ngân sách medium không schedule cho cải thiện trung vị khoảng **3,4%** so với tham số mặc định không schedule; ngân sách large kết hợp schedule hình thang cho cải thiện trung vị khoảng **5,2%**. Lợi ích giảm dần (diminishing returns) khi tăng ngân sách.

So sánh optimizer sau khi tune (Hình 4, Mục 3): trên 8 bài toán với ngân sách large, không schedule, **không có optimizer thống trị** mọi bài toán. Một số nhận xét cụ thể: NAG thường vượt Adam trên các bài toán phân loại ảnh, **trừ** hai bài toán VAE (P2, P5); RMSProp **nhất quán đứng đầu** cho bài toán mô hình ngôn ngữ ký tự (P8, War and Peace RNN). Lấy tốt nhất trong một tập nhỏ optimizer mặc định (ví dụ Adam và AdaBound) thường cho kết quả cạnh tranh, ngoại trừ 2 bài toán VAE.

5. Phân tích / ablation

- **Thứ hạng đổi theo ngân sách/schedule** (được paper tự nêu, dẫn tới Hình 14-16 phụ lục chưa đọc chi tiết trong ghi chú này), nhưng paper khẳng định "các xu hướng tổng thể vẫn nhất quán" dù thứ hạng chi tiết dao động.
- **Độ ổn định theo seed** là một thông số bị bỏ quên trong các benchmark khác nhưng được paper đo cẩn thận: optimizer "quá bất ổn" (cần nhiều seed để tune mới ổn định) sẽ bị điểm số tự nhiên trừng phạt, vì paper xem đây là "tính năng" phản ánh thực tế sử dụng, không phải lỗi của phương pháp đo.
- **Random search so với Bayesian optimization (tự nhận trong Mục 4):** paper chọn random search vì "các phương pháp phức tạp hơn, đặc biệt Bayesian optimization, sẽ nhân thêm số quyết định thiết kế

(kernel, acquisition function, prior...) và làm phức tạp đáng kể việc phân tích." Đây là một đánh đổi có ý thức giữa độ chính xác phân tích và khả năng diễn giải, khác với cách opt 32/33 (BayesOpt) hay opt 31 (Hyperband) tiếp cận bài toán tune.

- **Lựa chọn optimizer tương đương với lựa chọn schedule (giả thuyết của paper, Mục 3):** paper suy đoán rằng vì các optimizer có "implicit learning rate schedule" riêng (trích Agarwal et al. 2020), thử nhiều optimizer mặc định về bản chất tương tự thử nhiều schedule đã được kiểm định tốt, giải thích vì sao phát hiện 2 ("multiple default optimizers ~ tune 1 optimizer") lại đúng.

6. Giới hạn

Do tác giả tự nêu (Mục 4):

- **Khả năng tổng quát hoá hạn chế theo loại bài toán:** kết quả phản ánh chính xác các bài toán DeepOBS (CNN/VAE/RNN quy mô vừa-nhỏ), framework phần mềm (TensorFlow 1.15, DeepOBS v1.2.0-beta), và các tham số huấn luyện cố định đã chọn (ví dụ batch size, hệ số L2 regularization **không** được xem là siêu tham số tune, mà cố định theo gợi ý DeepOBS cho mỗi bài toán). Kết luận **không chắc tổng quát hoá** sang GAN, reinforcement learning, bài toán nhạy cảm bộ nhớ, hay quy mô lớn hơn (ImageNet, Transformer) mà paper **không** đưa vào benchmark.
- **Không bao gồm kỹ thuật tối ưu hoá khác** (weight averaging, ensemble methods) vì chúng có thể kết hợp với mọi optimizer, sẽ làm tăng chi phí tính toán của benchmark vốn đã rất lớn.
- **Ảnh hưởng của chiến lược tune:** như Choi et al. (2019, opt 28) và Sivaprasad et al. (2020) đã chỉ ra, phương pháp tune, ngân sách, và miền tìm kiếm ảnh hưởng mạnh tới kết quả; paper cố gắng định lượng bằng cách báo cáo 4 ngân sách khác nhau, nhưng tự nhận một miền tìm kiếm được chọn kém có thể khiến việc tune gần như bất khả thi, và việc tìm miền tìm kiếm phù hợp vẫn là một "khó khăn ngầm" mà bất kỳ người thực hành nào cũng gặp phải, không riêng gì benchmark này.
- Paper **tự nhận rõ** đây không phải "công trình định nghĩa cuối cùng" (the definitive work) về benchmark optimizer, mà là "một bước quan trọng đúng hướng."

Do chúng ta nhận thấy:

- Phụ lục (Bảng 2 danh sách optimizer, Bảng 3 danh sách schedule, Bảng 4 không gian tìm kiếm/giá trị mặc định, Hình 5-17 và các bảng 6-9 chi tiết theo bài toán) **không được đọc chi tiết** trong lượt làm ghi chú này (xem khai báo phạm vi đọc ở đầu file); các số liệu cụ thể trích ở trên đều lấy trực tiếp từ chính văn, không suy diễn thêm từ phụ lục chưa đọc.
- Một điểm đáng chú ý paper tự thừa nhận ở phần kết luận nhưng không nhấn mạnh đủ ở phần kết quả: "các optimizer khác nhau có phân phối hiệu năng khá giống nhau so với một **phương pháp duy nhất được tune lại hoặc chạy lại với seed khác**": nghĩa là biến động giữa các lần chạy lại của **cùng một** optimizer có thể lớn tới mức so sánh giữa các optimizer khác nhau trở nên khó phân biệt về ý nghĩa thống kê; đây là một cảnh báo phương pháp luận quan trọng cho bất kỳ ai muốn rút ra kết luận "optimizer X tốt hơn Y" từ một vài lần chạy.

7. Liên hệ với các bài khác

- **opt 28** (On Empirical Comparisons of Optimizers, Choi et al. 2019): bài này trích dẫn trực tiếp và mở rộng quy mô của opt 28 (15 optimizer, 8 bài toán, 4 ngân sách, 4 schedule so với quy mô nhỏ hơn của opt 28); đồng thời **không hoàn toàn đồng thuận**: opt 28 nhấn mạnh quan hệ bao hàm (inclusion) dự đoán đúng thứ hạng khi tune đủ kỹ, còn bài này (dùng một miền tìm kiếm thống nhất, không thiết kế riêng để kiểm tra bao hàm) quan sát hiện tượng rộng hơn: hiệu năng phụ thuộc bài toán mạnh, và "không có optimizer thắng tuyệt đối," một phát hiện bổ sung chứ không mâu thuẫn với opt 28.
- **opt 30** (AlgoPerf/MLCommons, Benchmarking Neural Network Training Algorithms): là lời đáp trực tiếp cho lời kêu gọi ở phần kết luận của bài này ("có đủ optimizer nghiên cứu rồi, cần tập trung vào

benchmark thật sự có ý nghĩa, không tăng dần nhỏ lẻ"); AlgoPerf chuẩn hoá protocol benchmark (ngân sách thời gian cố định, nhiều workload thực tế hơn DeepOBS) theo đúng hướng paper này đề xuất.

- **opt 31/32/33** (Hyperband, Bayesian optimization): paper này **chủ động từ chối** dùng Bayesian optimization cho việc tune (lý do: quá nhiều quyết định thiết kế bổ sung làm phức tạp phân tích), chọn random search đơn giản hơn; đây là một lựa chọn phương pháp luận trái ngược có chủ đích với cách opt 32/33 tiếp cận HPO, một điểm đối chiếu thú vị khi đọc song song hai nhóm bài.
- Đã kiểm tra bằng `ls` các thư mục cũ ("llm content/", "rl content/", "distillation content/", "pruning content/", "interp content/"): không thấy bài trực tiếp benchmark optimizer theo kiểu này.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, chạy trên GPU <= 8GB, batch 1). Nhận định dưới đây là suy luận:

1. **Phát hiện "nhiều optimizer mặc định ~ tune 1 optimizer" rất phù hợp với ràng buộc GPU 8GB của dự án**: thay vì đầu tư toàn bộ ngân sách tính toán hạn chế để tune kỹ một optimizer (ví dụ AdamW) cho bước distill/fine-tune, có thể thử nhanh 3-4 optimizer với tham số mặc định hợp lý (AdamW, NAdam, RAdam, SGD-Momentum với $\gamma = 0{,}9$) và chọn cái tốt nhất, theo đúng tinh thần phát hiện 2 của paper, với chi phí tương đương.
2. **"Adam vẫn là lựa chọn mạnh, không ai vượt trội rõ rệt"**: đây là lý do hợp lý để **không** tốn công thử nghiệm nhiều optimizer "mới" (AdaBelief, AdaBound...) cho dự án trừ khi có lý do cụ thể, và tập trung ngân sách tune vào các siêu tham số (learning rate, weight decay, ε như opt 28 chỉ ra) của AdamW/NAdam.
3. **Lợi ích của warm-up/trapezoidal schedule (~5,2% cải thiện trung vị)** là một "chiến thắng rẻ": thêm warm-up vào lịch learning rate cho bước distill hầu như không tốn thêm chi phí tính toán nhưng có thể cải thiện hiệu năng, nên áp dụng mặc định.
4. **Cảnh báo quan trọng nhất cho dự án**: do batch size của dự án là 1 (rất khác 8 bài toán DeepOBS, batch 50-256) và mô hình là VLM OCR với kiến trúc hybrid GatedDeltaNet (khác hẳn CNN/VAE/RNN/Wide ResNet trong DeepOBS), kết luận "hiệu năng phụ thuộc mạnh vào bài toán" của paper này cảnh báo rằng **không nên giả định** phát hiện "Adam mặc định tốt" hay "NAG thường vượt Adam" sẽ đúng cho bài toán của dự án; cần tự kiểm chứng bằng thí nghiệm nhỏ (xem Mục 9).
5. **Cảnh báo về biến động do seed**: như đã nêu ở Mục 6, hai lần chạy khác seed của cùng một optimizer có thể khác nhau đáng kể; khi so sánh optimizer trên dự án với ngân sách hạn chế (có thể chỉ chạy 1 seed mỗi cấu hình), cần cẩn trọng không rút ra kết luận "optimizer X tốt hơn Y" quá chắc chắn từ một lần chạy duy nhất.

9. Thí nghiệm gợi ý

1. **Tái hiện "ma trận one-shot vs. tuned" ở quy mô nhỏ**: trên một bước distill/fine-tune nhỏ của dự án, so sánh 3-4 optimizer (AdamW, NAdam, SGD-Momentum, RMSProp) với tham số mặc định so với mỗi optimizer đã tune nhẹ (10-20 lần random search). Đo xem default có "đủ tốt" như phát hiện của paper hay không trên chính bài toán OCR của dự án.
2. **So sánh schedule**: với một optimizer cố định (AdamW mặc định), thử 4 schedule (constant, cosine, cosine warm restarts, trapezoidal/warm-up) trên cùng một tác vụ distill nhỏ, đo validation loss cuối. Kỳ vọng theo paper: trapezoidal/warm-up cho cải thiện khiêm tốn nhưng nhất quán.
3. **Đo độ ổn định theo seed** cho cấu hình tốt nhất tìm được: chạy lại 3-5 seed khác nhau, tính độ lệch chuẩn của validation loss/accuracy cuối. So sánh độ lệch này với khoảng cách hiệu năng giữa các optimizer khác nhau đo ở thí nghiệm 1, để đánh giá liệu khác biệt giữa optimizer có "vượt qua nhiễu do seed" hay không, đúng cảnh báo phương pháp luận ở Mục 6.

4. **Kiểm tra ”quy tắc ngón tay cái” thích ứng=mặc định tốt** trên bài toán cụ thể của dự án: so sánh default AdamW/NAdam (thích ứng) với default SGD-Momentum/NAG (không thích ứng) không tune gì cả. Kỳ vọng theo paper: optimizer thích ứng có default tốt hơn optimizer không thích ứng.
5. **Đo lợi ích giảm dần theo ngân sách tune**: chạy one-shot, rồi 10, 25, 50 lần random search cho AdamW trên bài toán distill, vẽ đường cải thiện theo số lần thử. Kỳ vọng theo Hình 3 của paper: cải thiện nhanh ban đầu rồi chững lại (diminishing returns), giúp xác định ngân sách tune ”đủ” cho dự án mà không lãng phí GPU-giờ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Không gian tổ hợp thí nghiệm	15 optimizer × 8 bài toán × 4 ngân sách × 4 schedule $= 1920$ cấu hình (tự tính, khớp paper)
Ngân sách tune	one-shot (1), small (25), medium (50), large (75, thu hẹp từ top 20% medium)
Schedule	constant, cosine decay, cosine warm restarts (SGDR), trapezoidal (warm-up)
Cải thiện trung vị (medium, không schedule)	khoảng 3,4% so với mặc định
Cải thiện trung vị (large + trapezoidal)	khoảng 5,2% so với mặc định
Ví dụ Hình 2	AMSBound tuned (small) vượt SGD mặc định 2,4% trên 1 bài toán cụ thể
Phát hiện chính	nhiều optimizer mặc định, chọn tốt nhất $\approx$ tune kỹ 1 optimizer cố định
Tỉ lệ seed hỏng	dưới 0,5% tổng số lượt tune
Tổng quy mô benchmark	hơn 50.000 lượt chạy, 53.760 đường cong huấn luyện công khai

11. Kết luận ngắn

Qua benchmark quy mô lớn 15 optimizer trên 8 bài toán, 4 ngân sách và 4 schedule, paper không tìm ra một optimizer thống trị tuyệt đối, nhưng khẳng định Adam (và các biến thể gần nó như NAdam, RAdam) vẫn là lựa chọn an toàn, trong khi các phương pháp mới hơn không vượt trội nhất quán về ý nghĩa thống kê. Phát hiện thực hành quan trọng nhất là thử vài optimizer với tham số mặc định và chọn cái tốt nhất cho hiệu năng xấp xỉ tương đương với việc đầu tư toàn bộ ngân sách để tune kỹ một optimizer duy nhất, một chiến lược rẻ và hấp dẫn cho người thực hành có ngân sách tính toán hạn chế. Lời kêu gọi cuối bài (”có đủ optimizer rồi, cần nghiên cứu benchmark và tự động tune có ý nghĩa hơn thay vì phát minh thêm optimizer gia tăng nhỏ”) là động lực trực tiếp cho các benchmark chuẩn hoá ra đời sau như AlgoPerf (opt 30).

Nguồn gốc: https://arxiv.org/abs/2007.01547

Bài 30

Benchmarking Neural Network Training Algorithms (2023)

Tác giả: George E. Dahl, Frank Schneider, Zachary Nado, Naman Agarwal và 21 đồng tác giả khác (tổng 25 tác giả) thuộc MLCommons Algorithms Working Group (Google, University of Tübingen, Vector Institute, Dalhousie University, University of Toronto, Stanford University, Meta AI/FAIR, Dell Technologies).
Venue / arXiv: arXiv:2306.07179 (v2, 18/06/2025); trong toàn bộ phần chính văn đã đọc, paper không nêu rõ venue hội nghị/tạp chí nào, chỉ ghi ”AlgoPerf Authors” ở header lặp lại mỗi trang. Đây là một báo cáo kỹ thuật rất dài của MLCommons Algorithms Working Group, đồng thời đóng vai trò tài liệu đặc tả luật chơi cho benchmark AlgoPerf.
Nguồn: https://arxiv.org/abs/2306.07179
Chủ đề: benchmark cạnh tranh (competitive benchmark) đo tốc độ huấn luyện (time-to-result) cho ”training algorithm” nói chung (optimizer + tuning protocol + data selection), trên nhiều workload, với hardware cố định; đồng thời là một bài phê bình phương pháp luận so sánh optimizer.
Vị trí trong lộ trình: đọc sau opt 28 (Choi et al. 2019, ”On Empirical Comparisons of Optimizers”) và opt 29 (Schmidt et al. 2021, ”Descending through a Crowded Valley”) vì bài này tự nhận là phản hồi/mở rộng trực tiếp hai bài đó thành một benchmark chính thức có luật chơi, hạ tầng code, và điểm số chuẩn hoá; cũng liên hệ opt 31-33 (Hyperband, Bayesian Optimization) vì AlgoPerf chuẩn hoá chính ”tuning protocol” mà các phương pháp HPO đó thực hiện.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Đây là báo cáo kỹ thuật rất dài (chính văn 9 mục + Author Contributions + 5 phụ lục A-E + reference list); phạm vi đọc được nêu rõ ở cuối Mục 1.

1. Bối cảnh

Deep learning có hàng trăm ”training algorithm” (optimizer, update rule, lịch learning rate, cách chọn mẫu dữ liệu...) được đề xuất mỗi năm. Paper dẫn lại số liệu của chính opt 29 (Schmidt et al. 2021): hơn một trăm phương pháp, phần lớn công bố trong bảy năm gần nhất. Mỗi paper đề xuất đều tuyên bố phương pháp của mình tốt hơn, nhưng phần lớn cộng đồng deep learning không dùng bất kỳ kỹ thuật ”mới” nào trong số đó. Paper gọi đây là nghịch lý ”con gà và quả trứng”: một phương pháp bị coi là chưa hữu ích cho tới khi được dùng rộng rãi, nhưng nó không được dùng rộng rãi vì chưa được chứng minh hữu ích theo một chuẩn mà cộng đồng tin tưởng.

Nguyên nhân sâu xa được paper chỉ ra: paper đề xuất một optimizer mới thường cũng là nguồn bằng chứng thực nghiệm duy nhất cho optimizer đó, dẫn tới xung đột lợi ích khi chính nhóm đề xuất kiểm soát luôn baseline để so sánh. Các nghiên cứu so sánh thực nghiệm độc lập (như opt 28, opt 29) tuy hữu ích nhưng chưa giải quyết dứt điểm vấn đề, vì hai lý do: (1) chúng luôn có thể bị phản bác về chi tiết tuning (không có sự đồng thuận rộng về cách tune ”công bằng”), và (2) chúng có tính hồi cứu (retrospective), nên gánh nặng chứng minh không còn đặt lên vai người đề xuất phương pháp mới như trong một cuộc thi trực tiếp.

Giải pháp được đề xuất: học theo mô hình ”competitive benchmark” đã từng giúp deep learning được cộng đồng computer vision chấp nhận (ví dụ cuộc thi ImageNet, Russakovsky et al. 2015) thay vì chỉ thuyết phục bằng paper. MLCommons Algorithms Working Group ra đời với sứ mệnh xây dựng benchmark đo tốc độ huấn luyện (training speedup) do cải tiến thuật toán mang lại, và bài này là lần công bố đầu tiên của benchmark đó: **AlgoPerf: Training Algorithms**.

Phạm vi đọc đã thực hiện: người ghi chú đã đọc đầy đủ toàn bộ chính văn (Mục 1 đến Mục 9, từ Introduction tới Conclusion) và phần Author Contributions/Acknowledgments, cùng câu mở đầu của Phụ lục A (mô tả Hình

7 về hai loại learning-rate schedule). KHÔNG đọc chi tiết: phần còn lại Phụ lục A (chi tiết thí nghiệm của Mục 2), Phụ lục B (chi tiết target-setting), Phụ lục C (chi tiết baseline), Phụ lục D (đặc tả đầy đủ 8 workload: kiến trúc model, siêu tham số, biến thể workload cho Criteo, fastMRI, ImageNet, LibriSpeech, OGBG, WMT), Phụ lục E (thí nghiệm sơ bộ cho randomized workload), và danh mục References. Các phụ lục này ước tính dài hơn 1800 dòng text thô (so với khoảng 3470 dòng của chính văn), và theo mô tả trong chính văn chủ yếu là bảng số liệu/đặc tả kỹ thuật bổ sung cho các bảng đã trình bày trong chính văn (Bảng 7-15), không phải luận điểm phương pháp mới.

2. Ý tưởng chính

Paper xác định ba thử thách căn bản khiến so sánh thực nghiệm giữa training algorithm trở nên khó tin cậy, và đề xuất cách AlgoPerf giải quyết từng thử thách:

1. **Định nghĩa và đo "tốc độ huấn luyện" một cách chính xác.** Đường cong training/validation loss theo thời gian thường cắt nhau nhiều lần, nên so sánh trực tiếp hai đường cong là bài toán "ill-posed". AlgoPerf giải quyết bằng **time-to-result**: đặt một target error rate (trên validation/test set) cho mỗi workload, và tính thời gian wall-clock cần để lần đầu đạt target đó, trên hardware chuẩn hoá.
2. **Sự phụ thuộc vào workload.** Những thay đổi tưởng chừng nhỏ (kiến trúc, cách khởi tạo, vị trí layer normalization...) có thể đảo ngược hoàn toàn thứ hạng giữa các optimizer. AlgoPerf xử lý bằng cách cố định một tập workload đa dạng (8 fixed workload) và bổ sung **randomized/held-out workload**: các biến thể nhỏ của workload gốc, được lấy mẫu chỉ sau khi mọi submission đã "đóng băng", để phát hiện submission "giòn" (brittle) chỉ ăn gian trên đúng 8 workload cố định.
3. **Không thể so sánh "họ" thuật toán, chỉ so sánh các thể hiện cụ thể có tuning.** Vì mọi training algorithm thực chất là một "template" có siêu tham số tự do, AlgoPerf buộc phải xem **tuning protocol (bao gồm search space) là một phần của định nghĩa thuật toán**. Benchmark cung cấp hai luật chơi (ruleset) độc lập: *external tuning* (20 trial tìm siêu tham số, lặp 5 lần) và *self-tuning* (không có tuning ngoài, thuật toán phải tự thích nghi "trên đồng hồ", đổi lại được gấp 3 lần runtime budget).

Bốn đóng góp chính mà paper tự liệt kê: (a) phân tích và minh hoạ bằng thực nghiệm cụ thể ba thử thách trên (Mục 2); (b) đề xuất đầy đủ luật chơi AlgoPerf với quy trình đặt target, tính điểm tổng hợp qua nhiều workload, và cơ chế randomized workload (Mục 4); (c) xây dựng baseline cho 8 họ optimizer phổ biến/được chú ý gần đây, cho thấy benchmark khả thi và có khoảng cách rõ giữa các phương pháp (Mục 7); (d) xây dựng 24 biến thể workload (3 cho mỗi 8 workload cố định) để kiểm tra tính robust.

Paper tự nhận có "hai mục đích song song": vừa là báo cáo nghiên cứu (research report), vừa là tài liệu kỹ thuật đặc tả luật chơi cuộc thi (rules documentation) - độc giả muốn tham gia cuộc thi nên đọc kỹ Mục 4 như một bản tóm tắt cấp cao của luật đầy đủ trên GitHub, còn độc giả quan tâm khía cạnh nghiên cứu có thể lướt qua Mục 4 (trừ Mục 4.5 về cách tính điểm, cần để hiểu Mục 7).

3. Phương pháp chi tiết

3.1 Minh hoạ ba thử thách bằng thực nghiệm cụ thể (Mục 2 của paper)

Đo tốc độ huấn luyện. Hình 1 của paper cho hai lần chạy Adam trên ResNet-50/ImageNet đạt validation error cuối cùng 24.0% và 24.4%; đường cong cắt nhau nhiều lần, và ngay cả đường "best-so-far" (running minimum) cũng vẫn cắt nhau nhiều lần - đoạn giữa đường cam dẫn trước, đoạn cuối đường xanh vượt lên. Điều này cho thấy "ai nhanh hơn" phụ thuộc hoàn toàn vào định nghĩa "khi nào coi là xong".

Độ nhạy với workload. Ba thí nghiệm nhỏ minh hoạ:

- *Wide ResNet đổi stride* (CIFAR-10): với stride chuẩn 2×2, Nesterov cho test error 0.0376, AdamW cho 0.0407 (Nesterov thắng). Chỉ đổi stride của residual block cuối sang 1×1, Nesterov tụt xuống 0.0726 (tăng 3.5 điểm phần trăm) trong khi AdamW gần như không đổi (0.0420). Nguyên nhân: eigenvalue lớn

nhất của Hessian loss tại điểm khởi tạo tăng từ 32 lên 1052 chỉ vì đổi stride, gây mất ổn định huấn luyện ở learning rate lớn. Thêm 1000 bước linear warmup giúp Nesterov ổn định lại, đạt 0.0378 (”Nesterov + Warmup”).

- *Thêm batch normalization* vào ResNet-200: Nesterov từ test error 0.2090 (kiến trúc chuẩn) trở thành hoàn toàn không huấn luyện được (”No Feasible Trials”) khi thêm BN sau mỗi residual connection, vì độ cong ban đầu của loss vượt quá 10^{10}. AdamW chỉ giảm nhẹ xuống 0.2722. Thêm gradient clipping giúp Nesterov phục hồi về 0.2094.
- *Pre-LN vs Post-LN Transformer* (WMT, đo BLEU, 100 trial mỗi kiến trúc): chênh lệch BLEU giữa hai kiến trúc là 0.52 (AdamW), 0.45 (NadamW), 2.97 (Nesterov), 0.26 (Shampoo). *(Tự tính:* $2.9740/0.5208 \approx 5.7$*;* $2.9740/0.4487 \approx 6.6$*;* $2.9740/0.2569 \approx 11.6$*.)* Nesterov nhạy cảm với thay đổi kiến trúc khoảng 6-12 lần so với ba phương pháp còn lại.

Không thể so sánh ”họ” thuật toán. Paper định nghĩa độ suy giảm hiệu năng tệ nhất khi dùng một siêu tham số chung cho mọi workload thay vì tune riêng từng workload:

$$\phi(h, H) = \max_w \left| \frac{\text{val}(w, h) - \text{val}_H(w)}{\text{val}_H(w)} \right|, \qquad \Phi(H) = \min_{h \in H} \phi(h, H)$$

trong đó $\text{val}_H(w) = \min_{h \in H} \text{val}(w, h)$ là hiệu năng tốt nhất đạt được trên workload w trong tập siêu tham số đã lấy mẫu H. $\Phi(H) = 0$ nghĩa là tồn tại một siêu tham số thắng trên mọi workload; $\Phi(H)$ lớn nghĩa là buộc phải tune riêng từng workload mới đạt hiệu năng tốt. Kết quả đo trên 8 workload, H gồm 100 điểm quasirandom: $\Phi_{\text{AdamW}} = 0.195$, $\Phi_{\text{NadamW}} = 0.169$, $\Phi_{\text{Nesterov}} = 0.230$, $\Phi_{\text{HeavyBall}} = 0.239$. Nghĩa là với MỌI thuật toán trong bốn thuật toán này, luôn tồn tại ít nhất một workload mà siêu tham số ”tốt nhất chung” tệ hơn tối thiểu 16.9% so với siêu tham số tối ưu riêng cho workload đó.

Ngoài ra, bản thân **search space** (không phải chỉ giá trị siêu tham số) cũng ảnh hưởng tới kết quả: hai search space ”hợp lý” cho AdamW (Narrow: LR∈[2e-4,5e-3], WD∈[2e-2,0.5]; Broad: LR∈[5e-6,2e-2], WD∈[5e-6,2.0]) với cùng budget 20 trial mô phỏng (lấy median trên 1000 lần mô phỏng từ 100 điểm thật) cho AdamW Narrow thắng áp đảo trên toàn bộ 8 workload. Và **mục tiêu tuning** cũng quan trọng: hai lần tune AdamW cho ResNet-50/ImageNet với step budget khác nhau (186,666 vs 233,333 bước, cùng search space, cùng seed lấy mẫu) cho validation error gần như giống nhau (22.4%-23.0% tuỳ seed) nhưng ”Trial A” (budget ngắn hơn) nhanh hơn hẳn chỉ vì lịch cosine decay co giãn theo độ dài budget - minh hoạ rằng so sánh ”ai đạt target nhanh hơn” với ”ai đạt error thấp nhất trong một budget rộng rãi, tuỳ ý” là hai tiêu chí tune khác nhau, không thể trộn lẫn.

3.2 Bối cảnh liên quan (Mục 3 của paper)

Paper phân loại công trình liên quan thành ba nhóm: (a) benchmark hiện có - MLPerf™ Training (Closed Division đo hệ thống/hardware với thuật toán cố định; Open Division cho phép đổi cả model và thuật toán nên không tách được đóng góp của riêng thuật toán), DeepOBS (chỉ có 1 workload lớn là ResNet-50/ImageNet, còn lại là workload nhỏ), Benchopt (16 bài toán optimization nhưng chỉ 1 là deep learning), và chính opt 29 (Schmidt et al. 2021, dùng DeepOBS, không có điểm tổng hợp qua workload); (b) phê bình phương pháp luận - opt 28 (Choi et al. 2019b) và Sivaprasad et al. (2020) nhấn mạnh vai trò tuning budget/search space; (c) các tranh chấp nổi tiếng trong literature - Wilson et al. (2017) cho rằng SGD+Momentum tổng quát hoá tốt hơn Adam/AdaGrad/RMSProp, bị opt 28 phản biện là chỉ đúng khi cố định β_2/ε của Adam ở default và chỉ tune constant learning rate; tranh chấp RAdam (Liu et al. 2020) vs Adam+warmup được tune đúng cách (Ma và Yarats, 2021); tranh chấp optimizer dùng curvature off-diagonal (K-FAC, Shampoo) có thực sự cần thiết hay RMSProp tune đủ kỹ cũng gần bằng (Amid et al. 2022); và tranh chấp về optimizer ”batch lớn” LARS/LAMB có thực sự cần thiết hay SGD+Nesterov tune kỹ vẫn đạt kết quả tương đương tới batch size 32,768 (Nado et al. 2021).

3.3 Luật chơi AlgoPerf (Mục 4 của paper)

Time-to-result (4.1). Mỗi workload có validation target và test target. Đồng hồ ngừng ngay khi submission đạt cả hai target; nếu không đạt trong runtime tối đa cho phép, submission nhận thời gian vô hạn (∞) cho lần thử đó. Quy trình đặt target: chạy 4 thuật toán phổ biến (AdamW, NadamW, Heavy Ball, Nesterov) với quasirandom search, 200 trial mỗi thuật toán, trong 0.75× runtime tối đa cho phép submission; lấy cấu hình tốt nhất trên validation, chạy lại 20 seed khác nhau, lấy **median** validation error của 20 lần chạy làm validation target, và lấy **test error tệ nhất** trong số 10 lần chạy đạt validation target làm test target (tránh chọn một lần chạy ”ăn may”). Tổng runtime tối đa của 8 workload cố định được giới hạn khoảng 100 giờ trên hardware benchmark.

Đo runtime bằng wall-clock (4.1.1-4.1.2). Paper chọn đo thời gian thực (wall-clock) chứ không đếm step/gradient, vì submission được tự do định nghĩa lại ý nghĩa một ”step” (data echoing, batch size khác nhau...), nên đếm step là vô nghĩa với mục tiêu tổng quát. Hardware chuẩn hoá: một server với 8×NVIDIA V100 (16GB VRAM/card); chỉ cần dùng hardware này cho lần đo chính thức, còn việc tune siêu tham số có thể dùng hệ thống khác nhất quán.

Đặc tả training algorithm (4.2). Submission chỉ được định nghĩa 4 hàm: `update_params`, `data_selection`, `init_optimizer_state`, `get_batch_size`, cùng một search space siêu tham số (nếu dùng external tuning ruleset). Phần còn lại của pipeline (data pipeline, kiến trúc model, loss, tính validation/test metric, đo runtime) do benchmark cố định, nhằm cách ly đóng góp của riêng ”thuật toán huấn luyện”. Ranh giới model/algorithm: được phép tune độ mạnh regularization đã có sẵn trong model (ví dụ dropout rate), nhưng không được thêm layer dropout mới; được thêm L2 regularization ”agnostic với workload”; không được thêm/dời vị trí batch-normalization layer, nhưng được quyết định có cập nhật batch-norm running statistics hay không (quan trọng cho các phương pháp kiểu line-search). Không được tối ưu hoá kernel cấp thấp của JAX/PyTorch, phải dùng đúng version framework quy định; được phép thêm package cài đặt ý tưởng toán học mới (ví dụ BackPACK).

Để khuyến khích submission tổng quát (4.2.2), paper dùng hai câu hỏi hướng dẫn: ”thông tin nào được dùng?” và ”hành động nào được thực hiện dựa trên thông tin đó?”. Được phép: dùng shape/loại layer để chọn routine khác nhau (ví dụ update rule khác nhau cho conv layer vs BN layer vs fully-connected layer). Không được phép: dùng shape tham số như ”dấu vân tay” để nhận diện đúng workload cụ thể rồi tra cứu siêu tham số tối ưu đã tính sẵn (hard-code hành vi theo từng workload cụ thể là vi phạm). Được phép hard-code MỘT cấu hình siêu tham số mặc định tìm được qua tìm kiếm offline tốn kém, vì cấu hình đó phải hoạt động tốt đồng thời trên mọi workload nên có cơ sở để tổng quát hoá tới workload mới. Những trường hợp mơ hồ được phân xử bởi một ”spirit jury” (ban giám khảo xét tinh thần luật chơi) vì không thể viết luật bao quát hết mọi trường hợp.

Workload (4.3). Một workload = dataset + model + loss + target đo trên một evaluation metric. Bảng 7 (tóm tắt): 8 workload cố định phủ 6 loại tác vụ, gồm Criteo 1TB/DLRMsmall (click-through rate, CE), fastMRI/U-Net (tái tạo ảnh MRI, L1 → SSIM), ImageNet/ResNet-50 và ImageNet/ViT (phân loại ảnh, CE → Error Rate), LibriSpeech/Conformer và LibriSpeech/DeepSpeech (nhận dạng giọng nói, CTC → WER), OGBG/GNN (dự đoán tính chất phân tử, CE → mAP), WMT/Transformer (dịch máy, CE → BLEU). Mỗi workload cố định còn có một **randomized workload** tương ứng: một phân phối rời rạc trên 3 biến thể thủ công của workload gốc; chỉ sau khi toàn bộ submission ”đóng băng” mới lấy mẫu một biến thể cụ thể làm **held-out workload** dùng để chấm điểm.

Tuning (4.4). *External tuning ruleset*: 20 trial quasirandom search (hoặc một ”OptList” - danh sách cố định 20 điểm siêu tham số, lấy không hoàn lại) từ search space do submission khai báo; lặp lại 5 ”study” độc lập (tổng 100 trial); mỗi study, chọn trial đạt validation target nhanh nhất, rồi lấy thời gian đạt **test** target của chính trial đó làm điểm cho study; điểm cuối trên mỗi workload = **median** của 5 điểm theo study (mô phỏng 5 ”người dùng” độc lập cùng tune trên cùng search space). *Self-tuning ruleset*: không có tuning ngoài, thuật

toán phải tự thích nghi hoàn toàn ”trên đồng hồ” (ví dụ line search, freeze-thaw, hoặc dùng cùng một siêu tham số cố định cho mọi workload như Adam-default); chỉ 1 lần chạy mỗi study (vẫn 5 study, lấy median), nhưng runtime tối đa gấp 3 lần ruleset external để bù cho việc không có tuning song song.

Tính điểm (4.5). Với k submission trên n workload, performance ratio của submission $\bar{s}$ trên workload $\bar{w}$:

$$r_{\bar{s},\bar{w}} = \frac{t_{\bar{s},\bar{w}}}{\min_s t_{s,\bar{w}}} \qquad \text{(eq. 1)}$$

Performance profile là xác suất (trên workload lấy ngẫu nhiên) mà $r_{\bar{s},w} \leq \tau$:

$$\rho_{\bar{s}}(\tau) = \frac{1}{n} \left| \{ w : r_{\bar{s},w} \leq \tau \} \right| \qquad \text{(eq. 2)}$$

Benchmark score là diện tích chuẩn hoá dưới đường performance profile, tích phân tới $r_{\text{max}} = 4$:

$$B_{\bar{s}} = \frac{1}{r_{\text{max}} - 1} \int_1^{r_{\text{max}}} \rho_{\bar{s}}(\tau)\, d\tau \qquad \text{(eq. 3)}$$

$B_{\bar{s}} \in [0, 1]$, càng cao càng tốt; $B_{\bar{s}} = 1$ nghĩa là nhanh nhất trên mọi workload. Chọn $r_{\text{max}} = 4$ nghĩa là submission chậm hơn 4 lần so với submission nhanh nhất trên một workload sẽ không nhận điểm nào trên workload đó, coi như thất bại.

Held-out workload không tham gia trực tiếp vào benchmark score, nhưng đóng vai trò ”cổng chặn”: một submission chỉ nhận điểm hữu hạn trên một fixed workload nếu đồng thời thoả cả 4 điều kiện: (1) đạt target trên fixed workload trong runtime cho phép; (2) đạt trong vòng 4× thời gian của submission nhanh nhất; (3) đạt target trên held-out workload tương ứng trong runtime cho phép; (4) đạt trong vòng 4× thời gian của submission nhanh nhất trên held-out workload đó. Nếu một trong bốn điều kiện không thoả, submission bị coi là thất bại (thời gian vô hạn) trên CHÍNH fixed workload, dù nó có thể đã đạt target rất nhanh ở đó - đây là cơ chế chính để ngăn submission ”quá khớp” (overfit) vào đúng 8 workload cố định. Để đo tiến bộ qua các năm (không phải so sánh tương đối giữa submission trong cùng một lần benchmark), paper đề xuất dùng trung bình hình học (geometric mean) của raw training time của submission thắng cuộc qua các workload.

4. Thí nghiệm và kết quả chính

4.1 Kết quả đặt target (Mục 5)

Bốn thuật toán target-setting (AdamW, NadamW, Heavy Ball, Nesterov) được tune với search space khá rộng (Bảng 8), 200 trial mỗi thuật toán mỗi workload. So sánh giá trị validation tốt nhất của mỗi thuật toán trên 8 workload (Bảng 9) cho thấy: NadamW thắng trên Criteo (CE 0.123609), ImageNet-ViT (Error Rate 0.22534), LibriSpeech-Conformer (WER 0.077790), LibriSpeech-DeepSpeech (WER 0.113950) và WMT (BLEU 30.8534); Heavy Ball thắng ImageNet-ResNet-50 (Error Rate 0.22534); Nesterov thắng fastMRI (SSIM 0.734645) và OGBG (mAP 0.283124); AdamW không thắng workload nào. *(Tự tính: đối chiếu trực tiếp 32 số trong Bảng 9 theo từng cột higher-is-better/lower-is-better để xác định thuật toán thắng ở mỗi workload, rồi so với tên thuật toán ghi trong Bảng 10 - khớp hoàn toàn: NadamW thắng 5/8, Nesterov 2/8, Heavy Ball 1/8, đúng như câu ”NadamW cũng là thuật toán đặt target trên nhiều workload nhất (5/8)” mà paper phát biểu trực tiếp.)*

So với literature: target Criteo (val CE 0.123649) cạnh tranh với Sterbenz (2017, combined loss 0.1250, dùng model lớn hơn và huấn luyện hơn 3 lần lâu hơn); target ImageNet ResNet-50 (22.57%) tốt hơn paper gốc ResNet (24.7%, He et al. 2016) nhưng kém hơn các recipe hiện đại dùng augmentation mạnh hơn hoặc budget 2.67× (20.2%-21.9%, Bello et al. 2021; Wightman et al. 2021); target ViT (22.69%) gần với paper gốc ViT (22.09%, model nhích lớn hơn); target WER LibriSpeech (test: 0.068 DeepSpeech, 0.047 Conformer)

kém hơn DeepSpeech2 (0.0515, dùng beam search và dataset lớn hơn nhiều) và kém xa Conformer gốc (0.027, dùng beam search + model lớn hơn); target OGBG (mAP 0.28098) gần baseline gốc của OGB (0.2798) nhưng kém các phương pháp mới dùng model/khác hoặc thêm dữ liệu (0.30-0.33); target WMT (BLEU 30.72) kém hơn các paper dùng pretraining 2 chiều hoặc đổi kiến trúc attention (31.3-35.2).

4.2 Randomized workload (Mục 6)

Paper thử 9 nhóm biến đổi: đổi activation function (ReLU → GELU/SiLU/TanH), đổi Pre-LN ↔ Post-LN, đổi "attention temperature" (hệ số nhân c trong softmax$(cXW^Q(XW^K)^\top/\sqrt{D/H})$, mặc định $c = 1$, dùng để mô phỏng bất ổn huấn luyện thường gặp ở Transformer lớn), đổi scale khởi tạo (embedding layer hoặc batch-norm scale), đổi loại normalization layer, đổi width/depth/số channel, đổi cường độ SpecAugment (LibriSpeech), đổi cấu trúc residual connection, đổi loại pooling (global average → max).

Mâu thuẫn nội tại của paper (không tự "sửa" mà nêu rõ): phần văn xuôi Mục 6.2 viết nguyên văn, khi mô tả biến thể attention-temperature cho WMT Transformer: "In order to artificially induce instabilities... we set c = 1.6". Nhưng Bảng 11 (liệt kê giá trị cụ thể từng workload) lại ghi: biến thể Conformer (LibriSpeech) "Increases attention temp from 1 to 1.6", còn biến thể WMT Transformer "Increases attention temperature from 1 to 4.0". Vậy câu văn ở Mục 6.2 gán giá trị 1.6 cho WMT, trong khi bảng số liệu cụ thể lại cho WMT dùng 4.0 và gán 1.6 cho Conformer. Đây là một mâu thuẫn thật giữa văn bản mô tả và bảng số liệu của chính paper (nhiều khả năng câu văn ở Mục 6.2 là bản mô tả cũ (stale) chưa cập nhật theo giá trị cuối cùng trong Bảng 11), không phải lỗi trích xuất pdftotext. Ghi chú này ưu tiên số liệu trong Bảng 11 vì đó là bảng tổng hợp cuối cùng dùng để đặt target. NadamW được chọn làm "thước đo tính robust" vì siêu tham số tối ưu của nó transfer tốt nhất giữa workload gốc và biến thể (so với Heavy Ball/Nesterov rất nhạy). Để lượng hoá độ "khác biệt" của một biến thể, paper dùng chỉ số hạng (rank) của siêu tham số tối ưu khi áp sang workload khác:

$$\min\Big\{\text{rank}(h^*(w), H(w')),\ \text{rank}(h^*(w'), H(w))\Big\}$$

Phát hiện bất ngờ: rất khó tạo ra biến thể đủ làm thay đổi siêu tham số tối ưu của NadamW (một bằng chứng gián tiếp cho tính robust của optimizer có preconditioning), trong khi chỉ cần đổi stride hay thêm BN layer (Mục 2.2.1) là đủ làm Nesterov sụp. Một số thử nghiệm thất bại đáng chú ý: thay đổi tỉ lệ residual $\alpha x + (1-\alpha)F(x)$ cho $\alpha \in [0, 1]$ ảnh hưởng mạnh tới hiệu năng cuối (và $\alpha = 0.5$ không phải luôn là lựa chọn tối ưu) nhưng KHÔNG đổi learning rate tối ưu; tăng augmentation hoặc thêm label noise cũng không đổi được learning rate tối ưu của NadamW, chỉ làm hại hiệu năng. Một số biến thể thậm chí cải thiện hiệu năng so với workload gốc: ReLU → GELU (ResNet), ReLU → SiLU (ResNet, GNN), thêm GLU (ViT), bỏ residual connection (DeepSpeech). Bảng 11 tổng hợp 24 biến thể cuối cùng (3/workload × 8 workload) với validation/test target riêng cho từng biến thể.

4.3 Kết quả baseline (Mục 7)

Tám họ optimizer được dựng baseline: AdamW, NadamW, Nesterov, Heavy Ball, LAMB, Adafactor, SAM (kết hợp Adam), Distributed Shampoo. Quy trình xây search space 20-trial: lấy điểm tốt nhất trên mỗi workload từ 200-trial search gốc, dùng các điểm đó để khoanh biên search space hẹp hơn (loại outlier cho Heavy Ball/Nesterov vì khoảng quá rộng để lấy mẫu hiệu quả); cố định warmup 5% linear; với nhóm dùng Linear-Decay-Constant schedule thì decay factor chọn trong {1e-2, 1e-3}, decay steps cố định 0.9; dropout/aux. dropout buộc bằng nhau, chọn trong {0.0, 0.1}; label smoothing chọn trong {0.1, 0.2} khi workload hỗ trợ. Với mỗi thuật toán có tune β_1, paper còn tạo thêm baseline "OptList": chọn 20 điểm cụ thể bằng cách luân phiên lấy điểm tốt nhất từng workload từ 200-trial search.

Lưu ý về đo thời gian: 3/8 workload (Criteo, fastMRI, OGBG) bị nghẽn ở data pipeline (input pipeline chậm hơn tính gradient), nên optimizer "tốn thêm việc mỗi batch" (như SAM) có thể tận dụng thời gian accelerator đang rảnh. SAM và Distributed Shampoo tốn 1.5-2× thời gian/step so với các thuật toán còn lại. Distributed

Shampoo KHÔNG được đo chính thức trên hardware GPU chuẩn (8×V100) do lỗi memory/config tại thời điểm viết bài, nên bị loại khỏi so sánh theo runtime (chỉ xuất hiện trong so sánh theo số step, đo trên TPU). Adafactor chạy chậm hơn AdamW 10-20%/step trên GPU (nguyên nhân chưa được paper lý giải rõ), khiến baseline Adafactor bỏ lỡ một số target mà lẽ ra đạt được nếu chạy cùng số step như AdamW.

Bảng 13 cho điểm benchmark (runtime / steps): AdamW 0.600 / 0.596; **NadamW 0.850 / 0.830** (cao nhất trong số các baseline tuân thủ runtime budget); Nesterov 0.0 / 0.0; Heavy Ball 0.0 / 0.0 (không đạt target nào trong 20-trial budget với search space dạng ”box”); LAMB 0.249 / 0.248; Adafactor 0.236 / 0.476 (chênh lệch lớn giữa hai cách đo do chạy chậm/step); SAM(w. Adam) 0.120 / 0.732 (tương tự, bị phạt nặng theo runtime vì tốn 1.5-2×/step); Distributed Shampoo ”-” / 0.854 (không có điểm runtime vì chưa đo chính thức, nhưng theo số step là cao nhất). NadamW đạt target trên 7/8 workload (trượt duy nhất ImageNet-ResNet-50) và được paper gọi là ”state of the art tạm thời” (provisional SOTA) trong số các baseline hợp lệ về runtime; Distributed Shampoo cũng đạt 7/8 (trượt Criteo) nhưng có ”lợi thế không công bằng” vì chưa bị đo runtime thật. Heavy Ball/Nesterov từng thắng target-setting trên một số workload cụ thể (ResNet-50, fastMRI, OGBG) khi được tune rất kỹ (200 trial) nhưng hoàn toàn thất bại trong 20-trial budget với search space ”box” rộng - cho thấy khoảng cách giữa ”có thể thắng nếu tune cực kỹ” và ”thắng tin cậy trong một budget tuning thực tế, hợp lý”.

4.4 So sánh search space (Mục 7.3.1, Bảng 14)

Với AdamW/NadamW/Nesterov/Heavy Ball, paper so ba kiểu search space: tune β_1, cố định β_1=0.9, và OptList. NadamW tune-β_1 (0.850) vượt xa NadamW fixed-β_1 (0.600, giống hệt AdamW!) - tức lợi thế của NadamW so với AdamW ở budget này đến chủ yếu từ việc search space cho phép tune β_1. Ngược lại, AdamW tune-β_1 (0.600) chỉ nhích nhẹ so với fixed-β_1 (0.597) - tune β_1 ít quan trọng hơn với AdamW. OptList cho AdamW/NadamW (0.725/0.836) gần bằng search space ”box” tốt nhất của chính chúng, nhưng với Nesterov/Heavy Ball thì OptList (0.233/0.231) vượt xa hẳn so với box search space (0.0/0.0) - được paper giải thích là vì search space ”box” cho các optimizer không có preconditioning phải trải rộng để bao quát đủ workload, kéo theo nhiều điểm ”xấu” trong không gian đó, khiến 20 mẫu quasirandom khó trúng điểm tốt; còn một danh sách rời rạc gồm đúng 20 điểm ”đã biết là tốt” tránh được vấn đề này.

5. Phân tích / ablation

Về quy trình đặt target (8.1): chính paper tự nhận quy trình của họ là ”generic” nhưng giả định biết trước kiến trúc/dataset/loss/giới hạn runtime, và đủ compute để lấy mẫu hàng trăm trial - một giả định không hề nhẹ. Paper chỉ ra hai lỗ hổng nghiên cứu bị bỏ quên trong literature: (1) chưa có thuật toán nào xác định ”nên huấn luyện bao lâu” một cách nguyên tắc; (2) công cụ tune siêu tham số tự động hiện có vẫn cần con người xác định trước search space, mà việc thiết kế search space tốt cho một budget cho trước gần như chưa được nghiên cứu nghiêm túc. Hướng mở: xem việc đặt target và việc chấm benchmark như ”ảnh gương” của nhau, có thể luân phiên cải thiện cả hai qua các vòng benchmark.

Về randomized workload (8.2): việc tìm biến thể vừa thực tế, vừa huấn luyện được, vừa đủ khác biệt - được paper thừa nhận là ”bất ngờ khó và quá tốn công”. Đây là lời kêu gọi cho một ”lý thuyết transfer siêu tham số” (hyperparameter-transfer theory, liên hệ Yang et al. 2021 về μ-transfer theo model size) mà hiện vẫn thiếu.

Về baseline (8.3): luận điểm trung tâm của Mục 8.3, nói thẳng rằng cách làm phổ biến hiện nay - so sánh ”update rule trừu tượng” tách rời khỏi tuning protocol cụ thể hoá nó - ”chắc chắn sẽ tạo ra các kết quả mâu thuẫn vô tận”. Việc Heavy Ball/Nesterov thất bại hoàn toàn trong benchmark KHÔNG phải là bản án cho chính các update rule đó, mà là bằng chứng cho thấy chúng cần nhiều công sức tune theo-từng-workload hơn, và các tác giả đề xuất optimizer mới nên có trách nhiệm cung cấp hướng dẫn tune theo từng mức budget - điều mà literature hiện tại hầu như không làm.

Về chi phí (8.4, Bảng 15): đây là con số gây ấn tượng mạnh nhất của paper. Để ”Tune một submission” theo external tuning ruleset (100 trial = 20×5 study) tốn **23,222.61 giờ máy** trên 8×V100 (~2.65 năm-máy liên tục!). *(Tự tính: Bảng 15 cho ”một siêu tham số” = 232.23 giờ (chạy 1 lần qua cả 8 fixed + 8 held-out*

workload). "Chấm điểm một submission" = lặp lại 5 study $\Rightarrow 5 \times 232.23 \approx 1161.15$ *giờ, khớp với số 1161.13 mà paper ghi (sai số làm tròn). "Tune một submission" = mỗi study cần 20 trial × 5 study = 100 lần chạy "một siêu tham số"* $\Rightarrow 100 \times 232.23 \approx 23{,}223$ *giờ, khớp với số 23,222.61 giờ paper ghi. Việc hai cách tính độc lập này khớp nhau xác nhận cấu trúc chi phí 20×5=100 trial mà paper mô tả bằng lời là tự-nhất-quán.)* Để giảm rào cản cho nhóm ít compute, paper đề xuất "qualification set" (chỉ 3/8 workload: Criteo, OGBG, WMT, không có held-out) giúp giảm chi phí khoảng 10 lần (tune: 2064.75 giờ; chấm điểm: 103.24 giờ).

Bốn nhóm hạn chế được paper tự liệt kê: (a) **độ phủ** - external tuning ruleset chỉ cho phép quasirandom search thuần (không cho submission tự dùng thuật toán HPO thông minh hơn của riêng họ), API không hỗ trợ model parallelism hay truy cập thông tin kiến trúc chi tiết (nên không hỗ trợ optimizer kiểu K-FAC), chỉ hỗ trợ JAX/PyTorch; (b) **validity của thực nghiệm** - tỉ lệ 20 trial cho 8+8=16 workload có thể vẫn cho phép tune "quá khớp" vào đúng bộ workload này; các biến thể của cùng một workload gốc không được đảm bảo khác biệt lẫn nhau, và một số "họ" biến đổi (ví dụ đổi activation function) lặp lại ở nhiều workload gốc khác nhau, làm giảm độ đa dạng thực chất; huấn luyện neural network vốn nhiễu, đặc biệt khi siêu tham số tốt nhất thường nằm gần "edge of stability" (Cohen et al. 2022), nên số seed ít có thể gây nhiễu kết quả; (c) **phạm vi/liên hệ thực tế** - chỉ một "hạng cân" hardware (8×V100/16GB), nên batch size hiệu dụng của mọi workload gần như bị ép về mức tối đa vừa bộ nhớ, và hoàn toàn thiếu các modality quan trọng khác (object detection từ point cloud, dự báo thời tiết, language modeling, video, sinh ảnh) - rủi ro là optimizer phổ biến hiện nay có thể đã "cùng tiến hoá" với đúng các workload phổ biến hiện nay, khiến benchmark này có thể đang gia cố một "cực tiểu địa phương về phương pháp luận" thay vì thoát khỏi nó; (d) **khả năng tiếp cận** - chi phí 8×V100 như trên có thể là rào cản thật với nhóm nghiên cứu nhỏ, giải quyết tạm bằng tài trợ compute và "qualification set".

Hướng tương lai (8.5): phát triển "recipe huấn luyện agnostic với workload" tổng quát hơn cả quy trình đặt target hiện tại; và ý tưởng về một benchmark riêng "AlgoPerf Models" đo tốc độ do cải tiến MODEL (không phải thuật toán) mang lại, cố định một vài thuật toán huấn luyện chuẩn để cách ly đóng góp của kiến trúc.

6. Giới hạn

Ngoài 4 nhóm hạn chế paper tự liệt kê ở Mục 5 trên, cần nêu rõ thêm các điểm sau (phân biệt rõ cái gì là paper tự thừa nhận, cái gì là quan sát/suy luận của người ghi chú):

- **Distributed Shampoo chưa được đo thật** (paper tự thừa nhận): kết quả 7/8 target và điểm benchmark cao nhất theo số step của Distributed Shampoo dựa trên thời gian đo trên TPU rồi so sánh gián tiếp, KHÔNG phải đo trực tiếp trên hardware benchmark chính thức (8×V100) do lỗi kỹ thuật tại thời điểm viết bài. Mọi kết luận "Shampoo mạnh nhất" cần đi kèm điều kiện này.
- **Độ chậm của Adafactor có thể là lỗi cài đặt, không phải bản chất thuật toán** (paper tự thừa nhận chưa tìm ra nguyên nhân): baseline Adafactor chạy chậm hơn AdamW 10-20%/step trên GPU nhưng gần bằng AdamW trên TPU, và paper nói rõ "chưa tìm thấy vấn đề rõ ràng nào trong implementation" nhưng "muốn điều tra thêm trong tương lai". Vì vậy điểm runtime thấp của Adafactor trong Bảng 13 không nên được đọc như một kết luận khách quan về tốc độ nội tại của thuật toán Adafactor.
- **Benchmark không trực tiếp trả lời câu hỏi "phương pháp HPO nào tốt nhất"** (quan sát của người ghi chú, liên hệ opt 31-33): external tuning ruleset buộc dùng quasirandom search thuần làm cơ chế tune chính thức; AlgoPerf do đó chỉ so sánh cặp (optimizer, search space) dưới MỘT phương pháp tune cố định, không so sánh Hyperband/Bayesian Optimization với random search trên cùng workload. Một độc giả tới từ literature Hyperband/BayesOpt (opt 31-33) cần hiểu rằng AlgoPerf không "giải quyết" câu hỏi đó, dù hạ tầng của nó (search space cố định, budget cố định) rất thích hợp để làm thí nghiệm so sánh đó trong tương lai.
- **Ghi chú về trích xuất pdftotext:** công thức Φ(H) (Mục 2.3.1), công thức rank (Mục 6.2), và công thức performance profile/benchmark score (Mục 4.5) được trích xuất tương đối sạch; các ký hiệu bị biến dạng rõ nhất là "ˆ" (thay cho dấu nhân "×"), và mũi tên ngôn ngữ "Ñ" (ví dụ "De Ñ En" nghĩa

là ”Đức → Anh” trong Mục WMT). Các trường hợp này được diễn giải lại theo ngữ cảnh, không trích nguyên văn ký hiệu lỗi.

7. Liên hệ với các bài khác

- **opt 28** (Choi et al. 2019, ”On Empirical Comparisons of Optimizers for Deep Learning”): được AlgoPerf trích dẫn nhiều lần (Mục 3.2-3.3) làm nền tảng cho luận điểm trung tâm ”tuning protocol là một phần của thuật toán”, và cho cuộc tranh chấp Wilson et al. (2017) về SGD vs Adam. AlgoPerf hiện thực hoá trực tiếp thông điệp đó bằng cách buộc mọi submission phải khai báo VÀ ”trả giá” cho search space của chính nó dưới một tuning budget chung.
- **opt 29** (Schmidt et al. 2021, ”Descending through a Crowded Valley”): được nêu tên trực tiếp trong Mục 3.1 là nghiên cứu so sánh thực nghiệm gần nhất (qua DeepOBS), kèm phê bình rằng nó thiếu một điểm tổng hợp qua workload và chủ yếu dùng bài toán quy mô nhỏ. Cơ chế performance profile + benchmark score của AlgoPerf (Mục 4.5) chính là lời đáp cho đúng khoảng trống mà opt 29 (và cả opt 28) đã chỉ ra nhưng chưa tự giải quyết.
- **opt 31** (Hyperband): external tuning ruleset của AlgoPerf dùng quasirandom search thuần, KHÔNG dùng successive-halving/Hyperband hay bất kỳ early-stopping bandit nào - một lựa chọn đơn giản hoá mà chính Mục 8.4 của AlgoPerf gọi là hạn chế về độ phủ. Một submission muốn áp Hyperband-style early-stopping trong 20-trial budget sẽ phải nhúng logic đó vào bên trong thuật toán self-tuning của chính mình, vì hạ tầng tune chính thức không hỗ trợ.
- **opt 32 / opt 33** (Practical Bayesian Optimization / BayesOpt Tutorial): cùng quan sát như trên - search-space + budget cố định của AlgoPerf (20 trial, 5 study) là một ”sân chơi” tự nhiên để kiểm chứng liệu Bayesian Optimization có thắng quasirandom search trong đúng khuôn khổ 20-trial này hay không, nhưng bản thân paper AlgoPerf không thực hiện so sánh đó.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan: nén/post-training cho LightOnOCR-3-0.8B (VLM OCR, kiến trúc lai GatedDeltaNet + full attention, khoảng 3600 token/trang, chạy trên GPU ≤ 8GB, batch size 1). AlgoPerf được thiết kế cho một hệ sinh thái khác hẳn (8×V100 = 128GB VRAM tổng, đa workload, cuộc thi cộng đồng), nên không thể áp trực tiếp, nhưng nhiều nguyên tắc phương pháp luận của nó vẫn liên quan trực tiếp:

1. **Đo theo wall-clock, không đo theo step, khi so sánh optimizer cho giai đoạn recovery sau pruning.** Paper cho thấy SAM/Shampoo tốn 1.5-2×/step so với AdamW/NadamW; nếu dự án thử các optimizer ”nặng” hơn (second-order, SAM) để phục hồi mô hình sau khi pruning GatedDeltaNet, phải so sánh theo thời gian thực trên đúng GPU 8GB đang dùng, không so theo số step, nếu không sẽ lặp lại sai lầm mà cả opt 28/29/30 đều cảnh báo.
2. **Search space + tuning budget phải cố định khi so sánh optimizer cho fine-tuning phục hồi.** Nếu so AdamW với NadamW (hoặc SGD+Nesterov) để recovery-finetune model đã pruned, cần dùng CÙNG số trial tìm siêu tham số (ví dụ 5-10 trial quasirandom trên cùng search space) cho cả hai, theo đúng tinh thần ”tuning protocol là một phần của thuật toán” mà AlgoPerf vận hành; so sánh ”optimizer A không cần tune” với ”optimizer B đã tune kỹ” sẽ cho kết luận sai.
3. **NadamW-với-β_1-tune-được là một baseline mặc định hợp lý.** Trong benchmark đa-modal (ảnh, giọng nói, dịch, đồ thị) của AlgoPerf, NadamW đạt target trên 7/8 workload và là baseline mạnh nhất tuân thủ runtime budget. Dù không có bằng chứng trực tiếp cho VLM OCR, tính robust-đa-domain này gợi ý NadamW là lựa chọn khởi điểm hợp lý hơn AdamW thuần cho recovery fine-tuning của LightOnOCR, thay vì mặc định dùng AdamW chỉ vì nó phổ biến hơn.
4. **Cảnh báo cụ thể về curvature tăng vọt sau khi sửa kiến trúc:** thí nghiệm đổi stride của AlgoPerf cho thấy chỉ một thay đổi kiến trúc nhỏ có thể làm eigenvalue lớn nhất của Hessian tăng từ 32 lên 1052 (32×!), gây mất ổn định huấn luyện ở learning rate vốn ổn định trước đó. Pruning phần GatedDeltaNet hoặc attention head của LightOnOCR rất có thể tạo ra hiệu ứng tương tự (lớp còn lại đột ngột ”thấy” curvature

khác tại điểm sau-pruning); nên luôn thêm learning-rate warmup (và cân nhắc gradient clipping, theo đúng cách AlgoPerf phục hồi được Nesterov sau khi thêm BN layer) khi bắt đầu recovery fine-tuning, NGAY CẢ KHI recipe fine-tuning gốc (trước pruning) không cần warmup.

5. **Ràng buộc batch size = 1, GPU ≤ 8GB khiến "self-tuning ruleset" (không tune ngoài) là triết lý phù hợp hơn "external tuning".** Với tài nguyên hạn chế, không thể chạy 20 trial × 5 study như AlgoPerf; nên ưu tiên optimizer có hiệu năng mặc định tốt và ít nhạy với siêu tham số (NadamW/AdamW) hơn là Heavy Ball/Nesterov - vốn đã được chứng minh trong chính paper này là cần tune rất kỹ (200 trial) mới cạnh tranh được, một mức tune hoàn toàn không khả thi trên một GPU 8GB chạy batch size 1.

9. Thí nghiệm gợi ý

1. **So sánh time-to-target AdamW vs NadamW cho recovery fine-tuning**, trên chính GPU 8GB: cố định một target CER/WER trên tập validation OCR, đo wall-clock cần để đạt target đó với mỗi optimizer, dùng cùng budget tune nhỏ (ví dụ 5 trial quasirandom trên learning rate + weight decay cho mỗi optimizer) - tái hiện phương pháp luận time-to-result của AlgoPerf ở quy mô rất nhỏ.
2. **Kiểm tra tính robust của siêu tham số tối ưu khi đổi nhẹ kiến trúc sau-pruning:** tạo 2-3 biến thể nhỏ của bước recovery (ví dụ đổi scale khởi tạo của layer vừa được re-insert, hoặc đổi vị trí LayerNorm Pre/Post ở một block vừa fine-tune lại), rồi kiểm tra learning rate tối ưu của NadamW có transfer tốt giữa các biến thể hay không - trực tiếp tái hiện Mục 6 của paper trên model thật của dự án.
3. **Đo eigenvalue lớn nhất của Hessian ngay sau khi pruning** (vài phép Hessian-vector product qua power iteration, rẻ vì batch size chỉ là 1) so với trước khi pruning, để kiểm tra có xảy ra hiện tượng curvature tăng vọt kiểu "32 → 1052" như thí nghiệm stride của paper hay không; nếu có, xác nhận cần thêm warmup trước khi bắt đầu recovery fine-tuning.
4. **Dựng một "performance profile" thu nhỏ** trên vài sub-task OCR khác nhau (ví dụ các loại tài liệu/ngôn ngữ khác nhau trong tập eval), so 2-3 optimizer ứng viên cho recovery (AdamW, SGD+Nesterov+warmup, Adafactor để tiết kiệm bộ nhớ optimizer-state trên GPU 8GB), rồi tính benchmark score kiểu AlgoPerf (diện tích dưới performance profile) để chọn optimizer "ổn định nhất" qua nhiều sub-task, không chỉ nhanh nhất trên một sub-task.
5. **Kiểm tra lợi ích thật của việc tune β_1** cho AdamW trong bối cảnh recovery fine-tuning: so sánh "AdamW β_1 cố định 0.9" với "AdamW có tune β_1" dùng cùng số trial tune nhỏ, để xem việc thêm một chiều tune có đáng với chi phí trial bỏ ra hay không - mô phỏng thu nhỏ của Bảng 14 trong paper.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Time-to-result	Thời gian wall-clock để lần đầu đạt validation + test target trên một workload, trên hardware chuẩn hoá.
Fixed workload	Workload (dataset+model+loss+target) được đặc tả đầy đủ, công khai từ đầu (8 workload trong Bảng 7).
Randomized workload / held-out workload	Phân phối trên 3 biến thể thủ công của một fixed workload; một biến thể cụ thể (held-out) chỉ được lấy mẫu sau khi mọi submission đóng băng, dùng làm "cổng chặn" chống overfit vào đúng 8 fixed workload.
External tuning ruleset	20 trial quasirandom search (hoặc OptList gồm 20 điểm cố định) từ search space submission khai báo, lặp 5 study, điểm = median thời gian đạt test target của 5 study.
Self-tuning ruleset	Không có tuning ngoài; thuật toán phải hyperparameter-free hoặc tự tune "trên đồng hồ"; đổi lại runtime budget gấp 3 lần ruleset external.

Khái niệm	Ý nghĩa
OptList	Danh sách cố định các điểm siêu tham số (thay cho một search space liên tục), lấy mẫu không hoàn lại.
$\Phi(H)$	$\min_{h\in H} \max_w \lvert(\text{val}(w,h) - \text{val}_H(w))/\text{val}_H(w)\rvert$: độ suy giảm hiệu năng tệ nhất khi dùng MỘT siêu tham số chung cho mọi workload, so với tune riêng từng workload.
Performance ratio $r_{\bar{s},\bar{w}}$ (eq. 1)	$t_{\bar{s},\bar{w}}/\min_s t_{s,\bar{w}}$: thời gian của submission $\bar{s}$ trên workload $\bar{w}$, chia cho thời gian của submission nhanh nhất trên CHÍNH workload đó.
Performance profile $\rho_{\bar{s}}(\tau)$ (eq. 2)	Tỉ lệ workload mà submission $\bar{s}$ có performance ratio $\leq \tau$.
Benchmark score $B_{\bar{s}}$ (eq. 3)	$\frac{1}{r_{\max}-1}\int_1^{r_{\max}} \rho_{\bar{s}}(\tau)\, d\tau$, với $r_{\max} = 4$; càng gần 1 càng tốt.
Spirit jury	Ban giám khảo xét ”tinh thần luật chơi” cho các trường hợp API không bao quát hết được.
Qualification set	Tập con rẻ hơn (3/8 workload, không held-out) dùng để xét tài trợ compute cho nhóm ít tài nguyên.

11. Kết luận ngắn

AlgoPerf: Training Algorithms là benchmark cạnh tranh đầu tiên của MLCommons Algorithms Working Group nhằm trả lời câu hỏi ”training algorithm nào thực sự tốt nhất”, bằng cách buộc mọi so sánh phải đi kèm một tuning protocol cố định, đo bằng wall-clock time-to-target trên hardware chuẩn hoá, qua một bộ workload đa dạng có cả fixed và randomized/held-out. Thực nghiệm baseline trên 8 họ optimizer cho thấy NadamW (với search space tune được β_1) là ”state of the art tạm thời”, vượt rõ AdamW, còn các optimizer không có preconditioning (Heavy Ball, Nesterov) thất bại hoàn toàn trong khuôn khổ 20-trial dù từng thắng khi được tune cực kỹ, một minh chứng thực nghiệm trực tiếp cho luận điểm ”không thể so sánh update rule tách rời khỏi tuning protocol” mà opt 28 và opt 29 đã cảnh báo từ trước. Chi phí chạy đầy đủ benchmark (hàng chục nghìn giờ máy để tune một submission) và các hạn chế về độ phủ/phạm vi mà paper tự liệt kê cho thấy đây là một bước đi đầu quan trọng nhưng còn nhiều khoảng trống, đặc biệt là thiếu một lý thuyết transfer siêu tham số và thiếu so sánh với các phương pháp HPO thông minh hơn random search (Hyperband, Bayesian Optimization).

Nguồn gốc: ghi chú này được viết dựa trên toàn văn arXiv:2306.07179 (v2, 18/06/2025), tải và đọc trực tiếp bằng pdftotext -layout; không dùng bản dịch sẵn có.

Bài 31

Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization (2016)

Tác giả: Lisha Li (CMU), Kevin Jamieson (Univ. of Washington), Giulia DeSalvo (Google), Afshin Rostamizadeh (Google), Ameet Talwalkar (CMU, Determined AI).
Venue / arXiv: Journal of Machine Learning Research 18 (2018) 1-52; arXiv:1603.06560 (v4, 18/06/2018; bản v1 nộp 11/2016, có bản rút gọn trước đó tại Li et al. 2017).
Nguồn: https://arxiv.org/abs/1603.06560
Chủ đề: tối ưu siêu tham số (hyperparameter optimization) bằng early-stopping và phân bổ tài nguyên thích ứng (adaptive resource allocation), đóng khung thành bài toán ”pure-exploration non-stochastic infinite-armed bandit”; thuật toán Hyperband dựa trên SuccessiveHalving.
Vị trí trong lộ trình: đọc sau opt 32/33 (Bayesian optimization, hướng ”configuration selection”) để thấy một hướng hoàn toàn khác (”configuration evaluation”) giải quyết cùng bài toán tune siêu tham số; cũng liên hệ opt 34 (Population Based Training, thích ứng online trong một lần huấn luyện) và opt 28/opt 29/opt 30 (benchmark optimizer, cùng tinh thần đánh giá thực nghiệm nhiều phương pháp trên nhiều bài toán).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Hiệu năng của một mô hình machine learning phụ thuộc mạnh vào bộ siêu tham số (hyperparameter) của nó (tốc độ học, độ mạnh regularization, kiến trúc...). Cách làm mặc định (brute-force) là grid search hoặc random search (Bergstra & Bengio, 2012, Mục 1). Để làm tốt hơn, lĩnh vực hyperparameter optimization (HPO) gần đây bị ”chi phối” bởi các phương pháp **Bayesian optimization (BayesOpt)**: SMAC (random forest mô hình hoá $p(y \mid \lambda)$ dạng Gauss), TPE (tree-structured Parzen estimator), Spearmint (Gaussian Process, chính là opt 32/33 trong thư mục này). Các phương pháp này tập trung cải thiện **configuration selection**: chọn điểm thử tiếp theo thông minh hơn random (Hình 1a).

Paper chỉ ra một hướng **song song, trực giao** (orthogonal): cải thiện **configuration evaluation** (Hình 1b), tức là cấp tài nguyên (resource: số iteration, kích thước tập dữ liệu, số feature...) một cách thích ứng cho các cấu hình đã chọn, phân bổ nhiều tài nguyên hơn cho cấu hình có vẻ tốt và loại sớm cấu hình kém. Lợi ích: có thể thử **nhiều hơn hẳn về số lượng** cấu hình cùng một ngân sách tổng, so với huấn luyện mọi cấu hình tới khi hoàn tất. Một số công trình trước đã kết hợp BayesOpt với resource allocation (Swersky et al. 2013/2014; Domhan et al. 2015), nhưng paper này chọn làm việc với **random search** làm điểm khởi đầu đơn giản và có cơ sở lý thuyết (random search luôn hội tụ tiệm cận về cấu hình tối ưu nhờ lập luận ”phủ kín” không gian, bất kể độ mượt của hàm mục tiêu, trong khi BayesOpt không có cấu trúc bổ sung cũng chịu tốc độ hội tụ mũ theo số chiều, Kandasamy et al. 2015).

Các phương pháp ”halving”/loại dần trước đó (Sparks et al. 2015; Jamieson & Talwalkar 2015, dựa trên Karnin et al. 2013) gặp vấn đề **”n versus B/n”**: với ngân sách tổng B cố định, không rõ nên thử nhiều cấu hình n lớn (mỗi cấu hình ít tài nguyên B/n) hay ít cấu hình (mỗi cấu hình nhiều tài nguyên)? Đây chính là bài toán mà Hyperband giải quyết.

2. Ý tưởng chính

SuccessiveHalving (SH) (Jamieson & Talwalkar, 2015, Mục 3.1): với n cấu hình và ngân sách B, cấp đều B/n tài nguyên cho tất cả, đánh giá, loại nửa kém hơn, lặp lại cho tới khi còn 1 cấu hình; mỗi vòng cấp tài

nguyên tăng theo cấp số nhân cho các cấu hình còn lại. Nhược điểm: SH cần biết n trước, mà không có cách nào biết trước nên chọn n lớn (nhiều cấu hình, mỗi cấu hình ít tài nguyên, rủi ro loại nhầm cấu hình tốt vì chưa kịp ”lộ” chất lượng) hay n nhỏ (ít cấu hình, nhiều tài nguyên mỗi cấu hình, rủi ro không đủ đa dạng để tìm cấu hình tốt).

Hyperband giải quyết vấn đề này bằng cách **không chọn một giá trị n duy nhất**, mà chạy nhiều ”bracket” SuccessiveHalving khác nhau, mỗi bracket tương ứng một đánh đổi n so với B/n khác nhau (tức là làm một kiểu ”grid search” trên chính biến n), rồi lấy cấu hình tốt nhất qua tất cả brackets. Về lý thuyết, paper đóng khung bài toán thành **pure-exploration non-stochastic infinite-armed bandit (NIAB)**: mỗi ”cánh tay” (arm) là một cấu hình siêu tham số, kéo cánh tay i đúng k lần tương đương huấn luyện cấu hình đó k đơn vị tài nguyên và quan sát loss trung gian $\ell_{i,k}$; mục tiêu là tìm cánh tay có loss giới hạn $\nu_i = \lim_{k\to\infty} \ell_{i,k}$ nhỏ, dùng ít tổng số lần ”kéo” nhất.

3. Phương pháp chi tiết

3.1 Thuật toán Hyperband (Giải thuật 1, Mục 3.2)

```
input: R, eta (mặc định eta = 3)
s_max = floor(log_eta(R)),  B = (s_max + 1) * R
for s in {s_max, s_max-1, ..., 0}:
    n = ceil( B/R * eta^s / (s+1) ),   r = R * eta^{-s}
    T = get_hyperparameter_configuration(n)        # lấy n cấu hình i.i.d.
    for i in {0, ..., s}:
        n_i = floor(n * eta^{-i})
        r_i = r * eta^i
        L = { run_then_return_val_loss(t, r_i) : t in T }
        T = top_k(T, L, floor(n_i / eta))           # giữ lại 1/eta tốt nhất
return cấu hình có loss trung gian nhỏ nhất đã thấy
```

Hai siêu tham số **của chính Hyperband**: R (tài nguyên tối đa cấp cho một cấu hình) và η (tỉ lệ loại bỏ mỗi vòng, mặc định 3, điều chỉnh được). $s_{\max} = \lfloor \log_\eta(R) \rfloor$ quyết định số bracket $s_{\max} + 1$. Bracket $s = s_{\max}$ ”aggressive” nhất (nhiều cấu hình n, ít tài nguyên mỗi vòng ban đầu, early-stopping mạnh); bracket $s = 0$ tương đương **random search thuần** (mỗi cấu hình được cấp đủ R tài nguyên ngay từ đầu, không loại sớm). Một lần chạy Hyperband tốn ngân sách $(s_{\max} + 1)B$.

Ví dụ minh hoạ (Bảng 1 của paper, $R = 81$, $\eta = 3$, $s_{\max} = 4$, 5 bracket):

s	i=0	i=1	i=2	i=3	i=4
4	n=81, r=1	n=27, r=3	n=9, r=9	n=3, r=27	n=1, r=81
3	n=27, r=3	n=9, r=9	n=3, r=27	n=1, r=81	
2	n=9, r=9	n=3, r=27	n=1, r=81		
1	n=6, r=27	n=2, r=81			
0	n=5, r=81				

Kiểm tra bảo toàn ngân sách (tự tính), bracket $s = 4$: tổng tài nguyên tiêu thụ $= 81 \times 1 + 27 \times 3 + 9 \times 9 + 3 \times 27 + 1 \times 81 = 81 + 81 + 81 + 81 + 81 = 405$, đúng bằng $B = (s_{\max} + 1)R = 5 \times 81 = 405$. Với bracket $s = 3$: $27 \times 3 + 9 \times 9 + 3 \times 27 + 1 \times 81 = 81 + 81 + 81 + 81 = 324$, nhỏ hơn 405 (tự tính), cho thấy các bracket chỉ dùng ”xấp xỉ” B tài nguyên, không bắt buộc bằng nhau tuyệt đối. Bracket $s = 0$ (random search): $5 \times 81 = 405$, lại đúng bằng B.

Tốc độ tăng tốc tối đa (tự tính): nếu thời gian huấn luyện tăng tuyến tính theo tài nguyên, bracket khai phá nhiều nhất ($s = s_{\max}$) đánh giá R cấu hình với ngân sách $(s_{\max} + 1)R$, nên tốc độ tăng tốc tối đa so với

random search thuần là $R/(s_{\max}+1)$; với $R=81, \eta=3$: $81/5=16{,}2\times$ (tự tính). Paper tự báo cáo (Mục 4.4.1, không phải phép tính của người ghi chú) với bộ R, η dùng trong thí nghiệm mạng neural, tốc độ tăng tốc tối đa lý thuyết là khoảng $50\times$, còn tốc độ thực tế quan sát được dao động $6\times$ đến $70\times$ tuỳ bài toán.

3.2 Các loại "tài nguyên" (resource, Mục 3.4)

Hyperband tổng quát hoá cho nhiều định nghĩa "tài nguyên": **số iteration/epoch** (ví dụ LeNet trên MNIST, Mục 3.3), **thời gian huấn luyện** (hữu ích để chặn "straggler job" trong tính toán phân tán), **kích thước tập dữ liệu con** (data set subsampling, hiệu quả cho thuật toán có thời gian huấn luyện siêu tuyến tính như kernel method), **số lượng feature** (ví dụ số random feature xấp xỉ kernel RBF, Rahimi & Recht 2007).

3.3 Chọn R và η (Mục 3.5-3.6)

R nên có giới hạn tự nhiên theo bài toán (kích thước tập huấn luyện đầy đủ, giới hạn bộ nhớ, số epoch hợp lý theo kinh nghiệm). Nếu không biết R, paper cung cấp **phiên bản infinite horizon** (Mục 5), tăng gấp đôi ngân sách $B \in \{2, 4, 8, 16, \dots\}$ theo thời gian, tự suy ra R tăng dần. η kiểm soát độ "aggressive": η lớn hơn nghĩa là loại nhanh hơn, ít vòng hơn; lý thuyết tối ưu gợi ý $\eta = e \approx 2{,}718$, nhưng thực hành paper đề xuất $\eta=3$ hoặc 4 (kết quả "không nhạy cảm nhiều" với lựa chọn η). Số bracket $s_{\max}+1 = \lfloor \log_\eta(R) \rfloor + 1$; paper chọn η sao cho có khoảng 5 bracket cho các thí nghiệm của mình. Với bài toán tốn kém/chiều cao, có thể hạn chế chỉ chạy các bracket aggressive nhất (đặt $n_{\min}$) để giảm overhead.

4. Thí nghiệm và kết quả chính

Paper so sánh Hyperband với SMAC, TPE, Spearmint (BayesOpt), SuccessiveHalving (chỉ lặp lại bracket aggressive nhất $s=s_{\max}$), random search, và "random $2\times$" (random search với gấp đôi ngân sách).

4.1 Mạng neural sâu (CNN kiểu cuda-convnet, giống kiến trúc Snoek et al. 2012), 3 tập dữ liệu:

Tập dữ liệu	R	η	Kết quả chính
CIFAR-10 (40k/10k/10k)	300	4	Hyperband nhanh hơn đối thủ **hơn một bậc độ lớn** (order of magnitude)
MRBI (10k/2k/50k)	300	4	Hyperband nhanh hơn configuration-selection thuần hơn 1 bậc, nhanh hơn SMAC (early) $5\times$
SVHN (~600k/6k/26k)	600	4	SMAC (early, Domhan et al. 2015) **vượt** Hyperband; BayesOpt cạnh tranh được

Trên cả 3 tập, Hyperband và "SMAC (early)" là hai phương pháp duy nhất luôn vượt random $2\times$; Hyperband nhanh hơn random search thuần hơn $20\times$. Toàn bộ các thí nghiệm này tốn tương đương **hơn 1 năm GPU-giờ** trên NVIDIA GRID K520 (Amazon EC2 g2.8xlarge). Khi huấn luyện lại cấu hình tốt nhất do Hyperband tìm được trên dữ liệu train+validation gộp, 300 epoch, đạt lỗi test 17,0% trên CIFAR-10 (so với baseline chuyên gia 18% và kết quả tối ưu hoá siêu tham số công bố trước đó: 15,0% của Snoek et al. 2012 (paper chú thích **không tái lập được** con số 15,0% này dù đã nhận siêu tham số tối ưu qua liên hệ cá nhân với tác giả) và 17,2% của Domhan et al. 2015; các kết quả 15,0%/17,2% dùng nhiều dữ liệu hơn 25% và huấn luyện nhiều epoch hơn nên không so sánh ngang hàng trực tiếp).

4.2 Data set subsampling. (a) 117 tập dữ liệu OpenML (khung thực nghiệm của Feurer et al. 2015, 110 siêu tham số gộp 15 classifier + 14 phương pháp tiền xử lý feature + 4 phương pháp tiền xử lý dữ liệu): trên toàn bộ 117 tập, Hyperband vượt random search về test-error rank nhưng **kém hơn** về validation-error rank (dấu hiệu BayesOpt overfit lên validation set); **random** $2\times$ **vượt tất cả phương pháp khác** trên tập đầy đủ 117; trên tập con 21 tập dữ liệu có tốc độ tăng tốc rõ khi subsampling, Hyperband vượt mọi đối thủ kể cả random $2\times$ (chênh lệch nhỏ). (b) Kernel regularized least-squares classification trên CIFAR-10 ($R=400, \eta=4$): Hyperband đạt cấu hình tốt chỉ sau bracket SH đầu tiên (~20 phút), nhanh hơn BayesOpt hơn $30\times$ và random

search $70\times$; trong khoảng thời gian đó Hyperband đánh giá được hơn 250 cấu hình trong khi đối thủ chỉ đánh giá được 3.

4.3 Feature subsampling (random feature xấp xỉ kernel RBF, $R = 1000, \eta = 4$, không gian tìm kiếm chỉ 3 chiều): Hyperband nhanh hơn BayesOpt và random search khoảng $6\times$, mức tăng tốc nhỏ hơn hẳn các thí nghiệm khác vì không gian siêu tham số thấp chiều (dễ bao phủ bằng cả random search và BayesOpt).

5. Phân tích / ablation

Yếu tố ảnh hưởng tốc độ tăng tốc (Mục 4.4.1, do paper tự phân tích): (1) cách thời gian huấn luyện scale theo tài nguyên: nếu siêu tuyến tính (bậc $p > 1$, như kernel least-squares scale bậc 2), tốc độ tăng tốc tối đa tăng theo hệ số $\frac{\eta^{p-1}-1}{\eta^p-1}R$, giải thích vì sao thí nghiệm kernel RLS đạt $70\times$ cao hơn mức tuyến tính dự kiến; (2) chi phí overhead cố định (khởi tạo model, tính validation), giải thích vì sao thí nghiệm 117-datasets không tăng tốc nhiều (nhiều tập dữ liệu huấn luyện xong trong vài giây, overhead chiếm phần lớn); (3) độ khó bài toán (số chiều không gian tìm kiếm): bài toán "dễ" (cấu hình ngẫu nhiên hay tốt) ít cần Hyperband hơn bài toán "khó" nhiều chiều, giải thích vì sao thí nghiệm feature-subsampling 3 chiều chỉ tăng tốc $6\times$ còn mạng neural (nhiều chiều hơn) tăng tốc hơn 1 bậc độ lớn.

So sánh với SuccessiveHalving (bracket đơn, Mục 4.4.2): ngoại trừ thí nghiệm LeNet và 117-datasets, bracket aggressive nhất ($s = s_{\max}$, lặp lại nhiều lần) **vượt** Hyperband đầy đủ trong mọi thí nghiệm còn lại: nghĩa là "biết trước" nên chọn bracket nào sẽ tốt hơn chạy tất cả bracket, nhưng vì không biết trước bracket tốt nhất nên Hyperband phải thử hết (trả giá bằng hệ số $s_{\max} + 1$ overhead). Đây là điểm paper tự nhận "nhìn lại thì nên chỉ chạy bracket $s = 4$" cho các bài toán cụ thể này, nhưng nhấn mạnh **không biết trước** điều đó khi bắt đầu.

Finite horizon vs infinite horizon (Mục 3.5, 5.4): khi R đã biết, finite horizon Hyperband tốt hơn; thí nghiệm CIFAR-10 cho thấy infinite horizon Hyperband (phải tự ước lượng R dần) cạnh tranh được nhưng **không** bằng finite horizon trong cùng ngân sách $50R$.

Phân tích lý thuyết SuccessiveHalving vs. uniform allocation (Mục 5.3): với tham số hoá $\gamma(j) \sim j^{-1/\alpha}$ (tốc độ hội tụ) và $F(x) \sim (x - \nu_*)^\beta$ (độ hiếm của cấu hình tốt), ngân sách cần của uniform allocation scale như $\Delta^{-(\alpha+\beta)}$ còn SuccessiveHalving scale như $\Delta^{-\max\{\alpha,\beta\}}$, **tốt hơn hẳn** khi $\Delta \to 0$ (cần độ chính xác cao). Hyperband (không biết α, β trước) chỉ tốn thêm hệ số log so với SuccessiveHalving tối ưu (biết trước n, B lý tưởng) (Định lý 5, Mục 5.3.3), và trong trường hợp đặc biệt K-armed bandit ngẫu nhiên (Hệ quả 6), kết quả khớp với chặn trên/dưới đã biết trong lý thuyết bandit (Karnin et al. 2013; Kaufmann et al. 2015) tới hệ số log.

6. Giới hạn

Do tác giả tự nêu (Mục 4.4.3, Mục 6):

- **Siêu tham số phụ thuộc tài nguyên** (ví dụ độ sâu tối đa của random forest nên tăng theo lượng dữ liệu): trong trường hợp này, hiệu năng ở tài nguyên nhỏ không phản ánh tốt hiệu năng ở tài nguyên lớn, làm giảm lợi ích của early-stopping; paper khuyến nghị tìm siêu tham số thay thế ít phụ thuộc tài nguyên hơn (ví dụ "minimum samples per leaf" thay cho "max depth"), hoặc chuẩn hoá (ví dụ chia hệ số regularization theo kích thước tập huấn luyện).
- **Không biết trước bracket nào tốt nhất**, nên phải trả giá overhead chạy hết các bracket (Mục 4.4.2, như đã nêu ở Mục 5 của ghi chú này).
- **Song song hoá** còn là hướng mở: cách đơn giản nhất (phân brackets cho các máy khác nhau) có nhược điểm số lượng tác vụ giảm theo cấp số nhân khi cấu hình bị loại dần, cần hàng đợi ưu tiên phức tạp hơn.
- **Tốc độ hội tụ khác nhau giữa các họ mô hình** (ví dụ mạng sâu và cây quyết định cùng tham gia một lần tìm kiếm) chưa được xử lý tốt: nếu một họ mô hình hội tụ chậm hơn nhưng cho kết quả tốt hơn, Hyperband có thể loại nhầm nó sớm.

- Chưa tận dụng **cấu trúc tương quan giữa các cấu hình** (Hyperband coi mỗi cấu hình độc lập, chỉ định danh bằng chỉ số, không dùng thông tin hyperparameter cụ thể): kết hợp với configuration selection thích ứng (BayesOpt) là hướng để trong tương lai (paper trích dẫn Klein et al. 2017b làm được điều này **sau khi** bài Hyperband này được nộp).
- Footnote 9: tác giả **không tái lập được** kết quả 15,0% lỗi test CIFAR-10 của Snoek et al. 2012 dù nhận siêu tham số tối ưu qua liên hệ cá nhân.

Do chúng ta nhận thấy:

- Thí nghiệm 4.1 (mạng neural) tốn hơn 1 năm GPU-giờ cho tất cả trials gộp lại: bài báo là một benchmark rất tốn kém, không dễ tái lập đầy đủ trên hạ tầng nhỏ (1 GPU 8GB); các con số tăng tốc nên được hiểu theo tỉ lệ tương đối, không kỳ vọng tái lập chính xác tuyệt đối trên phần cứng khác.
- Trong thí nghiệm 117-datasets, chính **random** $2\times$ (không phải Hyperband hay BayesOpt) mới là phương pháp mạnh nhất về test-error rank trên toàn bộ 117 tập: đây là một kết quả "khiêm tốn hoá" quan trọng mà paper trình bày trung thực (không giấu), cần nhớ khi đánh giá độ ưu việt tổng thể của Hyperband.

7. Liên hệ với các bài khác

- **opt 32/opt 33** (Bayesian optimization, GP + Expected Improvement/Knowledge Gradient/Entropy Search): đây chính là nhóm phương pháp "configuration selection" mà Hyperband dùng làm đối thủ so sánh trực tiếp (SMAC, TPE, Spearmint); Hyperband thắng rõ trên hầu hết thí nghiệm có không gian tìm kiếm chiều cao nhưng thua trên SVHN và trên tập 117-datasets đầy đủ, cho thấy hai hướng (configuration selection vs. evaluation) bù trừ lẫn nhau tuỳ bài toán. opt 33 (Mục 5, multi-fidelity) có nhắc tới ý tưởng gần giống Hyperband (dùng fidelity thấp để tiết kiệm chi phí) nhưng theo hướng GP/KG, không phải bandit.
- **opt 34** (Population Based Training): PBT thích ứng **trong một lần huấn luyện duy nhất** (online, không chạy lại từ đầu), còn Hyperband vẫn chạy nhiều lần huấn luyện độc lập song song/tuần tự (dù có early-stopping); PBT tự liệt các phương pháp sequential optimisation (gồm BayesOpt) nhưng không so sánh trực tiếp với Hyperband.
- **opt 28/opt 29/opt 30** (benchmark optimizer: so sánh thực nghiệm nhiều optimizer trên nhiều bài toán, độ nhạy với protocol tuning): cùng tinh thần phương pháp luận với Mục 4 của Hyperband (benchmark nhiều phương pháp HPO trên nhiều bài toán, có baseline random search/random $2\times$); bài học chung là benchmark HPO rất nhạy với lựa chọn baseline và protocol (chính Hyperband cũng gặp trường hợp random $2\times$ thắng ở 117-datasets).
- **math 35** (A Tutorial on Thompson Sampling, theo titles.tsv, thư mục "math content/"): cùng gốc lý thuyết multi-armed bandit/pure-exploration mà Mục 5 của Hyperband dựa vào (Karnin et al. 2013; Kaufmann et al. 2015; Carpentier & Valko 2015).

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, chạy trên GPU <= 8GB, batch 1). Nhận định dưới đây là suy luận:

1. **Đây là một trong những bài liên quan trực tiếp nhất** tới việc tune siêu tham số rẻ tiền trên 1 GPU: "resource" = số bước/epoch huấn luyện fine-tune hoặc distillation khớp hoàn toàn với ví dụ LeNet/CNN của paper (Mục 3.3, 4.1). Có thể dùng Hyperband để dò learning rate, hệ số KD loss, tỉ lệ sparsity mục tiêu... bằng cách chỉ huấn luyện đầy đủ (resource R) cho số cấu hình ít nhất đã "sống sót" qua các vòng loại, tiết kiệm đáng kể GPU-giờ so với thử từng cấu hình tới khi hội tụ.
2. **Nguy cơ "siêu tham số phụ thuộc tài nguyên"** (Mục 4.4.3, Mục 6) rất đáng lưu ý cho dự án: nếu đang so sánh các **tỉ lệ sparsity/pruning** khác nhau (một dạng "siêu tham số"), chất lượng ở số bước huấn

luyện nhỏ (ít resource) có thể **không phản ánh đúng thứ hạng** ở số bước lớn (mô hình pruned nặng có thể cần nhiều bước phục hồi hơn mới "đuổi kịp" mô hình pruned nhẹ): đây chính là vấn đề mà các bài trong thư mục "pruning content/" và "distillation content/" (ngoài phạm vi 8 bài này, suy luận) có thể phải cân nhắc khi kết hợp với early-stopping kiểu Hyperband; nên kiểm tra tương quan rank giữa resource thấp và resource cao trước khi áp dụng.

3. **Data Set Subsampling** (Mục 3.4) có thể áp dụng khi tune trên một tập nhỏ các trang OCR mẫu trước khi scale lên full dataset, miễn là thời gian huấn luyện scale hợp lý theo kích thước dữ liệu.
4. **Kết quả 117-datasets (Mục 4.2.1)** là lời nhắc quan trọng: với không gian siêu tham số không quá phức tạp hoặc ngân sách GPU rất hạn chế (8GB, batch 1), **random search với ngân sách gấp đôi** có thể cạnh tranh hoặc vượt cả Hyperband và BayesOpt, nên luôn so sánh với baseline random $2\times$ đơn giản trước khi đầu tư triển khai Hyperband/BayesOpt phức tạp hơn.

9. Thí nghiệm gợi ý

1. **Chạy SuccessiveHalving đơn giản để tune 2-3 siêu tham số của một bước fine-tune/distill nhỏ** (learning rate, hệ số KD loss) trên GPU 8GB, dùng số bước huấn luyện làm resource (R = vài nghìn bước, $\eta = 3$ hoặc 4). Đo tổng GPU-giờ cần để tìm cấu hình tốt, so với thử tuần tự n cấu hình tới khi hội tụ đầy đủ.
2. **Chạy đầy đủ Hyperband (nhiều bracket) trên cùng bài toán ở thí nghiệm 1**, so sánh với chạy riêng bracket aggressive nhất ($s = s_{\max}$) lặp lại nhiều lần. Kỳ vọng theo Mục 5 của ghi chú này: bracket đơn có thể thắng nếu biết trước bài toán "dễ phân biệt sớm", nhưng Hyperband đầy đủ an toàn hơn khi không biết trước.
3. **Kiểm tra tương quan rank giữa resource thấp và resource cao** cho một siêu tham số nhạy với tài nguyên (ví dụ tỉ lệ sparsity pruning): huấn luyện 5-10 cấu hình ở 1/8 ngân sách và ở ngân sách đầy đủ, tính Spearman correlation giữa 2 lần xếp hạng. Nếu tương quan thấp, đây là tín hiệu cảnh báo (Mục 6) rằng Hyperband/SH có thể loại nhầm cấu hình tốt.
4. **So sánh Hyperband với random search** $2\times$ **ngân sách** trên cùng bài toán tune nhỏ. Đo test/validation error cuối cùng. Kỳ vọng dựa theo Mục 4.2.1: nếu không gian tìm kiếm đơn giản hoặc có nhiều overhead cố định, random $2\times$ có thể cạnh tranh ngang hoặc vượt Hyperband.
5. **Thử Data Set Subsampling**: tune một mô hình nhỏ dùng kích thước tập OCR mẫu làm resource (ví dụ 1k, 2k, 4k, 8k trang), so với dùng số bước huấn luyện làm resource; so sánh tốc độ tăng tốc giữa hai cách định nghĩa "tài nguyên" trên cùng ngân sách GPU-giờ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
SuccessiveHalving	cấp đều B/n cho n cấu hình, loại nửa kém, lặp tới khi còn 1
Hyperband (finite horizon)	$s_{\max} = \lfloor \log_\eta(R) \rfloor$, $B = (s_{\max}+1)R$, chạy $s_{\max}+1$ bracket SH
Bracket s	$n = \lceil \frac{B}{R}\frac{\eta^s}{s+1} \rceil$, $r = R\eta^{-s}$
Ví dụ $R = 81, \eta = 3$	$s_{\max} = 4$; bracket s=4: $(n, r) = (81, 1) \to (1, 81)$; bracket s=0 = random search thuần
Tốc độ tăng tốc tối đa (tuyến tính)	$R/(s_{\max}+1)$; với $R = 81, \eta = 3$: $\approx 16{,}2\times$ (tự tính)
NIAB (bài toán bandit)	arm i = cấu hình; $\nu_i = \lim_{k\to\infty} \ell_{i,k}$; mục tiêu tìm $\min \nu_i$ với ít "pulls" nhất
Tham số hoá lý thuyết	$\gamma(j) \sim j^{-1/\alpha}$ (tốc độ hội tụ), $F(x) \sim (x - \nu_*)^\beta$ (độ hiếm cấu hình tốt)
So sánh ngân sách cần (tiệm cận $\Delta \to 0$)	Uniform: $\Delta^{-(\alpha+\beta)}$; SuccessiveHalving: $\Delta^{-\max\{\alpha,\beta\}}$

Khái niệm	Công thức / giá trị
Kết quả thực nghiệm chính	nhanh hơn BayesOpt $5\times$-$30\times$ (CNN), $30\times$-$70\times$ (kernel RLS); $6\times$ (feature subsampling)

11. Kết luận ngắn

Hyperband giải quyết bài toán ”n versus B/n” của các phương pháp halving trước đó bằng cách chạy nhiều bracket SuccessiveHalving song song, mỗi bracket ứng với một đánh đổi $n/B/n$ khác nhau, đóng khung chặt chẽ như bài toán pure-exploration non-stochastic infinite-armed bandit với bảo đảm lý thuyết chỉ kém $O(\log)$ so với SuccessiveHalving tối ưu (biết trước n, B lý tưởng). Thực nghiệm cho thấy tăng tốc đáng kể (thường hơn một bậc độ lớn) so với BayesOpt (SMAC/TPE/Spearmint) trên nhiều bài toán deep learning và kernel method, nhưng kết quả không đồng nhất: SVHN và tập 117-datasets đầy đủ là hai trường hợp BayesOpt hoặc random $2\times$ đơn giản vẫn thắng, một điểm trung thực quan trọng của paper. Với một pipeline tune trên GPU 8GB, ý tưởng cốt lõi (dùng resource trung gian để loại sớm cấu hình kém) trực tiếp áp dụng được, nhưng cần cẩn trọng với siêu tham số ”phụ thuộc tài nguyên” (ví dụ tỉ lệ sparsity) nơi rank ở resource thấp có thể không phản ánh đúng rank ở resource cao.

Nguồn gốc: https://arxiv.org/abs/1603.06560

Bài 32

Practical Bayesian Optimization of Machine Learning Algorithms (2012)

Tác giả: Jasper Snoek, Hugo Larochelle, Ryan P. Adams (University of Toronto, Université de Sherbrooke, Harvard University).
Venue / arXiv: arXiv:1206.2944 (v2, 29/08/2012); paper không nêu venue hội nghị trong chính văn bản (không có dòng ”Advances in Neural Information Processing Systems...”; bài này được biết rộng rãi là NeurIPS/NIPS 2012 nhưng đây là nhận xét ngoài paper, không lấy từ text).
Nguồn: https://arxiv.org/abs/1206.2944
Chủ đề: dùng Bayesian optimization với Gaussian Process (GP) để tự động tune siêu tham số của thuật toán ML; ba đóng góp chính: xử lý Bayes đầy đủ cho siêu tham số của kernel GP (tích hợp bằng MCMC), mô hình hoá chi phí (thời gian) của mỗi lần đánh giá, và một thuật toán song song hoá dựa trên Monte Carlo cho các đánh giá đang chờ kết quả.
Vị trí trong lộ trình: đọc cùng nhóm với opt 31 (Hyperband, phương pháp bandit thay thế không cần GP) và opt 33 (A Tutorial on Bayesian Optimization của Frazier, nền lý thuyết tổng quát hơn cho acquisition function); nên đọc bài này trước hoặc sau opt 33 tuỳ ý, vì hai bài bổ sung cho nhau: bài này là ứng dụng thực hành (2012, sớm hơn), opt 33 là tutorial lý thuyết đầy đủ hơn (2018). Có thể đọc sau opt 06 đến opt 09 (Adam và các biến thể) để thấy một hướng ”tune bằng tay/bằng luật” đối lập với hướng tune tự động bằng BO ở đây.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Hầu hết thuật toán ML không ”không tham số”: có siêu tham số của regularizer, hyperprior của mô hình sinh, hay learning rate của một thủ tục tối ưu bậc gradient. Việc chọn các ”nút xoay” (knobs) này ảnh hưởng mạnh tới hiệu năng tổng quát hoá, nhưng theo paper, việc tune này thường là ”black art” dựa vào kinh nghiệm chuyên gia, luật ngầm, hoặc brute-force search.

Paper đề xuất xem việc tune siêu tham số như tối ưu một **hàm hộp đen** (black-box function) phản ánh hiệu năng tổng quát hoá, và áp dụng các thuật toán tối ưu hộp đen đã có: **Bayesian optimization** (Mockus, Tiesis, Zilinskas, 1978). Khác với tối ưu hộp đen thông thường, bài toán tune ML có hai đặc điểm riêng mà paper nhấn mạnh:

1. Mỗi lần đánh giá hàm có **thời gian khác nhau** (huấn luyện mạng 10 unit ẩn nhanh hơn mạng 1000 unit ẩn); cùng với điện toán đám mây, chi phí tiền bạc cũng khác nhau theo cấu hình.
2. Thí nghiệm ML thường chạy **song song** trên nhiều core/máy, nên muốn thuật toán BO tận dụng được song song hoá.

Bayesian optimization dùng Gaussian Process (GP) làm **prior** trên hàm mục tiêu, duy trì một **posterior** sau mỗi lần quan sát, và dùng một **acquisition function** để quyết định điểm tiếp theo cần thử, nhằm giảm số lần đánh giá hàm thật (rất đắt vì phải huấn luyện mô hình ML đến cuối).

2. Ý tưởng chính

Ba đóng góp thực hành (Mục 1, phần giới thiệu):

1. **Xử lý Bayes đầy đủ cho siêu tham số của kernel GP** (không chỉ tối ưu điểm, ”point estimate”): paper lập luận và thực nghiệm cho thấy việc tích hợp (marginalize) các siêu tham số của GP bằng Markov

chain Monte Carlo (MCMC) quan trọng hơn việc chọn đúng loại kernel.

2. **Mô hình hoá chi phí**: thêm một GP thứ hai cho log thời gian chạy $\ln c(x)$, rồi tối ưu "expected improvement mỗi giây" (EI per second) thay vì EI thường, để ưu tiên các điểm vừa có khả năng tốt vừa nhanh đánh giá.
3. **Song song hoá qua Monte Carlo**: khi có J thí nghiệm đang chạy (pending), paper tính một ước lượng Monte Carlo của acquisition function kỳ vọng trên các kết quả "giả tưởng" (fantasies) của các thí nghiệm đang chờ, để chọn điểm mới không trùng với các điểm đang chạy.

Kết quả chính: các biến thể BO này tune siêu tham số **nhanh hơn và/hoặc tốt hơn** so với chuyên gia người và so với grid search/TPE (Tree Parzen Algorithm) trên ba bài toán ML khác nhau, kể cả vượt qua kết quả của chuyên gia trên CIFAR-10.

3. Phương pháp chi tiết

3.1 Bayesian optimization với GP prior (Mục 2)

Mục tiêu: tìm $\min_{x\in\mathcal{X}} f(x)$, $\mathcal{X}\subset\mathbb{R}^D$. BO xây một mô hình xác suất cho f và dùng nó để chọn điểm đánh giá tiếp theo, tận dụng toàn bộ lịch sử quan sát (không chỉ gradient/Hessian cục bộ).

Gaussian Process (Mục 2.1): GP là phân phối tiên nghiệm trên hàm $f:\mathcal{X}\to\mathbb{R}$, định nghĩa bởi: với mọi tập hữu hạn N điểm $\{x_n\}_{n=1}^N$, các giá trị $\{f(x_n)\}$ tuân theo phân phối Gauss đa biến. GP được xác định bởi hàm trung bình m và hàm hiệp phương sai (kernel) K.

Acquisition function (Mục 2.2): giả sử $f\sim$ GP, quan sát $\{x_n,y_n\}_{n=1}^N$ với $y_n\sim\mathcal{N}(f(x_n),\nu)$. Hàm acquisition $a:\mathcal{X}\to\mathbb{R}_+$ chọn điểm tiếp theo $x_{next}=\arg\max_x a(x)$. Ba lựa chọn phổ biến, viết qua $\mu(x)$ (trung bình dự đoán) và $\sigma(x)$ (độ lệch chuẩn dự đoán), với $x_{best}=\arg\min_{x_n} f(x_n)$, Φ,ϕ là CDF/PDF chuẩn tắc:

- **Probability of Improvement** (Kushner, 1964; công thức 1): $a_{PI}(x)=\Phi(\gamma(x))$, với $\gamma(x)=\dfrac{f(x_{best})-\mu(x)}{\sigma(x)}$.
- **Expected Improvement** (EI, công thức 2): $a_{EI}(x)=\sigma(x)\big(\gamma(x)\Phi(\gamma(x))+\mathcal{N}(\gamma(x);0,1)\big)$.
- **GP Upper/Lower Confidence Bound** (Srinivas et al. 2010; công thức 3): $a_{LCB}(x)=\mu(x)-\kappa\,\sigma(x)$, cần tự chọn κ để cân bằng exploit/explore.

Paper chọn **EI** làm acquisition chính vì nó "hành xử tốt hơn" probability of improvement và không cần tham số tự chọn như κ của GP-UCB (dù tác giả có so sánh trực tiếp EI với GP-UCB ở Mục 4.1).

Ví dụ số (tự tính): giả sử tại một điểm x, GP dự đoán $\mu(x)=8$, $\sigma(x)=2$, và giá trị tốt nhất quan sát được là $f(x_{best})=10$ (bài toán minimize). Khi đó $\gamma(x)=(10-8)/2=1$. Dùng giá trị chuẩn $\Phi(1)\approx 0{,}8413$, $\phi(1)\approx 0{,}2420$: $a_{EI}(x)=2\times(1\times 0{,}8413+0{,}2420)=2\times 1{,}0833\approx 2{,}1666$ (tự tính, đơn vị cùng đơn vị với f). Nếu $\mu(x)=12>f(x_{best})$ (điểm có vẻ tệ hơn best hiện tại), $\gamma(x)=(10-12)/2=-1$, $\Phi(-1)\approx 0{,}1587$, $\phi(-1)=\phi(1)\approx 0{,}2420$, $a_{EI}=2\times(-1\times 0{,}1587+0{,}2420)=2\times 0{,}0833\approx 0{,}1666$ (tự tính): EI vẫn dương nhưng nhỏ hơn nhiều, phản ánh "còn ít kỳ vọng cải thiện" nhưng không bằng 0 vì vẫn còn bất định σ.

3.2 Covariance function và xử lý Bayes cho siêu tham số GP (Mục 3.1)

Kernel mặc định thường dùng là **ARD squared exponential** (công thức 4):

$$K_{SE}(x,x')=\theta_0\exp\Big(-\tfrac{1}{2}r^2(x,x')\Big),\quad r^2(x,x')=\sum_{d=1}^{D}(x_d-x'_d)^2/\theta_d^2.$$

Nhưng hàm mẫu từ kernel này ”mượt một cách không thực tế” cho bài toán tối ưu thực hành. Paper đề xuất **ARD Matérn 5/2** (công thức 5):

$$K_{M52}(x,x') = \theta_0\Big(1+\sqrt{5r^2(x,x')}+\tfrac{5}{3}r^2(x,x')\Big)\exp\big(-\sqrt{5r^2(x,x')}\big),$$

cho hàm mẫu khả vi hai lần (giả định tương ứng với các phương pháp quasi-Newton) mà không yêu cầu độ mượt vô hạn của squared exponential.

Với D chiều đầu vào, có $D+3$ siêu tham số GP: D length-scale $\theta_{1:D}$, biên độ θ_0, nhiễu quan sát ν, và trung bình hằng m. Cách thường làm là tối ưu hợp lý biên (marginal likelihood) để lấy một điểm ước lượng. Paper thay vào đó đề xuất **tích hợp acquisition function** trên toàn bộ phân phối hậu nghiệm của siêu tham số (công thức 6):

$$\hat{a}(x;\{x_n,y_n\}) = \int a(x;\{x_n,y_n\},\theta)\,p(\theta\mid\{x_n,y_n\})\,d\theta,$$

ước lượng bằng Monte Carlo qua các mẫu θ lấy bằng **slice sampling** (Murray & Adams, 2010). Chi phí tính toán của MCMC bị chi phối bởi cùng phép toán khối $O(N^3)$ (giải hệ tuyến tính N chiều) như khi tối ưu GP, nên cách làm Bayes đầy đủ này ”hợp lý” khi hàm mục tiêu thật còn đắt hơn nhiều. Phương pháp này được gọi là **GP EI MCMC**, đối lập với **GP EI Opt** (tối ưu điểm, point estimate).

3.3 Mô hình hoá chi phí (Mục 3.2)

Mục tiêu thực tế là tìm siêu tham số tốt càng nhanh (theo **wall-clock time**) càng tốt, không chỉ ít **số lần đánh giá**. Paper mô hình thêm hàm thời gian chạy $c(x):\mathcal{X}\to\mathbb{R}_+$ bằng một GP khác trên $\ln c(x)$ (giả định độc lập với $f(x)$ trong bài này, dù ghi nhận có thể mô hình đồng thời bằng multi-task GP). Từ đó tối ưu **expected improvement mỗi giây**:

$$a_{EI/sec}(x) = \frac{a_{EI}(x)}{\mathbb{E}[c(x)]} \quad \text{(suy ra từ Mục 3.2; paper viết là ”expected improvement per second”).}$$

Ý tưởng tổng quát hoá được sang chi phí khác (hoá chất, tiền).

3.4 Song song hoá bằng Monte Carlo (Mục 3.3)

Với N quan sát hoàn tất $\{x_n,y_n\}$ và J thí nghiệm đang chạy tại $\{x_j\}_{j=1}^J$ (chưa có kết quả), paper muốn chọn điểm mới không lặp lại điểm đang chạy. Vì ”roll-out” chính sách acquisition là bất khả thi, paper dùng chiến lược: tính kỳ vọng của $a(x)$ trên mọi kết quả khả dĩ $\{y_j\}$ của các thí nghiệm đang chờ (công thức 7):

$$\hat{a}(x;\{x_n,y_n\},\theta,\{x_j\}) = \int_{\mathbb{R}^J} a\big(x;\{x_n,y_n\},\theta,\{x_j,y_j\}\big)\,p\big(\{y_j\}\mid\{x_j\},\{x_n,y_n\}\big)\,dy_1\cdots dy_J,$$

tức kỳ vọng của $a(x)$ dưới một phân phối Gauss J chiều (trung bình/hiệp phương sai GP tính được trực tiếp). Mỗi mẫu ”giả tưởng” ($\{y_j\}$ lấy mẫu từ phân phối dự đoán tại các điểm đang chờ) gọi là một **fantasy**; lấy trung bình trên nhiều fantasy cho ước lượng Monte Carlo của $\hat{a}(x)$. Ký hiệu: $N\times$ GP EI MCMC nghĩa là chạy N bản song song của chiến lược này.

Paper ghi nhận một hướng tương tự được Ginsbourger và Le Riche (2010) nhắc tới nhưng bị coi là ”quá bất khả thi để đáng chú ý”; ở đây tác giả cho thấy Monte Carlo fantasy là ”rất hiệu quả trong thực hành”.

4. Thí nghiệm và kết quả chính

Paper không có một benchmark suite chuẩn hoá lớn (không giống các bài benchmark optimizer khác trong cùng thư mục như opt 28, opt 29); các thí nghiệm là 4 bài toán ML cụ thể, mỗi bài so sánh vài chiến lược BO với baseline (grid search, TPE, hoặc chuyên gia người). Tất cả thí nghiệm chạy trên máy giống nhau qua Amazon EC2 (chú thích 1, Mục 4).

Tên viết tắt (Mục 4): **GP EI MCMC** (EI tích hợp MCMC trên siêu tham số GP), **GP EI Opt** (EI với point-estimate siêu tham số), **GP EI per Second**, **N x GP EI MCMC** (song song N lần).

4.1 Branin-Hoo và logistic regression trên MNIST (Mục 4.1)

- Hàm Branin-Hoo: benchmark chuẩn cho BO, $x \in \mathbb{R}^2$, $0 \le x_1 \le 15$, $-5 \le x_2 \le 15$ (Jones, 2001).
- Logistic regression trên MNIST: 4 siêu tham số: learning rate SGD (log scale 0 đến 1), hệ số regularization ℓ_2 (0 đến 1), minibatch size (20 đến 2000), số epoch (5 đến 2000).
- Chạy Branin-Hoo 100 lần, logistic regression 10 lần, báo cáo trung bình và sai số chuẩn (standard error).
- Kết quả: tích hợp trên siêu tham số (GP EI MCMC) tốt hơn point-estimate (GP EI Opt); GP EI MCMC vượt rõ TPA (Tree Parzen Algorithm, Bergstra et al. 2011), tìm minimum trong **ít hơn một nửa số lần đánh giá** so với TPA, ở cả hai bài toán.

4.2 Online LDA trên Wikipedia (Mục 4.2)

- Online LDA (Hoffman et al. 2010): 2 siêu tham số học τ_0, κ (chi phối learning rate $\rho_t = (\tau_0 + t)^{-\kappa}$) và kích thước minibatch (tổng 3 siêu tham số).
- Dữ liệu: 249.560 bài Wikipedia ngẫu nhiên, chia train/validation/test = 200.000 / 24.560 / 25.000; từ vựng 7.702 từ. Số topic cố định 100, $\eta = \alpha = 0{,}01$ (prior Dirichlet đối xứng).
- Hoffman et al. (2010) dùng grid search kích thước $6 \times 6 \times 8 = 288$ cấu hình. Mỗi lần chạy mất 5 đến 10 giờ để hội tụ, nên: $288 \times 5/24 = 60$ ngày xử lý và $288 \times 10/24 = 120$ ngày xử lý (tự tính, khớp đúng với con số paper nêu "approximately 60 to 120 processor days").
- BO bị giới hạn đánh giá trên **đúng grid** này (để so sánh công bằng), lặp lại 100 lần (mỗi lần chọn 2 điểm ngẫu nhiên khác nhau để khởi tạo).
- Kết quả: tích hợp siêu tham số GP vẫn tốt hơn point-estimate. **GP EI MCMC** hiệu quả nhất theo số lần đánh giá, nhưng **GP EI MCMC song song** (3x, 5x, 10x) tìm bộ siêu tham số tốt nhất trong **thời gian wall-clock ít hơn nhiều**. Khi được phép chọn điểm **ngoài grid** (off-grid), các mô hình song song hoá tìm được giá trị nhỏ hơn (tốt hơn) so với kết quả grid search gốc của Hoffman et al., trong khi chỉ chạy một phần nhỏ số thí nghiệm.

4.3 Motif finding với M3E / structured SVM (Mục 4.3)

- Mô hình Max-Margin Min-Entropy (M3E, Miller et al. 2012), tổng quát hoá Latent Structured SVM (Yu & Joachims, 2009), áp dụng cho phân loại nhị phân chuỗi protein-DNA, tìm vị trí "motif" ẩn.
- Thí nghiệm trên 1 protein với khoảng 40.000 chuỗi (xấp xỉ theo text). Lưới tham số: 25 giá trị C (log scale 10^{-1} đến 10^{6}), 14 giá trị α (log scale 0,1 đến 5), và ngưỡng hội tụ $\epsilon \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ (4 giá trị).
- Tổng số cấu hình lưới: $25 \times 14 \times 4 = 1400$ (tự tính, khớp với số paper nêu "1,400 possible combinations"). Mỗi cấu hình đánh giá trên 5 lần chia train/test 50-50 ngẫu nhiên.
- So sánh randomized grid search với GP EI MCMC, GP EI per Second, và bản song song 3x của chúng, lặp lại 100 lần. BO hiệu quả hơn rõ rệt grid search. GP EI MCMC tốt hơn GP EI per Second theo **số lần đánh giá**, nhưng GP EI per Second nhanh hơn theo **wall-clock** (nó học cách dùng ngưỡng hội tụ ϵ lớn hơn, kém chặt hơn, ở giai đoạn đầu để khám phá các tham số khác nhanh hơn). 3x GP EI per Second kém hiệu quả nhất theo số lần đánh giá nhưng tìm tham số tốt nhanh nhất theo wall-clock trong số tất cả.
- So sánh kernel (Hình 5c): Matérn 5/2 ARD và Matérn 3/2 ARD tốt hơn squared exponential (có/không ARD); giả định khả vi vô hạn của squared exponential "quá hạn chế" cho bài toán này.

4.4 CNN trên CIFAR-10 (Mục 4.4)

- Mạng CNN 3 tầng theo kiến trúc Krizhevsky (2009), tune 9 siêu tham số: số epoch, learning rate, 4 hệ số weight cost (3 tầng + softmax output), và độ rộng/scale/power của response normalization ở tầng pooling.

- Baseline chuyên gia (Krizhevsky, 2009): **18% lỗi test**, khớp với state of the art lúc đó (Coates & Ng, 2011, không data augmentation).
- Mỗi chiến lược chạy 5 lần với khởi tạo ngẫu nhiên khác nhau, báo cáo trung bình và sai số chuẩn trên validation.
- Kết quả: siêu tham số tốt nhất tìm bởi **GP EI MCMC** đạt **14,98% lỗi test**, tốt hơn chuyên gia/SOTA hơn 3 điểm phần trăm. Tự tính: $18 - 14{,}98 = 3{,}02$ điểm phần trăm (khớp với câu paper "over 3% better"); cải thiện tương đối $(18 - 14{,}98)/18 \approx 16{,}8\%$ (tự tính).
- Chú thích 6: các siêu tham số tối ưu lệch khá nhiều so với cấu hình chuyên gia: hệ số weight cost bất đối xứng (tầng 2 nhỏ hơn tầng 1 khoảng một bậc độ lớn), learning rate nhỏ hơn hai bậc độ lớn so với chuyên gia, response normalization rộng hơn một chút, scale lớn hơn, power nhỏ hơn nhiều.

5. Phân tích / ablation

- **Tích hợp Bayes so với point-estimate siêu tham số GP** là phát hiện lặp lại xuyên suốt cả 4 bài toán (Branin-Hoo/logistic, Online LDA, CIFAR-10): GP EI MCMC luôn tốt hơn hoặc bằng GP EI Opt, được paper nhấn mạnh là đóng góp quan trọng nhất ("của critical importance to robust results").
- **Lựa chọn kernel quan trọng**: Mục 4.3, Hình 5c cho thấy khác biệt rõ giữa squared exponential và Matérn; đây là một dạng "ablation" về giả định độ mượt của hàm mục tiêu.
- **Đánh đổi wall-clock và số lần đánh giá**: GP EI per Second và các bản song song luôn hiệu quả hơn về wall-clock nhưng không nhất thiết về số lần đánh giá (Mục 4.2, 4.3); đây là phân tích trade-off giữa 2 tiêu chí.
- **Off-grid so với on-grid** (Mục 4.2, Hình 4c): khi được chọn tự do thay vì bị ép vào lưới cố định, BO song song tìm điểm tốt hơn hẳn so với kết quả gốc của Hoffman et al., cho thấy một phần lợi ích đến từ không gian tìm kiếm liên tục, không chỉ từ chiến lược chọn điểm.
- Không có thí nghiệm ablation tách riêng phần "mô hình hoá chi phí" (EI per second) khỏi "song song hoá" một cách độc lập trên cùng một bài toán đơn giản; hai đóng góp này được trộn lẫn trong các thí nghiệm 4.2-4.4.

6. Giới hạn

Do tác giả gợi ý/thừa nhận (rải trong Mục 3, 4):

- Giả định $f(x)$ và $c(x)$ (thời gian chạy) độc lập khi mô hình hoá chi phí (Mục 3.2), trong khi tác giả tự nhận có thể dùng multi-task GP để mô hình đồng thời nhưng không làm trong paper này.
- MCMC/slice sampling và tối ưu GP cùng bị chi phối bởi chi phí $O(N^3)$ (giải hệ tuyến tính N điểm quan sát); paper không bàn cách scale khi N lớn (rất nhiều lần đánh giá).
- Mỗi bài toán thực nghiệm chỉ có một vài chiều siêu tham số (2 đến 9); không có thử nghiệm ở không gian rất nhiều chiều.
- Online LDA và motif-finding bị ràng buộc vào lưới cố định ở một phần thí nghiệm (để so sánh công bằng với Hoffman et al.), nên chưa thấy hết tiềm năng khi không gian liên tục hoàn toàn (trừ phần "off-grid" ở Mục 4.2).

Do chúng ta nhận thấy:

- Không có so sánh trực tiếp với Hyperband/successive halving (khi đó chưa tồn tại, Hyperband ra sau năm 2016, xem opt 31); paper giả định mọi lần đánh giá chạy "đến cuối" (tới hội tụ hoặc số epoch cố định), không có early-stopping thích ứng.
- Không báo cáo độ lệch chuẩn hoặc kiểm định thống kê chính thức (significance test) giữa các chiến lược, chỉ sai số chuẩn trên biểu đồ.
- Venue hội nghị không được paper tự nêu rõ trong bản PDF (không có dòng in tiêu đề hội nghị); chỉ biết số arXiv và năm.

- Quy mô bài toán (vài nghìn tới vài chục nghìn mẫu, mạng CNN nhỏ theo chuẩn 2012) rất nhỏ so với deep learning hiện đại; không rõ GP EI MCMC scale thế nào khi mỗi lần đánh giá tốn hàng giờ GPU hiện đại hoặc khi có hàng chục siêu tham số.

7. Liên hệ với các bài khác

- **opt 33** (A Tutorial on Bayesian Optimization, Frazier): mở rộng lý thuyết các acquisition function (Expected Improvement tổng quát hoá cho nhiễu, Knowledge Gradient, Entropy Search) mà bài này chỉ dùng phiên bản EI/PI/UCB cơ bản; nên đọc cặp với bài này để hiểu rộng hơn các lựa chọn khác ngoài EI.
- **opt 31** (Hyperband): cùng chủ đề ”tune siêu tham số”, nhưng Hyperband không dùng mô hình xác suất GP mà dùng successive halving/bandit, rẻ hơn về tính toán mỗi bước nhưng cần chạy được ”ngân sách một phần” (vd số epoch) của thuật toán học, điều mà BO ở đây không yêu cầu.
- Trong cùng thư mục, các bài benchmark optimizer (opt 28, opt 29) nói về việc **so sánh optimizer** công bằng theo protocol tuning; BO ở bài này chính là một trong các ”protocol tuning” nghiêm túc có thể dùng để tune optimizer khi so sánh, nên liên quan trực tiếp đến thông điệp ”tuning budget quyết định kết quả so sánh” của hai bài đó.
- Đã kiểm tra bằng `ls` các thư mục cũ (”llm content/”, ”rl content/”, ”distillation content/”, ”pruning content/”, ”interp content/”): không thấy bài nào về Bayesian optimization, Gaussian Process hoặc bandit hyperparameter search khớp trực tiếp (thư mục ”rl content/” có bài về search nhưng là ”53_2025_search_r1.md”, một kỹ thuật suy luận LLM, không liên quan tới GP/Bayesian optimization), nên không trỏ tới các thư mục đó ở đây.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, chạy trên GPU <= 8GB, batch 1). Các hướng pruning, distillation, RL, interpretability đã có thư mục riêng; dưới đây chỉ là ý kiến suy luận về nơi Bayesian optimization GP có thể giúp pipeline này:

1. **Khi mỗi lần ”đánh giá” rất đắt.** Nếu mỗi lần thử một cấu hình (tỉ lệ sparsity mỗi layer, hệ số trộn loss distillation λ_{KD}, learning rate fine-tune) đòi hỏi chạy phục hồi/distill nhiều giờ trên GPU 8GB, GP EI MCMC (hoặc đơn giản hơn: GP EI Opt) là một ứng viên hợp lý để tune 2 đến 5 siêu tham số liên tục, vì BO tối thiểu hoá số lần ”đánh giá đắt” cần thử (suy luận, dựa trên Mục 4 của paper nơi BO vượt grid search rõ rệt khi mỗi lần đánh giá tốn hàng giờ).
2. **EI per second có thể không cần thiết ở đây** vì trên 1 GPU không có nhiều lựa chọn ”song song hoá” như paper giả định (paper giả định có nhiều core/máy); với 1 GPU, lợi ích chính của mô hình hoá chi phí (ưu tiên cấu hình nhanh hội tụ) vẫn có ích nếu các cấu hình sparsity khác nhau có thời gian huấn luyện khác biệt rõ (vd mô hình cắt nhiều tầng hơn chạy nhanh hơn), nhưng đây là suy luận, paper không thử trên pipeline nén.
3. **Song song hoá Monte Carlo (Mục 3.3) ít liên quan trực tiếp** vì dự án chỉ có 1 GPU, không có nhiều worker chạy đồng thời; nhược điểm này giống với PBT (opt 34).
4. **Khuyến nghị cụ thể (suy luận):** nếu cần tune khoảng 3-6 siêu tham số liên tục (vd ngưỡng sparsity theo layer, λ_{KD}, learning rate, số bước warmup) và mỗi lần thử tốn hàng chục phút đến vài giờ trên GPU 8GB, GP-EI đơn giản (không cần phiên bản MCMC phức tạp) đã đủ hiệu quả hơn random/grid search theo kết quả thực nghiệm của paper; nếu ngân sách đánh giá rất hạn chế (chỉ vài chục lần), nên kết hợp Hyperband (opt 31) để loại nhanh cấu hình tệ trước khi tinh chỉnh bằng BO.

9. Thí nghiệm gợi ý

1. **EI toy trên hàm 1D/2D.** Cài một GP đơn giản (vd dùng `scikit-learn` GaussianProcessRegressor hoặc viết tay với kernel RBF) và thuật toán EI (công thức 2) để tối ưu một hàm tổng hợp 1-2 chiều (vd hàm Branin-Hoo thu nhỏ hoặc hàm sin nhiều cực trị). Đo số lần đánh giá để đạt sai số dưới một ngưỡng, so với random search cùng ngân sách. Kỳ vọng: GP-EI cần ít lần đánh giá hơn random search đáng kể trên hàm có cấu trúc mượt.
2. **Tune thật 2 siêu tham số của một mạng nhỏ** (vd learning rate và weight decay của một CNN nhỏ trên tập con MNIST/CIFAR-10, train vài epoch, chạy được trên GPU 8GB trong vài phút mỗi lần). So sánh GP-EI với grid search có cùng số lần thử; đo validation error cuối cùng. Diễn giải: nếu GP-EI đạt error thấp hơn với cùng số lần thử, xác nhận lại phát hiện của paper ở quy mô nhỏ.
3. **So sánh point-estimate và tích hợp MCMC cho siêu tham số GP.** Lặp lại thí nghiệm 2 nhưng dùng hai biến thể: tối ưu marginal likelihood (point estimate) và lấy mẫu MCMC/slice sampling đơn giản (hoặc xấp xỉ bằng vài mẫu ngẫu nhiên từ posterior gần đúng). Đo xem biến thể "tích hợp" có ổn định hơn qua nhiều lần chạy lặp (ít phương sai kết quả) hay không, như paper quan sát.
4. **Chi phí biến đổi (EI per second) trên bài toán giả lập.** Tạo hàm mục tiêu tổng hợp có "chi phí" $c(x)$ khác nhau theo vùng không gian (vd mô phỏng: vùng tham số sparsity cao thì nhanh, sparsity thấp thì chậm). So sánh EI thường và EI/giây về tổng wall-clock cần để đạt một mức hàm mục tiêu cho trước.
5. **So sánh kernel SE và Matérn trên cùng bài toán tune thật.** Dùng lại thí nghiệm 2, thử kernel RBF (squared exponential) và Matérn 5/2; quan sát xem kết quả tune có khác biệt rõ hay không ở quy mô nhỏ (paper thấy khác biệt rõ ở motif-finding, Hình 5c).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Probability of Improvement	$a_{PI}(x) = \Phi(\gamma(x)),\ \gamma(x) = \dfrac{f(x_{best}) - \mu(x)}{\sigma(x)}$
Expected Improvement	$a_{EI}(x) = \sigma(x)(\gamma(x)\Phi(\gamma(x)) + \phi(\gamma(x)))$
GP (Lower) Confidence Bound	$a_{LCB}(x) = \mu(x) - \kappa\sigma(x)$
ARD squared exponential	$K_{SE} = \theta_0 \exp(-\frac{1}{2}r^2),\ r^2 = \sum_d (x_d - x'_d)^2/\theta_d^2$
ARD Matérn 5/2	$K_{M52} = \theta_0(1 + \sqrt{5r^2} + \frac{5}{3}r^2)\exp(-\sqrt{5r^2})$
Số siêu tham số GP	$D + 3$ (D length-scale, θ_0, nhiễu ν, trung bình m)
Tích hợp acquisition (Bayes đầy đủ)	$\hat{a}(x) = \int a(x;\theta)p(\theta \mid \text{data})\,d\theta$, MCMC/slice sampling
EI mỗi giây	$a_{EI/sec} \propto a_{EI}(x)/\mathbb{E}[c(x)]$, $c(x)$ mô hình bằng GP trên $\ln c(x)$
Song song hoá (fantasy)	$\hat{a}(x) = \mathbb{E}_{\{y_j\}}[a(x;\{x_j, y_j\})]$, lấy mẫu Monte Carlo J chiều
Kết quả nổi bật	Online LDA: grid 288 cấu hình, 60-120 processor-day; motif-finding: lưới 1400 cấu hình; CIFAR-10: 18% (chuyên gia) so với 14,98% (GP EI MCMC)

11. Kết luận ngắn

Paper đưa Bayesian optimization từ một kỹ thuật tối ưu hộp đen tổng quát thành một công cụ thực hành để tune siêu tham số ML, với ba cải tiến cốt lõi: tích hợp Bayes đầy đủ cho siêu tham số kernel GP (quan trọng nhất), mô hình hoá chi phí thời gian, và song song hoá bằng Monte Carlo fantasy. Trên bốn bài toán (Branin-Hoo/logistic regression, online LDA, motif-finding structured SVM, CNN CIFAR-10), các biến thể GP-EI vượt grid search, TPE, và cả chuyên gia người (CIFAR-10: 14,98% so với 18% lỗi test). Hạn chế chính là chi phí $O(N^3)$ của GP khi số lần đánh giá lớn và giả định độc lập giữa hàm mục tiêu và hàm chi phí; các bài sau trong lộ trình (opt 31 Hyperband, opt 33 Tutorial on BO) mở rộng hoặc thay thế các điểm này.

Nguồn gốc: https://arxiv.org/abs/1206.2944

Bài 33

A Tutorial on Bayesian Optimization (2018)

Tác giả: Peter I. Frazier (Cornell University).
Venue / arXiv: arXiv:1807.02811 (v1, 08/07/2018); paper không nêu venue hội nghị/tạp chí, đây là một bản tutorial/survey công bố trên arXiv.
Nguồn: https://arxiv.org/abs/1807.02811
Chủ đề: tutorial lý thuyết về Bayesian optimization (BayesOpt) cho tối ưu hàm hộp đen, không có gradient, tốn thời gian đánh giá: Gaussian Process (GP) regression làm mô hình thống kê, và ba acquisition function chính (Expected Improvement, Knowledge Gradient, Entropy Search/Predictive Entropy Search), cùng các mở rộng ”exotic” (nhiễu, song song, ràng buộc, multi-fidelity, multi-task, derivative).
Vị trí trong lộ trình: đọc cùng nhóm với opt 32 (Practical Bayesian Optimization, ứng dụng cụ thể cho ML, công bố sớm hơn 2012) và opt 31 (Hyperband, phương pháp thay thế không dùng GP); bài này là lý thuyết tổng quát hơn, nên có thể đọc sau opt 32 để thấy EI, GP, tích hợp Bayes đã được dùng thực tế thế nào, rồi đọc bài này để hiểu đầy đủ hơn các acquisition function khác (Knowledge Gradient, Entropy Search) và các bài toán ”exotic” mà opt 32 không đề cập. Cũng liên hệ opt 34 (Population Based Training), một hướng tune siêu tham số khác không dùng GP.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Bayesian optimization (BayesOpt) giải bài toán $\max_{x \in A} f(x)$ (công thức 1, Mục 1) khi:

- $x \in \mathbb{R}^d$ với d không quá lớn (paper ghi ”$d \leq 20$ trong hầu hết ứng dụng thành công”).
- Tập khả thi A đơn giản (hyper-rectangle hoặc simplex).
- f liên tục, ”đắt đánh giá” (vài trăm lần đánh giá là giới hạn thực tế, vì mỗi lần tốn phút/giờ, tiền, hoặc ”chi phí cơ hội”).
- f là ”hộp đen” (không có cấu trúc đặc biệt như lồi/tuyến tính để khai thác).
- Chỉ quan sát được $f(x)$, không có đạo hàm bậc 1/2 (”derivative-free”).
- Phần lớn bài viết giả định quan sát không nhiễu; Mục 5 mới nới ra cho phép nhiễu.
- Mục tiêu là tìm **tối ưu toàn cục**, không phải cực trị cục bộ.

BayesOpt bắt nguồn từ công trình Kushner (1964), Žilinskas (1975), Močkus (1975, 1978, 1989), nhưng chỉ phổ biến rộng sau khi Jones et al. (1998) đưa ra thuật toán ”Efficient Global Optimization” (EGO). Quan sát của Snoek et al. (2012) rằng BayesOpt hữu dụng để tune mạng neural sâu (chính là bài opt 32) đã tạo một làn sóng quan tâm mới trong machine learning, kéo theo nhiều cải tiến: multi-task optimization, multi-fidelity cho huấn luyện mạng sâu, các phương pháp song song hoá.

Tác giả phân biệt bài tutorial này với các tutorial/survey khác (Shahriari et al. 2016; Brochu et al. 2009; Sasena 2002; Frazier & Wang 2016) ở hai điểm: (i) bao quát các bài toán ”exotic” (không chuẩn) nhiều hơn; (ii) nhấn mạnh acquisition function hơn là GP regression; và tự nhận có một phân tích **mới** về expected improvement cho trường hợp đo nhiễu (dựa trên lập luận lý thuyết quyết định hình thức, khác các chỉnh sửa ad hoc trước đó).

2. Ý tưởng chính

BayesOpt gồm 2 thành phần (Mục 2, Giải thuật 1):

1. **Mô hình thống kê Bayes** cho f (hầu như luôn là Gaussian Process): cho một phân phối hậu nghiệm trên giá trị $f(x)$ tại điểm bất kỳ, cập nhật mỗi khi có quan sát mới.
2. **Acquisition function**: đo "giá trị" của việc đánh giá f tại một điểm mới x, dựa trên hậu nghiệm hiện tại; điểm tiếp theo là điểm tối đa hoá acquisition function.

Thuật toán tổng quát (Giải thuật 1): đặt GP prior lên f; quan sát f tại n_0 điểm theo thiết kế lấp đầy không gian (space-filling, thường ngẫu nhiên đều); lặp: cập nhật hậu nghiệm, chọn $x_n = \arg\max_x a(x)$ (acquisition), quan sát $y_n = f(x_n)$, tăng n, dừng khi đạt N lần đánh giá; trả về điểm có f lớn nhất đã quan sát hoặc điểm có hậu nghiệm trung bình lớn nhất.

Ba acquisition function chính được trình bày kỹ: **Expected Improvement (EI)** (phổ biến nhất, dễ dùng), **Knowledge Gradient (KG)** (xét ảnh hưởng lên toàn bộ hậu nghiệm, không chỉ điểm vừa đo), **Entropy Search/Predictive Entropy Search (ES/PES)** (đo giảm độ bất định entropy về vị trí cực đại toàn cục). KG, ES, PES đặc biệt hữu ích trong các bài toán "exotic" (Mục 5) nơi giả định cốt lõi của EI (lợi ích chính của việc đánh giá nằm ở cải thiện tại chính điểm đó) không còn đúng.

3. Phương pháp chi tiết

3.1 Gaussian Process regression (Mục 3)

GP là một phân phối tiên nghiệm Bayes trên hàm, xác định bởi: với một tập điểm hữu hạn $x_1, \dots, x_k$, vector $[f(x_1), \dots, f(x_k)]$ tuân theo phân phối Gauss đa biến (công thức 2):

$$f(x_{1:k}) \sim \text{Normal}\big(\mu_0(x_{1:k}), \Sigma_0(x_{1:k}, x_{1:k})\big),$$

với μ_0 là hàm trung bình tiên nghiệm, Σ_0 là kernel (hiệp phương sai). Khi quan sát không nhiễu $f(x_{1:n})$, hậu nghiệm tại điểm mới x là Gauss (công thức 3):

$$f(x) \mid f(x_{1:n}) \sim \text{Normal}(\mu_n(x), \sigma_n^2(x)),$$

$$\mu_n(x) = \Sigma_0(x, x_{1:n})\Sigma_0(x_{1:n}, x_{1:n})^{-1}\big(f(x_{1:n}) - \mu_0(x_{1:n})\big) + \mu_0(x),$$

$$\sigma_n^2(x) = \Sigma_0(x, x) - \Sigma_0(x, x_{1:n})\Sigma_0(x_{1:n}, x_{1:n})^{-1}\Sigma_0(x_{1:n}, x).$$

Trung bình hậu nghiệm là trung bình có trọng số giữa tiên nghiệm và dữ liệu; phương sai hậu nghiệm bằng phương sai tiên nghiệm trừ phần bị "giải thích" bởi dữ liệu. Về số: paper gợi ý dùng phân rã Cholesky thay nghịch đảo ma trận trực tiếp (ổn định số học hơn), và thêm 10^{-6} vào đường chéo Σ_0 để tránh giá trị riêng quá gần 0 khi các điểm quan sát gần nhau.

Kernel (Mục 3.1): hai ví dụ chính.

- Power exponential/Gaussian: $\Sigma_0(x, x') = \alpha_0 \exp(-\|x - x'\|^2)$, với $\|x - x'\|^2 = \sum_{i=1}^{d} \alpha_i (x_i - x'_i)^2$.
- Matérn: $\Sigma_0(x, x') = \alpha_0 \dfrac{2^{1-\nu}}{\Gamma(\nu)} \big(\sqrt{2\nu}\|x - x'\|\big)^{\nu} K_\nu\big(\sqrt{2\nu}\|x - x'\|\big)$, với K_ν là hàm Bessel biến đổi và ν là tham số độ mượt.

Chọn siêu tham số η (ký hiệu chung cho $\alpha_{0:d}, \nu, \mu$, Mục 3.2), ba cách, cùng cấu trúc với opt 32:

- **MLE** (maximum likelihood): $\hat{\eta} = \arg\max_\eta P(f(x_{1:n}) \mid \eta)$.
- **MAP** (maximum a posteriori, công thức qua luật Bayes): $\hat{\eta} = \arg\max_\eta P(f(x_{1:n}) \mid \eta) P(\eta)$; MLE là trường hợp đặc biệt khi $P(\eta)$ là phân phối hằng (degenerate).
- **Fully Bayesian**: tích hợp trên toàn bộ phân phối hậu nghiệm của η (công thức 5):

$$P(f(x) = y \mid f(x_{1:n})) = \int P(f(x) = y \mid f(x_{1:n}), \eta)\, P(\eta \mid f(x_{1:n}))\, d\eta,$$

xấp xỉ bằng Monte Carlo qua các mẫu $\hat{\eta}_j$ lấy bằng MCMC (vd slice sampling, Neal 2003), cùng kỹ thuật slice sampling mà opt 32 dùng (Murray & Adams 2010).

3.2 Expected Improvement (Mục 4.1)

Xây dựng bằng một ”thí nghiệm tưởng tượng”: nếu phải trả lời ngay bây giờ (không đánh giá thêm), lời giải tốt nhất là điểm đã quan sát có f lớn nhất, $f_n^* = \max_{m \le n} f(x_m)$. Nếu được đánh giá thêm 1 điểm x, cải thiện là $[f(x) - f_n^*]^+$ ($a^+ = \max(a, 0)$). Expected improvement (công thức 7):

$$\mathrm{EI}_n(x) := \mathbb{E}_n\big[[f(x) - f_n^*]^+\big].$$

Paper trình bày dạng đóng (công thức 8) qua $\Delta_n(x) := \mu_n(x) - f_n^*$ (chênh lệch kỳ vọng giữa điểm đề xuất và điểm tốt nhất hiện tại). **Ghi chú về trích xuất văn bản:** bản PDF-to-text (pdftotext) của công thức (8) cho ra dạng $\mathrm{EI}_n(x) = [\Delta_n(x)]^+ + \sigma_n(x)\phi(\Delta_n(x)/\sigma_n(x)) - |\Delta_n(x)|\Phi(\Delta_n(x)/\sigma_n(x))$; khi kiểm tra đại số (tự tính, với $\Delta > 0$), biểu thức này **không khớp** với dạng đóng tiêu chuẩn quen thuộc của EI (Jones et al. 1998), gợi ý nhiều khả năng là lỗi mất ký hiệu dưới/trên dòng khi trích xuất PDF (giống các lỗi đã gặp ở bài khác trong cùng batch, vd ”$3^2 = 9$” bị hiển thị thành ”32 = 9”), không phải lỗi của paper. Dạng đóng tiêu chuẩn, mà chính paper dẫn chiếu tới Jones et al. (1998)/Clark (1961), là:

$$\boxed{\mathrm{EI}_n(x) = \Delta_n(x)\,\Phi\Big(\frac{\Delta_n(x)}{\sigma_n(x)}\Big) + \sigma_n(x)\,\phi\Big(\frac{\Delta_n(x)}{\sigma_n(x)}\Big)}$$

(với quy ước $\Delta_n(x)/\sigma_n(x) = 0$ nếu $\sigma_n(x) = 0$). Ghi chú này được nêu rõ để không truyền đạt sai công thức cho người đọc; phần diễn giải định tính sau đây (EI tăng theo cả Δ_n và σ_n, bằng 0 tại điểm đã đánh giá) đúng với cả hai dạng viết.

Ví dụ số (tự tính), dùng dạng đóng tiêu chuẩn: giả sử $\mu_n(x) = 12$, $f_n^* = 10$ (đang tối đa hoá), $\sigma_n(x) = 3$. $\Delta_n(x) = 12 - 10 = 2$, $z = \Delta_n(x)/\sigma_n(x) = 2/3 \approx 0{,}667$. Dùng bảng chuẩn tắc: $\Phi(0{,}667) \approx 0{,}7476$, $\phi(0{,}667) \approx 0{,}3194$. $\mathrm{EI}_n(x) = 2 \times 0{,}7476 + 3 \times 0{,}3194 \approx 1{,}4952 + 0{,}9582 = 2{,}4534$ (tự tính). Nếu điểm đề xuất có $\mu_n(x) = 8 < f_n^* = 10$ (trông ”tệ hơn” best hiện tại) với cùng $\sigma_n(x) = 3$: $\Delta_n(x) = -2$, $z = -0{,}667$, $\Phi(-0{,}667) \approx 0{,}2524$, $\phi(-0{,}667) = \phi(0{,}667) \approx 0{,}3194$ (đối xứng): $\mathrm{EI}_n(x) = (-2) \times 0{,}2524 + 3 \times 0{,}3194 \approx -0{,}5048 + 0{,}9582 = 0{,}4534$ (tự tính): vẫn dương (do còn bất định σ) nhưng nhỏ hơn nhiều so với trường hợp $\Delta > 0$.

Thuật toán EI: $x_{n+1} = \arg\max_x \mathrm{EI}_n(x)$ (công thức 9), do EI_n rẻ và khả vi nên có thể tối ưu bằng L-BFGS-B. Hình 3 (contour của EI theo Δ_n, σ_n): EI tăng theo cả hai, thể hiện sự đánh đổi ”exploitation” (Δ_n lớn, gần tối ưu) và ”exploration” (σ_n lớn, chưa biết nhiều), cùng tinh thần với bài toán multi-armed bandit và đánh đổi exploration/exploitation trong RL.

3.3 Knowledge Gradient (Mục 4.2)

KG bỏ giả định ”chỉ được trả lời bằng điểm đã đánh giá”: cho phép trả lời bất kỳ điểm nào (kể cả chưa đánh giá), với giả định risk-neutral. Nếu dừng lại sau n mẫu, lời giải tốt nhất theo hậu nghiệm có giá trị kỳ vọng $\mu_n^* := \max_{x'} \mu_n(x')$. Nếu đánh giá thêm tại x, hậu nghiệm mới cho $\mu_{n+1}^* := \max_{x'} \mu_{n+1}(x')$. Knowledge gradient (công thức 10):

$$\mathrm{KG}_n(x) := \mathbb{E}_n\big[\mu_{n+1}^* - \mu_n^* \mid x_{n+1} = x\big].$$

Khác EI (chỉ xét hậu nghiệm tại điểm vừa đo), KG xét **toàn bộ** hậu nghiệm trên domain và cách mẫu mới làm thay đổi nó; điều này giúp ích nhiều trong các bài toán ”exotic” (nhiễu, multi-fidelity, derivative, Mục 5), nơi KG có thể vượt rõ EI. Tính KG bằng mô phỏng (Giải thuật 2: với mỗi vòng lặp $j = 1, \ldots, J$, mô phỏng $y_{n+1} \sim \mathrm{Normal}(\mu_n(x), \sigma_n^2(x))$, tính μ_{n+1}^* giả định quan sát đó, lấy trung bình $\mu_{n+1}^* - \mu_n^*$ qua J mô phỏng). Để tối ưu KG theo x hiệu quả hơn ở chiều cao, Wu & Frazier (2016) đề xuất **multistart stochastic gradient**

ascent (Giải thuật 3, tham số gợi ý: $R = 10$ điểm khởi tạo, $T = 10^2$ bước mỗi điểm, hệ số bước $a = 4$, $J = 10^3$ lần mô phỏng), dùng gradient ngẫu nhiên không chệch (Giải thuật 4) nhờ ”infinitesimal perturbation analysis” và định lý envelope (Milgrom & Segal, 2002).

3.4 Entropy Search và Predictive Entropy Search (Mục 4.3)

Gọi x^* là cực đại toàn cục của f (một biến ngẫu nhiên có phân phối suy ra từ hậu nghiệm GP). **Entropy Search (ES)** (Hennig & Schuler, 2012) đo giảm entropy sai phân của phân phối trên x^* do việc đánh giá tại x (công thức 11):

$$\mathrm{ES}_n(x) = H(P_n(x^*)) - \mathbb{E}_{f(x)}\big[H(P_n(x^* \mid f(x)))\big].$$

ES khó tính (entropy của argmax một GP không có dạng đóng, phải tính cho nhiều giá trị y, rồi tối ưu một hàm khó-đánh-giá; không có cách tính gradient ngẫu nhiên như KG). **Predictive Entropy Search (PES)** (Hernández-Lobato et al., 2014) dùng một công thức tương đương qua mutual information (công thức 12):

$$\mathrm{PES}_n(x) = \mathrm{ES}_n(x) = H(P_n(f(x))) - \mathbb{E}_{x^*}\big[H(P_n(f(x) \mid x^*))\big],$$

trong đó số hạng đầu có dạng đóng, số hạng sau cần xấp xỉ (lấy mẫu x^* từ hậu nghiệm, dùng expectation propagation, Minka 2001).

3.5 Acquisition function tối ưu nhiều bước (Mục 4.4)

Về bản chất, bài toán (1) là một **quyết định tuần tự**: chọn x_n, quan sát y_n, cuối cùng nhận ”reward” (giá trị điểm tốt nhất, hoặc giá trị tại điểm mới chọn theo hậu nghiệm, hoặc entropy...). EI, KG, ES, PES đều tối ưu khi $N = n + 1$ (chỉ còn 1 lần đánh giá) nhưng **không chắc tối ưu** khi $N > n + 1$ (còn nhiều lần đánh giá). Tính ”multi-step optimal” tổng quát cần quy hoạch động ngẫu nhiên (stochastic dynamic programming), nhưng ”lời nguyền chiều” (curse of dimensionality) làm việc này rất khó trong thực hành. Một vài bài toán hẹp hơn tính được lời giải nhiều bước tối ưu chính xác (Cashore et al. 2016; Waeber et al. 2013); paper trích: trong các bài toán đó, **KG đạt trong vòng 98% so với tối ưu nhiều bước** (Cashore et al. 2016), và entropy search chính là tối ưu nhiều bước trong bài toán root-finding của Waeber et al. (2013). Đây là **số liệu trích dẫn từ công trình khác**, không phải thí nghiệm của chính paper này.

3.6 Các bài toán ”exotic” (Mục 5)

- **Nhiễu đo:** thêm phương sai nhiễu vào đường chéo ma trận hiệp phương sai (công thức 3); KG/ES/PES áp dụng trực tiếp (vẫn one-step optimal); EI cần biến đổi heuristic (vd ”KGCP” của Scott et al. 2011, coi là xấp xỉ KG nhưng ràng buộc lời giải cuối phải là điểm đã đánh giá).
- **Đánh giá song song:** EI mở rộng thành ”multi-points EI” (công thức 14): $\mathrm{EI}_n(x^{(1:q)}) = \mathbb{E}_n[\max_i f(x^{(i)}) - f_n^*]^+$; tối ưu khó hơn, dùng xấp xỉ ”Constant Liar” (Ginsbourger et al. 2010) hoặc gradient ngẫu nhiên không chệch (Wang et al. 2016a), đã dùng cho tới $q = 128$ **lần đánh giá song song**.
- **Ràng buộc:** $\max_x f(x)$ với $g_i(x) \geq 0$ cũng đắt đánh giá; EI mở rộng tự nhiên khi không nhiễu (Schonlau et al. 1998; Gardner et al. 2014); PES cũng được nghiên cứu cho ràng buộc (Hernández-Lobato et al. 2015).
- **Multi-fidelity/multi-information source:** có $f(x, s)$ với s chỉ ”độ trung thực” (fidelity thấp hơn = rẻ hơn, kém chính xác hơn); mục tiêu $\max_x f(x, 0)$ với tổng chi phí $\sum_n c(s_n) \leq B$ (ngân sách B). EI khó dùng trực tiếp (đánh giá $s \neq 0$ không cải thiện giá trị tốt nhất quan sát ở $s = 0$); KG, ES, PES áp dụng trực tiếp.
- **Random environmental conditions/multi-task BO:** tối ưu $\max_x \int f(x, w)p(w)\,dw$ hoặc $\max_x \sum_w f(x, w)p(w)$; **ứng dụng machine learning trực tiếp được paper nêu**: w là một ”fold” cross-validation, $f(x, w)$ là hiệu năng test trên fold w của mô hình huấn luyện không dùng fold đó; mục tiêu là tối ưu hiệu năng CV trung bình.

- **Quan sát đạo hàm:** $\nabla f(x)$ (có thể nhiễu) đưa trực tiếp vào GP regression; Lizotte (2008) dùng với EI, cải thiện so với BFGS; KG tận dụng đạo hàm tốt hơn (Wu et al. 2017) vì EI "không đổi" khi có thêm quan sát đạo hàm tại cùng điểm (không tận dụng được việc đạo hàm hữu ích ở xa điểm đã đánh giá).

4. Thí nghiệm và kết quả chính

Đây là một **bài tutorial lý thuyết**, không có benchmark thực nghiệm riêng của tác giả: không có bảng số liệu so sánh nhiều phương pháp trên nhiều bài toán như opt 32 hay các bài benchmark optimizer khác trong thư mục này. Nội dung "thí nghiệm" của bài gồm:

- **Hình minh hoạ số** (Hình 1, 2, 3): Hình 1 minh hoạ một vòng lặp BayesOpt (GP posterior + EI) trên hàm 1 chiều tổng hợp; Hình 2 vẽ các hàm mẫu từ GP prior với kernel power-exponential, cho thấy tham số α_1 kiểm soát độ "gập ghềnh"; Hình 3 vẽ contour của EI theo (Δ_n, σ_n).
- **Kết quả trích dẫn từ công trình khác** (không phải thí nghiệm gốc của bài): Cashore et al. (2016) báo cáo KG đạt trong **98%** so với thuật toán multi-step tối ưu trên các bài toán feasibility-determination họ tính được; Bull (2011) chứng minh tốc độ hội tụ cho EI kết hợp lấy mẫu đều định kỳ.
- **Mục 6 (Software)**: liệt kê các thư viện GP/BayesOpt hiện có (không phải thí nghiệm, nhưng là thông tin thực hành quan trọng): DiceKriging/DiceOptim (R), GPyOpt (Python, trên GPy), Metrics Optimization Engine/MOE và Cornell MOE (C++/Python, hỗ trợ GPU, KG song song + đạo hàm), Spearmint (Python, cùng nhóm tác giả với opt 32), DACE (MATLAB), GPFlow (TensorFlow)/GPyTorch (PyTorch), laGP (R, hỗ trợ ràng buộc bất đẳng thức).

Vì không có benchmark thực nghiệm thật, các "kết quả chính" của bài này là các **công thức đóng** và **thuật toán** (EI, KG, ES/PES, các mở rộng exotic) đã trình bày ở Mục 3, không phải số liệu đo đạc.

5. Phân tích / ablation

Không có ablation thực nghiệm (không có thí nghiệm riêng), nhưng paper có các phân tích lý thuyết/so sánh định tính đóng vai trò tương đương:

- **EI so với KG:** KG luôn ít nhất bằng EI về mặt lý thuyết trong bài toán chuẩn không nhiễu (Frazier et al. 2009 cho thấy lợi ích nhỏ), nhưng KG "vượt trội đáng kể" trong bài toán có nhiễu, multi-fidelity, đạo hàm, hay cần tích hợp theo điều kiện môi trường (trích các công trình Wu et al. 2017; Poloczek et al. 2017; Wu & Frazier 2016; Toscano-Palmerin & Frazier 2018). Lý do định tính: KG xét thay đổi trên toàn hậu nghiệm, hữu ích khi "lợi ích chính không nằm ở điểm vừa đo" (vd đạo hàm cho biết hàm tăng ở hướng khác).
- **MLE so với MAP so với fully Bayesian** cho siêu tham số GP: MAP hữu ích khi MLE cho giá trị "không hợp lý" (hàm biến đổi quá nhanh/chậm); fully Bayesian (tích hợp MCMC) là lựa chọn tổng quát nhất, cùng tư tưởng với "GP EI MCMC" của opt 32.
- **ES so với PES:** về lý thuyết hai acquisition tương đương (cùng công thức entropy/mutual information), nhưng khác biệt trong **cách tính xấp xỉ** tạo ra khác biệt thực hành (PES có số hạng đầu dạng đóng, dễ hơn ES).
- **EI mở rộng cho nhiễu:** nhiều cách thay thế heuristic được liệt kê (thay phân phối của $f(x)$ bằng phân phối của $\mu_{n+1}(x)$, hay của y_{n+1}); tác giả tự đề xuất cách tiếp cận của Scott et al. (2011) là "tự nhiên nhất" vì giữ đúng giả định gốc của EI (chỉ trả lời điểm đã đánh giá) thay vì heuristic tuỳ ý.
- **One-step so với multi-step optimal:** trong các bài toán hẹp tính được chính xác, acquisition một bước (EI/KG/ES) đã gần tối ưu nhiều bước (KG trong 98%, ES bằng chính xác trong một trường hợp); tác giả nhận định "có thể một-bước đã đủ gần tối ưu để cải thiện thêm không còn ý nghĩa thực hành, hoặc có thể multi-step sẽ hữu ích đáng kể trong các bài toán khác chưa được nhận diện" (không kết luận chắc chắn).

6. Giới hạn

Do tác giả tự nêu (Mục 7, Kết luận và hướng nghiên cứu):

- Thiếu hiểu biết lý thuyết sâu: ngoài vài trường hợp hẹp, chưa có thuật toán multi-step tối ưu tính được trong thực hành; chưa có ”finite-time bound” giải thích vì sao acquisition một-bước lại gần tối ưu nhiều-bước trong thực nghiệm.
- Tốc độ hội tụ: ngay cả tiệm cận, biết rất ít; Bull (2011) chỉ chứng minh cho EI **kết hợp lấy mẫu đều định kỳ**, không rõ bỏ lấy mẫu đều có giữ cùng tốc độ hay không.
- GP (hoặc biến thể) được dùng hầu hết, nhưng có thể tồn tại lớp bài toán mô hình tốt hơn bằng phương pháp thống kê khác (không named cụ thể, chỉ là hướng mở).
- BayesOpt trong không gian **nhiều chiều** vẫn là thách thức lớn (hướng nghiên cứu: tận dụng cấu trúc, vd Wang et al. 2013/2016b, Kandasamy et al. 2015).
- Nhiều cấu trúc bài toán ”exotic” khác chưa được xét tới; tác giả gợi ý kết hợp phát triển phương pháp với áp dụng thực tế (bài toán thực thường lộ ra khó khăn chưa dự đoán được).
- Ứng dụng trong hoá học/khoa học vật liệu/khám phá thuốc còn ít người biết đến BayesOpt dù ”có vị trí tốt để tạo tác động”.

Do chúng ta nhận thấy:

- Công thức (8) cho EI khi trích xuất qua pdftotext không khớp đại số với dạng chuẩn Jones et al. (1998) ở trường hợp $\Delta_n(x) > 0$ (xem phân tích ở Mục 3.2); nhiều khả năng là lỗi trích xuất ký hiệu dưới/trên dòng của PDF, không phải lỗi gốc của paper, nhưng người đọc dùng bản PDF gốc nên kiểm tra lại hình ảnh công thức thay vì chỉ tin bản text trích xuất.
- Hoàn toàn không có số liệu thực nghiệm do chính tác giả tạo ra (khác opt 32); mọi ”con số” trong bài (vd 98% của Cashore et al.) là trích dẫn kết quả người khác, nên không thể dùng bài này để đánh giá hiệu năng BayesOpt theo nghĩa thực nghiệm.
- Không đề cập Hyperband/successive halving (paper 2016, cùng năm nộp bản v1 nhưng khác hướng) như một lựa chọn thay thế không cần GP; phần liệt kê các hướng liên quan (Mục 1) chỉ nói tới BayesOpt và các biến thể của nó.

7. Liên hệ với các bài khác

- **opt 32** (Practical Bayesian Optimization of Machine Learning Algorithms; file ”optimization content/32_2012_practical_bayesian_optimization.md” đã viết xong trong cùng thư mục): bài 32 là ứng dụng cụ thể, sớm hơn (2012), chỉ dùng EI với tích hợp Bayes MCMC cho siêu tham số GP (kernel Matérn 5/2); bài 33 đặt điều đó vào bối cảnh lý thuyết rộng hơn (EI là một trong 3 acquisition chính, có KG và ES/PES) và thêm nhiều bài toán ”exotic” mà bài 32 không xét (nhiễu, multi-fidelity, ràng buộc, derivative).
- **opt 31** (Hyperband): một hướng thay thế hoàn toàn không cần GP, dựa trên successive halving/bandit; hữu ích khi BayesOpt (dù dùng acquisition nào) vẫn còn đắt vì chi phí $O(n^3)$ của GP khi n lớn.
- **opt 34** (Population Based Training; file ”optimization content/34_2017_population_based_training.md” đã viết xong): PBT tự liệt BayesOpt (GP-UCB, TPE, Spearmint, SMAC) vào nhóm ”sequential optimisation” chậm vì cần nhiều lần chạy tuần tự; KG/ES/PES của bài này không giải quyết được nhược điểm ”tuần tự” đó, khác với PBT (thích ứng online trong một tiến trình huấn luyện).
- Đã kiểm tra bằng `ls` các thư mục cũ (”llm content/”, ”rl content/”, ”distillation content/”, ”pruning content/”, ”interp content/”): không thấy bài trực tiếp về Gaussian Process hay Bayesian optimization; ”math 35” (A Tutorial on Thompson Sampling, theo titles.tsv, thư mục ”math content/” đang được viết đồng thời) cùng họ bài toán exploration/exploitation qua acquisition/policy xác suất, có thể liên hệ về tư duy ”đánh đổi khai thác và khám phá” chung với EI.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, chạy trên GPU <= 8GB, batch 1). Nhận định dưới đây là suy luận:

1. **Phần lớn giá trị thực hành đã nằm ở opt 32** (EI + GP cho tune siêu tham số đắt-đánh-giá); bài 33 **ít thêm giá trị trực tiếp** cho pipeline 1-GPU đơn giản, vì các mở rộng mạnh nhất của nó (song song $q = 128$, multi-fidelity cho mạng sâu, ràng buộc) nhằm vào bối cảnh có nhiều máy/GPU hoặc bài toán phức tạp hơn một pipeline cắt+distill đơn.
2. **Ý tưởng "random environmental conditions/multi-task BO" đáng chú ý nhất cho dự án** (Mục 5): nếu đánh giá một cấu hình pruning/distill trên **nhiều fold hoặc nhiều tập dữ liệu con** (vd nhiều loại trang OCR: văn bản dày đặc, bảng, ảnh scan nhiễu), có thể coi mỗi loại là một "w" và muốn tối ưu hiệu năng **trung bình có trọng số** qua các loại; BayesOpt kiểu multi-task có thể tiết kiệm số lần thử so với đánh giá đầy đủ trên tất cả loại mỗi lần (suy luận, paper không thử trên OCR).
3. **Multi-fidelity cho huấn luyện mạng sâu** (Mục 5, trích Swersky et al. 2014; Klein et al. 2016): ý tưởng dùng s = số bước/epoch huấn luyện làm "fidelity" khớp trực tiếp với việc đánh giá nhanh một cấu hình distill bằng vài trăm bước trước khi quyết định có chạy đầy đủ không; đây gần với tinh thần Hyperband (opt 31) nhưng dùng GP đa-fidelity thay vì successive halving thuần.
4. **KG hữu ích hơn EI khi "tín hiệu" không trực tiếp nằm ở điểm vừa thử** (vd nếu đo được "đạo hàm gần đúng" của chất lượng theo tỉ lệ sparsity bằng cách thử nhiều mức sparsity gần nhau): có thể xem xét KG nếu EI không khai thác tốt thông tin dạng này, nhưng đây là ý tưởng suy luận bậc hai, không ưu tiên cao cho pipeline đơn giản.

9. Thí nghiệm gợi ý

1. **So sánh EI và KG trên hàm giả lập 1D không nhiễu.** Cài GP (vd `scikit-learn` hoặc viết tay) trên một hàm tổng hợp đa cực trị; so sánh số lần đánh giá EI và KG (KG tính bằng mô phỏng, Giải thuật 2 với J cỡ vài trăm) cần để đạt sai số dưới ngưỡng. Kỳ vọng theo paper: khác biệt nhỏ trong bài toán chuẩn không nhiễu.
2. **Thêm nhiễu Gauss vào hàm giả lập, lặp lại thí nghiệm 1.** Kỳ vọng: khoảng cách giữa KG và EI (dùng một trong các biến thể EI-cho-nhiễu heuristic) nới rộng ra, minh hoạ nhận định "KG vượt trội khi có nhiễu" của paper.
3. **Tune thật 2 siêu tham số của một mạng nhỏ bằng EI chuẩn (dạng đóng ở Mục 3.2) trên GPU 8GB,** so với random search cùng số lần thử; đo validation loss cuối. Đây tái khẳng định kết quả cốt lõi mà opt 32 đã thử ở quy mô lớn hơn.
4. **Mô phỏng "multi-task BO" với 2 "fold" giả lập** (vd 2 tập con dữ liệu OCR khác nhau cho cùng mạng nhỏ): tune 1 siêu tham số để tối ưu trung bình hiệu năng 2 fold, so sánh chiến lược "đánh giá đầy đủ cả 2 fold mỗi lần thử" với chiến lược "chỉ đánh giá 1 fold ngẫu nhiên mỗi lần thử, cập nhật GP theo cả x và w"; đo tổng số lần đánh giá (tính cả 2 fold) cần để đạt cùng mức hiệu năng trung bình.
5. **Tái hiện Hình 2 (hàm mẫu từ GP prior với các α_1 khác nhau)** bằng cách lấy mẫu từ GP với kernel RBF, vẽ vài hàm mẫu 1D cho 2-3 giá trị length-scale; quan sát trực quan cách length-scale ảnh hưởng độ "gập ghềnh", giúp chọn kernel hợp lý hơn trước khi áp vào tune thật.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Bài toán BayesOpt	$\max_{x \in A} f(x)$, f hộp đen, đắt đánh giá, không đạo hàm (công thức 1)
GP posterior	$\mu_n(x), \sigma_n^2(x)$ theo công thức (3), tuyến tính trong Σ_0

Khái niệm	Công thức / giá trị
Kernel Matérn	$\Sigma_0 = \alpha_0 \frac{2^{1-\nu}}{\Gamma(\nu)} (\sqrt{2\nu} r)^\nu K_\nu(\sqrt{2\nu} r)$
Expected Improvement (dạng chuẩn, Jones et al. 1998)	$\text{EI}_n(x) = \Delta_n(x)\Phi(\Delta_n(x)/\sigma_n(x)) + \sigma_n(x)\phi(\Delta_n(x)/\sigma_n(x))$
Knowledge Gradient	$\text{KG}_n(x) = \mathbb{E}_n[\mu^*_{n+1} - \mu^*_n \mid x_{n+1} = x]$ (công thức 10)
Entropy Search	$\text{ES}_n(x) = H(P_n(x^*)) - \mathbb{E}_{f(x)}[H(P_n(x^* \mid f(x)))]$ (công thức 11)
Predictive Entropy Search	$\text{PES}_n(x) = H(P_n(f(x))) - \mathbb{E}_{x^*}[H(P_n(f(x) \mid x^*))]$ (công thức 12)
EI song song (q điểm)	$\mathbb{E}_n[\max_i f(x^{(i)}) - f^*_n]^+$, dùng tới $q = 128$ (công thức 14)
multi-step vs one-step	KG trong 98% tối ưu nhiều bước trên bài toán Cashore et al. (2016)
Siêu tham số GP	MLE, MAP, hoặc fully Bayesian (tích hợp MCMC, công thức 5-6)

11. Kết luận ngắn

Bài tutorial trình bày đầy đủ khung Bayesian optimization: GP regression làm mô hình thống kê, ba acquisition function (Expected Improvement, Knowledge Gradient, Entropy Search/Predictive Entropy Search) với mức độ phức tạp tăng dần, và một loạt bài toán ”exotic” (nhiễu, song song, ràng buộc, multi-fidelity, multi-task, derivative) mà EI đơn giản không xử lý tốt nhưng KG/ES/PES có thể. Không có thí nghiệm gốc; giá trị của bài nằm ở việc hệ thống hoá lý thuyết và chỉ ra hướng nghiên cứu còn mở (bound lý thuyết, không gian nhiều chiều, cấu trúc exotic mới). So với opt 32 (ứng dụng cụ thể, chỉ EI), bài này mở rộng đáng kể bộ công cụ acquisition function nhưng ít thêm giá trị thực hành trực tiếp cho một pipeline tune đơn giản trên 1 GPU.

Nguồn gốc: https://arxiv.org/abs/1807.02811

Bài 34

Population Based Training of Neural Networks (2017)

Tác giả: Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, Koray Kavukcuoglu (DeepMind, London).
Venue / arXiv: arXiv:1711.09846 (v2, 28/11/2017); paper không nêu venue hội nghị/tạp chí trong chính văn bản.
Nguồn: https://arxiv.org/abs/1711.09846
Chủ đề: Population Based Training (PBT): một thuật toán bất đồng bộ, tối ưu đồng thời trọng số θ và siêu tham số h của một quần thể (population) mô hình đang huấn luyện song song, bằng hai cơ chế ”exploit” (sao chép mô hình đang tốt hơn) và ”explore” (nhiễu/resample siêu tham số), từ đó tự động khám phá một **lịch siêu tham số** thay vì một bộ giá trị cố định.
Vị trí trong lộ trình: cùng nhóm hyperparameter optimization với opt 31 (Hyperband, bandit/successive-halving) và opt 32, opt 33 (Bayesian optimization); nên đọc sau hai bài đó vì PBT được paper trình bày như một phương pháp lai giữa ”parallel search” (opt 31, 32, 33 đều thuộc nhóm này ở mức độ khác nhau) và ”sequential optimisation”, đồng thời bổ sung khả năng mà cả BO và Hyperband không có: **thích ứng siêu tham số trong lúc huấn luyện**. Có thể đọc liên hệ với rl 06 (A3C) vì các thí nghiệm RL của PBT xây trên agent kiểu A3C.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Mạng neural phụ thuộc mạnh vào việc tune đồng thời kiến trúc, dữ liệu, và chi tiết tối ưu hoá; mỗi thành phần này có siêu tham số riêng, và việc tune ngày càng nặng khi mô hình phức tạp hơn. Trong RL, bài toán còn khó hơn vì **môi trường học không ổn định** (non-stationary): siêu tham số tối ưu có thể cũng cần thay đổi theo thời gian, không thể định sẵn một lịch cố định.

Paper phân loại hai hướng tune hyperparameter truyền thống (Mục 1, Hình 1):

- **Parallel search** (grid search, random search): chạy nhiều tiến trình huấn luyện song song với siêu tham số khác nhau, lấy kết quả tốt nhất; tốn nhiều tài nguyên tính toán nhưng thời gian chờ chỉ bằng một lần huấn luyện.
- **Sequential optimisation** (hand tuning, Bayesian optimisation: GP-UCB, TPE, Spearmint, SMAC; xem opt 31 đến opt 33): dùng kết quả của lần chạy trước để chọn siêu tham số cho lần chạy sau; cho kết quả tốt nhất nhưng cần nhiều lần chạy **tuần tự**, không khả thi khi mỗi lần chạy rất dài.

Ngay cả Hyperband (Li et al. 2016, xem opt 31), dù có song song hoá sẵn, cũng cần nhiều tài nguyên tính toán ngay từ đầu nên không thể chạy ”trong đúng một tiến trình huấn luyện”. PBT được thiết kế để **không tốn thời gian hơn một lần huấn luyện đơn** (wall-clock không vượt một optimisation process), không cần chạy tuần tự, và dùng ít tài nguyên hơn random/grid search thuần.

2. Ý tưởng chính

PBT xây dựng trên ”thành công khó tin của random hyperparameter search” (Bergstra & Bengio, 2012): random search tốt ở việc tìm **vùng** siêu tham số tốt. PBT nhận diện vùng tốt đó và tập trung tính toán vào đó bằng cách **sao chép trọng số đã huấn luyện một phần** từ mô hình tốt sang mô hình tệ, tránh phải huấn luyện lại từ đầu như sequential optimisation.

Cơ chế cốt lõi, gọi độc lập trên từng worker trong quần thể (Mục 3):

- **exploit:** dựa vào hiệu năng của toàn quần thể, quyết định worker hiện tại nên giữ nguyên hay ”từ bỏ” lời giải hiện tại để chuyển sang một lời giải hứa hẹn hơn (vd sao chép trọng số và siêu tham số của một worker tốt hơn).
- **explore:** từ lời giải/siêu tham số hiện tại (sau exploit), đề xuất siêu tham số mới để tiếp tục khám phá không gian (vd nhiễu ngẫu nhiên hoặc resample từ phân phối tiên nghiệm).

Vì bước huấn luyện (gradient descent) chạy xen kẽ với exploit/explore trên từng worker không đồng bộ, PBT tạo ra một hệ học **hai mức thời gian** (two-timescale): mức nhanh là gradient descent cập nhật θ, mức chậm là ”tiến hoá” (giống genetic algorithm) cập nhật h và chọn mô hình. PBT không cần tiến trình trung tâm điều phối, chỉ cần một kho dữ liệu chia sẻ (hiệu năng, trọng số, siêu tham số hiện tại của mỗi worker).

3. Phương pháp chi tiết

3.1 Công thức hoá bài toán (Mục 3)

Cho mô hình f với tham số θ, tối ưu một hàm mục tiêu $\hat{Q}$ (vd loss huấn luyện) qua một thủ tục lặp:

$$\theta \leftarrow \text{step}(\theta \mid h) \qquad (2)$$

trong đó $h \in \mathcal{H}$ là siêu tham số điều kiện của bước cập nhật. Sau T bước:

$$\theta^* = \text{optimise}(\theta \mid h) = \text{optimise}(\theta \mid (h_t)_{t=1}^T) = \text{step}(\text{step}(\ldots \text{step}(\theta \mid h_1) \cdots \mid h_{T-1}) \mid h_T). \qquad (3)$$

Điểm quan trọng: **metric thật sự quan tâm** Q (vd accuracy validation, BLEU) thường khác hàm mục tiêu khả vi $\hat{Q}$ dùng để lấy gradient. PBT muốn tối ưu **đồng thời** θ và h trên chính Q:

$$\theta^* = \text{optimise}(\theta \mid h^*), \quad h^* = \arg\max_{h \in \mathcal{H}^T} \text{eval}(\text{optimise}(\theta \mid h)). \qquad (4)$$

Hàm eval không cần khả vi, không cần trùng với hàm dùng trong bước cập nhật (dù nên tương quan với nó).

Vì số giá trị h khả dĩ tăng theo cấp luỹ (mỗi bước một giá trị h_t), thực hành thường cố định h suốt huấn luyện hoặc đặt một lịch định trước (vd learning rate annealing), rồi chỉ search trên vài biến thể của toàn bộ lịch đó, công thức (4) ở mức ”tìm một $h \in \mathcal{H}^T$” rất khó giải trực tiếp.

3.2 Thuật toán PBT (Mục 3, Giải thuật 1)

PBT huấn luyện N mô hình $\{\theta^i\}_{i=1}^N$ tạo thành quần thể P, mỗi mô hình có siêu tham số riêng $\{h^i\}_{i=1}^N$, chạy **bất đồng bộ song song**. Giải thuật 1 (TRAIN(P)), viết lại ý chính:

```
với mỗi (theta, h, p, t) trong P (song song, bất đồng bộ):
    trong khi chưa hết huấn luyện:
        theta <- step(theta | h)              # một bước gradient
        p <- eval(theta)                       # đánh giá hiệu năng hiện tại
        nếu ready(p, t, P):                    # đủ điều kiện (vd đủ số bước)
            h', theta' <- exploit(h, theta, p, P)   # dùng cả quần thể để tìm lời giải
            ↪ tốt hơn
            nếu theta != theta':
                h, theta <- explore(h', theta', P)  # đề xuất siêu tham số mới
                p <- eval(theta)
        cập nhật P với (theta, h, p, t+1)
trả về theta có p cao nhất trong P
```

Tính chất quan trọng: PBT **tham lam (greedy)** ở mỗi bước exploit/explore, nhưng toàn bộ quá trình (meta-optimisation) vẫn hiệu quả nhờ chia sẻ thông tin liên tục qua quần thể. PBT không yêu cầu đồng bộ hoá tập

trung; chỉ cần dữ liệu hiệu năng/trọng số/siêu tham số hiện tại của mỗi thành viên khả dụng toàn cục (vd qua một key-value store, Phụ lục A.1).

3.3 Ví dụ toy minh hoạ (Mục 3, Hình 2)

Hàm mục tiêu thật $Q(\theta) = 1{,}2 - (\theta_0^2 + \theta_1^2)$ (không biết công thức), chỉ có surrogate khả vi $\hat{Q}(\theta \mid h) = 1{,}2 - (h_0\theta_0^2 + h_1\theta_1^2)$ để lấy gradient; có thể đọc Q ra để đánh giá. Khởi tạo $\theta_0 = [0{,}9,\ 0{,}9]$, chỉ có 2 worker. Grid/random search chỉ thử được $h = [1, 0]$ (đen) và $h = [0, 1]$ (đỏ): cả hai hội tụ xa điểm tối ưu thật ($Q(\theta) \approx 0{,}4$) vì không khám phá đủ không gian h (nếu chọn $h = [1, 1]$ sẽ khớp mục tiêu thật, nhưng giả định không biết trước điều này).

Khi áp PBT (mỗi 4 lần lặp, worker yếu copy lời giải của worker mạnh rồi nhiễu nhẹ hướng cập nhật), quần thể hội tụ về **tối ưu toàn cục**. Hai thí nghiệm ablation trên toy này: chỉ dùng explore hoặc chỉ dùng exploit cho thấy **phần lớn lợi ích đến từ exploit** (sao chép lời giải), explore chỉ cho cải thiện nhỏ thêm. Paper ghi nhận kết quả tương tự lặp lại trên các bài toán thật ở Mục 4.4.

3.4 Các biến thể exploit/explore dùng trong thí nghiệm

- **Exploit (a) T-test selection:** lấy mẫu ngẫu nhiên một worker khác trong quần thể, so sánh 10 reward episode gần nhất bằng kiểm định t của Welch (Welch, 1947); nếu worker được lấy mẫu có reward trung bình cao hơn và thoả t-test, sao chép trọng số + siêu tham số của nó.
- **Exploit (b) Truncation selection:** xếp hạng toàn quần thể theo reward; nếu worker hiện tại thuộc 20% dưới cùng, lấy mẫu ngẫu nhiên một worker trong 20% trên cùng và sao chép.
- **Exploit cho GAN:** thêm **binary tournament**: mỗi worker chọn ngẫu nhiên một worker khác, sao chép nếu worker đó điểm tốt hơn.
- **Explore (a) Perturb:** mỗi siêu tham số bị nhiễu độc lập bằng hệ số nhân 1,2 hoặc 0,8 (RL, dịch máy); với GAN dùng hệ số mạnh hơn 2,0 hoặc 0,5.
- **Explore (b) Resample:** mỗi siêu tham số được resample lại từ phân phối tiên nghiệm ban đầu với một xác suất nào đó.

4. Thí nghiệm và kết quả chính

4.1 Reinforcement learning (Mục 4.1)

Ba miền: UNREAL (Jaderberg et al. 2016) trên DeepMind Lab, Feudal Networks (FuN, Vezhnevets et al. 2017) trên Atari, và A3C đơn giản trên StarCraft II. Mỗi worker là một agent A3C bất đồng bộ (Mnih et al. 2016). Kích thước quần thể dùng: 10 đến 80.

- **UNREAL trên DM Lab:** PBT tune learning rate, entropy cost, unroll length. Ready khi đã trải qua 1×10^6 đến 10×10^6 bước agent kể từ lần ready trước. Với quần thể 40 worker trên 15 level, PBT tăng hiệu năng chuẩn hoá theo người (human-normalised performance) trung bình từ **93% lên 106%** (tự tính: tăng tuyệt đối 13 điểm phần trăm, tăng tương đối $(106 - 93)/93 \approx 14{,}0\%$).
- **FuN trên Atari (4 game: Amidar, Gravitar, Ms-Pacman, Enduro), quần thể 80 worker:** PBT tune learning rate, entropy cost, intrinsic reward cost. Hiệu năng chuẩn hoá trung bình tăng từ **147% lên 181%** (tự tính: tăng tuyệt đối 34 điểm phần trăm, tương đối $(181 - 147)/147 \approx 23{,}1\%$). Riêng ms-pacman: từ 6506 lên 9001 (tự tính: tăng tuyệt đối 2495, tương đối $2495/6506 \approx 38{,}4\%$), được paper gọi là state-of-the-art cho agent chỉ nhận pixel.
- **A3C trên StarCraft II (6 mini-game), quần thể 30 worker, chỉ tune learning rate:** hiệu năng chuẩn hoá trung bình tăng từ **36% lên 39%** (tự tính: tăng tuyệt đối 3 điểm phần trăm, tương đối $3/36 \approx 8{,}3\%$).
- Paper diễn giải PBT giúp qua 2 cơ chế: (i) siêu tham số được tập trung vào vùng tốt và thích ứng theo thời gian (vd learning rate được "anneal" tự động trên `lt_horse_shoe_color`; unroll length tăng dần trên `nav_maze_random_goal_01`, giúp agent học dùng bộ nhớ hồi quy tốt hơn, đạt hiệu năng siêu nhân

trên level này); (ii) việc sao chép trọng số lan truyền ”may rủi khám phá tốt” từ các worker gặp may sang các worker khác.

Chi tiết phụ lục A.2 (bảng số, mẫu một vài dòng để minh hoạ): FuN trên ms-pacman với 4 mức quần thể (Pop=10,20,40,80): không-PBT đạt 5635, 5280, 6177, 6506; có PBT đạt 5509, 5714, 8124, 9001 (điểm chuẩn hoá chưa nhân 100, đơn vị điểm game thô theo Bảng 1). UNREAL trên `nav_maze_random_goal_01` (Pop=40): không-PBT 114,2, có PBT 149,4; với Pop=20 còn có bản ablation `PBT-Hypers` (chỉ đổi siêu tham số, không copy trọng số) đạt 81,5 và `PBT-Weights` (chỉ copy trọng số, không đổi siêu tham số) đạt 74,2, so với PBT đầy đủ 96,1 và UNREAL gốc (Pop=20) 82,2 (Bảng 2).

4.2 Dịch máy (machine translation, Mục 4.2)

- Mô hình Transformer (Vaswani et al. 2017) ”base, single GPU” cho WMT 2014 English-to-German; baseline là lịch learning rate/dropout đã tune kỹ bằng tay + Bayesian optimisation.
- Quần thể 32 worker, mỗi worker 1 GPU, huấn luyện $400 \times 10^3 = 400.000$ bước. PBT tune learning rate, attention dropout, layer dropout, ReLU dropout.
- Ready mỗi $2 \times 10^3 = 2.000$ bước; exploit bằng T-test selection; explore bằng perturb (1,2/0,8).
- Kết quả: BLEU trên validation (newstest2012) tăng từ **23,71 lên 24,23** (tự tính: $+0{,}52$ điểm, tương đối $0{,}52/23{,}71 \approx 2{,}2\%$); BLEU trên test (newstest2014, tính bằng `multi-bleu.pl`) tăng từ **22,30 lên 22,65** (tự tính: $+0{,}35$ điểm, tương đối $0{,}35/22{,}30 \approx 1{,}6\%$). Để so sánh, cấu hình siêu tham số gốc (không PBT, không tune) huấn luyện cùng số bước chỉ đạt BLEU **21,23**, thấp hơn cả hai baseline.
- Mô hình ”base single-GPU” có cùng số tham số với bản chạy trên 8 GPU ($6{,}5 \times 10^7 \approx 65$ triệu tham số) nhưng mỗi bước chỉ thấy 2048 token so với $8 \times 4096 = 32.768$ token của bản 8 GPU: tỉ lệ $2048/32768 = 1/16$ (tự tính, khớp với mô tả ”1/16 số token” của paper, dù pdftotext hiển thị lệch thành ”16 1” do mất định dạng phân số).
- Lịch learning rate PBT tự tìm ra ”giống đáng ngạc nhiên” với lịch tune tay: ban đầu nhỏ, nhảy lên 3 bậc độ lớn, sau đó giảm dần kiểu hàm mũ; đồng thời PBT tự tìm ra lịch dropout phi đơn điệu (tăng gần 1 bậc độ lớn, giảm về giá trị đầu, rồi tăng lại chậm) mà chuyên gia không thiết kế.

4.3 Generative Adversarial Networks (Mục 4.3)

- Kiến trúc giống DCGAN (Radford et al. 2016), tối ưu bằng WGAN-GP (Gulrajani et al. 2017); mỗi bước gồm $K = 5$ lần cập nhật discriminator rồi 1 lần cập nhật generator (Adam, $\beta_1 = 0{,}5$, $\beta_2 = 0{,}999$, $\epsilon = 10^{-8}$, batch size 128, theo Phụ lục A.3).
- Quần thể 45 worker, mỗi worker huấn luyện 10^6 bước; PBT tune learning rate riêng cho generator và discriminator. Ready mỗi $5 \times 10^3 = 5.000$ bước; explore dùng perturb mạnh hơn (2,0/0,5).
- Khởi tạo learning rate từ $\{1, 2, 5\} \times 10^{-4}$ cho mỗi mạng, tổng $3^2 = 9$ tổ hợp ban đầu (pdftotext hiển thị nhầm thành ”32 = 9” do mất ký hiệu luỹ thừa), mỗi tổ hợp lặp lại 5 lần với khởi tạo trọng số ngẫu nhiên độc lập: $9 \times 5 = 45$ (tự tính, khớp đúng population size 45).
- Baseline tốt nhất (tìm bằng search kỹ các lịch anneal learning rate khác nhau, huấn luyện cùng 45 worker): đạt **CIFAR Inception score 6,39** (câu kết luận Mục 4.3.2) tương ứng ImageNet Inception 6,45; tuy nhiên Bảng 4 (Phụ lục A.3) ghi số liệu khác nhẹ: GAN baseline CIFAR Inception $6{,}38 \pm 0{,}03$, ImageNet (theo CIFAR) $6{,}45 \pm 0{,}05$ **(mâu thuẫn nhỏ 6,39 so với 6,38 giữa văn bản chính và Bảng 4, paper không giải thích; có thể là làm tròn khác nhau)**.
- PBT (truncation selection) đạt **CIFAR Inception 6,80** và ImageNet Inception tương ứng **6,89** (khớp cả văn bản chính và Bảng 4: $6{,}80 \pm 0{,}01$ và $6{,}89 \pm 0{,}04$); PBT (binary tournament) đạt CIFAR Inception $6{,}77 \pm 0{,}02$. Tự tính cải thiện tương đối CIFAR Inception theo Bảng 4: $(6{,}80 - 6{,}38)/6{,}38 \approx 6{,}6\%$; theo văn bản chính: $(6{,}80 - 6{,}39)/6{,}39 \approx 6{,}4\%$.
- ImageNet Inception ”peak” (điểm cao nhất đạt được tại bất kỳ thời điểm nào, không phải điểm chọn cuối huấn luyện) của PBT-GAN(TS) là $7{,}02 \pm 0{,}07$, cao hơn cả điểm ”theo CIFAR” cuối cùng.

4.4 Phân tích thiết kế PBT (Mục 4.4, dùng Hình 5)

- **Kích thước quần thể:** với FuN trên Atari, quần thể ≤ 10 cho phương sai cao, kết quả kém; quần thể 20-40 đủ để cải thiện ổn định; quần thể lớn hơn vẫn tốt hơn nhưng lợi ích giảm dần.
- **Loại exploit:** với GAN, truncation selection tốt hơn binary tournament (quan sát sơ bộ tương tự với FuN trên Atari).
- **Đối tượng PBT áp dụng:** trên UNREAL/DM Lab, so sánh (i) chỉ exploit/explore siêu tham số (không copy trọng số) và (ii) chỉ copy trọng số (không đổi siêu tham số): cả hai chỉ cải thiện một phần; **kết hợp cả hai** (PBT đầy đủ) cho hiệu năng tốt nhất.
- **Tính thích ứng của siêu tham số:** so sánh PBT đầy đủ (siêu tham số thay đổi suốt huấn luyện) với việc chỉ lấy **bộ siêu tham số PBT tìm được ở cuối** và dùng cố định từ đầu: PBT đầy đủ tốt hơn rõ rệt, cho thấy lợi ích chính nằm ở **khả năng thích ứng theo thời gian**, không chỉ ở việc tìm một điểm tốt trong không gian siêu tham số.

5. Phân tích / ablation

- Ablation toy (Mục 3.3, Hình 2): tách exploit và explore, thấy exploit (copy lời giải) đóng góp phần lớn cải thiện, explore chỉ thêm một lượng nhỏ, nhất quán với ablation PBT Targets trên UNREAL (Hình 5c).
- Cây phả hệ (phylogenetic tree, Hình 6, Mục 4.4 ”Phylogenetic trees”): tất cả agent cuối cùng đều là hậu duệ của **đúng một** agent khởi tạo (vì exploit chỉ dùng một agent khác tại một thời điểm, số tổ tiên gốc chỉ có thể giảm theo thời gian); tuy vậy, trước khi hội tụ về một ”người thắng”, có những nhánh phụ tồn tại lâu, cho thấy PBT có khám phá nhiều lời giải hứa hẹn trước khi chọn, khả năng mà ”most typical optimisers lack” theo paper.
- So sánh định tính GAN và Atari (Hình 6): ở GAN hầu hết thành viên cải thiện so với ”cha mẹ” sau khi copy; ở Atari, ngay cả muộn trong huấn luyện vẫn có bước explore làm giảm mạnh reward so với cha mẹ, một dấu hiệu bất ổn định của RL trên bài toán phức tạp mà PBT ”có vị trí tốt để sửa”.
- Lineage (Hình 7): lịch learning rate PBT tìm cho GAN và dịch máy có dạng tương tự lịch tune tay nhưng **mạnh hơn** (discriminator anneal gắt hơn, learning rate dịch máy nhảy sớm và dốc hơn); dropout dịch máy có lịch phi đơn điệu phức tạp mà ”vượt khỏi không gian lịch mà chuyên gia con người xem xét”.

6. Giới hạn

Do tác giả thừa nhận hoặc gợi ý (rải trong Mục 3, 4.4, Kết luận):

- PBT là thuật toán **tham lam** (greedy) ở mỗi bước exploit, có thể kẹt vào local optimum nếu quần thể không đủ đa dạng (liên hệ trực tiếp tới kết quả ”quần thể nhỏ cho phương sai cao” ở Mục 4.4).
- Cây phả hệ hội tụ về một tổ tiên duy nhất là hệ quả của cơ chế exploit chỉ dùng 1 agent khác; đây là tính chất cố hữu của thiết kế cụ thể này (không phải PBT nói chung), và paper không khám phá biến thể cho phép đa dạng hơn.
- Thí nghiệm trên dịch máy và GAN dùng mô hình/batch nhỏ hơn cấu hình tốt nhất đã biết (paper tự ghi ”small Transformer network using a reduced batch size, so is not representative of state of the art”), nên kết quả BLEU không nên dùng làm con số SOTA.
- Chưa có phân tích lý thuyết về độ hội tụ hay guarantee kiểu regret bound (khác với Hyperband, có phân tích lý thuyết).

Do chúng ta nhận thấy:

- Phát hiện mâu thuẫn nhỏ số liệu baseline GAN: văn bản chính Mục 4.3.2 ghi CIFAR Inception baseline là 6,39, nhưng Bảng 4 (Phụ lục A.3) ghi $6{,}38 \pm 0{,}03$; paper không giải thích sự khác biệt này (có thể là làm tròn hoặc lấy từ run khác), người đọc nên dùng số trong Bảng 4 làm chuẩn vì có kèm sai số.

- PBT đòi hỏi **nhiều worker chạy song song cùng lúc** (quần thể 10 đến 80 trong các thí nghiệm), nên chi phí tính toán **đồng thời** (không phải tổng compute) cao hơn nhiều so với huấn luyện một mô hình đơn; paper có đề cập chế độ "semi-serial/partial synchrony" (Phụ lục A.1) cho hạ tầng rẻ hơn nhưng không có số liệu thực nghiệm minh hoạ.
- Không so sánh trực tiếp bằng số với Hyperband hay Bayesian optimisation trên cùng bài toán (chỉ so sánh với random search cùng số worker và với baseline tune tay); khó đánh giá PBT tốt hơn các phương pháp opt 31-33 bao nhiêu trên cùng ngân sách tính toán.
- Phần "Ready" dùng ngưỡng khác nhau tuỳ miền (vd 1-10 $\times$ 10^6 bước cho RL, 2×10^3 bước cho dịch máy, 5×10^3 bước cho GAN) nhưng paper không cho hướng dẫn tổng quát để chọn ngưỡng này cho bài toán mới.

7. Liên hệ với các bài khác

- **opt 31** (Hyperband): cùng mục tiêu tune siêu tham số nhanh bằng song song hoá, nhưng dựa trên successive halving/bandit và cần phân bổ "ngân sách" (vd số epoch) ngay từ đầu cho nhiều cấu hình; PBT khác biệt ở việc **thích ứng liên tục** trong một tiến trình huấn luyện duy nhất, điều Hyperband không làm được (paper này tự so sánh ở Mục 2 "Related Work").
- **opt 32, opt 33** (Bayesian optimization; file "optimization content/32_2012_practical_bayesian_optimization.md" và "optimization content/33_2018_bayesian_optimization_tutorial.md" đã có trong cùng thư mục): PBT được paper liệt vào nhóm vấn đề mà sequential optimisation (BO) giải quyết nhưng BO cần nhiều lần chạy tuần tự; GP-UCB, TPE, Spearmint, SMAC được paper trích dẫn trực tiếp ở Mục 2 là các phương pháp BO PBT muốn tránh nhược điểm tuần tự của chúng.
- **rl 06** (A3C, đã kiểm tồn tại bằng `ls` thư mục "rl content/"): toàn bộ thí nghiệm RL của PBT (UNREAL trên DM Lab, A3C trên StarCraft II) xây trên agent kiểu A3C bất đồng bộ; hiểu A3C là điều kiện cần để hiểu vì sao PBT "chạy song song, bất đồng bộ" ăn khớp tự nhiên với framework A3C.
- Đã kiểm tra `ls` các thư mục cũ còn lại ("llm content/", "distillation content/", "pruning content/", "interp content/"): không thấy bài trực tiếp về population-based hyperparameter search hoặc PBT, nên không trỏ thêm.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1,6-2,1K token visual, chạy trên GPU <= 8GB, batch 1). Nhận định dưới đây là suy luận:

1. **PBT khó áp dụng trực tiếp vì cần nhiều worker song song.** Mọi thí nghiệm của paper dùng quần thể 9 đến 80 worker chạy **đồng thời**; trên 1 GPU 8GB, không thể chạy nhiều bản sao mô hình 0,8B cùng lúc với batch đủ lớn để huấn luyện hiệu quả. Đây là hạn chế giống với nhận xét ở opt 31-33 về song song hoá: PBT **ít liên quan trực tiếp** cho pipeline 1-GPU của dự án.
2. **Ý tưởng con có thể mượn: lịch siêu tham số thích ứng thay vì cố định.** Phát hiện cốt lõi của PBT (Mục 4.4, "Hyperparameter Adaptivity") là lợi ích chính nằm ở việc **learning rate/dropout thay đổi theo tiến trình huấn luyện** dựa trên tín hiệu hiệu năng thực tế, không chỉ ở việc tìm một giá trị tốt cố định. Khi fine-tune/distill mô hình đã cắt, có thể mô phỏng ý tưởng này bằng một lịch learning rate/hệ số λ_{KD} **phản hồi theo validation loss** (vd giảm lr khi loss chững lại), dù đây không phải PBT thật (không có quần thể để so sánh).
3. **"Giả PBT" tuần tự trên 1 GPU (suy luận):** có thể mô phỏng một quần thể nhỏ (vd 4-8 cấu hình) bằng cách chạy **tuần tự từng đoạn ngắn** mỗi cấu hình, rồi ở mỗi "checkpoint" so sánh hiệu năng và loại bỏ/nhân bản cấu hình kém giống exploit, trước khi tiếp tục; đây làm mất tính "không tốn thời gian hơn 1 lần huấn luyện" (ưu điểm chính của PBT) nên chỉ hợp khi ngân sách thời gian wall-clock không gấp, chỉ cần tiết kiệm số lần "đánh giá cuối cùng tốn kém" như GP-based BO.

4. **So với opt 31-33:** nếu chỉ có 1 GPU và cần tune vài siêu tham số liên tục, Hyperband (opt 31) hoặc Bayesian optimization (opt 32/33) khả thi hơn PBT vì không yêu cầu nhiều worker đồng thời, dù mất khả năng ”thích ứng online” mà PBT có.

9. Thí nghiệm gợi ý

1. **Tái hiện toy example của paper (Mục 3.3) trên máy cá nhân, không cần GPU:** cài $Q(\theta) = 1{,}2 - (\theta_0^2 + \theta_1^2)$ và $\hat{Q}(\theta \mid h) = 1{,}2 - (h_0\theta_0^2 + h_1\theta_1^2)$, chạy gradient ascent trên $\hat{Q}$ với 2-4 ”worker”, thêm bước exploit (copy worker tốt nhất) và explore (nhiễu h) mỗi vài vòng; đo $Q(\theta)$ cuối cùng so với chạy không có PBT (random h cố định). Kỳ vọng: PBT tiến gần $Q \approx 1{,}2$ hơn random cố định h.
2. **”Giả PBT” tuần tự tune learning rate cho mạng nhỏ (1 GPU 8GB):** mô phỏng quần thể 4 worker bằng cách huấn luyện tuần tự 4 cấu hình learning rate khác nhau trong N bước, rồi tại checkpoint, cấu hình tệ nhất ”copy” trọng số + nhiễu lr của cấu hình tốt nhất (tự triển khai bằng lưu/load checkpoint PyTorch); lặp lại k chu kỳ. So sánh với 4 cấu hình chạy độc lập không exploit/explore (tương đương random search). Đo validation loss cuối. Diễn giải: nếu ”giả PBT” tốt hơn 4 lần random độc lập ở cùng tổng số bước, xác nhận lợi ích của exploit ngay ở quy mô nhỏ.
3. **Đo ablation ”chỉ copy trọng số” và ”chỉ đổi siêu tham số”.** Lặp lại thí nghiệm 2 nhưng tách riêng: (a) chỉ copy trọng số khi exploit (không đổi lr), (b) chỉ đổi lr khi explore (không copy trọng số). So sánh cả ba biến thể với ”PBT đầy đủ”; kỳ vọng kết hợp cả hai tốt nhất, giống Hình 5c của paper.
4. **Perturb factor ảnh hưởng ra sao.** Với cùng thiết lập thí nghiệm 2, thử hệ số perturb nhẹ (1,2/0,8 như RL/dịch máy) so với mạnh (2,0/0,5 như GAN); đo độ ổn định (phương sai loss giữa các lần chạy) và tốc độ hội tụ.
5. **Lịch learning rate PBT tìm ra có giống lịch cosine/step decay thường dùng không?** Ghi lại lịch lr theo thời gian của worker thắng cuối cùng trong thí nghiệm 2, vẽ đồ thị, so sánh hình dạng với cosine schedule hoặc step decay thông thường (liên hệ opt 17, opt 18 nếu đã có trong thư mục).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Mục tiêu PBT	$\theta^* = \text{optimise}(\theta \mid h^*)$, $h^* = \arg\max_{h \in \mathcal{H}^T} \text{eval}(\text{optimise}(\theta \mid h))$ (công thức 4)
Bước cập nhật	$\theta \leftarrow \text{step}(\theta \mid h)$ (công thức 2)
exploit	sao chép (θ, h) từ worker tốt hơn: T-test selection hoặc truncation selection (bottom 20%/top 20%)
explore	nhiễu h (nhân 1,2/0,8 hoặc 2,0/0,5) hoặc resample từ prior
RL (DM Lab, UNREAL, pop 40)	93% $\rightarrow$ 106% human performance (tự tính $+14{,}0\%$ tương đối)
RL (Atari, FuN, pop 80)	147% $\rightarrow$ 181% human performance (tự tính $+23{,}1\%$ tương đối)
RL (StarCraft II, A3C, pop 30)	36% $\rightarrow$ 39% human performance (tự tính $+8{,}3\%$ tương đối)
Dịch máy (Transformer, pop 32)	BLEU validation $23{,}71 \rightarrow 24{,}23$; test $22{,}30 \rightarrow 22{,}65$; không-tune $= 21{,}23$
GAN (CIFAR, pop 45)	CIFAR Inception $6{,}38$-$6{,}39 \rightarrow 6{,}80$ (TS) hoặc $6{,}77$ (BT)
Kích thước quần thể hiệu quả	≥ 20: cải thiện ổn định; ≤ 10: phương sai cao

11. Kết luận ngắn

Population Based Training lai giữa parallel search và sequential optimisation bằng một quần thể worker bất đồng bộ, dùng exploit (sao chép trọng số/siêu tham số từ worker tốt hơn) và explore (nhiễu siêu tham số) để tự động khám phá một **lịch siêu tham số thích ứng**, không chỉ một bộ giá trị cố định. Trên RL (UNREAL, FuN, A3C), dịch máy (Transformer/WMT14), và GAN (CIFAR), PBT cải thiện rõ so với random search cùng

số worker và thậm chí vượt baseline tune tay kỹ (dịch máy, GAN). Hạn chế lớn nhất là yêu cầu **nhiều worker chạy đồng thời**, khiến PBT ít phù hợp với môi trường chỉ có 1 GPU như pipeline nén/distill của dự án OCR; các ý tưởng về lịch siêu tham số thích ứng theo tín hiệu validation vẫn có thể mượn ở quy mô nhỏ.

Nguồn gốc: https://arxiv.org/abs/1711.09846

Bài 35

Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer (2022)

Tác giả: Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, Jianfeng Gao (Microsoft Corporation, OpenAI).
Venue / arXiv: arXiv:2203.03466 (v2, 28/03/2022); paper không nêu venue hội nghị trong văn bản tải được (không có dòng ”Published as a conference paper at...” hay copyright hội nghị ở đầu trang 1 hoặc ở trang cuối, khác với nhiều bài ICLR/NeurIPS khác).
Nguồn: https://arxiv.org/abs/2203.03466
Chủ đề: Maximal Update Parametrization (viết tắt muP): một quy tắc scale khởi tạo, learning rate và các hệ số nhân (multiplier) theo width sao cho HP optimum (learning rate, init std, multiplier...) gần như không đổi khi width tăng; từ đó suy ra kỹ thuật muTransfer: tune hyperparameter trên một proxy model nhỏ rồi copy thẳng (zero-shot, không tune lại) sang model lớn.
Vị trí trong lộ trình: nên đọc sau khi đã quen với Adam (opt 06) và ít nhất một bài về scale learning rate theo batch size/model size (opt 10, LAMB) để thấy rõ muP giải quyết vấn đề gì mà các luật scale LR ”thủ công” trước đó chưa giải quyết triệt để. Đọc ngay trước opt 36 (Scaling Exponents Across Parameterizations and Optimizers), bài kiểm định lại và mở rộng chính kỹ thuật muP ở đây sang nhiều optimizer và quy mô lớn hơn.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Paper có rất nhiều công thức abc-parametrization; người ghi chú đã đọc toàn bộ thân bài và toàn bộ phụ lục A-J, nhưng với phần bảng công thức bị lỗi OCR (xem mục 3 và mục 6) đã ưu tiên đối chiếu lại bằng văn bản suy diễn rõ ràng trong Appendix J.2.1 thay vì đoán theo hình bảng.

1. Bối cảnh

Hyperparameter (HP) tuning (learning rate, init, batch size, momentum, các hệ số nhân...) là bước bắt buộc nhưng rất đắt trong deep learning. Với các mạng hàng trăm triệu đến hàng tỷ tham số, tune trực tiếp (grid/random search nhiều lần chạy full-scale) gần như không khả thi về compute. Thực hành phổ biến là ”đoán” HP của model lớn dựa trên model nhỏ, nhưng theo paper, cách đoán này ”hit-or-miss” vì **parametrization sai**: cách khởi tạo và scale learning rate theo width quyết định việc HP optimum có ổn định qua width hay không, và phần lớn framework deep learning mặc định dùng Standard Parametrization (SP, kiểu init LeCun/He mặc định của PyTorch) mà không scale theo width.

Nền tảng lý thuyết: bài trước trong series Tensor Programs ([57] = Yang & Hu, ”Feature learning in infinite-width neural networks”, còn gọi là TP4) chỉ ra rằng các parametrization khác nhau dẫn tới infinite-width limit khác nhau, và đề xuất Maximal Update Parametrization (muP) cho phép mọi layer học ”feature” một cách tối đa trong limit đó. Trực giác: muP đảm bảo mỗi layer được cập nhật cùng một ”tốc độ” (cùng bậc độ lớn) trong quá trình train, bất kể width, còn SP giữ activation có order 1 lúc **khởi tạo** nhưng lại khiến activation ”nổ” (blow up) theo width trong quá trình **train** do mất cân bằng update giữa các layer (ví dụ: input layer học quá chậm, output layer học quá nhanh, hoặc ngược lại).

Bài này là bài số 5 trong series Tensor Programs (TP I-IV xây nền lý thuyết trừu tượng hơn); tác giả mô tả nó ”self-contained”, hướng tới practitioner, và là ”payoff thực hành đầu tiên” của toàn bộ nền tảng lý thuyết trước đó.

Khái niệm cần nắm trước khi đọc tiếp:

- **Width**: số chiều của một layer ẩn (ví dụ d_{model} của Transformer, hoặc số hidden unit n của MLP). Paper phân loại mỗi chiều trong network là ”vô hạn” (scale theo width, ví dụ $d_{model}, d_{ffn}, n_{head}$) hay ”hữu hạn” (cố định, ví dụ vocab size, context size).
- **Parametrization**: không phải là chọn một giá trị HP cụ thể, mà là **quy tắc** scale HP khi width thay đổi (ví dụ ”init variance giảm một nửa khi width tăng gấp đôi”). SP và muP đều là parametrization theo nghĩa này.
- **Zero-shot hyperparameter transfer**: nếu một parametrization khiến HP optimum (theo một metric, ở đây là training loss) hội tụ khi width tăng, ta có thể tune HP ở width nhỏ rồi copy thẳng (không tune lại) sang width lớn và kỳ vọng gần tối ưu.

Câu hỏi trung tâm: có tồn tại một parametrization sao cho HP optimum ổn định qua width hay không, và nếu có thì tận dụng nó để giảm chi phí tune HP cho model khổng lồ như thế nào?

2. Ý tưởng chính

Paper dùng một ví dụ tối giản theo Central Limit Theorem (CLT) để minh hoạ ”parametrization đúng” nghĩa là gì. Xét $x_1, \dots, x_n$ là mẫu iid, zero-mean, unit-variance. Muốn tối thiểu hoá $F_n(c) = \mathbb{E}\, f(c(x_1 + \cdots + x_n))$ theo c. Nếu đặt $c = \alpha/\sqrt{n}$ (đúng theo CLT), thì $G_n(\alpha) = F_n(c)$ **ổn định** thành một hàm của α khi $n \to \infty$, và $\alpha_n^* = \arg\min_\alpha G_n(\alpha)$ gần đúng với α_N^* với $N > n$ bất kỳ (kể cả $N = \infty$), nghĩa là có thể copy α^* từ bài toán nhỏ sang bài toán lớn. Nếu đặt $c = 1/n$ (sai) thì $F_n(c) \to 0$; nếu đặt $c = 1$ (cũng sai) thì phương sai nổ theo n, cả hai trường hợp sai đều không có ”optimum ổn định” để copy.

Trong ngữ cảnh neural network: $x_1, \dots, x_n$ giống như tham số được khởi tạo ngẫu nhiên của một mạng width-n; c giống một HP như learning rate; f là hiệu năng sau khi train. muP chính là cách đặt $c = \alpha/\sqrt{n}$ ”đúng” này cho toàn bộ tham số của network (SP tương ứng cách đặt sai, kiểu c). Về lý thuyết, mạng muP có một infinite-width limit well-defined (giống $(x_1 + \cdots + x_n)/\sqrt{n} \to N(0,1)$), còn mạng SP không có limit tốt (nổ) hoặc rơi vào kernel regime không học feature (xem mục 3 và mục 6).

muTransfer = quy trình tận dụng tính chất này (Algorithm 1 trong paper):

1. Parametrize **target model** (model lớn muốn tune) theo muP.
2. Tune một bản **nhỏ hơn** (hẹp hơn và/hoặc ít tầng hơn) của chính target model đó.
3. Copy nguyên vẹn hyperparameter đã tune sang target model, **không tune lại**.

5 lợi ích tác giả nêu: (1) **hiệu năng tốt hơn** SP đã tune LR tối ưu (vì muP còn sửa đúng cả parameter multiplier và LR của input/output layer, không chỉ LR chung); (2) **tiết kiệm compute** rất lớn khi model đích hàng tỷ tham số; (3) **tune một lần cho cả họ model** (ví dụ BERT-base và BERT-large cùng lúc từ một proxy); (4) **tận dụng compute tốt hơn** (model nhỏ tune trên 1 GPU, model lớn train phân tán nhiều GPU riêng); (5) **chuyển từ giai đoạn explore sang scale-up không mất HP** đã tìm được ở model nhỏ.

Bảng phân loại HP (Table 1 và Table 2 của paper, dùng xuyên suốt phần còn lại):

Nhóm	Ví dụ HP
muTransferable (chuyển được qua width, có cơ sở lý thuyết; qua depth/batch/seq-len/thời gian train chỉ kiểm thực nghiệm)	learning rate, momentum, Adam beta, LR schedule, init variance từng layer, parameter multiplier
KHÔNG muTransfer tốt	HP chủ yếu để regularize: dropout, weight decay
HP được dùng làm ”trục” để transfer qua (không phải cái cần transfer)	width, depth*, batch size*, sequence length*, training time*

(dấu * = chỉ kiểm chứng thực nghiệm trên Transformer trong paper; width có thêm cơ sở lý thuyết đầy đủ).

3. Phương pháp chi tiết

3.1 abc-parametrization (Appendix A)

Mỗi tensor tham số W trong network có 3 ”núm chỉnh” theo định nghĩa của paper: một **multiplier** A (tensor thực chất lưu là $A\hat{W}$ với $\hat{W}$ mới là phần được học), một **init variance** B^2 (khởi tạo $\hat{W} \sim N(0, B^2)$), và một **learning rate** C. Một ”abc-parametrization” là quy tắc scale (A, B, C) theo width cho MỖI tensor riêng biệt (các HP khác giữ cố định theo width). SP và muP đều là abc-parametrization, chỉ khác ở quy tắc scale.

3.2 Ví dụ MLP minh hoạ (công thức (2)-(4) của paper)

MLP 2 lớp ẩn: $f(\xi) = W_3^\top \phi(W_2^\top \phi(W_1^\top \xi + b_1) + b_2)$, ϕ = ReLU, hidden width n. SP (công thức (2)): $W_1 \sim N(0, 1/d_{in})$, $W_2, W_3 \sim N(0, 1/n)$, bias $= 0$, learning rate chung η cho mọi tensor.

muP dạng cơ bản (công thức (3)): giữ $W_1 \sim N(0, 1/d_{in})$, $W_2 \sim N(0, 1/n)$ (không đổi so với SP), nhưng **đổi** $W_3 \sim N(0, 1/n^2)$ (output layer init nhỏ hơn SP theo width), và learning rate SGD: $\eta_{W_1} = \eta_{b_1} = \eta_{b_2} = \eta n$, $\eta_{W_2} = \eta$, $\eta_{W_3} = \eta n^{-1}$.

Để muP trùng khớp SP tại một base width n_0 (hữu ích khi muốn giữ hành vi cũ ở một kích thước quen thuộc), paper chèn thêm hệ số tỉ lệ width $\tilde{n} = n/n_0$ (công thức (4)): $W_3 \sim N(0, 1/(n\tilde{n}))$, $\eta_{W_1} = \eta_{b_1} = \eta_{b_2} = \eta\tilde{n}$, $\eta_{W_3} = \eta\tilde{n}^{-1}$. Tại $n = n_0$ thì $\tilde{n} = 1$ và công thức (4) trở về đúng SP (công thức (2)); khi n tăng, (4) lệch dần khỏi (2), và chính độ lệch này mới tạo ra khả năng transfer HP.

Tự tính minh hoạ bằng số cụ thể: lấy $n_0 = 256$, $n = 1024$ thì $\tilde{n} = 1024/256 = 4$. Theo công thức (4): $W_3 \sim N(0, 1/(1024 \times 4)) = N(0, 1/4096)$, trong khi SP tại $n = 1024$ sẽ là $N(0, 1/1024)$, std của muP nhỏ hơn SP đúng $\sqrt{4} = 2$ lần (tự tính). Learning rate của W_1, b_1, b_2 trong muP là 4η (gấp 4 lần base), còn của W_3 là $\eta/4$ (tự tính, theo đúng $\tilde{n} = 4$ và $\tilde{n}^{-1} = 1/4$).

3.3 Quy tắc muP tổng quát cho mọi kiến trúc (Table 3 của paper)

Paper phân loại tensor theo số **chiều vô hạn** (chiều scale theo width): **matrix-like** (2 chiều vô hạn, ví dụ hidden weights $n \times n$), **vector-like** (1 chiều vô hạn, ví dụ input/output weights và mọi bias), **scalar-like** (0 chiều vô hạn, ví dụ hệ số nhân learnable, giữ nguyên không đổi theo width). Dưới đây là bảng quy tắc, được người ghi chú dựng lại từ đúng phần suy diễn chữ (không phải đoán từ hình bảng bị lỗi OCR, xem lưu ý ở mục 6) trong Appendix J.2.1, và đối chiếu khớp với ví dụ MLP ở công thức (2)-(4) bên trên cho cột SGD LR:

Loại tham số	Init. Variance	SGD LR	Adam LR
Input weights & mọi bias	Θ(1/fan_in), nhưng fan_in ở đây hữu hạn (không đổi theo width) nên thực chất là Θ(1)	Θ(fan_out)	Θ(fan_out)
Output weights	Θ(1/fan_in²)	Θ(1/fan_in)	Θ(1/fan_in)
Hidden weights (ma trận vuông, fan_in = fan_out = width)	Θ(1/fan_in)	Θ(1)	Θ(1/fan_in)

(`fan_in`, `fan_out` tính theo đúng tensor đang xét: với input weight, `fan_in` = chiều vào hữu hạn, `fan_out` = width; với output weight thì ngược lại; với hidden weight cả hai đều là width.) So với SP (giữ init variance = `1/fan_in` cho mọi loại tensor và LR không đổi theo width): muP khác SP rõ nhất ở (a) init variance của **output weights** nhỏ hơn SP một bậc nữa (chia thêm `fan_in`), và (b) learning rate của **mọi** loại tensor đều được scale theo width (không còn là hằng số như SP).

Ba desiderata (Appendix J.2.1) mà muP cố gắng thoả, dùng để tự suy ra bảng trên: (1) mọi vector (pre-)activation phải có cỡ tọa độ Θ(1) (không nổ/không biến mất, quan trọng nhất với low-precision training); (2) output của network phải là O(1); (3) mọi tham số phải được update ”nhiều nhất có thể” theo width mà không

gây divergence (tức "maximal feature learning", mọi tham số đều đóng góp ý nghĩa trong infinite-width limit, không có tensor nào bị "kẹt" ở giá trị khởi tạo).

3.4 Transformer cụ thể (Definition 4.1 và Appendix B.1)

Attention logit dùng $q^\top k/d$ (chia cho d, không phải $\sqrt{d}$ như Transformer gốc). Lý do (footnote 8 của paper): lúc khởi tạo q, k độc lập nên $q^\top k$ scale theo CLT như $\sqrt{d}$ (giải thích gốc của $1/\sqrt{d}$ chuẩn), nhưng **trong quá trình train**, q và k trở nên tương quan nên $q^\top k$ lại scale theo Law of Large Numbers như d, do đó cần $1/d$ để giữ attention logit ổn định suốt quá trình train, không chỉ lúc init.

Áp theo Table 3: word embedding + positional embedding + layernorm weight/bias đi theo cột "input weights & bias" (vì map từ một chiều hữu hạn, như one-hot vocab, tới d_{model}); 4 ma trận attention W^Q, W^K, W^V, W^O và 2 ma trận MLP đi theo cột "hidden weights" (vì cả `fan_in` và `fan_out` đều là d_{model}/d_{ffn}, scale theo width); ma trận unembedding (dự đoán token tiếp theo) đi theo cột "output weights".

3.5 Hai formulation tương đương (Table 8, Table 9) và vì sao cần chúng

Paper đưa thêm 2 bảng tương đương về mặt toán học với Table 3 (chứng minh bằng Lemma J.1: đổi $(A, B, C) \to (A\theta, B/\theta, C/\theta^2)$ cho SGD hoặc $(A\theta, B/\theta, C/\theta)$ cho Adam với θ bất kỳ thì hành vi network không đổi):

- **Table 8** (bản mà package `mup` thực sự cài đặt, theo Appendix H): gộp cách scale của **output weights** lại để cùng dạng với input weights (cả hai đều có init variance Θ(1), Adam LR Θ(1)), và đưa phần khác biệt còn lại vào một **multiplier** riêng (Θ(1/fan_in) cho output). Lợi ích thực tế: code đơn giản hơn, và quan trọng hơn, **cho phép tie trọng số input/output embedding** (một thiết kế rất phổ biến ở Transformer) vì giờ hai loại weight này có cùng quy tắc init/LR. Paper chứng minh cụ thể trường hợp output weights: Table 3 có $(A, B, C) = (1,\ 1/\text{fan_in},\ \eta/\text{fan_in})$; lấy $\theta = 1/\text{fan_in}$ thì ra Table 8: multiplier $A' = 1/\text{fan_in}$, init std $B' = 1$ (variance Θ(1)), SGD LR $C' = \eta \cdot \text{fan_in}$, Adam LR $C' = \eta$ (Θ(1), hằng số).
- **Table 9**: bản viết theo đúng văn phong gốc của TP4 [57], dùng cho người đã quen notation cũ.

Người ghi chú chỉ trình bày chi tiết trường hợp **output weights** ở trên vì đây là trường hợp duy nhất paper viết ra bằng công thức chữ rõ ràng (không lệ thuộc hình bảng); các ô còn lại của Table 8/9 (đặc biệt cột input weights) bị mất định dạng superscript khi convert PDF sang text và người ghi chú không tự tin tái dựng đủ chính xác, nên không đưa vào đây để tránh sai (xem mục 6).

3.6 Optimizer khác và các mẹo cài đặt quan trọng (Appendix B.3, D.1, D.2)

- **AdamW**: giống Adam hoàn toàn trong các bảng trên, và có lợi thêm là weight decay tự động scale đúng; tuy nhiên weight decay **không tương thích** với muP Adam thường (cần dùng AdamW nếu muốn weight decay).
- **Adagrad, RMSProp**: giống Adam.
- **RAdam**: scale riêng từng giai đoạn (giai đoạn đầu kiểu SGD-momentum, giai đoạn sau kiểu Adam) theo đúng Table 3/8/9 tương ứng.
- **Optimizer "Frobenius normalization"** (chuẩn hoá update theo tỉ lệ $\|w\|_F/\|g\|_F$): paper liệt kê đích danh LARS, **Adafactor**, **LAMB**, Layca, Fromage, Nero. Những optimizer này cần learning rate của hidden weights scale thêm $1/\sqrt{\text{fan_in}}$ để tương thích muP, vì phép chuẩn hoá Frobenius-norm ngầm áp một kiểu scale Θ(1/√n) (gần đúng, chưa chuẩn) chứ không phải đúng Θ(1/n) mà muP cần. Paper suy đoán đây có thể giải thích một phần vì sao T5 (dùng Adafactor) train được từ 220M đến 11B tham số với một bộ HP cố định, còn GPT-3 (dùng Adam) phải giảm learning rate khi tăng kích thước model.
- **Coordinate checking** (D.1): kỹ thuật debug, đo |activation| trung bình của từng (pre-)activation vector qua vài step train đầu tiên, quét qua nhiều width; nếu implement muP đúng, các đường phải gần như "chồng" lên nhau theo width (flat); nếu có đường nổ/tụt về 0 theo width thì implementation sai. Paper

so sánh với ”gradient checking” khi implement autograd tay: về toán rất đơn giản nhưng dễ sai khi code.

- **Zero-init output layer và zero-init query projection** (D.2): ở init, một network muP xấp xỉ một Gaussian Process với variance $\Theta(\sigma^2/\text{width}) \to 0$ khi width lớn; nhưng ở **proxy model nhỏ**, variance này KHÔNG nhỏ, khiến quỹ đạo train ban đầu của proxy khác hẳn target, làm lệch HP optimum. Đặt σ=0 cho các layer đi từ một chiều ”vô hạn” về một chiều ”hữu hạn” (output logits, và attention logit qua ma trận query) xoá bỏ sự khác biệt này ở MỌI width; thực nghiệm không thấy giảm hiệu năng.

4. Thí nghiệm và kết quả chính

MLP trên CIFAR-10 (Fig. 3): với SP, optimal learning rate dịch khoảng 1 bậc độ lớn khi width tăng từ 256 lên 8192 (dùng LR tối ưu của model nhỏ nhất cho model lớn nhất gây hiệu năng rất tệ, có thể divergence); với muP, optimal LR ổn định qua toàn dải width, và tại một LR cố định, model rộng hơn luôn tốt hơn hoặc bằng.

Transformer trên Wikitext-2 (Fig. 1, Fig. 4): 2-layer pre-layernorm muP Transformer, 4 attention head, train 10k step, trung bình 5 seed, quét 4 HP (learning rate, output weight multiplier α_{output}, init std, LR schedule) qua width và depth, cả 4 HP đều ổn định qua width; riêng init std không ổn định tốt qua depth (xem mục 5, 6).

IWSLT14 De-En (Table 4): model 1x (fairseq default) có 40M tham số; proxy 0.25x (width chia 4) có 4M tham số.

Cách tune	Compute	Số sample	BLEU (percentile 25/50/75/100)
fairseq default	-	-	35.40
Tuning trực tiếp trên 1x	1x	5	33.62 / 35.00 / 35.35 / 35.45
Naive transfer từ 0.25x (SP)	1x	64	training diverged (toàn bộ)
muTransfer từ 0.25x (muP)	1x	64	35.27 / 35.33 / 35.45 / 35.53

Cùng một ngân sách FLOPs, muTransfer vừa **vượt** fairseq default, vừa vượt rõ ”tuning trực tiếp trên 1x” (vì được search với 64 sample rẻ thay vì chỉ 5 sample đắt), trong khi naive transfer (dùng SP thay muP) **diverge hoàn toàn**. Theo Pareto frontier compute-so-với-BLEU (Fig. 6), muTransfer luôn dominate conventional tuning; nếu so theo CÙNG SỐ sample (không so theo compute), tune trực tiếp trên model 1x nhích hơn một chút khi ít sample, nhưng khoảng cách biến mất khi tăng số sample: paper giải thích proxy model là một ”ước lượng noisy” của target, noise này bị triệt tiêu khi lấy nhiều sample.

WMT14 En-De (Table 5): target 211M tham số (Transformer lớn), proxy 15M tham số (giảm d_{model} từ 1024 → 256, d_{ffn} từ 4096 → 256, n_{head} từ 16 → 4).

Cách tune	Compute	Số sample	BLEU (worst/median/best)
fairseq default	-	-	26.40
Tuning trực tiếp trên 1x	1x	3	training diverged / - / 25.69
Naive transfer từ 0.25x (SP)	1x	64	training diverged
muTransfer từ 0.25x (muP)	1x	64	25.94 / 26.34 / 26.42

Ở cùng ngân sách, tune trực tiếp trên model lớn (chỉ đủ tiền cho 3 sample) có nguy cơ diverge; muTransfer khớp sát baseline đã tune kỹ (26.40) mà không cần tune model lớn.

BERT pretraining (Table 6): BERT-prototype (proxy) có 10 layer, $d_{model} = d_{ffn} = 256$, 8 head, $d_{head} = 32$ → 13M tham số, dùng để tune **đồng thời** cho cả BERT-base (110M) và BERT-large (350M). Search 256

combo, mỗi combo train 10^5 step; tổng chi phí tune ≈ chi phí pretrain 1 BERT-large đủ 10^6 step (kiểm lại ở mục 10).

Model	Cách	Model speedup	Total speedup	Test loss	MNLI (m/mm)	QQP
BERT-base	Megatron default	1x	1x	1.995	84.2/84.2	90.6
BERT-base	Naive transfer (SP)	4x	40x	training diverged	-	-
BERT-base	muTransfer	4x	40x	1.970	84.3/84.8	90.8
BERT-large	Megatron default	1x	1x	1.731	86.3/86.2	90.9
BERT-large	Naive transfer (SP)	22x	220x	training diverged	-	-
BERT-large	muTransfer	22x	220x	1.683	87.0/86.5	91.4

muTransfer cải thiện rõ so với baseline Megatron đã tune (test loss thấp hơn, MNLI/QQP cao hơn), trong khi naive transfer (SP) diverge hoàn toàn ở cả hai kích thước.

GPT-3 6.7B (§7.4, Table 7): target = 32 residual block, $d_{model} = 4096$, relative attention. Proxy: co width xuống 256 → khoảng 40 triệu tham số, nhỏ hơn target 168 lần (tự tính: $6.7 \times 10^9 / 4 \times 10^7 \approx 167.5$, khớp số "168 lần" paper nêu). Random search trên proxy ở 2 độ dài horizon (4 tỷ và 16 tỷ token): 467 run tổng (350 run @4B, 286 không diverge; 117 run @16B, 80 không diverge). Hai HP ảnh hưởng nhiều nhất: learning rate (chọn 0.006) và init scale (chọn 2.5); embedding multiplier chọn 10 (các multiplier khác giữ default 1). Khi train model 6.7B thật với các HP này, gặp divergence do underflow FP16 ở backward pass → phải chuyển sang train FP32; baseline so sánh ("re-run") vẫn train FP16 như gốc, và vô tình dùng absolute attention thay vì relative attention (lỗi phát hiện SAU khi mọi thí nghiệm đã hoàn tất, paper tự nêu rõ, xem mục 6).

Task / metric	6.7B + muP	6.7B re-run	6.7B gốc [7]
Validation loss (cross-entropy)	1.98	2.03	-
PTB (perplexity)	11.4	13.0	-
WikiText-103 (perplexity)	8.56	9.13	-
LAMBADA zero-shot (acc)	73.5	70.8	70.3
HellaSwag zero-shot (acc)	72.0	66.7	67.4

Chi phí tune chỉ xấp xỉ 7% tổng chi phí pretrain (công thức Appendix F.4: $s(t_1 N_1 + t_2 N_2)/(ST)$ với $s = 40$ triệu tham số proxy, $S = 6.7$ tỷ tham số target, $t_1 = 4$ tỷ token/$N_1 = 350$ trial, $t_2 = 16$ tỷ token/$N_2 = 117$ trial, $T = 300$ tỷ token huấn luyện target). Bảng 11 so model 6.7B+muP với model 13B gốc (gấp đôi tham số, không qua muTransfer): hiệu năng "tương đương" trên phần lớn task (HellaSwag zero-shot 72.0 so với 70.9), nhưng KHÔNG thắng tuyệt đối mọi nơi: ví dụ LAMBADA few-shot 6.7B+muP chỉ đạt 74.7 so với 81.3 của model 13B (xem thêm mục 6).

"Wider is better" suốt quá trình train (Section 8, Fig. 7-8): dưới muP, với MỘT bộ HP cố định, model rộng hơn đạt training loss tốt hơn hoặc bằng ở MỌI thời điểm trong quá trình train (không chỉ lúc cuối), ngược với SP (model rộng có thể tệ hơn tuỳ LR). Stress test trên một Transformer kiểu GPT-3, 4 layer, width từ 256 đến 32.768, cùng một bộ HP, train tới 1,4 tỷ token: rộng hơn luôn thắng hoặc bằng (trừ một đoạn ngắn quanh 10^8 token mà tác giả cho là nhiễu do chỉ chạy 1 seed).

ResNet (Appendix G.1, đọc tóm lược): trên CIFAR-10 (biến thể Davidnet), tune LR và α_{output} trên model 0.5x rồi transfer sang model 8x: muP đạt 95.29% so với SP 94.86% (chênh 0.43 điểm %, Table 12). Trên Wide-ResNet/ImageNet, tune 4 HP (LR, α_{output}, weight decay, momentum) trên model 0.125x rồi transfer sang model 1x: muP đạt 77.16% top-1 so với SP 76.75% (chênh 0.41 điểm %, Table 13).

5. Phân tích / ablation

- **Điều kiện tối thiểu để transfer tốt** (§6.1): trong thí nghiệm của paper, HP transfer tốt khi width, depth, batch size, sequence length, số step train đạt một mức sàn tối thiểu (paper nêu ví dụ width≥256, depth≥4, batch≥32, seqlen≥128, step≥5000) và kích thước đích nằm trong ”khoảng hợp lý” so với lúc tune, không phải transfer được ở bất kỳ khoảng cách scale nào.
- **Depth transfer chỉ hoạt động cho pre-layernorm Transformer**, không hoạt động tốt cho post-layernorm (§6.1); đây là một giới hạn quan trọng (nhắc lại ở mục 6).
- **Init std không transfer tốt qua depth** dù qua width thì ổn định hơn (Fig. 4, cột 3); thực hành thấy GIỮ init std cố định (không tune lại theo depth) khi tune các HP khác vẫn cho kết quả tốt (cách làm này được dùng thật ở thí nghiệm BERT, §7.3).
- **Activation function** (D.3, Fig. 9): ReLU tốt cho transfer; squashing activation (tanh, softmax) làm network hẹp bão hoà nhiều hơn network rộng, gây lệch gradient hệ thống, giảm chất lượng transfer; paper khuyến nghị thay activation ”squashing không thiết yếu” bằng ReLU khi có thể.
- **Enlarge** d_k (D.4, Fig. 10): nếu d_k (key/query dimension mỗi head) quá nhỏ trong proxy model (co cả d_{model} và d_{head} cùng lúc), HP landscape của multiplier attention α_{attn} rất ”nhiễu”, làm random search trên proxy kém tin cậy. Giải pháp: tách d_k khỏi d_{model}, giữ $d_k \gtrsim 32$ dù d_{model} co lại, làm mượt hẳn landscape mà tốc độ train/suy luận hầu như không đổi (nhờ tối ưu CUDA cho attention).
- **Gaussian tốt hơn non-Gaussian init** (D.5): init kiểu uniform đôi khi khiến model rộng tệ hơn model hẹp; điều này phù hợp với lý thuyết universality (non-Gaussian hội tụ CHẬM hơn về limit Gaussian khi width tăng).
- **Sequence length dài hơn giúp init std transfer tốt hơn qua width** (D.6); tác giả thẳng thắn ”không rõ vì sao”, để ngỏ cho nghiên cứu sau.
- **Thay đổi tỉ lệ width giữa các layer vẫn transfer tốt** (Appendix E.2, Fig. 12-13): LR landscape ổn định dù d_{ffn}/d_{model} thay đổi tới 32 lần (Fig. 12, IWSLT14), hoặc khi cố định d_{head} và chỉ tăng n_{head} (đúng cách GPT-3 thật scale, Fig. 13); điều này tăng tính linh hoạt khi thiết kế proxy model.
- **Reverse-muTransfer để debug** (Appendix I): lấy một tổ hợp HP gây mất ổn định ở model LỚN, chuyển ngược về model NHỎ bằng đúng công thức muP để tái tạo lại CHÍNH XÁC kiểu mất ổn định đó ở quy mô rẻ hơn rồi mới debug (2 case study thật: một Transformer width 8192 trên Wikitext-2 diverge do attention logit nổ, tái tạo ở width 256 để phân tích; một model 6 tỷ tham số thật trong một project khác, tái tạo lại ở model 100M tham số rồi fix bằng random search ở quy mô nhỏ).

6. Giới hạn

Tác giả tự nêu (Section 11 ”Venues of Improvement” và rải rác trong bài):

- Init std không transfer tốt qua depth; transfer qua depth nói chung còn thất bại với post-layernorm Transformer; tác giả đặt câu hỏi mở liệu có cần một parametrization theo chiều SÂU nguyên tắc hơn không.
- HP optimum vẫn dịch nhẹ theo scale (không hoàn toàn bất biến, Fig. 4), có thể do chưa có ”hiệu chỉnh width hữu hạn” (finite-width correction) cho muP, một hướng mở.
- **Section 9 ”theoretical puzzle”**: tác giả thừa nhận CHƯA có lý thuyết giải thích tại sao tồn tại một width N (của proxy) vừa đủ lớn để HP optimum hội tụ, nhưng vừa đủ nhỏ để hàm/loss của network CHƯA hội tụ (nếu cả hai hội tụ cùng tốc độ, muTransfer vô nghĩa vì train proxy coi như train luôn target). Đây là giới hạn lý thuyết thật, không phải lỗi, nhưng paper để ngỏ hoàn toàn cho tương lai.
- Chưa có cách transfer HP **regularization** (dropout, weight decay) theo cả model size và data size, quan trọng nhất khi finetune trên dataset nhỏ; paper nói thẳng ngay trong Introduction: ”in settings such as finetuning of models on small datasets, µTransfer may not be sufficient” (xem thêm mục 8).
- **Confound ở thí nghiệm GPT-3** (tác giả tự nêu rõ, §7.4 và footnote 16-17): (a) baseline ”re-run” vô tình dùng absolute attention thay vì relative attention như lẽ ra phải dùng (lỗi code, phát hiện SAU khi toàn bộ thí nghiệm đã chạy xong); (b) model muTransfer phải train FP32 (do FP16 underflow gây

diverge ở backward) còn baseline vẫn train FP16 gốc. Hai khác biệt này (loại attention + độ chính xác số) là confound thật: phần ”GPT-3+muP thắng GPT-3 gốc” ở Table 7 **không phải** một so sánh sạch chỉ-do-muTransfer; tác giả cũng nói họ không đủ tài nguyên để rerun baseline ở FP32 cho công bằng hơn.

- Chỉ verify thực nghiệm sâu trên Transformer và ResNet (dù khẳng định lý thuyết Tensor Programs áp dụng ”mọi kiến trúc”); domain thử gồm dịch máy, pretraining LM, phân loại ảnh; không có RL, VLM, audio, v.v.

Người ghi chú nhận xét:

- Table 3/8/9 trong file PDF bị mất ký hiệu superscript/subscript khi convert qua `pdftotext -layout` (ví dụ ”1/fan_in2” nghĩa là 1/fan_in², nhưng số 2 bị in dính liền không có dấu mũ, và các cột bị lệch hàng do khoảng trắng không đều). Người ghi chú đã **không** dùng trực tiếp bảng bị lỗi này; thay vào đó dựng lại Table 3 hoàn toàn từ phần suy diễn bằng chữ rõ ràng trong Appendix J.2.1 (mục 3.3 ở trên), và chỉ trích dẫn đúng MỘT ví dụ số cụ thể cho Table 8 (trường hợp output weights) mà chính paper viết ra bằng công thức tường minh (Lemma J.1). Với các ô còn lại của Table 8/9 (đặc biệt cột input weights của Table 8, nơi văn bản suy diễn J.2.1 và cấu trúc OCR của bảng có một điểm mâu thuẫn nhỏ về bậc scale của Adam LR mà người ghi chú không giải quyết được một cách chắc chắn trong thời gian đọc), ghi chú này KHÔNG khẳng định giá trị cụ thể để tránh lan truyền số sai.
- Tỉ lệ chi phí tune/pretrain của GPT-3: paper ghi ”≈0.07” (7%); tự tính lại đúng công thức ở Appendix F.4 với các số liệu paper cho ($s = 4\times10^7$, $S = 6,7\times10^9$, $t_1 = 4\times10^9$, $N_1 = 350$, $t_2 = 1,6\times10^{10}$, $N_2 = 117$, $T = 3\times10^{11}$) ra $(s(t_1N_1 + t_2N_2))/(ST) \approx 0,065$ (≈6,5%), hơi thấp hơn số ”7%” paper nêu. Chênh lệch nhỏ, nhiều khả năng do làm tròn ở dấu ”≈” của paper, không phải mâu thuẫn nghiêm trọng, nhưng nên hiểu ”7%” là số đã làm tròn lên.
- Bảng so sánh GPT-3 6.7B+muP với model 13B gốc (Table 11) **không phải** muP thắng tuyệt đối từng task: LAMBADA few-shot 6.7B+muP đạt 74.7 so với 81.3 của 13B gốc (thua khá xa); nên đọc ”hiệu năng tương đương” theo nghĩa trung bình trên nhiều task, không phải thắng toàn bộ.
- Không phát hiện mâu thuẫn số liệu nào khác giữa văn bản chính và các bảng (Table 4-7, 10-13) sau khi kiểm bằng `grep -a`; đây là một paper được viết cẩn thận về mặt số liệu thực nghiệm, vấn đề duy nhất là các bảng công thức toán bị hỏng định dạng khi OCR.

7. Liên hệ với các bài khác

- **opt 06 (Adam: A Method for Stochastic Optimization)**: muP cho Adam là một trong hai contribution chính của bài 35 so với bài TP4 trước đó (TP4 chỉ có SGD). Cần hiểu cơ chế moving-average + chuẩn hoá theo $\sqrt{v}$ của Adam (opt 06) để hiểu vì sao cột ”Adam LR” trong Table 3 lại khác cột ”SGD LR” ở hidden weights ($\Theta(1/\text{fan_in})$ so với $\Theta(1)$).
- **opt 10 (Large Batch Optimization for Deep Learning: Training BERT in 76 minutes, chính là LAMB)**: đây là reference [64] được chính bài 35 trích dẫn trong Appendix B.3; LAMB nằm trong nhóm optimizer ”Frobenius normalization” mà bài 35 chỉ ra cần thêm một hệ số $1/\sqrt{\text{fan_in}}$ để tương thích muP. Hai bài cùng nhắm tới mục tiêu ”pretrain BERT rẻ/nhanh hơn” nhưng theo hai trục khác nhau: LAMB tăng batch size + LR thích ứng theo từng layer, bài 35 giữ batch/model size cố định rồi tune HP một lần trên proxy nhỏ.
- **opt 36 (Scaling Exponents Across Parameterizations and Optimizers)**: bài đọc tiếp theo trong lộ trình, kiểm định lại và mở rộng phạm vi áp dụng của muP, so sánh với các parametrization/optimizer khác (Adam, Adafactor, Lion...) ở quy mô lớn hơn, một số khẳng định của bài 35 (HP optimum ổn định tuyệt đối theo width) được bài 36 làm rõ thêm điều kiện áp dụng.
- **opt 45 (Deconstructing What Makes a Good Optimizer for Language Models)**: cùng quan tâm ”cái gì thực sự quyết định HP tốt cho LLM pretraining”, nhưng khảo sát theo TRỤC optimizer (so Adam/Adafactor/Lion...) thay vì trục width như bài 35; đọc cùng nhau để thấy optimizer choice và parametrization là hai trục tương đối độc lập, cả hai đều ảnh hưởng HP optimum.

- **opt 11 (Symbolic Discovery of Optimization Algorithms, tức Lion)**: Lion được tìm bằng search tự động trên model/task nhỏ rồi áp dụng cho model lớn: đây là một kiểu ”transfer” khác (transfer CÔNG THỨC optimizer, không phải transfer GIÁ TRỊ HP như bài 35); so sánh hai cách ”mượn kết quả từ quy mô nhỏ” này khá thú vị.
- **llm 02 (Language Models are Few-Shot Learners, GPT-3)** (đã kiểm tra, file tồn tại trong `llm content/`): chính là model target ở §7.4: bài 35 áp dụng muTransfer để tune lại GPT-3 6.7B và so sánh trực tiếp với số liệu công bố trong bài GPT-3 gốc (LAMBADA, HellaSwag, PTB...). Nên đọc llm 02 trước để biết rõ các benchmark và baseline đang được so lại trong Table 7/10/11 của bài 35.

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU ≤ 8 GB, batch 1), bằng pruning + distillation + RL trên một **checkpoint đã pretrain sẵn**, không train from-scratch nhiều kích thước. Đây là điểm mấu chốt khiến bài 35 **ít áp dụng trực tiếp** được, vì hai lý do cốt lõi (suy luận, dựa trên đúng những gì paper phát biểu):

1. **muTransfer cần một trục ”width” thật, với nhiều kích thước cùng họ, train từ đầu theo muP.** Cốt lõi kỹ thuật là: tune trên proxy NHỎ rồi copy HP sang target LỚN HƠN, cả hai cùng parametrize theo muP từ lúc khởi tạo. Nếu người dùng chỉ có đúng MỘT kích thước cố định (0.8B, đã pretrain), không có ”model lớn hơn” nào để transfer HP TỚI; không có đối tượng đích cho bước 3 của Algorithm 1 (suy luận).
2. **Checkpoint đã pretrain sẵn không rõ có dùng muP hay không (rất có thể là SP/HF default).** muTransfer chỉ ”miễn phí” khi quy tắc scale init/LR được cài đúng từ ĐẦU; retrofit muP lên một checkpoint SP đã train xong không có ý nghĩa: các trọng số đã ”định hình” theo động lực SP, không thể coi như vừa mới khởi tạo lại theo muP (suy luận).
3. Chính paper cũng tự nói: *”in settings such as finetuning of models on small datasets, µTransfer may not be sufficient”*, tức tác giả thừa nhận bài toán fine-tune (gần giống việc hậu huấn luyện LightOnOCR của người dùng) không nằm trong phạm vi được verify của kỹ thuật này. Thêm vào đó, theo Table 1, chính dropout/weight decay/các HP mang tính regularize (những HP quan trọng khi fine-tune trên ít data, bao gồm rank/dropout của LoRA) lại là nhóm **KHÔNG** muTransfer tốt theo paper.

Những gì vẫn có thể mượn được (suy luận, mức thấp hơn so với áp dụng trực tiếp):

- Nếu người dùng từng cần huấn luyện một **proxy nhỏ from-scratch cùng kiến trúc hybrid** (ví dụ để làm teacher nhỏ hơn cho distillation, hoặc test một thay đổi kiến trúc trước khi áp lên 0.8B), Table 3 ở mục 3.3 là một checklist tức thì cho cách scale init/LR theo width mà không cần tune lại bằng tay mỗi lần đổi kích thước, giúp tiết kiệm đúng kiểu chi phí mà paper nhắm tới.
- **Coordinate checking** (D.1, mục 3.6) là một kỹ thuật debug tổng quát, không bắt buộc phải theo đúng muP: đo |activation| trung bình theo layer qua vài bước đầu khi thử một learning rate mới cho LoRA/full fine-tune, nếu một layer ”nổ” hoặc ”chết” ngay từ đầu, đó là dấu hiệu LR/init cho layer đó đang sai bậc độ lớn, bất kể có dùng muP hay không.
- **Bài học định tính về LR khác nhau cho các loại layer khác nhau** (input/embedding vs. hidden vs. output/LM-head, mục 3.3-3.4) vẫn hữu ích khi tinh chỉnh fine-tune: đừng dùng một LR toàn cục cho cả embedding/LM-head và các block Transformer/GatedDeltaNet; đây là trực giác chung rút ra từ bài 35, không phải một công thức cụ thể để copy (suy luận).
- Trong số các trục ”muTransferred Across” đã verify thực nghiệm (width, depth, batch size, sequence length, training time), **sequence length** gần với tình huống thực tế của người dùng nhất (trang OCR dài ~3.600 token): NẾU người dùng có một kiến trúc muP-compatible thật (ví dụ khi huấn luyện lại một model nhỏ từ đầu), có thể tune LR ở context ngắn rẻ rồi transfer sang context dài theo đúng kết quả đã verify ở Appendix G.2.1, nhưng đây vẫn là suy luận có điều kiện (chỉ đúng nếu model được parametrize đúng theo muP từ đầu, không áp dụng cho checkpoint SP có sẵn).

Kết luận cho mục này: bài 35 **ít liên quan trực tiếp** tới luồng công việc chính (pruning/distillation/RL trên checkpoint 0.8B có sẵn), vì thiếu cả hai điều kiện cần (nhiều width cùng họ train from-scratch theo muP, và một target lớn hơn để transfer HP tới). Giá trị thực tế lớn nhất nằm ở (a) checklist Table 3 nếu có ngày cần train một proxy/student nhỏ từ đầu, và (b) kỹ thuật coordinate-checking để debug LR/init nói chung.

9. Thí nghiệm gợi ý

1. **Tái tạo Fig. 3 ở quy mô toy** (đo trực tiếp ”HP transfer có hoạt động không”): huấn luyện MLP 2 lớp ẩn trên một tập nhỏ (ví dụ MNIST hoặc CIFAR-10 subset) ở 4-5 width (64, 128, 256, 512, 1024) dưới cả SP (init PyTorch default) và muP (cài theo công thức (3)-(4) ở mục 3.2), quét learning rate theo lưới log2. Đo: optimal LR tại mỗi width. Kỳ vọng: optimal LR của SP dịch dần theo width (độ dốc gần -1 trên log-log scale), còn optimal LR của muP gần như không đổi (đường thẳng ngang). Đây là thước đo trực tiếp nhất xem cài muP có đúng không.
2. **Coordinate checking** (tái tạo ý tưởng Fig. 5 ở mục 3.6): với đúng model/HP ở thí nghiệm 1, ghi lại mean |activation| của mỗi layer sau 1-4 step Adam đầu tiên, ở mỗi width. Đo: mean |activation| theo width. Kỳ vọng: SP cho đường tăng rõ theo width (blow up), muP cho đường gần như flat. Nếu muP mà vẫn ”nổ” theo width thì code cài sai công thức, cần soát lại Table 3.
3. **Zero-shot transfer thật sự (phiên bản mini của §7.1)**: với một Transformer nhỏ (2-4 layer, char-level hoặc token hoá đơn giản) cài đúng muP, tune LR bằng grid nhỏ (5-10 giá trị) ở width=64, lấy LR tốt nhất, rồi COPY THẲNG (không tune lại) sang width=512 hoặc 1024. So sánh loss cuối với (a) một model cùng width lớn được tune riêng bằng cùng lưới LR (nếu đủ compute) và (b) model lớn dùng LR mặc định không qua tune. Đo: loss cuối của 3 phương án. Kỳ vọng: (muTransfer) gần (model lớn tune riêng), và rõ ràng tốt hơn (b).
4. **Transfer qua sequence length** (liên quan trực tiếp tới mục 8, vì trang OCR của người dùng dài ~3.600 token): với model muP ở thí nghiệm 3, tune LR ở context ngắn (64 token) rồi transfer sang context dài hơn (256, 512 token) trên cùng kiến trúc. Đo: loss ở context dài khi dùng LR transfer so với LR tune riêng tại context dài (nếu đủ compute để tune riêng). Kỳ vọng (theo Appendix G.2.1): loss gần tương đương, chứng tỏ nguyên tắc ”tune ngắn, transfer dài” khả thi cho một proxy thật sự muP.
5. **Ablation enlarge** d_k (tái tạo ý tưởng Fig. 10 ở quy mô nhỏ, đo ”độ nhiễu” của HP landscape): với Transformer nhỏ muP, cố định d_{model} nhỏ (ví dụ 64) nhưng thử 2 cấu hình: (a) $d_{head} = d_{model}/n_{head}$ như thường (rất nhỏ nếu n_{head} lớn), (b) d_{head} cố định ≥32 bất kể n_{head}. Quét multiplier α_{attn} ở mỗi cấu hình, lặp lại với 3-5 seed. Đo: phương sai của optimal α_{attn} giữa các seed. Kỳ vọng: cấu hình (b) có phương sai thấp hơn rõ rệt, xác nhận claim ”enlarge d_k giảm nhiễu HP landscape”.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
abc-parametrization	quy tắc scale (multiplier A, init std B, learning rate C) theo width cho mỗi tensor; SP và muP là hai lựa chọn khác nhau của quy tắc này
Zero-shot HP transfer	một parametrization ”cho phép” transfer nếu HP optimum (theo một metric, ở đây là training loss) hội tụ khi width $\to \infty$
Algorithm 1 (muTransfer)	(1) parametrize target theo muP; (2) tune một bản nhỏ của target; (3) copy nguyên HP đã tune sang target, không tune lại
Table 1 (phân loại HP)	muTransferable: optim/init/multiplier. Không transfer tốt: dropout, weight decay. Trục để transfer qua: width, depth*, batch size*, seq length*, training time*
Table 3 (quy tắc muP, mục 3.3)	Input & bias: Init Θ(1/fan_in)=Θ(1), LR (SGD & Adam) Θ(fan_out). Output: Init Θ(1/fan_in²), LR Θ(1/fan_in). Hidden: Init Θ(1/fan_in), SGD LR Θ(1), Adam LR Θ(1/fan_in)

Khái niệm	Nội dung
Desiderata J.1	(1) mọi (pre-)activation cỡ Θ(1); (2) output network O(1); (3) mọi tham số được update tối đa theo width mà không gây divergence
Attention logit muP	$q^\top k/d$ (chia d, không phải $\sqrt{d}$ như Transformer gốc), vì q, k tương quan dần trong lúc train (Law of Large Numbers chứ không phải CLT)
Zero-init output/query layer (D.2)	đặt σ=0 cho layer ”vô hạn → hữu hạn” để xoá lệch Gaussian-Process ban đầu giữa proxy nhỏ và target lớn
Coordinate checking (D.1)	đo mean |activation| mỗi layer qua vài step đầu, quét width; implement đúng muP thì các đường phải flat
Tỉ lệ chi phí tune GPT-3 (Appendix F.4)	$\frac{s(t_1N_1+t_2N_2)}{ST}$; với $s=4\times10^7,\ S=6{,}7\times10^9,\ t_1=4\times10^9,\ N_1=350,\ t_2=1{,}6\times10^{10},\ N_2=117,\ T=3\times10^{11}$ → tự tính ≈0,065 (paper ghi ≈0,07)
Kiểm tra chi phí tune BERT (tự tính)	256 combo × 10^5 step trên proxy 13M tham số ≈ $2{,}56\times10^7$ ”step-tương-đương-proxy”; proxy rẻ hơn BERT-large (350M) theo tỉ lệ tham số ≈ 350/13 ≈ 26,9 lần mỗi step; tổng ≈ $2{,}56\times10^7/26{,}9\approx9{,}5\times10^5$ ”step-tương-đương-BERT-large”, gần đúng bằng 10^6 step pretrain thật của BERT-large, khớp với mô tả ”tổng chi phí tune ≈ chi phí pretrain 1 BERT-large” của paper
Wider-is-better (Section 8)	dưới muP, với một bộ HP cố định, model rộng hơn có training loss tốt hơn hoặc bằng ở MỌI thời điểm trong quá trình train

11. Kết luận ngắn

Maximal Update Parametrization (muP) là một quy tắc scale init/learning rate/multiplier theo width khiến HP optimum (đặc biệt learning rate) ổn định khi width tăng, trái với Standard Parametrization (SP) mặc định của các framework hiện nay, nơi optimal LR dịch cả bậc độ lớn theo width. Từ tính chất này, muTransfer cho phép tune HP trên một proxy model nhỏ rồi copy thẳng (zero-shot) sang model đích lớn hơn, được verify trên MT (IWSLT14, WMT14), pretraining LM (BERT, GPT-3 6.7B) và image classification (ResNet/CIFAR-10, ImageNet), ở quy mô GPT-3, chi phí tune chỉ còn khoảng 7% chi phí pretrain. Hạn chế lớn nhất: HP regularize (dropout, weight decay) không transfer tốt, depth-transfer còn mong manh với post-layernorm Transformer, và bản thân tác giả thừa nhận chưa có lý thuyết giải thích tại sao lại tồn tại một ”cửa sổ” width hữu dụng để transfer. Với một project chỉ hậu huấn luyện/nén một checkpoint 0.8B có sẵn (không train nhiều width from-scratch), kỹ thuật muTransfer gốc ít áp dụng trực tiếp được; giá trị còn lại chủ yếu nằm ở checklist scale init/LR (Table 3) và kỹ thuật debug coordinate-checking.

Nguồn gốc: https://arxiv.org/abs/2203.03466

Bài 36

Scaling Exponents Across Parameterizations and Optimizers (2024)

Tác giả: Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, Jeffrey Pennington (Google DeepMind; Katie Everett đồng thời ở MIT, Leslie Pack Kaelbling ở MIT; Wortsman, Novak và Sohl-Dickstein thực hiện nghiên cứu này khi còn ở Google DeepMind).
Venue / arXiv: arXiv:2407.05872 (v2, 16/07/2024); Proceedings of the 41st International Conference on Machine Learning (ICML 2024), Vienna, Austria, PMLR vol. 235 (ghi rõ ở footer trang 1 của paper).
Nguồn: https://arxiv.org/abs/2407.05872
Chủ đề: mở rộng lý thuyết ”parameterization” kiểu Tensor Programs / muP sang một không gian tổng quát hơn về giả định ”alignment” giữa tham số và dữ liệu; đo thực nghiệm alignment và tìm exponent learning rate theo width tốt hơn trên hàng chục nghìn model (3 optimizer x 4 parameterization, tới 26.8B tham số); phát hiện epsilon trong Adam gây ”underflow” ở model lớn và đề xuất Adam-atan2.
Vị trí trong lộ trình: đọc sau opt 06 (Adam, optimizer nền được phân tích) và opt 35 (Tensor Programs V / muP, khung parameterization và giả định alignment mà bài này mổ xẻ lại). Đọc cùng hoặc sau opt 45 (Deconstructing What Makes a Good Optimizer for Language Models, cùng năm 2024, góc nhìn thực nghiệm khác về optimizer cho LM) và opt 16 (Muon is Scalable for LLM Training, optimizer mới không nằm trong ba họ SGD/Adam/Adafactor mà bài này phân tích).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Đây là paper dài, nặng lý thuyết (63 trang kể cả appendix); phạm vi đọc cụ thể được ghi rõ ở mục 3.

1. Bối cảnh

Parameterization là gì? Khi huấn luyện một mạng neural, ta phải chọn: phương sai khởi tạo của từng lớp (layer), một ”parameter multiplier” (hằng số nhân vào output của phép nhân ma trận ở forward pass, không được cập nhật nhưng ảnh hưởng tới gradient truyền ngược), và learning rate (LR) của từng lớp. Một **parameterization** là một công thức quy định ba đại lượng này sẽ co giãn (scale) như thế nào theo **width** n (độ rộng, ví dụ số chiều ẩn d_{model} của Transformer) khi ta tăng model lên lớn hơn, cho từng loại lớp: embedding, hidden (các lớp ẩn giữa), readout (lớp cuối ra logits).

Vì sao cần quan tâm? Vì nếu công thức co giãn sai, khi scale model lên rất lớn, có thể xảy ra ”scaling mismatch” không thấy được ở quy mô nhỏ (ví dụ, dưới 1B tham số) nhưng hỏng ở quy mô lớn. Paper mở đầu bằng một ví dụ cụ thể: với standard parameterization (SP, cách khởi tạo và scale ”thông thường” kiểu Glorot/He) huấn luyện bằng SGD, LR tối ưu của lớp hidden lý thuyết phải co theo $O(1/\sqrt{n})$, còn LR của lớp readout phải co theo $O(1/n)$. Nếu dùng **một LR toàn cục** (global learning rate) cho mọi lớp như thực hành phổ biến hiện nay, ta buộc phải chọn một trong hai lựa chọn tệ: (a) LR co theo $O(1/\sqrt{n})$ thì lớp hidden học tốt nhưng lớp readout sẽ ”nổ” (explode) khi n tăng, hoặc (b) LR co theo $O(1/n)$ thì lớp readout ổn định nhưng update ở các lớp sớm hơn ”biến mất” (vanish) về 0 khi $n \to \infty$. Sai lệch này có thể âm thầm tồn tại tới khi model vượt một ngưỡng kích thước nào đó, rất rủi ro trong thời đại ”train once” (huấn luyện model siêu lớn chỉ một lần, không có cơ hội thử lại rẻ).

Tensor Programs / muP (bài opt 35, Yang & Hu 2021; Yang & Littwin 2023 cho Adam) đã giải bài toán này bằng cách suy ra các ràng buộc ”ổn định” (stability: activation có scale hằng số, logit không vượt scale hằng số) và ”không tầm thường” (nontriviality: logit phải thay đổi ít nhất ở scale hằng số sau khi học, nếu không model coi như không học được gì mới khi $n \to \infty$). Từ đó họ suy ra công thức LR theo từng lớp cho

muP, cho phép **hyperparameter transfer**: dò LR tối ưu trên model nhỏ rẻ rồi dùng lại (gần như) nguyên cho model lớn đắt.

Tuy nhiên, các ràng buộc này dựa trên một **giả định ngầm quan trọng gọi là "alignment"**: mức độ cập nhật tham số (`update`) tương quan (đồng hướng) với phân phối dữ liệu đến đâu. Yang & Hu (2021) và Yang & Littwin (2023) chọn một giá trị cố định cho mức alignment này (coi như luôn "alignment tối đa") ở mọi lớp trừ layer readout. Paper này đặt câu hỏi: giả định đó có đúng trong thực tế không, và nếu thay bằng giả định khác (hoặc đo trực tiếp) thì công thức LR tối ưu sẽ khác ra sao? Đây chính là câu hỏi trung tâm của bài.

2. Ý tưởng chính

Paper có bốn đóng góp chính, gắn chặt với nhau:

1. **Tổng quát hoá lý thuyết parameterization.** Thay vì cố định mức alignment như công trình trước, tác giả định nghĩa ba biến alignment liên tục α_l, ω_l, u_l cho mỗi lớp l (đo mức tương quan giữa ba thành phần khác nhau trong công thức khai triển activation sau update), rồi suy ra hệ ràng buộc ổn định/không-tầm-thường như một **hàm của các biến này** (Bảng 2 của paper) áp dụng cho cả ba họ optimizer SGD, Adam, Adafactor. Lý thuyết của Yang & Hu / Yang & Littwin chỉ là một điểm đặc biệt (khi $\alpha_l = 1,\ \omega_l = 1/2$ trừ $\omega_{L+1} = 1$; riêng u_l không bị ràng buộc gì thêm, xem mục 3.3) trong không gian tổng quát này.
2. **Đo alignment thực nghiệm.** Đề xuất một metric gọi là "log alignment ratio" và đo nó trong hàng chục nghìn lần chạy. Kết quả: alignment không cố định ở hai cực trị lý thuyết mà **nằm ở mức trung gian, biến đổi mạnh theo bước huấn luyện, theo loại lớp, theo parameterization**, và ít phụ thuộc vào optimizer. Hệ quả: giả định "alignment tối đa mọi lớp" của công trình trước có thể quá bảo thủ (quá "an toàn"/conservative), khiến nó loại bỏ nhiều parameterization-exponent thú vị mà lẽ ra vẫn ổn định.
3. **Mọi parameterization đều transfer được, không chỉ muP.** Với công thức LR theo từng lớp đúng (tức dùng đúng exponent lý thuyết cho layer đó, dưới một trong hai giả định "full alignment" hoặc "no alignment") cộng thêm việc tinh chỉnh một **hằng số nhân hằng (constant multiplier)** cho mỗi loại lớp, cả 4 parameterization (Standard/SP, NTK, muP, Mean-Field/MFP) đều đạt hyperparameter transfer qua width, không phải chỉ muP như công trình trước (muTransfer, Yang et al. 2022) nhấn mạnh.
4. **Phát hiện và sửa epsilon-underflow trong Adam.** Vì gradient co nhỏ dần theo width, hằng số epsilon trong mẫu số Adam (vốn chỉ để tránh chia 0) có thể trở thành số hạng **chi phối** mẫu số ở model đủ lớn, làm Adam mất tính scale-invariant. Tác giả đề xuất **Adam-atan2**: thay phép chia bằng hàm `atan2`, loại bỏ hoàn toàn epsilon, chỉ cần đổi một dòng code.

Kết quả nổi bật nhất (được nhấn mạnh ở abstract và mục 4.2.1): với Adam, công thức LR theo lớp tốt nhất tìm được không phải của muP mà của **Standard parameterization**: LR lớp embedding co theo $O(1)$ (không đổi theo n), LR lớp hidden và readout co theo $O(1/n)$. Cấu hình này đánh bại tất cả các tổ hợp parameterization + công thức LR khác cho Adam, kể cả vượt muP ở quy mô lớn hơn nhiều (15.3B Standard-param thắng 26.8B muP, xem mục 4).

3. Phương pháp chi tiết

Phạm vi đã đọc: toàn bộ phần thân paper (Abstract, Mục 1 Introduction, Mục 2 Background, Mục 3 Theoretical Contributions, Mục 4 Experiments gồm 4.1-4.3, Mục 5 Related Work, Mục 6 Limitations, Mục 7 Conclusions, trang 1-22). Ở Appendix: đọc đầy đủ Appendix A (author contributions), C (Experimental Details, gồm C.1-C.5 và các Bảng C1, C2, C3, công thức Adam-atan2), E (Additional Per-Layer LR Results, gồm Bảng E1), G (Weight Decay Experiments), H (Adafactor vs Adam+Parameter-Scaling), I (Fixed Step vs Compute Optimal, gồm Bảng I1). Đọc cấu trúc và các công thức tổng hợp của Appendix B (Theoretical Details: B.15 Summary of Constraints = Bảng 2 lặp lại, B.16 Tensor Programs as Special Case, B.17 Maximum Stable Learning Rates = Bảng 1 lặp lại) nhưng không triển khai từng bước chứng minh chi tiết B.1-B.14 (các bước đại số thuần tuý). Appendix D (grep xác nhận) chỉ chứa hình minh hoạ, không có bảng số hay đoạn văn mới. Appendix F có hai đoạn văn bản thực chất (độ nhạy epsilon khác nhau giữa các parameterization, dùng

ở mục 4/5) đã đọc đầy đủ cùng các hình minh hoạ của nó. Appendix J, K, L (mỗi phần hàng trăm dòng) là các biểu đồ learning-rate-sweep chi tiết cho từng tổ hợp optimizer x parameterization, chỉ dùng để vẽ lại Hình 3-5 và Bảng 3/E1 đã có trong phần thân, không đọc tuần tự toàn bộ ba phụ lục này.

3.1 Ký hiệu và mô hình

Theo ký hiệu của Yang & Hu (2021): xét một MLP (lý thuyết; thực nghiệm dùng Transformer) có L lớp ẩn, width n, input/output dimension d. Với mỗi lớp l, ba số (a_l, b_l, c_l) định nghĩa:

- parameter multiplier $= n^{-a_l}$,
- phương sai khởi tạo $W_l \sim \mathcal{N}(0, n^{-2b_l})$,
- learning rate $\eta_l \propto n^{-c_l}$ (hằng số tỉ lệ không phụ thuộc n bị bỏ qua ở ký hiệu này).

$\Delta W_l^t, \Delta z_l^t$ là độ thay đổi của tham số/activation ở lớp l, bước t, so với lúc khởi tạo. Activation sau khi update khai triển thành 4 số hạng:

$$z_l = n^{a_l}(W_l + \Delta W_l)(z_{l-1} + \Delta z_{l-1}) = n^{a_l}(W_l z_{l-1} + \Delta W_l z_{l-1} + W_l \Delta z_{l-1} + \Delta W_l \Delta z_{l-1}).$$

Số hạng đầu ($W_l z_{l-1}$) không có alignment (tham số khởi tạo ngẫu nhiên, độc lập dữ liệu). Ba số hạng còn lại **có thể** có alignment, với exponent tương ứng α_l (của $\Delta W_l z_{l-1}$, đo tương quan giữa update tham số và dữ liệu), ω_l (của $W_l \Delta z_{l-1}$, đo tương quan giữa tham số khởi tạo lớp sau và update ở lớp trước), u_l (của $\Delta W_l \Delta z_{l-1}$, pha trộn cả hai). Định nghĩa: $\alpha_l = 1/2$ là "không alignment" (hai vector ngẫu nhiên độc lập, tích vô hướng scale theo $O(\sqrt{\text{fan-in}})$ nhờ Central Limit Theorem), $\alpha_l = 1$ là "alignment cao" (hai vector tương quan/giống nhau, tích vô hướng scale theo $O(\text{fan-in})$ nhờ Law of Large Numbers).

3.2 Nhóm đối xứng và hai lớp tương đương (equivalence classes)

Có một phép biến đổi $a_l \leftarrow a_l + \theta,\ b_l \leftarrow b_l - \theta$ giữ nguyên forward pass (chỉ "chuyển" một hằng số từ khởi tạo sang parameter multiplier), kèm điều chỉnh LR tương ứng theo optimizer để giữ nguyên hiệu ứng backward pass: SGD cần $c_l \leftarrow c_l - 2\theta$, Adam cần $c_l \leftarrow c_l - \theta$, Adafactor giữ c_l nguyên. Nhờ phép biến đổi này, 4 parameterization (Standard, NTK, muP, Mean-Field) chỉ co lại thành **2 lớp tương đương** tại lớp readout: Standard và NTK có $a_{L+1} + b_{L+1} = 1/2$ (logit scale hằng số lúc khởi tạo); muP và Mean-Field có $a_{L+1} + b_{L+1} = 1$ (logit scale $1/\sqrt{n}$ lúc khởi tạo). Đây là khác biệt mấu chốt duy nhất giữa hai lớp. Lưu ý: tương đương này chỉ đúng ở độ chính xác số học vô hạn (infinite precision); mục 4.3 sẽ cho thấy ở độ chính xác hữu hạn thực tế (do epsilon trong Adam), hai lớp "tương đương lý thuyết" này có thể cho kết quả khác nhau rõ rệt.

3.3 Ràng buộc ổn định tổng quát và hai giả định alignment

Bảng 2 của paper (Appendix B.15 lặp lại là Bảng B1) viết hệ ràng buộc ổn định cho SGD, Adam, Adafactor dưới dạng hàm của α_l, ω_l, u_l và r_l (feature-learning residual: exponent âm của Δz_l; $r_l = 0$ nghĩa là lớp đó đang "học đặc trưng" thực sự ở infinite-width limit, $r_l > 0$ nghĩa là update ở lớp đó nhỏ dần về 0 khi $n \to \infty$, tức rơi vào "kernel regime" kiểu NTK tĩnh). Yang & Hu (2021)/Yang & Littwin (2023) tương ứng với lựa chọn cố định $\alpha_l = 1$ với mọi $l \in [2, L+1]$, $\omega_l = 1/2$ với $l < L+1$, và **đặc biệt** $\omega_{L+1} = 1$ (giả định alignment tối đa ở chính lớp readout). Appendix B.16 của paper lưu ý thêm: ràng buộc của công trình trước **không** đặt điều kiện gì lên u_l, vì $\alpha_l = 1$ đã là giá trị tối đa nên $\alpha_l \geq u_l$ luôn đúng, khiến số hạng $\Delta W_l \Delta z_{l-1}$ (do u_l chi phối) không bao giờ là số hạng chiếm ưu thế. Paper chỉ ra: chính giả định $\omega_{L+1} = 1$ là gốc rễ khiến Standard và NTK **không thể nào đạt feature learning** ($r_L = 0$) dù chọn LR thế nào, vì ràng buộc $a_{L+1} + b_{L+1} + r_L - \omega_{L+1} \geq 0$ với $a_{L+1} + b_{L+1} = 1/2 < \omega_{L+1} = 1$ buộc $r_L > 0$.

Paper **nới** ω_{L+1} **xuống** $1/2$ (không giả định alignment đặc biệt ở readout), và xét hai giả định cho α_l, u_l:

- **Full alignment**: $\alpha_l = u_l = 1,\ \omega_l = 1/2\ \forall l$.
- **No alignment**: $\alpha_l = \omega_l = u_l = 1/2\ \forall l$.

Dưới cả hai giả định (khác với công trình trước), **cả 4 parameterization đều đạt được feature learning** ($r_L = 0$) với công thức LR theo lớp phù hợp. Bảng 1 của paper (trích đầy đủ ở mục 10 của ghi chú này) cho exponent LR tối đa-ổn-định (maximum stable) của mỗi lớp, mỗi parameterization, mỗi optimizer, dưới hai giả định này.

Ví dụ cụ thể làm rõ phát hiện trung tâm: với Standard parameterization + Adam + full alignment, Bảng 1 cho LR lớp embedding $\propto n^0 = 1$ (không đổi theo n), LR lớp hidden và readout $\propto 1/n$. Đây chính xác là công thức được khuyến nghị ở mục 4.2.1 (và là kết quả tốt nhất toàn bài cho Adam, xem mục 4).

3.4 Metric alignment ratio

Để đo alignment thực sự xảy ra trong lúc train (không chỉ giả định), paper định nghĩa (Definition 3.1):

$$A_l^t = \log_{\text{fan-in}} \frac{\|W_l^t z_{l-1}^t\|_{RMS}}{\|W_l^t\|_{RMS}\,\|z_{l-1}^t\|_{RMS}} \in \mathbb{R},$$

với norm là RMS norm (căn trung bình bình phương của các phần tử). $A_l^t = 1/2$ tương ứng "không alignment" (CLT scaling $\sqrt{\text{fan-in}}$), $A_l^t = 1$ tương ứng "alignment cao" (LLN scaling fan-in). Metric này đo được ở kích thước hữu hạn (không cần infinite-width limit), và tác giả quan sát nó **hội tụ** (giống nhau) qua các kích thước model đã thử, cho thấy nó mang tính đại diện cho hành vi ở width lớn hơn.

Ví dụ minh hoạ nhanh (tự tính): xét fan-in $= 1024$. Nếu đo được $A = 0.5$ đúng như lý thuyết "không alignment" thì $\|Wz\|/(\|W\|\|z\|) = 1024^{0.5} = 32$ (tự tính: $\sqrt{1024} = 32$). Nếu đo được $A = 0.9$ (gần "alignment cao"), tỉ lệ này tăng lên $1024^{0.9} \approx 1024^{0.9}$; tự tính bằng ln: $0.9 \times \ln(1024) = 0.9 \times 6.931 = 6.238$, $e^{6.238} \approx 512$. Vậy chỉ cần A tăng từ 0.5 lên 0.9, tỉ số chuẩn hoá đã tăng khoảng $512/32 = 16$ lần (tự tính) dù vẫn còn cách xa cực đại lý thuyết $A = 1$ (tỉ số $= 1024$). Đây là lý do một alignment "trung gian" (ví dụ quanh 0.7-0.9 như đo được ở lớp readout của muP/MFP, mục 5) đã đủ để ảnh hưởng mạnh tới LR tối ưu, không cần đạt đúng cực trị $A = 1$ mới "đáng kể".

3.5 Công thức LR theo lớp + hằng số nhân, và hyperparameter transfer

Thực nghiệm chọn base model dim $b = 1024$, định nghĩa LR ở lớp l là $\eta_l = \beta_n \cdot (n/b)^{-c_l}$ (dò β_n ở mỗi width n bằng 1-D sweep), trong đó c_l lấy từ Bảng 1. Sau đó thêm hằng số nhân theo loại lớp γ_1 (embedding), γ_h (hidden), γ_{L+1} (readout): $\eta_l = \beta_n \cdot \gamma_l \cdot (n/b)^{-c_l}$, dò $(\gamma_1, \gamma_h, \gamma_{L+1})$ bằng Bayesian optimization 3 chiều (800 trial, Golovin et al. 2017) **chỉ tại base model dim** $b = 1024$, rồi **dùng lại nguyên** các hằng số này ở mọi width khác (đây là ý tưởng hyperparameter transfer: dò rẻ ở nhỏ, tái sử dụng ở lớn). Bảng C3 (Appendix) liệt kê các hằng số tối ưu tìm được, ví dụ cho Adam + Standard: $\gamma_1 = 5.357,\ \gamma_h = 1.133,\ \gamma_{L+1} = 2.596$; cho Adam + muP: $\gamma_1 = 5.303,\ \gamma_h = 1.258,\ \gamma_{L+1} = 11.723$ (paper nhận xét range hằng số cho Adam trải từ 0.15 đến 11.7, khá rộng, trong khi cho Adafactor/Adam+parameter-scaling range hẹp hơn, 0.5-2.6).

3.6 Epsilon underflow và Adam-atan2

Update Adam chuẩn: $\Delta\theta = -\eta \cdot m/(\sqrt{v} + \epsilon)$ (bỏ qua bias-correction), với ϵ thường là 10^{-8} hoặc 10^{-9}, ban đầu chỉ để tránh chia 0. Vì gradient co nhỏ theo width (xem cột "Gradient" ở Bảng 1, ví dụ Mean-Field hidden layer gradient co theo $1/n^{1.5}$, nhanh nhất trong 4 parameterization), đến một width đủ lớn, $\sqrt{v}$ (căn bậc hai bình phương trung bình gradient) rơi xuống dưới ϵ, khi đó ϵ không còn là "hằng số không đáng kể" mà **chi phối hoàn toàn mẫu số**, phá vỡ tính scale-invariant của Adam (lẽ ra nhân gradient với hằng số không đổi kết quả update).

Paper đề xuất **Adam-atan2**: thay dòng code

```
lambda m, v: m / (jnp.sqrt(v + eps_root) + eps)
```

bằng

```
lambda m, v: a * jnp.arctan2(m, b * jnp.sqrt(v))
```

với $a = b = 1$ trong mọi thí nghiệm. Hàm `atan2(x, y)` trả về $\arctan(x/y)$ đúng góc phần tư, xác định cả tại $(0, 0)$, và **scale-invariant tới hạn độ chính xác số** (vì chỉ phụ thuộc tỉ số x/y chứ không có số hạng cộng phá tỉ lệ). Do xấp xỉ góc nhỏ $\arctan\theta \approx \theta$ khi θ nhỏ, `atan2` gần giống phép chia thường khi $m \ll \sqrt{v}$, nhưng tiệm cận $\pm\pi/2$ khi $m/\sqrt{v} \to \pm\infty$ (thay vì chia cho ϵ rất nhỏ ra số khổng lồ). Hai cách khác được so sánh: giảm epsilon hằng số (ví dụ $10^{-9} \to 10^{-15}$), hoặc scale epsilon theo từng lớp tỉ lệ với gradient scale của lớp đó ($\epsilon_l = \epsilon_{base} \cdot (n/b)^{-g_l}$, g_l là exponent gradient ở Bảng 1).

Ví dụ số minh hoạ underflow (tự tính, không phải số của paper, chỉ dựng để thấy cơ chế): giả sử tại một lớp, $m = 10^{-9}$, $v = 10^{-20}$ (nên $\sqrt{v} = 10^{-10}$), $\epsilon = 10^{-8}$ (giá trị default phổ biến).

- Adam chuẩn: $m/(\sqrt{v} + \epsilon) = 10^{-9}/(10^{-10} + 10^{-8}) = 10^{-9}/(1.01 \times 10^{-8}) \approx 0.0990$ (tự tính).
- Tỉ số "đúng" nếu không có ϵ: $m/\sqrt{v} = 10^{-9}/10^{-10} = 10$.
- Vậy ϵ làm biên độ update tụt từ 10 xuống ≈ 0.099, tức co lại khoảng 101 lần (tự tính: $10/0.099 \approx 101$) chỉ vì ϵ lớn hơn $\sqrt{v}$ gần 100 lần, không phản ánh đúng "tín hiệu" gradient.
- Adam-atan2: $\arctan(m/\sqrt{v}) = \arctan(10) \approx 1.4711$ rad (tự tính).
- Bây giờ nhân cả m, v lên 1000 lần (giữ tỉ số không đổi): $m' = 10^{-6}$, $v' = 10^{-14}$ ($\sqrt{v'} = 10^{-7}$). Adam chuẩn: $m'/(\sqrt{v'} + \epsilon) = 10^{-6}/(10^{-7} + 10^{-8}) = 10^{-6}/(1.1 \times 10^{-7}) \approx 9.09$ (tự tính), khác hẳn 0.099 ở trên dù tỉ số gradient/uncertainty là như nhau, chỉ vì scale tuyệt đối đổi. Adam-atan2: $\arctan(m'/\sqrt{v'}) = \arctan(10) \approx 1.4711$ rad, **giống y hệt** giá trị trước (tự tính) vì atan2 chỉ phụ thuộc tỉ số $m/\sqrt{v}$. Đây minh hoạ trực tiếp cơ chế "scale-invariant" mà Adam-atan2 phục hồi được, khác với Adam chuẩn có kết quả lệch nhau gần 92 lần ($9.09/0.099 \approx 92$, tự tính) cho cùng một tỉ số gradient/nhiễu chỉ vì đổi scale tuyệt đối.

4. Thí nghiệm và kết quả chính

Thiết lập (Appendix C.1-C.3, xác nhận bằng grep số liệu). Kiến trúc NanoDO: decoder-only Transformer, learned positional embedding, pre-layer-norm, GeLU, không bias, không tie embedding/readout. Dữ liệu C4, tokenizer T5 SentencePiece, vocab size $V = 32{,}001$ (32.000 gốc + 1 token BOS). Batch size cố định 256, context length 512, depth $L = 8$ block Transformer. Head dimension cố định $h = 128$; co-scale D (model dim) và số head H sao cho $D = H \times h$; MLP width $F = 4D$. **14 kích thước width** từ $D = 128$ ($H = 1$, khoảng 9.9 triệu tham số) đến $D = 16{,}384$ ($H = 128$, khoảng **26.8 tỷ** tham số) - Bảng C1 liệt kê đủ cả 14 dòng, ví dụ $D = 4096$ ($H = 32$): tổng 1,875,750,912 tham số (~1.9B); $D = 12{,}288$ ($H = 96$): 15,290,929,152 (~15.3B). Huấn luyện 50.000 bước, warmup tuyến tính 1.000 bước rồi cosine decay về 0, **không weight decay** ở thí nghiệm chính (có kiểm tra riêng với weight decay ở Appendix G). Sweep LR với granularity $2^{0.25}$ hoặc $2^{0.5}$ (nhân/chia LR theo luỹ thừa của 2 với số mũ 0.25 hoặc 0.5 mỗi bước sweep). Tổng cộng: 3 optimizer (SGD+momentum, Adam, Adafactor/Adam+parameter-scaling) x 4 parameterization x nhiều giả định alignment x hơn chục LR x 14 width = **hàng chục nghìn lần chạy** (paper dùng đúng chữ "tens of thousands of experiments").

Bảng 3 của paper - kết quả chính cho model 26.8B (loss eval tốt nhất mỗi dòng):

Optimizer	Param.	Global LR + default const	Per-layer LR (full align) + default const	Per-layer LR (full align) + optimal const	Per-layer LR (no align) + optimal const
SGD	Standard	3.657	3.510	3.385	3.318
SGD	NTK	3.732	3.627	3.313	3.324
SGD	muP	4.224	4.184	3.809	4.222
SGD	Mean-Field	4.092	4.319	4.092	3.898
Adam	Standard	2.625	2.576	**2.572**	2.575

Optimizer	Param.	Global LR + default const	Per-layer LR (full align) + default const	Per-layer LR (full align) + optimal const	Per-layer LR (no align) + optimal const
Adam	NTK	2.592	2.615	2.591	2.581
Adam	muP	2.727	2.646	2.599	2.590
Adam	Mean-Field	2.638	2.630	2.633	2.638
Adam+PS	Standard	2.580	2.613	2.675	2.577
Adam+PS	NTK	2.570	2.623	2.667	**2.566**
Adam+PS	muP	2.574	2.655	2.606	2.575
Adam+PS	Mean-Field	2.624	2.640	2.772	2.623

(In đậm = tốt nhất toàn bảng cho mỗi optimizer, theo đúng cách paper đánh dấu ”bold + red”; Adam+PS = Adam + parameter scaling, đại diện họ Adafactor trong thí nghiệm chính vì Adafactor thật gặp lỗi kỹ thuật với FSDP.) **Kết quả chính:** với Adam, tốt nhất là Standard parameterization + per-layer LR ”full alignment” + hằng số tối ưu (2.572); với Adam+parameter-scaling, tốt nhất lại là NTK + per-layer LR ”no alignment” (tương đương global LR) + hằng số tối ưu (2.566) - paper gọi đây là ”quite unexpected” (khá bất ngờ), vì nó không khớp với trực giác ”muP luôn tốt nhất”.

So sánh công bằng Standard vs muP (mục 4.2.1). Khi cả hai được cho cùng cơ hội (per-layer LR + hằng số tối ưu), model Standard-param 15.3B tham số **vượt** model muP 26.8B tham số, dù chỉ có 57% số tham số (tự tính kiểm tra lại: $15{,}290{,}929{,}152/26{,}830{,}356{,}480 = 0.570$, khớp con số 57% mà paper nêu). Tác giả giải thích: các so sánh trước đây nghiêng về muP (như muTransfer gốc) vì so sánh không công bằng - muP được cho per-layer LR + hằng số đã tune, còn Standard chỉ có global LR.

Hyperparameter transfer (Hình 4). Với Adam + per-layer LR ”full alignment” + hằng số tối ưu, fit power-law exponent của LR tối ưu theo D: Standard -0.05, NTK -0.02, muP -0.06, Mean-Field $+0.26$. Exponent gần 0 nghĩa là **LR tối ưu gần như không đổi qua các width** (vì phần phụ thuộc width đã được ”gói” vào exponent c_l lý thuyết rồi) - đây là minh chứng transfer thành công cho mọi parameterization (không riêng muP), trái với tuyên bố của muTransfer (Yang et al. 2022) rằng chỉ muP mới transfer được.

Epsilon mitigations (Hình 7, mục 4.3). So 4 cách xử lý epsilon: default ($\epsilon = 10^{-9}$), constant nhỏ (10^{-15}), per-layer epsilon (base 10^{-12}), Adam-atan2. Cả 3 cách sửa cải thiện tương đương nhau; NTK và Mean-Field cải thiện rõ (gap tăng theo kích thước model), Standard và muP ở các kích thước đã thử hầu như không đổi (chưa chạm ngưỡng underflow). Sau khi sửa epsilon, hai cặp ”tương đương lý thuyết” (Standard~NTK, muP~Mean-Field, xem mục 3.2) mới thực sự cho performance xấp xỉ bằng nhau - trước khi sửa, chúng khác nhau rõ rệt dù lý thuyết nói chúng tương đương.

Fixed-step vs compute-optimal (Appendix I, Bảng I1). Lý thuyết ở mục 3 giả định số bước huấn luyện $T = O(1)$ (hằng số, không đổi theo width) để đơn giản hoá đạo hàm. Nhưng Chinchilla (bài llm 04, qua trích dẫn Hoffmann et al. 2022) cho biết ở chế độ compute-optimal thật, số token tối ưu tỉ lệ với số tham số, mà số tham số tỉ lệ n^2 (ma trận $n \times n$) nên số bước tối ưu phải tăng theo $\Theta(n^2)$, mâu thuẫn với giả định $T = O(1)$. Thực nghiệm đo exponent LR-tối-ưu-theo-D ở cả hai chế độ (50.000 bước cố định so với Chinchilla 20x compute-optimal): mọi tổ hợp optimizer x parameterization đều cho exponent **âm hơn** (LR phải giảm nhanh hơn theo width) ở chế độ compute-optimal. Ví dụ Adam+Standard: -0.95 (fixed step) so với -1.18 (compute-optimal); SGD+Standard: -0.38 so với -1.27. Mức chênh trung vị trên 12 tổ hợp là 0.46 (dao động 0.01 đến 0.89), nhỏ hơn nhiều mức chênh 2 mà phân tích lý thuyết ”tệ nhất” (worst-case, nếu chia LR cho n^2 để bù số bước) sẽ dự đoán - nhưng vẫn là một hiệu chỉnh thực, không thể bỏ qua.

5. Phân tích / ablation

- **Alignment đo được (Hình 2, mục 4.1).** Ở cả 4 parameterization, alignment ratio khởi đầu ở 0.5 (ngẫu nhiên, độc lập dữ liệu lúc init) rồi biến động mạnh trong lúc train. Với SGD: kết quả rất bất ổn, khó đọc; một mẫu hình nhất quán là NTK/Mean-Field gần như không có alignment còn Standard/muP có một ít. Với Adam và Adafactor: lớp readout của **muP và Mean-Field** đạt peak alignment rất cao (>0.9) sớm trong training rồi giảm về mức trung bình; lớp readout của Standard/NTK chỉ đạt mức trung bình (0.7-0.8); các lớp MLP ở mọi parameterization chỉ đạt alignment trung bình, không vượt 0.8. Tác giả giải thích: ở muP/MFP, scale của readout update lớn hơn scale readout khởi tạo nên update sớm ”lấn lướt” phần khởi tạo ngay từ đầu (alignment cao); ở Standard/NTK, scale update và khởi tạo tương đương nên dù update có thể alignment cao, nó không lấn lướt được khởi tạo để tạo alignment tổng thể cao.
- **Full alignment vs no alignment cho Adam (mục 4.2.1).** No-alignment cho eval loss nhúc nhích tốt hơn ở NTK/muP/Mean-Field, nhưng khi fit power-law cho LR tối ưu theo D dưới no-alignment, exponent rơi vào khoảng -0.58 đến -0.69 (xa 0 rất nhiều, nghĩa là **không** transfer tốt qua width), trong khi full-alignment cho exponent gần 0 (-0.05 đến -0.06, trừ Mean-Field $+0.26$). Kết luận: full-alignment ”gói” scaling vào exponent tốt hơn hẳn, nên được khuyến nghị dù loss tuyệt đối đôi khi nhúc nhích kém hơn no-alignment một chút.
- **Full alignment vs no alignment cho Adam+parameter-scaling, lý do ngược lại (mục 4.2.2).** Ở đây no-alignment (= global LR, vì mọi exponent c_l trở thành 0 dưới no-alignment cho họ Adafactor) lại **tốt hơn** full-alignment về cả ba mặt: (a) eval loss thấp hơn và khoảng cách càng rộng khi scale lên; (b) full-alignment có độ nhạy LR rất cao (đường cong loss-vs-LR cong gắt hơn, LR tối ưu gần sát LR-tối-đa-ổn-định, rủi ro mất ổn định); (c) hằng số tối ưu transfer tốt dưới no-alignment nhưng lại **hại** performance dưới full-alignment khi dùng hằng số mặc định. Thú vị là dù vậy, cả hai giả định đều cho exponent LR-vs-D gần 0 (từ -0.29 đến $+0.05$), khác hẳn mẫu hình rõ ràng (gần 0 cho full-align, xa 0 cho no-align) thấy ở Adam thường - cho thấy parameter scaling đưa vào một cơ chế co giãn phức tạp hơn lý thuyết width-scaling thuần tuý ở đây mô tả.
- **Vì sao Adam+parameter-scaling ít nhạy hằng số layer hơn Adam thường?** Parameter scaling (nhân update đã chuẩn hoá với norm hiện tại của tham số) tự động làm mỗi lớp có update ”đúng scale” so với tham số hiện có, bất kể parameterization - về khái niệm, nó tự làm một phần việc mà lý thuyết parameterization phải làm thủ công qua chọn LR. Đây lý giải vì sao Adafactor (và Adam+PS) ít nhạy với lựa chọn parameterization hơn SGD/Adam.
- **Weight decay (Appendix G).** Thêm decoupled weight decay 10^{-4} (kiểu AdamW) cải thiện eval loss ở mọi parameterization/kích thước nhưng **không** đổi dạng scaling của LR tối ưu theo width một cách đáng kể - tác giả dùng đây làm cross-check rằng kết luận chính nên transfer tốt sang chế độ có (một ít) weight decay thực tế.
- **Adafactor thật so với Adam+parameter-scaling (Appendix H).** Vì Adafactor thật gặp lỗi tensor-shape với FSDP của paper nên hầu hết thí nghiệm chính dùng Adam+PS làm đại diện. Cross-check trực tiếp hai optimizer trên các model tới 4B tham số cho thấy hành vi scaling tương tự (chênh lệch nhỏ), xác nhận Adam+PS là proxy hợp lý cho Adafactor trong nghiên cứu này.

6. Giới hạn

Paper tự nêu (mục 6 Limitations):

- Chỉ nghiên cứu width scaling một cách tách biệt; thực tế cần co-scale cùng lúc nhiều chiều (width, depth, batch size, training horizon, weight decay, learning rate) và bài toán co-scale đồng thời tất cả vẫn mở.
- Lý thuyết coi vocab size là $O(1)$ so với width, nhưng thực nghiệm dùng vocab cố định $32{,}000$ - con số này lớn tương đương chính các width lớn nhất đã thử ($D = 16{,}384$), nên ”thực tế hơn” có lẽ nên coi vocab là $O(n)$ chứ không phải $O(1)$; đây là một chỗ lý thuyết và thực nghiệm chưa khớp hoàn toàn.
- Giả định $T = O(1)$ bước huấn luyện (hằng số theo width) mâu thuẫn với chế độ compute-optimal thực

tế (cần $T \propto n^2$ theo lập luận Chinchilla); paper chỉ xử lý bằng thực nghiệm (Appendix I) chứ chưa có lý thuyết mới cho trường hợp này.
- Epsilon về lý thuyết nên được scale theo lớp (per-layer), nhưng ở các kích thước đã thử, tác giả **không** thấy optimal constant-epsilon phụ thuộc vào scale - để ngỏ khả năng điều này thay đổi ở model lớn hơn nữa.
- Future work được gợi ý: optimizer/scheduler "biết" alignment (alignment-aware), mở rộng phân loại feature-learning/kernel-regime sang không gian alignment tổng quát, và nghiên cứu depth/batch size scaling tương tự.

Người ghi chú nhận xét thêm:

- Toàn bộ thí nghiệm dùng **một** kiến trúc (NanoDO decoder-only, GeLU, pre-LN, không bias, không tie embedding), **một** dataset (C4), batch size cố định 256, context cố định 512. Không có gì đảm bảo các exponent đo được transfer sang kiến trúc khác hẳn (ví dụ các kiến trúc hybrid attention + linear-attention/SSM như trong llm 20-26), vì cơ chế lan truyền gradient/alignment có thể khác.
- Bản thân tác giả nhấn mạnh một điểm tinh tế dễ bị đọc lướt qua: "không chắc learning rate tối đa-ổn-định (maximum stable) và learning rate tối ưu (optimal) co giãn theo cùng exponent" - nghĩa là lý thuyết ở mục 3 chỉ cho một **trần an toàn** (ceiling) để biết "nên tìm LR trong vùng nào", không tự động cho đúng con số LR tối ưu; vẫn cần sweep thực nghiệm (như chính paper đã làm) để tìm hằng số nhân.
- Alignment metric (Definition 3.1) được định nghĩa ở kích thước hữu hạn như một **xấp xỉ** cho hành vi infinite-width; paper chỉ quan sát thực nghiệm rằng nó "hội tụ" qua các kích thước đã thử (không có chứng minh hội tụ hình thức).
- Kiểm tra chéo nội bộ: Hình 4 và Hình E1 của paper fit lại power-law cho cùng nhóm cấu hình "Adam + per-layer LR full-alignment + optimal constants" nhưng cho số khác nhau đôi chút. Với Standard: Hình 4 cho $LR = 2.38 \times 10^{-3} \cdot D^{-0.05}$, Hình E1 cho $LR = 2.43 \times 10^{-3} \cdot D^{-0.05}$ (chỉ lệch hằng số nhân, exponent giữ nguyên -0.05). Với NTK thì lệch cả exponent: Hình 4 cho $LR = 8.57 \times 10^{-2} \cdot D^{-0.02}$, Hình E1 cho $LR = 9.51 \times 10^{-2} \cdot D^{-0.03}$. Các lệch này nhỏ (trong sai số dò/làm tròn của việc fit hai lần trên dữ liệu tương tự) và không đổi kết luận chính ("gần như scale-invariant"), nhưng là một chỗ nên biết nếu đối chiếu số liệu chính xác giữa các hình của paper.

7. Liên hệ với các bài khác

- **opt 35 (Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer, muP).** Nền tảng trực tiếp: bài 36 dùng đúng khung "parameterization = (a_l, b_l, c_l)" của Yang & Hu (2021), chỉ mở rộng giả định alignment ($\omega_{L+1} = 1 \to 1/2$, và α_l, u_l từ cố định thành biến liên tục đo thực nghiệm). Kết luận gây bất ngờ nhất của bài 36 ("muP không phải lúc nào cũng tốt nhất, Standard-param với LR theo lớp còn thắng muP") là một phản biện trực tiếp vào đúng tuyên bố trung tâm của muTransfer.
- **opt 06 (Adam: A Method for Stochastic Optimization).** Optimizer được phân tích chi tiết nhất trong bài 36 (Adam chiếm phần lớn thí nghiệm chính và là nơi phát hiện epsilon-underflow). Hiểu cơ chế cập nhật $m/(\sqrt{v} + \epsilon)$ của Adam là tiền đề để hiểu vì sao Adam-atan2 (mục 3.6) hoạt động.
- **opt 45 (Deconstructing What Makes a Good Optimizer for Language Models).** Cùng năm 2024, cùng nhóm câu hỏi "cái gì thực sự quan trọng khi chọn optimizer cho LM", nhưng góc nhìn khác: opt 45 so sánh thực nghiệm nhiều optimizer (SGD, Adam, Adafactor+momentum, Lion, Signum) trên 4 quy mô model (150M đến 1.2B tham số, nhỏ hơn hẳn 26.8B của bài 36) với một LR toàn cục mỗi lần chạy, để tìm xem optimizer nào thực sự khác biệt (kết luận của opt 45: hầu hết tương đương nhau, trừ SGD). Bài 36 không so optimizer nào "tốt nhất" mà cố định 3 họ optimizer rồi tập trung vào **cách LR của từng lớp phải co giãn theo width** trong mỗi họ. Hai bài bổ sung nhau: một trả lời "optimizer nào", một trả lời "scale optimizer đó thế nào khi đổi kích thước".
- **opt 16 (Muon is Scalable for LLM Training).** Muon là optimizer mới (dựa trên orthogonalization của update, không thuộc gọn vào 3 họ SGD/Adam/Adafactor mà bài 36 xét) ra đời sau bài 36; bài 36 không

phân tích Muon, nhưng khung ”parameterization tổng quát theo alignment” của bài 36 là một công cụ tự nhiên để ai muốn suy ra exponent LR-theo-width cho Muon trong tương lai (câu hỏi mở, chưa có câu trả lời trong cả hai bài).

- **pruning 38 (Sheared LLaMA).** Pruning 38 thực hành continue-pretraining (giai đoạn ”recovery”) sau khi cắt cấu trúc, với một lịch LR/warmup riêng cho giai đoạn phục hồi. Bài 36 cho một góc nhìn lý thuyết bổ sung: sau khi cắt hidden width từ n xuống n', nếu muốn áp mô hình parameterization-exponent (ví dụ $\eta_{hidden} \propto 1/n$ cho Adam+Standard+full-align) cho giai đoạn phục hồi, LR hidden ”đúng” theo tỉ lệ width mới phải đổi theo hệ số n/n' - một gợi ý suy luận (paper 36 không thử nghiệm tình huống pruning-rồi-recovery) nhưng đáng kiểm tra thực nghiệm (xem mục 9).

8. Ứng dụng cho dự án (suy luận)

Dự án của người dùng là nén/hậu huấn luyện **LightOnOCR-3-0.8B** (VLM OCR, hybrid GatedDeltaNet + full attention, ~3.600 token/trang, GPU ≤ 8 GB, batch $= 1$). Cần nói thẳng: **phần lý thuyết trung tâm của bài 36 (width-scaling, hyperparameter transfer từ model nhỏ sang model lớn tới 26.8B) ít liên quan trực tiếp** tới dự án này, vì ba lý do:

1. Dự án không **scale width lên**: 0.8B đã là kích thước đích cố định, không cần dò LR trên proxy nhỏ rồi transfer lên bản lớn hơn như muP/bài 36 nhắm tới.
2. Toàn bộ lý thuyết của bài 36 (và cả Table 1) giả định huấn luyện **từ khởi tạo ngẫu nhiên** (”độc lập dữ liệu lúc init” là điểm neo để định nghĩa alignment $= 0.5$ ở bước đầu). Dự án fine-tune/distill/RL-tune một checkpoint **đã pretrain xong**, tức tham số đã alignment cao với dữ liệu ngay từ đầu giai đoạn hậu huấn luyện - không khớp với giả định nền của lý thuyết.
3. Mọi thí nghiệm dùng batch $= 256$ cố định; batch size scaling bị paper loại khỏi phạm vi nghiên cứu một cách rõ ràng (mục 6 Limitations liệt kê batch size là ”chiều khác cần scale” nhưng chưa làm). Không có tín hiệu nào từ bài 36 áp dụng trực tiếp cho batch $= 1$.

Tuy vậy, có vài ý **thực dụng** (suy luận, không phải kết luận của paper) vẫn đáng mang qua:

- **Per-layer LR nhóm theo loại tham số.** Bài học định tính ”các loại lớp khác nhau (embedding, hidden, readout/head) có gradient/update scale khác nhau nên muốn LR khác nhau” vẫn hợp lý ngay cả ở một width cố định, đặc biệt khi fine-tune/distill một model đã pruning (một số lớp bị cắt/khởi tạo lại, một số lớp giữ trọng số pretrain nguyên). Có thể cân nhắc dùng **LR nhóm** (ví dụ LR riêng cho embedding/LM head so với các block GatedDeltaNet/attention) khi tiếp tục train, thay vì một LR toàn cục - đây là cách làm phổ biến trong fine-tune LLM nói chung, và bài 36 cho một lý do định lượng (dù lý thuyết của nó nói về từ-đầu, không phải fine-tune) vì sao điều đó hợp lý.
- **Cảnh báo epsilon underflow khi train ở precision thấp.** Dù lý do gốc của bài 36 là width lớn, cơ chế cốt lõi (epsilon chi phối mẫu số khi $\sqrt{v}$ nhỏ) có thể xảy ra ở **bất kỳ** tình huống gradient nhỏ, kể cả ở model 0.8B nếu dùng bf16/fp16, batch=1 (gradient noisy, có thể có lớp gradient rất nhỏ), hoặc sau khi pruning làm một phần gradient bị dồn nén. Việc paper trích dẫn Wortsman et al. (2023, không phải số liệu của chính bài 36 này) rằng gradient norm của model Standard-param 1.2B từng tiến gần 10^{-8} là một lời nhắc hữu ích: nếu fine-tune không ổn định một cách khó hiểu, thử giảm epsilon (10^{-12} hoặc 10^{-15}) hoặc đổi sang Adam-atan2 (chỉ một dòng code, không cần tune epsilon) là một thử nghiệm rẻ, không cần hiểu hết lý thuyết width-scaling.
- **Rescale LR khi width thay đổi do pruning (suy luận, chưa kiểm chứng).** Nếu dự án từng cắt hidden width của một phần model (ví dụ giảm d_{model} hoặc state size của GatedDeltaNet) rồi continue-train để phục hồi, bài 36 gợi ý một quy tắc định tính để chọn LR mới: xem lớp bị cắt là ”lớp hidden” trong ngôn ngữ Bảng 1, và nếu dùng Adam, LR hidden lý thuyết (full-align) co theo $1/n$; giảm n (cắt width) theo lý thuyết này đề xuất LR hidden nên **tăng** theo tỉ lệ $n_{\text{cũ}}/n_{\text{mới}}$ so với LR gốc dùng cho model chưa cắt - ngược lại hoàn toàn với trực giác ”model nhỏ hơn thì cần LR nhỏ hơn cho an toàn”. Đây chỉ là suy luận ngoại suy có rủi ro cao (paper không thử nghiệm tình huống pruning), nên nếu áp dụng cần kiểm

chứng thực nghiệm nhỏ trước (xem thí nghiệm gợi ý #5 ở mục 9), không nên tin tưởng mù quáng.

9. Thí nghiệm gợi ý

1. **Tái hiện hyperparameter transfer trên MLP đồ chơi.** Huấn luyện MLP 2-3 lớp ẩn trên một bài hồi quy/phân loại tổng hợp, ở 4-5 width khác nhau (ví dụ $n = 64, 128, 256, 512, 1024$), với (a) LR toàn cục cố định theo width và (b) LR theo lớp dùng đúng exponent Bảng 1 cho Standard+Adam+full-alignment (embedding $O(1)$, hidden/readout $O(1/n)$). Ở mỗi width, sweep LR toàn cục (hoặc β_n cho cách (b)) để tìm optimal. Kỳ vọng: đường LR-tối-ưu-theo-width của (b) gần như nằm ngang (tái hiện rẻ tiền Hình 4 của paper), còn (a) lệch rõ theo width.
2. **Đo alignment ratio trong lúc train.** Cài Definition 3.1 ($A_l^t = \log_{fan\text{-}in}(\|W_l z_{l-1}\|_{RMS} / (\|W_l\|_{RMS}\|z_{l-1}\|_{RMS}))$) cho một MLP/Transformer nhỏ, log theo từng bước train với SGD và với Adam. Kỳ vọng: giá trị bắt đầu gần 0.5 ở bước 0 (độc lập vì vừa khởi tạo), tăng lên khi train; so sánh dạng đường cong giữa lớp readout và lớp hidden để kiểm tra lại quan sát "readout alignment có đỉnh sớm cao hơn ở các parameterization kiểu muP" (mục 5).
3. **Tái hiện epsilon-underflow và Adam-atan2.** Huấn luyện một MLP sâu (6-8 lớp) ở vài width với Adam, cố tình đẩy gradient về nhỏ (ví dụ đóng băng phần lớn lớp, hoặc dùng fan-in rất lớn ở một lớp), sweep epsilon trong $\{10^{-6}, 10^{-9}, 10^{-12}, 10^{-15}\}$; so sánh với việc đổi update rule sang `atan2(m, sqrt(v))` (một dòng code, xem mục 3.6) và không cần tune epsilon. Đo: eval loss cuối cùng theo epsilon (kỳ vọng dạng chữ U như Hình 6 của paper: quá nhỏ gây bất ổn định, quá lớn gây dưới-tối-ưu); Adam-atan2 kỳ vọng khớp với điểm tốt nhất của sweep epsilon mà không cần dò.
4. **LR theo nhóm tham số khi fine-tune model nhỏ đã pretrain.** Lấy một LM nhỏ đã pretrain (ví dụ GPT-2 small hoặc một checkpoint nội bộ cỡ tương đương chạy được trên GPU 8 GB), fine-tune trên một tập downstream nhỏ với (a) một LR toàn cục và (b) ba nhóm LR riêng (embedding, block giữa, lm-head) theo tỉ lệ gợi ý từ Bảng 1 (ví dụ embedding $\times 1$, hidden/readout nhân với một hệ số nhỏ hơn). So sánh tốc độ giảm loss và loss cuối cùng; liên hệ trực tiếp ý suy luận #1 ở mục 8.
5. **Rescale LR sau khi cắt width rồi phục hồi (kiểm chứng ý suy luận #3 ở mục 8).** Lấy một MLP/Transformer nhỏ, cắt width của các lớp hidden xuống một nửa (giữ phần trọng số còn lại, giống cắt cấu trúc đơn giản), rồi continue-train (recovery) với (a) LR giữ nguyên như model gốc, (b) LR hidden nhân với $n_{\text{cũ}}/n_{\text{mới}} = 2$ (theo gợi ý suy luận ở mục 8), (c) LR hidden nhân với $\sqrt{n_{\text{cũ}}/n_{\text{mới}}}$ (nếu dùng SGD thay Adam, theo cột SGD full-align $1/\sqrt{n}$ của Bảng 1). Đo tốc độ hồi phục loss ban đầu; nếu (b) hoặc (c) hồi phục nhanh hơn (a) một cách nhất quán, đó là tín hiệu thực nghiệm đầu tiên rằng logic width-scaling của bài 36 có ích ngoài phạm vi "train từ đầu" mà paper thiết kế.

10. Tóm tắt công thức / khái niệm

Ký hiệu parameterization: mỗi lớp l có (a_l, b_l, c_l): parameter multiplier n^{-a_l}, init std n^{-b_l} ($W_l \sim \mathcal{N}(0, n^{-2b_l})$), $\eta_l \propto n^{-c_l}$.

Phép biến đổi giữ nguyên forward pass: $a_l \leftarrow a_l + \theta$, $b_l \leftarrow b_l - \theta$; để giữ backward pass: SGD $c_l \leftarrow c_l - 2\theta$, Adam $c_l \leftarrow c_l - \theta$, Adafactor c_l không đổi.

Alignment ratio (Definition 3.1): $A_l^t = \log_{fan\text{-}in} \dfrac{\|W_l^t z_{l-1}^t\|_{RMS}}{\|W_l^t\|_{RMS}\|z_{l-1}^t\|_{RMS}}$; $A = 1/2$ = không alignment (CLT, $\sqrt{fan\text{-}in}$), $A = 1$ = alignment cao (LLN, $fan\text{-}in$).

Bảng 1 của paper (Max stable LR exponent theo width n, đầy đủ 4 parameterization):

Param.	Lớp	Init var	Param mult.	Gradient	SGD (full align)	Adam (full align)	Adafactor (full align)	SGD (no align)	Adam (no align)	Adafactor (no align)
Standard	Embedding	1	1	$1/\sqrt{n}$	$\sqrt{n}$	1	1	$\sqrt{n}$	1	1
Standard	Hidden	$1/n$	1	$1/\sqrt{n}$	$1/\sqrt{n}$	$1/n$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	1
Standard	Readout	$1/n$	1	1	$1/n$	$1/n$	$1/\sqrt{n}$	$1/\sqrt{n}$	$1/\sqrt{n}$	1
NTK	Embedding	1	1	$1/\sqrt{n}$	$\sqrt{n}$	1	1	$\sqrt{n}$	1	1
NTK	Hidden	1	$1/\sqrt{n}$	$1/n$	$\sqrt{n}$	$1/\sqrt{n}$	$1/\sqrt{n}$	n	1	1
NTK	Readout	1	$1/\sqrt{n}$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	$1/\sqrt{n}$	$\sqrt{n}$	1	1
muP	Embedding	1	$\sqrt{n}$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	1	1	$1/\sqrt{n}$	1
muP	Hidden	$1/n$	1	$1/n$	1	$1/n$	$1/\sqrt{n}$	$\sqrt{n}$	$1/\sqrt{n}$	1
muP	Readout	$1/n$	$1/\sqrt{n}$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	1	1	1	1
Mean-Field	Embedding	1	1	$1/n$	n	1	1	n	1	1
Mean-Field	Hidden	1	$1/\sqrt{n}$	$1/n^{1.5}$	n	$1/\sqrt{n}$	$1/\sqrt{n}$	$n^{1.5}$	1	1
Mean-Field	Readout	1	$1/n$	$1/n$	n	1	1	n	$\sqrt{n}$	1

(Cột ”LR” ghi exponent theo chiều **tăng dần theo** n của chính đại lượng đó, ví dụ ”$1/n$” nghĩa là $\eta \propto 1/n$, LR giảm khi n tăng.)

Công thức LR theo lớp + hằng số thực nghiệm: $\eta_l = \beta_n \cdot \gamma_l \cdot (n/b)^{-c_l}$, $b = 1024$ là base model dim, β_n dò theo width, γ_l (embedding/hidden/readout) dò một lần ở b rồi tái sử dụng (hyperparameter transfer).

Adam-atan2: thay $m/(\sqrt{v}+\epsilon)$ bằng $a \cdot \text{atan2}(m, b\cdot\sqrt{v})$ với $a = b = 1$; loại bỏ hoàn toàn ϵ, scale-invariant.

Kết quả tốt nhất cho Adam (26.8B, Bảng 3): Standard parameterization, LR embedding $O(1)$, LR hidden/readout $O(1/n)$, per-layer ”full alignment” + hằng số tối ưu $\rightarrow$ eval loss 2.572 (thấp nhất trong toàn Bảng 3 cho Adam).

Mức chênh fixed-step vs compute-optimal (Bảng I1): trung vị 0.46 đơn vị exponent (khoảng 0.01 đến 0.89) trên 12 tổ hợp optimizer x parameterization; luôn cùng chiều (compute-optimal cần LR giảm nhanh hơn theo width).

11. Kết luận ngắn

Bài này tổng quát hoá lý thuyết parameterization kiểu Tensor Programs/muP bằng cách thay giả định alignment cố định (tham số luôn ”thẳng hàng tối đa” với dữ liệu, trừ layer readout) bằng ba biến alignment liên tục đo được thực nghiệm, rồi suy lại toàn bộ hệ ràng buộc ổn định cho SGD, Adam, Adafactor. Đo alignment thật trong hàng chục nghìn lần chạy (tới 26.8B tham số) cho thấy giả định cũ quá bảo thủ: alignment thực tế nằm ở mức trung gian, biến động mạnh theo bước huấn luyện và theo layer. Dưới hai giả định mới (”full alignment”, ”no alignment”), **mọi** parameterization (không riêng muP) đều đạt hyperparameter transfer qua width nếu dùng đúng exponent LR theo lớp và tune thêm một hằng số nhân (dò rẻ ở model nhỏ, tái sử dụng ở model lớn). Phát hiện gây tranh luận nhất: với Adam, Standard parameterization với công thức LR theo lớp mới (embedding $O(1)$, hidden/readout $O(1/n)$) **vượt** muP, kể cả khi muP được cho gấp rưỡi số tham số. Song song đó, bài phát hiện epsilon trong Adam gây ”underflow” (mất tính scale-invariant) ở model đủ lớn và đề xuất Adam-atan2, một sửa đổi một-dòng-code loại bỏ hoàn toàn siêu tham số epsilon. Hạn chế lớn nhất: chỉ xét width scaling tách biệt (bỏ qua depth, batch size, training horizon cùng co giãn), và lý thuyết $T = O(1)$ bước không khớp chế độ compute-optimal thực tế (được bù bằng thực nghiệm riêng, không có lý thuyết mới). Với dự án nén/hậu huấn luyện một VLM OCR 0.8B cố định ở GPU nhỏ, phần lý thuyết ”transfer

từ nhỏ lên lớn” hầu như không áp dụng trực tiếp; cái mang qua được chỉ là vài heuristic suy luận cấp thấp (LR theo nhóm lớp, cẩn trọng epsilon ở precision thấp).

Nguồn gốc: https://arxiv.org/abs/2407.05872

Bài 37

Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification (2015)

Tác giả: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft Research).
Venue / arXiv: arXiv:1502.01852 (v1, 06/02/2015); paper không nêu venue hội nghị (không có dòng ”Published as...” hay tên hội nghị/tạp chí ở đầu bài trong bản PDF/text arXiv v1 tải được; bài này về sau được biết là in ở ICCV 2015, nhưng thông tin đó không xuất hiện trong văn bản đang đọc nên không ghi vào đây).
Nguồn: https://arxiv.org/abs/1502.01852
Chủ đề: đề xuất Parametric ReLU (PReLU), một bản tổng quát hoá của ReLU với hệ số dốc phần âm có thể học; từ đó suy ra một cách khởi tạo trọng số (”Kaiming init”) dựa trên phân tích variance forward/backward cho mạng rất sâu dùng ReLU/PReLU, cho phép huấn luyện trực tiếp từ đầu các mô hình 20-30 tầng mà ”Xavier init” không làm được.
Vị trí trong lộ trình: đọc khi đã quen khái niệm vanishing/exploding gradient nói chung; có thể đối chiếu với opt 40 (Pascanu và đồng nghiệp, phân tích kỹ hơn vấn đề này cho RNN) và opt 46 (Bengio, khuyến nghị thực hành gradient-based training, trong đó có cả init). Đọc trước opt 38 (Batch Normalization) và opt 39 (Layer Normalization): hai kỹ thuật chuẩn hoá runtime ra đời ngay sau bài này, dần thay vai trò ”phải tính đúng std khởi tạo” bằng việc chuẩn hoá lại activation ở mọi bước, bất kể init ban đầu. Nên đọc cùng ml 08 (VGG, kiến trúc nền mà paper này sửa lại) và trước ml 07 (ResNet, dùng trực tiếp PReLU-net/Kaiming init rồi giải quyết triệt để hơn vấn đề độ sâu bằng residual connection).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Paper mở đầu bằng việc liệt kê hai hướng tiến bộ của CNN tới 2015: (a) xây mô hình mạnh hơn (tăng độ sâu, mở rộng độ rộng, giảm stride, hoạt động hoá mới, thiết kế tầng phức tạp hơn), (b) chống overfitting tốt hơn (regularization, data augmentation mạnh, dữ liệu quy mô lớn). Trong số các tiến bộ về hoạt động hoá, **rectifier neuron** (ReLU và các biến thể) được paper gọi là ”một trong vài chìa khoá” cho thành công gần đây của mạng sâu: nó giúp tăng tốc hội tụ và dẫn tới nghiệm tốt hơn so với các đơn vị dạng sigmoid truyền thống. Nhưng tác giả nhận xét: mặc dù rectifier network rất phổ biến, các cải tiến kiến trúc gần đây và các hướng dẫn lý thuyết cho huấn luyện chúng ”hiếm khi tập trung vào đặc tính của chính rectifier”.

Hai câu hỏi cụ thể được đặt ra:

1. ReLU (và các biến thể) hiện tại là **không có tham số** (parameter-free: $f(y) = \max(0, y)$). Có thể tổng quát hoá nó thành một hoạt động hoá có tham số học được, cải thiện độ chính xác mà gần như không tốn thêm compute, không tăng rủi ro overfitting không?
2. Khi mạng ReLU/PReLU trở nên **rất sâu** (paper thử tới 30 tầng trọng số), cách khởi tạo trọng số ngẫu nhiên tiêu chuẩn thời đó (Gaussian cố định, ví dụ std $= 0.01$ theo AlexNet [16], hoặc ”Xavier init” của Glorot và Bengio [7]) có còn đủ tốt không? Glorot và Bengio [7] suy ra công thức khởi tạo dựa trên giả định **hoạt động hoá tuyến tính**, một giả định **không đúng** với ReLU/PReLU. Nhóm VGG [25] từng phải ”pre-train” một mạng 8 tầng conv rồi dùng nó để khởi tạo mạng sâu hơn, vì khởi tạo trực tiếp với std cố định không hội tụ cho mạng rất sâu; cách này tốn thêm thời gian huấn luyện và có thể dẫn tới nghiệm cục bộ kém hơn.

Bài này giải quyết cả hai câu hỏi trong một khung chung: nhìn kỹ vào **đặc tính phi tuyến cụ thể của rectifier** (phần âm bị cắt hoặc bị nhân một hệ số nhỏ) thay vì coi hoạt động hoá là ”hộp đen tuyến tính” như [7] làm.

2. Ý tưởng chính

Hai đóng góp, cả hai đều xuất phát từ việc ”lấy đúng rectifier” làm trung tâm phân tích:

- **PReLU (Parametric ReLU).** Thay hệ số 0 ở phần âm của ReLU bằng một hệ số a_i **học được** (có thể khác nhau theo channel, hoặc chia sẻ theo layer). Khi $a_i = 0$, PReLU chính là ReLU; PReLU chỉ thêm đúng một tham số cho mỗi channel (hoặc mỗi layer, ở biến thể channel-shared), nên chi phí tính toán thêm và nguy cơ overfit gần như bằng không. Khác với Leaky ReLU [20] (hệ số âm cố định nhỏ, ví dụ 0.01, chỉ để tránh gradient bằng 0), PReLU học hệ số này **cùng lúc** với toàn bộ mạng bằng backpropagation thông thường.
- **Kaiming init.** Thay vì giả định hoạt động hoá tuyến tính như Glorot và Bengio [7] (dẫn tới công thức Xavier $n_l \operatorname{Var}[w_l] = 1$), paper tính lại variance của tín hiệu lan truyền thuận và gradient lan truyền ngược **có tính đến việc ReLU cắt bỏ đúng một nửa tín hiệu** (vì y đối xứng quanh 0, $P(y > 0) = P(y \le 0) = 1/2$). Kết quả là một hệ số 2 khác với Xavier: $\frac{1}{2} n_l \operatorname{Var}[w_l] = 1$, tức std $= \sqrt{2/n_l}$ thay vì $\sqrt{1/n_l}$. Hệ số 2 này tưởng nhỏ nhưng khi luỹ tích qua hàng chục tầng (β^L với $\beta = \sqrt{2}$ hoặc $1/\sqrt{2}$ tuỳ chiều) lại tạo ra sự khác biệt sống còn giữa ”hội tụ” và ”hoàn toàn không học được gì” (xem ví dụ tự tính ở mục 3.2).

Với cách khởi tạo mới này, paper huấn luyện trực tiếp từ đầu (không cần pre-train mô hình nông rồi mở rộng như VGG) các mô hình sâu tới 22 tầng (model B/C) dùng rộng rãi trong thực nghiệm chính, và thử nghiệm riêng với một mô hình 30 tầng để kiểm chứng khả năng hội tụ. Kết hợp PReLU, khởi tạo mới, kiến trúc dựa trên VGG-19 có sửa đổi (dùng spatial pyramid pooling, SPP), và augmentation/testing nhiều scale, mô hình đơn (model C + PReLU) đạt 5.71% top-5 error trên validation; tổ hợp 6 mô hình đạt 4.94% top-5 error trên test set ImageNet 2012, vượt GoogLeNet (6.66%, vô địch ILSVRC 2014) khoảng 26% tương đối, và lần đầu tiên **vượt mức lỗi con người được báo cáo** (5.1%, theo Russakovsky và đồng nghiệp [22]).

3. Phương pháp chi tiết

3.1 PReLU: định nghĩa, gradient, và quy tắc tối ưu

Định nghĩa (công thức 1), áp dụng cho từng channel i với input y_i của hoạt động hoá:

$$f(y_i) = \begin{cases} y_i, & y_i > 0 \\ a_i y_i, & y_i \le 0 \end{cases} \tag{1}$$

tương đương $f(y_i) = \max(0, y_i) + a_i \min(0, y_i)$. $a_i = 0$: ReLU; a_i cố định nhỏ (paper ví dụ $a_i = 0.01$): Leaky ReLU [20]; a_i học được: PReLU. Paper không ràng buộc dấu/khoảng của a_i nên hoạt động hoá **có thể không đơn điệu** (non-monotonic) nếu $a_i < 0$ xuất hiện sau huấn luyện, dù thực nghiệm của họ hiếm khi thấy $|a_i| > 1$.

Vì f tuyến tính từng đoạn theo a_i, gradient suy trực tiếp từ chain rule. Gradient của a_i cho một layer (công thức 2-3), lấy tổng trên mọi vị trí y_i của feature map thuộc channel đó:

$$\frac{\partial E}{\partial a_i} = \sum_{y_i} \frac{\partial E}{\partial f(y_i)} \frac{\partial f(y_i)}{\partial a_i}, \qquad \frac{\partial f(y_i)}{\partial a_i} = \begin{cases} 0, & y_i > 0 \\ y_i, & y_i \le 0 \end{cases} \tag{2, 3}$$

Biến thể **channel-shared**: một hệ số a dùng chung cho mọi channel của cả layer, $f(y_i) = \max(0, y_i) + a \min(0, y_i)$; gradient của a cộng dồn trên cả y_i và trên mọi channel của layer. Đây chỉ thêm **một** tham số mỗi layer. Cập nhật a_i dùng momentum (công thức 4):

$$\Delta a_i := \mu \, \Delta a_i + (\text{tốc độ học}) \cdot \frac{\partial E}{\partial a_i} \tag{4}$$

(ký hiệu tốc độ học trong bản PDF gốc bị mất khi trích xuất văn bản, chỗ đó chỉ còn một khoảng trắng; nội dung công thức vẫn rõ: cập nhật kiểu momentum chuẩn, μ là hệ số momentum). Điểm quan trọng paper nhấn mạnh: **không dùng weight decay (L2) khi cập nhật** a_i, vì weight decay sẽ kéo a_i về 0, tức luôn "ưu tiên" ReLU, làm lệch kết quả học. Dù không có regularization, hệ số học được "hiếm khi" có độ lớn vượt 1 trong thực nghiệm của họ. Giá trị khởi tạo $a_i = 0.25$ được dùng xuyên suốt bài.

Số tham số thêm vào bằng đúng tổng số channel (biến thể channel-wise) hoặc số layer (biến thể channel-shared) nên "không đáng kể" so với tổng số trọng số, và tác giả kỳ vọng không tăng rủi ro overfit.

3.2 Phân tích variance lúc lan truyền thuận (forward)

Phần lõi của đóng góp thứ hai. Với một conv layer, response tại một pixel (công thức 5):

$$y_l = W_l x_l + b_l \tag{5}$$

x_l là vector k^2c-chiều (các pixel đồng vị trí trên c channel input, k là kích thước filter không gian), W_l là ma trận d-by-n ($n = k^2c$, d số filter, mỗi dòng là trọng số một filter), b_l là bias, y_l là response output. Quy ước $c_l = d_{l-1}$ (số channel input của layer l bằng số filter của layer $l-1$).

Giả định (giống [7]): các phần tử của W_l độc lập và cùng phân phối; các phần tử của x_l độc lập và cùng phân phối; x_l và W_l độc lập với nhau. Khi đó (công thức 6-7, dùng w_l, x_l, y_l là biến ngẫu nhiên đại diện cho một phần tử):

$$\mathrm{Var}[y_l] = n_l \,\mathrm{Var}[w_l x_l] = n_l \,\mathrm{Var}[w_l]\,\mathbb{E}[x_l^2] \tag{6, 7}$$

Chú ý $\mathbb{E}[x_l^2] \neq \mathrm{Var}[x_l]$ trừ khi x_l có kỳ vọng 0; đây chính là điểm khác với [7]. Với ReLU, $x_l = \max(0, y_{l-1})$ **không** có kỳ vọng 0 (chỉ giữ phần dương), nên không thể áp dụng trực tiếp giả định của Xavier. Nếu w_{l-1} có phân phối đối xứng quanh 0 và $b_{l-1} = 0$, thì y_{l-1} cũng đối xứng quanh 0 và có kỳ vọng 0; với một biến đối xứng quanh 0, một nửa khối lượng xác suất ở phần dương, nên $\mathbb{E}[x_l^2] = \frac{1}{2}\mathrm{Var}[y_{l-1}]$ khi f là ReLU. Thay vào (7):

$$\mathrm{Var}[y_l] = \frac{1}{2}\, n_l \,\mathrm{Var}[w_l]\,\mathrm{Var}[y_{l-1}] \tag{8}$$

Ghép L layer (công thức 9):

$$\mathrm{Var}[y_L] = \mathrm{Var}[y_1] \left(\prod_{l=2}^{L} \frac{1}{2}\, n_l \,\mathrm{Var}[w_l] \right) \tag{9}$$

Tích này là chìa khoá của toàn bộ thiết kế init: một khởi tạo tốt phải tránh cho tích đó khuếch đại hoặc triệt tiêu tín hiệu theo cấp số nhân. Điều kiện đủ (công thức 10):

$$\frac{1}{2}\, n_l \,\mathrm{Var}[w_l] = 1, \quad \forall l \tag{10}$$

tức W_l lấy từ phân phối Gaussian trung bình 0, độ lệch chuẩn $\boxed{\sqrt{2/n_l}}$, và bias $b = 0$. Với layer đầu tiên ($l = 1$), lẽ ra cần $n_1 \mathrm{Var}[w_1] = 1$ (không có ReLU trước input), nhưng vì hệ số $1/2$ chỉ tồn tại ở đúng một layer nên không ảnh hưởng tích luỹ (tương tự lý do [7] chấp nhận bỏ qua layer biên); để đơn giản, paper cũng dùng công thức (10) luôn cho layer đầu.

Ví dụ tự tính, minh hoạ trực tiếp tại sao hệ số $\frac{1}{2}$ lại quan trọng khi L lớn. Xét conv layer với $k = 3$, $c = 64$, nên $n = k^2c = 9 \times 64 = 576$. Giả sử $\text{Var}[y_0] = 1$ (chuẩn hoá input).

- Dùng **Xavier/Glorot**: $\text{Var}[w] = 1/n = 1/576$. Qua một layer ReLU: $\text{Var}[y_1] = \frac{1}{2} \cdot 576 \cdot \frac{1}{576} \cdot 1 = \frac{1}{2} \cdot 1 = 0.5$ (tự tính). Vậy mỗi layer ReLU dùng std Xavier làm variance **giảm đúng một nửa**. Qua $L = 10$ layer: $\text{Var}[y_{10}] = 0.5^{10} \approx 0.000977$ (tự tính), còn lại chưa tới 0.1% variance ban đầu; qua $L = 30$ layer: $0.5^{30} \approx 9.3 \times 10^{-10}$ (tự tính), tín hiệu coi như đã về 0 bằng số (và gradient ngược cũng tắt tương tự vì cùng hệ số $\frac{1}{2}$ xuất hiện ở công thức (12) bên dưới). Đây chính là cơ chế toán học phía sau hiện tượng "Xavier hoàn toàn stall" mà paper quan sát ở mô hình 30 tầng (Hình 3).
- Dùng **Kaiming init**: $\text{Var}[w] = 2/n = 2/576$. Qua một layer: $\text{Var}[y_1] = \frac{1}{2} \cdot 576 \cdot \frac{2}{576} \cdot 1 = 1 \cdot 1 = 1$ (tự tính) đúng bằng (10) đặt ra: tích số $\frac{1}{2} n \,\text{Var}[w] = 1$ nên variance được giữ nguyên qua mọi layer, dù L bao nhiêu.

Đây là đúng bản chất lập luận "Discussions" của paper: nếu mỗi layer làm tín hiệu bị nhân thêm một hệ số β cố định, sau L layer tín hiệu bị nhân β^L; $\beta > 1$ dẫn tới tràn số (diverge), $\beta < 1$ dẫn tới tín hiệu/gradient biến mất (stall); chỉ $\beta = 1$ (đúng điều kiện (10)/(14)) giữ được biên độ ổn định ở mọi độ sâu.

3.3 Phân tích variance lúc lan truyền ngược (backward) và điều kiện (14)

Gradient của một conv layer khi lan truyền ngược (công thức 11), với $\Delta x = \partial E/\partial x$, $\Delta y = \partial E/\partial y$:

$$\Delta x_l = \hat{W}_l \, \Delta y_l \tag{11}$$

Δy_l biểu diễn $k \times k$ pixel trên d channel, được reshape thành vector $\hat{n} = k^2 d$ chiều (chú ý $\hat{n} \neq n = k^2 c$: $\hat{n}_l$ dùng số filter d_l của **chính layer** l, còn n_l dùng số channel input $c_l = d_{l-1}$ của layer l, tức số filter của layer **trước**). $\hat{W}$ là ma trận c-by-$\hat{n}$ xếp lại từ W theo chiều lan truyền ngược (có thể reshape qua lại giữa W và $\hat{W}$). Giả định w_l và Δy_l độc lập; nếu w_l lấy từ phân phối đối xứng quanh 0 thì Δx_l có kỳ vọng 0 với mọi l.

Với ReLU, $f'(y_l)$ chỉ nhận giá trị 0 hoặc 1 với xác suất bằng nhau (giả định $f'(y_l)$ và Δx_{l+1} độc lập), nên $\mathbb{E}[\Delta y_l] = \mathbb{E}[\Delta x_{l+1}]/2 = 0$ và $\mathbb{E}[(\Delta y_l)^2] = \text{Var}[\Delta y_l] = \frac{1}{2}\text{Var}[\Delta x_{l+1}]$. Từ đó (công thức 12):

$$\text{Var}[\Delta x_l] = \hat{n}_l \,\text{Var}[w_l] \,\text{Var}[\Delta y_l] = \frac{1}{2} \hat{n}_l \,\text{Var}[w_l] \,\text{Var}[\Delta x_{l+1}] \tag{12}$$

Cùng dạng hệ số $\frac{1}{2}$ như forward, nhưng paper lưu ý: "lý do xuất hiện hệ số $\frac{1}{2}$ ở (12) khác với ở (8)" (một bên do f' nhị phân 0/1 đều xác suất, bên kia do x chỉ giữ phần dương của một biến đối xứng). Ghép L layer (công thức 13):

$$\text{Var}[\Delta x_2] = \text{Var}[\Delta x_{L+1}] \left(\prod_{l=2}^{L} \frac{1}{2} \hat{n}_l \,\text{Var}[w_l] \right) \tag{13}$$

Điều kiện đủ để gradient không bị khuếch đại/triệt tiêu theo cấp số nhân (công thức 14):

$$\frac{1}{2} \hat{n}_l \,\text{Var}[w_l] = 1, \quad \forall l \quad \Rightarrow \quad \text{std}(w_l) = \sqrt{2/\hat{n}_l} \tag{14}$$

Paper chỉ ra: **chỉ cần dùng một trong hai**, (10) hoặc (14), không cần cả hai cùng lúc. Nếu dùng riêng (14) cho mọi layer, tích trong (9) (dùng n_l thay $\hat{n}_l$) trở thành $\prod n_l/\hat{n}_l = c_2/d_L$, một số **không** tiêu biến trong các kiến trúc thông thường (vì c_2, số channel ảnh input ở layer 2, và d_L, số filter layer cuối, đều là hằng số hữu hạn không phụ thuộc L); tức nếu khởi tạo đúng cho chiều ngược thì chiều thuận cũng tự động ổn, và ngược lại. Với layer đầu ($l = 1$), không cần tính Δx_1 (vì đó là domain ảnh input), nhưng cùng lý do như forward

(một layer không làm tích luỹ theo hàm luỹ thừa), vẫn áp dụng (14) cho layer đầu để đơn giản. Tác giả khẳng định cả hai cách (chỉ dùng (10), hoặc chỉ dùng (14)) đều giúp mọi mô hình trong bài hội tụ.

3.4 Mở rộng cho PReLU (công thức 15) và so sánh trực tiếp với Xavier

Tổng quát hoá (10) cho PReLU với hệ số dốc âm khởi tạo a (công thức 15):

$$\frac{1}{2}(1+a^2)\, n_l \,\mathrm{Var}[w_l] = 1, \quad \forall l \tag{15}$$

(và tương tự $\frac{1}{2}(1+a^2)\hat{n}_l \mathrm{Var}[w_l] = 1$ cho chiều ngược). Khi $a = 0$: trở lại trường hợp ReLU (10)/(14). Khi $a = 1$ (tuyến tính hoàn toàn, không cắt gì): trở lại đúng công thức Xavier/Glorot $n_l \mathrm{Var}[w_l] = 1$. Đây là cách paper **đặt công thức của mình và của [7] vào cùng một họ**, khác biệt duy nhất là hệ số $(1+a^2)/2$ thay cho 1: khi hoạt động hoá cắt bỏ nhiều hơn (càng gần ReLU, a nhỏ), hệ số nhân với $n_l \mathrm{Var}[w_l]$ càng nhỏ, nên cần **tăng** $\mathrm{Var}[w_l]$ để bù lại phần tín hiệu bị cắt mất. *Lưu ý người ghi chú:* paper trình bày (15) như một hệ quả toán học tức thời của phần dẫn ReLU, nhưng **không nói rõ ràng** là các thực nghiệm PReLU-net trong bài (model A/B/C + PReLU, mục 4) có thực sự dùng (15) với $a = 0.25$ để tính std khởi tạo, hay vẫn dùng công thức ReLU (10)/(14) ($a = 0$) rồi để a_i tự học dần lên 0.25 và xa hơn trong quá trình train. Văn bản không có câu khẳng định minh bạch về điểm này; người đọc nên coi (15) là phần mở rộng lý thuyết, còn chi tiết cài đặt thực tế của riêng PReLU-net là một khoảng chưa chắc.

So sánh trực tiếp với Xavier (mục "Comparisons with Xavier Initialization"): công thức [7] dùng $n_l \mathrm{Var}[w_l] = 1$ cho trường hợp forward, tương đương std $= \sqrt{1/n_l}$; với L layer, std hiệu dụng của Xavier chỉ bằng $1/\sqrt{2}^{\,L}$ lần std theo cách của paper này (vì thiếu hệ số $\sqrt{2}$ mỗi layer). Paper thử nghiệm: trên mô hình 14 tầng (Table 2, dùng ReLU), Xavier cho 33.90/13.44 top-1/top-5, còn Kaiming init cho 33.82/13.34; "chưa quan sát được ưu thế rõ rệt của cách nào về độ chính xác" ở độ sâu này. Khác biệt chỉ lộ rõ ở độ sâu cực lớn: mô hình 22 tầng (Hình 2) cả hai đều hội tụ nhưng Kaiming init giảm lỗi sớm hơn; mô hình 30 tầng (Hình 3, 27 conv + 3 fc) Xavier **hoàn toàn đứng im** (tác giả xác nhận đã kiểm tra gradient và thấy chúng tiêu biến dần), trong khi Kaiming init vẫn hội tụ.

Một ví dụ số cụ thể trong chính paper, liên quan tới "model B" của VGG [25] (10 conv layer 3×3, số filter d_l: 64 cho 2 layer đầu, 128 cho 2 layer kế, 256 cho 2 layer kế, 512 cho 4 layer còn lại). Std theo (14) ($\sqrt{2/\hat{n}_l}$, với $\hat{n}_l = k^2 d_l = 9 d_l$) là $0.059,\ 0.042,\ 0.029,\ 0.021$ tương ứng với $d_l = 64, 128, 256, 512$ (tự tính kiểm lại: $\sqrt{2/(9\times 64)} = \sqrt{2/576} \approx 0.0589$; $\sqrt{2/(9\times 128)} = \sqrt{2/1152} \approx 0.0417$; $\sqrt{2/(9\times 256)} \approx 0.0295$; $\sqrt{2/(9\times 512)} \approx 0.0209$, khớp các số paper nêu khi làm tròn). Nếu khởi tạo bằng std cố định 0.01 (kiểu AlexNet) thay vì các giá trị trên, tỉ lệ std-cần-có trên std-thực-dùng là $5.9,\ 4.2,\ 2.9,\ 2.1$; paper tính std của gradient lan truyền từ conv10 về conv2 chỉ còn $1/(5.9 \times 4.2^2 \times 2.9^2 \times 2.1^4) = 1/(1.7\times 10^4)$ so với cách tính của họ (tự tính kiểm lại: $5.9 \times 4.2^2 \times 2.9^2 \times 2.1^4 = 5.9 \times 17.64 \times 8.41 \times 19.45 \approx 1.70 \times 10^4$, khớp số paper đưa ra). Đây được paper dùng để **giải thích tại sao** std cố định 0.01 khiến một số mạng rất sâu bị "diminishing gradient" trong thực nghiệm của nhóm VGG.

Một chi tiết thực hành khác được nêu: khi input ảnh không chuẩn hoá (ví dụ nằm trong khoảng $[-128, 128]$), biên độ tín hiệu lan truyền thuận có thể lớn tới mức softmax bị overflow; paper xử lý bằng cách **giảm std thủ công** ở layer đầu và hai lớp fc đầu/cuối (dùng std 0.01 cho hai fc đầu, 0.001 cho fc cuối, nhỏ hơn giá trị lý thuyết $\sqrt{2/4096} \approx 0.022$), coi như rải một hệ số $\beta^{1/L}$ nhỏ lên một vài layer để bù lại việc input không chuẩn hoá, mà không ảnh hưởng tới tích luỹ exponent tổng thể (do chỉ áp dụng ở một số ít layer).

3.5 Kiến trúc mô hình và chi tiết huấn luyện/kiểm thử

Mô hình nhỏ dùng để so sánh ReLU/PReLU (Table 1, 2): mạng 14 tầng trọng số ("model E" trong [10], He và Sun 2014), gồm conv1 (7×7, 64, stride 2) → pool1 → 4 conv (2×2, 128) → pool2 → 6 conv (2×2, 256) → SPP {6,3,2,1} → fc1 (4096) → fc2 (4096) → fc3 (1000). PReLU (khi dùng) áp cho tất cả conv layer và 2 fc đầu, tức đúng 13 layer có hoạt động hoá; đây giải thích tại sao biến thể channel-shared chỉ thêm đúng **13**

tham số (mỗi layer một hệ số) so với baseline ReLU, một con số khớp hoàn toàn với số layer có hoạt động hoá (tự tính kiểm chéo: $1 + 4 + 6 + 2 = 13$).

Mô hình lớn (Table 3): baseline là mô hình A, 19 tầng, sửa từ VGG-19 theo ba điểm: (i) layer đầu dùng filter 7×7 stride 2 (thay vì hai layer 3×3); (ii) ba conv layer còn lại ở hai feature map lớn nhất (224, 112) được ”dời” xuống các feature map nhỏ hơn (56, 28, 14), giữ tổng độ sâu và độ phức tạp tính toán gần như không đổi (vì các layer sâu hơn có nhiều filter hơn làm compute tương đương); (iii) dùng SPP (4 mức: 7×7, 3×3, 2×2, 1×1, tổng 63 bin) trước fc đầu tiên thay cho max-pool cuối. Mô hình B thêm 3 conv layer nữa (tổng 22 tầng). Mô hình C giữ cấu trúc như B nhưng tăng số filter ở các feature map nhỏ (384/768/896 so với 256/512/512 của B), tăng độ phức tạp lên khoảng 2.3 lần.

Mô hình	Độ sâu (conv+fc)	Complexity ($\times 10^{10}$ ops)
VGG-19 [25]	19	1.96
A	19	1.90
B	22	2.32
C	22	5.30

Tác giả nêu rõ **không có bằng chứng** model A tốt hơn VGG-19 về mặt kiến trúc (chỉ là kết quả số của A tốt hơn số VGG-19 *đã công bố* trong [25]); mục đích chính của A là tốc độ (2.6s/minibatch 128 so với 3.0s của VGG-19 tái lập, trên 4 GPU K20). Huấn luyện A/B trên 4 GPU K20, hoặc C trên 8 GPU K40, mất khoảng 3-4 tuần.

Về việc không tăng độ sâu hơn nữa: paper thử thêm 3-9 conv layer lên model B, theo dõi 20 epoch đầu và thấy cả train/test error đều xấu đi so với B gốc (nhưng không chạy tới hết do giới hạn thời gian, nên ”chưa có bằng chứng chắc chắn” rằng các mô hình lớn và quá sâu này cuối cùng sẽ kém hơn). Vì khả năng suy giảm này, họ chọn **tăng độ rộng (width)** thay vì độ sâu cho model C, dựa trên quan sát từ các nghiên cứu khác rằng độ chính xác nên cải thiện theo số tham số conv (phụ thuộc cả độ sâu và độ rộng).

Riêng với một mô hình 30 tầng (27 conv + 3 fc, cộng thêm 16 conv layer 256-filter 2×2 vào model trong Table 1) được dựng chỉ để kiểm tra khả năng hội tụ (Hình 3): paper xác nhận nó hội tụ được bằng khởi tạo của họ, nhưng kết quả cuối (38.56/16.59 top-1/top-5) **rõ ràng kém hơn** chính mô hình 14 tầng nhỏ hơn (33.82/13.34). Tác giả suy đoán nguyên nhân có thể do cách tăng độ sâu ”không phù hợp” hoặc bài toán nhận dạng ”chưa đủ phức tạp” để cần tới độ sâu đó, và nhấn mạnh: dù thí nghiệm riêng này chưa cho thấy lợi ích của cực-sâu, phương pháp khởi tạo ”đặt nền cho nghiên cứu sâu hơn về việc tăng độ sâu” trong tương lai.

Huấn luyện: crop ngẫu nhiên 224×224 từ ảnh đã resize cạnh ngắn về s (jitter ngẫu nhiên trong $[256, 512]$, áp dụng **từ đầu** quá trình train, khác với [25] chỉ jitter khi fine-tune); trừ mean theo pixel; lật ngang một nửa mẫu; color augmentation ngẫu nhiên kiểu AlexNet. Khác với VGG (khởi tạo mô hình sâu bằng một mô hình nông đã train trước), paper huấn luyện trực tiếp mô hình sâu bằng khởi tạo của họ (dùng công thức (14)), cho rằng cách train end-to-end này giúp tránh nghiệm cục bộ kém hơn. Weight decay 0.0005, momentum 0.9, dropout 50% ở hai fc đầu, batch size cố định 128, learning rate $10^{-2} \to 10^{-3} \to 10^{-4}$ (đổi khi error chững lại), khoảng 80 epoch mỗi mô hình. Không tăng batch size vượt 128 vì (theo [15]) độ chính xác có thể giảm.

Kiểm thử: multi-view trên feature map (kiểu SPP-net [11]) kết hợp dense sliding window (kiểu Overfeat/VGG [24, 25]): áp conv lên toàn ảnh đã resize để lấy feature map cuối, pool mỗi cửa sổ 14×14 bằng SPP layer, áp fc lên các vùng đã pool để tính điểm; làm tương tự trên ảnh lật ngang; trung bình điểm của mọi sliding window; kết hợp thêm nhiều scale.

Đa GPU: biến thể của phương pháp Krizhevsky [15]: ”data parallelism” cho các conv layer, đồng bộ GPU trước fc đầu tiên, fc chạy trên **một** GPU duy nhất (không song song hoá fc vì chi phí thấp). Cài trên bản Caffe sửa đổi. Speedup đo được: 3.8× với 4 GPU, 6.0× với 8 GPU.

4. Thí nghiệm và kết quả chính

Dữ liệu: ImageNet 2012, 1000 lớp, khoảng 1.2 triệu ảnh train, 50.000 ảnh validation, 100.000 ảnh test (nhãn test không công bố). Chỉ dùng dữ liệu được cung cấp để train. Mọi kết quả đánh giá trên validation set, trừ kết quả cuối cùng (Table 7) đánh giá trên test set qua server ILSVRC. Metric chính thức để xếp hạng là top-5 error.

So sánh nhỏ, ReLU vs PReLU (Table 2, mô hình 14 tầng, 10-view test, resize cạnh ngắn 256, view 224×224, 75 epoch mỗi mô hình):

Hoạt động hoá	top-1	top-5
ReLU	33.82	13.34
PReLU, channel-shared	32.71	12.87
PReLU, channel-wise	32.64	12.75

PReLU channel-wise giảm top-1 1.2 điểm % so với baseline; channel-shared (chỉ thêm 13 tham số) giảm 1.1 điểm %, cho thấy ”một số lượng tham số rất nhỏ vẫn đóng vai trò quan trọng”. Hai biến thể PReLU gần tương đương nhau. Bảng hệ số học được của PReLU (Table 1 gốc) cho thấy hai hiện tượng: (i) conv1 có hệ số lớn hẳn (channel-shared 0.681, channel-wise 0.596) vì filter đầu chủ yếu là bộ lọc kiểu Gabor (biên, texture) nên cả phần đáp ứng dương và âm đều mang thông tin; (ii) ở các conv sâu hơn, hệ số **trung bình nhỏ dần** (tự tính, gộp trung bình các layer paper cho theo nhóm: conv21-24 channel-shared trung bình ≈ 0.248, conv31-36 channel-shared trung bình ≈ 0.103), tức hoạt động hoá ”ngày càng giống ReLU/phi tuyến hơn” ở các tầng sâu, gợi ý mô hình giữ nhiều thông tin ở tầng đầu và trở nên phân biệt/chọn lọc hơn ở tầng sâu.

So sánh trên mô hình lớn A, dense testing nhiều scale (Table 4):

Scale s	ReLU top-1/top-5	PReLU top-1/top-5
256	26.25 / 8.25	25.81 / 8.08
384	24.77 / 7.26	24.20 / 7.03
480	25.46 / 7.63	24.83 / 7.39
multi-scale	24.02 / 6.51	22.97 / 6.28

Scale đơn tốt nhất là 384 (gần giữa khoảng jitter $[256, 512]$). Multi-scale: PReLU giảm top-1 1.05 điểm %, top-5 0.23 điểm % so với ReLU, ”với chi phí tính toán thêm gần như bằng không”, nhất quán với Table 2.

Kết quả single-model (Table 5, 10-view; Table 6, multi-scale + multi-view/dense, validation set):

Mô hình	top-1	top-5
VGG-16 [25] (tự test)	28.07†	9.33†
GoogLeNet [29]	-	9.15
A, ReLU	26.48 (10-view) / 24.02 (multi-scale+dense)	8.59 / 6.51
A, PReLU	25.59 / 22.97	8.23 / 6.28
B, PReLU	25.53 / 22.85	8.13 / 6.27
C, PReLU	24.27 / 21.59	7.38 / 5.71

(† = đánh giá từ test set theo paper VGG, còn lại là validation set). Mô hình đơn tốt nhất (C, PReLU) đạt 5.71% top-5, ”tốt hơn tất cả kết quả multi-model công bố trước đó” tại thời điểm đó. So A+PReLU với B+PReLU: 19 tầng và 22 tầng cho kết quả tương đương (tăng độ sâu từ 19 lên 22 không giúp nhiều ở quy mô này); so B

với C: tăng độ rộng vẫn cải thiện rõ, gợi ý rằng khi mô hình đã đủ sâu, **độ rộng** trở thành yếu tố quan trọng hơn.

Kết quả multi-model trên test set (Table 7, chính thức qua server ILSVRC):

Nhóm	top-5 (test)
GoogLeNet [29] (vô địch ILSVRC14)	6.66
VGG [25] (arXiv v5)	6.8
Baidu [32]	5.98
MSRA, PReLU-nets (6 mô hình)	**4.94**

Kết quả 4.94% là cải thiện tuyệt đối 1.7 điểm % so với GoogLeNet (6.66%), tương đương cải thiện tương đối $\approx 26\%$ (tự tính: $1.72/6.66 \approx 0.258$, khớp "26%" paper nêu), và cải thiện tương đối $\approx 17\%$ so với Baidu 5.98% (tự tính: $1.04/5.98 \approx 0.174$). Tổ hợp gồm 6 mô hình nhưng chỉ có **một** mô hình kiến trúc C (các mô hình khác kém hơn đáng kể); tác giả suy đoán có thể đạt kết quả tốt hơn nếu dùng ít mô hình hơn nhưng mỗi mô hình mạnh hơn.

So với con người: Russakovsky và đồng nghiệp [22] báo cáo lỗi top-5 của con người là 5.1% (một người được huấn luyện kỹ trên ảnh validation, có giao diện đặc biệt hiển thị 13 ảnh mẫu mỗi lớp, ước lượng trên 1500 ảnh test ngẫu nhiên). Kết quả 4.94% của paper vượt mức này; theo hiểu biết của tác giả, đây là lần đầu tiên công bố vượt lỗi con người trên benchmark này. [22] chỉ ra hai loại lỗi chính của con người: nhận diện fine-grained (ví dụ phân biệt 120 loài chó) và không biết hết các lớp; paper gợi ý máy có thể làm tốt hơn ở phần fine-grained, nhưng **tự cảnh báo** rõ: điều này không có nghĩa máy vượt người về nhận diện vật thể nói chung, vì với các lớp vật thể "sơ cấp" đời thường (kiểu Pascal VOC), máy vẫn mắc lỗi hiển nhiên với người.

Phân tích theo lớp: 113/1000 lớp có lỗi top-5 bằng 0 trên test set; ba lớp lỗi cao nhất là "letter opener" (49%), "spotlight" (38%), "restaurant" (36%), do nhiều vật thể trong ảnh, vật thể nhỏ, hoặc biến thiên nội lớp lớn (Hình 5 cho ví dụ ảnh sai). So với kết quả trong lúc thi ILSVRC 2014 của chính nhóm (trung bình 8.06%), kết quả cuối (4.94%) giảm lỗi ở 824 lớp, không đổi ở 127 lớp, tăng ở 49 lớp (tự tính kiểm tổng: $824 + 127 + 49 = 1000$, khớp đúng 1000 lớp của ImageNet, cho thấy số liệu nhất quán).

5. Phân tích / ablation

Vì đây là paper thực nghiệm về kiến trúc/khởi tạo (không có benchmark tách riêng theo nghĩa "so nhiều optimizer"), phần "ablation" chủ yếu nằm ở các so sánh có kiểm soát đã nêu ở mục 3-4. Gom lại các điểm phân tích quan trọng:

- **PReLU vs ReLU tại cùng kiến trúc, cùng ngân sách train:** luôn thắng nhẹ (khoảng 0.2-1.2 điểm % top-1 tuỳ scale/mô hình) với chi phí thêm không đáng kể, nhất quán ở cả mô hình nhỏ (Table 2) và mô hình lớn (Table 4). Đây là bằng chứng mạnh cho claim "cải thiện gần như miễn phí", nhưng cỡ cải thiện khá nhỏ so với cải thiện từ kiến trúc (A→B→C) hay từ multi-scale/multi-view testing.
- **Channel-wise vs channel-shared PReLU:** gần tương đương (0.07 điểm % top-1 khác biệt trên mô hình nhỏ), cho thấy phần lớn lợi ích đến từ việc "có một hệ số học được" hơn là từ việc hệ số đó khác nhau theo channel.
- **Kaiming init vs Xavier, theo độ sâu:** đây là ablation quan trọng nhất về mặt lý thuyết của bài. Ở độ sâu vừa (14 tầng), hai cách **cho độ chính xác xấp xỉ nhau** (33.82/13.34 vs 33.90/13.44), chỉ khác tốc độ hội tụ ban đầu (Hình 2, 22 tầng: Kaiming giảm lỗi sớm hơn Xavier dù cả hai đều hội tụ). Ở độ sâu cực lớn (30 tầng, Hình 3), khác biệt trở thành nhị phân: Xavier **hoàn toàn không học được** (gradient tắt dần, xác nhận bằng theo dõi trực tiếp), Kaiming init vẫn hội tụ. Kết luận quan trọng: lợi ích của việc "tính đúng hệ số 2" chỉ **lộ rõ ở độ sâu rất lớn**; ở độ sâu vừa phải, sai khác hằng số nhỏ giữa hai công thức init không quyết định được convergence.

- **Hội tụ được không đồng nghĩa có lợi về accuracy:** đây là một nhận xét paper tự đưa ra và cần đọc kỹ. Mô hình 30 tầng hội tụ nhờ Kaiming init nhưng **kết quả tệ hơn** mô hình 14 tầng nhỏ. Vậy ”khả năng huấn luyện mô hình rất sâu từ đầu” (đóng góp kỹ thuật chính mục 2.2) và ”mô hình rất sâu tốt hơn mô hình nông” là hai claim khác nhau; paper chỉ chứng minh chắc chắn claim đầu, còn claim sau bị chính thí nghiệm của họ bác lại ở quy mô dữ liệu/mô hình cụ thể này.
- **Độ sâu (B) vs độ rộng (C):** tăng từ 19 (A) lên 22 tầng (B) cải thiện rất ít; tăng độ rộng ở cùng 22 tầng (B→C) cải thiện rõ rệt hơn nhiều (ví dụ top-5 single-model multi-scale: B 6.27% → C 5.71%). Trong phạm vi thí nghiệm này, độ rộng ”ăn tiền” hơn độ sâu.

6. Giới hạn

Do tác giả nêu hoặc có thể suy ra trực tiếp từ văn bản:

- Mô hình 30 tầng hội tụ được nhưng **kết quả tệ hơn** mô hình 14 tầng (38.56/16.59 so với 33.82/13.34); tác giả tự nhận ”chưa quan sát được lợi ích từ việc train mô hình cực sâu” trong thí nghiệm ImageNet của họ, dù vẫn tin phương pháp khởi tạo ”đặt nền” cho nghiên cứu sau này.
- Thử nghiệm mở rộng độ sâu hơn nữa (thêm 3-9 layer lên model B) chỉ theo dõi 20 epoch đầu (chưa chạy hết) do giới hạn thời gian; bản thân paper nói ”chưa có bằng chứng chắc chắn” các mô hình này sẽ suy giảm nếu train đủ lâu, tức kết luận ”tăng độ sâu gây suy giảm” ở quy mô lớn **chưa được kiểm chứng đầy đủ**, chỉ là quan sát sớm.
- Với độ sâu vừa phải (14 tầng, Table 2 ReLU), Kaiming init và Xavier cho độ chính xác **gần như không khác biệt**; lợi ích về accuracy của công thức mới chỉ rõ ở độ sâu cực lớn, nơi Xavier hoàn toàn không hội tụ (tức ở đó không có ”accuracy” để so sánh).
- Như đã nêu ở mục 3.4, paper **không nói rõ** liệu khởi tạo PReLU trong thực nghiệm có dùng công thức tổng quát (15) với $a = 0.25$ hay dùng công thức ReLU (10)/(14); đây là một khoảng mập mờ về chi tiết cài đặt.
- Công thức Δa_i (4) bị mất ký hiệu tốc độ học khi trích xuất văn bản PDF (chỉ còn khoảng trắng ở vị trí đó); nội dung toán học (momentum update) vẫn rõ ràng, nhưng người đọc cần biết đây là lỗi hiển thị do convert PDF, không phải lỗi gốc của paper.
- Toàn bộ phân tích variance (mục 2.2) dựa trên các giả định độc lập/cùng phân phối giữa phần tử của W và x, và giả định đối xứng quanh 0 của w, y_{l-1}; đây là xấp xỉ hợp lý lúc khởi tạo (trước khi học), **không** còn đúng chính xác khi trọng số đã được cập nhật nhiều bước (phân phối trọng số lúc đó không còn đối xứng/độc lập theo giả định ban đầu). Paper không bàn tới hành vi variance *trong quá trình* huấn luyện, chỉ bàn tại thời điểm khởi tạo.
- So sánh với VGG-19 dùng kết quả VGG-19 **đã công bố** trong [25] cho model A, chứ chưa hẳn cùng điều kiện augmentation/testing đầy đủ; tác giả tự nói ”không có bằng chứng model A là kiến trúc tốt hơn VGG-19”, và với augmentation ít hơn, A và VGG-19 (tái lập, cùng SPP và cùng init của paper này) cho kết quả tương đương.
- Cả nghiên cứu chỉ trên ảnh tự nhiên ImageNet; không kiểm tra trên domain khác (văn bản, âm thanh, hay ảnh scan/OCR).

Người ghi chú nhận xét thêm:

- Giả định ReLU (xác suất $y > 0$ đúng bằng $1/2$) chỉ chính xác khi y đối xứng quanh 0 với kỳ vọng 0; một khi mạng dùng bias khác 0 hoặc bị lệch do dữ liệu (ví dụ ảnh không chuẩn hoá tốt), điều kiện này chỉ còn xấp xỉ, nhưng paper không định lượng mức sai lệch khi giả định này bị vi phạm nhẹ.
- Lợi ích thực đo được của PReLU (0.2 đến 1.2 điểm % top-1) khá nhỏ nếu so với lợi ích từ multi-scale testing (nhiều điểm %) hay từ tăng độ rộng mô hình (B→C, hơn 0.5 điểm % top-5); bài báo không tách bạch rõ ”đóng góp của khởi tạo” so với ”đóng góp của kiến trúc/augmentation” trong kết quả 4.94% cuối cùng, vì các yếu tố được cộng dồn cùng lúc.

7. Liên hệ với các bài khác

- **ml 08 (VGG, Simonyan và Zisserman 2014):** là baseline kiến trúc trực tiếp; model A/B/C của bài này là bản sửa của VGG-19 (SPP, filter 7×7/2 ở layer đầu, dời layer). Paper này giải quyết đúng vấn đề mà VGG gặp phải khi train sâu (phải pre-train mô hình nông rồi mở rộng) bằng một cách khởi tạo tốt hơn.
- **ml 07 (ResNet, He và đồng nghiệp 2015):** cùng nhóm tác giả (Kaiming He), ra đời ngay sau bài này, dùng trực tiếp PReLU-net/Kaiming init làm điểm xuất phát rồi **giải quyết triệt để hơn** vấn đề huấn luyện mạng rất sâu bằng residual connection (skip connection), thay vì chỉ dựa vào việc tính đúng variance khởi tạo. Có thể coi bài này là bước "khởi tạo đúng để mở cửa cho độ sâu", còn ResNet là bước "sửa kiến trúc để độ sâu thực sự có lợi" (điều mà chính bài này chưa chứng minh được ở mô hình 30 tầng).
- **opt 38 (Batch Normalization, Ioffe và Szegedy):** giải quyết cùng vấn đề (ổn định phân phối activation/gradient qua các tầng sâu) nhưng bằng chuẩn hoá **tại runtime** (theo batch) ở mọi bước forward, thay vì chỉ chuẩn hoá đúng **một lần** lúc khởi tạo như bài này; sau khi có BatchNorm, độ nhạy của mạng với lựa chọn std khởi tạo giảm đi nhiều (dù không biến mất hoàn toàn).
- **opt 39 (Layer Normalization, Ba và đồng nghiệp):** một biến thể chuẩn hoá khác (theo feature thay vì theo batch), cùng mục tiêu ổn định huấn luyện sâu, phổ biến hơn BatchNorm trong kiến trúc Transformer hiện đại.
- **opt 40 (Pascanu, Mikolov, Bengio, "On the difficulty of training Recurrent Neural Networks"):** phân tích vanishing/exploding gradient cho RNN bằng công cụ khác (chuẩn của Jacobian qua thời gian, gradient clipping) nhưng cùng bản chất toán học: tích luỹ một hệ số nhân qua nhiều "tầng" (ở đây là nhiều timestep) có thể làm gradient nổ hoặc tắt theo cấp số nhân, giống hệt cơ chế β^L ở mục 3.2-3.3 của bài này, chỉ khác domain (RNN theo thời gian, CNN theo độ sâu).
- **opt 46 (Bengio, "Practical recommendations for gradient-based training of deep architectures"):** tổng hợp các khuyến nghị thực hành bao gồm cả khởi tạo trọng số, learning rate, cho mạng sâu nói chung (trước giai đoạn ReLU/BatchNorm thống trị); hữu ích để đối chiếu xem khuyến nghị init kiểu "pre-2015" khác công thức có nguyên tắc của bài này ra sao.

8. Ứng dụng cho dự án (suy luận)

Dự án đang hướng tới là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, backbone hybrid Gated-DeltaNet + full attention, khoảng 3.600 token/trang, GPU $\leq$ 8 GB, batch 1), bắt đầu từ một checkpoint **đã được huấn luyện sẵn**, không train from scratch. Toàn bộ phân tích variance của paper này (mục 2.2) chỉ có ý nghĩa tại **thời điểm khởi tạo ngẫu nhiên** của một mạng chưa học gì; một khi trọng số đã được train hội tụ, phân phối của chúng không còn thoả các giả định (đối xứng quanh 0, độc lập, cùng phân phối) mà công thức (10)/(14) dựa vào, nên Kaiming init **không áp dụng để "sửa" trọng số đã train**. Do đó, phần lớn nội dung bài này **ít liên quan trực tiếp** đến việc fine-tune một checkpoint có sẵn. Tuy vậy có một số điểm chạm cụ thể, đều là suy luận:

1. **Khởi tạo các lớp mới thêm vào.** Khi fine-tune, rất có thể sẽ thêm module mới chưa có trong checkpoint gốc: một adapter/LoRA, một projection layer mới nối giữa phần visual encoder và phần ngôn ngữ, hoặc một classification/pooling head mới cho một tác vụ phụ trợ. Các tham số mới này **chưa học gì**, đúng đúng tình huống mà công thức (10)/(14) nhắm tới: nên khởi tạo theo $\sqrt{2/n}$ (hoặc công thức tương ứng với hoạt động hoá thực sự dùng, GELU/SiLU/SwiGLU thay vì ReLU thuần) hơn là dùng một std cố định tuỳ ý, đặc biệt khi lớp mới đó nằm sâu trong một stack phi tuyến và chỉ có rất ít bước fine-tune (ngân sách GPU 8 GB, batch 1 không cho phép "chờ" hàng chục nghìn bước để tự sửa một init tệ).
2. **Thực ra PyTorch đã "mặc định" làm gần đúng điều này.** `nn.Linear`/`nn.Conv2d` trong PyTorch khởi tạo mặc định bằng một biến thể của `kaiming_uniform_`; tức bất cứ layer mới nào được tạo bằng framework chuẩn, không chỉnh gì thêm, đã gần như áp dụng đúng tinh thần bài này. Hệ quả thực tế (suy luận): điều quan trọng không phải là "tự chọn implement lại công thức (14)" mà là **kiểm tra** rằng không có đoạn code nào vô tình ghi đè init mặc định bằng một hằng số tuỳ ý (ví dụ copy code cũ dùng

`std=0.02` cố định kiểu GPT-2 cho một module hoàn toàn khác về fan-in), và rằng hoạt động hoá thực tế của module mới (nếu không phải ReLU/GELU mà là một gating phi tuyến khác, ví dụ trong khối GatedDeltaNet) được tính hệ số đúng kiểu công thức (15) nếu cần.

3. **Khởi tạo lại phần bị thay thế sau khi pruning.** Nếu hướng nén dùng cách ”cắt rồi hàn lại” (ví dụ thay nguyên một khối attention/SSM bằng một khối nhỏ hơn mới dựng, khác với cách chỉ cắt bớt neuron/head của khối cũ), khối mới đó cần một init hợp lý trước khi chạy giai đoạn phục hồi (distillation/fine-tune ngắn); tư duy ”tính variance theo fan-in và theo đặc tính hoạt động hoá cụ thể của khối đó” từ bài này áp dụng được trực tiếp, thay vì copy std của khối cũ (fan-in khác) hay dùng một hằng số bất kỳ. Việc này nằm ngoài phạm vi các thư mục pruning/distillation hiện có (chúng tập trung vào *chọn gì để cắt*, không bàn *khởi tạo lại phần mới thêm sau khi cắt*).
4. **PReLU và việc không weight-decay tham số hoạt động hoá** là một chi tiết kỹ thuật nhỏ nhưng dễ bị bỏ sót khi viết optimizer config: nếu một bước nén/tinh chỉnh nào đó thêm một hoạt động hoá có tham số học được (gating có hệ số dốc, hoặc một nonlinearity thay thế sau pruning), cần nhớ loại tham số đó khỏi param-group có weight decay, đúng như lưu ý của paper (weight decay kéo $a_i \to 0$, thiên vị âm thầm về phía ReLU).
5. **Chuẩn hoá (LayerNorm/RMSNorm) trong LightOnOCR gần như chắc chắn đã ”gánh” phần lớn vai trò ổn định variance** mà bài này giải quyết bằng init thuần (xem opt 38/39); vì vậy nếu chỉ fine-tune các lớp đã có norm bao quanh, độ nhạy với init ban đầu của chúng thấp hơn nhiều so với bối cảnh gốc của paper (mạng ReLU 2015, không có BatchNorm/LayerNorm). Đây là lý do chính khiến bài học chính của paper (variance qua rất nhiều tầng) **ít cấp bách** cho phần backbone đã train sẵn, và chỉ còn cấp bách cho các thành phần mới, chưa có norm bảo vệ, được thêm vào sau.

Điều **ít liên quan**: toàn bộ phần kiến trúc VGG-sửa-đổi (model A/B/C), chi tiết multi-view/multi-scale testing, và kết luận ”tăng độ rộng tốt hơn tăng độ sâu ở quy mô ImageNet 2012 2015” không chuyển thẳng sang một backbone hybrid GatedDeltaNet + attention đã có sẵn kiến trúc cố định.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây chỉ cần PyTorch thuần, chạy được trên một GPU $\leq$ 8 GB (hoặc cả CPU với mạng nhỏ), nhằm cảm nhận trực tiếp hiện tượng mục 3.2-3.3, không cần tập ImageNet đầy đủ:

1. **Đo variance activation theo độ sâu, không train.** Dựng một ”mạng” toy gồm $L = 30$ layer Linear (hoặc Conv2d 3×3) + ReLU, fan-in cố định (ví dụ $n = 256$), input ngẫu nhiên chuẩn hoá variance $= 1$. Forward một lần (không backward, không cần dữ liệu thật), log $\mathrm{Var}[y_l]$ sau mỗi layer với ba cách khởi tạo: std cố định 0.01, Xavier ($\sqrt{1/n}$), Kaiming ($\sqrt{2/n}$). Kỳ vọng: đường Kaiming gần như phẳng (variance giữ nguyên quanh 1) đúng như ví dụ tự tính ở mục 3.2; đường Xavier giảm theo 0.5^l; đường std cố định giảm nhanh hơn nhiều. Đối chiếu số đo với công thức (9).
2. **Đo variance gradient theo chiều ngược.** Cùng mạng trên, gắn một loss giả (ví dụ tổng output), backward, log $\mathrm{Var}[\Delta x_l]$ theo (13) cho cả ba cách init. Kỳ vọng thấy cùng pattern như thí nghiệm 1 nhưng ở chiều ngược; đây là cách tái hiện trực tiếp ”gradient diminishing” mà paper mô tả cho mô hình 30 tầng, không cần chờ train thật.
3. **Hội tụ thật trên mạng sâu, không BatchNorm.** Dựng CNN 20-30 tầng (Conv3x3 + ReLU, không BatchNorm, không residual) trên CIFAR-10 (hoặc một tập con nhỏ để chạy nhanh trên 8 GB). Train vài epoch với Xavier vs Kaiming init, log training loss theo step. Kỳ vọng tái hiện tinh thần Hình 2/3 của paper: ở độ sâu vừa cả hai hội tụ (Kaiming nhanh hơn chút), ở độ sâu đủ lớn Xavier có thể hoàn toàn không giảm loss trong khi Kaiming vẫn giảm.
4. **ReLU vs PReLU (channel-shared) ở ngân sách nhỏ.** Dùng cùng CNN 5-10 tầng trên CIFAR-10, so sánh ReLU thường với PReLU channel-shared (a khởi tạo 0.25, không weight decay cho a). Đo accuracy cuối và số tham số thêm vào (kỳ vọng đúng bằng số layer có hoạt động hoá, như cách tính 13 ở mục 4). Thử thêm biến thể ”có weight decay cho a” để kiểm tra claim của paper rằng weight decay kéo $a \to 0$: log giá trị a qua các epoch ở hai cấu hình.

5. **Khởi tạo một lớp mới thêm vào mạng đã train (liên hệ mục 8).** Lấy một mạng nhỏ đã train sẵn trên CIFAR-10, đóng băng toàn bộ, chỉ thêm một lớp Linear mới ở cuối (mô phỏng ”thêm head mới khi fine-tune”). So sánh tốc độ học của lớp mới khi khởi tạo nó bằng Kaiming (theo fan-in thật của lớp) với khi khởi tạo bằng một std cố định sai lệch nhiều so với fan-in (ví dụ quá nhỏ hoặc quá lớn 10 lần). Kỳ vọng: với std sai lệch mạnh, vài epoch đầu gradient của lớp mới rất nhỏ (gần như không học) hoặc loss dao động mạnh; đây minh hoạ trực tiếp cho gợi ý ở mục 8.1 rằng lớp mới thêm vào khi fine-tune vẫn cần init hợp lý dù phần backbone đã train sẵn.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
PReLU	$f(y_i) = \max(0, y_i) + a_i \min(0, y_i)$ (1)	$a_i = 0$: ReLU; a_i học được, khởi tạo 0.25
Gradient PReLU	$\partial E/\partial a_i = \sum_{y_i}(\partial E/\partial f(y_i))(\partial f(y_i)/\partial a_i)$ (2, 3)	không dùng weight decay cho a_i
Variance forward 1 layer	$\mathrm{Var}[y_l] = \frac{1}{2} n_l \mathrm{Var}[w_l] \mathrm{Var}[y_{l-1}]$ (8)	$n_l = k_l^2 c_l$, hệ số $\frac{1}{2}$ từ việc ReLU cắt bỏ nửa tín hiệu
Điều kiện forward	$\frac{1}{2} n_l \mathrm{Var}[w_l] = 1 \; \forall l$ (10)	std $= \sqrt{2/n_l}$, $b = 0$ (”Kaiming init”)
Variance backward 1 layer	$\mathrm{Var}[\Delta x_l] = \frac{1}{2} \hat{n}_l \mathrm{Var}[w_l] \mathrm{Var}[\Delta x_{l+1}]$ (12)	$\hat{n}_l = k_l^2 d_l$ (d_l = số filter layer l)
Điều kiện backward	$\frac{1}{2} \hat{n}_l \mathrm{Var}[w_l] = 1 \; \forall l$ (14)	std $= \sqrt{2/\hat{n}_l}$; chỉ cần dùng (10) hoặc (14), không cần cả hai
PReLU tổng quát	$\frac{1}{2}(1 + a^2) n_l \mathrm{Var}[w_l] = 1$ (15)	$a = 0$: trường hợp ReLU; $a = 1$: trường hợp Xavier/Glorot [7]
Xavier/Glorot [7]	$n_l \mathrm{Var}[w_l] = 1$, std $= \sqrt{1/n_l}$	giả định hoạt động hoá tuyến tính, không đúng cho ReLU
Hệ quả luỹ tích	tín hiệu/gradient nhân β^L qua L layer	$\beta > 1$: tràn số; $\beta < 1$: tiêu biến; cần $\beta = 1$

11. Kết luận ngắn

Paper đưa ra hai đóng góp gắn liền với nhau qua việc ”lấy đúng rectifier” làm trọng tâm phân tích, thay vì coi hoạt động hoá là tuyến tính như Glorot và Bengio [7]: (1) PReLU, một tổng quát hoá gần như miễn phí của ReLU với hệ số dốc âm học được, cho cải thiện nhỏ nhưng nhất quán (khoảng 0.2-1.2 điểm % top-1) ở cả mô hình nhỏ và lớn; (2) một công thức khởi tạo trọng số (std $= \sqrt{2/n}$, khác Xavier đúng một hệ số $\sqrt{2}$) suy ra từ phân tích variance forward/backward có tính đến việc ReLU cắt bỏ một nửa tín hiệu, cho phép huấn luyện trực tiếp từ đầu các mô hình tới 30 tầng mà Xavier init hoàn toàn không hội tụ. Kết hợp hai đóng góp với kiến trúc dựa trên VGG-19 (có SPP) và augmentation/testing đa scale, mô hình đơn đạt 5.71% top-5 error, tổ hợp 6 mô hình đạt 4.94%, lần đầu vượt mức lỗi con người được báo cáo (5.1%). Điểm cần nhớ khi đọc lại: ”hội tụ được ở độ sâu lớn” và ”độ sâu lớn tốt hơn” là hai claim tách biệt, paper chỉ chứng minh chắc chắn claim đầu (mô hình 30 tầng của chính họ cho kết quả tệ hơn mô hình 14 tầng); lợi ích thực sự về độ chính xác của bộ ba PReLU + Kaiming init + kiến trúc sửa đổi tới từ 19-22 tầng và từ việc tăng độ rộng, không phải từ việc đẩy độ sâu lên cực đại.

Nguồn gốc: https://arxiv.org/abs/1502.01852

Bài 38

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift (2015)

Tác giả: Sergey Ioffe, Christian Szegedy (Google Inc.).
Venue / arXiv: arXiv:1502.03167 (v3, 02/03/2015); paper không nêu venue (không có dòng "Published as...", không có nhãn hội nghị/tạp chí ở đầu trang 1, chỉ có nhãn arXiv).
Nguồn: https://arxiv.org/abs/1502.03167
Chủ đề: chuẩn hoá (normalize) activation của mỗi tầng theo thống kê mini-batch (mean, variance), kèm một phép scale/shift học được (γ, β), để giảm "internal covariate shift" và cho phép huấn luyện nhanh, ổn định hơn với learning rate cao hơn.
Vị trí trong lộ trình: nên đọc sau khi đã nắm SGD/momentum cơ bản (opt 01-03) và lý tưởng là cùng lúc hoặc ngay sau opt 37 (Delving Deep into Rectifiers, chính là bài He et al. 2015 được paper này trích dẫn làm mốc SOTA 4.94% top-5; cùng thời điểm 2015, giải quyết vấn đề "huấn luyện mạng sâu ổn định" bằng hướng khởi tạo trọng số thay vì chuẩn hoá activation). Đọc trước opt 39 (Layer Normalization, biến thể thay trục chuẩn hoá từ batch sang feature để bỏ sự phụ thuộc vào batch) và ngay trước opt 44 (How Does Batch Normalization Help Optimization?, bài phân tích lại và nghi ngờ chính lời giải thích "internal covariate shift" ở đây).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Năm 2015, SGD (và các biến thể như momentum, Adagrad) là công cụ chính để huấn luyện mạng sâu, nhưng đòi hỏi dò learning rate cẩn thận và khởi tạo tham số kỹ. Paper chỉ ra nguyên nhân sâu hơn: **đầu vào của mỗi tầng bị ảnh hưởng bởi tham số của TẤT CẢ các tầng trước nó**, nên chỉ một thay đổi nhỏ ở tầng đầu cũng khuếch đại thành thay đổi lớn ở các tầng sau khi mạng càng sâu.

Khái niệm nền: **covariate shift** (Shimodaira, 2000) là khi phân phối đầu vào của một hệ thống học thay đổi, thường xử lý bằng domain adaptation. Paper mở rộng khái niệm này cho một tầng/sub-network bên trong mạng: xét $\ell = F_2(F_1(u, \Theta_1), \Theta_2)$, việc học Θ_2 tương đương việc huấn luyện một mạng con F_2 nhận đầu vào $x = F_1(u, \Theta_1)$; nếu phân phối của x giữ cố định theo thời gian, Θ_2 không phải "điều chỉnh lại để bù" cho sự thay đổi phân phối đầu vào đó.

Paper định nghĩa **Internal Covariate Shift (ICS)**: sự thay đổi của phân phối các activation nội bộ trong mạng, do tham số của các tầng trước thay đổi trong quá trình huấn luyện.

Ví dụ cụ thể minh hoạ tác hại: với tầng sigmoid $z = g(Wu + b)$, $g(x) = 1/(1 + \exp(-x))$, khi $|x|$ lớn thì $g'(x) \to 0$. Vì $x = Wu + b$ bị ảnh hưởng bởi tham số của mọi tầng dưới, các thay đổi trong lúc huấn luyện dễ đẩy nhiều chiều của x vào vùng bão hoà của sigmoid, làm gradient truyền xuống u gần như biến mất (vanishing gradient), hiệu ứng này tăng theo độ sâu. Thực tế lúc đó khắc phục bằng ReLU, khởi tạo cẩn thận, và learning rate nhỏ. Câu hỏi đặt ra: nếu giữ phân phối đầu vào của mỗi tầng ổn định hơn trong suốt quá trình huấn luyện, liệu có huấn luyện nhanh và ổn định hơn mà không cần các biện pháp vòng ngoài đó?

2. Ý tưởng chính

Paper đề xuất **Batch Normalization (BN)**: chuẩn hoá đầu vào của mỗi tầng (hoặc mỗi activation trước phi tuyến) về mean 0, variance 1, dùng **thống kê của mini-batch hiện tại** (không phải toàn bộ tập huấn luyện, vì tính trên toàn tập là quá đắt và không khả vi theo cách thuận tiện). Để không làm mất khả năng biểu diễn của

mạng (ví dụ chuẩn hoá sigmoid về quanh 0 sẽ ép nó vào vùng gần tuyến tính), paper thêm một phép **scale và shift học được** $\gamma^{(k)}, \beta^{(k)}$ cho mỗi activation, để mạng có thể tự quyết định giữ nguyên hay không giữ nguyên phân phối chuẩn hoá đó.

Điểm mấu chốt về thiết kế: phép chuẩn hoá được đưa **vào chính kiến trúc mạng** (một phép biến đổi khả vi, tham gia đầy đủ vào backpropagation), không phải một bước xử lý rời rạc nằm ngoài vòng lặp gradient. Nhờ vậy BN:

- Giảm internal covariate shift, giúp các tầng sau ”học trên một phân phối đầu vào ổn định hơn”.
- Cho phép dùng **learning rate cao hơn nhiều** mà không có rủi ro divergence (Sec. 3.3).
- Có tác dụng như một **regularizer**, trong một số trường hợp có thể bỏ hoặc giảm Dropout (Sec. 3.4).
- Cho phép huấn luyện mạng với các phi tuyến bão hoà (sigmoid) mà bình thường rất khó huấn luyện.

Áp dụng vào mạng Inception tốt nhất thời điểm đó cho ImageNet, paper đạt cùng độ chính xác với **14 lần ít bước huấn luyện hơn**, vượt độ chính xác gốc đáng kể, và một ensemble các mạng BN đạt top-5 error 4.9% (test error 4.82% theo máy chủ ILSVRC), vượt độ chính xác ước tính của con người.

3. Phương pháp chi tiết

3.1 Vì sao không whitening toàn tập, và vì sao không chuẩn hoá ”ngoài” bước gradient

Cách cũ (LeCun et al., 1998b) biết rằng mạng hội tụ nhanh hơn nếu input được ”whitened”: biến đổi tuyến tính về mean 0, variance 1, và decorrelated. Áp ý tưởng này cho mọi tầng nghĩa là whitening activation ở mỗi bước huấn luyện, nhưng việc này có hai vấn đề:

1. **Tính toán đắt.** Whitening đầy đủ cần ma trận hiệp phương sai $\text{Cov}[x] = \mathbb{E}_{x\in X}[xx^T] - \mathbb{E}[x]\mathbb{E}[x]^T$ và nghịch đảo căn bậc hai của nó, cùng đạo hàm của phép biến đổi này cho backprop; quá đắt nếu làm lại ở mọi bước.
2. **Nếu tính thống kê ”ngoài” vòng lặp gradient (không coi là một phần của đồ thị tính đạo hàm), gradient descent không biết rằng chuẩn hoá sẽ thay đổi theo tham số**, nên có thể cập nhật tham số theo cách làm phản tác dụng. Paper minh hoạ bằng ví dụ: tầng có $x = u+b$, chuẩn hoá bằng $\hat{x} = x-\mathbb{E}[x]$ (tính $\mathbb{E}[x]$ trên toàn tập, coi là hằng số với b). Nếu một bước gradient cập nhật $b \leftarrow b + \Delta b$ mà bỏ qua việc $\mathbb{E}[x]$ cũng phụ thuộc b, thì $u + (b + \Delta b) - \mathbb{E}[u + (b + \Delta b)] = u + b - \mathbb{E}[u + b]$: đầu ra của tầng (và loss) **không đổi**, nhưng b cứ tăng vô hạn trong khi loss giữ nguyên. Paper ghi rõ: họ đã quan sát thực nghiệm mô hình ”blow up” (bùng nổ tham số) khi các tham số chuẩn hoá được tính ngoài bước gradient descent.

Kết luận của mục này: chuẩn hoá phải là một phép biến đổi **khả vi, tham gia đầy đủ vào backprop**, và không cần phân tích lại toàn bộ tập huấn luyện sau mỗi lần cập nhật tham số.

3.2 Hai đơn giản hoá để tạo ra Batch Normalization

Đơn giản hoá 1: thay vì whitening đồng thời (joint) các chiều của x, chuẩn hoá **từng chiều một cách độc lập** về mean 0, variance 1. Với input d-chiều $x = (x^{(1)}, \ldots, x^{(d)})$:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

(expectation/variance tính trên toàn tập huấn luyện trong công thức lý tưởng này). Paper nhấn mạnh: dù chỉ chuẩn hoá từng chiều riêng (không decorrelate), vẫn tăng tốc hội tụ, theo (LeCun et al., 1998b).

Đơn giản hoá 2: vì huấn luyện bằng mini-batch, dùng **thống kê của mini-batch** để ước lượng mean/variance thay cho toàn tập; cách này cho phép thống kê chuẩn hoá tham gia đầy đủ vào backprop. Paper cũng giải

thích lý do chọn chuẩn hoá **theo từng chiều** (per-dimension variance) thay vì hiệp phương sai chung: cỡ mini-batch m thường **nhỏ hơn** số activation cần whiten, nên ma trận hiệp phương sai chung sẽ singular (không khả nghịch) nếu không regularize thêm.

Vấn đề mất khả năng biểu diễn: chuẩn hoá thuần có thể đổi những gì tầng có thể biểu diễn (ví dụ ép input sigmoid vào vùng tuyến tính gần 0). Để giải quyết, với mỗi activation $x^{(k)}$, thêm một cặp tham số học được $\gamma^{(k)}, \beta^{(k)}$, scale và shift giá trị đã chuẩn hoá:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

Các tham số này học cùng với tham số gốc của mô hình, và **khôi phục khả năng biểu diễn**: nếu đặt $\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$ và $\beta^{(k)} = \mathbb{E}[x^{(k)}]$, có thể khôi phục lại chính activation gốc (nếu đó là lựa chọn tối ưu).

3.3 Thuật toán BN Transform (Algorithm 1)

Với một mini-batch $B = \{x_1, \ldots, x_m\}$ của **một** activation $x^{(k)}$ (bỏ chỉ số k cho gọn), ϵ là hằng số nhỏ cộng vào variance để tránh chia 0:

```
Input:  gia tri x tren 1 mini-batch B = {x_1 ... x_m}; tham so can hoc: gamma, beta
Output: {y_i = BN_{gamma,beta}(x_i)}

mu_B      <- (1/m) * sum_i x_i                        // mini-batch mean
sigma_B^2 <- (1/m) * sum_i (x_i - mu_B)^2              // mini-batch variance
x_hat_i   <- (x_i - mu_B) / sqrt(sigma_B^2 + eps)      // normalize
y_i       <- gamma * x_hat_i + beta  =  BN_{gamma,beta}(x_i)   // scale and shift
```

Đây là **Algorithm 1** của paper, áp dụng cho một activation bất kỳ ($y = BN_{\gamma,\beta}(x)$), có thể chèn vào bất cứ đâu trong mạng. Backprop qua BN tính gradient theo chuỗi (chain rule) qua bốn đại lượng trung gian theo thứ tự ngược: $\partial \ell/\partial \hat{x}_i \to \partial \ell/\partial \sigma_B^2 \to \partial \ell/\partial \mu_B \to \partial \ell/\partial x_i$, cộng với $\partial \ell/\partial \gamma = \sum_i (\partial \ell/\partial y_i)\hat{x}_i$ và $\partial \ell/\partial \beta = \sum_i \partial \ell/\partial y_i$. Paper viết đầy đủ các công thức này (hơi rối vì nhiều chỉ số); điểm quan trọng không phải nhớ từng công thức mà là: **mọi đại lượng trung gian (μ_B, σ_B^2, $\hat{x}$) đều nằm trong đồ thị tính gradient**, nên BN không gặp vấn đề "mù" trước chuẩn hoá như ví dụ ở mục 3.1.

Tính chất của $\hat{x}$: bỏ qua ϵ, với mọi mini-batch (miễn các phần tử cùng phân phối), $\sum_i \hat{x}_i = 0$ và $(1/m)\sum_i \hat{x}_i^2 = 1$, tức mean 0 variance 1 đúng theo thiết kế.

Ví dụ số (tự tính). Lấy mini-batch $m = 4$ giá trị của một activation: $x = \{1, 2, 3, 6\}$.

i	x_i	$x_i - \mu_B$	$(x_i - \mu_B)^2$
1	1	-2	4
2	2	-1	1
3	3	0	0
4	6	3	9

$\mu_B = (1+2+3+6)/4 = 3$ (tự tính). $\sigma_B^2 = (4+1+0+9)/4 = 14/4 = 3.5$ (tự tính). Với ϵ bỏ qua (xem như $\epsilon \approx 0$), $\sqrt{\sigma_B^2} \approx 1.8708$, nên:

$$\hat{x} = \{-2/1.8708,\ -1/1.8708,\ 0/1.8708,\ 3/1.8708\} \approx \{-1.069,\ -0.535,\ 0,\ 1.604\} \text{ (tự tính)}$$

Kiểm tra: tổng $\hat{x}_i \approx 0$, trung bình $\hat{x}_i^2 \approx (1.143 + 0.286 + 0 + 2.572)/4 \approx 1.0003 \approx 1$ (tự tính, sai số do bo tròn). Chọn $\gamma = 2, \beta = 1$ (ví dụ), $y_i = \gamma \hat{x}_i + \beta$: $y \approx \{-1.138,\ -0.069,\ 1.000,\ 4.207\}$ (tự tính). Đây chính là bước "chuẩn hoá tay" một mini-batch nhỏ theo đúng Algorithm 1.

3.4 Huấn luyện và suy luận (inference) với mạng Batch-Normalized: Algorithm 2

Để BN hoá một mạng: chọn một tập activation $\{x^{(k)}\}_{k=1}^{K}$, chèn BN transform trước mỗi activation đó (theo Algorithm 1), sửa các tầng tiêu thụ $x^{(k)}$ để nhận $y^{(k)}$ thay vào. Paper lưu ý rõ: ”A model employing Batch Normalization can be trained using batch gradient descent, or Stochastic Gradient Descent **with a mini-batch size** $m > 1$, or with any of its variants such as Adagrad”. Tức BN được định nghĩa và huấn luyện dựa trên tiền đề **có nhiều hơn 1 mẫu trong batch**.

Sau khi huấn luyện xong mạng N_{BN}^{tr}, cần một mạng suy luận N_{BN}^{inf} có **tham số chuẩn hoá cố định** (vì ở inference, muốn đầu ra chỉ phụ thuộc deterministic vào input, không phụ thuộc các mẫu khác tình cờ nằm cùng batch). Thủ tục (Algorithm 2, bước 7-12): sau huấn luyện, chạy qua nhiều mini-batch B (kích thước m) và lấy trung bình:

```
E[x]   <- E_B[mu_B]                        // trung binh cac mini-batch mean
Var[x] <- (m / (m-1)) * E_B[sigma_B^2]     // uoc luong variance khong lech (unbiased)
```

rồi thay transform $y = BN_{\gamma,\beta}(x)$ bằng một phép **affine cố định duy nhất**:

$$y = \frac{\gamma}{\sqrt{\mathrm{Var}[x] + \epsilon}} \cdot x + \left(\beta - \frac{\gamma\, \mathbb{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} \right)$$

Vì $\mathrm{Var}[x]$ dùng hệ số hiệu chỉnh $m/(m-1)$ (ước lượng variance không lệch từ các sample variance σ_B^2, mỗi cái vốn chia cho m thay vì $m-1$), còn mean/variance ở đây đã **cố định sau huấn luyện**, phép biến đổi lúc inference chỉ là một phép tuyến tính đơn giản trên x, hoàn toàn deterministic, không phụ thuộc mini-batch nào nữa. Paper cũng nói có thể dùng **moving average** thay cho trung bình trên nhiều batch như trên, để theo dõi độ chính xác của mô hình trong lúc đang huấn luyện (đây là cách cài đặt phổ biến trong thực hành hiện đại, nhưng bản thân Algorithm 2 của paper trình bày phép trung bình qua nhiều mini-batch sau khi huấn luyện xong là quy trình chính).

Vì lúc inference phép chuẩn hoá chỉ là một affine cố định, nó **có thể hợp nhất (fold) vào tầng linear/convolution đứng trước** thành một phép biến đổi tuyến tính duy nhất, loại bỏ hoàn toàn chi phí runtime của BN lúc suy luận (điểm này rất hữu ích cho nén mô hình, xem mục 8).

3.5 Áp dụng cho tầng convolution

Cho một tầng tổng quát $z = g(Wu + b)$ (W, b là tham số học được, g là phi tuyến như sigmoid hoặc ReLU; công thức này bao hàm cả fully-connected và convolution), paper chèn BN **ngay trước phi tuyến**, chuẩn hoá $x = Wu + b$. Lý do không chuẩn hoá trực tiếp u (input của tầng): u thường là output của một phi tuyến trước đó, nên dạng phân phối của nó dễ thay đổi trong lúc huấn luyện, chuẩn hoá hai moment đầu không đủ loại bỏ covariate shift; trong khi $Wu + b$ thường có dạng phân phối đối xứng, không sparse, ”giống Gaussian hơn” (theo Hyvärinen & Oja, 2000), nên chuẩn hoá nó dễ cho ra activation có phân phối ổn định hơn.

Vì chuẩn hoá $Wu + b$, **bias** b **có thể bỏ qua** (tác dụng của nó bị huỷ bởi phép trừ mean; vai trò của bias được β trong BN thay thế). Nên $z = g(Wu + b)$ trở thành $z = g(BN(Wu))$.

Với tầng convolution, cần tôn trọng tính chất convolution: các vị trí khác nhau của cùng một feature map phải được chuẩn hoá theo cùng một cách. Paper giải quyết bằng cách để B là **tập mọi giá trị trong một feature map trên cả mini-batch và mọi vị trí không gian**: với mini-batch cỡ m và feature map kích thước $p \times q$, mini-batch hiệu dụng có cỡ $m' = |B| = m \cdot p \cdot q$. Mỗi feature map chỉ học **một cặp** $\gamma^{(k)}, \beta^{(k)}$ (không phải một cặp cho mỗi vị trí không gian).

Ví dụ (tự tính): với mini-batch $m = 32$ và feature map 28×28 (như các tầng trong Inception ở thí nghiệm, xem mục 4), mini-batch hiệu dụng $m' = 32 \times 28 \times 28 = 25\,088$ giá trị dùng để tính một cặp μ_B, σ_B^2 cho feature map đó.

3.6 Vì sao BN cho phép learning rate cao hơn (Sec. 3.3)

Với learning rate quá cao trong mạng thường, gradient có thể explode/vanish hoặc kẹt ở local minima xấu. BN giúp vì backprop qua một tầng **không còn phụ thuộc vào scale của tham số đó**: với một hệ số vô hướng a,

$$BN(Wu) = BN((aW)u)$$

và có thể chỉ ra $\partial BN((aW)u)/\partial u = \partial BN(Wu)/\partial u$ (không đổi theo a), còn $\partial BN((aW)u)/\partial(aW) = (1/a) \cdot \partial BN(Wu)/\partial W$ (nhỏ hơn khi a lớn). Hệ quả: trọng số lớn hơn tạo ra gradient nhỏ hơn, nên BN **tự ổn định tốc độ tăng của tham số**.

Paper còn **giả thuyết** (conjecture, không chứng minh chặt): BN có thể khiến Jacobian của mỗi tầng có singular value gần 1, điều này được biết là có lợi cho huấn luyện (Saxe et al., 2013). Lập luận: với hai tầng liên tiếp có input đã chuẩn hoá $\hat{x}, \hat{z} = F(\hat{x})$; nếu giả định $\hat{x}, \hat{z}$ là Gaussian, không tương quan, và F là biến đổi tuyến tính $\hat{z} \approx J\hat{x}$, thì cả $\hat{x}, \hat{z}$ có hiệp phương sai đơn vị, nên $I = \mathrm{Cov}[\hat{z}] = J\,\mathrm{Cov}[\hat{x}]\,J^T = JJ^T$, suy ra mọi singular value của J đều bằng 1. Paper tự thừa nhận: trong thực tế biến đổi không tuyến tính, và giả trị chuẩn hoá không chắc Gaussian hay độc lập, nên đây chỉ là kỳ vọng định tính, không phải kết quả chứng minh cho mạng thật.

3.7 BN như một regularizer (Sec. 3.4)

Khi huấn luyện với BN, một mẫu luôn được xử lý **cùng với các mẫu khác trong mini-batch nó rơi vào**, nên mạng huấn luyện không còn cho ra giá trị deterministic với một mẫu cố định (giá trị chuẩn hoá phụ thuộc vào ai ở cùng batch). Paper thấy hiệu ứng này có lợi cho generalization, và trong thực nghiệm của họ **Dropout có thể bị loại bỏ hoặc giảm cường độ** khi dùng BN.

4. Thí nghiệm và kết quả chính

4.1 MNIST: minh hoạ Internal Covariate Shift (Sec. 4.1)

Mạng rất đơn giản: input ảnh nhị phân 28×28, 3 tầng fully-connected ẩn với 100 activation mỗi tầng, mỗi tầng tính $y = g(Wu + b)$ với g là sigmoid, W khởi tạo Gaussian ngẫu nhiên nhỏ; tầng ẩn cuối nối với một tầng fully-connected 10 activation (1 cho mỗi lớp) và cross-entropy loss. Huấn luyện 50.000 bước, 60 mẫu mỗi mini-batch. BN được thêm vào mỗi tầng ẩn. Paper nói rõ mục tiêu chỉ là **so sánh baseline với BN**, không nhằm đạt SOTA trên MNIST.

Kết quả (Hình 1): mạng có BN đạt test accuracy cao hơn mạng gốc theo số bước huấn luyện (Hình 1a). Quan sát phân phối input của một sigmoid điển hình ở tầng ẩn cuối, vẽ theo percentile {15, 50, 85} qua thời gian huấn luyện (Hình 1b,c): ở mạng gốc, phân phối này **thay đổi đáng kể cả mean và variance** theo thời gian (phức tạp hoá việc học của các tầng sau); ở mạng có BN, phân phối **ổn định hơn nhiều** suốt quá trình huấn luyện, minh hoạ trực quan cho tuyên bố "BN giảm internal covariate shift".

4.2 ImageNet: Inception và các biến thể Batch-Normalized (Sec. 4.2)

Mạng cơ sở: một biến thể của Inception (Szegedy et al., 2014), huấn luyện trên tác vụ phân loại ImageNet (ILSVRC, 1000 lớp), tầng convolution dùng ReLU. Khác biệt chính so với Inception gốc: các tầng convolution 5×5 được thay bằng hai tầng convolution 3×3 liên tiếp với tối đa 128 filter; mạng có 13.6×10^6 tham số, không có tầng fully-connected nào ngoài tầng softmax cuối. Theo phụ lục: việc thay $5 \times 5 \to$ hai 3×3 tăng độ sâu tối đa thêm 9 tầng có trọng số, tăng số tham số 25%, tăng chi phí tính toán khoảng 30%; số module Inception 28×28 tăng từ 2 lên 3; không có tầng pooling đồng loạt giữa các module Inception (thay bằng convolution/pooling stride-2 trước khi nối các filter ở module 3c, 4e); tầng convolution đầu tiên dùng separable convolution với depth multiplier 8 (giảm chi phí tính toán, tăng bộ nhớ lúc huấn luyện).

Huấn luyện bằng SGD có momentum, mini-batch size 32, trên hạ tầng phân tán lớn (tương tự Dean et al., 2012). BN được áp vào input của mọi phi tuyến, theo cách convolutional (mục 3.5 ở trên), giữ nguyên phần còn lại của kiến trúc.

Các mạng được so sánh (Sec. 4.2.1-4.2.2), tất cả huấn luyện trên dữ liệu train LSVRC2012, đánh giá trên tập validation:

- **Inception:** mạng gốc, learning rate khởi đầu 0.0015.
- **BN-Baseline:** giống Inception, thêm BN trước mỗi phi tuyến.
- **BN-x5:** Inception + BN + các thay đổi ở mục 4.2.1 dưới đây; learning rate khởi đầu tăng 5 lần lên 0.0075. Paper ghi rõ: tăng learning rate tương tự cho Inception gốc (không BN) làm tham số mô hình ”đạt tới machine infinity” (NaN/Inf).
- **BN-x30:** như BN-x5 nhưng learning rate khởi đầu 0.045 (30 lần Inception gốc).
- **BN-x5-Sigmoid:** như BN-x5 nhưng dùng sigmoid thay ReLU. Paper cũng thử huấn luyện Inception gốc với sigmoid: mô hình giữ nguyên ở độ chính xác tương đương đoán ngẫu nhiên (không bao giờ đạt hơn 1/1000).

Các thay đổi cộng thêm cho BN-x5/BN-x30 (Sec. 4.2.1), áp dụng cùng lúc như một ”gói”:

1. Tăng learning rate (nhờ Sec. 3.3, không có tác dụng phụ xấu quan sát được).
2. Bỏ Dropout (BN đã đảm nhiệm một phần vai trò regularize).
3. Giảm hệ số L2 weight regularization đi 5 lần (cải thiện accuracy trên validation).
4. Decay learning rate nhanh hơn 6 lần (vì mạng BN huấn luyện nhanh hơn).
5. Bỏ Local Response Normalization (không cần thiết khi có BN).
6. Xáo trộn (shuffle) dữ liệu huấn luyện kỹ hơn (within-shard shuffling), tránh các mẫu luôn xuất hiện cùng nhau trong một mini-batch; riêng thay đổi này được paper định lượng: cải thiện khoảng **1%** validation accuracy, nhất quán với lập luận BN là regularizer (ngẫu nhiên hoá phát huy tác dụng tốt nhất khi mỗi mẫu bị ảnh hưởng khác nhau mỗi lần gặp).
7. Giảm méo quang trắc (photometric distortion) của ảnh, vì mạng BN huấn luyện nhanh và nhìn mỗi mẫu ít lần hơn nên cho trainer tập trung vào ảnh ”thật” hơn.

Kết quả số bước để đạt 72.2% (độ chính xác tối đa của Inception gốc) và độ chính xác tối đa đạt được (Hình 3):

Mô hình	Số bước để đạt 72.2%	Độ chính xác tối đa
Inception	31.0×10^6	72.2%
BN-Baseline	13.3×10^6	72.7%
BN-x5	2.1×10^6	73.0%
BN-x30	2.7×10^6	74.8%
BN-x5-Sigmoid	(không đạt)	69.8%

Diễn giải bằng lời của paper: chỉ thêm BN (BN-Baseline) đã khớp accuracy của Inception trong **chưa đến một nửa** số bước ($13.3/31.0 \approx 0.43$, tự tính). Thêm các thay đổi 4.2.1, BN-x5 cần **14 lần ít bước hơn** Inception để đạt 72.2% ($31.0/2.1 \approx 14.76$, tự tính, khớp số ”14” paper nêu ở abstract; phần Introduction diễn đạt cùng ý này dưới dạng ”chỉ 7% số bước huấn luyện”, và $2.1/31.0 \approx 6.8\%$ (tự tính), khớp). Tăng learning rate hơn nữa (BN-x30) làm mạng huấn luyện **chậm hơn một chút lúc đầu**, nhưng đạt accuracy cuối cao hơn: 74.8% sau 6×10^6 bước, tức **5 lần ít bước hơn** Inception cần để đạt 72.2% ($31.0/6 \approx 5.17$, tự tính). Lưu ý: cột ”số bước để đạt 72.2%” của BN-x30 trong bảng (2.7×10^6) là một mốc trung gian (lúc nó đi qua ngưỡng 72.2%), khác với số bước để đạt chính đỉnh của nó (74.8% sau 6×10^6 bước); đây là hai mốc khác nhau, không mâu thuẫn.

BN còn cho phép huấn luyện mạng với phi tuyến bão hoà: BN-x5-Sigmoid đạt 69.8% (không bao giờ đạt 72.2% trong phạm vi thí nghiệm), còn Inception gốc với sigmoid thì **không học được gì** (ở mức đoán ngẫu nhiên).

4.3 Ensemble classification (Sec. 4.2.3)

Mốc SOTA trước đó: Deep Image ensemble (Wu et al., 2015) và ensemble của He et al. (2015) với top-5 error **4.94%** (theo máy chủ ILSVRC; He et al. 2015 chính là bài opt 37, ”Delving Deep into Rectifiers”). Paper này báo cáo: ensemble 6 mạng (mỗi mạng dựa trên BN-x30, có thay đổi thêm: tăng trọng số khởi tạo ở tầng convolution; dùng Dropout xác suất 5% hoặc 10% so với 40% của Inception gốc; dùng BN theo kiểu non-convolutional, per-activation ở các tầng ẩn cuối) đạt top-5 validation error **4.9%**, test error **4.82%** (theo máy chủ ILSVRC). Mỗi mạng thành phần đạt accuracy tối đa sau khoảng 6×10^6 bước; dự đoán ensemble là trung bình số học (arithmetic average) xác suất lớp của các mạng thành phần.

Bảng so sánh (Hình 4, copy từ paper):

Mô hình	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	variable	-	-	-	5.98%
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	4.9%*

(*) BN-Inception ensemble đạt 4.82% top-5 error trên 100.000 ảnh của tập test ImageNet, theo máy chủ test. Đây vượt độ chính xác ước tính của người đánh giá con người theo (Russakovsky et al., 2014), theo đúng tuyên bố ở abstract.

5. Phân tích / ablation

Cần nói thẳng: **paper này không có một bảng ablation tách riêng từng yếu tố** theo đúng nghĩa (kiểu ”bật/tắt từng thay đổi một”). Những gì thực sự có:

- **BN-Baseline vs BN-x5/BN-x30:** chỉ thêm BN đã cải thiện tốc độ và accuracy; thêm ”gói” 7 thay đổi ở mục 4.2 (confound, không tách được đóng góp riêng, **trừ** shuffle: khoảng 1% validation accuracy) cải thiện xa hơn nữa.
- **BN-x5 vs BN-x30 (ablation về learning rate):** tăng learning rate từ 5 lần lên 30 lần so với Inception gốc làm chậm giai đoạn đầu nhưng nâng đỉnh cuối (73.0% $\rightarrow$ 74.8%), cho thấy một đánh đổi tốc độ-học-ban-đầu và accuracy-cuối theo quy mô learning rate, nhất quán với lập luận Sec. 3.3 rằng BN mở ra không gian learning rate lớn hơn an toàn hơn.
- **BN-x5-Sigmoid vs Inception+sigmoid (ablation về phi tuyến):** đây là ablation rõ ràng nhất của paper, cách ly đúng một biến (có BN hay không, cùng dùng sigmoid): không BN thì mạng sigmoid kẹt ở mức đoán ngẫu nhiên; có BN thì huấn luyện thành công, đạt 69.8%. Đây là bằng chứng mạnh nhất, tách biệt tốt nhất, cho tuyên bố ”BN giúp tránh kẹt ở vùng bão hoà của phi tuyến”.
- **Hình 1(b,c) (MNIST, định tính):** so percentile {15,50,85} của input một sigmoid theo thời gian, cho bằng chứng trực quan (không phải số liệu thống kê) rằng BN làm phân phối input ổn định hơn qua các bước huấn luyện.
- **Lập luận lý thuyết Sec. 3.3 (bất biến scale):** $BN(Wu) = BN((aW)u)$ là một đẳng thức hình thức (không phải thực nghiệm đo trên mạng thật), dùng để giải thích vì sao BN ”cho phép” learning rate cao hơn, nhưng không kèm số liệu đo trực tiếp norm gradient theo a trên Inception.

- **Shuffle training (+1%):** là số liệu ablation cụ thể duy nhất được tách riêng trong toàn bộ Sec. 4.2.1, và paper dùng nó làm bằng chứng gián tiếp cho giả thuyết ”BN là regularizer” (Sec. 3.4): shuffle tốt hơn khiến mỗi mẫu bị chuẩn hoá cùng các mẫu khác nhau mỗi lần gặp, tăng độ ngẫu nhiên có lợi.

6. Giới hạn

Paper tự nêu (Sec. 5, Conclusion):

- Chưa khám hết ”toàn bộ dải khả năng” mà BN mở ra; để lại cho tương lai: áp dụng cho Recurrent Neural Network (nơi internal covariate shift và vanishing/exploding gradient có thể nghiêm trọng hơn); khả năng dùng cho domain adaptation (có thể chỉ cần tính lại population mean/variance, Algorithm 2, cho phân phối mới); và **”further theoretical analysis of the algorithm”** để hiểu sâu và cải thiện hơn.
- Về cơ chế giúp gradient propagation: paper viết rõ ”**the precise effect of Batch Normalization on gradient propagation remains an area of further study**”, và lập luận về singular value ≈ 1 của Jacobian (mục 3.6) chỉ là một **giả thuyết (conjecture)** dựa trên giả định Gaussian/tuyến tính không chắc đúng với mạng thật, bản thân paper cũng thừa nhận điều này.
- Việc bỏ/giảm Dropout chỉ được xác nhận ”trong trường hợp của họ” (Modified BN-Inception); paper không khảo sát tổng quát khi nào BN đủ để thay Dropout.

Người ghi chú nhận thấy thêm:

- **Yêu cầu $m > 1$ (đã trích ở mục 3.4) là nền tảng, không phải chi tiết phụ:** mọi lợi ích (tốc độ, regularization, giảm ICS) đều dựa trên có nhiều hơn 1 mẫu để μ_B, σ_B^2 có ý nghĩa thống kê; đây là điểm nghẽn khi áp dụng BN ở batch=1 (mục 8).
- **Số liệu lệch nhỏ giữa abstract và thân bài (rounding, không phải mâu thuẫn thực chất):** abstract ghi ”4.9% top-5 validation error (and 4.8% test error)”, còn Sec. 4.2.3 và Hình 4 ghi rõ test error **4.82%**. Đây là làm tròn (4.82% $\rightarrow$ 4.8%), nên không trích ”4.8%” như một con số chính xác riêng.
- **Các tỉ lệ tốc độ paper nêu (14 lần, ”7% số bước”, ”chưa đến một nửa”, 5 lần) đều là số làm tròn của các phép chia nguyên** (xem các phép tính ”tự tính” ở mục 4.2, ví dụ $31.0/2.1 \approx 14.76$ chứ không đúng 14); không phải mâu thuẫn, chỉ là làm tròn.
- **Lỗi/thiếu sót nhỏ trong trình bày của paper:** công thức sigmoid ở BN-x5-Sigmoid viết $g(t) = 1/(1 + \exp(-x))$ (biến t ở vế trái nhưng x ở vế phải, không khớp biến); hàng ”Deep Image high-res” trong Hình 4 ghi top-1 error là ”24.88” không kèm dấu %, trong khi các hàng khác đều có % (giữ nguyên ”24.88” theo paper ở mục 4.3, không tự thêm % vì không chắc đây là lỗi in hay quy ước khác).
- **Các thay đổi ở Sec. 4.2.1 confound với nhau** (đã nói ở mục 5, gói 7 thay đổi): không tách được đóng góp riêng của BN khỏi bảy thay đổi kèm theo ở BN-x5/BN-x30; đóng góp riêng của BN (không thay đổi gì khác) chỉ thấy ở BN-Baseline (cải thiện 72.2% $\rightarrow$ 72.7%, nhanh hơn khoảng 2.3 lần số bước, $31.0/13.3 \approx 2.33$ tự tính, khiêm tốn hơn nhiều so với BN-x30).
- **Không có khảo sát định lượng ảnh hưởng của cỡ mini-batch m lên chất lượng chuẩn hoá** (ví dụ $m = 8$ so với $m = 256$): toàn bộ thí nghiệm ImageNet dùng $m = 32$ cố định, nên paper không cho biết thực nghiệm BN ”xuống cấp” ra sao khi m nhỏ, một câu hỏi về sau trở nên quan trọng (và là một phần động lực cho Layer Normalization, opt 39).
- **Không so sánh với phương án chuẩn hoá khác** (vì tại thời điểm 2015 LayerNorm, GroupNorm, Weight Normalization chưa tồn tại); paper chỉ so với ”standardization layer” của Gülçehre & Bengio (2013), áp dụng sau phi tuyến (khác vị trí, mục tiêu khác: tạo activation sparse hơn) nên không phải một so sánh cùng điều kiện.
- Mô hình Inception ở đây có 13.6×10^6 tham số: nhỏ so với tiêu chuẩn hiện nay, kết quả tốc độ huấn luyện có thể không ngoại suy trực tiếp cho mô hình vài tỷ tham số (paper không bàn tới quy mô này, hợp lý với thời điểm 2015).

7. Liên hệ với các bài khác

- **opt 37 (Delving Deep into Rectifiers, He et al. 2015):** chính là bài được paper này trích dẫn trực tiếp làm mốc SOTA trước đó (top-5 error 4.94%) mà BN-Inception ensemble vượt qua (4.9%/4.82%). Hai bài giải quyết cùng một nỗi lo ”huấn luyện mạng sâu ổn định” nhưng theo hai hướng khác nhau: opt 37 sửa **khởi tạo trọng số** (He initialization) cho ReLU, còn bài này sửa **phân phối activation trong lúc huấn luyện** bằng chuẩn hoá theo batch; hai kỹ thuật không loại trừ nhau, có thể dùng cùng lúc.
- **opt 39 (Layer Normalization):** biến thể trực tiếp, thay trục tính thống kê từ ”theo batch, mỗi feature” sang ”theo feature, mỗi mẫu” (tính μ, σ^2 trên các chiều feature của **một** mẫu, không cần nhiều mẫu). Nhờ vậy LayerNorm hoạt động giống nhau ở train và inference, không cần Algorithm 2 (population statistics), và không cần $m > 1$, giải quyết trực tiếp hạn chế $m > 1$ nêu ở mục 6. Nên so sánh song song hai bài để thấy rõ đánh đổi (batch statistics mạnh hơn về mặt ”chuẩn hoá chéo mẫu” khi m lớn, nhưng phụ thuộc batch; LayerNorm yếu hơn về mặt đó nhưng độc lập batch).
- **opt 44 (How Does Batch Normalization Help Optimization?):** bài phân tích lại chính paper này, đặt câu hỏi trực diện vào lời giải thích ”internal covariate shift” ở mục 1-2 trên: bài 44 thực nghiệm cho thấy việc giảm ICS (theo định nghĩa của chính paper này) không tương quan tốt với tốc độ huấn luyện nhanh hơn, và đề xuất lời giải thích khác (làm mượt loss landscape / cho phép learning rate hiệu dụng lớn hơn một cách khác). Đây là ví dụ rõ về việc lời giải thích ”tại sao nó hoạt động” của một bài nền tảng bị bài sau chất vấn, dù kỹ thuật (BN transform) vẫn được giữ nguyên và dùng rộng rãi.
- **llm content/09_2020_pre_ln_transformer.md:** các kiến trúc Transformer (và phần lớn LLM/VLM hiện đại) dùng LayerNorm (hoặc biến thể RMSNorm) ở vị trí Pre-LN hay Post-LN, **không dùng Batch Normalization**. Lý do một phần chính là các hạn chế mục 6 ở trên (phụ thuộc batch, cần $m > 1$, hành vi train/inference khác nhau); so bài 09 với bài này cho thấy lịch sử: cộng đồng vision (CNN, 2015-2018) dùng BN rất thành công, nhưng cộng đồng sequence model/Transformer (2017 trở đi) chọn LayerNorm ngay từ đầu, và không quay lại BN.

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B, một VLM OCR dùng kiến trúc hybrid GatedDeltaNet (linear-attention/gated-delta kiểu SSM) + full attention, khoảng 3.600 token/trang (1.500 text + 1.6-2.1K visual), chạy trên GPU ≤ 8 GB, **batch 1**. Dưới đây là suy luận (không có quyền truy cập trực tiếp vào mã nguồn LightOnOCR để xác nhận), không phải trích dẫn từ paper BN hay từ tài liệu LightOnOCR.

1. **BN gần như chắc chắn không phù hợp ở chế độ batch=1 của dự án, như đã trích ở mục 3.4/6 (”mini-batch size $m > 1$”).** Với một tầng fully-connected/attention (không có trục không gian), ở batch=1, σ_B^2 của chính activation đó bằng 0 (chỉ một mẫu, không có độ lệch quanh ”mean của chính nó”), phép chia $(x_i - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$ suy biến (luôn ra 0 vì $x_i = \mu_B$, hoặc rất nhạy ϵ); đây là hệ quả toán học trực tiếp từ Algorithm 1 khi $m = 1$, không phải lỗi cài đặt. (Với tầng convolution thì đỡ hơn một chút vì $m' = p \cdot q > 1$ nhờ các vị trí không gian, nhưng vẫn là ước lượng rất nhiễu nếu chỉ dựa trên 1 ảnh.) Nếu dự án phải fine-tune một module như vậy ở batch=1, running statistics sẽ được cập nhật từ các mẫu đơn lẻ liên tiếp, rất nhiễu, dễ gây mất ổn định đúng theo cơ chế mục 3.1.
2. **Vì sao LayerNorm/RMSNorm (rất có khả năng LightOnOCR dùng, suy luận) tránh được vấn đề này.** LayerNorm/RMSNorm chuẩn hoá theo trục **feature, trong một mẫu/một token**, nên với batch=1 (hay bất kỳ batch size nào) phép tính không đổi dạng, không cần Algorithm 2 (population statistics tách biệt train/inference). Suy luận: vì Transformer hiện đại và các biến thể linear-attention/SSM như GatedDeltaNet đều kế thừa truyền thống LayerNorm/RMSNorm, LightOnOCR hầu như chắc chắn **không chứa tầng BatchNorm nào** trong backbone ngôn ngữ hay khối GatedDeltaNet/attention; nếu có BN, nhiều khả năng chỉ nằm ở một conv stem thị giác kiểu cũ (thiết kế đã lỗi thời với VLM hiện đại).
3. **Ý tưởng duy nhất từ paper này còn chuyển giao được: thủ thuật ”fold BN thành một affine” ở**

Algorithm 2 (mục 3.4). Nếu một module con nào đó của pipeline OCR (ví dụ một vision encoder phụ trợ cũ) có BN, trước khi prune/quantize nên **hợp nhất BN vào trọng số của tầng linear/conv đứng ngay trước nó** (công thức $y = \frac{\gamma}{\sqrt{\text{Var}[x]+\epsilon}}x + (\beta - \frac{\gamma\mathbb{E}[x]}{\sqrt{\text{Var}[x]+\epsilon}})$): loại hoàn toàn chi phí runtime của BN, hữu ích khi đo latency hoặc pruning dựa trên trọng số đã fuse.

4. **Dùng γ như chỉ số "độ quan trọng" channel để pruning (kiểu Network Slimming):** đây **không** phải điều paper này đề xuất ("ngoài paper, kiến thức chung" từ các bài pruning sau); chỉ liệt kê để không nhầm là ý của bài gốc.

Tóm lại (suy luận): BN ít liên quan trực tiếp tới huấn luyện/serving batch=1 của LightOnOCR; giá trị còn lại là nền tảng lịch sử (vì sao LayerNorm ra đời) và thủ thuật fold-affine ở điểm 3.

9. Thí nghiệm gợi ý

1. **Tái lập thí nghiệm MNIST-sigmoid của paper ở quy mô nhỏ (1 GPU 8 GB, vài phút):** huấn luyện MLP 3 tầng ẩn x100 unit, sigmoid, trên MNIST, có và không có BN, 5.000-10.000 bước, batch 60 (giống paper). Đo: test accuracy theo số bước. Kỳ vọng: curve có BN tăng nhanh và cao hơn curve không BN, tái lập định tính Hình 1(a); vẽ thêm percentile {15,50,85} của input một sigmoid ở tầng cuối để tái lập Hình 1(b,c).
2. **Khảo sát BN theo cỡ batch (trực tiếp kiểm chứng lập luận mục 8):** cùng một CNN nhỏ (ví dụ 3 tầng conv) trên MNIST/Fashion-MNIST, train với BN ở các batch size $\{1, 2, 4, 8, 32, 128\}$. Đo: validation accuracy cuối cùng, và giá trị σ_B^2 trung bình quan sát mỗi bước. Kỳ vọng: batch 1 huấn luyện bất ổn hoặc hội tụ rất kém (biến thiên cao, có thể NaN nếu ϵ quá nhỏ); batch lớn ổn định; đây là bằng chứng thực nghiệm trực tiếp cho giới hạn "$m > 1$" nêu ở mục 6/8.
3. **So train-mode và eval-mode (frozen population stats) của BN ở batch=1:** lấy mô hình đã huấn luyện ổn ở batch 32 (bài 2), chạy forward một mẫu đơn (batch=1) ở train-mode (dùng thống kê chính mẫu đó, suy biến) và ở eval-mode (dùng population statistics cố định theo Algorithm 2). Đo: độ lệch logit/accuracy giữa hai mode. Kỳ vọng: train-mode ở batch=1 cho kết quả rất khác (và kém) so với eval-mode, minh hoạ trực tiếp vì sao BN cần một "chế độ suy luận" riêng mà LayerNorm không cần (so sánh thêm với bản LayerNorm của cùng mạng, không có sự khác biệt train/eval).
4. **Tái lập hiệu ứng "BN cho phép huấn luyện phi tuyến bão hoà" (BN-x5-Sigmoid) ở quy mô nhỏ:** một CNN nhỏ dùng sigmoid làm phi tuyến, trên Fashion-MNIST hoặc CIFAR-10 subset, có và không BN. Đo: accuracy sau N epoch cố định. Kỳ vọng: không BN mạng kẹt gần mức đoán ngẫu nhiên (giống Inception+sigmoid trong paper); có BN mạng học được, dù có thể không bằng bản ReLU.
5. **Fold BN vào conv và kiểm tra tương đương số học (vận hành trực tiếp mục 8, điểm 4):** huấn luyện CNN nhỏ có BN, sau đó tính lại trọng số conv đã hợp nhất BN theo công thức Algorithm 2, chạy forward trên cùng batch dữ liệu bằng (a) mô hình gốc ở eval-mode và (b) mô hình đã fold. Đo: sai số tuyệt đối giữa hai output (kỳ vọng xấp xỉ 0, chỉ còn sai số floating-point) và thời gian inference trước/sau fold (kỳ vọng fold nhanh hơn một chút do bớt một bước tính toán).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Internal Covariate Shift	sự thay đổi phân phối activation nội bộ do tham số các tầng trước thay đổi trong lúc huấn luyện
BN transform (Algorithm 1)	$\hat{x}_i = (x_i - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$; $y_i = \gamma\hat{x}_i + \beta$
μ_B, σ_B^2	mean, variance tính trên mini-batch hiện tại (biased, chia cho m)
γ, β	tham số học được, khôi phục khả năng biểu diễn; $\gamma = \sqrt{\text{Var}[x]}, \beta = \mathbb{E}[x]$ khôi phục activation gốc

Khái niệm	Công thức / ý nghĩa
Inference (Algorithm 2)	$\text{Var}[x] = \frac{m}{m-1}\mathbb{E}_B[\sigma_B^2]$ (unbiased); $y = \frac{\gamma}{\sqrt{\text{Var}[x]+\epsilon}}x + (\beta - \frac{\gamma\mathbb{E}[x]}{\sqrt{\text{Var}[x]+\epsilon}})$, một affine cố định
Conv BN	chuẩn hoá theo feature map, B gồm mọi vị trí không gian và mọi mẫu trong batch, $m' = m \cdot p \cdot q$; một γ, β cho mỗi feature map
Bất biến scale	$BN(Wu) = BN((aW)u)$; gradient theo W co lại $1/a$ khi a tăng, ổn định tăng trưởng tham số
Giả thuyết singular value	nếu $\hat{x}, \hat{z}$ Gaussian không tương quan và F tuyến tính J, thì $JJ^T = I$: mọi singular value của J bằng 1 (chỉ là lập luận định tính, không chứng minh cho mạng thật)
Yêu cầu huấn luyện	"mini-batch size $m > 1$" (trích nguyên văn); nền của mọi lợi ích BN
Kết quả chính ImageNet	BN-x5: 14 lần ít bước hơn Inception để đạt 72.2% ($31.0 \times 10^6 \to 2.1 \times 10^6$); ensemble BN-x30 (6 mạng): top-5 4.9%/4.82%

11. Kết luận ngắn

Batch Normalization chuẩn hoá activation theo thống kê mini-batch (μ_B, σ_B^2 từng chiều), kèm scale/shift học được γ, β để giữ khả năng biểu diễn, đưa thẳng vào kiến trúc mạng để tham gia đầy đủ backpropagation. Nhờ đó learning rate có thể tăng mạnh, Dropout có thể giảm/bỏ, và cả phi tuyến bão hoà (sigmoid) cũng huấn luyện được. Trên Inception/ImageNet: nhanh hơn 14 lần để đạt cùng accuracy, ensemble đạt top-5 error 4.9%/4.82%, vượt SOTA trước đó (4.94%, opt 37) và ước tính độ chính xác con người. Hạn chế cốt lõi, cả paper tự nhận (lý thuyết ICS/gradient chưa chặt) và quan sát về sau (yêu cầu nền tảng $m > 1$, hành vi train/inference khác nhau), là động lực trực tiếp cho Layer Normalization (opt 39) và các phân tích lại cơ chế này (opt 44).

Nguồn gốc: https://arxiv.org/abs/1502.03167

Bài 39

Layer Normalization (2016)

Tác giả: Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton (University of Toronto; Hinton đồng thời ở Google Inc.).
Venue / arXiv: arXiv:1607.06450 (v1, 21/07/2016); paper không nêu venue (đầu trang 1 chỉ có nhãn "arXiv:1607.06450v1 [stat.ML] 21 Jul 2016", không có dòng hội nghị/tạp chí; bài này thực tế chưa từng được công bố chính thức ở hội nghị lớn nào, chỉ tồn tại dưới dạng arXiv preprint).
Nguồn: https://arxiv.org/abs/1607.06450
Chủ đề: Layer Normalization (LN), chuẩn hoá tổng đầu vào (summed input) của các neuron trong một layer theo chiều feature, tính riêng cho từng training case (không theo batch như Batch Normalization), nên không phụ thuộc batch size và áp dụng tự nhiên cho RNN.
Vị trí trong lộ trình: đọc sau opt 38 (Batch Normalization, bài LN so sánh trực tiếp và kế thừa toàn bộ khung công thức); đọc trước opt 40 (khó khăn huấn luyện RNN: vanishing/exploding gradient) và opt 42 (Mixed Precision Training, liên quan tới việc mean/variance của LN nên tính ở độ chính xác nào). Nên đọc cùng llm 09 (Pre-LN Transformer) và llm 10 (RMSNorm, biến thể đơn giản hoá của chính LN).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng cho dự án" là suy luận.

1. Bối cảnh

Huấn luyện mạng neural sâu hiện đại (2016) bằng SGD rất tốn thời gian, thường mất nhiều ngày. Có hai hướng tăng tốc: (a) song song hoá trên nhiều máy, nhưng tốn communication và dễ gặp "diminishing returns"; (b) sửa chính phép tính ở forward pass để việc học dễ hơn, hướng mà Batch Normalization (BN, Ioffe và Szegedy 2015, xem opt 38) đi theo.

BN chuẩn hoá tổng đầu vào của mỗi neuron bằng mean/variance ước lượng từ **mini-batch**: với neuron i ở layer l, gọi a_i^l là tổng đầu vào (trước phi tuyến),

$$\bar{a}_i^l = \frac{g_i^l}{\sigma_i^l}(a_i^l - \mu_i^l), \qquad \mu_i^l = \mathbb{E}_{x\sim P(x)}[a_i^l], \qquad \sigma_i^l = \sqrt{\mathbb{E}_{x\sim P(x)}\left[(a_i^l - \mu_i^l)^2\right]} \tag{2}$$

Kỳ vọng lý tưởng là trên toàn bộ dữ liệu, nhưng vì không thể tính exact, μ, σ được ước lượng từ mini-batch. Hai hạn chế paper nêu ngay đầu bài:

1. **RNN:** tổng đầu vào của recurrent unit đổi theo độ dài chuỗi, nên BN "theo cách hiển nhiên" cần thống kê riêng cho mỗi time-step; chuỗi test dài hơn mọi chuỗi train sẽ không có thống kê tương ứng.
2. **Online learning / mô hình phân tán lớn:** khi mini-batch buộc phải nhỏ (kể cả batch size 1), ước lượng μ, σ từ batch rất nhiễu.

Khái niệm cần nắm: **mini-batch** (tập con nhỏ training case xử lý cùng lúc); **"covariate shift" nội bộ** (tham số layer dưới đổi làm phân phối tổng đầu vào layer trên trôi theo); **gain** g và **bias** b (hai tham số học được, nhân/cộng sau khi chuẩn hoá, cho mô hình tự điều chỉnh lại scale/vị trí).

Câu hỏi: có cách chuẩn hoá nào giữ lợi ích của BN nhưng không phụ thuộc batch size và dùng tự nhiên cho RNN không?

2. Ý tưởng chính

Ý tưởng cốt lõi là **đổi chiều lấy thống kê**: BN lấy mean/variance theo chiều **batch** (cố định một neuron, trung bình qua nhiều training case); LN lấy theo chiều **feature/layer** (cố định một training case, trung bình qua H neuron trong layer đó). Paper gọi đây là "transpose batch normalization into layer normalization".

Hệ quả trực tiếp:

- **Không ràng buộc batch size:** μ, σ chỉ cần chính training case đó, dùng được ở batch size 1 ("pure online regime").
- **Train và test giống nhau tuyệt đối:** BN cần ước lượng thống kê population (running average) để dùng ở test; LN không có vấn đề này, công thức ở test giống hệt lúc train.
- **Tự nhiên cho RNN:** chỉ cần tính lại μ^t, σ^t từ tổng đầu vào tại time-step t, không cần thống kê riêng theo vị trí thời gian.
- Mỗi neuron vẫn có **gain, bias riêng**, áp sau chuẩn hoá, trước phi tuyến.

Trực giác giảm "covariate shift": thay đổi ở output layer dưới gây thay đổi tương quan cao ở tổng đầu vào layer trên (đặc biệt ReLU); cố định mean/variance của toàn bộ tổng đầu vào trong layer (trên một sample) giảm nhiễu này mà không cần nhìn sang sample khác.

Điểm quan trọng (Mục 4, Related work): LN **không phải** một phép tái tham số hoá (re-parameterization) của mạng gốc, khác BN/weight norm dùng thống kê kỳ vọng đầy đủ (vốn tương đương cách viết khác của cùng một mạng). Vì khác bản chất, LN có tính bất biến khác hẳn, phân tích ở Mục 3.3.

3. Phương pháp chi tiết

3.1 Công thức LN (công thức 3)

Với layer l có H hidden unit, $a^l = (a_1^l, \ldots, a_H^l)$ là vector tổng đầu vào của layer đó cho **một** training case:

$$\boxed{\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l, \qquad \sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2}} \tag{3}$$

Khác biệt duy nhất so với (2) là **trục lấy trung bình**: BN có μ_i, σ_i riêng cho mỗi unit nhưng dùng chung qua các case trong batch; LN có μ^l, σ^l chung cho mọi unit trong layer, nhưng riêng theo từng case. Dạng tổng quát (công thức 5, áp cho cả BN/WN/LN):

$$h_i = f\Big(\frac{g_i}{\sigma_i}(a_i - \mu_i) + b_i\Big) \tag{5}$$

với g_i, b_i là gain/bias riêng của neuron i (không chia sẻ, khác μ, σ).

3.2 LN cho mạng hồi quy (công thức 4)

Trong RNN chuẩn, $a^t = W_{hh}h^{t-1} + W_{xh}x^t$. LN áp ngay lên a^t:

$$h^t = f\Big(\frac{g}{\sigma^t} \odot (a^t - \mu^t) + b\Big), \qquad \mu^t = \frac{1}{H}\sum_{i=1}^{H} a_i^t, \qquad \sigma^t = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(a_i^t - \mu^t)^2} \tag{4}$$

μ^t, σ^t chỉ phụ thuộc tổng đầu vào **tại chính bước** t, nên chỉ cần **một bộ gain/bias** g, b **duy nhất** dùng chung mọi time-step (khác BN-RNN cần thống kê riêng mỗi bước). Trong RNN chuẩn, biên độ trung bình tổng đầu

vào có xu hướng tăng/co dần theo thời gian (gốc của exploding/vanishing gradient); LN làm mô hình bất biến với việc re-scale toàn bộ tổng đầu vào của layer, nên hidden-to-hidden dynamics ổn định hơn.

Ví dụ tự tính: layer có $H = 4$ unit, tổng đầu vào $a = (2, 4, 6, 8)$.

$$\mu = \frac{2+4+6+8}{4} = 5, \qquad \sigma = \sqrt{\frac{9+1+1+9}{4}} = \sqrt{5} \approx 2{,}236$$

Chuẩn hoá (gain $g = 1$, bias $b = 0$): $(a-\mu)/\sigma \approx (-1{,}342;\ -0{,}447;\ 0{,}447;\ 1{,}342)$ (tự tính). Nhân mọi phần tử của a lên 10 lần, $a' = (20, 40, 60, 80)$: $\mu' = 10\mu$, $\sigma' = 10\sigma$, kết quả chuẩn hoá giữ nguyên bốn số trên (tự tính) - minh hoạ "bất biến với re-scale một training case" (công thức 7, Mục 3.3).

3.3 Bất biến (invariance) dưới biến đổi weight/dữ liệu (Bảng 1)

Ba cách chuẩn hoá theo công thức (5): BN, LN dùng μ, σ theo (2)/(3); **weight normalization** (WN, Salimans và Kingma 2016) có $\mu = 0$, $\sigma = \|w\|_2$ (chuẩn weight vector, không phụ thuộc dữ liệu).

Tính chất bất biến	Batch norm	Weight norm	Layer norm
Re-scale toàn bộ weight matrix	Invariant	Invariant	Invariant
Re-center toàn bộ weight matrix	No	No	Invariant
Re-scale một weight vector (1 neuron)	Invariant	Invariant	No
Re-scale toàn bộ dataset	Invariant	No	Invariant
Re-center toàn bộ dataset	Invariant	No	No
Re-scale một training case đơn lẻ	No	No	Invariant

Hai ý chính (Mục 5.1): **(1) weight matrix re-scale/re-center (công thức 6):** nếu $W' = \delta W + \mathbf{1}\gamma^\top$ (scale δ và shift chung γ cho mọi dòng W), dưới LN hai mô hình cho cùng output, vì scale/shift tác động **đồng nhất lên mọi unit trong layer của cùng một case**: $\mu' = \delta\mu + \gamma^\top x$, $a'_i - \mu' = \delta(a_i - \mu)$, $\sigma' = \delta\sigma$, hai δ triệt tiêu và phần shift bị trừ khi tính $a'_i - \mu'$. BN/WN tính μ, σ riêng theo từng unit nên không có tính chất "triệt tiêu theo unit" này. **(2) re-scale một training case (công thức 7):** $x' = \delta x$ thì $a'_i = \delta a_i$ với mọi i (weight không đổi), nên $\mu' = \delta\mu, \sigma' = \delta\sigma$, output giữ nguyên - đúng như minh hoạ số ở Mục 3.2.

Ngược lại, re-scale **riêng một weight vector** của một neuron chỉ đổi a_i của chính neuron đó, còn $H - 1$ unit khác giữ nguyên; vì μ^l, σ^l gộp chung cả H unit, LN không bất biến với biến đổi này, còn BN/WN (tính μ_i, σ_i riêng theo từng unit, không liên quan unit khác) lại bất biến ngay.

Diễn giải thực hành: bất biến với re-scale một training case đơn lẻ là lý do LN phù hợp xử lý từng mẫu độc lập (batch size 1, decode tự hồi quy): scale của một input không ảnh hưởng output qua layer LN, bất kể input đó lớn/nhỏ ra sao so với các input khác đã thấy.

3.4 Hình học không gian tham số: Fisher information (Mục 5.2)

Mục này lý giải "vì sao chuẩn hoá giúp học ổn định" bằng information geometry. Với một mô hình xác suất, "khoảng cách" giữa hai điểm tham số $\theta, \theta + \delta$ là KL divergence giữa hai phân phối output; khai triển Taylor bậc hai, khoảng cách xấp xỉ bằng dạng toàn phương qua ma trận Fisher $F(\theta)$ (Amari, 1998):

$$ds^2 = D_{KL}\big[P(y \mid x;\theta)\,\|\,P(y \mid x;\theta{+}\delta)\big] \approx \tfrac{1}{2}\,\delta^\top F(\theta)\,\delta, \qquad F(\theta) = \mathbb{E}_{x,y}\Big[\Big(\frac{\partial \log P}{\partial \theta}\Big)\Big(\frac{\partial \log P}{\partial \theta}\Big)^\top\Big] \quad (8{,}9)$$

Trên mô hình tuyến tính tổng quát (GLM, output thuộc họ hàm mũ), ma trận Fisher của GLM chuẩn (chưa chuẩn hoá) có dạng tích Kronecker (công thức 12): $F(\theta) = \mathbb{E}_x\big[\frac{\mathrm{Cov}[y|x]}{\varphi^2} \otimes \left(\begin{smallmatrix} xx^\top & x \\ x^\top & 1 \end{smallmatrix}\right)\big]$. Khi thêm gain và chuẩn

hoá a qua μ, σ, ma trận Fisher $\bar{F}(\theta)$ phức tạp hơn (công thức 13-14, có số hạng χ_i đo mức μ, σ "trôi" theo w_i). Hai kết luận định tính paper nhấn mạnh:

(a) Tăng chuẩn (norm) weight vector tự giảm learning rate hiệu dụng: nếu $\|w_i\|$ tăng gấp đôi (output không đổi vì σ_i cũng tăng gấp đôi bù lại), độ cong (curvature) theo hướng w_i đổi theo hệ số $1/2$ (paper nói nguyên văn). Weight vector lớn dần tự "hãm" tốc độ học của chính nó, tạo hiệu ứng giống early stopping ngầm.

(b) Học gain ít phụ thuộc scale input/weight hơn, dưới BN/LN: công thức (32)-(33) cho thấy ds^2 khi đối gain chỉ phụ thuộc $\text{Cov}[y \mid x]/\varphi^2$, không có x hay norm weight; GLM chuẩn chưa chuẩn hoá (công thức 35) lại phụ thuộc $a_i a_j/(\|w_i\|\|w_j\|)$, tức phụ thuộc scale thật của input/weight.

(Người ghi chú nhận xét: công thức (34) của weight norm vẫn còn số hạng $a_i^2/\|w_i\|^2$ phụ thuộc $a_i = w_i^\top x$, tức WN chưa hẳn "robust" như câu kết luận chung của paper gợi ý, khớp với Bảng 1 cho WN "No" ở cả hai cột liên quan dataset. Ngoài ra, toàn Mục 5.2 chứng minh trên GLM đơn, mở rộng cho DNN chỉ qua xấp xỉ "block-diagonal Fisher theo từng neuron", không phải định lý chặt cho mạng sâu; và hiệu ứng "early-stopping ngầm" ở (a) không gắn riêng với việc LN thực ra không bất biến với re-scale một weight vector đơn lẻ, Mục 3.3, nên nên đọc như minh hoạ trực giác qua GLM, không phải chứng minh riêng cho LN.)

4. Thí nghiệm và kết quả chính

Paper không dùng một benchmark thống nhất; thử LN trên 6-7 tác vụ, chủ yếu RNN. Mặc định khởi tạo: gain $= 1$, bias $= 0$.

#	Tác vụ	Mô hình / dữ liệu	Kết quả chính (so baseline không LN)
6.1	Order embeddings ảnh-câu	GRU + VGG (pretrained) trên MS-COCO	Hội tụ tới best validation model trong 60% thời gian baseline; test set tốt hơn
6.2	Attentive reader (QA)	LSTM trên CNN corpus (Hermann et al. 2015)	Hội tụ nhanh hơn và validation error tốt hơn cả BN-LSTM, BN-everywhere
6.3	Skip-thought vectors	Sentence encoder 2400-dim trên BookCorpus	Tốt hơn baseline trên hầu hết 5 downstream task
6.4	Binarized MNIST (DRAW)	DRAW, 64 glimpse, 256 LSTM unit	Hội tụ gần 2 lần nhanh hơn; log-likelihood test 82,09 nats so với 82,36
6.5	Handwriting generation	3 lớp x 400 LSTM cell, IAM-OnDB	NLL tương đương baseline, hội tụ nhanh hơn rõ rệt
6.6	Permutation-invariant MNIST	MLP 784-1000-1000-10	Robust với batch size (128 và 4), hội tụ nhanh hơn BN áp toàn mạng
6.7	ConvNet (thử nghiệm sơ bộ)	không nêu kiến trúc cụ thể	LN nhanh hơn baseline không chuẩn hoá, nhưng kém hơn BN

6.1 (Bảng 2, trung bình 5 test split, MS-COCO):

	Caption				Image			
Model	R@1	R@5	R@10	Mean r	R@1	R@5	R@10	Mean r
Sym (Vendrov et al. 2016)	45,4	-	88,7	5,8	36,3	-	85,8	9,0
OE (Vendrov et al. 2016)	46,7	-	88,9	5,7	37,9	-	85,9	8,1
OE (ours, không LN)	46,6	79,3	89,1	5,2	37,8	73,6	85,7	7,9

Model	Caption R@1	R@5	R@10	Mean r	Image R@1	R@5	R@10	Mean r
OE + LN	48,5	80,6	89,8	5,1	38,9	74,3	86,3	7,6

(Hai dòng đầu để trống R@5 vì paper gốc Vendrov et al. không báo cáo chỉ số này, không phải lỗi trích xuất.) OE+LN tăng R@1 caption từ 46,6 lên 48,5, tức +1,9 điểm, tương đối $\approx 4{,}1\%$ (tự tính: $1{,}9/46{,}6 \approx 0{,}0408$).

6.2: so LSTM baseline, BN-LSTM, BN-everywhere (lấy từ Cooijmans et al. 2016) và LN-LSTM trên validation error rate tới 800 nghìn bước (Hình 2); LN-LSTM hội tụ nhanh nhất, sai số thấp nhất. Thí nghiệm riêng: LN với gain init $= 1{,}0$ tốt hơn đáng kể so với $= 0{,}1$ (ngược với BN hồi quy của Cooijmans et al., vốn phải chọn $0{,}1$ mới tốt); LN không nhạy scale khởi tạo gain theo kiểu đó.

6.3 (Bảng 3, BookCorpus, checkpoint mỗi 50.000 bước):

Model	SICK r	SICK ρ	SICK MSE	MR	CR	SUBJ	MPQA
Original (Kiros et al. 2015)	0,848	0,778	0,287	75,5	79,3	92,1	86,9
Ours (không LN)	0,842	0,767	0,298	77,3	81,8	92,6	87,9
Ours + LN (1M bước)	0,854	0,785	0,277	79,5	82,6	93,4	89,0
Ours + LN, 1 tháng ($\approx$ 1,7M bước)	0,858	0,788	0,270	79,4	83,1	93,7	89,3

Train thêm (1M $\rightarrow$ 1,7M bước) cải thiện mọi task **trừ MR** (79,5 $\rightarrow$ 79,4), khớp câu paper "gains across all but one task". Original so Ours+LN(1M) trên CR: $+3{,}3$ điểm, tương đối $\approx 4{,}2\%$ (tự tính: $3{,}3/79{,}3 \approx 0{,}0416$).

6.4: sau 200 epoch, baseline đạt $82{,}36$ nats, LN đạt $82{,}09$ nats (thấp hơn tốt hơn), chênh $0{,}27$ nats, tương đối $\approx 0{,}33\%$ (tự tính: $0{,}27/82{,}36 \approx 0{,}00328$), kèm hội tụ nhanh gần gấp đôi.

6.5: $\approx$ 3,7 triệu tham số (kiến trúc Graves 2013), minibatch 8, sequence length 500; LN đạt log-likelihood tương đương baseline với ít update hơn rõ rệt.

6.6: MLP 784-1000-1000-10, Adam, 55.000 ảnh train; so BN, LN ở batch 128 và 4 (BN batch nhỏ dùng ước lượng variance không chệch). LN hội tụ nhanh ở cả hai batch size; BN áp toàn mạng kém ổn định hơn ở batch 4.

6.7: quan sát sơ bộ, không có bảng số: LN nhanh hơn baseline không chuẩn hoá trên ConvNet, nhưng BN vẫn vượt trội hơn LN.

5. Phân tích / ablation

- **Batch-size robustness (Hình 6, Mục 6.6):** ablation trung tâm. Ở batch 128, BN và LN gần tương đương; ở batch 4, BN (dù đã dùng ước lượng variance không chệch) suy giảm rõ, LN giữ nguyên tốc độ hội tụ, bằng chứng thực nghiệm trực tiếp cho tuyên bố ở Mục 3.
- **Nhạy cảm scale khởi tạo gain (Mục 6.2):** BN hồi quy cần gain init $= 0{,}1$ để tốt (chi tiết triển khai "mong manh"); LN tốt hơn với gain init $= 1{,}0$, ít nhạy hơn lựa chọn này.
- **So 3 kiểu chuẩn hoá cùng lúc (Hình 4, Mục 6.4):** DRAW baseline, Weight Norm, Layer Norm so trực tiếp trên cùng bài toán sinh ảnh; LN vẫn vượt baseline, bổ sung so sánh ba chiều.
- **ConvNet là trường hợp thất bại tương đối (Mục 6.7, liên hệ Mục 3.3):** với fully-connected layer, các unit trong layer đóng góp "giống nhau" vào output, re-center/re-scale chung cả layer hợp lý. Với ConvNet, nhiều unit gần biên ảnh hiếm khi kích hoạt, thống kê khác hẳn phần còn lại layer; giả định "vai trò tương tự giữa các unit" không còn đúng, khiến gộp chung μ, σ qua cả layer kém hiệu quả hơn BN (chuẩn hoá theo channel, qua batch và không gian).

6. Giới hạn

Paper tự nêu:

- LN kém hơn BN trên ConvNet (”further research is needed to make layer normalization work well in ConvNets”, Mục 6.7), hạn chế tác giả tự công nhận rõ nhất.
- Một số thí nghiệm (6.2, 6.3) chỉ báo cáo trên validation set, theo đúng protocol các bài gốc so sánh.
- Phân tích Fisher (Mục 5.2) là xấp xỉ bậc hai của KL-divergence, chỉ chứng minh chặt cho GLM đơn; với mạng sâu chỉ ”có thể áp dụng” qua xấp xỉ block-diagonal.

Người ghi chú nhận xét:

- Công thức (3), (4), (15), (16) không có hằng số ϵ cộng vào σ để chống chia cho 0 (luôn có trong mọi implementation hiện đại, ví dụ tham số `eps` của PyTorch `LayerNorm`). Nếu H unit của một sample có giá trị gần bằng nhau (variance rất nhỏ), công thức gốc có nguy cơ chia gần 0, một khoảng trống thực hành.
- **Mâu thuẫn nhỏ văn bản/hình (Mục 6.4):** văn bản nói Hình 4 vẽ ”the first 100 epoch”, nhưng câu ngay sau trích số liệu ”After 200 epoches” (82,36/82,09 nats). Hai số này nằm ngoài phạm vi epoch mà Hình 4 thực sự vẽ; có thể lỗi đánh máy (100 thay vì 200) hoặc model được train tiếp sau khi hình đã chụp, paper không giải thích.
- Tập thí nghiệm gần như toàn RNN/LSTM/GRU cho NLP (chuỗi 10-40, trừ handwriting dài hơn) cộng một mô hình sinh ảnh nhỏ và một MLP nhỏ; không kiến trúc nào gần Transformer (bài ra đời một năm trước Attention Is All You Need), nên không nói trực tiếp về LN trong self-attention hay huấn luyện quy mô tỷ tham số.
- Không có nghiên cứu overhead tính toán của LN so với BN, hay ảnh hưởng tới suy luận ở độ chính xác thấp (FP16/FP8); các câu hỏi này chỉ quan trọng ở các bài sau (Mục 7, 8).

7. Liên hệ với các bài khác

- **opt 38 (Batch Normalization):** bài gốc LN ”transpose” trục chuẩn hoá; công thức (2)-(5), Bảng 1 của LN đều định nghĩa qua đối chiếu trực tiếp với BN, nên đọc trước bài này.
- **opt 44 (How Does Batch Normalization Help Optimization?):** phản biện chính cách lý giải ”giảm covariate shift” mà cả BN gốc và LN (Mục 1-2) dùng làm động lực, cho rằng lợi ích thật nằm ở làm phẳng loss landscape; nên đọc lại Mục 1-2 bài này với góc nhìn phản biện đó.
- **opt 42 (Mixed Precision Training):** khi huấn luyện FP16, phép reduction tính mean/variance như công thức (3) cần accumulate ở FP32 để tránh lỗi số học; LN với H lớn (hidden size LLM hiện đại) hay được giữ FP32 ngay trong pipeline FP16/BF16.
- **llm 09 (Pre-LN Transformer):** đặt LayerNorm trước block attention/FFN (”Pre-LN”) để ổn định huấn luyện Transformer sâu, một lý do LN thành chuẩn hoá mặc định của Transformer, thay BN.
- **llm 10 (RMSNorm):** đơn giản hoá trực tiếp công thức (3): bỏ phần re-centering (không trừ μ), chỉ chia theo root-mean-square của a; biến thể hầu hết LLM/VLM hiện đại (LLaMA, Mistral, Qwen...) dùng thay LN gốc.

8. Ứng dụng cho dự án (suy luận)

Dự án của bạn là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, $\approx$ 3.600 token/trang gồm 1.500 text + 1.6-2.1K visual, GPU $\leq$ 8 GB, batch 1). Đây là bài liên quan trực tiếp, vì hai lý do cốt lõi:

1. **Batch size 1 đúng chế độ LN sinh ra để giải quyết.** Mục 1-2: BN ”không rõ cách áp dụng cho RNN” và ”không dùng được cho online learning khi mini-batch buộc phải nhỏ”. Serving/fine-tune batch 1 trên GPU 8 GB chính là chế độ đó. Hình 6 (Mục 6.6, batch 4 vs 128) cho bằng chứng BN suy giảm khi batch nhỏ, LN thì không, vì μ, σ chỉ cần một sample. Đây cũng là lý do gần như mọi LLM/VLM hiện

đại dùng LayerNorm/RMSNorm chứ không BatchNorm: suy luận production luôn cần hoạt động tốt ở batch nhỏ (decode tự hồi quy từng token, OCR xử lý từng trang).

2. **Train/test giống nhau, không cần running statistics.** BN cần running mean/var tích luỹ qua quá trình train để dùng ở test; pipeline pruning + distillation liên tục đổi cấu trúc mạng sẽ làm running statistics đó lệch ngay khi cấu trúc đổi. LN không có vấn đề này, giúp pipeline nén/fine-tune đơn giản hơn khi kiến trúc thay đổi.
3. **GatedDeltaNet kế thừa trực tiếp động lực RNN của bài này.** Mục 3.1-3.2: LN chặn hiện tượng biên độ hidden state phình/co qua từng bước thời gian. GatedDeltaNet cập nhật state tuần tự qua hàng nghìn token/trang, về bản chất là "RNN tuyến tính có gate"; nhiều khả năng LightOnOCR dùng RMSNorm (biến thể LN, llm 10) ngay sau mỗi bước state-update hoặc quanh gate, đúng vai trò "ổn định hidden-to-hidden dynamics" mà Mục 3.2 mô tả.
4. **Khi pruning, cẩn trọng với H trong công thức (3).** Nếu bước nén cắt bớt hidden unit (giảm H) mà vẫn giữ lớp norm cũ, μ^l, σ^l tính trên ít unit hơn, phân phối activation đưa vào norm đổi; nên fine-tune/recalibrate lại gain g, bias b sau pruning thay vì giữ nguyên tham số norm cũ với H mới.

Điều ít liên quan: phân tích hình học Fisher (Mục 3.4) là lý thuyết nền, không cho công thức hành động cụ thể cho việc nén mô hình; thí nghiệm ConvNet (Mục 6.7) không áp dụng cho backbone attention/SSM như của bạn.

9. Thí nghiệm gợi ý

1. **Tái hiện Hình 6 ở quy mô nhỏ:** MLP nhỏ (784-256-256-10) trên MNIST, ba biến thể không chuẩn hoá/BatchNorm/LayerNorm, batch $\in \{1, 4, 128\}$. Đo train loss/test error theo epoch. Kỳ vọng: BN suy giảm rõ (đặc biệt batch 1), LN ổn định ở mọi batch size.
2. **Ổn định hidden state LSTM theo thời gian:** LSTM nhỏ trên tác vụ chuỗi dài (copy task hoặc char-level LM) với/không LN ở gate/cell (công thức 20-22, phụ lục). Đo $\|h^t\|$ và gradient norm theo t qua vài trăm bước; kỳ vọng bản không-LN có norm trôi rõ rệt, bản LN ổn định hơn.
3. **Kiểm tra giới hạn "thiếu epsilon":** cài LayerNorm hai phiên bản (không ϵ đúng công thức gốc, và có $\epsilon = 10^{-5}$), cho input variance rất nhỏ chạy ở FP16/FP32, quan sát khi nào bản không-ϵ cho NaN/Inf.
4. **Recalibrate gain/bias sau khi "prune" hidden unit:** lấy MLP nhỏ đã train với LayerNorm, cắt 50% hidden unit một layer giữa, so accuracy giữ nguyên gain/bias cũ với accuracy sau khi fine-tune lại vài bước ngắn. Kỳ vọng giữ nguyên tham số norm cũ cho kết quả tệ hơn.
5. **So LayerNorm với RMSNorm:** cài hai lớp norm, một có trừ μ (công thức 3), một bỏ μ (chỉ chia root-mean-square), cùng kiến trúc/siêu tham số, đo tốc độ hội tụ và loss cuối. Kỳ vọng hai phiên bản gần tương đương.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Ghi chú
Tổng đầu vào	$a_i^l = w_i^{l\top} h^l$	công thức (1), trước phi tuyến
BN (theo batch)	$\bar{a}_i = \frac{g_i}{\sigma_i}(a_i - \mu_i)$, μ_i, σ_i qua $x \sim P(x)$	công thức (2), ước lượng bằng mini-batch
LN (theo layer)	$\mu^l = \frac{1}{H}\sum_i a_i^l$, $\sigma^l = \sqrt{\frac{1}{H}\sum_i (a_i^l - \mu^l)^2}$	công thức (3), trên H unit của một sample
LN cho RNN	$h^t = f\big(\frac{g}{\sigma^t} \odot (a^t - \mu^t) + b\big)$	công thức (4), μ^t, σ^t chỉ từ bước t
Dạng chung 3 phương pháp	$h_i = f\big(\frac{g_i}{\sigma_i}(a_i - \mu_i) + b_i\big)$	công thức (5); WN: $\mu = 0, \sigma = \|w\|_2$
Bất biến weight matrix	re-scale + re-center	chỉ LN có cả hai (công thức 6)

Khái niệm	Công thức	Ghi chú
Bất biến dữ liệu	re-scale 1 training case	chỉ LN (công thức 7); BN/WN bất biến re-scale/re-center cả dataset
Riemannian metric	$ds^2 \approx \frac{1}{2}\delta^\top F(\theta)\delta$	công thức (8), xấp xỉ KL bậc hai qua Fisher info
Hiệu ứng early-stopping ngầm	$\|w_i\| \times 2 \Rightarrow$ curvature $\times \frac{1}{2}$	Mục 5.2.2, phát biểu của paper
Robust scale khi học gain	BN/LN: $ds^2 \propto \mathrm{Cov}[y \mid x]/\varphi^2$ (không phụ thuộc x)	công thức (32)-(33), đối lập công thức (35)

11. Kết luận ngắn

Layer Normalization chuẩn hoá tổng đầu vào của neuron theo chiều H (số hidden unit trong layer), tính riêng cho từng training case, thay vì theo chiều batch như Batch Normalization. Đổi trục này giúp LN không phụ thuộc batch size (dùng được ở batch 1), tính giống nhau ở train/test, và áp dụng tự nhiên cho RNN. Phân tích bất biến (Bảng 1) cho thấy LN có tập tính chất khác hẳn BN/weight norm; phân tích hình học Fisher đưa lý giải định tính cho việc chuẩn hoá giúp học ổn định hơn. Thực nghiệm trên 6-7 tác vụ (chủ yếu RNN cho NLP, cộng DRAW và MNIST) cho thấy LN tăng tốc hội tụ và thường cải thiện kết quả cuối, trừ ConvNet nơi BN vẫn vượt trội. Những năm sau, LN (và biến thể RMSNorm) trở thành lớp chuẩn hoá mặc định của Transformer và phần lớn LLM/VLM hiện đại, chính nhờ tính không-phụ-thuộc-batch là mục tiêu thiết kế trung tâm của bài báo này.

Nguồn gốc: https://arxiv.org/abs/1607.06450

Bài 40

On the difficulty of training Recurrent Neural Networks (2012)

Tác giả: Razvan Pascanu (Université de Montréal), Tomas Mikolov (Brno University of Technology), Yoshua Bengio (Université de Montréal).
Venue / arXiv: arXiv:1211.5063 (v2, 16/02/2013; ID arXiv tháng 11/2012); paper không nêu venue (không có dòng ”Proceedings of...” hay copyright hội nghị ở đầu bài trong bản text tải được).
Nguồn: https://arxiv.org/abs/1211.5063
Chủ đề: phân tích nguyên nhân (đại số, hình học, hệ động lực) của vanishing/exploding gradient khi huấn luyện RNN bằng BPTT; đề xuất gradient norm clipping cho exploding gradient và một regularization term (soft constraint) cho vanishing gradient.
Vị trí trong lộ trình: đọc sau opt 37 (Delving Deep into Rectifiers, một hướng khác chống gradient không ổn định bằng khởi tạo trọng số) và có thể đọc cùng opt 06 (Adam, optimizer thích ứng cũng giảm nhẹ vấn đề này); đọc trước opt 41 (giải thích lý thuyết chặt hơn cho chính kỹ thuật clipping giới thiệu ở bài này).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Một **recurrent neural network (RNN)** nhận input u_t và duy trì state x_t qua thời gian:

$$x_t = F(x_{t-1}, u_t, \theta) \tag{1}$$

Paper dùng xuyên suốt phần lý thuyết một tham số hóa cụ thể (công thức 11, tương đương $x_t = \sigma(W_{rec}x_{t-1} + W_{in}u_t + b)$ nhưng viết dưới dạng thuận tiện hơn cho đạo hàm):

$$x_t = W_{rec}\,\sigma(x_{t-1}) + W_{in}u_t + b \tag{2}$$

với $\theta = \{W_{rec}, W_{in}, b\}$, σ là hàm phi tuyến theo từng phần tử (tanh hoặc sigmoid), x_0 cho trước hoặc học. Chi phí tổng $E = \sum_{1\le t\le T} E_t$ với $E_t = L(x_t)$.

Để tính gradient, paper dùng **Backpropagation Through Time (BPTT)**: RNN được ”trải” (unroll) thành một mạng nhiều lớp, mỗi lớp một bước thời gian, rồi backprop như mạng feedforward (số lớp không giới hạn vì chiều dài sequence có thể rất lớn). Hai vấn đề cổ điển, nêu lần đầu ở Bengio et al. (1994): **vanishing gradient** (gradient tắt về 0 theo cấp số nhân khi lan truyền ngược, mô hình không học được phụ thuộc xa) và **exploding gradient** (gradient tăng vọt theo cấp số nhân, bước cập nhật phá hỏng quá trình học).

Paper viết lại công thức gradient dưới dạng tổng-của-tích để làm rõ hai hiện tượng này:

$$\frac{\partial E}{\partial \theta} = \sum_{1\le t\le T} \frac{\partial E_t}{\partial \theta} \tag{3}$$

$$\frac{\partial E_t}{\partial \theta} = \sum_{1\le k\le t} \frac{\partial E_t}{\partial x_t}\frac{\partial x_t}{\partial x_k}\frac{\partial^+ x_k}{\partial \theta} \tag{4}$$

$$\frac{\partial x_t}{\partial x_k} = \prod_{t \geq i > k} \frac{\partial x_i}{\partial x_{i-1}} = \prod_{t \geq i > k} W_{rec}^T \, \text{diag}(\sigma'(x_{i-1})) \tag{5}$$

Ký hiệu $\partial^+ x_k / \partial \theta$ là đạo hàm "trực tiếp" (coi x_{k-1} là hằng số theo θ). Mỗi số hạng $\frac{\partial E_t}{\partial x_t} \frac{\partial x_t}{\partial x_k} \frac{\partial^+ x_k}{\partial \theta}$ trong (4) gọi là một **temporal contribution**: đo mức θ tại bước k ảnh hưởng tới cost tại bước $t > k$. Khoảng cách $t - k$ lớn gọi là **long term**, nhỏ là **short term**. Nhân tố $\partial x_t / \partial x_k$ (công thức 5) là tích của $t - k$ ma trận Jacobian; giống như tích của $t - k$ số thực có thể co về 0 hoặc nổ ra vô hạn, tích ma trận này cũng vậy theo một hướng v nào đó. Câu hỏi của bài là: khi nào tích này co lại (vanishing), khi nào nó nổ (exploding), và làm gì để kiểm soát cả hai.

2. Ý tưởng chính

Paper tiếp cận vấn đề từ ba góc nhìn rồi suy ra hai giải pháp kỹ thuật:

1. **Góc nhìn đại số (eigenvalue):** với σ tuyến tính hóa cục bộ, điều kiện vanishing/exploding quy về so sánh giá trị riêng lớn nhất (spectral radius) λ_1 của W_{rec} với một ngưỡng $1/\gamma$, trong đó γ chặn trên $|\sigma'(x)|$.
2. **Góc nhìn hệ động lực (dynamical systems):** exploding gradient tương ứng với việc trạng thái "nhảy" qua ranh giới giữa hai basin of attraction (hoặc một bifurcation boundary); một thay đổi rất nhỏ ở input/tham số có thể tạo ra một thay đổi rất lớn ở state cuối.
3. **Góc nhìn hình học (error surface):** khi gradient nổ, độ cong (curvature) của error surface theo hướng đó cũng nổ cùng lúc, tạo ra một "vách đứng" (wall) hẹp. Nếu đúng vậy, chỉ cần **giới hạn độ dài bước đi** (không cần đổi hướng bằng phương pháp bậc hai) là đủ để tránh nhảy ra ngoài vùng "thung lũng" quanh vách.

Từ quan sát (3), paper đề xuất **gradient norm clipping**: khi chuẩn (norm) của gradient toàn phần vượt một ngưỡng, scale lại toàn bộ vector gradient để norm đúng bằng ngưỡng, giữ nguyên hướng. Đây là giải pháp cho exploding gradient: đơn giản, rẻ, không cần đổi kiến trúc.

Cho vanishing gradient, paper đề xuất một **regularization term** (soft constraint) khuyến khích norm của Jacobian $\partial x_{k+1} / \partial x_k$ không co lại quá nhiều theo hướng mà error signal thực sự lan truyền, để tín hiệu lỗi "sống sót" đủ lâu khi đi ngược về quá khứ.

3. Phương pháp chi tiết

3.1 Điều kiện đại số cho vanishing/exploding (§2.1)

Giả sử σ bị chặn đạo hàm: $|\sigma'(x)| \leq \gamma$, do đó $\|\text{diag}(\sigma'(x_k))\| \leq \gamma$ (chuẩn 2 trên ma trận). Với tanh, $\gamma = 1$; với sigmoid, $\gamma = 1/4$ (đạo hàm sigmoid lớn nhất tại $x = 0$ bằng 0.25).

Điều kiện đủ cho vanishing gradient. Giả sử $\lambda_1 < 1/\gamma$, với λ_1 là giá trị tuyệt đối của giá trị riêng lớn nhất của W_{rec} (dùng tham số hóa công thức 2, Jacobian $\partial x_{k+1} / \partial x_k = W_{rec}^T \text{diag}(\sigma'(x_k))$). Khi đó:

$$\left\| \frac{\partial x_{k+1}}{\partial x_k} \right\| \leq \|W_{rec}^T\| \cdot \|\text{diag}(\sigma'(x_k))\| \leq \lambda_1 \gamma < 1 \tag{6}$$

Đặt $\eta = \lambda_1 \gamma < 1$ là chặn trên chung cho mọi k. Bằng quy nạp:

$$\left\| \frac{\partial E_t}{\partial x_t} \left(\prod_{i=k}^{t-1} \frac{\partial x_{i+1}}{\partial x_i} \right) \right\| \leq \eta^{t-k} \left\| \frac{\partial E_t}{\partial x_t} \right\| \tag{7}$$

Vì $\eta < 1$, các long term contribution ($t - k$ lớn) co về 0 theo cấp số nhân theo $t - k$.

Điều kiện cần cho exploding gradient: đảo lại chứng minh trên, nếu không (6) không đủ thì các thành phần long-term sẽ vanish thay vì explode, nên **cần** $\lambda_1 > 1/\gamma$ để gradient có thể nổ. Với tanh ($\gamma = 1$): cần $\lambda_1 > 1$; với sigmoid ($\gamma = 1/4$): cần $\lambda_1 > 4$.

Phụ lục ("Analytical analysis...") chứng minh điều kiện cần này chặt hơn bằng power iteration cho trường hợp tuyến tính ($\sigma = \text{id}$). Với $l = t - k$: $\partial x_t/\partial x_k = (W_{rec}^T)^l$ (công thức 12). Viết $\partial E_t/\partial x_t = \sum_{i=1}^n c_i q_i^T$ theo cơ sở riêng q_i (giá trị riêng $\lambda_1 > \cdots > \lambda_n$ của W_{rec}). Nếu j là chỉ số nhỏ nhất với $c_j \neq 0$, dùng $q_i^T W_{rec}^{Tl} = \lambda_i^l q_i^T$:

$$\frac{\partial E_t}{\partial x_t}\frac{\partial x_t}{\partial x_k} = c_j \lambda_j^l q_j^T + \lambda_j^l \sum_{i=j+1}^{n} c_i \frac{\lambda_i^l}{\lambda_j^l} q_i^T \approx c_j \lambda_j^l q_j^T \tag{13}$$

vì $(\lambda_i/\lambda_j)^l \to 0$ khi $l \to \infty$ với $i > j$. Nếu $|\lambda_j| > 1$, số hạng này nổ theo hướng q_j. Mở rộng cho toàn gradient (công thức 14): chọn j, k sao cho $c_j q_j^T \, \partial^+ x_k/\partial\theta \neq 0$ và $|\lambda_j|$ lớn nhất có thể; nếu $|\lambda_j| > 1$ số hạng này áp đảo tổng và kéo cả $\partial E_t/\partial\theta$ nổ theo t. Đây là điều kiện **cần** (spectral radius của W_{rec} phải > 1), không phải điều kiện đủ tổng quát cho mọi σ phi tuyến.

3.2 Góc nhìn hệ động lực (§2.2)

Với $\sigma = \text{id}$, hệ tự hồi quy không input là một **dynamical system**: state x_t hội tụ về một trong các **attractor**, mỗi attractor có một **basin of attraction**; khi θ đổi chậm, hành vi tiệm cận đổi trơn trừ tại các **bifurcation boundary** (attractor xuất hiện/biến mất/đổi dạng). Paper dùng lại ví dụ một hidden unit của Doya (1993) (Hình 3, recurrent weight cố định $= 5.0$, bias b thay đổi): trong khoảng (b_1, b_2) có hai attractor; ranh giới giữa hai basin và hai bifurcation boundary tại b_1, b_2 là hai loại điểm "nhạy cảm".

Hai quan sát chính: (a) vượt ranh giới giữa hai basin là sự kiện **cục bộ** và **đủ** để gradient nổ (thay đổi Δb rất nhỏ gây Δx_∞ rất lớn vì state bị hút về attractor khác); (b) vượt bifurcation boundary thì **không cần cũng không đủ** để gradient nổ, vì đó là sự kiện toàn cục, có thể không ảnh hưởng cục bộ nếu state ở basin của attractor không đổi dạng khi crossing. Paper mở rộng quan sát của Doya (chỉ nói bifurcation gây nổ) thành điều kiện đủ rộng hơn: crossing boundary giữa hai basin (kể cả không phải bifurcation) là đủ để gradient nổ.

Với mô hình có input, paper phân rã $F_t(x) = W_{rec}\sigma(x) + b + W_{in}u_t = \tilde{F}(x) + U_t(x)$: $\tilde{F}$ cố định (phân tích được bằng lý thuyết hệ động lực chuẩn), U_t biến đổi theo thời gian do input, có thể đẩy state qua basin boundary của $\tilde{F}$. Quan sát bổ sung cho vanishing: nếu $\partial x_t/\partial x_k \to 0$ khi $t - k$ lớn, x_t không còn phụ thuộc x_k, tức đã gần hội tụ về một attractor. Vậy **vanishing** ứng với state gần tâm basin (ổn định), **exploding** ứng với state gần ranh giới basin (không ổn định).

3.3 Góc nhìn hình học (§2.3)

Xét mô hình một hidden unit $x_t = w\sigma(x_{t-1}) + b$ (công thức 8), huấn luyện để đạt giá trị mục tiêu sau 50 bước, $x_0 = 0.5$, σ sigmoid. Hình 6 vẽ error surface $E_{50} = (\sigma(x_{50}) - 0.7)^2$ theo (w, b), cho thấy các "vách đứng" (steep walls) rất cao.

Để phân tích định lượng, paper đơn giản về trường hợp tuyến tính: $\sigma = \text{id}$, $b = 0$, nên $x_t = x_0 w^t$, do đó:

$$\frac{\partial x_t}{\partial w} = t x_0 w^{t-1}, \qquad \frac{\partial^2 x_t}{\partial w^2} = t(t-1) x_0 w^{t-2}$$

Cả hai chứa thừa số w^{t-1} hoặc w^{t-2}: khi đạo hàm bậc một "nổ", đạo hàm bậc hai cũng nổ cùng cấp. Tổng quát hơn (với trường hợp tuyến tính, hướng nổ v là eigenvector của giá trị riêng lớn nhất của W_{rec}), paper giả thuyết (chưa chứng minh chặt cho σ phi tuyến tổng quát) rằng curvature theo hướng v cũng nổ cùng lúc, tạo "vách" vuông góc với gradient. Hệ quả thực hành: một bước gradient thường (không chặn) tại gần vách sẽ nhảy xuyên "thung lũng" ra khỏi vùng tốt, phá hỏng việc học; **giới hạn độ dài bước** (clip norm) chỉ đẩy trở lại vùng mượt sát vách, nơi SGD vẫn khám phá được các hướng khác. Vì "thung lũng" quanh vách được

giả định **rộng**, chỉ cần clipping bậc một là đủ, không cần phương pháp bậc hai; paper còn dùng giả thuyết này để giải thích vì sao Hessian-Free hoạt động tốt với RNN (dùng Hessian đầy đủ, ước lượng lại mỗi bước, bắt được thay đổi curvature đột ngột).

3.4 Các giải pháp trước đó (§3.1, điểm qua)

- **L1/L2 penalty trên W_{rec}:** giữ spectral radius < 1 nên ngăn exploding, nhưng giới hạn mô hình về chế độ chỉ một attractor tại gốc, thông tin phải tắt theo cấp số nhân, không giữ được long-term memory.
- **Pre-programming / teacher forcing (Doya, 1993):** khởi tạo mô hình sẵn ở chế độ tiệm cận đúng, hoặc dùng target thật thay output mỗi bước; nhược điểm cần biết trước chế độ cần có, và vẫn không ngăn được vượt basin boundary.
- **LSTM (Hochreiter & Schmidhuber, 1997):** thêm unit tự hồi quy cố định $= 1$ cùng cổng input/output, giải quyết vanishing qua kiến trúc nhưng **không trực tiếp** xử lý exploding.
- **Hessian-Free + structural damping (Sutskever et al., 2011):** dùng toàn bộ Hessian, cập nhật lại mỗi bước; structural damping ép $\partial x_t/\partial\theta$ nhỏ, giúp cả exploding gradient.
- **Echo State Networks / leaky units:** không học W_{rec}, W_{in} (spectral radius < 1 sẵn), tránh cả hai vấn đề nhưng khó học long-term dependency; leaky units là bộ lọc thông thấp.
- **Clipping kiểu Mikolov (luận án PhD, 2012):** clip **từng phần tử** khi vượt ngưỡng tuyệt đối, tiền đề trực tiếp cho bài này; khác biệt là bài này clip **theo norm toàn vector**, dù "cả hai biến thể hoạt động tương tự trong thực tế".

3.5 Gradient norm clipping (§3.2, Algorithm 1)

```
Algorithm 1: norm clipping gradient khi nổ
  g_hat <- dE/dtheta
  if ||g_hat|| >= threshold then
      g_hat <- (threshold / ||g_hat||) * g_hat
  end if
```

Khác với clip-từng-phần-tử của Mikolov, cách này rescale **toàn vector gradient** theo một hệ số chung, nên hướng cập nhật luôn giữ nguyên là **hướng giảm (descent direction)** đối với mini-batch hiện tại; đây là lý do tác giả nêu để "cho nền tảng lý thuyết tốt hơn" so với bản gốc, dù hai biến thể hoạt động tương tự trong thực nghiệm.

Hạn chế được nêu: thêm một siêu tham số, **threshold**. Heuristic đề xuất: nhìn vào thống kê norm gradient trung bình qua nhiều update để chọn ngưỡng. Thực nghiệm của tác giả cho thấy với cùng task/kích thước mô hình, việc huấn luyện không nhạy với giá trị cụ thể của threshold, và thuật toán vẫn hoạt động tốt với threshold khá nhỏ. Có thể coi clipping như một dạng **điều chỉnh learning rate tự động** theo norm gradient tại mỗi bước; khác các phương pháp thích ứng learning rate dựa trên thống kê gradient tích lũy (ví dụ Duchi et al. 2011, AdaGrad), clipping dùng **gradient tức thời**, nên phản ứng được với các thay đổi norm rất đột ngột mà các phương pháp thống kê tích lũy không theo kịp.

Ví dụ số (tự tính). Giả sử tại một bước, gradient (thu gọn về 2 chiều để minh họa) là $\hat{g} = (3, 4)$, vậy $\|\hat{g}\| = \sqrt{3^2 + 4^2} = 5$. Đặt threshold $= 2$. Vì $5 \geq 2$, áp dụng Algorithm 1:

$$\hat{g} \leftarrow \frac{2}{5}(3, 4) = (1.2,\ 1.6)$$

Kiểm tra lại: $\|(1.2, 1.6)\| = \sqrt{1.2^2 + 1.6^2} = \sqrt{1.44 + 2.56} = \sqrt{4} = 2$, đúng bằng threshold, hướng $(3, 4)/5 = (0.6, 0.8)$ được giữ nguyên. Nếu thay vào đó $\hat{g} = (1, 1)$ ($\|\hat{g}\| = \sqrt{2} \approx 1.41 < 2$), điều kiện `if` không thỏa, gradient giữ nguyên không đổi.

3.6 Regularization term cho vanishing gradient (§3.3)

Ý tưởng: ưu tiên các θ sao cho error signal lan truyền ngược **không co cũng không giãn quá nhiều** về norm. Vì tăng norm $\partial x_t/\partial x_k$ đồng nghĩa error tại t nhạy hơn với mọi input $u_k, \dots, u_{t-1}$ (một số trong đó có thể là nhiễu không liên quan), paper lập luận: không thể vừa tăng norm Jacobian vừa luôn đi theo hướng giảm cost E (một phương pháp bậc hai sẽ không tự làm điều này), nên cần ép bằng một **regularization term** riêng:

$$\Omega = \sum_k \Omega_k = \sum_k \left(\frac{\left\| \frac{\partial E}{\partial x_{k+1}} \frac{\partial x_{k+1}}{\partial x_k} \right\|}{\left\| \frac{\partial E}{\partial x_{k+1}} \right\|} - 1 \right)^2 \tag{9}$$

Để tính hiệu quả, chỉ dùng đạo hàm ”trực tiếp” của Ω theo W_{rec} (coi x_k và $\partial E/\partial x_{k+1}$ là hằng số khi lấy đạo hàm của Ω_k):

$$\frac{\partial^+ \Omega}{\partial W_{rec}} = \sum_k \frac{\partial^+ \Omega_k}{\partial W_{rec}} \tag{10}$$

(biểu thức cụ thể dùng lại $\partial E/\partial x_{k+1}$ đã có sẵn từ BPTT nên không cần tính lại; paper dùng Theano để autodiff biểu thức này.) Hai điểm paper tự lưu ý: (1) ràng buộc chỉ áp vào **hướng của error signal** $\partial E/\partial x_{k+1}$ đang truyền, không ép mọi eigenvalue của Jacobian về gần 1; (2) đây là **soft constraint** (một số hạng cộng vào loss, không phải ràng buộc cứng), nên **không đảm bảo** gradient không nổ. Thật vậy, dưới góc nhìn hệ động lực: ngăn vanishing gradient nghĩa là đẩy state ra xa attractor hơn (không hội tụ quá nhanh, vì nếu hội tụ thì gradient vanish), tức là đẩy state **gần basin boundary hơn**, làm **tăng khả năng gradient nổ**. Vì vậy regularization này thường phải **đi kèm** clipping (mục 3.5) để kiểm soát phần exploding mà chính nó có thể gây ra; paper gọi cấu hình kết hợp cả hai là **SGD-CR** (Clipping + Regularization).

4. Thí nghiệm và kết quả chính

4.1 Bài toán tổng hợp bệnh lý (pathological, §4.1, theo thiết lập của Martens & Sutskever 2011, gốc từ Hochreiter & Schmidhuber 1997)

Ba biến thể thuật toán so sánh: **SGD** (thường), **SGD-C** (SGD + clipping), **SGD-CR** (SGD + clipping + regularization). Tiêu chí thành công: lỗi $\leq 1\%$ trên 10.000 mẫu test (hoặc sai số < 0.04 với bài toán hồi quy).

- **Temporal order problem** (bài toán điển hình): chuỗi symbol rời rạc dài T, hai vị trí ngẫu nhiên mang symbol $\in \{A, B\}$, phải đoán đúng thứ tự (AA/AB/BA/BB) cuối chuỗi. Hình 7: SGD-CR đạt **100% thành công tới chiều dài 200** (độ dài tối đa dùng trong Martens & Sutskever 2011), huấn luyện **một mô hình duy nhất** cho các độ dài 50-200, tổng quát hóa tới **400 bước** (gấp đôi) lúc test. Với chuỗi dài hơn 20 bước, **vanishing gradient đã đủ** để cả SGD và SGD-C thất bại (clipping chỉ xử lý exploding); phải có regularization mới vượt ngưỡng này.
- **Các bài toán bệnh lý khác** (cũng Hochreiter & Schmidhuber 1997): addition, multiplication, 3-bit temporal order, random permutation, noiseless memorization (pattern nhị phân dài 5 bit, và biến thể mở rộng pattern dài 10 ký hiệu lấy từ bảng 5 ký hiệu, tức hơn 20 bit thông tin). SGD-CR đạt 100% trên tất cả **trừ** random permutation (chỉ **1/8 lần thử** thành công, bài khó nhất). Bốn bài đầu dùng một mô hình duy nhất cho chiều dài tới 200; noiseless memorization dùng mô hình riêng cho mỗi chiều dài (50, 100, 150, 200). Paper báo cáo vượt Martens & Sutskever (2011, Hessian-Free) về tỉ lệ thành công và/hoặc độ dài chuỗi trên hầu hết các bài.
- Siêu tham số điển hình: addition problem 50 hidden units tanh, lr $= 0.01$, $\alpha = 0.5$, ngưỡng clip $= 6$, khởi tạo $\mathcal{N}(0, 0.1^2)$; temporal order 50 hidden units, lr $= 0.001$, $\alpha = 2$, clip $= 6$; 3-bit temporal order tương tự nhưng 100 units; random permutation 100 units, lr $= 0.001$, $\alpha = 1$, clip $= 6$.

4.2 Bài toán tự nhiên (§4.2)

Polyphonic music prediction (Piano-midi.de, Nottingham, MuseData, theo Boulanger-Lewandowski et al. 2012), đo bằng negative log-likelihood trung bình mỗi bước thời gian (thấp hơn = tốt hơn):

Dataset	Fold	SGD	SGD+C	SGD+CR
Piano-midi.de	train	6.87	6.81	7.01
Piano-midi.de	test	7.56	7.53	**7.46**
Nottingham	train	3.67	3.21	3.24
Nottingham	test	3.80	3.48	**3.46**
MuseData	train	8.25	6.54	**6.51**
MuseData	test	7.11	7.00	**6.99**

SGD-CR cho cải thiện **có ý nghĩa thống kê** so với state-of-the-art RNN trên các bài polyphonic music, **trừ** MuseData (bằng đúng kết quả của Bengio et al. 2012, nhưng đó dùng kiến trúc khác).

Language modelling (Penn Treebank, cấp ký tự), đo bằng entropy (bit/ký tự), cả cho next-character (1 bước) và dự đoán ký tự thứ 5 trong tương lai (5 bước):

Task	Fold	SGD	SGD+C	SGD+CR
1 bước	train	1.46	1.34	1.36
1 bước	test	1.50	1.42	**1.41**
5 bước	train	N/A	3.76	3.70
5 bước	test	N/A	3.89	**3.74**

("N/A" ở SGD thường cho tác vụ 5 bước: paper không báo cáo số, hàm ý huấn luyện không hội tụ đủ tốt để so sánh.) Kết quả trên Penn Treebank đạt mức state-of-the-art của Mikolov et al. (2012, một biến thể clipping tương tự nhưng khác chi tiết), được diễn giải là bằng chứng hai cách clipping hoạt động tương đương. Mô hình có regularization đạt hiệu suất **tương đương** Hessian-Free. Vì cả train và test error cùng cải thiện (không đánh đổi kiểu regularizer thường thấy), paper kết luận **clipping giải quyết một vấn đề tối ưu hóa, không đóng vai trò regularizer**.

Siêu tham số: polyphonic music dùng sigmoid units, chuỗi 200 bước, ngưỡng clip chung $= 8$; language modelling dùng 500 sigmoid hidden units không bias, chuỗi 200 bước, ngưỡng clip $= 45$ (tổng cost trên cả chuỗi, không chia trung bình).

5. Phân tích / ablation

- **Clipping đơn lẻ xử lý exploding nhưng không xử lý vanishing.** Hình 7: SGD-C không hơn SGD khi chuỗi dài hơn 20 bước vì thất bại ở đó do vanishing, không phải exploding; chỉ SGD-CR (thêm regularization) vượt qua được.
- **Regularization giúp nhiều hơn cho tác vụ cần long-term hơn.** Trên Penn Treebank, cải thiện của SGD-CR so với SGD-C ở tác vụ 5-bước-tương lai (test $3.89 \to 3.74$, tự tính $(3.89-3.74)/3.89 \approx 3.9\%$) rõ rệt hơn so với tác vụ 1-bước (test $1.42 \to 1.41$, tự tính $\approx 0.7\%$), khớp giả thuyết: tác vụ 5-bước cần phụ thuộc dài hơn nên lợi nhiều hơn từ việc giữ norm error signal.
- **Lịch giảm dần α hữu ích cho bài toán tự nhiên.** Paper quan sát (chưa giải thích đầy đủ bằng lý thuyết) rằng nên để hệ số regularization α_t giảm theo epoch (ví dụ $\alpha_t = 1/(2t)$ hoặc $1/t$): regularization ép mô hình ưu tiên tương quan dài hạn, giảm dần α cho phép mô hình sau đó tận dụng lại thông tin ngắn hạn.

- **Không nhạy với threshold clipping trong một tác vụ cố định**, dù giá trị threshold khác nhau khá nhiều giữa các tác vụ (6 cho bài toán tổng hợp, 8 cho music, 45 cho language modelling); ”không nhạy” nên hiểu cục bộ, không phải một ngưỡng dùng chung mọi nơi.
- **So với Hessian-Free + structural damping:** tác giả giả thuyết Hessian-Free trị vanishing tốt vì ở không gian nhiều chiều thành phần long/short-term thường gần trực giao nên Hessian rescale độc lập được, nhưng tự nhận không đảm bảo tính chất này luôn đúng.

6. Giới hạn

Tác giả tự nêu:

- Regularization (eq. 9) là **soft constraint**, không đảm bảo chống exploding; có thể còn làm tăng khả năng exploding (đẩy state gần basin boundary hơn), nên phải kết hợp với clipping.
- Phân tích đại số chặt (mục 3.1) và phân tích hình học (mục 3.3, ”gradient nổ thì curvature cũng nổ”) chỉ chứng minh hình thức cho trường hợp **tuyến tính** hoặc **một hidden unit**; với σ phi tuyến tổng quát, phần hình học chỉ là **giả thuyết**, chưa chứng minh.
- Phân tích hệ động lực (mục 3.2) chủ yếu áp dụng cho hệ **tự hồi quy không input**; mở rộng sang mô hình có input (phân rã $F_t = \tilde{F} + U_t$) mang tính trực giác hơn là kết quả hình thức chặt.
- Random permutation chỉ thành công 1/8 lần thử, không có giải thích chi tiết tại sao khó hơn các bài khác; lịch giảm α cho bài toán tự nhiên chọn theo quan sát thực nghiệm, không có công thức lý thuyết cho lịch tối ưu.
- Mọi siêu tham số (threshold, α, kích thước mô hình) chọn qua grid search riêng cho từng tác vụ, không có cấu hình ”mặc định dùng mọi nơi”.

Người ghi chú nhận thấy:

- Threshold dao động khá rộng giữa các tác vụ (6, 8, 45), nên ”không nhạy với threshold” chỉ nên hiểu cục bộ trong một cấu hình đã chọn sẵn quy mô, không phải ”chọn threshold nào cũng được”.
- Bảng 1, 2 không báo cáo độ lệch chuẩn/nhiều seed, nên khó đánh giá ý nghĩa thống kê của khác biệt nhỏ ngoài tuyên bố định tính ”statistically significant” của paper.
- Đây là bài về RNN ”vanilla” (công thức 2); không so sánh trực tiếp với LSTM trên cùng bộ task, nên không rõ nếu dùng LSTM thì regularization/clipping còn cần tới mức nào.

7. Liên hệ với các bài khác

- **opt 41** (Why gradient clipping accelerates training: A theoretical justification for adaptivity): đọc sau trực tiếp, cho lý thuyết chặt hơn (giả định độ dốc kiểu ”(L_0, L_1)-smoothness”) cho lý do clipping hội tụ nhanh hơn GD thường, thay lập luận hình học bán-trực giác ở mục 3.3 bài này.
- **opt 37** (Delving Deep into Rectifiers): hướng khác cho gradient không ổn định, chọn **khởi tạo trọng số** (Kaiming init) giữ variance activation/gradient ổn định qua các lớp, thay vì can thiệp lúc backward như clipping.
- **opt 06** (Adam): optimizer thích ứng scale từng tham số theo moment bậc một/hai của gradient; không giải quyết trực tiếp vanishing/exploding theo thời gian của RNN, nhưng trong thực hành thường **dùng cùng** gradient clipping, không thay thế nhau.
- **ml basics 13** (GRU vs LSTM) và **llm 24** (Gated DeltaNet): LSTM là giải pháp cấu trúc cho vanishing gradient mà paper này nhắc tới ở §3.1; Gated DeltaNet là kiến trúc linear-attention/SSM gated hiện đại, vẫn truyền gradient qua nhiều bước cập nhật state cùng họ vấn đề với $\partial x_t / \partial x_k$ ở bài này (xem mục 8).

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện **LightOnOCR-3-0.8B**: VLM OCR dạng hybrid, xen lớp **Gated-DeltaNet** (một kiến trúc linear-RNN/SSM có gating, có trạng thái recurrent cập nhật qua từng token) với

lớp **full attention**, xử lý khoảng 3.600 token/trang (1.500 text + 1.600-2.100 visual), chạy trên GPU ≤ 8 GB, batch 1.

1. **Attention không mắc vanishing/exploding kiểu RNN cổ điển, nhưng GatedDeltaNet có thể vẫn liên quan.** Các lớp full attention tính trực tiếp tương tác giữa mọi cặp token (không qua chuỗi tích Jacobian theo thời gian như công thức 5), nên phân tích của bài này **không áp dụng trực tiếp** cho attention (suy luận). Phần **GatedDeltaNet** vẫn có trạng thái recurrent cập nhật tuần tự qua 3.600 token/trang; gradient ngược qua đó về nguyên tắc vẫn là tích nhiều Jacobian liên tiếp, cùng họ vấn đề, dù cơ chế gate hiện đại (residual/decay học được) có thể giữ eigenvalue hiệu dụng an toàn hơn RNN ”vanilla” năm 2012 (suy luận, bài này không thực nghiệm trên kiến trúc gated hiện đại).
2. **Gradient clipping vẫn rất phổ biến khi huấn luyện LLM/VLM hiện đại, không chỉ cho RNN** (ngoài paper, kiến thức chung: bài này chỉ thực nghiệm trên RNN/LSTM-era). Global gradient norm clipping (thường quanh ngưỡng 1.0, áp trước bước optimizer) vẫn dùng như biện pháp ổn định chung, chặn spike gradient hiếm gặp (batch bất thường, số học fp16/bf16...), không riêng cho vanishing/exploding kiểu RNN. Với LightOnOCR trên GPU nhỏ, batch 1 (gradient mỗi bước nhiễu hơn vì ít trung bình hóa), giữ global-norm clipping khi fine-tune/phục hồi sau pruning (đặc biệt chạm vào lớp GatedDeltaNet) là biện pháp an toàn rẻ, kế thừa trực tiếp Algorithm 1 của bài này (suy luận).
3. **Regularization term (công thức 9) khó áp dụng trực tiếp:** nó thiết kế cho huấn luyện RNN **từ đầu**, còn dự án chỉ fine-tune ngắn mô hình đã pretrain; cài một soft constraint tương tự riêng cho tham số gate của GatedDeltaNet tốn công cài đặt không nhỏ so với lợi ích kỳ vọng ở quy mô fine-tune ngắn (suy luận, không có trong paper).
4. **Gợi ý chẩn đoán rẻ:** nếu fine-tune/phục hồi sau pruning gặp loss nhảy vọt hoặc NaN, theo dõi norm gradient qua các bước; norm tăng vọt đúng lúc loss nhảy là dấu hiệu ”exploding” kiểu bài này mô tả, và hạ threshold clip hoặc giảm learning rate tạm thời là phản ứng hợp lý đầu tiên (suy luận).

9. Thí nghiệm gợi ý

1. **Tái lập mini temporal-order problem.** Cài RNN tanh nhỏ (50 hidden units) bằng PyTorch, huấn luyện trên bài toán temporal order (dữ liệu tổng hợp như mục 4.1) với 3 cấu hình: SGD thường, SGD+clip, SGD+clip+regularization (công thức 9, autograd). Đo tỉ lệ thành công theo log(độ dài chuỗi) từ 5 đến 150 bước, chạy CPU/1 GPU nhỏ; kỳ vọng tái lập hình dạng Hình 7 (SGD/SGD-C sụp ở chuỗi dài, SGD-CR giữ được lâu hơn).
2. **Đo và tự tính threshold clipping.** Log norm gradient $\|\hat{g}\|$ mỗi bước trong 200-500 bước đầu (không clip) của một RNN/khối GatedDeltaNet nhỏ trên dữ liệu toy, vẽ histogram, chọn threshold ở phân vị 90-95%, rồi tự tính tay vài vector mẫu (như ví dụ mục 3.5) để kiểm tra code clip đúng.
3. **Kiểm tra điều kiện eigenvalue bằng số.** Khởi tạo W_{rec} ngẫu nhiên với spectral radius $\lambda_1 \in \{0.5, 0.9, 1.0, 1.5, 4, 8\}$ ($\sigma = \tanh$, $\gamma = 1$), chạy forward không input qua 200 bước, đo $\|x_t\|$ theo t; kiểm tra $\lambda_1 < 1$ dẫn tới state co lại, $\lambda_1 > 1$ dẫn tới state có thể bùng (điều kiện cần cho exploding).
4. **Ablation clip-chỉ vs reg-chỉ vs cả hai** trên bài toán addition (mục 4.1), chiều dài chuỗi cố định lớn (ví dụ 100), đo accuracy cuối cùng; kỳ vọng xác nhận mục 5: clip-chỉ không đủ cho chuỗi dài do vanishing, chỉ cả hai mới ổn định.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
RNN (tham số hóa dùng trong lý thuyết)	$x_t = W_{rec}\sigma(x_{t-1}) + W_{in}u_t + b$	công thức (2)/(11)
Gradient qua BPTT (dạng tích)	$\partial x_t/\partial x_k = \prod_{t\geq i>k} W_{rec}^T \text{diag}(\sigma'(x_{i-1}))$	công thức (5)

Khái niệm	Công thức / giá trị	Ghi chú
Điều kiện đủ vanishing	$\lambda_1 < 1/\gamma$ (λ_1: spectral radius W_{rec}, γ: chặn $\|\sigma'\|$)	tanh: $\gamma = 1$; sigmoid: $\gamma = 1/4$
Điều kiện cần exploding	$\lambda_1 > 1/\gamma$	đảo lại điều kiện trên
Norm clipping (Algorithm 1)	$\hat{g} \leftarrow (threshold/\|\hat{g}\|)\,\hat{g}$ nếu $\|\hat{g}\| \geq threshold$	§3.2
Regularization chống vanishing	$\Omega = \sum_k \left(\frac{\|\partial E/\partial x_{k+1} \cdot \partial x_{k+1}/\partial x_k\|}{\|\partial E/\partial x_{k+1}\|} - 1 \right)^2$	công thức (9), soft constraint
SGD-C, SGD-CR	SGD + clipping; SGD + clipping + regularization	tên gọi trong thực nghiệm
Kết quả chính tổng hợp	SGD-CR: 100% temporal order tới 200 bước, tổng quát tới 400	Hình 7, §4.1
Kết quả chính tự nhiên	cải thiện NLL polyphonic music và bit/char Penn Treebank so với SGD/SGD+C	Bảng 1, 2, §4.2

11. Kết luận ngắn

Bài báo phân tích vanishing/exploding gradient trong RNN từ ba góc nhìn (đại số qua eigenvalue của Jacobian, hệ động lực qua basin/bifurcation, hình học qua curvature của error surface), rồi từ góc nhìn hình học suy ra một giải pháp đơn giản cho exploding gradient: **gradient norm clipping** (Algorithm 1), và từ nhu cầu giữ norm error signal, đề xuất một **regularization term** (soft constraint, công thức 9) cho vanishing gradient. Thực nghiệm trên các bài toán tổng hợp bệnh lý (temporal order, addition, multiplication, ...) và bài toán tự nhiên (polyphonic music, Penn Treebank) cho thấy kết hợp hai kỹ thuật (SGD-CR) vượt rõ SGD thường và chỉ-clipping trên các tác vụ cần phụ thuộc dài, đạt hoặc vượt mức state-of-the-art cùng thời (kể cả so với Hessian-Free). Clipping, dù đơn giản, trở thành một thành phần tiêu chuẩn trong huấn luyện mạng neural nói chung (không chỉ RNN) về sau; regularization term ít được dùng rộng rãi hơn, một phần vì cài đặt phức tạp hơn và là soft constraint không đảm bảo tuyệt đối.

Nguồn gốc: https://arxiv.org/abs/1211.5063

Bài 41

Why Gradient Clipping Accelerates Training: A Theoretical Justification for Adaptivity (2019)

Tác giả: Jingzhao Zhang, Tianxing He, Suvrit Sra, Ali Jadbabaie (Massachusetts Institute of Technology).
Venue / arXiv: arXiv:1905.11881 (v2, 10/02/2020; bản đầu nộp 05/2019); venue: ”Published as a conference paper at ICLR 2020” (dòng này lặp lại ở đầu hầu hết các trang của bản PDF).
Nguồn: https://arxiv.org/abs/1905.11881
Chủ đề: đề xuất điều kiện độ trơn (smoothness) ”(L_0, L_1)-smoothness”, tổng quát hơn Lipschitz-smoothness thường gặp, để chứng minh gradient clipping (và normalized gradient descent) hội tụ nhanh hơn gradient descent bước cố định trên lớp hàm không lồi này, kèm giải thích vì sao kết quả gợi ý một phần cho hiệu quả của các optimizer ”thích ứng” (adaptive).
Vị trí trong lộ trình: đọc sau opt 40 (On the difficulty of training RNNs, bài giới thiệu heuristic clipping mà bài này lý giải chặt hơn bằng lý thuyết) và opt 06 (Adam, ví dụ optimizer thích ứng dùng làm động lực mở đầu); đọc cùng hoặc ngay sau opt 43 (The Marginal Value of Adaptive Gradient Methods), vì cả hai bàn câu hỏi ”khi nào/ tại sao phương pháp thích ứng tốt hơn GD” từ hai góc nhìn khác nhau (thực nghiệm generalization so với lý thuyết hội tụ).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Gradient descent (GD) bước cố định là thuật toán nền cho tối ưu không lồi: $x_{k+1} = x_k - \eta \nabla f(x_k)$. Lý thuyết hội tụ chuẩn của GD dựa trên giả thiết **L-smoothness** (gradient Lipschitz): $\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\|$ với mọi x, y, tương đương (với hàm khả vi hai lần) $\|\nabla^2 f(x)\| \le L$ với mọi x. Dưới giả thiết này, Carmon et al. (2017) chứng minh GD với $\eta = 1/L$ đạt điểm ϵ-dừng ($\|\nabla f(x)\| \le \epsilon$) trong $O(L(f(x_0) - f^*)\epsilon^{-2})$ bước, và tốc độ này tối ưu tới hằng số.

Vấn đề: giả thiết tồn tại một hằng số L **toàn cục** chặn biến thiên của gradient là rất hạn chế. Một đa thức đơn giản như $f(x) = x^3$ đã vi phạm nó (xem mục 3.2). Cách ”chữa cháy” thường dùng là giả sử L chỉ tồn tại trên một miền compact rồi chứng minh quỹ đạo không thoát khỏi miền đó, nhưng khi đó L có thể rất lớn và làm tốc độ hội tụ lý thuyết chậm đi nhiều.

Trong thực hành, nhất là huấn luyện RNN/LSTM cho language modeling (LM), gradient có thể bùng nổ (exploding gradient, xem opt 40: Pascanu, Mikolov, Bengio 2012/2013). Giải pháp phổ biến là **gradient clipping**: giới hạn chuẩn gradient trước khi cập nhật. Đây là một mẹo trực giác, không có nền tảng lý thuyết chặt cho tới bài này. Song song đó, các optimizer **thích ứng** (Adagrad, Adam, RMSProp) hội tụ nhanh hơn hẳn SGD thường trên các tác vụ NLP, kể cả sau khi đã dò kỹ siêu tham số của SGD; các giải thích lý thuyết trước đó (Adagrad thích ứng với gradient thưa, Duchi et al. 2011; bền với biến thiên siêu tham số, Ward et al. 2018) không khớp vì gradient trong thực tế thường **dày đặc** (dense), không thưa.

Động lực thực nghiệm then chốt của bài báo (Hình 1): đo độ trơn cục bộ (ước lượng bằng sai phân hữu hạn của gradient dọc quỹ đạo huấn luyện) và vẽ nó theo chuẩn gradient tại cùng điểm, trên một mô hình AWD-LSTM huấn luyện LM trên Penn Treebank (PTB). Kết quả: độ trơn cục bộ dao động rất mạnh qua các bước, và có **tương quan dương** với chuẩn gradient, đặc biệt rõ trong thí nghiệm LM. Điều này mâu thuẫn trực tiếp với giả thiết ”L là một hằng số”, và đặt ra câu hỏi trung tâm của bài: có thể tìm một điều kiện độ trơn tinh hơn (dựa trên quan sát này) để vừa thiết kế thuật toán nhanh về lý thuyết, vừa nhanh trong thực nghiệm hay không?

2. Ý tưởng chính

Tác giả đề xuất (L_0, L_1)**-smoothness**: $\|\nabla^2 f(x)\| \le L_0 + L_1\|\nabla f(x)\|$, cho phép độ trơn cục bộ tăng theo chuẩn gradient (khớp với Hình 1). Đây là điều kiện **tổng quát hơn** (yếu hơn) L-smoothness theo hai nghĩa: (a) khi $L_1 \to 0$ nó trở lại đúng L-smoothness với $L = L_0$; (b) tồn tại hàm (L_0, L_1)-smooth **toàn cục** nhưng **không** L-smooth với bất kỳ L hữu hạn nào (mọi đa thức bậc ≥ 3, Lemma 2 của paper, minh hoạ ở mục 3.2).

Kết quả chính: dưới điều kiện mới này, **clipped GD** (và **normalized GD**, chứng minh gần tương đương clipped GD tới một hằng số nhân trong bước học) hội tụ tới điểm ϵ-dừng với độ phức tạp lặp **không phụ thuộc** vào M (chặn trên của chuẩn gradient đạt được dọc quỹ đạo, Assumption 4), trong khi GD bước cố định **bắt buộc phải phụ thuộc** vào M để hội tụ (tác giả chứng minh $M < \infty$ là điều kiện **cần**, không chỉ đủ, cho GD). Khi $L_1 M$ lớn (ví dụ khởi tạo xấu khiến gradient có thể rất lớn ở đâu đó trên quỹ đạo), GD bước cố định có thể chậm hơn clipped GD **một cách tuỳ ý** (arbitrarily slower).

Cần nói rõ phạm vi để tránh hiểu quá tay tiêu đề ”adaptivity”: paper **chỉ** phân tích hai thuật toán, clipped GD và normalized GD (NGD), hai thuật toán này thích ứng theo nghĩa đơn giản nhất, bước học co lại tỉ lệ nghịch với **chuẩn** L_2 **toàn cục** của gradient. Paper **không** chứng minh định lý nào cho Adam, Adagrad hay RMSProp; mục Discussion (mục 6 của paper) liệt kê rõ ”coordinate-wise learning rates (nói chung hơn là preconditioning)” là hướng **để ngỏ cho tương lai**. Adam/Adagrad/RMSProp chỉ xuất hiện ở phần mở đầu như động lực thực nghiệm (tại sao cần hiểu ”thích ứng” nói chung), chứ không phải đối tượng được chứng minh. Mục 6 và 8 dưới đây sẽ nhắc lại ranh giới này.

Thực nghiệm minh hoạ (không phải benchmark lớn, xem mục 4): tương quan độ trơn và chuẩn gradient thật sự xuất hiện khi huấn luyện LSTM trên PTB với clipping và learning rate lớn, khớp với việc clipping được dùng rộng rãi trong NLP; tương quan yếu hơn nhiều khi huấn luyện ResNet trên CIFAR10 theo công thức mặc định (bước nhỏ, có momentum), khớp với việc clipping ít phổ biến hơn trong thị giác máy tính theo công thức huấn luyện thông thường.

3. Phương pháp chi tiết

3.1 Định nghĩa (L_0, L_1)-smoothness

Định nghĩa 1 (công thức 2). Hàm f khả vi hai lần là (L_0, L_1)-smooth nếu tồn tại $L_0, L_1 > 0$ sao cho

$$\|\nabla^2 f(x)\| \le L_0 + L_1\|\nabla f(x)\| \quad \text{với mọi } x. \tag{2}$$

Ghi chú 1 của paper: không nhất thiết Hessian và chuẩn gradient phải thoả đúng quan hệ tuyến tính này; chỉ cần chúng **tương quan dương** là đủ để chứng minh clipping nhanh hơn GD bước cố định; dạng tuyến tính được chọn vì đơn giản cho trình bày (nghĩa là các hằng số L_0, L_1 ước lượng từ dữ liệu thật, như ở Hình 1, chỉ là xấp xỉ heuristic, không phải hằng số cấu trúc chính xác của hàm loss).

3.2 Ví dụ: tổng quát hơn Lipschitz-smoothness thật sự (tự tính)

Paper nêu Lemma 2 (đa thức bậc $d \ge 3$ là (L_0, L_1)-smooth nhưng không L-smooth) và chỉ cho chứng minh tóm tắt dựa trên giới hạn của tỉ số f'/f'' khi $x \to \pm\infty$ (bản text trích xuất từ PDF bị lỗi hiển thị phân số ở đoạn này). Để kiểm chứng cụ thể, lấy trường hợp đơn giản nhất $d = 3$: $f(x) = x^3$, nên $f'(x) = 3x^2$, $f''(x) = 6x$.

Không L-smooth toàn cục: $\sup_x |f''(x)| = \sup_x |6x| = \infty$, nên với bất kỳ L hữu hạn nào, chọn $x > L/6$ sẽ có $|f''(x)| > L$, vi phạm định nghĩa L-smoothness.

Nhưng (L_0, L_1)-smooth toàn cục với $L_0 = 3, L_1 = 1$ (tự tính): cần kiểm $|6x| \le 3 + 1 \cdot |3x^2| = 3 + 3x^2$ với mọi x. Bất đẳng thức này tương đương

$$3x^2 - 6|x| + 3 \geq 0 \iff 3(|x| - 1)^2 \geq 0,$$

luôn đúng (dấu bằng xảy ra tại $|x| = 1$). Vậy $f(x) = x^3$ là $(3, 1)$-smooth trên toàn bộ $\mathbb{R}$ dù không L-smooth với bất kỳ L nào, minh hoạ trực tiếp lớp hàm mà điều kiện mới bao phủ còn L-smoothness thì không.

3.3 Trực giác vì sao mạng neural có tính chất này

Với mạng tuyến tính ℓ lớp, loss bình phương $\mathcal{L}(Y, f(X)) = \|Y - W_\ell \cdots W_1 X\|^2$, paper dùng công thức đạo hàm bậc một và bậc hai từ Kawaguchi (2016) để chỉ ra gradient và Hessian **chia sẻ nhiều thành phần chung** (cùng tích ma trận trọng số qua các lớp), và nếu chặn trên các chuẩn này bằng Cauchy-Schwarz thì cả hai chặn trên đều tăng đơn điệu theo $\|W_i\|$ và $\|f(X) - Y\|$. Đây là lập luận trực giác (tác giả gọi là "conjecture"), không phải định lý chặt cho mạng phi tuyến tổng quát.

3.4 Giả thiết và thuật toán

Bài toán: $\min_{x \in \mathbb{R}^d} f(x)$ không lồi, tìm điểm ϵ-dừng $\|\nabla f(x)\| \leq \epsilon$. Giả thiết 1: f bị chặn dưới bởi f^*. Giả thiết 2: f khả vi hai lần (có thể nới về khả vi bậc một bằng dạng lim sup, paper không khai thác thêm). Giả thiết 3: (L_0, L_1)-smoothness toàn cục hoặc trên tập mức dưới (sublevel set)

$$S := \{x \mid \exists y : f(y) \leq f(x_0),\ \|x - y\| \leq 1\}. \tag{3}$$

Ba thuật toán được phân tích:

$$\text{GD: } x_{k+1} = x_k - \eta \nabla f(x_k) \tag{4}$$

$$\text{Clipped GD: } x_{k+1} = x_k - h_c \nabla f(x_k), \quad h_c := \min\left\{\eta_c, \frac{\gamma \eta_c}{\|\nabla f(x_k)\|}\right\} \tag{5}$$

$$\text{NGD: } x_{k+1} = x_k - h_n \nabla f(x_k), \quad h_n := \frac{\eta_n}{\|\nabla f(x_k)\| + \beta} \tag{6}$$

Trực giác (5): khi $\|\nabla f\| \leq \gamma$ (gradient nhỏ), $h_c = \eta_c$ như GD thường; khi $\|\nabla f\| > \gamma$ (gradient lớn), bước cập nhật có chuẩn cố định $h_c \|\nabla f\| = \gamma \eta_c$, tức là **chuẩn gradient sau clip bị chặn ở ngưỡng** γ. Paper chứng minh clipped GD và NGD **gần tương đương**: nếu đặt $\gamma \eta_c = \eta_n$ và $\eta_c = \eta_n / \beta$ thì $\frac{1}{2} h_c \leq h_n \leq 2 h_c$, nên mọi kết quả hội tụ chứng minh cho clipped GD áp dụng được cho NGD (chỉ khác hằng số).

3.5 Vì sao phân tích cổ điển thất bại, và cách sửa

Phân tích GD chuẩn dựa trên "descent lemma":

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \tfrac{L}{2} \|y - x\|^2 \tag{10}$$

Dưới (L_0, L_1)-smoothness, số hạng bậc hai này có thể **tăng theo hàm mũ** theo $\|y - x\|$ (vì L cục bộ phụ thuộc chính gradient tại điểm trung gian, Appendix B). Lời giải của paper (Lemma 9, Appendix C) là chặn mức gradient có thể thay đổi trong một bước di chuyển ngắn ($\|x^+ - x\| \leq \min\{1/L_1, 1\}$) bằng **bất đẳng thức Grönwall** (dạng tích phân): $\|\nabla f(x^+)\| \leq 4(L_0/L_1 + \|\nabla f(x)\|)$. Từ đó suy ra một "descent inequality" cục bộ hợp lệ cho mỗi bước clipped GD, và cộng dồn (telescoping sum) ra độ phức tạp lặp (Theorem 3, mục 4). Với GD bước cố định, thách thức khác: phải chứng minh bằng phản chứng rằng gradient không vượt quá M trong suốt một bước (Theorem 6); còn với bản ngẫu nhiên (stochastic), bước học h_k lại phụ thuộc chính

gradient ngẫu nhiên g_k (tương quan step-size và nhiễu), được giải quyết bằng cách chia thành hai trường hợp "gradient lớn" và "gradient nhỏ" (Appendix B, F).

4. Thí nghiệm và kết quả chính

Đây là paper lý thuyết: không có benchmark lớn theo nghĩa thông thường. "Kết quả chính" là các **Theorem/bound** về độ phức tạp lặp; phần thực nghiệm (mục 5 của paper) chỉ là minh hoạ nhỏ (một mô hình LM, một mô hình ảnh, một hàm tổng hợp), dùng để kiểm tra định tính các dự đoán lý thuyết, không phải để so sánh SOTA.

Trường hợp tất định (deterministic).

Định lý	Thuật toán	Độ phức tạp lặp (số bước để $\|\nabla f\| \le \epsilon$)
Theorem 3	Clipped GD, $\eta_c = \frac{1}{10L_0}$, $\gamma = \min\{\frac{1}{\eta_c}, \frac{1}{10L_1\eta_c}\}$	$\frac{20L_0(f(x_0)-f^*)}{\epsilon^2} + \frac{20\max\{1,L_1^2\}(f(x_0)-f^*)}{L_0}$ (không phụ thuộc M)
Theorem 4 (lower bound)	GD bước cố định, mọi η	$\ge \frac{L_1 M(f(x_0)-f^*-5\epsilon/8)}{8\epsilon^2(\log M+1)}$
Theorem 6 (upper bound)	GD bước cố định, $\eta = \frac{1}{2(ML_1+L_0)}$	$\le 4(ML_1+L_0)(f(x_0)-f^*)\epsilon^{-2}$

Remark 5 tổng hợp: GD không thể nhanh hơn $\Omega\big((L_1M/\log M + L_0)(f(x_0)-f^*)\epsilon^{-2}\big)$, trong khi clipped GD đạt $O\big(L_0(f(x_0)-f^*)\epsilon^{-2} + L_1^2(f(x_0)-f^*)L_0^{-1}\big)$; cận dưới và cận trên của GD khớp nhau tới một hệ số $\log M$, và khi L_1M lớn, GD có thể chậm hơn clipped GD **tuỳ ý**.

Minh hoạ số (tự tính). Lấy $L_0 = 1$, $L_1 = 10$, $M = 1000$ (đại diện một khởi tạo xấu, gradient có thể lớn), $f(x_0) - f^* = 100$, $\epsilon = 0.1$:

- Clipped GD (Theorem 3): $\frac{20\cdot 1\cdot 100}{0.01} + \frac{20\cdot 100\cdot 100}{1} = 200000 + 200000 = 4\times 10^5$ bước.
- GD, cận dưới bắt buộc (Theorem 4): $\frac{10\cdot 1000\cdot(100-0.0625)}{8\cdot 0.01\cdot(\ln 1000+1)} \approx \frac{999375}{0.633} \approx 1.58\times 10^6$ bước.
- GD, cận trên đạt được (Theorem 6): $4\cdot(1000\cdot 10+1)\cdot 100/0.01 \approx 4.0\times 10^8$ bước.

Ngay cả ở mức "ít nhất" (cận dưới của GD), GD đã cần nhiều hơn khoảng 4 lần so với mức "nhiều nhất" clipped GD cần (cận trên); và trong kịch bản cận trên cụ thể của Theorem 6, chênh lệch lên tới khoảng **1000 lần** (tự tính). Đây chính là ý nghĩa định lượng của "arbitrarily faster" khi $L_1M \to \infty$.

Trường hợp ngẫu nhiên (stochastic, Assumption 5: nhiễu bị chặn $\|\nabla\hat{f}(x)-\nabla f(x)\| \le \tau$ h.c.c.): Theorem 7 cho stochastic clipped GD một chặn không phụ thuộc M (chỉ phụ thuộc $L_0, L_1, \tau, \Delta = f(x_0)-f^*+(5L_0+2L_1\tau)\tau^2+9\tau L_0^2/L_1$); Theorem 8 cho SGD thường một chặn $O\big((f(x_0)-f^*+(5L_0+4L_1M)(M+\tau)^2)^2\epsilon^{-4}\big)$, phụ thuộc M. Paper không chứng minh được cận dưới riêng cho SGD (vì lower bound cho SGD nói chung chưa được biết ngay cả ở trường hợp L-smooth thường), nên dùng lại cận dưới tất định của Theorem 4 làm cận dưới "có thể lỏng".

Thí nghiệm minh hoạ trên mạng thật (mục 5 của paper):

- **LM:** AWD-LSTM 3 lớp (hidden 1150, embedding 400, dropout 0.4, DropConnect 0.5) huấn luyện trên PTB (10k từ vựng, 887k/70k/78k từ train/val/test), clip 0.25, lr 30, 500 epoch, đạt test perplexity 56.5 (gần SOTA thời điểm đó). Hình 2 cho thấy tương quan dương rõ giữa chuẩn gradient và độ trơn cục bộ khi có clipping và lr lớn.

- **Ảnh:** ResNet20 trên CIFAR10 (50k train/10k test), baseline SGD+momentum (lr 0.1, momentum 0.9, weight decay 5×10^{-4}, 200 epoch, giảm lr 10 lần ở epoch 100 và 150) đạt test accuracy 95.0% (Appendix H.2); abstract nêu riêng "95.2%" cho (clipped) GD lr lớn, không đặt cạnh 95.0% để so trực tiếp (xem mục 6).
- **Tổng hợp (Hình 4):** GD/clipped GD với lr lớn hội tụ loss huấn luyện nhanh nhất ở cả hai tác vụ; với LM, mô hình dùng clipping đạt validation loss tốt nhất (vì clipping là công thức chuẩn cho recipe này); với ảnh, clipped GD đạt tốc độ hội tụ nhanh nhất và độ chính xác test gần bằng SGD+momentum.
- **Thí nghiệm tổng hợp (Appendix I):** tối ưu $f(x) = x^4$ từ $x_0 = 30$. Trên $[-30, 30]$: $f''(x) \le 12 \times 30^2 = 1.08 \times 10^4$ (ước lượng L địa phương cho GD thường) trong khi có thể lấy $L_1 = 10, L_0 = 0.1$ cho clipped GD (vì $f''(x) \approx 10|f'(x)| + 0.1$ xấp xỉ trên miền này). Dò step size riêng cho từng thuật toán, clipped GD hội tụ nhanh hơn GD rất nhiều (Hình 7 của paper).
- **Appendix J, so lý thuyết với thực nghiệm:** ở thí nghiệm tổng hợp, lý thuyết dự đoán tỉ lệ cải thiện $f'(x_{GD})/f'(x_{clipped}) \approx (ML_1 + L_0)/L_0 \approx 10^5$, thực nghiệm đo được $\approx 10^7$ (lý thuyết đúng hướng nhưng bảo thủ, conservative). Ở một LSTM nhỏ hơn (2 lớp, hidden 512, embedding 200, clip 0.25 lr 10 so với GD lr 2), lý thuyết dự đoán tỉ lệ $\approx 25/5 = 5$, thực nghiệm đo ≈ 3, cùng bậc độ lớn.

5. Phân tích / ablation

- **Trường hợp đặc biệt $L_1 = 0$:** (L_0, L_1)-smoothness trở lại đúng L-smoothness với $L = L_0$; khi đó bound Theorem 3 còn $20L_0\Delta/\epsilon^2$, cùng dạng $O(L\Delta/\epsilon^2)$ với kết quả cổ điển (Carmon et al. 2017), chỉ khác hằng số 20 so với hằng số tối ưu đã biết: hợp lý vì kết quả mới phải chứa kết quả cũ như một trường hợp riêng.
- **Vai trò của M (Assumption 4):** Theorem 4 xây dựng ví dụ đối kháng gồm hai hàm một chiều, một hàm tăng kiểu mũ (buộc η phải rất nhỏ để không phân kỳ) và một hàm "chậm" dạng đa thức bậc bốn từng khúc (buộc số bước cần thiết tỉ lệ với $L_1 M/(\epsilon^2 \log M)$ khi dùng đúng η nhỏ đó). Ghép hai vế chứng minh $M < \infty$ là điều kiện cần cho GD hội tụ với bất kỳ η cố định nào, không chỉ là một giả thiết kỹ thuật thuận tiện.
- **Kiểm tra chéo hằng số lý thuyết với thực nghiệm (tự tính):** với $L_0 = 0.1, L_1 = 10$ dùng ở Appendix I, Theorem 3 cho $\eta_c = \frac{1}{10L_0} = 1$, $\gamma = \min\{\frac{1}{\eta_c}, \frac{1}{10L_1\eta_c}\} = \min\{1, \frac{1}{100}\} = 0.01$. Ngưỡng clip hiệu dụng của thuật toán (5) khi gradient lớn là $\gamma\eta_c = 0.01 \times 1 = 0.01$, **khớp đúng** với "threshold 0.01" mà Appendix I chọn khi chạy thí nghiệm. Đây là một dấu hiệu tốt rằng công thức lý thuyết không chỉ trừu tượng mà còn tái tạo đúng lựa chọn thực nghiệm của chính các tác giả.
- **Tương quan độ trơn/chuẩn gradient chỉ rõ khi có clip + lr lớn:** Hình 2 và 3 cho thấy nếu không clip, optimizer buộc phải dùng lr nhỏ để không phân kỳ, và khi đó quỹ đạo chỉ loanh quanh vùng mà độ trơn cục bộ ít biến thiên, nên tương quan yếu đi; clipping cho phép quỹ đạo "đi qua" vùng không trơn một cách ổn định, bộc lộ rõ tương quan dự đoán bởi (L_0, L_1)-smoothness. Đây cũng là lý do ResNet mặc định (lr nhỏ, có momentum) ít cho thấy hiện tượng này, trong khi LM (lr lớn theo recipe chuẩn) cho thấy rất rõ.
- **Khoảng cách lý thuyết và thực nghiệm (Appendix J):** hai trường hợp lệch theo hai hướng ngược nhau, nhưng cùng bậc độ lớn: ở thí nghiệm tổng hợp, lý thuyết ($\approx 10^5$) **thấp hơn** thực tế đo được ($\approx 10^7$, bound bảo thủ); ở LSTM nhỏ, lý thuyết dự đoán tỉ lệ 5 lại **cao hơn** thực tế đo được (≈ 3). Tác giả cho rằng nhiễu (noise) trong huấn luyện mạng thật là một phần nguyên nhân của độ lệch (các ước lượng $ML_1 + L_0$ thô không tính nhiễu).

6. Giới hạn

Tác giả nêu (mục 6, Discussion): (a) chưa rõ có tồn tại điều kiện độ trơn tốt hơn vừa khớp thực nghiệm vừa cho phân tích sạch hơn; (b) chỉ phân tích clipped GD/NGD, chưa mở rộng cho momentum, **coordinate-wise learning rate / preconditioning** (tức các thuật toán kiểu Adam, Adagrad, RMSProp), hay variance reduction, đây đều là hướng để ngỏ; (c) không có lower bound cho SGD (phải dùng tạm lower bound tất định, có thể

lỏng); (d) nhắc nhở rằng baseline được tinh chỉnh tốt (well-tuned) đôi khi đã đủ cạnh tranh với thuật toán ”mới” được đề xuất gần đây, nên cần kiểm tra baseline cẩn thận.

Người ghi chú nhận thấy thêm:

- Dạng tuyến tính (2) chỉ là lựa chọn để đơn giản hoá (Remark 1 tự thừa nhận); các hằng số L_0, L_1 ước lượng từ Hình 1 nên hiểu là tham số **fit heuristic** chứ không phải hằng số cấu trúc chính xác của hàm loss mạng neural.
- Số ”95.2%” ở abstract (ResNet20/CIFAR10, GD/clipped GD lr lớn) và ”95.0%” ở Appendix H.2 (baseline SGD+momentum) không được paper đặt cạnh nhau để so trực tiếp; không nên coi hai số là cùng một đại lượng mà không kiểm tra lại cấu hình.
- Ví dụ số ở Appendix I ($L_0 = 0.1, L_1 = 10$ cho $f(x) = x^4$ trên $[-30, 30]$) hơi lỏng tại vùng gradient nhỏ: kiểm tra trực tiếp (tự tính) tại $x = 0.2$, $f''(x) = 12 \times 0.04 = 0.48$ trong khi $10|f'(x)| + 0.1 = 10 \times 4 \times 0.008 + 0.1 = 0.42$, tức $0.48 > 0.42$, bất đẳng thức (L_0, L_1)-smoothness với $L_0 = 0.1$ bị vi phạm nhẹ đúng tại điểm này (cần $L_0 \gtrsim 0.16$ mới đủ); không ảnh hưởng tới kết luận định tính của thí nghiệm (clipped GD vẫn nhanh hơn GD rất nhiều) nhưng là một chỗ số liệu phụ lục chưa khít tuyệt đối.
- Các ví dụ đối kháng trong Theorem 4 là hàm một chiều dựng sẵn; paper không chứng minh chúng đại diện cho cảnh quan loss (loss landscape) của mạng neural thật ngoài bằng chứng gián tiếp là tương quan thực nghiệm ở Hình 1 đến 3.
- Vì phạm vi chỉ giới hạn ở clipped GD/NGD, tên bài ”A Theoretical Justification for Adaptivity” dễ bị hiểu nhầm là chứng minh cho Adam nói chung; cần nhắc lại ranh giới này khi trích dẫn.

7. Liên hệ với các bài khác

- **opt 40 (On the difficulty of training Recurrent Neural Networks, Pascanu et al. 2012/2013):** bài giới thiệu gradient norm clipping như một ”intuitive work around” cho exploding gradient; chính paper 41 trích dẫn trực tiếp và cung cấp chứng minh lý thuyết cho đúng heuristic này.
- **opt 06 (Adam):** optimizer thích ứng được paper 41 dùng làm động lực mở đầu (Adam hội tụ nhanh hơn SGD trên NLP dù gradient dày đặc); paper 41 không phân tích Adam, chỉ mượn hiện tượng này để đặt câu hỏi nghiên cứu.
- **opt 43 (The Marginal Value of Adaptive Gradient Methods in Machine Learning, Wilson et al. 2017):** được paper 41 trích dẫn ngay ở Introduction cho nhận xét ”GD đạt hiệu năng thực nghiệm tốt”; opt 43 lập luận GD/SGD có thể tổng quát hoá tốt hơn Adam trên một số bài toán, cho một góc nhìn bổ sung/đối lập về giá trị của ”thích ứng” so với câu chuyện tốc độ hội tụ ở bài 41.
- **opt 08 (On the Convergence of Adam and Beyond, Reddi, Kale, Kumar 2019):** paper 41 trích dẫn đây là ”điểm khởi đầu” (one starting point) cho làn sóng nghiên cứu lý thuyết hội tụ của các phương pháp thích ứng từ 2018 trở đi (Related Work, Appendix A); AMSGrad sửa lỗi hội tụ của Adam, trong khi paper 41 đi theo hướng khác: đổi hẳn giả thiết độ trơn thay vì sửa thuật toán.
- Thư mục khác (đã kiểm tra bằng ls, không thấy bài nào phân tích đúng điều kiện (L_0, L_1)-smoothness hay smoothness biến thiên dọc quỹ đạo trong ”math content/” hay ”ml basics content/” hiện có, nên không trỏ thêm để tránh gán sai liên hệ).

8. Ứng dụng cho dự án (suy luận)

Dự án: nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, hybrid GatedDeltaNet + full attention, khoảng 3600 token/trang gồm 1500 text + 1.6-2.1K visual, GPU ≤ 8 GB, batch 1). Paper này không phải về OCR hay nén mô hình, nhưng kết quả về ổn định huấn luyện có liên hệ thực tế rõ ràng, ở mức suy luận:

1. **Batch 1 làm nhiễu gradient (τ trong Assumption 5) lớn hơn.** Theorem 7 cho thấy chặn hội tụ của stochastic clipped GD chứa các số hạng $(5L_0 + 2L_1\tau)\tau^2$ tăng theo τ; batch 1 nghĩa là không có trung bình hoá nhiễu giữa các mẫu trong một batch, nên τ (biên độ sai số gradient ước lượng so với gradient thật) có xu hướng lớn hơn so với batch cỡ lớn. Suy luận: đây là lý do thực tế để luôn bật gradient

clipping khi fine-tune/distill ở batch 1 trên GPU nhỏ, kể cả với các run ngắn, chứ không chỉ cho huấn luyện từ đầu quy mô lớn.

2. **GatedDeltaNet là khối kiểu hồi quy (recurrent-like/linear-attention), gần với LSTM hơn ResNet.** Thí nghiệm của paper (Hình 2 so với Hình 3) cho thấy tương quan "độ trơn tăng theo chuẩn gradient" rõ rệt ở LSTM LM nhưng yếu ở ResNet huấn luyện mặc định. Suy luận (chưa được paper kiểm chứng trực tiếp, vì paper không thử kiến trúc linear-attention/SSM nào): phần tham số thuộc khối GatedDeltaNet nhiều khả năng có hành vi độ trơn gần với LSTM hơn phần full-attention, nên gradient clipping có thể hữu ích hơn (không phải kém hữu ích hơn) khi fine-tune riêng các khối này, đặc biệt nếu learning rate đặt tương đối lớn cho nhánh đó.
3. **Chọn ngưỡng clip threshold.** Paper không cho công thức chọn ngưỡng tiên nghiệm (prior) nếu không biết trước L_0, L_1; ước lượng L_0, L_1 thực nghiệm (bộ ước lượng độ trơn công thức 17, dựa trên Santurkar et al. 2018) tốn thêm forward/backward, khó đáng công cho một dự án nhỏ. Suy luận: cách thực dụng hơn là theo dõi histogram chuẩn gradient trong vài trăm bước đầu rồi đặt ngưỡng gần bách phân vị 90-95%, sau đó kiểm tra lại sau vài epoch xem ngưỡng có còn hợp lý không (kỹ thuật thông dụng, không phải kết luận của paper).
4. **Quan hệ với Adam/AdamW.** Paper không phân tích clipping kết hợp Adam (để ngỏ ở mục Discussion). AdamW đã chia mỗi toạ độ cho một ước lượng moment bậc hai riêng, tức đã là một dạng thích ứng (thô hơn, có độ trễ) theo từng tham số. Suy luận: gradient-norm clipping toàn cục đặt thêm lên trên AdamW nên hiểu là một lớp **an toàn bổ sung, không thay thế**, dùng để chặn đúng kịch bản "một batch xấu" (ví dụ một trang OCR bị lỗi scan, văn bản bị lặp bất thường khiến loss/gradient đột biến), điều mà huấn luyện batch 1 đặc biệt dễ gặp vì không có các mẫu khác trong cùng batch để pha loãng. Giữ `clip_grad_norm_` chủ động cùng AdamW khi fine-tune, với ngưỡng theo kinh nghiệm chung thường trong khoảng 1 đến 5 (suy luận thực hành, không lấy từ paper), là lựa chọn hợp lý.
5. Kết quả ResNet20/CIFAR10 (mục 4) không cho thấy clipping làm giảm độ chính xác cuối cùng so với SGD+momentum đã tinh chỉnh kỹ; suy luận mở rộng: công dụng chính của clipping vẫn là ổn định huấn luyện, không phải cải thiện chất lượng cuối.

9. Thí nghiệm gợi ý

1. **Tái tạo thí nghiệm tổng hợp Appendix I** trên 1 GPU nhỏ: tối ưu $f(x) = x^3$ hoặc $f(x) = x^4$ từ x_0 lớn (ví dụ 30), so sánh GD và clipped GD ở nhiều step size, đo số bước để $|f'(x)| \leq \epsilon$. Kỳ vọng: clipped GD cần ít hơn nhiều bậc độ lớn so với GD tốt nhất, khớp hướng minh hoạ số ở mục 4; diễn giải: nếu không thấy chênh lệch rõ, có thể do chọn ϵ hoặc khoảng step size chưa đủ cực trị.
2. **Ước lượng độ trơn cục bộ trên một LSTM/GatedDeltaNet nhỏ.** Huấn luyện một mô hình ngôn ngữ ký tự nhỏ (hoặc một khối hồi quy tương tự dùng trong dự án) với và không có clipping; mỗi vài chục bước, ước lượng độ trơn bằng công thức (17) (sai phân hữu hạn gradient dọc hướng cập nhật) và vẽ theo chuẩn gradient. Kỳ vọng: nếu kiến trúc hồi quy có hành vi giống LSTM trong paper, sẽ thấy tương quan dương rõ khi có clip + lr lớn, yếu khi lr nhỏ; diễn giải: xác nhận hay bác bỏ suy luận ở mục 8.2.
3. **Kiểm tra "batch xấu" ở batch 1.** Trên một tập nhỏ các trang OCR khó (nhiễu, văn bản lặp bất thường), fine-tune batch 1 với và không có `clip_grad_norm_`, đếm số bước loss/gradient đột biến (NaN/Inf hoặc spike lớn hơn X lần trung vị). Kỳ vọng: số bước đột biến giảm rõ khi bật clip; diễn giải: đo trực tiếp giả thuyết "clipping chặn batch xấu" ở mục 8.1/8.4 thay vì chỉ suy luận lý thuyết.
4. **So sánh AdamW đơn thuần với AdamW + clip.** Cùng một run fine-tune nhỏ, log tỉ lệ bước thực sự bị clip (clip rate) và loss cuối cùng. Kỳ vọng: nếu clip rate gần 0%, clipping gần như không đổi gì (ngưỡng đặt quá cao, không phải "an toàn" thật sự); nếu clip rate đáng kể mà loss cuối không đổi hoặc tốt hơn, xác nhận vai trò "lớp an toàn" nêu ở mục 8.4; diễn giải kèm theo: nên báo cáo cả hai số (clip rate và loss) chứ không chỉ loss cuối.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Nguồn
L-smoothness	$\|\nabla f(x) - \nabla f(y)\| \le L\|x-y\| \iff \|\nabla^2 f(x)\| \le L$	Mục 2.1, công thức (1)
(L_0, L_1)-smoothness	$\|\nabla^2 f(x)\| \le L_0 + L_1\|\nabla f(x)\|$	Định nghĩa 1, công thức (2)
Ví dụ không L-smooth nhưng (L_0, L_1)-smooth	$f(x) = x^3$ là $(3, 1)$-smooth (tự tính, mục 3.2)	Suy từ Lemma 2
GD	$x_{k+1} = x_k - \eta\nabla f(x_k)$	Công thức (4)
Clipped GD	$x_{k+1} = x_k - h_c\nabla f(x_k)$, $h_c = \min\{\eta_c, \gamma\eta_c/\|\nabla f(x_k)\|\}$	Công thức (5)
NGD	$h_n = \eta_n/(\|\nabla f(x_k)\| + \beta)$, tương đương clipped GD tới hằng số	Công thức (6)
Clipped GD, tất định (Theorem 3)	$O(L_0\Delta\epsilon^{-2} + L_1^2\Delta L_0^{-1})$, không phụ thuộc M	Theorem 3
GD, tất định (Theorem 4 + 6)	$\Omega((L_1 M/\log M + L_0)\Delta\epsilon^{-2})$ đến $O((ML_1 + L_0)\Delta\epsilon^{-2})$	Theorem 4, 6, Remark 5
Stochastic clipped GD / SGD	Theorem 7 (không phụ thuộc M) so với Theorem 8 (phụ thuộc M)	Mục 4.2
Ước lượng độ trơn thực nghiệm	$\hat{L}(x_k) = \max_{\gamma\in\{\delta,\dots,1\}} \|\nabla f(x+\gamma d) - \nabla f(x)\|/\|\gamma d\|$	Công thức (17), Appendix H.3

11. Kết luận ngắn

Paper đề xuất (L_0, L_1)-smoothness, một điều kiện độ trơn tổng quát hơn Lipschitz-smoothness, được động lực hoá trực tiếp từ quan sát thực nghiệm rằng độ trơn cục bộ của mạng neural (nhất là LSTM cho language modeling) tăng theo chuẩn gradient. Dưới điều kiện này, clipped GD (và NGD tương đương) có độ phức tạp lặp không phụ thuộc vào chặn trên gradient M, trong khi GD bước cố định bắt buộc phụ thuộc M và có thể chậm hơn tuỳ ý khi $L_1 M$ lớn, cả ở chế độ tất định lẫn ngẫu nhiên. Đóng góp chính là lý thuyết (các Theorem 3, 4, 6, 7, 8), được minh hoạ chứ không kiểm chứng bằng benchmark lớn trên LSTM/PTB, ResNet20/CIFAR10 và một hàm tổng hợp. Giới hạn quan trọng nhất cần nhớ: phạm vi chứng minh chỉ dừng ở clipping/normalized GD theo chuẩn toàn cục, chưa chạm tới các optimizer thích ứng theo từng toạ độ như Adam, vốn được nêu rõ là hướng nghiên cứu tương lai trong chính bài báo.

Nguồn gốc: https://arxiv.org/abs/1905.11881

Bài 42

Mixed Precision Training (2017)

Tác giả: Sharan Narang, Gregory Diamos, Erich Elsen (Baidu Research); Paulius Micikevicius, Jonah Alben, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, Hao Wu (NVIDIA). Narang và Micikevicius đóng góp ngang nhau (equal contribution), theo chú thích của paper.
Venue / arXiv: arXiv:1710.03740 (v3, 15/02/2018); venue: ”Published as a conference paper at ICLR 2018” (dòng này có ghi rõ ở đầu mỗi trang trong văn bản).
Nguồn: https://arxiv.org/abs/1710.03740
Chủ đề: huấn luyện mạng neural bằng số thực nửa độ chính xác (FP16, half-precision) kết hợp giữ một bản sao trọng số FP32 (master weights) và kỹ thuật ”loss scaling”, để đạt độ chính xác (accuracy) ngang với huấn luyện FP32 mà giảm gần một nửa bộ nhớ và tăng tốc tính toán trên GPU hiện đại.
Vị trí trong lộ trình: đọc sau opt 37-39 (khởi tạo Kaiming, Batch Normalization, Layer Normalization: các kỹ thuật giữ ổn định số học khi lan truyền qua mạng sâu) và opt 40-41 (gradient clipping: ổn định khi gradient quá lớn). Bài 42 chuyển sang một khía cạnh khác của ”huấn luyện ổn định”: biểu diễn số (số bit, dải giá trị) chứ không phải kiến trúc hay thuật toán optimizer. Đọc cùng nhóm hạ tầng/tốc độ huấn luyện với opt 21 (Accurate, Large Minibatch SGD) và opt 10 (LAMB, train BERT trong 76 phút); là tiền đề thực hành cho mọi optimizer/kiến trúc LLM hiện đại sau này (ví dụ opt 16, Muon) vì gần như tất cả đều huấn luyện ở độ chính xác thấp hơn FP32.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Mô hình deep learning ngày càng lớn: paper dẫn ví dụ mạng nhận dạng giọng nói trong Hannun et al. (2014) có 11 triệu tham số, tăng lên khoảng 67 triệu cho RNN hai chiều, rồi 116 triệu cho mô hình GRU một chiều mới nhất trong Amodei et al. (2016). Mô hình lớn hơn cần nhiều compute và memory hơn để huấn luyện.

Theo paper, hiệu năng (tốc độ) của một chương trình bất kỳ, kể cả huấn luyện/suy luận mạng neural, bị giới hạn bởi một trong ba yếu tố: arithmetic bandwidth (băng thông tính toán), memory bandwidth (băng thông bộ nhớ), hoặc latency. Giảm độ chính xác số (reduced precision) giải quyết hai yếu tố đầu: dùng ít bit hơn để lưu cùng một lượng giá trị giúp giảm áp lực memory bandwidth; nhiều bộ xử lý (GPU) cho throughput cao hơn với toán học độ chính xác thấp hơn. Cụ thể, paper nêu: ”half-precision math throughput in recent GPUs is 2× to 8× higher than for single-precision” (thông lượng toán FP16 trên GPU gần đây cao hơn FP32 từ 2 đến 8 lần).

Hệ thống huấn luyện deep learning thời điểm đó dùng định dạng số thực đơn độ chính xác (FP32, single-precision) làm chuẩn. Chuẩn IEEE 754 định nghĩa FP32 gồm 1 bit dấu, 8 bit exponent (số mũ), 23 bit mantissa (phần định trị); FP16 (half-precision) gồm 1 bit dấu, 5 bit exponent, 10 bit mantissa (đây là cấu trúc bit chuẩn IEEE 754, ngoài paper, kiến thức chung, nhưng số ”10 bit mantissa” được paper nhắc lại trực tiếp ở phần loss scaling). Vì exponent chỉ có 5 bit, FP16 có dải giá trị (dynamic range) hẹp hơn FP32 rất nhiều: paper nói rõ ”FP16 exponent bias centers the range of normalized value exponents to [-14, 15]”, và giá trị lớn nhất biểu diễn được trong FP16 là 65.504. Dải hẹp này là nguồn gốc của mọi vấn đề mà paper phải giải quyết: nhiều giá trị gradient trong huấn luyện mạng sâu có độ lớn rất nhỏ (exponent rất âm), dễ bị tràn dưới (underflow) thành 0 khi ép về FP16.

Trước bài này đã có nhiều hướng giảm độ chính xác (lượng tử hoá cực đoan: binary, ternary, vài bit) nhưng đều đánh đổi mất accuracy hoặc phải thay đổi kiến trúc/hyperparameter. Câu hỏi bài 42 trả lời: có thể dùng FP16

cho *toàn bộ* forward, backward (tất cả tensor và phép tính, không chỉ một phần) mà *không* đổi hyperparameter và *không* mất accuracy so với FP32 không?

2. Ý tưởng chính

Paper đề xuất phương pháp ”mixed precision training”: lưu trọng số (weights), activation, và gradient ở FP16 trong hầu hết quá trình forward/backward (để giảm bộ nhớ, tăng tốc), nhưng bù lại ba vấn đề mất thông tin do dải FP16 hẹp bằng ba kỹ thuật:

1. **FP32 master copy of weights**: giữ một bản sao trọng số ở FP32, cộng gradient vào bản sao này mỗi bước optimizer; mỗi iteration tạo lại bản FP16 của trọng số để dùng cho forward/backward.
2. **Loss scaling**: nhân giá trị loss với một hệ số s trước khi backward, để ”đẩy” các giá trị gradient nhỏ lên vùng FP16 biểu diễn được, tránh chúng bị làm tròn về 0; sau backward chia lại (unscale) trước khi update trọng số.
3. **Tích lũy (accumulate) ở FP32 cho một số phép tính**: dot-product (tích vô hướng) và các phép reduction (tổng theo nhiều phần tử, ví dụ trong batch normalization, softmax) cộng dồn kết quả trung gian ở FP32 rồi mới chuyển về FP16 khi lưu, để tránh lỗi làm tròn tích lũy qua nhiều phép cộng.

Kết quả: paper chứng minh phương pháp này hoạt động trên ”wide variety of tasks and modern large scale (exceeding 100 million parameters) model architectures” mà không cần đổi hyperparameter, với 7 nhóm bài toán khác nhau (phân loại ảnh, object detection, nhận dạng giọng nói, dịch máy, mô hình ngôn ngữ, GAN).

3. Phương pháp chi tiết

3.1 FP32 master copy of weights

Trong mixed precision training, trọng số, activation và gradient được lưu ở FP16. Để khớp accuracy của mạng FP32, một bản sao trọng số FP32 (master copy) được giữ và cập nhật bằng weight gradient trong bước optimizer; mỗi iteration, bản sao FP16 của master weights được tạo ra để dùng cho forward và backward, giúp giảm một nửa lưu trữ/bandwidth so với huấn luyện FP32 thuần.

Paper nêu hai lý do khiến cần master copy FP32 (không thể chỉ giữ trọng số ở FP16):

Lý do 1 (update quá nhỏ bị mất): giá trị update (weight gradient nhân learning rate) có thể nhỏ hơn ngưỡng biểu diễn được của FP16: ”any value whose magnitude is smaller than 2^{-24} becomes zero in FP16”. Paper quan sát (Figure 2b) rằng khoảng 5% giá trị weight gradient của mô hình tiếng Quan Thoại (Mandarin speech) có exponent nhỏ hơn -24. Những gradient nhỏ này sẽ thành 0 khi nhân với learning rate nếu optimizer làm việc ở FP16, làm hại accuracy.

Tự tính: $2^{-24} = 1/16.777.216 \approx 5{,}96 \times 10^{-8}$. Đây là ngưỡng dưới của dải FP16 denormalized; bất kỳ update nhỏ hơn giá trị này coi như biến mất hoàn toàn nếu không có master copy FP32.

Lý do 2 (tỉ lệ trọng số / update quá lớn): dù update bản thân biểu diễn được trong FP16, nó có thể thành 0 khi phép cộng dịch phải (right-shift) để căn chỉnh binary point với trọng số. Hiện tượng này xảy ra khi độ lớn trọng số (đã chuẩn hoá) lớn hơn ít nhất 2.048 lần độ lớn update. Vì FP16 có 10 bit mantissa, bit ẩn (implicit bit) phải bị dịch phải từ 11 vị trí trở lên để có thể tạo ra 0 (một số trường hợp làm tròn vẫn cứu được giá trị); khi tỉ lệ lớn hơn 2.048, bit ẩn bị dịch 12 vị trí trở lên, lúc này update chắc chắn thành 0 và không thể cứu. Tỉ lệ càng lớn, hiện tượng này còn xảy ra với cả số denormalized.

Tự tính minh hoạ cơ chế này (số tự chọn để làm ví dụ, không phải số của paper): giả sử trọng số $w = 1{,}0$ (exponent 0) và update $\Delta w = 2^{-11} \approx 0{,}000488$ (tỉ lệ $w/\Delta w = 2^{11} = 2.048$, đúng ngưỡng biên paper nêu). Khi cộng $w + \Delta w$, số mũ của Δw phải dịch phải 11 bit để căn với w; vì mantissa chỉ có 10 bit cộng 1 bit ẩn (11 bit hiệu dụng), dịch đúng 11 bit đẩy bit có nghĩa cuối cùng ra khỏi vùng lưu được, update ở biên này rất dễ mất; nếu $\Delta w = 2^{-12}$ (tỉ lệ 4.096, dịch 12 bit) thì chắc chắn mất, đúng như paper mô tả.

Để minh hoạ, paper dùng mô hình tiếng Quan Thoại (mục 4.3), huấn luyện trên khoảng 800 giờ dữ liệu giọng nói, 20 epoch: khi cập nhật bản sao FP32 sau forward/backward FP16, kết quả khớp huấn luyện FP32; còn khi cập nhật trực tiếp trọng số FP16, accuracy giảm tương đối 80% (”updating FP16 weights results in 80% relative accuracy loss”).

Về chi phí bộ nhớ: giữ thêm bản sao FP32 làm tăng 50% bộ nhớ dành cho trọng số so với chỉ lưu FP32 một lần, nhưng vì bộ nhớ huấn luyện bị chi phối bởi activation (do batch size lớn và phải lưu activation của mỗi layer để dùng lại ở backward), và activation vẫn được lưu ở FP16, nên tổng bộ nhớ huấn luyện ”roughly halved” (giảm gần một nửa).

Tự tính kiểm tra khẳng định ”giảm gần một nửa” này: gọi W là kích thước bộ nhớ trọng số ở FP32 gốc, A là kích thước bộ nhớ activation ở FP32 gốc. Baseline FP32: tổng $= W + A$. Mixed precision: trọng số cần $0{,}5W$ (bản FP16) $+W$ (master FP32) $= 1{,}5W$ (đúng ”tăng 50%” paper nêu); activation cần $0{,}5A$. Tổng MP $= 1{,}5W + 0{,}5A$. Nếu $A = 9W$ (giả định minh hoạ, activation gấp 9 lần trọng số, một tỉ lệ hợp lý khi batch size lớn): baseline $= 10W$, MP $= 1{,}5W + 4{,}5W = 6W$, tỉ lệ MP/baseline $= 0{,}6$ (giảm 40%). Nếu $A = 19W$: baseline $= 20W$, MP $= 1{,}5W + 9{,}5W = 11W$, tỉ lệ $= 0{,}55$ (giảm 45%). Khi $A/W \to \infty$, tỉ lệ MP/baseline $\to 0{,}5$. Vậy ”roughly halved” là một xấp xỉ đúng ở giới hạn khi activation chi phối hoàn toàn bộ nhớ, còn mức giảm thực tế phụ thuộc tỉ lệ A/W của từng mô hình/batch size cụ thể, không phải một con số cố định; đây là nhận xét của người ghi chú, không có trong paper.

3.2 Loss scaling

FP16 exponent bias đặt trung tâm dải exponent chuẩn hoá ở $[-14, 15]$, nhưng gradient trong thực tế thường bị chi phối bởi các giá trị có độ lớn nhỏ (exponent rất âm). Paper minh hoạ bằng histogram gradient activation của mạng Multibox SSD (Figure 3): phần lớn dải biểu diễn FP16 không được dùng tới, trong khi nhiều giá trị nằm dưới dải biểu diễn tối thiểu và trở thành 0. Mạng SSD này phân kỳ (diverges) hoàn toàn nếu không scale gradient; chỉ cần scale bằng hệ số 8 (tăng exponent thêm 3) là đủ để khớp accuracy đạt được khi huấn luyện FP32. Paper nêu cụ thể: giá trị gradient activation dưới 2^{-27} về độ lớn là ”không liên quan” (irrelevant) tới huấn luyện mạng này, nhưng giá trị trong khoảng $[2^{-27}, 2^{-24})$ là ”quan trọng cần giữ lại”.

Tự tính minh hoạ cơ chế loss scaling bằng chính các số trên: lấy một gradient $g = 2^{-26}$ (nằm trong khoảng ”quan trọng” $[2^{-27}, 2^{-24})$ paper nêu). Vì $2^{-26} < 2^{-24}$ (ngưỡng underflow FP16), nếu không scale, giá trị này bị làm tròn về 0 và mất hoàn toàn. Nhân loss với $s = 8 = 2^3$ trước backward: theo chain rule, mọi gradient (kể cả g) bị nhân theo cùng hệ số, nên giá trị đã scale là $g \cdot s = 2^{-26} \cdot 2^3 = 2^{-23}$, lớn hơn ngưỡng 2^{-24} nên biểu diễn được trong FP16. Sau backward, chia lại cho 8 để ”unscale” trước khi cộng vào master weight FP32 (nơi 2^{-26} hoàn toàn biểu diễn được), khôi phục đúng giá trị gradient gốc mà không làm thay đổi ý nghĩa của learning rate hay các hyperparameter khác.

Cách thực hiện: scale giá trị loss tính được ở forward pass, trước khi bắt đầu backward; do chain rule, mọi giá trị gradient tự động bị scale theo cùng hệ số, không cần thêm phép tính nào trong backward. Weight gradient phải được unscale trước khi update trọng số, để giữ đúng độ lớn update như khi huấn luyện FP32; nên thực hiện unscale ngay sau backward pass nhưng trước gradient clipping hoặc các phép tính khác liên quan tới gradient, để không phải chỉnh lại các hyperparameter khác (ngưỡng clipping, weight decay, ...).

Về cách chọn hệ số scale: cách đơn giản nhất là một hệ số cố định (constant). Paper thử nhiều mạng với hệ số từ 8 đến 32K (nhiều mạng không cần scale). Có thể chọn hệ số theo kinh nghiệm, hoặc nếu có thống kê gradient, chọn hệ số sao cho tích của nó với giá trị gradient tuyệt đối lớn nhất vẫn dưới 65.504 (giá trị lớn nhất biểu diễn được trong FP16). Không có bất lợi khi chọn hệ số lớn, miễn là không gây tràn trên (overflow) trong backward; overflow sẽ sinh ra infinity/NaN trong weight gradient, gây hại không thể phục hồi cho trọng số sau một lần update. Overflow có thể phát hiện hiệu quả bằng cách kiểm tra weight gradient đã tính (ví dụ ngay khi unscale); một lựa chọn là bỏ qua lần update đó khi phát hiện overflow, chuyển sang iteration kế tiếp. Paper chỉ mô tả việc chọn hệ số cố định (constant scaling factor) là kỹ thuật chính dùng trong các thí nghiệm;

việc ”tự động hoá” lựa chọn hệ số (tăng/giảm linh hoạt theo phát hiện overflow) chỉ được nêu như hướng tương lai ở mục 5 (Conclusions and Future Work), chưa phải phương pháp chính được thí nghiệm trong bài.

3.3 Độ chính xác số học (Arithmetic Precision)

Paper chia phép tính mạng neural thành ba loại, mỗi loại cần xử lý độ chính xác khác nhau:

- **Tích vô hướng (vector dot-products)**: để giữ accuracy, phép tích vô hướng FP16 cần tích lũy (accumulate) các tích phần (partial products) vào một giá trị FP32, rồi mới chuyển về FP16 khi lưu vào bộ nhớ. Nếu không tích lũy ở FP32, một số mô hình FP16 không khớp accuracy của mô hình baseline. GPU đời trước chỉ hỗ trợ phép nhân-cộng (multiply-add) ở FP16; GPU NVIDIA Volta giới thiệu Tensor Core: nhân các ma trận đầu vào FP16, tích lũy kết quả vào đầu ra FP16 hoặc FP32.
- **Reduction lớn (tổng nhiều phần tử của vector)**: nên thực hiện ở FP32. Loại phép này thường xuất hiện ở lớp batch normalization (khi tích lũy thống kê mean/variance) và lớp softmax. Cả hai loại lớp này vẫn đọc/viết tensor FP16 từ bộ nhớ, nhưng thực hiện phép tính số học ở FP32; việc này không làm chậm quá trình huấn luyện vì các lớp này bị giới hạn bởi memory bandwidth, không nhạy với tốc độ số học.
- **Phép tính theo từng phần tử (point-wise)**: ví dụ hàm phi tuyến (non-linearity), tích ma trận theo từng phần tử, bị giới hạn bởi memory bandwidth; độ chính xác số học không ảnh hưởng tới tốc độ các phép này, nên có thể dùng FP16 hoặc FP32 tuỳ ý.

4. Thí nghiệm và kết quả chính

Paper chạy hai dạng thí nghiệm cho mỗi ứng dụng: **Baseline (FP32)**: lưu trữ và số học hoàn toàn FP32; **Mixed Precision (MP)**: lưu trữ và số học FP16, dùng master copy FP32 cho update, loss-scaling dùng cho một số ứng dụng, phép tính FP16 dùng Tensor Core với tích lũy FP32 cho convolution, fully-connected, và matrix multiply trong lớp recurrent. Baseline chạy trên GPU Maxwell hoặc Pascal của NVIDIA; MP chạy trên Volta V100 (tích lũy tích FP16 vào FP32). Thí nghiệm nhận dạng giọng nói MP (mục 4.3) dùng GPU Maxwell chỉ với lưu trữ FP16 (”Pseudo FP16”, mô phỏng hoạt động Tensor Core trên hardware không phải Volta); paper xác nhận cách này cho kết quả tương đương MP chạy thật trên Volta V100.

4.1 CNN cho phân loại ILSVRC: huấn luyện AlexNet, VGG-D, GoogLeNet (Inception v1), Inception v2, Inception v3, ResNet-50 pre-activation, tất cả khớp top-1 accuracy baseline FP32 với cùng hyperparameter. Dùng framework Caffe (sửa để dùng Volta TensorOps), trừ ResNet50 dùng PyTorch.

Mô hình	Baseline	Mixed Precision
AlexNet	56,77%	56,93%
VGG-D	65,40%	65,43%
GoogLeNet (Inception v1)	68,33%	68,43%
Inception v2	70,03%	70,02%
Inception v3	73,85%	74,13%
ResNet50	75,92%	76,04%

Loss-scaling *không* cần thiết cho các mạng này: toàn bộ tensor forward/backward ở FP16, chỉ master copy weight ở FP32 (mục 3.1) là đủ.

4.2 CNN cho detection: Faster-RCNN và Multibox-SSD, backbone VGG-16, đo mAP trên Pascal VOC 2007 test (Faster-RCNN huấn luyện trên VOC2007 train, SSD trên hợp VOC2007+2012, lý do khác biệt mAP baseline giữa hai mô hình).

Mô hình	Baseline	MP không loss-scale	MP có loss-scale
Faster R-CNN	69,1%	68,6%	69,7%
Multibox SSD	76,9%	phân kỳ (diverges)	77,1%

SSD *không* huấn luyện được ở FP16 nếu thiếu loss-scaling: mất các giá trị gradient nhỏ về 0 làm trọng số học kém, dẫn tới phân kỳ; hệ số loss-scale bằng 8 (như nêu ở mục 3.2) khôi phục mAP khớp FP32.

4.3 Nhận dạng giọng nói: mô hình DeepSpeech 2 cho tiếng Anh và tiếng Quan Thoại. Mô hình tiếng Anh: 2 lớp convolution 2D, 3 lớp recurrent GRU, 1 lớp row convolution, lớp cost CTC (Connectionist Temporal Classification), khoảng 115 triệu tham số, huấn luyện trên dữ liệu nội bộ 6.000 giờ tiếng Anh. Mô hình tiếng Quan Thoại kiến trúc tương tự, 215 triệu tham số, huấn luyện trên 2.600 giờ dữ liệu nội bộ. Cả hai huấn luyện 20 epoch với Nesterov SGD, cùng hyperparameter cho baseline và pseudo FP16.

Mô hình/dataset	Baseline (CER)	Mixed Precision (CER)
Tiếng Anh (test WSJ'92)	2,20	1,99
Tiếng Quan Thoại (test nội bộ)	15,82	15,01

CER (Character Error Rate) của mixed precision thấp hơn baseline khoảng 5-10%, tức *tốt hơn* chứ không chỉ "bằng". Paper suy đoán: định dạng lưu trữ nửa độ chính xác có thể đóng vai trò như một dạng regularizer trong huấn luyện ("the half-precision storage format may act as a regularizer during training").

4.4 Dịch máy: các biến thể mô hình trong TensorFlow tutorial dịch Anh-Pháp, vocabulary 100K (Anh) và 40K (Pháp), 3 hoặc 5 tầng encoder/decoder, mỗi tầng 1024 LSTM cell, optimizer SGD, dữ liệu WMT15. Có biến động đáng kể giữa các lần huấn luyện FP32 khác nhau với cùng cấu hình (3 đường cong FP32 tham chiếu trong Figure 4 cho mô hình 3 tầng). Mixed-precision có loss-scaling khớp kết quả FP32; không loss-scaling thì kết quả giảm nhẹ. Mô hình 5 tầng có hành vi tương tự.

4.5 Mô hình ngôn ngữ: bigLSTM (Jozefowicz et al., 2016) trên tập 1 tỷ từ. Mô hình gồm 2 tầng 8192 LSTM cell, chiếu (projection) xuống embedding 1024 chiều, huấn luyện 50 epoch với Adagrad, vocabulary 793K từ, dùng sampled softmax với 8K negative sample, batch size tổng hợp qua 4 GPU là 1024. Để khớp perplexity FP32, cần loss-scaling (Figure 5): không scale thì perplexity FP16 phân kỳ so với FP32 sau 300K iteration; hệ số scale 128 khôi phục toàn bộ giá trị gradient liên quan và khớp baseline.

4.6 DCGAN: sinh ảnh khuôn mặt 128x128 pixel trên dataset CelebFaces (phương pháp DCGAN). Generator: 7 tầng tích chập dải phân đoạn (fractionally-strided convolution), 6 tầng leaky ReLU + 1 tầng tanh. Discriminator: 6 tầng convolution + 2 tầng fully-connected, leaky ReLU trừ tầng cuối (sigmoid). Batch normalization áp dụng cho mọi tầng trừ tầng fully-connected cuối của discriminator. Optimizer Adam, huấn luyện 100K iteration. Paper tự nhận GAN không có thước đo định lượng được công nhận rộng rãi; so sánh chất lượng chỉ định tính (qualitative), kết quả FP32 và mixed-precision "comparable". Mạng này *không* cần loss-scaling.

5. Phân tích / ablation

Nhìn tổng hợp 6 nhóm ứng dụng, có một mẫu hình rõ: **loss-scaling cần hay không phụ thuộc loại bài toán**, không phải một thuộc tính cố định của "mixed precision nói chung":

- **Không cần loss-scaling**: phân loại ảnh ILSVRC (6 kiến trúc CNN, mục 4.1), DCGAN (mục 4.6).
- **Cần loss-scaling để khớp/không phân kỳ**: detection SSD (mục 4.2, SSD phân kỳ hoàn toàn nếu thiếu), mô hình ngôn ngữ bigLSTM (mục 4.5, phân kỳ sau 300K iteration nếu thiếu), dịch máy (mục 4.4, giảm nhẹ chất lượng nếu thiếu, không phân kỳ hẳn).

- **Faster R-CNN**: là trường hợp trung gian, MP không loss-scale giảm nhẹ (69,1% → 68,6%) nhưng không phân kỳ, loss-scale đưa về mức tốt hơn cả baseline (69,7%).

Lý do hợp lý (suy luận của người ghi chú dựa trên phần 3.2 của paper, không phải khẳng định trực tiếp của paper): các bài toán phân loại ảnh dùng cross-entropy với gradient tương đối ”đậm” (nhiều giá trị không quá nhỏ); còn object detection (regression toạ độ bounding box) và các mô hình recurrent lan truyền qua rất nhiều time-step (RNN/LSTM) có xu hướng tạo nhiều gradient rất nhỏ (nhân nhiều số nhỏ liên tiếp qua các bước thời gian), dễ rơi xuống dưới ngưỡng 2^{-24} của FP16 hơn.

Một điểm phân tích quan trọng khác là tính tương đương ”Pseudo FP16” (lưu FP16, tính bằng phần cứng không có Tensor Core) với ”MP thật” (Tensor Core trên Volta, tích lũy FP16→FP32): paper xác nhận hai cách cho accuracy tương đương cho mô hình giọng nói, hỗ trợ lập luận rằng lợi ích chính của phương pháp này đến từ việc lưu trữ ở FP16 và từ việc tích lũy ở FP32 trong lúc tính (mục 3.3), không phụ thuộc vào có Tensor Core phần cứng cụ thể hay không, miễn là nguyên tắc ”accumulate ở FP32” được giữ đúng.

Kết quả bất ngờ nhất (và paper chỉ nêu như một quan sát, không đi sâu phân tích cơ chế) là CER của nhận dạng giọng nói với mixed precision *tốt hơn* baseline 5-10%. Đây là một quan sát thực nghiệm chưa được paper giải thích đầy đủ bằng lý thuyết (chỉ có một câu suy đoán ngắn về regularization), nên nên coi là một gợi ý cần kiểm chứng thêm, không phải kết luận chắc chắn.

6. Giới hạn

- Paper không báo cáo số liệu tốc độ (wall-clock speedup) thực tế cho các mô hình case-study chính (CNN, detection, speech, dịch máy, mô hình ngôn ngữ, GAN) ở mục 4; phần tốc độ duy nhất được định lượng là benchmark vi mô (micro-benchmark) các phép toán DNN riêng lẻ bằng công cụ DeepBench trên GPU Volta, cho tốc độ tăng 2-6 lần so với FP32 khi bị giới hạn bởi memory hoặc arithmetic bandwidth (thấp hơn khi bị giới hạn bởi latency). Paper tự nhận ”speedups depend on library and framework optimizations... and are a focus of future work” và rằng thí nghiệm trong bài dùng ”early versions of both libraries and frameworks” (thư viện/framework giai đoạn đầu), nên tốc độ thực tế lúc đó thấp hơn tiềm năng lý thuyết.
- Phương pháp chọn hệ số loss-scaling trong paper là hằng số cố định, chọn bằng tay/kinh nghiệm hoặc dựa trên quan sát thống kê gradient trước đó; việc tự động điều chỉnh hệ số (tăng/giảm linh hoạt dựa trên phát hiện overflow) chỉ được gợi ý ở phần Future Work, chưa được thí nghiệm hay đánh giá trong bài.
- GAN (mục 4.6) được đánh giá hoàn toàn định tính, không có chỉ số định lượng nào so sánh FP32 với MP, vì theo paper ”GANs do not have a widely-accepted quantification of their result quality”.
- Paper chỉ xét định dạng FP16 (half-precision theo chuẩn IEEE 754); không đề cập tới định dạng BF16 (bfloat16, cùng 8 bit exponent như FP32 nhưng chỉ 7 bit mantissa) vì BF16 chưa phổ biến trên GPU ở thời điểm bài được công bố (2017-2018). Đây là một giới hạn về thời điểm, không phải lỗi của paper, nhưng cần lưu ý khi áp dụng cho phần cứng/framework hiện đại (BF16 có dải giá trị rộng như FP32 nên thường không cần loss-scaling, khác biệt lớn so với FP16 được paper phân tích).
- Các thí nghiệm MP ”thật” (không phải Pseudo FP16) đều chạy trên GPU Volta V100 với Tensor Core cụ thể; paper không khảo sát liệu kết luận có giữ nguyên trên các kiến trúc GPU khác (ví dụ không có Tensor Core, hoặc Tensor Core thế hệ khác) hay không.
- Phần phân tích ”FP32 master copy” và ”right-shift làm mất update” (mục 3.1) chỉ đưa ra lập luận định tính kèm ngưỡng số (2.048, 11/12 bit dịch); paper không chứng minh chặt bằng công thức toán tổng quát cho mọi trường hợp rounding mode, chỉ mô tả cơ chế ở mức trực giác.

7. Liên hệ với các bài khác

- **opt 38** (Batch Normalization, đã có trong thư mục ”optimization content/”): liên hệ trực tiếp vì paper 42 nói rõ các phép reduction trong batch normalization (tích lũy thống kê mean/variance) cần thực hiện ở

FP32 (mục 3.3) để không mất accuracy khi huấn luyện mixed precision; opt 38 giải thích chi tiết chính các thống kê đó.

- **opt 39** (Layer Normalization, đã có trong thư mục): cùng lý do, LayerNorm cũng là một phép reduction (tính mean/variance theo chiều feature) nên cùng chịu ràng buộc ”accumulate FP32” nêu ở mục 3.3 của bài 42; so sánh BN và LN giúp hiểu rõ hơn vì sao cả hai kiểu normalization đều nhạy với độ chính xác số khi tích lũy thống kê.
- **opt 21** (Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour, theo titles.tsv; đã có file trên đĩa là ”21_2017_large_minibatch_sgd.md”): cùng nhóm ”huấn luyện nhanh/hiệu quả ở quy mô lớn”; cả hai bài đều nhằm giảm thời gian huấn luyện mà không đổi kết quả cuối, một bằng kỹ thuật batch size/learning rate, một bằng kỹ thuật số học.
- **opt 10** (Large Batch Optimization for Deep Learning: Training BERT in 76 minutes, theo titles.tsv; file trên đĩa ”10_2019_lamb.md”): huấn luyện BERT nhanh ở quy mô lớn trong thực tế thường kết hợp cả large-batch optimizer (LAMB) và mixed precision; hai kỹ thuật bổ sung cho nhau (một giảm số bước, một giảm chi phí mỗi bước).
- **opt 16** (Muon is Scalable for LLM Training, theo titles.tsv, chưa có file lúc viết bài này): các pipeline huấn luyện LLM hiện đại, bất kể dùng optimizer nào, hầu như luôn kết hợp với huấn luyện độ chính xác thấp (FP16/BF16); bài 42 là nền tảng kỹ thuật cho điều đó.
- **llm content/09_2020_pre_ln_transformer.md** (đã ls-verify tồn tại): Pre-LN giúp ổn định gradient khi mạng sâu/scale lớn; ổn định số học (ít giá trị cực lớn/cực nhỏ) cũng là điều kiện thuận lợi để huấn luyện mixed precision không cần loss-scaling phức tạp, nên hai kỹ thuật có tác động hỗ trợ lẫn nhau trong thực hành huấn luyện Transformer lớn (suy luận liên hệ của người ghi chú, hai bài không trực tiếp trích dẫn nhau).

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm khoảng 1.500 token text + 1.600-2.100 token visual, chạy trên GPU <= 8 GB, batch size 1). Đây là bài rất liên quan tới việc fine-tune/distill thực tế trên phần cứng hạn chế, với vài suy luận cụ thể (tất cả đều là suy luận, không phải nội dung paper):

- Trên GPU <= 8 GB, dùng độ chính xác thấp hơn FP32 gần như là điều kiện cần (không phải chỉ ”nên”) để vừa model 0.8B tham số cùng activation cho context dài (~3.600 token/trang) ở batch size 1. Theo cơ chế mục 3.1, nên giữ optimizer state/master weight ở FP32 (hoặc ít nhất ở độ chính xác cao hơn FP16 thuần) để tránh mất mát update nhỏ như paper mô tả, đặc biệt khi fine-tune với learning rate nhỏ (update nhỏ dễ underflow hơn).
- Các GPU hiện đại (Ampere trở lên) hỗ trợ BF16, có dải exponent giống FP32 (8 bit) nên ít cần loss-scaling hơn FP16 (chỉ mất độ chính xác mantissa, không mất dải giá trị). Nếu GPU 8 GB của dự án thuộc dòng hỗ trợ BF16, nên ưu tiên BF16 thay FP16 để tránh toàn bộ phần phức tạp ”chọn hệ số loss-scale” ở mục 3.2 (paper này viết trước khi BF16 phổ biến nên không đề cập, như đã nêu ở mục 6). Nếu chỉ có FP16 (GPU cũ hơn), cần bật loss-scaling (tự động, kiểu ”dynamic loss scaling” mà paper chỉ gợi ý ở Future Work, nay là chuẩn mặc định trong hầu hết framework hiện đại như PyTorch AMP) khi thấy loss NaN/Inf hoặc gradient dường như ”biến mất” bất thường.
- Theo mục 3.3, các phép reduction trong normalization layer (GatedDeltaNet/attention trong LightOnOCR nhiều khả năng dùng RMSNorm hoặc LayerNorm, xem opt 39) và trong softmax (attention) nên accumulate ở FP32 dù lưu trữ ở FP16/BF16; hầu hết framework hiện đại (PyTorch autocast) đã tự làm điều này, nhưng nếu tự viết custom kernel cho GatedDeltaNet (một kiến trúc gated linear-RNN/SSM ít phổ biến hơn Transformer chuẩn) thì cần tự đảm bảo nguyên tắc này, không mặc định framework lo hết.
- Kết quả mục 4.3 (mixed precision cho CER tốt hơn baseline ở speech) gợi ý rằng lo lắng ”giảm precision sẽ luôn làm giảm chất lượng” là chưa chắc đúng; với fine-tune nhẹ (ví dụ chỉ vài epoch, learning rate

nhỏ) trên 0.8B model, có thể kỳ vọng mixed precision không làm giảm (thậm chí đôi khi cải thiện nhẹ) chất lượng OCR, nhưng đây chỉ là loại suy từ một domain khác (speech, không phải OCR/VLM), cần tự kiểm chứng trên task cụ thể.

9. Thí nghiệm gợi ý

1. **Đo ngưỡng underflow FP16 bằng thực nghiệm nhỏ**: dùng PyTorch, tạo tensor gradient giả với nhiều giá trị ở các exponent khác nhau (ví dụ 2^{-20} đến 2^{-30}), ép kiểu `torch.float16`, đếm bao nhiêu giá trị thành 0. Kỳ vọng: giá trị dưới khoảng 2^{-24} bắt đầu mất, khớp với ngưỡng paper nêu ở mục 3.1; diễn giải: xác nhận trực tiếp cơ chế underflow mà không cần GPU mạnh, chỉ cần CPU.
2. **Tái tạo hiện tượng "right-shift mất update"**: trong PyTorch/NumPy với dtype `float16`, lấy $w = 1{,}0$ và cộng lần lượt các $\Delta w = 2^{-k}$ với k tăng dần từ 9 đến 13, in ra $w + \Delta w - w$ (giá trị update "còn lại" sau phép cộng). Kỳ vọng: với $k \geq 12$ (tỉ lệ $w/\Delta w \geq 4096 > 2048$), hiệu số về 0 hoàn toàn, khớp ngưỡng paper mô tả ở mục 3.1; với $k = 11$ (ranh giới 2048) có thể còn cứu được nhờ rounding. Diễn giải: hiểu trực quan vì sao cần giữ master weight ở FP32 khi learning rate nhỏ hoặc model đã gần hội tụ (update nhỏ so với weight).
3. **So sánh accuracy/memory FP32 vs FP16 (không loss-scale) vs FP16 (có loss-scale)** trên một mô hình toy (ví dụ CNN nhỏ hoặc LSTM nhỏ) cho một bài toán có gradient rất nhỏ (ví dụ RNN chạy nhiều time-step, hoặc regression với target scale nhỏ, mô phỏng tình huống giống SSD mục 4.2). Đo: train loss có phân kỳ không, test accuracy cuối, bộ nhớ GPU dùng (`torch.cuda.max_memory_allocated`). Kỳ vọng: FP16 không scale có thể phân kỳ hoặc kém hơn rõ rệt, FP16 có scale khớp FP32 nhưng dùng ít bộ nhớ hơn. Diễn giải: xác nhận bài học chính của paper (loss-scaling cần cho bài toán gradient nhỏ) trên quy mô 1 GPU 8 GB.
4. **Thử `torch.cuda.amp.autocast` với BF16 so với FP16** trên cùng mô hình toy (ví dụ fine-tune một mạng nhỏ vài epoch), không dùng loss-scaling cho cả hai. Đo: FP16 không scale có thể NaN/phân kỳ ở learning rate cao, BF16 không scale vẫn ổn định hơn (do dải exponent rộng như FP32). Diễn giải: minh hoạ trực tiếp lý do nên ưu tiên BF16 khi phần cứng hỗ trợ, như nêu ở mục 8.
5. **Đo tốc độ (token/s hoặc sample/s) và bộ nhớ** khi fine-tune một mô hình nhỏ (ví dụ 100-300 triệu tham số, giả lập quy mô nhỏ của LightOnOCR) ở FP32 thuần so với mixed precision (FP16/BF16 autocast, optimizer state FP32) trên 1 GPU 8 GB, với batch size cố định = 1 (giống ràng buộc thực tế của dự án). Kỳ vọng: mixed precision cho phép tăng batch size hoặc sequence length tối đa trước khi hết bộ nhớ (out of memory), và tốc độ mỗi bước nhanh hơn. Diễn giải: định lượng trực tiếp lợi ích paper tuyên bố ("nearly halves memory... speeds up arithmetic") trên đúng ràng buộc phần cứng của dự án.

10. Tóm tắt công thức / khái niệm

- **Dải biểu diễn FP16**: exponent chuẩn hoá trong $[-14, 15]$ (paper); giá trị lớn nhất biểu diễn được 65.504 (paper); ngưỡng dưới (underflow hoàn toàn) $2^{-24} \approx 5{,}96 \times 10^{-8}$ (tự tính từ con số paper nêu).
- **Cập nhật trọng số (FP32 master copy)**:

$$w_{32} \leftarrow w_{32} - \eta \cdot g_{32}$$

  trong đó w_{32} là trọng số FP32 (master), η là learning rate, g_{32} là gradient đã tính ở FP16 (forward/backward) rồi đưa về FP32 (và unscale nếu có loss-scaling) trước khi cộng vào w_{32}. Bản FP16 của w_{32} được tạo lại mỗi iteration để dùng cho forward/backward tiếp theo.
- **Loss scaling**: với hệ số scale s,

$$L' = s \cdot L, \qquad g_w^{scaled} = s \cdot g_w \text{ (tự động do chain rule)}, \qquad g_w = \frac{g_w^{scaled}}{s} \text{ (unscale trước khi update)}$$

- **Điều kiện chọn s (paper gợi ý)**: chọn s sao cho

$$s \cdot \max_i |g_i| < 65.504$$

  để tránh overflow trong backward, với g_i là các giá trị gradient trước khi scale.
- **Ngưỡng "right-shift mất update"**: update bị mất hoàn toàn khi

$$\frac{|w|}{|\Delta w|} > 2.048 = 2^{11}$$

  (tỉ lệ độ lớn trọng số so với update), vì FP16 có 10 bit mantissa + 1 bit ẩn, chỉ chịu được dịch tối đa khoảng 11 bit trước khi mất thông tin không thể cứu bằng rounding.
- **Ba loại phép tính cần xử lý khác nhau (mục 3.3)**: dot-product (accumulate FP32), reduction lớn như BN/softmax (tính ở FP32, đọc/viết FP16), point-wise (FP16 hoặc FP32 tuỳ ý, không ảnh hưởng tốc độ).

11. Kết luận ngắn

Mixed Precision Training chứng minh rằng có thể huấn luyện mạng neural hoàn toàn bằng FP16 cho forward/backward (giảm gần một nửa bộ nhớ, tăng tốc số học 2-8 lần theo throughput GPU) mà không mất accuracy và không cần đổi hyperparameter, với điều kiện áp dụng ba kỹ thuật bù trừ: giữ master weight FP32, loss-scaling để tránh mất gradient nhỏ, và accumulate FP32 cho dot-product/reduction. Paper kiểm chứng trên 6 nhóm ứng dụng khác nhau (phân loại ảnh, detection, nhận dạng giọng nói, dịch máy, mô hình ngôn ngữ, GAN), cho thấy nhu cầu loss-scaling khác nhau rõ rệt theo loại bài toán (RNN/regression cần hơn CNN phân loại). Đây là một trong những nền tảng kỹ thuật quan trọng nhất cho việc huấn luyện mô hình lớn hiện đại trên phần cứng hạn chế, bao gồm cả việc fine-tune/distill các VLM nhỏ như trong dự án đang quan tâm.

Nguồn gốc: https://arxiv.org/abs/1710.03740

Bài 43

The Marginal Value of Adaptive Gradient Methods in Machine Learning (2017)

Tác giả: Ashia C. Wilson, Rebecca Roelofs, Mitchell Stern (University of California, Berkeley); Nathan Srebro (Toyota Technological Institute at Chicago); Benjamin Recht (University of California, Berkeley).
Venue / arXiv: arXiv:1705.08292 (v2, 22/05/2018); venue: ”31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA” (dòng này ghi ở footer trang 1 của văn bản).
Nguồn: https://arxiv.org/abs/1705.08292
Chủ đề: so sánh khả năng tổng quát hoá (generalization) giữa các optimizer thích ứng (AdaGrad, RMSProp, Adam) và các optimizer không thích ứng (SGD, SGD với momentum/Heavy Ball), qua cả một ví dụ lý thuyết dựng sẵn và thực nghiệm trên 4 bài toán deep learning.
Vị trí trong lộ trình: đọc sau opt 06 (Adam, đã có file) vì bài 43 phản biện trực tiếp chính thuật toán này; đọc cùng opt 07 (AdamW) và opt 08 (AMSGrad), hai bài cũng sửa/phê bình Adam nhưng theo hướng khác (weight decay và hội tụ lý thuyết, không phải generalization thực nghiệm); liên hệ opt 41 (Why gradient clipping accelerates training, bàn về adaptivity) ở góc nhìn lý thuyết. Bài 43 không đề xuất optimizer mới mà đặt câu hỏi nền tảng ”optimizer thích ứng có luôn tốt hơn không”, nên đọc trước khi đánh giá các optimizer phức tạp hơn xuất hiện sau trong lộ trình (opt 11 Lion, opt 13 Sophia, opt 16 Muon) để có góc nhìn phản biện.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận.

1. Bối cảnh

Theo paper, ngày càng nhiều nhà nghiên cứu huấn luyện mạng neural bằng các optimizer ”thích ứng” (adaptive gradient methods) vì tốc độ hội tụ trên tập train nhanh; Adam nói riêng ”has become the default algorithm used across many deep learning frameworks” (đã trở thành thuật toán mặc định trong nhiều framework deep learning). Tuy vậy, hành vi tổng quát hoá (generalization, khả năng làm tốt trên dữ liệu chưa thấy) và out-of-sample của các phương pháp này ”remains poorly understood” (còn chưa được hiểu rõ).

Một quan sát quan trọng paper nêu ở Introduction: khi số tham số vượt số điểm dữ liệu (overparameterized), lựa chọn thuật toán có thể ảnh hưởng mạnh tới việc mô hình nào được học ra, vì có vô số nghiệm đạt loss train bằng 0. Câu hỏi trung tâm: cho hai nghiệm cực tiểu khác nhau của cùng một bài toán tối ưu, có thể nói gì về khả năng tổng quát hoá tương đối của chúng?

Khái niệm cần nắm trước: ”generalization” ở đây là hiệu năng của nghiệm w trên một tập dữ liệu rộng hơn (ngoài tập train), thường đo bằng một loss khác với loss dùng để train (ví dụ train bằng cross-entropy nhưng đánh giá generalization bằng classification error, paper nêu rõ ở mục 2). ”Adaptive gradient methods” (AdaGrad, RMSProp, Adam) là các optimizer dùng một ma trận khoảng cách cục bộ (local distance metric) xây dựng từ toàn bộ lịch sử các iterate, khác với SGD/momentum dùng cùng một ”hình học” ℓ_2 cố định cho mọi toạ độ.

2. Ý tưởng chính

Paper đưa ra hai bằng chứng độc lập cho cùng một luận điểm: **các optimizer thích ứng thường hội tụ về nghiệm tổng quát hoá kém hơn SGD/SGD+momentum, dù huấn luyện nhanh hơn trên tập train.**

1. **Lý thuyết (mục 3):** paper dựng một bài toán phân loại nhị phân bằng bình phương tối thiểu (least-squares classification), trong đó dữ liệu quần thể là phân tách tuyến tính (tồn tại nghiệm margin lớn),

nhưng AdaGrad, RMSProp, Adam hội tụ về một nghiệm phân loại sai dữ liệu mới với xác suất tiến gần $1/2$ (gần như đoán ngẫu nhiên), trong khi SGD/GD tìm được nghiệm có lỗi test bằng 0.

2. **Thực nghiệm (mục 4):** trên 4 bài toán deep learning hiện đại (phân loại ảnh CIFAR-10, mô hình ngôn ngữ ký tự, hai tác vụ parsing), với cùng mức độ công sức tinh chỉnh hyperparameter cho mọi thuật toán, SGD và Heavy Ball (HB, SGD+momentum) luôn cho hiệu năng dev/test bằng hoặc tốt hơn AdaGrad/RMSProp/Adam, kể cả khi các optimizer thích ứng đạt training loss bằng hoặc thấp hơn.

Một luận điểm phụ nhưng quan trọng: paper bác bỏ quan niệm phổ biến "Adam không cần tinh chỉnh nhiều"; thực tế tinh chỉnh learning rate ban đầu và lịch decay cho Adam mang lại cải thiện đáng kể so với cấu hình mặc định, trong mọi trường hợp thử nghiệm.

3. Phương pháp chi tiết

3.1 Khung công thức chung

Paper viết lại các optimizer phổ biến dưới một khung chung. Gradient method và biến thể mini-batch:

$$w_{k+1} = w_k - \alpha_k \nabla \tilde{f}(w_k) \tag{2.1}$$

Phương pháp momentum (bao gồm Heavy Ball của Polyak với $\gamma_k = 0$, và Nesterov Accelerated Gradient với $\gamma_k = \beta_k$):

$$w_{k+1} = w_k - \alpha_k \nabla \tilde{f}(w_k + \gamma_k(w_k - w_{k-1})) + \beta_k(w_k - w_{k-1}) \tag{2.2}$$

Các phương pháp thích ứng (AdaGrad, RMSProp, Adam) dùng một ma trận xác định dương H_k xây dựng từ toàn bộ lịch sử iterate:

$$w_{k+1} = w_k - \alpha_k H_k^{-1} \nabla \tilde{f}(w_k + \gamma_k(w_k - w_{k-1})) + \beta_k H_k^{-1} H_{k-1}(w_k - w_{k-1}) \tag{2.3}$$

với H_k thường là ma trận đường chéo:

$$H_k = \text{diag}\left(\left(\sum_{i=1}^{k} \eta_i \, g_i \circ g_i \right)^{1/2} \right) \tag{2.4}$$

trong đó $\circ$ là tích Hadamard (theo từng phần tử), $g_k = \nabla \tilde{f}(w_k + \gamma_k(w_k - w_{k-1}))$, và η_i là hệ số riêng của từng thuật toán. Nói cách khác, H_k là ma trận đường chéo mà mỗi phần tử là căn bậc hai của một tổ hợp tuyến tính các bình phương gradient trong quá khứ, theo từng toạ độ riêng. SGD tương đương với việc đặt $H_k = I$ (dùng chung hình học ℓ_2 cho mọi toạ độ), còn các phương pháp thích ứng cố điều chỉnh theo hình học riêng của dữ liệu ở từng toạ độ.

Paper cho bảng cụ thể các hệ số (Table 1 của paper, $G_k := H_k \circ H_k$, $D_k := \text{diag}(g_k \circ g_k)$):

Thuật toán	G_k	α_k	β_k	γ
SGD	I	α	0	0
HB (Heavy Ball)	I	α	β	0
NAG (Nesterov)	I	α	β	β
AdaGrad	$G_{k-1} + D_k$	α	0	0
RMSProp	$\beta_2 G_{k-1} + (1-\beta_2) D_k$	α	0	0
Adam	(xem chú thích)	(xem chú thích)	(xem chú thích)	0

Ghi chú quan trọng: dòng Adam trong Table 1 của paper bị lỗi hiển thị khi convert PDF sang text (cột α_k và β_k của Adam bị pdftotext trộn chữ, các phân số xếp lệch dòng/cột so với các thuật toán khác, một phần ký hiệu căn

bậc hai bị mất). Vì vậy, thay vì trích "theo đúng văn bản bị lỗi hiển thị" có nguy cơ sai, ở đây dùng công thức bias-correction tiêu chuẩn của Adam (kiến thức chung, đã trình bày ở opt 06): $\alpha_k = \alpha\sqrt{1-\beta_2^k}/(1-\beta_1^k)$ và $\beta_k = \beta_1(1-\beta_1^{k-1})/(1-\beta_1^k)$; mảnh ký tự còn đọc được trong bản OCR của paper (ví dụ cụm "$\beta_1(1-\beta_1^{k-1})$" trên "$1-\beta_1^k$") khớp với công thức β_k này, nên nhiều khả năng đây đúng là nội dung gốc của Table 1, nhưng không khẳng định chắc 100% vì không đọc được bản PDF gốc (chỉ có text đã convert).

3.2 Bài toán tối ưu minh hoạ: phân loại nhị phân bằng bình phương tối thiểu

Để so sánh nghiệm mà các thuật toán tìm ra, paper dùng bài toán đơn giản nhất có thể giải closed-form:

$$\min_w R_S[w] := \tfrac{1}{2}\|Xw - y\|_2^2 \tag{3.1}$$

với X là ma trận đặc trưng $n \times d$, $y \in \{-1, 1\}^n$. Khi $d > n$ (nhiều tham số hơn số điểm dữ liệu, giống deep learning) và tồn tại nghiệm loss 0, có vô số nghiệm cực tiểu toàn cục. Câu hỏi: thuật toán nào tìm nghiệm nào, và nghiệm đó làm tốt trên dữ liệu mới ra sao?

Nghiệm của phương pháp không thích ứng (mục 3.1 của paper): mọi gradient hoặc stochastic gradient của R_S đều nằm trong row-span của X. Nếu khởi tạo trong row-span của X (ví dụ $w_0 = 0$) và chỉ dùng tổ hợp tuyến tính của các gradient/iterate trước, nghiệm cuối cũng nằm trong row-span của X. Nghiệm duy nhất trong row-span thoả $Xw = y$ chính là nghiệm có chuẩn Euclid (ℓ_2) nhỏ nhất:

$$w_{SGD} = X^T(XX^T)^{-1}y$$

Hầu hết phương pháp không thích ứng (SGD, SGD+momentum, mini-batch SGD, GD, Nesterov, conjugate gradient) hội tụ về đúng nghiệm chuẩn nhỏ nhất này, và nghiệm chuẩn nhỏ nhất cũng là nghiệm có margin lớn nhất trong số các nghiệm của $Xw = y$. Đây là lý do GD "tự nhiên" tìm nghiệm max-margin, một tính chất được paper mô tả là "a pleasant surprise" (một sự bất ngờ thú vị).

Nghiệm của phương pháp thích ứng (mục 3.2 của paper, Lemma 3.1): với H_k đường chéo, nếu tồn tại một hằng số c sao cho $X\,\text{sign}(X^Ty) = cy$, thì khi khởi tạo tại $w_0 = 0$, AdaGrad, Adam, RMSProp đều hội tụ về cùng một nghiệm duy nhất $w \propto \text{sign}(X^Ty)$ (tỉ lệ với dấu của X^Ty, nghĩa là mọi thành phần chỉ nhận giá trị $\pm\tau$ cho một hằng số τ nào đó, không phân biệt độ lớn tương đối giữa các toạ độ). Paper chứng minh bằng quy nạp rằng quỹ đạo $w_k = \lambda_k\,\text{sign}(X^Ty)$ cho một dãy vô hướng λ_k, tận dụng việc H_k cũng tỉ lệ với $|X^Ty|$ theo từng toạ độ khi điều kiện trên đúng.

3.3 Khi tính thích ứng gây overfit: ví dụ dựng sẵn

Paper xây dựng một mô hình sinh dữ liệu cụ thể thoả điều kiện Lemma 3.1, cho thấy AdaGrad có thể tổng quát hoá tuỳ ý kém. Với n mẫu, nhãn $y_i = 1$ với xác suất $p > 1/2$ và $y_i = -1$ với xác suất $1-p$. Đặc trưng x_i (vô hạn chiều để đơn giản việc tính toán, nhưng paper ghi chú có thể lấy chiều hữu hạn $6n$; paper cũng nhận xét deep learning thường có số tham số bằng $25n$ hoặc hơn, nên đây "không phải một ví dụ chiều cao bất thường so với thực tế hiện nay"):

- Đặc trưng 1: chính là nhãn y_i.
- Đặc trưng 2 và 3: luôn bằng 1 (hằng số, không mang thông tin phân biệt).
- Một khối đặc trưng riêng cho từng i: nếu $y_i = 1$ thì có đúng 1 đặc trưng riêng bằng 1; nếu $y_i = -1$ thì có 5 đặc trưng riêng bằng 1. Các đặc trưng riêng này chỉ "bật" cho đúng mẫu i đó, không mẫu nào khác.
- Mọi đặc trưng khác bằng 0.

Chỉ đặc trưng 1 (chính nhãn) là có ích để phân loại dữ liệu ngoài tập train; các đặc trưng riêng-theo-mẫu là nhiễu thuần (chỉ "khớp" đúng 1 mẫu train, vô dụng cho dữ liệu mới), còn đặc trưng 2-3 là hằng số.

Paper chứng minh (đặt $b = \sum_i y_i > 0$, đúng với xác suất cao khi n lớn) điều kiện Lemma 3.1 được thoả,

nên nghiệm AdaGrad là $w_{ada} \propto \text{sign}(X^T y)$, và vì w_{ada} có 3 thành phần đầu cùng dấu nhau (do cùng dấu với $u = X^T y$ ở các toạ độ 1-3), với một điểm test mới x_{test} (chỉ có đặc trưng 1-3 khác 0, các đặc trưng riêng-theo-mẫu khác đều bằng 0 với điểm mới), ta có:

$$\langle w_{ada}, x_{test} \rangle = \tau(y^{(test)} + 2) > 0 \quad \text{luôn đúng, bất kể } y^{(test)} \text{ là } 1 \text{ hay } -1$$

Vì vậy AdaGrad sẽ gán nhãn dương cho **mọi** điểm dữ liệu chưa thấy, tức lỗi phân loại bằng chính xác $1 - p$ (tỉ lệ nhãn âm thật trong quần thể).

Ngược lại, nghiệm chuẩn nhỏ nhất (SGD) có dạng $w_{SGD} = \sum_{i \in P} \alpha_+ x_i + \sum_{j \in N} \alpha_- x_j$ (với P, N là tập mẫu dương/âm, $n_+ = |P|$, $n_- = |N|$), giải hệ $XX^T \alpha = y$ cho nghiệm closed-form:

$$\alpha_+ = \frac{4n_- + 3}{9n_+ + 3n_- + 8n_+ n_- + 3}, \qquad \alpha_- = \frac{4n_+ + 1}{9n_+ + 3n_- + 8n_+ n_- + 3} \tag{3.2}$$

(đạo hàm đầy đủ bằng kernel trick nằm ở Appendix A của paper). Với điểm test mới, $\langle w_{SGD}, x_{test} \rangle = y^{(test)}(n_+\alpha_+ - n_-\alpha_-) + 2(n_+\alpha_+ + n_-\alpha_-)$; thay (3.2) vào, paper chỉ ra: **bất cứ khi nào $n_+ > n_-/3$, nghiệm SGD phân loại đúng hoàn toàn (không lỗi).**

Appendix C hình thức hoá bằng một phân phối sinh dữ liệu thật (không chỉ cố định n mẫu), đưa ra chặn: lỗi SGD $\leq n^2/(2N)$, lỗi AdaGrad $\geq (1-p)(1 - n/N)(1 - n^2/(2N))$, với N là kích thước "vũ trụ" mẫu có thể và n là số mẫu train rút ra (i.i.d). Khi $N \to \infty$ với n cố định, lỗi SGD $\to 0$ còn lỗi AdaGrad $\to 1 - p$, khớp với kết quả cực đoan ở trên.

Tự tính minh hoạ bằng số cụ thể theo đúng hai công thức chặn trên: lấy $p = 0{,}9$ (90% mẫu là nhãn dương), $n = 10$ mẫu train, $N = 10.000$ (kích thước quần thể/vũ trụ mẫu có thể). Chặn trên lỗi SGD: $n^2/(2N) = 100/20.000 = 0{,}005$, tức lỗi SGD $\leq 0{,}5\%$. Chặn dưới lỗi AdaGrad: $(1-p)(1 - n/N)(1 - n^2/(2N)) = 0{,}1 \times (1 - 0{,}001) \times (1 - 0{,}005) = 0{,}1 \times 0{,}999 \times 0{,}995 \approx 0{,}0994$, tức lỗi AdaGrad $\geq$ khoảng $9{,}94\%$. Vậy ở quy mô N vừa phải này, AdaGrad đã có lỗi tối thiểu gần 20 lần lỗi tối đa của SGD ($0{,}0994/0{,}005 \approx 19{,}9$, tự tính); nếu N tiếp tục tăng, khoảng cách này còn nới rộng thêm, tiến tới SGD $\to 0\%$ và AdaGrad $\to 10\%$ ($= 1 - p$).

4. Thí nghiệm và kết quả chính

Paper thử nghiệm trên 4 bài toán deep learning (Table 2 của paper): **C1** phân loại ảnh CIFAR-10 (CNN sâu, kiến trúc `cifar.torch`, framework Torch); **L1** mô hình ngôn ngữ ký tự trên tiểu thuyết "War and Peace" (LSTM 2 tầng, `torch-rnn`, Torch); **L2** parsing phân biệt (discriminative, LSTM 2 tầng + feedforward, `span-parser`, DyNet, Penn Treebank); **L3** parsing sinh (generative, LSTM 3 tầng, `emnlp2016`, TensorFlow, Penn Treebank). So sánh 2 phương pháp không thích ứng (SGD, Heavy Ball) với 3 phương pháp thích ứng (AdaGrad, RMSProp, Adam). Mỗi thí nghiệm chạy 5 lần từ điểm khởi tạo ngẫu nhiên khác nhau, với ngân sách epoch cố định; khi có dev set thì chọn cấu hình tốt nhất trên dev set, CIFAR-10 không có dev set riêng nên chọn cấu hình có training loss thấp nhất.

Về tinh chỉnh hyperparameter (mục 4.1): chỉ learning rate ban đầu và lịch decay được tinh chỉnh kỹ (các siêu tham số khác giữ mặc định framework, do grid search toàn bộ không khả thi); dùng lưới 5 giá trị learning rate theo thang log, mở rộng lưới nếu điểm tốt nhất nằm ở biên. Hai lịch decay: "dev-decay" (giảm learning rate theo hệ số δ mỗi khi không đạt kết quả dev tốt nhất mới) và "fixed-decay" (giảm theo hệ số δ cố định sau mỗi k epoch); paper khuyến nghị dev-decay vì ít siêu tham số hơn và cho kết quả tương đương hoặc tốt hơn.

C1 (CIFAR-10, VGG+BN+Dropout): baseline tham khảo từ blog Torch là $7{,}55\%$ lỗi test. Lỗi test tốt nhất của từng thuật toán (annotation trong Figure 1 của paper):

Thuật toán	Lỗi test (%)
SGD	$7{,}65 \pm 0{,}14$
HB	$7{,}74 \pm 0{,}25$
RMSProp	$9{,}60 \pm 0{,}19$
Adam (đã tune)	$9{,}78 \pm 0{,}25$
AdaGrad	$11{,}34 \pm 0{,}46$
Adam (mặc định)	$12{,}02 \pm 0{,}16$

Giai đoạn đầu, các phương pháp thích ứng có vẻ tốt hơn, nhưng từ epoch 50 trở đi, SGD/HB vượt qua chúng trên test dù training error của phương pháp thích ứng vẫn thấp hơn; tới epoch 100, SGD/HB vượt tất cả phương pháp thích ứng trên cả train và test. AdaGrad là thuật toán ”flatline” (dừng cải thiện) sớm nhất trong nhóm thích ứng. Tăng learning rate giúp phương pháp thích ứng cải thiện nhanh hơn trong ~50 epoch đầu, nhưng làm hiện tượng flatline tệ hơn; không có lịch decay nào khắc phục được.

L1 (War and Peace, char-level LM, 200 epoch): test loss cuối cùng (Figure 2b):

Thuật toán	Test loss
SGD	$1{,}212 \pm 0{,}001$
RMSProp	$1{,}214 \pm 0{,}005$
HB	$1{,}218 \pm 0{,}002$
Adam (đã tune)	$1{,}229 \pm 0{,}004$
AdaGrad	$1{,}233 \pm 0{,}004$
Adam (mặc định)	$1{,}269 \pm 0{,}007$

AdaGrad tiến bộ nhanh ban đầu rồi flatline. Các phương pháp thích ứng nhạy hơn với cách khởi tạo, phương sai (variance) cao hơn trên cả train/test. RMSProp bất ngờ bám sát SGD về test loss, cho thấy phương pháp thích ứng không phải lúc nào cũng tổng quát hoá kém; nhưng paper cũng ghi nhận có cấu hình RMSProp đạt training loss thấp hơn SGD mà lại gây hành vi ”erratic” (bất thường) trên test, đẩy test loss RMSProp lên cao hơn cả Adam ở cấu hình đó.

L2 (Parsing phân biệt, Penn Treebank, 50 epoch, DyNet): RMSProp không có trong bản DyNet dùng lúc thí nghiệm, nên bị loại khỏi so sánh này. F1 dev tốt nhất (Figure 2d):

Thuật toán	F1 dev
SGD	$91{,}24 \pm 0{,}11$
Adam (đã tune)	$91{,}11 \pm 0{,}09$
HB	$91{,}16 \pm 0{,}12$
AdaGrad	$90{,}18 \pm 0{,}03$
Adam (mặc định)	$90{,}79 \pm 0{,}13$

SGD cho hiệu năng dev tốt nhất, theo sát là HB và Adam, AdaGrad kém xa. Điểm thú vị: Adam đạt F1 dev tốt nhất $91{,}11$ chỉ sau 6 epoch, trong khi SGD cần 18 epoch để đạt mức đó và phải tới epoch 31 mới đạt F1 tốt nhất $91{,}24$ của nó; Adam tiếp tục cải thiện trên train rất lâu sau khi đã đạt điểm dev tốt nhất, còn với SGD hai đỉnh train/dev gần trùng thời điểm hơn.

L3 (Parsing sinh, Penn Treebank, 100 epoch, TensorFlow, hidden dim giảm từ 1500 xuống 500 và dropout giảm từ 0,7 xuống 0,5 để giảm chi phí tính toán, paper ghi hiệu năng chỉ giảm nhẹ): perplexity dev (Figure 2f, càng thấp càng tốt):

Thuật toán	Perplexity dev
SGD	$5{,}09 \pm 0{,}04$
HB	$5{,}13 \pm 0{,}01$
RMSProp	$5{,}28 \pm 0{,}00$
AdaGrad	$5{,}24 \pm 0{,}02$
Adam (đã tune)	$5{,}35 \pm 0{,}01$
Adam (mặc định)	$5{,}47 \pm 0{,}02$

SGD và HB đạt perplexity dev tốt nhất (SGD nhích hơn chút); Adam có một trong các đường training tốt nhất nhưng perplexity dev lại **tệ nhất** trong số các phương pháp đã tune (dù vẫn tốt hơn chính nó ở cấu hình mặc định). Đáng chú ý, ở bài toán này AdaGrad ($5{,}24$) lại nhích hơn Adam đã tune ($5{,}35$), cho thấy thứ hạng "ai kém nhất" giữa các phương pháp thích ứng không cố định qua các bài toán.

5. Phân tích / ablation

Ba dạng "ablation" chính được paper tự thực hiện qua thiết kế thí nghiệm:

- **Tune vs không tune Adam:** so sánh "Adam (mặc định)" với "Adam (đã tune learning rate/decay)" ở cả 4 bài toán cho thấy tune mang lại cải thiện rõ rệt trong mọi trường hợp (ví dụ CIFAR-10: $12{,}02\% \to 9{,}78\%$ lỗi test; L2: F1 $90{,}79 \to 91{,}11$). Đây là bằng chứng trực tiếp phản bác quan niệm "Adam không cần tune nhiều": lượng công sức tune cần cho Adam, trong các thí nghiệm này, không ít hơn công sức cần cho SGD.
- **Tăng learning rate cho phương pháp thích ứng ở CIFAR-10:** giúp tăng tốc 50 epoch đầu nhưng làm hiện tượng flatline xấu hơn về sau, và không có lịch decay nào khắc phục nổi; đây là một ablation theo learning rate cho thấy vấn đề tổng quát hoá của phương pháp thích ứng không chỉ do chọn learning rate chưa tối ưu (dù đã thử mở rộng lưới tìm kiếm), mà có vẻ là một đặc tính cấu trúc của chính cơ chế thích ứng.
- **Cấu hình RMSProp "quá khớp train" ở L1:** paper tìm được cấu hình RMSProp đạt training loss thấp hơn cả SGD, nhưng test loss lại tệ hơn Adam ở cấu hình đó; đây là một minh chứng nội tại (within-method) cho luận điểm trung tâm của bài: training loss thấp không đảm bảo generalization tốt, ngay cả khi so sánh các cấu hình khác nhau của cùng một thuật toán, không chỉ giữa các thuật toán khác nhau.
- **dev-decay so với fixed-decay:** paper báo cáo hai lịch decay cho kết quả tương đương (trong cùng một độ lệch chuẩn) ở bài War and Peace, ủng hộ khuyến nghị dùng dev-decay (ít siêu tham số hơn) khi có dev set.

Nhìn tổng thể cả lý thuyết và thực nghiệm: mẫu hình lặp lại nhiều lần là optimizer thích ứng tiến bộ nhanh trên train giai đoạn đầu rồi chững lại trên dev/test sớm hơn SGD/HB, trong khi SGD/HB tiếp tục cải thiện dev/test lâu hơn dù tiến bộ ban đầu chậm hơn. Cơ chế lý thuyết ở mục 3.3 (các đặc trưng chỉ xuất hiện ở một mẫu train bị optimizer thích ứng cho trọng số tương đương các đặc trưng thực sự tổng quát, vì H_k chuẩn hoá theo từng toạ độ riêng, không theo độ lớn tương đối toàn cục như chuẩn ℓ_2) đưa ra một lời giải thích khả dĩ cho mẫu hình thực nghiệm này, dù paper không chứng minh trực tiếp rằng đúng cơ chế đó xảy ra trong các mạng neural sâu ở mục 4 (đây là suy luận của người ghi chú dựa trên cấu trúc bài, không phải khẳng định trực tiếp của paper).

6. Giới hạn

- Ví dụ lý thuyết ở mục 3.3 là một cấu trúc "đối nghịch" (adversarial) được paper tự thiết kế để minh hoạ hành vi cực đoan ("this generative model was chosen to illustrate extreme behavior"), không phải một chứng minh rằng optimizer thích ứng luôn tổng quát hoá kém trên mọi dữ liệu thực tế; nó chỉ chứng minh điều này **có thể** xảy ra, một cách chặt chẽ, trong ít nhất một thiết lập tự nhiên.

- Lemma 3.1 (nghiệm thích ứng có dạng đóng $\propto$ sign$(X^T y)$) chỉ áp dụng khi tồn tại hằng số c thoả X sign$(X^T y) = cy$, một điều kiện đại số đặc biệt, không đúng cho dữ liệu tổng quát; ví dụ mục 3.3 được xây dựng riêng để thoả điều kiện này.
- Bốn bài toán thực nghiệm ở mục 4 đều thuộc kiến trúc/giai đoạn trước 2017 (CNN VGG cho CIFAR-10, LSTM cho mô hình ngôn ngữ/parsing); không có Transformer, không có pretraining quy mô lớn kiểu LLM hiện đại. Kết luận ”SGD tổng quát hoá tốt hơn Adam” có thể không chuyển trực tiếp sang bối cảnh pretraining LLM ngày nay, nơi AdamW vẫn là lựa chọn mặc định phổ biến nhất trên thực tế (quan sát của người ghi chú, không phải nội dung paper).
- Paper chỉ tune learning rate ban đầu và lịch decay; các siêu tham số khác (hệ số ϵ làm mượt số học, cách khởi tạo bộ tích lũy gradient, β_1, β_2) giữ nguyên mặc định của từng framework (Appendix B liệt kê sự khác biệt mặc định giữa Torch/TensorFlow/DyNet). Đây là lựa chọn phương pháp hợp lý để tiết kiệm chi phí tính toán, nhưng cũng là một giới hạn: không loại trừ khả năng một số kết quả sẽ khác nếu các siêu tham số khác được tune kỹ hơn (nhận xét của người ghi chú).
- CIFAR-10 (C1) không có dev set riêng nên tiêu chí chọn cấu hình là training loss thấp nhất, khác với 3 bài toán còn lại (chọn theo hiệu năng dev tốt nhất); paper tự nêu rõ sự khác biệt này (dòng 347-348 của văn bản) nhưng đây vẫn là một điểm không đồng nhất về phương pháp giữa các thí nghiệm.
- RMSProp bị loại khỏi so sánh ở bài toán L2 (parsing phân biệt) vì không có trong phiên bản DyNet lúc đó, làm so sánh ở bài toán này thiếu một phương pháp thích ứng.
- Phần Conclusion của paper suy đoán vì sao Adam vẫn phổ biến (phù hợp ”ngẫu nhiên” với các bài toán không phải tối ưu thuần, như GAN hoặc Q-learning) nhưng chính paper nói rõ ”we are not sure exactly as to why”, nghĩa là đây là một giả thuyết mở, chưa được kiểm chứng trong bài.

7. Liên hệ với các bài khác

- **opt 06** (Adam, đã có file trong ”optimization content/”): đối tượng bị phê bình trực tiếp trong bài 43; đọc lại định nghĩa Adam gốc giúp hiểu rõ vì sao cơ chế chuẩn hoá theo toạ độ của nó lại dẫn tới hành vi mô tả ở mục 3.
- **opt 07** (Decoupled Weight Decay Regularization / AdamW, đã có file): một hướng sửa Adam khác, tập trung vào vấn đề weight decay bị trộn sai với scaling thích ứng, không trực tiếp giải quyết vấn đề generalization mà bài 43 nêu, nhưng cùng dòng nghiên cứu ”Adam có vấn đề gì và sửa thế nào”.
- **opt 08** (On the Convergence of Adam and Beyond / AMSGrad, đã có file): phê bình Adam từ góc độ hội tụ lý thuyết (Adam có thể không hội tụ trong một số bài toán convex online), khác góc độ generalization thực nghiệm của bài 43; đọc hai bài cùng nhau cho thấy Adam bị phê bình từ (ít nhất) hai hướng độc lập.
- **opt 41** (Why gradient clipping accelerates training: A theoretical justification for adaptivity, theo titles.tsv, có thể chưa có file khi bài này được viết): bàn về ”adaptivity” dưới một góc nhìn tích cực hơn (clipping gradient hoạt động giống một dạng thích ứng theo toạ độ có lợi dưới điều kiện (L_0, L_1)-smoothness); đối chiếu hai bài giúp thấy ”tính thích ứng” không phải lúc nào cũng xấu, bối cảnh/bài toán cụ thể quan trọng.
- **ml basics content/01_2016_rethinking_generalization.md** (đã ls-verify tồn tại): bài ”Rethinking generalization” (Zhang et al.) cùng chủ đề nền ”optimizer/mạng neural tổng quát hoá như thế nào”, một góc nhìn bổ sung (mạng neural có thể khớp hoàn toàn nhãn ngẫu nhiên, implicit bias của optimizer vẫn quan trọng để tổng quát hoá tốt trên nhãn thật), liên hệ trực tiếp tới lập luận ”implicit regularization” mà bài 43 trích dẫn ở mục 2.1 (Related Work).

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm khoảng 1.500 token text + 1.600-2.100 token visual, GPU <= 8 GB, batch size 1). Một vài suy luận (tất cả là suy luận, bài 43 không nói về VLM/LLM hiện đại):

- Trong thực tế huấn luyện/fine-tune LLM và VLM hiện nay, AdamW (biến thể của Adam, xem opt 07) vẫn là lựa chọn mặc định áp đảo, khác hẳn kết luận năm 2017 của bài 43 (vốn dựa trên CNN/LSTM thời đó). Điều này gợi ý rằng luận điểm ”SGD tổng quát hoá tốt hơn” có thể không áp dụng trực tiếp cho pretraining/fine-tuning các mô hình Transformer/attention quy mô lớn như LightOnOCR, do khác biệt lớn về kiến trúc, quy mô dữ liệu, và chế độ huấn luyện (ít epoch, một lượt qua dữ liệu lớn) so với các thí nghiệm CIFAR-10/LSTM nhiều epoch trong bài 43; nên không nên vội chuyển sang SGD cho fine-tune LightOnOCR chỉ dựa vào kết luận bài này.
- Tuy vậy, cơ chế lý thuyết ở mục 3 (optimizer thích ứng có thể cho trọng số ”quá lớn” tới các đặc trưng chỉ xuất hiện hiếm/đặc thù cho một vài mẫu train) có thể là một lời cảnh báo hữu ích khi fine-tune trên tập dữ liệu OCR nhỏ, hẹp (ví dụ chỉ vài nghìn trang tài liệu riêng của một domain cụ thể): nếu thấy mô hình fine-tune bằng AdamW khớp rất tốt trên tập train nhỏ này nhưng lại tổng quát kém sang tài liệu dạng khác, có thể là dấu hiệu của hiện tượng tương tự (dù không chắc, đây chỉ là loại suy, không phải cùng cơ chế toán học chính xác).
- Có thể thử nghiệm nhỏ: chỉ fine-tune phần ”nông” của model (ví dụ linear head cho OCR, hoặc một vài layer cuối) bằng cả SGD+momentum và AdamW với cùng công sức tune learning rate/decay (theo đúng phương pháp mục 4.1 của bài 43: lưới log-scale, mở rộng nếu điểm tốt nhất ở biên), rồi so sánh hiệu năng trên một tập test ”lệch domain” so với tập train (ví dụ train trên tài liệu in rõ, test trên ảnh chụp nghiêng/mờ) để xem optimizer nào tổng quát hoá tốt hơn trong trường hợp cụ thể của dự án; đây là một thí nghiệm giá rẻ, không cần train lại toàn bộ 0.8B tham số.
- Với GPU <= 8 GB và batch size 1 (rất nhỏ, nhiều nhiễu gradient theo từng mẫu), cả AdaGrad/RMSProp/Adam vốn được thiết kế để bớt nhạy với nhiễu gradient batch nhỏ (chuẩn hoá theo lịch sử); nếu chuyển sang SGD/HB như bài 43 gợi ý, nhiều khả năng cần tăng số bước accumulate gradient hoặc learning rate warmup kỹ hơn để bù lại, vì SGD thường nhạy hơn với batch size nhỏ và cần tune cẩn thận hơn (quan sát/suy luận chung, không phải nội dung trực tiếp của bài 43).

9. Thí nghiệm gợi ý

1. **Tái dựng ví dụ lý thuyết mục 3.3 bằng NumPy/PyTorch quy mô nhỏ** (ví dụ $n = 50$, vài trăm chiều thay vì vô hạn chiều): sinh dữ liệu đúng theo công thức, chạy AdaGrad và SGD (đóng vai trò pseudo-inverse/min-norm) tới hội tụ trên loss train, rồi đo lỗi phân loại trên dữ liệu test sinh mới. Kỳ vọng: AdaGrad gán gần hết nhãn dương cho test (lỗi gần $1 - p$), SGD gần như không lỗi khi $n_+ > n_-/3$. Diễn giải: xác nhận trực tiếp bằng số rằng cơ chế lý thuyết của paper tái lập được trên máy cá nhân, không cần GPU.
2. **So sánh AdaGrad vs SGD trên một mạng nhỏ (ví dụ MLP 2 tầng) với dữ liệu có đặc trưng ”rò” theo mẫu** (thêm một số đặc trưng one-hot ngẫu nhiên gắn theo chỉ số mẫu train, mô phỏng tinh thần ví dụ mục 3.3 nhưng cho mạng neural thật, không chỉ bình phương tối thiểu). Đo: train accuracy vs test accuracy theo epoch cho cả hai optimizer. Kỳ vọng: AdaGrad/Adam đạt train accuracy cao nhanh hơn nhưng test accuracy có thể thấp hơn SGD nếu đặc trưng rò đủ mạnh. Diễn giải: kiểm tra liệu hiện tượng mục 3 có ”lan” tới mạng neural phi tuyến, không chỉ bài toán tuyến tính closed-form.
3. **Lặp lại tinh thần thí nghiệm CIFAR-10 (mục 4.2) ở quy mô nhỏ, 1 GPU 8 GB:** huấn luyện một CNN nhỏ (ví dụ ResNet-9 hoặc một CNN tự viết vài triệu tham số) trên CIFAR-10 với SGD+momentum, RMSProp, Adam, mỗi thuật toán dò learning rate trên lưới log-scale 5 điểm như paper mô tả. Đo train/test error theo epoch. Kỳ vọng: thấy lại mẫu hình ”thích ứng tiến bộ nhanh đầu, chững sớm trên test” như Figure 1 của paper, trên quy mô nhỏ hơn nhiều. Diễn giải: xác nhận mẫu hình có lặp lại ở quy mô rất nhỏ hay chỉ xuất hiện ở mạng đủ lớn.
4. **So sánh Adam mặc định vs Adam đã tune learning rate/decay** trên một tác vụ fine-tune nhỏ (ví dụ fine-tune một mô hình ngôn ngữ nhỏ có sẵn, hoặc một phần nhỏ của pipeline OCR của dự án) trên 1 GPU 8 GB. Đo validation loss cuối cùng của cả hai cấu hình. Kỳ vọng: tune learning rate/decay cho Adam vẫn mang lại cải thiện đáng kể so với mặc định, giống phát hiện (iv) của paper ở mục 4, dù đây là bối cảnh fine-tune LLM/VLM hiện đại khác hẳn CNN/LSTM 2017.

5. **Đo "độ nhạy với khởi tạo"**: chạy cùng một mô hình nhỏ 5 lần với 5 seed khác nhau, cho cả SGD+momentum và một optimizer thích ứng (Adam hoặc RMSProp), đo độ lệch chuẩn (standard deviation) của test accuracy/loss cuối cùng giữa các seed. Kỳ vọng: optimizer thích ứng có phương sai giữa các seed cao hơn SGD, giống quan sát "adaptive methods appear more sensitive to the initialization" ở thí nghiệm L1 (War and Peace) của paper. Diễn giải: kiểm tra độ ổn định (không chỉ hiệu năng trung bình) khi chọn optimizer cho một pipeline cần tái lập kết quả.

10. Tóm tắt công thức / khái niệm

- **Cập nhật tổng quát (phương pháp thích ứng)**:

$$w_{k+1} = w_k - \alpha_k H_k^{-1} \nabla \tilde{f}(w_k + \gamma_k(w_k - w_{k-1})) + \beta_k H_k^{-1} H_{k-1}(w_k - w_{k-1})$$

  với $H_k = \text{diag}\big((\sum_{i=1}^{k} \eta_i \, g_i \circ g_i)^{1/2}\big)$; SGD là trường hợp đặc biệt $H_k = I$.
- **Bài toán minh hoạ (least-squares classification)**: $\min_w \frac{1}{2}\|Xw - y\|_2^2$, $X \in \mathbb{R}^{n \times d}$, $y \in \{-1, 1\}^n$, $d > n$.
- **Nghiệm không thích ứng (min-norm, max-margin)**: $w_{SGD} = X^T(XX^T)^{-1}y$.
- **Nghiệm thích ứng (Lemma 3.1)**: nếu $\exists c : X \,\text{sign}(X^T y) = cy$, thì $w_{ada} \propto \text{sign}(X^T y)$ khi khởi tạo tại 0.
- **Nghiệm closed-form cho ví dụ mục 3.3**:

$$\alpha_+ = \frac{4n_- + 3}{9n_+ + 3n_- + 8n_+n_- + 3}, \qquad \alpha_- = \frac{4n_+ + 1}{9n_+ + 3n_- + 8n_+n_- + 3}$$

  SGD không lỗi khi $n_+ > n_-/3$; AdaGrad luôn gán nhãn dương cho mọi điểm mới, lỗi $= 1 - p$.
- **Chặn lỗi theo N (Appendix C)**: lỗi SGD $\leq n^2/(2N)$; lỗi AdaGrad $\geq (1-p)(1-n/N)(1-n^2/(2N))$; khi $N \to \infty$ (n cố định): lỗi SGD $\to 0$, lỗi AdaGrad $\to 1 - p$.
- **Khái niệm chính**: generalization gap giữa adaptive và non-adaptive methods; "implicit bias" của optimizer (hướng nghiệm tự nhiên khi có nhiều nghiệm cực tiểu); dev-decay vs fixed-decay cho learning rate schedule.

11. Kết luận ngắn

Bài 43 đưa ra cả chứng minh lý thuyết chặt (một bài toán phân loại tuyến tính nơi AdaGrad/Adam/RMSProp hội tụ về nghiệm tổng quát hoá tuỳ ý kém, còn SGD tìm đúng nghiệm max-margin) và bằng chứng thực nghiệm trên 4 bài toán deep learning (CIFAR-10, mô hình ngôn ngữ ký tự, 2 tác vụ parsing) cho thấy SGD/Heavy Ball nhất quán tổng quát hoá bằng hoặc tốt hơn AdaGrad/RMSProp/Adam khi được tune công sức tương đương, dù các phương pháp thích ứng thường hội tụ nhanh hơn trên train. Paper cũng phản bác quan niệm "Adam không cần tune", cho thấy tune learning rate/decay cho Adam mang lại cải thiện lớn trong mọi thí nghiệm. Kết luận cần được đọc trong bối cảnh năm 2017 (kiến trúc CNN/LSTM, không phải Transformer/LLM hiện đại nơi AdamW vẫn thống trị thực tế), nhưng cơ chế lý thuyết về cách optimizer thích ứng có thể "quá tin" vào các đặc trưng hiếm/đặc thù vẫn là một lời nhắc hữu ích khi chọn và tune optimizer cho bất kỳ bài toán nào.

Nguồn gốc: https://arxiv.org/abs/1705.08292

Bài 44

How Does Batch Normalization Help Optimization? (2018)

Tác giả: Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, Aleksander Madry (MIT, đóng góp ngang nhau cho 3 tác giả đầu).
Venue / arXiv: arXiv:1805.11604 (v5, 15/04/2019; v1 tháng 05/2018 suy ra từ mã arXiv, ngoài paper); venue ghi trong paper (footnote trang 1): 32nd Conference on Neural Information Processing Systems (NeurIPS 2018), Montréal, Canada.
Nguồn: https://arxiv.org/abs/1805.11604
Chủ đề: lật lại lời giải thích phổ biến ”Batch Normalization (BatchNorm) hoạt động vì nó giảm internal covariate shift (ICS)”; bằng thực nghiệm và lý thuyết, paper chỉ ra BatchNorm thực chất làm **landscape tối ưu mượt hơn** (tốt hơn về Lipschitzness và beta-smoothness), khiến gradient dự đoán được hơn và cho phép dùng learning rate lớn hơn.
Vị trí trong lộ trình: theo titles.tsv, bài này đọc sau opt 38 (Batch Normalization gốc của Ioffe và Szegedy, đưa ra giả thuyết ICS mà bài này kiểm định lại) và opt 39 (Layer Normalization, một biến thể không cần batch), sau cả opt 40-43 (khó khăn khi train RNN, gradient clipping, mixed precision, ”marginal value of adaptive gradient methods”, tức các vấn đề optimization thực tế). Bài 44 đóng vai trò ”giải phẫu” cơ chế thật của một kỹ thuật đã quá quen thuộc, chuẩn bị cho opt 45 (Deconstructing What Makes a Good Optimizer for Language Models) vốn cũng theo tinh thần thực nghiệm kiểm định giả thuyết tương tự nhưng cho optimizer của LLM. Đã kiểm bằng `ls`: các file opt 38 và opt 39 đều có trong thư mục ”optimization content/”; mục 7 dưới đây chỉ trỏ tới các file đã xác nhận tồn tại bằng `ls`.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Phạm vi đã đọc: toàn văn paper arXiv v5 (26 trang, gồm Phụ lục A-B và chứng minh Phụ lục C); số liệu hình lấy từ chú thích và text trích xuất, riêng Hình 7 và các công thức Định lý 4.2, 4.4 được kiểm thêm trên ảnh trang. Nhận định về các bài khác (mục 7) không đối chiếu với text gốc của các bài đó.

1. Bối cảnh

Batch Normalization (BatchNorm, Ioffe và Szegedy 2015) là một trong những cải tiến kiến trúc thành công nhất của deep learning: tính đến lúc paper này viết, nó đã được trích dẫn hơn 6.000 lần và được dùng mặc định trong hầu hết mô hình thực tế. Ý tưởng kỹ thuật của BatchNorm rất đơn giản: với mỗi lớp, chuẩn hoá đầu ra (activation) sao cho trên một mini-batch, trung bình bằng 0 và phương sai bằng 1, rồi nhân với một hệ số học được γ và cộng thêm một độ lệch học được β để giữ khả năng biểu diễn của mô hình. Phép chuẩn hoá này áp dụng trước phi tuyến (non-linearity) của lớp trước.

Lời giải thích gốc của Ioffe và Szegedy, và cũng là động cơ thiết kế ban đầu, là BatchNorm làm giảm **internal covariate shift (ICS)**: hiện tượng phân phối đầu vào của một lớp thay đổi liên tục trong quá trình huấn luyện do tham số của các lớp trước đó được cập nhật. Giả thuyết là sự thay đổi liên tục này gây hại cho việc huấn luyện, và BatchNorm giúp bằng cách ổn định lại phân phối đó.

Vấn đề mà paper này nêu ra: mặc dù hiệu quả thực nghiệm của BatchNorm là không thể chối, chúng ta lại chưa có bằng chứng cụ thể nào cho thấy ICS (theo định nghĩa đó) thật sự liên quan đến hiệu năng huấn luyện. Nhiều phương pháp thay thế BatchNorm đã ra đời (Layer Norm, Group Norm, SELU...) nhưng không phương pháp nào làm sáng tỏ thêm câu hỏi ”tại sao BatchNorm hoạt động”. Mục tiêu của bài là trả lời trực tiếp câu hỏi đó.

Khái niệm cần nắm trước (giải thích cho người mới):

- **Mini-batch:** một tập con nhỏ (ví dụ 128 ảnh) được lấy ra từ toàn bộ dữ liệu huấn luyện để tính một bước gradient descent (SGD). BatchNorm tính trung bình/phương sai trên các mẫu trong cùng một mini-batch này.
- **Lipschitz continuity:** hàm f gọi là L-Lipschitz nếu $|f(x_1) - f(x_2)| \leq L\|x_1 - x_2\|$ với mọi x_1, x_2: nghĩa là đầu ra của f không thể thay đổi "quá nhanh" so với đầu vào; L nhỏ nghĩa là hàm "phẳng" hơn.
- **β-smoothness:** hàm f gọi là β-smooth nếu gradient của nó là β-Lipschitz, tức $\|\nabla f(x_1) - \nabla f(x_2)\| \leq \beta\|x_1 - x_2\|$. Đây chính là độ "cong" tối đa của hàm mất mát: β nhỏ nghĩa là bước gradient hiện tại vẫn còn đúng hướng khi ta đi một đoạn khá xa.
- **Landscape tối ưu (optimization landscape):** cách hình dung hàm mất mát như một "địa hình" nhiều chiều mà gradient descent phải di chuyển trên đó; landscape có nhiều "kink" (gấp khúc), vùng phẳng (dẫn tới vanishing gradient) hoặc cực tiểu sắc nhọn (dẫn tới exploding gradient khi bước đi quá lớn) sẽ làm training khó và nhạy với learning rate.

2. Ý tưởng chính

Paper tiến hành theo hai bước rõ ràng:

Bước 1 (phản biện giả thuyết ICS): Tác giả thiết kế một thí nghiệm "tiêm nhiễu" (noise injection) trực tiếp vào sau lớp BatchNorm để tạo ra ICS rất nặng một cách có chủ đích, rồi so sánh hiệu năng huấn luyện. Nếu ICS thực sự là nguyên nhân, mạng "BatchNorm nhiễu" này phải huấn luyện kém hơn nhiều so với BatchNorm chuẩn. Kết quả: hiệu năng gần như không đổi, dù phân phối activation mất ổn định hơn cả mạng không dùng BatchNorm. Ngược lại, paper cũng đo trực tiếp ICS (qua một định nghĩa toán học mới, Định nghĩa 2.1 ở mục 2.2 của paper) và thấy mạng có BatchNorm nhiều khi có ICS **bằng hoặc tệ hơn** mạng không có BatchNorm, trong khi vẫn train tốt hơn rõ rệt. Hai quan sát này cùng chỉ về một kết luận: liên hệ giữa ICS và hiệu quả của BatchNorm là rất yếu, nếu có.

Bước 2 (tìm cơ chế thật): Tác giả đề xuất và kiểm chứng (thực nghiệm + lý thuyết) rằng cơ chế thật của BatchNorm là nó **tái tham số hoá (reparametrize)** bài toán tối ưu để landscape mượt hơn: hàm mất mát có Lipschitz constant nhỏ hơn (gradient không đổi đột ngột về độ lớn) và "effective" β-smoothness tốt hơn (gradient không đổi đột ngột về hướng khi đi một bước dài). Khi gradient "đáng tin" hơn như vậy, ta có thể dùng learning rate lớn hơn mà không sợ nhảy vào vùng loss tệ, từ đó train nhanh hơn và ổn định hơn. Quan trọng hơn, tác giả chỉ ra hiệu ứng làm mượt này **không phải đặc quyền riêng của BatchNorm**: các chuẩn hoá tự nhiên khác (chuẩn theo ℓ_1, ℓ_2, ℓ_∞ norm) cũng cho hiệu ứng tương tự hoặc mạnh hơn, dù chúng không kiểm soát phân phối activation theo kiểu Gauss như BatchNorm. Điều này gợi ý thành công của BatchNorm có phần "ngẫu nhiên, hên" (serendipitous) hơn là nhờ đúng cơ chế mà tác giả gốc đề ra.

3. Phương pháp chi tiết

3.1 Hai kiến trúc dùng xuyên suốt bài

- **VGG trên CIFAR-10:** một mạng conv kiểu VGG (Simonyan và Zisserman), train bằng SGD thường (không momentum/Adam đề cập ở phần chính), 15.000 bước, batch size 128, learning rate cố định 0.1 (trừ khi nói khác), không dùng data augmentation (vì mục tiêu là đo khả năng *optimize*/fit training set, không phải generalization). Mạng đạt test accuracy 83% với BatchNorm và 80% không có BatchNorm (nếu thêm augmentation thì 92% và 88% tương ứng). Tác giả chủ động không dùng ResNet vì ResNet đã có hiệu ứng tương tự BatchNorm theo một paper khác (Hardt và Ma, "Identity matters in deep learning"), sẽ gây nhiễu kết luận.
- **Deep Linear Network (DLN) 25 lớp:** bài toán hồi quy tổng hợp $\min_{W_1,\ldots,W_k} \|W_1 \cdots W_k x - Ax\|_2^2$ với $k = 25$ lớp fully-connected vuông (không phi tuyến), A là ma trận Gauss ngẫu nhiên 10×10, x sinh từ phân phối Gauss, cỡ dữ liệu 1000 (bằng batch size, tức full-batch gradient descent), 10.000 bước, learning rate 10^{-6} (trừ khi nói khác). DLN giúp loại bỏ nhiễu do phi tuyến và tính ngẫu nhiên của SGD (vì train full-batch), cô lập đúng hiệu ứng tái tham số hoá của BatchNorm. Cả hai mạng dùng

khởi tạo Glorot chuẩn, và learning rate được chọn bằng hyperparameter search để mỗi mạng đạt hiệu năng training tốt nhất trong phạm vi khảo sát.

3.2 Thí nghiệm "Noisy BatchNorm" (kiểm định ICS, mục 2.1 của paper)

Ý tưởng: thêm nhiễu thời gian-biến-đổi (time-varying) ngay sau lớp BatchNorm để **chủ động** phá vỡ sự ổn định phân phối mà BatchNorm vừa tạo ra, nhưng vẫn giữ nguyên các lợi ích tính toán khác của BatchNorm. Với $a_{i,j}$ là activation thứ j của mẫu i trong batch (sau BatchNorm), tại mỗi bước t (với μ_j^t và σ_j^t lấy mẫu độc lập cho từng unit j, theo Thuật toán 1 của paper):

$$\mu_j^t \sim U(-n_\mu, n_\mu), \qquad \sigma_j^t \sim U(1, n_\sigma)$$
$$m_{i,j}^t \sim U(\mu_j^t - r_\mu, \mu_j^t + r_\mu), \qquad s_{i,j}^t \sim \mathcal{N}(\sigma_j^t, r_\sigma)$$
$$a_{i,j}^t \leftarrow s_{i,j}^t \cdot a_{i,j} + m_{i,j}^t$$

với $n_\mu = 0.5$, $n_\sigma = 1.25$, $r_\mu = r_\sigma = 0.1$ (giá trị cụ thể paper dùng, Phụ lục A.2.1). Nói cách khác: mỗi activation bị nhân/cộng với một nhiễu ngẫu nhiên có mean và variance **khác 0/1 và thay đổi theo thời gian** , chủ đích tạo ra một dạng ICS "nhân tạo" rất nặng, nặng hơn cả mạng không dùng BatchNorm. Nếu đem đúng lượng nhiễu này cộng vào activation của mạng *không* có BatchNorm, mạng đó không train được (paper nói rõ "prevents it from training entirely").

3.3 Định nghĩa đo ICS theo hướng optimization (Định nghĩa 2.1)

Vì cách đo ICS kiểu "phân phối activation" (mean/variance theo thời gian) cho thấy không liên hệ trực tiếp với hiệu năng (mục 3.2 trên), tác giả đưa ra một định nghĩa khác, gắn chặt với bài toán tối ưu hơn là với phân phối thống kê. Gọi L là loss, $W_1^{(t)}, \ldots, W_k^{(t)}$ là tham số của k lớp tại bước t, $(x^{(t)}, y^{(t)})$ là batch dữ liệu dùng ở bước t. Định nghĩa:

$$G_{t,i} = \nabla_{W_i^{(t)}} L\big(W_1^{(t)}, \ldots, W_k^{(t)}; x^{(t)}, y^{(t)}\big)$$
$$G'_{t,i} = \nabla_{W_i^{(t)}} L\big(W_1^{(t+1)}, \ldots, W_{i-1}^{(t+1)}, W_i^{(t)}, W_{i+1}^{(t)}, \ldots, W_k^{(t)}; x^{(t)}, y^{(t)}\big)$$

ICS của lớp i tại bước t được định nghĩa là $\|G_{t,i} - G'_{t,i}\|_2$. $G_{t,i}$ là gradient "đồng loạt" dùng để cập nhật mọi lớp cùng lúc (như vẫn làm trong thực tế); $G'_{t,i}$ là gradient tính lại *sau khi* các lớp phía trước $(1, \ldots, i-1)$ đã được cập nhật sang giá trị mới, còn lớp i và các lớp sau vẫn giữ giá trị cũ. Hiệu số này đo đúng mức độ "bài toán của lớp i thay đổi bao nhiêu" chỉ vì các lớp trước thay đổi, tức đúng bản chất optimization của ICS, không lệ thuộc vào việc activation có phân phối Gauss ổn định hay không.

3.4 Đo độ mượt của landscape (mục 3.2 và Phụ lục A.2.2)

Tại một bước huấn luyện cho trước, lấy gradient hiện tại g, rồi đo loss $L(\theta - \eta g)$ khi thay đổi η (độ dài bước) trong một khoảng cho trước (VGG: $\eta \in [1/2, 4]\times$ step size đang dùng; DLN: $\eta \in [1/100, 30]\times$ step size). Ba đại lượng được vẽ theo bước huấn luyện:

1. **Loss landscape:** khoảng biến động của loss khi η thay đổi (đo Lipschitzness của loss).
2. **Gradient predictiveness:** khoảng biến động ℓ_2 của gradient mới $\nabla L(\theta - \eta g)$ so với gradient cũ g khi η thay đổi.
3. **"Effective" β-smoothness:** tỉ số lớn nhất giữa độ biến động gradient (mục 2) và khoảng cách đã di chuyển $\eta\|g\|$, tức một xấp xỉ cục bộ, theo đúng hướng gradient, của hằng số β lý thuyết.

Ba phép đo này **không phải quy trình huấn luyện**, mà là một phép "quét" riêng để khảo sát hình dạng landscape tại từng bước huấn luyện thật.

3.5 Khung lý thuyết (mục 4, Hình 5 của paper)

Xét một lớp fully-connected W tuỳ ý trong mạng, đầu vào x, đầu ra $y = Wx$. So sánh hai mạng giống nhau hệt về tham số W: (a) mạng chuẩn (loss L); (b) mạng chèn thêm một lớp BatchNorm ngay sau W (loss $\hat{L}$, dùng mũ để phân biệt với mạng chuẩn), với $\hat{y} = \text{BN}_{0,1}(y) = \frac{y-\mu}{\sigma}$ (chuẩn hoá theo batch, σ là độ lệch chuẩn theo batch của y_j), rồi $z = \gamma\hat{y} + \beta$ (γ, β coi là hằng số trong phân tích theo giả thiết của paper; paper không nêu lý do, nên lý do sau đây là suy luận: để tách hiệu ứng tái tham số hoá khỏi việc scale lại cuối cùng). Phần sau lớp này (có thể gồm nhiều lớp phi tuyến khác) là giống nhau giữa hai mạng.

4. Thí nghiệm và kết quả chính

(a) ICS không giải thích được vì sao BatchNorm hoạt động (mục 2, Hình 1-3). VGG với BatchNorm train nhanh và tốt hơn rõ rệt so với không có BatchNorm (Hình 1a,b), nhưng khi quan sát trực tiếp phân phối activation theo thời gian (Hình 1c), sự khác biệt về độ ổn định phân phối giữa hai mạng là ”marginal” (rất nhỏ). Thí nghiệm ”Noisy BatchNorm” (Hình 2) cho kết quả gây sốc: mạng BatchNorm bị tiêm nhiễu phân phối rất mạnh vẫn đạt hiệu năng **gần như trùng khớp** với BatchNorm chuẩn, và tốt hơn hẳn mạng không BatchNorm, mặc dù về mặt phân phối nó ”mất ổn định” còn hơn cả mạng chuẩn. Dùng Định nghĩa 2.1, Hình 3 cho thấy mạng có BatchNorm có ICS (đo bằng cosine angle và ℓ_2-difference của gradient trước/sau update) **bằng hoặc tệ hơn** mạng chuẩn, rõ nhất ở DLN (gradient gần như không tương quan ở mạng BatchNorm, trong khi mạng chuẩn gần như không có ICS suốt quá trình train), trong khi BatchNorm vẫn thắng về accuracy/loss. Kết luận: liên hệ ICS, hiệu năng là ”tenuous at best” (nói chữ paper dùng), và BatchNorm có khi còn **không** làm giảm ICS.

(b) BatchNorm làm landscape mượt hơn rõ rệt (mục 3, Hình 4, 9). Đo trực tiếp loss landscape, gradient predictiveness, và ”effective” β-smoothness cho VGG (Hình 4) và DLN (Hình 9): ở cả ba chỉ số, mạng BatchNorm tốt hơn mạng chuẩn một cách nhất quán (paper: ”consistent differences”), đặc biệt ở giai đoạn đầu huấn luyện (chênh nhau tới gần hai bậc độ lớn về gradient predictiveness, theo lời paper). BatchNorm còn tiếp tục cho hiệu ứng làm mượt ở khoảng bước đi lớn hơn nhiều so với mạng chuẩn (mạng chuẩn hoạt động kém đi khi bước quá dài, theo chú thích Hình 4).

(c) Hiệu ứng làm mượt không phải đặc quyền của BatchNorm (mục 3.3, Hình 11-14). Thử chuẩn hoá theo ℓ_1, ℓ_2, ℓ_∞-norm (cố định mean bằng 0 như BatchNorm, nhưng chia theo norm trung bình thay vì theo độ lệch chuẩn) cho VGG và DLN. Các biến thể này có phân phối activation **không còn Gauss**, và thực ra gây dịch chuyển phân phối (distributional shift) còn lớn hơn mạng không chuẩn hoá (Hình 14), nhưng vẫn cho hiệu năng huấn luyện tương đương hoặc tốt hơn BatchNorm (với DLN, chuẩn hoá ℓ_1 còn tốt hơn BatchNorm, Hình 13b), và vẫn cải thiện độ mượt landscape tương tự BatchNorm (Hình 11, 12).

(d) Lý thuyết xác nhận quan sát thực nghiệm (mục 4). Định lý 4.1 chứng minh $\|\nabla_{y_j}\hat{L}\|^2 \le \frac{\gamma^2}{\sigma_j^2}\Big(\|\nabla_{y_j}L\|^2 - \frac{1}{m}\langle 1, \nabla_{y_j}L\rangle^2 - \frac{1}{m}\langle \nabla_{y_j}L, \hat{y}_j\rangle^2\Big)$: bình phương gradient theo activation (tức Lipschitz constant của loss theo activation) ở mạng BatchNorm bị chặn trên bởi γ^2/σ_j^2 nhân với bình phương gradient của mạng chuẩn, rồi trừ đi hai số hạng không âm. Vì hệ số γ^2/σ_j^2 có thể lớn hơn 1, bất đẳng thức chỉ đảm bảo không vượt mạng chuẩn khi $\sigma_j \ge \gamma$ (suy luận); paper nhấn mạnh rằng phần trừ đi có tác dụng ngay cả khi $\sigma_j = \gamma$, và rằng σ_j thường lớn trong thực tế (Phụ lục, Hình 8). Mức giảm càng lớn khi activation tương quan với gradient hoặc gradient có trung bình khác 0: paper nhận xét gradient hiếm khi không tương quan với chính biến của nó, và số hạng theo trung bình đáng kể vì $\langle 1, \nabla_{y_j}L\rangle^2$ tăng theo bình phương số chiều (paper, ngay sau Định lý 4.1). Định lý 4.2 cho một bất đẳng thức tương tự cho số hạng Hessian (liên quan tới β-smoothness): nếu Hessian giữ chuẩn tương đối giữa $\hat{g}_j$ và $\nabla_{y_j}\hat{L}$, thì $\nabla_{y_j}\hat{L}^\top \frac{\partial \hat{L}}{\partial y_j \partial y_j}\nabla_{y_j}\hat{L} \le \frac{\gamma^2}{\sigma^2}\Big(\hat{g}_j^\top H_{jj}\hat{g}_j - \frac{1}{m\gamma}\langle \hat{g}_j, \hat{y}_j\rangle \Big\|\frac{\partial \hat{L}}{\partial y_j}\Big\|^2\Big)$, nghĩa là gradient dự đoán được tốt hơn, miễn Hessian của loss (theo activation chuẩn hoá) là positive semi-definite (đúng khi mạng dùng phi tuyến tuyến tính-từng-đoạn và loss cuối lồi, ví dụ softmax cross-entropy) và $\langle \hat{y}_j, \hat{g}_j\rangle > 0$ (gradient âm hướng về cực tiểu). Định lý 4.4 nâng kết quả Lipschitz (Định lý 4.1) lên bound minimax theo trọng số W (không chỉ activation; bản minimax cho độ mượt là Định lý C.1 ở Phụ lục C):

$\hat{g}_j \leq \frac{\gamma^2}{\sigma_j^2}\left(g_j^2 - m\mu_{g_j}^2 - \lambda^2\langle\nabla_{y_j}L, \hat{y}_j\rangle^2\right)$ với $g_j = \max_{\|X\|\leq\lambda}\|\nabla_W L\|^2$. Quan sát 4.3 chỉ ra BatchNorm không chỉ đơn thuần ”rescale” lại cùng một landscape: với mọi W, tồn tại một bộ (W, γ, β) cho mạng BatchNorm tạo ra **đúng** activation y_j như mạng chuẩn (khi $\gamma = \sigma_j$), nghĩa là mọi cực tiểu của mạng chuẩn vẫn là cực tiểu hợp lệ trong landscape BatchNorm, chỉ là landscape xung quanh các cực tiểu đó mượt hơn.

5. Phân tích / ablation

- **Theo learning rate (Hình 3 và Hình 6, Phụ lục B):** Hình 3 đo ICS ở VGG với LR 0.1 và ở DLN với LR 10^{-7}; Hình 6 thêm VGG với LR 0.01 và DLN với LR 10^{-6}. Kết luận ”BatchNorm không giảm ICS” giữ nguyên ở các LR đó, nên không phải hiện tượng chỉ xảy ra ở một LR cụ thể.
- **Theo kiến trúc (VGG có phi tuyến + SGD mini-batch, so với DLN tuyến tính + full-batch):** cả hai kiến trúc cho cùng kết luận về ICS (Hình 3) và độ mượt (Hình 4, 9). DLN được dùng để tách hiệu ứng khỏi phi tuyến và nhiễu SGD (Phụ lục A.1), và hiệu ứng làm mượt vẫn xuất hiện ở DLN (mục 3.2; Hình 9, 12). Suy luận: vì vậy hiệu ứng này không chỉ do phi tuyến hay nhiễu minibatch; paper không kết luận riêng rằng nó đến từ phép tái tham số hoá tuyến tính.
- **Ablation ”mức độ nhiễu” của Noisy BatchNorm (Hình 7):** vẽ $|\mu^{t+1} - \mu^t|$ và $|\sigma_{t+1}^2 - \sigma_t^2|$ (biến động mean/variance giữa hai bước liên tiếp) cho ba mạng (chuẩn, BatchNorm, Noisy BatchNorm): Noisy BatchNorm có biến động này **lớn hơn rõ rệt** các mạng còn lại ở mọi layer được khảo sát (layer 2, 9, 13) (Hình 7a; chú thích paper: ”significantly higher ICS than standard networks”), xác nhận lượng nhiễu tiêm vào là thực sự đáng kể, không phải nhiễu ”vô hại”.
- **Ablation họ chuẩn hoá ℓ_p (Hình 11-13):** so ℓ_1, ℓ_2, ℓ_∞-normalization với BatchNorm trên cả VGG và DLN; tất cả các biến thể đều cải thiện performance và độ mượt so với mạng chuẩn, dù mức cải thiện giữa các biến thể có khác nhau (DLN: ℓ_1 > BatchNorm; VGG: các biến thể tương đương hoặc gần BatchNorm).
- **Phân tích khởi tạo (Bổ đề 4.5):** nhờ tính scale-invariance của BatchNorm (với mọi $a > 0$, $\text{BN}((aW)x) = \text{BN}(Wx)$, nên mọi nhân tử vô hướng của một cực tiểu W^* của mạng chuẩn cũng là cực tiểu $\hat{W}^* = kW^*$ hợp lệ của mạng BatchNorm, với k chọn tối ưu), với mọi điểm khởi tạo W_0 sao cho $\langle W_0, W^*\rangle > 0$, khoảng cách từ W_0 tới cực tiểu gần nhất **trong landscape BatchNorm không thể lớn hơn** khoảng cách tới W^* trong landscape chuẩn. Đây là một lợi thế khởi tạo tách biệt hoàn toàn với lý do ”làm mượt landscape” đã nói ở trên.

6. Giới hạn

- Lý thuyết (mục 4) chỉ phân tích **một** lớp BatchNorm chèn ngay sau **một** lớp fully-connected W tuỳ ý, không phải toàn bộ một mạng sâu nhiều lớp BatchNorm xếp chồng; các định lý là bound cục bộ theo từng lớp; paper không đưa ra bound toàn cục cho cả mạng (suy luận từ phạm vi định lý; chú thích 3 nói riêng về β-smoothness). Paper cũng coi β và γ là hằng số trong phân tích (mục 4.1), và phiên bản rút gọn của Định lý 4.2 cần thêm giả thiết Hessian giữ chuẩn tương đối.
- Chính tác giả ghi chú ở footnote 3: do sự tồn tại của phi tuyến, ”one should not expect the β-smoothness to be bounded in an absolute, global sense” (không nên kỳ vọng β-smoothness bị chặn tuyệt đối toàn cục); các con số đo đạc trong Hình 4, 9 chỉ là ”effective” (cục bộ, theo đúng hướng gradient tại bước đó), không phải hằng số Lipschitz toàn cục đúng nghĩa.
- Thực nghiệm giới hạn ở VGG (CIFAR-10, không augmentation cho các thí nghiệm landscape) và DLN tổng hợp (ma trận 10×10, 25 lớp); tác giả chủ động loại ResNet ra khỏi khảo sát (footnote 4) để tránh nhiễu. Nghĩa là (suy luận) kết luận chưa được kiểm chứng trực tiếp trên kiến trúc có residual connection, Transformer, hay ở quy mô dữ liệu/mô hình lớn hơn nhiều (paper năm 2018). Paper cũng nói các thí nghiệm sơ bộ trên kiến trúc khác ”gave similar results” (Phụ lục A.1) nhưng không trình bày. Với VGG, paper ghi nhận mạng rất nhạy learning rate nên chỉ line search trong một khoảng hẹp, và bước tối đa được chọn nhỏ hơn LR mà mạng chuẩn phân kỳ (Phụ lục A.2.2).
- Định nghĩa ICS theo ”khoảng cách gradient trước/sau update” (Định nghĩa 2.1) là một lựa chọn cụ thể

của nhóm tác giả để bắt đúng bản chất optimization; paper chỉ nói ICS không liên hệ trực tiếp với hiệu năng "ít nhất là khi gắn ICS với ổn định mean và variance" (mục 2.2), nên có thể tồn tại định nghĩa ICS khác vẫn liên hệ với hiệu năng (ví dụ theo thống kê bậc cao hơn; suy luận), còn hai định nghĩa paper thử (phân phối và gradient) đều không ủng hộ giả thuyết ICS gốc.
- Liên hệ giữa độ mượt landscape và **generalization** (khả năng khái quát hoá, không chỉ tốc độ train) chỉ được paper nêu như một hướng tương lai ở Kết luận ("chúng tôi hy vọng công việc tương lai sẽ khảo sát khả năng này"), chưa có bằng chứng thực nghiệm hay lý thuyết trong chính bài.
- Vì sao chuẩn hoá ℓ_p tổng quát lại tạo ra hiệu ứng làm mượt tương tự BatchNorm về mặt lý thuyết (không chỉ thực nghiệm) không được chứng minh hình thức trong paper, chỉ dừng ở quan sát thực nghiệm (mục 3.3) kèm diễn giải định tính.

7. Liên hệ với các bài khác

(Đã kiểm tra tồn tại bằng `ls` các thư mục "llm content/" và "rl content/" trước khi trỏ.) Các nhận định về ba bài này dưới đây là ngoài paper 1805.11604 và chưa đối chiếu với text gốc của các bài đó (kiến thức chung).

- **llm content/09_2020_pre_ln_transformer.md** (Pre-LN Transformer): bài đó cho thấy việc đặt LayerNorm trước hay sau sub-layer ảnh hưởng trực tiếp tới độ lớn gradient ở các lớp sâu và tới việc có cần warm-up learning rate hay không, một biểu hiện khác của chính câu hỏi "chuẩn hoá ảnh hưởng thế nào tới landscape/gradient" mà bài 44 trả lời ở mức cơ chế tổng quát hơn (Lipschitz, β-smoothness) cho BatchNorm.
- **llm content/10_2019_rmsnorm.md** (RMSNorm): RMSNorm bỏ bước trừ mean của LayerNorm, chỉ chuẩn hoá theo RMS, biện minh một phần bằng việc thành phần re-scaling (không phải re-centering) mới là phần quan trọng cho hiệu quả tối ưu hoá; (suy luận) đây gần như là một phiên bản "thực nghiệm độc lập" ủng hộ đúng thông điệp của bài 44 rằng chính việc tái tham số hoá độ lớn (scale), không phải việc kiểm soát chính xác phân phối thống kê, mới là thứ quyết định.
- **rl content/08_2020_implementation_matters_ppo_trpo.md** (Implementation Matters in Deep RL): cùng tinh thần phương pháp luận "giải phẫu" một thuật toán tưởng đã hiểu rõ (PPO) bằng cách tách bạch code-level optimization tricks khỏi phần thuật toán chính, phát hiện phần lớn hiệu quả thực tế nằm ở các chi tiết cài đặt chứ không phải ở lý thuyết được công bố, một cấu trúc luận điểm gần như song song với bài 44 (hiệu quả BatchNorm không nằm ở ICS như công bố gốc, mà ở độ mượt landscape).
- Trong cùng bộ "opt" (titles.tsv), opt 38 (BatchNorm gốc, nêu giả thuyết ICS) và opt 39 (Layer Normalization) đứng trước bài 44 trong danh sách này, nên đọc cùng bài 44 để thấy rõ vòng lặp: đề xuất kỹ thuật, một giả thuyết cơ chế, rồi bài 44 kiểm định lại giả thuyết đó.

8. Ứng dụng cho dự án (suy luận)

Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1.6-2.1K token visual, chạy trên GPU ≤ 8 GB, batch size 1).

(Suy luận) Mức độ liên quan trực tiếp: thấp. BatchNorm hầu như không xuất hiện trong các kiến trúc Transformer/SSM hiện đại (bao gồm GatedDeltaNet và attention block của LightOnOCR-3): chuẩn mực hiện nay là LayerNorm hoặc RMSNorm, vì BatchNorm cần thống kê theo chiều batch, điều này gãy hoàn toàn khi batch size = 1 (như mô tả của dự án) hoặc khi suy luận tuần tự autoregressive từng token một (thống kê theo batch không ổn định hoặc không có ý nghĩa). Vì vậy bản thân kỹ thuật BatchNorm, và cả Theorem 4.1/4.2 (viết riêng cho chuẩn hoá theo batch dimension), không áp dụng trực tiếp lên LightOnOCR-3.

(Suy luận) Mức độ liên quan ở tầm khung lý thuyết: trung bình, hữu ích khi debug. Bài học cốt lõi, rằng vai trò thật của một lớp chuẩn hoá là cải thiện Lipschitzness/β-smoothness của landscape (không phải kiểm soát thống kê activation), vẫn là một lăng kính hữu ích khi quan sát RMSNorm trong GatedDeltaNet sau khi pruning hoặc distillation: nếu sau khi cắt bớt head/kênh mà optimizer (ví dụ AdamW khi fine-tune lại

mô hình đã nén) trở nên bất ổn (loss dao động mạnh, cần learning rate rất nhỏ mới train được), kỹ thuật đo "effective β-smoothness" của bài 44 (đo biến động gradient khi đi dọc theo hướng gradient hiện tại, mục 3.4) có thể dùng làm công cụ chẩn đoán cụ thể: nếu landscape sau khi nén trở nên "gồ ghề" hơn (độ mượt giảm) so với mô hình gốc, đó là dấu hiệu định lượng cho việc cắt tỉa đã làm hỏng cấu trúc tái tham số hoá tốt mà các lớp norm đang giữ, hơn là chỉ nhìn loss curve một cách định tính.

(Suy luận) Liên hệ với Bổ đề 4.5 (khởi tạo): khi distill hoặc warm-start mô hình nén từ trọng số gốc, tính scale-invariance của RMSNorm (giống BatchNorm ở việc $\text{norm}(aWx) = \text{norm}(Wx)$ với norm theo kênh) có nghĩa phần "hướng" của trọng số quan trọng hơn phần "độ lớn"; nếu sau khi prune một phần kênh mà không reset lại scale (γ) của RMSNorm cho phù hợp, điểm khởi tạo mới có thể đã ở xa cực tiểu gần nhất hơn cần thiết, một hiện tượng tương tự điều Bổ đề 4.5 mô tả cho BatchNorm. Đây là gợi ý debug, không phải kết luận đã kiểm chứng cho project cụ thể.

9. Thí nghiệm gợi ý

1. **Tái lập "Noisy BatchNorm" ở quy mô nhỏ.** Trên một MLP 4-6 lớp với BatchNorm, train trên một tập con MNIST hoặc CIFAR-10 (vài nghìn ảnh, chạy được trên CPU hoặc GPU 8 GB trong vài phút), thêm đúng công thức nhiễu ở mục 3.2 ($n_\mu = 0.5, n_\sigma = 1.25, r_\mu = r_\sigma = 0.1$) sau lớp BatchNorm. Đo training accuracy theo bước cho 3 mạng: chuẩn, BatchNorm, Noisy BatchNorm. Kỳ vọng: Noisy BatchNorm gần khớp BatchNorm chuẩn, cả hai vượt xa mạng chuẩn, qua đó tự kiểm chứng kết luận chính của paper trước khi tin vào nó.
2. **Đo "effective β-smoothness" cho RMSNorm so với không-norm.** Lấy một block Transformer nhỏ (1-2 layer, hidden size nhỏ, kiểu toy GPT) có RMSNorm và một bản không có norm, cùng khởi tạo, train vài trăm bước trên một tác vụ ngôn ngữ toy (ví dụ copy task hoặc một đoạn text nhỏ), tại một vài bước huấn luyện lấy gradient hiện tại, quét $\eta \in [0.1, 4]\times$ step size, đo biến động loss và gradient theo đúng quy trình mục 3.4. Kỳ vọng: bản có RMSNorm cho đường cong landscape và gradient-predictiveness phẳng hơn (giống Hình 4 của paper), xác nhận hiệu ứng làm mượt cũng xảy ra với RMSNorm, không chỉ BatchNorm.
3. **Tái lập chuẩn hoá ℓ_p so với BatchNorm trên DLN toy.** Cài chính xác bài toán DLN của Phụ lục A.1, mục 2 ($k = 25$ lớp vuông, A ngẫu nhiên 10×10, full-batch GD, 10.000 bước, learning rate 10^{-6}, không cần GPU vì kích thước rất nhỏ): so 4 biến thể (không chuẩn hoá, BatchNorm, ℓ_1-norm, ℓ_2-norm). Đo training loss theo bước. Kỳ vọng: cả 3 biến thể chuẩn hoá đều train nhanh hơn hẳn bản không chuẩn hoá, và ℓ_1-norm có thể nhanh hơn cả BatchNorm, đúng như Hình 13b của paper.
4. **Chẩn đoán landscape trước/sau pruning trên mô hình thật (liên hệ trực tiếp dự án).** Lấy một checkpoint nhỏ dạng GatedDeltaNet hoặc một Transformer nhỏ có RMSNorm (có thể dùng mô hình toy, không cần chính LightOnOCR-3-0.8B), prune bớt một phần kênh/head theo một tiêu chí đơn giản (magnitude pruning), rồi fine-tune lại vài trăm bước. Đo "effective β-smoothness" (như thí nghiệm 2) ở mô hình gốc và mô hình đã prune, tại cùng một vài bước fine-tune tương ứng. Kỳ vọng và cách diễn giải: nếu mô hình prune có β-smoothness tệ hơn rõ rệt ở giai đoạn đầu fine-tune, đó là bằng chứng định lượng rằng learning rate cần giảm (hoặc cần warm-up dài hơn) khi fine-tune mô hình đã nén, thay vì chỉ "đoán" learning rate từ mô hình gốc.
5. **Khảo sát độ nhạy learning rate, lấy cảm hứng từ Hình 10 phụ lục B.** Hình 10 đo biến động gradient theo độ dài bước chứ không quét LR, nên thí nghiệm quét LR này là mở rộng của người ghi chú. Trên cùng cặp mạng chuẩn/BatchNorm (hoặc không-norm/RMSNorm) của thí nghiệm 1 hoặc 2, train với nhiều learning rate khác nhau (quét theo bậc 2, ví dụ 10^{-4} đến 10^{0}) và ghi lại learning rate lớn nhất mà mạng còn train ổn định (không diverge). Kỳ vọng: mạng có chuẩn hoá chịu được dải learning rate rộng hơn hẳn, minh hoạ trực tiếp hệ quả thực hành quan trọng nhất của toàn bộ lý thuyết trong bài (gradient dự đoán được hơn cho phép learning rate lớn hơn).

10. Tóm tắt công thức / khái niệm

- **Định nghĩa ICS theo optimization (Định nghĩa 2.1):** $\text{ICS}_{t,i} = \|G_{t,i} - G'_{t,i}\|_2$, với $G_{t,i}$ là gradient "đồng loạt" và $G'_{t,i}$ là gradient tính lại sau khi các lớp trước đã cập nhật. ICS nhỏ nghĩa là bài toán của lớp i ít bị xáo trộn bởi cập nhật của các lớp trước.
- **Lipschitz continuity:** $|f(x_1) - f(x_2)| \leq L\|x_1 - x_2\|$.
- **β-smoothness:** gradient của f là β-Lipschitz, tức $\|\nabla f(x_1) - \nabla f(x_2)\| \leq \beta\|x_1 - x_2\|$.
- **Định lý 4.1 (Lipschitzness của loss):**

$$\|\nabla_{y_j}\hat{L}\|^2 \leq \frac{\gamma^2}{\sigma_j^2}\left(\|\nabla_{y_j}L\|^2 - \tfrac{1}{m}\langle 1, \nabla_{y_j}L\rangle^2 - \tfrac{1}{m}\langle \nabla_{y_j}L, \hat{y}_j\rangle^2\right)$$

  Bình phương gradient theo activation của mạng BatchNorm bị chặn trên bởi γ^2/σ_j^2 nhân với bình phương gradient của mạng chuẩn, rồi trừ đi hai số hạng không âm. Khi $\sigma_j \geq \gamma$ (suy luận từ công thức), chặn này không vượt gradient của mạng chuẩn; paper còn nhấn mạnh phần trừ đi có tác dụng ngay cả khi $\sigma_j = \gamma$, và nói rằng kết quả đến từ tái tham số hoá chứ không phải một phép scale đơn giản.
- **Định lý 4.2 (độ mượt/Hessian), bản đơn giản khi Hessian giữ chuẩn tương đối:**

$$(\nabla_{y_j}\hat{L})^\top \frac{\partial^2\hat{L}}{\partial y_j \partial y_j}(\nabla_{y_j}\hat{L}) \leq \frac{\gamma^2}{\sigma^2}\left(\hat{g}_j^\top H_{jj}\hat{g}_j - \tfrac{1}{m\gamma}\langle \hat{g}_j, \hat{y}_j\rangle\left\|\frac{\partial\hat{L}}{\partial y_j}\right\|^2\right)$$

  (Công thức theo bản in của paper, dạng rút gọn của Định lý 4.2.) Diễn giải: số hạng bậc hai (liên quan β-smoothness) cũng bị thu nhỏ tương tự, nên gradient còn dự đoán được tốt hơn cả về hướng, không chỉ độ lớn.
- **Định lý 4.4 (bound minimax trong không gian trọng số):** với $g_j = \max_{\|X\|\leq\lambda}\|\nabla_W L\|^2$, $\hat{g}_j = \max_{\|X\|\leq\lambda}\|\nabla_W\hat{L}\|^2$: $\hat{g}_j \leq \frac{\gamma^2}{\sigma_j^2}\left(g_j^2 - m\mu_{g_j}^2 - \lambda^2\langle\nabla_{y_j}L, \hat{y}_j\rangle^2\right)$ (đúng như bản in của paper; ghi chú suy luận: theo (17)-(18) của chứng minh, số hạng g_j^2 lẽ ra là g_j, nên có thể là lỗi in của paper), tức bound Lipschitz theo activation (Định lý 4.1) được nâng lên thành bound worst-case theo trọng số W thật, chứng minh bằng việc gradient theo W là một phép biến đổi tuyến tính của gradient theo y qua ma trận dữ liệu X ($\partial\hat{L}/\partial W_{\cdot j} = X^\top(\partial\hat{L}/\partial y_j)$).
- **Quan sát 4.3 (BatchNorm không chỉ là rescale):** với mọi W, chọn $\gamma = \sigma_j$ thì (W, γ, β) cho ra đúng activation y_j như mạng chuẩn, nên mọi cực tiểu của mạng chuẩn vẫn là cực tiểu hợp lệ của landscape BatchNorm.
- **Bổ đề 4.5 (khởi tạo thuận lợi), kèm ví dụ tự tính:** gọi $k = \dfrac{\langle W^*, W_0\rangle}{\|W^*\|^2}$ (hệ số chiếu W_0 lên W^*), $\hat{W}^* = kW^*$ là cực tiểu tương ứng trong landscape BatchNorm (do tính scale-invariance $\text{BN}((aW)x) = \text{BN}(Wx)$ với mọi $a > 0$). Khi $\langle W_0, W^*\rangle > 0$:

$$\|W_0 - \hat{W}^*\|^2 = \|W_0 - W^*\|^2 - \|W^*\|^2(1-k)^2 \leq \|W_0 - W^*\|^2$$

  Tự tính: lấy $W^* = (1, 0)$, $W_0 = (2, 1)$ (không gian 2 chiều cho dễ hình dung). Ta có $\langle W^*, W_0\rangle = 1 \times 2 + 0 \times 1 = 2$, $\|W^*\|^2 = 1$, nên $k = 2/1 = 2$ và $\hat{W}^* = kW^* = (2, 0)$. Tính trực tiếp: $\|W_0 - W^*\|^2 = \|(1, 1)\|^2 = 1^2 + 1^2 = 2$; $\|W_0 - \hat{W}^*\|^2 = \|(0, 1)\|^2 = 0^2 + 1^2 = 1$. Kiểm tra lại bằng công thức bổ đề: $\|W^*\|^2(1-k)^2 = 1 \times (1-2)^2 = 1$, và $2 - 1 = 1$, khớp đúng với $\|W_0 - \hat{W}^*\|^2 = 1$ tính trực tiếp. Vậy bình phương khoảng cách tới cực tiểu giảm từ 2 xuống 1 (tự tính), minh hoạ cụ thể bằng số cho Bổ đề 4.5: điểm khởi tạo luôn được "lợi" (hoặc hoà, khi $k = 1$) khi chuyển sang landscape BatchNorm.
- **Đo độ mượt thực nghiệm (mục 3.4):** với gradient hiện tại g và bước η, theo dõi $L(\theta - \eta g)$ (loss landscape), $\|\nabla L(\theta - \eta g) - g\|$ (gradient predictiveness), và tỉ số lớn nhất của biến động gradient trên khoảng cách di chuyển $\eta\|g\|$ (effective β-smoothness).

11. Kết luận ngắn

Paper chứng minh bằng cả thực nghiệm (thí nghiệm tiêm nhiễu, đo ICS theo gradient, đo độ mượt landscape trên VGG và DLN) và lý thuyết (4 định lý/bổ đề cho một lớp BatchNorm chèn sau một lớp fully-connected) rằng lời giải thích kinh điển ”BatchNorm hoạt động vì giảm internal covariate shift” không được dữ liệu ủng hộ; cơ chế thật là BatchNorm tái tham số hoá landscape tối ưu để Lipschitz constant và ”effective” β-smoothness của loss tốt hơn, khiến gradient dự đoán được hơn và cho phép learning rate lớn hơn, train nhanh và ổn định hơn. Hiệu ứng làm mượt này không duy nhất thuộc về BatchNorm: các chuẩn hoá ℓ_p khác cũng cho hiệu ứng tương tự hoặc mạnh hơn dù không kiểm soát phân phối Gauss như BatchNorm, gợi ý nên nhìn việc thiết kế lớp chuẩn hoá qua lăng kính ”độ mượt landscape” thay vì ”kiểm soát thống kê activation”.

Nguồn gốc: https://arxiv.org/abs/1805.11604

Bài 45

Deconstructing What Makes a Good Optimizer for Language Models (2024)

Tác giả: Rosie Zhao (Harvard University), Depen Morwani (Harvard University), David Brandfonbrener (Kempner Institute at Harvard University), Nikhil Vyas (Harvard University), Sham Kakade (Kempner Institute at Harvard University). Bốn tác giả có dấu * (Zhao, Morwani, Brandfonbrener, Vyas) đóng góp ngang nhau, thứ tự tác giả được xáo trộn ngẫu nhiên.
Venue / arXiv: arXiv:2407.07972 (v2, 28/02/2025; ID 2407.* cho thấy nộp lần đầu 07/2024, theo quy ước arXiv, kiến thức chung). Trang bìa bản v2 (ICLR 2025) có tiêu đề dài hơn, thêm cụm ”for Autoregressive Language Models”; tiêu đề H1 của file này giữ tên ngắn hơn. Bản tải về có dòng ”Published as a conference paper at ICLR 2025” ở đầu mỗi trang, tức đây là bản camera-ready ICLR 2025.
Nguồn: https://arxiv.org/abs/2407.07972
Chủ đề: so sánh thực nghiệm có hệ thống nhiều optimizer (SGD, Adam, Adafactor có momentum, Lion, Signum, Sophia) cho pretraining language model decoder-only, trên nhiều quy mô và nhiều siêu tham số; sau đó ”giải phẫu” Adam bằng hai biến thể đơn giản hoá (Signum, Adalayer) để tìm xem phần nào của tính thích nghi (adaptivity) thực sự cần thiết.
Vị trí trong lộ trình: đọc sau opt 06 (Adam) và opt 07 (AdamW decoupled weight decay) nếu đã có trong thư mục này, vì bài này lấy Adam làm chuẩn để so sánh và giải thích tại sao nó hoạt động tốt. Đọc cùng nhóm với opt 48 (Old Optimizer, New Norm), vì cả hai cùng hỏi ”cái gì bên trong Adam mới quan trọng”. Trong thư mục này, đọc trước hoặc sau opt 44 (BatchNorm giúp optimization ra sao) không bắt buộc thứ tự, nhưng cả hai cùng bàn về chuẩn hoá (LayerNorm ở đây, BatchNorm ở opt 44) ảnh hưởng tới optimization.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Phạm vi đọc: toàn bộ văn bản paper (21 trang), gồm thân bài, Mục 4 (Discussion and Limitations), Phụ lục A-G, Bảng 1, tiêu đề và chú thích của Hình 1-20, và chú thích chân trang. Hình 7 được đối chiếu trực tiếp từ ảnh render (giá trị đọc từ hình là xấp xỉ); các hình khác được đối chiếu qua chú thích và lời văn.

1. Bối cảnh

Từ 2015, Adam (Kingma & Ba) vẫn là optimizer chủ đạo để huấn luyện language model, mặc dù rất nhiều optimizer mới được đề xuất với tuyên bố ”tốt hơn Adam” (Lion tìm ra bằng symbolic search, Adafactor xấp xỉ hạng 1 để tiết kiệm bộ nhớ, Shampoo và Sophia được chú ý gần đây; mô tả ”dùng thông tin bậc hai” cho Shampoo và Sophia là kiến thức chung, ngoài paper). Vấn đề: các paper đề xuất optimizer mới thường chỉ so với Adam trên kiến trúc/task họ chọn, không có sự đồng thuận chung về hiệu năng tương đối. Theo paper, Kaddour et al. (2024) là công trình duy nhất so sánh các optimizer này cùng lúc, nhưng chỉ làm trên masked language modeling và một quy mô model duy nhất.

Hai câu hỏi paper muốn trả lời: (1) niềm tin rằng Adam luôn tốt nhất có đúng không, khi so sánh công bằng nhiều optimizer trên autoregressive LM, nhiều siêu tham số, nhiều kiến trúc, nhiều quy mô? (2) Nếu nhiều optimizer thực ra tương đương Adam, thì **thành phần nào** trong Adam mới là thứ tạo ra hiệu quả đó?

Khái niệm cần nhớ: **tính thích nghi (adaptivity)** của optimizer nghĩa là learning rate hiệu dụng cho mỗi tham số tự điều chỉnh theo lịch sử gradient của tham số đó (ví dụ Adam chia gradient cho căn bậc hai trung bình bình phương gradient quá khứ), khác với SGD dùng cùng một learning rate cho mọi tham số.

2. Ý tưởng chính

Bước 1 (Mục 2 của paper): so sánh 5 optimizer, SGD, Adam, Adafactor (bản tự thêm momentum), Lion, Signum, trên 4 quy mô model decoder-only (150m, 300m, 600m, 1.2b tham số không tính embedding), quét learning rate rồi quét các siêu tham số khác tại learning rate tối ưu. Kết quả bất ngờ: **trừ SGD, tất cả các optimizer còn lại gần như tương đương nhau**, cả về hiệu năng tối ưu (validation loss thấp nhất đạt được) lẫn độ ổn định khi lệch khỏi siêu tham số tối ưu.

Bước 2 (Mục 2.6): để hiểu vì sao, paper xét **Signum** (signed momentum, một trường hợp đặc biệt của Lion khi $\beta_1 = \beta_2$). Paper dẫn một lemma có sẵn trong literature (Balles & Hennig, 2018; xem Mục 4.5): tỉ lệ giữa update Adam và update Signum là $|\mathbb{E}[m]|/\sqrt{\mathbb{E}[v^2]}$, nên khi $\beta_1 = \beta_2$ hai update chỉ gần nhau theo trực giác, không đúng tuyệt đối vì tỉ lệ thay đổi theo từng tham số; paper kiểm chứng bằng thực nghiệm (Hình 6). Từ đó paper đưa ra **giả thuyết** rằng lợi ích chính của Adam so với Signum nằm ở việc Adam tách được β_1 (đà ngắn hạn cho dấu gradient) khỏi β_2 (trung bình động bình phương gradient), tức hai tốc độ này độc lập nhau.

Bước 3 (Mục 3): để hiểu **mức độ chi tiết (granularity)** của tính thích nghi cần đến đâu, paper đưa ra **Adalayer**, một biến thể của Adam chỉ lưu **một số vô hướng cho mỗi layer** (so với Adam lưu ma trận đường chéo đầy đủ, Adafactor lưu xấp xỉ hạng 1). Sau khi sửa một chi tiết ở layer cuối (lớp chiếu ra logit), Adalayer* phục hồi gần hết hiệu năng và độ ổn định của Adam. Từ đó, paper tiếp tục tháo rời: layer nào thực sự cần thích nghi mức tinh (không thể thay bằng SGD)? Câu trả lời: **lớp cuối (last layer/logit) và các tham số LayerNorm**; các ma trận còn lại có thể dùng một tỉ lệ learning rate cố định (gần như SGD có tỉ lệ theo lớp) mà vẫn giữ được hầu hết hiệu năng.

3. Phương pháp chi tiết

3.1 Thiết lập chung

- Dữ liệu: **C4**, token hoá bằng tokenizer T5 (Raffel et al., 2020). Đo bằng validation loss.
- Kiến trúc: decoder-only transformer, codebase OLMo (Groeneveld et al., 2024). 4 quy mô: 150m, 300m, 600m, 1.2b tham số (không tính embedding). Paper ghi (Mục 2.2) "The models have widths of 1024, 1024, and 1408 and depths of 12, 24, 24": chỉ có 3 cặp width/depth cho 4 quy mô, và paper **không ghi rõ cặp nào ứng với quy mô nào**, cũng không có bảng kích thước model. Cách ghép dưới đây là suy luận của người ghi chú: 150m = 1024/12, 300m = 1024/24, 600m = 1408/24. Tự tính (ước lượng gần đúng, mỗi block khoảng $12d^2$ tham số với MLP 4 lần width, bỏ qua embedding, bias và LayerNorm): 1024/12 cho khoảng 0,151 tỷ; 1024/24 cho khoảng 0,302 tỷ; 1408/24 cho khoảng 0,571 tỷ (gần mốc 600m). Không cặp nào trong ba cặp cho ra khoảng 1,2 tỷ, nên **paper không cung cấp width/depth cho model 1.2b**; ghi chú không tự suy ra con số còn thiếu.
- MLP hidden = 4 lần width. Activation GeLU. Positional encoding RoPE. Attention head luôn có dimension 64. LayerNorm chuẩn của PyTorch, không học bias cho linear layer hay LayerNorm (theo Wortsman et al., 2024). Huấn luyện mixed precision bfloat16.
- Hai biến thể kiến trúc: **standard** (có QK-LayerNorm theo Dehghani et al., 2023, và z-loss hệ số 10^{-4} theo Chowdhery et al., 2023/PaLM) và **không QK norm / không z-loss**.
- Batch size 256, độ dài chuỗi 512. Số bước mặc định xấp xỉ "chinchilla-optimal" (khoảng 20 lần số tham số về token): 150m huấn luyện 25k bước ($\approx$ 3.3 tỷ token), 300m 50k bước, 600m 100k bước, 1.2b 200k bước; ngoài ra có biến thể "150m-long" huấn luyện 100k bước (4x dài hơn bản chuẩn).
- Siêu tham số mặc định: weight decay = 0; lịch learning rate warmup 10% số bước rồi cosine decay về mức tối thiểu bằng 10% learning rate lớn nhất; $\beta_2 = 0.95$, $\epsilon = 10^{-15}$ (theo Wortsman et al., 2024).

3.2 Các optimizer được so sánh

- **AdamW**: cài đặt PyTorch chuẩn.
- **SGDW**: cài đặt timm (Wightman, 2019).
- **Lion**: cài đặt OLMo (Groeneveld et al., 2024).

- **Adafactor có momentum**: paper tự cài lại theo Zhai et al. (2022), vẫn giữ ước lượng mô men bậc hai dạng factor hoá (rank-1, theo hàng/cột) như Adafactor gốc nhưng thêm momentum, tương đương Adam với mô men bậc hai được xấp xỉ factor hoá.
- **Signum**: vì Signum $\equiv$ Lion với $\beta_1 = \beta_2$, paper dùng lại cài đặt Lion, chỉ đặt $\beta_1 = \beta_2$.
- **Sophia** (Liu et al., 2024): thử trong Appendix G, nhưng vì không vượt được Signum (Signum đạt được từ Sophia bằng cách đặt $\rho = 0$) nên không đưa vào các hình chính.

3.3 Quy trình quét siêu tham số (Mục 2.1)

Bắt đầu từ siêu tham số khuyến nghị trong literature (ví dụ $\beta_1 = 0.9$). Đầu tiên quét learning rate để tìm learning rate tối ưu cho mỗi optimizer. Sau đó, **cố định learning rate tối ưu đó** và quét từng siêu tham số khác (momentum, weight decay, warmup, batch size, ϵ, β_2) theo kiểu một chiều (1D sweep). Paper tự nhận hạn chế: cách này bỏ qua tương tác hai chiều giữa các siêu tham số (ví dụ batch size và learning rate có tương tác đã biết từ Shallue et al., 2019; Porian et al., 2024), nhưng nói đây là phương án khả thi về compute.

Khoảng quét learning rate (nhân theo bội số $\sqrt{10}$): Adam và Adafactor từ 10^{-4} đến 1; Lion và Signum từ 10^{-5} đến 10^{-1}; SGD từ 10^{-3} đến 10. Trong lượt quét learning rate, $\beta_1 = 0.9$ cho mọi optimizer trừ SGD dùng $\beta_1 = 0.98$.

3.4 Adalayer (Mục 3, thuật toán 1)

Ba optimizer Adam, Adafactor, Adalayer đều (xấp xỉ) lưu ma trận mô men bậc hai theo đường chéo, nhưng với độ chi tiết (granularity) giảm dần: với một layer kích thước $m \times n$, Adam lưu nguyên ma trận mn số; Adafactor lưu trung bình theo hàng và theo cột (xấp xỉ hạng 1); **Adalayer lưu đúng một số vô hướng cho cả layer** (trung bình bình phương gradient toàn layer). Công thức cập nhật Adalayer cho layer l có p tham số, tại bước t:

$$g_t^l \leftarrow \nabla_l L(w_t); \qquad v_t^l \leftarrow \beta_2 v_{t-1}^l + (1-\beta_2)\, p^{-1/2} \|g_t^l\|_2^2$$

$$m_t^l \leftarrow \beta_1 m_{t-1}^l + (1-\beta_1) g_t^l; \qquad w_{t+1}^l \leftarrow w_t^l - \eta \frac{m_t^l}{\sqrt{v_t^l} + \epsilon}$$

(bias correction giữ như Adam, paper không viết lại chi tiết). Đây là một trường hợp đặc biệt của optimizer BAGM (Zheng & Kwok, 2019) đã biết trước, không phải đóng góp hoàn toàn mới về thuật toán; đóng góp của paper là **dùng Adalayer như một công cụ phân tích** để xem độ chi tiết thích nghi nào là cần thiết.

Paper nhận thấy Adalayer ”trần” (naive) kém ổn định và kém hiệu năng hơn Adam (Hình 10, Appendix C). Nguyên nhân nghi ngờ: lớp cuối (lớp chiếu ra logit) có các token với tần suất rất khác nhau nên có quy mô gradient rất khác nhau giữa các logit; gộp chung một số vô hướng cho cả lớp cuối làm mất thông tin quan trọng. Sửa: tách **tập trọng số đi vào mỗi logit** của lớp cuối thành một ”block” riêng khi tính mô men bậc hai (các layer khác giữ nguyên một scalar/layer). Biến thể này gọi là **Adalayer*** và phục hồi gần hết hiệu năng + ổn định của Adam.

4. Thí nghiệm và kết quả chính

4.1 So sánh 5 optimizer qua 4 quy mô (Hình 1, Bảng 1)

Hình 1 vẽ validation loss cuối cùng theo learning rate (đã dịch trục để learning rate tối ưu của mỗi optimizer trùng nhau), cho 4 quy mô 150m/300m/600m/1.2b. Kết luận chính: **trừ SGD, bốn optimizer còn lại (Adam, Adafactor, Lion, Signum) đạt hiệu năng tối ưu tương đương nhau VÀ độ ổn định với learning rate cũng tương đương nhau**, nhất quán qua cả 4 quy mô.

Bảng 1 (Phụ lục B, Table 1; learning rate tối ưu tìm được từ Hình 1):

Optimizer	150m	300m	600m	1.2b
Adam	3.16e-3	1e-3	1e-3	1e-3
Adafactor	3.16e-3	1e-3	1e-3	1e-3
Lion	3.16e-4	3.16e-4	3.16e-4	1e-4
Signum	3.16e-4	3.16e-4	3.16e-4	3.16e-4

Nhận xét của paper: learning rate tối ưu của Adam và Adafactor gần như trùng nhau; learning rate tối ưu của Lion/Signum nhỏ hơn một bậc độ lớn so với Adam/Adafactor (paper không giải thích lý do; ngoài paper, kiến thức chung: update của Lion/Signum có biên độ cố định do lấy dấu, còn Adam chia theo căn bậc hai mô men bậc hai).

Tự tính kiểm tra số token: paper nói 150m huấn luyện 25.000 bước, batch 256 chuỗi, chuỗi dài 512 token, và ước tính ra "≈ 3.3 tỷ token". Tự tính: $25000 \times 256 \times 512 = 3.276.800.000 \approx 3,28$ tỷ token, khớp với số paper nêu. Tương tự cho 300m: $50000 \times 256 \times 512 = 6.553.600.000 \approx 6,55$ tỷ token (số "20 lần tham số" lý thuyết là 6 tỷ, gần đúng); 600m: $100000 \times 256 \times 512 = 13.107.200.000 \approx 13,1$ tỷ token (20x600m=12 tỷ); 1.2b: $200000 \times 256 \times 512 = 26.214.400.000 \approx 26,2$ tỷ token (20x1.2b=24 tỷ). Các số tự tính này đều hơi lớn hơn mốc "20x tham số" lý thuyết một chút, phù hợp cách paper nói "approximately chinchilla-optimal".

4.2 Ảnh hưởng kiến trúc và thời gian huấn luyện (Hình 2, 3)

Bỏ QK-LayerNorm và z-loss (Hình 2): model kém ổn định hơn nói chung, nhưng **thứ tự so sánh giữa các optimizer không đổi**. Huấn luyện 150m lâu hơn 4 lần (100k bước, Hình 3): hiệu năng tốt hơn và ổn định hơn với learning rate, nhưng vẫn không đổi thứ tự xếp hạng optimizer.

4.3 Quét momentum (Hình 4)

Cố định learning rate ở mức tối ưu riêng cho mỗi optimizer, quét β_1. Trong khoảng $\beta_1 \in [0.8, 0.98]$, Adam/Adafactor/Lion/Signum có độ ổn định tương đương; β_1 cao thì Lion lợi hơn, β_1 thấp thì Adam/Adafactor lợi hơn; SGD nhạy cảm hơn hẳn với momentum (vẫn cần momentum cao mới cạnh tranh được).

4.4 Các siêu tham số khác (Hình 5)

Quét weight decay, độ dài warmup, batch size (hàng trên) và ϵ, β_2 (hàng dưới), tại learning rate và momentum tối ưu. Theo Mục 2.2, các ablation này được paper thực hiện trên model 150m standard. Nhận xét: các optimizer không-SGD nói chung rất ổn định với các siêu tham số này (khác hẳn độ nhạy với learning rate/momentum); có thể cải thiện tới 0,02 validation loss nếu dùng weight decay cao hơn, warmup dài hơn, hoặc β_2 cao hơn so với mặc định. Hai ngoại lệ: Lion kém hơn ở giá trị β_2 cực trị (vì trong Lion, β_2 đóng vai trò "đà dài hạn" khác với vai trò "trung bình động bình phương gradient" của β_2 trong Adam/Adafactor); và **hiệu năng của mọi optimizer đều xấu đi khi batch size lớn** (hiệu ứng giảm dần lợi ích khi tăng batch size, đã biết từ Shallue et al. 2019, Zhang et al. 2024a).

4.5 Signum ≈ Adam (Mục 2.6, Hình 6)

Lemma 1 (trích từ Balles & Hennig, 2018, không phải đóng góp mới của paper này): xét một tham số có lịch sử gradient $g_t, g_{t-1}, \ldots$. Gọi m là biến ngẫu nhiên nhận giá trị $g_{t-\tau}$ với xác suất $(1-\beta_1)\beta_1^\tau$, và v là biến ngẫu nhiên nhận giá trị $g_{t-\tau}$ với xác suất $(1-\beta_2)\beta_2^\tau$ (tức m, v là cách viết lại trung bình động theo β_1, β_2 dưới dạng kỳ vọng). Khi đó update Adam và update Signum liên hệ bởi:

$$\delta_{Adam} = \delta_{Signum} \cdot \frac{|\mathbb{E}[m]|}{\sqrt{\mathbb{E}[v^2]}}$$

Nếu $\beta_1 = \beta_2$ thì $m = v$, và tỉ lệ giữa update Adam/Signum chỉ còn là tỉ lệ giữa trung bình và căn bậc hai mô men bậc hai của **cùng một** biến m, nên về trực giác hai update sẽ gần nhau. Paper kiểm chứng (Hình 6): quét β_1 cho cả Adam (đặt $\beta_2 = \beta_1$) và Signum, hai đường cong hiệu năng/ổn định rất giống nhau. Vì setting chuẩn dùng $\beta_2 = 0.95, \beta_1 = 0.9$ (hai giá trị gần nhau), đây có thể giải thích vì sao Adam và Signum hoạt động tương tự ở Hình 1. Paper suy ra **giả thuyết**: lợi ích chính của Adam so với Signum nằm ở việc Adam cho phép β_2 khác β_1 (quan sát thực nghiệm: cố định β_2 và quét β_1 riêng cho Adam ổn định hơn Signum).

5. Phân tích / ablation

5.1 Adalayer* phục hồi gần hết Adam (Mục 3.1, Hình 7, 8, 10)

Hình 7 vẽ phân vị (quantile) của **learning rate hiệu dụng** $\eta_t/(\sqrt{v_t'} + \epsilon)$ theo bước huấn luyện cho 3 nhóm: các hàng của lớp cuối (last layer blocks), các ma trận khác, và các tham số LayerNorm, khi huấn luyện model 150m bằng Adalayer*. Kết quả (đọc từ hình, xấp xỉ): learning rate hiệu dụng của các hàng lớp cuối trải trên **nhiều bậc độ lớn**, từ khoảng 10^{-1} ở đầu huấn luyện đến khoảng 2×10^4 quanh bước 5000 ở quantile cao nhất; khoảng cách giữa quantile 0 và 0.8 khoảng 3 bậc. Các ma trận khác có khoảng cách quantile khoảng 1 bậc và khá đồng nhất trong nhóm của chúng (paper gọi là "remarkably similar"); LayerNorm có learning rate hiệu dụng nhỏ hơn, khoảng 10^0 đến 10^2. Đây là bằng chứng cho thấy lớp cuối cần được chia nhỏ ra để precondition riêng từng logit, trong khi các ma trận khác có thể dùng một tỉ lệ chung.

5.2 Chỉ cần thích nghi ở lớp cuối + LayerNorm (Mục 3.2, Hình 8, 9, 11)

Thử nghiệm: huấn luyện lớp cuối bằng Adalayer* (learning rate cố định 3.16×10^{-3}), còn lại dùng SGD, cả hai với $\beta_1 = 0.9$. Kết quả (Hình 8 trái): hiệu năng tốt hơn SGD thuần, nhưng **chưa ổn định** như Adalayer*/Adam đầy đủ. *Paper quy nguyên nhân mất ổn định này cho các block LayerNorm (vì chúng có learning rate hiệu dụng nhỏ theo Hình 7, dễ "nổ" ở learning rate SGD cao). Thêm LayerNorm vào nhóm dùng Adalayer* (Hình 8 phải): phục hồi cả hiệu năng và ổn định của Adalayer* đầy đủ. Hình 11 (phụ lục) cho thấy xu hướng này vẫn đúng ở quy mô 300m và 600m.

Một kiểm tra chặt hơn (Hình 9, Mục 3.2 đoạn cuối): để loại khả năng "khoảng learning rate khác nhau giữa các nhóm" (vì SGD cần learning rate cao hơn Adalayer* nhiều bậc) là nguyên nhân gây ổn định giả, paper làm thí nghiệm khác: huấn luyện **tất cả** layer bằng Adalayer* (quét learning rate η chung), nhưng **đóng băng mô men bậc hai** (tức dùng tỉ lệ learning rate cố định từ lúc khởi tạo, hiệu quả tương đương SGD có tỉ lệ riêng theo layer) cho mọi layer **trừ** lớp cuối và LayerNorm. Cách làm: chạy 1000 batch đầu để ước lượng mô men bậc hai ban đầu cho mọi layer mà không cập nhật trọng số, sau đó train bình thường. Kết quả: với model 150m, cách này gần như phục hồi hoàn toàn hiệu năng và ổn định của Adalayer*; với 300m còn vượt hiệu năng đỉnh của Adalayer* và gần bằng Adam. Đây là bằng chứng mạnh hơn cho việc **chỉ lớp cuối và LayerNorm cần thích nghi liên tục**, phần còn lại có thể dùng tỉ lệ cố định ngay từ đầu.

5.3 Các ablation phụ (Phụ lục D, E, F, G)

- Dùng Adalayer* chỉ cho các ma trận (LR cố định 10^{-3}, không áp cho lớp cuối/LayerNorm), còn lớp cuối + LayerNorm dùng SGD: hiệu năng tốt hơn SGD nhưng **không** đạt hiệu năng đỉnh của Adalayer*, và vẫn mất ổn định ở learning rate cao (Hình 12). Tức là lớp cuối + LayerNorm mới là nơi cần thích nghi, không phải các ma trận lớn.
- Thiết lập như Hình 8 trái (Adalayer* chỉ cho lớp cuối, learning rate cố định 3.16×10^{-3}; SGD cho các ma trận), nhưng tắt việc train tham số LayerNorm (giữ nguyên giá trị khởi tạo): ổn định hơn SGD thường nhưng vẫn chưa đạt hiệu năng Adalayer* (Hình 13), cho thấy việc **học** (không chỉ chuẩn hoá) các tham số LayerNorm cũng có ích, rõ hơn ở model lớn.
- Thay Adalayer* bằng **Adafactor** cho lớp cuối + LayerNorm (còn lại SGD): hiệu năng/ổn định tương đương Adafactor đầy đủ nhưng không vượt (Hình 14).
- Thay SGD (cho các ma trận) bằng **AdaSGD** (Wang & Wiens, 2020, một learning rate thích nghi toàn cục dùng chung cho mọi ma trận) kết hợp Adalayer* cho lớp cuối/LayerNorm: khớp gần như hoàn toàn

hiệu năng Adalayer* đầy đủ, nhưng vẫn còn một khoảng cách nhỏ tới Adam (Hình 16; Hình 15 cho thấy AdaSGD đứng riêng vẫn chưa ổn định theo learning rate). Paper cho rằng khoảng cách này có thể do một số ma trận vẫn cần độ chi tiết thích nghi cao hơn một scalar toàn cục.
- Đóng băng tỉ lệ Adalayer* cho **toàn bộ** layer (kể cả lớp cuối, LayerNorm) từ lúc khởi tạo: phân kỳ nhanh, không đạt hiệu năng hay ổn định nào (Hình 17). Chỉ giữ cập nhật một trong hai (lớp cuối hoặc LayerNorm), phần còn lại đóng băng: đạt hiệu năng đỉnh nhưng vẫn mất ổn định ở learning rate cao (Hình 18). Giữ cập nhật lớp cuối, đóng băng tỉ lệ của các layer còn lại và tắt học LayerNorm: ổn định hơn nhưng kém hiệu năng hơn (Hình 19).
- Phụ lục G: Signum là trường hợp đặc biệt của Sophia (Liu et al., 2024) khi $\rho = 0$; paper thử Sophia với siêu tham số gốc (kể cả trick scale attention theo nghịch đảo chỉ số layer) nhưng không thấy vượt Signum trên model 150m (Hình 20).

6. Giới hạn

Paper tự liệt kê (Mục 4, Discussion and Limitations) các hạn chế sau. Bốn hạn chế đầu nằm trong câu ”Of course, there are several limitations...”; hạn chế thứ năm là phạm vi của kết luận, cũng được paper nêu trong cùng mục:

- Chỉ ablate một số ít lựa chọn kiến trúc (chủ yếu QK-LayerNorm và z-loss); nhiều lựa chọn kiến trúc khác không được xét.
- Chỉ quét siêu tham số theo kiểu một chiều (1D), không xét tương tác nhiều chiều (ví dụ learning rate với batch size), dù paper nhận biết các tương tác này có khả năng tồn tại.
- Cố định batch size (256) trong toàn bộ thí nghiệm chính.
- Chỉ xét autoregressive language modeling trên một dataset duy nhất (C4); không xét domain khác hay kiến trúc LM khác.
- Chỉ so sánh các optimizer dùng **preconditioning đường chéo** (Adam, Adafactor, Lion, Signum, SGD); các optimizer phi đường chéo như Shampoo (đã được dẫn là có hiệu năng tốt hơn Adam trong công trình khác, Gupta et al., 2018a) không được đưa vào so sánh thực nghiệm chính, nên paper chỉ kết luận rằng lựa chọn optimizer ít có triển vọng cải thiện hiệu năng (paper: ”unlikely to be a promising approach”) trong phạm vi các optimizer đường chéo.

Nhận xét thêm của người ghi chú (ngoài paper, suy luận):

- Quy mô lớn nhất thử nghiệm là 1.2 tỷ tham số, với số bước xấp xỉ ”chinchilla-optimal” (không phải rất dài); người ghi chú nhận xét (không phải paper nói) rằng mô hình production thường pretrain lâu hơn và lớn hơn nhiều, nên các so sánh optimizer có thể không giữ nguyên thứ hạng ở quy mô đó.
- Như đã nêu ở Mục 3.1, paper không có bảng kích thước model: chỉ có 3 cặp width/depth cho 4 quy mô, nên không có width/depth cho 1.2b. Đây là điểm trình bày chưa đầy đủ, không phải lỗi phương pháp, nhưng gây khó khăn khi muốn tái lập chính xác.

7. Liên hệ với các bài khác

- **llm content/10_2019_rmsnorm.md** (RMSNorm): phát hiện trung tâm của bài này là lớp cuối (logit) và tham số LayerNorm cần thích nghi đặc biệt trong Adam; bài RMSNorm bàn về một biến thể chuẩn hoá khác (bỏ phần tái tâm/re-centering) đang được dùng rộng rãi hơn LayerNorm trong LLM hiện đại, nên khi đọc cùng nhau sẽ thấy câu hỏi ”các tham số chuẩn hoá cần optimizer đối xử thế nào” áp dụng cho cả hai loại norm.
- **llm content/09_2020_pre_ln_transformer.md** (Pre-LN Transformer): bài đó giải thích vì sao đặt LayerNorm trước sub-layer (thay vì sau) giúp gradient ổn định hơn khi scale depth; đọc cùng bài 45 này để thấy hai góc nhìn bổ sung cho nhau về vai trò của chuẩn hoá trong việc optimizer hoạt động tốt.
- **llm content/03_2020_scaling_laws_kaplan.md** (Scaling laws): bài 45 thử nghiệm ở 4 quy mô (150m đến 1.2b) và quan tâm liệu kết luận có giữ nguyên qua quy mô; đây đúng là câu hỏi trọng tâm của

scaling laws (số liệu có ngoại suy theo quy mô hay không), nên hai bài bổ trợ trực tiếp.

- **llm content/24_2025_gated_deltanet.md** (GatedDeltaNet): vì dự án đang dùng kiến trúc hybrid GatedDeltaNet + full attention, bài này giúp đặt câu hỏi liệu các gate/norm trong GatedDeltaNet cũng cần được ”đối xử đặc biệt” như LayerNorm trong bài 45 không (xem Mục 8 bên dưới).

(Lưu ý: khi kiểm tra ngày 09/11/2026 bằng `ls`, các file opt 01-51 trong ”optimization content/” đã có trên đĩa; bài này chưa liên kết tới chúng. Các bài opt 06 (Adam), opt 07 (AdamW), opt 44 (BatchNorm) và opt 48 (Old Optimizer, New Norm) có thể được thêm liên kết vào mục này nếu muốn.)

8. Ứng dụng cho dự án (suy luận)

Đây là suy luận của người ghi chú, dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm 1.500 token văn bản và 1.6-2.1K token thị giác, chạy trên GPU $\leq$ 8 GB, batch size 1).

- **Chọn optimizer khi fine-tune/distill:** vì paper cho thấy Adam, AdamW, Adafactor (với momentum), Lion, Signum đều cho hiệu năng và độ ổn định tương đương (chỉ SGD kém hơn rõ rệt), lựa chọn optimizer cho LightOnOCR có thể dựa chủ yếu vào **ràng buộc bộ nhớ** như paper gợi ý ở phần Discussion, không cần lo mất hiệu năng. Với GPU $\leq$ 8 GB và model 0.8B, Adafactor (bộ nhớ optimizer state thấp hơn Adam vì xấp xỉ hạng 1) hoặc Lion/Signum (không cần lưu mô men bậc hai, chỉ cần dấu + một mô men) là lựa chọn hợp lý để dành bộ nhớ cho activation/kích thước batch, đặc biệt khi chuỗi dài tới 3.600 token/trang.
- **Ưu tiên tinh chỉnh learning rate + momentum, giảm công quét các siêu tham số khác:** paper cho thấy hiệu năng nhạy với learning rate và momentum hơn hẳn so với weight decay/warmup/β_2/ϵ (Hình 4, 5). Với ngân sách tune hạn chế (1 GPU), nên dồn effort vào quét learning rate (và có thể momentum) trước, để mặc định hợp lý cho các siêu tham số còn lại.
- **Cân nhắc đối xử đặc biệt với lớp cuối và các tham số chuẩn hoá khi fine-tune với ngân sách hẹp:** phát hiện cốt lõi của bài là chỉ lớp cuối (đầu ra logit) và LayerNorm cần thích nghi mịn; nếu muốn cắt giảm bộ nhớ optimizer khi fine-tune LightOnOCR (ví dụ dùng Adam đầy đủ chỉ cho lớp output head/token embedding và các lớp norm, còn áp dụng một learning rate cố định/một scalar chung cho phần lớn ma trận attention/MLP), có cơ sở lý thuyết tương tự để thử. Đây là suy luận ngoại suy từ kiến trúc decoder-only chuẩn (LayerNorm) sang kiến trúc hybrid GatedDeltaNet (có các gate riêng và RMSNorm/chuẩn hoá khác), **chưa được paper kiểm chứng trực tiếp**; cần thử nghiệm riêng trước khi tin tưởng.
- **Hạn chế khi ngoại suy:** paper chỉ test tới 1.2B tham số pretraining từ đầu trên C4 (text thuần), còn LightOnOCR là VLM huấn luyện tiếp/distill trên dữ liệu OCR đa phương thức ở quy mô nhỏ hơn 1B; các kết luận về ”optimizer nào tương đương optimizer nào” có thể không giữ nguyên khi fine-tune ngắn trên domain hẹp, nên nên xem đây là gợi ý hướng thử nghiệm, không phải quy tắc chắc chắn.

9. Thí nghiệm gợi ý

1. **Tái lập Hình 1 ở quy mô nhỏ:** huấn luyện một LM decoder-only nhỏ (1-5 triệu tham số, kiểu nanoGPT) trên một tập văn bản nhỏ (ví dụ TinyShakespeare hoặc một phần WikiText) trên 1 GPU 8 GB, quét learning rate cho SGD, AdamW, và Signum (cài bằng Lion với $\beta_1 = \beta_2$). Đo: validation loss tốt nhất và độ rộng của ”vùng learning rate tốt” cho mỗi optimizer. Kỳ vọng: SGD có vùng ổn định hẹp hơn và loss tối ưu cao hơn AdamW/Signum; AdamW và Signum tương đối giống nhau, giống Hình 1 của paper.
2. **Cài Adalayer thủ công và kiểm tra hiệu ứng lớp cuối:** trên cùng LM nhỏ ở trên, viết một optimizer tối giản lưu đúng một số vô hướng mô men bậc hai mỗi layer (theo công thức Mục 3.4), so sánh (a) Adalayer trần, (b) Adalayer có sửa lớp cuối theo từng hàng logit, (c) AdamW đầy đủ. Đo validation loss theo learning rate. Kỳ vọng: (a) kém và kém ổn định hơn (c); (b) gần với (c) hơn hẳn (a), minh

hoạ trực tiếp phát hiện chính của paper ở quy mô nhỏ.

3. **Đo learning rate hiệu dụng theo hàng của lớp output head:** khi train LM nhỏ bằng AdamW, ghi lại $\eta/(\sqrt{v}+\epsilon)$ trung bình cho từng hàng trọng số của lớp cuối (ứng với từng token trong vocab) ở một vài bước huấn luyện, rồi vẽ histogram/log-scale. Vì ngôn ngữ (và cả "từ vựng" OCR gồm ký tự + token thị giác) có phân phối tần suất rất lệch (Zipf), kỳ vọng thấy learning rate hiệu dụng khác nhau nhiều bậc giữa token phổ biến và hiếm, tương tự Hình 7 của paper, hỗ trợ (hoặc phản bác) ý tưởng ứng dụng ở Mục 8.
4. **Thử "đóng băng" mô men bậc hai sau warmup cho phần lớn tham số:** trên LM nhỏ, chạy AdamW như thường trong N bước đầu để mô men bậc hai ổn định, sau đó đóng băng mô men bậc hai (coi như learning rate hiệu dụng cố định) cho mọi layer trừ lớp cuối và LayerNorm, tiếp tục train, so với AdamW đầy đủ cùng tổng số bước. Đo validation loss cuối. Nếu kết quả gần AdamW đầy đủ (giống Hình 9 của paper ở quy mô 150m/300m), đây là gợi ý thực nghiệm nhỏ cho việc cắt giảm bộ nhớ optimizer khi fine-tune trên GPU hẹp.
5. **Sweep weight decay và batch size trên LM nhỏ:** cố định learning rate/momentum tối ưu, quét weight decay (0, 0.01, 0.1) và batch size (nhỏ, trung bình, lớn so với kích thước data) cho AdamW. Đo validation loss. Kỳ vọng: ảnh hưởng của weight decay nhỏ hơn ảnh hưởng của learning rate (khớp Hình 5), và hiệu năng giảm dần khi batch size tăng quá cao so với kích thước bộ nhớ của LightOnOCR (batch 1 hiện tại), giúp hiểu rõ hơn đánh đổi giữa batch size hiệu dụng (qua gradient accumulation) và chất lượng optimization trên 1 GPU.

10. Tóm tắt công thức / khái niệm

- **Cập nhật Adalayer** (một scalar mô men bậc hai mỗi layer l, p tham số):

$$v_t^l = \beta_2 v_{t-1}^l + (1-\beta_2)\, p^{-1/2} \|g_t^l\|_2^2, \qquad w_{t+1}^l = w_t^l - \eta \frac{m_t^l}{\sqrt{v_t^l}+\epsilon}$$

- **Learning rate hiệu dụng** của một layer l tại bước t: $\eta_t/(\sqrt{v_t^l}+\epsilon)$; đây là đại lượng paper dùng để chẩn đoán layer nào "cần" thích nghi tinh (biến động nhiều bậc độ lớn giữa các hàng) và layer nào không.
- **Lemma liên hệ Adam và Signum** (Balles & Hennig, 2018, trích dẫn lại trong paper): với m là trung bình động mũ của gradient theo β_1 và v theo β_2 (viết dưới dạng kỳ vọng trên lịch sử gradient), $\delta_{Adam} = \delta_{Signum} \cdot |\mathbb{E}[m]|/\sqrt{\mathbb{E}[v^2]}$; khi $\beta_1 = \beta_2$ thì $m = v$ và hai update có xu hướng gần nhau.
- **Signum** = Lion với $\beta_1 = \beta_2$ = signed momentum: $w_{t+1} = w_t - \eta\,\text{sign}(m_t)$ với $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$ (paper không viết lại công thức Lion gốc trong văn bản chính, chỉ dẫn chiếu Chen et al., 2023, nên công thức sign-momentum này là kiến thức chung về Lion/Signum, không phải trích trực tiếp từ công thức trong bài).
- **Chinchilla-optimal token count (quy ước paper dùng, không phải định nghĩa toán học chặt)**: số token huấn luyện $\approx 20\times$ số tham số không-embedding; tự tính minh hoạ ở Mục 4.1.
- **AdaSGD** (Wang & Wiens, 2020, paper mô tả bằng lời trong Phụ lục E, không đưa công thức): một SGD có learning rate toàn cục tự thích nghi theo thời gian huấn luyện, tương đương "Adalayer*" với một block duy nhất cho toàn bộ ma trận (không tính lớp cuối và LayerNorm).

11. Kết luận ngắn

Khi so sánh công bằng (nhiều quy mô, nhiều siêu tham số, cả hiệu năng tối ưu và độ ổn định), các optimizer phổ biến cho pretraining autoregressive LM (Adam, Adafactor có momentum, Lion, Signum) gần như tương đương nhau, chỉ SGD thực sự kém hơn; lựa chọn optimizer trong nhóm đó nên dựa vào ràng buộc thực dụng như bộ nhớ, không phải vì khác biệt hiệu năng. Với Signum, paper đặt **giả thuyết** rằng phần lợi ích chính của Adam nằm ở việc tách được β_1 và β_2 (dựa trên quan sát ở Hình 6). Và quan trọng nhất cho việc hiểu Adam: tính thích nghi mịn theo từng tham số không cần thiết ở mọi nơi trong mạng, chỉ **lớp cuối (logit) và**

các tham số LayerNorm cần nó để giữ cả hiệu năng và ổn định; phần lớn các ma trận attention/MLP có thể dùng một tỉ lệ learning rate thô hơn (theo layer, hoặc thậm chí cố định từ lúc khởi tạo) mà không mất nhiều so với Adalayer* (vẫn còn một khoảng cách nhỏ tới Adam, Phụ lục E).

Nguồn gốc: https://arxiv.org/abs/2407.07972

Bài 46

Practical Recommendations for Gradient-Based Training of Deep Architectures (2012)

Tác giả: Yoshua Bengio (được viết là chương sách, không ghi tên trường/viện trong văn bản chính; lời cảm ơn cuối bài ghi hỗ trợ từ NSERC, FQRNT, CIFAR, Canada Research Chairs).
Venue / arXiv: arXiv:1206.5533 (v2, 16/09/2012). Bài tự giới thiệu là ”a practical guide” / ”this chapter”, nối tiếp ”the previous edition of this book” và dẫn tới ”another chapter (Hinton, 2013)” về họ Boltzmann machine; paper không nêu rõ tên sách hay venue hội nghị/tạp chí trong văn bản chính (được biết rộng rãi ngoài paper là chương sách ”Neural Networks: Tricks of the Trade”, nhưng vì tên sách không xuất hiện trực tiếp trong full text đã tải, không ghi chắc như trích dẫn từ paper).
Nguồn: https://arxiv.org/abs/1206.5533
Chủ đề: hướng dẫn thực hành (không phải một thí nghiệm/đề xuất thuật toán mới) về cách chọn và tinh chỉnh siêu tham số khi huấn luyện mạng neural/deep architecture bằng gradient: learning rate, mini-batch, momentum, weight decay, khởi tạo trọng số, tìm kiếm siêu tham số (grid/random/greedy theo layer), gradient checking, trực quan hoá, và các câu hỏi mở về vì sao mạng sâu khó huấn luyện.
Vị trí trong lộ trình: đọc như một bài ”tổng quan thực hành” có thể đọc sớm trong thư mục optimization, trước các bài cụ thể hơn như opt 06 (Adam), opt 44 (BatchNorm) hay opt 47 (disciplined hyper-parameters, cùng chủ đề learning rate/batch size/momentum nhưng với thực nghiệm hệ thống hơn 6 năm sau). Nhiều khái niệm ở đây (learning rate schedule, momentum, khởi tạo Glorot) là nền cho opt 18 (SGDR) và opt 45 (deconstructing optimizer) đọc sau.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Đây là bài dạng ”practical tutorial” dài (28 trang nội dung + tài liệu tham khảo); người ghi chú đã đọc toàn bộ Mục 1 đến Mục 6 (Introduction, Gradients, Hyper-Parameters, Debugging and Analysis, Other Recommendations, Open Questions, Conclusion), không đọc chi tiết danh sách tài liệu tham khảo ở cuối. Ghi chú tập trung vào các khuyến nghị thực hành cốt lõi, không lặp lại từng câu trích dẫn phụ.

1. Bối cảnh

Năm 2012, huấn luyện mạng neural sâu (deep learning) đã hồi sinh từ bước ngoặt 2006 (Hinton et al. 2006; Bengio et al. 2007; Ranzato et al. 2007) dựa trên **pretraining không giám sát theo từng lớp (greedy layer-wise unsupervised pretraining)**: huấn luyện từng lớp một bằng một thuật toán học biểu diễn không giám sát (RBM, auto-encoder, hoặc biến thể sparse coding), dùng biểu diễn học được của lớp dưới làm đầu vào cho lớp trên, sau đó có thể fine-tune có giám sát toàn mạng.

Vấn đề thực hành mà bài này giải quyết: các thuật toán học sâu có **rất nhiều siêu tham số** (learning rate, kích thước lớp ẩn, hệ số weight decay, cách khởi tạo, v.v.), và theo paper, các nhóm nghiên cứu khác nhau không luôn đồng thuận về cách thực hành tốt nhất, trong khi nhiều khuyến nghị ”chưa được kiểm chứng hình thức”. Bài này là một chương sách tổng hợp kinh nghiệm thực hành (và một phần biện minh lý thuyết) cho việc huấn luyện và debug mạng neural nhiều lớp, không giới hạn ở một thuật toán cụ thể.

Khái niệm nền cần nhớ: **hyper-parameter** là biến phải đặt trước khi chạy thuật toán học A trên dữ liệu, không do chính A chọn trong quá trình học (ví dụ learning rate, số lớp ẩn); việc chọn giá trị hyper-parameter tương đương bài toán **model selection**.

2. Ý tưởng chính

Paper không đề xuất một thuật toán mới, mà tổ chức lại tri thức thực hành thành các nhóm:

1. **Cơ chế gradient và học online** (Mục 2): SGD là cách tối ưu hoá chủ đạo; quan trọng là hiểu SGD với dữ liệu i.i.d. thực chất đang tối ưu ước lượng không chệch của gradient của **lỗi tổng quát hoá (generalization error)**, không chỉ lỗi huấn luyện.
2. **Danh sách và cách chọn siêu tham số** (Mục 3): chia hai nhóm, siêu tham số của bộ tối ưu hoá (learning rate, lịch learning rate, batch size, số bước huấn luyện, momentum) và siêu tham số của model/criterion (số unit ẩn, hệ số weight decay, hệ số sparsity, loại non-linearity, cách khởi tạo trọng số, tiền xử lý dữ liệu). Kèm theo chiến lược tìm kiếm: grid search, coordinate descent, random search, và tìm kiếm tham lam theo từng lớp khi pretraining.
3. **Debug và trực quan hoá** (Mục 4): gradient checking bằng sai phân hữu hạn, theo dõi train/validation loss, trực quan hoá filter và quỹ đạo học.
4. **Các khuyến nghị khác** (Mục 5): tận dụng GPU/BLAS, xử lý input sparse, chia sẻ tham số (embedding, multi-task, multi-relational learning).
5. **Câu hỏi mở** (Mục 6): vì sao mạng sâu khó huấn luyện hơn mạng nông, và các hướng dùng learning rate thích nghi / phương pháp bậc hai để giảm sự phụ thuộc vào learning rate thủ công.

Tinh thần xuyên suốt, tác giả nhắc lại ở phần mở đầu và kết luận: đây là ”living practice”, nên được **thử và thách thức**, không phải quy tắc bất biến.

3. Phương pháp chi tiết

3.1 Gradient và learning rate (Mục 2.1)

SGD online cập nhật theo mỗi ví dụ hoặc mini-batch kích thước B (công thức (1) của paper):

$$\theta^{(t)} \leftarrow \theta^{(t-1)} - \epsilon_t \frac{1}{B} \sum_{t'=Bt+1}^{B(t+1)} \frac{\partial L(z_{t'}, \theta)}{\partial \theta}$$

$B = 1$ là SGD online thuần; $B =$ kích thước tập huấn luyện là ”batch gradient descent”. Paper nói learning rate tối ưu thường gần (trong khoảng hệ số 2) với learning rate lớn nhất mà không gây phân kỳ (divergence) của tiêu chí huấn luyện; từ đó gợi ý **heuristic**: bắt đầu với learning rate lớn, nếu phân kỳ thì thử lại với learning rate nhỏ hơn 3 lần, lặp lại tới khi không còn phân kỳ. Một chú thích của paper nói ngưỡng phân kỳ gần đúng bằng 2 lần trị riêng lớn nhất của ma trận Hessian trung bình của loss (không chứng minh chi tiết trong văn bản chính, chỉ là nhận xét).

Về kích thước mini-batch B: tăng B giúp tận dụng tích ma trận-ma trận (nhanh hơn tích ma trận-vector, paper nói thường được khoảng 2x trên thực tế), nhưng giảm số lần cập nhật trên mỗi epoch, nên có ”điểm ngọt” (sweet spot) trung gian. Paper cũng nhắc: hướng gradient (ngay cả gradient thật trên toàn bộ tập) chỉ là hướng giảm dốc nhất **cục bộ**, không nhất thiết là hướng tốt khi bước đi xa, vì tiêu chí huấn luyện không phải hàm bậc hai (quadratic) theo tham số; do đó không nên tốn nhiều tính toán để ước lượng gradient cực chính xác, mà nên cập nhật thường xuyên hơn.

3.2 Tính gradient và vi phân tự động (Mục 2.2)

Gradient có thể tính tay hoặc bằng vi phân tự động (automatic differentiation). Paper khuyến nghị cấu trúc tính toán thành **flow graph** có hướng không chu trình (DAG): mỗi node có một phép toán cơ bản (cộng, nhân, non-linearity...), và gradient được tính ngược theo quy tắc chuỗi: $g_a = \sum_s g_s \frac{\partial o_s}{\partial o_a}$ với tổng trên các node kế tiếp trực tiếp s của a. Paper nhấn mạnh lợi ích của việc làm điều này **symbolic** (như thư viện Theano, so với Torch/Lush chỉ làm numeric theo kiểu forward/backward method trên object): cho phép tính đạo hàm bậc hai dễ dàng, định nghĩa tiêu chí huấn luyện liên quan tới gradient (ví dụ CAE dùng norm của Jacobian), và tính tích ma trận Hessian với một vector mà không cần dựng toàn bộ ma trận Hessian (R-operator, Pearlmutter 1994).

3.3 Danh sách siêu tham số (Mục 3.1, 3.2)

Nhóm tối ưu hoá:

- **Learning rate khởi tạo** ϵ_0: paper gọi đây là "thường là siêu tham số quan trọng nhất", nên luôn tune (ít nhất tới hệ số 2). Giá trị điển hình với input đã chuẩn hoá: trong khoảng $(10^{-6}, 1)$; mặc định 0.01 thường ổn cho MLP chuẩn nhưng "sẽ là dại dột nếu chỉ dựa vào giá trị mặc định này".
- **Lịch learning rate**: mặc định hằng số ($\tau \to \infty$). Ví dụ lịch $O(1/t)$ (công thức (2)):

$$\epsilon_t = \frac{\epsilon_0 \tau}{\max(t, \tau)}$$

  giữ learning rate hằng trong τ bước đầu rồi giảm dần $O(1/t^\alpha)$; lý thuyết cổ điển (trường hợp lồi) gợi ý $\alpha = 1$, nhưng paper dẫn Bach & Moulines (2011) cho thấy $\alpha < 1$ có thể tốt hơn ở trường hợp không lồi, đặc biệt khi dùng kỹ thuật trung bình hoá gradient (momentum). Cũng có cách thích nghi: giữ ϵ_t hằng tới khi tiêu chí huấn luyện ngừng giảm đáng kể (theo ngưỡng cải thiện tương đối), hoặc dùng heuristic của Bottou (2013): định kỳ thử vài learning rate trên một tập con nhỏ cố định, giữ giá trị tốt nhất.
- **Kích thước mini-batch** B: thường 1 đến vài trăm, ví dụ $B = 32$ là giá trị mặc định tốt. Paper nói ảnh hưởng của B chủ yếu là **tính toán** (B lớn tính nhanh hơn nhưng cần nhiều ví dụ hơn để đạt cùng lỗi), nên có thể tối ưu B riêng (so sánh đường cong lỗi theo thời gian tính toán) sau khi đã chọn các siêu tham số khác (trừ momentum).
- **Số bước huấn luyện** T: tối ưu gần như miễn phí bằng **early stopping**: theo dõi lỗi trên validation set, dừng khi không còn cải thiện. Paper mô tả heuristic "patience" (độ kiên nhẫn) từ Deep Learning Tutorials: khởi tạo patience = 10.000 ví dụ (trong MLP tutorial); mỗi khi tìm được điểm cực tiểu validation mới ở bước t, cập nhật $\hat{T} \leftarrow t$ và tăng patience (cộng hoặc nhân với hằng số) để tiếp tục huấn luyện thêm trước khi chấp nhận dừng.
- **Momentum** β: làm trơn gradient ngẫu nhiên theo thời gian bằng trung bình động: $\bar{g} \leftarrow (1-\beta)\bar{g} + \beta g$. **Lưu ý quan trọng về quy ước**: trong công thức này của paper, $\beta = 1$ nghĩa là **không có momentum** (dùng thẳng gradient tức thời), còn β nhỏ hơn 1 nghĩa là trung bình hoá nhiều hơn (nhiều momentum hơn). Đây là quy ước **ngược** với β_1 trong Adam (opt 06), nơi β_1 gần 1 mới là nhiều momentum; người đọc cần phân biệt rõ để tránh nhầm khi đối chiếu hai bài.
- **Siêu tham số tối ưu hoá theo từng layer**: có thể dùng learning rate khác nhau cho từng lớp (đặc biệt tự nhiên khi pretraining từng lớp riêng rẽ) hoặc cho từng loại tham số (trọng số vs bias).

Nhóm model/criterion:

- **Số unit ẩn** n_h: paper tổng kết hai quan sát thực nghiệm (Larochelle et al., 2009): các lớp cùng kích thước thường tốt hơn hoặc bằng kiến trúc hình chóp tăng/giảm dần; một lớp ẩn đầu tiên "overcomplete" (nhiều unit hơn số chiều input) thường tốt hơn "undercomplete"; khi có pretraining không giám sát, n_h tối ưu thường lớn hơn nhiều (từ hàng trăm lên hàng nghìn unit).
- **Hệ số weight decay** λ: L2 thêm $\lambda \sum_i \theta_i^2$ (tương đương Gaussian prior, phương sai $\sigma^2 = 1/(2\lambda)$), L1 thêm $\lambda \sum_i |\theta_i|$ (tương đương Laplace prior, scale $s = 1/\lambda$). Vì tiêu chí thật là tổng trên toàn bộ T ví dụ còn mỗi bước SGD chỉ thấy một mini-batch, paper khuyến nghị **nhân thêm hệ số** B/T vào gradient của phần regularizer mỗi lần cập nhật mini-batch, để giữ ước lượng không chệch. L1 tạo sparsity tham số (gần 0 nhưng không chính xác bằng 0 với SGD), có thể dùng như một dạng chọn đặc trưng; có thể chỉ phạt trọng số, không phạt bias, để giữ khả năng hội tụ về bộ dự đoán hằng số tối ưu khi regularization mạnh.
- **Hệ số sparsity hoạt hoá** α: phạt activation của unit ẩn (khác bản chất với L1 trên trọng số vì phụ thuộc phân phối dữ liệu), ví dụ phạt KL-divergence giữa trung bình activation $\bar{h}_j$ và mục tiêu ρ (paper dẫn $\rho = 0.05$ theo Hinton, 2010).
- **Non-linearity**: sigmoid, tanh, rectifier $\max(0, a)$, hard-tanh. Paper nói sigmoid ở lớp ẩn trên cùng của mạng sâu giám sát (không pretraining) gây khó khăn tối ưu (Glorot & Bengio, 2010); rectifier tốt cho

hidden units (mặc dù trực giác ban đầu lo ngại việc saturate ở 0 chặn gradient) nhưng **không nên dùng cho output units** vì khi saturate thì không có gradient nào lan truyền để sửa lỗi.

- **Hệ số scale khởi tạo trọng số**: bias có thể khởi tạo 0; trọng số cần ngẫu nhiên để phá vỡ đối xứng giữa các unit cùng lớp. Công thức Glorot & Bengio (2010) được paper trích dẫn trực tiếp: lấy mẫu Uniform$(-r, r)$ với

$$r = \sqrt{\frac{6}{\text{fan-in} + \text{fan-out}}} \quad \text{(đơn vị tanh)}, \qquad r = 4\sqrt{\frac{6}{\text{fan-in} + \text{fan-out}}} \quad \text{(đơn vị sigmoid)}$$

  Với RBM, paper nói nên dùng Gaussian trung tâm 0, độ lệch chuẩn nhỏ (0.1 hay 0.01).

Tự tính minh hoạ công thức Glorot: giả sử một lớp fully-connected có fan-in = 100, fan-out = 50, dùng đơn vị tanh. $r = \sqrt{6/(100+50)} = \sqrt{6/150} = \sqrt{0.04} = 0.2$. Vậy trọng số được lấy mẫu đều trong $[-0.2,\ 0.2]$ (tự tính, không phải số paper đưa sẵn, chỉ là ví dụ áp dụng công thức).

- **Random seed**: ảnh hưởng thường nhỏ; có thể lấy trung bình 5-10 seed cho cấu hình tốt nhất để ”vét” thêm hiệu năng, hoặc dùng model averaging (giống Bagging).
- **Tiền xử lý**: chuẩn hoá (trừ trung bình, chia độ lệch chuẩn), PCA (giảm chiều, thêm siêu tham số số thành phần giữ lại), uniformization (biến đổi theo hàm phân phối tích luỹ ước lượng, Mesnil et al., 2011).

3.4 Chiến lược tìm kiếm siêu tham số (Mục 3.3, 3.4)

- **Nguyên tắc chung**: với siêu tham số số (numerical), nên dò theo **thang log** (vì ảnh hưởng tương đối, không phải tuyệt đối, mới có ý nghĩa, ví dụ 0.01 so với 0.011 khác rất ít nhưng 0.001 so với 0.002 khác nhiều); kiểm tra nếu giá trị tốt nhất nằm ở biên khoảng tìm kiếm thì phải mở rộng khoảng; có thể dùng ước lượng rẻ (ít dữ liệu, ít epoch) để sàng lọc trước khi đánh giá đầy đủ.
- **Coordinate descent / multi-resolution search**: khi chỉ có một máy, đổi từng siêu tham số một lần (ưu tiên tinh chỉnh learning rate thường xuyên hơn); bắt đầu tìm với bước nhảy lớn/khoảng rộng, sau đó tinh chỉnh quanh điểm tốt tìm được.
- **Grid search**: liệt kê K giá trị log-đều cho mỗi siêu tham số rồi thử mọi tổ hợp; dễ song song hoá nhưng **bùng nổ tổ hợp theo cấp số nhân**: paper ví dụ 5 siêu tham số, mỗi cái 6 giá trị, ra $6^5 = 7776$ tổ hợp (tự tính kiểm tra: $6 \times 6 \times 6 \times 6 \times 6 = 7776$, khớp số paper nêu).
- **Random search** (Bergstra & Bengio, 2012): thay lưới đều bằng lấy mẫu ngẫu nhiên (thường log-uniform) độc lập cho mỗi siêu tham số; hiệu quả hơn grid search nhiều lần khi số siêu tham số vượt 2-3, vì không lặp lại cùng giá trị của một siêu tham số trong nhiều tổ hợp của các siêu tham số khác; vẫn song song hoá tốt, và có thể thêm trial mới mà không ”lãng phí” các trial cũ (khác grid search, nơi kết hợp hai lưới không tương thích thì khó).
- **Tìm kiếm tham lam theo layer** (Mục 3.3.4, Algorithm 1): khi pretraining từng lớp, chọn siêu tham số cho lớp 1 (đánh giá bằng một bộ phân loại đơn giản trên biểu diễn học được), giữ K cấu hình tốt nhất, dùng làm điểm khởi đầu để chọn siêu tham số lớp 2, lặp lại; sau đó tìm siêu tham số cho giai đoạn fine-tuning có giám sát.

3.5 Debug bằng gradient checking (Mục 4.1)

Xấp xỉ sai phân hữu hạn bậc một: $\frac{\partial f(x)}{\partial x} \approx \frac{f(x+\epsilon)-f(x)}{\epsilon} + o(\epsilon)$. Paper nhấn mạnh công thức bậc hai (tính đối xứng quanh x), chính xác hơn với cùng chi phí gấp đôi:

$$\frac{\partial f(x)}{\partial x} \approx \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon} + o(\epsilon^2)$$

vì các số hạng bậc hai trong khai triển Taylor của $f(x+\epsilon)$ và $f(x-\epsilon)$ triệt tiêu nhau. Paper dùng $\epsilon = 10^{-4}$ trong thực hành (đủ nhỏ để phát hiện lỗi, nhưng không quá nhỏ vì sai số số học (finite precision) có thể làm sai lệch tăng lên nếu ϵ quá nhỏ). Sau khi gradient đúng, một sanity check khác: kiểm tra có thể **overfit một tập**

huấn luyện nhỏ bằng một phương pháp bậc hai mạnh như L-BFGS, để tách lỗi code/hyperparameter SGD khỏi lỗi kiến trúc/capacity.

Tự tính minh hoạ sai phân hữu hạn: lấy $f(x) = x^3$, tại $x = 2$, đạo hàm thật $f'(2) = 3 \cdot 2^2 = 12$. Với $\epsilon = 0.01$: xấp xỉ bậc một $\frac{f(2.01)-f(2)}{0.01} = \frac{8{,}120601-8}{0.01} = 12,0601$ (sai số $\approx 0,0601$); xấp xỉ bậc hai (trung tâm) $\frac{f(2.01)-f(1.99)}{0.02} = \frac{8{,}120601-7{,}880599}{0.02} = 12,0001$ (sai số $\approx 0,0001$), nhỏ hơn hẳn, minh hoạ đúng tính chất $o(\epsilon)$ so với $o(\epsilon^2)$ mà paper nói (tự tính).

4. Thí nghiệm và kết quả chính

Đây là một bài hướng dẫn thực hành (practical guide), **không có một bộ thí nghiệm định lượng trung tâm với bảng số liệu so sánh** như paper thực nghiệm thông thường; paper không nêu benchmark số liệu cụ thể cho các khuyến nghị (ví dụ không có bảng ”learning rate X cho độ chính xác Y%”). Các ”kết quả” được trình bày dưới dạng quan sát thực nghiệm tổng hợp từ nhiều nghiên cứu trước (của tác giả và người khác), ví dụ:

- Quan sát từ Larochelle et al. (2009) (nghiên cứu so sánh lớn, paper trích dẫn nhưng không lặp lại số liệu chi tiết trong văn bản): các lớp cùng kích thước thường tốt hơn kiến trúc hình chóp.
- Quan sát GPU: trên máy 512 core, paper nói tác giả ”thường đạt tốc độ 4x đến 40x cho mạng neural lớn” (số liệu do paper tự báo cáo như trải nghiệm thực hành, không có bảng chi tiết theo kiến trúc/kích thước).
- Quan sát về sampled reconstruction (Dauphin et al., 2011) cho auto-encoder với input sparse: tốc độ tăng tuyến tính theo độ sparsity, lỗi trung bình vẫn tối ưu tốt như tính đầy đủ (paper dẫn lại kết luận của Dauphin et al., không tự làm thí nghiệm trong chương này).
- Quan sát về trực quan hoá quỹ đạo học (Erhan et al., 2010b): các lần khởi tạo khác nhau luôn kết thúc ở cực tiểu cục bộ khác nhau (trong không gian hàm, qua t-SNE/Isomap); cực tiểu đạt được khi có pretraining không giám sát ở xa (trong không gian hàm) so với cực tiểu đạt từ khởi tạo ngẫu nhiên thuần, và tổng quát hoá tốt hơn.

Vì đây là một ”survey thực hành” tổng hợp, mục này chủ yếu liệt kê các quan sát định tính mà paper coi là đủ chắc để đưa vào khuyến nghị, không phải một thí nghiệm mới của chính paper.

5. Phân tích / ablation

Không có ablation theo nghĩa thí nghiệm có đối chứng trong bài. Thay vào đó, paper tổ chức các ”phân tích” ở dạng **phân loại trường hợp / trade-off**, có thể xem như tương đương ablation về mặt tư duy:

- **Trade-off mini-batch size**: B nhỏ giúp nhiều update/epoch hơn (khám phá nhanh hơn) và có hiệu ứng điều chuẩn (regularization) từ nhiễu gradient, nhưng tính toán kém hiệu quả hơn trên phần cứng song song; B lớn ngược lại.
- **Trade-off L1 vs L2**: L2 phạt giá trị lớn mạnh hơn (tương đương SVM khi số unit ẩn tiến tới vô hạn, theo paper); L1 đẩy tham số về đúng 0 (tương đương boosting ở giới hạn đó), hữu ích như công cụ chọn đặc trưng nhưng SGD không tạo ra số 0 chính xác.
- **Trade-off non-linearity vs depth** (Mục 6.1): sigmoid ở lớp ẩn trên cùng gây plateau khi không có pretraining; tanh (đối xứng) không gặp vấn đề này; softsign ($s(a) = a/(1 + |a|)$, không có đuôi mũ) còn tốt hơn; rectifier (không đối xứng, ”hard-limiting”) lại hoạt động tốt một cách bất ngờ so với trực giác ban đầu, paper đưa giả thuyết (không chứng minh chặt) rằng gradient vẫn lan truyền tốt qua các ”đường” được chọn (path) khi chỉ một phần unit bị saturate, có thể làm bài toán gán công (credit assignment) sắc nét hơn và điều kiện số của Hessian tốt hơn.
- **Trade-off tốc độ hội tụ sớm vs muộn giữa SGD và phương pháp bậc hai** (Mục 6.2): SGD hội tụ nhanh ở pha đầu nhưng chậm hơn ở pha cuối so với phương pháp bậc hai; do đó một ”mẹo” thực hành paper nêu là bắt đầu bằng một vài epoch SGD rồi mới chuyển sang phương pháp bậc hai (nếu dùng).

- **Grid search vs random search**: grid search dễ phân tích định tính (cố định mọi siêu tham số khác khi nhìn một chiều) nhưng kém hiệu quả hơn khi số siêu tham số lớn; random search hội tụ nhanh hơn theo số trial nhưng cho hình ảnh ”nhiễu” hơn khi cố định các siêu tham số khác (vì chúng dao động ngẫu nhiên giữa các trial).

6. Giới hạn

Paper tự nêu nhiều giới hạn/hạn chế, không phải suy luận của người ghi chú:

- Phần lớn khuyến nghị ”come out of a living practice... but very often have not been formally validated” (lời mở đầu); tác giả chủ động cảnh báo người đọc nên **thử và thách thức** chứ không coi là chân lý.
- Nhiều hiện tượng chỉ có giải thích **giả thuyết, chưa chứng minh chặt**: ví dụ vì sao rectifier hoạt động tốt dù saturate (Mục 6.1, ”a plausible partial answer”), vì sao mạng sâu khó huấn luyện hơn (Mục 6.1, ”this is clearly still an open question”).
- Phương pháp bậc hai và natural gradient được paper mô tả là ”appealing conceptually... but have not yet become a standard... and need to be validated” (Mục 6.2); nghĩa là paper không khẳng định các phương pháp này đã sẵn sàng thay SGD.
- Phần kiến trúc convolutional bị paper chủ động loại ra, nhường cho ”another chapter (LeCun, 2013)”; nên các khuyến nghị về khởi tạo/kiến trúc trong bài này chủ yếu nhắm tới mạng fully-connected.
- Paper viết trước khi BatchNorm (2015, Ioffe & Szegedy, nếu đã có trong thư mục này là opt 38), LayerNorm, Adam (2015), và kiến trúc Transformer tồn tại; do đó nhiều khuyến nghị (khởi tạo Glorot cho sigmoid/tanh, learning rate mặc định 0.01, lịch $O(1/t)$) được thiết kế cho MLP/mạng nông-trung bình thời 2012, **người ghi chú nhận xét** (không phải paper nói) rằng không nên áp dụng trực tiếp cho Transformer/LLM hiện đại mà không kiểm chứng lại, vì các kiến trúc hiện đại dùng chuẩn hoá (LayerNorm/RMSNorm), residual connection, và optimizer thích nghi (Adam/AdamW) làm thay đổi đáng kể động lực huấn luyện mà paper 2012 chưa biết tới.
- Quy ước β cho momentum (trong đó $\beta = 1$ là không có momentum) **khác quy ước phổ biến hiện nay** (ví dụ β_1 trong Adam, nơi gần 1 là nhiều momentum); đây không phải lỗi của paper (quy ước nội tại paper nhất quán), nhưng là điểm dễ gây nhiễm chéo khi đọc cùng tài liệu hiện đại, cần người đọc tự lưu ý.

7. Liên hệ với các bài khác

- **optimization content/44_2018_batchnorm_helps_optimization.md** (đã xác nhận tồn tại bằng `ls`): bài 46 nêu câu hỏi mở ”vì sao mạng sâu khó huấn luyện hơn” (Mục 6.1) và liệt kê các yếu tố giả thuyết (khởi tạo, non-linearity, tín hiệu huấn luyện cục bộ); bài opt 44 trả lời một phần câu hỏi này bằng phân tích cụ thể vai trò của BatchNorm trong việc làm loss landscape ”mượt” hơn, ra đời đúng để giải quyết các khó khăn mà bài 46 chỉ mới đặt câu hỏi.
- **optimization content/45_2024_deconstructing_llm_optimizer.md** (đã xác nhận tồn tại): bài 46 Mục 6.2 nêu ý tưởng learning rate thích nghi theo layer/parameter như một hướng mở (dẫn Bordes et al. 2009, LeCun et al. 1998a); bài opt 45 (2024) chính là một nghiên cứu hiện đại thực hiện chính xác ý tưởng đó (Adalayer: thích nghi theo layer) và trả lời câu hỏi ”chi tiết (granularity) nào thực sự cần thiết”, 12 năm sau khi bài 46 đặt vấn đề.
- **optimization content/18_2016_sgdr.md** (đã xác nhận tồn tại): nối tiếp trực tiếp ý tưởng ”lịch learning rate” của bài 46 (Mục 3.1.1, công thức (2) dạng $O(1/t)$); SGDR thay bằng lịch cosine có ”restart”, là một phát triển cụ thể hơn nhiều so với heuristic chung mà bài 46 mô tả.
- **optimization content/10_2019_lamb.md** (đã xác nhận tồn tại): liên quan tới chủ đề mini-batch size ở Mục 3.1.1 của bài 46 (ảnh hưởng chủ yếu là tính toán, có thể tối ưu tách biệt); LAMB là một optimizer thiết kế riêng để mini-batch rất lớn vẫn hội tụ tốt, vấn đề mà bài 46 (2012) chưa đặt ra ở quy mô đó.

(Lưu ý: đã `ls` thư mục ”optimization content/” tại thời điểm viết bài này và chỉ thấy 4 file: 10, 18, 44, 45 (các

số khác trong opt 01-43, 47-51 chưa tồn tại do đang được viết song song bởi các agent khác), nên không trỏ tới chúng.)

8. Ứng dụng cho dự án (suy luận)

Đây là suy luận của người ghi chú. Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang, GPU $\leq$ 8 GB, batch size 1).

- **Quy trình debug còn hữu dụng gần như nguyên vẹn**: gradient checking bằng sai phân hữu hạn trung tâm (Mục 3.5), sanity check "overfit một tập nhỏ trước" (ví dụ vài chục trang OCR), theo dõi riêng train loss và validation loss để phân biệt lỗi tối ưu hoá với overfitting, đều là thực hành tốt áp dụng được trực tiếp khi debug pipeline fine-tune/distill LightOnOCR, không phụ thuộc kiến trúc cụ thể.
- **Early stopping kiểu "patience"** (Mục 3.1.1) là một cách rẻ để tránh overfit khi fine-tune trên một tập dữ liệu OCR nhỏ hơn nhiều so với pretraining gốc; vì GPU hạn chế nên mỗi lần đánh giá validation cũng tốn thời gian, có thể áp dụng patience theo số ví dụ (không phải epoch) như paper khuyến nghị, phù hợp khi dữ liệu OCR có kích thước không chuẩn.
- **Khởi tạo Glorot/Xavier (Mục 3.3) không áp dụng trực tiếp cho các layer đã pretrain** của LightOnOCR (chỉ hữu ích nếu thêm layer mới, ví dụ một adapter/LoRA head mới cho OCR); với các layer GatedDeltaNet/attention đã huấn luyện sẵn, nên dùng trọng số pretrained, không khởi tạo lại.
- **Mini-batch size và learning rate**: vì dự án chạy batch size 1 (hạn chế GPU), khuyến nghị của paper (Mục 3.1.1) rằng B chủ yếu ảnh hưởng tốc độ tính toán, không nhất thiết ảnh hưởng chất lượng cuối (nếu đủ update), gợi ý rằng có thể bù batch size nhỏ bằng gradient accumulation hoặc huấn luyện nhiều bước hơn, miễn chấp nhận thời gian huấn luyện lâu hơn; đây là suy luận ngoại suy, paper 2012 không nói về gradient accumulation (kỹ thuật phổ biến hơn trong kỷ nguyên GPU nhớ hạn chế sau này).
- **Random search cho siêu tham số** (Mục 3.4) thực tế hơn grid search khi chỉ có 1 GPU và vài chục lần thử là khả thi; nên ưu tiên quét learning rate (và nếu dùng, hệ số LoRA rank/alpha) bằng log-uniform sampling thay vì lưới đều, theo đúng khuyến nghị của bài.
- **Hạn chế khi ngoại suy**: các khuyến nghị cụ thể về non-linearity, khởi tạo, momentum của bài 46 nhắm tới MLP/mạng thời 2012 không có LayerNorm/residual phức tạp như Transformer hiện đại; với LightOnOCR (Transformer hybrid, đã có pretraining ổn định), phần giá trị thực tế nhất của bài 46 cho dự án là **quy trình debug và tìm kiếm siêu tham số**, không phải các con số/mặc định cụ thể.

9. Thí nghiệm gợi ý

1. **Kiểm tra gradient checking trên một mô-đun tự viết**: nếu có tuỳ biến (ví dụ viết lại một gate của GatedDeltaNet hoặc một loss OCR tuỳ chỉnh) bằng PyTorch, so sánh gradient autograd với sai phân hữu hạn trung tâm (công thức Mục 3.5) ở $\epsilon = 10^{-4}$, trên vài tham số ngẫu nhiên. Đo: sai số tương đối. Kỳ vọng: sai số rất nhỏ (dưới 10^{-4} tương đối) nếu code đúng; sai số lớn/có dấu sai chỉ ra bug, đúng như paper mô tả.
2. **So sánh lịch learning rate hằng số vs $O(1/t)$ (công thức (2))**: trên một mô hình nhỏ (ví dụ fine-tune một adapter nhỏ của LightOnOCR trên vài trăm trang OCR), huấn luyện với ϵ_t hằng so với $\epsilon_t = \epsilon_0 \tau / \max(t, \tau)$ với vài giá trị τ. Đo validation loss cuối. Kỳ vọng: lịch giảm dần ổn định hơn ở giai đoạn cuối, đặc biệt nếu learning rate khởi tạo hơi lớn.
3. **Early stopping với patience**: cài thuật toán patience (Mục 3.1.1) khi fine-tune, với patience khởi tạo = N ví dụ, tăng theo hệ số cố định mỗi khi có validation loss tốt nhất mới. Đo: số bước huấn luyện thực tế trước khi dừng, so với huấn luyện cố định số bước. Kỳ vọng: patience dừng sớm hơn nếu model đã hội tụ, tiết kiệm thời gian GPU mà không mất nhiều chất lượng.
4. **Random search log-uniform cho learning rate**: lấy 10-20 giá trị learning rate ngẫu nhiên theo thang log trong một khoảng rộng (ví dụ $[10^{-5}, 10^{-2}]$), huấn luyện ngắn (vài trăm bước) một adapter nhỏ, vẽ validation loss theo learning rate. Đo: có thấy dạng đường cong U rõ không, giá trị tối ưu có nằm giữa khoảng không hay sát biên (nếu sát biên, mở rộng khoảng theo đúng khuyến nghị Mục 3.4).

5. **Trực quan hoá học theo "age" (số ví dụ đã thấy) thay vì epoch**: khi so sánh hai lần fine-tune với batch size hiệu dụng khác nhau (ví dụ nhờ gradient accumulation), vẽ lại train/validation loss theo số ví dụ đã xử lý (age = số update $\times$ batch size) thay vì theo epoch, như paper gợi ý ở Mục 4.2. Quan sát: liệu hai đường cong có thẳng hàng tốt hơn khi dùng "age" làm trục hoành, minh hoạ đúng lý do paper đưa ra khuyến nghị này.

10. Tóm tắt công thức / khái niệm

- **SGD mini-batch** (công thức (1)): $\theta^{(t)} \leftarrow \theta^{(t-1)} - \epsilon_t \frac{1}{B} \sum \frac{\partial L(z_{t'}, \theta)}{\partial \theta}$.
- **Lịch learning rate** $O(1/t)$ (công thức (2)): $\epsilon_t = \epsilon_0 \tau / \max(t, \tau)$.
- **Momentum (quy ước riêng của paper)**: $\bar{g} \leftarrow (1-\beta)\bar{g} + \beta g$; $\beta = 1$ là không momentum (ngược quy ước Adam).
- **Weight decay**: L2 $\lambda \sum_i \theta_i^2$ (Gaussian prior, $\sigma^2 = 1/(2\lambda)$); L1 $\lambda \sum_i |\theta_i|$ (Laplace prior, scale $1/\lambda$); nhân thêm B/T vào gradient của regularizer khi cập nhật theo mini-batch.
- **Khởi tạo Glorot/Xavier**: Uniform$(-r, r)$ với $r = \sqrt{6/(\text{fan-in} + \text{fan-out})}$ (tanh) hoặc $r = 4\sqrt{6/(\text{fan-in} + \text{fan-out})}$ (sigmoid).
- **Sai phân hữu hạn trung tâm (gradient checking)**: $\frac{\partial f(x)}{\partial x} \approx \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon} + o(\epsilon^2)$, chính xác hơn công thức một phía $o(\epsilon)$.
- **Số tổ hợp grid search**: với k siêu tham số, mỗi cái m giá trị, số tổ hợp là m^k (ví dụ $6^5 = 7776$).
- **Flow graph / quy tắc chuỗi cho gradient**: $g_a = \sum_s g_s \, \partial o_s / \partial o_a$, tổng trên các node kế tiếp trực tiếp s của node a.

11. Kết luận ngắn

Bài là một bản tổng hợp thực hành (không phải nghiên cứu thực nghiệm mới) về cách chọn, tìm kiếm, và debug siêu tham số khi huấn luyện mạng neural sâu bằng gradient, viết ở thời điểm 2012 ngay sau làn sóng pretraining không giám sát theo lớp. Các khuyến nghị cốt lõi, learning rate là siêu tham số quan trọng nhất và nên dò theo thang log, mini-batch chủ yếu ảnh hưởng tốc độ tính toán, khởi tạo trọng số cần theo fan-in/fan-out, gradient checking bằng sai phân hữu hạn trung tâm, random search hiệu quả hơn grid search khi nhiều siêu tham số, vẫn còn giá trị thực hành cho ngày nay, dù nhiều chi tiết cụ thể (non-linearity sigmoid/tanh, lịch $O(1/t)$) đã được các kỹ thuật sau này (LayerNorm, Adam, warmup+cosine, v.v.) thay thế hoặc bổ sung. Tác giả tự coi đây là tri thức "đang sống", cần tiếp tục được kiểm chứng và thách thức, không phải quy tắc cố định.

Nguồn gốc: https://arxiv.org/abs/1206.5533

Bài 47

A Disciplined Approach to Neural Network Hyper-Parameters: Part 1 - Learning Rate, Batch Size, Momentum, and Weight Decay (2018)

Tác giả: Leslie N. Smith (US Naval Research Laboratory, Washington DC, USA).
Venue / arXiv: arXiv:1803.09820 (v2, 24/04/2018); paper tự ghi là ”US Naval Research Laboratory Technical Report 5510-026”, không phải bài đăng hội nghị/tạp chí có peer review (paper không nêu venue hội nghị nào khác).
Nguồn: https://arxiv.org/abs/1803.09820
Chủ đề: quy trình thực hành (”checklist”) để chọn learning rate, batch size, momentum, và weight decay cho mạng neural, dựa trên việc đọc ”đầu mối” (clue) từ đường cong validation/test loss sớm trong huấn luyện, thay vì grid/random search đầy đủ; đây là **Part 1** của một báo cáo hai phần, Part 2 (không thuộc phạm vi bài này) dự kiến bàn kiến trúc, dữ liệu, và các dạng regularization khác.
Vị trí trong lộ trình: đọc sau opt 46 (Bengio, 2012, ”Practical recommendations...”, được chính paper này trích dẫn là công trình liên quan gần nhất) và opt 19 (Super-Convergence, Smith & Topin 2017, mà Mục 4.1 của bài này ”revisit” và mở rộng) để thấy paper này tổng hợp và hệ thống hoá lại. Đọc cùng nhóm với opt 18 (SGDR) vì cùng chủ đề lịch learning rate theo chu kỳ nhưng với công thức khác (cosine restart so với 1cycle tuyến tính ở đây).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục ”Ứng dụng cho dự án” là suy luận. Bài có 2 phần tiếp theo (Part 2, Part 3) do cùng tác giả công bố sau; bài này chỉ ghi nội dung của **Part 1** (học tốc độ, batch size, momentum, weight decay), không lấy nội dung Part 2/3 dù paper gốc có nhắc sơ lược kế hoạch Part 2 ở Mục 6.

1. Bối cảnh

Chọn siêu tham số cho mạng neural sâu, theo paper, ”vẫn là một black art” cần nhiều năm kinh nghiệm; grid search hay random search (Bergstra & Bengio, 2012, tức opt 32 nếu đã viết) trên toàn không gian siêu tham số rất tốn tính toán. Thực tế nhiều người dùng trực tiếp file siêu tham số có sẵn trong ”model zoo” của một kiến trúc chuẩn (ví dụ ResNet), nhưng các giá trị đó thường không tối ưu cho dữ liệu của riêng họ.

Paper xây dựng trên các công trình trước của chính tác giả: cyclical learning rate (CLR) và LR range test (Smith, 2015; Smith, 2017), và super-convergence (Smith & Topin, 2017, opt 19 nếu đã tồn tại) là mạng có thể huấn luyện rất nhanh với learning rate rất lớn trong một chu kỳ. Bài này đặt câu hỏi rộng hơn: làm sao chọn **đồng thời và nhanh** learning rate, batch size, momentum, weight decay, tận dụng đúng các ”đầu mối” quan sát được trong vài epoch đầu, để không cần chạy đầy đủ grid/random search.

2. Ý tưởng chính

Ý tưởng trung tâm, đặt tên là ”the unreasonable effectiveness of validation/test loss” (Mục 3): **đường cong test/validation loss sớm trong huấn luyện mang nhiều thông tin về underfitting/overfitting hơn test accuracy hay training loss**, nên có thể dùng vài epoch chạy ngắn để chẩn đoán và chỉnh siêu tham số, thay vì phải chạy xong toàn bộ rồi so sánh kết quả cuối (Remark 1).

Từ đó, paper đề xuất một bộ quy tắc cho 4 siêu tham số:

- **Learning rate**: dùng **LR range test** để tìm learning rate lớn nhất khả dụng, sau đó dùng chính sách **1cycle** (một biến thể của cyclical learning rate do bài này đặt tên, kế tục ý tưởng super-convergence).
- **Batch size**: ngược với khuyến nghị cũ (batch nhỏ để có hiệu ứng điều chuẩn), bài này khuyến nghị dùng batch size **lớn nhất mà bộ nhớ GPU cho phép**, kết hợp với 1cycle, để có thể dùng learning rate lớn hơn.
- **Momentum**: khác learning rate, **momentum range test không hữu ích** để tìm giá trị tối ưu; nhưng dùng **cyclical momentum giảm dần khi learning rate tăng** (kết hợp với 1cycle) cho kết quả tốt và ổn định hơn momentum hằng số.
- **Weight decay**: khác learning rate/momentum, giá trị tối ưu của weight decay nên **giữ hằng số** suốt quá trình huấn luyện (không cyclical); nhưng giá trị tối ưu đó **phụ thuộc chặt vào learning rate** đang dùng (learning rate lớn hơn tạo thêm regularization, nên cần weight decay nhỏ hơn).

Nguyên lý tổng hợp (Remark 3, Remark 6): **lượng regularization tổng thể phải được cân bằng cho từng tập dữ liệu/kiến trúc**; learning rate lớn, batch nhỏ, weight decay, dropout đều là các ”nguồn” regularization khác nhau, tăng một nguồn thì phải giảm nguồn khác.

3. Phương pháp chi tiết

3.1 Đọc đầu mối underfitting/overfitting từ test loss (Mục 3)

Underfitting: test loss **tiếp tục giảm liên tục** (không có điểm phẳng) trong một run ngắn (Hình 3: mạng 3 lớp trên Cifar-10 với learning rate 0.001 vẫn giảm, trong khi learning rate 0.004 giảm nhanh rồi phẳng ra, chỉ báo config tốt hơn và đúng là đạt accuracy cuối cao hơn; tương tự trên Imagenet, resnet-50 và inception-resnet-v2 ở 100.000 iteration ($\approx$ 3 epoch) đều còn giảm nhưng inception-resnet-v2 giảm nhiều hơn và gần phẳng hơn, chỉ báo ít underfit hơn).

Overfitting: thông thường test loss **tăng trở lại** sau một điểm cực tiểu (Hình 2, mô hình lý thuyết underfitting/overfitting kiểu U/chữ V). Nhưng paper cảnh báo thực tế phức tạp hơn: ở Hình 1, learning rate **nhỏ** (0.01-0.04) cũng cho thấy dấu hiệu overfitting (test loss tăng) trong khi learning rate cao hơn vẫn tiếp tục giảm như đang underfit, ngụ ý learning rate quá nhỏ cũng có thể gây ”giống overfitting”. Hình 4 cho ví dụ cụ thể: mạng 3 lớp Cifar-10, weight decay 10^{-4} đạt cực tiểu ở LR$\approx$ 0.002 rồi tăng (overfit), còn weight decay 4×10^{-3} ổn định trên khoảng LR rộng hơn và đạt loss thấp hơn, tức WD lớn hơn ở đây tốt hơn; paper cũng lưu ý: tăng loss đột ngột ở LR rất lớn (gần 0.005 hay 0.008 trong hình) là **mất ổn định số học do LR quá lớn**, không phải overfitting, cần phân biệt hai hiện tượng.

3.2 LR range test và chính sách 1cycle (Mục 4.1)

LR range test: chạy một lần huấn luyện (pre-training run) với learning rate tăng **tuyến tính** từ một giá trị nhỏ tới một giá trị lớn qua toàn bộ run. Khi LR còn nhỏ, test loss giảm (hội tụ); khi LR đủ lớn, test loss bắt đầu tăng trở lại (và accuracy giảm). Giá trị LR tại điểm đổi hướng đó là **cận trên (max bound)** dùng được cho cyclical learning rate; nếu dùng learning rate hằng số thì phải chọn nhỏ hơn giá trị này. Cách chọn cận dưới (min bound): (1) nhỏ hơn cận trên 3-4 lần; (2) nhỏ hơn 10-20 lần nếu chỉ dùng một chu kỳ; (3) chạy thử vài trăm iteration với vài LR nhỏ, chọn LR lớn nhất mà việc hội tụ bắt đầu mà không có dấu hiệu overfitting.

Super-convergence (Smith & Topin, 2017, dẫn lại và mở rộng): với resnet sâu trên Cifar-10/100, test loss/accuracy gần như không đổi suốt LR range test dù LR rất lớn, cho phép huấn luyện rất nhanh với một chu kỳ LR rất lớn. Ví dụ paper nêu lại (Hình 5a): huấn luyện xong trong 10.000 iteration với LR lên tới 3.0, so với 80.000 iteration cần thiết nếu dùng LR hằng số 0.1 (số liệu của paper, không phải tự paper này đo lại từ đầu mà kế tục Smith & Topin 2017).

Chính sách 1cycle (đóng góp đặt tên của chính bài này): dùng **một chu kỳ** nhỏ hơn tổng số iteration/epoch huấn luyện; sau khi LR tăng rồi giảm trở lại mức ban đầu trong chu kỳ đó, cho phép LR **giảm tiếp vài bậc độ lớn** dưới mức ban đầu trong phần iteration còn lại. Paper nhận xét 1cycle giống sự kết hợp giữa curriculum

learning (Bengio et al., 2009) và simulated annealing (Aarts & Korst, 1988). Hình 5b: weight decay $\leq 10^{-4}$ cho phép dùng LR lớn tới 3.0, còn weight decay 10^{-3} thì không, minh hoạ nguyên lý cân bằng regularization.

3.3 Batch size (Mục 4.2)

Vấn đề đo lường: so sánh batch size theo **số epoch cố định** thiệt cho batch lớn (bỏ qua hiệu quả tính toán của batch lớn); so theo **số iteration cố định** lại lợi quá nhiều cho batch lớn. Paper chọn so sánh theo **thời gian thực thi gần như cố định** (26 phút trên máy IBM Power8 của họ, xác định từ một grid search nhỏ tìm số epoch tối thiểu để TBS=128 đạt accuracy tối ưu, rồi suy ra số epoch tương ứng cho các TBS khác để giữ thời gian chạy tương đương).

3.4 Cyclical momentum (Mục 4.3)

Nhắc lại công thức SGD với momentum (công thức (1)-(3) của paper):

$$\theta_{iter+1} = \theta_{iter} - \epsilon\, \delta L(F(x,\theta),\theta) \quad \text{(không momentum, công thức 1)}$$

$$v_{iter+1} = \alpha v_{iter} - \epsilon\, \delta L(F(x,\theta),\theta), \qquad \theta_{iter+1} = \theta_{iter} + v \quad \text{(công thức 2-3, có momentum)}$$

với ϵ là learning rate và α là hệ số momentum (ở đây dùng quy ước giống Adam/opt 06: α gần 1 là nhiều momentum, **khác quy ước của Bengio 2012/opt 46** nơi $\beta = 1$ nghĩa là không momentum). Paper chỉ ra velocity v là trung bình động của gradient, nên momentum tác động lên update với **cùng cấp độ lớn** như learning rate.

Khác với learning rate, **momentum range test không hữu ích**: test loss tiếp tục giảm (accuracy tăng) khi momentum tăng từ 0.7 tới gần 1, không có điểm cực tiểu rõ rệt (Hình 7b). Nhưng momentum quá lớn (0.99) gây dấu hiệu overfit rồi phân kỳ, quan sát được sớm (Hình 7a). Giải pháp đề xuất (Remark 5): dùng **cyclical momentum (CM) giảm dần khi LR tăng** (ví dụ 0.95 xuống 0.85), kết hợp 1cycle; nếu dùng LR hằng số, dùng momentum hằng lớn (0.9-0.99) thay vì CM.

3.5 Weight decay (Mục 4.4)

Khác learning rate/momentum, weight decay tối ưu **nên giữ hằng số** (cyclical weight decay không hữu ích theo thực nghiệm của paper, dù chưa test hết các dạng regularization khác). Quy trình: nếu chưa có ý tưởng, thử $10^{-3}, 10^{-4}, 10^{-5}, 0$; dữ liệu/kiến trúc nhỏ cần weight decay lớn hơn, dữ liệu/kiến trúc lớn/sâu cần weight decay nhỏ hơn (giả thuyết của paper: dữ liệu phức tạp đã tự mang tính "regularize"). Thủ thuật tìm giá trị: **chia đôi theo số mũ (bisect the exponent), không chia đôi theo giá trị**, ví dụ giữa 10^{-4} và 10^{-3}, điểm giữa là $10^{-3.5} = 3,16 \times 10^{-4}$.

Tự tính kiểm tra công thức bisect exponent: $10^{-3.5} = 10^{-4} \times 10^{0.5} = 10^{-4} \times \sqrt{10} \approx 10^{-4} \times 3,1623 = 3,1623 \times 10^{-4}$, khớp với số $3,16 \times 10^{-4}$ paper nêu (tự tính). Tương tự, điểm giữa giữa 10^{-3} và 10^{-2} là $10^{-2.5} \approx 3,16 \times 10^{-3}$ (paper dùng đúng giá trị này trong Hình 9, 10). Một ví dụ khác paper dùng: $10^{-2.75} = 10^{-3} \times 10^{0.25} \approx 10^{-3} \times 1,778 = 1,78 \times 10^{-3}$, khớp với "$1,8 \times 10^{-3}$" paper nêu khi tinh chỉnh thêm giữa $3,16 \times 10^{-3}$ và 10^{-3} (tự tính).

3.6 Danh sách khuyến nghị tóm tắt (đầu Mục 5)

Paper gói toàn bộ phương pháp thành "checklist" 4 bước:

1. **LR**: chạy LR range test tới LR "lớn" (tuỳ kiến trúc, ví dụ 0.01 với mạng 3 lớp nông, 3.0 với resnet); dùng 1cycle với LR tối đa từ test đó, LR tối thiểu $\approx$ 1/10 LR tối đa làm điểm khởi đầu hợp lý.
2. **TBS (total batch size)**: dùng batch lớn nhất bộ nhớ GPU cho phép (nhân thêm số GPU nếu multi-GPU); nhớ rằng batch nhỏ thêm regularization, batch lớn thì ít hơn, cần cân bằng; thường lợi hơn khi dùng batch lớn + LR lớn hơn.

3. **Momentum**: chạy ngắn với momentum $\in \{0.99, 0.97, 0.95, 0.9\}$ để chọn giá trị tốt nhất; nếu dùng 1cycle, dùng cyclical momentum bắt đầu ở giá trị đó, giảm dần xuống 0.8-0.85 (giá trị tối thiểu ít ảnh hưởng).
4. **Weight decay**: grid search, thường chỉ cần đúng tới 1 chữ số có nghĩa; dữ liệu phức tạp hơn thì thử giá trị nhỏ hơn ($10^{-4}, 10^{-5}, 10^{-6}, 0$), kiến trúc nông hơn thì thử giá trị lớn hơn ($10^{-2}, 10^{-3}, 10^{-4}$).

4. Thí nghiệm và kết quả chính

Tất cả thí nghiệm dùng Cifar-10, Cifar-100, MNIST, hoặc Imagenet; phần cứng chính là cụm IBM Power8 (32 node tính toán, 20 core/node, 4 GPU Tesla P100/node, 255 GB/node) và một cụm phụ 64 node với 8 GPU Titan Black/node (Phụ lục A.1). Mỗi đường trong các hình là **trung bình của 4 lần chạy** với batch size và cách khởi tạo (msra, xavier) khác nhau đôi chút, để giảm phụ thuộc vào lựa chọn ngẫu nhiên đó.

Bảng 2 (kết quả chính, so sánh step learning rate policy truyền thống với 1cycle):

Dataset / kiến trúc	Chính sách	Epoch	Accuracy (%)
Cifar-10, wide resnet (32 lớp, x4 channel)	step	800	90,3 ± 1,0
Cifar-10, wide resnet	1cycle	100	91,9 ± 0,2
Cifar-10, densenet (40 lớp)	step	400	92,7 ± 0,2
Cifar-10, densenet	1cycle	150	92,8 ± 0,1
MNIST, LeNet	step/inv (mặc định Caffe)	85	99,03 ± 0,04
MNIST, LeNet	1cycle	12	99,25 ± 0,03
MNIST, LeNet	1cycle (LR rộng hơn)	85	99,35 ± 0,03
Cifar-100, resnet-56	step	400	61,0 ± 0,2
Cifar-100, resnet-56	1cycle	25	65,4 ± 0,2
Cifar-100, resnet-56	1cycle	100	69,0 ± 0,4

Tự tính tốc độ giảm số epoch cần thiết (lấy đúng số trong Bảng 2): wide resnet $800/100 = 8$ lần ít epoch hơn mà accuracy còn cao hơn (90,3% → 91,9%); densenet $400/150 \approx 2,67$ lần; MNIST/LeNet $85/12 \approx 7,08$ lần ít epoch hơn mà accuracy vẫn cao hơn (99,03% → 99,25%) (tự tính, các tỉ lệ này không phải số paper viết trực tiếp mà tính lại từ Bảng 2 của paper).

Imagenet (Mục 5.5, 1,24 triệu ảnh, 1000 lớp): với resnet-50, baseline theo Szegedy et al. (2017) (momentum 0,9, LR 0,045 giảm theo hàm mũ 0,94 mỗi 2 epoch, WD 10^{-4}) đạt 63,7% sau 130 epoch (chạy chưa hết do giới hạn thời gian, paper tự ngoại suy ”near 65%”); với 1cycle (LR 0,05 → 1,0 → 0,00005 trong 20 epoch, phải **giảm weight decay** xuống 3×10^{-6} đến 10^{-5} để chịu được LR lớn) đạt **67,6%** chỉ trong 20 epoch. Với inception-resnet-v2: baseline 100 epoch → 67,6% (ngoại suy 69-70%); 1cycle 20 epoch, WD giảm xuống 3×10^{-6} đến 10^{-6}, tốt nhất ở 3×10^{-6} → **74,0%**. Paper tự ghi chú (footnote) kết quả Imagenet chỉ là trung bình của **2 lần chạy** (do chi phí tính toán cao), nên độ tin cậy thống kê thấp hơn các thí nghiệm Cifar/MNIST (4 lần chạy).

5. Phân tích / ablation

5.1 Weight decay phụ thuộc vào có dùng LR range hay không (Hình 11)

Trên resnet-56/Cifar-10 (momentum hằng 0,95, TBS=1030): với **LR hằng 0,1**, WD=10^{-4} bị overfit, WD=10^{-3} tốt hơn; nhưng với **LR range 0,1-1,0**, WD=10^{-4} lại là tốt nhất (nhỏ hơn 10 lần so với trường hợp LR hằng). Đây là minh chứng trực tiếp cho nguyên lý ”regularization phải cân bằng”: LR lớn hơn (dùng LR range/cyclical) tự mang thêm regularization, nên cần WD nhỏ hơn để tổng regularization không quá mức.

5.2 Weight decay tìm từ snapshot để tiết kiệm thời gian (Hình 12)

Thay vì chạy lại từ đầu cho mỗi giá trị WD, paper gợi ý: chạy một lần tới khi loss ”phẳng” (plateau), lưu snapshot, rồi từ đó **restart** nhiều lần tiếp tục với các giá trị WD khác nhau. Ví dụ trên mạng 3 lớp Cifar-10: chạy đầu với WD=10^{-3} (chưa tối ưu) tới điểm plateau, sau đó 3 lần tiếp tục với WD $\in \{10^{-3}, 3\times10^{-3}, 10^{-2}\}$ cho thấy 3×10^{-3} tốt nhất; trên resnet-56, 3 lần tiếp tục với WD $\in \{10^{-3}, 3 \times 10^{-4}, 10^{-4}\}$ cho thấy 10^{-4} tốt nhất. Cách này **tiết kiệm compute** so với chạy lại từ đầu mỗi giá trị, đồng thời vẫn cho thông tin cả về LR range nếu đang chạy kết hợp LR range test.

5.3 Momentum tối ưu khác nhau theo kiến trúc (Mục 5.2 vs 5.1)

Với resnet-56 (Hình 7, Bảng 1), momentum cao (0,95) thường tốt hơn; nhưng với densenet 40 lớp (Hình 13b), momentum **cao 0,99 làm phân kỳ**, và momentum **nhỏ hơn** (giảm dần 0,9 xuống 0,85) lại tốt hơn 0,95/0,97. Paper tự nhận đây là khác biệt theo kiến trúc, nhấn mạnh phải lặp lại quy trình tìm momentum riêng cho mỗi kiến trúc, không có một giá trị chung tối ưu.

5.4 Nghịch lý validation loss vs accuracy theo batch size (Hình 6)

Trên resnet-56/Cifar-10 với 1cycle: batch size lớn hơn (TBS=1024 so với 128) cho **accuracy cuối cao hơn**, nhưng **validation loss cuối lại cao hơn** (tệ hơn) khi batch size tăng, ngược hướng với accuracy. Paper gọi đây là ”kind of discrepancy... unusual” trong toàn bộ thí nghiệm của họ, tức một quan sát bất thường mà họ không giải thích triệt để, chỉ ghi nhận.

5.5 Cyclical momentum giúp ổn định hơn là tăng accuracy tối đa (Bảng 1)

So sánh momentum cyclical 0,95-0,85 với momentum hằng tốt nhất (0,95) trên resnet-56: cyclical đạt $92,0 \pm 0,2$%, hằng 0,95 đạt $92,1 \pm 0,1$%, tức **gần như tương đương**, không có cải thiện accuracy rõ rệt từ cyclical. Giá trị thật của cyclical momentum, theo paper, là cho phép huấn luyện ổn định trên **khoảng learning rate rộng hơn** (ít nhạy với LR quá lớn) hơn là tăng accuracy tối đa so với một hằng số đã được tune kỹ.

6. Giới hạn

Paper tự nêu các điểm sau, không phải suy luận của người ghi chú:

- Mục 6 (Discussion) tự nhận: literature có nhiều khuyến nghị **mâu thuẫn nhau** vì mỗi paper chỉ xét một vài yếu tố tách biệt (”reminiscent of the parable of the blind men and the elephant”); bài này cũng chỉ là Part 1 trong kế hoạch 2+ phần, chưa xét kiến trúc, dữ liệu, data augmentation, hay các dạng regularization khác (dropout, stochastic depth) một cách đầy đủ, những thứ dự kiến dành cho Part 2.
- Kết quả Imagenet chỉ trung bình **2 lần chạy** (footnote 1, Mục 5.5) do chi phí tính toán, trong khi Cifar/MNIST dùng 4 lần; độ tin cậy thống kê của phần Imagenet do đó thấp hơn.
- Kết luận ”weight decay không nên cyclical” paper tự nói ”appears to be generally so for regularization but was not tested for all regularization methods” (Mục 4.4), tức chưa kiểm tra đầy đủ cho các dạng regularization khác (dropout, stochastic depth).
- Nhiều giá trị số cụ thể (LR tối đa, WD tối ưu) được paper tự nhận **phụ thuộc kiến trúc** (ví dụ densenet cần momentum nhỏ hơn, weight decay nhỏ hơn hẳn so với resnet-56 cùng dataset Cifar-10), nên các số trong Bảng 2 không nên áp dụng trực tiếp cho kiến trúc khác mà không chạy lại quy trình LR range test/grid WD.
- Phần so sánh batch size (Mục 4.2) tự nhận khó khách quan hoàn toàn: paper chọn tiêu chí ”thời gian thực thi gần như cố định” nhưng đây là một lựa chọn trong nhiều lựa chọn hợp lý khác (so theo epoch hay theo iteration cũng có thể biện minh được, tuỳ mục tiêu).
- Baseline Imagenet (momentum 0,9, LR 0,045 decay 0,94 mỗi 2 epoch, WD 10^{-4}) được paper lấy nguyên từ Szegedy et al. (2017) và **không chạy tới hội tụ hoàn toàn** (130 epoch rồi ngoại suy), nên con số ”65% ngoại suy” là ước lượng, không phải số đo trực tiếp; khi so sánh với 67,6% của 1cycle (đo trực tiếp ở 20 epoch), người đọc nên lưu ý một bên là số đo thật, một bên là ngoại suy.

7. Liên hệ với các bài khác

- **optimization content/19_2017_super_convergence.md** (đã xác nhận tồn tại bằng `ls`): bài 47 Mục 4.1 trực tiếp ”revisit” và đặt tên chính thức ”1cycle” cho chính sách learning rate mà Smith & Topin (2017) giới thiệu hiện tượng super-convergence; nên đọc opt 19 trước để hiểu nguồn gốc hiện tượng, rồi đọc bài 47 để thấy nó được hệ thống hoá thành quy trình chọn 4 siêu tham số.
- **optimization content/18_2016_sgdr.md** (đã xác nhận tồn tại): cùng chủ đề ”lịch learning rate theo chu kỳ” nhưng cơ chế khác: SGDR dùng cosine annealing với warm restart (nhiều chu kỳ, có thể tăng độ dài chu kỳ dần), còn 1cycle ở bài 47 dùng đúng **một** chu kỳ tuyến tính có chủ đích không lặp lại; so sánh hai bài giúp thấy ”cyclical learning rate” là một họ ý tưởng với nhiều biến thể khác nhau về số chu kỳ và hình dạng.
- **optimization content/46_2012_practical_recommendations_deep_architectures.md** (đã xác nhận tồn tại): được chính bài 47 trích dẫn (Bengio, 2012) là công trình liên quan gần nhất về hyper-parameter thực hành; bài 47 mở rộng cụ thể hơn nhiều cho 4 siêu tham số với thực nghiệm hệ thống trên nhiều kiến trúc/dataset, và quy ước α cho momentum ở bài 47 ngược với quy ước β ở bài 46 (xem Mục 3.4 bên trên), một điểm cần đọc kỹ khi so sánh hai bài.
- **optimization content/44_2018_batchnorm_helps_optimization.md** (đã xác nhận tồn tại): cùng năm 2018, cùng quan tâm tới việc learning rate lớn liên hệ thế nào với độ ổn định/regularization của huấn luyện; BatchNorm là một cơ chế kiến trúc giúp chịu được learning rate lớn hơn, bổ sung cho các kỹ thuật thuần về lịch siêu tham số ở bài 47.

(Lưu ý: đã `ls` thư mục ”optimization content/” tại thời điểm viết bài này, thấy thêm các file opt 06 (Adam), opt 10 (LAMB), opt 11 (Lion), opt 32 (Practical Bayesian Optimization) đã tồn tại nhưng không đủ liên quan trực tiếp bằng 3 bài trên để đưa vào mục này; opt 01-05, 07-09, 12-17, 20-31, 33-43, 48-51 chưa tồn tại lúc viết, không trỏ tới.)

8. Ứng dụng cho dự án (suy luận)

Đây là suy luận của người ghi chú. Dự án liên quan là nén/hậu huấn luyện LightOnOCR-3-0.8B (VLM OCR, kiến trúc hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang, GPU $\leq$ 8 GB, batch size 1).

- **LR range test là công cụ rẻ và trực tiếp áp dụng được**: chạy một lần fine-tune ngắn (vài trăm bước) với learning rate tăng tuyến tính từ rất nhỏ tới khá lớn, theo dõi validation loss, để tìm nhanh ”LR lớn nhất chấp nhận được” cho LightOnOCR trước khi fine-tune/distill đầy đủ, tốn ít GPU-hour hơn nhiều so với quét nhiều lần fine-tune đầy đủ.
- **Batch size 1 trên GPU $\leq$ 8 GB là tình huống ”batch nhỏ bắt buộc”, không phải lựa chọn**: khuyến nghị ”dùng batch lớn nhất GPU cho phép” của paper không áp dụng được trực tiếp; thay vào đó nên dùng **gradient accumulation** để đạt batch size hiệu dụng lớn hơn nếu muốn tận dụng được các khuyến nghị LR lớn hơn của 1cycle (suy luận ngoại suy: paper không bàn gradient accumulation, công nghệ phổ biến hơn sau 2018).
- **Weight decay cần giảm khi dùng LR lớn hơn (Mục 5.1, Hình 11)**: nếu áp dụng ý tưởng 1cycle cho fine-tune LightOnOCR với LR tăng tạm thời cao hơn bình thường, nên đồng thời giảm weight decay so với fine-tune LR hằng, tránh ”chồng regularization” làm underfit trên bộ dữ liệu OCR vốn đã nhỏ hơn nhiều so với pretraining gốc.
- **Cyclical momentum ít quan trọng hơn LR/WD cho dự án nhỏ**: vì lợi ích chính của cyclical momentum (Mục 5.5) là ổn định ở LR rất lớn, còn khi fine-tune nhẹ (LR nhỏ, ít bước) trên 1 GPU, momentum hằng số đã tune (0,9-0,95, giống AdamW mặc định β_1) nhiều khả năng đủ tốt; đây là suy luận ưu tiên hoá công sức tune cho dự án có ngân sách hạn chế.
- **Hạn chế khi ngoại suy**: toàn bộ thí nghiệm của paper dùng SGD + momentum cổ điển trên CNN (resnet, densenet, LeNet), không dùng optimizer thích nghi (Adam/AdamW) hay kiến trúc Transformer/SSM như LightOnOCR; các số liệu cụ thể (LR tối đa, WD tối ưu) chắc chắn không chuyển trực tiếp, chỉ **quy**

trình chẩn đoán bằng test loss sớm và **nguyên lý cân bằng regularization** là có thể mang sang, cần tự đo lại toàn bộ con số cho kiến trúc/optimizer của dự án.

9. Thí nghiệm gợi ý

1. **LR range test trên một mô hình nhỏ 1 GPU**: huấn luyện một CNN nhỏ (ví dụ ResNet-20 thu nhỏ) trên Cifar-10 hoặc một subset nhỏ, với learning rate tăng tuyến tính từ 10^{-4} tới 1 qua vài trăm bước. Vẽ test loss/accuracy theo LR. Đo: LR tại điểm test loss bắt đầu tăng trở lại. Kỳ vọng: tái lập dạng đường cong giống Hình 1a/3 của paper, xác nhận LR range test hoạt động như mô tả.
2. **So sánh 1cycle với LR hằng ở cùng tổng số bước**: trên cùng model/dataset trên, huấn luyện với (a) LR hằng (giá trị "an toàn" nhỏ) và (b) 1cycle (LR tối đa lấy từ bài 1, rồi giảm dần), cùng tổng số epoch. Đo validation loss/accuracy cuối. Kỳ vọng: 1cycle đạt accuracy bằng hoặc tốt hơn với cùng ngân sách epoch, minh hoạ hiện tượng super-convergence ở quy mô nhỏ.
3. **Quét weight decay theo bisect-exponent**: với LR và momentum cố định ở giá trị tốt tìm được, thử WD $\in \{10^{-2}, 3\times10^{-3}, 10^{-3}\}$ (bisect theo số mũ như Mục 3.5), vẽ validation loss. Đo: giá trị WD cho loss thấp nhất, có nằm giữa khoảng hay ở biên (nếu ở biên, mở rộng khoảng tìm kiếm theo đúng khuyến nghị của paper 46/47).
4. **Lặp lại thí nghiệm WD ở hai chế độ LR khác nhau (tái lập Mục 5.1)**: chạy quét WD một lần với LR hằng, một lần với LR range/1cycle (cùng kiến trúc/dataset). Đo WD tối ưu ở mỗi chế độ. Kỳ vọng: WD tối ưu ở chế độ LR range nhỏ hơn WD tối ưu ở chế độ LR hằng, xác nhận trực tiếp nguyên lý cân bằng regularization trên quy mô nhỏ, trước khi tin tưởng áp dụng ý tưởng này khi fine-tune LightOnOCR.
5. **Kiểm tra momentum range test có "không có cực tiểu rõ" như paper nói**: quét momentum $\{0.5, 0.7, 0.8, 0.9, 0.95, 0.99\}$ ở LR cố định tối ưu, vẽ validation loss/accuracy theo momentum. Đo: đường cong có đơn điệu (không có cực tiểu rõ như Hình 7b của paper) hay có điểm tối ưu rõ (khác paper). Nếu khác, đó là tín hiệu cho thấy kết luận "momentum range test không hữu ích" của paper có thể không tổng quát cho mọi kiến trúc/optimizer, một manh mối hữu ích khi áp dụng ý tưởng này sang kiến trúc hybrid GatedDeltaNet.

10. Tóm tắt công thức / khái niệm

- **SGD với momentum** (công thức (1)-(3)): $v_{iter+1} = \alpha v_{iter} - \epsilon\,\delta L$, $\theta_{iter+1} = \theta_{iter} + v$ (α gần 1 là nhiều momentum, giống quy ước Adam).
- **LR range test**: tăng LR tuyến tính qua một run ngắn, LR tối đa dùng được = LR tại điểm test loss ngừng giảm/bắt đầu tăng.
- **1cycle**: một chu kỳ LR (tăng rồi giảm) ngắn hơn tổng số bước, sau đó LR giảm tiếp vài bậc độ lớn dưới mức khởi điểm cho phần còn lại; thường đi kèm cyclical momentum giảm khi LR tăng.
- **Nguyên lý cân bằng regularization** (Remark 3, 6): tổng các nguồn regularization (LR lớn, batch nhỏ, weight decay, dropout...) cho một dataset/kiến trúc nên ở một mức cân bằng; tăng nguồn này phải giảm nguồn khác.
- **Bisect theo số mũ cho weight decay**: giữa 10^a và 10^b, thử $10^{(a+b)/2}$ trước khi thử các giá trị trung gian khác; ví dụ $10^{-3.5} \approx 3,16\times10^{-4}$.
- **Khuyến nghị "checklist" 4 bước** (đầu Mục 5): LR range test → 1cycle; batch lớn nhất bộ nhớ cho phép; momentum ngắn $\{0.99, 0.97, 0.95, 0.9\}$ rồi cyclical nếu dùng 1cycle; weight decay grid search với độ chính xác 1 chữ số có nghĩa, lớn hơn cho kiến trúc nông/dữ liệu đơn giản, nhỏ hơn cho kiến trúc sâu/dữ liệu phức tạp.

11. Kết luận ngắn

Bài đề xuất một quy trình thực hành để chọn nhanh learning rate, batch size, momentum, và weight decay bằng cách đọc đầu mối underfitting/overfitting từ validation/test loss **sớm** trong huấn luyện, thay cho grid/random search đầy đủ. Bốn siêu tham số không đối xứng về cách xử lý: learning rate và momentum nên dùng dạng

cyclical (1cycle, cyclical momentum giảm khi LR tăng), còn weight decay nên giữ hằng nhưng giá trị tối ưu phụ thuộc chặt vào mức LR đang dùng; batch size nên lớn (giới hạn bởi bộ nhớ) khi dùng 1cycle. Thực nghiệm trên Cifar-10/100, MNIST, Imagenet với nhiều kiến trúc (mạng nông, resnet, wide resnet, densenet, inception-resnet) cho thấy 1cycle thường đạt accuracy bằng hoặc vượt chính sách LR cổ điển với **số epoch ít hơn nhiều lần** (8x cho wide resnet, khoảng 7x cho MNIST theo tự tính ở Mục 4), minh hoạ hiện tượng super-convergence là khá tổng quát, không chỉ riêng cho một kiến trúc. Nguyên lý xuyên suốt và tổng quát nhất của bài, có lẽ hữu dụng hơn cả các con số cụ thể, là: **phải cân bằng tổng lượng regularization cho từng dataset/kiến trúc**, không tune từng siêu tham số một cách độc lập.

Nguồn gốc: https://arxiv.org/abs/1803.09820

Bài 48

Old Optimizer, New Norm: An Anthology (2024)

Tác giả: Jeremy Bernstein, Laker Newhouse (MIT CSAIL, Hoa Kỳ).
Venue / arXiv: arXiv:2409.20325 (v2, 06/12/2024); paper tự ghi đầu trang là ”OPT2024: 16th Annual Workshop on Optimization for Machine Learning” (workshop đi cùng NeurIPS 2024), không phải bài main-track hội nghị/tạp chí có peer review đầy đủ.
Nguồn: https://arxiv.org/abs/2409.20325
Chủ đề: diễn giải lại ba optimizer phổ biến (Adam, Shampoo, Prodigy), khi tắt exponential moving average (EMA), như các biến thể của ”steepest descent” (hạ dốc nhất) dưới một chuẩn (norm) toán tử cụ thể; từ đó đề xuất khung ”modular norm” để gán chuẩn khác nhau cho từng tensor trọng số theo vai trò của nó trong mạng.
Vị trí trong lộ trình: đọc sau opt 06 (Adam), opt 11 (Lion), opt 45 (Deconstructing what makes a good optimizer for language models, chính paper này trích dẫn làm bằng chứng thực nghiệm) và opt 36 (scaling exponents/parameterizations cho optimizer, cùng tinh thần ”chuẩn hoá theo layer” như muP); đọc trước opt 49 (Gradient Descent Happens in a Tiny Subspace) vì bài này cho một lăng kính lý thuyết thống nhất ”tại sao” nhiều optimizer hoạt động dựa trên hình học (norm) của không gian trọng số, trước khi sang phân tích hình học của chính quỹ đạo gradient (Hessian, subspace) ở bài 49.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu, định lý, công thức lấy từ toàn văn arXiv:2409.20325 (bản v2, tải và đọc toàn bộ ngày 09/11/2026, gồm cả Prologue, 3 ”Story”, Epilogue, Appendix A và Appendix B chứng minh). Các phép tính đánh dấu ”tự tính” là của người ghi chú, không có trong paper. Đây là paper lý thuyết/toán: paper không chạy benchmark huấn luyện mạng neural quy mô lớn của riêng mình, Mục 4 dưới đây nói rõ điều này và chỉ liệt kê các kết quả lý thuyết (định lý/mệnh đề) cùng vài dẫn chứng thực nghiệm mà paper trích dẫn từ công trình khác.

1. Bối cảnh

Theo Prologue của paper, các optimizer deep learning thường được ”bán” (motivate) qua hai lăng kính lý thuyết: (a) lý thuyết tối ưu lồi (convex optimization), ví dụ Adam gốc (Kingma và Ba, 2015) phân tích qua ”regret bound” kiểu online convex optimization; (b) lý thuyết xấp xỉ bậc hai (approximate second-order), ví dụ xem Adam hay Shampoo như đang xấp xỉ nghịch đảo Hessian theo đường chéo hoặc theo khối Kronecker.

Hai tác giả cho rằng cả hai lăng kính này, dù hữu ích để chứng minh hội tụ, không giải thích trực quan ”vì sao” các optimizer này tốt cho deep learning, một bài toán vốn không lồi và nơi xấp xỉ Hessian rất tốn kém. Paper đề xuất một lăng kính thứ ba, đơn giản hơn: lý thuyết bậc một (first-order) thuần túy, không cần giả định lồi, dựa trên khái niệm cũ ”steepest descent dưới một chuẩn” (steepest descent under a norm). Ý tưởng cốt lõi: nếu tắt EMA (momentum và trung bình động bậc hai) của Adam, Shampoo, và Prodigy, mỗi thuật toán thu gọn thành đúng nghiệm của một bài toán steepest descent dưới một chuẩn toán tử cụ thể, không cần bất kỳ giả định lồi hay xấp xỉ Hessian nào.

Bối cảnh rộng hơn: cuối 2023 - 2024 là giai đoạn optimizer cho LLM được xem xét lại nhiều (Lion xem opt 11, Adafactor opt 12, Sophia opt 13, bài Deconstructing optimizers opt 45, nghiên cứu muP/scaling exponents opt 36), vì chi phí tuning optimizer cho mô hình tỷ tham số rất lớn nên hiểu ”vì sao” một optimizer hoạt động có giá trị thực tiễn (tiết kiệm tuning) chứ không chỉ học thuật. Bài này nằm trong mạch đó, nhắm tới câu hỏi: nếu hiểu rõ Adam/Shampoo như lựa chọn chuẩn, liệu có thể thiết kế optimizer tốt hơn bằng cách chọn chuẩn có chủ đích (modular norm) thay vì ngẫu nhiên kế thừa lịch sử thuật toán.

2. Ý tưởng chính

Công cụ trung tâm là **steepest descent dưới một chuẩn** (Proposition 1). Cho gradient $g \in \mathbb{R}^n$, một chuẩn $\|\cdot\|$ trên $\mathbb{R}^n$ với chuẩn đối (dual norm) $\|\cdot\|_*$, và một hệ số "độ sắc" (sharpness) $\lambda > 0$, bài toán:

$$\arg\min_{\Delta w \in \mathbb{R}^n} \; g^T \Delta w + \frac{\lambda}{2}\|\Delta w\|^2$$

có nghiệm tách thành hai phần độc lập: độ lớn bước đi (chuẩn đối của gradient chia cho sharpness) và hướng đi (vector đơn vị tối đa hoá tích trong với gradient):

$$\Delta w^\star = -\frac{\|g\|_*}{\lambda} \cdot \arg\max_{\|t\|=1} g^T t.$$

Mạch chính của bài là: các optimizer nổi tiếng khác nhau chẳng qua là chọn chuẩn $\|\cdot\|$ khác nhau cho bài toán trên (sau khi tắt EMA):

- Chuẩn Euclid ℓ_2 trên không gian trọng số phẳng $\Rightarrow$ gradient descent thường (SGD).
- Chuẩn vô cùng ℓ_∞ trên không gian trọng số phẳng $\Rightarrow$ sign descent, đúng là Adam khi tắt EMA ($\beta_1 = \beta_2 = 0$).
- Chuẩn phổ (spectral norm, tức chuẩn toán tử $\ell_2 \to \ell_2$ của ma trận) áp theo từng lớp $\Rightarrow$ Shampoo khi tắt tích luỹ (accumulation) của hai preconditioner trái/phải.
- "Chuẩn modular" (modular norm): max có trọng số của chuẩn riêng từng lớp $\Rightarrow$ khung tổng quát hoá cả hai trường hợp Adam và Shampoo, cho phép mỗi tensor trọng số có chuẩn riêng theo "vai trò" của nó (ví dụ lớp embedding khác lớp linear thường, dù cùng là không gian ma trận $\mathbb{R}^{m\times n}$).
- Prodigy (tắt EMA) hoá ra dùng **cùng chuẩn** ℓ_∞ như Adam (vẫn là sign descent), nhưng thay bước đi $\eta = \|g\|_*/\lambda$ cố định bằng một quy tắc khởi động (warmup) tự động gọi là đạt "tốc độ thoát" (escape velocity). Điều này cho thấy Proposition 1 tách rời rõ hai quyết định thiết kế optimizer: chọn chuẩn (hướng đi) và chọn cách đặt sharpness/step size (độ lớn bước), hai việc độc lập nhau.

Một điểm quan trọng paper nêu rõ: EMA (tức $\beta_1, \beta_2 \neq 0$ trong Adam, hay tích luỹ L_t, R_t trong Shampoo) không nằm trong khung lý thuyết "steepest descent dưới chuẩn" này; EMA được paper mô tả như một phép "làm mượt" (smoothing) giúp optimizer bền hơn với nhiễu minibatch, nhưng vai trò chính xác của EMA, theo chính câu chữ của paper, "nailing down the precise role of EMA is perhaps still an open problem" (vẫn là câu hỏi mở). Đây là giới hạn quan trọng cần nhớ xuyên suốt: toàn bộ lý thuyết trong bài chỉ áp dụng cho phiên bản **tắt EMA** của ba optimizer, không phải phiên bản thật dùng trong thực hành (luôn có EMA).

3. Phương pháp chi tiết

3.1. Định nghĩa chuẩn toán tử cảm sinh (induced operator norm)

Cho ma trận $M \in \mathbb{R}^{d_{out}\times d_{in}}$ và hai chuẩn vector $\|\cdot\|_\alpha$ trên $\mathbb{R}^{d_{in}}$, $\|\cdot\|_\beta$ trên $\mathbb{R}^{d_{out}}$, chuẩn toán tử "α sang β" (Definition 1):

$$\|M\|_{\alpha\to\beta} = \max_{x\neq 0} \frac{\|Mx\|_\beta}{\|x\|_\alpha}.$$

Thay đổi α, β cho ra một họ lớn chuẩn ma trận, và tương ứng một họ lớn optimizer steepest-descent. Đây là "trục tự do" chính mà paper khai thác.

Tự tính (ví dụ minh hoạ Proposition 1): lấy $g = (3, 4) \in \mathbb{R}^2$, $\lambda = 2$.

- Dưới chuẩn Euclid ℓ_2: $\|g\|_2 = \sqrt{3^2+4^2} = 5$ (chuẩn đối của ℓ_2 cũng là ℓ_2). Hướng tối ưu $t^\star = \arg\max_{\|t\|_2=1} g^T t = g/\|g\|_2 = (0.6, 0.8)$. Độ lớn $c = \|g\|_2/\lambda = 5/2 = 2.5$. Vậy $\Delta w = -c \cdot t^\star = -2.5 \cdot (0.6, 0.8) = (-1.5, -2.0)$. Kiểm tra: đây đúng bằng $-g/\lambda = (-1.5, -2.0)$, khớp công thức SGD ở Bảng 1 (mục 10 dưới).
- Dưới chuẩn vô cùng ℓ_∞: chuẩn đối của ℓ_∞ là ℓ_1, $\|g\|_1 = 3 + 4 = 7$. Hướng tối ưu là $t^\star = \text{sign}(g) = (1, 1)$ (vì tối đa hoá $g^T t$ với $|t_i| \le 1$ thì đặt $t_i = \text{sign}(g_i)$ cho mọi i, không phụ thuộc độ lớn $|g_i|$). Độ lớn $c = \|g\|_1/\lambda = 7/2 = 3.5$. Vậy $\Delta w = -3.5 \cdot (1, 1) = (-3.5, -3.5)$.

So sánh hai kết quả tự tính trên: cùng một gradient g, cùng sharpness λ, nhưng đổi chuẩn thì **hướng** cập nhật đổi hẳn, từ $(-1.5, -2.0)$ (tỉ lệ với gradient, như SGD) sang $(-3.5, -3.5)$ (chỉ phụ thuộc dấu, như sign descent/Adam không EMA). Đây chính là minh hoạ bằng số cho Hình 1 của paper (đổi chuẩn làm đổi hướng nghiệm khi gradient không nằm trên trục toạ độ).

3.2. Story I: Adam là steepest descent dưới chuẩn ”max-of-max”

Bỏ bias correction và ổn định số, Adam gồm:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad w_{t+1} = w_t - \eta\, m_t/\sqrt{v_t}.$$

Khi $\beta_1 = \beta_2 = 0$ (tắt EMA), công thức rút gọn đúng thành sign descent: $w_{t+1} = w_t - \eta \cdot \text{sign}(g_t)$. Đây không phải phát hiện mới của paper này: RMSprop (Tieleman và Hinton, 2012, tiền thân của Adam) từng được chính các tác giả gốc gọi là ”phiên bản minibatch của chỉ-dùng-dấu-gradient”, và RMSprop lại kế thừa RPROP (Riedmiller và Braun, 1993).

Mệnh đề 2 (Proposition 2) chỉ ra sign descent chính là steepest descent dưới chuẩn ℓ_∞ (phần đã tự tính ở trên). Nhưng câu hỏi paper đặt ra tiếp: tại sao chuẩn ℓ_∞ trên toàn bộ vector trọng số **đã làm phẳng** (flatten) lại có ý nghĩa gì với deep learning, khi không gian trọng số thực chất có cấu trúc theo lớp (ma trận $W_1, \ldots, W_L$)?

Câu trả lời của paper dựa vào tính chất ”max của max là max”: nếu $\text{row}_r(W_l)$ là hàng thứ r của ma trận lớp l, và w là vector đã làm phẳng toàn bộ các W_l, thì:

$$\|w\|_\infty = \max_l \max_r \|\text{row}_r(W_l)\|_\infty = \max_l \|W_l\|_{\ell_1 \to \ell_\infty}.$$

Tức chuẩn ℓ_∞ của vector phẳng trùng với chuẩn toán tử $\ell_1 \to \ell_\infty$ lớn nhất trong các lớp, gọi là ”chuẩn max-of-max”. Mệnh đề 3 chứng minh: bài toán steepest descent nhiều ma trận dưới chuẩn max-of-max này có nghiệm là sign descent áp riêng từng lớp, $\Delta W_l = -\eta \cdot \text{sign}(G_l)$ cho mọi l, với một **bước đi toàn cục duy nhất** $\eta = \frac{1}{\lambda} \sum_{l=1}^{L} \|G_l\|^*_{\ell_1 \to \ell_\infty}$ dùng chung cho mọi lớp (lấy từ tổng chuẩn đối của gradient qua các lớp).

(Nhận xét người ghi chú): cần phân biệt rõ công thức η vừa nêu (kết quả suy ra từ bài toán tối ưu Mệnh đề 3, một đại lượng phụ thuộc gradient hiện tại, tính lại mỗi bước) với learning rate η thực tế mà Adam/Lion dùng trong huấn luyện (một siêu tham số cố định hoặc theo lịch, do người dùng chỉnh, xem opt 06, opt 11). Paper dùng Mệnh đề 3 để biện minh cho **hướng đi** (sign theo từng lớp) là tối ưu dưới một chuẩn hợp lý, không khẳng định Adam/Lion trong thực hành thực sự tính η theo công thức tổng dual-norm đó; đây là một khoảng cách giữa lý thuyết và thuật toán triển khai thật mà paper không nói rõ.

3.3. Story II: Shampoo là steepest descent dưới chuẩn phổ (spectral norm)

Shampoo (Gupta và cộng sự, 2017/2018), tại mỗi bước và mỗi lớp, với gradient ma trận G_t:

$$L_t = L_{t-1} + G_t G_t^T, \quad R_t = R_{t-1} + G_t^T G_t, \quad W_{t+1} = W_t - \eta \cdot L_t^{-1/4} G_t R_t^{-1/4}.$$

Nếu tắt tích luỹ, đặt $L_t = G_t G_t^T$, $R_t = G_t^T G_t$ (chỉ dùng gradient hiện tại, không cộng dồn lịch sử), và thay SVD rút gọn $G_t = U_t \Sigma_t V_t^T$, công thức rút gọn thành:

$$W_{t+1} = W_t - \eta \cdot U_t V_t^T.$$

Mệnh đề 4 chứng minh UV^T chính là ma trận bán-trực-giao (semi-orthogonal, $AA^T = I$ hoặc $A^T A = I$) **gần** G **nhất theo chuẩn Frobenius** (một dạng bài toán Procrustes trực giao). Nếu G có hạng đầy đủ (full rank), nghiệm UV^T là duy nhất; nếu không, có nhiều nghiệm tốt như nhau (ví dụ paper cho công thức nghiệm thay thế ở Phương trình 47 khi có giá trị suy biến $\sigma_i = 0$).

Mệnh đề 5 chỉ ra: điều này tương đương steepest descent dưới **chuẩn phổ** $\|\cdot\|_{\ell_2 \to \ell_2}$ áp theo max qua các lớp, với bước đi $\eta = \frac{1}{\lambda} \sum_l \operatorname{tr} \Sigma_l$ (tổng các giá trị suy biến, tức chuẩn vết/nuclear norm, là chuẩn đối của chuẩn phổ).

Tự tính (ví dụ Mệnh đề 4, trường hợp dễ): lấy $G = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$. Ma trận này đã ở dạng SVD với $U = I$, $\Sigma = \operatorname{diag}(2, 1)$, $V = I$ (hai giá trị suy biến dương, không cần xoay). Theo công thức, nghiệm bán-trực-giao gần nhất là $UV^T = I \cdot I = I$ (ma trận đơn vị). Trực giác: cập nhật Shampoo (không tích luỹ) ”trắng hoá” (whiten) sự chênh lệch độ lớn giữa hai hướng suy biến (2 so với 1), chỉ giữ hướng, giống tinh thần chuẩn hoá entry-wise của Adam nhưng áp cho toàn ma trận qua chuẩn phổ thay vì cho từng phần tử qua chuẩn ℓ_∞.

Về động cơ ”tại sao chuẩn phổ”: Mệnh đề 6 (phần quan trọng nhất về mặt lý thuyết của Story II) xét mô hình tuyến tính $y = Wx$, $W \in \mathbb{R}^{d_{out} \times d_{in}}$, dữ liệu chuẩn hoá $\|x_i\|_2 = \sqrt{d_{in}}$, hàm mất bình phương $L(W) = \frac{1}{2n} \sum_i \frac{1}{d_{out}} \|y_i - W x_i\|_2^2$. Paper chứng minh cận trên bậc hai đúng (exact quadratic upper bound, không xấp xỉ):

$$L(W + \Delta W) \le L(W) + \langle \nabla_W L(W), \Delta W \rangle + \frac{1}{2} \cdot \frac{d_{out}}{d_{in}} \cdot \|\Delta W\|_{\ell_2 \to \ell_2}^2.$$

Chọn ΔW để tối thiểu hoá chính cận trên này **đúng bằng** bài toán steepest descent dưới chuẩn phổ. Đây là lý do paper cho là chuẩn phổ ”có ý nghĩa” với deep learning hơn là sự ”trùng hợp” của chuẩn ℓ_∞ trong Story I: có một chuỗi suy luận majorization-minimization (suy ra cận trên đúng của loss, rồi tối thiểu hoá cận trên đó) dẫn trực tiếp tới chuẩn phổ, trong khi với Adam paper chỉ chỉ ra sự trùng khớp đại số (max-of-max) chứ chưa suy ra từ một cận trên loss tương tự.

Paper cũng dẫn Phụ lục A (Appendix A) về cách tính UV^T trong thực hành: (1) SVD đầy đủ; (2) sketching ngẫu nhiên; (3) Newton iteration cho nghịch đảo căn bậc p của ma trận; (4) một ”Newton-Schulz iteration” do chính hai tác giả trình bày, dùng lặp $X_{t+1} = \frac{3}{2} X_t - \frac{1}{2} X_t X_t^T X_t$ xuất phát từ $X_0 = G / \|G\|_{\ell_2 \to \ell_2}$, hội tụ về UV^T chỉ bằng vài phép nhân ma trận, không cần SVD tường minh *(ngoài paper, kiến thức chung: đây chính là phép ”trực giao hoá Newton-Schulz” được optimizer Muon dùng sau này cho các kỷ lục tốc độ huấn luyện nanoGPT, tuy bài này không nhắc tên ”Muon”).*

3.4. Story III: Prodigy, tự động tính ”tốc độ thoát” (escape velocity)

Prodigy (Mishchenko và Defazio, 2023), bỏ ổn định số và lịch learning rate, có hệ cập nhật (ký hiệu g_t là gradient, w_0 là trọng số khởi tạo):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \eta_t g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) \eta_t^2 g_t^2,$$
$$r_t = \beta_2 r_{t-1} + (1 - \beta_2) \eta_t^2 g_t^T (w_0 - w_t), \quad s_t = \beta_2 s_{t-1} + (1 - \beta_2) \eta_t^2 g_t,$$
$$\eta_{t+1} = \max\left(\eta_t, \frac{r_t}{\|s_t\|_1}\right), \qquad w_{t+1} = w_t - \eta_t \cdot m_t / \sqrt{v_t}.$$

Khi $\beta_1 = \beta_2 = 0$, hệ này rút gọn thành sign descent với bước đi động:

$$\eta_{t+1} = \max\left(\eta_t,\ \frac{g_t^T(w_0 - w_t)}{\|g_t\|_1}\right), \qquad w_{t+1} = w_t - \eta_t \cdot \text{sign}(g_t).$$

Vẫn là chuẩn ℓ_∞ (giống Adam), nhưng η_t không còn là hằng số mà được cập nhật theo một thủ tục mà paper mô tả bằng phép ẩn dụ "tốc độ thoát" (escape velocity, mượn từ vật lý): (1) chọn η_0 rất nhỏ, chắc chắn nhỏ hơn $\eta^\star$ tối ưu; (2) ở mỗi bước, nếu trọng số w_t chưa "thoát" khỏi vùng tuyến tính hoá quanh w_0 (đạo hàm có hướng $(w_1 - w_0)^T g_1$ vẫn còn âm, nghĩa là còn có thể bước xa hơn mà loss còn giảm theo xấp xỉ tuyến tính), thì nhân đôi $\eta_{t+1} = 2\eta_t$; (3) khi đã thoát, ngừng tăng.

Paper biến đổi công thức η_{t+1} bằng hai giả định: (a) gradient "dày" (dense), $\|g_t\|_2/\|g_t\|_1 \approx 1/\sqrt{n}$; (b) w_t còn gần w_0 nên góc θ giữa g_t và $w_0 - w_t$ gần 0 ($\cos\theta \approx 1$). Khi đó $\eta_{t+1} \approx \max(\eta_t, \|w_t - w_0\|_{RMS})$, với chuẩn RMS định nghĩa $\|\cdot\|_{RMS} := \frac{1}{\sqrt{n}}\|\cdot\|_2$. Kết hợp với cập nhật sign descent, paper suy ra $\|w_{t+2} - w_{t+1}\|_{RMS} \approx \max(\eta_t, \|w_t - w_0\|_{RMS}) \geq \|w_t - w_0\|_{RMS}$, tức dưới hai giả định trên, bước đi ở thời điểm $t+1$ **xấp xỉ bằng toàn bộ quãng đường đã đi tới thời điểm** t, ngụ ý η_t tăng theo cấp số nhân (gấp đôi liên tục) cho tới khi giả định (b) không còn đúng (tức đã "thoát" đủ xa).

Paper cũng lưu ý đây chỉ là một trong một họ phương pháp "line search" khá dĩ (trích Armijo 1966; Riedmiller và Braun 1993; Kenneweg và cộng sự, 2024), và tự nhận quy tắc của Prodigy "chỉ tăng, không bao giờ giảm η_t" có thể là chưa tối ưu; paper nêu một biến thể khác, $\eta_{t+1} = \eta_t \times (1 + \cos\theta)$, được nói là "hoạt động tốt trong một số thí nghiệm sơ bộ" (preliminary experiments) nhưng **không cho số liệu cụ thể nào** (không nêu dataset, không nêu con số loss/accuracy). Đây là một khẳng định định tính, thiếu kiểm chứng định lượng trong chính bài, cần ghi nhận là một điểm "không chắc" của paper (xem Mục 6).

3.5. Chuẩn modular (modular norm) và các chuẩn toán tử khả tính

Epilogue tổng hợp ba câu chuyện bằng một khung chung hơn, "chuẩn modular" (Mệnh đề 7, mượn từ Large và cộng sự, 2024, "Scalable optimization in the modular norm", arXiv:2405.14813): cho các hệ số vô hướng $s_1, \ldots, s_L > 0$ và chuẩn riêng $\|\cdot\|_1, \ldots, \|\cdot\|_L$ cho từng lớp, định nghĩa:

$$\|(W_1, \ldots, W_L)\|_{\text{modular}} = \max\{s_1\|W_1\|_1, \ldots, s_L\|W_L\|_L\}.$$

Nghiệm steepest descent dưới chuẩn này (chứng minh ở Appendix B, qua đổi biến $\Delta W_l = c_l T_l$ với $\|T_l\|_l = 1$): mọi lớp dùng **cùng một "vận tốc" chuẩn hoá** η/s_l (với $\eta = \frac{1}{\lambda}\sum_k \frac{1}{s_k}\|G_k\|_k^*$ tính một lần, dùng chung), nhưng độ lớn cập nhật tuyệt đối từng lớp $c_l = \eta/s_l$ khác nhau tuỳ hệ số s_l do người thiết kế chọn, còn hướng $T_l = \arg\max_{\|T_l\|_l=1}\langle G_l, T_l\rangle$ tối ưu riêng theo chuẩn riêng của lớp đó. Đặt $s_l = 1$ và chuẩn $\ell_1 \to \ell_\infty$ cho mọi lớp thì khôi phục đúng Story I (Adam); đặt chuẩn phổ cho mọi lớp thì khôi phục đúng Story II (Shampoo).

Mệnh đề 8 cho công thức đóng (không cần giải tối ưu số) cho hai họ chuẩn toán tử: $\|M\|_{\ell_1\to\ell_p} = \max_j \|\text{col}_j(M)\|_p$ (chuẩn lớn nhất trong các cột) và $\|M\|_{\ell_p\to\ell_\infty} = \max_i \|\text{row}_i(M)\|_{p/(p-1)}$ (chuẩn đối lớn nhất trong các hàng). Nhờ đó nhiều chuẩn (không chỉ $\ell_1 \to \ell_\infty$ và phổ) có thể tính rẻ, mở ra không gian thiết kế lớn hơn hẳn hai ví dụ Adam/Shampoo.

Khuyến nghị thực hành mà paper đưa ra (không kèm thực nghiệm định lượng riêng, chỉ là lập luận): lớp **linear thường** nên dùng chuẩn toán tử "RMS sang RMS" (tương đương một chuẩn phổ được co giãn lại, theo Yang và cộng sự, 2023) vì nó thường ánh xạ giữa các vector có chuẩn RMS xấp xỉ 1; lớp **embedding** nên dùng chuẩn "ℓ_1 sang RMS" (tương đương chuẩn toán tử $\ell_1 \to \ell_2$ co giãn lại) vì đầu vào là vector one-hot còn đầu ra cần chuẩn RMS đơn vị. Hai lớp này cùng là không gian ma trận $\mathbb{R}^{m\times n}$ nhưng, theo paper, nên có chuẩn khác nhau vì "vai trò" khác nhau.

4. Thí nghiệm và kết quả chính

Đây là paper lý thuyết, không có benchmark huấn luyện mạng neural quy mô lớn do chính tác giả thực hiện. Toàn bộ ”kết quả chính” là 8 mệnh đề (Proposition 1-8) cùng chứng minh đầy đủ ở Appendix B, tóm tắt lại ở Mục 3 trên. Không có bảng loss/accuracy, không có learning curve, không có so sánh optimizer trên một bộ dữ liệu cụ thể nào xuất phát từ thí nghiệm của chính hai tác giả.

Các dẫn chứng ”thực nghiệm” duy nhất trong bài đều là trích dẫn công trình khác, không phải số liệu paper tự đo:

- Một biến thể Shampoo thắng track ”external tuning” của cuộc thi AlgoPerf: Training Algorithms 2024 (Dahl và cộng sự, 2023): đây là bằng chứng gián tiếp paper dùng để lập luận chuẩn phổ ”đáng quan tâm trong thực hành”, không phải thí nghiệm của bài này.
- Zhao và cộng sự (2024, arXiv:2407.07972, chính là opt 45 trong lộ trình) và Large và cộng sự (2024) được trích dẫn khi paper suy đoán ”việc chuẩn hoá gradient theo từng ma trận (giống Adam/Lion) có thể là lý do chính khiến các optimizer này vượt vanilla gradient descent khi huấn luyện LLM”: paper diễn đạt rõ đây là suy đoán (”may be a major reason”), không phải kết luận paper tự kiểm chứng.
- Yang và cộng sự (2023) và Large và cộng sự (2024) được trích dẫn cho hiện tượng ”learning rate transfer across scale” khi dùng chuẩn tốt cho từng lớp: lại là kết quả của paper khác, không phải của bài 48.
- Prodigy: câu ”chúng tôi thấy một quy tắc... hoạt động tốt trong một số thí nghiệm sơ bộ” không kèm bảng số liệu, dataset, hay độ lớn mô hình nào.

Vì vậy, ”kết quả chính” cần hiểu đúng bản chất là **kết quả toán học** (đẳng thức/đồng nhất thức giữa optimizer đã biết và một bài toán steepest descent tường minh), không phải kết quả thực nghiệm theo nghĩa thông thường của các paper deep learning khác trong lộ trình (ví dụ opt 44, opt 45, opt 46, opt 47 đều có bảng số liệu huấn luyện thật).

5. Phân tích / ablation

Paper không có bảng ablation theo nghĩa thông thường (so sánh có/không một thành phần trên benchmark), nhưng có một ”ablation toán học” nhất quán xuyên suốt cả ba câu chuyện: **tắt EMA** ($\beta_1 = \beta_2 = 0$ với Adam/Prodigy, hoặc tắt tích luỹ L_t, R_t với Shampoo) chính là phép thao tác cho phép quy mỗi optimizer về đúng một bài toán steepest descent tường minh. Đây không phải ablation thực nghiệm (không có số liệu đo loss khi on/off EMA), mà là một phép ”rút gọn lý thuyết” (algebraic reduction): paper chứng minh bằng đại số rằng dạng tắt-EMA của Adam/Shampoo/Prodigy trùng khớp chính xác với nghiệm của một bài toán tối ưu có công thức đóng, không phải một quan sát thực nghiệm.

Hệ quả của cách ”ablation” này: toàn bộ lý giải ”tại sao Adam/Shampoo/Prodigy hoạt động tốt” trong bài chỉ chứng minh được cho phiên bản **không có EMA**, còn phiên bản **có EMA** (tức phiên bản thật sự dùng trong mọi pipeline huấn luyện thực tế, bao gồm cả AlgoPerf-winning Shampoo) thì paper tự nhận chưa có lý thuyết tương đương, chỉ dừng ở suy đoán EMA ”làm mượt”/”giảm nhiễu minibatch”. Nói cách khác, phần ”giải thích được bằng toán” (tắt EMA) và phần ”thực sự dùng trong thực hành” (có EMA) là hai chế độ khác nhau, và khoảng cách giữa chúng chưa được lấp đầy trong bài.

Một dạng phân tích khác xuất hiện ở Appendix A (các chiến lược tính UV^T cho Shampoo: SVD đầy đủ, sketching, Newton iteration, Newton-Schulz): đây gần như một ”ablation về phương pháp tính”, so sánh định tính (không có bảng thời gian chạy thật) bốn cách tính cùng một đại lượng toán học, với nhận xét chi phí phụ thuộc số điều kiện (condition number) của G và phần cứng sẵn có, nhưng không có số liệu benchmark cụ thể kèm theo.

6. Giới hạn

- **EMA chưa được lý thuyết hoá**: paper tự thừa nhận ”nailing down the precise role of EMA is perhaps still an open problem”. Toàn bộ các Mệnh đề 1-8 chỉ đúng khi tắt EMA; phiên bản thật của Adam/Shampoo/Prodigy (luôn có EMA khi dùng trong huấn luyện thật) không được các mệnh đề này mô tả trực tiếp.
- **Cận loss chỉ chứng minh cho mô hình tuyến tính**: Mệnh đề 6 (lý do chọn chuẩn phổ) chỉ chứng minh cho một mạng tuyến tính một lớp với hàm mất bình phương, một trường hợp rất đơn giản hoá so với mạng neural sâu phi tuyến thật. Paper có nhắc ”generalizing the pattern to arbitrary network architectures and loss functions requires more advanced machinery” và trích các công trình khác (Automatic Gradient Descent, Bernstein và cộng sự 2023; Streeter 2023; Large và cộng sự 2024) nhưng không tự chứng minh phần tổng quát này trong bài.
- **Khoảng cách giữa công thức bước đi suy ra và bước đi thực dùng**: như đã nêu ở Mục 3.2, công thức η trong Mệnh đề 3/5/7 là một đại lượng tính lại mỗi bước từ chuẩn đối của gradient hiện tại, khác về bản chất với learning rate hằng số/theo lịch mà Adam, Shampoo, Lion dùng trong thực hành. Paper không làm rõ mối liên hệ định lượng giữa hai đại lượng này.
- **Suy luận Prodigy dựa trên hai giả định có thể vi phạm**: (a) gradient ”dày” ($\|g\|_2/\|g\|_1 \approx 1/\sqrt{n}$, không đúng nếu gradient thưa/tập trung ở vài chiều); (b) w_t còn gần w_0 ($\cos\theta \approx 1$, chỉ đúng ở giai đoạn đầu huấn luyện). Khi hai giả định này không còn đúng (huấn luyện đã chạy lâu), phép xấp xỉ $\eta_{t+1} \approx \max(\eta_t, \|w_t - w_0\|_{RMS})$ không còn được đảm bảo, và paper không phân tích hành vi ngoài vùng xấp xỉ này.
- **Khẳng định định tính không kèm số liệu**: quy tắc thay thế $\eta_{t+1} = \eta_t(1 + \cos\theta)$ được nói ”hoạt động tốt trong một số thí nghiệm sơ bộ” nhưng không có bảng số, dataset, hay cấu hình mô hình nào kèm theo; đây là một khẳng định chưa kiểm chứng được từ chính văn bản.
- **Tự sửa giữa các phiên bản (điểm paper tự mâu thuẫn, cần nêu rõ)**: ở Appendix A, đoạn mô tả Newton-Schulz iteration (Phương trình 36), bản thân đoạn văn có câu ”After posting the first version of this paper on arXiv, we found out that the iteration, at least for fixed coefficients, is classical (Kovarik, 1970; Björck and Bowie, 1971).” Nói cách khác, ở bản v1, phép lặp này được trình bày như một đóng góp mới (”we developed a Newton-Schulz iteration”), nhưng tới bản v2 (bản đọc cho ghi chú này), các tác giả tự đính chính rằng kỹ thuật (với hệ số cố định) thực ra đã có từ 1970-1971, không phải phát hiện mới của bài. Đây là một ví dụ rõ về paper tự điều chỉnh/giảm nhẹ một tuyên bố mới lạ (novelty) giữa các phiên bản, cần ghi nhận khi trích dẫn phần ”Newton-Schulz” của bài này ở nơi khác.
- **Chưa phải peer review đầy đủ**: paper là bài workshop (OPT2024, NeurIPS workshop series), không phải bài main-track hội nghị/tạp chí lớn, nên quy trình review có thể ít nghiêm ngặt hơn so với các paper main-track khác trong lộ trình (ví dụ Adam ICLR 2015, hoặc BatchNorm ICML 2015 ở opt 44).

7. Liên hệ với các bài khác

(Đã kiểm tra tồn tại file bằng lệnh `ls` thư mục ”optimization content/” trước khi trích; tại thời điểm viết đã có các file: 06, 07, 10, 11, 12, 13, 18, 19, 21, 32, 34, 36, 37, 44, 45, 46, 47.)

- **optimization content/06_2014_adam.md** (Adam gốc, Kingma và Ba 2015): bài 48 phân tích lại chính thuật toán của bài 06 (khi tắt EMA) như steepest descent dưới chuẩn ℓ_∞/max-of-max; nên đọc 06 trước để có công thức Adam gốc làm đối chiếu khi đọc Story I.
- **optimization content/11_2023_lion.md** (Lion, Chen và cộng sự 2023): bài 48 trích dẫn đúng paper Lion khi lập luận ”sign descent theo từng lớp có thể là lý do Adam/Lion vượt vanilla GD cho LLM”; Lion vốn cũng là một dạng sign-based update, nên khung chuẩn ℓ_∞/max-of-max của bài 48 áp dụng gần như trực tiếp cho Lion.
- **optimization content/45_2024_deconstructing_llm_optimizer.md** (Zhao và cộng sự 2024): chính paper này được bài 48 trích dẫn hai lần (ở Story I và ở Epilogue) làm bằng chứng thực nghiệm gián tiếp cho việc chuẩn hoá gradient theo ma trận giúp ích cho huấn luyện LLM; nên đọc 45 để có số liệu thực

nghiệm thật mà bài 48 (lý thuyết) không tự cung cấp.

- **optimization content/36_2024_scaling_exponents_parameterizations_optimizers.md** (scaling exponents/parameterizations, liên quan muP): cùng tinh thần ”chuẩn hoá/co giãn theo từng lớp để optimizer ổn định khi tăng quy mô” như khuyến nghị ”RMS-to-RMS cho linear, ℓ_1-to-RMS cho embedding” ở Epilogue bài 48; hai bài bổ sung cho nhau, bài 36 cho góc nhìn parameterization/learning-rate-transfer, bài 48 cho góc nhìn ”chuẩn toán tử” tổng quát hơn.
- **optimization content/10_2019_lamb.md** (LAMB, layerwise adaptive): LAMB cũng chuẩn hoá cập nhật theo từng lớp (trust ratio theo norm của trọng số/gradient mỗi lớp), là một tiền lệ thực hành gần với tinh thần ”modular norm” của bài 48 dù LAMB không đặt vấn đề dưới dạng steepest-descent-dưới-chuẩn một cách hình thức.

8. Ứng dụng cho dự án (suy luận)

(Toàn bộ mục này là suy luận của người ghi chú, không có trong paper, áp cho bài toán nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B, VLM OCR hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1.600-2.100 token hình ảnh, chạy trên GPU ≤ 8GB, batch 1.)

- **Chuẩn khác nhau cho tensor khác nhau theo vai trò**: ý tưởng cốt lõi của Epilogue (modular norm, gán chuẩn theo vai trò tensor) gợi ý rằng khi fine-tune/distill lại LightOnOCR-3-0.8B sau khi pruning, có thể không cần dùng cùng một optimizer config (learning rate, weight decay) cho mọi loại lớp. Ví dụ: patch embedding của nhánh vision (ánh xạ từ patch ảnh, tương tự ”one-hot-ish” input như embedding token) có thể hưởng lợi từ một chuẩn kiểu $\ell_1 \to$RMS (như khuyến nghị cho embedding layer), còn các ma trận chiếu Q/K/V của full-attention layer (phần xử lý token text, ít patch, nhạy với spectral norm vì liên quan trực tiếp tới attention score) có thể phù hợp hơn với một update kiểu ”spectral-aware” (Shampoo-flavored) để ổn định hơn khi fine-tune với rất ít dữ liệu/epoch.
- **Newton-Schulz orthogonalization giá rẻ cho GPU nhỏ**: do dự án giới hạn GPU ≤ 8GB, batch 1, Shampoo đầy đủ (tích luỹ L_t, R_t và nghịch đảo căn bậc 4 ma trận) tốn bộ nhớ/tính toán không phù hợp. Phép lặp Newton-Schulz ở Appendix A ($X_{t+1} = \frac{3}{2}X_t - \frac{1}{2}X_t X_t^T X_t$, chỉ cần vài phép nhân ma trận, không cần SVD hay nghịch đảo tường minh) có thể là cách khả thi để thử một ”spectral-norm-aware update” (không tích luỹ, giống Story II) ngay trên các ma trận chiếu attention của LightOnOCR-3-0.8B khi fine-tune post-pruning, với chi phí bộ nhớ thấp hơn SVD đầy đủ.
- **Tách hướng và bước đi khi phục hồi sau pruning**: Proposition 1 tách biệt rõ ”chọn chuẩn (hướng)” và ”chọn sharpness (bước đi)”. Khi phục hồi mô hình sau khi cắt head/channel (pruning), các lớp vừa bị cắt có gradient landscape khác (có thể sắc hơn) so với lớp chưa chạm tới; suy luận: có thể cần λ (tương đương nghịch đảo learning rate hiệu dụng) khác nhau cho các lớp mới bị pruning so với lớp giữ nguyên, tương tự cách modular norm cho phép hệ số s_l khác nhau từng lớp, thay vì một learning rate toàn cục như cấu hình optimizer thông thường.
- **Liên hệ với Lion (opt 11) cho fine-tune OCR**: vì Story I cho thấy Adam-tắt-EMA chính là sign descent theo từng lớp, và Lion về bản chất cũng là sign-based update, suy luận rằng Lion (tiết kiệm bộ nhớ hơn Adam vì không cần lưu v_t) có thể là lựa chọn hợp lý để thử khi fine-tune LightOnOCR-3-0.8B trên GPU 8GB, với kỳ vọng hành vi tương tự về ”hướng” cập nhật như phân tích chuẩn ℓ_∞/max-of-max ở Story I, dù chưa có số liệu kiểm chứng riêng cho bài toán OCR.

9. Thí nghiệm gợi ý

(5 thí nghiệm nhỏ, chạy được trên 1 GPU 8GB, không cần huấn luyện LLM/VLM đầy đủ.)

1. **So sánh trực quan hướng cập nhật theo chuẩn**: trên một MLP nhỏ (2-3 lớp, vài nghìn tham số, ví dụ hồi quy tổng hợp hoặc MNIST thu nhỏ), cài cả ba update steepest descent (Euclid/SGD, ℓ_∞/sign, phổ qua Newton-Schulz không tích luỹ) theo đúng Proposition 1/2/5, và vẽ đường loss theo epoch cho cùng một mô hình/khởi tạo, nhằm quan sát trực tiếp hiệu ứng ”đổi chuẩn đổi hướng” như minh hoạ tự tính ở

Mục 3.1, nhưng trên một mạng thật thay vì một vector 2 chiều.

2. **Benchmark Newton-Schulz vs SVD trên kích thước ma trận thật của dự án**: lấy kích thước ma trận chiếu Q/K/V hoặc FFN của LightOnOCR-3-0.8B (ví dụ một ma trận vài trăm x vài trăm hoặc lớn hơn tuỳ kiến trúc thật), tạo ma trận ngẫu nhiên cùng kích thước, đo thời gian và bộ nhớ của phép lặp Newton-Schulz (Phương trình 36, chạy vài chục bước) so với gọi `torch.linalg.svd` trực tiếp, trên GPU 8GB, để kiểm chứng tính khả thi thực tế của gợi ý ở Mục 8.
3. **Ablation EMA on/off trên mạng nhỏ**: trên cùng mạng nhỏ ở thí nghiệm 1, so sánh Adam chuẩn ($\beta_1 = 0.9, \beta_2 = 0.999$) với Adam tắt EMA ($\beta_1 = \beta_2 = 0$, tức sign descent thuần) trên cùng bài toán, đo tốc độ hội tụ và độ ổn định, để có cảm nhận thực nghiệm (dù quy mô rất nhỏ) về câu hỏi mở ”vai trò của EMA” mà paper tự nhận chưa giải quyết.
4. **Kiểm tra quy tắc ”escape velocity” của Prodigy trên hàm lồi đồ chơi**: cài quy tắc $\eta_{t+1} = \max(\eta_t, g_t^T(w_0 - w_t)/\|g_t\|_1)$ trên một hàm bậc hai lồi đơn giản (ví dụ $L(w) = \frac{1}{2}w^T A w$ với A xác định dương nhỏ), so sánh đường đi của η_t và tốc độ hội tụ với một SGD learning-rate-cố-định đã dò tối ưu bằng tay, để kiểm tra trực tiếp xem cơ chế ”tăng gấp đôi tới khi thoát vùng tuyến tính hoá” có thực sự tiến gần learning rate tối ưu hay không trên một bài toán nhỏ, dễ kiểm soát.
5. **Thử gán hệ số s_l theo vai trò lớp trên một block GatedDeltaNet/attention nhỏ**: lấy một block nhỏ tương tự kiến trúc hybrid của LightOnOCR (một lớp GatedDeltaNet + một lớp full attention), gán hệ số s_l lớn hơn cho patch-embedding/projection đầu vào so với các lớp attention nội bộ (theo tinh thần modular norm ở Mục 3.5), rồi so sánh đường loss khi fine-tune trên một tập nhỏ dữ liệu OCR tổng hợp, đối chiếu với baseline dùng Adam cấu hình đồng nhất (cùng learning rate mọi lớp), nhằm kiểm tra sơ bộ (suy luận, quy mô rất nhỏ) liệu ý tưởng ”chuẩn theo vai trò” có mang lại lợi ích thực tế cho đúng kiến trúc của dự án hay không.

10. Tóm tắt công thức / khái niệm

- **Steepest descent dưới chuẩn (Proposition 1):** $\arg\min_{\Delta w} g^T\Delta w + \frac{\lambda}{2}\|\Delta w\|^2 = -\frac{\|g\|_*}{\lambda}\arg\max_{\|t\|=1} g^T t$. Tách bài toán thành độ lớn ($\|g\|_*/\lambda$) và hướng (argmax tích trong trên mặt cầu đơn vị của chuẩn). Đã tự tính ở Mục 3.1 với $g = (3, 4)$, $\lambda = 2$: chuẩn ℓ_2 cho $\Delta w = (-1.5, -2.0)$, chuẩn ℓ_∞ cho $\Delta w = (-3.5, -3.5)$.
- **Chuẩn toán tử cảm sinh (Definition 1):** $\|M\|_{\alpha\to\beta} = \max_{x\neq 0}\|Mx\|_\beta/\|x\|_\alpha$. Hai công thức đóng hữu ích (Proposition 8): $\|M\|_{\ell_1\to\ell_p} = \max_j \|\text{col}_j(M)\|_p$; $\|M\|_{\ell_p\to\ell_\infty} = \max_i \|\text{row}_i(M)\|_{p/(p-1)}$.
- **Bảng tổng hợp ba cặp (chuẩn, optimizer)** (tái hiện từ Table 1 của paper):

Không gian	Chuẩn	Nghiệm Δ	Optimizer tương ứng
$\mathbb{R}^n$	Euclid ℓ_2	$\Delta w = -\|g\|_2\, g/(\lambda\|g\|_2) = -g/\lambda$	vanilla GD / SGD
$\mathbb{R}^n$	vô cùng ℓ_∞	$\Delta w = -(\|g\|_1/\lambda)\,\text{sign}(g)$	sign descent / Adam (tắt EMA)
$\mathbb{R}^{m\times n}$	Frobenius S_2	$\Delta W = -\|G\|_F\, G/(\lambda\|G\|_F) = -G/\lambda$	vanilla GD / SGD (ma trận)
$\mathbb{R}^{m\times n}$	phổ S_∞	$\Delta W = -(\text{tr}\,\Sigma/\lambda)\,UV^T$	spectral descent / Shampoo (tắt tích luỹ)

- **Max-of-max norm (Proposition 3):** $\|w\|_\infty = \max_l \|W_l\|_{\ell_1\to\ell_\infty}$ trên vector phẳng; nghiệm steepest descent là sign descent từng lớp với bước đi toàn cục $\eta = \frac{1}{\lambda}\sum_l \|G_l\|^*_{\ell_1\to\ell_\infty}$.
- **Chuẩn modular (Proposition 7):** $\max\{s_1\|W_1\|_1, \ldots, s_L\|W_L\|_L\}$; nghiệm cho mỗi lớp $\Delta W_l = -\frac{\eta}{s_l}\arg\max_{\|T_l\|_l=1}\langle G_l, T_l\rangle$ với $\eta = \frac{1}{\lambda}\sum_k \frac{1}{s_k}\|G_k\|^*_k$ chung cho mọi lớp.
- **Cận loss majorization-minimization (Proposition 6):** $L(W+\Delta W) \le L(W) + \langle\nabla_W L(W), \Delta W\rangle + \frac{1}{2}\cdot\frac{d_{out}}{d_{in}}\cdot\|\Delta W\|^2_{\ell_2\to\ell_2}$ cho mô hình tuyến tính, hàm mất bình phương, dữ liệu chuẩn hoá $\|x_i\|_2 = \sqrt{d_{in}}$.
- **Escape velocity (Prodigy, Story III):** $\eta_{t+1} = \max(\eta_t, g_t^T(w_0 - w_t)/\|g_t\|_1)$, xấp xỉ $\approx \max(\eta_t, \|w_t -$

$w_0\|_{RMS}$) dưới hai giả định (gradient dày, w_t gần w_0); cùng chuẩn ℓ_∞ như Adam, khác ở cách chọn η_t (tự động, không cố định).

11. Kết luận ngắn

Paper (workshop OPT2024) đóng góp một lăng kính diễn giải, không phải một optimizer mới: chứng minh bằng đại số rằng Adam, Shampoo, Prodigy, khi tắt EMA, đều là nghiệm của đúng một bài toán steepest descent dưới một chuẩn toán tử cụ thể (lần lượt là ℓ_∞/max-of-max, phổ, và ℓ_∞ với bước đi tự động), và tổng quát hoá thành khung ”chuẩn modular” cho phép gán chuẩn khác nhau theo vai trò từng tensor trong mạng. Giá trị chính là tư duy thiết kế (tách hướng và bước đi, chọn chuẩn có chủ đích), không phải số liệu thực nghiệm (bài không có benchmark huấn luyện riêng, và tự nhận vai trò EMA là câu hỏi mở). Với dự án nén/hậu huấn luyện LightOnOCR-3-0.8B, ý tưởng hữu ích nhất để mang sang là khả năng gán cấu hình optimizer khác nhau theo vai trò lớp (embedding/patch-embedding khác attention/FFN) và dùng Newton-Schulz orthogonalization như một cách rẻ để thử ý tưởng ”spectral-aware update” trên GPU nhỏ, cả hai đều ở mức suy luận cần kiểm chứng thêm, không phải kết luận có sẵn từ paper.

Nguồn gốc: https://arxiv.org/abs/2409.20325

Bài 49

Gradient Descent Happens in a Tiny Subspace (2018)

Tác giả: Guy Gur-Ari (Institute for Advanced Study, Princeton, đồng tác giả chính), Daniel A. Roberts (Facebook AI Research, đồng tác giả chính), Ethan Dyer (Johns Hopkins University).
Venue / arXiv: arXiv:1812.04754 (v1, 12/12/2018); paper không nêu venue hội nghị/tạp chí nào trong toàn văn đọc được (chỉ có header arXiv, không có dòng ”Published as...” hay tên hội nghị).
Nguồn: https://arxiv.org/abs/1812.04754
Chủ đề: quan sát thực nghiệm (trên nhiều kiến trúc/bộ dữ liệu) rằng gradient của hàm mất trong deep learning hội tụ nhanh vào một không gian con rất nhỏ (”tiny subspace”), trương bởi k eigenvector lớn nhất của ma trận Hessian (với k = số lớp phân loại), và không gian con này được bảo toàn qua thời gian dài huấn luyện; kèm một mô hình đồ chơi giải được chính xác để minh hoạ cơ chế.
Vị trí trong lộ trình: đọc sau opt 48 (Old Optimizer, New Norm) vì cùng mạch ”hiểu hình học không gian trọng số qua ma trận/chuẩn toán tử”, nhưng bài 48 nhìn qua lăng kính chuẩn (norm) còn bài 49 nhìn trực tiếp qua phổ Hessian; đọc sau opt 06 (Adam) và opt 18 (SGDR) để có nền về SGD/Adam làm optimizer được khảo sát. Đọc trước các bài về weight decay (opt 50, opt 51) vì hiểu ”không gian con top-Hessian nhỏ, phần còn lại rất phẳng” giúp hình dung tại sao weight decay/L2 lại tác động mạnh tới một phần nhỏ chiều không gian trọng số.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu, định lý, công thức lấy từ toàn văn arXiv:1812.04754 (bản v1, tải và đọc toàn bộ ngày 09/11/2026, gồm phần chính và các Phụ lục A-E). Các phép tính đánh dấu ”tự tính” là của người ghi chú, không có trong paper. Paper này có cả thực nghiệm thật (MNIST/CIFAR10/CIFAR100, nhiều kiến trúc) và một mô hình đồ chơi (toy model) giải tích được, Mục 4 dưới đây trình bày đầy đủ cả hai loại, không chỉ lý thuyết thuần.

1. Bối cảnh

Theo Mục 1 của paper, stochastic gradient descent (SGD) và các biến thể được dùng để huấn luyện hầu như mọi mô hình deep learning quy mô lớn, nhưng việc SGD hoạt động tốt trên một loss landscape phi tuyến, không lồi (nonconvex) vẫn còn ”có phần là một điều bí ẩn” (nguyên văn: ”its success remains somewhat of a mystery”). Paper xuất phát từ quan sát lý thuyết: trong các mô hình ”overparameterized” (nhiều tham số hơn hẳn số mẫu huấn luyện), người ta kỳ vọng loss landscape có nhiều ”hướng phẳng” (flat directions): hướng trong không gian tham số mà loss thay đổi rất ít hoặc không đổi. Một cách đo độ phẳng cục bộ là qua phổ (spectrum) của ma trận Hessian: eigenvalue nhỏ hoặc gần 0 ứng với hướng phẳng (Hochreiter và Schmidhuber, 1997, về ”flat minima”).

Công trình trước đó của Sagun và cộng sự (2016; 2017) đã phân tích phổ Hessian cho các hàm mất cross-entropy trong deep learning và phát hiện bằng thực nghiệm rằng phổ tách thành hai phần: một phần ”bulk” (số lượng lớn eigenvalue nhỏ, gần 0) và một phần ”top” (ít eigenvalue lớn hơn hẳn, dương). Số chiều của phần ”top” đúng bằng k, số lớp (classes) trong bài toán phân loại. Đây chính là điểm khởi đầu của paper 49: nếu không gian tangent tại mỗi điểm trong quá trình huấn luyện có hai phần trực giao (top và bulk) như vậy, thì gradient đi theo quỹ đạo nào trong hai phần này?

Bối cảnh rộng hơn: paper nằm trong một mạch nghiên cứu cuối 2016-2018 cố gắng dùng hình học của Hessian (saddle point, độ cong, phổ eigenvalue) để lý giải vì sao deep learning huấn luyện tốt dù loss landscape không lồi (Pascanu và cộng sự 2014; Dauphin và cộng sự 2014 về saddle point; Chaudhari và cộng sự 2016, Entropy-SGD). Bài 48 trong lộ trình (Old Optimizer, New Norm) cũng dùng Hessian/chuẩn toán tử để lý giải optimizer, nhưng theo hướng ”chọn chuẩn cho bước cập nhật”; bài 49 đi theo hướng khác và bổ sung: mô tả trực tiếp hình học phổ Hessian và vị trí gradient trong phổ đó suốt quá trình huấn luyện thật, không giả định một optimizer cụ thể.

2. Ý tưởng chính

Paper nêu rõ hai quan sát thực nghiệm mới (nguyên văn gọi là ”two novel observations”):

1. **Gradient hội tụ nhanh vào top subspace**: trong quá trình huấn luyện, gradient của loss nhanh chóng dịch chuyển để nằm (gần như hoàn toàn) trong không gian con ”top” của Hessian (trương bởi k eigenvector có eigenvalue lớn nhất). Trong không gian con này, hướng của gradient dường như không có cấu trúc đặc biệt, ”ngẫu nhiên” so với hệ cơ sở eigenvector.
2. **Top subspace được bảo toàn qua thời gian dài**: các eigenvector ”top” của Hessian tiến triển không tầm thường theo thời gian (không cố định tuyệt đối), nhưng có xu hướng **không trộn lẫn** với các eigenvector ”bulk”, dù qua hàng trăm bước huấn luyện hoặc hơn. Nói cách khác, không gian con ”top” là một đối tượng xấp xỉ bất biến, tiến triển rất chậm.

Kết hợp hai quan sát: dù mô hình deep learning có số tham số khổng lồ, việc học dường như diễn ra chủ yếu trong một không gian con rất nhỏ, tiến triển chậm. Paper đưa ra lập luận đơn giản để thấy vì sao điều này có ý nghĩa cho optimization: với một bước gradient descent $-\eta g$, độ giảm loss ở xấp xỉ bậc một là $\delta L = -\eta \|g\|^2$. Nếu g hầu như nằm trong top subspace (gọi phần chiếu là g_{top}), thì làm gradient descent chỉ với g_{top} cũng cho mức giảm loss tương tự (miễn xấp xỉ tuyến tính còn đúng). Paper tự nhận đây chỉ là ”a simple argument [that] suggests” (một lập luận đơn giản gợi ý), không phải chứng minh chặt cho mạng neural tổng quát; ngôn ngữ trong abstract (”may happen”, ”suggests”) thể hiện rõ mức độ dè dặt này.

Hai quan sát này được kiểm chứng thực nghiệm trên nhiều kiến trúc (mạng fully-connected, convolutional network, ResNet-18) và bộ dữ liệu (MNIST, CIFAR10, CIFAR100, hồi quy tổng hợp), bền vững qua nhiều biến thể cấu hình (SGD và Adam, nhiều learning rate/batch size, có/không dropout và batch-norm, activation trơn (softplus) thay ReLU, và cả trường hợp gán nhãn ngẫu nhiên hoàn toàn, tức chế độ ”ghi nhớ” theo Zhang và cộng sự 2016). Paper cũng xây một mô hình đồ chơi (softmax regression trên hỗn hợp Gaussian) tái tạo được đầy đủ các hiệu ứng trên dưới dạng giải tích chính xác, dùng để hiểu cơ chế, không nhằm là lời giải thích dứt điểm cho deep learning tổng quát.

3. Phương pháp chi tiết

3.1. Định nghĩa và độ đo chính

Mô hình có p tham số θ, hàm mất cross-entropy $L(\theta)$, gradient $g(\theta) = \nabla L(\theta)$, Hessian $H(\theta) = \nabla\nabla^T L(\theta)$. Cập nhật gradient descent với learning rate η ở bước t: $\theta^{(t+1)} = \theta^{(t)} - \eta\, g(\theta^{(t)})$.

Gọi V_{top} là không gian con trương bởi k eigenvector lớn nhất của Hessian (k = số lớp), V_{bulk} là không gian con trực giao (phần còn lại). Gradient tách thành $g = g_{top} + g_{bulk}$. **Tỉ lệ gradient trong top subspace** (Phương trình 2):

$$f_{top} \equiv \frac{\|g_{top}\|^2}{\|g\|^2}.$$

Vì tính trực tiếp k eigenvector tốn kém (cần phân tích phổ của ma trận $p \times p$), paper dùng hai công cụ để đo rẻ hơn, đều dựa trên phép nhân Hessian-vector Hv (không cần dựng tường minh ma trận Hessian $p \times p$): tính $a = \nabla L^T v$ (một đại lượng vô hướng) rồi lấy gradient của a theo θ, cho đúng $\nabla a = Hv$ (Phụ lục A, kỹ thuật autodiff ”double backprop” chuẩn). Từ đó paper dùng phương pháp Lanczos (Lehoucq và cộng sự, 1998) để lấy các eigenvector lớn nhất một cách lặp, chỉ cần nhiều lần tính Hv.

Overlap gradient-Hessian (Phương trình 3), một proxy rẻ hơn nữa (chỉ cần một lần tính Hg):

$$\text{overlap}(g, Hg) \equiv \frac{g^T H g}{\|g\| \cdot \|Hg\|} \in [-1, 1].$$

Paper lập luận và kiểm chứng: overlap lớn (gần 1) là dấu hiệu gradient nằm chủ yếu trong top subspace. Để có một ước lượng định lượng, paper xét một vector ngẫu nhiên w giới hạn trong top subspace k chiều, với hệ số $w_i = \pm 1$ chọn ngẫu nhiên ở một đỉnh của hình hộp đơn vị (theo cơ sở eigenvector), và tính (Phương trình 4):

$$\text{overlap}(w, Hw) = \frac{\sum_i \lambda_i w_i^2}{\sqrt{\left(\sum_j w_j^2\right)\left(\sum_l \lambda_l^2 w_l^2\right)}} = \frac{\sum_i \lambda_i}{\sqrt{k \sum_j \lambda_j^2}}$$

(đẳng thức thứ hai dùng $w_i^2 = 1$). Lấy xấp xỉ thô $\lambda_i \propto i$ (quan sát thực nghiệm thô cho $k = 10$) và giới hạn k lớn (liên tục hoá tổng), paper ước lượng giá trị này $\approx \sqrt{3/4} \approx 0.87$.

(Tự tính) kiểm lại bằng tổng hữu hạn với $k = 10$, $\lambda_i = i$: $\sum_{i=1}^{10} i = 55$; $\sum_{i=1}^{10} i^2 = 385$. Vậy overlap $= 55/\sqrt{10 \times 385} = 55/\sqrt{3850} \approx 55/62{,}05 \approx 0{,}886$. Giá trị tự tính hữu hạn ($\approx 0{,}886$) rất gần giá trị xấp xỉ liên tục mà paper nêu ($\sqrt{3/4} \approx 0{,}866$), chỉ lệch khoảng $0{,}02$, xác nhận công thức của paper đúng và hội tụ nhanh về giới hạn liên tục ngay ở $k = 10$.

(Tự tính) minh hoạ hai trường hợp biên của overlap: lấy $H = \text{diag}(2, 0)$ (một chiều "top" dương, một chiều "bulk" hoàn toàn phẳng). Nếu $g = (1, 0)$ (gradient nằm hoàn toàn trong chiều top): $Hg = (2, 0)$, overlap $= \frac{g^T Hg}{\|g\|\|Hg\|} = \frac{2}{1 \times 2} = 1$, đúng giá trị biên trên, khớp trực giác "gradient hoàn toàn trong top subspace cho overlap bằng 1". Nếu $g = (0, 1)$ (gradient nằm hoàn toàn trong chiều bulk/phẳng): $Hg = (0, 0)$, mẫu số bằng 0, overlap không xác định; đây khớp với giải thích của paper rằng hướng hoàn toàn phẳng có gradient triệt tiêu đúng bằng 0 do một đối xứng tịnh tiến của loss (Mục 3.5 dưới), nên tình huống "gradient khác 0 nhưng thuần bulk" không xảy ra trong thực tế.

3.2. Theo dõi sự tiến triển của top subspace theo thời gian

Gọi $P_{top}^{(t)}$ là phép chiếu trực giao lên $V_{top}^{(t)}$ tại bước t (với $\left(P_{top}^{(t)}\right)^2 = P_{top}^{(t)}$). Paper định nghĩa overlap giữa hai không gian con ở hai thời điểm $t < t'$ (Phương trình 5):

$$\text{overlap}\left(V_{top}^{(t)}, V_{top}^{(t')}\right) \equiv \frac{\text{Tr}\left(P_{top}^{(t)} P_{top}^{(t')}\right)}{\sqrt{\text{Tr}(P_{top}^{(t)})\,\text{Tr}(P_{top}^{(t')})}} = \frac{1}{k} \sum_{i=1}^{k} \left\| P_{top}^{(t)} v_i^{(t')} \right\|^2.$$

Giá trị bằng 1 nếu hai không gian con trùng nhau, bằng 0 nếu trực giao hoàn toàn. Paper quan sát (Hình 2): với $t_1 \approx 100$ bước, overlap giữa $V_{top}^{(t_1)}$ và $V_{top}^{(t_2)}$ vẫn lớn dù $t_2 - t_1 \gg t_1$ (hàng trăm bước sau), trong khi không gian con trương bởi k eigenvector **kế tiếp** (hạng $k{+}1$ đến $2k$) thì KHÔNG có tính chất này: overlap của nó suy giảm nhanh ngay cả với t_1 lớn. Điều này cho thấy tính "đông cứng" (preserved) không phải là đặc điểm chung của mọi không gian con chậm biến đổi, mà đặc trưng riêng cho đúng k chiều "top".

Phụ lục C mở rộng: quét số chiều subspace từ 2 đến 20 (Hình 3, 9, 10), thấy overlap trung bình giữ khá bằng phẳng (ổn định) tới số chiều $d = 9$, rồi **giảm dần đơn điệu** từ $d = 10$ trở đi (với bài toán $k = 10$ lớp). Đây là bằng chứng cho việc có một "đặc điểm" ở đúng số chiều bằng số lớp. Tuy nhiên paper tự ghi chú (chú thích 9) rằng việc xác định chính xác điểm chuyển tiếp là k hay $k - 1$ là "không thể kết luận chắc chắn từ phổ" (inconclusive), dù "khá rõ nét" trong Hình 3, đây là một điểm paper tự nhận không chắc.

Phụ lục C cũng cho thêm bằng chứng số chiều k gắn với **bài toán phân loại** (nhãn), không phải gắn với **cấu trúc nội tại của dữ liệu**: gán lại nhãn MNIST theo chẵn/lẻ (2 lớp, dù dữ liệu vẫn có 10 cụm chữ số) thì chỉ còn 2 eigenvalue lớn, không phải 10; còn khi hoán vị ngẫu nhiên toàn bộ nhãn (phá tương quan input-label, mạng chỉ "ghi nhớ" theo nghĩa Zhang và cộng sự 2016) thì vẫn có đúng 10 eigenvalue lớn (vì bài toán vẫn là phân loại 10 lớp về mặt cấu trúc, dù không có tín hiệu thật).

3.3. Mô hình đồ chơi giải tích (toy model)

Để hiểu cơ chế, paper xây một bài toán phân loại 2 lớp, n mẫu $x_a \in \mathbb{R}^d$ lấy từ hỗn hợp hai phân phối Gaussian $\mathcal{N}(\mu_1, \sigma^2 I)$ và $\mathcal{N}(\mu_2, \sigma^2 I)$, huấn luyện bằng softmax regression với tham số $\theta_{y,i}$ ($y = 1, 2$; $i = 1, \ldots, d$), hàm mất cross-entropy (Phương trình 6). Hai giả định đơn giản hoá: (a) giới hạn $\sigma^2 \to 0$ (mẫu co về đúng μ_1, μ_2, bài toán trở thành học từ 2 điểm mẫu); (b) $\mu_1 \perp \mu_2$ (xấp xỉ tốt khi d lớn, vì hai vector đơn vị ngẫu nhiên trên mặt cầu S^{d-1} có tích trong kỳ vọng $\approx d^{-1/2}$).

Dưới hai giả định này, loss có $2d - 2$ hướng phẳng hoàn toàn (flat), nên Hessian có hạng (rank) đúng bằng 2: không gian con top đúng 2 chiều, không gian bulk là hạt nhân (kernel) của Hessian (toàn bộ eigenvalue bằng 0 chính xác, không gần 0), và gradient luôn nằm **chính xác** (không chỉ gần đúng) trong top subspace 2 chiều này.

Giải phương trình gradient descent ở giới hạn thời gian liên tục, nghiệm thời điểm muộn (Phương trình 7-9, c_1, c_2 là hằng số dương tuỳ ý, $\tilde{\theta}_{1,2}, \tilde{\theta}_0$ là các vector tuỳ ý trực giao với $\mu_{1,2}$):

$$\theta_{1,2}(t) = \tilde{\theta}_{1,2} + \tilde{\theta}_0 \pm \frac{\mu_1}{2} \log(\eta t + c_1) \mp \frac{\mu_2}{2} \log(\eta t + c_2),$$

$$g_{\theta_1}(t) = \frac{2(\mu_2 - \mu_1)}{\eta t} + O(t^{-2}), \qquad g_{\theta_1}(t) = -g_{\theta_2}(t),$$

$$H(t) = \frac{1}{2\eta t} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} \otimes \left(\mu_1 \mu_1^T + \mu_2 \mu_2^T\right) + O(t^{-2}).$$

Hessian có đúng hai eigenvalue dương (top subspace), bằng nhau, cùng bằng $(\eta t)^{-1}$ (suy biến/degenerate trong mô hình đối xứng đơn giản này); các eigenvalue còn lại triệt tiêu đúng bằng 0. Top subspace không đổi theo thời gian (trương bởi hai vector cố định $(\mu_1, -\mu_1)$ và $(\mu_2, -\mu_2)$), còn gradient tiến triển theo t (giảm dần như $1/t$) nhưng luôn nằm chính xác trong 2 chiều này.

Paper kiểm chứng thêm bằng thực nghiệm số (không chứng minh giải tích đầy đủ trong bài chính, hẹn công bố ở Kirilin và cộng sự, ”forthcoming”):

- Thêm nhiễu mẫu nhỏ ($\sigma^2 > 0$ nhỏ): phần bulk có eigenvalue nhỏ khác 0 (bị ”dập” theo σ^2), nhưng gradient vẫn chảy vào top subspace như trước, tức nhiễu chỉ là nhiễu loạn nhỏ, không đổi kết luận định tính.
- Thêm bias vào tham số: phá vỡ tính suy biến của hai eigenvalue trong top subspace; gradient trong quá trình huấn luyện trở nên **thẳng hàng với eigenvector có eigenvalue NHỎ HƠN** trong hai eigenvalue đã tách biệt (một kết quả khá ngược trực giác, giải thích ở Mục 3.4 dưới).
- Tổng quát hoá cho $k > 2$ lớp (hỗn hợp Gaussian k thành phần): top subspace trở thành k chiều với k eigenvalue dương, khớp đúng với quan sát thực nghiệm chính của paper (k = số lớp); nếu phá suy biến bằng bias, có $k - 1$ eigenvalue lớn và một eigenvalue nhỏ hơn mà gradient thẳng hàng theo.

3.4. Vì sao gradient thẳng hàng với eigenvalue nhỏ hơn trong top subspace: lập luận tuyến tính hoá

Dưới một bước gradient descent, khai triển bậc một theo η (Phương trình 10):

$$g^{(t+1)} = g\left(\theta^{(t)} - \eta g^{(t)}\right) = \left(I - \eta H^{(t)}\right) g^{(t)} + O(\eta^2).$$

Nếu xấp xỉ tuyến tính này còn đúng và η đủ nhỏ, lặp lại phép biến đổi này đẩy g về phía eigenvector của H có eigenvalue **dương nhỏ nhất** (trong không gian top, nơi mọi eigenvalue dương, khác không gian bulk hoàn toàn bằng 0). Lý do: thành phần gradient theo hướng eigenvalue λ bị nhân với hệ số co $(1 - \eta\lambda)$ mỗi bước; với $0 < \eta\lambda < 2$ đây là một phép co, và hệ số càng gần 1 (co chậm hơn, ”sống sót” lâu hơn) khi λ càng nhỏ.

(Tự tính) minh hoạ bằng số: lấy $\lambda_1 = 1$, $\lambda_2 = 4$, $\eta = 0{,}1$. Hệ số co mỗi bước: $(1 - \eta\lambda_1) = 0{,}9$ và $(1 - \eta\lambda_2) = 0{,}6$. Sau 10 bước: $0{,}9^{10} \approx 0{,}349$ và $0{,}6^{10} \approx 0{,}006$. Tỉ lệ giữa hai thành phần còn lại: $0{,}349/0{,}006 \approx 58$ lần, dù ban đầu hai thành phần bằng nhau. Con số tự tính này xác nhận trực giác của paper: thành phần gradient dọc theo eigenvalue **nhỏ hơn (nhưng khác 0)** sống sót lâu hơn nhiều và dần áp đảo, đúng như cơ chế giải thích việc gradient ”chọn” eigenvector có eigenvalue nhỏ hơn trong top subspace khi tính suy biến bị phá vỡ (ví dụ do thêm bias).

Paper đặc biệt cảnh báo không nên áp dụng ngây thơ lập luận này cho **toàn bộ** phổ Hessian (nếu áp dụng máy móc sẽ dẫn tới kết luận sai là gradient phải trôi vào bulk, vì bulk có eigenvalue còn nhỏ hơn/bằng 0). Hai lý do paper đưa ra: (1) các hướng bulk là hướng **phẳng tuyệt đối** (eigenvalue bằng 0 chính xác, do một đối xứng tịnh tiến của loss trong các hướng đó), nên gradient đã triệt tiêu đúng bằng 0 ở đó từ đầu, không có động lực nào ”đẩy” một đại lượng đã bằng 0 đi đâu cả; (2) để tin vào kết luận ”gradient hội tụ về eigenvalue nhỏ nhất”, cần tin xấp xỉ tuyến tính (Phương trình 10) đúng ở thời điểm muộn, nhưng không có lý do để giả định các hiệu chỉnh bậc cao không trở nên quan trọng trên toàn phổ. Paper cũng ghi chú cơ chế này là **chính xác** (không chỉ xấp xỉ) cho trường hợp đặc biệt hồi quy tuyến tính với hàm mất bình phương (Hessian hằng số, không có hiệu chỉnh bậc cao).

3.5. Ứng dụng gợi ý: Newton’s method rút gọn hạng thấp

Phần Discussion (Mục 4 của paper) nêu một gợi ý thực hành (không kiểm chứng thực nghiệm trong bài): phương pháp Newton cổ điển $\theta^{(t+1)} = \theta^{(t)} - H^{-1}g^{(t)}$ có hai chỉ trích truyền thống: (a) nghịch đảo Hessian $p \times p$ bất khả thi về tính toán cho mạng neural lớn, và vì Hessian rất suy biến (nhiều eigenvalue gần 0) nên phép nghịch đảo không được định nghĩa tốt; (b) Newton’s method không ”descend” (giảm) một cách chặt chẽ, mà di chuyển tới bất kỳ điểm tới hạn nào (cực tiểu, cực đại, hay điểm yên ngựa).

Vì gradient sống chủ yếu trong top subspace nhỏ, có điều kiện tốt (mọi eigenvalue dương), paper đề xuất giới hạn bước Newton vào đúng không gian con này (Phương trình 11):

$$\theta^{(t+1)} = \theta^{(t)} - \left(H_{top}^{(t)}\right)^{-1} g_{top}^{(t)}.$$

Nghịch đảo một ma trận $k \times k$ (hoặc hạng thấp) được định nghĩa tốt và tính toán rẻ; vì mọi eigenvalue trong top subspace dương chặt, bước này đảm bảo giảm loss (không đi tới điểm yên ngựa). Paper gọi đây là ”vài bình luận sơ bộ” (preliminary comments) và để việc đánh giá thực sự ý tưởng này cho công trình tương lai, **không** triển khai hay benchmark trong bài này.

4. Thí nghiệm và kết quả chính

Khác với paper lý thuyết thuần (như opt 48), bài này có cả thực nghiệm thật và một mô hình đồ chơi giải tích, tách bạch rõ: **không có một optimizer/thuật toán mới nào được đề xuất và benchmark** (Newton rút gọn ở Mục 3.5 chỉ là gợi ý, chưa cài đặt), mà là các đo đạc chẩn đoán (diagnostic measurements) trên mô hình huấn luyện bình thường.

- **Hình 1** (phần chính): f_{top} (Phương trình 2) đo trong giai đoạn đầu huấn luyện cho ba cấu hình: (a,b) mạng fully-connected 2 lớp ẩn, 100 neuron mỗi lớp, trên MNIST, SGD batch 64, $\eta = 0{,}1$; (c,d) mạng convolutional nhỏ (theo Chollet và cộng sự, 2015, Keras) trên CIFAR10, cùng optimizer; (e,f) ResNet-18 (He và cộng sự, 2016) trên CIFAR10. Cả ba đều cho thấy f_{top} bắt đầu nhỏ, sau đó tăng nhanh lên gần 1 trong giai đoạn đầu huấn luyện.
- **Bảng 1** (số liệu overlap trung bình trên 1000 bước cuối, batch 64, $\eta = 0{,}1$ trừ khi ghi khác, huấn luyện vài epoch), tái hiện đầy đủ:

Dataset	Model	Ghi chú	Overlap trung bình
MNIST	Softmax	(mặc định)	0,96
MNIST	FC	Activation softplus	0,96
MNIST	FC	$\eta = 0{,}01$	0,96
MNIST	FC	Batch size 256	0,97
MNIST	FC	Nhãn ngẫu nhiên	0,86
CIFAR10	ConvNet	Nhãn ngẫu nhiên	0,86
CIFAR10	ConvNet	Dropout, batch-norm, thêm 1 lớp dense	0,93
CIFAR10	ConvNet	Dùng Adam	0,89
Regression	FC	Batch size 100	0,99

Toàn bộ giá trị đều cao (0,86 đến 0,99), kể cả với nhãn ngẫu nhiên (chế độ ghi nhớ) và dùng Adam thay SGD.

- **Hình 2**: overlap giữa top subspace ở hai thời điểm khác nhau ($V_{top}^{(t_1)}$ và $V_{top}^{(t_2)}$), cho cả FC-MNIST, ConvNet-CIFAR10, ResNet18-CIFAR10; sau khoảng $t_1 = 100$ bước, overlap vẫn cao dù $t_2 - t_1$ lớn hơn nhiều t_1; trong khi subspace trương bởi k eigenvector **kế tiếp** (hạng 11-20) suy giảm overlap nhanh, không được bảo toàn.
- **Hình 3** (chi tiết ở Phụ lục C): quét số chiều subspace từ 2 đến 20 cho mạng FC-MNIST và ResNet18-CIFAR10, cho thấy ”khúc gãy” (kink) rõ nét ở số chiều gần bằng số lớp (10 cho hai bộ dữ liệu này).
- **Phụ lục B** (Hình 4-8): nghiên cứu sâu một mạng FC nhỏ (2 lớp ẩn, 32 neuron mỗi lớp) huấn luyện 40 epoch trên MNIST, phổ eigenvalue cho thấy rõ 10 eigenvalue lớn tách biệt khỏi phần còn lại (Hình 4). Với mạng FC lớn hơn (256, 256 neuron) trên CIFAR100 (100 lớp), biểu đồ mật độ (trung bình trên 200 lần thực hiện) cho thấy một ”vùng loãng” (underdensity) rõ ở khoảng eigenvalue thứ 100 (Hình 5), khớp số lớp; paper suy đoán phần phổ trong top subspace khớp tốt với phân phối Gaussian, có thể là log-normal, nhưng tự nhận ”chỉ là gợi ý, cần nhiều bằng chứng và giải thích hơn” (một điểm paper tự nhận còn chưa chắc). Kiểm tra giả thuyết ”eigenvector top chỉ là trọng số lớp cuối”: sai, eigenvector lớn nhất có hệ số khác 0 lan toả khắp mọi lớp của mạng, lớp cuối chỉ lớn hơn lớp ẩn đầu tiên khoảng $\sim 4\times$, không áp đảo (Hình 6, 7). Eigenvalue lớn nhất có xu hướng giảm dần (gợi ý $\sim 1/t$ ở cuối huấn luyện, khớp mô hình đồ chơi) nhưng paper tự nhận đồ thị ”quá thô để nói gì cụ thể về dạng hàm suy giảm” (một điểm paper tự nhận còn chưa chắc, Phụ lục B, chú thích 16).
- **Phụ lục C** (Hình 9-11): mở rộng quét số chiều cho FC-MNIST và ResNet18-CIFAR10, xác nhận số chiều đặc biệt $\approx k$ (hoặc $k - 1$, không khẳng định chắc được chiều nào đúng hơn). Phân tích theo từng eigenvector riêng lẻ (Phương trình 12, $c_i^2 = \|v_i \cdot g\|^2 / \|g\|^2$): tại bước 0, $\sum_{i=1}^{20} c_i^2 = 0{,}20$ (gradient chưa tập trung); sau 50 bước, $\sum_{i=1}^{20} c_i^2 = 0{,}94$, cụ thể $\sum_{i=1}^{10} c_i^2 = 0{,}93$ so với $\sum_{i=11}^{20} c_i^2 = 0{,}01$: một sự phân tách rất rõ giữa 10 chiều top và 10 chiều kế tiếp, và cũng cho thấy gradient **không** chỉ là một eigenvector đơn lẻ, mà trải trên cả 10 chiều top.
- **Phụ lục D** (Hình 12): kiểm tra độ bền của f_{top} qua learning rate ($\eta = 0{,}01$ so với $0{,}1$), batch size (256 so với 64), bỏ hết lớp ẩn (chỉ softmax), đổi activation (softplus so với ReLU), nhãn ngẫu nhiên, và đổi optimizer (Adam so với SGD). Mọi trường hợp đều cho f_{top} hội tụ gần 1.
- **Mô hình đồ chơi** (Phụ lục E): không phải thực nghiệm số mà một ví dụ giải được chính xác (Mục 3.3-3.4 trên); paper nói rõ ”không nhằm là lời giải thích dứt điểm” cho hiện tượng, chỉ minh hoạ cơ chế trong một trường hợp đơn giản hoá, kiểm soát được.

5. Phân tích / ablation

Các ”ablation” (thay đổi một thành phần, giữ phần còn lại) xuất hiện rải trong Bảng 1 và Hình 12, không phải một bảng ablation tập trung duy nhất:

- **Learning rate** ($0{,}1$ so với $0{,}01$): f_{top} và overlap không đổi đáng kể (Bảng 1 dòng MNIST FC $\eta = 0{,}01$:

0,96, bằng dòng mặc định).

- **Batch size** (64 so với 256): overlap tăng nhẹ (0,96 $\rightarrow$ 0,97), không giảm.
- **Độ sâu** (bỏ hết lớp ẩn, chỉ softmax, so với FC có lớp ẩn): hiệu ứng vẫn giữ (Bảng 1 dòng "MNIST Softmax" = 0,96, cùng mức với các biến thể FC có lớp ẩn; Hình 12c).
- **Activation** (softplus trơn so với ReLU không trơn): không đổi đáng kể (0,96 so với baseline).
- **Regularization** (thêm dropout + batch-norm + một lớp dense trên ConvNet CIFAR10): overlap 0,93, hơi thấp hơn các dòng MNIST (0,96-0,97) nhưng vẫn cao; paper **không** đưa dòng baseline "ConvNet CIFAR10, không chỉnh gì" với chú thích trống để so sánh trực tiếp mức độ ảnh hưởng của riêng dropout/batch-norm (một khoảng trống dữ liệu, xem Mục 6).
- **Optimizer** (SGD so với Adam, trên ConvNet CIFAR10): overlap 0,89 khi dùng Adam, vẫn cao, cho thấy hiệu ứng không đặc thù riêng cho SGD.
- **Nhãn** (nhãn thật so với nhãn ngẫu nhiên hoàn toàn): overlap giữ ở mức cao tương đương (0,86 cho cả MNIST FC và CIFAR10 ConvNet với nhãn ngẫu nhiên) dù mạng khi đó chỉ đang "ghi nhớ" dữ liệu không có tương quan thật với nhãn; kết luận quan trọng của paper từ đây: hiệu ứng "gradient tập trung vào top-k subspace" gắn với **cấu trúc bài toán phân loại** k **lớp** (dạng hàm mất, dạng đầu ra), không phụ thuộc việc dữ liệu có cấu trúc/tín hiệu thật hay không.
- **Số chiều subspace** (từ 2 đến 20, Hình 3/9/10): đây là ablation cấu trúc quan trọng nhất, cho thấy chính xác đúng k (hay $k-1$) chiều là nơi tính chất "bảo toàn qua thời gian" đạt cực đại/ổn định, tăng số chiều quá k làm overlap giảm dần.
- **Mô hình đồ chơi**: các "ablation" lý thuyết đi kèm thực nghiệm số nhỏ, gồm bật/tắt nhiễu mẫu ($\sigma^2 = 0$ so với σ^2 nhỏ dương) và bật/tắt bias, đều xác nhận các hiệu ứng chính (gradient trong top subspace, và khi phá suy biến thì thẳng hàng theo eigenvalue nhỏ hơn) vẫn giữ nguyên về chất.

6. Giới hạn

- **Mô hình đồ chơi chỉ chứng minh chặt cho trường hợp đơn giản nhất**: lời giải giải tích đầy đủ (Phụ lục E) chỉ cho $k = 2$ lớp, $\sigma^2 = 0$, không bias, hai mean trực giao tuyệt đối. Các mở rộng quan trọng hơn ($k > 2$ lớp, $\sigma^2 > 0$, có bias) chỉ được **kiểm tra bằng thực nghiệm số nhỏ** (paper tự ghi $d = 1000$, $k = 2, 5, 10$, $\sigma = 0$ hoặc $0{,}02$), chưa có chứng minh giải tích trong bài, và paper dẫn một công trình "sắp công bố" (Kirilin và cộng sự, forthcoming) cho phần tổng quát hoá này; tại thời điểm đọc bài 49, phần đó chưa có để đối chiếu.
- **Paper tự thừa nhận một lỗi ở bản trước**: chú thích 11 ghi "We thank Vladimir Kirilin for pointing out a mistake in an earlier version of this paper" (cảm ơn Vladimir Kirilin đã chỉ ra một lỗi trong bản trước của paper này), liên quan tới nghiệm giải tích của mô hình đồ chơi ở Mục 3.3. Đây là một điểm tự sửa đã được công khai thừa nhận, cần lưu ý khi dùng lại các công thức ở Phương trình 7-9 (bản đọc cho ghi chú này là v1 trên arXiv, đã có sửa so với một bản nháp trước đó không công khai).
- **Ranh giới** k **hay** $k-1$ **chưa chốt được**: chú thích 9 tự nhận việc xác định số chiều chính xác của "khúc gãy" (k hay k-1) là "không thể kết luận chắc chắn từ phổ" dù hình vẽ (Hình 3) cho thấy một khúc gãy khá rõ.
- **Giả thuyết log-normal cho phổ eigenvalue chỉ là gợi ý**: paper tự nói "chỉ là gợi ý, cần nhiều bằng chứng và giải thích hơn" (Phụ lục B).
- **Dạng hàm suy giảm eigenvalue lớn nhất chưa xác định**: paper tự nhận đồ thị quá thô để khẳng định dạng hàm cụ thể (dù mô hình đồ chơi gợi ý $\sim 1/t$), đề xuất cần lấy trung bình nhiều lần chạy trong tương lai để kết luận chắc hơn (Phụ lục B, chú thích 16).
- **Gradient/Hessian được đo trên toàn bộ tập huấn luyện, không phải minibatch**: dù mô hình được huấn luyện bằng SGD (dùng gradient minibatch, có nhiễu), các đại lượng f_{top}, overlap, phổ Hessian trong Hình 1-3 đều được tính lại trên **toàn bộ** tập huấn luyện tại mỗi điểm $\theta^{(t)}$ quan sát. *(Nhận xét người ghi chú)*: paper không thảo luận rõ liệu kết luận "gradient tập trung vào top subspace" có còn đúng cho chính gradient minibatch nhiễu thực sự dùng trong mỗi bước cập nhật hay không, đây là một khoảng trống giữa đại lượng được đo (full-batch) và đại lượng thực sự điều khiển động lực huấn luyện

(minibatch).

- **Gợi ý Newton rút gọn hạng thấp (Phương trình 11) chưa được cài đặt/benchmark**: paper chỉ nêu ý tưởng ở phần Discussion, không có số liệu nào cho thấy phương pháp này thực sự cải thiện tốc độ/chất lượng huấn luyện so với SGD/Adam thông thường.
- **Thiếu dòng baseline rõ cho một số ablation** *(nhận xét người ghi chú)*: Bảng 1 không có dòng "CIFAR10 ConvNet, không chỉnh gì" với chú thích trống để so sánh trực tiếp hiệu ứng riêng của dropout/batch-norm hay của Adam, khiến khó tách bạch chính xác độ lớn ảnh hưởng của từng thay đổi so với một baseline rõ ràng (chỉ có thể so sánh gián tiếp qua các bộ MNIST).
- **Chưa rõ venue công bố cuối cùng**: văn bản đọc được chỉ có header arXiv, không có dòng ghi hội nghị/tạp chí.

7. Liên hệ với các bài khác

(Đã kiểm tra tồn tại file bằng `ls` các thư mục "optimization content/", "pruning content/" trước khi trích.)

- **optimization content/48_2024_old_optimizer_new_norm.md**: đọc ngay trước, cùng mạch dùng hình học không gian trọng số/Hessian để hiểu optimizer; bài 48 dùng "chuẩn toán tử" để diễn giải Adam/Shampoo, còn bài 49 mô tả trực tiếp phổ Hessian và vị trí gradient trong phổ đó, hai góc nhìn bổ sung cho nhau trên cùng đối tượng (Hessian của mạng neural).
- **optimization content/14_2018_shampoo.md** (Shampoo gốc): Shampoo xấp xỉ Hessian theo cấu trúc Kronecker; gợi ý Newton rút gọn hạng thấp ở Mục 3.5 bài 49 (chỉ nghịch đảo Hessian trong top subspace nhỏ) có cùng động cơ với Shampoo (tránh nghịch đảo Hessian đầy đủ, rất tốn kém), dù hai bài chọn cách xấp xỉ khác nhau (Shampoo dùng cấu trúc ma trận theo lớp, bài 49 dùng cấu trúc phổ theo top-k eigenvector toàn cục).
- **optimization content/24_2016_sharp_minima.md** (sharp vs. flat minima): cùng chủ đề "hướng phẳng" (flat directions) trong loss landscape nhìn qua phổ Hessian; bài 24 tập trung câu hỏi "cực tiểu phẳng có tổng quát hoá tốt hơn không", còn bài 49 tập trung câu hỏi "gradient descent di chuyển ở đâu trong không gian đó"; nên đọc cùng nhau để có hai góc nhìn trên cùng hiện tượng phổ Hessian có cấu trúc bulk/top.
- **pruning content/01_1989_1993_optimal_brain_damage_surgeon.md** (Optimal Brain Damage/Surgeon): dùng trực tiếp thông tin Hessian (đường chéo cho OBD, đầy đủ hơn cho OBS) để chọn trọng số nào an toàn để cắt bỏ. Quan sát của bài 49, rằng phần lớn không gian tham số là "bulk" gần phẳng (Hessian gần 0) còn chỉ một không gian con nhỏ "top" có độ cong đáng kể, củng cố trực giác nền tảng của OBD/OBS: hầu hết trọng số nằm trong vùng ít ảnh hưởng tới loss (có thể cắt an toàn), chỉ một số ít hướng (tương ứng top subspace) thực sự nhạy.
- **pruning content/22_2022_optimal_brain_compression.md** (Optimal Brain Compression, phiên bản hiện đại dùng Hessian cho pruning LLM): áp cùng logic Hessian-based pruning của OBD/OBS lên quy mô LLM; liên hệ trực tiếp tới câu hỏi dự án (Mục 8 dưới) về việc không gian con "quan trọng" sau pruning có trùng với top-Hessian-subspace hay không.

8. Ứng dụng cho dự án (suy luận)

(Toàn bộ mục này là suy luận của người ghi chú, không có trong paper, áp cho bài toán nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B, VLM OCR hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1.600-2.100 token hình ảnh, chạy trên GPU ≤ 8GB, batch 1.)

- **Theo dõi f_{top}/overlap trong quá trình fine-tune phục hồi sau pruning**: sau khi cắt head/channel của LightOnOCR-3-0.8B, suy luận rằng đo overlap(g, Hg) (Phương trình 3, chỉ cần 1 Hessian-vector product, rẻ) trong vài trăm bước fine-tune đầu có thể cho tín hiệu sớm về việc mô hình đã "định hình lại" được một top subspace mới (ổn định) hay chưa; nếu overlap thấp/dao động kéo dài bất thường so với mô hình chưa pruning, có thể là dấu hiệu quá trình phục hồi chưa hội tụ về một vùng tốt.

- **Số chiều top subspace và số lớp OCR**: bài 49 cho thấy k (số chiều top subspace) gắn với số lớp đầu ra (classification head). Với LightOnOCR (đầu ra là phân phối trên vocabulary token, không phải k lớp cố định nhỏ như MNIST/CIFAR), suy luận rằng cấu trúc "top subspace" tương tự khó áp trực tiếp (vocabulary lớn hơn rất nhiều so với 10-100 lớp trong paper); tuy nhiên có thể đáng thử trên một đầu ra phụ (ví dụ nếu dự án có phân loại loại-trang hoặc layout-class nhỏ) để kiểm tra hiệu ứng tương tự ở quy mô nhỏ hơn, nơi việc gán k lớp rõ ràng hơn.
- **Gợi ý Newton rút gọn hạng thấp cho fine-tune nhẹ trên GPU nhỏ**: Phương trình 11 (nghịch đảo Hessian chỉ trong top-k subspace) có thể là một hướng rẻ để thử cải thiện bước cuối của việc phục hồi sau pruning (fine-tune ít bước, ít dữ liệu, trên GPU 8GB), vì top subspace nhỏ (ước tính k cỡ vài chục tới vài trăm, tuỳ cách định nghĩa "lớp" cho OCR) có thể tính và nghịch đảo được với Lanczos + Hessian-vector product theo đúng kỹ thuật Phụ lục A, không cần dựng Hessian đầy đủ của mô hình 0,8B tham số. Đây hoàn toàn là suy luận, paper gốc không benchmark ý tưởng này cho bất kỳ mô hình nào.
- **Liên hệ với OBS/OBC (pruning content 01, 22)**: nếu phần lớn tham số LightOnOCR-3-0.8B nằm trong "bulk" gần phẳng theo nghĩa bài 49, suy luận rằng các phương pháp pruning dựa trên độ nhạy Hessian (OBS/OBC, cũng dùng Hessian-vector hoặc xấp xỉ Hessian) có cơ sở lý thuyết tốt để chọn đúng các trọng số "bulk" (ít ảnh hưởng loss) mà cắt, và ngược lại cần đặc biệt cẩn trọng với các trọng số có đóng góp lớn vào top subspace (ít về số lượng nhưng ảnh hưởng loss nhiều) khi quyết định pruning.

9. Thí nghiệm gợi ý

(5 thí nghiệm nhỏ, chạy được trên 1 GPU 8GB, không cần huấn luyện VLM đầy đủ.)

1. **Đo f_{top} trên một mạng nhỏ tự huấn luyện**: huấn luyện một mạng FC nhỏ (2 lớp ẩn, 50-100 neuron) trên MNIST hoặc một bộ con CIFAR10 bằng SGD, cài Lanczos đơn giản (hoặc dùng thư viện có sẵn) qua Hessian-vector product theo đúng thủ thuật Phụ lục A (`a = (grad * v).sum()`, rồi `torch.autograd.grad(a, params)`), tái tạo đường cong f_{top} giống Hình 1, để tự kiểm chứng hiện tượng trước khi áp cho mô hình OCR thật.
2. **Đo overlap(g,Hg) qua các giai đoạn fine-tune phục hồi pruning**: trên một checkpoint nhỏ đã pruning của LightOnOCR (hoặc một mô hình thay thế nhỏ hơn nếu 0,8B quá nặng để thử nhanh), đo overlap(g, Hg) mỗi vài chục bước trong quá trình fine-tune phục hồi, so sánh với overlap của mô hình gốc chưa pruning ở cùng giai đoạn huấn luyện tương đối, để xem pruning có làm overlap giảm/dao động nhiều hơn (dấu hiệu landscape "khó" hơn) hay không.
3. **Quét số chiều subspace (giống Hình 3) cho một đầu phân loại phụ nhỏ**: nếu pipeline OCR có một đầu phân loại rời rạc nhỏ (ví dụ phân loại hướng trang, ngôn ngữ, hay loại tài liệu, với k vài lớp), đo overlap giữa top-subspace tại hai thời điểm khác nhau theo các số chiều subspace khác nhau (2 đến khoảng $2k$), kiểm tra có xuất hiện "khúc gãy" gần k như paper hay không trên chính kiến trúc hybrid GatedDeltaNet + attention của dự án.
4. **Thử bước Newton rút gọn hạng thấp (Phương trình 11) trên mạng nhỏ**: trên mạng FC nhỏ ở thí nghiệm 1, cài đặt bước cập nhật $\theta^{(t+1)} = \theta^{(t)} - (H_{top}^{(t)})^{-1} g_{top}^{(t)}$ (tính H_{top} bằng vài vector Lanczos), so sánh tốc độ hội tụ (số bước để đạt một mức loss cho trước) với SGD/Adam thông thường cùng mạng, để có một đánh giá thực nghiệm sơ bộ (mà chính paper không cung cấp) cho ý tưởng gợi ý ở Mục 3.5/8.
5. **Kiểm tra tính bảo toàn của top subspace qua các epoch fine-tune OCR**: trên một checkpoint LightOnOCR nhỏ, đo overlap giữa $V_{top}^{(t_1)}$ và $V_{top}^{(t_2)}$ (Phương trình 5, với k chọn xấp xỉ theo heuristic nào đó, ví dụ 10-20) ở các mốc t_1, t_2 cách nhau vài trăm bước trong một lần fine-tune ngắn trên dữ liệu OCR, kiểm tra xem top subspace có "đông cứng" tương tự các kiến trúc FC/ConvNet/ResNet trong paper hay không trên kiến trúc hybrid attention + GatedDeltaNet, một kiến trúc khác hẳn các kiến trúc paper đã thử.

10. Tóm tắt công thức / khái niệm

- **Tỉ lệ gradient trong top subspace**: $f_{top} = \|g_{top}\|^2/\|g\|^2$ (Phương trình 2), với V_{top} = span của k eigenvector lớn nhất Hessian, k = số lớp.
- **Overlap gradient-Hessian** (proxy rẻ): $\text{overlap}(g, Hg) = g^T Hg/(\|g\|\|Hg\|) \in [-1, 1]$ (Phương trình 3). Đã tự tính hai trường hợp biên ở Mục 3.1: gradient thuần top cho overlap $= 1$; gradient thuần bulk cho overlap không xác định $(0/0)$.
- **Ước lượng overlap cho vector ngẫu nhiên trong top subspace**: $\text{overlap}(w, Hw) = \sum_i \lambda_i/\sqrt{k\sum_j \lambda_j^2}$ (Phương trình 4); với $\lambda_i \propto i$, giới hạn liên tục cho $\approx \sqrt{3/4} \approx 0{,}87$; đã tự tính với $k = 10$ hữu hạn cho $\approx 0{,}886$, khớp sát.
- **Overlap giữa hai không gian con theo thời gian**: $\text{overlap}(V_{top}^{(t)}, V_{top}^{(t')}) = \frac{1}{k}\sum_i \|P_{top}^{(t)} v_i^{(t')}\|^2$ (Phương trình 5), bằng 1 nếu trùng nhau, 0 nếu trực giao.
- **Tiến triển tuyến tính hoá của gradient**: $g^{(t+1)} = (I - \eta H^{(t)})g^{(t)} + O(\eta^2)$ (Phương trình 10); thành phần theo eigenvalue λ co theo hệ số $(1 - \eta\lambda)$ mỗi bước, thành phần với λ nhỏ hơn (nhưng dương) sống sót lâu hơn (đã tự tính bằng số ở Mục 3.4: $\lambda = 1$ so với $\lambda = 4$, $\eta = 0{,}1$, sau 10 bước tỉ lệ sống sót ≈ 58 lần).
- **Mô hình đồ chơi (2 lớp, $\sigma^2 \to 0$, $\mu_1 \perp \mu_2$)**: Hessian hạng 2, eigenvalue $(\eta t)^{-1}$ (suy biến), gradient $\sim 2(\mu_2 - \mu_1)/(\eta t)$, luôn nằm chính xác trong top subspace 2 chiều (Phương trình 7-9).
- **Newton rút gọn hạng thấp (gợi ý, chưa benchmark)**: $\theta^{(t+1)} = \theta^{(t)} - (H_{top}^{(t)})^{-1} g_{top}^{(t)}$ (Phương trình 11).

11. Kết luận ngắn

Paper đưa ra bằng chứng thực nghiệm nhất quán trên nhiều kiến trúc (FC, ConvNet, ResNet-18) và bộ dữ liệu (MNIST, CIFAR10, CIFAR100, hồi quy) rằng gradient trong huấn luyện deep learning hội tụ nhanh vào một không gian con rất nhỏ (kích thước bằng số lớp phân loại) trương bởi các eigenvector lớn nhất của Hessian, và không gian con này được bảo toàn qua thời gian dài, dù chính các eigenvector trong đó vẫn tiến triển. Một mô hình đồ chơi giải được chính xác minh hoạ cơ chế (gradient tuyến tính hoá hội tụ về eigenvalue dương nhỏ nhất trong không gian khác-không, không bao giờ "rò" vào các hướng phẳng tuyệt đối do đối xứng của loss). Đóng góp chính là quan sát và cơ chế giải thích sơ bộ, không phải một optimizer mới; gợi ý thực hành (Newton rút gọn hạng thấp) được nêu nhưng chưa kiểm chứng. Với dự án nén/hậu huấn luyện LightOnOCR-3-0.8B, ý tưởng hữu ích nhất để mang sang là dùng $\text{overlap}(g, Hg)$ như một chỉ báo rẻ cho "độ khó"/mức hội tụ của loss landscape khi fine-tune phục hồi sau pruning, và khả năng thử một bước Newton giới hạn trong top subspace nhỏ như một cách xấp xỉ bậc hai rẻ cho GPU hạn chế, cả hai đều ở mức suy luận cần kiểm chứng thêm.

Nguồn gốc: https://arxiv.org/abs/1812.04754

Bài 50

Why Do We Need Weight Decay in Modern Deep Learning? (2023)

Tác giả: Francesco D'Angelo, Maksym Andriushchenko (hai tác giả đóng góp ngang nhau), Aditya Varre, Nicolas Flammarion (Theory of Machine Learning Lab, EPFL, Lausanne, Thuỵ Sĩ).
Venue / arXiv: arXiv:2310.04415 (v2, 04/11/2024); paper tự ghi ở chân trang: "38th Conference on Neural Information Processing Systems (NeurIPS 2024)".
Nguồn: https://arxiv.org/abs/2310.04415
Chủ đề: xem lại vai trò thực sự của weight decay (WD) trong deep learning hiện đại qua hai chế độ huấn luyện khác nhau (huấn luyện nhiều epoch cho ảnh, và huấn luyện gần đúng 1 epoch cho LLM), kết luận WD hiếm khi là một "regularizer" tường minh theo nghĩa cổ điển mà chủ yếu thay đổi động lực tối ưu (optimization dynamics) theo hướng có lợi.
Vị trí trong lộ trình: đọc sau opt 49 (Gradient Descent Happens in a Tiny Subspace, cùng dùng chuẩn Jacobian/Hessian để mô tả động lực huấn luyện) và sau opt 07 (AdamW) để có công thức decoupled weight decay làm nền; đọc trước opt 51 (How to set AdamW's weight decay as you scale model and dataset size) vì bài 51 kế tục trực tiếp câu hỏi "chọn λ bao nhiêu khi scale mô hình/dữ liệu" dựa trên đúng cơ chế (ELR, bias-variance) mà bài 50 trình bày.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu, công thức, hình vẽ lấy từ toàn văn arXiv:2310.04415 (bản v2, tải và đọc toàn bộ ngày 09/11/2026, gồm Mục 1-4 và các Phụ lục A-D; phần cuối bài là bảng checklist tiêu chuẩn của NeurIPS, không có nội dung khoa học mới, không được tóm tắt lại). Các phép tính đánh dấu "tự tính" là của người ghi chú, không có trong paper. Đây là paper có thực nghiệm thật (ResNet/VGG trên CIFAR/Tiny-ImageNet, GPT-2 trên OpenWebText), không phải paper lý thuyết thuần; Mục 4 trình bày đầy đủ các thí nghiệm, kèm một "conjecture" (giả thuyết) được kiểm chứng thực nghiệm chứ không chứng minh toán học chặt.

1. Bối cảnh

Theo Mục 1 của paper, huấn luyện mạng neural hiện đại rơi vào hai chế độ (regime) rộng: **over-training** (nhiều lượt đi qua dữ liệu, "multipass", cần chiến lược regularization để tránh overfit) và **under-training** (ít lượt đi qua dữ liệu, do lượng dữ liệu và ràng buộc tính toán lớn, theo tinh thần Hoffmann và cộng sự, 2022, tức Chinchilla). Kiến trúc ResNet cho thị giác máy tính (He và cộng sự, 2016) là ví dụ điển hình của over-training; huấn luyện LLM (Brown và cộng sự, 2020, GPT-3) là ví dụ điển hình của under-training. Cả hai chế độ đều dùng weight decay rộng rãi, nhưng vai trò thực sự của nó vẫn còn tranh luận: Zhang và cộng sự (2016) cho thấy mạng vẫn có thể ghi nhớ hoàn toàn dữ liệu dù có WD, nghi ngờ tính chất "regularizer" của WD; còn ở chế độ under-training, về lý thuyết regularization gần như không cần thiết vì số lượt đi qua dữ liệu hạn chế đã tự ngăn overfit.

Thí nghiệm mở đầu (Hình 1 của paper): huấn luyện ResNet18 trên các tập con của CIFAR-5m (Nakkiran và cộng sự, 2020) với kích thước từ 10.000 đến 5 triệu mẫu, giữ cố định tổng số bước huấn luyện là 5 triệu (tương đương số lượt đi qua dữ liệu từ 500 lượt xuống 1 lượt). Kết quả: WD có lợi ở **cả hai** chế độ, nhưng paper cho rằng lý do khác nhau giữa hai chế độ, cần phân tích riêng.

Bối cảnh tổng quát hơn: khái niệm phạt ℓ_2 lên trọng số có nguồn gốc từ các nghiên cứu về ổn định nghiệm cho bài toán ill-posed (Tikhonov, 1943) và hồi quy ridge (Hoerl, 1962; Hoerl và Kennard, 1970), được Krogh và Hertz (1991) áp dụng có hệ thống cho mạng neural. Loshchilov và Hutter (2019, chính là opt 07 trong lộ trình, bài AdamW) tách biệt rõ WD và ℓ_2 regularization, đặc biệt cho Adam. Nhiều công trình khác (Van Laarhoven, 2017; Zhang và cộng sự, 2018; Li và Arora, 2019) đã nghiên cứu WD như một công cụ ảnh hưởng

tới **động lực tối ưu** (qua khái niệm "effective learning rate", ELR) chứ không chỉ là một ràng buộc dung lượng (capacity constraint), nhưng các phân tích này phần lớn dựa vào tính **scale-invariance** (bất biến theo tỉ lệ) của mạng, một tính chất không áp dụng cho mọi kiến trúc tổng quát. Bài 50 đặt mục tiêu giải thích cơ chế của WD mà không cần giả định scale-invariance.

2. Ý tưởng chính

Thông điệp xuyên suốt của paper: **weight decay hiếm khi có giá trị như một regularizer tường minh; thay vào đó, nó tạo ra những thay đổi có lợi trong động lực tối ưu**, và cơ chế cụ thể của sự thay đổi này khác nhau giữa hai chế độ huấn luyện:

- **Chế độ over-training (ảnh, nhiều epoch, SGD)**: WD kết hợp với learning rate (LR) lớn giúp duy trì **nhiễu SGD không triệt tiêu** (non-vanishing noise) qua một cơ chế gọi là "loss stabilization" (ổn định hoá loss): WD ngăn chuẩn trọng số tăng không giới hạn, điều này (với hàm mất cross-entropy, có đuôi mũ) ngăn luôn việc loss giảm tiếp xuống gần 0, khiến loss "mắc kẹt" ở một mức xấp xỉ hằng số. Khi loss không giảm thêm được, nhiễu gradient minibatch không triệt tiêu, cho phép implicit regularization (điều chuẩn ẩn) của SGD "mở ra" và kiểm soát chuẩn Jacobian của mạng, cải thiện generalization.
- **Chế độ under-training (LLM, gần 1 epoch, AdamW)**: WD **không** mang hiệu ứng regularization cổ điển nào (generalization gap gần như bằng 0 dù có WD hay không), mà tương đương với việc điều chỉnh một **effective learning rate (ELR)**, qua đó cân bằng lại **trade-off bias-variance** trong tối ưu stochastic, dẫn tới training loss cuối cùng thấp hơn. Thêm vào đó, WD có một lợi ích thực hành quan trọng: giúp huấn luyện ổn định ở độ chính xác số bfloat16 (ngăn loss "nổ"/phân kỳ).

Hai góc nhìn này được paper trình bày như một "lăng kính thống nhất" (unifying perspective) từ ResNet trên bài toán thị giác tới LLM: trong cả hai trường hợp, WD hoạt động qua việc thay đổi quỹ đạo tối ưu (không phải qua một số hạng phạt tĩnh trong hàm mất), chỉ khác nhau ở cơ chế cụ thể (noise-driven implicit regularization với over-training, ELR/bias-variance với under-training).

3. Phương pháp chi tiết

3.1. Thiết lập chung

Dữ liệu $(x_i, y_i)_{i=1}^N$, mạng $h_w : \mathbb{R}^d \to \mathbb{R}^c$ với c lớp, hàm mất cross-entropy ℓ, loss huấn luyện $L(w) = \frac{1}{N}\sum_i \ell(y_i, h_w(x_i))$, loss có phạt ℓ_2: $L_\lambda(w) = L(w) + \frac{\lambda}{2}\|w\|^2$. SGD với WD (batch size 1, có hoàn lại, Phương trình 1):

$$w_{t+1} = w_t - \eta \nabla_w \ell(y_{i_t}, h(w_t, x_{i_t})) - \eta\lambda w_t.$$

Paper theo dõi 3 chuỗi: w_t (LR lớn, khai thác nhiễu SGD), $\tilde{w}_t$ (fine-tuning, bắt đầu từ w_t rồi dùng LR nhỏ), và $\bar{w}_t$ (exponential moving average, EMA, trên chuỗi LR lớn).

3.2. Cơ chế "loss stabilization" (over-training)

Thí nghiệm nền (Hình 2): ResNet18 trên CIFAR-10 và Tiny-ImageNet, so sánh có/không WD, LR nhỏ/lớn. Quan sát quan trọng: các đường có cùng λ (cùng độ lớn $\|w\|_2$ cuối cùng, cùng train CE sau khi giảm LR, Hình 2b-c) nhưng LR khác nhau trong giai đoạn đầu lại cho **test error khác nhau rõ rệt** (Hình 2a). Vậy chỉ riêng việc tối thiểu hoá L_λ (tức việc dùng chuẩn ℓ_2 cố định) không giải thích được lợi ích của WD; chuẩn trọng số cuối cùng cũng không phải yếu tố quyết định. Yếu tố quyết định là **lựa chọn LR sau khi chuẩn đã bị WD ràng buộc**.

Cơ chế được paper đề xuất dựa trên tính chất của loss cross-entropy: khi dữ liệu phân tách được (separable) và không có WD, cận dưới (infimum) của CE là tại vô cực, khiến chuẩn trọng số tăng không giới hạn (Soudry và cộng sự, 2018; Ji và Telgarsky, 2019, về implicit bias của gradient descent trên dữ liệu phân tách được).

WD, bằng cách ngăn chuẩn trọng số tăng, **ngăn luôn CE giảm tiếp** (với LR cố định), khiến optimization giống như đang diễn ra trên mặt một hình cầu có bán kính xấp xỉ cố định. Phát biểu cơ chế (nguyên văn được đóng khung trong paper):

> Ràng buộc chuẩn trọng số cản trở việc CE giảm tiếp, qua đó cho phép nhiễu SGD không triệt tiêu. Điều này cho phép implicit regularization của SGD phát huy tác dụng và dẫn dắt quỹ đạo tối ưu.

3.3. Hình thức hoá: quá trình điều khiển bởi nhiễu (the noise driven process)

Viết lại cập nhật SGD bằng cách tách phần nhiễu $g_t := \nabla_w L(w_t) - \nabla_w \ell(y_{i_t}, h(w_t, x_{i_t}))$ (Phương trình 2):

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta\nabla L(w_t) + \eta g_t.$$

Định nghĩa Jacobian mạng $J(x_i, w) := \nabla h_w(x_i) \in \mathbb{R}^p$ và chuẩn Frobenius trung bình trên tập dữ liệu $\|J(w)\|_F^2 = \frac{1}{N}\sum_i \mathrm{Tr}(\nabla h_w(x_i)\nabla h_w(x_i)^\top)$. Dùng xấp xỉ Gaussian cho nhiễu SGD (đã được nhiều công trình trước kiểm chứng, Li và cộng sự 2020, 2021a; Smith và cộng sự 2020) và một "xấp xỉ decoupling" (giả định đạo hàm bậc một của loss $\ell_i'(w_t)$ xấp xỉ bằng nhau trên mọi điểm dữ liệu, gọi chung là $\sigma_{\eta,\lambda}(w_t)$, kiểm chứng thực nghiệm ở Phụ lục C.6 bằng Stochastic Lanczos Quadrature so sánh phổ ma trận hiệp phương sai thật và xấp xỉ, Hình 16-18, cho thấy "chồng lấp đáng kể"), paper thu được một dạng cập nhật SGD xấp xỉ (Phương trình 4) và từ đó nêu:

Giả thuyết 1 (Conjecture 1): quá trình $(h(w_t, x))_t$ hội tụ về một phân phối dừng $\mu_{\eta,\lambda}^{\infty}(x)$ có trung bình $\bar{\mu}_{\eta,\lambda}(x) = h(w_{\eta,\lambda}^*, x)$, trong đó $w_{\eta,\lambda}^*$ là điểm tới hạn bậc một của loss điều chuẩn (Phương trình 5):

$$\tilde{L}_\lambda(w) := L_\lambda(w) + \eta\sigma_{\eta,\lambda}^2\|J(w)\|_F^2.$$

Nói cách khác: ngoài số hạng phạt ℓ_2 tường minh $\frac{\lambda}{2}\|w\|^2$, nhiễu SGD (với quy mô $\sigma_{\eta,\lambda}$ phụ thuộc cả η và λ) tạo ra một **số hạng điều chuẩn ẩn thứ hai** tỉ lệ với chuẩn Jacobian bình phương $\|J(w)\|_F^2$. Đây là cơ chế chính giải thích lợi ích generalization của WD: không phải số hạng ℓ_2 tường minh, mà chính độ lớn nhiễu (do WD kết hợp LR lớn tạo ra qua loss stabilization) quyết định mức điều chuẩn Jacobian.

(Tự tính) minh hoạ bias-variance trade-off (ý tưởng dùng ở Mục 3.5 để giải thích LLM, nhưng cùng loại lập luận với cơ chế trên): dùng cận lý thuyết chuẩn cho SGD (Moulines và Bach, 2011), Excess Risk $\lesssim (1 - \eta\mu)^t\|w_0 - w^*\|^2 + \eta\sigma^2$. Lấy $\mu = 1$, $\sigma^2 = 1$, $\|w_0 - w^*\|^2 = 100$, so sánh $\eta = 0{,}01$ và $\eta = 0{,}1$ tại $t = 100$: với $\eta = 0{,}01$: $(0{,}99)^{100} \approx 0{,}366$ nên bias $\approx 36{,}6$, variance $= \eta\sigma^2 = 0{,}01$, tổng $\approx 36{,}61$. Với $\eta = 0{,}1$: $(0{,}9)^{100} \approx 2{,}66 \times 10^{-5}$ nên bias $\approx 0{,}00266$, variance $= 0{,}1$, tổng $\approx 0{,}103$. Ở $t = 100$, η lớn hơn (dù variance lớn hơn 10 lần) vẫn thắng rõ vì bias còn thống trị. Tại $t = 10.000$: cả hai bias đều xấp xỉ 0 (bias đã "quên" hoàn toàn điều kiện đầu), nên tổng xấp xỉ đúng bằng variance: $0{,}01$ (cho $\eta = 0{,}01$) so với $0{,}1$ (cho $\eta = 0{,}1$), lúc này η **nhỏ hơn** lại thắng. Phép tính tự tính này minh hoạ chính xác lập luận paper dùng ở Mục 3.5: ELR lớn thắng ở giai đoạn đầu (bias còn chi phối), ELR nhỏ thắng khi huấn luyện đã chạy đủ lâu (variance chi phối).

3.4. EMA và fine-tuning

Để "khai thác" độ lệch (bias) hữu ích mà nhiễu SGD tạo ra, cần giảm nhiễu bằng một trong hai cách: EMA ($\bar{w}_t$, hệ số $\beta = 0{,}999$) hoặc giảm LR (fine-tuning, $\tilde{w}_t$). Paper nhận thấy EMA hiệu quả hơn về mặt thực hành (không cần thêm bước gradient hay tinh chỉnh siêu tham số) và vượt nhẹ nhưng nhất quán so với fine-tuning trong các thí nghiệm (Hình 5b, 2a, 2d). Để so sánh công bằng dọc theo quỹ đạo fine-tuning (các điểm khác thời điểm t có loss khác nhau nên không so sánh trực tiếp $\|J(\tilde{w}_t)\|_F$ được), paper chiếu w_t lên một "đa tạp loss không đổi" bằng cách fine-tune LR nhỏ không WD tới khi $L(\tilde{w}_t) \approx$ hằng số với mọi t (Hình 5a); khi đó $\|J(\tilde{w}_t)\|_F$ giảm dần theo t (Hình 5c), xác nhận quỹ đạo gốc w_t có xu hướng dần thiên về nghiệm điều chuẩn tốt hơn.

Paper tự ghi một điểm bất thường (không chắc, xem Mục 6): ”trong Hình 5c chúng tôi quan sát các điểm cuối của chuỗi fine-tune thể hiện xu hướng NGƯỢC so với EMA (LR nhỏ hơn cho $\|J\|_F$ lớn hơn)”, một sự khác biệt mà paper cho là có thể do bản chất phụ thuộc trạng thái (state-dependent) của hiệp phương sai nhiễu SGD trong Phương trình 3, chưa giải thích dứt điểm.

3.5. Hai hiệu ứng của WD trong chế độ under-training (LLM)

Thiết lập: GPT-2-124M (NanoGPT, Karpathy 2023) trên OpenWebText, AdamW, LR mặc định $0{,}0006$, warmup 400 bước, 10x cosine LR decay, batch 256, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}95$, huấn luyện 50.000 bước (thay 600.000 bước mặc định, để tiết kiệm tính toán học thuật), độ dài ngữ cảnh giảm từ 1024 xuống 256 trừ thí nghiệm bfloat16 (Hình 8, cần ngữ cảnh dài để thấy phân kỳ).

Hiệu ứng 1 (tối ưu training loss tốt hơn): tái hiện quan sát của Hoffmann và cộng sự (2022, Chinchilla, hình A7 của họ) ở quy mô nhỏ hơn nhiều (Hình 6): WD = 0,1 và 0,3 cho training loss cuối thấp hơn WD = 0 (dù WD=0,1/0,3 khởi đầu loss CAO HƠN, chỉ ”bắt kịp và vượt” về cuối). Cơ chế giải thích: WD kết hợp sign-SGD (dùng như đại diện cho Adam, theo Balles và Hennig, 2018) tương đương projected SGD trên mặt cầu với **effective learning rate** $\eta_{\text{eff}} \propto \eta_t / \|w_t\|_2$ (chú ý: mũ 1, không phải mũ 2 như ELR cổ điển $\eta/\|w\|_2^2$ của Van Laarhoven/Zhang và cộng sự 2018 cho plain SGD). Paper xác nhận bằng thực nghiệm khớp ELR: huấn luyện KHÔNG WD nhưng với một lịch LR thủ công khớp đúng $\eta_t/\|w_t\|_2$ của chạy có WD=0,1/0,3 sẽ cho ra **toàn bộ** đường cong loss giống nhau (Hình 7, chỉ đúng với float32; với bfloat16 chạy khớp-ELR-nhưng-không-WD vẫn phân kỳ giữa huấn luyện).

(Tự tính) minh hoạ sự khác biệt giữa hai công thức ELR: lấy $\|w\|_2 = 10$, $\eta = 0{,}01$. ELR kiểu plain-SGD (mũ 2): $\eta/\|w\|_2^2 = 0{,}01/100 = 0{,}0001$. ELR kiểu sign-SGD/Adam (mũ 1, công thức paper dùng cho AdamW): $\eta/\|w\|_2 = 0{,}01/10 = 0{,}001$, lớn hơn **10 lần**. Con số tự tính này cho thấy rõ vì sao paper nhấn mạnh việc dùng đúng công thức mũ 1 là ”tới hạn” (critical) khi khớp ELR cho AdamW: nếu dùng nhầm công thức mũ 2 (đúng cho SGD thường), độ lớn LR suy ra sẽ sai hẳn một hệ số $\|w\|_2$ (ở đây là 10 lần), paper ghi rõ dùng nhầm công thức khiến ”chạy phân kỳ rất sớm trong huấn luyện, ngay cả với độ chính xác float32”.

Giải thích hình dạng đường loss qua lý thuyết bias-variance cổ điển (Moulines và Bach, 2011; công thức đã nêu ở Mục 3.3): đầu huấn luyện, ELR lớn (do WD giữ $\|w\|$ nhỏ) làm variance term chi phối, khiến loss (tạm thời) cao hơn chạy không WD; cuối huấn luyện, khi ELR và variance term đã giảm ở cả hai chạy, chạy có WD ”bắt kịp” vì trong suốt quá trình nó duy trì ELR cao hơn (vì $\|w\|$ không tăng vô hạn như chạy không WD), nên bias term co lại nhanh hơn. Điều này cũng giải thích vì sao EMA cho LLM có lợi nhất khi dùng với LR lớn (Sanyal và cộng sự, 2023; Hình 23 của paper): khi variance chi phối, giảm variance bằng trung bình giúp nhiều.

Hiệu ứng 2 (ổn định huấn luyện bfloat16): với độ dài ngữ cảnh 1024, LR vừa phải $0{,}001$ (LR mặc định của Adam) KHÔNG có WD gây ”nổ” loss cuối huấn luyện ở **mọi** random seed khi dùng bfloat16 (Hình 8); mô hình **không hồi phục** sau cú nổ này (khác với hiện tượng ”Edge of Stability” của Cohen và cộng sự 2021/2022, nơi các cú tăng vọt loss thường tự phục hồi). Dùng float32 ở cùng LR thì hoàn toàn ổn định. Giảm LR ($0{,}001 \to 0{,}0006$) tránh được phân kỳ nhưng huấn luyện chậm hơn (Hình 24, phụ lục). Cách hiệu quả nhất: giữ LR cao ($0{,}001$) **và** thêm WD, vừa ổn định bfloat16 vừa cho training loss cuối tốt hơn. Paper tự nhận đây là một lợi ích của WD **không được giải thích đầy đủ** bởi cơ chế ELR ở trên (vì chạy khớp-ELR-không-WD vẫn phân kỳ ở bfloat16 dù khớp hoàn hảo ở float32), tức WD còn mang lại gì đó ngoài việc chỉ thay đổi ELR.

4. Thí nghiệm và kết quả chính

Đây là paper có thực nghiệm thật trên cả hai chế độ, không phải lý thuyết thuần:

- **Hình 1**: ResNet18 trên CIFAR-5m, kích thước tập huấn luyện từ 10.000 đến 5 triệu, tổng số bước cố định 5 triệu; WD cải thiện test error ở cả vùng nhiều epoch và vùng gần 1 epoch.

- **Hình 2**: ResNet18 trên CIFAR-10 và Tiny-ImageNet, so sánh có/không WD, LR nhỏ/lớn, kèm EMA; LR lớn + WD cho test error tốt nhất dù train CE và chuẩn ℓ_2 cuối cùng giống các cấu hình khác cùng λ.
- **Hình 3, 4**: heatmap test error và chuẩn Jacobian EMA trên lưới (η, λ) cho Tiny-ImageNet; tồn tại một giá trị tối ưu của tích $\eta \times \lambda$ (paper nêu $\approx 0{,}005$ cho Tiny-ImageNet), vượt quá gây "over-regularization", thấp hơn gây "under-regularization"; chuẩn Jacobian giảm đơn điệu theo $\eta\lambda$ tăng (khớp Giả thuyết 1); quy mô nhiễu $\sigma_{\eta,\lambda}$ tăng đơn điệu theo train loss và theo $\eta \times \lambda$.
- **Hình 5**: so sánh EMA và fine-tuning trên ResNet18/CIFAR-10, cố định $\lambda = 0{,}0125$, nhiều LR; EMA nhất quán tốt hơn fine-tuning nhẹ; chuẩn Jacobian của chuỗi fine-tune giảm dần theo thời gian huấn luyện gốc.
- **Phụ lục C.3-C.5 (Hình 10-15)**: lặp lại các thí nghiệm chính cho VGG/ResNet trên CIFAR-10/100, và cho mạng scale-invariant chiếu lên mặt cầu đơn vị (Phương trình 8-9, Giả thuyết 3 là biến thể của Giả thuyết 1 không có số hạng λ tường minh); cùng hiện tượng test error dạng chữ U theo LR trong giai đoạn LR lớn.
- **Phụ lục C.6 (Hình 16-18)**: dùng Stochastic Lanczos Quadrature kiểm chứng "decoupling approximation" (phổ hiệp phương sai SGD thật so với xấp xỉ khớp khá tốt) trên nhiều cặp (η, λ).
- **Phụ lục C.7 (Bảng 2, 3)**: snapshot ensemble (trung bình trong function space mỗi 10 epoch) khớp sát với EMA cả về test error và Total Variation Distance của softmax output, thêm bằng chứng cho Giả thuyết 1.
- **Hình 6**: GPT-2-124M trên OpenWebText, tái hiện quan sát của Hoffmann và cộng sự (2022) ở quy mô nhỏ: WD=0,1 và 0,3 cho training loss cuối thấp hơn WD=0.
- **Hình 7**: ELR ($\eta_t / \|w_t\|_2$) của các chạy ở Hình 6 (trái); một lịch LR khớp ELR đó (giữa); khớp ELR tái tạo toàn bộ đường loss nếu dùng float32 (phải).
- **Hình 8**: bfloat16, ngữ cảnh 1024, LR=0,001 không WD phân kỳ ở mọi seed; có WD thì ổn định và loss cuối tốt hơn.
- **Hình 19 (Phụ lục D)**: generalization gap gần 0 cho mọi λ, cả mô hình 124M và **774M** tham số, xác nhận WD không mang hiệu ứng regularization cổ điển trong chế độ này.
- **Hình 20-22 (Phụ lục D)**: WD trên cả tham số LayerNorm cho hiệu ứng tương tự ở λ nhỏ nhưng kém hơn ở λ lớn; dùng ℓ_2 penalty cộng vào loss (không decoupled) cho hiệu ứng **giống** decoupled WD của AdamW; hiệu ứng cải thiện training loss cũng xuất hiện với SGD có momentum, không riêng Adam.

5. Phân tích / ablation

- **Ablation LR tại λ cố định (Hình 2, 10, 11)**: giữ λ cố định, chỉ LR lớn mới cho test error tốt, dù train CE/chuẩn cuối giống nhau; cho thấy chính η (không chỉ λ) quyết định mức điều chuẩn ẩn.
- **Ablation $\eta \times \lambda$ (Hình 3, 4, 10b)**: các cặp (η, λ) có cùng tích cho test error và chuẩn Jacobian tương tự, gợi ý tích $\eta\lambda$ (không phải η hay λ riêng lẻ) là đại lượng điều khiển chính mức điều chuẩn.
- **Ablation EMA so với fine-tuning (Hình 5)**: cả hai đều cải thiện performance nhưng EMA nhất quán nhẹ hơn fine-tuning; cả hai chỉ hiệu quả khi kết hợp với loss stabilization (nếu không có pha LR lớn/WD trước đó, không có "bias" tích lũy nào để khai thác).
- **Ablation decoupled WD so với ℓ_2 thường (Hình 21, chế độ LLM)**: cho kết quả **giống nhau** đối với hiệu ứng cải thiện training loss, tức việc "decouple" WD khỏi bước cập nhật thích ứng của Adam (như Loshchilov và Hutter, 2019, chủ trương) **không cần thiết** riêng cho hiệu ứng này; đây là một điểm thú vị cần đối chiếu với động cơ ban đầu của chính AdamW (xem Mục 6).
- **Ablation optimizer (Hình 22, SGD+momentum so với AdamW)**: hiệu ứng cải thiện training loss vẫn xảy ra với SGD có momentum, gợi ý cơ chế không đặc thù riêng cho learning rate thích ứng (adaptive) của Adam.
- **Ablation WD trên LayerNorm (Hình 20)**: phạt cả tham số LayerNorm cho kết quả giống ở λ nhỏ (0,1) nhưng kém hơn ở λ lớn (0,3); paper ghi chú việc này làm thay đổi giá trị tối ưu của objective.
- **Ablation precision (Hình 8, 24, 25)**: bfloat16 với LR vừa phải không WD phân kỳ; float32 cùng LR ổn định; giảm LR (không cần WD) cũng tránh phân kỳ nhưng chậm hơn; kết hợp LR cao + WD là lựa

chọn tốt nhất trong các phối hợp đã thử.

- **Ablation Hessian trace so với Jacobian norm (Phụ lục C.4, Hình 14)**: dọc theo chuỗi fine-tune, thứ hạng (ranking) giữa các LR theo trace Hessian **ngược lại** với kỳ vọng lý thuyết (LR lớn hơn lẽ ra phải điều chuẩn mạnh hơn/chuẩn nhỏ hơn); dọc theo EMA, trace Hessian không cho thứ hạng rõ ràng còn Jacobian norm thì có (thấp hơn cho LR cao hơn, đúng kỳ vọng). Đây là một bất đồng được paper tự ghi nhận, chưa giải thích hết (xem Mục 6).

6. Giới hạn

- **Giả thuyết 1 (Conjecture 1) không được chứng minh toán học**, chỉ được hỗ trợ bởi một chuỗi xấp xỉ (Gaussian noise approximation, decoupling approximation) và kiểm chứng thực nghiệm (Phụ lục C.6). Paper tự nhận: ”chúng tôi không có kết quả lý thuyết độc lập mới như định lý hay bổ đề... chúng tôi cố gắng đưa ra một bức tranh thực nghiệm rõ ràng và các giải thích tổng quát” (Mục 4, kết luận).
- **Tính toán hạn chế, không có thực nghiệm quy mô lớn thật sự**: paper tự nhận ”với nguồn lực tính toán hạn chế, chúng tôi không thực hiện thực nghiệm quy mô lớn thực sự”. Mô hình LLM lớn nhất được huấn luyện đầy đủ theo đúng thiết lập thí nghiệm là GPT-2-124M; mô hình 774M chỉ được dùng cho kiểm tra sanity-check generalization gap (Hình 19), không chạy đầy đủ các thí nghiệm ELR/bfloat16 ở quy mô đó.
- **Số bước huấn luyện LLM giảm mạnh so với thực tế**: 50.000 bước thay vì 600.000 bước mặc định của NanoGPT (để vừa ngân sách học thuật), và độ dài ngữ cảnh giảm từ 1024 xuống 256 cho hầu hết thí nghiệm (trừ thí nghiệm bfloat16 cần ngữ cảnh dài để thấy phân kỳ); kết luận có thể không khái quát hoàn toàn cho các cấu hình huấn luyện LLM production-scale thật.
- **Lợi ích bfloat16 của WD chưa được giải thích bởi chính cơ chế ELR của paper**: chạy khớp ELR nhưng không có WD thật vẫn phân kỳ ở bfloat16 (chỉ khớp đường loss khi dùng float32), cho thấy WD đóng góp một điều gì đó về ổn định số học **ngoài** phạm vi giải thích bằng ELR; paper tự nhận và không đưa ra cơ chế thay thế cho hiện tượng này.
- **Bất đồng tự ghi nhận giữa trace Hessian và Jacobian norm** (Phụ lục C.4): thứ hạng theo trace Hessian dọc theo chuỗi fine-tune bị đảo ngược so với lý thuyết kỳ vọng; paper chọn phân tích Jacobian norm một phần ”vì nó không phụ thuộc tường minh vào giá trị training loss, cho phép so sánh thuận tiện giữa các nghiệm khác nhau”, một lựa chọn vì tiện lợi phân tích hơn là vì đã chứng minh được đây là đại lượng ”đúng” cho loss cross-entropy (với square loss thì trace Hessian và Jacobian norm bằng nhau chính xác tại điểm interpolate, nhưng với CE chỉ là xấp xỉ).
- **Bất đồng tự ghi nhận giữa EMA và fine-tuning** (Mục 3.4, Hình 5c): điểm cuối của chuỗi fine-tune cho xu hướng ngược với EMA (LR nhỏ hơn cho Jacobian norm lớn hơn), paper chỉ đưa giả thiết (chưa xác nhận) rằng nguyên nhân là tính phụ thuộc trạng thái của hiệp phương sai nhiễu.
- **Ablation decoupled WD vs ℓ_2 (Hình 21) có thể gây hiểu nhầm nếu đọc tách khỏi ngữ cảnh**: kết quả ”giống nhau” này chỉ áp dụng cho **hiệu ứng cải thiện training loss** trong chế độ under-training mà bài 50 đang xét; nó không phủ nhận các khác biệt khác giữa WD decoupled và ℓ_2 thường mà Loshchilov và Hutter (2019, opt 07) đã chỉ ra ở các khía cạnh khác (ví dụ tương tác với Adam’s adaptive scaling khi tune nhiều siêu tham số cùng lúc); *(nhận xét người ghi chú)*: hai paper không thực sự mâu thuẫn vì đang trả lời hai câu hỏi hơi khác nhau, nhưng người đọc cần phân biệt rõ phạm vi của mỗi kết luận.
- **Mô hình phân tách rạch ròi 2 chế độ có thể là một sự đơn giản hoá**: thực tế một pipeline huấn luyện có thể nằm giữa hai chế độ (ví dụ fine-tune nhiều epoch trên dữ liệu ít), paper không thảo luận trực tiếp vùng chuyển tiếp này (dù thí nghiệm Hình 1 với số lượt đi qua dữ liệu trung gian có gợi ý một phần).

7. Liên hệ với các bài khác

(Đã kiểm tra tồn tại file bằng `ls` các thư mục ”optimization content/”, ”llm content/” trước khi trích.)

- **optimization content/07_2017_adamw.md** (AdamW, Loshchilov và Hutter 2019): bài 50 phân tích trực tiếp công thức decoupled weight decay mà bài 07 đề xuất, và đối chiếu lại động cơ ban đầu của

AdamW (giải thích là regularizer tốt hơn) với phát hiện mới (WD chủ yếu là ELR/optimization, không phải regularizer, trong chế độ LLM); nên đọc 07 trước để hiểu rõ công thức đang được ”giải phẫu lại”.

- **optimization content/49_2018_gradient_descent_tiny_subspace.md**: đọc ngay trước, cùng dùng chuẩn Jacobian/Hessian (qua Hessian-vector product) để mô tả hình học của quá trình huấn luyện; bài 49 nói về nơi gradient ”sống” trong phổ Hessian, bài 50 nói về việc WD điều chuẩn chính chuẩn Jacobian đó qua nhiễu SGD, hai góc nhìn trên cùng các đại lượng toán học.
- **optimization content/24_2016_sharp_minima.md** (sharp vs. flat minima, implicit regularization của batch size/noise): liên hệ trực tiếp tới giả thuyết trung tâm của bài 50 rằng nhiễu SGD (không triệt tiêu nhờ WD+LR lớn) chính là nguồn điều chuẩn ẩn giúp tổng quát hoá tốt hơn.
- **optimization content/19_2017_super_convergence.md** (super-convergence, LR theo chu kỳ lớn-rồi-nhỏ): liên hệ tới cấu trúc hai pha ”LR lớn rồi fine-tuning LR nhỏ” mà bài 50 dùng để phân tích (Mục 3.4), dù hai bài có mục tiêu khác nhau (tốc độ huấn luyện so với cơ chế WD).
- **llm content/04_2022_chinchilla.md** (Hoffmann và cộng sự, 2022): chính là công trình mà bài 50 trích dẫn trực tiếp và tái hiện lại ở quy mô nhỏ (quan sát WD cải thiện training loss cho AdamW, chỉ sau khoảng 80% số bước huấn luyện theo Chinchilla gốc); bài 50 là lời giải thích cơ chế (ELR, bias-variance) cho quan sát mà Chinchilla chỉ ”briefly observe” (quan sát ngắn, không giải thích).

8. Ứng dụng cho dự án (suy luận)

(Toàn bộ mục này là suy luận của người ghi chú, không có trong paper, áp cho bài toán nén/hậu huấn luyện mô hình LightOnOCR-3-0.8B, VLM OCR hybrid GatedDeltaNet + full attention, khoảng 3.600 token/trang gồm ~1.500 token text và ~1.600-2.100 token hình ảnh, chạy trên GPU ≤ 8GB, batch 1.)

- **Xác định đang ở chế độ nào trước khi chọn WD**: suy luận rằng việc fine-tune phục hồi LightOnOCR-3-0.8B sau pruning cần xác định rõ mình đang ở gần chế độ ”over-training” (nếu dữ liệu phục hồi nhỏ, lặp lại nhiều epoch) hay ”under-training” (nếu dùng một corpus OCR lớn, gần 1 epoch). Nếu là trường hợp đầu, cơ chế loss-stabilization (Mục 3.2) gợi ý nên dùng WD **cùng với** LR tương đối lớn trong giai đoạn đầu phục hồi (không chỉ WD đơn độc) để kích hoạt điều chuẩn ẩn qua nhiễu; nếu là trường hợp sau, nên kỳ vọng lợi ích chính của WD là tối ưu training loss nhanh hơn (qua ELR) và ổn định số học, không phải generalization.
- **Ổn định bfloat16/fp16 khi fine-tune trên GPU 8GB**: do GPU 8GB gần như bắt buộc dùng mixed-precision (bf16/fp16) để vừa bộ nhớ, phát hiện ở Hình 8 (WD giúp tránh loss ”nổ” không hồi phục ở bfloat16, đặc biệt với LR không quá nhỏ) gợi ý nên **luôn giữ WD khác 0** khi fine-tune phục hồi LightOnOCR ở độ chính xác thấp, kể cả nếu mục tiêu chính không phải generalization mà chỉ là optimization/ổn định, vì paper cho thấy hạ LR để tránh phân kỳ (thay vì thêm WD) làm chậm huấn luyện, một đánh đổi không mong muốn khi ngân sách GPU đã hạn chế.
- **Theo dõi $\|w\|_2$ và ELR suy ra trong quá trình phục hồi sau pruning**: pruning loại bỏ trực tiếp một phần trọng số (giảm $\|w\|_2$ đột ngột), suy luận rằng điều này tương đương một ”cú nhảy” ELR ($\eta/\|w\|_2$ tăng đột ngột ngay sau pruning vì $\|w\|$ giảm) mà không được WD ”dự đoán” trước; có thể cần giảm LR tạm thời ngay sau pruning hoặc tăng WD một chút trong vài bước đầu để tránh ELR hiệu dụng quá cao gây mất ổn định early-training, tương tự tinh thần Hình 7/Mục 3.5 nhưng áp cho tình huống trọng số bị thay đổi đột ngột bởi pruning chứ không phải do chính SGD.
- **Cân nhắc decoupled WD so với ℓ_2 đơn giản cho dự án**: vì Hình 21 cho thấy hai cách này cho hiệu ứng giống nhau (ít nhất cho hiệu ứng cải thiện training loss trong chế độ gần 1-epoch), suy luận rằng với fine-tune phục hồi ngắn (ít bước), không cần quá cầu kỳ triển khai decoupled WD đúng chuẩn AdamW nếu codebase sẵn có chỉ hỗ trợ ℓ_2 penalty thường; tuy vậy đây là suy luận ngoại suy từ GPT-2/OpenWebText, chưa kiểm chứng cho kiến trúc hybrid GatedDeltaNet + attention của LightOnOCR.

9. Thí nghiệm gợi ý

(5 thí nghiệm nhỏ, chạy được trên 1 GPU 8GB, không cần huấn luyện VLM đầy đủ.)

1. **Tái hiện thí nghiệm loss-stabilization (Hình 2) trên một mạng nhỏ**: huấn luyện một CNN nhỏ (hoặc ResNet rút gọn) trên một tập con CIFAR-10, so sánh 4 cấu hình (WD=0/khác 0, LR nhỏ/lớn), đo test error, train CE, và $\|w\|_2$ theo epoch, để tự kiểm chứng hiện tượng "cùng λ, cùng chuẩn cuối, nhưng LR khác nhau cho test error khác nhau" trước khi suy luận áp dụng cho OCR.
2. **Tái hiện thí nghiệm ELR cho một mô hình ngôn ngữ nhỏ (nanoGPT-char hoặc một decoder nhỏ của dự án)**: huấn luyện với AdamW, WD $\in \{0,0{,}1, 0{,}3\}$ trên một corpus nhỏ gần 1 epoch, đo $\eta_t/\|w_t\|_2$ theo bước huấn luyện (giống Hình 7 trái), kiểm tra xem mô hình nhỏ của dự án (hoặc một decoder Transformer đồ chơi cùng kiến trúc attention) có tái hiện hiệu ứng "WD cho ELR cao hơn, training loss thấp hơn về cuối" hay không.
3. **Kiểm tra ổn định bfloat16 cho một lần fine-tune LightOnOCR rút gọn**: fine-tune một vài trăm bước trên một checkpoint LightOnOCR nhỏ (hoặc mô hình thay thế nhỏ hơn) với LR tương đối cao, so sánh WD=0 so với WD khác 0, ở cả bfloat16 và float32, theo dõi có xảy ra loss spike không hồi phục (giống Hình 8) hay không trên kiến trúc hybrid GatedDeltaNet + attention, một kiến trúc khác hẳn Transformer chuẩn của GPT-2.
4. **Đo chuẩn Jacobian xấp xỉ dọc theo fine-tune phục hồi sau pruning**: dùng kỹ thuật Hessian/Jacobian-vector product (đã dùng ở opt 49) để ước lượng $\|J(w)\|_F$ (hoặc một xấp xỉ rẻ của nó) tại vài điểm trong quá trình fine-tune phục hồi mô hình đã pruning, với vài giá trị λ khác nhau, kiểm tra xem $\|J(w)\|_F$ có giảm đơn điệu theo $\eta\lambda$ tăng như Giả thuyết 1 dự đoán hay không, trên đúng bài toán nén của dự án.
5. **Ablation decoupled WD so với ℓ_2 thường trên một fine-tune nhỏ của dự án**: lặp lại ablation kiểu Hình 21 (so sánh AdamW decoupled WD với Adam + ℓ_2 penalty cộng vào loss) trên một lần fine-tune ngắn của LightOnOCR rút gọn, kiểm tra xem hai cách có cho training loss cuối tương đương hay không trên chính kiến trúc và dữ liệu OCR của dự án, trước khi quyết định có cần triển khai đúng chuẩn decoupled WD hay dùng ℓ_2 đơn giản cho đủ.

10. Tóm tắt công thức / khái niệm

- **SGD với weight decay** (Phương trình 1): $w_{t+1} = w_t - \eta\nabla_w \ell(y_{i_t}, h(w_t, x_{i_t})) - \eta\lambda w_t$.
- **Tách phần nhiễu** (Phương trình 2): $w_{t+1} = (1-\eta\lambda)w_t - \eta\nabla L(w_t) + \eta g_t$.
- **Chuẩn Jacobian trung bình**: $\|J(w)\|_F^2 = \frac{1}{N}\sum_i \mathrm{Tr}\big(\nabla h_w(x_i)\nabla h_w(x_i)^\top\big)$.
- **Giả thuyết 1 (điều chuẩn ẩn qua nhiễu, over-training)** (Phương trình 5): điểm dừng trung bình của quá trình ngẫu nhiên là điểm tới hạn bậc một của $\bar{L}_\lambda(w) = L_\lambda(w) + \eta\sigma^2_{\eta,\lambda}\|J(w)\|_F^2$; quy mô nhiễu $\sigma_{\eta,\lambda}$ phụ thuộc cả η và λ, điều khiển mức điều chuẩn.
- **Giả thuyết 3 (biến thể trên mặt cầu, scale-invariant)** (Phương trình 10): $\bar{L}(w) = L(w) + \eta\sigma^2_\eta\|J(w)\|_F^2$ (không có số hạng λ tường minh vì chuẩn đã cố định bằng 1 do chiếu lên mặt cầu).
- **Effective learning rate (ELR) cho sign-SGD/Adam (dưới WD)**: $\eta_{\text{eff}} \propto \eta_t/\|w_t\|_2$ (mũ 1), khác ELR cổ điển cho plain SGD scale-invariant $\eta/\|w\|_2^2$ (mũ 2). Đã tự tính ở Mục 3.5: với $\|w\|_2 = 10$, $\eta = 0{,}01$, hai công thức cho $0{,}001$ so với $0{,}0001$ (khác 10 lần).
- **Cận bias-variance cho SGD** (Mục 3.5, trích Moulines và Bach 2011): Excess Risk $\lesssim (1-\eta\mu)^t\|w_0 - w^*\|^2 + \eta\sigma^2$. Đã tự tính ở Mục 3.3: $\mu = 1, \sigma^2 = 1, \|w_0 - w^*\|^2 = 100$; ở $t = 100$, $\eta = 0{,}1$ thắng ($\approx 0{,}103$ so với $\approx 36{,}61$ cho $\eta = 0{,}01$); ở $t = 10.000$, $\eta = 0{,}01$ thắng ($\approx 0{,}01$ so với $\approx 0{,}1$).
- **Phân rã Gauss-Newton của Hessian** (Phụ lục C.4): $\nabla^2 L(w) = \sum_i G_i(w) + \sum_i E_i(w)$, với G_i (số hạng ngoài tích, liên hệ Jacobian) thường chiếm ưu thế (Sagun và cộng sự, 2017); với square loss, $\mathrm{Tr}(\nabla^2 L) \sim \|J\|_F^2$ chính xác tại điểm interpolate, với CE chỉ là xấp xỉ.
- **Bảng so sánh với các công trình điều chuẩn ẩn liên quan** (Bảng 1, Phụ lục A): Damian và cộng sự (2021)/Li và cộng sự (2021b) dùng square loss/CE+label smoothing với label noise GD, điều chuẩn ẩn = trace Hessian; Blanc và cộng sự (2020) dùng square loss với label noise GD, điều chuẩn ẩn = Jacobian norm; Li và cộng sự (2020) dùng scale-invariant loss với SGD, chưa có điều chuẩn ẩn cụ thể; Andriushchenko và cộng sự (2023) dùng square loss với SGD LR lớn, điều chuẩn ẩn = Jacobian norm; **bài 50** dùng regularized CE với SGD LR lớn, điều chuẩn ẩn = Jacobian norm, tức mở rộng dòng nghiên cứu này từ square loss sang CE có ℓ_2 penalty.

11. Kết luận ngắn

Paper (NeurIPS 2024) đưa ra một bức tranh thống nhất về weight decay qua thực nghiệm trên cả ResNet/ảnh (chế độ nhiều epoch) và GPT-2/LLM (chế độ gần 1 epoch): WD hiếm khi hoạt động như một regularizer tường minh cổ điển; trong chế độ nhiều epoch, nó cho phép nhiễu SGD không triệt tiêu qua cơ chế ”loss stabilization”, từ đó kích hoạt một điều chuẩn ẩn lên chuẩn Jacobian của mạng (Giả thuyết 1, chưa chứng minh toán học, chỉ kiểm chứng thực nghiệm); trong chế độ gần 1 epoch, WD tương đương việc tăng effective learning rate, cải thiện cân bằng bias-variance, và còn mang lại ổn định số học cho huấn luyện bfloat16 (lợi ích này chưa được giải thích đầy đủ bởi chính cơ chế ELR). Với dự án nén/hậu huấn luyện LightOnOCR-3-0.8B, bài học thực tế quan trọng nhất là: luôn giữ WD khác 0 khi fine-tune ở độ chính xác thấp (bf16) trên GPU hạn chế để tránh phân kỳ, và cần ý thức rõ WD đang đóng vai trò gì (regularizer qua nhiễu, hay chỉnh ELR, hay ổn định số học) tuỳ vào việc quá trình phục hồi sau pruning gần với chế độ nhiều epoch hay gần 1 epoch, thay vì áp dụng một giá trị λ mặc định mà không suy xét cơ chế.

Nguồn gốc: https://arxiv.org/abs/2310.04415

Bài 51

How to Set AdamW's Weight Decay as You Scale Model and Dataset Size (2024)

Tác giả: Xi Wang (Đại học Johns Hopkins, Hoa Kỳ), Laurence Aitchison (Đại học Bristol, Anh).
Venue / arXiv: arXiv:2405.13698 (v3, 01/06/2025); paper tự ghi ở chân trang: "Proceedings of the 42nd International Conference on Machine Learning (ICML), Vancouver, Canada. PMLR 267, 2025".
Nguồn: https://arxiv.org/abs/2405.13698
Chủ đề: diễn giải trọng số học được của AdamW như một exponential moving average (EMA, trung bình trượt theo hàm mũ) của các bước cập nhật gần đây; từ đó rút ra "timescale" (thang thời gian) của EMA này là $\tau_{iter} = 1/(\eta\lambda)$, và đề xuất quy tắc scale weight decay λ theo kích thước dữ liệu và kích thước mô hình để giữ timescale này (tính theo epoch) gần như không đổi, qua đó khôi phục khả năng chuyển giao learning rate (LR transfer) của muP cho AdamW.
Vị trí trong lộ trình: đọc sau opt 50 (Why Do We Need Weight Decay, D'Angelo và cộng sự, được chính bài 51 trích dẫn là "concurrent work" và còn dùng lại code NanoGPT của nhóm bài 50 cho thí nghiệm scale theo dữ liệu) và sau opt 35/36 (Tensor Programs V/muP, scaling exponents) để có nền về công thức $\eta \propto 1/\text{fan_in}$; đây là bài cuối cùng trong loạt opt 44-51 của lộ trình, tổng kết câu hỏi thực hành "chọn λ bao nhiêu khi scale mô hình hoặc dữ liệu".

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu, công thức, hình vẽ lấy từ toàn văn arXiv:2405.13698 (bản v3, tải và đọc toàn bộ ngày 09/11/2026, gồm Mục 1-6 và Phụ lục A-H đầy đủ, kể cả tất cả chú thích Hình 1-19 và bảng hyperparameter ở Phụ lục G). Các phép tính đánh dấu "tự tính" là của người ghi chú, không có trong paper. Paper có thực nghiệm thật (ResNet-18/ViT trên CIFAR-10 và ImageNet 32x32 thu nhỏ; NanoGPT 124M và transformer 8 lớp trên OpenWebText) kèm một định lý (Theorem 1) chứng minh chặt cho một họ mạng scale-invariant đặc biệt (không phải kiến trúc chuẩn mặc định); không phải paper lý thuyết thuần nhưng cũng không phải thuần thực nghiệm.

1. Bối cảnh

Theo Mục 1-2 của paper, muP (maximal update parameterization, Yang và cộng sự 2022, chính là opt 35 "Tensor Programs V" trong lộ trình) cho một quy tắc để chọn learning rate (LR) η khi scale độ rộng (width) của mạng, sao cho động lực huấn luyện ở các độ rộng khác nhau "khớp" nhau và hyperparameter tối ưu tìm được ở một mô hình nhỏ có thể chuyển giao (transfer) sang mô hình lớn hơn mà không cần tune lại. Quy tắc này xuất phát từ hai yêu cầu (desiderata) ở gần điểm khởi tạo: (a) đầu ra ban đầu $W_0 x$ phải là $O(1)$, buộc độ lệch chuẩn khởi tạo $\sigma_0 \propto 1/\sqrt{\text{fan_in}}$; (b) ảnh hưởng của bước cập nhật đầu tiên lên đầu ra, $\Delta W\, x$, cũng phải là $O(1)$. Vì cập nhật Adam/AdamW có dạng $\hat{m}/\sqrt{\hat{v}+\epsilon}$, một đại lượng đã được chuẩn hoá theo từng toạ độ để có độ lớn $O(1)$ (Balles và Hennig, 2018, "Dissecting Adam" lập luận Adam gần giống sign descent về độ lớn bước), nên $\Delta W_{ij} \sim \eta$; để $\Delta W_{ij} \cdot \text{fan_in} \sim O(1)$ cần $\eta \propto 1/\text{fan_in}$. Đây chính là quy tắc LR chuẩn của muP cho Adam/AdamW (opt 36, "scaling exponents", là một công trình tiếp nối theo hướng này).

Vấn đề mà bài 51 chỉ ra: lập luận trên của muP chỉ xét vài bước cập nhật đầu tiên gần điểm khởi tạo ngẫu nhiên, và hoàn toàn **không nói gì về weight decay** λ, vì theo paper, WD "chỉ trở nên quan trọng sau rất nhiều bước huấn luyện" (khi trọng số đã đi xa khỏi init). Do đó thực hành phổ biến là giữ λ **cố định** khi scale độ rộng mô hình theo muP. Bài 51 cho thấy cách làm này sai đối với AdamW: nó phá vỡ chính tính chất LR transfer mà muP hướng tới. Bối cảnh liên quan khác: paper AdamW gốc (Loshchilov và Hutter, 2019, chính là opt 07) tham số hoá qua (η, γ) với $\gamma = \eta\lambda$, và từng quan sát thực nghiệm rằng γ "tách biệt" (decoupled) tốt hơn với η so với chính λ, nhưng không giải thích lý do hay đưa ra quy tắc scale cụ thể. Ngoài ra còn một câu hỏi thực hành riêng (không liên quan muP): khi scale **kích thước dữ liệu** pretraining N (theo tinh thần

Chinchilla, kiểu scaling-law), nên chọn λ thế nào? Trước bài 51, không có khuyến nghị có nguyên lý rõ ràng cho câu hỏi này; thực hành thường là giữ λ cố định hoặc tune lại từ đầu ở mỗi quy mô, rất tốn kém.

2. Ý tưởng chính

Ý tưởng cốt lõi: viết lại chính công thức cập nhật trọng số của AdamW để lộ ra nó **chính là** một EMA (trung bình trượt hàm mũ) của các "bước cập nhật" gần đây, và thang thời gian (timescale) của EMA này, $\tau_{iter} = 1/(\eta\lambda)$, là đại lượng điều khiển then chốt.

AdamW dạng PyTorch (Phương trình 4 của paper): $w_t = (1 - \eta_t\lambda)w_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t+\epsilon}}$. So với một EMA tổng quát (Phương trình 5): $\text{ema}_t = (1 - \frac{1}{\tau})\text{ema}_{t-1} + \frac{1}{\tau}q_t$. Khớp hai công thức bằng cách đặt $1/\tau_{iter} = \eta_t\lambda$ (tốc độ "quên"), $\text{ema}_t = w_t$, và $q_t = -\frac{1}{\lambda}\frac{\hat{m}_t}{\sqrt{\hat{v}_t+\epsilon}}$ (Phương trình 6), ta thấy đệ quy AdamW **chính xác** là một EMA theo dõi đại lượng q_t với thang thời gian τ_{iter}. Trực giác: trọng số tại bước t gần như là trung bình có trọng số của khoảng τ_{iter} giá trị q_t gần nhất (các giá trị cũ hơn bị chiết khấu theo hàm mũ). τ_{iter} do đó là siêu tham số thực sự điều khiển "bộ nhớ" của trọng số AdamW, không phải η hay λ riêng lẻ.

Hai hệ quả thực hành chính rút ra từ đó:

- **Theo kích thước dữ liệu**: đo τ_{iter} theo đơn vị epoch, $\tau_{epoch} = \tau_{iter}/M$ với $M = N/B$ (số iteration mỗi epoch, N=kích thước dataset, B=batch size). Phát hiện thực nghiệm trung tâm của paper: τ_{epoch} **tối ưu gần như không đổi khi dataset size** N **thay đổi** (giữ η, B cố định). Vì M tăng theo N, giữ τ_{epoch} không đổi buộc λ phải **giảm** khi N tăng (Phương trình 11: $\lambda = 1/(\eta M\tau_{epoch}) \propto 1/N$).
- **Theo kích thước mô hình**: dưới quy ước muP $\eta = \eta_{base}/s$ (s=hệ số scale độ rộng so với mô hình gốc), nếu giữ $\lambda = \lambda_{base}$ cố định, $\tau_{iter} = 1/(\eta\lambda)$ tăng tỉ lệ s (Phương trình 14) hoàn toàn chỉ do thay đổi độ rộng, một hiệu ứng "ăn theo" không liên quan gì tới động lực học thực sự, phá vỡ giả định ngầm của muP rằng việc scale giữ nguyên động lực huấn luyện. Cách khắc phục: đặt $\lambda = s \cdot \lambda_{base}$ (Phương trình 15), khi đó τ_{iter} không đổi (Phương trình 16), và LR transfer của muP được khôi phục.

Theorem 1 hình thức hoá điều này cho một họ mạng được làm **scale-invariant hoàn toàn** (đầu ra mọi lớp bất biến với việc scale lại trọng số đầu vào của lớp đó, đạt được qua 3-4 thay đổi kiến trúc, xem Mục 3): hai lần chạy AdamW với $(\eta, \lambda, \sigma, \epsilon)$ và $(\eta/c, c\lambda, \sigma/c, c\epsilon)$ (cùng init w_0/c) cho ra **cùng một quỹ đạo hàm số** ở mọi bước, miễn là $\eta\lambda = \eta'\lambda'$ (cùng τ_{iter}) và $\eta/\sigma = \eta'/\sigma'$ (cùng tỉ lệ bước đầu so với quy mô init). Nói cách khác: chỉ τ_{iter} và tỉ lệ này mới thực sự quan trọng; mọi tham số khác chỉ là "nuisance" (không ảnh hưởng quỹ đạo).

3. Phương pháp chi tiết

Dạng tổng có trọng số (Phương trình 7, suy ra ở Phụ lục B). Khai triển đệ quy EMA với tốc độ quên biến đổi theo thời gian $\eta_t\lambda$ (Phương trình 36-37 Phụ lục B): w_t bằng tổng của phần "tàn dư" của w_0 (nhân với tích các hệ số $(1 - \eta_\tau\lambda)$) cộng tổng các $q_{t'}$ với trọng số là tích $(1 - \eta_\tau\lambda)$ cho τ từ $t' + 1$ đến t. Khi $\tau_{iter} \gg 1$, xấp xỉ $(1 - \eta\lambda) \approx e^{-\eta\lambda}$, biến tích thành $\exp(-\sum \eta_\tau\lambda) \approx e^{-(t-t')/\tau_{iter}}$ (khi η xấp xỉ không đổi), cho Phương trình 7: $w_t \approx \frac{1}{\tau_{iter}} \sum_{t'} e^{-(t-t')/\tau_{iter}} q_{t'}$, đúng là công thức trung bình trượt hàm mũ theo $q_{t'}$.

(tự tính) Kiểm tra độ chính xác của xấp xỉ $(1 - 1/\tau) \approx e^{-1/\tau}$ khi $\tau = 100$ và lặp 10 bước: $(1 - 1/100)^{10} = 0{,}99^{10} \approx 0{,}9044$; còn $e^{-10/100} = e^{-0{,}1} \approx 0{,}9048$. Hai số chỉ lệch khoảng $0{,}0004$ (sai số tương đối $\approx 0{,}04\%$), xác nhận xấp xỉ hàm mũ rất tốt với $\tau_{iter} \gg 1$ ngay cả sau khi lặp lại 10 lần, đúng là cơ sở của Phương trình 7.

Ở đây $\hat{m}_t$ là moment bậc một đã hiệu chỉnh bias (EMA của gradient qua hệ số β_1), $\hat{v}_t$ là moment bậc hai đã hiệu chỉnh bias (EMA của gradient bình phương qua β_2), và $q_t = -\frac{1}{\lambda}\frac{\hat{m}_t}{\sqrt{\hat{v}_t+\epsilon}}$. Theo Balles và Hennig (2018), vì $\hat{m}/\sqrt{\hat{v} + \epsilon}$ đã được chuẩn hoá theo từng toạ độ để có độ lớn $O(1)$ (Adam hành xử gần giống sign-of-gradient descent về biên độ bước, dù không hoàn toàn là dấu), nên q_t có độ lớn $O(1/\lambda)$, **không phụ thuộc** vào độ lớn gradient thật. Điều này trực tiếp giải thích tại sao độ lớn trọng số học được W_{ij} nên có quy mô $O(1/\lambda)$: vì w_t xấp xỉ là trung bình trượt của các đại lượng cỡ $O(1/\lambda)$, trọng số sẽ "kế thừa" quy mô đó khi huấn luyện đủ lâu ($t \gg \tau_{iter}$, để EMA "quên" điều kiện khởi tạo).

Công thức τ_{epoch} (Phương trình 10): $\tau_{epoch} = \tau_{iter}/M = \dfrac{1}{\eta\lambda M} = \dfrac{1}{\eta\lambda N/B}$, với $M = N/B$ là số iteration mỗi epoch.

(tự tính) Minh hoạ quy tắc scale theo dataset size với số tròn: chọn $\eta = 0{,}001$, $\lambda = 0{,}1$, batch $B = 100$, $N = 1.000.000 \Rightarrow M = 10.000$ iteration/epoch; $\tau_{iter} = 1/(\eta\lambda) = 1/0{,}0001 = 10.000$; $\tau_{epoch} = \tau_{iter}/M = 10.000/10.000 = 1$ (epoch). Giả sử N tăng 4 lần thành $4.000.000$ ($M = 40.000$), muốn giữ $\tau_{epoch} = 1$ không đổi (giữ η không đổi): cần $\tau_{iter} = \tau_{epoch} \times M = 1 \times 40.000 = 40.000$, suy ra $\lambda = 1/(\eta\tau_{iter}) = 1/(0{,}001 \times 40.000) = 0{,}025$. Vậy λ mới $= 0{,}025 = \lambda_{\text{cũ}}/4$, đúng tỉ lệ nghịch với N (N tăng 4 lần, λ giảm đúng 4 lần), khớp chính xác với Phương trình 11 ($\lambda \propto 1/N$) khi τ_{epoch} giữ đúng là hằng số.

Công thức scale theo kích thước mô hình (Phương trình 12-16). Quy ước muP: $\eta = \eta_{base}/s$. Cách "trực tiếp"/ngây thơ: giữ $\lambda = \lambda_{base}$ cố định $\Rightarrow \tau_{iter} = \dfrac{1}{(\eta_{base}/s)\lambda_{base}} = s \cdot \tau_{iter,base}$ (Phương trình 14, tăng tuyến tính theo s). Cách đề xuất: đặt $\lambda = s \cdot \lambda_{base}$ (Phương trình 15) $\Rightarrow \tau_{iter} = \dfrac{1}{(\eta_{base}/s)(s\lambda_{base})} = \dfrac{1}{\eta_{base}\lambda_{base}} = \tau_{iter,base}$ (Phương trình 16, không đổi, độc lập với s).

(tự tính) Với $s = 4$ (mô hình rộng gấp 4, theo hệ số fan_in), $\eta_{base} = 0{,}001$, $\lambda_{base} = 0{,}1 \Rightarrow \tau_{iter,base} = 1/(0{,}001 \times 0{,}1) = 10.000$. Theo muP, $\eta = \eta_{base}/s = 0{,}00025$. Cách trực tiếp ($\lambda = 0{,}1$ cố định): $\tau_{iter} = 1/(0{,}00025 \times 0{,}1) = 1/0{,}000025 = 40.000 = 4 \times 10.000 = s \cdot \tau_{iter,base}$, khớp Phương trình 14. Cách đề xuất ($\lambda = s\lambda_{base} = 0{,}4$): $\tau_{iter} = 1/(0{,}00025 \times 0{,}4) = 1/0{,}0001 = 10.000 = \tau_{iter,base}$, không đổi, khớp Phương trình 16.

Theorem 1 (phát biểu và sơ đồ chứng minh). Với mạng scale-invariant hoàn toàn ở mọi lớp, đạt được qua các thay đổi kiến trúc ở Phụ lục A.3: (i) lớp đầu ra chuẩn hoá ("normed output") thay cho lớp tuyến tính chuẩn, để đầu ra toàn mạng không phụ thuộc quy mô trọng số tổng thể; (ii) learning rate riêng ("decoupled") cho tham số scale của BatchNorm/LayerNorm, thay vì dùng chung η với các trọng số khác; (iii) quy mô khởi tạo phụ thuộc η ("rescaled init"), tức σ_{init} gắn với η thay vì cố định; và với ViT, thêm (iv) một thay đổi cấu trúc nữa để đảm bảo bất biến tỉ lệ hoàn toàn trong residual stream (gọi là "SI structure", Phụ lục A.3.4): hai cấu hình AdamW liên hệ qua hệ số c như trên cho ra quỹ đạo đầu ra **giống nhau tuyệt đối** ở mọi bước, miễn chia sẻ τ_{iter} và tỉ lệ η/σ_{init}. Chứng minh bằng quy nạp theo t (Phụ lục A.1): nếu ở bước t có $m'_t = c \cdot m_t$, $v'_t = c^2 \cdot v_t$, $w'_t = w_t/c$ (quan hệ scale giữa hai hệ thống "nguyên bản" và "có dấu phẩy"), quan hệ này được bảo toàn ở bước $t+1$ qua thay thế trực tiếp vào công thức cập nhật Adam/AdamW, dùng đúng giả định $\eta\lambda = \eta'\lambda'$ và $\eta/\sigma = \eta'/\sigma'$; trường hợp cơ sở $t = 0$ đúng theo cách dựng init đã rescale.

Cần nhấn mạnh rõ: Theorem 1 là một phát biểu toán học cho mạng thoả **giả định lý tưởng hoá** (scale-invariant hoàn toàn); nó **không** tự động đúng cho ResNet/ViT/Transformer kiến trúc chuẩn (vốn có lớp đầu ra tuyến tính thường, η chung cho tham số norm-layer, và init cố định không phụ thuộc η). Chính Hình 7-10 của paper (Phụ lục) cho thấy mạng kiểu chuẩn **không** thể hiện tính bất biến theo η ở τ_{epoch} cố định như lý thuyết dự đoán, cho tới khi áp đủ cả 3 (hoặc 4 với ViT) thay đổi; thêm từng phần một chỉ thu hẹp một phần khoảng cách, chỉ tổ hợp **đầy đủ** mới đạt tính bất biến sạch như Theorem 1 tiên đoán.

4. Thí nghiệm và kết quả chính

Scale theo kích thước dữ liệu (Mục 4.1). ResNet-18 và ViT trên các tập con ImageNet 32x32 thu nhỏ (Chrabaszcz và cộng sự, 2017) kích thước $\{80\text{k}, 160\text{k}, 320\text{k}, 640\text{k}, 1{,}28\text{M} = \text{đầy đủ}\}$, batch size 100, LR khởi tạo 10^{-3} giảm dần theo cosine về $0{,}1$ lần, 100 epoch, quét λ trên khoảng rộng ($\lambda = (2{,}5 \times 10^{-6}) \times 2^i$, $i = 0, \ldots, 15$, theo Phụ lục G.2). Hình 1: λ tối ưu giảm khoảng 2 bậc độ lớn khi N tăng từ 80K lên 1,28M, nhưng τ_{epoch} tối ưu nằm trong một khoảng tương đối hẹp, gần như không đổi qua cả 5 kích thước dataset. Kiểm tra độ bền với schedule khác: LR cố định (Hình 13, phụ lục) và cosine-decay-về-chính-xác-0 (Hình 14, phụ lục) cho mẫu hình tương tự, không phải hiện tượng chỉ xảy ra với schedule cụ thể trong hình chính.

NanoGPT 124M trên các tập con OpenWebText $\{1/4, 1/2, \text{đầy đủ}\}$ (dùng lại chính code của D'Angelo và

cộng sự, 2024, tức repo của opt 50), batch size cố định 122.880 token, LR $6 \times 10^{-4} \to 6 \times 10^{-5}$ theo cosine, huấn luyện khoảng 4 lượt qua mỗi tập con, λ quét theo luỹ thừa của 2 từ 2^{-8} đến 2. Hình 2: cùng mẫu hình (λ tối ưu giảm theo N, τ_{epoch} tối ưu ổn định).

Đối chiếu với tài liệu gần đây: Bjorck và cộng sự (2024) và Schaipp và cộng sự (2025) phát hiện rằng ở λ và batch size **cố định**, η tối ưu giảm khi chạy huấn luyện dài hơn (nhiều dữ liệu/bước hơn). Paper tái lập điều này (Hình 12 phụ lục, dòng trên: λ cố định, η tối ưu giảm theo N) và chỉ ra một hướng khác đạt hiệu ứng ổn định tương tự: scale $\lambda \propto 1/N$ **thay vì** động tới η cho η tối ưu ổn định qua các kích thước dataset (Hình 12 phụ lục, dòng dưới). Tức có hai cách giữ τ_{epoch} không đổi (giảm η, hoặc giảm λ); khuyến nghị của paper cụ thể là hướng λ vì nó tách biệt một hiệu ứng "sạch" hơn (xem thêm Mục 5/7).

Kiểm chứng tiên đoán quy mô trọng số (cuối Mục 4.2, Phụ lục D, Hình 11). ResNet-18/CIFAR-10, AdamW, LR giảm cosine qua 1000 epoch, quét λ từ 10^{-3} đến 1 với $\eta = 10^{-5}/\lambda$ cho mỗi lần chạy (tức $\tau_{iter} = 1/(\eta\lambda) = 1/10^{-5} = 10^5$ giữ **cố định** suốt toàn bộ quét, nhằm tách riêng hiệu ứng weight decay lên quy mô trọng số cuối cùng khỏi nhiễu do timescale thay đổi). Đo $E[|W_{ij}|]$ trung bình cuối cùng ở các lớp conv/linear theo λ: theo một đường thẳng độ dốc -1 trên thang log-log, khớp gần như chính xác với đường tham chiếu $y = 0{,}01/x$, tức thực nghiệm $E[|W_{ij}|] \approx 0{,}01/\lambda$ (hằng số $\approx 0{,}01$ là khớp thực nghiệm riêng cho kiến trúc/init này, không phải paper suy ra dạng đóng). Phần phải Hình 11 còn cho thấy diễn tiến theo thời gian: quy mô trọng số bắt đầu ở mức init (chung cho mọi λ), rồi dần hội tụ về mức tiệm cận $O(1/\lambda)$, xác nhận trực quan cơ chế "EMA quên điều kiện khởi tạo theo thang τ_{iter}" ở Mục 3.

Scale theo kích thước mô hình (Mục 4.2). ResNet-18, hệ số độ rộng $s \in \{0{,}5, 1, 2\}$ qua scale độ rộng chuẩn của thư viện mup, trên tập con ImageNet 320K, $\eta_{base} = 10^{-3}$, quét λ_{base} khoảng 10^{-2} đến 10, 50 epoch. Hình 3: dòng trên (muP trực tiếp, λ cố định theo Phương trình 13) cho thấy λ_{base} tối ưu lệch đáng kể qua các độ rộng; dòng dưới (scale đề xuất $\lambda = s \cdot \lambda_{base}$, Phương trình 15) cho λ_{base} tối ưu (và do đó τ_{iter}) ổn định/chồng lấp hơn nhiều qua ba độ rộng. Paper ghi nhận dữ liệu cũng phù hợp với một quan hệ **siêu tuyến tính** khả dĩ giữa λ_{base} tối ưu và s, nhưng nói rõ "cần thêm thực nghiệm lớn hơn để xác định dứt điểm tính siêu tuyến tính này", coi đây là nằm ngoài phạm vi paper.

Transformer 8 lớp kiểu GPT, độ rộng thay đổi qua số head $\{4, 8, 16\}$ (giữ cố định head dim 64) dùng codebase nanoGPT-mup, OpenWebText, hai lựa chọn $\eta_{base} \in \{3 \times 10^{-4}, 3 \times 10^{-5}\}$, 250K iteration, λ quét theo luỹ thừa 2 từ 2^{-6} đến 1. Hình 4: cùng câu chuyện, λ_{base} tối ưu ổn định dưới scale s đề xuất, không ổn định dưới λ cố định.

Kết quả trung tâm "muP hỏng, rồi được sửa" (Hình 5). ResNet-18 trên CIFAR-10 và trên tập con ImageNet-32 320K, nhiều hệ số độ rộng s, λ_{base} **cố định** ở một giá trị (paper chọn $\lambda_{base} = 1{,}0$), quét η_{base} rộng (10^{-5} đến $\sim 10^{-2}$ cho CIFAR; $(2{,}5 \times 10^{-6}) \times 2^i$, $i = 0, \ldots, 11$, cho ImageNet). Dòng trên: quy ước muP chuẩn ($\eta = \eta_{base}/s$, λ cố định) cho thấy η_{base} tối ưu lệch dữ dội (nhiều bậc độ lớn) qua các độ rộng, tức tính LR transfer nổi tiếng của muP **bị phá vỡ** cho AdamW một khi weight decay có mặt và không được scale. Dòng dưới: scale đề xuất $\lambda = s \cdot \lambda_{base}$ (Phương trình 15) khôi phục η_{base} tối ưu ổn định, không phụ thuộc độ rộng, tức LR transfer của muP được **khôi phục**. Phiên bản tương tự cho ViT trên ImageNet (Phụ lục E/Hình 15) cho cùng kết luận. Paper coi đây là minh chứng thực nghiệm quan trọng nhất và lợi ích thực hành rõ ràng nhất của toàn bộ quy tắc scale, và ghi nhận phát hiện này được củng cố độc lập bởi quan sát ở quy mô LLM lớn (Lingle, 2024; sau này Blake và cộng sự, 2024).

Paper không đặt mục tiêu đạt độ chính xác state-of-the-art; đây là paper về phương pháp/quy tắc scale, các số liệu accuracy chỉ dùng để xác nhận độ ổn định của hyperparameter transfer, không nhằm công bố SOTA mới.

5. Phân tích / ablation

Ablation các giả định kiến trúc của Theorem 1 (Hình 7-10, đã nêu ở Mục 3). Từng thay đổi (lớp đầu ra chuẩn hoá, init phụ thuộc η, LR riêng cho BN/LN, và với ViT thêm "SI structure") được thêm dần một; chỉ tổ hợp **đầy đủ** mới loại bỏ sự phụ thuộc vào η của quỹ đạo huấn luyện ở τ_{epoch} cố định. Đây là một ablation

có giá trị thông tin riêng: nó xác định chính xác những "dư lượng" phụ thuộc tỉ lệ nào của kiến trúc chuẩn là nguyên nhân tạo khoảng cách giữa lý thuyết lý tưởng hoá (Theorem 1) và thực hành.

Ablation độ bền với schedule (Hình 13 phụ lục LR cố định, Hình 14 cosine-về-đúng-0). Phát hiện "τ_{epoch} tối ưu gần như không đổi" sống sót qua thay đổi schedule LR, tức không phải hiện tượng chỉ của schedule cosine-về-0,1-lần dùng ở hình chính.

Ablation nút λ so với nút η (Hình 12 phụ lục). Đối chiếu trực tiếp "giữ λ cố định, để η tối ưu giảm theo N" (tái lập Bjorck và cộng sự 2024 / Schaipp và cộng sự 2025) với "giữ η cố định, để λ giảm theo N" (khuyến nghị của paper 51); cả hai đều là con đường hợp lệ tới cùng điểm tối ưu τ_{epoch} không đổi, nhưng scale λ "sạch" hơn vì động tới η còn làm thay đổi động lực gần init (theo điều kiện thứ hai của Theorem 1, tỉ lệ η/σ_{init}), trộn lẫn hai hiệu ứng tách biệt về logic, trong khi động tới λ riêng tách được đúng hiệu ứng "thang thời gian EMA". Điều này được dùng trong thảo luận liên quan (Mục 5) để giải thích vì sao quan sát cũ của Loshchilov và Hutter (AdamW gốc, rằng $\gamma = \eta\lambda$ "decoupled" tốt hơn λ) lại đúng: γ (tương đương $1/\tau_{iter}$) điều khiển chế độ "dài hạn" của EMA, còn η với γ cố định riêng điều khiển chế độ "ngắn hạn, gần init", đúng là hai điều kiện của Theorem 1.

Ablation quy mô trọng số (Hình 11) đóng vai trò một ablation gián tiếp/cấu trúc xác nhận chính **cơ chế** EMA-của-cập-nhật (không chỉ hệ quả thực hành của quy tắc scale): nếu trọng số AdamW thực sự hành xử như EMA của các đại lượng cỡ $O(1/\lambda)$, quy mô trọng số cuối cùng **phải** scale theo $1/\lambda$, độc lập với bất kỳ thảo luận scale theo model/dataset size nào; điều này được xác nhận với τ_{iter} giữ cố định suốt quét để tách riêng đúng hiệu ứng này.

Kiểm tra nhất quán nối lại với chính desiderata của muP (cuối Mục 4.2): dưới scale đề xuất $\lambda \propto s$, trọng số học được $|W_{ij}| \sim 1/\lambda \sim 1/s \sim 1/\text{fan_in}$ (vì s tỉ lệ với fan_in theo quy ước độ rộng của muP), **đúng** là quy mô trọng số mà lý thuyết muP yêu cầu để pre-activation ổn định ở $O(1)$. Paper nhấn mạnh đây là một kiểm tra nhất quán nội bộ đáng mừng, không phải một phép suy ra độc lập: nó cho thấy hai desiderata (scale init/update của muP, và lập luận thang thời gian EMA cho scale WD) khớp nhau, nhưng không chứng minh cái này từ cái kia.

6. Giới hạn

- **(a) Giả định kiến trúc của Theorem 1 không khớp kiến trúc chuẩn.** Theorem 1 chỉ đúng chặt cho mạng có 3-4 thay đổi kiến trúc không nhỏ mà ResNet/ViT/Transformer chuẩn thường **không** có sẵn (lớp đầu ra tuyến tính thường, η chung cho tham số scale của norm-layer, init độc lập quy mô). Chính Hình 7B/8B và 9B/10B của paper (các dòng chưa áp đủ sửa) cho thấy mạng kiểu chuẩn không thoả tính bất biến theo η ở τ_{epoch} cố định như lý thuyết dự đoán. Đây là khoảng cách giữa lý thuyết sạch và thực hành thông thường mà paper tự thừa nhận.
- **(b) Tính siêu tuyến tính chưa xác định của λ_{base} theo s.** Ở Mục 4.2 (Hình 3), paper thừa nhận xu hướng có thể là siêu tuyến tính thay vì quy tắc tỉ lệ thuận đơn giản $\lambda \propto s$ mà paper đề xuất, nhưng nói rõ việc xác nhận/bác bỏ cần "thêm thực nghiệm lớn hơn", để ngoài phạm vi bài.
- **(c) Khác biệt quan trọng giữa chế độ nhiều-epoch (được kiểm chứng trong bài) và chế độ một-epoch thực tế của LLM pretraining.** Toàn bộ thực nghiệm của paper 51 đều ở chế độ **nhiều epoch** (ResNet 100-1000 epoch, NanoGPT khoảng 4 lượt qua tập con), tức dataset được xem lại nhiều lần. Pretraining LLM thực tế lại gần với chế độ **đúng một epoch**. Paper tự nêu rõ trong phần liên quan (Mục 5, "công trình sau bài này") rằng kết luận "τ_{epoch} tối ưu gần như không đổi" được rút ra ở chế độ nhiều epoch và có thể không chuyển trực tiếp sang chế độ một epoch; paper trích dẫn công trình nối tiếp (Bergsma và cộng sự, 2025a, "Power lines: scaling laws for weight decay and batch size in LLM pre-training") chuyên nghiên cứu đúng chế độ một epoch, được paper 51 báo cáo là tìm thấy τ_{epoch} tối ưu **không** giữ nguyên ở đó, mà theo một quy luật luỹ thừa, τ_{epoch} tối ưu $\approx (\text{TPP})^{-0,527}$ (TPP = tokens-per-parameter), giảm khi TPP tăng, và luôn nhỏ hơn 1 khi TPP lớn hơn 1. Đây là một điểm cần

lưu ý quan trọng, có khả năng làm giảm giá trị của tuyên bố trung tâm "τ_{epoch} tối ưu gần như hằng số" của chính bài 51, đúng ngay ở ứng dụng LLM pretraining mà bài 51 tự nêu là động lực chính.

- **(d) Batch size B được giữ cố định xuyên suốt mọi thí nghiệm.** Tương tác giữa scale B và quy tắc scale λ đề xuất được paper nói rõ để lại cho công trình liên quan/nối tiếp (quy tắc $\eta \propto B$ kiểu McCandlish và cộng sự 2018, hoặc $\lambda \propto B$ theo Bergsma và cộng sự 2025a), không được nghiên cứu trong bài này.
- **(e) Cạm bẫy thực hành của thư viện mup (Phụ lục F).** Thư viện mup của Microsoft có tham số từ khoá tên là `decoupled`, nhưng phải đặt `decoupled=False` (không phải `True`, dù tên nghe như lựa chọn "đúng/cẩn thận" hơn) để đạt đúng quy tắc scale λ đề xuất (Phương trình 15) của paper; paper cảnh báo rõ người dùng phải kiểm tra trực tiếp source code thư viện, không nên tin vào tên tham số, một cạm bẫy thực hành thật cho ai muốn tái lập hoặc áp dụng khuyến nghị này.
- **(f) Nguy cơ nhiễu loạn từ AutoAugment/label smoothing trong thí nghiệm ViT (Phụ lục G.1), ghi nhận của người viết ghi chú.** ViT được huấn luyện có AutoAugment và label smoothing $\alpha = 0{,}1$, được paper mô tả là "crucial" để đạt accuracy tham chiếu đã công bố; đây là nhiễu loạn (confound) khả dĩ chưa được tách bạch hoàn toàn khỏi chính các kết luận về scale WD/LR đang được kiểm định trên ViT, vì các kỹ thuật augmentation/regularization này cũng tương tác với động lực huấn luyện hiệu quả. Paper không tự nêu điểm này; đây là suy luận riêng của người viết ghi chú.
- **(g) Hằng số $O(1/\lambda)$ ở Hình 11 chỉ là khớp thực nghiệm.** $E[|W|] \approx c/\lambda$ với $c \approx 0{,}01$ là khớp thực nghiệm riêng cho kiến trúc/init đã thử, không phải giá trị paper suy ra dạng đóng; nên hiểu công thức này ở mức định tính (độ dốc đúng) hơn là định lượng phổ quát.
- **(h) Chứng minh Theorem 1 dùng đệ quy AdamW đã đơn giản hoá.** Chứng minh ở Phụ lục A.1 là một lập luận quy nạp cho đệ quy AdamW lý tưởng hoá theo một cách xử lý bias-correction/epsilon cụ thể; đây là một lập luận toán học cho một họ mạng lý tưởng hoá, chưa được kiểm chứng đối chiếu với từng chi tiết số học dấu phẩy động thực tế của các cài đặt optimizer trong framework, một đơn giản hoá hợp lý nhưng không được paper nêu rõ ràng (quan sát riêng của người viết ghi chú).

7. Liên hệ với các bài khác

Đã kiểm tra tồn tại bằng `ls` trước khi trích dẫn (thư mục `optimization content/` và `llm content/`):

- **optimization content/50_2023_why_weight_decay.md**: liên hệ trực tiếp nhất. Bài 51 gọi chính xác bài 50 là "concurrent work" (D'Angelo và cộng sự, 2024), còn tái dùng code NanoGPT của nhóm bài 50 cho thí nghiệm scale theo dataset size (Mục 4.1). Bài 50 lập luận WD kiểm soát quy mô nhiễu minibatch qua $\eta\lambda$ (cùng đại lượng $1/\tau_{iter}$ của bài 51); bài 51 tự nhận định khung EMA của mình "cung cấp hướng dẫn cụ thể hơn cho cách đặt λ khi scale model/dataset size", tức mở rộng cơ chế định tính của bài 50 thành quy tắc định lượng.
- **optimization content/07_2017_adamw.md**: công thức AdamW gốc (Phương trình 4 của bài 51 chính là công thức này) và quan sát $\gamma = \eta\lambda$ "decoupled" của Loshchilov và Hutter, được bài 51 giải thích cơ chế ở Mục 5/7.
- **optimization content/35_2022_tensor_programs_v_mutransfer.md**: nguồn gốc muP và quy tắc $\eta \propto 1/\text{fan_in}$ mà bài 51 xây dựng trên và chỉ ra lỗ hổng (thiếu quy tắc cho λ).
- **optimization content/36_2024_scaling_exponents_parameterizations_optimizers.md**: công trình tiếp nối về scaling exponents/parameterization cho optimizer, cùng hướng muP-cho-Adam mà bài 51 đặt câu hỏi tiếp theo (WD).
- **optimization content/06_2014_adam.md**: định nghĩa gốc của $\hat{m}_t, \hat{v}_t$ dùng trực tiếp trong Phương trình 4-6 của bài 51.
- **llm content/43_2026_lightonocr.md**: bài nền của mô hình dự án (LightOnOCR), dùng cho liên hệ ở Mục 8 dưới đây.

8. Ứng dụng cho dự án (suy luận)

Toàn bộ mục này là **suy luận/speculation** của người viết ghi chú; bài 51 không nghiên cứu OCR, VLM, kiến trúc hybrid GatedDeltaNet + full attention, pruning, hay batch size 1.

- **Phục hồi sau pruning như một lần huấn luyện ngắn, cần chọn lại τ_{iter} hợp lý.** Fine-tune/recovery sau pruning thường là một run ngắn, ít epoch. Nếu pruning làm co hẹp fan_in hiệu dụng của một lớp (ví dụ giảm state dimension của GatedDeltaNet hay số head attention) với hệ số $s < 1$ nhưng recovery fine-tune thực tế vẫn dùng một η toàn cục đơn giản (không tái suy ra scale muP theo từng lớp), lập luận ở Mục 3/4.2 gợi ý λ nên được điều chỉnh **cùng hướng** với thay đổi độ rộng (giảm λ khi fan_in co lại sau pruning) để giữ τ_{iter} gần khớp với thang thời gian ngầm định của lần pretraining gốc, thay vì tái sử dụng mù quáng λ của mô hình gốc.
- **Dataset fine-tune nhỏ hơn rất nhiều so với pretraining, nên tăng λ tương ứng.** Dataset phục hồi/fine-tune OCR thường nhỏ hơn rất nhiều so với corpus pretraining gốc (N giảm mạnh), nên quy tắc Mục 4.1 ($\lambda \propto 1/N$, giữ η cố định) gợi ý λ nên **tăng** so với giá trị dùng ở pretraining đầy đủ, không nên giữ nguyên, khi chạy một recovery run ngắn vài nghìn bước trên tập OCR nhỏ hơn nhiều, vì nếu không, thang thời gian ngầm τ_{epoch} sẽ "phình to" quá mức so với số epoch thực sự dự định cho tập fine-tune nhỏ này.
- **Batch size 1 (ràng buộc GPU ≤8GB) khiến $M = N/B$ rất lớn, công thức τ_{epoch} nhạy đặc biệt với B.** Vì dự án huấn luyện ở batch size 1, công thức $\tau_{epoch} = 1/(\eta\lambda N/B)$ rất nhạy với B trong chế độ batch siêu nhỏ này; λ có thể cần đặt khác đáng kể (nhiều khả năng lớn hơn, vì số "iteration mỗi sample" hiệu dụng tăng mạnh ở $B = 1$) so với các khuyến nghị dựa trên batch lớn, điều này nên được kiểm tra trực tiếp bằng thực nghiệm nhỏ (xem Mục 9) thay vì sao chép hyperparameter từ recipe batch lớn.

9. Thí nghiệm gợi ý

1. **Kiểm tra quy mô trọng số (tái lập logic Hình 11 ở quy mô nhỏ):** fine-tune (hoặc huấn luyện vài trăm bước) một checkpoint LightOnOCR đã pruning với AdamW, quét $\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}, 1{,}0\}$ trong khi giữ $\tau_{iter} = 1/(\eta\lambda)$ cố định (điều chỉnh η nghịch theo λ, đúng cách paper làm); đo $E[|W|]$ cuối cùng ở vài lớp linear; kiểm tra liệu xu hướng $\approx 1/\lambda$ còn giữ trong bối cảnh rất khác (mô hình đã pruning, recovery run ngắn) hay không.
2. **Ablation τ_{epoch}-không-đổi cho recovery fine-tune:** chạy cùng một quy trình phục hồi sau pruning ở hai kích thước dataset OCR rất khác nhau (ví dụ 10K và 40K trang tài liệu), lần đầu giữ λ cố định, lần sau scale $\lambda \propto 1/N$ theo Phương trình 11; so sánh η tối ưu (hoặc độ chính xác/CER cuối cùng) có ổn định hơn qua hai kích thước dataset dưới cách scale λ hay không, mô phỏng đúng phương pháp của Hình 1/2 ở quy mô nhỏ hơn nhiều.
3. **Tái lập mini "muP hỏng rồi sửa" trên một sweep độ rộng do pruning tạo ra:** lấy 2-3 phiên bản của cùng một block transformer với độ rộng khác nhau do pruning (ví dụ thay đổi state/head dimension của GatedDeltaNet), áp quy tắc muP chuẩn $\eta \propto 1/\text{fan_in}$ với λ cố định, quét η_{base}, xem η_{base} tối ưu có lệch qua các độ rộng (tái lập kiểu "hỏng" ở Hình 5) hay không; sau đó lặp lại với $\lambda \propto s$, xem liệu η_{base} tối ưu có ổn định hơn, từ đó quyết định có nên áp cách sửa của paper cho các thí nghiệm width-pruning của dự án hay không.
4. **Kiểm tra xấp xỉ EMA cho đúng cấu hình optimizer thực tế của dự án:** thay trực tiếp các giá trị $(\eta, \lambda, \text{batch size}, \text{dataset size})$ thực tế đang dùng cho recovery vào $\tau_{iter} = 1/(\eta\lambda)$ và $\tau_{epoch} = \tau_{iter} \cdot B/N$, xem "cửa sổ nhớ" ngầm định của run phục hồi hiện tại đang rơi vào đâu (ví dụ có "nhớ" nhiều hơn hay ít hơn một lượt qua tập fine-tune OCR); cho biết trực tiếp liệu λ hiện tại có hợp lý theo chẩn đoán của paper, không cần huấn luyện lại, rất rẻ để tính.
5. **Kiểm tra tương tác với bfloat16 (nối lại câu hỏi mở của bài 50):** chạy chính pipeline recovery sau pruning của dự án với WD bật/tắt ở cùng τ_{iter} (điều chỉnh η để bù), huấn luyện ở bfloat16, so sánh độ ổn định (loss spike); kiểm tra trực tiếp liệu khung "bộ nhớ EMA" của bài 51 có đóng góp thêm gì so với giải thích (chưa đầy đủ) của bài 50 về lợi ích ổn định bf16 của WD, trong đúng bối cảnh kiến trúc

hybrid-attention đã pruning của dự án.

10. Tóm tắt công thức / khái niệm

- **Cập nhật AdamW** (Phương trình 4): $w_t = (1 - \eta_t\lambda)w_{t-1} - \eta_t \dfrac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$.
- **Khớp EMA** (Phương trình 5-6): $\text{ema}_t = (1 - \frac{1}{\tau})\text{ema}_{t-1} + \frac{1}{\tau}q_t$, với $1/\tau_{iter} = \eta_t\lambda$ và $q_t = -\frac{1}{\lambda}\dfrac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$.
- **Dạng tổng có trọng số** (Phương trình 7): $w_t \approx \dfrac{1}{\tau_{iter}} \sum_{t'} e^{-(t-t')/\tau_{iter}} q_{t'}$. Đã tự tính: $(0{,}99)^{10} \approx 0{,}9044$ so với $e^{-0{,}1} \approx 0{,}9048$, sai số $\approx 0{,}04\%$.
- **Thang thời gian theo epoch** (Phương trình 10): $\tau_{epoch} = \tau_{iter}/M = \dfrac{1}{\eta\lambda M} = \dfrac{1}{\eta\lambda N/B}$, $M = N/B$.
- **Quy tắc scale theo dataset size** (Phương trình 11): giữ τ_{epoch} không đổi (ở η, B cố định) $\Rightarrow \lambda \propto 1/N$. Đã tự tính: N tăng 4 lần, λ giảm đúng 4 lần ($0{,}1 \to 0{,}025$).
- **Quy tắc scale theo kích thước mô hình** (Phương trình 12-16): với muP $\eta = \eta_{base}/s$; λ cố định $\Rightarrow \tau_{iter} = s \cdot \tau_{iter,base}$ (hỏng LR transfer); $\lambda = s \cdot \lambda_{base} \Rightarrow \tau_{iter} = \tau_{iter,base}$ (khôi phục). Đã tự tính với $s = 4$: τ_{iter} từ 40.000 (cách cũ) về lại 10.000 (cách đề xuất, bằng $\tau_{iter,base}$).
- **Theorem 1 (tóm gọn):** với mạng scale-invariant đầy đủ, hai cấu hình AdamW $(\eta, \lambda, \sigma, \epsilon)$ và $(\eta/c, c\lambda, \sigma/c, c\epsilon)$ (cùng w_0/c) cho cùng quỹ đạo hàm số ở mọi bước, miễn $\eta\lambda = \eta'\lambda'$ và $\eta/\sigma = \eta'/\sigma'$.
- **Quy mô trọng số học được** (Phụ lục D, thực nghiệm, Hình 11): $E[|W_{ij}|] \approx c/\lambda$ với $c \approx 0{,}01$ khớp thực nghiệm trên ResNet-18/CIFAR-10 (không phải hằng số suy từ lý thuyết).
- **Liên hệ muP:** dưới scale đề xuất, $|W| \sim 1/\lambda \sim 1/s \sim 1/\text{fan_in}$, khớp đúng yêu cầu pre-activation $O(1)$ của muP, một kiểm tra nhất quán (không phải một chứng minh độc lập).

11. Kết luận ngắn

Bài 51 (ICML 2025) cho một cách đọc mới, gọn gàng của trọng số AdamW: chúng chính là một EMA của các cập nhật gần đây với thang thời gian $\tau_{iter} = 1/(\eta\lambda)$; từ đó suy ra hai quy tắc thực hành, $\lambda \propto 1/N$ khi scale dataset size và $\lambda \propto s$ (tỉ lệ độ rộng) khi scale model size dưới muP, cả hai được xác nhận bằng thực nghiệm trên ResNet/ViT (CIFAR-10, ImageNet thu nhỏ) và transformer kiểu GPT nhỏ, cùng một định lý chứng minh chặt cho một họ mạng scale-invariant lý tưởng hoá. Giới hạn quan trọng nhất cần nhớ: toàn bộ xác nhận thực nghiệm nằm ở chế độ nhiều epoch, còn công trình nối tiếp (Bergsma và cộng sự, 2025a) cho thấy trong chế độ một epoch thực tế của LLM pretraining, τ_{epoch} tối ưu không hằng định mà theo một luật luỹ thừa theo tokens-per-parameter, một điểm cần cân nhắc kỹ trước khi áp dụng máy móc quy tắc $\lambda \propto 1/N$ cho pretraining LLM thực sự. Với dự án nén/hậu huấn luyện LightOnOCR-3-0.8B, bài học thực tế (suy luận, chưa kiểm chứng) là: khi phục hồi sau pruning ở một dataset nhỏ hơn và một batch size cực nhỏ (batch 1), nên chủ động tính lại τ_{iter}/τ_{epoch} ngầm định của chính cấu hình đang dùng, thay vì tái sử dụng λ của checkpoint gốc mà không xét quy mô mới của dữ liệu và mô hình.

Nguồn gốc: https://arxiv.org/abs/2405.13698